

Palm OS® Programmer's API Reference

CONTRIBUTORS

Written by Christopher Bey, Elly Freeman, Gary Hillerson, Jean Ostrem, Ruben Rodriguez, and Greg Wilson
Production by <dot>**PS** Document Production Services
Engineering contributions by Jesse Donaldson, Noah Gibbs, Lee Taylor, Danny Epstein, Peter Epstein, Ludovic Ferrandis, Gilles Fabre, David Fedor, Roger Flores, Steve Lemke, Bob Ebert, Ken Krugler, Paul Plaquette, Bruce Thompson, Tim Wiegman, Gavin Peacock, Ryan Robertson, and Waddah Kudaimi

Copyright © 1996 - 2001, Palm, Inc. All rights reserved. This documentation may be printed and copied solely for use in developing products for Palm OS software. In addition, two (2) copies of this documentation may be made for archival and backup purposes. Except for the foregoing, no part of this documentation may be reproduced or transmitted in any form or by any means or used to make any derivative work (such as translation, transformation or adaptation) without express written consent from Palm, Inc.

Palm, Inc. reserves the right to revise this documentation and to make changes in content from time to time without obligation on the part of Palm, Inc. to provide notification of such revision or changes. PALM, INC. MAKES NO REPRESENTATIONS OR WARRANTIES THAT THE DOCUMENTATION IS FREE OF ERRORS OR THAT THE DOCUMENTATION IS SUITABLE FOR YOUR USE. THE DOCUMENTATION IS PROVIDED ON AN "AS IS" BASIS. PALM, INC. MAKES NO WARRANTIES, TERMS OR CONDITIONS, EXPRESS OR IMPLIED, EITHER IN FACT OR BY OPERATION OF LAW, STATUTORY OR OTHERWISE, INCLUDING WARRANTIES, TERMS, OR CONDITIONS OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND SATISFACTORY QUALITY.

TO THE FULL EXTENT ALLOWED BY LAW, PALM, INC. ALSO EXCLUDES FOR ITSELF AND ITS SUPPLIERS ANY LIABILITY, WHETHER BASED IN CONTRACT OR TORT (INCLUDING NEGLIGENCE), FOR DIRECT, INCIDENTAL, CONSEQUENTIAL, INDIRECT, SPECIAL, OR PUNITIVE DAMAGES OF ANY KIND, OR FOR LOSS OF REVENUE OR PROFITS, LOSS OF BUSINESS, LOSS OF INFORMATION OR DATA, OR OTHER FINANCIAL LOSS ARISING OUT OF OR IN CONNECTION WITH THIS DOCUMENTATION, EVEN IF PALM, INC. HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Palm Computing, Palm OS, Graffiti, HotSync, and Palm Modem are registered trademarks, and Palm III, Palm IIIe, Palm IIIx, Palm V, Palm Vx, Palm VII, Palm, Palm Powered, More connected., Simply Palm, the Palm logo, Palm Computing platform logo, Palm III logo, Palm IIIx logo, Palm V logo, and HotSync logo are trademarks of Palm, Inc. or its subsidiaries. All other product and brand names may be trademarks or registered trademarks of their respective owners.

IF THIS DOCUMENTATION IS PROVIDED ON A COMPACT DISC, THE OTHER SOFTWARE AND DOCUMENTATION ON THE COMPACT DISC ARE SUBJECT TO THE LICENSE AGREEMENT ACCOMPANYING THE COMPACT DISC.

Palm OS Programmer's API Reference
Document Number 3003-004
July 23, 2001
For the latest version of this document, visit
http://www.palmos.com/dev/tech/docs/.

Palm, Inc.
5470 Great America Pkwy.
Santa Clara, CA 95052
USA
www.palmos.com

Table of Contents

About This Document **xv**
Palm OS SDK Documentation xv
What This Volume Contains xv
Additional Resources xvi

Part I: User Interface

1 Application Launch Codes **3**
Launch Codes 6
Launch Flags 34

2 Palm OS Resources **35**
System Resources 35
Resource Types 37
Alert Resource 39
Button Resource 41
Check Box Resource 43
Field Resource 45
Form Resource 48
Form Bitmap Resource 50
Gadget Resource 51
Graffiti Shift Indicator Resource 52
Label Resource 53
List Resource 53
Menus and Menu Bars 55
Popup Trigger Resource 57
Push Button Resource 58
Repeating Button Resource 59
Scroll Bar Resource 61
Selector Trigger Resource 62
String Resource 63
Table Resource 64

3 Palm OS Events **67**
Event Data Structures 68
Event Reference 71

4 Notifications **97**
Notification Data Structures 99
Notification Reference 100

5 Categories **121**
Category Data Structures 121
Category Constants 122
Category Functions 123

6 Clipboard **139**
Clipboard Data Structures 139
Clipboard Functions 140

7 Controls **143**
Control Data Structures 143
Control Resources 153
Control Functions. 153

8 Date and Time Selector **171**
Date and Time Selections Data Structures 171
Date and Time Selection Functions 172

9 Fields **181**
Field Data Structures 181
Field Resources. 189
Field Functions 189

10 Find **227**
Find Functions 227

11 Forms **231**
Form Data Structures 231
Form Constants 247

Form Resources 247
Form Functions. 248
Application-Defined Functions 293

12 Graffiti Shift **297**
GraffitiShift Functions 297

13 Insertion Point **301**
Insertion Point Functions 301

14 Lists **305**
List Data Structures 305
List Resources 309
List Functions 309
Application-Defined Function 319

15 Menus **323**
Menu Data Structures 323
Menu Constants 333
Menu Resources 334
Menu Functions 334

16 Private Records **351**
Private Record Data Structures 351
Private Record Functions 352

17 Progress Manager **355**
Progress Manager Functions 355
Application-Defined Functions 360

18 Scroll Bars **365**
Scroll Bar Data Structures 365
Scroll Bar Resources. 369
Scroll Bar Functions 369

19 System Dialogs **375**
System Dialog Functions 375

20 Tables **377**
Table Data Structures 377
Table Constants 389
Table Resource 389
Table Functions 390
Application-Defined Functions 427

21 UI Color List **431**
UI Color Data Types 431
UI Color Functions 435

22 UI Controls **439**
UI Control Functions 439

23 Miscellaneous User Interface Functions **443**
Miscellaneous User Interface Data Structures 443
Miscellaneous User Interface Functions 447

Part II: System Management

24 Alarm Manager **455**
Alarm Manager Functions 455
Application-Defined Functions 459

25 Attention Manager **461**
Attention Manager Data Structures 461
Attention Manager Constants 471
Attention Manager Functions 474
Application-Defined Functions 485

26 Bitmaps **487**
Bitmap Data Structures 487
Bitmap Constants 496
Bitmap Resources 496
Bitmap Functions 497

27 Character Attributes **509**
Character Attribute Functions 509

28 Data and Resource Manager **513**
Data Manager Data Structures 513
Data Manager Constants. 514
Data Manager Functions. 522
Application-Defined Functions 583

29 Error Manager **585**
ERROR_CHECK_LEVEL Define 585
Error Manager Data Structures 586
Error Manager Functions 586

30 Expansion Manager **593**
Expansion Manager Data Structures. 593
Expansion Manager Constants 594
Expansion Manager Functions 596

31 Feature Manager **603**
Feature Manager Functions 603

32 File Streaming **609**
File Streaming Constants 609
File Streaming Functions. 611
File Streaming Error Codes. 627

33 Float Manager **629**
Float Manager Data Structures 629
Float Manager Functions 631

34 Fonts **643**
Font Functions 643

35 Graffiti Manager **655**
Graffiti Manager Functions 655

36 Helper API **665**
Helper Data Structures 665
Helper Constants 673

37 Key Manager **675**
Key Manager Functions 675

38 Locale Manager **677**
Locale Manager Data Types 677
Locale Manager Constants 679
Locale Manager Functions 682

39 Memory Manager **689**
Memory Manager Functions 689

40 Notification Manager **711**
Notification Constants. 711
Notification Functions. 712
Application-Defined Functions 719

41 Overlay Manager **721**
Overlay Manager Data Structures. 721
Overlay Manager Constants 722
Overlay Manager Functions 723

42 Password **731**
Password Functions. 731

43 Pen Manager **733**
Pen Manager Functions 733

44 Preferences **735**
Preferences Data Types 735
Preferences Constants 747
Preferences Functions 750

45 Rectangles **759**
Rectangle Data Structures 759
Rectangle Functions. 760

46 Sound Manager **765**
Sound Manager Data Structures 765
Sound Manager Functions 773
Application-Defined Functions 784

47 Standard IO **787**
Standard IO Functions 787
Standard IO Provider Functions 800
Application-Defined Function 803

48 String Manager **805**
String Manager Functions 805

49 System Event Manager **825**
System Event Manager Data Structures 825
System Event Manager Functions 825

50 System Manager **841**
System Manager Data Structures 841
System Functions 842
Application-Defined Functions 868

51 Text Manager **869**
Text Manager Data Structures 869
Text Manager Functions 870

52 Text Services Manager **909**
Text Services Manager Data Structures. 909
Text Services Manager Functions 910

53 Time Manager **913**
Time Manager Data Structures 913
Time Manager Constants 922
Time Manager Functions 923

54 Virtual File System Manager **941**
VFS Manager Data Structures 941
VFS Manager Constants 945
VFS Manager Functions 951
Application-Defined Functions 1007

55 Windows **1009**
Window Data Structures 1009
Window Functions 1021

56 Miscellaneous System Functions **1081**

Part III: Communications

57 Connection Manager **1091**
Connection Manager Data Types 1091
Connection Manager Constants. 1092
Connection Manager Functions. 1101

58 Exchange Manager **1123**
Exchange Manager Data Structures 1123
Exchange Manager Constants 1132
Exchange Manager Functions 1135
Application-Defined Functions 1175

59 Exchange Library **1179**
Exchange Library Functions 1179

60 IR Library **1195**
IR Library Data Structures 1195
IR Library Constants 1200
IR Stack Callback Events. 1202
IR Library Functions 1205
IAS Functions 1219
Application-Defined Functions 1227

61 Modem Manager **1229**
Modem Manager Functions 1229

62 Net Library **1231**
Net Library Data Structures 1231
Net Library Constants 1237
Net Library Functions 1238

63 Network Utilities **1303**
Network Utility Functions 1303

64 Script Plugin **1307**
Script Plugin Data Types 1307
Script Plugin Constants 1312
Script Plugin Functions 1314

65 Virtual Drivers **1319**
Driver Data Structures 1319
Driver Constants 1331
Virtual Driver-Defined Functions 1333
Serial Manager Queue Functions 1341

66 Serial Manager **1345**
Serial Manager Data Structures 1345
Serial Manager Constants 1351
Serial Manager Functions 1357
Serial Manager Application-Defined Functions 1382

67 Old Serial Manager **1383**
Serial Manager Data Structures 1383
Serial Manager Functions 1385

68 Serial Link Manager **1399**
Serial Link Manager Functions 1399

69 Telephony Basic Services **1407**
Telephony Service Types 1407
Telephony Data Structures 1409
Telephony Constants 1415
Telephony Functions 1423
Feature Support Functions 1461

70 Telephony Security and Configuration **1467**
Telephony Security and Configuration Data Structures 1467
Telephony Security and Configuration Constants 1470
Telephony Security and Configuration Functions 1471

71 Telephony Network **1481**
Telephony Network Data Structures 1481
Telephony Network Constants 1484
Telephony Network Functions 1485

72 Telephony Calls **1499**
Telephony Calls Data Structures 1499
Telephony Calls Functions 1504

73 Telephony SMS **1539**
Telephony SMS Data Structures 1539
Telephony SMS Constants 1566
Telephony SMS Functions 1569

74 Telephony Phone Book **1599**
Telephony Phone Book Data Structures 1599
Telephony Phone Book Constants 1604
Telephony Phone Book Functions 1605

Part IV: Libraries

75 Internet Library 1623
Internet Library Data Structures . . . 1623
Internet Library Constants . . . 1634
Internet Library Functions . . . 1636

76 PalmOSGlue Library 1671
PalmOSGlue Functions . . . 1672

77 SMS Exchange Library 1687
SMS Exchange Library Data Structures . . . 1687
SMS Exchange Library Constants . . . 1701

78 Personal Data Interchange Library 1705
PDI Library Data Structures . . . 1705
PDI Library Constants . . . 1709
PDI Library Functions . . . 1721

79 Unified Data Access Manager 1745
UDA Manager Data Structures . . . 1746
UDA Manager Constants . . . 1749
UDA Manager Functions . . . 1750
UDA Object Creation Functions . . . 1756

Part V: Appendixes

A System Use Only Functions 1761

B Compatibility Guide 1767
2.0 New Feature Set . . . 1768
3.0 New Feature Set . . . 1771
3.1 New Feature Set . . . 1775
3.2 New Feature Set . . . 1778

International Feature Set 1779
Japanese Feature Set 1781
Wireless Internet Feature Set 1781
New Serial Manager Feature Set 1784
Connection Manager Feature Set 1786
3.5 New Feature Set 1787
Notification Feature Set 1793
4.0 New Feature Set 1793
Expansion Manager Feature Set 1799
VFS Manager Feature Set 1800

C 1.0 Float Manager **1803**
Float Manager Functions 1803

Index **1809**

About This Document

Palm OS Programmer's API Reference is part of the Palm OS® Software Development Kit. This introduction provides an overview of SDK documentation, discusses what materials are included in this document, and what conventions are used.

Palm OS SDK Documentation

The following documents are part of the SDK:

Document	Description
Palm OS Programmer's API Reference	An API reference document that contains descriptions of all Palm OS function calls and important data structures.
Palm OS Programmer's Companion	A multi-volume guide to application programming for the Palm OS. This guide contains conceptual and "how-to" information that complements the Reference.
Constructor for Palm OS	A guide to using Constructor to create Palm OS resource files.
Palm OS Programming Development Tools Guide	A guide to writing and debugging Palm OS applications with the various tools available.

What This Volume Contains

This section provides an overview of this volume.

- Part I, "User Interface," documents the API contained in the header files in the `\Incs\Core\UI\` folder. This part contains chapters covering subjects such as application launch codes, user interface resources, events, and all window, form, and field object managers.

- Part II, "System Management," documents the API contained in the header files in the `\Incs\Core\System\` folder. This part contains chapters covering subjects such as the alarm manager, data and resource manager, feature manager, float manager, graffiti manager, key manager, memory manager, preferences manager, sound manager, string manager, and system manager.
- Part III, "Communications," documents the API related to communications, such as the exchange manager, IR library, net library, serial manager, and serial drivers.
- Part IV, "Libraries," documents the API contained in the header files in the `\Incs\Libraries\` folder. This part contains chapters covering the Internet Library and the Palm OS Glue library.

Additional Resources

- Documentation

  Palm publishes its latest versions of this and other documents for Palm OS developers at

  http://www.palmos.com/dev/tech/docs/

- Training

  Palm and its partners host training classes for Palm OS developers. For topics and schedules, check

  http://www.palmos.com/dev/tech/support/classes/

- Knowledge Base

  The Knowledge Base is a fast, web-based database of technical information. Search for frequently asked questions (FAQs), sample code, white papers, and the development documentation at

  http://www.palmos.com/dev/tech/kb/



Part I:
User Interface

1

Application Launch Codes

This chapter provides detailed information about the predefined application launch codes. Launch codes are declared in the header file `SystemMgr.h`. The associated parameter blocks are declared in `AppLaunchCmd.h`, `AlarmMgr.h`, `ExgMgr.h`, and `Find.h`.

Table 1.1 lists all Palm OS® standard launch codes. More detailed information is provided immediately after the table:

- Launch Codes
- Launch Flags

To learn what a launch code is and how to use it, see the chapter titled "Application Startup and Stop" in the *Palm OS Programmer's Companion*, vol. I.

Table 1.1 Palm OS Launch Codes

Code	Request
`scptLaunchCmdExecuteCmd`	Execute the specified Network login script plugin command.
`scptLaunchCmdListCmds`	Provide information about the commands that your Network script plugin executes.
`sysAppLaunchCmdAddRecord`	Add a record to a database.
`sysAppLaunchCmdAlarmTriggered`	Schedule next alarm or perform quick actions such as sounding alarm tones.
`sysAppLaunchCmdAttention`	Perform the action requested by the attention manager.

Table 1.1 Palm OS Launch Codes *(continued)*

Code	Request
sysAppLaunchCmdCardLaunch	Launch the application. This launch code signifies that the application is being launched from an expansion card.
sysAppLaunchCmdCountryChange	Respond to country change.
sysAppLaunchCmdDisplayAlarm	Display specified alarm dialog or perform time-consuming alarm-related actions.
sysAppLaunchCmdExgAskUser	Let application override display of dialog asking user if they want to receive incoming data via the Exchange Manager.
sysAppLaunchCmdExgGetData	Notify application that it should send data using the Exchange Manager.
sysAppLaunchCmdExgPreview	Notify application that it should display a preview using the Exchange Manager.
sysAppLaunchCmdExgReceiveData	Notify application that it should receive incoming data using the Exchange Manager.
sysAppLaunchCmdFind	Find a text string.
sysAppLaunchCmdGoto	Go to a particular record, display it, and optionally select the specified text.
sysAppLaunchCmdGoToURL	Launch an application and open a URL.
sysAppLaunchCmdInitDatabase	Initialize database.

Table 1.1 Palm OS Launch Codes *(continued)*

Code	Request
sysAppLaunchCmdLookup	Look up data. In contrast to sysAppLaunchCmdFind, a level of indirection is implied. For example, look up a phone number associated with a name.
sysAppLaunchCmdNormalLaunch	Launch normally.
sysAppLaunchCmdNotify	Notify about an event.
sysAppLaunchCmdOpenDB	Launch application and open a database.
sysAppLaunchCmdPanelCalledFromApp	Tell preferences panel that it was invoked from an application, not the Preferences application.
sysAppLaunchCmdReturnFromPanel	Tell an application that it's restarting after preferences panel had been called.
sysAppLaunchCmdSaveData	Save data. Often sent before find operations.
sysAppLaunchCmdSyncNotify	Notify applications that a HotSync® has been completed.
sysAppLaunchCmdSystemLock	Sent to the Security application to request that the system be locked down.
sysAppLaunchCmdSystemReset	Respond to system reset. No UI is allowed during this launch code.
sysAppLaunchCmdTimeChange	Respond to system time change.
sysAppLaunchCmdURLParams	Launch an application with parameters from the Web Clipping Application Viewer.

Launch Codes

This section provides supplemental information about launch codes. For some launch codes, it lists the parameter block, which in some cases provides additional information about the launch code.

sysAppLaunchCmdAddRecord

Add a record to an application's database.

This launch code is used to add a message to the Mail or iMessenger™ (on the Palm VII™ organizer) application's outbox. You pass information about the message such as address, body text, etc. in the parameter block. For iMessenger, you can set the `edit` field of the parameter block to control whether or not the iMessenger editor is displayed. Set it to `true` to display the editor or `false` not to display it.

For more information on sending messages via iMessenger, see "Sending Email Messages" on page 172 in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

IMPORTANT: Implemented for iMessenger only if Wireless Internet Feature Set is present. Implemented for Mail only on OS version 3.0 or later.

sysAppLaunchCmdAddRecord Parameter Block for Mail Application

Prototype

```
typedef enum {
   mailPriorityHigh,
   mailPriorityNormal,
   mailPriorityLow
} MailMsgPriorityType;

typedef struct {
   Boolean secret;
   Boolean signature;
   Boolean confirmRead;
   Boolean confirmDelivery;
   MailMsgPriorityType priority;
   UInt8 padding
   Char* subject;
```

```
   Char* from;
   Char* to;
   Char* cc;
   Char* bcc;
   Char* replyTo;
   Char* body;
} MailAddRecordParamsType;
```

Fields

`secret`	`True` means that the message should be marked secret.
`signature`	`True` means that the signature from the Mail application's preferences should be attached to the message.
`confirmRead`	`True` means that a confirmation should be sent when the message is read.
`confirmDelivery`	`True` means that a confirmation should be sent when the message is delivered.
`priority`	Message priority. Specify one of the `MailMsgPriorityType` enumerated types.
`padding`	Reserved for future use.
`subject`	Message's subject, a null-terminated string (optional).
`from`	Message's sender, a null-terminated string (not used on outgoing mail).
`to`	Address of the recipient, a null-terminated string (required).
`cc`	Addresses of recipients to be copied, a null-terminated string (optional).
`bcc`	Addresses of recipients to be blind copied, a null-terminated string (optional).
`replyTo`	Reply to address, a null-terminated string (optional).
`body`	The text of the message, a null-terminated string (required).

sysAppLaunchCmdAddRecord Parameter Block for iMessenger Application

Prototype

```
typedef struct {
   UInt16 category;
   Boolean edit;
   Boolean signature;
   Char *subject;
   Char *from;
   Char *to;
   Char *replyTo;
   Char *body;
} MsgAddRecordParamsType;
```

Fields

`category`	Category in which to place the message. Specify one of the following categories:
	`MsgInboxCategory`
	`MsgOutboxCategory`
	`MsgDeletedCategory`
	`MsgFiledCategory`
	`MsgDraftCategory`

`edit`	`True` means that the message should be opened in the editor. `False` means that the message should simply be placed into the outbox and the editor not opened. You can specify `true` only if the category is set to `MsgOutboxCategory`.
`signature`	`True` means that the signature from the iMessenger application preferences should be attached to the message.
`subject`	Message's subject, a null-terminated string (optional).
`from`	Message's sender, a null-terminated string (not used on outgoing mail).
`to`	Address of the recipient, a null-terminated string (required).

`replyTo`	Reply to address, a null-terminated string (optional).
`body`	The text of the message, a null-terminated string (required).

sysAppLaunchCmdAlarmTriggered

Performs quick action such as scheduling next alarm or sounding alarm.

This launch code is sent as close to the actual alarm time as possible. An application may perform any quick, non-blocking action at this time. Multiple alarms may be pending at the same time for multiple applications, and one alarm display shouldn't block the system and prevent other applications from receiving their alarms in a timely fashion. An opportunity to perform more time-consuming actions will come when `sysAppLaunchCmdDisplayAlarm` is sent.

sysAppLaunchCmdAlarmTriggered Parameter Block

Prototype

```
typedef struct SysAlarmTriggeredParamType {
   UInt32     ref;
   UInt32     alarmSeconds;
   Boolean    purgeAlarm;
   UInt8      padding;
} SysAlarmTriggeredParamType;
```

Fields

`-> ref`	The caller-defined value specified when the alarm was set with `AlmSetAlarm`.
`-> alarmSeconds`	The date/time specified when the alarm was set with `AlmSetAlarm`. The value is given as the number of seconds since 1/1/1904.
`<- purgeAlarm`	Upon return, set to `true` if the alarm should be removed from the alarm table. Use this as an optimization to prevent the application from receiving `sysAppLaunchCmdDisplayAlarm` if you don't wish to perform any other processing for this alarm. If you do want to receive the launch code, set this field to `false`.
`padding`	Not used.



New sysAppLaunchCmdAttention

Perform the action requested by the attention manager. This launch code is accompanied by a value of the `AttnCommand` type; this type specifies the set of possible commands that can be sent to the application that requested the alarm.

```
typedef UInt16 AttnCommand;
```

The following table lists the values that `AttnCommand` can assume.

Table 1.2 sysAppLaunchCmdAttention Commands

Constant	Value	Description
`AttnCommand_drawDetail`	`((AttnCommand) 1)`	Indicates that the application needs to draw the detailed contents of the attention dialog. The command arguments parameter points to a structure of type `AttnCommandDrawDetailArgsType`.
`AttnCommand_drawList`	`((AttnCommand) 2)`	Indicates that the application needs to draw the appropriate list item in the attention dialog. The command arguments parameter points to a structure of type `AttnCommandDrawListArgsType`.
`AttnCommand_customEffect`	`((AttnCommand) 3)`	Indicates that the Attention Manager is doing something to get the user's attention, and any application-specific special effect should be done. This command is only sent to attention items that set the `AttnFlags_CustomEffectBit` when they call `AttnGetAttention`, which most applications won't do.

Table 1.2 sysAppLaunchCmdAttention Commands

Constant	Value	Description
`AttnCommand_goThere`	`((AttnCommand) 4)`	Tells the application to navigate to the item. The command arguments parameter is `NULL`. An application commonly calls `SysAppLaunch` upon receipt of this command to have itself launched.
`AttnCommand_gotIt`	`((AttnCommand) 5)`	Tells the application that the user is dismissing the item. The command arguments parameter is `NULL`. The application may choose to clean up memory at this point.
`AttnCommand_snooze`	`((AttnCommand) 6)`	Indicates to the application that the user is snoozing. The command arguments parameter is `NULL`. Most applications do nothing upon receipt of this command. This command is passed to each and every item currently pending, insistent or subtle. Applications with more than one attention item pending are called more than once.
`AttnCommand_iterate`	`((AttnCommand) 7)`	This command is passed to the application during the enumeration of attention items. This command is particularly useful after HotSync operations, as it allows the application to examine each item, updating or removing those that are stale or invalid.

AttnCommandDrawDetailArgsType

When `AttnCommand_drawDetail` is passed to the application, either via the callback function or as a parameter accompanying the `sysAppLaunchCmdAttention` launch code, the application needs to draw the detailed contents of the attention dialog. The `AttnCommandDrawDetailArgsType` structure accompanies the `AttnCommand_drawDetail` command, and provides the information needed to draw the contents of that dialog.

```
typedef struct {
  RectangleType bounds;
   Boolean firstTime;
   AttnFlagsType flags;
} AttnCommandDrawDetailArgsType;
```

Field Descriptions

`bounds`	Contains the window-relative bounding box for the area to draw. The clipping region is also set to the dimensions of this box to prevent accidental drawing outside.
`firstTime`	Set to `true` if the user has not yet seen this item. The value of this field could be used, for example, to display attentions that the user hasn't seen before in some special way.
`flags`	The global user preferences for this attention attempt combined with the custom flags passed in by the developer. For example, if the global preference is to mute sounds, and the developer flags are both zero, then the `AttnFlags_NoSound` flag is on and the `AttnFlags_AlwaysSound` flag is off.

AttnCommandDrawListArgsType

When `AttnCommand_drawList` is passed to the application, either via the callback function or as a parameter accompanying the `sysAppLaunchCmdAttention` launch code, the application is to draw the appropriate list item in the attention dialog. The `AttnCommandDrawListArgsType` structure accompanies the `AttnCommand_drawList` command, and provides the information needed to draw the contents of that dialog.

```
typedef struct {
  RectangleType bounds;
  Boolean firstTime;
  AttnFlagsType flags;
} AttnCommandDrawListArgsType;
```

Field Descriptions

`bounds`	Contains the window-relative bounding box for the area to draw. The clipping region is also set to the dimensions of this box to prevent accidental drawing outside.
`firstTime`	Set to `true` if the user has not yet seen this item. The value of this field could be used, for example, to trigger a custom sound the first time this attention item is presented to the user.
`flags`	The global user preferences for this attention attempt combined with the custom flags passed in by the developer. For example, if the global preference is to mute sounds, and the developer flags were both zero, then the `AttnFlags_NoSound` flag is on and the `AttnFlags_AlwaysSound` flag is off.

AttnCommandGotItArgsType

When `AttnCommand_gotIt` is passed to the application, either via the callback function or as a parameter accompanying the `sysAppLaunchCmdAttention` launch code, it is accompanied by an `AttnCommandGotItArgsType` structure. This structure indicates whether the `AttnCommand_gotIt` command was generated because the user dismissed the attention, or whether the system is simply informing your application that `AttnForgetIt` was called. Your application normally ignores the latter case if your application made the call to `AttnForgetIt`.

```
typedef struct {
  Boolean dismissedByUser;
} AttnCommandGotItArgsType;
```

Field Descriptions

`dismissedByUser`	`true` indicates that the user dismissed the attention. `false` indicates that the `AttnCommand_gotIt` command was generated by a call to `AttnForgetIt`.

New sysAppLaunchCmdCardLaunch

This launch code is sent to applications that are being run from an expansion card. The application is copied into the device's main memory prior to being sent this launch code. If the application doesn't respond to `sysAppLaunchCmdCardLaunch`, it is then sent a `sysAppLaunchNormalLaunch` launch code. Applications that can profit from the knowledge that they are being launched from an expansion card may want to consult the fields in the parameter block that accompanies `sysAppLaunchCmdCardLaunch`.

When the Launcher sends `sysAppLaunchCmdCardLaunch` to an application, it also sends `sysAppLaunchFlagNewGlobals` and `sysAppLaunchFlagUIApp` flags. These two flags are not sent to `start.prc`, however. Applications should never interact with the user upon receiving this launch code, and should not depend on

globals being available. This launch code is intended to notify the application that it is being launched from a card. Applications typically save some state information upon receiving this launch code and do the bulk of their processing when they receive `sysAppLaunchNormalLaunch`.

sysAppLaunchCmdCardLaunch Parameter Block

Prototype

```
typedef struct {
   Err     err;
   UInt16  volRefNum;
   const Char *path;
   UInt16  startFlags;
   } SysAppLaunchCmdCardType;
```

Fields

`<- err` Initially set to `expErrUnsupportedOperation`, applications that recognize `sysAppLaunchCmdCardLaunch` and that don't want to receive the subsequent `sysAppLaunchNormalLaunch` launch code should set this field to `errNone`.

`-> volRefNum` The reference number of the volume from which the application is being launched.

`-> path` The complete path to the application being launched.

`<-> startFlags` This field is made up of a combination of the following flags.

`sysAppLaunchStartFlagAutoStart`
: Indicates that the application is being run automatically upon card insertion.

`sysAppLaunchStartFlagNoUISwitch`
: Set this bit to prevent a UI switch to the application.

`sysAppLaunchStartFlagNoAutoDelete`
: Set this bit to prevent the VFS Manager from deleting the copy of the application in main memory when the associated volume is unmounted.

sysAppLaunchCmdCountryChange

Responds to country change.

Applications should change the display of numbers to use the proper number separators. To do this, call LocGetNumberSeparators, StrLocalizeNumber, and StrDelocalizeNumber.

sysAppLaunchCmdDisplayAlarm

Performs full, possibly blocking, handling of alarm.

This is the application's opportunity to handle an alarm in a lengthy or blocking fashion. Alert dialogs are usually displayed when this launch code is received. This work should be done here, not when sysAppLaunchCmdAlarmTriggered is received. Multiple alarms may be pending at the same time for multiple applications, and one alarm display shouldn't block the system and prevent other applications from receiving their alarms in a timely fashion.

sysAppLaunchCmdDisplayAlarm Parameter Block

Prototype

```
typedef struct SysDisplayAlarmParamType {
   UInt32     ref;
   UInt32     alarmSeconds;
   Boolean    soundAlarm;
   UInt8      padding;
   } SysDisplayAlarmParamType;
```

Fields

`-> ref` The caller-defined value specified when the alarm was set with AlmSetAlarm.

`-> alarmSeconds` The date/time specified when the alarm was set with AlmSetAlarm. The value is given as the number of seconds since 1/1/1904.

`-> soundAlarm` true if the alarm should be sounded, false otherwise. This value is currently not used.

`padding` Not used.

sysAppLaunchCmdExgAskUser

The Exchange Manager sends the `sysAppLaunchCmdExgAskUser` launch code to the application when data has arrived for that application. This launch code allows the application to tell the Exchange Manager not to display the exchange dialog, which it uses to have the user confirm the receipt of data. If the application does not handle this launch code, the default course of action is that the Exchange Manager displays the exchange dialog.

Applications may want to respond to this launch code under these circumstances:

- To reject all incoming data or to reject data under certain circumstances without first prompting the user. To reject incoming data, set the `result` field of the parameter block to `exgAskCancel` and then return.
- To receive incoming data without confirmation. To automatically receive incoming data, set the result field to `exgAskOk`.
- To provide a user confirmation dialog with extra functionality. This is described in more detail below.

Starting with Palm OS 3.5, the Exchange Manager allows applications to provide extra functionality in the exchange dialog. You can have the dialog include a category pop-up list from which the user chooses a category in which to file the incoming data. If you want to provide a category pop-up list, call the `ExgDoDialog` function in response to this launch code and pass it a database that contains the categories to be listed. See the description of that function for more information.

Applications may also bypass the call to `ExgDoDialog` altogether and provide their own dialogs.

If an application responds to this launch code, it must set the `result` field in the parameter block to the appropriate value. Possible values are:

`exgAskDialog`	Display the default exchange dialog provided by Exchange Manager.
`exgAskOk`	Accept the incoming data.
`exgAskCancel`	Reject the incoming data.

On Palm OS 3.5 or higher if you don't use the default version of the dialog, return `exgAskOk` if the user confirmed or `exgAskCancel` if the user canceled. If you don't set the `result` field properly, two dialogs are displayed.

IMPORTANT: Implemented only if 3.0 New Feature Set is present.

sysAppLaunchCmdExgAskUser Parameter Block

Prototype

```
typedef struct {
   ExgSocketPtr          socketP;
   ExgAskResultType  result;
   UInt8                 reserved;
   } ExgAskParamType;
```

Fields

`<-> socketP`	Socket pointer (see `ExgSocketType`)
`<- result`	Show dialog, auto-confirm, or auto-cancel
`-> reserved`	Reserved for future use



New sysAppLaunchCmdExgGetData

The Exchange Manager sends the `sysAppLaunchCmdExgGetData` launch code when the exchange library requests data to be sent to a remote device. That is, an application on a remote device has performed an `ExgGet` function to request data, and the Exchange Manager has determined that the launched application should handle the request.

To respond to this launch code, applications should initiate a connection with `ExgPut`, use `ExgSend` to send the data, and call `ExgDisconnect` when finished.

The parameter block sent with this launch code is a pointer to the `ExgSocketType` structure corresponding to the Exchange Manager connection on which the data is to be sent. You pass this socket pointer to `ExgPut`. For more details, see the "Exchange Manager" chapter.

IMPORTANT: Implemented only if 4.0 New Feature Set is present.



sysAppLaunchCmdExgPreview

Following the launch code `sysAppLaunchCmdExgAskUser`, the Exchange Manager sends the `sysAppLaunchCmdExgPreview` launch code to have the application display the preview in the exchange dialog.

sysAppLaunchCmdExgPreview Parameter Block

Prototype

```
typedef struct {
   UInt16             version;
   ExgSocketType      *socketP;
   UInt16             op;
   Char               *string;
   UInt32             size;
   RectangleType      bounds;
   UInt16             types;
   Err                error;
} ExgPreviewInfoType;
```

Fields

`-> version` Set this field to 0 to specify version 0 of this structure.

`-> socketP` A pointer to the socket structure (see `ExgSocketType`). The `libraryRef` field must point to the exchange library from which preview data should be received.

`-> op` A constant that identifies the operation. This can be one of the following:

`exgPreviewDialog`
: Display a form or modal dialog containing the preview. This constant is only used in situations where one application launches another to display data.

	`exgPreviewDraw`
	Draw the preview as a graphic in the `bounds` rectangle.
	`exgPreviewLongString`
	Return the preview as a long string in the `string` field.
	`exgPreviewQuery`
	Return the list of preview modes the application supports in the `types` field.
	`exgPreviewLongString`
	Return the preview as a short string in the `string` field.

<- `string`	A buffer into which the preview string is placed if one of the string preview operations is specified.
-> `size`	The allocated size of the `string` field.
-> `bounds`	The bounds of the rectangle in which to draw the graphic if the preview operation is `exgPreviewDraw`.
<- `types`	Upon return from `exgPreviewQuery`, a bit field identifying the types of previews the library supports (`exgPreviewDraw`, `exgPreviewLongString`, or `exgPreviewShortString`).
<- `error`	The error code returned from the library. If this is `errNone`, the preview operation was successful.

Applications that respond to this launch code should check the parameter block's `op` field and respond as described above.

Applications can define and use their own constants for the preview operation. Operations specific to an application are numbered starting at `exgPreviewFirstUser` and should be no greater than `exgPreviewLastUser`.

Applications respond to this launch in much the same way they respond to `sysAppLaunchCmdExgReceiveData`. Use `ExgAccept` to accept the preview connection, `ExgReceive` to

receive the data, and then ExgDisconnect to end the connection. The only difference is what is done with the data when it is received. With this launch code, the application should return the data in the `string` field or draw it in the `bounds` rectangle. With the `sysAppLaunchCmdExgReceiveData` launch code, the application stores the received data.

IMPORTANT: Implemented only if 4.0 New Feature Set is present.

sysAppLaunchCmdExgReceiveData

The Exchange Manager sends the sysAppLaunchCmdExgReceiveData launch code following the sysAppLaunchCmdExgAskUser and sysAppLaunchCmdExgPreview launch codes to notify the application that it should receive the data (assuming that the application and/or the user has indicated the data should be received).

The application should use Exchange Manager functions to receive the data and store it or do whatever it needs to with the data. Specifically, most applications should respond to this launch code by calling ExgAccept to accept the connection and then ExgReceive to receive the data.

Note that the application may not be the active application, and thus may not have globals available when it is launched with this launch code. You can check if you have globals by using this code in the PilotMain routine:

```
Boolean appIsActive = launchFlags & sysAppLaunchFlagSubCall;
```

The `appIsActive` value is `true` if your application is active and globals are available; otherwise, you won't be able to access any of your global variables during the receive operation.

The parameter block sent with this launch code is a pointer to the ExgSocketType structure corresponding to the Exchange Manager connection on which the data is arriving. Pass this pointer

to the `ExgAccept` function to begin receiving the data. For more details, refer to the "Exchange Manager" chapter.

IMPORTANT: Implemented only if 3.0 New Feature Set is present.

sysAppLaunchCmdFind

This launch code is used to implement the global find. When the user enters a text string in the Find dialog, the system sends this launch code with the `FindParamsType` parameter block to each application. The application should search for the string that the user entered and return any records matching the find request.

The system displays the results of the query in the Find results dialog. The system continues the search with each application until it has a full screen of matching records or until all of the applications on the device have had a chance to respond. If the screen is full, a Find More button appears at the bottom of the dialog. If the user clicks the Find More button, the search resumes. Applications can test whether the current find is a continuation of a previous one by checking the `continuation` field in the parameter block.

Most applications that use text records should support this launch code. When they receive it, they should search all records for matches to the find string and return all matches. Functions that you can use to respond to this launch code are described in Chapter 10, "Find."

An application can also integrate the find operation in its own user interface and send the launch code to a particular application.

Applications that support this launch code should support `sysAppLaunchCmdSaveData` and `sysAppLaunchCmdGoto` as well.

sysAppLaunchCmdFind Parameter Block

Prototype

```
typedef struct {
   UInt16        dbAccesMode;
   UInt16        recordNum;
   Boolean       more;
```

```
   Char          strAsTyped [maxFindStrLen+1];
   Char          strToFind [maxFindStrLen+1];
   UInt8         reserved1;
   UInt16        numMatches;
   UInt16        lineNumber;
   Boolean       continuation;
   Boolean       searchedCaller;
   LocalID       callerAppDbID;
   UInt16        callerAppCardNo;
   LocalID       appDbID;
   UInt16        appCardNo;
   Boolean       newSearch;
   UInt8         reserved2;
   DmSearchStateType       searchState;
   FindMatchType           match [maxFinds];
} FindParamsType;
```

Fields

`dbAccesMode` Mode in which to open the application's database. Pass this directly to `DmOpenDatabase` as the `mode` parameter. Its value is either `dmModeReadOnly` or `dmModeReadOnly | dmModeShowSecret`. (See "Open Mode Constants" for more information.)

`recordNum` Index of last record that contained a match. Start the search from this location. Do not set this value directly. Instead, call `FindSaveMatch` when you have a matching record.

`more` If `true`, the Find results dialog displays the Find More button indicating that there may be more results to display.

Typically `FindSaveMatch` handles setting the `more` field. Applications with large databases to search may want to periodically check for and stop the search if an event is pending. If so, they should set this field to `true` before stopping.

`strAsTyped` Search string as the user entered it.

`strToFind`	Normalized version of the search string. The method by which a search string is normalized varies depending on the version of Palm OS and the character encoding supported by the device. You pass `strToFind` directly to the search function (either `FindStrInStr`, `TxtFindString`, or `TxtGlueFindString`).
`reserved1`	Reserved for future use.
`numMatches`	The current number of matches. Do not set this field directly. Instead, call `FindSaveMatch`, which increments it for you.
`lineNumber`	Line number of the next line that displays the results. Do not set this field directly. It is incremented by a call to `FindDrawHeader`.
`continuation`	If `true`, the launch code is being sent as part of a continuation of a previous Find. If `false`, this is a new Find. Do not set this field directly; the system sets it when the Find results dialog is full.
`searchedCaller`	If `true`, the application that was active at the time the user tapped the Find button has responded to this launch code. This application is always searched before any others.
`callerAppDbID`	Database ID of the application that was active when the user tapped the Find button. Do not change the value of this field; the system sets it and uses it when searching for application databases.
`callerAppCardNo`	Card number of the application that was active when the user tapped the Find button. Do not change the value of this field; the system sets it and uses it when searching for application databases.

`appDbID`	The ID of your application's resource database. Do not set this field directly; the system sets it and uses it when searching for application databases.
`appCardNo`	The card number of your application's resource database. Do not set this field directly; the system sets it and uses it when searching for application databases.
`newSearch`	System use only.
`reserved2`	Reserved for future use.
`searchState`	System use only.
`match`	System use only.

sysAppLaunchCmdGoto

Sent in conjunction with `sysAppLaunchCmdFind` or `sysAppLaunchCmdExgReceiveData` to allow users to actually inspect the record that the global find returned or that was received by the Exchange Manager.

Applications should do most of the normal launch actions, then display the requested item. The application should continue running unless explicitly closed.

An application launched with this code does have access to global variables, static local variables, and code segments other than segment 0 (in multi-segment applications).

Applications that receive this launch code should test the `sysAppLaunchFlagNewGlobals` launch flag to see if they need to initialize global variables. `sysAppLaunchFlagNewGlobals` indicates that the system has just allocated your global variables.

For example:

```
case sysAppLaunchCmdGoTo:
  if (launchFlags & sysAppLaunchFlagNewGlobals)
    StartApplication();
```

Note that you shouldn't automatically initialize the global variables in response to this launch code. Test the launch flag first. Your application receives this launch code when the user selects a record

in the global find results. If your application was the current application before the user selected the Find command, the launch flag is clear to indicate that your globals should not be re-initialized.

sysAppLaunchCmdGoto Parameter Block

Prototype

```
typedef struct {
   Int16      searchStrLen;
   UInt16     dbCardNo;
   LocalID    dbID;
   UInt16     recordNum;
   UInt16     matchPos;
   UInt16     matchFieldNum;
   UInt32     matchCustom;
} GoToParamsType;
```

Fields

`searchStrLen`	Length of normalized search string. This is **not** the length of the matching string. See below for a full explanation.
`dbCardNo`	Card number of the database to open.
`dbID`	Local ID of the database to open.
`recordNum`	Index of record containing a match.
`matchPos`	Position of the match within the field.
`matchFieldNum`	Field number string was found in.
`matchCustom`	Application-specific information.

Often, applications highlight the search string when displaying the resulting record. Localizable applications commonly store the length of the string to select in the `matchCustom` field for this purpose. Some multi-byte character encodings represent certain characters both as a single-byte character and a multi-byte character. When the search is performed, the single-byte character is accurately matched against its multi-byte equivalent. For this reason, the length of the string searched for does not always equal the length of the matching string. Applications that support being localized to multi-byte character sets often set the `matchCustom` field to the length of the matching string in the call to `FindSaveMatch` so that they know the length of the string to select.

sysAppLaunchCmdGoToURL

Applications can respond to this launch code to retrieve and display the specified URL.

The parameter block for this launch command is simply a pointer to a string containing the URL.

This launch code may be sent in the following instances:

- If the Wireless Internet Feature Set is, applications can send this launch code directly to the Web Clipping Application Viewer application.
- If 4.0 New Feature Set is present, the `ExgRequest` function launches an application with this launch code if it cannot find an exchange library that is registered for the URL it has received. To receive the launch code, the application must first use `ExgRegisterDatatype` to register for a URL scheme.

IMPORTANT: Implemented only if Wireless Internet Feature Set is present.

sysAppLaunchCmdInitDatabase

This launch code is sent by the Desktop Link server in response to a request to create a database. It is sent to the application whose creator ID matches that of the requested database.

The most frequent occurrence of this is when a `'data'` database is being installed or restored from the desktop. In this case, HotSync creates a new database on the device and passes it to the application via a `sysAppLaunchCmdInitDatabase` command, so that the application can perform any required initialization. HotSync will then transfer the records from the desktop database to the device database.

When a Palm OS application crashes while a database is installed using HotSync, the reason may be that the application is not handling the `sysAppLaunchCmdInitDatabase` command properly. Be especially careful not to access global variables.

The system will create a database and pass it to the application for initialization. The application must perform any initialization required, then pass the database back to the system, unclosed.

sysAppLaunchCmdInitDatabase Parameter Block

Prototype

```
typedef struct {
   DmOpenRef     dbP;
   UInt32        creator;
   UInt32        type;
   UInt16        version;
} SysAppLaunchCmdInitDatabaseType;
```

Fields

`dbP`	Database reference.
`creator`	Database creator.
`type`	Database type.
`version`	Database version.

sysAppLaunchCmdLookup

The system or an application sends this launch command to retrieve information from another application. In contrast to Find, there is a level of indirection; for example, this launch code could be used to retrieve the phone number based on input of a name.

This functionality is currently supported by the standard Palm OS Address Book.

Applications that decide to handle this launch code must search their database for the string the user entered and perform the match operation specified in the launch code's parameter block.

If an application wants to allow its users to perform lookup in other applications, it has to send it properly, including all information necessary to perform the match. An example for this is in `Address.c` and `AppLaunchCmd.h`, which are included in your SDK.

sysAppLaunchCmdLookup Parameter Block

The parameter block is defined by the application that supports this launch code. See `AppLaunchCmd.h` for an example.

IMPORTANT: Implemented only if 2.0 New Feature Set is present.

sysAppLaunchCmdNotify

The system or an application sends this launch code to notify applications that an event has occurred. The parameter block specifies the type of event that occurred, as well as other pertinent information. To learn which notifications are broadcast by the system, see the chapter titled "Notifications" in this book.

IMPORTANT: Implemented only if Notification Feature Set is present.

sysAppLaunchCmdNotify Parameter Block

The `SysNotifyParamType` structure declared in `NotifyMgr.h` defines the format of this launch code's parameter block. See its description in the "Notifications" chapter.

sysAppLaunchCmdOpenDB

You can send this launch code to the Web Clipping Application Viewer application to launch the application and cause it to open and display a Palm™ query application stored on the device. This is the same mechanism that the Launcher uses to launch query applications.

IMPORTANT: Implemented only if Wireless Internet Feature Set is present.

sysAppLaunchCmdOpenDB Parameter Block

Prototype

```
typedef struct {
   UInt16     cardNo;
   LocalID    dbID;
   } SysAppLaunchCmdOpenDBType;
```

Fields

`cardNo`	Card number of database to open.
`dbID`	Database id of database to open.

sysAppLaunchCmdPanelCalledFromApp

`sysAppLaunchCmdPanelCalledFromApp` and `sysAppLaunchCmdReturnFromPanel` allow an application to let users change preferences without switching to the Preferences application. For example, for the calculator, you may launch the Formats preferences panel, set up a number format preference, then directly return to the calculator that then uses the new format.

`sysAppLaunchCmdPanelCalledFromApp` lets a preferences panel know whether it was switched to from the Preferences application or whether an application invoked it to make a change. The panel may be a preference panel owned by the application or a system preferences panel.

Examples of these system panels that may handle this launch code are:

- Network panel (called from network applications)
- Modem panel (called if modem selection is necessary)

All preferences panels must handle this launch code. If a panel is launched with this command, it should:

- Display a Done button.
- **Not** display the panel-switching pop-up trigger used for navigation within the preferences application.

IMPORTANT: Implemented only if 2.0 New Feature Set is present.

sysAppLaunchCmdReturnFromPanel

This launch code is used in conjunction with `sysAppLaunchCmdPanelCalledFromApp`. It informs an application that the user is done with a called preferences panel. The system passes this launch code to the application when a previously-called preferences panel exists.

IMPORTANT: Implemented only if 2.0 New Feature Set is present.

sysAppLaunchCmdSaveData

Instructs the application to save all current data. For example, before the system performs a global find, an application should save all data.

Any application that supports the Find command and that can have buffered data should support this launch code. The system sends this launch code to the currently active application before it begins the search. The application receiving this launch code should respond by saving all buffered data so that the search is able to find matches in the text just entered.

sysAppLaunchCmdSaveData Parameter Block

Prototype

```
typedef struct {
   Boolean    uiComing;
   UInt8      reserved1;
} SysAppLaunchCmdSaveDataType;
```

Fields

`uiComing`	`true` if the system dialog is displayed before launch code arrives.
`reserved1`	Reserved for future use.

sysAppLaunchCmdSyncNotify

This launch code is sent to applications to inform them that a HotSync operation has occurred.

This launch code is sent only to applications whose databases were changed during the HotSync operation. (Installing the application database itself is considered a change.) The record database(s) must have the same creator ID as the application in order for the system to know which application to send the launch code to.

This launch code provides a good opportunity to update, initialize, or validate the application's new data, such as resorting records, setting alarms, and so on.

Because applications only receive `sysAppLaunchCmdSyncNotify` when their databases are updated, this launch code is not a good place to perform any operation that must occur after every HotSync operation. Instead, you may register to receive the `sysNotifySyncFinishEvent` on systems that have the Notification Feature Set. This notification is sent at the end of a HotSync operation, and it is sent to all applications registered to receive it, whether the application's data changed or not. Note that there is also a `sysNotifySyncStartEvent` notification.

sysAppLaunchCmdSystemLock

Launch code sent to the system-internal security application to lock the device.

As a rule, applications don't need to do respond to this launch code. If an application replaces the system-internal security application, it must handle this launch code.

IMPORTANT: Implemented only if 2.0 New Feature Set is present.

sysAppLaunchCmdSystemReset

Launch code to respond to system soft or hard reset.

Applications can respond to this launch code by performing initialization, indexing, or other setup that they need to do when the system is reset. For more information about resetting the device, see "System Boot and Reset" in the *Palm OS Programmer's Companion*, vol. I.

sysAppLaunchCmdSystemReset Parameter Block

Prototype

```
typedef struct {
   Boolean    hardReset;
   Boolean    createDefaultDB;
} SysAppLaunchCmdSystemResetType;
```

Fields

`hardReset` `true` if system was `hardReset`. `false` if system was `softReset`.

`createDefaultDB`
: If `true`, application has to create default database.

sysAppLaunchCmdTimeChange

Launch code to respond to a time change initiated by the user.

Applications that are dependent on the current time or date need to respond to this launch code. For example, an application that sets alarms may want to cancel an alarm or set a different one if the system time changes.

On systems that have the Notification Feature Set, applications should register to receive the `sysNotifyTimeChangeEvent` notification instead of responding to this launch code. The `sysAppLaunchCmdTimeChange` launch code is sent to all applications. The `sysNotifyTimeChangeEvent` notification is sent only to applications that have specifically registered to receive it, making it more efficient than `sysAppLaunchCmdTimeChange`.

sysAppLaunchCmdURLParams

This launch code is sent from the Web Clipping Application Viewer application to launch another application.

The parameter block consists of a pointer to a special URL string, which the application must know how to parse. The string is the URL used to launch the application and may contain encoded parameters.

An application launched with this code may or may not have access to global variables, static local variables, and code segments other than segment 0 (in multi-segment applications). It depends on the URL that caused the Web Clipping Application Viewer to send this launch code. If this launch code results from a `palm` URL, then globals are available. If the launch code results from a `palmcall` URL, then globals are not available.

The best way to test if you have global variable access is to test the `sysAppLaunchFlagNewGlobals` launch flag sent with this launch code. If this is flag is set, then you have global variable access.

IMPORTANT: Implemented only if Wireless Internet Feature Set is present.

Launch Flags

When an application is launched with any launch command, it also is passed a set of launch flags.

An application may decide to ignore the flags even if it handles the launch code itself. For applications that decide to use the launch flags, the following table provides additional information:

Table 1.3 Launch Flags

Flag	Functionality
`sysAppLaunchFlagNewGlobals`	Set when the system has created and initialized a new globals world for the application. Implies new owner ID for memory chunks.
`sysAppLaunchFlagUIApp`	Set when a UI application is being launched.
`sysAppLaunchFlagSubCall`	Set when the application is calling its entry point as a subroutine call. This tells the launch code that it's OK to keep the A5 (globals) pointer valid through the call. If this flag is set, it indicates that the application is already running as the current application.

IMPORTANT: Applications should never set launch flags when sending a launch code to another application. They should only be set by the system. In particular, note that you should never pass `sysAppLaunchFlagNewGlobals` as a launch flag for `SysAppLaunch`. If you do and you make repeated calls to `SysAppLaunch`, the system eventually runs out of owner IDs, and the new application fails to launch.

2

Palm OS Resources

Palm OS® User Interface resources are the elements of an application's GUI (graphical user interface). This chapter provides reference material you can use when creating user interface resources in Metrowerks Constructor. It provides detailed guidelines for using each resource, and it provides descriptions of the attributes you set in Metrowerks Constructor.

NOTE: For more information see the following manuals:
The *Palm OS Tutorial* provides more detailed instruction on how to create a GUI using the Constructor tool.
The *Constructor for Palm OS* manual in the CodeWarrior Documentation folder provides detailed reference-style documentation as well as information on how to use each individual resource.

System Resources

Every application running under Palm OS must have certain minimum system (not UI) resources defined to be recognized by the Palm OS system software. These required resources are created for your application by the development environment. You may find that you need additional, application-specific resources. The required resources are 'code' #1, 'code' #0, and 'data' #0. All other system resources are optional. This section describes both the required and optional resources.

The 'code' #1 Resource

The system creates a `'code'` #1 resource for every application. This resource is the entry point for the application and is where application initialization is performed. When the Palm Powered™ device launches an application, it starts executing at the first byte of

the ‘code’ #1 resource. All of the application code that you provide is included in this resource as well.

Typically, some startup code provided with the Palm OS development environment is linked in with your application code. This startup code works as follows:

- The startup code performs application setup and initialization.
- The startup code calls your `main` routine.
- When your `main` routine exits, control is returned to the startup code, which performs any necessary cleanup of your application and returns control to the Palm OS system software.

The ‘code’ #0 and ‘data’ #0 Resources

The `'code'` #0 and `'data'` #0 resources contain the required size of your global data and an image of the initialized area of that global data. When your application is launched, the system allocates a memory chunk in the dynamic heap that’s big enough to hold all your globals. The `'data'` #0 resource is then used to initialize those globals.

The ‘pref’ #0 Resource

The system creates a `'pref'` #0 resource for every application. This resource contains startup information for launching your application. The resource includes

- Required stack size
- Dynamic heap space required (not currently used)
- Task priority (not currently used)

This resource applies only to Palm OS 3.0 and higher. It is ignored on older versions of Palm OS.

Resource Types

Metrowerks Constructor divides resources into two types: catalog resources and project resources.

Catalog Resources

Catalog resources are available in Constructor's Catalog window and can be dragged directly on a form. Table 2.1 lists the available catalog resources. The Macintosh ResEdit resource name is included for reference only; it's not needed by developers who use Constructor exclusively, and not relevant for Windows developers.

Table 2.1 Catalog Resources

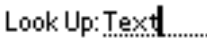
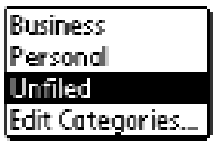
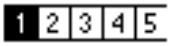
Name	Resource	Resource
`tBTN`	Button Resource	
`tCBX`	Check Box Resource	
`tFLD`	Field Resource	
`tFBM`	Form Bitmap Resource	(container for Bitmap resource)
`tGDT`	Gadget Resource	(application defined)
`tGSI`	Graffiti Shift Indicator Resource	
`tLBL`	Label Resource	(container for a String)

Table 2.1 Catalog Resources *(continued)*

Name	Resource	Resource
tLST	List Resource	
tPUT	Popup Trigger Resource	
tPBN	Push Button Resource	
tREP	Repeating Button Resource	
	Scroll Bar Resource	(see below)
tSLT	Selector Trigger Resource	
tTBL	Table Resource	

Project Resources

Project resources are instantiated from the projects window.

Table 2.2 lists the project resources. The Macintosh ResEdit resource name is included for reference only; it's not needed by developers who use Constructor exclusively, and not relevant for Windows developers.

Table 2.2 Project Resources

Name	Resource	UI Name
`Talt`	Alert Resource	Alert
`tFRM`	Form Resource	Form
	Menu Resource	Menu
	Menu bar Resource	Menu bar
`tSTR`	String Resource	String
	Icons	
	Bitmaps	

Alert Resource

Example



Overview

The alert resource defines a modal dialog that displays a message, an icon, and one or more buttons.

A small icon indicates the category of the dialog box; for example, an exclamation mark for an error message. The icon appears on the left side of the dialog. The text is justified left but placed to the right of the dialog icon.

Type	Icon	Definition	Button	Example
Information	i	Lowest-level warning. Action shouldn't or can't be completed but doesn't generate an error or risk data loss.	OK	An alarm setting must be between 1 and 99.
Confirmation	?	Confirm an action or suggest options.	OK, Cancel	Change settings before switching applications? (For example, when pressing an application key with an open dialog box.)
Warning	!	Ask if user wishes to continue a potentially dangerous action.	OK, Cancel	Are you sure you want to delete this entry?
Error	(stop sign)	Attempted action generated error and/or cannot be completed.	OK	Disk full.

The Alert resource has the following attributes.

Attributes

Alert Type — Determines the sound played and the icon displayed when the alert is drawn. There are four possible icons:

- InformationAlert (Alert Number 0)
- ConfirmationAlert (Alert Number 1)
- WarningAlert (Alert Number 2)
- ErrorAlert (Alert Number 3)

Help ID — The ID of a String resource that's the help text for the alert dialog box. If you provide a value, the system displays an "i" in the top right corner of the alert box.

Default Button ID	The number of a button that the system assumes is selected if the user switches to another application, forcing the form to go away without making a selection.
Title	Title of the alert form.
Message	Message displayed by the alert dialog. May contain ^1, ^2, ^3 as substitution variables for use in conjunction with `FrmCustomAlert`.
Button Text	Text of the button (e.g. OK or Cancel), determined by an entry in the resource of each button. To add a button, select Item Text 0, and type Cmd-K.

Button Resource

UI Structure `ControlType`

Overview A button is a clickable UI object, often used to trigger events in an application. A button displays as a text label surrounded by a rectangular frame. The frame has rounded corners. The label may be regular text or a glyph from one of the symbol fonts provided with your development environment, for example, an arrow.

Examples



Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Button ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of button. Valid values: 0 – 159
Top Origin	Form-relative position of top of object. Valid values: 0 – 159

Width	Width of button in pixels. Size the buttons to allow 3-6 pixels of white space at each end of the label. Valid values: 0 – 160
Height	Height of the button in pixels. Should be 3 pixels larger than the font size, for example, height =12 for 9-point labels. Valid values: 1 – 160
Usable	A nonusable object is not considered part of the application's interface and doesn't draw. Nonusable object can programmatically be set to usable. If checked, the object is usable.
Anchor Left	Controls how the object resizes itself when its text label is changed. If checked, the left bound of the object is fixed; if unchecked, the right bound is fixed.
Frame	If checked, a rectangular frame with rounded corners is drawn around the button. Most buttons have frames. Buttons whose labels are single symbol characters such as scroll buttons don't have frames.
Non-bold Frame	If checked, a one-pixel-wide rectangular frame with rounded corners is drawn around the button. If unchecked, a bold frame (two pixels wide) is drawn around the button. Non-bold frames are the default.
Font	Font used to draw the text label of the button. Choose from the pop-up menu to select one of the fonts.
Label	Text displayed inside the button: one line of text or a single character from a symbol font to create an increment arrow.

Comments

The label is centered in the button. If the label text is wider than the button, the whole label is centered and both the right and left sides are clipped.

Place command buttons at the bottom of table views and dialog boxes. Leave three pixels between the dialog bottom and buttons.

Increment arrows are a special case; they are buttons that let users increment the value displayed in a data field.

To create an increment arrow, use an arrow character from the Symbol font as a label. Several arrow styles and sizes are available.

Tip Making a Button with a Bitmap Label

It's not possible to make a bitmap the label of a button; the label always has to be a text string. However, the same effect can be achieved by

- Creating a bitmap the same size of a button
- Placing them at the same location.

Make sure the bitmap is a Form Bitmap, selected from the catalog.

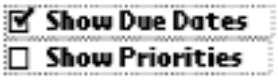
When the user selects the button, the system inverts the bitmap graphic as well.

Check Box Resource

UI Structure `ControlType`

Overview A check box is a small, square UI object with an optional text label to the right.

Example The figure below shows a checked and an unchecked check box with a label to the right (the default).



Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Check Box ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of object. Valid values: 0 – 159
Top Origin	Form-relative position of top of object. Valid values: 0 – 159

Width	Width of the picking area around the check box. Valid values: 0 – 160
Height	Height of the picking area around the check box. Valid values: 1– 160
Usable	If this box is checked, the object is usable. A nonusable object is not considered part of the application interface and doesn't draw. Nonusable objects can programmatically be set to usable.
Selected	Initial selection state of the checkbox. If the box is checked (the default), the checkbox is initially checked.
Group ID	Group ID of a check box that is part of an exclusive group. Ungrouped (nonexclusive) check boxes have 0 as a group ID. Valid values: 0 – 65535
Font	Font used to draw the text label of the button. Choose from the pop-up menu to select one of the fonts.
Label	Text displayed to the right of the check box. This text is part of the activation area. To create a (nonactive) label to the left of the check box, leave this attribute blank and create a separate Label resource.

Comments Make sure that only one check box in a group is initially checked.

All check boxes are the same size. The Height and Width determine the toggle area, which is the screen area the user needs to press to check or uncheck the box.

If a label attribute is defined, it's part of the activation area.

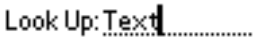
Field Resource

UI Structure `FieldType`

Overview The field UI object is for user data entry in an application. It displays one or more lines of text. A field can be underlined, justified left or right, and editable or uneditable.

Text fields can be located anywhere except in menus and in the command button area.

The following is an underlined, left-justified field containing data:



Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Field ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of object. Valid values: 0 – 159
Top Origin	Form-relative position of top of object. Valid values: 0 – 159
Width	Width of the object in pixels. Valid values: 0 – 160
Height	Height of the object in pixels. Valid values: 1– 160
Usable	If this box is checked, the object is usable. A nonusable object is not considered part of the application interface and doesn't draw. Nonusable objects can programmatically be set to usable.

Editable	If this box is checked, the field is editable. Noneditable fields don't accept user input but can be changed programmatically. Noneditable text fields are useful when you want to display text on a form but don't want users to edit it.
Underline	If set, each line of text is underlined with a gray line.
Single Line	If checked, the field doesn't scroll horizontally and doesn't accept Return or Tab characters. Only a single line of text is displayed. If the user attempts to enter text beyond this, the text display is clipped. In a multiline text field, the text wraps when its length on the line exceeds the display width of the field. An empty field may display one or more blank lines; for example, records in a To Do list or a text page.
Dynamic Size	If checked, a `fldHeightChanged` event is generated whenever the number of lines needs to increase or decrease. Your application needs to respond to this event by adjusting the size of the field's bounding box. Set this attribute to false if the Single Line attribute is set.

NOTE: The table code automatically handles the fldHeightChanged event if you have this attribute enabled for fields in tables.

Left Justified	Text justification. Supported only for fields that have the Single Line attribute checked. Valid values: checked (left-justified)—recommended unchecked (right-justified) Note that the user cannot enter Tabs into right-justified fields.
Max characters	Maximum number of bytes that the user can enter into the field. Note that the number of bytes is not the same as the number of characters when multi-byte characters (for example, Japanese) are being used. Valid values: 0 – 32767
Font	Font used to draw the text label of the button. Choose from the pop-up menu to select one of the fonts.
Auto-Shift	If checked, 2.0 (and later) auto-shift rules are applied.
Has Scrollbar	If checked, the field has a scroll bar. The system sends more frequent `fldHeightChangedEvent`s so the application can adjust the height appropriately.
Numeric	If checked, only the characters 0 through 9 and associated separators are allowed to be entered in the field. The associated separators are the thousands separator and the decimal character. The values of these two characters depend on the settings in the Formats prefs panel.

Form Resource

Overview

A form is a container for one or more of the Catalog Resources.

Applications usually contain several different forms that the user triggers by tapping buttons or other control UI objects. Most UI objects are displayed only if they are contained within a form.

Example

The example below shows a modal form. A form can also be as large as the screen.



Attributes

Left Origin	Window-relative position of left side of form. Valid values: 0 – 159
Top Origin	Window-relative position of top of form. Valid values: 0 – 159
Width	Width of the form in pixels. Valid values: 0 – 160
Height	Height of the form in pixels. Valid values: 1– 160
Usable	Not currently supported for forms.
Modal	If checked, form is modal. Modal forms ignore pen events outside their boundaries. Used for dialogs.
Save Behind	If checked, the region obscured by the form is saved when it's drawn and restored when it's erased. Used for dialogs.
Form ID	Form ID assigned by Constructor.

Help ID	ID number of a string that's displayed when the user taps the "i" icon. The system adds the icon to the form when you provide a value for this property. Currently, only modal dialogs have help resources.
Menu Bar ID	Contains the ID of a menu bar resource to be associated with this form.
Default Button ID	ID number of a button that the system assumes is selected if the user switches to another application, forcing the form to go away without making a selection.
Form Title	Title of that form. Use titles for dialogs, menu bars for views. By convention, the name of the application and the name of the screen, if possible, for example Address List or Address Edit. The title must be one line; it uses about 13 pixels of the top of the form.

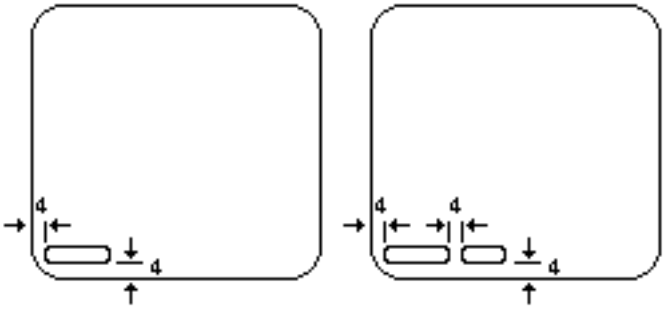
Comments

The total display on the Palm™ device is 160 pixels by 160 pixels. If you want your whole form to be seen, make sure it fits within this display area. For pop-up dialogs, you can make the form smaller. Align a popup dialog with the bottom of the screen.

Here are some general design guidelines:

- Each form should have a title that displays the name or view of the application, or both.
- Scroll bars in fields and tables appear and disappear dynamically if you've selected that option for that UI element. Place them to the right of command buttons.
- Dialogs always occupy the full width of the screen and are justified to the bottom of the screen.
- Modal dialogs hide the command buttons of the base application but don't obscure the title bar of the base application if possible. There should be a minimum of three pixels between the top of the modal dialog title bar and the bottom of the application title bar. If the dialog is too large to accommodate this, the entire application title bar should be obscured.

- Screen command buttons should always be at the bottom of the screen.
- Dialog command buttons appear four pixels above the bottom of the dialog box frame. The two-pixel default ring is three pixels above the bottom, and the baseline of the text within the buttons should be aligned.
- Command buttons should be grouped on the left, with four pixels between adjacent buttons as illustrated below. This four-pixel spacing can be reduced when necessary to accommodate large buttons or a large number of smaller buttons. Alternatively, the white space on either side of the button text can be reduced, making the button itself smaller. Either way, try to reduce the white space evenly so that the buttons don't look unbalanced.



- Command buttons should ideally be just wide enough to accommodate the text within, plus five pixels of white space on either side of the text. Because characters already include a single row of white pixels along the right-hand edge, the actual button size should usually be the width of the button text plus nine pixels (five on the left, four on the right), or `FntCharWidth(label) + 9` pixels wide.

Form Bitmap Resource

Overview Places predefined bitmaps on a given form. Used for icons in Alert dialogs to indicate a warning, error, information, and so on. You have to associate a Bitmap with the Form Bitmap to actually make a picture appear.

Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Left Origin.	Left bounds of bitmap.
Top Origin	Top bounds of bitmap.
Bitmap Resource ID	ID of a PICT resource containing the graphic. You can also assign an ID number, then click on Create and draw the picture in the bitmap editor that appears.
Usable	Checked if the bitmap should be drawn.

Gadget Resource

Name tGDT

UI Name Gadget

Overview A gadget object lets developers implement a custom UI gadget. The gadget resource contains basic information about the custom gadget, which is useful to the gadget writer for drawing and processing user input.

Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Gadget ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of object. Valid values: 0 – 159
Top Origin	Form-relative position of top of object. Valid values: 0 – 159

Width	Width of the gadget in pixels. Valid values: 0 – 160
Height	Height of the gadget in pixels. Valid values: 1– 160
Usable	If this box is checked, the object is usable. A nonusable object is not considered part of the application interface and doesn't draw. Nonusable objects can programmatically be set to usable.

Graffiti Shift Indicator Resource

Overview

The Graffiti® Shift Indicator resource specifies the window-relative or form-relative position of the Graffiti shift state indicator. The different shift states are punctuation, symbol, uppercase shift, and uppercase lock. These indicators will appear at the position of the Graffiti Shift resource.

For Japanese systems, the Graffiti Shift Indicator also indicates the front end processor modes, including on/off and Hiragana/ Katakana.

Note: By convention, Graffiti Shift indicators are placed at the bottom-right of every form that has an editable text field.

Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Left Origin	Form-relative position of left side of object. Valid values: 0 – 159
Top Origin	Form-relative position of top of object. Valid values: 0 – 159
Object ID	ID of the object (assigned by Constructor).

Label Resource

Overview The label resource displays noneditable text or labels on a form (dialog box or full-screen). It's used, for example, to have text appear to the left of a checkbox instead of the right.

Comments Pressing Return in a label wraps the text to the next line.

Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Label ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of object. Valid values: 0 – 159
Top Origin	Form-relative position of top of object. Valid values: 0 – 159
Usable	If this box is checked, the object is usable. A nonusable object is not considered part of the application interface and doesn't draw. Nonusable objects can programmatically be set to usable.
Font	Font used to draw the text label of the button. Choose from the pop-up menu to select one of the fonts.
Text	Text of the label.

List Resource

UI Structure `ListType`

Example



Overview

A list provides a box with a list of choices to the user. The list is scrollable if the choices don't all fit in the box.

The list box appears as a vertical list of choices surrounded by a rectangular frame. The current selection of the list is inverted. Arrows for scrolling the list appear in the right margin if necessary.

Lists can appear as popup lists when used with popup triggers. See Popup Trigger Resource.

Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
List ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of object. Valid values: 0 – 159
Top Origin	Form-relative position of top of object. Valid values: 0 – 159
Width	Width of the list. Valid values: 0 – 160
Usable	If this box is checked, the object is usable. A nonusable object is not considered part of the application interface and doesn't draw. Nonusable objects can programmatically be set to usable.
Font	Font used to draw the text label of the button. Choose from the pop-up menu to select one of the fonts.
Visible items	Height of the list box, in items (choices). For example, if the list has six items but only four fit, specify four.
Items	Items in the list.

Comments

Errors may occur if the number of visible items is greater than the actual number of items. An item's text is not clipped against the list box's borders. Set a list to not usable if it's linked to a popup trigger.

Use a list to let users choose between items of data; use a menu to activate a command.

If a list becomes too tall to fit below the trigger, it's justified up. If it becomes to large for the screen, it scrolls.

Menus and Menu Bars

Overview



A menu assembly consists of a menu bar, menu names indicating the available menus, and the menus themselves with their commands:

- **Menu bar**. The menu bar at the top of the screen contains the names of the available menus. Each application has different sets of menu names; within an application, different views may have different menus.
- **Menu name**. Each menu is displayed below the menu name. The following menu names are commonly found:
  - Record—Place Record to the left of Edit (if applicable).
  - Edit—Screens that allow editing need an Edit menu. Note, however, that most editing is edit-in-place.
  - Options—Typically, the last menu. The About command, which provides version and creator information, should always be an Options command under Palm OS.
- **Menu**. The menus themselves consist of menu items and optional shortcuts. Under Palm OS, menu items should **not** duplicate functionality available via command buttons. Menus justify left with the active heading of the menu name when invoked. If the menu doesn't fit, it's justified to the right border of the screen.

NOTE: For each menu, provide shortcuts for all commands or for none at all. Don't assign the same shortcut twice within one application.

Menu Bar and Menu Resources

The only information provided for the menu and menu bar resource is the resource name and resource ID.

Menu User Interaction

- **Default Menu and Menu Item.** A pen-up on the menu icon displays the menu bar. The first time a menu is invoked after an application is launched, no menus are displayed unless there is only one menu available. Afterwards the menu and menu item of the last command executed from the menu are displayed. Graffiti command equivalents are ignored.

  For example, if the user selects Edit > Copy, the Edit menu is popped down and the Copy command is highlighted the next time the menu bar is displayed. This expedites execution of commonly used commands or of grouped commands (e.g., Copy/Paste). The last menu heading is not saved if the user switches to a different view or a different application.

- **View-specific Menus.** Each view within an application can have a unique menu, that is, different menu headings and items.

- **Menu Display.** As a rule, a Palm OS application should try to have the menu visible on screen as rarely as possible:
  - After a menu command is executed, the menu bar is dismissed.
  - The menu bar is active when the menu headings in it are active. When not active, the menu bar is not visible.
  - There are no grayed-out menu headings or grayed-out menu items. A command not accessible in a certain mode doesn't appear at all.

- **Size.** The vertical active area of menu headings is 2 pixels beyond the ascender and 1 pixel below a potential descender of the menu heading text. The horizontal active area covers half the distance to the next menu heading, leaving no gaps between the headings. If the menu headings aren't as wide as the menu bar, part of the menu bar may be inactive.

- **Active Area.** The entire area of the menu, excluding the border, is active. Divider lines and status items on the launcher menu are inactive; that is, they do not highlight when tapped.

Popup Trigger Resource

UI structure `ControlType`

Overview The popup trigger shows the selection of a list. The user can press the popup trigger to pop up the list and change the selection.

A popup trigger displays a text label and a triangle to the left of the label that indicates the object is a popup trigger.

When the user selects a popup trigger, a list of items pops up.

Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Popup ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of button. Valid values: 0 – 159
Top Origin	Form-relative position of top of button. Valid values: 0 – 159
Width	Width of the button's picking area in pixels. Valid values: 1 – 160
Height	Height of the button's picking area in pixels. Valid values: 1 – 160
Usable	If this box is checked, the object is usable. A nonusable object is not considered part of the application interface and doesn't draw. Nonusable objects can programmatically be set to usable.
Left anchor	Controls how the object resizes itself when its text label is changed. Valid values: • checked (left bound fixed) • unchecked (right bound fixed)

Font	Font used to draw the text label of the button. Choose from the pop-up menu to select one of the fonts.
Label	Text displayed in the popup trigger (right of the arrow).
List ID	ID of the List object that pops up when the user taps the pop-up trigger.

Push Button Resource

UI Structure `ControlType`

Overview Push buttons allow users to select an option from a group of items. The choices should have few characters; if the choices are long; check boxes are preferable.

Push buttons display a text label surrounded by a 1-pixel-wide rectangular frame. They appear in a horizontal or vertical row with no pixels between the buttons. The buttons share a common border so there appears to be a one pixel line between two controls. The current selection is highlighted.





The List By dialog of the Address Book and the Details dialog of the ToDo List contain examples of rows of push buttons.

Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Button ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of button. Valid values: 0 – 159
Top Origin	Form-relative position of top of button. Valid values: 0 – 159

Width	Width of the button in pixels. Should be size of label plus two pixels at each end. Valid values: 1 – 160
Height	Height of the button in pixels. Should be font size plus two pixels. Valid values: 1 – 160
Usable	If this box is checked, the object is usable. A nonusable object is not considered part of the application interface and doesn't draw. Nonusable objects can programmatically be set to usable.
Group ID	Group ID of a push button that is part of an exclusive group. Only one push button in an exclusive group may be depressed at a time. Ungrouped (nonexclusive) push buttons have zero as a group ID. This feature must be enforced by the application. Valid values: 0 – 65535
Font	Font used to draw the text label of the button. Choose from the pop-up menu to select one of the fonts.
Label	Text displayed inside the push button.

Comment To create a row of push buttons, create a number of individual push button resources with the same height and align them by specifying the same top position for each button.

Repeating Button Resource

Overview The repeating button object is identical to the button object in its appearance. The repeating button is used for buttons that need to be triggered continuously by holding the pen down on them.

A good example for a repeating button is the scroll arrow, which moves text as long as it's held down.

Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Button ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of button. Valid values: 0 – 159
Top Origin	Form-relative position of top of button. Valid values: 0 – 159
Width	Width of the button in pixels. Valid values: 1 – 160
Height	Height of the button in pixels. Valid values: 1 – 160
Usable	If this box is checked, the object is usable. A nonusable object is not considered part of the application interface and doesn't draw. Nonusable objects can programmatically be set to usable.
Anchor Left	Controls how the object resizes itself when its text label is changed. If checked, the left bound of the object is fixed; if unchecked, the right bound is fixed.
Frame	If checked, a rectangular frame with rounded corners is drawn around the button.
Non-bold Frame	Determines the width of the rectangular frame drawn around the object. Valid values: • checked (1-pixel-wide frame) • unchecked (2-pixel-wide frame)
Font	Font used to draw the text label of the button. Choose from the pop-up menu to select one of the fonts.
Label	Text displayed inside the button.

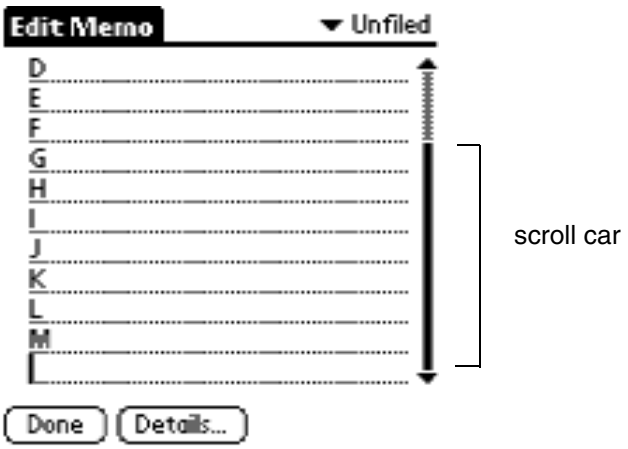
Comments The attributes match those of the Button Resource (tBTN); the behavior differs.

You can also use repeating buttons to create increment arrows. See Button Resource for more information.

Scroll Bar Resource

Overview The scroll bar resource helps developers to provide scrolling behavior for fields and tables.

Example



Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Scrollbar ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of the scroll bar. Valid values: 0 – 159
Top Origin	Form-relative position of top of the scroll bar. Valid values: 0 – 159
Width	Width of the scroll bar in pixels. 7 (the default) is strongly recommended.

Height	Height of the scroll bar in pixels. Valid values:1 – 160
Usable	If this box is checked, the object is usable. A nonusable object is not considered part of the application interface and doesn't draw. Nonusable objects can programmatically be set to usable.
Value	Current top value of the scroll bar's car (movable piece).
Min. Value	Position of the scroll car when the scroll bar is at the top. Default should usually be 0.
Max Value	Position of the scroll car when the scroll bar is at the bottom. To compute this value, use the formula: Number of lines – Page size + Overlap.
Page Size	Number of lines to scroll at one time.

Selector Trigger Resource

UI Structure `ControlType`

Overview Users can tap a selector trigger to pop up a dialog that lets them select an item. The selected item becomes the label of the selector trigger. For example, a selector trigger for time pops up a time selector. The selected time is entered into the selector trigger.

A selector trigger displays a text label surrounded by a gray rectangular frame, as shown below:



Attributes

Object Identifier	Name of the object. Assigned by developer and used by Constructor during header file generation and update.
Selector Trigger ID	ID of the object (assigned by Constructor).

Left Origin	Form-relative position of the left side of the object. Valid values: 0 – 159
Top Origin	Form-relative position of top of object. Valid values: 0 – 159
Width	Width of the object in pixels. Valid values: 1– 160
Height	Height of the object in pixels. Height extends two pixels above and one pixel below the 9-point plain font. Height is one pixel above command buttons to accommodate the gray frame. Valid values: 1– 160
Usable	If this box is checked, the object is usable. A nonusable object is not considered part of the application interface and doesn't draw. Nonusable objects can programmatically be set to usable.
Anchor Left	Controls how the object resizes itself when its text label is changed. If checked, the left bound of the object is fixed, if unchecked, the right bound is fixed. Valid values: • checked (left bound fixed) • unchecked (right bound fixed)
Font	Font used to draw the text label of the button. Choose from the pop-up menu to select one of the fonts.
Label	Text in the selector trigger.

String Resource

Name Strings

Overview Stores data strings used by the program. String resources may be entered as text strings or as a series of hexadecimal characters.

String	The text string to be stored. The string is stored using the character encoding you have specified for resource data.

Comments The string resource uses either the string or data. If both are entered, they are concatenated.

WARNING! It is risky to store data in a string resource, since Constructor and PalmRez both can transliterate string data in an attempt to map between different character sets. You should store data in other kinds of resources.

Table Resource

Overview The table object allows the developer to organize a collection of objects on the display. For example, a table might contain a column of labels that correspond to a column of fields. Under some circumstances, a one-column table may be preferable to a list.

Comments Since tables are scrollable, they may be larger than the display.

Example



Attributes

Object Identifier	Name of the object. Assigned by developer, used by Constructor during header file generation/update.
Table ID	ID of the object (assigned by Constructor).
Left Origin	Form-relative position of left side of the object. Valid values: 0 – 159
Top Origin	Form-relative position of top of object. Valid values: 0 – 159

Width	Width of the object in pixels. Valid values: 1– 160
Height	Height of the object in pixels. Valid values: 1–160
Editable	If the user can modify the table.
Rows	Number of rows in the table.
Columns	Number of columns in the table.
Column width	Width of the nth column.

3

Palm OS Events

Palm OS® events are structures (defined in the header files `Event.h`, `SysEvent.h`, and `INetMgr.h`) that the system passes to the application when the user interacts with the graphical user interface. Chapter 4, "Event Loop," on page 67, in the *Palm OS Programmer's Companion*, vol. I discusses in detail how this works. This chapter provides reference-style information about each event. First it shows the types used by Palm OS events. Then it discusses the following events in alphabetical order:

Event	UI Object
appStopEvent	N.A.
ctlEnterEvent, ctlExitEvent, ctlRepeatEvent, ctlSelectEvent	Control
daySelectEvent	N.A.
fldChangedEvent, fldEnterEvent, fldHeightChangedEvent	Field
frmCloseEvent, frmGotoEvent, frmLoadEvent, frmOpenEvent, frmSaveEvent, frmUpdateEvent, frmTitleEnterEvent, frmTitleSelectEvent	Form
frmGadgetEnterEvent, frmGadgetMiscEvent	Extended gadget
inetSockReadyEvent, inetSockStatusChangeEvent	N.A. (INetLib)
keyDownEvent	N.A.
lstEnterEvent, lstExitEvent, lstSelectEvent	List
menuEvent, menuOpenEvent, menuCloseEvent, menuCmdBarOpenEvent	Menu
nilEvent	N.A.
penDownEvent, penMoveEvent, penUpEvent	N.A. (pen)

Event	UI Object
popSelectEvent	Popup (Control)
sclEnterEvent, sclRepeatEvent, sclExitEvent	Scroll bar
tblEnterEvent, tblExitEvent, tblSelectEvent	Table
winEnterEvent, winExitEvent	Window

Event Data Structures

eventsEnum

The `eventsEnum` enum specifies the possible event types.

```
enum events {
  nilEvent = 0,
  penDownEvent,
  penUpEvent,
  penMoveEvent,
  keyDownEvent,
  winEnterEvent,
  winExitEvent,
  ctlEnterEvent,
  ctlExitEvent,
  ctlSelectEvent,
  ctlRepeatEvent,
  lstEnterEvent,
  lstSelectEvent,
  lstExitEvent,
  popSelectEvent,
  fldEnterEvent,
  fldHeightChangedEvent,
  fldChangedEvent,
  tblEnterEvent,
  tblSelectEvent,
  daySelectEvent,
  menuEvent,
  appStopEvent = 22,
  frmLoadEvent,
```

```
    frmOpenEvent,
    frmGotoEvent,
    frmUpdateEvent,
    frmSaveEvent,
    frmCloseEvent,
    frmTitleEnterEvent,
    frmTitleSelectEvent,
    tblExitEvent,
    sclEnterEvent,
    sclExitEvent,
    sclRepeatEvent,
    tsmFepModeEvent,

    menuCmdBarOpenEvent = 0x0800,
    menuOpenEvent,
    menuCloseEvent,
    frmGadgetEnterEvent,
    frmGadgetMiscEvent,

    firstINetLibEvent = 0x1000,
    firstWebLibEvent = 0x1100,

    firstUserEvent = 0x6000,
    lastUserEvent  = 0x7FFF
} eventsEnum;
```

Each of these event types is discussed in alphabetical order below.

EventType

The `EventType` structure contains all the data associated with a system event. All event types have some common data. Most events also have data specific to those events. The specific data uses a union that is part of the `EventType` data structure. The union can have up to 8 words of specific data.

The common data is documented below the structure. The Event Reference section gives details on the important data associated with each type of event.

```
typedef struct {
  eventsEnum   eType;
  Boolean      penDown;
  UInt8        tapCount;
  Int16        screenX;
  Int16        screenY;
  union{
    ...
  } data;
} EventType;
```

Common Field Descriptions

`eType` One of the `eventsEnum` constants. Specifies the type of the event.

`penDown` `true` if the pen was down at the time of the event, otherwise `false`.

`tapCount` The number of taps received at this location. This value is used mainly by fields. When the user taps in a text field, two taps selects a word, and three taps selects the entire line.

`screenX` Window-relative position of the pen in pixels (number of pixels from the left bound of the window).

`screenY` Window-relative position of the pen in pixels (number of pixels from the top left of the window).

`data` The specific data for an event, if any. The data is a union, and its exact contents depend on the `eType` field. The Event Reference section in this chapter shows what the `data` field contains for each event.

NOTE: Remember that the `data` field is part of the access path to an identifier in the `EventType` structure. As an example, the code to access the `controlID` field of a `ctlEnterEvent` would be:

```
EventType *event;
//...
if (event->data.ctlEnter.controlID ==
              MyAppLockButton)
```

Compatibility The `tapCount` field is only defined if 3.5 New Feature Set is present. Because of the `tapCount` field, it's particularly important that you clear the event structure before you use it to add a new event to the queue in Palm OS 3.5 and higher. Otherwise, the `tapCount` value may be incorrect for the new event.

EventPtr

The `EventPtr` defines a pointer to an `EventType`.

```
typedef EventType *EventPtr;
```

Event Reference

appStopEvent

When the system wants to launch a different application than the one currently running, the event manager sends this event to request the current application to terminate. In response, an application has to exit its event loop, close any open files and forms, and exit.

If an application doesn't respond to this event by exiting, the system can't start the other application.

ctlEnterEvent

The control routine CtlHandleEvent sends this event when it receives a penDownEvent within the bounds of a control.

For this event, the data field contains the following structure:

```
struct ctlEnter {
  UInt16 controlID;
  struct ControlType *pControl;
} ctlEnter;
```

Field Descriptions

controlID Developer-defined ID of the control.

pControl Pointer to a control structure (ControlType).

ctlExitEvent

The control routine CtlHandleEvent sends this event. When CtlHandleEvent receives a ctlEnterEvent, it tracks the pen until the pen is lifted from the display. If the pen is lifted within the bounds of a control, a ctlSelectEvent is added to the event queue; if not, a ctlExitEvent is added to the event queue. The penDown, screenX, and screenY fields of the EventType structure are set appropriately for the ctlExitEvent. As well, the data field contains the following structure:

```
struct ctlExit {
  UInt16 controlID;
  struct ControlType *pControl;
} ctlExit;
```

Field Descriptions

controlID Developer-defined ID of the control.

pControl Pointer to a control structure (ControlType).

ctlRepeatEvent

The control routine CtlHandleEvent sends this event. When CtlHandleEvent receives a ctlEnterEvent in a repeating

button (tREP) or a feedback slider control (tslf), it sends a `ctlRepeatEvent`. When `CtlHandleEvent` receives a `ctlRepeatEvent` in a repeating button, it sends another `ctlRepeatEvent` if the pen remains down within the bounds of the control for 1/2 second beyond the last `ctlRepeatEvent`.

When `CtlHandleEvent` receives a `ctlRepeatEvent` in a feedback slider control, it sends a `ctlRepeatEvent` each time the slider's thumb moves by at least one pixel. Feedback sliders do not send `ctlRepeatEvents` at regular intervals like repeating buttons do.

If you return `true` in response to a `ctlRepeatEvent`, it stops the `ctlRepeatEvent` loop. No further `ctlRepeatEvents` are sent.

For this event, the `data` field contains the following structure:

```
struct ctlRepeat {
  UInt16 controlID;
  struct ControlType *pControl;
  UInt32 time;
  UInt16 value;
} ctlRepeat;
```

Field Descriptions

`controlID`	Developer-defined ID of the control.
`pControl`	Pointer to a control structure (`ControlType`).
`time`	System-ticks count when the event is added to the queue.
`value`	Current value if the control is a feedback slider.

Compatibility The `value` field is only present if 3.5 New Feature Set is present.

ctlSelectEvent

The control routine `CtlHandleEvent` sends this event. When `CtlHandleEvent` receives a `ctlEnterEvent`, it tracks the pen until the pen is lifted. If the pen is lifted within the bounds of the same control it went down in, a `cltSelectEvent` is added to the event queue; if not, a `ctlExitEvent` is added to the event queue.

For this event, the `data` field contains the following structure:

```
struct ctlSelect {
  UInt16  controlID;
  struct ControlType *pControl;
  Boolean on;
  UInt8   reserved1;
  UInt16  value;
} ctlSelect;
```

Field Descriptions

`controlID`	Developer-defined ID of the control.
`pControl`	Pointer to a control structure (`ControlType`).
`on`	`true` when the control is depressed; otherwise, `false`.
`reserved1`	Unused.
`value`	Current value if the control is a slider.

Compatibility The `value` field is only present if 3.5 New Feature Set is present.

daySelectEvent

The system-internal `DayHandleEvent` routine, which handles events in the day selector object, handles this event. When the day selector object displays a calendar month, the user can select a day by tapping on it.

This event is sent when the pen touches and is lifted from a day number.

For this event, the `data` field contains the following structure:

```
struct daySelect {
  struct DaySelectorType *pSelector;
  Int16 selection;
  Boolean useThisDate;
  UInt8 reserved1;
} daySelect;
```

Field Descriptions

`pSelector`	Pointer to a day selector structure (`DaySelectorType`).
`selection`	Not used.
`useThisDate`	Set to `true` to automatically use the selected date.
`reserved1`	Unused.

fldChangedEvent

The field routine `FldHandleEvent` sends this event when the text of a field has or might have been scrolled. This event actually can be triggered from any call to the field code that causes scrolling to happen; most often, this happens during `FldHandleEvent`.

When `FldHandleEvent` receives a `fldEnterEvent`, it positions the insertion point and tracks the pen until it's lifted. Text is selected (highlighted) appropriately as the pen is dragged.

For this event, the `data` field contains the following structure:

```
struct fldChanged {
  UInt16 fieldID;
  struct FieldType *pField;
} fldChanged;
```

Field Descriptions

`fieldID`	Developer-defined ID of the field.
`pField`	Pointer to a field structure (`FieldType`).

fldEnterEvent

The field routine `FldHandleEvent` sends this event when the field receives a `penDownEvent` within the bounds of a field. For this event, the `data` field contains the following structure:

```
struct fldEnter {
  UInt16 fieldID;
  struct FieldType *pField;
} fldEnter;
```

Field Descriptions

`fieldID` Developer-defined ID of the field.

`pField` Pointer to a field structure (`FieldType`).

fldHeightChangedEvent

Several field routines, including `FldHandleEvent`, send this event when the number of lines in the field changes. These functions send a `fldHeightChangedEvent` to notify your application that the height of a field needs to change.

If the field is contained in a table, the table's code handles the `fldHeightChangedEvent`. If the field is directly on a form, your application code should handle the `fldHeightChangedEvent` itself. The form code does not handle the event for you.

For this event, the `data` field contains the following structure:

```
struct fldHeightChanged {
  UInt16   fieldID;
  struct FieldType *pField;
  Int16    newHeight;
  UInt16   currentPos;
} fldHeightChanged;
```

Field Descriptions

`fieldID` Developer-defined ID of the field.

`pField` Pointer to a field structure (`FieldType`).

`newHeight` New visible height of the field, in number of lines.

`currentPos` Current position of the insertion point.

frmCloseEvent

The form routines `FrmGotoForm` and `FrmCloseAllForms` send this event. `FrmGotoForm` sends a `frmCloseEvent` to the currently active form; `FrmCloseAllForms` sends a `frmCloseEvent` to all forms an application has loaded into memory. If an application doesn't intercept this event, the routine `FrmHandleEvent` erases the specified form and releases any memory allocated for it.

For this event, the `data` field contains the following structure:

```
struct frmClose {
  UInt16 formID;
} frmClose;
```

Field Descriptions

`formID`	Developer-defined ID of the form.

frmGadgetEnterEvent

The function `FrmHandleEvent` sends this event when there is a `penDownEvent` within the bounds of an extended gadget. The gadget handler function (see `FormGadgetHandler`) should handle this event.

For this event, the `data` field contains the following structure:

```
struct gadgetEnter {
  UInt16 gadgetID;
  struct FormGadgetType *gadgetP;
} gadgetEnter;
```

Field Descriptions

`gadgetID`	Developer-defined ID of the gadget.
`gadgetP`	Pointer to the `FormGadgetType` object representing this gadget.

Compatibility Implemented only if 3.5 New Feature Set is present.

frmGadgetMiscEvent

An application may choose to send this event when it needs to pass information to an extended gadget. The `FrmHandleEvent` function passes `frmGadgetMiscEvents` on to the extended gadget's handler function (see `FormGadgetHandler`).

For this event, the `data` field contains the following structure:

```
struct gadgetMisc {
  UInt16 gadgetID;
  struct FormGadgetType *gadgetP;
  UInt16 selector;
  void *dataP;
} gadgetMisc;
```

Field Descriptions

`gadgetID`	Developer-defined ID of the gadget.
`gadgetP`	Pointer to the `FormGadgetType` object representing this gadget.
`selector`	Any necessary integer value to pass to the gadget handler function.
`dataP`	A pointer to any necessary data to pass to the gadget handler function.

Compatibility Implemented only if 3.5 New Feature Set is present.

frmGotoEvent

An application may choose to send itself this event when it receives a `sysAppLaunchCmdGoto` launch code. `sysAppLaunchCmdGoto` is generated when the user selects a record in the global find facility. Like `frmOpenEvent`, `frmGotoEvent` is a request that the application initialize and draw a form, but this event provides extra information so that the application may display and highlight the matching string in the form.

The application is responsible for handling this event.

For this event, the `data` field contains the following structure:

```
struct frmGoto {
  UInt16 formID;
  UInt16 recordNum;
  UInt16 matchPos;
  UInt16 matchLen;
```

```
  UInt16 matchFieldNum;
  UInt32 matchCustom;
} frmGoto;
```

Field Descriptions

`formID`	Developer-defined ID of the form.
`recordNum`	Index of record containing the match string.
`matchPos`	Position of the match.
`matchLen`	Length of the matched string.
`matchFieldNum`	Number of the field the matched string was found in.
`matchCustom`	Application-specific information. You might use this if you need to provide extra information to locate the matching string within the record.

frmLoadEvent

The form routines `FrmGotoForm` and `FrmPopupForm` send this event. It's a request that the application load a form into memory.

The application is responsible for handling this event.

For this event, the `data` field contains the following structure:

```
struct frmLoad {
  UInt16 formID;
} frmLoad;
```

Field Descriptions

`formID`	Developer-defined ID of the form.

frmOpenEvent

The form routines `FrmGotoForm` and `FrmPopupForm` send this event. It is a request that the application initialize and draw a form.

The application is responsible for handling this event.

For this event, the `data` field contains the following structure:

```
struct frmOpen {
  UInt16 formID;
} frmOpen;
```

Field Descriptions

`formID` Developer-defined ID of the form.

frmSaveEvent

The form routine `FrmSaveAllForms` sends this event. It is a request that the application save any data stored in a form.

The application is responsible for handling this event.

No data is passed with this event.

frmTitleEnterEvent

The control routine `FrmHandleEvent` sends this event when it receives a `penDownEvent` within the bounds of the title of the form. Note that only the written title, not the whole title bar is active.

For this event, the `data` field contains the following structure:

```
struct frmTitleEnter {
  UInt16 formID;
  } frmTitleEnter;
```

Field Descriptions

`formID` Developer-defined ID of the form.

frmTitleSelectEvent

The control routine `FrmHandleEvent` sends this event. `FrmHandleEvent` receives a `frmTitleEnterEvent`, it tracks the pen until the pen is lifted. If the pen is lifted within the bounds of the active same title bar region, a `frmTitleSelectEvent` is added to the event queue.

For this event, the `data` field contains the following structure:

```
struct frmTitleSelect {
  UInt16 formID;
} frmTitleSelect;
```

Field Descriptions

`formID` Developer-defined ID of the form.

Compatibility In Palm OS version 3.5 and higher, `FrmHandleEvent` responds to `frmTitleSelectEvent`. Its response is to enqueue a `keyDownEvent` with a `vchrMenu` character to display the form's menu.

frmUpdateEvent

The form routine `FrmUpdateForm`, or in some cases the routine `FrmEraseForm`, sends this event when it needs to redraw the region obscured by the form being erased.

Generally, the region obscured by a form is saved and restored by the form routines without application intervention. However, in cases where the system is running low on memory, the form's routine may not save obscured regions itself. In that case, the application adds a `frmUpdateEvent` to the event queue. The form receives the event and redraws the region using the `updateCode` value.

An application can define its own `updateCode` and then use this event to also trigger behavior in another form, usually when changes made to one form need to be reflected in another form.

For this event, the `data` field contains the following structure:

```
struct frmUpdate {
  UInt16 formID;
  UInt16 updateCode;
} frmUpdate;
```

Field Descriptions

`formID`	Developer-defined ID of the form.
`updateCode`	The reason for the update request. `FrmEraseForm` sets this code to `frmRedrawUpdateCode`, which indicates that the entire form needs to be redrawn. Application developers can define their own `updateCode`. The `updateCode` is passed as a parameter to `FrmUpdateForm`.

inetSockReadyEvent

This event is returned only by `INetLibGetEvent` (not `EvtGetEvent`) when the Internet library determines that a socket has data ready for an `INetLibSockRead`.

For this event, the `data` field contains the following structure:

```
struct  {
  MemHandle  sockH;
  UInt32     context;
  Boolean    inputReady;
  Boolean    outputReady;
} inetSockReady;
```

Field Descriptions

`sockH`	Socket handle of the socket that this event refers to.
`context`	Not used.
`inputReady`	`true` when the socket has data ready for the `INetLibSockRead` call.
`outputReady`	Not used.

The `penDown`, `tapCount`, `screenX` and `screenY` fields are **not** valid for Internet library events and should be ignored.

Compatibility Implemented only if Wireless Internet Feature Set is present.

inetSockStatusChangeEvent

This event is returned only by `INetLibGetEvent` (not `EvtGetEvent`) when the Internet library determines that a socket has data ready for an `INetLibSockRead`.

For this event, the `data` field contains the following structure:

```
struct  {
  MemHandle  sockH;
  UInt32     context;
  UInt16     status;
  Err        sockErr;
}inetSockStatusChange;
```

Field Descriptions

`sockH`	Socket handle of the socket that this event refers to.
`context`	Not used.
`status`	Current status of the socket. This is one of the `INetStatusEnum` constants.
`sockErr`	Reason for failure of the last operation, if any. The current socket error can be cleared by calling `INetLibSockStatus`.

The `penDown`, `tapCount`, `screenX` and `screenY` fields are **not** valid for Internet library events and should be ignored.

Compatibility Implemented only if Wireless Internet Feature Set is present.

keyDownEvent

This event is sent by the system when the user enters a Graffiti® character, presses one of the buttons below the display, or taps one of the icons in the icon area; for example, the Find icon.

For this event, the `data` field contains the following structure:

```
struct _KeyDownEventType {
  WChar    chr;
  UInt16   keyCode;
  UInt16   modifiers;
};
```

Field Descriptions

`chr`	The character code.
`keyCode`	Unused.
`modifiers`	0, or one or more of the following values:
`shiftKeyMask`	Graffiti is in case-shift mode.
`capsLockMask`	Graffiti is in cap-shift mode.
`numLockMask`	Graffiti is in numeric-shift mode.
`commandKeyMask`	The Graffiti glyph was the menu command glyph or a virtual key code.
`optionKeyMask`	Not implemented. Reserved.
`controlKeyMask`	Not implemented. Reserved.
`autoRepeatKeyMask`	Event was generated due to auto-repeat.
`doubleTapKeyMask`	Not implemented. Reserved.
`poweredOnKeyMask`	The key press caused the system to be powered on.
`appEvtHookKeyMask`	System use only.
`libEvtHookKeyMask`	System use only.

lstEnterEvent

The list routine `LstHandleEvent` sends this event when it receives a `penDownEvent` within the bounds of a list object.

For this event, the `data` field contains the following structure:

```
struct lstEnter {
  UInt16 listID;
  struct ListType *pList;
  Int16 selection;
} lstEnter;
```

Field Descriptions

`listID`	Developer-defined ID of the list.
`pList`	Pointer to a list structure (`ListType`).
`selection`	Unused.

lstExitEvent

The list routine `LstHandleEvent` sends this event. When `LstHandleEvent` receives a `lstEnterEvent`, it tracks the pen until the pen is lifted. If the pen is lifted within the bounds of a list, a `lstSelectEvent` is added to the event queue; if not, a `lstExitEvent` is added to the event queue.

For this event, the `data` field contains the following structure:

```
struct lstExit {
  UInt16 listID;
  struct ListType *pList;
} lstExit;
```

Field Descriptions

`listID`	Developer-defined ID of the list.
`pList`	Pointer to a list structure (`ListType`).

lstSelectEvent

The list routine `LstHandleEvent` sends this event. When `LstHandleEvent` receives a `lstEnterEvent`, it tracks the pen until the pen is lifted. If the pen is lifted within the bounds of a list, a `lstSelectEvent` is added to the event queue; if not, a `lstExitEvent` is added to the event queue.

For this event, the `data` field contains the following structure:

```
struct lstSelect {
  UInt16 listID;
  struct ListType *pList;
  Int16 selection;
} lstSelect;
```

Field Descriptions

`listID`	Developer-defined ID of the list.
`pList`	Pointer to a list structure (`ListType`).
`selection`	Item number (zero-based) of the new selection.

menuCloseEvent

This event is not currently used.

menuCmdBarOpenEvent

The menu routine `MenuHandleEvent` sends this event when the user enters the menu shortcut keystroke, causing the command toolbar to be displayed at the bottom of the screen. Applications might respond to this event by calling `MenuCmdBarAddButton` to add custom buttons to the command toolbar. Shared libraries or other non-application code resources can add buttons to the toolbar by registering to receive the `sysNotifyMenuCmdBarOpenEvent` notification.

For this event, the `data` field contains the following structure:

```
struct menuCmdBarOpen {
  Boolean  preventFieldButtons;
  UInt8    reserved;
} menuCmdBarOpen;
```

Field Descriptions

`preventFieldButtons`	If `true`, the field manager does not add the standard cut, copy, paste, and undo buttons when the focus is on a field. If `false`, the field adds the buttons.
`reserved`	Unused.

To prevent the command toolbar from being displayed, respond to this event and return `true`. Returning `true` prevents the form manager from displaying the toolbar.

Compatibility Implemented only if 3.5 New Feature Set is present.

menuEvent

The menu routine `MenuHandleEvent` sends this event:

- When the user selects an item from a pull-down menu
- When the user selects a menu command using the Graffiti command keystroke followed by an available command; for example, Command-C for copy
- When the user taps one of the buttons on the command toolbar and the button is set up to generate a `menuEvent`.

For this event, the `data` field contains the following structure:

```
struct menu {
  UInt16 itemID;
} menu;
```

Field Descriptions

`itemID` Item ID of the selected menu command.

menuOpenEvent

The menu routine `MenuHandleEvent` sends this event when a new active menu has been initialized. A menu becomes active the first time the user taps the Menu silk-screen button or taps the form's titlebar, and it might need to be re-initialized and reactivated several times during the life of an application.

A menu remains active until one of the following happens:

- A `FrmSetMenu` call changes the active menu on the form.
- A new form, even a modal form or alert panel, becomes active.

Suppose a user selects your application's About item from the Options menu then clicks the OK button to return to the main form. When the About dialog is displayed, it becomes the active form,

which causes the main form's menu state to be erased. This menu state is not restored when the main form becomes active again. The next time the user requests the menu, it must be initialized again, so `menuOpenEvent` is sent again.

Applications might respond to this event by adding, hiding, or un-hiding menu items using the functions `MenuAddItem`, `MenuHideItem`, or `MenuShowItem`.

A `menuCloseEvent` is defined by the system, but it is not currently sent. If you need to perform some cleanup (such as closing a resource) after the menu item you added is no longer needed, do so in response to `frmCloseEvent`.

For this event, the `data` field contains the following structure:

```
struct menuOpen {
  UInt16   menuRscID;
  Int16    cause;
} menuOpen;
```

Field Descriptions

`menuRscID` Resource ID of the menu.

`cause` Reason for opening the menu. If `menuButtonCause`, the user tapped the Menu silkscreen button or tapped the form's titlebar, and the menu is going to be displayed. If `menuCommandCause`, the user entered the command keystroke, so the menu is becoming active without being displayed.

Compatibility Implemented only if 3.5 New Feature Set is present.

nilEvent

A `nilEvent` is useful for animation, polling, and similar situations.

The event manager sends this event when there are no events in the event queue. This can happen if the routine `EvtGetEvent` is passed a time-out value (a value other than `evtWaitForever`, -1). If `EvtGetEvent` is unable to return an event in the specified time, it returns a `nilEvent`. Different Palm OS versions and different

devices can send `nilEvents` under different circumstances, so you might receive a `nilEvent` even before the timeout has expired.

penDownEvent

The event manager sends this event when the pen first touches the digitizer.

The following data is passed with the event:

Field Descriptions

`penDown`	Always `true`.
`tapCount`	The number of taps received at this location.
`screenX`	Window-relative position of the pen in pixels (number of pixels from the left bound of the window).
`screenY`	Window-relative position of the pen in pixels (number of pixels from the top left of the window).

penMoveEvent

The event manager sends this event when the pen is moved on the digitizer. Note that several kinds of UI objects, such as controls and lists, track the movement directly, and no `penMoveEvent` is generated.

The following data is passed with the event:

Field Descriptions

`penDown`	Always `true`.
`tapCount`	The number of taps received at this location.
`screenX`	Window-relative position of the pen in pixels (number of pixels from the left bound of the window).
`screenY`	Window-relative position of the pen in pixels (number of pixels from the top left of the window).

penUpEvent

The event manager sends this event when the pen is lifted from the digitizer. Note that several kinds of UI objects, such as controls and lists, track the movement directly, and no `penUpEvent` is generated.

For this event, the `data` field contains the following structure:

```
struct _PenUpEventType {
  PointType start;
  PointType end;
};
```

Field Descriptions

`start`	Display-relative start point of the stroke.
`end`	Display-relative end point of the stroke.

In addition, the following data is passed with this event:

`penDown`	Always `false`.
`tapCount`	The number of taps received at this location.
`screenX`	Window-relative position of the pen in pixels (number of pixels from the left bound of the window).
`screenY`	Window-relative position of the pen in pixels (number of pixels from the top left of the window).

popSelectEvent

The form routine `FrmHandleEvent` sends this event when the user selects an item in a popup list.

For this event, the `data` field contains the following structure:

```
struct popSelect {
  UInt16 controlID;
  struct ControlType *controlP;
  UInt16 listID;
  struct ListType *listP;
  Int16 selection;
  Int16 priorSelection;
} popSelect;
```

Field Descriptions

`controlID`	Developer-defined ID of the resource.
`controlP`	Pointer to the control structure (`ControlType`) of the popup trigger object.
`listID`	Developer-defined ID of the popup list object.
`listP`	Pointer to the list structure (`ListType`) of the popup list object.
`selection`	Item number (zero-based) of the new list selection.
`priorSelection`	Item number (zero-based) of the prior list selection.

sclEnterEvent

The routine `SclHandleEvent` sends this event when it receives a `penDownEvent` within the bounds of a scroll bar.

Applications usually don't have to handle this event.

For this event, the `data` field contains the following structure:

```
struct sclEnter {
  UInt16 scrollBarID;
  struct ScrollBarType *pScrollBar;
} sclEnter;
```

Field Descriptions

`scrollBarID`	Developer-defined ID of the scroll bar resource.
`pScrollBar`	Pointer to the scroll bar structure.

sclExitEvent

The routine `SclHandleEvent` sends this event when the user lifts the pen from the scroll bar.

Applications that want to implement non-dynamic scrolling should wait for this event, then scroll the text using the values provided in `value` and `newvalue`.

Note that this event is sent regardless of previous `sclRepeatEvents`. If, however, the application has implemented dynamic scrolling, it doesn't have to catch this event.

For this event, the `data` field contains the following structure:

```
struct sclExit {
  UInt16 scrollBarID;
  struct ScrollBarType *pScrollBar;
  Int16 value;
  Int16 newValue;
} sclExit;
```

Field Descriptions

`scrollBarID`	Developer-defined ID of the scroll bar resource.
`pScrollBar`	Pointer to the scroll bar structure.
`value`	Initial position of the scroll bar
`newvalue`	New position of the scroll bar. Given `value` and `newValue`, you can actually tell how much you have scrolled.

sclRepeatEvent

The routine `SclHandleEvent` sends this event when the pen is continually held within the bounds of a scroll bar.

Applications that implement dynamic scrolling should watch for this event. In dynamic scrolling, the display is updated as the user drags the scroll bar (not after the user releases the scroll bar).

For this event, the `data` field contains the following structure:

```
struct sclRepeat {
  UInt16 scrollBarID;
  struct ScrollBarType *pScrollBar;
  Int16 value;
  Int16 newValue;
  Int32 time;
} sclRepeat;
```

Field Descriptions

`scrollBarID`	Developer-defined ID of the scroll bar resource.
`pScrollBar`	Pointer to the scroll bar structure.
`value`	Initial position of the scroll bar.
`newValue`	New position of the scroll bar. Given `value` and `newValue`, you can actually tell how much you have scrolled.
`time`	System-ticks count when the event is added to the queue to determine when the next event should occur.

tblEnterEvent

The table routine `TblHandleEvent` sends this event when it receives a `penDownEvent` within the bounds of an active item in a table object.

For this event, the `data` field contains the following structure:

```
struct tblEnter {
  UInt16 tableID;
  struct TableType *pTable;
  Int16 row;
  Int16 column;
} tblEnter;
```

Field Descriptions

`tableID`	Developer-defined ID of the table.
`pTable`	Pointer to a table structure (`TableType`).
`row`	Row of the item.
`column`	Column of the item.

tblExitEvent

The table routine TblHandleEvent sends this event. When TblHandleEvent receives a tblEnterEvent, it tracks the pen until it's lifted from the display. If the pen is lifted within the bounds of the same item it went down in, a tblSelectEvent is added to the event queue; if not, a tblExitEvent is added to the event queue.

For this event, the data field contains the following structure:

```
struct tblExit {
  UInt16 tableID;
  struct TableType *pTable;
  Int16 row;
  Int16 column;
} tblExit;
```

Field Descriptions

tableID	Developer-defined ID of the table.
pTable	Pointer to a table structure (TableType).
row	Row of the item.
column	Column of the item.

tblSelectEvent

The table routine TblHandleEvent sends this event. When TblHandleEvent receives a tblEnterEvent, it tracks the pen until the pen is lifted from the display. If the pen is lifted within the bounds of the same item it went down in, a tblSelectEvent is added to the event queue; if not, a tblExitEvent is added to the event queue.

For this event, the data field contains the following structure:

```
struct tblSelect {
  UInt16 tableID;
  struct TableType *pTable;
  Int16 row;
  Int16 column;
} tblSelect;
```

Field Descriptions

`tableID` Developer-defined ID of the table.

`pTable` Pointer to a table structure (`TableType`).

`row` Row of the item.

`column` Column of the item.

winEnterEvent

The event manager sends this event when a window becomes the active window. This can happen in two ways: a call to `WinSetActiveWindow` is issued (`FrmSetActiveForm` calls this routine), or the user taps within the bounds of a window that is visible but not active. All forms are windows, but not all windows are forms; for example, the menu bar is a window but not a form.

For this event, the `data` field contains the following structure:

```
struct _WinEnterEventType {
  WinHandle enterWindow;
  WinHandle exitWindow;
};
```

Field Descriptions

`enterWindow` Handle to the window we are entering. If the window is a form, then this is a pointer to a `FormType` structure; if not, it's a pointer to a `WindowType` structure.

`exitWindow` Handle to the window we are exiting, if there is currently an active window, or zero if there is no active window. If the window is a form, then this is a pointer to a `FormType` structure; if not, it's a pointer to a `WindowType` structure.

winExitEvent

This event is sent by the event manager when a window is deactivated. A window is deactivated when another window becomes the active window (see `winEnterEvent`).

For this event, the `data` field contains the following structure:

```
struct _WinExitEventType {
  WinHandle enterWindow;
  WinHandle exitWindow;
};
```

Field Descriptions

`enterWindow` Handle to the window we are entering. If the window is a form, then this is a pointer to a `FormType` structure; if not, it's a pointer to a `WindowType` structure.

`exitWindow` Handle to the window we are exiting. If the window is a form, then this is a pointer to a `FormType` structure; if not, it's a pointer to a `WindowType` structure.

4

Notifications

This chapter provides detailed information about the notifications declared in the header file `NotifyMgr.h`. Notifications are broadcast to inform applications, shared libraries, system extensions, or other code resources of certain system-level or application-level events.

Notifications are similar to application launch codes, but they differ from launch codes in the following ways:

- The system broadcasts notifications only to interested parties. To register to receive a notification, use `SysNotifyRegister`.
- Notifications can be sent to non-applications.

See the "Notification Manager" chapter in this book and the section "Notifications" on page 46 of the *Palm OS Programmer's Companion*, vol. I for more information on receiving and handling notifications.

Table 4.1 Notification Constants

Constant	Description
`cncNotifyProfileEvent`	The connection profile used by the Connection Panel has changed.
`sysExternalConnectorAttachEvent`	A device has been attached to an external connector.
`sysExternalConnectorDetachEvent`	A device has been detached from an external connector.
`sysNotifyAntennaRaisedEvent`	The antenna has been raised on a Palm VII™ series device.
`sysNotifyCardInsertedEvent`	An expansion card has been inserted into the expansion slot.
`sysNotifyCardRemovedEvent`	An expansion card has been removed from the expansion slot.

Table 4.1 Notification Constants *(continued)*

Constant	Description
sysNotifyDBDeletedEvent	A database has been deleted.
sysNotifyDeleteProtectedEvent	The Launcher has attempted to delete a protected database.
sysNotifyDeviceUnlocked	The user has unlocked the device.
sysNotifyDisplayChangeEvent	The color table or bit depth has changed.
sysNotifyEarlyWakeupEvent	The system is starting to wake up.
sysNotifyForgotPasswordEvent	The user has tapped the Lost Password button in the Security application.
sysNotifyGotUsersAttention	The Attention Manager has informed the user of an event.
sysNotifyHelperEvent	An application has requested that a particular service be performed.
sysNotifyIrDASniffEvent	Not used.
sysNotifyLateWakeupEvent	The system has finished waking up.
sysNotifyLocaleChangedEvent	The system locale has changed.
sysNotifyMenuCmdBarOpenEvent	The system is about to display the menu command toolbar.
sysNotifyNetLibIFMediaEvent	The system has been connected to or disconnected from the network.
sysNotifyPhoneEvent	Reserved for future use.
sysNotifyPOSEMountEvent	System use only.
sysNotifyResetFinishedEvent	The system has finished a reset.
sysNotifyRetryEnqueueKey	The Attention Manager has failed to post a virtual character to the key queue.
sysNotifySleepNotifyEvent	The system is about to go to sleep.
sysNotifySleepRequestEvent	The system has decided to go to sleep.

Table 4.1 Notification Constants *(continued)*

Constant	Description
sysNotifySyncFinishEvent	A HotSync® operation has just completed.
sysNotifySyncStartEvent	A HotSync operation is about to begin.
sysNotifyTimeChangeEvent	The system time has just changed.
sysNotifyVolumeMountedEvent	A file system has been mounted.
sysNotifyVolumeUnmountedEvent	A file system has been unmounted.

Notification Data Structures

SysNotifyParamType

The `SysNotifyParamType` structure contains all of the data associated with a notification. This structure is passed as the parameter block for the `sysAppLaunchCmdNotify` launch code or as a parameter to the notification callback function. All notifications have some common data. Most notifications also have data specific to that notification. The specific data is pointed to by the `notifyDetailsP` field.

The common data for each notification is documented below the following structure declaration. The Notification Reference section gives details on the important data associated with each type of notification.

```
typedef struct SysNotifyParamType {
  UInt32   notifyType;
  UInt32   broadcaster;
  void *   notifyDetailsP;
  void *   userDataP;
  Boolean  handled;
  UInt8    reserved2;
} SysNotifyParamType;
```

Field Descriptions

`notifyType`	The type of event that occurred. See Notification Reference.
`broadcaster`	The creator ID of the application that broadcast the notification, or `sysNotifyBroadcasterCode` if the system broadcast the event.
`notifyDetailsP`	Pointer to data specific to this notification.
`userDataP`	Custom data that your notification handler requires. You create this data and pass it to `SysNotifyRegister`.
`handled`	Set to `true` if the notification has been handled; set to `false` otherwise. In some cases, `handled` is treated as a bit field that notification handlers can use to indicate that certain conditions are true.
`reserved2`	Reserved for future use.

Notification Reference



New cncNotifyProfileEvent

The `cncNotifyProfileEvent` is broadcast whenever a connection profile has been created, modified, or deleted and after a request has been made to update the connection profile list.

The `notifyDetailsP` field informs the notification handler of the type of change that was made. Register for the `cncNotifyProfileEvent` if your application maintains its own list of connection profiles that it should keep current or if it should help the Connection Panel maintain its list.

cncNotifyProfileEvent Specific Data

`notifyDetailsP` points to a `CncProfileNotifyDetailsType` structure.

Prototype

```
typedef struct _CncProfileNotifyDetailsTag {
  UInt16 version;
  UInt32 profileID;
  UInt16 deviceKind;
  UInt16 request;
} CncProfileNotifyDetailsType;
```

Fields

`version` The current version of this structure. Use the `kCncProfileNotifyCurrentVersion` constant to find out what the current version is.

`profileID` The ID of the modified connection profile.

`deviceKind` Device kind of the profile. This can be one of the following constants:

`kCncDeviceKindSerial`
: Serial connection profile.

`kCncDeviceKindModem`
: Modem profile.

`kCncDeviceKindPhone`
: Phone profile.

`kCncDeviceKindLocalNetwork`
: LAN profile.

`request` The action that was performed. This can be one of the following constants:

`kCncNotifyCreateRequest`
: The profile has been created.

`kCncNotifyDeleteRequest`
: The profile is about to be deleted.

`kCncNotifyModifyRequest`
: The profile has been modified.

`kCncNotifyUpdateListRequest`
: A HotSync operation or system reset has just occurred. The notification handler should update the Connection Panel's list.

If a profile has been created or modified, the `request` field also contains a flag indicating how the current profile is to be set:

`kCncBecomeCurrentModifier`
: The new profile should be made the current profile.

`kCncNotifyAlertUserModifier`
: The user is prompted to set the current profile.

Compatibility Implemented only if 4.0 New Feature Set is present.



New sysExternalConnectorAttachEvent

The `sysExternalConnectorAttachEvent` is broadcast when a USB cradle, RS-232 cradle or peripheral, a power cable, or a modem is attached to the universal connector. This notification is broadcast only on devices that have the universal connector.

sysExternalConnectorAttachEvent Specific Data

The `notifyDetailsP` field points to a `UInt16` that identifies which type of device was attached.

Compatibility Implemented only if 4.0 New Feature Set is present.



New sysExternalConnectorDetachEvent

The `sysExternalConnectorDetachEvent` is broadcast when a USB cradle, a RS-232 cradle or peripheral, a power cable, or a modem is detached from the universal connector. This notification is only broadcast on devices that have the universal connector.

sysExternalConnectorDetachEvent Specific Data

The `notifyDetailsP` field points to a `UInt16` that identifies which type of device was detached.

Compatibility Implemented only if 4.0 New Feature Set is present.

sysNotifyAntennaRaisedEvent

The `sysNotifyAntennaRaisedEvent` is broadcast by `SysHandleEvent` when the antenna is raised on a Palm VII series device.

Register for this notification if you want to handle the antenna key down event. To ensure that no other code handles the antenna key down event after yours, set the `handled` parameter of the `SysNotifyParamType` structure to `true`.

sysNotifyAntennaRaisedEvent Specific Data

None.

Compatibility Implemented only if Notification Feature Set is present.

New

sysNotifyCardInsertedEvent

The `sysNotifyCardInsertedEvent` is broadcast when an Expansion Manager card is inserted into a slot. When a new card is inserted, the Expansion Manager attempts to mount the volume on that card and plays a sound (indicating success or failure) once the attempt is complete.

Most applications will want to register for `sysNotifyVolumeMountedEvent` instead of this notification. Register for `sysNotifyCardInsertedEvent` if you need to know when a card is inserted or if you want to prevent the Expansion Manager from performing its default handling of the notification.

To prevent the Expansion Manager from mounting the volume, set the `expHandledVolume` bit in the `handled` field. To prevent the

Expansion Manager from playing the sound, set the `expHandledSound` bit in the `handled` field. For example:

```
cmdPBP->handled |= expHandledSound;
```

sysNotifyCardInsertedEvent Specific Data

`notifyDetailsP` points to a `UInt16` containing the slot reference number.

Compatibility Implemented only if 4.0 New Feature Set is present.



New sysNotifyCardRemovedEvent

The `sysNotifyCardRemovedEvent` is broadcast when an Expansion Manager card is removed from a slot. When a card is removed, the Expansion Manager responds to this notification by playing a goodbye sound and then attempting to unmount the volume.

Most applications will want to register for `sysNotifyVolumeUnmountedEvent` instead of this notification. Register for `sysNotifyCardRemovedEvent` if you need to know when a card is removed or if you want to prevent the Expansion Manager from performing its default handling of the notification.

To prevent the Expansion Manager from unmounting the volume, set the `expHandledVolume` bit in the `handled` field. To prevent the Expansion Manager from playing the sound, set the `expHandledSound` bit in the `handled` field. For example:

```
cmdPBP->handled |= expHandledSound;
```

sysNotifyCardRemovedEvent Specific Data

`notifyDetailsP` points to a `UInt16` containing the slot reference number.

Compatibility Implemented only if 4.0 New Feature Set is present.

New sysNotifyDBDeletedEvent

The `sysNotifyDBDeletedEvent` is broadcast **after** a database is removed from the device.

Register for this notification if you keep an internal list of databases that needs to be updated upon removal of a database. For example, the Attention Manager and Connection Manager register for this notification to maintain their internal lists of databases.

sysNotifyDBDeletedEvent Specific Data

`notifyDetailsP` points to a `SysNotifyDBDeletedType` structure.

Prototype

```
typedef struct SysNotifyDBDeletedTag {
  LocalID oldDBID;
  UInt16 cardNo;
  UInt16 attributes;
  Char dbName[dmDBNameLength];
  UInt32 creator;
  UInt32 type;
} SysNotifyDBDeletedType;
```

Fields

`oldDBID`	The local ID of the deleted database. This ID is no longer valid.

WARNING! The ID in `oldDBID` is invalid by the time the notification is broadcast. If you try to pass it to a Data Manager function, the system will crash.

`cardNo`	The number of the card on which the database resided.
`attributes`	The deleted database's attributes.
`dbName`	The name of the deleted database.
`creator`	The creator ID of the deleted database.
`type`	The type of the deleted database.

Compatibility

Implemented only if 4.0 New Feature Set is present.



New sysNotifyDeleteProtectedEvent

The `sysNotifyDeleteProtectedEvent` is broadcast when the Launcher attempts to delete a database that has the protected flag set. The Launcher broadcasts the notification and then attempts to delete the database again. Any third party application that deletes databases should broadcast this notification as well.

Register for this notification if you have a protected database but you still want to allow users to delete your application or other code resource if they choose. A notification handler should check the information in the `notifyDetailsP` struct to see if its database is the one being deleted. If so, it should respond to this notification to perform any necessary cleanup and to clear the protected flag. In this way, when the Launcher attempts to delete the database again, it will succeed. Note that if an application has multiple protected databases, this notification may be sent out more than once.

sysNotifyDeleteProtectedEvent Specific Data

`notifyDetailsP` points to a `SysNotifyDBInfoType` structure.

Prototype

```
typedef struct SysNotifyDBInfoTag {
  LocalID dbID;
  UInt16 cardNo;
  UInt16 attributes;
  Char dbName[dmDBNameLength];
  UInt32 creator;
  UInt32 type;
} SysNotifyDBInfoType;
```

Fields

`dbID`	The local ID of the database to be deleted.
`cardNo`	The number of the card on which the database resides.
`attributes`	The database's attributes.
`dbName`	The name of the database to be deleted.
`creator`	The creator ID of the database to be deleted.
`type`	The type of the database to be deleted.

Compatibility Implemented only if 4.0 New Feature Set is present.



New sysNotifyDeviceUnlocked

The `sysNotifyDeviceUnlocked` notification is broadcast by the Security application when the user unlocks the device. The notification is broadcast immediately after the device has finished unlocking.

If you display UI in response to the `sysNotifyLateWakeupEvent` notification, you should also register to receive the `sysNotifyDeviceUnlocked` notification. When a locked device receives the `sysNotifyLateWakeupEvent`, your UI should not be displayed if the device is waiting for the user to enter the password. The `sysNotifyDeviceUnlocked` notification is broadcast after the password is entered, which indicates that the user interface is ready.

sysNotifyDeviceUnlocked Specific Data

None.

Compatibility Implemented only if 4.0 New Feature Set is present.

sysNotifyDisplayChangeEvent

The `sysNotifyDisplayChangeEvent` is broadcast whenever the display mode changes. That is, either the color table has been set to use a specific palette using the `WinPalette` function or the bit depth has changed using the `WinScreenMode` function.

The `notifyDetailsP` field indicates how the bit depth changed. If the two values in the struct are equal, it means that the color palette has changed instead of the bit depth.

sysNotifyDisplayChangeEvent Specific Data

`notifyDetailsP` points to a `SysNotifyDisplayChangeDetailsType` structure.

Prototype

```
typedef struct {
  UInt32 oldDepth;
  UInt32 newDepth;
} SysNotifyDisplayChangeDetailsType;
```

Fields

`oldDepth`	The old bit depth.
`newDepth`	The new bit depth.

Compatibility Implemented only if Notification Feature Set is present.

sysNotifyEarlyWakeupEvent

The `sysNotifyEarlyWakeupEvent` is broadcast during `SysHandleEvent` immediately after the system has finished sleeping. The screen may still be turned off, and the system may not fully wake up. It may simply handle an alarm or a battery charger event and go back to sleep. Most applications that need notification of a wakeup event will probably want to register for `sysNotifyLateWakeupEvent` instead.

IMPORTANT: This notification is **not** guaranteed to be broadcast. Thus, it is not suitable for applications where external hardware must be turned on when the system is powered on.

sysNotifyEarlyWakeupEvent Specific Data

None.

Compatibility Implemented only if Notification Feature Set is present.

sysNotifyForgotPasswordEvent

The `sysNotifyForgotPasswordEvent` is broadcast after the user taps the Lost Password button in the Security application. The notification is sent after the user has confirmed that all private records should be deleted but before the deletion actually occurs.

sysNotifyForgotPasswordEvent Specific Data

None.

Compatibility Implemented only if Notification Feature Set is present.



New sysNotifyGotUsersAttention

The `sysNotifyGotUsersAttention` notification is broadcast when the Attention Manager has finished displaying or sounding its attention indicators (blinking, playing sounds, vibrating, and so on).

System extensions or shared libraries should register for this notification if they want to perform some extra effect or if they simply want to be informed of when the user's attention was received.

sysNotifyGotUsersAttention Specific Data

`notifyDetailsP` points to an `AttnNotifyDetailsType` structure.

Prototype

```
typedef struct {
  AttnFlagsType flags;
} AttnNotifyDetailsType;
```

Fields

`flags`	The attention indicators that were used to get the user's attention. See `AttnFlagsType`.

Compatibility Implemented only if 4.0 New Feature Set is present.



New sysNotifyHelperEvent

The `sysNotifyHelperEvent` is broadcast by applications to request a service from another application. For example, the Address Book application broadcasts this notification to request that the Dial application dial a phone number. For the `sysNotifyHelperEvent`, the notification client (that is, the application or shared library that registers for the notification) is called a **helper**.

The application that broadcasts this notification specifies one of the action codes listed in Table 36.1 in Chapter 36, "Helper API." These action codes request all helper applications to enumerate (list the services they perform), validate (ensure that the service will succeed), and execute (perform the action). The helper responds to the notification by returning the required data in the appropriate portion of the `notifyDetailsP` structure and by setting the `handled` field to `true` or `false` to indicate the success or failure of the action.

For more information on this notification, see the section "Helper Notifications" on page 53 in the *Palm OS Programmer's Companion*, vol. I.

sysNotifyHelperEvent Specific Data

`notifyDetailsP` points to a `HelperNotifyEventType` structure.

Compatibility Implemented only if 4.0 New Feature Set is present.

sysNotifyLateWakeupEvent

The `sysNotifyLateWakeupEvent` is broadcast during `SysHandleEvent` immediately after the device has finished waking up. This notification is sent at the late stage of wakeup, after the screen has been turned on. When this notification is broadcast, the system is guaranteed to fully wake up. Register for this notification if you need to perform startup tasks each time the system wakes up.

IMPORTANT: This notification is **not** guaranteed to be broadcast. Thus, it is unsuitable for applications where external hardware must be powered on when the device wakes up.

When the device receives this notification, it may be locked and waiting for the user to enter the password. If this is the case, you must wait for the user to unlock the device before you display a user interface. Therefore, if you intend to display a user interface when the device wakes up, you should make sure the device is not locked. If the device is locked, you should register for

sysNotifyDeviceUnlocked notification and display your user interface when it is received. For example:

```
case sysNotifyLateWakeupEvent:
  if ((Boolean)
      PrefGetPreference(prefDeviceLocked)) {
    SysNotifyRegister(myCardNo, myDbID,
      sysNotifyDeviceUnlocked, NULL,
      sysNotifyNormalPriority, NULL);
  } else {
    HandleDeviceWakeup();
  }
case sysNotifyDeviceUnlocked:
  HandleDeviceWakeup();
```

Note that the sysNotifyDeviceUnlocked notification is only broadcast on Palm OS 4.0 and higher.

sysNotifyLateWakeupEvent Specific Data

None.

Compatibility Implemented only if Notification Feature Set is present.



New sysNotifyLocaleChangedEvent

The sysNotifyLocaleChangedEvent is broadcast immediately after the system locale has changed. Currently, the user has the opportunity to change the locale only when the device first starts up and after a hard reset.

RAM-based applications and other code resources should obtain locale information by passing the prefLocale constant to PrefGetPreference. They should not register for this notification. This notification is used by the built-in applications, which respond to it by rebuilding their default databases to use the newly selected language and character set.

sysNotifyLocaleChangedEvent Specific Data

notifyDetailsP points to a SysNotifyLocaleChangedType structure.

Prototype

```
typedef struct SysNotifyLocaleChangedTag {
  LmLocaleType oldLocale;
  LmLocaleType newLocale;
} SysNotifyLocaleChangedType;
```

Fields

`oldLocale`	The old locale. See `LmLocaleType`.
`newLocale`	The new locale.

Compatibility Implemented only if 4.0 New Feature Set is present.

sysNotifyMenuCmdBarOpenEvent

The `sysNotifyMenuCmdBarOpenEvent` is broadcast during `MenuHandleEvent` when it is about to display the menu shortcut command bar.

Register for this notification if you are writing a system extension (such as a "hack" installed with the HackMaster program) that needs to add a button to the menu command bar or to suppress the menu command bar. To add a button, call `MenuCmdBarAddButton`. To suppress the command toolbar, set the `handled` field to `true`.

Applications that need to add their own buttons to the menu command bar should do so in response to a `menuCmdBarOpenEvent`. They should **not** register for this notification because an application should only add buttons if it is already the active application. The notification is sent after the event has been received, immediately before the command toolbar is displayed.

sysNotifyMenuCmdBarOpenEvent

None.

Compatibility Implemented only if Notification Feature Set is present.

New sysNotifyNetLibIFMediaEvent

The `sysNotifyNetLibIFMediaEvent` is broadcast at the top of the event loop whenever the network interface makes the network connection active or inactive. The Network Panel uses this notification to decide whether the Connect button should be active.

Register for this notification if you need to know when the network connection is currently active.

sysNotifyNetLibIFMediaEvent Specific Data

`notifyDetailsP` contains a `SysNotifyNetLibIFMediaType` structure.

Prototype

```
typedef struct SysNotifyNetLibIFMediaTag {
  NetLibIFMediaEventNotificationTypeEnum eType;
  UInt32 ifCreator;
  UInt16 ifInstance;
} SysNotifyNetLibIFMediaType;
```

Fields

`eType` One of the following values:

`netIFMediaUp`
: The network connection is active. This is usually sent after the network interface has displayed UI indicating that a connection attempt is in progress.

`netIFMediaDown`
: The network connection is inactive. This is usually sent after the network interface has brought the connection down because an inactivity timeout value was reached.

`ifCreator` Creator ID of the network interface

`ifInstance` Instance number of the network interface.

Compatibility Implemented only if 4.0 New Feature Set is present.

sysNotifyResetFinishedEvent

The `sysNotifyResetFinishedEvent` is broadcast immediately after the system has finished a reset.

Because the notification registry is cleared upon a reset, only internal system components use this notification. Applications that need to be informed of a system reset can respond to the `sysAppLaunchCmdSystemReset` launch code.

sysNotifyResetFinishedEvent Specific Data

None.

Compatibility Implemented only if Notification Feature Set is present.



New sysNotifyRetryEnqueueKey

The `sysNotifyRetryEnqueueKey` notification is broadcast at the top of the event loop if the Attention Manager has attempted to post a virtual character to the key queue and failed because the queue is full. The notification signals that the Attention Manager is going to retry enqueuing the virtual character until it is successful.

Most applications do not need to register for this notification. It is used only by the Attention Manager to schedule retries of enqueuing the virtual character. When enqueueing a virtual character fails, the Attention Manager retries at the top of the event loop. It uses this notification to schedule retries so that they occur even if the user switches applications.

sysNotifyRetryEnqueueKey Specific Data

`notifyDetailsP` points to a `WChar` containing the virtual character to be enqueued.

Compatibility Implemented only if 4.0 New Feature Set is present.

sysNotifySleepNotifyEvent

The `sysNotifySleepNotifyEvent` is broadcast during `SysHandleEvent` immediately before the system is put to sleep. After the broadcast is complete, the system is put to sleep.

Register for this notification if you have a small amount of cleanup that needs to be performed before the system goes to sleep. It is recommended that you not perform any sort of prolonged activity, such as displaying an alert panel that requests confirmation, in response to a sleep notification. If you do, the alert might be displayed long enough to trigger another auto-off event, which could be detrimental to other handlers of this notification.

If your code is in the middle of a lengthy computation and needs to defer sleep, it should register for the `sysNotifySleepRequestEvent` instead.

IMPORTANT: This notification is **not** guaranteed to be broadcast. For example, if the system goes to sleep because the user removes the batteries, sleep notifications are not sent. Thus, these notifications are unsuitable for applications where external hardware must be shut off to conserve power before the system goes to sleep.

sysNotifySleepNotifyEvent Specific Data

None.

Compatibility Implemented only if Notification Feature Set is present.

sysNotifySleepRequestEvent

The `sysNotifySleepRequestEvent` is broadcast during `SysHandleEvent` processing when the system has decided to go to sleep.

Register for this notification if you need to delay the system from going to sleep while your code performs a lengthy operation, such as disconnecting from the network. The system checks the `deferSleep` value when each notification handler returns. If it is nonzero, it cancels the sleep event.

After you defer sleep, your code is free to finish what it was doing. When it is finished, you must allow the system to continue with the sleep event. To do so, create a `keyDownEvent` with the `resumeSleepChr` and the command key bit set (to signal that the character is virtual) and add it to the event queue. When the system receives this event, it will again broadcast the `sysNotifySleepRequestEvent` to all clients. If `deferSleep` is 0 after all clients return, then the system knows it is safe to go to sleep, and it broadcasts the `sysNotifySleepNotifyEvent` to all of its clients.

Note that you may receive this notification several times before the system goes to sleep because notification handlers can delay the system sleep and resume it later.

IMPORTANT: This notification is **not** guaranteed to be broadcast. For example, if the system goes to sleep because the user removes the batteries, sleep notifications are not sent. Thus, these notifications are unsuitable for applications where external hardware must be shut off to conserve power before the system goes to sleep.

sysNotifySleepRequestEvent Specific Data

`notifyDetailsP` points to a `SleepEventParamType` structure.

Prototype

```
typedef struct {
  UInt16 reason;
  UInt16 deferSleep;
} SleepEventParamType;
```

Fields

`reason` The reason the system is going to sleep. The possible values are:

`sysSleepAutoOff`
: The idle time limit has been reached.

`sysSleepPowerButton`
: The user pressed the power off button.

`sysSleepResumed`
: The sleep event was deferred by one of the notification handlers but has been resumed through the use of the `resumeSleepChr`.

`sysSleepUnknown`
: Unknown reason.

`deferSleep` — Initially set to 0. If a notification handler wants to defer sleep, then it should increment this value. When `deferSleep` is greater than 0, the system waits before going to sleep.

Compatibility Implemented only if Notification Feature Set is present.

sysNotifySyncFinishEvent

The `sysNotifySyncFinishEvent` is broadcast immediately after a HotSync operation has completed. Register for this notification if you need to perform post-processing after HotSync operations.

sysNotifySyncFinishEvent Specific Data

None.

Compatibility Implemented only if Notification Feature Set is present.

sysNotifySyncStartEvent

The `sysNotifySyncStartEvent` is broadcast immediately before a HotSync operation is begun. Register for this notification if you need to perform preprocessing before a HotSync operation.

sysNotifySyncStartEvent Specific Data

None.

Compatibility Implemented only if Notification Feature Set is present.

sysNotifyTimeChangeEvent

The `sysNotifyTimeChangeEvent` notification is broadcast just after the system time has been changed using `TimSetSeconds`. Register for this notification if you need to know when the time has changed.

sysNotifyTimeChangeEvent Specific Data

None.

Compatibility Implemented only if Notification Feature Set is present.



New

sysNotifyVolumeMountedEvent

The `sysNotifyVolumeMountedEvent` is broadcast when a Virtual File System Manager volume is mounted. When a volume is mounted, the VFS Manager activates the `start.prc` application on the newly mounted volume and switches applications to the Launcher or to the `start.prc` application on that volume if it has a user interface.

Register for this notification if you need to know when a volume is mounted or if you want to prevent the default behavior of the VFS Manager.

To prevent the VFS Manager from activating the `start.prc` application, set the `vfsHandledStartPrc` bit in the `handled` field. To prevent the VFS Manager from switching applications, set the `vfsHandledUIAppSwitch` bit.

sysNotifyVolumeMountedEvent Specific Data

`notifyDetailsP` points to a `VFSSlotMountParamType` or `VFSPOSEMountParamType` structure.

Compatibility Implemented only if 4.0 New Feature Set is present.



New sysNotifyVolumeUnmountedEvent

The `sysNotifyVolumeUnmountedEvent` is broadcast when a Virtual File System Manager volume is unmounted. Register for this notification if you need to know when a volume is unmounted.

sysNotifyVolumeUnmountedEvent Specific Data

`notifyDetailsP` points to a `UInt16` containing the volume reference number.

Compatibility Implemented only if 4.0 New Feature Set is present.

5

Categories

This chapter describes the category API as declared in the header file `Category.h`. It discusses the following topics:

- Category Data Structures
- Category Constants
- Category Functions

For more information on categories, see the section "Categories" on page 109 in the *Palm OS Programmer's Companion*, vol. I.

Category Data Structures

AppInfoPtr

The `AppInfoPtr` defines a pointer to an `AppInfoType` structure.

```
typedef AppInfoType *AppInfoPtr;
```

AppInfoType

The `AppInfoType` structure shown below maps category names to category indexes and unique IDs. To use the category API described in this chapter, a database's application info block must either be an `AppInfoType` structure, or it must have an `AppInfoType` structure as its first field.

```
typedef struct {
  UInt16   renamedCategories;
  Char    categoryLabels [dmRecNumCategories]
            [dmCategoryLength];
  UInt8    categoryUniqIDs[dmRecNumCategories];
  UInt8    lastUniqID;
  UInt8    padding;
} AppInfoType;
```

Allocate the application info block in the storage heap and use the DmSetDatabaseInfo function to set the database's application info ID to the local ID of this structure. Then, use the CategoryInitialize function to initialize it with a localized list of strings containing the category names.

Field Descriptions

`renamedCategories`	Used by CategorySetName as a bit field indicating which categories have been renamed. Usually cleared by a conduit.
`categoryLabels`	An array of strings containing the category names. The maximum size of the array is `dmRecNumCategories`, and the maximum length of each string in the array is `dmCategoryLength`. Both of these constants are defined in `DataMgr.h`.
`categoryUniqIDs`	Category IDs used for synchronization with the desktop database. Unique IDs generated by the device are between 0 and 127. Unique IDs generated by the desktop computer are between 128 and 255.
`lastUniqID`	Used for sorting and assigning unique IDs.

Category Constants

The following category constants are defined:

Constant	Value	Description
`categoryHideEditCategory`	10000	Used to suppress the "Edit Categories" item.
`categoryDefaultEditCategoryString`	10001	Used to show the default "Edit Categories" item.

NOTE: These constants look like system resource IDs, but they are not. To use a non-default string for the "Edit Categories" item you pass a resource ID of a string containing your title. If you want to use the default or hide the item, you pass one of these constants. They are within the system resource ID range (that is, they are greater than 10000) so that they don't conflict with any other possible value for that parameter.

Compatibility Both `categoryHideEditCategory` and `categoryDefaultEditCategoryString` are defined only if the 3.5 New Feature Set is present.

Category Functions

CategoryCreateList

Purpose Populate a popup list with a database's categories.

Prototype

```
void CategoryCreateList (DmOpenRef db,
ListType *listP, UInt16 currentCategory,
Boolean showAll, Boolean showUneditables,
UInt8 numUneditableCategories,
UInt32 editingStrID, Boolean resizeList)
```

Parameters

-> `db` Open database containing the category information you want to read.

<- `listP` Pointer to the `ListType` structure that should display the categories.

-> `currentCategory` Index of the category to select. The index is the index into the `categoryLabels` array. The default is to have the "Unfiled" category selected.

-> `showAll` `true` to include an "All" list item.

-> `showUneditables`
`true` to show uneditable categories.

-> `numUneditableCategories`
The number of categories that the user cannot edit. You should store uneditable categories at the beginning of the `categoryLabels` array. For example, it's common to have an "Unfiled" category at position zero that is not editable. This function displays the uneditable categories at the end of the popup list.

-> `editingStrID` The resource ID of a `tSTR` resource to use as the Edit Categories list item. To use the default string ("Edit Categories") pass the constant `categoryDefaultEditCategoryString`.

If you don't want users to edit categories, pass the `categoryHideEditCategory` constant.

-> `resizeList` `true` to resize the list to the number of categories. Set to `true` for popups, `false` otherwise.

Result Returns nothing.

Comments The "All" item is first in the list (if the `showAll` parameter is `true`), followed by the editable categories in the database and then the categories that cannot be edited. The option to edit categories is last in the list and can be suppressed if desired.

You rarely call this function directly. Instead, most applications use `CategorySelect`, which calls this function and fully manages the user's selection of a category in the popup list. Use `CategoryCreateList` only if you want more control over the category popup list.

This function obtains the `db` parameter's `appInfoID`, reads the `AppInfoType` structure at that location, and uses the information in it to initialize the `listP`'s items array with the names of the database's categories. You must have already allocated the structure pointed to by `listP`. `CategoryCreateList` does not display the list; use `LstPopupList` or `LstDrawList` to do so.

You must balance a call to `CategoryCreateList` with a call to `CategoryFreeList`. The `CategoryCreateList` function locks the resources for the category names. It also allocates the `listP` items array. `CategoryFreeList` unlocks all resources locked by `CategoryCreateList` and frees all memory allocated by `CategoryCreateList`.

Compatibility Implemented only if 2.0 New Feature Set is present.

The constants `categoryDefaultEditCategoryString` and `categoryHideEditCategory` are defined only if 3.5 New Feature Set is present. In earlier versions, you can suppress the Edit Categories string by passing 0 for the `editingStrID` parameter, or include the item by passing `categoryEditStrID`.

See Also `CategoryCreateListV10`

CategoryCreateListV10

Purpose Read a database's categories and set the category list.

Prototype

```
void CategoryCreateListV10 (DmOpenRef db,
ListType *lst, UInt16 currentCategory,
Boolean showAll)
```

Parameters

-> `db`	Open database containing the category information you want to read.
<- `lst`	Pointer to the `ListType` that should display the categories.
-> `currentCategory`	Index of the category to select. The index is the index into the `categoryLabels` array. The default is to have the "Unfiled" category selected.
-> `showAll`	`true` to include an "All" list item.

Result Returns nothing.

Compatibility This function corresponds to the Palm OS® 1.0 version of `CategoryCreateList`. It is obsolete.

CategoryEdit

Purpose Event handler for the Edit Categories dialog.

Prototype

```
Boolean CategoryEdit (DmOpenRef db,
UInt16 *category, UInt32 titleStrID,
UInt8 numUneditableCategories)
```

Parameters

-> `db` Open database containing the categories to be edited.

<- `category` Upon return, the index of the last category selected before the dialog was closed.

-> `titleStrID` The resource ID of a `tSTR` resource to use as the dialog's title. To use the default string ("Edit Categories"), pass the constant `categoryDefaultEditCategoryString`.

-> `numUneditableCategories` The number of categories that the user cannot edit. You should store uneditable categories at the beginning of the `categoryLabels` array. For example, it's common to have an "Unfiled" category at position zero that is not editable.

Result Returns `true` if any of the following conditions are `true`:

- The current category is renamed.
- The current category is deleted.
- The current category is merged with another category.

Comments You rarely call this function directly. The `CategorySelect` function calls it when the user chooses the Edit Category list item.

This function both displays the Edit Categories dialog and handles the result of the user actions. It updates the `AppInfoType` structure's list of categories and reassigns database records to new categories as needed. If a user deletes a category, `CategoryEdit` moves all of the records belonging to that category to the Unfiled category. If a category is renamed to be the same as an existing

category, this function moves all of the old category's records to the new category.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also CategoryEditV20, CategoryEditV10, DmMoveCategory

CategoryEditV20

Purpose Event handler for the Edit Categories dialog.

Prototype

```
Boolean CategoryEditV20 (DmOpenRef db,
UInt16 *category, UInt32 titleStrID)
```

Parameters

-> db	Database containing the categories to be edited.
<- category	Upon return, the last category selected before the dialog was closed.
-> titleStrID	The resource ID of a tSTR resource to use as the dialog's title.

Result Returns true if any of the following conditions are true:

- The current category is renamed.
- The current category is deleted.
- The current category is merged with another category.

Compatibility This function corresponds to the Palm OS 2.0 version of CategoryEdit. Implemented only if 2.0 New Feature Set is present. This function is obsolete.

See Also CategoryEdit, CategoryEditV10

CategoryEditV10

Purpose Event handler for the Edit Categories dialog.

Prototype
```
Boolean CategoryEditV10 (DmOpenRef db,
UInt16 *category)
```

Parameters

-> `db`	Open database containing the categories to be edited.
<- `category`	Upon return, the last category selected before the dialog was closed.

Result Returns `true` if any of the following conditions are true:

- The current category is renamed.
- The current category is deleted.
- The current category is merged with another category.

Compatibility This function corresponds to the Palm OS 1.0 version of `CategoryEdit`. It is obsolete.

See Also `CategoryEdit`, `CategoryEditV20`

CategoryFind

Purpose Return the index of a category given its name.

Prototype
```
UInt16 CategoryFind (DmOpenRef db,
const Char *name)
```

Parameters

-> `db`	Open database to search.
-> `name`	Category name. Pass the empty string to find the first unused category.

Result Returns the index of the category's entry in the `categoryLabels` array (see `AppInfoType`). Returns `dmAllCategories` if the category does not exist.

CategoryFreeList

Purpose Unlock or free memory locked or allocated by `CategoryCreateList`.

Prototype

```
void CategoryFreeList (DmOpenRef db,
ListType *listP, Boolean showAll,
UInt32 editingStrID)
```

Parameters

-> `db`	Open database containing the categories.
-> `listP`	Pointer to the category list. (See `ListType`.)
-> `showAll`	`true` if the list was created with an "All" category.
-> `editingStrID`	The resource ID that you passed as the `editingStrID` parameter to `CategoryCreateList`. This function unlocks the resource.

Result Returns nothing.

Comments You only need to call this function if you explicitly call `CategoryCreateList`. Typical applications call `CategorySelect`, which handles both the creation and deletion of the list.

This function frees the items in the popup list `listP`'s items array and it unlocks other resources that `CategoryCreateList` may have locked.

This function does **not** remove the categories from the passed database, and it does **not** free the `ListType` structure pointed to by `listP`. (Typically, a list is freed when its form is freed.)

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `CategoryFreeListV10`

CategoryFreeListV10

Purpose Unlock or free memory locked or allocated by `CategoryCreateListV10`.

Prototype
```
void CategoryFreeListV10 (DmOpenRef db,
ListType *lst)
```

Parameters

-> `db`	Open database containing the categories.
-> `listP`	Pointer to the category list. (See `ListType`.)

Result Returns nothing.

Compatibility This function corresponds to the Palm OS 1.0 version of `CategoryFreeList`. It is obsolete.

See Also `CategoryFreeList`

CategoryGetName

Purpose Return the name of the specified category.

Prototype
```
void CategoryGetName (DmOpenRef db, UInt16 index,
Char *name)
```

Parameters

-> `db`	Database that contains the categories.
-> `index`	Category index. This is the index into the `categoryLabels` array in the `AppInfoType` structure. You can retrieve this index from a database record's attribute word.
<- `name`	Buffer to hold category name. Buffer should be `dmCategoryLength` in size.

Result Stores the category name in the `name` buffer passed.

May display a fatal error message if the index is out of range.

Comments You can use this function to find out the name of a given database record's category. Use the DmRecordInfo call to obtain the category index from the given record. For example:

```
DmOpenRef myDB;
UInt16 record, attr, category;
Char *name;

DmRecordInfo(myDB, record, &attr, NULL, NULL);
category = attr & dmRecAttrCategoryMask;
CategoryGetName(myDB, category, name);
```

The category's name is copied into the variable you pass for the `name` parameter.

See Also CategorySetName

CategoryGetNext

Purpose Return the index of the next category after a given category.

Prototype

```
UInt16 CategoryGetNext (DmOpenRef db,
UInt16 index)
```

Parameters

-> `db`	Open database containing the categories.
-> `index`	Category index.

Result Category index of next category.

Comments The intended use of this function is to allow your users to cycle through categories. For example, the built-in applications cycle through categories when the user presses the corresponding hard-key button. (See the `ListViewNextCategory` function in the Address Book sample application for an example.) Note that categories are not displayed in the same order as they are stored.

Do not use this function for searching for a particular category or iterating through a category list.

Compatibility In Palm OS 1.0, the system chose `Unfiled` as one category.

In Palm OS 2.0 and later, the system skips both `Unfiled` and categories with empty records.

CategoryInitialize

Purpose Initialize the category names, IDs, and flags.

Prototype

```
void CategoryInitialize (AppInfoPtr appInfoP,
UInt16 localizedAppInfoStrID)
```

Parameters

`->appInfoP` Pointer to the locked application info block. See `AppInfoType`.

`->localizedAppInfoStrID` Resource ID of the localized category names. This must be a resource of the type `appInfoStringsRsc ('tAIS')`.

Result Returns nothing.

Comments Call this function at database creation time to initialize the database's categories from a list of localized strings.

`CategoryInitialize` initializes the `AppInfoType` structure that is associated with your database. It does not create the structure. To create the structure, you must allocate it in the storage heap (using `DmNewHandle`) and associate it with your database using `DmSetDatabaseInfo`.

Compatibility Implemented only if 2.0 New Feature Set is present.

CategorySelect

Purpose Process the selection and editing of categories.

Prototype

```
Boolean CategorySelect (DmOpenRef db,
const FormType *frm, UInt16 ctlID, UInt16 lstID,
Boolean title, UInt16 *categoryP,
Char *categoryName, UInt8 numUneditableCategories,
UInt32 editingStrID)
```

Parameters

-> `db`	Open database containing the categories.
-> `frm`	Form that contains the category popup list.
-> `ctlID`	ID of the popup trigger.
-> `lstID`	ID of the popup list.
-> `title`	`true` to have an "All" list item. (In general, if the trigger is in the form's title bar, it should have an "All" item. If the trigger is elsewhere in the form, it should not.)
<-> `categoryP`	Index of the selected category. The index is the index into the `categoryLabels` array.
<-> `categoryName`	Name of the selected category.
-> `numUneditableCategories`	The number of categories that the user cannot edit. You should store uneditable categories at the beginning of the `categoryLabels` array. For example, it's common to have an "Unfiled" category at position zero that is not editable. This function displays the uneditable categories at the end of the popup list.
-> `editingStrID`	The resource ID of a `tSTR` resource to use as the Edit Categories list item. To use the default string ("Edit Categories"), pass the constant `categoryDefaultEditCategoryString`. If you don't want users to edit categories, pass the `categoryHideEditCategory` constant.

Result

Returns `true` if any of the following conditions are true:

- The current category is renamed.
- The current category is deleted.
- The current category is merged with another category.

Comments

Call this function when the user taps the category popup trigger. This function handles all aspects of displaying the popup list and managing the user selection—It creates the popup list using

CategoryCreateList, displays the popup list, calls CategoryEdit if the user selects the Edit Categories item, uses CategorySetTriggerLabel to set the trigger label to the item the user selected, and then calls CategoryFreeList to free the list items array. Your application is responsible for checking the value of categoryP upon return and updating the display or changing the record's category to the new selection.

Compatibility Implemented only if 2.0 New Feature Set is present.

The constants categoryDefaultEditCategoryString and categoryHideEditCategory are defined only if 3.5 New Feature Set is present. In earlier versions, you can suppress the Edit Categories string by passing 0 for the editingStrID parameter, or include the item by passing categoryEditStrID.

See Also CategorySelectV10

CategorySelectV10

Purpose Process the selection and editing of categories.

Prototype

```
Boolean CategorySelectV10 (DmOpenRef db,
const FormType *frm, UInt16 ctlID, UInt16 lstID,
Boolean title, UInt16 *categoryP,
Char *categoryName)
```

Parameters

-> db	Open database containing the categories.
-> frm	Form that contains the category popup list.
-> ctlID	ID of the popup trigger.
-> lstID	ID of the popup list.
-> title	true to have an "All" list item. (In general, if the trigger is in the form's title bar, it should have an "All" item. If the trigger is elsewhere in the form, it should not.)

<-> `categoryP`	Index of the selected category. The index is the index into the `categoryLabels` array.
<-> `categoryName`	Name of the selected category.

Result Returns `true` if any of the following conditions are true:

- The current category is renamed.
- The current category is deleted.
- The current category is merged with another category.

Compatibility This function corresponds to the Palm OS 1.0 version of `CategorySelect`. It is obsolete.

CategorySetName

Purpose Change the category name in the `AppInfoType` structure, or delete a category.

Prototype

```
void CategorySetName (DmOpenRef db, UInt16 index,
const Char *nameP)
```

Parameters

-> `db`	Open database containing the category to change.
-> `index`	Index of category to rename.
-> `nameP`	The new category name (null-terminated), or `NULL` to delete the category.

Result Returns nothing.

Comments The `CategoryEdit` function calls this function when a user creates a new category or renames an existing category in the Edit Categories dialog. Your application does not have to call it directly.

Compatibility Implemented only if 2.0 New Feature Set is present.

CategorySetTriggerLabel

Purpose Set the label displayed by the category popup trigger.

Prototype

```
void CategorySetTriggerLabel (ControlType *ctl,
Char *name)
```

Parameters

<-> `ctl`	Pointer to control object (popup trigger) to relabel.
<-> `name`	Pointer to the name of the new category.

Result Returns nothing.

Comments The `CategorySetTriggerLabel` function calls the `CategoryTruncateName` function to truncate the category name to the maximum length. The maximum length varies, depending upon which ROM is installed in the device. (The resource `maxCategoryWidthID` contains this value. You can check the value of this resource using `ResLoadConstant`.)

NOTE: This function passes the `name` parameter to the `CategoryTruncateName` function, which means that the `name` value must be modifiable. `CategorySetTriggerLabel` does not make a copy of the string passed, so you must ensure that the string remains valid until the form is closed.

See Also `CtlSetLabel`

CategoryTruncateName

Purpose Truncate a category name so that it's short enough to display. The category name is truncated if it's longer than `maxWidth`.

Prototype

```
void CategoryTruncateName (Char *name,
UInt16 maxWidth)
```

Parameters

<-> `name`	Category name to truncate. Upon return, contains the truncated name.
-> `maxWidth`	Maximum size, in pixels, of truncated category (including ellipsis).

Result Returns nothing.

6

Clipboard

This chapter provides reference material for the clipboard API defined in `Clipboard.h`. It covers:

- Clipboard Data Structures
- Clipboard Functions

Clipboard Data Structures

ClipboardFormatType

The `ClipboardFormatType` enum specifies the type of data to add to the clipboard or retrieve from the clipboard.

```
enum clipboardFormats {
  clipboardText,
  clipboardInk,
  clipboardBitmap };
typedef enum clipboardFormats
  ClipboardFormatType;
```

Value Descriptions

`clipboardText`	Textual data. This is the most commonly used clipboard.
`clipboardInk`	Reserved.
`clipboardBitmap`	Bitmap data.

Clipboards for each type of data are separately maintained. That is, if you add a string of text to the clipboard, then add a bitmap, then ask to retrieve a `clipboardText` item from the clipboard, you will receive the string you added before the bitmap; the bitmap does not overwrite textual data and vice versa.

Clipboard Functions

ClipboardAddItem

Purpose Add the item passed to the specified clipboard. Replaces the current item (if any) of that type.

Prototype

```
void ClipboardAddItem
(const ClipboardFormatType format,
const void *ptr, UInt16 length)
```

Parameters

-> `format`	Text, ink, bitmap, etc. See `ClipboardFormatType`.
-> `ptr`	Pointer to the item to place on the clipboard.
-> `length`	Size in bytes of the item to place on the clipboard.

Result Returns nothing.

Comments The clipboard makes a copy of the data that you pass to this function. Thus, you may free any data that you've passed to the clipboard without destroying the contents of the clipboard. You may also add constant data or stack-based data to the clipboard.

See Also `FldCut`, `FldCopy`

ClipboardAppendItem

Purpose

Append data to the item on the clipboard.

Prototype

```
Err ClipboardAppendItem
(const ClipboardFormatType format,
const void *ptr, UInt16 length)
```

Parameters

-> `format`	Text, ink, bitmap, etc. See `ClipboardFormatType`. This function is intended to be used only for the `clipboardText` format.
-> `ptr`	Pointer to the data to append to the item on the clipboard.
-> `length`	Size in bytes of the data to append to the clipboard.

Result

0 upon success or `memErrNotEnoughSpace` if there is not enough space to append the data to the clipboard.

Comments

This function differs from `ClipboardAddItem` in that it does not overwrite data already on the clipboard. It allows you to create a large text item on the clipboard from several small disjointed pieces. When other applications retrieve the text from the clipboard, it's retrieved as a single unit.

This function simply appends the specified item to the item already on the clipboard without attempting to parse the format. It's assumed that you'll call it several times over a relatively short interval and that no other application will attempt to retrieve text from the clipboard before your application is finished appending.

Compatibility

Implemented only if 3.2 New Feature Set is present.

ClipboardGetItem

Purpose Return the handle of the contents of the clipboard of a specified type and the length of a clipboard item.

Prototype

```
MemHandle ClipboardGetItem
(const ClipboardFormatType format, UInt16 *length)
```

Parameters

-> `format`	Text, ink, bitmap, etc. See `ClipboardFormatType`.
<- `length`	The length in bytes of the clipboard item is returned here.

Result Handle of the clipboard item.

Comments The handle returned is a handle to the actual clipboard chunk. It is not suitable for passing to any API that modifies memory (such as `FldSetTextHandle`). Consider this to be read-only access to the chunk. Copy the contents of the clipboard to your application's own storage as soon as possible and use that reference instead of the handle returned by this function.

Don't free the handle returned by this function; it is freed when a new item is added to the clipboard.

Text retrieved from the clipboard does not have a null terminator. You must use the `length` parameter to determine the length in bytes of the string you've retrieved.

7

Controls

This chapter describes the control object API as declared in the header file `Control.h`. It discusses the following topics:

- Control Data Structures
- Control Resources
- Control Functions

For more information on controls, see the section "Controls" in the *Palm OS Programmer's Companion*, vol. I.

Control Data Structures

ButtonFrameType

The `ButtonFrameType` enum specifies the border style for the button. It defines values for the `frame` field of `ControlAttrType`.

```
enum buttonFrames {noButtonFrame,
      standardButtonFrame, boldButtonFrame,
      rectangleButtonFrame};
typedef enum buttonFrames ButtonFrameType;
```

Value Descriptions

`noButtonFrame`	The button has no border.
`standardButtonFrame`	Standard button rectangular border with rounded corners.
`boldButtonFrame`	Bolded rectangular border with rounded corners.
`rectangleButtonFrame`	Rectangular border with square corners.

ControlAttrType

The `ControlAttrType` bit field specifies the control's visible characteristics. It is defined as follows:

```
typedef struct  {
  UInt8 usable      :1;
  UInt8 enabled     :1;
  UInt8 visible     :1;
  UInt8 on          :1;
  UInt8 leftAnchor  :1;
  UInt8 frame       :3;
  UInt8 drawnAsSelected : 1;
  UInt8 graphical   :1;
  UInt8 vertical    :1;
} ControlAttrType;
```

WARNING! Palm, Inc. does not support or provide backward compatibility for the `ControlAttrType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

Your code should treat the `ControlAttrType` structure as opaque. Use the functions specified in the descriptions below to retrieve and set each value. Do not attempt to change structure member values directly.

Field Descriptions

`usable` — If 0, the control is not considered to be part of the interface of the current application, and it doesn't appear on screen. You can use `CtlSetUsable`, `CtlShowControl`, or `CtlHideControl` to set or clear this value.

`enabled` — If 0, the control is visible but doesn't respond to the pen. This value is set by `CtlSetEnabled` and returned by `CtlEnabled`.

`visible`	Set and cleared internally when the control is drawn (`CtlDrawControl`) and erased (`CtlEraseControl`).
`on`	If set, the control has the value "on." For example, a check box that has the on value has a check mark displayed in it. Use `CtlGetValue` and `CtlSetValue` to retrieve and set this value.
`leftAnchor`	Used by controls that expand and shrink their width when the label is changed. If this attribute is set, the left bound of the control is fixed.
`frame`	The type of frame drawn around the button controls. See `ButtonFrameType` for possible values. Only button controls use this attribute; for all other controls, the `ControlStyleType` determines the frame.
`drawnAsSelected`	Used on Palm OS® release 3.5 for button controls that contain no text (indicating that the button is displayed on top of a bitmap). If set, the button is drawn as inverted. If clear, the button is drawn normally.
`graphical`	If set, the control is a graphical control, slider, or feedback slider.
`vertical`	Not currently used.

Compatibility The `drawnAsSelected`, `graphical`, and `vertical` attributes are only present if 3.5 New Feature Set is present.

ControlPtr

The `ControlPtr` is a pointer to a `ControlType` structure.

```
typedef ControlType* ControlPtr;
```

ControlStyleType

The `ControlStyleType` enum specifies values for the `ControlType` `style` field, which specifies the type of the control (button, push button, and so on).

```
enum controlStyles {buttonCtl, pushButtonCtl,
    checkboxCtl, popupTriggerCtl,
    selectorTriggerCtl, repeatingButtonCtl,
    sliderCtl, feedbackSliderCtl};
typedef enum controlStyles ControlStyleType;
```

Value Descriptions

`buttonCtl`	Button. Buttons display a text label in a box. The `ButtonFrameType` specifies the type of box.
`pushButtonCtl`	Push button. Selecting a push button inverts its display so that it appears highlighted.
`checkboxCtl`	Check box. Check boxes display a setting of either on (checked) or off (unchecked)
`popupTriggerCtl`	Popup trigger. Popup triggers display a graphic element followed by a text label. They are used to display popup lists.
`selectorTriggerCtl`	Selector trigger. Selector triggers display a text label surrounded by a gray rectangular frame. The control expands or contracts to the width of the new label.
`repeatingButtonCtl`	Repeating button. Repeating buttons look like buttons; however, a repeating button is repeatedly selected if the user holds the pen on it.

`sliderCtl`	Slider. Sliders display two bitmaps: one representing the current value (the thumb), and another representing the scale of available values. The user can slide the thumb to the left or the right to change the value.
`feedbackSliderCtl`	Feedback slider. A feedback slider looks like a slider; however, a feedback slider sends events each time the thumb moves while the pen is still down. A regular slider sends an event only when the user releases the pen.

Compatibility The `sliderCtl` and `feedbackSliderCtl` values are only defined if 3.5 New Feature Set is present.

ControlType

The `ControlType` structure defines the type and characteristics of a control. It is defined as follows:

```
typedef struct {
  UInt16              id;
  RectangleType       bounds;
  Char *              text;
  ControlAttrType     attr;
  ControlStyleType    style;
  FontID              font;
  UInt8               group;
  UInt8               reserved;
} ControlType;
```

WARNING! Palm, Inc. does not support or provide backward compatibility for the `ControlType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

Your code should treat the `ControlType` structure as opaque. The fields in the struct are set by values you specify when you create the control resource, and they typically do not change. Use the functions specified in the descriptions below to retrieve and set the values. Do not attempt to change structure member values directly.

Field Descriptions

`id` ID value you specified when you created the control resource.

`bounds` Bounds of the control, in window-relative coordinates. The control's text label is clipped to the control's bounds. The control's frame is drawn around (outside) the bounds of the control. `FrmGetObjectBounds` and `FrmSetObjectBounds` retrieve and set this value.

`text` Pointer to the control's label. If `text` is `NULL`, the control has no label. Use `CtlGetLabel` and `CtlSetLabel` to retrieve and set this value.

`attr` Control attributes. See `ControlAttrType`.

`style` Style of the control. See `ControlStyleType`.

`font` Font to use to draw the control's label.

`group` Group ID of a push button or a check box that is part of an exclusive group. The control routines don't automatically turn one control off when another is selected. It's up to the application or a higher-level object, like a dialog box, to manage this.

`reserved` Reserved for future use.

GraphicControlType

The `GraphicControlType` struct defines a graphical control. A graphical control is like any other control except that it displays a bitmap in place of the text label.

```
typedef struct GraphicControlType {
  UInt16              id;
  RectangleType       bounds;
  DmResID             bitmapID;
  DmResID             selectedBitmapID;
  ControlAttrType     attr;
  ControlStyleType    style;
  FontID              unused;
  UInt8               group;
  UInt8               reserved;
} GraphicControlType;
```

WARNING! Palm, Inc. does not support or provide backward compatibility for the `GraphicControlType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

Your code should treat the `GraphicControlType` structure as opaque. The fields in the struct are set by values you specify when you create the control resource, and they typically do not change. Use the functions specified in the descriptions below to retrieve and set the values. Do not attempt to change structure member values directly.

Field Descriptions

`id`	ID value you specified when you created the control resource.
`bounds`	Bounds of the control, in window-relative coordinates. The control's frame is drawn around (outside) the bounds of the control. `FrmGetObjectBounds` and `FrmSetObjectBounds` retrieve and set this value.
`bitmapID`	Resource ID of the bitmap to display in the button. You can use `CtlSetGraphics` to change this value.

`selectedBitmapID`	If the button should show a different bitmap when selected, this field contains the resource ID of that bitmap. You typically use this field for push buttons or repeating buttons. `CtlSetGraphics` can change this value.
`attr`	Control attributes. See `ControlAttrType`. For a graphical control, the `graphical` attribute must be set. The APIs described in the `ControlAttrType` section can be used to access the bitfields here. Because the `ControlAttrType` APIs take a `ControlType*` as an argument, the `GraphicControlType*` should be cast to a `ControlType*` when making the API calls.
`style`	Style of the control. See `ControlStyleType`. A graphical control can be any type of control other than `checkboxCtl`.
`unused`	Unused.
`group`	Group ID of a push button that is part of an exclusive group. The control routines don't automatically turn one control off when another is selected. It's up to the application or a higher-level object, like a dialog box, to manage this.
`reserved`	Reserved for future use.

Compatibility This struct is defined only if 3.5 New Feature Set is present.

SliderControlType

The `SliderControlType` struct defines a slider control or a feedback slider control.

```
typedef struct SliderControlType {
  UInt16                id;
  RectangleType         bounds;
  DmResID               thumbID;
  DmResID               backgroundID;
  ControlAttrType       attr;
  ControlStyleType      style;
  UInt8                 reserved;
  Int16                 minValue;
  Int16                 maxValue;
  Int16                 pageSize;
  Int16                 value;
  MemPtr                activeSliderP;
} SliderControlType;
```

WARNING! Palm, Inc. does not support or provide backward compatibility for the `SliderControlType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

Your code should treat the `SliderControlType` structure as opaque. The fields in the struct are set by values you specify when you create the control resource, and they typically do not change. You can use `CtlSetSliderValues` to set new minimum, maximum, page size, and current values, and `CtlGetSliderValues` to retrieve these values. Do not attempt to change structure member values directly.

Field Descriptions

`id`	ID value you specified when you created the control resource.
`bounds`	Bounds of the control, in window-relative coordinates. `FrmGetObjectBounds` and `FrmSetObjectBounds` retrieve and set this value.

`thumbID`	Resource ID of the bitmap to use for the slider knob (called the "thumb"). If `NULL`, the default bitmap is used.
`backgroundID`	Resource ID of the bitmap to use for the slider background. If `NULL`, the default bitmap is used.
`attr`	Control attributes. See `ControlAttrType`. For a slider, the `graphical` attribute is set. The APIs described in the `ControlAttrType` section can be used to access the bitfields here. Because the `ControlAttrType` APIs take a `ControlType*` as an argument, the `SliderControlType*` should be cast to a `ControlType*` when making the API calls.
`style`	Style of the control. See `ControlStyleType`. Must be `sliderCtl` or `feedbackSliderCtl`.
`reserved`	Reserved for future use.
`minValue`	Value of the slider when the thumb is all the way to the left.
`maxValue`	Value of the slider when the thumb is all the way to the right.
`pageSize`	Amount by which to increase or decrease the slider value when the user taps to the right or left of the thumb.
`value`	Current value represented by the slider. Use `CtlGetValue` and `CtlSetValue` to retrieve and set this value.
`activeSliderP`	Pointer to a memory location used when the slider is active. A slider is active if it is currently being drawn or if it is tracking the pen. If the slider is inactive, this pointer is `NULL`.

Compatibility This struct is defined only if 3.5 New Feature Set is present.

Control Resources

Different resources are associated with different controls, as follows:

- Button—Button Resource (tBTN)
- Popup trigger— Popup Trigger Resource (tPUT)
- Selector trigger—Selector Trigger Resource (tSLT)
- Repeat control—Repeating Button Resource (tREP)
- Push button—Push Button Resource (tPBN)
- Check box—Check Box Resource (tCBX)
- Slider— Slider Resource (tsld)
- Feedback slider— Feedback Slider Resource (tslf)

Control Functions

CtlDrawControl

Purpose Draw a control object (and the text or graphic in it) on screen.

Prototype

```
void CtlDrawControl (ControlType *controlP)
```

Parameters

`-> controlP` Pointer to the control object to draw. (See `ControlType`.)

Result Returns nothing.

Comments The control is drawn only if its `usable` attribute is `true`. This function sets the `visible` attribute to `true`.

Compatibility In releases prior to Palm OS® 3.5, it is common to create graphical buttons by drawing a button with no text label on top of a bitmap. If 3.5 New Feature Set is present, you should use graphical controls instead. (See `GraphicControlType`.) `CtlDrawControl` attempts to provide backward compatibility for the old-style graphical buttons.

See Also `CtlSetUsable`, `CtlShowControl`

CtlEnabled

Purpose Return `true` if the control responds to the pen.

Prototype
```
Boolean CtlEnabled (const ControlType *controlP)
```

Parameters `-> controlP` Pointer to control object. (See `ControlType`.)

Result Returns `true` if the controls object responds to the pen; `false` if not.

Comments This function provides no indication of whether the control is visible on the screen. A control that doesn't respond to the pen may be visible, and if so, its appearance is no different from controls that do respond to the pen. You might use such a control to display some state of your application that cannot be modified.

See Also `CtlSetEnabled`

CtlEraseControl

Purpose Erase a usable and visible control object and its frame from the screen.

Prototype
```
void CtlEraseControl (ControlType *controlP)
```

Parameters `-> controlP` Pointer to control object to erase. (See `ControlType`.)

Comments This function sets the `visible` attribute to `false`. If 3.5 New Feature Set is present, it also sets the `drawnAsSelected` attribute to `false`.

Don't call this function directly; instead, use `FrmHideObject`, which calls this function.

CtlGetLabel

Purpose Return a character pointer to a control's text label.

Prototype
```
const Char *CtlGetLabel
(const ControlType *controlP)
```

Parameters

`-> controlP`	Pointer to control object. (See `ControlType`.)

Result Returns a pointer to a null-terminated string.

Comments Make sure that `controlP` is not a graphical control or a slider control. The graphical control and slider control structures do not contain a text label field.

See Also `CtlSetLabel`

CtlGetSliderValues

Purpose Return current values used by a slider control.

Prototype
```
void CtlGetSliderValues (const ControlType *ctlP,
UInt16 *minValueP, UInt16 *maxValueP,
UInt16 *pageSizeP, UInt16 *valueP)
```

Parameters

-> `ctlP`	Pointer to a control object. (See `ControlType`.)
<- `minValueP`	The slider's minimum value. Pass `NULL` if you don't want to retrieve this value.
<- `maxValueP`	The slider's maximum value. Pass `NULL` if you don't want to retrieve this value.
<- `pageSizeP`	The slider's page size value. Pass `NULL` if you don't want to retrieve this value.
<- `valueP`	The slider's current value. Pass `NULL` if you don't want to retrieve this value.

Result Returns nothing. The slider's values are returned in the parameters to this function.

Comments If `ctlP` is not a slider or a feedback slider, this function immediately returns.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `CtlSetSliderValues`, `SliderControlType`

CtlGetValue

Purpose Return the current value of the specified control.

Prototype

```
Int16 CtlGetValue (const ControlType *controlP)
```

Parameters

`-> controlP` Pointer to a control object. (See `ControlType`.)

Result Returns the current value of the control. For most controls the return value is either 0 (off) or 1 (on). For sliders, this function returns the value of the `value` field.

See Also `CtlSetValue`, `FrmGetControlGroupSelection`, `FrmSetControlGroupSelection`, `FrmGetControlValue`, `FrmSetControlValue`

CtlHandleEvent

Purpose Handle event in the specified control object.

Prototype

```
Boolean CtlHandleEvent (ControlType *controlP,
EventType *pEvent)
```

Parameters

`-> controlP` Pointer to control object. (See `ControlType`.)

`-> pEvent` Pointer to an `EventType` structure.

Result Returns `true` if an event is handled by this function. Events that are handled are:

- `penDownEvent` — If the pen is within the bounds of the control

- ctlEnterEvent, ctlRepeatEvent, and ctlExitEvent— If the control ID in the event data matches the control's ID.

Comments The control object must be usable, visible, and respond to the pen for this function to handle the event.

When this routine receives a `penDownEvent`, it checks if the pen position is within the bounds of the control object. If it is, a `ctlEnterEvent` is added to the event queue and the routine exits.

When this routine receives a `ctlEnterEvent`, the control object is redrawn as necessary as either selected or deselected, depending on its previous state.

When this routine receives a `ctlEnterEvent` or `ctlRepeatEvent`, it checks that the control ID in the passed event record matches the ID of the specified control. If they match, this routine tracks the pen until it comes up or until it leaves the object's bounds. When that happens, `ctlSelectEvent` is sent to the event queue if the pen came up in the bounds of the control. If the pen exits the bounds, a `ctlExitEvent` is sent to the event queue.

CtlHideControl

Purpose Set a control's `usable` attribute to `false` and erase the control from the screen.

Prototype
```
void CtlHideControl (ControlType *controlP)
```

Parameters

`-> controlP`	Pointer to the control object to hide. (See ControlType.)

Result Returns nothing.

Comments A control that is not usable doesn't draw and doesn't respond to the pen.

This function is the same as CtlEraseControl except that it also sets `usable` to `false` (in addition to setting `visible` to `false`).

Don't call this function directly; instead, use FrmHideObject, which performs the same function and works for all user interface objects.

See Also CtlShowControl

CtlHitControl

Purpose Simulate tapping a control. This function adds a ctlSelectEvent to the event queue.

Prototype

```
void CtlHitControl (const ControlType *controlP)
```

Parameters

`-> controlP`	Pointer to a control object. (See ControlType.)

Result Returns nothing.

Comments Useful for testing.

CtlNewControl

Purpose Create a new control object dynamically and install it in the specified form.

Prototype

```
ControlType *CtlNewControl (void **formPP,
UInt16 ID, ControlStyleType style,
const Char *textP, Coord x, Coord y, Coord width,
Coord height, FontID font, UInt8 group,
Boolean leftAnchor)
```

Parameters

`<-> formPP`	Pointer to the pointer to the form in which the new control is installed. This value is not a handle; that is, the `formPP` value may change if the object moves in memory. In subsequent calls, always use the new `formPP` value returned by this function.
`-> ID`	Symbolic ID of the control.

`-> style`	A `ControlStyleType` value specifying the kind of control to create: button, push button, repeating button, check box, popup trigger, or popup selector. To create a graphical control or slider control dynamically, use `CtlNewGraphicControl` or `CtlNewSliderControl`, respectively.
`-> textP`	Pointer to the control's label text. If `textP` is `NULL`, the control has no label. Only buttons, push buttons, and text boxes have text labels. Because the contents of this pointer are copied into their own buffer, you can free the `textP` pointer any time after the `CtlNewControl` function returns. The buffer into which this string is copied is freed automatically when you remove the control from the form or delete the form.
`-> x`	Horizontal coordinate of the upper-left corner of the control's boundaries, relative to the window in which it appears.
`-> y`	Vertical coordinate of the upper-left corner of the control's boundaries, relative to the window in which it appears.
`-> width`	Width of the control, expressed in pixels. Valid values are 1–160. If the value of either of the `width` or `height` parameters is 0, the control is sized automatically as necessary to display the text passed as the value of the text parameter.
`-> height`	Height of the control, expressed in pixels. Valid values are 1–160. If the value of either of the `width` or `height` parameters is 0, the control is sized automatically as necessary to display the text passed as the value of the text parameter.
`-> font`	Font used to draw the control's label.

`-> group`	Group ID of a push button or a check box that is part of an exclusive group. The control routines don't turn one control off automatically when another is selected. It's up to the application or a higher-level object, such as a dialog box, to manage this.
`-> leftAnchor`	`true` specifies that the left bound of this control is fixed. This attribute is used by controls that resize dynamically in response to label text changes.

Result Returns a pointer to the new control.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `CtlValidatePointer`, `FrmRemoveObject`

CtlNewGraphicControl

Purpose Create a new graphical control dynamically and install it in the specified form.

Prototype

```
GraphicControlType *CtlNewGraphicControl
(void **formPP, UInt16 ID, ControlStyleType style,
DmResID bitmapID, DmResID selectedBitmapID,
Coord x, Coord y, Coord width, Coord height,
UInt8 group, Boolean leftAnchor)
```

Parameters

`<-> formPP`	Pointer to the pointer to the form in which the new control is installed. This value is not a handle; that is, the `formPP` value may change if the object moves in memory. In subsequent calls, always use the new `formPP` value returned by this function.
`-> ID`	Symbolic ID of the control.

`-> style`	A `ControlStyleType` value specifying the kind of control to create: button, push button, popup trigger, repeating button, or popup selector. Graphic controls cannot be check boxes.
`-> bitmapID`	Resource ID of the bitmap to display on the control.
`-> selectedBitmapID`	Resource ID of the bitmap to display when the control is selected, if different from `bitmapID`.
`-> x`	Horizontal coordinate of the upper-left corner of the control's boundaries, relative to the window in which it appears.
`-> y`	Vertical coordinate of the upper-left corner of the control's boundaries, relative to the window in which it appears.
`-> width`	Width of the control, expressed in pixels. Valid values are 1–160.
`-> height`	Height of the control, expressed in pixels. Valid values are 1–160.
`-> group`	Group ID of a push button that is part of an exclusive group. The control routines don't turn one control off automatically when another is selected. It's up to the application or a higher-level object, such as a dialog box, to manage this.
`-> leftAnchor`	`true` specifies that the left bound of this control is fixed.

Result Returns a pointer to the new graphical control. See `GraphicControlType`.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `CtlNewSliderControl`, `CtlNewControl`, `CtlValidatePointer`, `FrmRemoveObject`

CtlNewSliderControl

Purpose Create a new slider or feedback slider dynamically and install it in the specified form.

Prototype

```
SliderControlType *CtlNewSliderControl
(void **formPP, UInt16 ID, ControlStyleType style,
DmResID thumbID, DmResID backgroundID, Coord x,
Coord y, Coord width, Coord height,
UInt16 minValue, UInt16 maxValue, UInt16 pageSize,
UInt16 value)
```

Parameters

<-> `formPP`	Pointer to the pointer to the form in which the new control is installed. This value is not a handle; that is, the `formPP` value may change if the object moves in memory. In subsequent calls, always use the new `formPP` value returned by this function.
-> `ID`	Symbolic ID of the slider.
-> `style`	Either `sliderCtl` or `feedbackSliderCtl`. See `ControlStyleType`.
-> `thumbID`	Resource ID of the bitmap to display as the slider thumb. The slider thumb is the knob that the user can drag to change the slider's value. To use the default thumb bitmap, pass `NULL` for this parameter.
-> `backgroundID`	Resource ID of the bitmap to display as the slider background. To use the default background bitmap, pass `NULL` for this parameter.
-> `x`	Horizontal coordinate of the upper-left corner of the slider's boundaries, relative to the window in which it appears.
-> `y`	Vertical coordinate of the upper-left corner of the slider's boundaries, relative to the window in which it appears.

-> `width`	Width of the slider, expressed in pixels. Valid values are 1–160.
-> `height`	Height of the slider, expressed in pixels. Valid values are 1–160.
-> `minValue`	Value of the slider when its thumb is all the way to the left.
-> `maxValue`	Value of the slider when its thumb is all the way to the right.
-> `pageSize`	Amount by which to increase or decrease the slider's value when the user clicks to the right or left of the thumb.
-> `value`	The initial value to display in the slider.

Result Returns a pointer to the new slider control. See `SliderControlType`.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `CtlNewGraphicControl`, `CtlNewControl`, `CtlValidatePointer`, `FrmRemoveObject`

CtlSetEnabled

Purpose Set a control as enabled or disabled. Disabled controls do not respond to the pen.

Prototype

```
void CtlSetEnabled (ControlType *controlP,
Boolean usable)
```

Parameters

-> `controlP`	Pointer to a control object. (See `ControlType`.)
-> `usable`	`true` to enable the control; `false` to disable the control.

Result Returns nothing.

Comments If you disable a visible control, the control is still displayed, and its appearance is no different from controls that do respond to the pen. You might use such a control to inform your users of some state of your application that cannot be modified.

See Also CtlEnabled

CtlSetGraphics

Purpose Set the bitmaps for a graphical control and redraw the control if it is visible.

Prototype

```
void CtlSetGraphics (ControlType *ctlP,
DmResID newBitmapID, DmResID newSelectedBitmapID)
```

Parameters

-> `ctlP` Pointer to a graphical control object. (See GraphicControlType.)

-> `newBitmapID` Resource ID of a new bitmap to display on the control, or `NULL` to use the current bitmap.

-> `newSelectedBitmapID` Resource ID of a new bitmap to display when the control is selected, or `NULL` to use the current selected bitmap.

Result Returns nothing.

Comments If `ctlP` is not a graphical control, this function immediately returns.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also GraphicControlType

CtlSetLabel

Purpose Set the current label for the specified control object and redraw the control if it is visible.

Prototype

```
void CtlSetLabel (ControlType *controlP,
const Char *newLabel)
```

Parameters

`-> controlP`	Pointer to a control object. (See `ControlType`.)
`-> newLabel`	Pointer to the new text label. Must be a `NULL`-terminated string.

Result Returns nothing.

Comments This function resizes the width of the control to the size of the new label.

This function stores the `newLabel` pointer in the control's data structure. It doesn't make a copy of the string that is passed in. Therefore, if you use `CtlSetLabel`, you must manage the string yourself. You must ensure that it persists for as long as it is being displayed (that is, for as long as the control is displayed or until you call `CtlSetLabel` with a new string), and you must free the string after it is no longer in use (typically after the form containing the control is freed).

If you never use `CtlSetLabel`, you do not need to worry about freeing a control's label.

Make sure that `controlP` is not a graphical control or a slider control. The graphical controls and slider control structures do not contain a text label field, so attempting to set one will crash your application.

See Also `CtlGetLabel`

CtlSetSliderValues

Purpose Change a slider control's values and redraw the slider if it is visible.

Prototype

```
void CtlSetSliderValues (ControlType *ctlP,
const UInt16 *minValueP, const UInt16 *maxValueP,
const UInt16 *pageSizeP, const UInt16 *valueP)
```

Parameters

-> `ctlP`	Pointer to an inactive slider or feedback slider control. (See `SliderControlType`.)
-> `minValueP`	Pointer to a new value to use for the slider's minimum or `NULL` if you don't want to change this value.
-> `maxValueP`	Pointer to a new value to use for the slider's maximum, or `NULL` if you don't want to change this value.
-> `pageSizeP`	Pointer to a new value to use for the slider's page size, or `NULL` if you don't want to change this value.
-> `valueP`	Pointer to a new value to use for the current value, or `NULL` if you don't want to change this value.

Result Returns nothing.

Comments The control's style must be `sliderCtl` or `feedbackSliderCtl`, and it not be currently tracking the pen. If the slider is currently tracking the pen, use `CtlSetValue` to set the `value` field.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `CtlGetSliderValues`, `SliderControlType`

CtlSetUsable

Purpose Set a control to usable or not usable.

Prototype
```
void CtlSetUsable (ControlType *controlP,
Boolean usable)
```

Parameters

`-> controlP`	Pointer to a control object. (See `ControlType`.)
`-> usable`	`true` to have the control be usable; `false` to have the control be not usable.

Result Returns nothing.

Comments A control that is not usable doesn't draw and doesn't respond to the pen.

This function doesn't usually update the control.

See Also `CtlEraseControl`, `CtlHideControl`, `CtlShowControl`

CtlSetValue

Purpose Set the current value of the specified control. If the control is visible, it's redrawn.

Prototype
```
void CtlSetValue (ControlType *controlP,
Int16 newValue)
```

Parameters

`-> controlP`	Pointer to a control object. (See `ControlType`.)
`-> newValue`	New value to set for the control. For sliders, specify a value between the slider's minimum and maximum. For graphical controls, push buttons, or check boxes, specify 0 for off, nonzero for on.

Result Returns nothing.

Comments This function works only with graphical controls, sliders, push buttons, and check boxes. If you set the value of any other type of control, the behavior is undefined.

Compatibility Sliders and graphical controls are only supported if 3.5 New Feature Set is present.

See Also CtlGetValue, FrmGetControlGroupSelection, FrmSetControlGroupSelection, FrmGetControlValue, FrmSetControlValue

CtlShowControl

Purpose Set a control's `usable` attribute to `true` and draw the control on the screen. This function calls CtlDrawControl.

Prototype

```
void CtlShowControl (ControlType *controlP)
```

Parameters `-> controlP` Pointer to a control object. (See ControlType.)

Result Returns nothing.

Comments If the control is already usable, this function is the functional equivalent of `CtlDrawControl`.

Sets the `visible` and the `usable` attributes to `true`. (See ControlAttrType.)

Don't use this function directly; instead use FrmShowObject, which does the same thing.

See Also CtlHideControl

CtlValidatePointer

Purpose Returns `true` if the specified pointer references a valid control object.

Prototype
```
Boolean CtlValidatePointer
(const ControlType *controlP)
```

Parameters -> `controlP` Pointer to a control. (See `ControlType`.)

Result Returns `true` when passed a valid pointer to a control; otherwise, returns `false`.

Comments For debugging purposes; do not include this function in commercial products. In debug builds, this function displays a dialog and waits for the debugger when an error occurs.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FrmValidatePtr`, `WinValidateHandle`

8

Date and Time Selector

The Palm OS® UI provides two system resources for accepting date and time input values. These resources are dialog boxes that contain UI gadgetry for entering dates and times. The Palm OS UI also provides routines to manage the interaction with these resources. This chapter describes those functions.

The API described in this chapter is declared in the header files `Day.h`, `SelDay.h`, `SelTime.h`, and `SelTimeZone.h`.

Date and Time Selections Data Structures

SelectDayType

```
typedef enum {
  selectDayByDay,      // return d/m/y
  selectDayByWeek,  // return d/m/y with d as
                     // same day of the week
  selectDayByMonth // return d/m/y with d as
                    // same day of the month
} SelectDayType;
```

DaySelectorType

```
typedef struct DaySelectorType {
  RectangleType  bounds;
  Boolean        visible;
  UInt8          reserved1;
  Int16         visibleMonth; // month actually
                               // displayed
```

```
  Int16          visibleYear;  // year actually
                                 // displayed
  DateTimeType   selected;
  SelectDayType  selectDayBy;
  UInt8          reserved2;
} DaySelectorType;
```

HMSTime

```
typedef struct {
  UInt8 hours;
  UInt8 minutes;
  UInt8 seconds;
  UInt8 reserved;
} HMSTime;
```

Date and Time Selection Functions

DayDrawDays

Purpose Draw only the days-of-the-month portion of a day selector control object.

Prototype

```
void DayDrawDays
(const DaySelectorType *selectorP)
```

Parameters `-> selectorP` Pointer to the control object to draw.

Result Nothing.

Comments This function is used when the year or month changes. Only drawing the portion of the control that presents the days of the month avoids the flicker that would occur if the week titles were redrawn.

See Also DayDrawDaySelector

DayDrawDaySelector

Purpose Draw a day selector control object on screen.

Prototype
```
void DayDrawDaySelector
(const DaySelectorType *selectorP)
```

Parameters

`-> selectorP` Pointer to the control object to draw.

Result Nothing.

Comments The control is drawn only if it is usable.

See Also DayDrawDays

DayHandleEvent

Purpose Handle event in the specified control. This routine handles two types of events, penDownEvent and ctlEnterEvent.

Prototype
```
Boolean DayHandleEvent
(DaySelectorType *selectorP,
const EventType *pEvent)
```

Parameters

`-> selectorP` Pointer to control object.

`-> pEvent` Pointer to an `EventType` structure.

Result `true` if the event was handled or `false` if it was not.

Posts a daySelectEvent with information on whether to use the date.

Comments A date is used if the user selects a day in the visible month.

When this routine receives a penDownEvent, it checks if the pen position is within the bounds of the control object. If it is, a `dayEnterEvent` is added to the event queue and the routine exits.

When this routine receives a `dayEnterEvent`, it checks that the control id in the event record matches the id of the control specified. If they match, this routine will track the pen until it comes up in the bounds in which case daySelectEvent is sent.

If the pen exits the bounds a `dayExitEvent` is sent.

SelectDay

Purpose Display a form showing a date; allow user to select a different date.

Prototype

```
Boolean SelectDay
(const SelectDayType selectDayBy, Int16 *month,
Int16 *day, Int16 *year, const Char *title)
```

Parameters

`selectDayBy` The method by which the user should choose the day. Possible values are `selectDayByDay`, `selectDayByWeek`, and `selectDayByMonth`. See SelectDayType

`<-> month, day, year` Month, day, and year selected.

`-> title` String title for the dialog.

Result `true` if the OK button was pressed. If `true`, `month`, `day`, and `year` contain the new date.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `SelectDayV10`

SelectDayV10

Purpose Display a form showing a date, allow user to select a different date.

Prototype
```
Boolean SelectDayV10 (Int16 *month, Int16 *day,
Int16 *year, const Char *title)
```

Parameters

`<-> month, day, year` Month, day, and year selected. The initial values passed in these parameters must be valid.

`-> title` String title for the dialog.

Result Returns `true` if the OK button was pressed. In that case, the parameters passed are changed.

Compatibility This function corresponds to the 1.0 version of `SelectDay`.

See Also `SelectDay`

SelectOneTime

Purpose Display a form showing the time and allow the user to select a different time.

Prototype
```
Boolean SelectOneTime (Int16 *hour, Int16 *minute,
const Char *titleP)
```

Parameters

`<-> hour` The hour selected in the form.

`<-> minute` The minute selected in the form.

`-> titleP` A pointer to a string to display as the title. Doesn't change as the function executes.

Result Returns `true` if the user selects OK and `false` otherwise. If `true` is returned, the values in `hour` and `minute` have probably been changed.

Comments Use this function instead of `SelectTime` if you want to display a dialog that specifies a single point in time, not a range of time from start to end.

Compatibility Implemented only if 3.1 New Feature Set is present.

See Also `SelectTimeV33`

SelectTime

Purpose Display a form showing a start and end time. Allow the user to select a different time.

Prototype

```
Boolean SelectTime (TimeType *startTimeP,
TimeType * EndTimeP, Boolean untimed, const Char
*titleP, Int16 startOfDay, Int16 endOfDay,
Int16 startOfDisplay)
```

Parameters

`<-> startTimeP, EndTimeP`
Pointers to values of type `TimeType`. Pass values to display in these two parameters. If the user makes a selection and taps the OK button, the selected values are returned here.

`-> untimed` Pass in `true` to indicate that no time is selected. If the user sets the time to no time then `startTimeP` and `EndTimeP` are both set to the constant `noTime (-1)`.

`-> titleP` A pointer to a string to display as the title. Doesn't change as the function executes.

`-> startOfDay` The hour that the hour list displays at its top. To see earlier hours, the user can scroll the list up. The value must be between 0 to 12, inclusive.

`-> endOfDay` `The` hour used when the "All Day" button is selected.

`-> startOfDisplay`
First hour initially visible.

Result Returns `true` if the user selects OK and `false` otherwise. If `true` is returned, the values in `hour` and `minute` have probably been changed.

Comments This version of `SelectTime` adds the `endOfDay` and `startOfDisplay` functionality.

Compatibility Implemented if 3.5 New Feature Set is present.

See Also `SelectDay`, `SelectOneTime`

SelectTimeV33

Purpose Display a form showing the time and allow the user to select a different time.

This function is obsolete and should not be used.

Prototype

```
Boolean SelectTimeV33 (TimeType *startTimeP,
TimeType *EndTimeP, Boolean untimed,
const Char *titleP, Int16 startOfDay)
```

Parameters

`<-> startTimeP, EndTimeP` Pointers to values of type `TimeType`. Pass values to display in these two parameters. If the user makes a selection and taps the OK button, the selected values are returned here.

`-> untimed` Pass in `true` to indicate that no time is selected. If the user sets the time to no time then `startTimeP` and `EndTimeP` are both set to the constant `noTime (-1)`.

`-> titleP` A pointer to a string to display as the title. Doesn't change as the function executes.

`-> startOfDay` The hour that the hour list displays at its top. To see earlier hours, the user can scroll the list up. The value must be between 0 to 12, inclusive.

Result Returns `true` if the user selects OK and `false` otherwise. If `true` is returned, the values in `hour` and `minute` have probably been changed.

Comments

NOTE: Obsolete functions are provided ONLY for backward compatibility; for example, so a 1.0 application will work on 3.x OS releases. New code should not call these routines!

See Also SelectDay, SelectOneTime



New SelectTimeZone

Purpose

Display a form that allows the user to select a different time zone.

Prototype

```
Boolean SelectTimeZone (Int16 *ioTimeZoneP,
LmLocaleType *ioLocaleInTimeZoneP,
const Char *titleP, Boolean showTimes,
Boolean anyLocale)
```

Parameters

<-> `ioTimeZoneP`

A pointer to the time zone, given as minutes east of Greenwich Mean Time (GMT). The initial value is used as the initial selection in the form. Upon return, this parameter contains a pointer to the new time zone that the user selected.

<-> `ioLocaleInTimeZoneP`

A pointer to a locale (see LmLocaleType) that identifies the time zone country. This parameter is used for countries that share a time zone, such as Canada and Chile.

If the time zone specified by `ioTimeZoneP` is specific to a particular country, you do not have to initialize this parameter. Instead, set the `anyLocale` parameter to `true` to have this parameter ignored upon entry.

-> `titleP`

A string to use as the title for the dialog. Pass `NULL` to use the default title, which is "Set Time Zone".

-> `showTimes`	If `true`, the dialog shows the correct times in both the current and newly selected time zones. If `false`, the dialog doesn't show the current time. Using `false` provides a larger area for the list of time zones.
-> `anyLocale`	If `true`, ignore `ioLocaleInTimeZoneP` upon entry.

Result Returns `true` if the user clicked the OK button in the dialog to change the time zone, or `false` if the user clicked the Cancel button.

Comments The time zones displayed in the form are listed by country. For this reason, if the time zone specified by `ioTimeZoneP` is shared by several countries, you need to supply a value for `ioLocaleTimeZoneP` to identify which country should be selected when the list is first displayed. You can use the constant `lmAnyLanguage` as the value for the language field of the structure pointed to by this parameter.

If you don't care which value is initially selected, pass `true` for the `anyLocale` parameter. In this case, the first country that matches the GMT offset given in `ioTimeZoneP` is selected.

You might want to use the current time zone stored in the system preferences as the initial value for the `ioTimeZoneP` parameter. To obtain this time zone, do the following:

```
Int16 timeZone =
  (Int16)PrefGetPreference(prefTimeZone);
CountryType timeZoneCountry = (CountryType)
  PrefGetPreference(prefTimeZoneCountry);
LmLocaleType timeZoneLocale;
Boolean change;

timeZoneLocale.country = timeZoneCountry;
timeZoneLocale.language = lmAnyLanguage;
change = SelectTimeZone(&timeZone,
  &timeZoneLocale, NULL, true, false);
```

Compatibility Implemented if 4.0 New Feature Set is present.

9

Fields

This chapter provides the following information about field objects:

- Field Data Structures
- Field Resources
- Field Functions

The header file `Field.h` declares the API that this chapter describes. For more information on fields, see the section "Fields" in the *Palm OS Programmer's Companion*, vol. I.

Field Data Structures

FieldAttrType

The `FieldAttrType` bit field defines the visible characteristics of the field. The functions `FldGetAttributes` and `FldSetAttributes` return and set these values. There are other functions that retrieve or set individual attributes defined here. Those functions are noted below.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FieldAttrType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16 usable        :1;
  UInt16 visible       :1;
  UInt16 editable      :1;
  UInt16 singleLine    :1;
  UInt16 hasFocus      :1;
  UInt16 dynamicSize   :1;
  UInt16 insPtVisible  :1;
```

```
    UInt16 dirty         :1;
    UInt16 underlined    :2;
    UInt16 justification :2;
    UInt16 autoShift     :1;
    UInt16 hasScrollBar  :1;
    UInt16 numeric       :1;
  } FieldAttrType;
```

Field Descriptions

`usable` — If not set, the field object is not considered part of the current interface of the application, and it doesn't appear on screen. The function `FldSetUsable` sets this value, but it is better to use `FrmShowObject`.

`visible` — Set or cleared internally when the field object is drawn with `FldDrawField` or `FrmShowObject`, and erased with `FldEraseField` or `FrmHideObject`.

`editable` — If not set, the field object doesn't accept Graffiti® input or editing commands and the insertion point cannot be positioned with the pen. The text can still be selected and copied.

`singleLine` — If set, the field is a single line of text high and the text does not wrap when it exceeds the width of the field. If not set, the text wraps to fill multiple lines.

`hasFocus` — Set internally when the field has the current focus. The blinking insertion point appears in the field that has the current focus. Use the function `FrmSetFocus` and `FldReleaseFocus` to set this value.

`dynamicSize` — If set, a `fldHeightChangedEvent` is generated whenever the number of lines needs to increase or decrease. Your application needs to respond to this event by adjusting the size of the field's bounding box. If not set, the text wraps to fill more (or less) lines as required, but the event is not generated. *Note that this bit does not cause the field to change size automatically*; your application must respond to the `fldHeightChangedEvent` and resize the field itself.

Set this attribute to `false` if the Single Line attribute is set.

`insPtVisible` — If set, the insertion point is scrolled into view. This attribute is set and cleared internally.

`dirty` — If set, the user has modified the field. The functions `FldDirty` and `FldSetDirty` retrieve this field's value.

`underlined` — If set each line of the field, including blank lines, is underlined. Possible values are defined by the `UnderlineModeType` defined in `Window.h`:

```
noUnderline
grayUnderline
solidUnderline
colorUnderline
```

Editable text fields generally use `grayUnderline` as the value.

The `solidUnderline` value is only valid for Palm OS 3.1 and higher.

The `colorUnderline` value is only valid for Palm OS 3.5 and higher.

`justification`	Specifies the text alignment. Possible values are `leftAlign` and `rightAlign`. (left or right justification only; `centerAlign` justification is not supported).
`autoShift`	If set, Graffiti auto-shift rules are applied.
`hasScrollBar`	If set, the field has a scrollbar. The system sends more frequent `fldChangedEvent`s so the application can adjust the height appropriately.
`numeric`	If set, only the characters 0 through 9 and associated separators are allowed to be entered in the field. The associated separators are the thousands separator and the decimal character. The values of these two characters depend on the settings in the Formats prefs panel.

FieldPtr

The `FieldPtr` type defines a pointer to a `FieldType` structure.

```
typedef FieldType *FieldPtr;
```

You pass the `FieldPtr` as an argument to all field functions. You can obtain the `FieldPtr` using the function `FrmGetObjectPtr` in this way:

```
fldPtr = FrmGetObjectPtr(frm,
  FrmGetObjectIndex(frm, fldID));
```

where `fldID` is the resource ID assigned when you created the field.

FieldType

The `FieldType` structure represents a field.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FieldType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16              id;
  RectangleType       rect;
  FieldAttrType       attr;
  Char                *text;
  MemHandle           textHandle;
  LineInfoPtr         lines;
  UInt16              textLen;
  UInt16              textBlockSize;
  UInt16              maxChars;
  UInt16              selFirstPos;
  UInt16              selLastPos;
  UInt16              insPtXPos;
  UInt16              insPtYPos;
  FontID              fontID;
  UInt8               reserved;
} FieldType;
```

Your code should treat the `FieldType` structure as opaque. Use the functions specified in the descriptions below to retrieve and set each value. Do not attempt to change structure member values directly.

Field Descriptions

`id` — ID value you specified when you created the field resource. This ID value is included as part of the event data of `fldEnterEvent`.

`rect` — Position and size of the field object. The functions `FldGetBounds`, `FrmGetObjectBounds`, `FldSetBounds`, and `FrmSetObjectBounds` retrieve and set this value.

`attr` — Field object attributes. (See `FieldAttrType`.)

`text` — Pointer to the `NULL`-terminated string that is displayed by the field object. The functions `FldGetTextPtr` and `FldSetTextPtr` retrieve and set this value (see below).

Never set the contents of this string directly; for example, do not pass this pointer as the destination value to a function such as `StrCopy`.

`textHandle`	Handle to the stored text or to a database record containing the stored text. The functions `FldGetTextHandle` and `FldSetTextHandle` retrieve and set this value. If `textHandle` is defined, the field calculates the `text` pointer when it locks the handle. In general, you should only use `FldGetTextPtr` and `FldSetTextPtr` on text fields that aren't editable. On editable text fields, use `FldGetTextHandle` and `FldSetTextHandle`. Also note that editable text fields allocate the text handle as necessary. If a user starts typing in a field that doesn't have a text handle allocated, the field will allocate one. The field also resizes the text's memory block as necessary when the user adds more text.
`lines`	Pointer to an array of `LineInfoType` structures. There is one entry in this array for each visible line of the text. (See `LineInfoType`.) The field code maintains this array internally; you should never change the `lines` array yourself. Note that this value is `NULL` for single line fields, and for fields that do not have an allocated text handle.
`textLen`	Length in bytes of the string currently displayed by the field object; the null terminator is excluded. You can retrieve this value with `FldGetTextLength`.
`textBlockSize`	Allocated size of the memory block that holds the field object's text string. You can retrieve this value with `FldGetTextAllocatedSize`. Fields allocate memory for the field text as needed, several bytes at a time.

Note that `textBlockSize` may be different from the size of the chunk pointed to by `textHandle`. The `textHandle` may point to a database record that contains, in part, the text displayed by the field. If you called `MemHandleSize` on such a `textHandle`, the number returned may be greater than `textBlockSize`.

`maxChars` — Maximum number of bytes the field object accepts. The functions `FldGetMaxChars` and `FldSetMaxChars` retrieve and set this value.

Note the difference between `textLen`, `textBlockSize`, and `maxChars`. `textLen` is the number of bytes of character data that `text` actually holds. `textBlockSize` is the amount of memory currently allocated for the text (which must be greater than or equal to `textLen`), and `maxChars` sets the maximum value that `textBlockSize` and `textLen` can expand to.

For example, if you've created a text field for users to enter their first names in, you might specify that the maximum length of this field is 20 bytes. If a user enters "John" in this field, `textLen` is 4, `textBlockSize` is 16, and `maxChars` is 20.

`selFirstPos` — Starting character offset in bytes of the current selection. Use `FldGetSelection` and `FldSetSelection` to retrieve and set this value and the `selLastPos` value.

`selLastPos` — Ending character offset in bytes of the current selection. When `selFirstPos` equals `selLastPos`, there is no selection.

`insPtXPos`	Horizontal location of the insertion point, given as the offset in bytes into the line indicated by `insPtYPos`. The functions `FldGetInsPtPosition` and `FldSetInsPtPosition` retrieve and set this value.
`insPtYPos`	Vertical location of the insertion point, given as the display line where the insertion point is positioned. The first display line is zero. The first display line may be different from the first line of text in the field if the field has been scrolled. The functions `FldGetInsPtPosition` and `FldSetInsPtPosition` retrieve and set this value.
`fontID`	Font ID for the field. See `Font.h` for more information. The functions `FldGetFont` and `FldSetFont` retrieve and set this value.
`reserved`	Reserved for future use.

LineInfoPtr

The `LineInfoPtr` type defines a pointer to the `LineInfoType`.

```
typedef LineInfoType *LineInfoPtr;
```

LineInfoType

The `LineInfoType` structure defines an element in the field's `lines` array. The `lines` array contains the field's word wrapping information. There is one element in the array per visible line in the field, including visible lines that contain no text. The field code maintains this array internally; you should never change the `lines` array yourself.

The functions `FldCalcFieldHeight`, `FldGetVisibleLines`, `FldRecalculateField`, and `FldGetNumberOfBlankLines` retrieve or set information in this structure. The scrolling functions `FldGetScrollPosition`, `FldGetScrollValues`, `FldScrollField`, and `FldSetScrollPosition` also retrieve or set information in this structure.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `LineInfoType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16       start;
  UInt16       length;
} LineInfoType;
```

Field Descriptions

`start` The byte offset into the `FieldType`'s `text` field of the first character displayed by this line. If the line is blank, `start` is equal to `textLen` and `length` is 0.

`length` The length in bytes of the portion of the string displayed on this line. If the line is blank, the length is 0.

Field Resources

The Field Resource (tFLD) represents a field on screen.

Field Functions

FldCalcFieldHeight

Purpose Determine the height of a field for a string.

Prototype

```
UInt16 FldCalcFieldHeight (const Char *chars,
UInt16 maxWidth)
```

Parameters

-> `chars` Pointer to a null-terminated string.

-> `maxWidth` Maximum line width in pixels.

Result Returns the total number of lines needed to draw the string passed.

Comments The width of a field is contained in the `rect` member of the `FieldType` structure. You can retrieve this value in the following way:

```
FrmGetObjectBounds(frm,
  FrmGetObjectIndex(frm, fldID),
  &myRect);
fieldWidth = myRect.extent.x;
FldCalcFieldHeight(myString, fieldWidth);
```

See Also `FldWordWrap`

FldCompactText

Purpose Compact the memory block that contains the field's text to release any unused space.

Prototype

```
void FldCompactText (FieldType *fldP)
```

Parameters

-> `fldP` Pointer to a field object (`FieldType` structure).

Result Returns nothing.

Comments As characters are added to the field's text, the block that contains the text is grown. The block is expanded several bytes at a time so that it doesn't have to expand each time a character is added. This expansion may result in some unused space in the text block.

Applications should call this function on field objects that edit data records in place before the field is unlocked, or at any other time when a compact field is desirable; for example, before writing to the storage heap.

See Also `FldGetTextAllocatedSize`, `FldSetTextAllocatedSize`

FldCopy

Purpose Copy the current selection to the text clipboard.

Prototype
```
void FldCopy (const FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).

Result Returns nothing.

Comments This function leaves the current selection highlighted.

This function replaces anything previously in the text clipboard if there is text to copy. If no text is selected, the function beeps and the clipboard remains intact.

See Also `FldCut`, `FldPaste`

FldCut

Purpose Copy the current selection to the text clipboard, delete the selection from the field, and redraw the field.

Prototype
```
void FldCut (FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).

Result Returns nothing.

Comments If text is selected, the text is removed from the field, the field's dirty attribute is set, and anything previously in the text clipboard is replaced by the selected text.

If there is no selection or if the field is not editable, this function beeps.

See Also `FldCopy`, `FldPaste`, `FldUndo`

FldDelete

Purpose Delete the specified range of characters from the field and redraw the field.

Prototype
```
void FldDelete (FieldType *fldP, UInt16 start,
UInt16 end)
```

Parameters

-> `fldP`	Pointer to the field object (`FieldType` structure) to delete from.
-> `start`	The beginning of the range of characters to delete given as a valid byte offset into the field's `text` string.
-> `end`	The end of the range of characters to delete given as a valid byte offset into the field's `text` string. On systems that support multi-byte characters, this position must be an inter-character boundary. That is, it must not point to a middle byte of a multi-byte character.

Result Returns nothing.

Comments This function deletes all characters from the starting offset up to the ending offset and sets the field's dirty attribute. It does not delete the character at the ending offset.

If `start` or `end` point to an intra-character boundary, `FldDelete` attempts to move the offset backward, toward the beginning of the text, until the offset points to an inter-character boundary (i.e., the start of a character).

`FldDelete` posts a `fldChangedEvent` to the event queue. If you call this function repeatedly, you may overflow the event queue with `fldChangedEvents`. An alternative is to remove the text handle from the field, change the text, and then set the field's handle again. See `FldGetTextHandle` for a code example.

See Also `FldInsert`, `FldEraseField`, `TxtCharBounds`

FldDirty

Purpose Return `true` if the field has been modified since the text value was set.

Prototype
```
Boolean FldDirty (const FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).

Result Returns `true` if the field has been modified either by the user or through calls to certain functions such as `FldInsert` and `FldDelete`, `false` if the field has not been modified.

See Also `FldSetDirty`, `FieldAttrType`

FldDrawField

Purpose Draw the text of the field.

Prototype
```
void FldDrawField (FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).

Result Returns nothing.

Comments The field's `usable` attribute must be `true` or the field won't be drawn.

This function doesn't erase the area behind the field before drawing.

If the field has the focus, the blinking insertion point is displayed in the field.

See Also `FldEraseField`

FldEraseField

Purpose Erase the text of a field and turn off the insertion point if it's in the field.

Prototype
```
void FldEraseField (FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).

Result Returns nothing.

Comments You rarely need to call this function directly. Instead, use `FrmHideObject`, which calls `FldEraseField` for you.

This function visibly erases the field from the display, but it doesn't modify the contents of the field or free the memory associated with it.

If the field has the focus, the blinking insertion point is turned off.

This function sets the `visible` attribute to `false`. (See `FieldAttrType`.)

See Also `FldDrawField`

FldFreeMemory

Purpose Release the handle-based memory allocated to the field's text and the associated word-wrapping information.

Prototype
```
void FldFreeMemory (FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).

Result Returns nothing. May raise a fatal error message if the text associated with the field is actually a record in a database.

Comments This function releases

- The memory allocated to the text of a field—the memory block that the `textHandle` member of the `FieldType` data structure points to.

If the field's `textHandle` is `NULL` but there is a `text` string associated with that field (which is often the case with noneditable text fields), the text string is not freed.

- The memory allocated to hold the word-wrapping information—the memory block that the `lines` member of the `FieldType` data structure points to.

This function doesn't affect the display of the field. Fields allocate memory for the text string as needed, so it is not an error to call this function while the field is still displayed. That is, if `text` is `NULL` and the user starts typing in the field, the field simply allocates memory for text and continues.

FldGetAttributes

Purpose Return the attributes of a field.

Prototype
```
void FldGetAttributes (const FieldType *fldP,
FieldAttrPtr attrP)
```

Parameters

-> `fldP`	Pointer to a `FieldType` structure.
<- `attrP`	Pointer to the `FieldAttrType` structure.

Result Returns the field's attributes in the `attrP` parameter.

See Also `FldSetAttributes`

FldGetBounds

Purpose Return the current bounds of a field.

Prototype
```
void FldGetBounds (const FieldType *fldP,
RectanglePtr rect)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
<- `rect`	Pointer to a `RectangleType` structure.

Result Returns nothing. Stores the field's bounds in the `RectangleType` structure reference by `rect`.

Comments Returns the `rect` field of the `FieldType` structure.

See Also `FldSetBounds`, `FrmGetObjectBounds`

FldGetFont

Purpose Return the ID of the font used to draw the text of a field.

Prototype
```
FontID FldGetFont (const FieldType *fldP)
```

Parameters -> `fldP` Pointer to a field object (`FieldType` structure).

Result Returns the ID of the font.

See Also `FldSetFont`

FldGetInsPtPosition

Purpose Return the insertion point position within the string.

Prototype
```
UInt16 FldGetInsPtPosition (const FieldType *fldP)
```

Parameters -> `fldP` Pointer to a field object (`FieldType` structure).

Result Returns the byte offset of the insertion point.

Comments The insertion point is to the left of the byte offset that this function returns. That is, if this function returns 0, the insertion point is to the left of the first character in the string. In multiline fields, line feeds are counted as a single character in the string, and the byte offset after the line feed character is the beginning of the next line.

See Also `FldSetInsPtPosition`

FldGetMaxChars

Purpose Return the maximum number of bytes the field accepts.

Prototype
```
UInt16 FldGetMaxChars (const FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).

Result Returns the maximum length in bytes of characters the user is allowed to enter. This is the `maxChars` field in `FieldType`.

See Also `FldSetMaxChars`

FldGetNumberOfBlankLines

Purpose Return the number of blank lines that are displayed at the bottom of a field.

Prototype
```
UInt16 FldGetNumberOfBlankLines
(const FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a `FieldType` structure.

Result Returns the number of blank lines visible.

Comments This routine is useful for updating a scroll bar after characters have been removed from the text in a field. See the `NoteViewScroll` function in the Address sample application for an example.

Compatibility Implemented only if 2.0 New Feature Set is present.

FldGetScrollPosition

Purpose Return the offset of the first character in the first visible line of a field.

Prototype
```
UInt16 FldGetScrollPosition
(const FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).

Result Returns the offset of the first visible character.

See Also `FldSetScrollPosition`, `LineInfoType`

FldGetScrollValues

Purpose Return the values necessary to update a scroll bar.

Prototype
```
void FldGetScrollValues (const FieldType *fldP,
UInt16 *scrollPosP, UInt16 *textHeightP,
UInt16 *fieldHeightP)
```

Parameters

-> `fldP`	Pointer to a `FieldType` structure.
<- `scrollPosP`	The line of text that is the topmost visible line. Line numbering starts with 0.
<-`textHeightP`	The number of lines needed to display the field's text, given the width of the field.
<-`fieldHeightP`	The number of visible lines in the field.

Result Returns nothing. Stores the position, text height, and field height in the parameters passed in.

Comments Use the values returned by this function to calculate the values you send to `SclSetScrollBar` to update the scroll bar. For example:

```
FldGetScrollValues (fldP, &scrollPos,
  &textHeight,  &fieldHeight);

if (textHeight > fieldHeight)
```

```
    maxValue = textHeight - fieldHeight;
  else if (scrollPos)
    maxValue = scrollPos;
  else
    maxValue = 0;

  SclSetScrollBar (bar, scrollPos, 0, maxValue,
    fieldHeight-1);
  }
```

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also FldSetScrollPosition

FldGetSelection

Purpose Return the current selection of a field.

Prototype

```
void FldGetSelection (const FieldType *fldP,
UInt16 *startPosition, UInt16 *endPosition)
```

Parameters

-> `fldP` Pointer to a field object (`FieldType` structure).

<- `startPosition` Pointer to the start of the selected characters range, given as the byte offset into the field's text.

<- `endPosition` Pointer to end of the selected characters range given as the byte offset into the field's text.

Result Returns the starting and ending byte offsets in `startPosition` and `endPosition`.

Comments The first character in a field is at offset zero.

If the user has selected the first five characters of a field, `startPosition` will contain the value 0 and `endPosition` the value 5, assuming all characters are a single byte long.

See Also FldSetSelection

FldGetTextAllocatedSize

Purpose Return the number of bytes allocated to hold the field's text string. Don't confuse this number with the actual length of the text string displayed in the field.

Prototype
```
UInt16 FldGetTextAllocatedSize
(const FieldType *fldP)
```

Parameters -> `fldP` Pointer to a field object.

Result Returns the number of bytes allocated for the field's text. This is the `textBlockSize` field in `FieldType`.

See Also `FldSetTextAllocatedSize`

FldGetTextHandle

Purpose Return a handle to the block that contains the text string of a field.

Prototype
```
MemHandle FldGetTextHandle (const FieldType *fldP)
```

Parameters -> `fldP` Pointer to a field object (`FieldType` structure).

Result Returns the handle to the text string of a field or `NULL` if no handle has been allocated for the field pointer.

Comments The handle returned by this function is not necessarily the handle to the start of the string. If you've used `FldSetText` to set the field's text to a string that is part of a database record, the text handle points to the start of that record. You'll need to compute the offset from the start of the record to the start of the string. You can either store the offset that you passed to `FldSetText` or you can compute the offset by performing pointer arithmetic on the pointer you get by locking this handle and the pointer returned by `FldGetTextPtr`.

If you are obtaining the text handle so that you can edit the field's text, you must remove the handle from the field before you do so. If

you change the text while it is being used by a field, the field's internal structures specifying the text length, allocated size, and word wrapping information can become out of sync. To avoid this problem, remove the text handle from the field, change the text, and then set the field's text handle again. For example:

```
/* Get the handle for the string and unlock */
/* it by removing it from the field. */
textH = FldGetTextHandle(fldP);
FldSetTextHandle (fldP, NULL);

/* Insert code that modifies the string here.*/
/* The basic steps are: */
/* resize the chunk if necessary,*/
/* lock the chunk, write to it, and then */
/* unlock the chunk. If the text is in a */
/* database record, use Data Manager calls. */

/* Update the text in the field. */
FldSetTextHandle (fldP, textH);
FldDrawField(fldP);
```

See Also FldSetTextHandle, FldGetTextPtr

FldGetTextHeight

Purpose Return the height in pixels of the number of visible lines that are not empty.

Prototype

```
UInt16 FldGetTextHeight (const FieldType *fldP)
```

Parameters

-> `fldP` Pointer to a field object (FieldType structure).

Result Returns the height in pixels of the number of visible lines that are not empty.

Comments Empty lines are all of the lines in the field following the last byte of text. Note that lines that contain only a linefeed are not empty. Also note that only lines that are visible are counted.

See Also FldCalcFieldHeight

FldGetTextLength

Purpose Return the length in bytes of the field's text.

Prototype
```
UInt16 FldGetTextLength (const FieldType *fldP)
```

Parameters -> `fldP` Pointer to a field object (FieldType structure).

Result Returns the length in bytes of a field's text, not including the terminating null character. This is the `textLen` field of `FieldType`.

FldGetTextPtr

Purpose Return a locked pointer to the field's text string.

Prototype
```
Char *FldGetTextPtr (const FieldType *fldP)
```

Parameters -> `fldP` Pointer to a field object (FieldType structure).

Result Returns a locked pointer to the field's text string or `NULL` if the field is empty.

Comments The pointer returned by this function can become invalid if the user edits the text after you obtain the pointer.

Do not modify the contents of the pointer yourself. If you change the text while it is being used by a field, the field's internal structures specifying the text length, allocated size, and word wrapping information can become out of sync. To avoid this problem, follow the instructions given under FldGetTextHandle.

WARNING! The pointer returned by this function is "owned" by the field until you specify a different pointer for the field. You should not store this pointer for future use, since the field can modify the size of the string, which can cause the pointer to become invalid.

See Also FldSetTextPtr, FldGetTextHandle

FldGetVisibleLines

Purpose Return the number of lines that can be displayed within the visible bounds of the field, regardless of what text is stored in the field.

Prototype
```
UInt16 FldGetVisibleLines (const FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (FieldType structure).

Result Returns the number of lines the field displays. (This is the size of the `lines` array in the `FieldType` structure.)

See Also FldGetNumberOfBlankLines, FldCalcFieldHeight

FldGrabFocus

Purpose Turn the insertion point on (if the specified field is visible) and position the blinking insertion point in the field.

Prototype
```
void FldGrabFocus (FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (FieldType structure).

Result Returns nothing.

Comments You rarely need to call this function directly. Instead, use FrmSetFocus, which calls `FldGrabFocus` for you.

One instance where you need to call `FldGrabFocus` directly is to programmatically set the focus in a field that is contained in a table cell.

This function sets the field attribute `hasFocus` to `true`. (See `FieldAttrType`.)

See Also `FrmSetFocus`, `FldReleaseFocus`

FldHandleEvent

Purpose Handles events that affect the field, including the following: `keyDownEvent`, `penDownEvent`, and `fldEnterEvent`.

Prototype

```
Boolean FldHandleEvent (FieldType *fldP,
EventType *eventP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
-> `eventP`	Pointer to an event (`EventType` data structure).

Result Returns `true` if the event was handled.

Comments When a `keyDownEvent` occurs in an editable text field, the keystroke appears in the field if it's a printable character or manipulates the insertion point if it's a "movement" character. The field is automatically updated.

When a `penDownEvent` occurs, the field sends a `fldEnterEvent` to the event queue.

When a `fldEnterEvent` occurs, the field grabs the focus. If the user has tapped twice in the current location, the word at that location is selected. If the user has tapped three times, the entire line is selected. Otherwise, the insertion point is placed in the specified position.

When a `menuCmdBarOpenEvent` occurs, the field adds paste, copy, cut, and undo buttons to the command toolbar. These buttons are only added if they make sense in the current context. That is, the cut button is only added if the field is editable, the paste button is only

added if there is text on the clipboard and the field is editable, and the undo button is only added if there is an action to undo.

If the event alters the contents of the field, this function visually updates the field.

This function doesn't handle any events if the field is not editable or usable.

Compatibility Double-tapping to select a word and triple-tapping to select a line are only supported if 3.5 New Feature Set is present.

`FldHandleEvent` only handles the `menuCmdBarOpenEvent` if 3.5 New Feature Set is present.

FldInsert

Purpose Replace the current selection if any with the specified string and redraw the field.

Prototype

```
Boolean FldInsert (FieldType *fldP,
const Char *insertChars, UInt16 insertLen)
```

Parameters

-> `fldP`	Pointer to the field object (`FieldType` structure) to insert to.
-> `insertChars`	Text string to be inserted.
-> `insertLen`	Length in bytes of the text string to be inserted, not counting the trailing null character.

Result Returns `true` if string was successfully inserted. Returns `false` if:

- The `insertLen` parameter is 0.
- The field is not editable.
- Adding the text would exceed the field's size limit (the `maxChars` value).
- More memory must be allocated for the field, and the allocation fails.

Comments If there is no current selection, the string passed is inserted at the position of the insertion point.

This function sets the field's dirty attribute and posts a `fldChangedEvent` to the event queue. If you call this function repeatedly, you may overflow the event queue with `fldChangedEvents`. An alternative is to remove the text handle from the field, change the text, and then set the field's handle again. See `FldGetTextHandle` for a code example.

See Also `FldPaste`, `FldDelete`, `FldCut`, `FldCopy`

FldMakeFullyVisible

Purpose Generates an event to cause a dynamically resizable field to expand its height to make its text fully visible.

Prototype

```
Boolean FldMakeFullyVisible (FieldType *fldP)
```

Parameters

-> `fldP` Pointer to a field object (`FieldType` structure).

Result Returns `true` if the field is dynamically resizable and was not fully visible; `false` otherwise.

Comments Use this function on a field whose `dynamicSize` attribute is `true` (see `FieldAttrType`).

This function does not actually resize the field. Instead, it computes how big the field should be to be fully visible and then posts this information to the event queue in a `fldHeightChangedEvent`.

NOTE: The event does not get generated if the number of lines in the field is equal to or greater than the value of the maximum lines attribute for the field.

If the field is contained in a table, the table's code handles the `fldHeightChangedEvent`. If the field is directly on a form, your application code should handle the `fldHeightChangedEvent` itself. The form code does not handle the event for you. Note that

the constant `maxFieldLines` defines the maximum number of lines a field can expand to if the field is using the standard font.

See Also `TblHandleEvent`

FldNewField

Purpose Create a new field object dynamically and install it in the specified form.

Prototype

```
FieldType *FldNewField (void **formPP, UInt16 id,
Coord x, Coord y, Coord width, Coord height,
FontID font, UInt32 maxChars, Boolean editable,
Boolean underlined, Boolean singleLine,
Boolean dynamicSize,
JustificationType justification,
Boolean autoShift, Boolean hasScrollBar,
Boolean numeric)
```

Parameters

<-> `formPP`	Pointer to the pointer to the form in which the new field is installed. This value is not a handle; that is, the old form pointer value is not necessarily valid after this function returns. In subsequent calls, always use the new form pointer value returned by this function.
-> `id`	Symbolic ID of the field, specified by the developer. By convention, this ID should match the resource ID (not mandatory).
-> `x`	Horizontal coordinate of the upper-left corner of the field's boundaries, relative to the window in which it appears.
-> `y`	Vertical coordinate of the upper-left corner of the field's boundaries, relative to the window in which it appears.
-> `width`	Width of the field, expressed in pixels.
-> `height`	Height of the field, expressed in pixels.
-> `font`	Font to use to draw the field's text.

-> `maxChars`	Maximum number of bytes held by the field this function creates.
-> `editable`	Pass `true` to create a field in which the user can edit text. Pass `false` to create a field that cannot be edited.
-> `underlined`	Pass `noUnderline` for no underline, or `grayUnderline` to have the field underline the text it displays. On Palm OS® version 3.1 and higher, pass `solidUnderline` to use a solid underline instead of a dotted underline.
-> `singleLine`	Pass `true` to create a field that can display only a single line of text.
-> `dynamicSize`	Pass `true` to create a field that resizes dynamically according to the amount of text it displays.
-> `justification`	Pass either of the values `leftAlign` or `rightAlign` to specify left justification or right justification, respectively. The `centerAlign` value is not supported.
-> `autoShift`	Pass `true` to specify the use of Palm OS 2.0 (and later) auto-shift rules.
-> `hasScrollBar`	Pass `true` to attach a scroll bar control to the field this function creates.
-> `numeric`	Pass `true` to specify that only characters in the range of 0 through 9 are allowed in the field.

Result Returns a pointer to the new field object or `NULL` if there wasn't enough memory to create the field. Out of memory situations could be caused by memory fragmentation.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FrmValidatePtr`, `WinValidateHandle`, `CtlValidatePointer`, `FrmRemoveObject`

FldPaste

Purpose Replace the current selection in the field, if any, with the contents of the text clipboard.

Prototype
```
void FldPaste (FieldType *fldP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).

Result Returns nothing

Comments The function performs these actions:

- Scrolls the field, if necessary, so the insertion point is visible.
- Inserts the clipboard text at the position of the insertion point if there is no current selection.
- Positions the insertion point after the last character inserted.
- Doesn't delete the current selection if there is no text in the clipboard.

See Also `FldInsert`, `FldDelete`, `FldCut`, `FldCopy` `FldUndo`

FldRecalculateField

Purpose Update the structure that contains the word-wrapping information for each visible line.

Prototype
```
void FldRecalculateField (FieldType *fldP,
Boolean redraw)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
-> `redraw`	If `true`, redraws the field.

Result Returns nothing.

Comments This function will allocate the memory block that contains the displayed lines information if, and only if, the block does not yet exist.

You should call this function when you change the field width or text length of the field. Do not call this function after changing the font or field height.

Note that many of the field functions, including `FldSetTextHandle`, `FldInsert`, and `FldDelete`, recalculate the word-wrapping information for you.

Compatibility In releases prior to Palm OS 4.0, the word-wrapping information is only updated if the `redraw` parameter is set to `true`. As of Palm OS 4.0 it is updated whenever `FldRecalculateField` is called, regardless of the value of the `redraw` parameter.

FldReleaseFocus

Purpose Turn the blinking insertion point off if the field is visible and has the current focus, reset the Graffiti state, and reset the undo state.

Prototype
```
void FldReleaseFocus (FieldType *fldP)
```

Parameters -> `fldP` Pointer to a field object (`FieldType` structure).

Result Returns nothing.

Comments This function sets the field attribute `hasFocus` to `false`. (See `FieldAttrType`.)

Usually, you don't need to call this function. If the field is in a form or in a table that doesn't use custom drawing functions, the field code releases the focus for you when the focus changes to some other control. If your field is in any other type of object, such as a table that uses custom drawing functions or a gadget, you must call `FldReleaseFocus` when the focus moves away from the field.

See Also `FldGrabFocus`

FldScrollable

Purpose Return `true` if the field is scrollable in the specified direction.

Prototype
```
Boolean FldScrollable (const FieldType *fldP,
WinDirectionType direction)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
-> `direction`	The direction to test. `DirectionType` is defined in `Window.h`. It is an enum defining the constants `up` and `down`.

Result Returns `true` if the field is scrollable in the specified direction; `false` otherwise.

See Also `FldScrollField`

FldScrollField

Purpose Scroll a field up or down by the number of lines specified.

Prototype
```
void FldScrollField (FieldType *fldP,
UInt16 linesToScroll, WinDirectionType direction)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
-> `linesToScroll`	Number of lines to scroll.
-> `direction`	The direction to scroll. `DirectionType` is defined in `Window.h`. It is an enum defining, among others, the constants `winUp` and `winDown`.

Result Returns nothing.

Comments This function can't scroll horizontally, that is, right or left.

The field object is redrawn if it's scrolled; however, the scrollbar is not updated. Use SclSetScrollBar to update the scrollbar. For example:

```
FldScrollField (fldP, linesToScroll,
direction);

// Update the scroll bar.
SclGetScrollBar (bar, &value, &min, &max,
  &pageSize);

if (direction == winUp)
  value -= linesToScroll;
else
  value += linesToScroll;

SclSetScrollBar (bar, value, min, max,
  pageSize);
```

If the field is not scrollable in the direction indicated, this function returns without performing any work. You can use FldScrollable before calling this function to see if the field can be scrolled.

See Also FldScrollable, FldSetScrollPosition

FldSendChangeNotification

Purpose Send a fldChangedEvent to the event queue.

Prototype

```
void FldSendChangeNotification
(const FieldType *fldP)
```

Parameters

-> `fldP` Pointer to a field object.

Result Returns nothing.

Comments This function is used internally by the field code. You normally never call it in application code.

FldSendHeightChangeNotification

Purpose Send a `fldHeightChangedEvent` to the event queue.

Prototype

```
void FldSendHeightChangeNotification
(const FieldType *fldP, UInt16 pos,
Int16 numLines)
```

Parameters

-> `fldP`	Pointer to a field object.
-> `pos`	Character position of the insertion point.
-> `numLines`	New number of lines in the field.

Result Returns nothing.

Comments This function is used internally by the field code. You normally never call it in application code.

FldSetAttributes

Purpose Set the attributes of a field.

Prototype

```
void FldSetAttributes (FieldType *fldP,
const FieldAttrType *attrP)
```

Parameters

-> `fldP`	Pointer to a `FieldType` structure.
-> `attrP`	Pointer to the attributes.

Result Returns nothing.

Comments This function does not do anything to make the new attribute values take effect. For example, if you use this function to change the value of the underline attribute, you won't see its effect until you call `FldDrawField`.

You usually do not have to modify field attributes at runtime, so you rarely need to call this function.

WARNING! You must not call this function to change any attributes that are noted as "for internal use only."

The proper way to use `FldSetAttributes` is to:

1. Call `FldGetAttributes` to retrieve the attributes.
2. Set the specific flags that you want to modify.
3. Call `FldSetAttributes` to make the modifications.

WARNING! You must not call any field routines between calling `FldGetAttributes` and `FldSetAttributes`; this can cause the attributes to be out of sync, with unpredictable results.

See Also `FldGetAttributes`, `FieldAttrType`

FldSetBounds

Purpose Change the position or size of a field.

Prototype

```
void FldSetBounds (FieldType *fldP,
const RectangleType *rP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
-> `rP`	Pointer to a `RectangleType` structure that contains the new bounds of the display.

Result Returns nothing. May raise a fatal error message if the memory block that contains the word-wrapping information needs to be resized and there is not enough space to do so.

Comments If the field is visible, the field is redrawn within its new bounds.

NOTE: You can change the height or location of the field while it's visible, but do not change the width.

The memory block that contains the word-wrapping information (see LineInfoType) will be resized if the number of visible lines is changed. The insertion point is assumed to be off when this routine is called.

Make sure that `rect` is at least as tall as a single line in the current font. (You can determine this value by calling FntLineHeight.) If it's not, results are unpredictable.

See Also FldGetBounds, FrmSetObjectBounds

FldSetDirty

Purpose Set whether the field has been modified.

Prototype
```
void FldSetDirty (FieldType *fldP, Boolean dirty)
```

Parameters

-> `fldP`	Pointer to a field object (FieldType structure).
-> `dirty`	`true` if the text is modified.

Result Returns nothing.

Comments You typically call this function when you want to clear the dirty attribute. The dirty attribute is set when the user enters or deletes text in the field. It is also set by certain field functions, such as FldInsert and FldDelete.

See Also FldDirty

FldSetFont

Purpose Set the font used by the field, update the word-wrapping information, and draw the field if the field is visible.

Prototype
```
void FldSetFont (FieldType *fldP, FontID fontID)
```

Parameters

-> `fldP`	Pointer to a field object (FieldType structure).
-> `fontID`	ID of new font.

Result Returns nothing.

See Also FldGetFont, FieldAttrType

FldSetInsertionPoint

Purpose Set the location of the insertion point based on a specified string position.

Prototype

```
void FldSetInsertionPoint (FieldType *fldP,
UInt16 pos)
```

Parameters

-> fldP	Pointer to a FieldType structure.
-> pos	New location of the insertion point, given as a valid offset in bytes into the field's text. On systems that support multi-byte characters, you must make sure that this specifies an inter-character boundary (does not specify the middle or end bytes of a multi-byte character).

Result Nothing.

Comments This routine differs from FldSetInsPtPosition in that it doesn't make the character position visible. FldSetInsertionPoint also doesn't make the field the current focus of input if it was not already.

If pos indicates a position beyond the end of the text in the field, the insertion point is set to the end of the field's text.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also TxtCharBounds

FldSetInsPtPosition

Purpose Set the location of the insertion point for a given string position.

Prototype
```
void FldSetInsPtPosition (FieldType *fldP,
UInt16 pos)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
-> `pos`	New location of the insertion point, given as a valid offset in bytes into the field's text. On systems that support multi-byte characters, you must make sure that this specifies an inter-character boundary (does not specify the middle or end bytes of a multi-byte character).

Result Returns nothing.

Comments If the position is beyond the visible text, the field is scrolled until the position is visible.

See Also `FldGetInsPtPosition`, `TxtCharBounds`

FldSetMaxChars

Purpose Set the maximum number of bytes the field accepts (the `maxChars` value).

Prototype
```
void FldSetMaxChars (FieldType *fldP,
UInt16 maxChars)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
-> `maxChars`	Maximum size in bytes of the characters the user may enter. You may specify any value up to `maxFieldTextLen`.

Result Returns nothing.

Comments Line feed characters are counted when the length of characters is determined.

See Also FldGetMaxChars



FldSetMaxVisibleLines

Purpose Allows the creation of tables and fields smaller than 121 pixels tall that still drag-select when there are more lines of text than will fit in the space provided.

Prototype

```
void FldSetMaxVisibleLines (FieldType *fldP,
UInt8 maxLines)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
-> `maxLines`	Maximum number of lines to which the field will visually grow.

Result Returns nothing.

Comments A field can be dynamically expandable. When it is, the field package needs to know the maximum number of lines that should be visible so it can prevent the field from being expanded further. Since field expansion is actually handled by enclosing objects—tables or forms—this function's primary purpose is to allow the enclosing object to tell the field how big it can get.

By default, tables assume that the field can get as big as the table.

If you don't call this function, fields expect to be at least 121 pixels tall and try to grow repeatedly until they are.

FldSetScrollPosition

Purpose Scroll the field such that the character at the indicated offset is the first character on the first visible line. Redraw the field if necessary.

Prototype
```
void FldSetScrollPosition (FieldType *fldP,
UInt16 pos)
```

Parameters

-> `fldP` Pointer to a field object (`FieldType` structure).

-> `pos` Byte offset into the field's text string of first character to be made visible. On systems that support multi-byte characters, you must make sure that this specifies an inter-character boundary (does not specify the middle or end bytes of a multi-byte character).

Result Returns nothing.

Comments This function scrolls the field but does not update the field's scrollbar. You should update the scrollbar after calling this function. To do so, first call `FldGetScrollValues` to determine the values to use, and then call `SclSetScrollBar`.

See Also `FldGetScrollPosition`, `FldScrollField`, `TxtCharBounds`

FldSetSelection

Purpose Set the current selection in a field and highlight the selection if the field is visible.

Prototype
```
void FldSetSelection (FieldType *fldP,
UInt16 startPosition, UInt16 endPosition)
```

Parameters

-> `fldP` Pointer to a field object (`FieldType` structure).

-> `startPosition` Starting offset of the character range to highlight, given as a byte offset into the field's text.

	-> `endPosition`	Ending offset of the character range to highlight. The ending offset should be greater than or equal to the starting offset. On systems that support multi-byte characters, this position must be an inter-character boundary. That is, it must not point to a middle byte of a multi-byte character.

Result Returns nothing.

Comments To cancel a selection, set both `startPosition` and `endPosition` to the same value. If `startPosition` equals `endPosition`, then the current selection is unhighlighted.

If either `startPosition` or `endPosition` point to an intra-character boundary, `FldSetSelection` attempts to move that offset backward, toward the beginning of the string, until the offset points to an inter-character boundary (i.e., the start of a character).

See Also `TxtCharBounds`

FldSetText

Purpose Set the text value of the field without updating the display.

Prototype

```
void FldSetText (FieldType *fldP,
MemHandle textHandle, UInt16 offset, UInt16 size)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
-> `textHandle`	Unlocked handle of a block containing a null-terminated text string. Pass `NULL` for this parameter to remove the association between the field and the string it is currently displaying so that the string is not freed with the field when the form is deleted.
-> `offset`	Offset from start of block to start of the text string.

-> `size`	The allocated size of the text string. This is not the string length, and should not be set to `0`, unless you are setting the text to the empty string.

Result Returns nothing.

Comments This function allows applications to perform editing in place in memory. You can use it to point the field to a string in a database record so that you can edit that string directly using field routines.

The handle that you pass to this function is assumed to contain a null-terminated string starting at `offset` bytes in the memory chunk. The string should be between 0 and `size` - 1 bytes in length. The field does not make a copy of the memory chunk or the string data; instead, it stores the handle to the record in its structure.

`FldSetText` updates the word-wrapping information and places the insertion point after the last visible character, but it does not update the display. You must call `FldDrawField` after calling this function to update the display.

`FldSetText` increments the lock count for `textHandle` and decrements the lock count of its previous text handle (if any).

Because `FldSetText` (and `FldSetTextHandle`) may be used to edit database records, they do not free the memory associated with the previous text handle. If the previous text handle points to a string on the dynamic heap and you want to free it, use `FldGetTextHandle` to obtain the handle before using `FldSetText` and then free that handle after using `FldSetText`. (See `FldSetTextHandle` for a code example.)

If the field points to a database record, you want the memory associated with the text handle to persist; however, this memory and all other memory associated with the field is freed when the field itself is freed, which happens when the form is closed. If you don't want the memory associated with the text handle freed when the field is freed, use `FldSetText` and pass `NULL` for the text handle immediately before the form is closed. Passing `NULL` removes the association between the field and the text handle that you want retained. That text handle is unlocked as a result of the

`FldSetText` call, and when the field is freed, there is no text handle to free with it.

See Also `FldSetTextPtr`, `FldSetTextHandle`

FldSetTextAllocatedSize

Purpose Set the number of bytes allocated to hold the field's text string. Don't confuse this with the actual length of the text string displayed in the field.

Prototype

```
void FldSetTextAllocatedSize (FieldType *fldP,
UInt16 allocatedSize)
```

Parameters

-> `fldP` Pointer to a field object (`FieldType` structure).

-> `allocatedSize` Number of bytes to allocate for the text.

Result Returns nothing.

Comments This function generally is not used. It does not resize the field's allocated memory for the text string; it merely sets the `textBlockSize` field of the `FieldType` structure. The value of this field is computed and maintained internally by the field, so you should not have to call `FldSetTextAllocatedSize` directly.

See Also `FldGetTextAllocatedSize`, `FldCompactText`

FldSetTextHandle

Purpose Set the text value of a field to the string associated with the specified handle. Does not update the display.

Prototype

```
void FldSetTextHandle (FieldType *fldP,
MemHandle textHandle)
```

Parameters

-> `fldP` Pointer to a field object (`FieldType` structure).

-> `textHandle`	Unlocked handle of a field's text string. Pass `NULL` for this parameter to remove the association between the field and the string it is currently displaying so that the string is not freed with the field when the form is deleted.

Result Returns nothing.

Comments This function differs from `FldSetText` in that it uses the entire memory chunk pointed to by `textHandle` for the string. In fact, this function simply calls `FldSetText` with an offset of 0 and a size equal to the entire length of the memory chunk. Use it to have the field edit a string in a database record if the entire record consists of that string, or use it to have the field edit a string in the dynamic heap.

`FldSetTextHandle` updates the word-wrapping information and places the insertion point after the last visible character, but it does not update the display. You must call `FldDrawField` after calling this function to update the display.

`FldSetTextHandle` increments the lock count for `textHandle` and decrements the lock count of its previous text handle (if any).

Because `FldSetTextHandle` (and `FldSetText`) may be used to edit database records, they do not free the memory associated with the previous text handle. If the previous text handle points to a string on the dynamic heap and you want to free it, use `FldGetTextHandle` to obtain the handle before using `FldSetText` and then free that handle after using `FldSetText`. For example:

```
/* get the old text handle */
oldTxtH = FldGetTextHandle(fldP);

/* change the text and update the display */
FldSetTextHandle(fldP, txtH);
FldDrawField(fldP);

/* free the old text handle */
if (oldTxtH != NULL)
  MemHandleFree(oldTxtH);
```

If the field points to a database record, you want the memory associated with the text handle to persist; however, this memory and all other memory associated with the field is freed when the field itself is freed, which happens when the form is closed. If you don't want the memory associated with the text handle freed when the field is freed, use `FldSetTextHandle` and pass `NULL` for the text handle immediately before the form is closed. Passing `NULL` removes the association between the field and the text handle that you want retained. That text handle is unlocked as a result of the `FldSetTextHandle` call, and when the field is freed, there is no text handle to free with it.

See Also `FldSetTextPtr`, `FldSetText`

FldSetTextPtr

Purpose Set a noneditable field's text to point to the specified text string.

Prototype

```
void FldSetTextPtr (FieldType *fldP, Char *textP)
```

Parameters

-> `fldP`	Pointer to a field object (`FieldType` structure).
-> `textP`	Pointer to a null-terminated string.

Result Returns nothing. May display an error message if passed an editable text field.

Comments Do not call `FldSetTextPtr` with an editable text field. Instead, call `FldSetTextHandle` for editable text fields. `FldSetTextPtr` is intended for displaying noneditable text in the user interface.

If the field has more than one line, use `FldRecalculateField` to recalculate the word wrapping.

This function does **not** visually update the field. Use `FldDrawField` to do so.

The field never frees the string that you pass to this function, even when the field itself is freed. You must free the string yourself. Before you free the string, make sure the field is not still displaying it. Set the field's string pointer to some other string or call

`FldSetTextPtr(fldP, NULL)` before freeing a string you have passed using this function.

See Also `FldSetTextHandle`, `FldGetTextPtr`

FldSetUsable

Purpose Set a field to usable or nonusable.

Prototype
```
void FldSetUsable (FieldType *fldP,
Boolean usable)
```

Parameters

`fldP`	Pointer to a `FieldType` structure.
`usable`	`true` to set usable; `false` to set nonusable.

Result Returns nothing.

Comments A nonusable field doesn't display or accept input.

Use `FrmHideObject` and `FrmShowObject` instead of using this function.

See Also `FldEraseField`, `FldDrawField`, `FieldAttrType`

FldUndo

Purpose Undo the last change made to the field object, if any. Changes include typing, backspaces, delete, paste, and cut.

Prototype
```
void FldUndo (FieldType *fldP)
```

Parameters

`fldP`	Pointer to the field (`FieldType` structure) that has the focus.

Result Returns nothing.

See Also `FldPaste`, `FldCut`, `FldDelete`, `FldInsert`

FldWordWrap

Purpose Given a string and a width, return the number of bytes of characters that can be displayed using the current font.

Prototype

```
UInt16 FldWordWrap (const Char *chars,
Int16 maxWidth)
```

Parameters

-> `chars`	Pointer to a null-terminated string.
-> `maxWidth`	Maximum line width in pixels.

Result Returns the length in bytes of the characters that can be displayed.

See Also `FntWordWrap`

10

Find

This chapter describes the global find facility API declared in the header file `Find.h`.

Find Functions

FindDrawHeader

Purpose Draw the header line that separates, by application, the list of found items.

Prototype

```
Boolean FindDrawHeader (FindParamsPtr findParams,
Char const* title)
```

Parameters

-> `findParams`	Pointer to the `sysAppLaunchCmdFind` launch code's parameter block.
-> `title`	String to display as the title for the current application.

Result Returns `true` if Find screen is filled up. Applications should exit from the search if this occurs.

Comments Call this function once at the beginning of your application's response to the `sysAppLaunchCmdFind` launch code. This function draws a header for your application's Find results. The header separates the search results from your application with the search results from another application.

If your application searches multiple databases, you may also use `FindDrawHeader` as a separator between databases.

FindGetLineBounds

Purpose Returns the bounds of the next available line for displaying a match in the Find results dialog.

Prototype

```
void FindGetLineBounds
(const FindParamsType *findParams, RectanglePtr r)
```

Parameters

-> `findParams`	Pointer to the `sysAppLaunchCmdFind` launch code's parameter block.
<- `r`	The bounds of the area that should contain the next line of results.

Result Returns nothing.

FindSaveMatch

Purpose Saves the record and position within the record of a text search match. This information is saved so that it's possible to later navigate to the match.

Prototype

```
Boolean FindSaveMatch (FindParamsPtr findParams,
UInt16 recordNum, UInt16 pos, UInt16 fieldNum,
UInt32 appCustom, UInt16 cardNo, LocalID dbID)
```

Parameters

-> `findParams`	Pointer to the `sysAppLaunchCmdFind` launch code's parameter block.
-> `recordNum`	Record index. This parameter sets the `recordNum` field in the `sysAppLaunchCmdGoto`'s parameter block.
-> `pos`	Offset of the match string from start of record. This parameter sets the `matchPos` field in the `sysAppLaunchCmdGoto`'s parameter block.
-> `fieldNum`	Field number that the string was found in. This parameter sets the `matchFieldNum` field in the `sysAppLaunchCmdGoto`'s parameter block.

-> appCustom	Extra data the application can save with a match. This parameter sets the matchCustom field in the sysAppLaunchCmdGoto's parameter block.
-> cardNo	Card number of the database that contains the match. This parameter sets the dbCardNo field in the sysAppLaunchCmdGoto's parameter block.
-> dbID	Local ID of the database that contains the match. This parameter sets the dbID field in the sysAppLaunchCmdGoto's parameter block.

Result Returns true if Find screen is filled up. Applications should exit from the search if this occurs.

Comments Call this function when your application finds a record with a matching string (FindStrInStr or TxtFindString returns true). This function saves the information you pass. If the user clicks this selection in the Find results dialog, the information is retrieved and used to set up the sysAppLaunchCmdGoto launch code's parameter block.

You can use the appCustom field for any application-specific data that might be needed to navigate to the record if the user selects it. It's common for localizable applications to set appCustom to the length of the matching string because the global find facility cannot correctly determine the length of the matching string on systems with multi-byte character sets. In some character encodings, one character may be accurately represented as either a single-byte character or a multi-byte character. The TxtFindString function accurately matches single-byte characters against their multi-byte equivalents and returns the length of the matching string. If you pass TxtFindString's return value as the appCustom parameter to FindSaveMatch, the matchCustom field of the sysAppLaunchCmdGoTo parameter block contains the length of the matching string.

If your application requires more custom information, you can store the information in a feature and store the feature number in the appCustom field. See the "Feature Manager" chapter for more information.

FindStrInStr

Purpose Perform a case-blind prefix search for a string in another string. This function assumes that the string to find has already been normalized for searching.

Prototype

```
Boolean FindStrInStr (Char const *strToSearch,
Char const *strToFind, UInt16 *posP)
```

Parameters

-> `strToSearch`	String to search.
-> `strToFind`	Normalized version of the text string to be found.
<- `posP`	If a match is found, contains the offset of the match within `strToSearch`.

Result Returns `true` if the string was found. `FindStrInStr` matches the beginnings of words only; that is, `strToFind` must be a prefix of one of the words in `strToSearch` for `FindStrInStr` to return `true`.

Comment Don't use this function on systems that support the text manager. Instead, use `TxtFindString`, which performs searches on strings that contain multi-byte characters and returns the length of the matching text.

For backward compatibility with systems that don't support the text manager, use `TxtGlueFindString`, found in the PalmOSGlue library. `TxtGlueFindString` calls `TxtFindString` if the text manager is present, or `FindStrInStr` if it is not present. For more information, see Chapter 76, "PalmOSGlue Library."

The method by which a search string is normalized varies depending on the version of Palm OS® and the character encoding supported by the device. The string passed to your application in the `strToFind` field of the `sysAppLaunchCmdFind` launch code parameter block has already been normalized. It can be passed directly to `FindStrInStr`, `TxtFindString`, or `TxtGlueFindString`. If you need to create your own normalized search string, use `TxtGluePrepFindString`, also in the PalmOSGlue library.

11

Forms

This chapter provides the following information about form objects:

- Form Data Structures
- Form Constants
- Form Resources
- Form Functions
- Application-Defined Functions

The header file `Form.h` declares the API that this chapter describes. For more information on forms, see the section "Forms, Windows, and Dialogs" in the *Palm OS Programmer's Companion*, vol. I.

Form Data Structures

FormAttrType

The `FormAttrType` bit field defines the visible characteristics of the form.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FormAttrType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16 usable      :1;
  UInt16 enabled     :1;
  UInt16 visible     :1;
  UInt16 dirty       :1;
  UInt16 saveBehind  :1;
  UInt16 graffitiShift :1;
  UInt16 globalsAvailable : 1;
```

```
    UInt16 doingDialog : 1;
    UInt16 exitDialog  : 1;
    UInt16 reserved    :7;
    UInt16 reserved2;
  } FormAttrType;
```

Your code should treat the `FormAttrType` bit field as opaque. Do not attempt to change bit field member values directly.

Field Descriptions

`usable`	Not set if the form is not considered part of the current interface of the application, and it doesn't appear on screen.
`enabled`	Not used.
`visible`	Set or cleared internally when the field object is drawn or erased.
`dirty`	Not used.
`saveBehind`	Set if the bits behind the form are saved when the form is drawn.
`graffitiShift`	Set if the graffiti shift indicator is supported.
`globalsAvailable`	System use only.
`doingDialog`	System use only.
`exitDialog`	System use only.
`reserved`	Reserved for system use.
`reserved2`	Reserved for system use.

Compatibility

The `globalsAvailable`, `doingDialog`, and `exitDialog` flags are present only if 3.5 New Feature Set is present.

FormBitmapType

The `FormBitmapType` structure defines the visible characteristics of a bitmap on a form.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FormBitmapType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  FormObjAttrType  attr;
  PointType        pos;
  UInt16           rscID;
} FormBitmapType;
```

Field Descriptions

`attr`	See `FormObjAttrType`.
`pos`	Location of the bitmap.
`rscID`	Resource ID of the bitmap. If you use `DmGetResource` with this value as the resource ID, it returns a pointer to a `BitmapType` structure.

FormFrameType

The `FormFrameType` structure defines a frame that appears on the form.

```
typedef struct {
  UInt16           id;
  FormObjAttrType  attr;
  RectangleType    rect;
  UInt16           frameType;
} FormFrameType;
```

Field Descriptions

`id` ID of the frame.

`attr` See `FormObjAttrType`.

`rect` Location and size of the frame.

`frameType` The type of frame.

FormGadgetAttrType

The `FormGadgetAttrType` bit field defines a gadget's attributes.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FormGadgetAttrType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16 usable : 1;
  UInt16 extended : 1;
  UInt16 visible : 1;
  UInt16 reserved : 13;
} FormGadgetAttrType;
```

Your code should treat the `FormGadgetAttrType` structure as opaque. Use the functions specified in the descriptions below to retrieve and set each value. Do not attempt to change structure member values directly.

Field Descriptions

`usable` Not set if the gadget is not considered part of the current interface of the application, and it doesn't appear on screen. This is set by `FrmShowObject` and cleared by `FrmHideObject`.

`extended` If set, the gadget is an extended gadget. Extended gadgets are supported if 3.5 New Feature Set is present. An extended gadget has the `handler` field defined in its `FormGadgetType`. If not set, the gadgets is a standard gadget compatible with all releases of Palm OS®.

`visible` Set or cleared when the gadget is drawn or erased. `FrmHideObject` clears this value. You should set it explicitly in the gadget's callback function (if it has one) in response to a draw request.

`reserved` Reserved for future use.

Many form functions (`FrmGetObjectType`, `FrmHideObject`, and `FrmGetObjectBounds`, for example) take an object index as one of their arguments. The most common way to get an object's index is to call `FrmGetObjectIndex`. `FrmGetObjectIndex` takes a form ID and returns the form object's index. This is the routine one should use in most cases, because the application usually knows the object ID. However, gadgets and specifically extended gadgets, have APIs with callbacks that pass back the gadget pointer and not the ID. In those cases, the only way to get the object index (so one can use the `FrmGetObject*` APIs) is to use the function `FrmGetObjectIndexFromPtr`.

If you need the same functionality on pre-Palm OS 4.0 systems then you can accomplish the same thing with the following code snippet.

```
UInt16 index;
UInt16 objIndex = frmInvalidObjectId;
UInt16 numObjects = FrmGetNumberOfObjects(frmP)
for (index = 0; index < numObjects; index++) {
     if (FrmGetObjectPtr(index) == myObjPtr) {
          // Found it
          objIndex = index;
          break;
    }
}
```

Compatibility This type is defined only if 3.5 New Feature Set is present.

FormGadgetType

The `FormGadgetType` structure defines a gadget object that appears on a form.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FormGadgetType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct{
  UInt16                  id;
  FormGadgetAttrType      attr;
  RectangleType           rect;
  const void              *data;
  FormGadgetHandlerType   *handler;
}FormGadgetType;
```

Your code should treat the `FormGadgetType` structure as opaque. Use the functions specified in the descriptions below to retrieve and set each value. Do not attempt to change structure member values directly.

Field Descriptions

`id`	ID of the gadget resource.
`attr`	See `FormGadgetAttrType`.
`rect`	Location and size of the object.
`data`	Pointer to any specific data that needs to be stored. You can set and retrieve the value of this field with `FrmGetGadgetData` and `FrmSetGadgetData`.
`handler`	Pointer to a callback function that controls the gadget's behavior and responds to events. You can set this field with `FrmSetGadgetHandler`.

Many form functions (`FrmGetObjectType`, `FrmHideObject`, and `FrmGetObjectBounds`, for example) take an object index as one of their arguments. The most common way to get an object's index is to call `FrmGetObjectIndex`. `FrmGetObjectIndex` takes a form ID

and returns the form object's index. This is the routine one should use in most cases, because the application usually knows the object ID. However, gadgets and specifically extended gadgets, have APIs with callbacks that pass back the gadget pointer and not the ID. In those cases, the only way to get the object index (so one can use the `FrmGetObject*` APIs) is to use the function `FrmGetObjectIndexFromPtr`.

If you need the same functionality on pre-Palm OS 4.0 systems then you can accomplish the same thing with the following code snippet.

```
UInt16 index;
UInt16 objIndex = frmInvalidObjectId;
UInt16 numObjects = FrmGetNumberOfObjects(frmP)
for (index = 0; index < numObjects; index++) {
     if (FrmGetObjectPtr(index) == myObjPtr) {
          // Found it
          objIndex = index;
          break;
     }
}
```

Compatibility

In Palm OS® releases prior to 3.5, the `attr` field was of type `FormObjAttrType` and the `handler` field did not exist.

FormLabelType

The `FormLabelType` structure defines a label that appears on a form.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FormLabelType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16           id;
  PointType        pos;
  FormObjAttrType  attr;
  FontID           fontID;
  UInt8            reserved;
  Char             *text;
} FormLabelType;
```

Your code should treat the `FormLabelType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`id`	Resource ID of the label.
`pos`	Location of the label.
`attr`	See `FormObjAttrType`.
`fontID`	Font ID of the font used for the label.
`reserved`	Reserved for future use.
`text`	Text of the label.

FormLineType

The `FormLineType` structure defines a line appearing on a form.

```
typedef struct {
  FormObjAttrType    attr;
  PointType          point1;
  PointType          point2;
} FormLineType;
```

Your code should treat the `FormLineType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`attr`	See `FormObjAttrType`.
`point1`	Starting point of the line.
`point2`	Ending point of the line.

FormObjAttrType

The `FormObjAttrType` bit field defines a form object's attributes.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FormObjAttrType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16 usable : 1;
  UInt16 reserved : 15;
} FormObjAttrType;
```

Your code should treat the `FormObjAttrType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`usable` Not set if the object is not considered part of the current interface of the application, and it doesn't appear on screen.

`reserved` Reserved for future use.

FormObjectKind

The `FormObjectKind` enum specifies values for the `objectType` field of the `FormObjListType`. It specifies how to interpret the `object` field.

```
enum formObjects {
  frmFieldObj,
  frmControlObj,
  frmListObj,
  frmTableObj,
  frmBitmapObj,
  frmLineObj,
  frmFrameObj,
  frmRectangleObj,
  frmLabelObj,
  frmTitleObj,
  frmPopupObj,
  frmGraffitiStateObj,
  frmGadgetObj,
  frmScrollbarObj,
};
typedef enum formObjects FormObjectKind;
```

Value Descriptions

`frmFieldObj`	Text field
`frmControlObj`	Control
`frmListObj`	List
`frmTableObj`	Table
`frmBitmapObj`	Form bitmap
`frmLineObj`	Line
`frmFrameObj`	Frame
`frmRectangleObj`	Rectangle
`frmLabelObj`	Label
`frmTitleObj`	Form title
`frmPopupObj`	Popup list
`frmGraffitiStateObj`	Graffiti® state indicator
`frmGadgetObj`	Gadget (custom object)
`frmScrollbarObj`	Scrollbar

FormObjectType

The `FormObjectType` union points to the C structure for a user interface object that appears on the form.

```
typedef union {
  void                  *ptr;
  FieldType             *field;
  ControlType           *control;
  GraphicControlType    *graphicControl;
  SliderControlType     *sliderControl;
  ListType              *list;
  TableType             *table;
  FormBitmapType        *bitmap;
  FormLabelType         *label;
  FormTitleType         *title;
  FormPopupType         *popup;
```

```
    FormGraffitiStateType    *grfState;
    FormGadgetType           *gadget;
    ScrollBarType            *scrollBar;
  } FormObjectType;
```

Your code should treat the `FormObjectType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`ptr`	Used when the object's type is not one of those specified below.
`field`	Text field's structure. See `FieldType`.
`control`	Control's structure. See `ControlType`.
`graphicControl`	Graphic button structure. See `GraphicControlType`.
`sliderControl`	Slider control structure. See `SliderControlType`.
`list`	List object's structure. See `ListType`.
`table`	Table structure. See `TableType`.
`bitmap`	Form bitmap's structure. See `FormBitmapType`.
`label`	Label's structure. See `FormLabelType`.
`title`	Form title's structure. See `FormTitleType`.
`popup`	Popup list's structure. See `FormPopupType`.
`grfState`	Graffiti shift indicator's structure. See `FrmGraffitiStateType`.
`gadget`	Gadget (custom UI resource) structure. See `FormGadgetType`.
`scrollbar`	Scroll bar's structure. See `ScrollBarType`.

Compatibility

The `graphicControl` and `sliderControl` fields are only defined if 3.5 New Feature Set is present.

FormObjListType

The `FormObjectListType` structure specifies a user interface object that appears on the form.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FormObjListType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  FormObjectKind   objectType;
  UInt8            reserved;
  FormObjectType   object;
} FormObjListType;
```

Your code should treat the `FormObjListType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`objectType`	Specifies the type of the object (control, field, etc.). See `FormObjectKind`.
`reserved`	Reserved for future use.
`object`	The C data structure that defines the object. See `FormObjectType`.

FormPopupType

The `FormPopupType` structure defines a popup list that appears on a form.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FormPopupType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16           controlID;
  UInt16           listID;
} FormPopupType;
```

Your code should treat the `FormPopupType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`controlID` Resource ID of the popup trigger control that triggers the list's display.

`listID` Resource ID of the list object that defines the popup list.

FormPtr

The `FormPtr` type defines a pointer to a `FormType` structure.

```
typedef FormType *FormPtr;
```

FormRectangleType

The `FormRectangleType` structure defines a rectangle that appears on the form.

```
typedef struct {
  FormObjAttrType  attr;
  RectangleType    rect;
} FormRectangleType;
```

Your code should treat the `FormRectangleType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`attr` See `FormObjAttrType`.

`rect` Location and size of the rectangle.

FormTitleType

The `FormTitleType` structure defines the title of the form.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FormTitleType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  RectangleType  rect;
  char           *text;
} FormTitleType;
```

Your code should treat the `FormTitleType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`rect` The location and size of the title area.

`text` Text of the title.

FormType

The `FormType` structure and supporting structures are defined below.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FormType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  WindowType                window;
  UInt16                    formId;
  FormAttrType              attr;
  WinHandle                 bitsBehindForm;
  FormEventHandlerType      *handler;
  UInt16                    focus;
```

```
    UInt16                  defaultButton;
    UInt16                  helpRscId;
    UInt16                  menuRscId;
    UInt16                  numObjects;
    FormObjListType         *objects;
} FormType;
```

Your code should treat the `FormType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`window` — Structure of the window object that corresponds to the form. See `WindowType`. Access this field with `FrmGetWindowHandle`.

`formId` — ID number of the form, specified by the application developer. This ID value is part of the event data of `frmOpenEvent`. The ID should match the form's resource ID. Access this field with `FrmGetFormId`.

`attr` — Form object attributes. See `FormAttrType`.

`bitsBehindForm` — Used to save all the bits behind the form so the screen can be properly refreshed when the form is closed. This field is for internal use only by modal forms.

`handler` — Routine called when the form needs to handle an event. You typically set this in your application's event handling function by calling `FrmSetEventHandler`.

`focus` — Index of a field or table object within the form that contains the focus. Any `keyDownEvent` is passed to the object that has the focus. Set to `noFocus` if no object has the focus. Set this field with `FrmSetFocus`.

`defaultButton` — Resource ID of the object defined as the default button. This value is used by the routine `FrmDoDialog`.

`helpRscId`	Resource ID number of the help resource. The help resource is a String resource (type tSTR).
`menuRscId`	ID number of a menu bar to use if the form has a menu, or zero if the form doesn't have a menu.
`numObjects`	Number of objects contained within the form. Access this field with `FrmGetNumberOfObjects`.
`objects`	Pointer to the array of objects contained within the form. See `FormObjListType`.

FrmGraffitiStateType

The `FrmGraffitiStateType` structure defines the graffiti shift indicator.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FrmGraffitiStateType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct{
  PointerType    pos;
}FrmGraffitiStateType;
```

Your code should treat the `FrmGraffitiStateType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`pos`	Location of the graffiti shift indicator.

Form Constants

The following form constants are defined:

Constant	Value	Description
`noFocus`	0xffff	No form object has the focus
`frmRedrawUpdateCode`	0x8000	Indicates that the form should be redrawn; flag in a `frmUpdateEvent`.
`frmNoSelectedControl`	0xff	Returned by `FrmGetControlGroupSelection` if no control is selected.
`frmResponseCreate`	1974	Passed to `FormCheckResponseFunc` to indicate that the function should perform initialization.
`frmResponseQuit`	0xBEEF	Passed to `FormCheckResponseFunc` to indicate that the function should perform cleanup.

Form Resources

The following resources are associated with forms and with the objects on a form whose data structures are defined above:

- Form—Form Resource (tFRM)
- Alert dialog— Alert Resource (Talt)
- Bitmap—Form Bitmap Resource (tFBM)
- Button—Button Resource (tBTN)
- Check box—Check Box Resource (tCBX)
- Field—Field Resource (tFLD)
- Gadget (custom object)— Gadget Resource (tGDT)
- Graffiti shift indicator —Graffiti Shift Indicator Resource (tGSI)
- Label—Label Resource (tLBL)
- List—List Resource (tLST)

- Popup trigger—Popup Trigger Resource (tPUT)
- Push button—Push Button Resource (tPBN)
- Repeating button—Repeating Button Resource (tREP)
- Scrollbar—Scroll Bar Resource (tSCL)
- Selector trigger—Selector Trigger Resource (tSLT)
- Table—Table Resource (tTBL)

Form Functions

FrmAlert

Purpose Create a modal dialog from an alert resource and display it until the user selects a button in the dialog.

Prototype
```
UInt16 FrmAlert (UInt16 alertId)
```

Parameters -> `alertId` ID of the alert resource.

Result Returns the item number of the button the user selected. A button's item number is determined by its order in the alert dialog; the first button has the item number 0 (zero).

NOTE: A default button press is simulated if the user switches to a different application while a modal dialog is active.

See Also `FrmDoDialog`, `FrmCustomAlert`, `FrmCustomResponseAlert`

FrmCloseAllForms

Purpose Send a `frmCloseEvent` to all open forms.

Prototype
```
void FrmCloseAllForms (void)
```

Parameters None.

Result Returns nothing.

Comments Applications can call this function to ensure that all forms are closed cleanly before exiting `PilotMain`; that is, before termination.

See Also `FrmSaveAllForms`

FrmCopyLabel

Purpose Copy the passed string into the data structure of the specified label object in the active form.

Prototype
```
void FrmCopyLabel (FormType *formP,
UInt16 labelID, const Char *newLabel)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `labelID`	ID of form label object.
-> `newLabel`	Pointer to a `NULL`-terminated string.

Result Returns nothing.

Comments The size of the new label **must not** exceed the size of the label defined in the resource. When defining the label in the resource, specify an initial size at least as big as any of the strings that will be assigned dynamically. This function redraws the label if the form's `usable` attribute and the label's `visible` attribute are set.

This function redraws the label but does not erase the old one first. If the new label is shorter than the old one, the end of the old label

will still be visible. To avoid this, you can hide the label using FrmHideObject, then show it using FrmShowObject, after using FrmCopyLabel.

Note that FrmCopyLabel copies the passed string into memory already allocated for the label. Thus, the string doesn't need to remain in existence once FrmCopyLabel returns.

See Also FrmGetLabel

FrmCopyTitle

Purpose Copy a new title over the form's current title. If the form is visible, the new title is drawn.

Prototype

```
void FrmCopyTitle (FormType *formP,
const Char *newTitle)
```

Parameters

-> formP	Pointer to the form object (FormType structure).
-> newTitle	Pointer to the new title string.

Result Returns nothing.

Comments The size of the new title **must not** exceed the title size defined in the resource. When defining the title in the resource, specify an initial size at least as big as any of the strings to be assigned dynamically.

See Also FrmGetTitle, FrmSetTitle

FrmCustomAlert

Purpose Create a modal dialog from an alert resource and display the dialog until the user taps a button in the alert dialog.

Prototype

```
UInt16 FrmCustomAlert (UInt16 alertId,
const Char *s1, const Char *s2, const Char *s3)
```

Parameters

-> `alertId`	Resource ID of the alert.
-> `s1, s2, s3`	Strings to replace ^1, ^2, and ^3 (see Comments).

Result Returns the number of the button the user tapped (the first button is zero).

Comments A button's item number is determined by its order in the alert template; the first button has the item number zero.

Up to three strings can be passed to this routine. They are used to replace the variables ^1, ^2 and ^3 that are contained in the message string of the alert resource.

If the variables ^1, ^2, and ^3 occur in the message string, do not pass `NULL` for the arguments `s1`, `s2`, and `s3`. If you want an argument to be ignored, pass the empty string (`""`). In Palm OS 2.0 or below, pass a string containing a space (`" "`) instead of the empty string.

NOTE: A default button press is simulated if the user switches to a different application while a modal dialog is active.

See Also `FrmAlert`, `FrmDoDialog`, `FrmCustomResponseAlert`

FrmCustomResponseAlert

Purpose Create a modal dialog with a text field from an alert resource and display it until the user taps a button in the alert dialog.

Prototype

```
UInt16 FrmCustomResponseAlert (UInt16 alertId,
const Char *s1, const Char *s2, const Char *s3,
Char *entryStringBuf, Int16 entryStringBufLength,
FormCheckResponseFuncPtr callback)
```

Parameters

-> `alertId` Resource ID of the alert.

-> `s1, s2, s3` Strings to replace ^1, ^2, and ^3. See the Comments in `FrmCustomAlert` for more information.

<- `entryStringBuf`
The string the user entered in the text field.

-> `entryStringBufLength`
The maximum length for the string in `entryStringBuf`.

-> `callback` A callback function that processes the string. See `FormCheckResponseFunc`. Pass `NULL` if there is no callback.

Result Returns the number of the button the user tapped (the first button is zero).

Comments This function differs from `FrmCustomAlert` in these ways:

- The dialog it displays contains a text field for user entry. The text that the user enters is returned in the `entryStringBuf` parameter.
- When the user taps a button, the `callback` function is called and is passed the button number and `entryStringBuf`. The dialog is only dismissed if the callback returns `true`. This behavior allows you to perform error checking on the string that the user entered and give the user a chance to re-enter the string.

The callback function is also called with special constants when the alert dialog is being initialized and when it is being deallocated. This allows the callback to perform any necessary initialization and cleanup.

NOTE: A default button press is simulated if the user switches to a different application while a modal dialog is active.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also FrmAlert, FrmDoDialog

FrmDeleteForm

Purpose Release the memory occupied by a form. Any memory allocated to objects in the form is also released.

Prototype
```
void FrmDeleteForm (FormType *formP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns nothing.

Comments This function doesn't modify the display.

Compatibility If 3.5 New Feature Set is present and the form contains an extended gadget, this function calls the gadget's callback with `formGadgetDeleteCmd`. See `FormGadgetHandler`.

See Also FrmInitForm, FrmReturnToForm

FrmDispatchEvent

Purpose Dispatch an event to the application's handler for the form.

Prototype
```
Boolean FrmDispatchEvent (EventType *eventP)
```

Parameters

-> `eventP`	Pointer to an event.

Result Returns the Boolean value returned by the form's event handler or `FrmHandleEvent`. (If the form's event handler returns `false`, the event is passed to `FrmHandleEvent`.) This function also returns false if the form specified in the event is invalid.

Comments The event is dispatched to the current form's handler unless the form ID is specified in the event data, as, for example, with `frmOpenEvent` or `frmGotoEvent`. A form's event handler (`FormEventHandler`) is registered by `FrmSetEventHandler`.

Note that if the form does not have a registered event handler, this function causes a fatal error.

FrmDoDialog

Purpose Display a modal dialog until the user taps a button in the dialog.

Prototype
```
UInt16 FrmDoDialog (FormType *formP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns the resource ID of the button the user tapped.

NOTE: A default button press is simulated if the user switches to a different application while a modal dialog is active.

See Also `FrmInitForm`, `FrmCustomAlert`, `FrmCustomResponseAlert`

FrmDrawForm

Purpose Draw all objects in a form and the frame around the form.

Prototype
```
void FrmDrawForm (FormType *formP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns nothing.

Comments If the `saveBehind` form attribute is set and the form is visible, this function saves the bits behind the form using the `bitsBehindForm` field in the `FormType` structure.

You should call this function in response to a `frmOpenEvent`.

If you do any custom drawing, you should do so after you call this function not before. If you do custom drawing, respond to `frmUpdateEvent` as well as `frmOpenEvent`, and be sure to return true to specify that the `frmUpdateEvent` was handled. The default event handler for `frmUpdateEvent` calls `FrmDrawForm`, so if you allow the event to fall through by returning `false`, your custom drawing is erased.

Compatibility If 3.5 New Feature Set is present, `FrmDrawForm` erases the form's window before performing any drawing. Thus, it is especially important to do any custom drawing after this function call on Palm OS 3.5 and higher.

If 3.5 New Feature Set is present and the form contains an extended gadget, this function calls the gadget's callback with `formGadgetDrawCmd`. See `FormGadgetHandler`.

See Also `FrmEraseForm`, `FrmInitForm`

FrmEraseForm

Purpose Erase a form from the display.

Prototype
```
void FrmEraseForm (FormType *formP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns nothing.

Comments If the region obscured by the form was saved by `FrmDrawForm`, this function restores that region.



New FrmGetActiveField

Purpose Return the active field for a specified form.

Prototype
```
FieldType *FrmGetActiveField
(const FormType *formP)
```

Parameters

`-> formP`	Pointer to the form for which the active field should be returned, or `NULL` if the active field on the active form is desired.

Result Returns a pointer to the field object of the active field, or `NULL` if the form doesn't have an active field or if there is no active form.

Comments This function will most often be called with a `NULL` parameter to obtain the active field on the active form.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `FrmGetActiveForm`

FrmGetActiveForm

Purpose Return the currently active form.

Prototype
```
FormType *FrmGetActiveForm (void)
```

Parameters None.

Result Returns a pointer to the form object of the active form.

Comments You should not call the `FrmGetActiveForm` function when a popup window is open. There is no active form while a popup is displayed, and the value returned from `FrmGetActiveForm` in this situation has no meaning.

See Also `FrmGetActiveField`, `FrmGetActiveFormID`, `FrmSetActiveForm`

FrmGetActiveFormID

Purpose Return the ID of the currently active form.

Prototype
```
UInt16 FrmGetActiveFormID (void)
```

Parameters None.

Result Returns the active form's ID number.

See Also `FrmGetActiveForm`

FrmGetControlGroupSelection

Purpose Return the item number of the control selected in a group of controls.

Prototype
```
UInt16 FrmGetControlGroupSelection
(const FormType *formP, UInt8 groupNum)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `groupNum`	Control group number.

Result Returns the item number of the selected control; returns `frmNoSelectedControl` if no item is selected.

Comments The item number is the index into the form object's data structure.

> **NOTE:** `FrmSetControlGroupSelection` sets the selection in a control group based on an object ID, **not** its index, which `FrmGetControlGroupSelection` returns.

Compatibility On versions prior to 3.5, this function returned a `Byte` instead of `UInt16`.

See Also `FrmGetObjectId`, `FrmGetObjectPtr`, `FrmSetControlGroupSelection`

FrmGetControlValue

Purpose Return the current value of a control.

Prototype
```
Int16 FrmGetControlValue (const FormType *formP,
UInt16 objIndex)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

	-> `objIndex`	Index of the control object in the form object's data structure. You can obtain this by using `FrmGetObjectIndex`.

Result Returns the current value of the control. For most controls the return value is either 0 (off) or 1 (on). For sliders, this function returns the value of the `value` field.

Comments The caller must specify a valid index. This function is valid only for push button and check box control objects.

See Also `FrmSetControlValue`

FrmGetFirstForm

Purpose Return the first form in the window list.

Prototype

```
FormType *FrmGetFirstForm (void)
```

Parameters None.

Result Returns a pointer to a form object, or `NULL` if there are no forms.

Comments The window list is a LIFO stack. The last window created is the first window in the window list.

FrmGetFocus

Purpose Return the item (index) number of the object that has the focus.

Prototype

```
UInt16 FrmGetFocus (const FormType *formP)
```

Parameters

	-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns the index of the object (UI element) that has the focus, or returns `noFocus` if none does. To convert the object index to an ID, use `FrmGetObjectId`.

See Also `FrmGetObjectPtr`, `FrmSetFocus`

FrmGetFormBounds

Purpose Return the visual bounds of the form; the region returned includes the form's frame.

Prototype

```
void FrmGetFormBounds (const FormType *formP,
RectangleType *rP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
<- `rP`	Pointer to a `RectangleType` structure where the bounds is returned.

Result Returns nothing. The bounds of the form are returned in `r`.

FrmGetFormId

Purpose Return the resource ID of a form.

Prototype

```
UInt16 FrmGetFormId (const FormType *formP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns form resource ID.

See Also `FrmGetFormPtr`

FrmGetFormPtr

Purpose Return a pointer to the form that has the specified ID.

Prototype
```
FormType *FrmGetFormPtr (UInt16 formId)
```

Parameters

-> `formId`	Form ID number.

Result Returns a pointer to the form object, or `NULL` if the form is not in memory.

See Also `FrmGetFormId`

FrmGetGadgetData

Purpose Return the value stored in the `data` field of the gadget object.

Prototype
```
void *FrmGetGadgetData (const FormType *formP,
UInt16 objIndex)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of the gadget object in the form object's data structure. You can obtain this by using `FrmGetObjectIndex`.

Result Returns a pointer to the custom gadget's data.

Comments Gadget objects provide a way for an application to attach custom gadgetry to a form. In general, the `data` field of a gadget object contains a pointer to the custom object's data structure.

See Also `FrmSetGadgetData`, `FrmSetGadgetHandler`

FrmGetLabel

Purpose Return pointer to the text of the specified label object in the specified form.

Prototype
```
const Char *FrmGetLabel (const FormType *formP,
UInt16 labelID)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `labelID`	ID of the label object.

Result Returns a pointer to the label string.

Comments Does not make a copy of the string; returns a pointer to the string. The object must be a label.

See Also `FrmCopyLabel`

FrmGetNumberOfObjects

Purpose Return the number of objects in a form.

Prototype
```
UInt16 FrmGetNumberOfObjects
(const FormType *formP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns the number of objects in the specified form.

See Also `FrmGetObjectPtr`, `FrmGetObjectId`

FrmGetObjectBounds

Purpose Retrieve the bounds of an object given its form and index.

Prototype
```
void FrmGetObjectBounds (const FormType *formP,
UInt16 objIndex, RectangleType *rP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of an object in the form. You can obtain this by using `FrmGetObjectIndex`.
<- `rP`	Pointer to a `RectangleType` structure where the object bounds are returned. The bounds are in window-relative coordinates.

Result Returns nothing. The object's bounds are returned in `r`.

See Also `FrmGetObjectPosition`, `FrmSetObjectPosition`

FrmGetObjectId

Purpose Return the ID of the specified object.

Prototype
```
UInt16 FrmGetObjectId (const FormType *formP,
UInt16 objIndex)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of an object in the form. You can obtain this by using `FrmGetObjectIndex`.

Result Returns the ID number of an object or `frmInvalidObjectId` if the `objIndex` parameter is invalid.

See Also `FrmGetObjectPtr`

FrmGetObjectIndex

Purpose Return the index of an object in the form's objects list.

Prototype
```
UInt16 FrmGetObjectIndex (const FormType *formP,
UInt16 objID)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objID`	ID of an object in the form.

Result Returns the index of the specified object (the index of the first object is 0), or `frmInvalidObjectId` if the supplied object ID is invalid.

Comments Bitmaps use a different mechanism for IDs than the rest of the form objects. When finding a bitmap with `FrmGetObjectIndex`, you need to pass the bitmap's resource ID, not the ID of the form bitmap object. (Passing the ID of the form bitmap object may or may not give you the right object back, depending on how you created the objects.)

This means that if you've got the same bitmap in two different form bitmap objects on the same form, you won't be able to use `FrmGetObjectIndex` to get at the second one; it'll always return the first.

See Also `FrmGetObjectPtr`, `FrmGetObjectId`



New FrmGetObjectIndexFromPtr

Purpose Return an object's index.

Prototype
```
UInt16 FrmGetObjectIndexFromPtr
(const FormType *formP, void *objP)
```

Parameters

-> `formP`	Pointer to a `FormType`.
-> `objP`	Pointer to an object.

Result Returns the object's index. `frmInvalidObjectId` is returned if `objP` is not associated with the form.

Comments Many form functions (`FrmGetObjectType`, `FrmHideObject`, and `FrmGetObjectBounds`, for example) take an object index as one of their arguments. The most common way to get an object's index is to call `FrmGetObjectIndex`. `FrmGetObjectIndex` takes a form ID and returns the form object's index. This is the routine one should use in most cases, because the application usually knows the object ID. However, gadgets and specifically extended gadgets, have APIs with callbacks that pass back the gadget pointer and not the ID. In those cases, the only way to get the object index (so one can use the `FrmGetObject*` APIs) is to use the function `FrmGetObjectIndexFromPtr`.

If you need the same functionality on pre-Palm OS 4.0 systems then you can accomplish the same thing with the following code snippet.

```
UInt16 index;
UInt16 objIndex = frmInvalidObjectId;
UInt16 numObjects = FrmGetNumberOfObjects(frmP)
for (index = 0; index < numObjects; index++) {
     if (FrmGetObjectPtr(index) == myObjPtr) {
          // Found it
          objIndex = index;
          break;
     }
}
```

FrmGetObjectPosition

Purpose Return the coordinates of the specified object relative to the form.

Prototype

```
void FrmGetObjectPosition (const FormType *formP,
UInt16 objIndex, Coord *x, Coord *y)
```

Parameters

-> `formP` Pointer to the form object (`FormType` structure).

-> `objIndex` Index of an object in the form. You can obtain this by using `FrmGetObjectIndex`.

	`<- x, y`	Pointers where the window-relative x and y positions of the object are returned. These locate the top-left corner of the object.

Result Returns nothing.

See Also `FrmGetObjectBounds`, `FrmSetObjectPosition`

FrmGetObjectPtr

Purpose Return a pointer to the data structure of an object in a form.

Prototype

```
void *FrmGetObjectPtr (const FormType *formP,
UInt16 objIndex)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of an object in the form. You can obtain this by using `FrmGetObjectIndex`.

Result Returns a pointer to an object in the form.

See Also `FrmGetObjectId`

FrmGetObjectType

Purpose Return the type of an object.

Prototype

```
FormObjectKind FrmGetObjectType
(const FormType *formP, UInt16 objIndex)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

	-> `objIndex`	Index of an object in the form. You can obtain this by using `FrmGetObjectIndex`.

Result Returns `FormObjectKind` of the item specified. See `FormObjectKind`.

FrmGetTitle

Purpose Return a pointer to the title string of a form.

Prototype
```
const Char *FrmGetTitle (const FormType *formP)
```

Parameters

	-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns a pointer to title string, or `NULL` if there is no title string or there is an error finding it.

Comments This is a pointer to the internal structure itself, **not** to a copy.

See Also `FrmCopyTitle`, `FrmSetTitle`

FrmGetWindowHandle

Purpose Return the window handle of a form.

Prototype
```
WinHandle FrmGetWindowHandle
(const FormType *formP)
```

Parameters

	-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns the handle of the memory block that contains the form data structure. Since the form structure begins with the `WindowType`, this is also a `WinHandle`.

FrmGotoForm

Purpose Send a `frmCloseEvent` to the current form; send a `frmLoadEvent` and a `frmOpenEvent` to the specified form.

Prototype
```
void FrmGotoForm (UInt16 formId)
```

Parameters

-> `formId`	ID of the form to display.

Result Returns nothing.

Comments The default form event handler (`FrmHandleEvent`) erases and disposes of a form when it receives a `frmCloseEvent`.

See Also `FrmPopupForm`

FrmHandleEvent

Purpose Handle the event that has occurred in the form.

Prototype
```
Boolean FrmHandleEvent (FormType *formP,
EventType *eventP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `eventP`	Pointer to the event data structure (`EventType`).

Result Returns `true` if the event was handled.

Comments Never call this function directly. Call `FrmDispatchEvent` instead. `FrmDispatchEvent` passes events to a form's custom event handler and then, if the event was not handled, to this function.

WARNING! You should never call this function directly. You should call the FrmDispatchEvent function instead.

Table 11.1 provides an overview of how `FrmHandleEvent` handles different events.

Table 11.1 FrmHandleEvent Actions

When FrmHandleEvent receives...	FrmHandleEvent performs these actions...
`ctlEnterEvent`	Passes the event and a pointer to the object the event occurred in to `CtlHandleEvent`. The object pointer is obtained from the event data. If the control is part of an exclusive control group, it deselects the currently selected control of the group first.
`ctlRepeatEvent`	Passes the event and a pointer to the object the event occurred in to `CtlHandleEvent`. The object pointer is obtained from the event data.
`ctlSelectEvent`	Checks if the control is a Popup Trigger Control. If it is, the list associated with the popup trigger is displayed until the user makes a selection or touches the pen outside the bounds of the list. If a selection is made, a `popSelectEvent` is added to the event queue.
`fldEnterEvent` or `fldHeightChangedEvent`	Checks if a field object or a table object has the focus and passes the event to the appropriate handler (`FldHandleEvent` or `TblHandleEvent`). The table object is also a container object, which may contain a field object. If `TblHandleEvent` receives a field event, it passes the event to the field object contained within it.
`frmCloseEvent`	Erases the form and releases any memory allocated for it.
`frmGadgetEnterEvent`	Passes the event to the gadget's callback function if the gadget has one. See `FormGadgetHandler`.
`frmGadgetMiscEvent`	Passes the event to the gadget's callback function if the gadget has one. See `FormGadgetHandler`.

Table 11.1 FrmHandleEvent Actions *(continued)*

When FrmHandleEvent receives...	FrmHandleEvent performs these actions...
frmTitleEnterEvent	Tracks the pen until it is lifted. If it is lifted within the bounds of the form title, adds a frmTitleSelectEvent event to the event queue.
frmTitleSelectEvent	Adds a keyDownEvent with the vchrMenu character to the event queue.
frmUpdateEvent	Calls FrmDrawForm to redraw the form.
keyDownEvent	Passes the event to the handler for the object that has the focus. If no object has the focus, the event is ignored.
lstEnterEvent	Passes the event and a pointer to the object the event occurred in to LstHandleEvent. The object pointer is obtained from the event data.
menuCmdBarOpenEvent	Checks if a field object or a table object has the focus and passes the event to the appropriate handler (FldHandleEvent or TblHandleEvent), broadcasts the notification sysNotifyMenuCmdBarOpenEvent, and then displays the command toolbar.
menuEvent	Checks if the menu command is one of the system edit menu commands. The system provides a standard edit menu that contains the commands Undo, Cut, Copy, Paste, Select All, and Keyboard. FrmHandleEvent responds to these commands.
penDownEvent; pen position in the bounds of the form object	Checks the list of objects contained by the form to determine if the pen is within the bounds of one. If it is, the appropriate handler is called to handle the event, for example, if the pen is in a control, CtlHandleEvent is called. If the pen isn't within the bounds of an object, the event is ignored by the form. If the pen is within the bounds of the help icon, it is tracked until it is lifted, and if it's still within the help icon bounds, the help dialog is displayed.

Table 11.1 FrmHandleEvent Actions *(continued)*

When FrmHandleEvent receives...	FrmHandleEvent performs these actions...
popSelectEvent	Sets the label of the popup trigger to the current selection of the popup list.
sclEnterEvent or sclRepeatEvent	Passes the event and a pointer to the object the event occurred in to SclHandleEvent.
tblEnterEvent	Passes the event and a pointer to the object the event occurred in to TblHandleEvent. The object pointer is obtained from the event data.

Compatibility FrmHandleEvent only handles frmTitleSelectEvent, menuCmdBarOpenEvent, frmGadgetEnterEvent, and frmGadgetMiscEvent if 3.5 New Feature Set is present.

See Also FrmDispatchEvent

FrmHelp

Purpose Display the specified help message until the user taps the Done button in the help dialog.

Prototype
```
void FrmHelp (UInt16 helpMsgId)
```

Parameters -> helpMsgId Resource ID of help message string.

Result Returns nothing.

Comments The help message is displayed in a modal dialog that supports scrolling the text if necessary.

FrmHideObject

Purpose Erase the specified object and set its attribute data (`usable` bit) so that it does not redraw or respond to the pen.

Prototype

```
void FrmHideObject (FormType *formP,
UInt16 objIndex)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of an object in the form. You can obtain this by using `FrmGetObjectIndex`.

Result Returns nothing.

Compatibility Prior to OS version 3.2, this function did not set the `usable` bit of the object attribute data to `false`. On an OS version prior to 3.2 you can work around this bug by directly setting this bit to `false` yourself.

On versions of Palm OS prior to 3.5 this function doesn't affect lists or tables. On Palm OS 3.5 it operates correctly on lists but doesn't have any effect on tables. On Palm OS 4.0 it operates correctly on both lists and tables.

If 3.5 New Feature Set is present and the object is an extended gadget, this function calls the gadget's callback with `formGadgetEraseCmd`. See `FormGadgetHandler`.

See Also `FrmShowObject`

FrmInitForm

Purpose Load and initialize a form resource.

Prototype
```
FormType *FrmInitForm (UInt16 rscID)
```

Parameters

-> `rscID` Resource ID of the form.

Result Returns a pointer to the form data structure.

When using debug ROMs, `FrmInitForm` displays an error message if the form has already been initialized.

Comments This function does not affect the display (use `FrmDrawForm` to draw the form) nor make the form active (use `FrmSetActiveForm` to make it active).

For each initialized form, you must call `FrmDeleteForm` to release the form memory when you are done with the form. Alternatively, you can free the active form by calling `FrmReturnToForm`.

See Also `FrmDoDialog`, `FrmDeleteForm`, `FrmReturnToForm`

FrmNewBitmap

Purpose Create a new form bitmap dynamically.

Prototype
```
FormBitmapType *FrmNewBitmap (FormType **formPP,
UInt16 ID, UInt16 rscID, Coord x, Coord y)
```

Parameters

<-> `formPP` Pointer to a pointer to the form in which the new bitmap is installed. This value is not a handle; that is, the old `formPP` value is not necessarily valid after this function returns because the form may be moved in memory. In subsequent calls, always use the new `formPP` value returned by this function.

-> `ID`	Symbolic ID of the bitmap, specified by the developer. By convention, this ID should match the resource ID (not mandatory).
-> `rscID`	Numeric value identifying the resource that provides the bitmap. This value must be unique within the application scope.
-> `x`	Horizontal coordinate of the upper-left corner of the bitmap's boundaries, relative to the window in which it appears.
-> `y`	Vertical coordinate of the upper-left corner of the bitmap's boundaries, relative to the window in which it appears.

Result Returns a pointer to the new bitmap, or 0 if the call did not succeed. The most common cause of failure is lack of memory.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FrmRemoveObject`

FrmNewForm

Purpose Create a new form object dynamically.

Prototype

```
FormType *FrmNewForm (UInt16 formID,
const Char *titleStrP, Coord x, Coord y,
Coord width, Coord height, Boolean modal,
UInt16 defaultButton, UInt16 helpRscID,
UInt16 menuRscID)
```

Parameters

-> `formID`	Symbolic ID of the form, specified by the developer. By convention, this ID should match the resource ID (not mandatory).
-> `titleStrP`	Pointer to a string that is the title of the form.
-> `x`	Horizontal coordinate of the upper-left corner of the form's boundaries, relative to the window in which it appears.

-> `y`	Vertical coordinate of the upper-left corner of the form's boundaries, relative to the window in which it appears.
-> `width`	Width of the form, expressed in pixels. Valid values are 1 -160.
-> `height`	Height of the form, expressed in pixels.Valid values are 1 -160.
-> `modal`	`true` specifies that the form ignores pen events outside its boundaries.
-> `defaultButton`	Symbolic ID of the button that provides the form's default action, specified by the developer.
-> `helpRscID`	Symbolic ID of the resource that provides the form's online help, specified by the developer. Only modal dialogs can have help resources.
-> `menuRscID`	Symbolic ID of the resource that provides the form's menus, specified by the developer.

Result Returns a pointer to the new form object, or 0 if the call did not succeed. The most common cause of failure is lack of memory.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FrmValidatePtr`, `WinValidateHandle`, `FrmRemoveObject`

FrmNewGadget

Purpose Create a new gadget dynamically and install it in the specified form.

Prototype

```
FormGadgetType *FrmNewGadget (FormType **formPP,
UInt16 id, Coord x, Coord y, Coord width,
Coord height)
```

Parameters

<-> `formPP`	Pointer to a pointer to the form in which the new gadget is installed. This value is not a handle; that is, the old `formPP` value is not necessarily valid after this function returns because the form may be moved in memory. In subsequent calls, always use the new `formPP` value returned by this function.
-> `id`	Symbolic ID of the gadget, specified by the developer. By convention, this ID should match the resource ID (not mandatory).
-> `x`	Horizontal coordinate of the upper-left corner of the gadget's boundaries, relative to the window in which it appears.
-> `y`	Vertical coordinate of the upper-left corner of the gadget's boundaries, relative to the window in which it appears.
-> `width`	Width of the gadget, expressed in pixels. Valid values are 1 - 160.
-> `height`	Height of the gadget, expressed in pixels.Valid values are 1 - 160.

Result Returns a pointer to the new gadget object or 0 if the call did not succeed. The most common cause of failure is lack of memory.

Comments A gadget is a custom user interface object. For more information, see "Gadget Resource" on page 51.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FrmRemoveObject`

FrmNewGsi

Purpose Create a new Graffiti shift indicator dynamically and install it in the specified form.

Prototype

```
FrmGraffitiStateType *FrmNewGsi
(FormType **formPP, Coord x, Coord y)
```

Parameters

<-> `formPP`	Pointer to a pointer to the form in which the new Graffiti shift indicator is installed. This value is not a handle; that is, the old `formPP` value is not necessarily valid after this function returns because the form may be moved in memory. In subsequent calls, always use the new `formPP` value returned by this function.
-> `x`	Horizontal coordinate of the upper-left corner of the Graffiti shift indicator's boundaries, relative to the window in which it appears.
-> `y`	Vertical coordinate of the upper-left corner of the Graffiti shift indicator's boundaries, relative to the window in which it appears.

Result Returns a pointer to the new gadget object or 0 if the call did not succeed. The most common cause of failure is lack of memory.

Comments In normal operation, the Graffiti shift indicator is drawn in the lower-right portion of the screen when the user enters the shift keystroke. You use this function if the Graffiti shift indicator needs to be drawn in a nonstandard location. For example, the form manager uses it to draw the shift indicator in a custom alert dialog that contains a text field (`FrmCustomResponseAlert`).

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `FrmRemoveObject`

FrmNewLabel

Purpose Create a new label object dynamically and install it in the specified form.

Prototype

```
FormLabelType *FrmNewLabel (FormType **formPP,
UInt16 ID, const Char *textP, Coord x, Coord y,
FontID font)
```

Parameters

<-> `formPP`	Pointer to a pointer to the form in which the new label is installed. This value is not a handle; that is, the old `formPP` value is not necessarily valid after this function returns because the form may be moved in memory. In subsequent calls, always use the new `formPP` value returned by this function.
-> `ID`	Symbolic ID of the label, specified by the developer. By convention, this ID should match the resource ID (not mandatory).
-> `textP`	Pointer to a string that provides the label text. This string is copied into the label structure.
-> `x`	Horizontal coordinate of the upper-left corner of the label's boundaries, relative to the window in which it appears.
-> `y`	Vertical coordinate of the upper-left corner of the label's boundaries, relative to the window in which it appears.
-> `font`	Font with which to draw the label text.

Result Returns a pointer to the new label object or 0 if the call did not succeed. The most common cause of failure is lack of memory.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `CtlValidatePointer`, `FrmRemoveObject`

FrmPointInTitle

Purpose Check if a coordinate is within the bounds of the form's title.

Prototype
```
Boolean FrmPointInTitle (const FormType *formP,
Coord x, Coord y)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `x, y`	Window-relative x and y coordinates.

Result Returns `true` if the specified coordinate is in the form's title.

Compatibility Implemented only if 2.0 New Feature Set is present.

FrmPopupForm

Purpose Queues a `frmLoadEvent` and a `frmOpenEvent` for the specified form.

Prototype
```
void FrmPopupForm (UInt16 formId)
```

Parameters

-> `formID`	Resource ID of form to open.

Result Returns nothing.

Comments This routine differs from `FrmGotoForm` in that the current form is not closed. You can call `FrmReturnToForm` to close a form opened by `FrmPopupForm`.

FrmRemoveObject

Purpose Remove the specified object from the specified form.

Prototype

```
Err FrmRemoveObject (FormType **formPP,
UInt16 objIndex)
```

Parameters

<-> `formPP` Pointer to a pointer to the form from which this function removes an object. This value is not a handle; that is, the old `formPP` value is not necessarily valid after this function returns. In subsequent calls, always use the new `formPP` value returned by this function.

-> `objIndex` The object to remove, specified as an index into the list of objects installed in the form. You can use the `FrmGetObjectIndex` function to discover this value.

Result Returns 0 if no error.

Comments You can use this function to remove any form object (a bitmap, control, list, and so on) and free the memory allocated to it within the form data structure. The data structures for most form objects are embedded within the form data structure memory chunk. This function frees that memory and moves the other objects, if necessary, to close up the memory "hole" and decrease the size of the form chunk.

Note that this function does not free memory outside the form data structure that may be allocated to an object, such as the memory allocated to the string in an editable field object.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FrmNewBitmap`, `FrmNewForm`, `FrmNewGadget`, `FrmNewLabel`, `CtlNewControl`, `FldNewField`, `LstNewList`

FrmRestoreActiveState

Purpose Macro that restores the active window and form state.

Prototype
```
FrmRestoreActiveState (stateP)
```

Parameters

-> `stateP`	A pointer to the `FormActiveStateType` structure that you passed to `FrmSaveActiveState` when you saved the state.

Result Returns zero on success.

Comments Use this function to restore the state of displayed forms to the state that existed before you dynamically showed a new modal form. You must have previously called `FrmSaveActiveState` to save the state.

Compatibility Implemented only if 3.0 New Feature Set is present.

FrmReturnToForm

Purpose Erase and delete the currently active form and make the specified form the active form.

Prototype
```
void FrmReturnToForm (UInt16 formId)
```

Parameters

-> `formID`	Resource ID of the form to return to.

Result Returns nothing.

Comments It is assumed that the form being returned to is already loaded into memory and initialized. Passing a form ID of 0 returns to the first form in the window list, which is the last form to be loaded.

`FrmReturnToForm` does not generate a `frmCloseEvent` when called from a modal form's event handler. It assumes that you have

already handled cleaning up your form's variables since you are explicitly calling `FrmReturnToForm`.

See Also `FrmGotoForm`, `FrmPopupForm`

FrmSaveActiveState

Purpose Macro that saves the active window and form state.

Prototype

```
FrmSaveActiveState (stateP)
```

Parameters

<-> `stateP`	A pointer to a `FormActiveStateType` structure that is used to save the state. Pass the same pointer to `FrmRestoreActiveState` to restore the state. Treat the structure like a black box; that is, don't attempt to read it or write to it.

Result Returns zero on success.

Comments Use this function to save the state of displayed forms before dynamically showing a new modal form. Call `FrmRestoreActiveState` to restore the state after you remove the modal form.

Compatibility Implemented only if 3.0 New Feature Set is present.

FrmSaveAllForms

Purpose Send a `frmSaveEvent` to all open forms.

Prototype

```
void FrmSaveAllForms (void)
```

Parameters None.

Result Returns nothing.

See Also `FrmCloseAllForms`

FrmSetActiveForm

Purpose Set the active form. All input (key and pen) is directed to the active form and all drawing occurs there.

Prototype
```
void FrmSetActiveForm (FormType *formP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns nothing.

Comments A `penDownEvent` outside the form but within the display area is ignored.

Compatibility In Palm OS releases earlier than 3.5, this function generated a `winEnterEvent` for the new form immediately following the `winExitEvent` for the old form. Starting in Palm OS 3.5, `FrmSetActiveForm` does not generate the `winEnterEvent`. The `winEnterEvent` does not occur until the newly active form is drawn.

See Also `FrmGetActiveForm`

FrmSetCategoryLabel

Purpose Set the category label displayed on the title line of a form. If the form's `visible` attribute is set, redraw the label.

Prototype
```
void FrmSetCategoryLabel (const FormType *formP,
UInt16 objIndex, Char *newLabel)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of an object in the form. You can obtain this by using `FrmGetObjectIndex`.
-> `newLabel`	Pointer to the name of the new category.

Result Returns nothing.

Comments The pointer to the new label (`newLabel`) is saved in the object.

FrmSetControlGroupSelection

Purpose Set the selected control in a group of controls.

Prototype

```
void FrmSetControlGroupSelection
(const FormType *formP, UInt8 groupNum,
UInt16 controlID)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `groupNum`	Control group number.
-> `controlID`	ID of control to set.

Result Returns nothing.

Comments This function unsets all the other controls in the group. The display is updated.

NOTE: `FrmGetControlGroupSelection` returns the selection in a control group as an object index, **not** as an object ID, which `FrmSetControlGroupSelection` uses to set the selection.

See Also `FrmGetControlGroupSelection`

FrmSetControlValue

Purpose Set the current value of a control. If the control is visible, it's redrawn.

Prototype
```
void FrmSetControlValue (const FormType *formP,
UInt16 objIndex, Int16 newValue)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of the control in the form. You can obtain this by using `FrmGetObjectIndex`.
-> `newValue`	New value to set for the control. For sliders, specify a value between the slider's minimum and maximum. For graphical controls, push buttons, or check boxes, specify 0 for off, nonzero for on.

Result Returns nothing.

Comments This function works only with graphical controls, sliders, push buttons, and check boxes. If you set the value of any other type of control, the behavior is undefined.

See Also `FrmGetControlValue`

FrmSetEventHandler

Purpose Registers the event handler callback routine for the specified form.

Prototype
```
void FrmSetEventHandler (FormType *formP,
FormEventHandlerType *handler)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `handler`	Address of the form event handler function, `FormEventHandler`.

Result Returns nothing.

Comments FrmDispatchEvent calls this handler whenever it receives an event for a specific form.

FrmSetEventHandler must be called right after a form resource is loaded. The callback routine it registers is the mechanism for dispatching events to an application. The tutorial explains how to use callback routines.

FrmSetFocus

Purpose Set the focus of a form to the specified object.

Prototype
```
void FrmSetFocus (FormType *formP,
UInt16 fieldIndex)
```

Parameters

-> formP	Pointer to the form object (FormType structure).
-> fieldIndex	Index of the object to get the focus in the form. You can obtain this by using FrmGetObjectIndex. You can pass the constant noFocus so that no object has the focus.

Result Returns nothing.

Comments You can set the focus to a field or table object. If the focus is set to a field object, this function turns on the insertion point in the field by calling FldGrabFocus internally.

See Also FrmGetFocus

FrmSetGadgetData

Purpose Store a data value in the `data` field of the gadget object.

Prototype

```
void FrmSetGadgetData (FormType *formP,
UInt16 objIndex, const void *data)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of an object in the form. You can obtain this by using `FrmGetObjectIndex`.
-> `data`	Application-defined value. This value is stored into the `data` field of the gadget data structure (`FormGadgetType`).

Result Returns nothing.

Comments Gadget objects provide a way for an application to attach custom gadgetry to a form. Typically, the `data` field of a gadget object contains a pointer to the custom object's data structure.

See Also `FrmGetGadgetData`, `FrmSetGadgetHandler`

FrmSetGadgetHandler

Purpose Registers the gadget event handler callback routine for the specified gadget on the specified form.

Prototype

```
void FrmSetGadgetHandler (FormType *formP,
UInt16 objIndex, FormGadgetHandlerType *attrP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of a gadget object in the form. You can obtain this by using `FrmGetObjectIndex`.
-> `attrP`	Address of the callback function. See `FormGadgetHandler`.

Result Returns nothing.

Comments This function sets the application-defined function that controls the specified gadget's behavior. This function is called when the gadget needs to be drawn, erased, deleted, or needs to handle an event.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also FrmGetGadgetData, FrmSetGadgetData

FrmSetMenu

Purpose Change a form's menu bar and make the new menu active.

Prototype

```
void FrmSetMenu (FormType *formP,
UInt16 menuRscID)
```

Parameters

-> formP	Pointer to the form object (FormType structure).
-> menuRscID	Resource ID of the menu.

Result Returns nothing.

Compatibility Implemented only if 2.0 New Feature Set is present.

FrmSetObjectBounds

Purpose Set the bounds or position of an object.

Prototype

```
void FrmSetObjectBounds (FormType *formP,
UInt16 objIndex, const RectangleType *bounds)
```

Parameters

-> formP	Pointer to the form object (FormType structure).
-> objIndex	Index of an object in the form. You can obtain this by using FrmGetObjectIndex.

	-> `bounds`	Window-relative bounds. For the following objects, this sets only the position of the top-left corner: label, bitmap, and Graffiti state indicator.

Result Returns nothing.

Comments Doesn't update the display.

Compatibility Implemented only if 2.0 New Feature Set is present.

FrmSetObjectPosition

Purpose Set the position of an object.

Prototype

```
void FrmSetObjectPosition (FormType *formP,
UInt16 objIndex, Coord x, Coord y)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of an object in the form. You can obtain this by using `FrmGetObjectIndex`.
-> `x`	Window-relative horizontal coordinate.
-> `y`	Window-relative vertical coordinate.

Result Returns nothing.

See Also `FrmGetObjectPosition`, `FrmGetObjectBounds`

FrmSetTitle

Purpose Set the title of a form. If the form is visible, draw the new title.

Prototype

```
void FrmSetTitle (FormType *formP, Char *newTitle)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

	-> `newTitle`	Pointer to the new title string.

Result Returns nothing.

Comments This function draws the title if the form is visible.

This function saves the pointer passed in `newTitle`; it does **not** make a copy. The value of `newTitle` must not be a pointer to a stack-based object.

Compatibility Earlier versions of this function redrew the title without first erasing the old one. This problem was corrected in version 3.0 of Palm OS.

See Also `FrmGetTitle`, `FrmCopyTitle`, `FrmCopyLabel`

FrmShowObject

Purpose Set a form object as usable. If the form is visible, draw the object.

Prototype

```
void FrmShowObject (FormType *formP,
UInt16 objIndex)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `objIndex`	Index of an object in the form. You can obtain this by using `FrmGetObjectIndex`.

Result Returns nothing.

Compatibility On versions of Palm OS prior to 3.5 this function doesn't affect lists or tables. On Palm OS 3.5 it operates correctly on lists but doesn't have any effect on tables. On Palm OS 4.0 it operates correctly on both lists and tables.

If 3.5 New Feature Set is present and the object is an extended gadget, this function calls the gadget's callback with `formGadgetDrawCmd`. See `FormGadgetHandler`.

See Also `FrmHideObject`

FrmUpdateForm

Purpose Send a `frmUpdateEvent` to the specified form.

Prototype

```
void FrmUpdateForm (UInt16 formId,
UInt16 updateCode)
```

Parameters

-> `formId`	Resource ID of form to update.
-> `updateCode`	An application-defined code that can be used to indicate what needs to be updated. Specify the code `frmRedrawUpdateCode` to indicate that the whole form should be redrawn.

Result Returns nothing.

Comments If the `frmUpdateEvent` posted by this function is handled by the default form event handler, `FrmHandleEvent`, the `updateCode` parameter is ignored. `FrmHandleEvent` always redraws the form.

If you handle the `frmUpdateEvent` in a custom event handler, you can use the `updateCode` parameter any way you want. For example, you might use it to indicate that only a certain part of the form needs to be redrawn. If you do handle the `frmUpdateEvent`, be sure to return `true` from your event handler so that the default form handler does not also redraw the whole form.

If you do handle the `frmUpdateEvent` in a custom event handler, be sure to handle the case where `updateCode` is set to `frmRedrawUpdateCode`, and redraw the whole form. This event (and code) is sent by the system when the whole form needs to be redrawn because the display needs to be refreshed.

FrmUpdateScrollers

Purpose Visually update (show or hide) the field scroll arrow buttons.

Prototype

```
void FrmUpdateScrollers (FormType *formP,
UInt16 upIndex, UInt16 downIndex,
Boolean scrollableUp, Boolean scrollableDown)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).
-> `upIndex`	Index of the up-scroller button. You can obtain this by using `FrmGetObjectIndex`.
-> `downIndex`	Index of the down-scroller button. You can obtain this by using `FrmGetObjectIndex`.
-> `scrollableUp`	Set to `true` to make the up scroll arrow active (shown), or `false` to hide it.
-> `scrollableDown`	Set to `true` to make the down scroll arrow active (shown), or `false` to hide it.

Result Returns nothing.

FrmValidatePtr

Purpose Return `true` if the specified pointer references a valid form.

Prototype

```
Boolean FrmValidatePtr (const FormType *formP)
```

Parameters

-> `formP`	Pointer to be tested.

Result Returns `true` if the specified pointer is a non-`NULL` pointer to an object having a valid form structure.

Comments This function is intended for debugging purposes only. Do not include it in released code.

To distinguish between a window and a form in released code, instead of using this function, look at the flag `windowFlags.dialog` in the `WindowType` structure. This flag is `true` if the window is a form.

Compatibility Implemented only if 3.0 New Feature Set is present.

FrmVisible

Purpose Return `true` if the form is visible (is drawn).

Prototype
```
Boolean FrmVisible (const FormType *formP)
```

Parameters

-> `formP`	Pointer to the form object (`FormType` structure).

Result Returns `true` if the form is visible; `false` if it is not visible.

See Also `FrmDrawForm`, `FrmEraseForm`

Application-Defined Functions

FormCheckResponseFunc

Purpose Callback function for `FrmCustomResponseAlert`.

Prototype
```
Boolean FormCheckResponseFuncType (Int16 button,
Char *attempt)
```

Parameters

-> `button`	The ID of the button that the user tapped.
-> `attempt`	The string that the user entered in the alert dialog.

Result Return `true` if the dialog should be dismissed. Return `false` if the dialog should not be dismissed.

Comments This function is called at these times during the `FrmCustomResponseAlert` routine:

- At the beginning of `FrmCustomResponseAlert`, this function is called with a button ID of `frmResponseCreate`. This constant indicates that the dialog is about to be displayed, and your function should perform any necessary initialization. For example, on a Japanese system, a password dialog might need to disable the Japanese FEP. So it would call `TsmSetFepMode(NULL, tsmFepModeOff)` in this function.
- When the user has tapped a button on the dialog. The function should process the `attempt` string. If the string is valid input, the function should return `true`. If not, it should return `false` to give the user a chance to re-enter the string.
- At the end of `FrmCustomResponseAlert`, this function is called with a button ID of `frmResponseQuit`. This gives the callback a change to perform any cleanup, such as re-enabling the Japanese FEP.

Compatibility Implemented only if 3.5 New Feature Set is present.

FormEventHandler

Purpose The event handler callback routine for a form.

Prototype
```
Boolean FormEventHandlerType (EventType *eventP)
```

Parameters

-> `eventP`	Pointer to the form event (`FormType` structure).

Result Must return `true` if this routine handled the event, otherwise `false`.

Comments `FrmDispatchEvent` calls this handler whenever it receives an event for the form.

This callback routine is the mechanism for dispatching events to particular forms in an application. The callback is registered by the routine `FrmSetEventHandler`.

FormGadgetHandler

Purpose The event handler callback for an extended gadget.

Prototype

```
Boolean (FormGadgetHandlerType)
(struct FormGadgetType *gadgetP, UInt16 cmd,
void *paramP)
```

Parameters

-> `gadgetP` Pointer to the gadget structure. See `FormGadgetType`.

-> `cmd` A constant that specifies what action the handler should take. This can be one of the following:

`formGadgetDeleteCmd`
: Sent by `FrmDeleteForm` to indicate that the gadget is being deleted and must clean up any memory it has allocated or perform other cleanup tasks.

`formGadgetDrawCmd`
: Sent by `FrmDrawForm` and `FrmShowObject` to indicate that the gadget must be drawn or redrawn.

`formGadgetEraseCmd`
: Sent by `FrmHideObject` to indicate that the gadget is going to be erased. `FrmHideObject` clears the `visible` and `usable` flags for you. If you return `false`, it also calls `WinEraseRectangle` to erase the gadget's bounds.

`formGadgetHandleEventCmd`
: Sent by `FrmHandleEvent` to indicate that a gadget event has been received. The `paramP` parameter contains the pointer to the `EventType` structure.

-> `paramP`	`NULL` except if `cmd` is `formGadgetHandleEventCmd`. In that case, this parameter holds the pointer to the `EventType` structure containing the event.

Result Return `true` if the event was handed successfully; `false` otherwise.

Comments If this function performs any drawing in response to the `formGadgetDrawCmd`, it should set the gadget's `visible` attribute flag. (`gadgetP->attr.visible = true`). This flag indicates that the gadget appears on the screen. If you don't set the `visible` flag, the gadget won't be erased when `FrmHideObject` is called. (`FrmHideObject` immediately returns if the object's `visible` flag is `false`.)

Note that if the function receives the `formGadgetEraseCmd`, it may simply choose to perform any necessary cleanup and return `false`. If the function returns `false`, `FrmHideObject` erases the gadget's bounding rectangle. If the function returns `true`, it must erase the gadget area itself.

If this function receives a `formGadgetHandleEventCmd`, `paramP` points one of two events: `frmGadgetEnterEvent` or `frmGadgetMiscEvent`. The `frmGadgetEnterEvent` is passed when there is a `penDownEvent` within the gadget's bounds. This function should track the pen and perform any necessary highlighting. The `frmGadgetMiscEvent` is never sent by the system. Your application may choose to use it if at any point it needs to send data to the extended gadget. In this case, the event has one or both of these fields defined: `selector`, an unsigned integer, and `dataP`, a pointer to data.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `FrmSetGadgetHandler`

12

Graffiti Shift

This chapter provides reference material for the Graffiti® Shift facility, declared in the header file `GraffitiShift.h`.

GraffitiShift Functions

GsiEnable

Purpose Enable or disable the Graffiti-shift state indicator.

Prototype
```
void GsiEnable (const Boolean enableIt)
```

Parameters

`enableIt`	`true` to enable, `false` to disable.

Result Returns nothing.

Comments Enabling the indicator makes it visible, disabling it makes the insertion point invisible.

GsiEnabled

Purpose Return `true` if the Graffiti-shift state indicator is enabled, or `false` if it's disabled.

Prototype
```
Boolean GsiEnabled (void)
```

Parameters None.

Result `true` if enabled, `false` if not.

GsiInitialize

Purpose Initialize the global variables used to manage the Graffiti-shift state indicator.

Prototype
```
void GsiInitialize (void)
```

Parameters None.

Result Returns nothing.

GsiSetLocation

Purpose Set the display-relative position of the Graffiti-shift state indicator.

Prototype
```
void GsiSetLocation (const Int16 x, const Int16 y)
```

Parameters

`x, y`	Coordinate of left side and top of the indicator.

Result Returns nothing.

Comments The indicator is not redrawn by this routine.

GsiSetShiftState

Purpose Set the Graffiti-shift state indicator.

Prototype
```
void GsiSetShiftState (const UInt16 lockFlags,
const UInt16 tempShift)
```

Parameters

`lockFlags`	`glfCapsLock` or `glfNumLock`.
`tempShift`	The current temporary shift.

Result Returns nothing.

Comments This function affects only the state of the UI element, not the underlying Graffiti engine.

See Also GrfSetState

13

Insertion Point

This chapter provides reference material for the insertion point API, declared in the header file `InsPoint.h`.

For more information on the insertion point, see the section "Insertion Point" in the *Palm OS Programmer's Companion*, vol. I.

Insertion Point Functions

InsPtEnable

Purpose Enable or disable the insertion point. When the insertion point is disabled, it's invisible; when it's enabled, it blinks.

Prototype

```
void InsPtEnable (Boolean enableIt)
```

Parameters

`enableIt`	`true` = enable; `false` = disable

Result Returns nothing.

Comments This function is called by the Form functions when a text field loses or gains the focus, and by the Windows function when a region of the display is copied (`WinCopyRectangle`).

See Also `InsPtEnabled`

InsPtEnabled

Purpose Return `true` if the insertion point is enabled or `false` if the insertion point is disabled.

Prototype
```
Boolean InsPtEnabled (void)
```

Parameters None.

Result Returns `true` if the insertion point is enabled (blinking); returns `false` if the insertion point is disabled (invisible).

See Also `InsPtEnable`

InsPtGetHeight

Purpose Return the height of the insertion point.

Prototype
```
Int16 InsPtGetHeight (void)
```

Parameters None.

Result Returns the height of the insertion point, in pixels.

InsPtGetLocation

Purpose Return the screen-relative position of the insertion point.

Prototype
```
void InsPtGetLocation (Int16 *x, Int16 *y)
```

Parameters

`x, y`	Pointer to top-left position of insertion point's x and y coordinate.

Result Returns nothing. Stores the location in `x` and `y`.

Comments This function is called by the Field functions. An application would not normally call this function.

InsPtSetHeight

Purpose Set the height of the insertion point.

Prototype
```
void InsPtSetHeight (const Int16 height)
```

Parameters

`height`	Height of the insertion point in pixels.

Result Returns nothing.

Comments Set the height of the insertion point to match the character height of the font used in the field that the insertion point is in. When the current font is changed, the insertion point height should be set to the line height of the new font.

If the insertion point is visible when its height is changed, it's erased and redrawn with its new height.

See Also `InsPtGetHeight`

InsPtSetLocation

Purpose Set the screen-relative position of the insertion point.

Prototype
```
void InsPtSetLocation (const Int16 x,
const Int16 y)
```

Parameters

`x, y`	Number of pixels from the left side (top) of the display.

Result Returns nothing.

Comments The position passed to this function is the location of the top-left corner of the insertion point.

This function should be called only by the Field functions.

See Also `InsPtGetLocation`

14

Lists

This chapter provides information about list objects by discussing these topics:

- List Data Structures
- List Resources
- List Functions
- Application-Defined Function

The header file `List.h` declares the API that this chapter describes. For more information on lists, see the section "Lists" in the *Palm OS Programmer's Companion*, vol. I.

List Data Structures

ListAttrType

The `ListAttrType` bit field defines the visible characteristics of the list.

```
typedef struct {
  UInt16 usable       :1;
  UInt16 enabled      :1;
  UInt16 visible      :1;
  UInt16 poppedUp     :1;
  UInt16 hasScrollBar :1.
  UInt16 search       :1;
  UInt16 reserved      :2;
} ListAttrType;
```

Field Descriptions

`usable`	Set to make the list usable. If not set, the list is not considered part of the current interface of the application, and does not appear on screen.
`enabled`	Not used.
`visible`	Set when the list object is drawn, and cleared when the list object is erased. This attribute is set and cleared internally.
`poppedUp`	Set to indicate that the choices are displayed in a popup window. This attribute is set and cleared internally.
`hasScrollBar`	Set to indicate that the list has a scroll bar.
`search`	Set to enable incremental search. If incremental search is enabled, when the list is displayed the user can navigate the list by entering up to five characters. The list will scroll to present the first list item that matches the entered characters. This feature only works for popup lists, and only works if the list is sorted and the list items are available to the List Manager (that is, you don't pass `NULL` to `LstSetListChoices`).
`reserved`	Reserved for system use.

ListType

The `ListType` structure is defined below.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `ListType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16                id;
  RectangleType         bounds;
  ListAttrType          attr;
  Char                  **itemsText;
  Int16                 numItems;
  Int16                 currentItem;
  Int16                 topItem;
  FontID                font;
  UInt8                 reserved;
  WinHandle             popupWin;
  ListDrawDataFuncPtr   drawItemCallback;
} ListType;
```

Field Descriptions

`id` — The ID value, specified by the application developer. This ID value is part of the event data of `lstEnterEvent` and `lstSelectEvent`.

`bounds` — The bounds of the list, relative to the window. For example, to access the bounds of an object in a form whose ID is kObjectID:

```
{
RectangleType rect;
FormPtr formP = FrmGetActiveForm();

FrmGetObjectBounds(formP,
FrmGetObjectIndex(formP, kObjectID),
&rect);
}
```

`attr` — The list attributes. See `ListAttrType`.

`itemsText` A pointer to an array of pointers to the text of the choices. Access with `LstGetSelectionText`.
For example, to access the string specified by `itemNum` in the list whose ID is `kChoiceList` use the following:

```
{
Char *string;
Int16 itemNum;
...
string =
LstGetSelectionText(GetObjectPtr(kChoic
esList), itemNum);
}
```

where `GetObjectPtr` is the following:

```
static void *GetObjectPtr(UInt16
rsrcID){
FormPtr formP;

formP = FrmGetActiveForm();
return FrmGetObjectPtr(formP,
FrmGetObjectIndex(formP, rsrcID));
}
```

If you use a callback routine to draw the list items, note that the `itemsText` pointer you supply to `LstSetListChoices` is passed to your callback routine. See the comments under `ListDrawDataFuncType` for tips on using `itemsText` with a callback routine.

`numItems` The number of choices in the list. Access with `LstGetNumberOfItems`.

`currentItem` The currently-selected list choice (0 = first choice). Access with `LstGetSelection`.

`topItem` The first choice displayed in the list. Set this field with `LstSetTopItem`.

`font` The ID of the font used to draw all list text strings.

`reserved`	Reserved for future use.
`popupWin`	The handle of the window created when a list is displayed by calling `LstPopupList`.
`drawItemCallback`	Function used to draw an item in the list. If `NULL`, the default drawing routine is used instead. Set this field with `LstSetDrawFunction`. See Application-Defined Function.

List Resources

The List Resource (tLST), and Popup Trigger Resource (tPUT) are used together to represent an active list.

List Functions

LstDrawList

Purpose Sets the visible attribute of the list object to `true`, and draws the list object if it is usable.

Prototype

```
void LstDrawList (ListType *listP)
```

Parameters

`-> listP`	Pointer to a list object (ListType).

Result Returns nothing.

Comments If there are more choices than can be displayed, this function ensures that the current selection is visible. The current selection is highlighted. Note that this function does not ensure the current selection is visible; if you need to do this, call the `LstMakeItemVisible` function.

If the list is disabled, it's drawn grayed-out (strongly discouraged). If it's empty, nothing is drawn. If it's not usable, nothing is drawn.

See Also FrmGetObjectPtr, LstPopupList, LstEraseList

LstEraseList

Purpose Erase a list object.

Prototype
```
void LstEraseList (ListType *listP)
```

Parameters

`-> listP`	Pointer to a list object (ListType).

Result Returns nothing.

Comments The `visible` attribute is set to `false` by this function.

See Also FrmGetObjectPtr, LstDrawList

LstGetNumberOfItems

Purpose Return the number of items in a list.

Prototype
```
Int16 LstGetNumberOfItems (const ListType *listP)
```

Parameters

`-> listP`	Pointer to a list object (ListType).

Result Returns the number of items in a list.

See Also FrmGetObjectPtr, LstSetListChoices

LstGetSelection

Purpose Return the currently selected choice in the list.

Prototype
```
Int16 LstGetSelection (const ListType *listP)
```

Parameters

`-> listP`	Pointer to a list object.

Result Returns the item number of the current list choice. The list choices are numbered sequentially, starting with 0; Returns `noListSelection` if none of the items are selected.

See Also `FrmGetObjectPtr`, `LstSetListChoices`, `LstSetSelection`, `LstGetSelectionText`

LstGetSelectionText

Purpose Return a pointer to the text of the specified item in the list, or `NULL` if no such item exists.

Prototype
```
Char *LstGetSelectionText (const ListType *listP,
Int16 itemNum)
```

Parameters

`-> listP`	Pointer to a list object.
`-> itemNum`	Item to select (0 = first item in list).

Result Returns a pointer to the text of the current selection, or `NULL` if out of bounds.

Comments This is a pointer within ListType, not a copy. This function is only usable if you supplied an array of strings and a count to `LstSetListChoices`; if your application uses a callback function that dynamically generates the list text, this function returns `NULL`.

See Also `FrmGetObjectPtr`

LstGetTopItem

Purpose Returns the topmost visible item.

Prototype
```
Int16 LstGetTopItem (const ListType *listP)
```

Parameters

`-> listP`	Pointer to a list object.

Result Returns the item number of the top item visible.

See Also `LstGetSelection`, `LstSetTopItem`

LstGetVisibleItems

Purpose Return the number of visible items.

Prototype
```
Int16 LstGetVisibleItems (const ListType *listP)
```

Parameters

`-> listP`	Pointer to a list object.

Result The number of items visible.

Compatibility Implemented only if 2.0 New Feature Set is present.

LstHandleEvent

Purpose Handle event in the specified list; the list object must have its `usable` and `visible` attribute set to `true`.This routine handles two type of events, `penDownEvent` and `lstEnterEvent`; see Comments.

Prototype
```
Boolean LstHandleEvent (ListType *listP,
const EventType *eventP)
```

Parameters

`-> listP`	Pointer to a list object (ListType).
`-> eventP`	Pointer to an `EventType` structure.

Result Return `true` if the event was handled. The following cases will result in a return value of `true`:

- A `penDownEvent` within the bounds of the list
- A `lstEnterEvent` with a list ID value that matches the list ID in the list data structure

Comments When this routine receives a `penDownEvent`, it checks if the pen position is within the bounds of the list object. If it is, this routine tracks the pen until the pen comes up. If the pen comes up within the bounds of the list, a `lstEnterEvent` is added to the event queue, and the routine is exited.

When this routine receives a `lstEnterEvent`, it checks that the list ID in the event record matches the ID of the specified list. If there is a match, this routine creates and displays a popup window containing the list's choices and the routine is exited.

If a `penDownEvent` is received while the list's popup window is displayed and the pen position is outside the bounds of the popup window, the window is dismissed. If the pen position is within the bounds of the window, this routine tracks the pen until it comes up. If the pen comes up outside the list object, a `lstEnterEvent` is added to the event queue.

LstMakeItemVisible

Purpose Make an item visible, preferably at the top. If the item is already visible, make no changes.

Prototype

```
void LstMakeItemVisible (ListType *listP,
Int16 itemNum)
```

Parameters

`-> listP`	Pointer to a list object (ListType).
`-> itemNum`	Item to select (0 = first item in list).

Result Returns nothing.

Comments Does ***not*** visually update the list. You must call `LstDrawList` to update it.

See Also `FrmGetObjectPtr`, `LstSetSelection`, `LstSetTopItem`, `LstDrawList`

LstNewList

Purpose Create a new list object dynamically and install it in the specified form.

Prototype

```
Err LstNewList (void **formPP, UInt16 id, Coord x,
Coord y, Coord width, Coord height, FontID font,
Int16 visibleItems, Int16 triggerId)
```

Parameters

<--> `formPP`	Pointer to the pointer to the form in which the new list is installed. This value is not a handle; that is, the old `formPP` value is not necessarily valid after this function returns. In subsequent calls, always use the new `formPP` value returned by this function.
`-> id`	Symbolic ID of the list, specified by the developer. By convention, this ID should match the resource ID (not mandatory).
`-> x`	Horizontal coordinate of the upper-left corner of the list's boundaries, relative to the window in which it appears.
`-> y`	Vertical coordinate of the upper-left corner of the list's boundaries, relative to the window in which it appears.
`-> width`	Width of the list, expressed in pixels. Valid values are 1 – 160.
`-> height`	Height of the list, expressed in pixels.Valid values are 1 – 160.
`-> visibleItems`	Number of list items that can be viewed together.

`-> triggerId`	Symbolic ID of the popup trigger associated with the new list. This ID is specified by the developer; by convention, this ID should match the resource ID (not mandatory).

Result Returns 0 if no error.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `LstDrawList`, `FrmRemoveObject`

LstPopupList

Purpose Display a modal window that contains the items in the list.

Prototype

```
Int16 LstPopupList (ListType *listP)
```

Parameters

`-> listP`	Pointer to a list object.

Result Returns the list item selected, or -1 if no item was selected.

Comments Saves the previously active window. Creates and deletes the new popup window.

See Also `FrmGetObjectPtr`

LstScrollList

Purpose Scroll the list up or down a number of times.

Prototype

```
Boolean LstScrollList (ListType *listP,
WinDirectionType direction, Int16 itemCount)
```

Parameters

`-> listP`	Pointer to a list object.
`-> direction`	Direction to scroll.
`-> itemCount`	Items to scroll in direction.

Result Returns `true` when the list is actually scrolled, `false` otherwise. May return `false` if a scroll past the end of the list is requested.

Compatibility Implemented only if 2.0 New Feature Set is present.

LstSetDrawFunction

Purpose Set a callback function to draw each item instead of drawing the item's text string.

Prototype

```
void LstSetDrawFunction (ListType *listP,
ListDrawDataFuncPtr func)
```

Parameters

`-> listP` Pointer to a list object.

`-> func` Pointer to a function that draws items.

Result Returns nothing.

Comments This function also adjusts `topItem` to prevent a shrunken list from being scrolled down too far. Use this function for custom draw functionality.

See Also `FrmGetObjectPtr`, `LstSetListChoices`, `ListDrawDataFuncType`

LstSetHeight

Purpose Set the number of items visible in a list.

Prototype

```
void LstSetHeight (ListType *listP,
Int16 visibleItems)
```

Parameters

`-> listP` Pointer to a list object.

`-> visibleItems` Number of choices visible at once.

Result Returns nothing.

Comments This function doesn't redraw the list if it's already visible.

See Also FrmGetObjectPtr

LstSetListChoices

Purpose Set the items of a list to the array of text string pointers passed to this function. This functions erases the old list items.

Prototype

```
void LstSetListChoices (ListType *listP,
Char **itemsText, Int16 numItems)
```

Parameters

`-> listP`	Pointer to a list object.
`-> itemsText`	Pointer to an array of text strings.
`-> numItems`	Number of choices in the list.

Result Returns nothing.

Comments You need to call the `LstDrawList` function to update the list if it is displayed when you call this function.

NOTE: This function does not copy the strings in the `itemsText` array, which means that you need to ensure that the array is not moved or deallocated until after you are done with the list.

If you use a callback routine to draw the items in your list, the `itemsText` pointer is simply passed to that callback routine and is not otherwise used by the List Manager code. See the comments under `ListDrawDataFuncType` for tips on using the `itemsText` parameter with a callback routine.

See Also FrmGetObjectPtr, LstSetSelection, LstSetTopItem, LstDrawList, LstSetHeight, LstSetDrawFunction

LstSetPosition

Purpose Set the position of a list.

Prototype
```
void LstSetPosition (ListType *listP, Coord x,
Coord y)
```

Parameters

`-> listP`	Pointer to a list object
-> x	Left bound.
`-> y`	Top bound.

Result Returns nothing.

Comments The list is not redrawn. Don't call this function when the list is visible.

See Also FrmGetObjectPtr

LstSetSelection

Purpose Set the selection for a list.

Prototype
```
void LstSetSelection (ListType *listP,
Int16 itemNum)
```

Parameters

`-> listP`	Pointer to a list object.
`-> itemNum`	Item to select (0 = first item in list; -1 = none).

Result Returns nothing.

Comments The old selection, if any, is unselected. If the list is visible, the selected item is visually updated. The list is scrolled to the selection, if necessary, as long as the list object is both visible and usable.

See Also FrmGetObjectPtr , LstSetTopItem

LstSetTopItem

Purpose Set the item visible. The item cannot become the top item if it's on the last page.

Prototype
```
void LstSetTopItem (ListType *listP,
Int16 itemNum)
```

Parameters

`-> listP`	Pointer to a list object.
`-> itemNum`	Item to select (0 = first item in list). This must be a valid item number.

Result Returns nothing.

Comments Does *not* update the display.

NOTE: The value you specify for itemNum must be in the range `0` to max-number-of-items.

See Also `FrmGetObjectPtr`, `LstSetSelection`, `LstGetTopItem`, `LstMakeItemVisible`, `LstDrawList`, `LstEraseList`

Application-Defined Function

If you need to perform special drawing for items in the list, call `LstSetDrawFunction` to set the list drawing callback function. The `ListDrawDataFuncType` section documents the prototype for the callback function you provide for drawing list items.

ListDrawDataFuncType

Purpose Callback function that you provide for drawing items in a list. This function is called whenever the Palm OS needs to draw an element in the list. Your callback function declaration must match the prototype shown here.

Prototype

```
void ListDrawDataFuncType(Int16 itemNum,
RectangleType *bounds, Char **itemsText)
```

Parameters

`-> itemNum`	The number of the item to draw.
`-> bounds`	The bounds of the list, relative to the window.
`-> itemsText`	A pointer to an array of pointers to the text of the list items. This is the pointer that you supplied when calling `LstSetListChoices`.

Result

Returns nothing.

Comments

You can call `LstSetDrawFunction` to register your callback function for the list, which means that your function will be called to draw the list items, rather than using the built-in draw functionality, which displays each item's text string.

Your callback function is called whenever an item in the list needs to be drawn. When it is called, the value of the `itemNum` parameter specifies which item the function is to draw. The `itemsText` parameter, which is what was supplied to `LstSetListChoices`, doesn't actually need to point to a list of strings: you can pass `NULL`, or you can pass a pointer to anything that will be useful to your drawing function. Note, however, that if you pass anything other than a pointer to a list of strings when you call `LstSetListChoices`, you must ensure that `LstGetSelectionText` is never called since it assumes that this pointer indicates an array of text items. In particular, if your list is associated with a pop-up trigger you must handle the `popSelectEvent` yourself before `FrmHandleEvent` gets a chance at it since `FrmHandleEvent` calls `LstGetSelectionText`.

WARNING! If the list is a popup list, the form that owns the list is not active while the list is in a window. This means that you cannot call the `FrmGetActiveForm` function. Instead, use itemsText pointer to access any information that you need for drawing. If you must access the form, use the `FrmGetFormPtr` function.

Note that the list object manages which colors are used to draw the items and how to draw selected versus unselected items. In almost all circumstances, your drawing function does not have to be concerned with these details.

However, if you do need to determine if the item is selected, you can rely on the fact that the system has set the pen color to one of two colors prior to calling your draw function:

- If the item is selected, the foreground color is `UIObjectSelectedForeground`.
- If the item is not selected, the foreground color is `UIObjectForeground`.

You can determine the foreground color that is in effect for the list item by calling the `WinSetForeColor` function, which returns the previous value of the foreground color. Remember to call `WinSetForeColor` again to reset the foreground color to what it was. For example:

```
itemColor = WinSetForeColor(myColor)
WinSetForeColor(itemColor)
selectColor = UiColorGetTableEntryIndex(
                    UIObjectSelectedForeground)
if itemColor == selectColor
...
```

See Also `LstSetDrawFunction`, `UIColorGetTableEntryIndex`, `WinSetForeColor`

15

Menus

This chapter describes the menu API as declared in the header file `Menu.h`. It discusses the following topics:

- Menu Data Structures
- Menu Constants
- Menu Resources
- Menu Functions

For more information on menus, see the section "Menus" on page 101 in the *Palm OS Programmer's Companion*, vol. I.

Menu Data Structures

MenuBarAttrType

The `MenuBarAttrType` bit field defines some characteristics of the menu bar.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `MenuBarAttrType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16 visible : 1;
  UInt16 commandPending : 1;
  UInt16 insPtEnabled : 1;
  UInt16 needsRecalc : 1;
} MenuBarAttrType;
```

Your code should treat the `MenuBarAttrType` structure as opaque. Use the functions specified in the descriptions below to retrieve and set each value. Do not attempt to change structure member values directly.

Field Descriptions

`visible`	If set, the menu bar is drawn and visible on the screen. This attribute is set as part of `MenuDrawMenu`, which is called when the menu is drawn.
`commandPending`	If set, a menu command shortcut is in progress. This bit is set during `MenuHandleEvent` if the menu shortcut keystroke is received. If the next key is received before the timeout value is reached, the key is examined to see if it is a valid menu command.
`insPtEnabled`	Stores the state of the insertion point at the time the menu was drawn so that it can be restored when the menu is erased.
`needsRecalc`	If set, recalculate menu dimensions.

Compatibility The `needsRecalc` constant is present only if 3.5 New Feature Set is present.

MenuCmdBarButtonType

The `MenuCmdBarButtonType` structure defines a button to be displayed on the command toolbar. The `buttonsData` field of the `MenuCmdBarType` structure contains an array of structures of this type.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `MenuCmdBarButtonType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16   bitmapId;
  Char     name[menuCmdBarMaxTextLength];
  MenuCmdBarResultType resultType;
  UInt8    reserved;
  UInt32   result;
} MenuCmdBarButtonType;
```

Your code should treat the `MenuCmdBarButtonType` structure as opaque. Do not attempt to change structure member values directly. Instead, use `MenuCmdBarAddButton` to add a button to the display. For the most part, the parameters to `MenuCmdBarAddButton` are the same as the fields in the `MenuCmdBarButtonType`, so there should be no need to alter these fields directly.

`MenuCmdBarGetButtonData` can be called to access information about command bar buttons.

Field Descriptions

`bitmapId`	Resource ID of the bitmap to display on the button. This bitmap should be 13 pixels high by 16 pixels wide.
`name`	Text to display in the status message when the user taps the button.
`resultType`	Specifies what type of data is contained in the `result` field. See `MenuCmdBarResultType`.
`reserved`	Reserved for future use.
`result`	Specifies the data to send when the user clicks the button. The data is interpreted as specified by the `resultType` field. The result can be a shortcut character to enqueue in a `keyDownEvent`, a menu item ID to enqueue in a `menuEvent`, or a notification to be broadcast.

Compatibility This structure is defined only if 3.5 New Feature Set is present.

MenuCmdBarResultType

The `MenuCmdBarResultType` enum specifies how the `result` field in the `MenuCmdBarButtonType` structure should be interpreted.

```
typedef enum {
  menuCmdBarResultNone,
  menuCmdBarResultChar,
  menuCmdBarResultMenuItem,
  menuCmdBarResultNotify
} MenuCmdBarResultType;
```

Value Descriptions

`menuCmdBarResultNone`	Send nothing.
`menuCmdBarResultChar`	The result is a character to send in a `keyDownEvent`.
`menuCmdBarResultMenuItem`	The result is the ID of the menu item to send in a `menuEvent`.
`menuCmdBarResultNotify`	The result is a notification constant to be broadcast using `SysNotifyBroadcastDeferred`

Compatibility This enum is defined only if 3.5 New Feature Set is present.

MenuCmdBarType

The `MenuCmdBarType` structure defines the command toolbar. This command toolbar is allocated and displayed when the user draws the shortcut stroke in the Graffiti® area. It is deallocated when `MenuEraseStatus` is called, which occurs most frequently when the timeout value has elapsed.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `MenuCmdBarType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct MenuCmdBarType {
  WinHandle    bitsBehind;
  Int32        timeoutTick;
  Coord        top;
  Int16        numButtons;
  Boolean      insPtWasEnabled;
  Boolean      gsiWasEnabled;
  Boolean      feedbackMode;
  MenuCmdBarButtonType *buttonsData;
} MenuCmdBarType;
```

Your code should treat the `MenuCmdBarType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`bitsBehind`	Handle for the window that contains the region obscured by the command toolbar.
`timeoutTick`	Timeout value given in system ticks. If the user hasn't specified a command after this many ticks, the command toolbar is erased from the screen and deallocated from memory. This value also specifies how long the status message is displayed after the user successfully enters a command.
`top`	The top bound of the command toolbar given in display-relative coordinates. The command toolbar is as wide as the screen and displays at the bottom of the screen.
`numButtons`	Number of buttons displayed on the command toolbar.
`insPtWasEnabled`	If `true`, the insertion point was enabled before the command toolbar was displayed and should be re-enabled when the command toolbar is erased. If `false`, the insertion point was disabled.

`gsiWasEnabled`	If `true`, the Graffiti shift indicator was enabled before the command toolbar was displayed and should be re-enabled when the command toolbar is erased. If `false`, the Graffiti shift indicator was disabled.
`feedbackMode`	If `true`, the command toolbar is currently displaying a status message. The status message is displayed to tell the user what command is being performed. If `false`, the command toolbar is awaiting input.
`buttonsData`	The list of buttons to display on the command toolbar. See `MenuCmdBarButtonType`. Buttons are stored in this list sequentially with the rightmost button at index 0. Access with `MenuCmdBarGetButtonData`.

Compatibility This structure is defined only if 3.5 New Feature Set is present.

MenuBarPtr

The `MenuBarPtr` type defines a pointer to a `MenuBarType`.

```
typedef MenuBarType *MenuBarPtr;
```

MenuBarType

The `MenuBarType` structure defines the menu bar. There is one menu bar per form.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `MenuBarType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  WinHandle         barWin;
  WinHandle         bitsBehind;
  WinHandle         savedActiveWin;
```

```
    WinHandle         bitsBehindStatus;
    MenuBarAttrType   attr;
    Int16             curMenu;
    Int16             curItem;
    Int32             commandTick;
    Int16             numMenus;
    MenuPullDownPtr   menus;
  } MenuBarType;
```

Your code should treat the `MenuBarType` structure as opaque. Do not attempt to change structure member values directly.

Field Descriptions

`barWin`	Handle for the window that contains the menu bar.
`bitsBehind`	Handle for the window that contains the region obscured by the menu bar.
`savedActiveWin`	Handle where the currently active window is saved so that it can be restored when the menu is erased.
`bitsBehindStatus`	Handle where the bits behind the status message are saved so that when the message display terminates, the bits can be restored. The status message is displayed when the user activates the menu through the use of the command keystroke.
`attr`	Menu bar attributes. See `MenuBarAttrType`.
`curMenu`	Menu number for the currently visible menu. Menus are numbered sequentially starting with 0. The value is preserved when the menu bar is dismissed. A value of `noMenuSelection` indicates that there is no current pull-down menu.

`curItem`	Item number of the currently highlighted menu item. The items in each menu are numbered sequentially, starting with zero. A value of `noMenuItemSelection` indicates that there is no current item selected.
`commandTick`	Tick count at which the status message should be erased.
`numMenus`	Number of pull-down menus on the menu bar.
`menus`	Array of `MenuPullDownType` structures.

Compatibility

If 3.5 New Feature Set is present, the `bitsBehindStatus` and `commandTick` fields are defined but are not used. Instead, the `bitsBehind` and `timeoutTick` fields in `MenuCmdBarType` define the save-behind window and the timeout value for the command toolbar.

MenuItemType

The `MenuItemType` structure defines a specific item within a menu. The `items` array in the `MenuPullDownType` structure contains one `MenuItemType` structure for each menu item in the pull-down menu.

If 3.5 New Feature Set is present, you can add a menu item to a pull-down menu programmatically using `MenuAddItem`.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `MenuItemType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16   id;
  Char     command;
  UInt8    hidden: 1;
  UInt8    reserved: 7;
  Char     *itemStr;
} MenuItemType;
```

Field Descriptions

id	ID value you specified when you created the menu item. This ID value is included as part of the event data of a `menuEvent`.
`command`	Shortcut key. If you provide shortcuts, make sure that each shortcut is unique among all commands available at that time.
`hidden`	If `true`, the menu item is hidden. If `false`, it is displayed. You can set and clear this value using `MenuHideItem` and `MenuShowItem`.
`reserved`	Reserved for future use.
`itemStr`	Pointer to the text to display for this menu item, including the shortcut key. To include a shortcut key, begin the string with the item's text, then type a tab character, and then the item's shortcut key. To create a separator bar, create a one-character string containing the `MenuSeparatorChar` constant.

Compatibility The `hidden` and `reserved` fields are defined only if 3.5 New Feature Set is present.

MenuPullDownPtr

The `MenuPullDownPtr` type defines a pointer to a `MenuPullDownType`.

```
typedef MenuPullDownType *MenuPullDownPtr;
```

MenuPullDownType

The `MenuPullDownType` structure defines a specific menu accessed from the menu bar. The menus array in the `MenuBarType` structure contains one `MenuPullDownType` structure for each pull-down menu associated with the menu bar.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `MenuPullDownType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  WinHandle         menuWin;
  RectangleType     bounds;
  WinHandle         bitsBehind;
  RectangleType     titleBounds;
  Char              *title;
  UInt16            hidden : 1;
  UInt16            numItems : 15;
  MenuItemType      *items;
} MenuPullDownType;
```

Field Descriptions

`menuWin`	Handle for the window that contains the menu.
`bounds`	Position and size, in pixels, of the pull-down menu.
`bitsBehind`	Handle of a window that contains the region obscured by the menu.

`title`	The menu title (null-terminated string) displayed in the menu bar.
`titleBounds`	Position and size, in pixels, of the title in the menu bar.
`hidden`	If `true`, the menu is hidden; if `false`, it is displayed. This field is not currently used.
`numItems`	Number of items in the menu. Separators count as items.
`items`	Array of `MenuItemType` structures.

Compatibility The `hidden` field is defined only if 3.5 New Feature Set is present.

Menu Constants

Constant	Value	Description
`noMenuSelection`	-1	The `curMenu` field of `MenuBarType` is set to this when there is no currently selected menu.
`noMenuItemSelection`	-1	The `curItem` field of `MenuBarType` is set to this when there is no currently selected menu item.
`separatorItemSelection`	-2	The `curItem` field of `MenuBarType` is set to this when a menu separator item is selected.
`MenuSeparatorChar`	'–'	Special character indicating that the menu item is a bar used to separate groups of related menu items. The first character of the `itemStr` string in `MenuItemType` is set to this.

Menu Resources

The menu bar (MBAR) and pull-down menu (MENU) resources are used jointly to represent a menu object on screen. See "Menus and Menu Bars" in Chapter 2, "Palm OS Resources."

Menu Functions

MenuAddItem

Purpose Adds an item to the currently active menu.

Prototype

```
Err MenuAddItem (UInt16 positionId, UInt16 id,
Char cmd, const Char *textP)
```

Parameters

-> `positionId`	ID of an existing menu item. The new menu item is added after this menu item.
-> `id`	ID value to use for the new menu item.
-> `cmd`	Shortcut key. If you provide shortcuts, make sure that each shortcut is unique among all commands available at that time.
-> `textP`	Pointer to the text to display for this menu item, including the shortcut key. To include a shortcut key, begin the string with the item's text, then type a tab character, and then the item's shortcut key. To create a separator bar, create a one-character string containing the `MenuSeparatorChar` constant.

Result Returns 0 upon success or one of the following if an error occurs:

`menuErrNoMenu`	The `textP` parameter is `NULL`.
`menuErrSameId`	The menu already contains an item with the ID `id`.

`menuErrNotFound`
: The menu doesn't contain an item with the ID `positionId`.

May display a fatal error message if there is no current menu.

Comments

This function creates a new `MenuItemType` structure and adds it to the `MenuBarType`'s item list.

You should call this function only in response to a `menuOpenEvent`. This event is generated when the menu is first made active. In general, a form's menu becomes active the first time a `keyDownEvent` with a `vchrMenu` or `vchrCommand` is generated, and it remains active until a new form (including a modal form or alert panel) is displayed or until `FrmSetMenu` is called to change the form's menu. Palm OS® user interface guidelines discourage adding or hiding menu items at any time other than when the menu is first made active.

Compatibility

Implemented only if 3.5 New Feature Set is present.

MenuCmdBarAddButton

Purpose

Defines a button to be displayed on the command toolbar.

Prototype

```
Err MenuCmdBarAddButton (UInt8 where,
UInt16 bitmapId, MenuCmdBarResultType resultType,
UInt32 result, Char *nameP)
```

Parameters

-> `where`
: Either `menuCmdBarOnLeft` to add the button to the left of the other buttons on the command toolbar, `menuCmdBarOnRight` to add it to the right of the other buttons, or a number indicating the exact position of the button. Button positions are numbered from right to left, and the rightmost position is number 1.

-> `bitmapId`
: Resource ID of the bitmap to display on the button. The bitmap's dimensions should be 13 pixels high by 16 pixels wide.

-> `resultType`	The type of data contained in the `result` parameter. See `MenuCmdBarResultType`.
-> `result`	The data to send when the user taps this button. This can be a character, a menu item ID, or a notification constant.
-> `nameP`	Pointer to the text to display in the status message if the user taps the button. If `NULL`, the text is taken from the menu item that matches the ID or shortcut character contained in `result`, if a match is found. If you supply a text buffer for this parameter, `MenuCmdBarAddButton` makes a copy of the buffer.

Result Returns 0 upon success, or one of the following error codes:

`menuErrOutOfMemory`
There is not enough memory available to perform the operation.

`menuErrTooManyItems`
The command toolbar already has the maximum number of buttons allowed (currently 8).

Comments Call this function in response to a `menuCmdBarOpenEvent` or to the notification `sysNotifyMenuCmdBarOpenEvent`. Both of these signal that the user has entered the command keystroke and the command toolbar is about to open. Your response should be to add buttons to the toolbar and to return `false`, indicating that you have not completely handled the event.

The `sysNotifyMenuCmdBarOpenEvent` notification is intended to be used only by shared libraries, system extensions, and other code resources that do not use an event loop. If you're writing an application, always respond to the event instead of the notification; an application should only add buttons to the toolbar if it is the current application. If you register for the notification, you receive it each time the command toolbar is displayed, whether your application is active or not.

Note that the command toolbar is allocated each time it is opened and is deallocated when it is erased from the screen.

There is a limited amount of space in which to display buttons on the command toolbar. You should limit the number of buttons to four or five. The maximum allowed by the system is eight, but you should leave space for the status message that appears after the user chooses an action. Buttons should be contextual; for example, the field code only displays a paste button if there is text on the clipboard. Bitmaps for the buttons should be 16 X 13 pixels.

If a field has focus when the command toolbar is opened, the field manager adds buttons for cut, copy, paste, and undo. If your application does not want this default behavior, set the `preventFieldButtons` field in the `menuCmdBarOpenEvent` structure to `true`. (Note that there is no way to prevent the field buttons from being drawn from within a notification handler.)

The following bitmaps for command toolbar buttons are defined in `UIResources.h`. The system and the built-in applications use these bitmaps to represent the commands listed in the table. Your application should also use them if it performs the same actions. If you use any of these buttons, add them in the order shown from right to left. (For example, `BarDeleteBitmap`, if used, should always be the rightmost button.)

Bitmap	Command
`BarDeleteBitmap`	Delete record.
`BarPasteBitmap`	Paste clipboard contents at insertion point.
`BarCopyBitmap`	Copy selection.
`BarCutBitmap`	Cut selection.
`BarUndoBitmap`	Undo previous action.
`BarSecureBitmap`	Show Security dialog.
`BarBeamBitmap`	Beam current record.
`BarInfoBitmap`	Show Info dialog (Launcher).

It is best to add buttons on the left side. If you add buttons to the right, this function moves all existing buttons over one position to the left. You can also specify an exact position for the `where` parameter. The positions are numbered from right to left with the rightmost position being 1. If you specify an exact position, the function leaves space for the other buttons. For example, if you specify position 3 and there are no buttons displayed at positions 1 and 2, there will be blank spots to the right of your button.

The `result` and `resultType` parameters specify what the result should be if the user taps the button. `result` contains the actual data, and `resultType` contains a constant that specifies the type of data in `result`. Typically, the result is to enqueue a `menuEvent`. In this case, `resultType` is `menuCmdBarResultMenuItem` and the `result` is the ID of the menu item that should included in the event.

You may also specify the shortcut character instead of the menu ID; however, doing so is inefficient. When `result` is a shortcut character, the `MenuHandleEvent` function enqueues a `keyDownEvent` with the character in `result`. During the next cycle of the event loop, `MenuHandleEvent` enqueues a `menuEvent` in response to the `keyDownEvent`. Thus, it is better to have your button enqueue the `menuEvent` directly.

If you call `MenuCmdBarAddButton` outside of an application, you might not know of any menu items in the active menu (unless your code has added one using `MenuAddItem`). In this case, specify a notification to be broadcast. The notification is broadcast at the top of the next event loop, and it must contain no custom data. (Applications may also use the notification result type.)

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `MenuCmdBarDisplay`, `MenuCmdBarGetButtonData`

MenuCmdBarDisplay

Purpose Displays the command toolbar.

Prototype
```
void MenuCmdBarDisplay (void)
```

Parameters None

Result Returns nothing.

Comments This function displays the command toolbar when the user enters the command keystroke. You normally do not call this function in your own code. The form manager calls it at the end of its handling of `menuCmdBarOpenEvent`.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `MenuCmdBarAddButton`, `MenuCmdBarGetButtonData`

MenuCmdBarGetButtonData

Purpose Gets the data for a given command button.

Prototype
```
Boolean MenuCmdBarGetButtonData
(Int16 buttonIndex, UInt16 *bitmapIdP,
MenuCmdBarResultType *resultTypeP,
UInt32 *resultP, Char *nameP)
```

Parameters

-> `buttonIndex`	Index of the button for which you want to obtain information. Buttons are ordered from right to left, with the rightmost button at index 0.
<- `bitmapIdP`	The resource ID of the bitmap displayed on the button. Pass `NULL` if you don't want to retrieve this value.
<- `resultTypeP`	The type of action this button takes. Pass `NULL` if you don't want to retrieve this value.

<- `resultP`	The result of tapping the button. Pass `NULL` if you don't want to retrieve this information.
<- `nameP`	The text displayed in the status message when this button is tapped. Pass `NULL` if you don't want to retrieve this information. If not `NULL`, `nameP` must point to a string of `menuCmdBarMaxTextLength` size.

Result Returns `true` if the information was retrieved successfully, `false` if there is no command toolbar or if there is no button at `buttonIndex`.

Comments You can use this function to retrieve information about the buttons that are displayed on the command toolbar. If the command toolbar has not yet been initialized, this function returns `false`.

Note that the command toolbar is allocated when the user enters the command keystroke and deallocated when `MenuEraseStatus` is called. Thus, the only logical place to call `MenuCmdBarGetButtonData` is in response to a `menuCmdBarOpenEvent` or `sysNotifyMenuCmdBarOpenEvent` notification.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `MenuCmdBarDisplay`, `MenuCmdBarAddButton`

MenuDispose

Purpose Releases any memory allocated to the menu and the command status and restore any saved bits to the screen.

Prototype
```
void MenuDispose (MenuBarType *menuP)
```

Parameters

-> `menuP`	Pointer to the menu object to dispose. (See `MenuBarType`.) If `NULL`, this function returns immediately.

Result Returns nothing.

Comments Most applications do not need to call this function directly. `MenuDispose` is called by the system when the form that contains the menu is no longer the active form, when the form that contains the menu is freed, and when `FrmSetMenu` is called to change the form's menu bar.

See Also `MenuInit`, `MenuDrawMenu`

MenuDrawMenu

Purpose Draws the current menu bar and the last pull-down that was visible.

Prototype
```
void MenuDrawMenu (MenuBarType *menuP)
```

Parameters -> `menuP` Pointer to a `MenuBarType`. Must not be `NULL`.

Result Returns nothing.

Comments Most applications do not need to call this function directly. `MenuHandleEvent` calls `MenuDrawMenu` when the user taps the Menu silk-screen button (or taps the form's title on Palm OS 3.5 and higher).

The menu bar and the pull-down menu are drawn in front of all the application windows. The state of the insertion point, the bits that are obscured by the menu bar and the pull-down menu, and the currently active window are saved before the menu is drawn. These are all restored when the menu is erased.

A menu keeps track of the last pull-down menu displayed for as long as the menu is active. A menu loses its active status under these conditions:

- When `FrmSetMenu` is called to change the active menu on the form.
- When a new form, even a modal form or alert panel, becomes active.

Suppose a user selects your application's About item from the Options menu then clicks the OK button to return to the main form. When the About dialog is displayed, it becomes the active form,

which causes the main form's menu state to be erased. This menu state is not restored when the main form becomes active again. The next time `MenuDrawMenu` is called (that is, the next time the user taps the Menu silk-screen button), the menu bar is displayed as it was before, and the first pull-down menu listed in the menu bar is displayed instead of the Options pull-down menu.

See Also `MenuInit`, `MenuDispose`

MenuEraseStatus

Purpose Erases the menu command status.

Prototype

```
void MenuEraseStatus (MenuBarType *menuP)
```

Parameters

-> `menuP` Pointer to a `MenuBarType`, or `NULL` for the current menu.

Result Returns nothing.

Comments When the user selects a menu command using the command keystroke, the command toolbar or status message is displayed at the bottom of the screen. `MenuEraseStatus` erases the toolbar or status message.

Under most circumstances, you do not need to call this function explicitly—just let the current menu command status remove itself automatically. Otherwise, you may cause text to be erased before the user has a chance to see it.

You need to call `MenuEraseStatus` explicitly only if the command toolbar covers something that is going to be changed by the menu command the user has selected. For example, if the user selects a command that displays a new form, call `MenuEraseStatus` before executing the command. Also, if the command performs some drawing in the lower portion of the window, call `MenuEraseStatus` before performing the drawing function.

Compatibility The exact behavior when a menu shortcut character is entered depends on which version of the operating system is running. In

versions prior to release 3.5, the system displays the string "Command:" in the lower-left portion of the screen when the user enters the Graffiti command keystroke.

In Palm OS 3.5 and higher, entering the Graffiti command keystroke displays the command toolbar. This toolbar is the entire width of the screen and it displays buttons that the user can tap instead of entering another keystroke. If the user taps a button or enters a character that matches a shortcut character for an item on the active menu, a status message is displayed in the toolbar while the command is executed. Calling `MenuEraseStatus` on Palm OS 3.5 and higher deallocates the command toolbar data structure as well as erasing the command toolbar from the screen.

Because the command toolbar takes up more of the display than the pre-Palm OS 3.5 status message, you may find you need to call `MenuEraseStatus` more frequently in Palm OS 3.5 than in earlier versions.

See Also `MenuInit`

MenuGetActiveMenu

Purpose Returns a pointer to the currently active menu.

Prototype

```
MenuBarType *MenuGetActiveMenu (void)
```

Parameters None.

Result Returns a pointer to the currently active menu, or `NULL` if there is none.

Comments An active menu is not necessarily visible on the screen. A menu might be active but not visible, for example, if a command shortcut has been entered. In general, a form's menu becomes active the first time a `keyDownEvent` with a `vchrMenu` or `vchrCommand` is generated, and it remains active until a new form (including a modal form or alert panel) is displayed or until `FrmSetMenu` is called to change the form's menu.

If you want to know if the menu is visible rather than merely active, there are two options:

- You can check the `visible` attribute. For example:

```
myMenu = MenuGetActiveMenu();
if (myMenu && myMenu->attr.visible) {
  // menu is visible
}
```

- You can check for `winEnterEvent` and `winExitEvent`.

  When the system draws a menu, the menu's window becomes the active drawing window. The system generates a `winExitEvent` for the previous active drawing window and a `winEnterEvent` for the menu's window. When the menu is erased, the system generates a `winExitEvent` for the menu's window and a `winEnterEvent` for the window that was active before the menu was drawn.

  It's common to want to check if the menu is visible in applications that perform custom drawing to a window. Such applications want to make sure that they don't draw on top of the menu. The recommended way to do this is to stop drawing when you receive a `winExitEvent` matching your drawing window and resume drawing when you receive the corresponding `winEnterEvent`. For example, the following code is excerpted from the Reptoids example application's main event loop:

```
EvtGetEvent (&event, TimeUntillNextPeriod());

if (event.eType == winExitEvent) {
  if (event.data.winExit.exitWindow ==
    (WinHandle) FrmGetFormPtr(MainView)) {
      // stop drawing.
  }
}


else if (event.eType == winEnterEvent) {
  if (event.data.winEnter.enterWindow ==
    (WinHandle) FrmGetFormPtr(MainView) &&
    event.data.winEnter.enterWindow ==
```

```
        (WinHandle) FrmGetFirstForm ()) {
          // start drawing
      }
    }
```

Note that the second method of checking to see if a menu is visible is preferred because it is not specific to menus—your application should stop drawing if any window obscures your drawing window, and it will do so if you check for `winEnterEvent` and `winExitEvent`.

See Also MenuHandleEvent, MenuSetActiveMenu

MenuHandleEvent

Purpose Handles events in the current menu. This routine handles two types of events, penDownEvent and keyDownEvent.

Prototype

```
Boolean MenuHandleEvent (MenuBarType *menuP,
EventType *event, UInt16 *error)
```

Parameters

-> `menuP`	Pointer to a `MenuBarType` data structure.
-> `event`	Pointer to an `EventType` structure.
-> `error`	Error (or 0 if no error). Currently this function always sets `error` to zero.

Result Returns `true` if the event is handled; that is, if the event is a penDownEvent within the menu bar or the menu, or the event is a keyDownEvent that the menu supports. Returns `false` on any other event.

Comments When a penDownEvent is received in the menu bar, `MenuHandleEvent` tracks the pen until it comes up. If the pen comes up within the bounds of the menu bar, the selected title is inverted and the appropriate pull-down menu is drawn. Any previous pull-down menu is erased. If the pen comes up outside of the menu bar and pull-down menu, the menu is erased.

When a penDownEvent is received in a pull-down menu, MenuHandleEvent tracks the pen until it comes up. If the pen comes up within the bounds of the menu, a menuEvent containing the resource ID of the selected menu item is added to the event queue. If the pen comes up outside of the bounds of the menu and menu bar, the menu is erased.

If a keyDownEvent is received with a vchrMenu, the menu is drawn if it is not visible and erased if it is visible.

If a keyDownEvent is received with a vchrCommand, a command shortcut is in progress. Command shortcuts are handled differently depending on which version of Palm OS is running. On versions earlier than 3.5, the next keyDownEvent is checked to see if it is a valid menu shortcut. If so, a menuEvent is added to the event queue.

If a keyDownEvent is received with a vchrCommand on Palm OS 3.5 and higher, MenuHandleEvent enqueues a menuCmdBarOpenEvent if the command toolbar is not already open. The menuCmdBarOpenEvent provides a chance for applications to add their own buttons to the command toolbar. The next event might be either a keyDownEvent with a character that completes the shortcut or a penDownEvent on one of the buttons on the toolbar. The keyDownEvent is processed as with the earlier releases— if it is a valid menu shortcut, a menuEvent is added to the event queue. If the next event is a penDownEvent, the pen is tracked until it comes up. If the pen comes up within the bounds of a button, the appropriate action is taken. See the description of MenuCmdBarAddButton for more information.

In Palm OS version 3.5 or higher, if either the vchrMenu or the vchrCommand event causes a menu to be activated and initialized for the first time, a menuOpenEvent containing the reason the menu was initialized (menuButtonCause for a vchrMenu or menuCommandCause for a vchrCommand) is added to the event queue, and then the current event is added after it.

MenuHideItem

Purpose Makes the specified menu item hidden.

Prototype
```
Boolean MenuHideItem (UInt16 id)
```

Parameters -> `id` ID of the menu item to hide.

Result Returns `true` if the `hidden` attribute of the specified item was successfully enabled, `false` otherwise.

Comments You should call this function only in response to a `menuOpenEvent`. This event is generated when the menu is first made active. In general, a form's menu becomes active the first time a `keyDownEvent` with a `vchrMenu` or `vchrCommand` is generated, and it remains active until a new form (including a modal form or alert panel) is displayed or until `FrmSetMenu` is called to change the form's menu. Palm OS user interface guidelines discourage adding or hiding menu items at any time other than when the menu is first made active.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `MenuShowItem`

MenuInit

Purpose Loads a menu resource from a resource file.

Prototype
```
MenuBarType *MenuInit (UInt16 resourceId)
```

Parameters -> `resourceId` ID that identifies the menu resource.

Result Returns the pointer to a memory block allocated to hold the menu resource (a pointer to a `MenuBarType`).

Comments The menu is not usable until `MenuSetActiveMenu` is called.

Typically, you do not need to call this function directly. A form stores the resource ID of the menu associated with it and initializes that menu as necessary. If you want to change the form's menu, call FrmSetMenu, which handles disposing of the form's current menu, associating the new menu with the form, and initializing when needed.

See Also MenuSetActiveMenu, MenuDispose

MenuSetActiveMenu

Purpose Sets the current menu.

Prototype

```
MenuBarType *MenuSetActiveMenu
(MenuBarType *menuP)
```

Parameters

-> `menuP` Pointer to the memory block that contains the new menu, or `NULL` for none.

Result Returns a pointer to the menu that was active before the new menu was set, or `NULL` if no menu was active.

Comments This function sets the active menu but does not associate it with a form. It's recommended that you call FrmSetMenu instead of `MenuSetActiveMenu`. `FrmSetMenu` sets the active menu, frees the old menu, and associates the newly active menu with the form, which means the menu will be freed when the form is freed.

See Also MenuGetActiveMenu

MenuSetActiveMenuRscID

Purpose Sets the current menu by resource ID.

Prototype
```
void MenuSetActiveMenuRscID (UInt16 resourceId)
```

Parameters -> `resourceId` Resource ID of the menu to be made active.

Result Returns nothing.

Comments This function is similar to `MenuSetActiveMenu` except that you pass the menu's resource ID instead of a pointer to a menu structure. It is used as an optimization; with `MenuSetActiveMenu`, you must initialize the menu before making it active. Potentially, the application may exit before the menu is needed, making this memory allocation unnecessary. `MenuSetActiveMenuRscID` simply stores the resource ID. The next time a menu is requested, the menu is initialized from this resource.

It's recommended that you call `FrmSetMenu` instead of calling this function for the reasons given in `MenuSetActiveMenu`.

Compatibility Implemented only if 2.0 New Feature Set is present.

MenuShowItem

Purpose Makes the specified menu item visible.

Prototype
```
Boolean MenuShowItem (UInt16 id)
```

Parameters -> `id` ID of the menu item to display.

Result Returns `true` if the `hidden` attribute of the specified item was successfully disabled, `false` otherwise.

Comments You should call this function only in response to a `menuOpenEvent`. This event is generated when the menu is first made active. In general, a form's menu becomes active the first time a `keyDownEvent` with a `vchrMenu` or `vchrCommand` is generated, and it remains active until a new form (including a modal form or alert panel) is displayed or until `FrmSetMenu` is called to change the form's menu. Palm OS user interface guidelines discourage adding or hiding menu items at any time other than when the menu is first made active.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `MenuHideItem`

16

Private Records

This chapter describes the private records API as declared in `PrivateRecords.h`. It discusses the following topics:

- Private Record Data Structures
- Private Record Functions

Private Record Data Structures

privateRecordViewEnum

The `privateRecordViewEnum` enumerated type provides the available choices for displaying private records.

```
typedef enum privateRecordViewEnum {
  showPrivateRecords = 0x00,
  maskPrivateRecords,
  hidePrivateRecords
} privateRecordViewEnum;
```

Value Descriptions

`showPrivateRecords`	Display private records in the user interface.
`maskPrivateRecords`	Show a shaded rectangle in place of a private record.
`hidePrivateRecords`	Hide private records and provide no indication in the user interface that they exist.

Private Record Functions

SecSelectViewStatus

Purpose Display a form that allows the user to select whether to hide, show, or mask private records.

Prototype

```
privateRecordViewEnum SecSelectViewStatus (void)
```

Parameters None

Result Returns a constant that indicates which option the user selected. See `privateRecordViewEnum`.

Comments This function displays a dialog that allows users to change the preference `prefShowPrivateRecords`, which controls how private records are displayed.

When the user taps the OK button in this dialog, `SecVerifyPW` is called to see if the user changed the preference setting and, if so, to prompt the user to enter the appropriate password.

After calling this function, your code should check the return value or the value of `prefShowPrivateRecords` and mask, display, or hide the private records accordingly. See the description of `TblSetRowMasked` for a partial example.

Compatibility Implemented only if 3.5 New Feature Set is present.

SecVerifyPW

Purpose Display a password dialog, verify the password, and change the private records preference.

Prototype
```
Boolean SecVerifyPW
(privateRecordViewEnum newSecLevel)
```

Parameters

-> `newSecLevel` The security level (display, hide, or mask) selected on the private records dialog.

Result Returns `true` if the `prefShowPrivateRecords` preference was successfully changed, `false` if not.

Comments This function checks `newSecLevel` against the current value for the preference. If the two values differ and `newSecLevel` indicates a decrease in security, a dialog is displayed prompting the user to enter a password. (Hidden is considered the most secure, followed by masked. Showing private records is considered the least secure.) If the password is entered successfully, the preference is changed.

This function also displays an alert message if the security level has changed to either hidden or masked.

Compatibility Implemented only if 3.5 New Feature Set is present.

17

Progress Manager

This chapter provides reference material for the Progress Manager.

- Progress Manager Functions
- Application-Defined Functions

The header file `Progress.h` declares the API that this chapter describes. For more information on the progress manager, see the section "Progress Dialogs" in the *Palm OS Programmer's Companion*, vol. I.

Progress Manager Functions

PrgHandleEvent

Purpose Handles events related to the active progress dialog.

Prototype

```
Boolean PrgHandleEvent (ProgressPtr prgGP,
EventType *eventP)
```

Parameters

-> `prgGP` Pointer to a progress structure created by `PrgStartDialog`.

-> `eventP` Pointer to an event. You can pass a `NULL` event to cause this function to immediately call your `PrgCallbackFunc` function and then update the dialog (for example, after you call `PrgUpdateDialog`).

Result Returns `true` if the system handled the event. If it returns `false`, you should check if the user canceled the dialog by calling `PrgUserCancel`.

Comments Use this function instead of SysHandleEvent when you have a progress dialog. PrgHandleEvent internally calls SysHandleEvent as needed.

Note that the auto power-off feature of the system is automatically disabled when you use this function, unless the dialog is just displaying an error. This function also prevents appStopEvent events.

If an update to the dialog is pending (from a call to PrgUpdateDialog, for example) this function handles that event and causes the dialog to be updated. As part of this process, the textCallback function you specified in your call to PrgStartDialog is called. Your textCallback function should set the textP buffer in the PrgCallbackData structure with the new message to be displayed in the progress dialog. Optionally, you can also set the bitmapId field to include an icon in the update dialog. For more information about the textCallback function, see the section "Application-Defined Functions."

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also PrgStartDialog, PrgStopDialog, PrgUpdateDialog, PrgUserCancel

PrgStartDialog

Purpose Displays a progress dialog that can be updated.

Prototype

```
ProgressPtr PrgStartDialog (const Char *title,
PrgCallbackFunc textCallback, void *userDataP)
```

Parameters

-> title — Pointer to a title for the progress dialog. Must be a NULL-terminated string that is no longer than progressMaxTitle (20).

-> textCallback — Pointer to a callback function that supplies the text and icons for the current progress state. See PrgCallbackFunc.

-> `userDataP`	A pointer to data that you need to access in the callback function. This address gets passed directly to your function.

Result A pointer to a progress structure. This pointer must be passed to other progress manager functions and **must** be released by calling `PrgStopDialog`. `NULL` is returned if the system is unable to allocate the progress structure.

Comments The dialog created by this function can be updated by another process via the `PrgUpdateDialog` function. The dialog can contain a Cancel or OK button. The initial dialog defaults to stage 0 and calls the `textCallback` function to get the initial text and icon data for the progress dialog.

This function saves the screen bits behind the progress dialog, and these are restored when you call `PrgStopDialog`. Because of this, you should minimize changes to the screen while the progress dialog is displayed, otherwise, the restored bits may not match with what is currently being displayed.

Compatibility This version of the function is available only if 3.2 New Feature Set is present. On earlier systems, `PrgStartDialog` has the prototype shown for `PrgStartDialogV31`.

See Also `PrgHandleEvent`, `PrgStopDialog`, `PrgUpdateDialog`, `PrgUserCancel`

PrgStartDialogV31

Purpose Displays a progress dialog that can be updated.

Prototype

```
ProgressPtr PrgStartDialogV31 (const Char *title,
PrgCallbackFunc textCallback)
```

Parameters

-> `title`	Pointer to a title for the progress dialog. Must be a `NULL`-terminated string that is no longer than `progressMaxTitle` (20).

	-> `textCallback`	Pointer to a callback function that supplies the text and icons for the current progress state. See `PrgCallbackFunc`.

Result A pointer to a progress structure. This pointer must be passed to other progress manager functions and **must** be released by calling `PrgStopDialog`. `NULL` is returned if the system is unable to allocate the progress structure.

Compatibility This function corresponds to version of `PrgStartDialog` available on Palm OS® 3.0 and Palm OS 3.1.

See Also `PrgHandleEvent`, `PrgStopDialog`, `PrgUpdateDialog`, `PrgUserCancel`

PrgStopDialog

Purpose Releases memory used by the progress dialog and restores the screen to its initial state.

Prototype

```
void PrgStopDialog (ProgressPtr prgP,
Boolean force)
```

Parameters

-> `prgP`	Pointer to a progress structure created by `PrgStartDialog`.
-> `force`	`true` removes the progress dialog immediately, `false` causes the system to wait until the user confirms an error, if one is displayed.

Result Returns nothing.

Comments If the progress dialog is in a state where it is displaying an error message to the user, this function normally waits for the user to confirm the dialog before it removes the dialog. If you specify `true` for the `force` parameter, this causes the system to remove the dialog immediately.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also PrgHandleEvent, PrgStartDialog, PrgUpdateDialog, PrgUserCancel

PrgUpdateDialog

Purpose Updates the status of the current progress dialog.

Prototype

```
void PrgUpdateDialog (ProgressPtr prgGP,
UInt16 err, UInt16 stage, const Char *messageP,
Boolean updateNow)
```

Parameters

-> `prgGP`	Pointer to a progress structure created by PrgStartDialog.
-> `err`	If non-zero, causes the dialog to go into error mode, to display an error message with only an OK button.
-> `stage`	Current stage of progress. The callback function can use this to determine the correct string to display in the updated dialog.
-> `messageP`	Extra text that may be useful in displaying the progress for this stage. Used by the callback function, which can append it to the base message that is based on the stage.
-> `updateNow`	If `true`, the dialog is immediately updated. Otherwise, the dialog is updated on the next call to PrgHandleEvent.

Result Returns nothing.

Comments For more information about how the parameters are used and the callback function, see the section "Application-Defined Functions."

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also PrgHandleEvent, PrgStartDialog, PrgStopDialog, PrgUserCancel

PrgUserCancel

Purpose Macro that returns `true` if the user cancelled the process via the progress dialog.

Prototype
```
PrgUserCancel (prgP)
```

Parameters

-> `prgP`	Pointer to a progress structure (`ProgressPtr`) created by `PrgStartDialog`.

Result Returns the value of the `cancel` field in the progress structure (as a `UInt16`).

Comments This is a macro you can use to check if the user cancelled the process. If the user did cancel, you can change the progress dialog text to something like "Cancelling," or "Disconnecting," or whatever is appropriate for your application. Then you should cancel the process, end the communication session, or do whatever processing is necessary.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `PrgHandleEvent`, `PrgStartDialog`, `PrgStopDialog`, `PrgUpdateDialog`

Application-Defined Functions

PrgCallbackFunc

Purpose Supplies the text and icons for the current progress state.

Prototype
```
Boolean (*PrgCallbackFunc)
(PrgCallbackDataPtr cbP)
```

Parameters

<-> `cbP`	A pointer to a `PrgCallbackData` structure. See below.

Result Returns `true` if the progress dialog should be updated using the values you specified in the `PrgCallbackData` structure. If you specify `false`, the dialog is still updated, but with default status messages. (Returning `false` is not recommended.)

Comments This is a callback function that you specify when you call `PrgStartDialog`. The callback function is called by `PrgHandleEvent` when it needs current progress information to display in the progress dialog.

The system passes this function one parameter, a pointer to a `PrgCallbackData` structure. Here are the important fields in that data structure (note that -> indicates you set the field in the `textCallback` function):

`<- UInt16 stage` Current stage (passed from `PrgUpdateDialog`).

`<-> Char *textP` Buffer to hold the text to display in the updated dialog. You might want to look up a message in a resource file, based on the value in the `stage` field. Also, you should append the additional text in the `message` field, to form the full string to display. Be sure to include a null terminator at the end of the string you return, and don't exceed the length in `textLen`.

`<- UInt16 textLen`
Maximum length of the text buffer `textP`. Note that this value is set for you by the caller. Be careful not to exceed this length in `textP`.

`<- Char *message`
Additional text to display in the dialog (from the `messageP` parameter to `PrgUpdateDialog`). This should be no longer than `progressMaxMessage` (128).

`<- Err error` Current error (passed from the `err` parameter to `PrgUpdateDialog`).

`-> UInt16 bitmapId`
Resource ID of the bitmap to display in the progress dialog, if any.

`<- UInt16 canceled:1`
`true` if user has pressed the cancel button.

`<- UInt16 showDetails:1`
`true` if user pressed the down arrow button on the Palm™ device for more details. (Because this is a non-standard user interface technique, you shouldn't use this feature to display details that users need under normal conditions. It's more for debugging purposes.)

`-> UInt16 textChanged:1`
If `true`, then update text (defaults to `true`). You can set this to `false` to avoid an update to the text.

`<- UInt16 timedOut:1`
`true` if update caused by a timeout.

`<-> UInt32 timeout`
Timeout in ticks to force next update. After this number of ticks, an update is automatically triggered (which sets the `timedOut` flag). You can use this feature to do a simple animation effect. Note that you must set the timeout for `EvtGetEvent` to a value that is equal to or less than this value, otherwise you won't get update events as frequently as you expect.

`-> UInt16 delay:1`
If `true`, delay for one second after updating the dialog. Use this value when you are displaying the final progress message so that users have a chance to see the message before the dialog closes. This field is available only if 3.2 New Feature Set is present.

`<- void *userDataP`

A pointer to any application-defined data that the function needs to access. You specify this pointer as a parameter to `PrgStartDialog` if the callback function needs to access some application data but does not have access to application globals. This field is available only if 3.2 New Feature Set is present.

In this function, you should set the value of the `textP` buffer to the string you want to display in the progress dialog when it is updated. You can use the value in the `stage` field to look up a message in a string resource. You also might want to append the text in the `message` field to your base string. Typically, the `message` field would contain more dynamic information that depends on a user selection, such as a phone number, device name, or network identifier, etc.

For example, the `PrgUpdateDialog` function might have been called with a `stage` of 1 and a `messageP` parameter value of a phone number string, "555-1212". Based on the stage, you might find the string "Dialing" in a string resource, and append the phone number, to form the final text "Dialing 555-1212" that you place in the text buffer `textP`.

Keeping the static strings corresponding to various stages in a resource makes it easier to localize your application. More dynamic information can be passed in via the `messageP` parameter to `PrgUpdateDialog`.

NOTE: This function is called only if the parameters passed to `PrgUpdateDialog` have changed from the last time it was called. If `PrgUpdateDialog` is called twice with exactly the same parameters, the `textCallback` function is called only once.

18

Scroll Bars

This chapter provides reference material for the scroll bar API.

- Scroll Bar Data Structures
- Scroll Bar Resources
- Scroll Bar Functions

The header file `ScrollBar.h` declares the API that this chapter describes. For more information on scroll bars, see the section "Scroll Bars" on page 122 in the *Palm OS Programmer's Companion*, vol. I.

Scroll Bar Data Structures

ScrollBarAttrType

The `ScrollBarAttrType` bit field defines a scroll bar's visible characteristics.

```
typedef struct {
  UInt16 usable  :     1;
  UInt16 visible  :    1;
  UInt16 hilighted  :  1;
  UInt16 shown  :      1;
  UInt16 activeRegion: 4;
} ScrollBarAttrType;
```

Field Descriptions

`usable` If not set, the scroll bar object is not considered part of the current interface of the application, and it doesn't appear on screen.

`visible` If set, the scroll bar is allowed to be displayed on the screen. If both `visible` and `shown` are `true`, then the scroll bar is actually displayed on the screen.

`hilighted` `true` if either the up arrow or the down arrow is highlighted.

`shown` Set if the scroll bar is visible and if `maxValue` > `minValue`. (See `ScrollBarType`.)

`activeRegion` The region of the scroll bar that is receiving the pen down events. Possible values are:

`sclUpArrow`	The up arrow.
`sclDownArrow`	The down arrow.
`sclUpPage`	The region between the scroll car and the up arrow.
`sclDownPage`	The region between the scroll car and the down arrow.
`sclCar`	The scroll car.

ScrollBarPtr

The `ScrollBarPtr` type defines a pointer to a `ScrollBarType` structure.

```
typedef ScrollBarType *ScrollBarPtr;
```

You pass the `ScrollBarPtr` as an argument to all scroll bar functions. You can obtain the `ScrollBarPtr` using the function `FrmGetObjectPtr` in this way:

```
scrollBarPtr = FrmGetObjectPtr(frm,
  FrmGetObjectIndex(frm, scrollBarID));
```

where `scrollBarID` is the resource ID assigned when you created the scroll bar.

ScrollBarType

The `ScrollBarType` represents a scroll bar.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `ScrollBarType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  RectangleType         bounds;
  UInt16                id;
  ScrollBarAttrType     attr;
  Int16                 value;
  Int16                 minValue;
  Int16                 maxValue;
  Int16                 pageSize;
  Int16                 penPosInCar;
  Int16                 savePos;
} ScrollBarType;
```

Your code should treat the `ScrollBarType` structure as opaque. Use the functions described in this chapter to retrieve and set each value. Do not attempt to change structure member values directly.

Field Descriptions

`bounds` Position (using absolute coordinates) and size (in pixels) of the scroll bar on the screen. For example, to access the bounds of an object in a form whose ID is `kObjectID`:

```
{
RectangleType rect;
FormPtr formP = FrmGetActiveForm();

FrmGetObjectBounds(formP,
FrmGetObjectIndex(formP,
kObjectID), &rect);
}
```

`id` ID value you specified when you created the scroll bar object.

`attr`	Scroll bar's attributes. See `ScrollBarAttrType`.
`value`	Current value of the scroll bar. This value is used to determine where to position the scroll car (the dark region in the scroll bar that indicates the position in the document). Access with `SclGetScrollBar`. The number given is typically a number relative to `minValue` and `maxValue`. These values have nothing to do with any physical characteristics of the object that the scroll bar is attached to, such as the number of lines in the object. This value is typically set to 0 initially and then adjusted programmatically with `SclSetScrollBar`.
`minValue`	Minimum value. When `value` equals `minValue`, the scroll car is positioned at the very top of the scrolling region. This value is typically 0. Access with `SclGetScrollBar`.
`maxValue`	Maximum value. When `value` equals `maxValue`, the scroll car is positioned at the very bottom of the scrolling region. This value is typically set to 0 initially and then adjusted programmatically with `SclSetScrollBar`. Access with `SclGetScrollBar`.
`pageSize`	Number of lines to scroll when user scrolls one page. Access with `SclGetScrollBar`.
`penPosInChar`	Used internally.
`savePos`	Used internally.

Scroll Bar Resources

The Scroll Bar Resource (tSCL) represents a scroll bar.

Scroll Bar Functions

SclDrawScrollBar

Purpose Draw a scroll bar.

Prototype
```
void SclDrawScrollBar (ScrollBarType *bar)
```

Parameters

-> `bar`	Pointer to a scroll bar structure (see `ScrollBarType`).

Result Returns nothing.

Comments This function is called internally by `SclSetScrollBar` and `FrmDrawForm`. You rarely need to call it yourself.

Compatibility Implemented only if 2.0 New Feature Set is present.

SclGetScrollBar

Purpose Retrieve a scroll bar's current position, its range, and the size of a page.

Prototype
```
void SclGetScrollBar (const ScrollBarType *bar,
Int16 *valueP, Int16 *minP, Int16 *maxP,
Int16 *pageSizeP)
```

Parameters

-> `bar`	Pointer to a scroll bar structure (see `ScrollBarType`).
<- `valueP`	A value representing the scroll car's current position. (The scroll car is the dark region that indicates the position in the document.)

<-minP	A value representing the top of the user interface object.
<-maxP	A value representing the bottom of the user interface object.
<-pageSizeP	Pointer to size of a page (used when page scrolling).

Result Returns the scroll bar's current values in `valueP`, `minP`, `maxP`, and `pageSizeP`.

Comments You might use this function immediately before calling `SclSetScrollBar` to update the scroll bar. `SclGetScrollBar` returns the scroll bar's current values, which you can then adjust as necessary and pass to `SclSetScrollBar`.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `SclSetScrollBar`

SclHandleEvent

Purpose Handles events that affect a scroll bar.

Prototype

```
Boolean SclHandleEvent (ScrollBarType *bar,
const EventType *event)
```

Parameters

-> bar	Pointer to a scroll bar structure (see `ScrollBarType`).
-> event	Pointer to an event (EventType).

Result Returns `true` if the event was handled.

Comment When a `penDownEvent` occurs, the scroll bar sends an `sclEnterEvent` to the event queue.

When an `sclEnterEvent` occurs, the scroll bar determines what its new value should be based on which region of the scroll bar is

receiving the pen down events. It then sends either an `sclRepeatEvent` or an `sclExitEvent` to the event queue.

When the user holds and drags the scroll bar with the pen, the scroll bar sends a `sclRepeatEvent`. Applications that implement dynamic scrolling should catch this event and move the text each time one arrives.

When the user releases the pen from the scroll bar, the scroll bar sends a `sclExitEvent`. Applications that implement non-dynamic scrolling should catch this event and move the text when `sclExitEvent` arrives. Applications that implement dynamic scrolling can ignore this event.

Compatibility Implemented only if 2.0 New Feature Set is present.

SclSetScrollBar

Purpose Set the scroll bar's current position, its range, and the size of a page. If the scroll bar is visible and its minimum and maximum values are not equal, it's redrawn.

Prototype

```
void SclSetScrollBar (ScrollBarType *bar,
Int16 value, Int16 min, Int16 max, Int16 pageSize)
```

Parameters

-> `bar`	Pointer to a scroll bar structure (see `ScrollBarType`).
-> `value`	The position the scroll car should move to. (The scroll car is the dark region that indicates the position in the document.)
-> `min`	Minimum value.
-> `max`	Maximum value.
-> `pageSize`	Number of lines of text that can be displayed on a the screen at one time (used when page scrolling).

Result Returns nothing. May display a fatal error message if the `min` parameter is greater than the `max` parameter.

Comments

Call this function when the user adds or deletes text in a field or when a table row is added or deleted.

For scrolling fields, your application should catch the `fldChangedEvent` and update the scroll bar at that time.

The `max` parameter is computed as:

number of lines of text – page size + overlap

where number of lines of text is the total number of lines or rows needed to display the entire object, page size is the number of lines or rows that can be displayed on the screen at one time, and overlap is the number of lines or rows from the bottom of one page to be visible at the top of the next page.

For example, if you have 100 lines of text and 10 lines show on a page, the `max` value would be 90 or 91, depending on the overlap. So if value is greater than or equal to 90 or 91, the scroll car is at the very bottom of the scrolling region.

You can use the `FldGetScrollValues` function to compute the values you pass for `value`, `min`, and `max`. For example:

```
FldGetScrollValues (fld, &scrollPos,
  &textHeight,  &fieldHeight);

if (textHeight > fieldHeight)
  maxValue = textHeight - fieldHeight;
else if (scrollPos)
  maxValue = scrollPos;
else
  maxValue = 0;

SclSetScrollBar (bar, scrollPos, 0, maxValue,
  fieldHeight-1);
```

In this case, `textHeight` is the number of lines of text and `fieldHeight` is the page size. No lines overlap when you scroll one page. Notice that if the page size is greater than the lines of text, then `max` equals `min`, which means that the scroll bar is not displayed.

For scrolling tables, there is no equivalent to `FldGetScrollValues`. Your application must scroll the table itself and keep track of the scroll values. See the `ListViewUpdateScrollers` function in the Memo example application (`MemoMain.c`) for an example of setting scroll bar values for a table.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `SclGetScrollBar`

19

System Dialogs

This chapter provides reference material for system dialogs declared in the header files `FatalAlert.h`, `Launcher.h`, `GraffitiReference.h`, and `GraffitiUI.h`.

System Dialog Functions

SysAppLauncherDialog

Purpose Display the launcher popup, get a choice, ask the system to launch the selected application, clean up, and leave. If there are no applications to launch, nothing happens.

Prototype

```
void SysAppLauncherDialog()
```

Parameters None.

Result The system may be asked to launch an application.

Comments Typically, this routine is called by the system as necessary. Most applications do not need to call this function themselves.

In Palm OS® version 3.0 and higher the launcher is an application, rather than a popup. This function remains available for compatibility purposes only.

See Also `SysAppLaunch`, the "Application Launcher" section in the *Palm OS Programmer's Companion*, vol. I

SysFatalAlert

Purpose Display a fatal alert until the user taps a button in the alert.

Prototype
```
UInt16 SysFatalAlert (const Char *msg)
```

Parameters

`msg`	Message to display in the dialog.

Result The button tapped; first button is zero.

SysGraffitiReferenceDialog

Purpose Pop up the Graffiti® Reference Dialog.

Prototype
```
void SysGraffitiReferenceDialog
(ReferenceType referenceType)
```

Parameters

`referenceType`	Which reference to display. See `GraffitiReference.h` for more information.

Result Nothing returned.

20

Tables

This chapter describes the table API as declared in the header file `Table.h`. It discusses the following topics:

- Table Data Structures
- Table Constants
- Table Resource
- Table Functions
- Application-Defined Functions

For more information on tables, see the section "Tables" in the *Palm OS Programmer's Companion*, vol. I.

Table Data Structures

TableAttrType

The `TableAttrType` bit field defines the visible characteristics of the table.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `TableAttrType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16 visible:1;
  UInt16 editable:1;
  UInt16 editing:1;
  UInt16 selected:1;
```

```
    UInt16 hasScrollBar:1;
    UInt16 reserved:11;
  } TableAttrType;
```

Your code should treat the `TableAttrType` bit field as opaque. Use the functions specified in the descriptions below to retrieve and set each value. Do not attempt to change member values directly.

Field Descriptions

`visible`	If set, the table is drawn on the screen. The value of this field is set by `TblDrawTable` and cleared by `TblEraseTable`.
`editable`	If set, the user can modify the table. You specify this when you create the table resource.
`editing`	If set, the table is in edit mode. The table is in edit mode while the user edits a text item. The value of this field is returned by `TblEditing`.
`selected`	If set, the current item (as identified by the `TableType` fields `currentRow` and `currentColumn`) is selected. Use `TblGetSelection` to retrieve this value.
`hasScrollBar`	If set, the table has a scroll bar. Note that this attribute can only be set programmatically. See `TblHasScrollBar`.

TableColumnAttrType

The `TableColumnAttrType` structure defines a column in a table.

WARNING! Palm, Inc. Palm, Inc. does not support or provide backward compatibility for the `TableColumnAttrType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  Coord                 width;
```

```
    UInt16                  reserved1 : 5;
    UInt16                  masked : 1;
    UInt16                  editIndicator : 1;
    UInt16                  usable : 1;
    UInt16                  reserved2 : 8;
    Coord                   spacing;
    TableDrawItemFuncPtr    drawCallback;
    TableLoadDataFuncPtr    loadDataCallback;
    TableSaveDataFuncPtr    saveDataCallback;
  } TableColumnAttrType;
```

Your code should treat the `TableColumnAttrType` structure as opaque. Use the functions specified in the descriptions below to retrieve and set each value. Do not attempt to change structure member values directly.

Field Descriptions

`width` — The column's width in pixels. See `TblGetColumnWidth` and `TblSetColumnWidth`.

`reserved1` — Reserved for future use.

`masked` — If `true` and the item's row also has a `masked` attribute of `true`, the table cell is drawn on the screen but is shaded to obscure the information that it contains. See `TblSetColumnMasked`.

`editIndicator` — If `true`, items in the column should be highlighted if selected while in edit mode. If `false`, items in the column should not be highlighted. By default, text field items are highlighted in edit mode, but all other types of items are not highlighted. The default can be overridden with `TblSetColumnEditIndicator`.

`usable` — If `false`, the column is not considered part of the current interface of the application, and it doesn't appear on screen. See `TblSetColumnUsable`.

`reserved2`	Reserved for future use.
`spacing`	The spacing in pixels between this column and the next column. See `TblGetColumnSpacing` and `TblSetColumnSpacing`.
`drawCallback`	Pointer to a function that draws custom items in the column. This function is called during `TblDrawTable` and `TblRedrawTable`. See `TblSetCustomDrawProcedure`.
`loadDataCallback`	Pointer to a function that loads data into the column. This function is called during `TblDrawTable` and `TblRedrawTable`. See `TblSetLoadDataProcedure`.
`saveDataCallback`	Pointer to a function that saves the data in the column. Called when the focus moves from one table cell to another and when the table loses focus entirely. See `TblSetSaveDataProcedure`.

Compatibility The `masked` field is defined only if 3.5 New Feature Set is present.

TableItemPtr

A `TableItemPtr` points to a `TableItemType`.

```
typedef TableItemType *TableItemPtr;
```

TableItemType

The `TableItemType` structure defines an item, or cell, within the table.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `TableItemType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  TableItemStyleType   itemType;
  FontID               fontID;
  Int16                intValue;
  Char                 *ptr;
} TableItemType;
```

Your code should treat the `TableItemType` structure as opaque. Use the functions specified in the descriptions below to retrieve and set each value. Do not attempt to change structure member values directly.

NOTE: None of the table items create memory that you need to free. The table manager handles all of the allocating and deallocating of memory for table items. The only memory you are responsible for freeing is the memory handle containing the text that you want displayed in editable text fields. (See `TableLoadDataFuncType`.)

Field Descriptions

`itemType` The type of the item, such as a control, a text label, and so on. `TblSetItemStyle` sets this value. The rest of the fields in this struct are either used or not used depending on the `itemType`. See Table 20.1.

`fontID` ID of the font used to display the item's text. `TblGetItemFont` and `TblSetItemFont` retrieve and set this value.

`intValue` Integer value of the item. `TblGetItemInt` and `TblSetItemInt` retrieve and set this value.

`ptr` Pointer to the item's text. `TblGetItemPtr` and `TblSetItemPtr` retrieve and set this value. All text items have a maximum of `tableMaxTextItemSize`.

The following table lists the possible values for the `itemType` field, describes how each type is drawn, describes which other fields are used for each `itemType`, and provides special instructions for

setting those fields. Note in particular that the `fontID` field is often not used. Instead, certain items are displayed in a standard font. These are noted in the last column of this table.

Table 20.1 TableItemType fields

itemType	Description	TableItemType Fields Used
`checkboxTableItem`	A checkbox control.	`intValue`
`customTableItem`	Application-defined cell.	None. Custom items are drawn using the custom drawing function that you implement. See `TableDrawItemFuncType`. If you want, you can store data in the `intValue` and `ptr` fields.

`dateTableItem`	Non-editable date in the form *month/day*, or a dash if the date value is -1. The date is followed by an exclamation point if it has past.	`intValue`
		The `intValue` field should be a value that can be cast to `DateType`. `DateType` is currently defined as a 16-bit number:
		`yyyyyyymmmmddddd`
		The first 7 bits are the year given as the offset since 1904, the next 4 bits are the month, and the next 5 bits are the day. Dates are always drawn in the current font.

`labelTableItem`	Non-editable text.	`ptr` Labels are displayed in the system's default font.
`numericTableItem`	Non-editable number.	`intValue` Numbers are displayed in the system's default bold font.

Table 20.1 TableItemType fields *(continued)*

itemType	Description	TableItemType Fields Used

`popupTriggerTableItem`	A list with a pop-up trigger.	`intValue`
		`ptr`
		`intValue` is the index of the list item that should be displayed.
		`ptr` is a pointer to the list. Lists are displayed in the system's default font.

`textTableItem`	Editable text field.	`fontID`
		`ptr`
		For this item type, implement the callback function `TableLoadDataFuncType` to load text into the table cell and implement the callback `TableSaveDataFuncType` to save data before the field is freed.

`textWithNoteTableItem`	Editable text field and a note icon to the right of the text.	`fontID`
		`ptr`
		For this item type, implement the callback function `TableLoadDataFuncType` to load text into the table cell and implement the callback `TableSaveDataFuncType` to save data before the field is freed.

Table 20.1 TableItemType fields *(continued)*

itemType	Description	TableItemType Fields Used
`timeTableItem`	Not implemented.	

`narrowTextTableItem`	Editable text with space reserved on the right side of the cell.	`fontID`
		`ptr`
		`intValue`
		`intValue` is the number of pixels to reserve on the right side of the cell.
		For this item type, implement the callback function `TableDrawItemFuncType` to draw in the space reserved on the right side of the cell, the `TableLoadDataFuncType` callback function to load text into the table cell, and the callback function `TableSaveDataFuncType` to save data before the field is freed.

TablePtr

The `TablePtr` type defines a pointer to a `TableType`.

```
typedef TableType *TablePtr;
```

You pass the table's pointer as an argument to all table functions. You can obtain the table's pointer using the function `FrmGetObjectPtr` in this way:

```
tblPtr = FrmGetObjectPtr(frm,
  FrmGetObjectIndex(frm, tblID));
```

where `tblID` is the resource ID assigned when you created the table.

TableRowAttrType

The `TableRowAttrType` structure defines a row in a table.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `TableRowAttrType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct {
  UInt16       id;
  Coord        height;
  UInt32       data;
  UInt16       reserved1 : 7;
  UInt16       usable : 1;
  UInt16       reserved2 : 4;
  UInt16       masked : 1;
  UInt16       invalid : 1;
  UInt16       staticHeight : 1;
  UInt16       selectable : 1;
  UInt16       reserved3;
} TableRowAttrType;
```

Your code should treat the `TableRowAttrType` structure as opaque. Use the functions specified in the descriptions below to retrieve and set each value. Do not attempt to change structure member values directly.

Field Descriptions

`id`	The ID of this row. See `TblFindRowID`, `TblGetRowID`, and `TblSetRowID`.
`height`	Height of the row in pixels. The functions `TblSetRowHeight` and `TblGetRowHeight` set and retrieve this value.

`data`	Any application-specific value you want to store in this row. For example, the Datebook and ToDo applications use this field to store the unique ID of the database record that is displayed in this table row. See `TblFindRowData`, `TblGetRowData`, and `TblSetRowData`.
`reserved1`	Reserved for future use.
`usable`	If `false`, the row is not considered part of the current interface of the application, and it doesn't appear on screen. Table rows have `usable` set to `false` when they are scrolled off the screen. See `TblRowUsable` and `TblSetRowUsable`. The function `TblGetLastUsableRow` returns the row that appears at the bottom of the screen.
`masked`	If `true` and the item's column also has a `masked` attribute of `true`, the table cell is drawn on the screen but is shaded to obscure the information that it contains. See `TblSetRowMasked` and `TblRowMasked`.
`reserved2`	Reserved for future use.
`invalid`	If `true`, the row needs to be redrawn. See `TblRowInvalid`, `TblMarkRowInvalid`, and `TblMarkTableInvalid`.
`staticHeight`	`true` if the row height cannot be changed, `false` otherwise. If `false`, text fields in this table row will dynamically resize to multiple lines as necessary. See `TblSetRowStaticHeight`.
`selectable`	If `true`, the user can select individual cells in this row. See `TblSetRowSelectable` and `TblRowSelectable`.
`reserved3`	Reserved for future use.

Compatibility The `masked` field is defined only if 3.5 New Feature Set is present.

TableType

The `TableType` structure represents a table.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `TableType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct TableType {
  UInt16                id;
  RectangleType         bounds;
  TableAttrType         attr;
  Int16                 numColumns;
  Int16                 numRows;
  Int16                 currentRow;
  Int16                 currentColumn;
  Int16                 topRow;
  TableColumnAttrType   *columnAttrs;
  TableRowAttrType      *rowAttrs;
  TableItemPtr          items;
  FieldType             currentField;
} TableType;
```

Your code should treat the `TableType` structure as opaque. Use the functions specified in the descriptions below to retrieve and set each value. Do not attempt to change structure member values directly.

Field Descriptions

`id`	ID value you specified when you created the table resource. This ID is included as part of the event data of `tblEnterEvent`.
`bounds`	Position and size of the table object. The functions `TblGetBounds`, `FrmGetObjectBounds`, `TblSetBounds`, and `FrmSetObjectBounds` retrieve and set this value.
`attr`	The table's attributes. See `TableAttrType`.

`numColumns`	Number of columns displayed by the table object. You specify the number of columns when you create the table resource, and this value cannot be changed. Access with `TblGetNumberOfColumns`.
`numRows`	Maximum number of visible rows in the table object. You specify this value when you create the table resource, and it does not change; however, the total number of rows in a table can change if you insert new rows in a table, and even the number of currently visible rows can change if a text field within a table cell is dynamically resized. The function `TblGetNumberOfRows` returns the value of this field.
`currentRow`	Row index of the currently selected table cell. Rows are numbered from top to bottom starting with 0. `TblGetSelection` and `TblSetSelection` retrieve and set the values of `currentRow`.
`currentColumn`	Column index of the currently selected table cell. Columns are numbered from left to right starting with 0. If the `TableAttrType` `selected` is `true`, then this table cell is highlighted. If `selected` is `false`, the table still considers this the "current" item, but it is not highlighted. `TblGetSelection` and `TblSetSelection` retrieve and set the values of `currentColumn`.
`topRow`	First visible row of the table object. Access with `TblGetTopRow`.
`columnAttrs`	An array of each table column's attributes. See `TableColumnAttrType`.

`rowAttrs`	An array of each row's attributes, such as its ID, height, and whether or not it is usable, selectable, or invalid. See `TableRowAttrType`.
`items`	An array of each item's (table cell's) attributes, such as the item type, font ID, an integer value, and a character pointer. See `TableItemType`.
`currentField`	Field object the user is currently editing. The function `TblGetCurrentField` retrieves the value of this item.

Table Constants

Constant	Value	Description
`tableDefaultColumnSpacing`	1	Never used.
`tableNoteIndicatorHeight`	11	The height in pixels of the note indicator for tables items of type `textWithNoteTableItem`.
`tableNoteIndicatorWidth`	7	The width in pixels of the note indicator for tables items of type `textWithNoteTableItem`.
`tableMaxTextItemSize`	255	The maximum length of an editable text field within a table cell.
`tblUnusableRow`	0xffff	Value returned by `TblGetLastUsableRow` if none of the table's rows are usable. This value is only available in version 3.5 and higher.

Table Resource

The Table Resource (tTBL) represents a table on screen.

Table Functions

TblDrawTable

Purpose Draw a table.

Prototype
```
void TblDrawTable (TableType *tableP)
```

Parameters -> `tableP` Pointer to a table object. (See `TableType`.)

Result Returns nothing.

Comments This function is called as part of `FrmDrawForm` when the form contains a table object.

This function draws the entire table, marking all rows valid before drawing. See the `TableItemType` struct description for more information about how each type of table cell is drawn.

When drawing cells with editable text fields (`textTableItem`, `textWithNoteTableItem`, or `narrowTextTableItem`), this function uses the `TableLoadDataFuncType` callback function to load the text into the table cells. The text field does not retain the text handle that your `TableLoadDataFunc` returns, meaning that you are responsible for freeing the memory that you load into the table.

When drawing `narrowTextTableItem` cells or `customTableItem` cells, this function uses the `TableDrawItemFuncType` callback function to draw the extra pixels after the text or to draw the entire cell.

On color systems, tables are always drawn using the same color as is used for the field background color.

When the user has set the security setting to mask private records, table cells that contain private database records are drawn as shaded cells to obscure the information they contain. You must explicitly tell the table which cells are masked by making the appropriate calls to `TblSetRowMasked` and `TblSetColumnMasked`.

Compatibility Color support and masked private records are only supported in Palm OS® version 3.5 and higher.

In versions earlier than 3.5, this function did not erase table cells before it drew them. In earlier releases, consider calling `TblEraseTable` before calling this function, particularly if the entire table has changed, as the visual effect of drawing over a white background may be more pleasing.

See Also `TblEraseTable`, `TblRedrawTable`, `TblSetCustomDrawProcedure`

TblEditing

Purpose Check whether a table is in edit mode.

Prototype

```
Boolean TblEditing (const TableType *tableP)
```

Parameters -> `tableP` Pointer to a table object. (See `TableType`.)

Result Returns `true` if the table is in edit mode, `false` otherwise.

Comments The table is in edit mode while the user edits a text item. More specifically, the table is in edit mode when a `tblEnterEvent` is received on an editable table cell (`textTableItem`, `textWithNoteTableItem`, or `narrowTextTableItem`), or when `TblGrabFocus` is called.

The table is taken out of edit mode when a the user places the pen on a note in a `textWithNoteTableItem` or when the table releases the focus (`TblReleaseFocus`).

TblEraseTable

Purpose Erase a table object.

Prototype
```
void TblEraseTable (TableType *tableP)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)

Result Returns nothing.

Comments This function sets the table's `visible` and `selected` attributes to `false`. It does not invalidate table rows.

See Also `TblDrawTable`, `TblSetCustomDrawProcedure`, `TblRedrawTable`

TblFindRowData

Purpose Return the number of the row that contains the specified data value.

Prototype
```
Boolean TblFindRowData (const TableType *tableP,
UInt32 data, Int16 *rowP)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `data`	Row data to find.
<- `rowP`	Pointer to the row number (return value).

Result Returns `true` if a match was found, `false` otherwise.

Comments This function searches for a row whose attributes contain the specified data. The data is any application-specific data that you have set with `TblSetRowData`.

See Also `TblGetRowData`, `TblFindRowID`, `TableRowAttrType`

TblFindRowID

Purpose Return the number of the row with the specified ID.

Prototype
```
Boolean TblFindRowID (const TableType *tableP,
UInt16 id, Int16 *rowP)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `id`	Row ID to find.
<- `rowP`	Pointer to the row number (return value).

Result Returns `true` if a match was found, `false` otherwise.

See Also `TblSetRowID`, `TblFindRowData`, `TableRowAttrType`

TblGetBounds

Purpose Return the bounds of a table.

Prototype
```
void TblGetBounds (const TableType *tableP,
RectangleType *rP)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
<- `rP`	A `RectangleType` structure in which the bounds are returned.

Result Returns nothing. The `rP` parameter contains the bounds.

See Also `TblGetItemBounds`

TblGetColumnSpacing

Purpose Return the spacing after the specified column.

Prototype
```
Coord TblGetColumnSpacing
(const TableType *tableP, Int16 column)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `column`	Column number (zero-based).

Result Returns the spacing after column (in pixels).

This function may display a fatal error message if the `column` parameter is invalid.

See Also `TblGetColumnWidth`, `TblSetColumnSpacing`, `TblSetColumnUsable`

TblGetColumnWidth

Purpose Return the width of the specified column.

Prototype
```
Coord TblGetColumnWidth (const TableType *tableP,
Int16 column)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `column`	Column number (zero-based).

Result Returns the width of a column (in pixels). This function may display a fatal error message if the `column` parameter is invalid.

See Also `TblGetColumnSpacing`, `TblSetColumnWidth`, `TblSetColumnUsable`

TblGetCurrentField

Purpose Return a pointer to the FieldType in which the user is currently editing a text item.

Prototype
```
FieldPtr TblGetCurrentField
(const TableType *tableP)
```

Parameters

-> tableP	Pointer to a table object. (See TableType.)

Result Returns a pointer to the currently selected field, or NULL if the table is not in edit mode.

See Also TblGetSelection

TblGetItemBounds

Purpose Return the bounds of an item in a table.

Prototype
```
void TblGetItemBounds (const TableType *tableP,
Int16 row, Int16 column, RectangleType *rP)
```

Parameters

-> tableP	Pointer to a table object. (See TableType.)
-> row	Row number of the item (zero-based).
-> column	Column number of the item (zero-based).
<- rP	Pointer to a structure that holds the bounds of the item.

Result Returns nothing. Stores the bounds in rP. This function may raise a fatal exception if the row or column parameter specifies a row or column that does not appear on screen.

TblGetItemFont

Purpose Return the font used to display a table item.

Prototype
```
FontID TblGetItemFont (const TableType *tableP,
Int16 row, Int16 column)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number of the item (zero-based).
-> `column`	Column number of the item (zero-based).

Result Returns the ID of the font used for the table item at the row and column indicated. This function may display a fatal error message if the `row` or `column` parameter specifies a row or column that is not on the screen.

Comments This function returns the value stored in the `fontID` field for this table item. Only certain types of table items use the font specified by the `fontID` field when they are displayed. The `TableItemType` description specifies what font is used to display each type of table item.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `TblSetItemFont`

TblGetItemInt

Purpose Return the integer value stored in a table item.

Prototype
```
Int16 TblGetItemInt (const TableType *tableP,
Int16 row, Int16 column)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number of the item (zero-based).
-> `column`	Column number of the item (zero-based).

Result Returns the integer value. This function may display a fatal message if the `row` or `column` does not appear on the screen.

Comments This function returns the value stored in the `intValue` field for this table item. Certain types of table items display the value stored in `intValue`, and other types display the value pointed to by the `ptr` field. See the `TableItemType` description for details. If the `intValue` was never set for this table item, this function returns 0.

See Also `TblSetItemInt`, `TblGetItemPtr`

TblGetItemPtr

Purpose Return the pointer value stored in a table item

Prototype

```
void *TblGetItemPtr (const TableType *tableP,
Int16 row, Int16 column)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number of the item (zero-based).
-> `column`	Column number of the item (zero-based).

Result Returns the item's pointer value or `NULL` if the item does not have a pointer value. This function may display a fatal message if the `row` or `column` parameter is invalid.

Comments This function returns the value stored in the `ptr` field for this table item. Certain types of table items display the value pointed to by the `ptr`, and other types display the value stored in the `intValue` field. See the `TableItemType` description for details. An application may have set the value of the `ptr` field anyway, even for items that use the `intValue`. This function always returns that value.

Compatibility Implemented only if 3.5 New Feature Set is present. In earlier versions, you can implement this function using the following code:

```
return tableP->items[row * tableP->numColumns +
  column].ptr;
```

See Also `TblSetItemPtr`

TblGetLastUsableRow

Purpose Return the last row in a table that is usable (visible).

Prototype
```
Int16 TblGetLastUsableRow
(const TableType *tableP)
```

Parameters -> `tableP` Pointer to a table object. (See `TableType`.)

Result Returns the row index (zero-based) or `tblUnusableRow` if there are no usable rows.

See Also `TblGetRowData`, `TblGetRowID`



New TblGetNumberOfColumns

Purpose Return the number of columns in a table.

Prototype
```
Int16 TblGetNumberOfColumns
(const TableType *tableP)
```

Parameters -> `tableP` Pointer to a `TableType`.

Result This function returns the number of columns in a table.

Compatibility Implemented only if 4.0 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TblGlueGetNumberOfColumns`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TblGetTopRow`, `TblSetSelection`

TblGetNumberOfRows

Purpose Return the number of rows in a table.

Prototype
```
Int16 TblGetNumberOfRows (const TableType *tableP)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)

Result Returns the maximum number of visible rows in the specified table.

Comments Note that even though you can add and remove rows to and from a table, the value returned by this function does not change. The value returned by this function indicates the maximum number of rows that can be displayed on the screen at one time. It is set when you create the table resource.

TblGetRowData

Purpose Return the data value of the specified row.

Prototype
```
UInt32 TblGetRowData (const TableType *tableP,
Int16 row)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Number of the row (zero-based).

Result Returns the application-specific data stored for this row, if any. Returns 0 if there is no application-specific data value.

This function may display a fatal error message if the `row` parameter is invalid.

See Also `TblFindRowData`, `TblSetRowData`, `TableRowAttrType`

TblGetRowHeight

Purpose Return the height of the specified row.

Prototype
```
Coord TblGetRowHeight (const TableType *tableP,
Int16 row)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Number of the row (zero-based).

Result Returns the height in pixels. This function may display a fatal error message if the `row` parameter is invalid.

See Also `TblGetItemBounds`, `TblSetRowHeight`

TblGetRowID

Purpose Return the ID value of the specified row.

Prototype
```
UInt16 TblGetRowID (const TableType *tableP,
Int16 row)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Number of the row (zero-based).

Result Returns the ID value of the row in the table.

This function may display a fatal error message if the `row` parameter is invalid.

See Also `TblGetRowData`, `TblSetRowID`, `TblFindRowID`, `TableRowAttrType`

TblGetSelection

Purpose Return the row and column of the currently selected table item.

Prototype
```
Boolean TblGetSelection (const TableType *tableP,
Int16 *rowP, Int16 *columnP)
```

Parameters

-> `tableP` Pointer to a table object. (See `TableType`.)

<- `rowP, columnP` The row and column indexes (zero-based) of the currently selected item.

Result Returns `true` if the item is highlighted, `false` if not.

See Also `TblSetRowSelectable`



New TblGetTopRow

Purpose Return the top row visible row of a table.

Prototype
```
Int16 TblGetTopRow(const TableType *tableP)
```

Parameters

-> `tableP` Pointer to a `TableType`.

Result This function returns the top visible row in a table.

Compatibility Implemented only if 4.0 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TblGlueGetToRow`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TblGetNumberOfColumns`, `TblSetSelection`

TblGrabFocus

Purpose Put a table into edit mode.

Prototype

```
void TblGrabFocus (TableType *tableP, Int16 row,
Int16 column)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Current row to be edited (zero-based).
-> `column`	Current column to be edited (zero-based).

Result Returns nothing. This function may display a fatal error message if the table already has the focus or if the `row` or `column` parameter is invalid.

Comments This function puts the table into edit mode and sets the current item to the one at the row and column passed in. An editable field must exist in the coordinates passed to this function.

You must call `FrmSetFocus` before calling this function. `FrmSetFocus` releases the focus from the object that previously had it and sets the form's internal structures. After calling this function, you must call `FldGrabFocus` to display the insertion point in the field. (You can use `TblGetCurrentField` to obtain a pointer to the field.)

For example, the following function from the Address Book application sets the focus in an editable field within a table:

```
static void EditViewRestoreEditState () {
  Int16      row;
  FormPtr    frm;
  TablePtr   table;
  FieldPtr   fld;

  if (CurrentFieldIndex == noFieldIndex)
    return;

  // Find the row that the current field is in.
  table = GetObjectPtr (EditTable);
```

```
    if ( ! TblFindRowID (table,
      CurrentFieldIndex, &row) )
      return;

    frm = FrmGetActiveForm ();
    FrmSetFocus (frm, FrmGetObjectIndex (frm,
      EditTable));
    TblGrabFocus (table, row, editDataColumn);

    // Restore the insertion point position.
    fld = TblGetCurrentField (table);
    FldSetInsPtPosition (fld, EditFieldPosition);
    FldGrabFocus (fld);
  }
```

See Also TblReleaseFocus

TblHandleEvent

Purpose Handle an event for the table.

Prototype

```
Boolean TblHandleEvent (TableType *tableP,
EventType *event)
```

Parameters

-> `tableP`	Pointer to a table object. (See TableType.)
-> `event`	The event to be handled.

Result Returns `true` if the event was handled, `false` if it was not.

Comments Returns `false` if the table is not an editable table.

If the table is editable, this function passes along any keyDownEvent, fldEnterEvent, or menuCmdBarOpenEvent to the currently selected field.

When a fldHeightChangedEvent occurs, this function changes the height of the specified field as indicated by the event. If the field being resized is going to scroll off the bottom of the screen, then instead the table scrolls the rows above it up off the top. Otherwise,

the table is scrolled downward and rows below the current row are scrolled off the bottom as necessary.

Note that the `fldHeightChangedEvent` is only handled for dynamically sized fields. See the descriptions of `FieldAttrType` and `FldMakeFullyVisible` for more information.

When a `penDownEvent` occurs, the table checks to see if the focus is being changed. If it is and the user was previously editing a text field within the table, it saves the data in the table cell using the `TableSaveDataFuncType` callback function, then it enqueues a `tblEnterEvent` with the new row and column that are selected.

When a `tblEnterEvent` occurs, this function tracks the pen until it is lifted. If the pen is lifted within the bounds of the same item it went down in, a `tblSelectEvent` is added to the event queue; if not, a `tblExitEvent` is added to the event queue.

TblHasScrollBar

Purpose Set the `hasScrollBar` attribute in the table. (See `TableAttrType`.)

Prototype

```
void TblHasScrollBar (TableType *tableP,
Boolean hasScrollBar)
```

Parameters

-> `tableP` Pointer to a table object. (See `TableType`.)

-> `hasScrollBar` `true` to set the attribute, `false` to unset it.

Result Returns nothing.

Comments Your application must scroll the table itself and keep track of the scroll values. See the `ListViewUpdateScrollers` function in the Memo example application (`MemoMain.c`) for an example of setting scroll bar values for a table.

Compatibility Implemented only if 2.0 New Feature Set is present.

TblInsertRow

Purpose Insert a row into the table before the specified row.

Prototype
```
void TblInsertRow (TableType *tableP, Int16 row)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row to insert (zero-based).

Result Returns nothing.

Comments The number of rows in a table is the maximum number of rows displayed on the screen. Unlike a multi-line text field, there is no notion of a table that is bigger than the available screen. For this reason, this function does not increase the number of table rows.

Instead of keeping track of a total number of rows in the table and a number of rows displayed on the screen, tables mark any row that isn't currently displayed with a usable value of `false`. (See `TableRowAttrType`.)

The newly inserted row is marked as invalid, unusable, and not masked. If you want to display the newly inserted row, call `TblSetRowUsable` after making sure that the row displays a value and then call `TblRedrawTable` when you are ready to draw the table.

See Also `TblRemoveRow`, `TblSetRowUsable`, `TblSetRowSelectable`

TblMarkRowInvalid

Purpose Mark the row invalid.

Prototype
```
void TblMarkRowInvalid (TableType *tableP,
Int16 row)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number (zero-based).

Result Returns nothing.

Comments After calling this function, call TblRedrawTable to redraw all rows marked invalid.

This function may display a fatal error message if the `row` parameter is invalid.

See Also TblRemoveRow, TblSetRowUsable, TblSetRowSelectable, TblMarkTableInvalid, TblRowInvalid, TableRowAttrType

TblMarkTableInvalid

Purpose Mark all the rows in a table invalid.

Prototype
```
void TblMarkTableInvalid (TableType *tableP)
```

Parameters -> `tableP` Pointer to a table object. (See TableType.)

Result Returns nothing.

Comments After calling this function, you must call TblRedrawTable to redraw all rows.

See Also TblEraseTable, TblRedrawTable, TableRowAttrType

TblRedrawTable

Purpose Redraw the rows of the table that are marked invalid.

Prototype
```
void TblRedrawTable (TableType *tableP)
```

Parameters -> `tableP` Pointer to a table object. (See TableType.)

Result Returns nothing.

Comments This function draws the invalid rows in the table. See the `TableItemType` struct description for more information about how each type of table cell is drawn.

When drawing cells with editable text fields (`textTableItem`, `textWithNoteTableItem`, or `narrowTextTableItem`), this function uses the `TableLoadDataFuncType` callback function to load the text into the table cells. The text field does not retain the text handle that your `TableLoadDataFunc` returns, meaning that you are responsible for freeing the memory that you load into the table.

When drawing `narrowTextTableItem` cells or `customTableItem` cells, this function uses the `TableDrawItemFuncType` callback function to draw the extra pixels after the text or to draw the entire cell.

On color systems, tables are always drawn using the same color as is used for the field background color.

When the user has set the security setting to mask private records, table cells that contain private database records are drawn as shaded cells to obscure the information they contain. You must explicitly tell the table which cells are masked by making the appropriate calls to `TblSetRowMasked` and `TblSetColumnMasked`.

Compatibility Color support and masked private records are only supported in Palm OS version 3.5 and higher.

See Also `TblMarkTableInvalid`, `TblMarkRowInvalid`, `TblDrawTable`

TblReleaseFocus

Purpose Release the focus.

Prototype

```
void TblReleaseFocus (TableType *tableP)
```

Parameters -> `tableP` Pointer to a table object.

Result Returns nothing.

Comments You typically do not call this function yourself. Instead, call FrmSetFocus with an object index of noFocus to notify the form that the table has lost focus. The form code calls TblReleaseFocus for you.

If the current item is a text item, the TableSaveDataFuncType callback function is called to save the text in the currently selected field, the memory allocated for editing is released, and the insertion point is turned off.

Also note that you might have to call FldReleaseFocus if the focus is in an editable text field and that field uses a custom drawing function (TableDrawItemFuncType).

See Also TblGrabFocus

TblRemoveRow

Purpose Remove the specified row from the table.

Prototype
```
void TblRemoveRow (TableType *tableP, Int16 row)
```

Parameters

-> tableP	Pointer to a table object. (See TableType.)
-> row	Row to remove (zero-based).

Result Returns nothing. This function may raise a fatal error message if an invalid row is specified.

Comments The number of rows in the table is not decreased; instead, this row is moved from its current spot to the end of the table and is marked unusable so that it won't be displayed when the table is redrawn.

This function does not visually update the display. To update the display, call TblRedrawTable.

See Also TblInsertRow, TblSetRowUsable, TblSetRowSelectable, TblMarkRowInvalid

TblRowInvalid

Purpose Return whether a row is invalid.

Prototype
```
Boolean TblRowInvalid (const TableType *tableP,
Int16 row)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number (zero-based).

Result Returns `true` if the row is invalid, `false` if it's valid. This function may raise a fatal error message if the `row` parameter is invalid.

Comments Invalid rows need to be redrawn. Use `TblRedrawTable` to do so.

See Also `TblMarkRowInvalid`, `TblMarkTableInvalid`

TblRowMasked

Purpose Return whether a row is masked.

Prototype
```
Boolean TblRowMasked (const TableType *tableP,
Int16 row)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number (zero-based).

Result Returns `true` if the row is masked, `false` otherwise.

Comments Your code should set a row to masked if it contains a private database record and the user has set the display preference for private records to masked. Masked cells are displayed as shaded.

Note that a table cell is not masked unless both its row and column are masked. This allows non-private information in the row item to remain visible. For example, the Datebook application shows the time for a private appointment, but it does not show the description.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also TblSetRowMasked, TblSetColumnMasked, TableRowAttrType, SecSelectViewStatus

TblRowSelectable

Purpose Return whether the specified row is selectable.

Prototype
```
Boolean TblRowSelectable (const TableType *tableP,
Int16 row)
```

Parameters

-> `tableP`	Pointer to a table object. (See TableType.)
-> `row`	Row number (zero-based).

Result Returns `true` if the row is selectable, `false` if it's not.

Comments Rows that are not selectable don't highlight when touched.

See Also TableRowAttrType

TblRowUsable

Purpose Determine whether the specified row is usable.

Prototype
```
Boolean TblRowUsable (const TableType *tableP,
Int16 row)
```

Parameters

-> `tableP`	Pointer to a table object. (See TableType.)
-> `row`	Row number (zero-based).

Result Returns `true` if the row is usable, `false` if it's not.

This function may display a fatal error message if the `row` parameter is invalid.

Comments Rows that are not usable do not display.

See Also TblRowSelectable, TblGetLastUsableRow, TblSetRowUsable

TblSelectItem

Purpose Select (highlight) the specified item. If there is already a selected item, it is unhighlighted.

Prototype

```
void TblSelectItem (TableType *tableP, Int16 row,
Int16 column)
```

Parameters

-> `tableP`	Pointer to a table object. (See TableType.)
-> `row`	Row of the item to select (zero-based).
-> `column`	Column of the item to select (zero-based).

Result Returns nothing.

This function may display a fatal error message if the `column` or `row` parameter point to an item that is not on the screen.

Comments If `row` contains a masked private database record, then the item remains unselected.

This function cannot highlight an entire row; it can only highlight one cell in a row, and it always unhighlights the previously selected table cell. If you want to select an entire row, either create a table that has a single column, or write your own selection code.

If the selected item is a multi-line text field or a text field with a nonstandard height, this function only highlights the top eleven pixels. If you want a larger area highlighted, you must write your own selection code.

See Also TblRowSelectable, TblGetItemBounds, TblGetItemInt

TblSetBounds

Purpose Sets the bounds of a table.

Prototype
```
void TblSetBounds (TableType *tableP,
const RectangleType *rP)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `rP`	Pointer to a `RectangleType` structure that specifies the bounds for the table.

Result Returns nothing.

Compatibility Implemented only if 2.0 New Feature Set is present.

TblSetColumnEditIndicator

Purpose Set the column attribute that controls whether a column highlights when the table is in edit mode.

Prototype
```
void TblSetColumnEditIndicator (TableType *tableP,
Int16 column, Boolean editIndicator)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `column`	Column number (zero based).
-> `editIndicator`	`true` to highlight, `false` to turn off highlight.

Result Returns nothing.

Comments The edit indicator controls whether the item in the column is highlighted when it is selected. By default, text field items have the `editIndicator` value of `true`, and all other table item types have an edit indicator of `false`.

When the table is drawn, only the leftmost contiguous set of items with the edit indicator set are drawn as highlighted. That is, if

columns 1, 2, and 4 have an edit indicator of `true` and column 3 has an edit indicator of `false`, only the items in column 1 and 2 are drawn as highlighted when selected. Column 4 items are not drawn as highlighted.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `TableColumnAttrType`

TblSetColumnMasked

Purpose Set whether the column is masked.

Prototype

```
void TblSetColumnMasked (TableType *tableP,
Int16 column, Boolean masked)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `column`	Column number (zero-based).
-> `masked`	`true` to have the column be masked, `false` otherwise.

Result Returns nothing.

Comments Masked cells are displayed as shaded. You should set a column to masked if its contents should be hidden when it contains information from a private database record and the user has set the display preference for private records to masked.

A table cell is not masked unless both its row and column are masked. This allows non-private information in the row item to remain visible. For example, the Datebook application shows the time for a private appointment, but it does not show the description.

Because the number of columns is static, you only need to call this function once per column when you first set up the table. The masked attribute on the row will determine if the contents of the table cell are actually displayed as shaded.

Compatibility Implemented only if 3.5 New Feature Set if present.

See Also TblRowMasked, TblSetRowMasked, TableColumnAttrType, SecSelectViewStatus

TblSetColumnSpacing

Purpose Set the spacing after the specified `column`.

Prototype
```
void TblSetColumnSpacing (TableType *tableP,
Int16 column, Coord spacing)
```

Parameters

-> `tableP`	Pointer to a table object. (See TableType.)
-> `column`	Column number (zero-based).
-> `spacing`	Spacing after the column in pixels.

Result Returns nothing.

This function may display a fatal error message if the `column` parameter is invalid.

See Also TblSetColumnUsable, TableColumnAttrType

TblSetColumnUsable

Purpose Set a column in a table to usable or unusable.

Prototype
```
void TblSetColumnUsable (TableType *tableP,
Int16 column, Boolean usable)
```

Parameters

-> `tableP`	Pointer to a table object. (See TableType.)
-> `column`	Column number (zero-based).
-> `usable`	`true` for usable or `false` for not usable.

Result Returns nothing.

This function may display a fatal error message if the `column` parameter is invalid.

Comments Columns that are not usable do not display.

See Also `TblMarkRowInvalid`, `TableColumnAttrType`

TblSetColumnWidth

Purpose Set the width of the specified column.

Prototype

```
void TblSetColumnWidth (TableType *tableP,
Int16 column, Coord width)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `column`	Column number (zero-based).
-> `width`	Width of the column (in pixels).

Result Returns nothing.

This function may display a fatal error message if the `column` parameter is invalid.

See Also `TblGetColumnWidth`, `TableColumnAttrType`

TblSetCustomDrawProcedure

Purpose Set the custom draw callback procedure for the specified column.

Prototype

```
void TblSetCustomDrawProcedure (TableType *tableP,
Int16 column, TableDrawItemFuncPtr drawCallback)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `column`	Column number.
-> `drawCallback`	Callback function.

Result Returns nothing.

Comments The custom draw callback function is used to draw table items with a `TableItemStyleType` of `customTableItem`. See the `TableItemType` description for more information.

This function may display a fatal error message if the `column` parameter is invalid.

See Also `TableDrawItemFuncType`, `TblDrawTable`, `TableColumnAttrType`

TblSetItemFont

Purpose Set the font used to display a table item.

Prototype
```
void TblSetItemFont (TableType *tableP, Int16 row,
Int16 column, FontID fontID)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number of the item (zero-based).
-> `column`	Column number of the item (zero-based).
-> `fontID`	ID of the font to be used.

Result Returns nothing.

Comments This function sets the value stored in the `fontID` field for this table item. Only certain types of table items use the font specified by the `fontID` field when they are displayed. The `TableItemType` description specifies what font is used to display each type of table item. It is not an error to set the `fontID` for a table item that does not use it.

This function may display a fatal error message if the `row` or `column` parameter specifies a row or column that is not on the screen.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `TblGetItemFont`

TblSetItemInt

Purpose Set the integer value of the specified item.

Prototype
```
void TblSetItemInt (TableType *tableP, Int16 row,
Int16 column, Int16 value)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number of the item (zero-based).
-> `column`	Column number of the item (zero-based).
-> `value`	Any byte value (an integer).

Result Returns nothing.

This function may display a fatal error message if the `row` or `column` parameter is invalid.

Comments You typically use this function when setting up and initializing a table for the first time to set the value of each table cell.

This function sets the value stored in the `intValue` field for this table item. Certain types of table items display the value stored in `intValue`, and other types display the value pointed to by the `ptr` field. See the `TableItemType` description for details. If you set the `intValue` of an item that displays its `ptr` value, it is not an error. An application can store whatever value it wants in the `intValue` field; however, be aware that this has nothing to do with the value displayed by such a table cell.

See Also `TblGetItemInt`, `TblSetItemPtr`

TblSetItemPtr

Purpose Set the item to the specified pointer value.

Prototype
```
void TblSetItemPtr (TableType *tableP, Int16 row,
Int16 column, void *value)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number of the item (zero-based).
-> `column`	Column number of the item (zero-based).
-> `value`	Pointer to data to display in the table item.

Result Returns nothing.

This function may display a fatal error message if the `row` or `column` parameter is invalid.

Comments This function sets the value stored in the `ptr` field for this table item. Certain types of table items display the value pointed to by `ptr`, and other types display the value stored in the `intValue` field. See the `TableItemType` description for details. If you set the `ptr` of an item that displays its `intValue`, it is not an error. An application can store whatever value it wants in the `ptr` field; however, be aware that this has nothing to do with the value displayed by such a table cell.

See Also `TblGetItemPtr`, `TblSetItemInt`

TblSetItemStyle

Purpose Set the type of item to display; for example, text, numbers, dates, and so on.

Prototype
```
void TblSetItemStyle (TableType *tableP,
Int16 row, Int16 column, TableItemStyleType type)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number of the item (zero-based).
-> `column`	Column number of the item (zero-based).
-> `type`	The type of item, such as an editable text field or a check box. See `TableItemType` for a list of possible values.

Result

Returns nothing.

This function may display a fatal error message if the `row` or `column` parameter is invalid.

Comments

You typically use this function when first setting up and initializing a table; you do not dynamically change item styles.

Follow this function with a call to either `TblSetItemInt` or `TblSetItemPtr` to set the value displayed by the table item. You should call one or the other of these functions depending on the type of table item you specified. See the table in the `TableItemType` description for details.

Note also that a table column always contains items of the same type. For example, you might initialize a table using this code:

```
for (row = 0; row < rowsInTable; row++) {
  TblSetItemStyle (table, row, completedColumn,
    checkboxTableItem);
  TblSetItemStyle (table, row, priorityColumn,
    numericTableItem);
  TblSetItemStyle (table, row, descColumn,
    textTableItem);
  TblSetItemStyle (table, row, dueDateColumn,
    customTableItem);
  TblSetItemStyle (table, row, categoryColumn,
    customTableItem);
}
```

See Also

`TblSetCustomDrawProcedure`

TblSetLoadDataProcedure

Purpose Set the load-data callback procedure for the specified column.

Prototype
```
void TblSetLoadDataProcedure (TableType *tableP,
Int16 column,
TableLoadDataFuncPtr loadDataCallback)
```

Parameters

-> `tableP` Pointer to a table object. (See `TableType`.)

-> `column` Column number (zero-based).

-> `loadDataCallback` Callback procedure. See `TableLoadDataFuncType`.

Result Returns nothing.

Comments The callback procedure is used to load the data values of a table item. See the `TableLoadDataFuncType` for more information on writing the callback function.

You typically use this function when first setting up and initializing a table.

See Also `TblSetCustomDrawProcedure`

TblSetRowData

Purpose Set the data value of the specified row. The data value is a placeholder for application-specific values.

Prototype
```
void TblSetRowData (TableType *tableP, Int16 row,
UInt32 data)
```

Parameters

-> `tableP` Pointer to a table object. (See `TableType`.)

-> `row` Row number (zero-based).

	-> `data`	Application-specific data value to store for this row. For example, the Datebook and ToDo applications use this field to store the unique ID of the database record displayed by this row.

Result Returns nothing.

This function may display a fatal error message if the `row` parameter is invalid.

See Also `TblGetRowData`, `TblFindRowData`

TblSetRowHeight

Purpose Set the height of the specified row.

Prototype

```
void TblSetRowHeight (TableType *tableP,
Int16 row, Coord height)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number (zero-based).
-> `height`	New height in pixels.

Result Returns nothing.

This function may display a fatal error message if the `row` parameter is invalid.

See Also `TblGetRowHeight`, `TblSetRowStaticHeight`

TblSetRowID

Purpose Set the ID value of the specified row.

Prototype

```
void TblSetRowID (TableType *tableP, Int16 row,
UInt16 id)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number (zero-based).
-> `id`	ID to identify a row.

Result Returns nothing.

This function may display a fatal error message if the `row` parameter is invalid.

See Also `TblGetRowID`, `TblFindRowID`, `TableRowAttrType`

TblSetRowMasked

Purpose Set a row in a table to masked or unmasked.

Prototype

```
void TblSetRowMasked (TableType *tableP,
Int16 row, Boolean masked)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number (zero-based).
-> `masked`	`true` to have the row be masked, `false` otherwise.

Result Returns nothing.

Comments Masked cells are displayed as shaded. You should call this function before drawing or redrawing the table. If a table row contains a private database record and the user has set the display preference for private records to masked, you must call this function on that row. For example:

```
UInt16 attr;
privateRecordViewEnum privateRecordStatus;
Boolean masked;

privateRecordStatus = (privateRecordViewEnum)
  PrefGetPreference(prefShowPrivateRecords);
....
```

```
DmRecordInfo (ToDoDB, recordNum, &attr, NULL,
  NULL);
masked = ((attr & dmRecAttrSecret) &&
  (privateRecordStatus == maskPrivateRecords));
TblSetRowMasked(tableP, row, masked);
```

Note that a table cell is not masked unless both its row and column are masked. This allows non-private information in the row item to remain visible. For example, the Datebook application shows the time for a private appointment, but it does not show the description.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also TblRowMasked, TblSetColumnMasked, TableRowAttrType, SecSelectViewStatus

TblSetRowSelectable

Purpose Set a row in a table to selectable or nonselectable.

Prototype

```
void TblSetRowSelectable (TableType *tableP,
Int16 row, Boolean selectable)
```

Parameters

-> `tableP`	Pointer to a table object. (See TableType.)
-> `row`	Row number (zero-based).
-> `selectable`	`true` or `false`.

Result Returns nothing.

This function may display a fatal error message if the `row` parameter is invalid.

Comments Rows that are not selectable don't highlight when touched.

See Also TblRowSelectable, TblSetRowUsable, TableRowAttrType

TblSetRowStaticHeight

Purpose Set the static height attribute of a row.

Prototype
```
void TblSetRowStaticHeight (TableType *tableP,
Int16 row, Boolean staticHeight)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number (zero-based).
-> `staticHeight`	`true` to set the static height, `false` to unset it.

Result Nothing.

This function may display a fatal error message if the `row` parameter is invalid.

Comments A row that has its static height attribute set will not expand or contract the height of the row as text is added or removed from a text item.

Compatibility Implemented only if 2.0 New Feature Set is present.

TblSetRowUsable

Purpose Set a row in a table to usable or unusable. Rows that are not usable do not display.

Prototype
```
void TblSetRowUsable (TableType *tableP,
Int16 row, Boolean usable)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number (zero-based).
-> `usable`	`true` or `false`.

Result Returns nothing.

This function may display a fatal error message if the `row` parameter is invalid.

See Also `TblRowUsable`, `TblSetRowSelectable`

TblSetSaveDataProcedure

Purpose Set the save-data callback procedure for the specified column.

Prototype

```
void TblSetSaveDataProcedure (TableType *tableP,
Int16 column,
TableSaveDataFuncPtr saveDataCallback)
```

Parameters

-> `tableP` Pointer to a table object. (See `TableType`.)

-> `column` Column number (zero-based).

-> `saveDataCallback` Callback function. See `TableSaveDataFuncType`.

Result Returns nothing.

This function may display a fatal error message if the `column` parameter is invalid.

Comments The callback procedure is called when the table object determines the data of a text object needs to be saved.

See Also `TblSetCustomDrawProcedure`



New TblSetSelection

Purpose Set a table item.

Prototype
```
void TblSetSelection (TableType *tableP,
Int16 row, Int16 column)
```

Parameters

-> `tableP`	Pointer to a `TableType`.
-> `row`	Table row.
-> `column`	Table column.

Result Returns nothing.

Comments This function sets a table item, determined by the row and column arguments, as the current selection. `TblDrawTable` must be called afterwards to update the UI.

Compatibility Implemented only if 4.0 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TblGlueSetSelection`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TblGetNumberOfColumns`, `TblGetTopRow`

TblUnhighlightSelection

Purpose Unhighlight the currently selected item in a table.

Prototype
```
void TblUnhighlightSelection (TableType *tableP)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)

Result Returns nothing.

Application-Defined Functions

TableDrawItemFuncType

Purpose Draw a custom table item.

Prototype

```
void TableDrawItemFuncType (void *tableP,
Int16 row, Int16 column, RectangleType *bounds)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number of the item to be drawn (zero-based).
-> `column`	Column number of the item to be drawn (zero-based).
-> `bounds`	The area of the screen in which the item is to be drawn.

Result Returns nothing.

Comments This function is called during `TblDrawTable` and `TblRedrawTable`.

You implement a custom drawing function when your table contains items of type `customTableItem` (to draw the entire item) or `narrowTextTableItem` (to draw whatever is required in the space between the text and the right edge of the table cell).

You may implement a custom drawing function to include any style of information in the table. If you don't like the way a predefined item is drawn, you may prefer to use a `customTableItem` instead. For example, if you want to include a date in your table but you want it to show the year as well as the month and day, you should implement a custom drawing function.

See Also `TblSetCustomDrawProcedure`, `TableItemType`

TableLoadDataFuncType

Purpose

Load data into a column.

Prototype

```
Err TableLoadDataFuncType (void *tableP,
Int16 row, Int16 column, Boolean editable,
MemHandle *dataH, Int16 *dataOffset,
Int16 *dataSize, FieldPtr fld)
```

Parameters

-> `tableP`	Pointer to a table object. (See `TableType`.)
-> `row`	Row number of the table item to load.
-> `column`	Column number of the table item to load.
-> `editable`	If `true`, the table is currently being edited. If `false`, the table is being drawn but not necessarily being edited.
<- `dataH`	Unlocked handle of a block containing a null-terminated text string.
<- `dataOffset`	Offset from start of block to start of the text string.
<- `dataSize`	Allocated size of text string, **not** the string length.
-> `fld`	Pointer to the text field in this table cell.

Result

Returns 0 upon success or an error if unsuccessful.

Comments

This function is called in two cases: when a text field item is being drawn (`TblDrawTable` or `TblRedrawTable`) and when a text field item is being selected (part of `TblHandleEvent`'s handling of `tblEnterEvent`). If this function returns an error (any nonzero value) and the item is being selected, then the item is not selected and the table's `editing` attribute is set to `false`.

You return the same values for `dataH`, `dataOffset`, and `dataSize` that you would pass to `FldSetText`. That is, you can use this function to point the table cell's text field to a string in a database record so that you can edit that string directly using text field routines. To do so, return the handle to a database record in

dataH, the offset from the start of the record to the start of the string in dataOffset, and the allocated size of the string in dataSize.

The handle that you return from this function is assumed to contain a null-terminated string starting at dataOffset bytes in the memory chunk. The string should be between 0 and dataSize - 1 bytes in length.

As with FldSetText, you are responsible for freeing the memory associated with the dataH parameter. You can do so in the TableSaveDataFuncType function, but it is only called for a cell that has been edited. For non-editable text cells or text cells that are editable but were never selected, free the memory when you close the form.

The fld pointer passed to your function has already been initialized with default values by the table code. If you want to override a field's attributes (for example, if you want to change the underline mode) you can do so in this function.

See Also TblDrawTable, TblHandleEvent, TableLoadDataFuncType

TableSaveDataFuncType

Purpose Save the data associated with a text field.

Prototype

```
Boolean TableSaveDataFuncType (void *tableP,
Int16 row, Int16 column);
```

Parameters

-> tableP	Pointer to a table object. (See TableType.)
-> row	Row number of the table item to load.
-> column	Column number of the table item to load.

Result Return true if the table should be redrawn, or false if the table does not need to be redrawn.

Comments This is called before the memory associated with the currently selected text field in a table cell is freed. Implement this function if you need to do any special cleanup before this memory is freed.

This function is called only when the currently selected editable text field is releasing the focus. You can use `TblGetCurrentField` to retrieve a pointer to this field. It is called only on the currently selected field, not on any other fields in the table.

Note that the table manager already disassociates the memory handle from the text field for you so that the memory associated with your data is not freed when the field is freed. The table manager also calls `FldCompactText` for you.

If the text handle you returned in your `TableLoadDataFuncType` callback points to a string on the dynamic heap, you should implement this callback function to store or free the handle. You can use `FldGetTextHandle` to obtain the handle.

If you return `true` from this function, `TblRedrawTable` is called. You should mark invalid any table rows that you want redrawn before returning.

See Also `TblSetSaveDataProcedure`

21

UI Color List

This chapter provides information about the UI color list by discussing the following topics:

- UI Color Data Types
- UI Color Functions

The header file `UIColor.h` declares the API that this chapter describes. For more information on the color list, see "Color and Grayscale Support" on page 128 in the *Palm OS Programmer's Companion*, vol. I.

UI Color Data Types

UIColorTableEntries

The `UIColorTableEntries` enum declares symbolic color constants for the various UI elements.

Do not confuse the UI color list with the system color table. The **system color table** (or **system palette**) defines all available colors for the display or draw window, whether they are in use or not. The **UI color list** defines the colors used to draw the interface objects.

```
typedef enum UIColorTableEntries {
  UIObjectFrame = 0,
  UIObjectFill,
  UIObjectForeground,
  UIObjectSelectedFill,
  UIObjectSelectedForeground,

  UIMenuFrame,
  UIMenuFill,
  UIMenuForeground,
  UIMenuSelectedFill,
  UIMenuSelectedForeground,
```

```
  UIFieldBackground,
  UIFieldText,
  UIFieldTextLines,
  UIFieldCaret,
  UIFieldTextHighlightBackground,
  UIFieldTextHighlightForeground,
  UIFieldFepRawText,
  UIFieldFepRawBackground,
  UIFieldFepConvertedText,
  UIFieldFepConvertedBackground,
  UIFieldFepUnderline,

  UIFormFrame,
  UIFormFill,

  UIDialogFrame,
  UIDialogFill,

  UIAlertFrame,
  UIAlertFill,

  UIOK,
  UICaution,
  UIWarning,

  UILastColorTableEntry
} UIColorTableEntries;
```

Field Descriptions

`UIObjectFrame`	Color for the border of user interface objects (such as command buttons, push buttons, selector triggers, menus, arrows checkboxes, and other controls).
`UIObjectFill`	The background color for a solid or "filled" user interface object. Note that UI objects in tables use the `UIField`... colors instead of the `UIObject`... colors.
`UIObjectForeground`	The color for foreground items (such as labels or graphics) in a user interface object.
`UIObjectSelectedFill`	The background color of the currently selected user interface object, whether that object is solid or not.
`UIObjectSelectedForeground`	The color of foreground items in a selected user interface object.
`UIMenuFrame`	The color of the border around the menu.
`UIMenuFill`	The background color of a menu item.
`UIMenuForeground`	The color of the menu's text.
`UIMenuSelectedFill`	The background color of a selected menu item.
`UIMenuSelectedForeground`	The color of the text of a selected menu item.
`UIFieldBackground`	The background color of an editable text field.
`UIFieldText`	The color of the text in the editable field.
`UIFieldTextLines`	The color of underlines in an editable field.
`UIFieldCaret`	The color of the cursor in an editable text field.
`UIFieldTextHighlightBackground`	The background color for selected text in an editable text field.

`UIFieldTextHighlightForeground`	The color of the selected text in an editable text field.
`UIFieldFepRawText`	Color used for unconverted text in the inline conversion area when a FEP is used as a text input method (for example, on Japanese devices). If the FEP colors are identical to field colors, unconverted text has a solid underline.
`UIFieldFepRawBackground`	The background color for unconverted text in the inline conversion area when a FEP is used as a text input method. If the FEP colors are identical to field colors, unconverted text has a solid underline.
`UIFieldFepConvertedText`	Color used for converted text in the inline conversion area when a FEP is used as a text input method (for example, on Japanese devices). If the FEP colors are identical to field colors, converted text has a double-thick underline.
`UIFieldFepConvertedBackground`	The background color used for converted text in the inline conversion area. If the FEP colors are identical to field colors, converted text has a double-thick underline.
`UIFieldFepUnderline`	The color used for underlines in the inline conversion area.
`UIFormFrame`	The color of the border and titlebar on a form.
`UIFormFill`	The background color of a form. White is recommended for this value.
`UIDialogFrame`	The color of a border and titlebar on a modal form.

`UIDialogFill`	The background color of a modal form.
`UIAlertFrame`	The color of the border and titlebar on an alert panel.
`UIAlertFill`	The background color of an alert panel.
`UIOK`	The color for an informational icon.
`UICaution`	The color for a caution icon.
`UIWarning`	The color for a warning icon. Palm OS® does not currently use the `UIOK`, `UICaution`, and `UIWarning` constants.
`UILastColorTableEntry`	Placeholder to indicate end of enum.

Compatibility Implemented only if 3.5 New Feature Set is present.

UI Color Functions

UIColorGetTableEntryIndex

Purpose Return the index value for a UI color for the current system palette.

Prototype
```
IndexedColorType UIColorGetTableEntryIndex
(UIColorTableEntries which)
```

Parameters

-> `which`	One of the symbolic color constants. See `UIColorTableEntries`.

Result Returns the system color table index of the color used for the specified symbolic color.

Comments One way to find out the indexes of all the colors that the OS is using is to loop through the UI color list, calling `UIColorGetTableEntryIndex` for each slot, and keep a list (excluding duplicates).

```
IndexedColorType
  colorsUsed[UILastColorTableEntry];
UInt16 numColors = 0;
...
for (i = 0; i < UILastColorTableEntry; i++) {
  IndexedColorType currentColor;
  Boolean isNew = true;

  currentColor = UIColorGetTableEntryIndex(i);

  for (j = 0; ((j < numColors) && isNew); j++)
    if (colorsUsed[j] == currentColor)
      isNew = false; /* exit loop */
  if (isNew) {
    numColors++;
    colorsUsed[j] = currentColor;
  }
```

To get the RGB values of the colors, do the same thing but call UIColorGetTableEntryRGB.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also IndexedColorType, WinIndexToRGB

UIColorGetTableEntryRGB

Purpose Return the RGB value for the UI color.

Prototype

```
void UIColorGetTableEntryRGB
(UIColorTableEntries which, RGBColorType *rgbP)
```

Parameters

-> which	One of the symbolic color constants. See UIColorTableEntries.
<- rgbP	Pointer to an RGB color value corresponding to the current color used for the symbolic color. (See RGBColorType.)

Result Returns nothing.

Comments In general, it is more efficient to work with indexed color entries instead of RGB color entries.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also UIColorGetTableEntryIndex, WinRGBToIndex

UIColorSetTableEntry

Purpose Change a value in the UI color list.

Prototype

```
Err UIColorSetTableEntry
(UIColorTableEntries which,
const RGBColorType *rgbP)
```

Parameters

-> `which`	One of the symbolic color constants. See UIColorTableEntries.
-> `rgbP`	The RGB value of the color that should be used for the specified UI object. (See RGBColorType.)

Result Returns 0 upon success.

Comments Sets the value of a UI color entry to the passed RGB value. Updates the index for that UI color entry to the current best fit for that RGB value according to the palette used by the current draw window.

It is best to use this function only if the draw window is currently onscreen. Otherwise, the best-fit algorithm may choose a color that is not available on the current screen.

See Also WinIndexToRGB, UIColorGetTableEntryIndex, UIColorGetTableEntryRGB

22

UI Controls

This chapter describes the UI controls API as declared in `UIControls.h`.

UI Control Functions

UIBrightnessAdjust

Purpose Displays the brightness adjust dialog.

Prototype

```
void UIBrightnessAdjust()
```

Parameters None

Result Returns nothing.

Comments On hardware that supports a brightness setting, this function displays a dialog that allows the user to change the brightness level. On hardware that has a backlight, this function toggles the backlight.

Compatibility Implemented only if 3.5 New Feature Set is present.

UIContrastAdjust

Purpose Displays the contrast adjust dialog (currently only available on the Palm V™ Connected Organizer).

Prototype

```
void UIContrastAdjust()
```

Parameters None.

Result Returns nothing.

Compatibility This function was renamed from `ContrastAdjust` to `UIContrastAdjust` in Palm OS® release 3.5. The `ContrastAdjust` function is available if 3.1 New Feature Set is present.

UIPickColor

Purpose Displays a dialog that allows the user to choose a color.

Prototype

```
Boolean UIPickColor (IndexedColorType *indexP,
RGBColorType *rgbP, UIPickColorStartType start,
const Char *titleP, const Char *tipP)
```

Parameters

<-> `indexP` Index value of the selected color. (See `IndexedColorType`.) Upon entry, this points to the index value of the color initially selected. Upon return, this points to the index value of the color the user selected. Pass `NULL` to not set or return this value.

<-> `rgbP` RGB value of the selected color. (See `RGBColorType`.) Upon entry, this points to the RGB value of the color initially selected when the dialog is displayed. Upon return, this points to the RGB value that the user selected. Pass `NULL` to not set or return this value.

-> `start`	Either `UIPickColorStartPalette` to display the system palette as a series of color squares or `UIPickColorStartRGB` to display individual sliders for the red, green, and blue values. This parameter is only used if both `indexP` and `rgbP` are not `NULL`.
-> `titleP`	String to display as the title of the dialog. Specify `NULL` to use the default title, which is "Pick Color."
-> `tipP`	Not used.

Result Returns `true` if a new color was selected, `false` otherwise.

Comments Use this function to allow users to choose a color used in your user interface. (The system never calls `UIPickColor`.)

This function can display two versions of the dialog: palette or RGB. The palette version of the dialog displays a series of squares, each containing a different color defined on the system palette. The `indexP` value contains the index of the square that is initially selected.

The RGB version of the dialog displays three sliders that allow the user to select the level of red, green, and blue in the color. The `rgbP` parameter contains the red, green, and blue values initially shown in the dialog. The sliders only allow values that are defined in the current system color table.

If `indexP` is initially `NULL`, only the RGB dialog is displayed. Similarly, if `rgbP` is `NULL`, only the palette version is displayed. If both parameters are non-`NULL`, the system adds a pull-down list that allows the user to switch between the palette dialog and the RGB dialog, and the `start` parameter controls which version of the dialog is initially shown. In this case, both `indexP` and `rgbP` contain the value of the user-selected color upon return.

Palm OS 3.5 supports a maximum of 256 colors. The number of possible RGB colors greatly exceeds this amount. For this reason, the chosen RGB may not have an exact match. If this is the case, the `indexP` parameter (if not `NULL`) contains the closest match using a luminance best-fit if the color lookup table is entirely grayscale (red,

green, and blue values for each entry are identical), or a shortest-distance fit in the RGB space is used if the palette contains colors.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinSetBackColor`, `WinSetForeColor`, `WinSetTextColor`, `UIColorSetTableEntry`

23

Miscellaneous User Interface Functions

This chapter provides descriptions of miscellaneous user interface functions. It covers the following topics:

- Miscellaneous User Interface Data Structures
- Miscellaneous User Interface Functions

You can find declarations for the functions described in this chapter in the header files `AppLaunchCmd.h`,`PhoneLookup.h`, and `UIResources.h`.

Miscellaneous User Interface Data Structures

The `PhoneNumberLookupCustom` function uses these data structures to look up contact information based upon the current cursor position.

AddressLookupFields

The `AddressLookupFields` enum specifies the fields you can search by and the corresponding fields to return using the `field1` and `field2` elements of the `AddrLookupParamsType` structure. For both `field1` and `field2` pass one of the values up to, but not including, `addrLookupFieldCount`.

```
typedef enum {
  addrLookupName,
  addrLookupFirstName,
  addrLookupCompany,
  addrLookupAddress,
  addrLookupCity,
  addrLookupState,
  addrLookupZipCode,
```

```
  addrLookupCountry,
  addrLookupTitle,
  addrLookupCustom1,
  addrLookupCustom2,
  addrLookupCustom3,
  addrLookupCustom4,
  addrLookupNote,
  addrLookupWork,
  addrLookupHome,
  addrLookupFax,
  addrLookupOther,
  addrLookupEmail,
  addrLookupMain,
  addrLookupPager,
  addrLookupMobile,
  addrLookupSortField,
  addrLookupListPhone,
  addrLookupFieldCount,

  addrLookupNoField = 0xff
} AddressLookupFields;
```

AddrLookupParamsType

Pass this structure to PhoneNumberLookupCustom to precisely control the phone number lookup dialog and paste process.

```
typedef struct {
  Char *title;
  Char *pasteButtonText;
  Char lookupString[addrLookupStringLength];
  AddressLookupFields field1;
  AddressLookupFields field2;
  Boolean field2Optional;
  Boolean userShouldInteract;
  Char *formatStringP;
  MemHandle resultStringH;
  UInt32 uniqueID;
} AddrLookupParamsType;

typedef AddrLookupParamsType
*AddrLookupParamsPtr;
```

Value Descriptions

`title`	Title to appear in the title bar. Supply `NULL` to use the default title.
`pasteButtonText`	Text to appear in paste button. Supply `NULL` to use the default, "paste".
`lookupString`	Buffer containing the string to look up. If the string matches only one record, that record is used without presenting the lookup dialog to the user. `PhoneNumberLookup` and `PhoneNumberLookupCustom` both set this field based upon the current selection or cursor position.
`field1`	Field to search by. This field appears on the left side of the lookup dialog. If the field is the sort field, the search is performed using a binary search. If the field isn't the sort field, searching is performed by a linear search, which can be slow. Supply one of the values in the `AddressLookupFields` enum to specify the field to search by.
`field2`	Field to display on the right. Often displays some information about the person. If it is a phone field and a record has multiple instances of the phone type then the person appears once per instance of the phone type. Either `field1` or `field2` may be a phone field, but not both. Supply one of the values in the `AddressLookupFields` enum to specify the field to display.
`field2Optional`	A value of `true` means that the record need not have `field2` in order to be listed. A value of `false` indicates that `field2` is required in order for the record to be listed.

`userShouldInteract` A value of `true` forces the user to resolve non-unique lookups. A `false` value means a non-unique and complete lookup causes `resultStringH` to be set to `NULL` and `uniqueID` to be set to 0.

`formatStringP` Controls the format of the paste string. All characters in the format string are literal unless they identify a field (signified by a caret (^) followed by the field name). For example, the format string "^first - ^home" might result in "Roger - 123-4567". Allowable field names are:

- name
- first
- company
- address
- city
- state
- zipcode
- country
- title
- custom1
- custom2
- custom3
- custom4
- work
- home
- fax
- other
- email
- main
- pager
- mobile
- listname

`resultStringH`	If there is a format string, a result string is allocated on the dynamic heap and its handle is returned here.
`uniqueID`	The unique ID of the found record, or 0 if none was found, is returned here.

Miscellaneous User Interface Functions

PhoneNumberLookup

Purpose Calls the Address Book application to look up a phone number.

Prototype

```
void PhoneNumberLookup (FieldType *fldP)
```

Parameters

`-> fldP`	Field object in which the text to match is found.

Result Nothing returned; it's locked.

Comments This function displays the user's phone list and inserts the chosen name and number (or company name, name, and number, if that's how the user's Address Book preferences indicate that the phone list should be sorted) into the specified field. When displaying the phone list, `PhoneNumberLookup` scrolls the list to that entry that best matches the supplied field. The match compares the field contents against the name or company name (depending on the user's preferences) as follows:

- If the field contains selected text, `PhoneNumberLookup` tries to match against the selected text. The selected text is then replaced with the text of the chosen address list entry.
- If there is no selected text in the field, `PhoneNumberLookup` matches against the word in which the cursor lies (the match will take place if the cursor is at the beginning, the end, or within a word). The matched word is replaced with the text of the chosen address list entry.

- If the cursor does not lie within or adjoin a word, `PhoneNumberLookup` displays the address list starting at the first entry, and the text of the chosen entry is inserted at the current position within the text field.

If the user chooses Cancel when the address list is displayed, the field contents are left unaltered. The paste operation takes place through the clipboard so that Undo can be used to restore the field to its previous state.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `PhoneNumberLookupCustom`



New PhoneNumberLookupCustom

Purpose Calls the Address Book application to look up a phone number.

Prototype

```
void PhoneNumberLookupCustom (FieldType *fldP,
AddrLookupParamsType *params,
Boolean useClipboard)
```

Parameters

`-> fldP` Field object in which the text to match is found.

`<-> params` A structure that allows full control over the find dialog and the format of the resulting paste string. See `AddrLookupParamsType` for a description of the fields in this structure.

`-> useClipboard` If true, `PhoneNumberLookupCustom` pastes the result into the field through the clipboard, thereby enabling undo.

Result Nothing returned; it's locked.

Comments

This function displays two fields from each record in the user's address list and inserts a formatted string based upon fields in the chosen record into the specified field. When displaying the address list, `PhoneNumberLookupCustom` scrolls the list to that entry that best matches the supplied field. The match compares the field contents against the field specified in the `params` structure's `field1` element as follows:

- If the field contains selected text, `PhoneNumberLookup` tries to match against the selected text. The selected text is then replaced with the text of the chosen address list entry.
- If there is no selected text in the field, `PhoneNumberLookup` matches against the word in which the cursor lies (the match will take place if the cursor is at the beginning, the end, or within a word). The matched word is replaced with the text of the chosen address list entry.
- If the cursor does not lie within or adjoin a word, `PhoneNumberLookup` displays the address list starting at the first entry, and the text of the chosen entry is inserted at the current position within the text field.

PhoneNumberLookupCustom copies the portion of the field used for the search—the selected text or the word in which the cursor lies—into the `lookupString` field in the `params` structure prior to replacing that part of the field with the user-selected entry.

If the user chooses Cancel when the address list is displayed, the field contents are left unaltered. Depending on the value of the `useClipboard` parameter, the paste operation can take place through the clipboard so that Undo can be used to restore the field to its previous state.

Compatibility

Implemented only if 4.0 New Feature Set is present.

ResLoadConstant

Purpose Load a constant from a `'tint'` resource and return its value.

Prototype

```
UInt32 ResLoadConstant (UInt16 rscID)
```

Parameters

-> `rscID` The ID of the `'tint'` resource (symbolically named `constantRscType`) to load.

Result The four-byte value of the constant in the resource, or 0 if the resource could not be found. The return value may be cast as necessary.

Comments Use this function to load constant values that are stored as `'tint'` resources. (All open resource databases are searched for the resource ID you specify.) You should store a constant value as a resource when its value changes depending on the locale.

As an example, consider the maximum length of the Alarm Sound trigger label in the Datebook application's preferences panel. The list displayed by this trigger uses the localized name for each sound stored in the system. Because localized names are used, the maximum length that the Datebook application allows for the label differs depending on the current locale. The maximum length is stored as a resource constant so that each overlay database can specify a different value for the constant.

Compatibility Implemented only if 3.5 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS®, link with the PalmOSGlue library and call `ResGlueLoadConstant`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `DmGetResource`, `DmGet1Resource`

ResLoadForm

Purpose Copy and initialize a form resource. The structures are complete except pointers updating. Pointers are stored as offsets from the beginning of the form.

Prototype
```
void* ResLoadForm (UInt16 rscID)
```

Parameters

`-> rscID`	The resource ID of the form.

Result The handle of the memory block that the form is in, since the form structure begins with the `WindowType`, this is also a `WinHandle`.

ResLoadMenu

Purpose Copy and initialize a menu resource. The structures are complete except pointers updating. Pointers are stored as offsets from the beginning of the menu.

Prototype
```
void* ResLoadMenu (UInt16 rscID)
```

Parameters

`-> rscID`	The resource ID of the menu.

Result The handle of the memory block that the form is in, since the form structure begins with the `WindowType` this is also a `WinHandle`.



Part II:
System Management

24

Alarm Manager

This chapter provides reference material for the alarm manager:

- Alarm Manager Functions
- Application-Defined Functions

The alarm manager API is declared in the header file `AlarmMgr.h`.

For more information on the Alarm Manager, see the section "Alarms" in the *Palm OS Programmer's Companion*, vol. I.

Alarm Manager Functions

AlmGetAlarm

Purpose Return the date and time for the application's alarm, if one is set.

Prototype

```
UInt32 AlmGetAlarm (UInt16 cardNo, LocalID dbID,
UInt32* refP)
```

Parameters

-> `cardNo`	Number of the storage card on which the application resides.
-> `dbID`	Local ID of the application.
<- `refP`	The alarm's reference value is returned here. This value is passed to the application when the alarm is triggered.

Result The date and time the alarm will trigger, given in seconds since 1/1/1904; if no alarm is active for the application, 0 is returned for the alarm seconds and the reference value is undefined.

See Also `AlmSetAlarm`

AlmGetProcAlarm

Purpose Macro that returns the date and time that a procedure alarm will trigger. Also returns the caller-defined alarm reference value.

Prototype
```
AlmGetProcAlarm (procP, refP)
```

Parameters

`-> procP`	Pointer to a function that will be called when alarm is triggered. See `AlmAlarmProcPtr`.
`<- refP`	A `UInt32` pointer to a location where the alarm's reference value is returned. This value is passed to the procedure when the alarm is triggered.

Result The date and time the alarm will trigger, given in seconds since 1/1/1904; if no alarm is active for the procedure, 0 is returned for the alarm seconds and the reference value is undefined.

Compatibility Implemented only if 3.2 New Feature Set is present.

See Also `AlmSetProcAlarm`

AlmSetAlarm

Purpose Set or cancel an alarm for the given application.

Prototype
```
Err AlmSetAlarm (UInt16 cardNo, LocalID dbID,
UInt32 ref, UInt32 alarmSeconds, Boolean quiet)
```

Parameters

`-> cardNo`	Number of the storage card on which the application resides.
`-> dbID`	Local ID of the application.
`-> ref`	Caller-defined value. This value is passed with the launch code that notifies the application that the alarm has been triggered.
`-> alarmSeconds`	Alarm date/time in seconds since 1/1/1904, or 0 to cancel the current alarm (if any).

	-> `quiet`	Reserved for future upgrade. This value is not currently used.

Result

`0`	No error.
`almErrMemory`	Insufficient memory.
`almErrFull`	Alarm table is full.

Comments

This function sets an alarm for the specified application. An application can have only one alarm set at a time. If an alarm for this application has already been set, it is replaced with the new alarm.

The `alarmSeconds` parameter specifies the time at which the alarm will be triggered. As soon as possible after this time, the alarm manager sends the `sysAppLaunchCmdAlarmTriggered` launch code to the specified application. If there is another alarm that should be set for this application, you can set it in response to this launch code. Following the `sysAppLaunchCmdAlarmTriggered` launch code, the alarm manager sends a `sysAppLaunchCmdDisplayAlarm` launch code. This is where your application should do things such as display a modal dialog indicating that the event has occurred. Read more about these launch codes in Chapter 1, "Application Launch Codes."

If your application needs access to any particular value to respond to the alarm, pass a pointer to that value in the `ref` parameter. The system will pass this pointer back to the application in the launch codes' parameter blocks.

See Also `AlmGetAlarm`

AlmSetProcAlarm

Purpose

Macro that sets or cancels a procedure alarm.

Prototype

```
AlmSetProcAlarm (procP, ref, alarmSeconds)
```

Parameters

-> `procP`	Pointer to a function that should be called when alarm is triggered. See `AlmAlarmProcPtr`.

-> `ref`	A caller-defined `UInt32` value. This value is passed with the launch code that notifies the application that the alarm has been triggered.
-> `alarmSeconds`	A `UInt32` indicating the alarm date/time in seconds since 1/1/1904, or 0 to cancel the current alarm (if any).

Result Returns one of the following error codes:

`0`	No error.
`almErrMemory`	Insufficient memory.
`almErrFull`	Alarm table is full.

Comments This macro is similar to the `AlmSetAlarm` function, but it specifies a procedure to be called at the specified date and time rather than an application to be launched. With this macro, you can set alarms that are independent of any application. For example, a shared library can set procedure alarms that call a procedure implemented in the library.

Procedure alarms also differ from regular system alarms in that if they trigger when the device is in sleep mode, the LCD does not light up. Thus, you can use procedure alarms to perform a scheduled task in a manner that is entirely hidden from the user.

IMPORTANT: Because the `procP` pointer is used to directly call the procedure, the pointer must remain valid from the time `AlmSetProcAlarm` is called to the time the alarm is triggered. If the procedure is in a shared library, you must keep the library open. If the procedure is in a separately loaded code resource, the resource must remain locked until the alarm is triggered. When you close a library or unlock a resource, you must remove any pending alarms. If you don't, the system will crash when the alarm is triggered.

Compatibility Implemented only if 3.2 New Feature Set is present.

See Also `AlmGetProcAlarm`

Application-Defined Functions

AlmAlarmProcPtr

Purpose A procedure to be executed when an alarm is triggered.

Prototype

```
void (*AlmAlarmProcPtr) (UInt16 almProcCmd,
SysAlarmTriggeredParamType *paramP)
```

Parameters

`-> almProcCmd` One of the `AlmProcCmdEnum` constants. These are commands that your function must handle. Possible values are:

`almProcCmdTriggered`
: The alarm's date and time has passed and the alarm has been triggered. The function should perform its main task in response to this command.

`almProcCmdReschedule`
: A system time change occurred, so the function must reschedule the alarm.

`-> paramP` Pointer to a `SysAlarmTriggeredParamType` structure. See below.

Result Returns nothing.

Comments `AlmAlarmProcPtr` procedures are called when an alarm set by `AlmSetProcAlarm` is triggered. Your implementation should check the value of `almProcCmd` and respond accordingly.

The `paramP` parameter is a pointer to a `SysAlarmTriggeredParamType` structure. This structure is defined as:

```
typedef struct SysAlarmTriggeredParamType {
  UInt32   ref;
  UInt32   alarmSeconds;
  Boolean purgeAlarm;
} SysAlarmTriggeredParamType;
```

`ref` and `alarmSeconds` are both values specified in `AlmSetProcAlarm` when the alarm is set. The `purgeAlarm` field specifies if the alarm will be removed from the alarm table when the function returns so that the `sysAppLaunchCmdDisplayAlarm` launch code is not triggered. This should be `true` for all procedure alarms; the alarm manager set it to `true` for you after your function returns.

If necessary, you can define new values for the `almProcCmd` parameter to call the procedure for something other than a triggered alarm or a system time change. The value you use must be greater than the constant `almProcCmdCustom` as defined in `AlarmMgr.h`.

Compatibility Implemented only if 3.2 New Feature Set is present.

See Also `AlmGetProcAlarm`

25

Attention Manager

This chapter provides reference material for the Attention Manager, and is divided into the following sections:

- Attention Manager Data Structures
- Attention Manager Constants
- Attention Manager Functions
- Application-Defined Functions

The Attention Manager API is declared in the header file `AttentionMgr.h`.

For more information about the attention manager, see the section "Getting the User's Attention" in the *Palm OS Programmer's Companion*, vol. I.

IMPORTANT: The Attention Manager was introduced in Palm OS® 4.0 and is not available in earlier versions of the operating system.

Attention Manager Data Structures



New

AttnCommandType

The `AttnCommandType` typedef specifies the set of possible commands that can be sent to the application requesting the user's attention, either as a parameter to the `AttnCallbackProc` callback function or as one of the arguments that accompanies a `sysAppLaunchCmdAttention` launch code.

```
typedef UInt16 AttnCommandType;
```

The following table lists the values that `AttnCommandType` can assume.

Constant	Description
`kAttnCommandDrawDetail`	Indicates that the application needs to draw the detailed contents of the attention dialog. The command arguments parameter points to a structure of type `drawDetail`.
`kAttnCommandDrawList`	Indicates that the application needs to draw the appropriate list item in the attention dialog. The command arguments parameter points to a structure of type `drawList`.
`kAttnCommandPlaySound`	Indicates that the application needs to play a sound. The command arguments parameter is `NULL`.
`kAttnCommandCustomEffect`	Indicates that the application needs to perform any application-specific special effects. This command is only sent for attention items that set the `kAttnFlagsCustomEffectBit` when they call `AttnGetAttention`, which most applications won't do.
`kAttnCommandGoThere`	Tells the application to navigate to the item. The command arguments parameter is `NULL`. An application commonly calls `SysAppLaunch` upon receipt of this command to have itself launched.
`kAttnCommandGotIt`	Tells the application that the user is dismissing the item. The command arguments parameter points to a structure of type `gotIt`. The application may choose to clean up memory at this point.
`kAttnCommandSnooze`	Indicates to the application that the user is snoozing. The command arguments parameter is `NULL`. Most applications do nothing upon receipt of this command. This command is passed to the appropriate application for each and every item currently pending, insistent or subtle. Applications with more than one attention item pending are called more than once.

Constant	Description
kAttnCommandIterate	This command is passed to the application during the enumeration of attention items. It is particularly useful after HotSync® operations, as it allows the application to examine each item, updating or removing those that are stale or invalid. The command arguments parameter points to a structure of type iterate.



AttnCommandArgsType

The AttnCommandArgsType structure is a union of C structures. How you interpret the union's contents depends on which command it accompanies. Not all commands are accompanied by an AttnCommandArgsType structure, as listed in the following table:

AttnCommandType	Accompanied By
kAttnCommandDrawDetail	drawDetail
kAttnCommandDrawList	drawList
kAttnCommandPlaySound	None
kAttnCommandCustomEffect	None
kAttnCommandGoThere	None
kAttnCommandGotIt	gotIt
kAttnCommandSnooze	None
kAttnCommandIterate	iterate

The structures that make up the AttnCommandArgsType union are described in the following sections.

drawDetail

When `kAttnCommandDrawDetail` is passed to the application, either via the callback function or as a parameter accompanying the `sysAppLaunchCmdAttention` launch code, the application needs to draw the detailed contents of the attention dialog. The `drawDetail` structure accompanies the `kAttnCommandDrawDetail` command, and provides the information needed to draw the contents of that dialog.

```
struct AttnCommandArgsDrawDetailTag {
  RectangleType bounds;
  Boolean firstTime;
  AttnFlagsType flags;
} drawDetail;
```

Field Descriptions

`bounds`	Contains the window-relative bounding box for the area to draw. The clipping region is also set to the dimensions of this box to prevent accidental drawing outside.
`firstTime`	Set to `true` if the user has not yet seen this item. The value of this field could be used to display attentions that the user hasn't seen before in some special way. `firstTime` also indicates to your application whether or not the area in which your application is to draw has been erased. If `firstTime` is `false`, the area is not guaranteed to be blank; your application will need to erase it.
`flags`	The global user preferences for this attention attempt combined with the custom flags passed in by the developer. Only the lower 16 bits of this field have meaning; use `kAttnFlagsSoundBit`, `kAttnFlagsLEDBit`, `kCustomFlagsVibrateBit`, and `kAttnFlagsCustomEffectBit` (described under "AttnFlagsType" on page 467) to interpret the contents of this field.

drawList

When `kAttnCommandDrawList` is passed to the application, either via the callback function or as a parameter accompanying the `sysAppLaunchCmdAttention` launch code, the application needs to draw the appropriate list item in the attention dialog. The `drawList` structure accompanies the `kAttnCommandDrawList` command, and provides the information needed to draw the contents of that dialog.

```
struct AttnCommandArgsDrawListTag {
  RectangleType bounds;
  Boolean firstTime;
  AttnFlagsType flags;
  Boolean selected;
} drawList;
```

Field Descriptions

`bounds`	Contains the window-relative bounding box for the area to draw. The clipping region is also set to the dimensions of this box to prevent accidental drawing outside.
`firstTime`	Set to `true` if the user has not yet seen this item. The value of this field could be used, for example, to trigger a custom sound the first time this attention item is presented to the user.
`flags`	The global user preferences for this attention attempt combined with the custom flags passed in by the developer. Only the lower 16 bits of this field have meaning; use `kAttnFlagsSoundBit`, `kAttnFlagsLEDBit`, `kCustomFlagsVibrateBit`, and `kAttnFlagsCustomEffectBit` (described under "AttnFlagsType" on page 467) to interpret the contents of this field.
`selected`	Set to `true` if the item has been selected. It is up to your code to draw the item appropriately (typically by changing the UI colors) based upon the value of this flag.

gotIt

When `kAttnCommandGotIt` is passed to the application, either via the callback function or as a parameter accompanying the `sysAppLaunchCmdAttention` launch code, it is accompanied by an `gotIt` structure. This structure indicates whether the `kAttnCommandGotIt` command was generated because the user dismissed the attention, or whether the system is simply informing your application that `AttnForgetIt` was called. Your application normally ignores the latter case if your application made the call to `AttnForgetIt`.

```
struct AttnCommandArgsGotItTag {
  Boolean dismissedByUser;
} gotIt;
```

Field Descriptions

`dismissedByUser`	`true` indicates that the user dismissed the attention. `false` indicates that the `kAttnCommandGotIt` command was generated by a call to `AttnForgetIt`.

iterate

When `kAttnCommandIterate` is passed to the application, either via the callback function or as a parameter accompanying the `sysAppLaunchCmdAttention` launch code, it is accompanied by an `iterate` structure. This structure contains any necessary data that the application may need in order to process the callback or launch code.

```
struct AttnCommandArgsIterateTag {
  UInt32 iterationData;
} iterate;
```

Field Descriptions

`iterationData`	Any necessary data that the application may need in order to process the callback or launch code. The value of this field is that which was originally passed to `AttnIterate`.



New AttnFlagsType

Pass a value of this type to `AttnGetAttention` and `AttnUpdate` to specify what means the device should or should not use to get the user's attention. A value of this type is also passed back to your code either as a parameter to the callback function or, if no callback was specified, as part of the structure passed with the `sysAppLaunchCmdAttention` launch code.

```
typedef UInt32 AttnFlagsType;
```

Note that more bits may be defined if necessary to accommodate additional hardware.

Constant	Value	Description
`kAttnFlagsSoundBit`	`0x0001`	Plays a sound.
`kAttnFlagsLEDBit`	`0x0002`	Blinks an LED, if the device is so equipped.
`kAttnFlagsVibrateBit`	`0x0004`	Triggers vibration, if the device is so equipped.
`kAttnFlagsCustomEffectBit`	`0x0008`	Triggers an application-specific custom effect.
`kAttnFlagsAllBits`	`0xFFFF`	Uses all available means to get the user's attention.
`kAttnFlagsUseUserSettings`	`0x0000`	System-wide preferences determine what means are used to get the user's attention.

The following constant values can be used to override the user's settings and force or prevent specific behaviors:

Constant	Value	Description
`kAttnFlagsAlwaysSound`	`kAttnFlagsSoundBit`	Play a sound, regardless of the user's settings.
`kAttnFlagsAlwaysLED`	`kAttnFlagsLEDBit`	Blink an LED, if the device is so equipped, regardless of the user's settings.
`kAttnFlagsAlwaysVibrate`	`kAttnFlagsVibrateBit`	Vibrate, if the device is so equipped, regardless of the user's settings.
`kAttnFlagsAlwaysCustomEffect`	`kAttnFlagsCustomEffectBit`	Trigger an application-specific custom effect.
`kAttnFlagsEverything`	`kAttnFlagsAllBits`	Use every available means to get the user's attention, regardless of the user's settings.
`kAttnFlagsNoSound`	`kAttnFlagsSoundBit << 16`	Prevent a sound from being played, regardless of the user's settings.
`kAttnFlagsNoLED`	`kAttnFlagsLEDBit << 16`	Prevent the LED from flashing, regardless of the user's settings.
`kAttnFlagsNoVibrate`	`kAttnFlagsVibrateBit << 16`	Prevent vibration, regardless of the user's settings.
`kAttnFlagsNoCustomEffect`	`kAttnFlagsCustomEffectBit << 16`	Prevent triggering of the application-specific custom effect.
`kAttnFlagsNothing`	`kAttnFlagsAllBits << 16`	Disable all attention-getting mechanisms, regardless of the user's settings.

These constants can be used in combination. For example, to disable both sound and the LED, use `kAttnFlagsNoSound | kAttnFlagsNoLED`.

If neither `kAttnFlagsAlwaysSound` nor `kAttnFlagsNoSound` is set for a given attention item, a sound plays if and only if the user's preference is to play a sound and the user's preference for alarm volume is non-zero.



AttnLaunchCodeArgsType

If a callback function is not specified in a call to `AttnGetAttention` and the Attention Manager needs your code to draw the details of your attention in the attention dialog or perform another attention-specific function, it sends a `sysAppLaunchCmdAttention` launch code to your application. Along with the launch code, it passes a pointer to the following structure, which indicates both what your code is expected to do and identifies the attention that triggered the launch code:

```
typedef struct {
  AttnCommandType command;
  UInt32 userData;
  AttnCommandArgsType *commandArgsP;
} AttnLaunchCodeArgsType;
```

Field Descriptions

`command`	Indicates what your code is being requested to do. The complete list of possible commands are described in the definition of `AttnCommandType`.
`userData`	Identifier that distinguishes the particular attention item from others made by this application. This identifier was specified when the attention item was created.
`commandArgsP`	Pointer to command-specific arguments. See the description of each command for a discussion of that command's arguments.

When processing the launch code be aware that your application doesn't have application globals available to it; it is important that anything necessary to draw or otherwise display be available through `commandArgsP`.



AttnLevelType

Attention attempts can either be insistent or subtle. Insistent attention attempts make a serious effort to get the user's attention, by both displaying a dialog and possibly by triggering one or more special effects, such as blinking a light, vibrating, or playing a sound. Other alerts are of a less serious nature and shouldn't disrupt the user. Consequently, subtle attention attempts typically make the attention indicator blink and may trigger one or more special effects, but don't display the Attention Manager dialog. The user can then work until a suitable time, at which point they can tap on the indicator to see what needs their attention. Subtle attention attempts might be used for telling the user that they have new e-mail, or perhaps that a holiday or birthday is coming up.

```
typedef UInt16 AttnLevelType;
```

The following table lists the two defined values for `AttnLevelType`:

Constant	Description
`kAttnLevelInsistent`	An insistent attention attempt. Make a serious effort to get the user's attention by displaying a dialog and optionally triggering one or more special effects.
`kAttnLevelSubtle`	A subtle attention attempt. Notify the user using special effects, but don't disrupt the user with the dialog if the device is in use.

Note that user preferences for the various special effects can't be set separately for subtle and insistent attention attempts. If your application needs to vary the effects based upon the `AttnLevelType`, pass a suitable value for the `flags` parameter in your `AttnGetAttention` call.

Attention Manager Constants

In addition to the constant values defined specifically for use with `AttnCommandType`, `AttnFlagsType`, and `AttnLevelType`, the Attention Manager defines the following constant types:

- Error Code Constants
- Attention Manager Drawing Constants
- Attention Manager Feature Constants

Error Code Constants

The Attention Manager returns the following error code under the circumstances described below.

Constant	Description
`attnErrMemory`	Returned by `AttnGetAttention` when there is insufficient memory to perform the requested operation.

Attention Manager Drawing Constants

The following four constants define the on-screen boundaries of the attention indicator:

Constant	Value	Description
`kAttnIndicatorLeft`	0	The left-hand edge of the attention indicator.
`kAttnIndicatorTop`	0	The top-most edge of the attention indicator.
`kAttnIndicatorWidth`	16	The width of the attention indicator.
`kAttnIndicatorHeight`	16	The height of the attention indicator.

The following two constants are used when drawing the list view. Applications should use these constants to format the display of information in the Attention Manager's list view. Draw the application's small icon centered within the first `kAttnListMaxIconWidth` pixels of the drawing bounds. Then draw two lines of text describing the attention, left-justified, starting

at `kAttnListTextOffset` from the left edge of the drawing bounds.

Constant	Value	Description
`kAttnListMaxIconWidth`	15	Maximum width of the application's icon. If the icon is narrower than this, it should be drawn centered within this width.
`kAttnListTextOffset`	17	Offset, from the left-hand edge of the drawing bounds, of the textual description of the attention.

Attention Manager Feature Constants

The Attention Manager defines a read-only feature (`'attn', 0`) that indicates the current user settings and capabilities of the hardware. The upper 16 bits of the feature indicate whether or not the hardware has the capability to perform that sort of alert. The lower 16 bits indicate whether the user has currently enabled that sort of alert.

Constant	Value	Description
`kAttnFtrCreator`	`'attn'`	Attention Manager feature creator.
`kAttnFtrCapabilities`	`0`	Attention Manager feature number.

When working with the value obtained with `FtrGet`, use the following two constants to separate those bits that contain the user settings from those bits that identify the device's capabilities:

Constant	Value	Description
`kAttnFlagsUserSettingsMask`	`kAttnFlagsAllBits`	Mask to isolate those bits that contain the user settings.
`kAttnFlagsCapabilitiesMask`	`kAttnFlagsAllBits << 16`	Mask to isolate those bits that contain the device capabilities.

These constants can be used to interpret the device capabilities (`kAttnFlagsHas...`) and the user settings (`kAttnFlagsUserWants...`):

Constant	Value	Description
`kAttnFlagsHasLED`	`kAttnFlagsLEDBit << 16`	The device has an LED that can be illuminated to indicate an alert.
`kAttnFlagsHasSound`	`kAttnFlagsSoundBit << 16`	The device is capable of playing a sound to indicate an alert.
`kAttnFlagsHasVibrate`	`kAttnFlagsVibrateBit << 16`	The device is capable of vibrating to indicate an alert.
`kAttnFlagsHasCustomEffect`	`kAttnFlagsCustomEffectBit << 16`	Not used.
`kAttnFlagsUserWantsLED`	`kAttnFlagsLEDBit`	The user wants the LED illuminated to signal an alert.
`kAttnFlagsUserWantsSound`	`kAttnFlagsSoundBit`	The user wants a sound played to signal an alert.
`kAttnFlagsUserWantsVibrate`	`kAttnFlagsVibrateBit`	The user wants the device to vibrate to signal an alert.
`kAttnFlagsUserWantsCustomEffect`	`kAttnFlagsCustomEffectBit`	Not used.

Attention Manager Functions



New AttnDoSpecialEffects

Purpose Triggers an Attention Manager special effect set.

Prototype

```
Err AttnDoSpecialEffects (AttnFlagsType flags)
```

Parameters

`-> flags` Specifies the behavior to be exhibited by this special effects request. See `AttnFlagsType` for the various bits that make up this flag. Note that the behavior is undefined if you set incompatible flags. Supply `kAttnFlagsUseUserSettings` to have this attention request follow the user's pre-set preferences.

Result Returns `errNone` if no problems were encountered. Returns `attnErrMemory` if there wasn't enough memory to accommodate the attention request.

Comments This routine is provided as a convenience for applications that need to trigger special effects. It does the equivalent of one "nag" of an Attention Manager special effect set.

Compatibility Implemented only if 4.0 New Feature Set is present.



New AttnForgetIt

Purpose

Provides a way for applications to tell the Attention Manager to forget about an attention item.

Prototype

```
Boolean AttnForgetIt (UInt16 cardNo, LocalID dbID,
UInt32 userData)
```

Parameters

`-> cardNo`	Card number on which the application making the request resides.
`-> dbID`	Database ID of the application making the request.
`-> userData`	Identifier that distinguishes the attention attempt from others made by the same application. This identifier can be an integer, a pointer, or any other 32-bit value.

Result

Returns `true` if the item was removed, `false` if a matching item was not found.

Comments

You typically call this function after your application has handled a "Go There" event and the user has read about the item. For example, if there is a subtle attention pending that says "you have three e-mail messages waiting" and you go to the e-mail application on your own and read your e-mail, the subtle notification must disappear. `AttnForgetIt` allows the application to do this.

Note that this call can be made when the Attention Manager dialog is on-screen (though presumably that is rare, since the application is probably not doing much at this point). If this call removes a list item, then the Attention Manager may call back into other items to redraw the list.

If this call removes the last item when any indicator is present, the indicator disappears. If this call removes the last unread item, but read items remain, the indicator switches from blinking to steady state.

Compatibility

Implemented only if 4.0 New Feature Set is present.



New AttnGetAttention

Purpose Requests the user's attention.

Prototype

```
Err AttnGetAttention (UInt16 cardNo, LocalID dbID,
UInt32 userData, AttnCallbackProc *callbackFnP,
AttnLevelType level, AttnFlagsType flags,
UInt16 nagRateInSeconds, UInt16 nagRepeatLimit)
```

Parameters

`-> cardNo` Card number on which the application making the request resides.

`-> dbID` Database ID of the application making the request.

`-> userData` Application-specific data that is later passed back to your code through the callback function. If no callback function is specified in the `callbackFnP` parameter, this data is included in what is passed along with a `sysAppLaunchCmdAttention` launch code. `userData` can be an integer, a pointer, or any other 32-bit value as needed by your application. Most applications pass the unique ID or other key for the record which caused the attention request. `userData` is also used to distinguish a given attention attempt from others made by the same application.

`-> callbackFnP`

Pointer to the function registered by the application to be called by the Attention Manager when the attention is displayed or removed. See `AttnCallbackProc`, below, for the callback function's parameters. Supply `NULL` to instead have a `sysAppLaunchCmdAttention` launch code sent to the application that made the attention request whenever the attention is displayed or removed.

`-> level` Indicates the annoyance level. Pass one of the values defined for `AttnLevelType`.

`-> flags` Behavior override, if necessary, for this attention request. This override allows, for instance, silent alarms or noisy alarms. See `AttnFlagsType` for the various bits that make up this flag. Note that the behavior is undefined if you set incompatible flags. Supply `kAttnFlagsUseUserSettings` to have this attention request follow the user's pre-set preferences.

`-> nagRateInSeconds`
How long to wait before nagging.

`-> nagRepeatLimit`
How many times to nag, excluding the first attempt.

Result Returns `errNone` if no problems were encountered. Returns `attnErrMemory` if there wasn't enough memory to accommodate the attention request.

Comments The combination of `cardNo`, `dbID` and `userData` uniquely identify an attention-getting attempt. If another call is made to `AttnGetAttention` with identical values for these arguments, an error is reported. To update or delete an existing attention item, pass these same values to `AttnUpdate` or `AttnForgetIt`, respectively.

In response to `AttnGetAttention`, the behavior of the operating system or application depends on whether there already are other demands and on the annoyance level passed in the `AttnGetAttention` call.

- No other demands, insistent attention request:

  The Attention Manager puts up a dialog that details the current attempt to get the user's attention.

- Other demands exist, insistent attention request:

  The Attention Manager adds a summary of the current attempt to get the user's attention to a list of things that need attention. If the dialog is currently in detail form—which is the case if just

one other demand exists—the view is refreshed, changing from detail to list form. In this case, the pen and key event queues are also flushed so that any user events that are happening while the display is changing are explicitly ignored. Two exceptions to this behavior exist: if all existing attentions are subtle, or if all existing insistent attentions were snoozed, the new insistent attention brings up the dialog in detail mode, rather than list mode.

- Subtle attention request:

  The Attention Manager starts the attention indicator blinking, and adds the item to its list for later display, unless the dialog is currently being displayed in list mode. In this event, the new subtle attention item simply appears in the list; the indicator does not blink to announce it.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `AttnUpdate`



New AttnGetCounts

Purpose Returns the number of attention items that are currently pending.

Prototype

```
UInt16 AttnGetCounts (UInt16 cardNo, LocalID dbID,
UInt16 *insistentCountP, UInt16 *subtleCountP)
```

Parameters

`-> cardNo`	If this value is zero, counts pending attention items from applications on all cards. Otherwise, counts only pending attention items from applications on the specified card.
`-> dbID`	If this value is zero, counts pending attention items from all applications. Otherwise, counts only pending attention items from applications with the specified database ID.

<- `insistentCountP`
Pointer to a 16-bit unsigned value that is filled in with the number of insistent items pending. Pass NULL for this parameter if you don't need to know the number of insistent items that are pending.

<- `subtleCountP` Pointer to a 16-bit unsigned value that is filled in with the number of subtle items pending. Pass NULL for this parameter if you don't need to know the number of subtle items that are pending.

Result Returns the total number of items, both insistent and subtle, that are currently pending.

Comments Call this function if you need to exhibit different behavior if attention items are already pending.

Compatibility Implemented only if 4.0 New Feature Set is present.



New AttnIndicatorEnable

Purpose Enables and disables the on-screen attention indicator.

Prototype
```
void AttnIndicatorEnable (Boolean enableIt)
```

Parameters -> `enableIt` `true` to enable the attention indicator, `false` to disable it.

Result Returns nothing.

Comments This function is used by applications to enable or disable the on-screen attention indicator. The indicator only blinks when all of the following are true:

- The indicator is enabled.

- The indicator is being asked to blink by the attention manager.
- The operating system isn't using the display in such a way as to prevent the attention indicator from showing, such as when the menu bar is being displayed or when a modal dialog is on top of the form.

The attention indicator is enabled by default. If your application controls the upper portion of the screen and needs to prevent the attention indicator from being displayed, call `AttnIndicatorEnable` and pass it a value of `false`.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `AttnIndicatorEnabled`



New AttnIndicatorEnabled

Purpose Returns whether the on-screen attention indicator is currently enabled.

Prototype

```
Boolean AttnIndicatorEnabled (void)
```

Parameters None.

Result Returns `true` if the on-screen attention indicator is currently enabled, `false` otherwise.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `AttnIndicatorEnable`



New AttnIterate

Purpose

Instructs the Attention Manager to check each attention item currently pending and, for those that match the specified card number and database ID, invoke the item's callback routine. If a callback routine was not specified in the request, the `sysAppLaunchCmdAttention` launch code is sent to the application that made the attention request.

Prototype

```
void AttnIterate (UInt16 cardNo, LocalID dbID,
UInt32 iterationData)
```

Parameters

`-> cardNo` Card number on which the application that made the request resides.

`-> dbID` Database ID of the application that made the request.

`-> iterationData` Any necessary data that the application may need in order to process the callback or launch code. See the description of the `AttnCallbackProc` function for more information on this parameter.

Result

Returns nothing.

Comments

This function iterates through all of the attention requests made by this application and uses the callback or launch code for each to inform the application about the attention request. When an application receives a `sysAppLaunchCmdSyncNotify` launch code, signifying that a HotSync occurred that affected that application's databases, the application typically calls `AttnIterate` so it can remove attention requests for records that may have been removed during the HotSync. Applications can also call `AttnGetAttention` after a HotSync, if necessary.

Note that you can call `AttnForgetIt` inside the iteration since it only marks the record for deletion and thus doesn't confuse the iteration.

Compatibility Implemented only if 4.0 New Feature Set is present.



New AttnListOpen

Purpose Displays the attention dialog in list mode and, after the user has dismissed it, acts accordingly based on how it was dismissed.

Prototype

```
void AttnListOpen (void)
```

Parameters None.

Result Returns nothing.

Comments This function allows applications that do not provide the blinking attention indicator to open the list, if necessary.

Compatibility Implemented only if 4.0 New Feature Set is present.



New AttnUpdate

Purpose Updates one or more aspects of a specified attention item.

Prototype

```
Boolean AttnUpdate (UInt16 cardNo, LocalID dbID,
UInt32 userData, AttnCallbackProc *callbackFnP,
AttnFlagsType *flagsP, UInt16 *nagRateInSecondsP,
UInt16 *nagRepeatLimitP)
```

Parameters

`-> cardNo` Card number on which the application that made the request resides.

`-> dbID` Database ID of the application that made the request.

`-> userData` Application-specific data that is passed back to your code through the callback function. If no callback function is specified in the `callbackFnP` parameter, this data is included in what is passed along with a `sysAppLaunchCmdAttention` launch code. `userData` can be an integer, a pointer, or any other 32-bit value. Most applications pass the unique ID or other key for the record which caused the attention request. The value of the `userData` parameter is also used to distinguish a given attention attempt from others made by the same application.

`-> callbackFnP`

Registers a new function to be called by the Attention Manager when the attention is displayed or removed. The function to which this parameter points should conform to `AttnCallbackProc`. Supply `NULL` to instead have a `sysAppLaunchCmdAttention` launch code sent to the application that made the attention request whenever the attention is displayed or removed.

IMPORTANT: Because `NULL` indicates that a launch code should be sent whenever the callback would otherwise be invoked, it isn't used in this instance to leave the original setting for the `callbackFnP` parameter intact. The value supplied for this parameter *always* overwrites the value supplied in the original attention request.

`-> flagsP` Pointer to a set of flags that can be used to override user-specified attention behavior; for instance, to force silent or noisy alarms. See `AttnFlagsType` for the various bits that make up this flag, and note that the behavior is undefined if you set incompatible flags. Pass `NULL` to leave the current flag settings unchanged.

`-> nagRateInSecondsP` Pointer to the length of time to wait before nagging. Pass `NULL` to leave the "nag rate" unchanged.

`-> nagRepeatLimitP` Pointer to the maximum number of times the user should be nagged. Pass `NULL` to leave the nag repeat limit unchanged.

Result Returns `true` if the update was successful, `false` if no matching attention item was found.

Comments This call may result in the callback function being called to re-display the item. If no callback function is specified, the `sysAppLaunchCmdAttention` launch code is instead sent to your application. It may also result in callbacks to other pending attention requests.

You call `AttnUpdate` to tell the Attention Manager to update, forcing it to call into all of its clients to redraw. This provides a way for an application to update the text of an attention item without tearing down and re-opening the Attention Manager dialog. For example, `AttnUpdate` could be used to update an existing email notification to say "You have three new email messages" when additional messages are received.

Although `AttnUpdate` may cause a given attention item to redraw, it does not cause that item's sound to play, and the item is not moved to the top of the list. If you want this to happen, call `AttnForgetIt` followed by `AttnGetAttention`.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also AttnGetAttention

Application-Defined Functions



New AttnCallbackProc

Purpose Provides a function prototype to be used by callback functions supplied to AttnGetAttention and AttnUpdate. The supplied function is invoked by the Attention Manager whenever the attention is displayed or removed.

Prototype

```
typedef Err AttnCallbackProc
(AttnCommandType command, UInt32 userData,
AttnCommandArgsType *commandArgsP)
```

Parameters

-> `command` Indicates what the callback function is being requested to do. The complete list of possible commands are described in the definition of AttnCommandType.

-> `userData` Identifier that distinguishes the particular attention item from others made by this application. This identifier was specified when the attention item was created.

-> `commandArgsP` Pointer to command-specific arguments. See the description of each command for a discussion of that command's arguments.

Result The callback function should return `errNone` if it correctly handled the command, or an appropriate error code otherwise. If the callback function returns an error code other than `errNone`, the attention is removed from the list of active attention items.

Comments For a given attention item, the Attention Manager calls back to the code resource that created that item whenever the Attention Manager needs the resource to draw the attention dialog contents or whenever it needs to inform the code resource of activity relating to the attention item. The Attention Manager calls back using one of two mechanisms:

- If a callback routine has been specified for a given attention item, the Attention Manager invokes the specified routine. This callback routine doesn't have application globals available to it, so it is important that anything necessary to draw or otherwise display be available through `commandArgsP`. A callback routine is typically used by libraries and system extensions.
- If a callback routine has not been specified for a given attention item, the Attention Manager instead sends a `sysAppLaunchCmdAttention` launch code to the application that registered the attention item. Accompanying that launch code is an `AttnLaunchCodeArgsType` structure containing the three parameters documented above. Applications typically use the launch-code mechanism due to the restrictions that are placed on callback routines.

IMPORTANT: It is your responsibility to ensure that the callback procedure is still in the same place when it gets called, dealing with the possibility that the code resource might be unlocked and moved in memory, and with the possibility that the database containing the code resource might be deleted. For the most part, these problems don't exist when using launch codes.

Compatibility Invoked only if 4.0 New Feature Set is present.

26

Bitmaps

This chapter provides information about bitmaps by discussing these topics:

- Bitmap Data Structures
- Bitmap Constants
- Bitmap Resources
- Bitmap Functions

The header file `Bitmap.h` declares the API that this chapter describes. For more information on bitmaps, see the section "Bitmaps" on page 118 in the *Palm OS Programmer's Companion*, vol. I.

Bitmap Data Structures

BitmapCompressionType

The `BitmapCompressionType` enum specifies possible bitmap compression types. These are the possible values for the `compressionType` field of `BitmapType`. You can compress or uncompress a bitmap using a call to `BmpCompress`.

```
typedef enum {
  BitmapCompressionTypeScanLine = 0,
  BitmapCompressionTypeRLE,
  BitmapCompressionTypePackBits,
  BitmapCompressionTypeEnd,
  BitmapCompressionTypeBest = 0x64,
  BitmapCompressionTypeNone = 0xFF
} BitmapCompressionType;
```

Value Descriptions

`BitmapCompressionTypeScanLine`	Use scan line compression. Scan line compression is compatible with Palm OS® 2.0 and higher.
`BitmapCompressionTypeRLE`	Use RLE compression. RLE compression is supported in Palm OS 3.5 and higher.
`BitmapCompressionTypePackBits`	Use PackBits compression. PackBits compression is supported in Palm OS 4.0 only.
`BitmapCompressionTypeEnd`	For internal use only.
`BitmapCompressionTypeBest`	For internal use only.
`BitmapCompressionTypeNone`	No compression is used. This value should only be used as an argument to `BmpCompress`.

Compatibility `BitmapCompressionType` is only defined if 3.5 New Feature Set is present. Earlier releases do support compressed bitmaps, but in scan line format only.



New

BitmapDirectInfoType

For direct color bitmaps—each pixel is represented by an RGB triplet rather than a palette index—the `BitmapDirectInfoType` structure follows the color table if one is present, or immediately follows the `BitmapType` if a color table is not present. For direct color bitmaps, only 16 bits per pixel is supported, 5 bits for red, 6 bits for green, and 5 bits for blue.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `BitmapDirectInfoType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct BitmapDirectInfoType {
  UInt8 redBits;
  UInt8 greenBits;
  UInt8 blueBits;
  UInt8 reserved;
  RGBColorType transparentColor;
} BitmapDirectInfoType;
```

Field Descriptions

`redBits`	Number of bits used by the red component in each pixel.
`greenBits`	Number of bits used by the green component in each pixel.
`blueBits`	Number of bits used by the blue component in each pixel.
`reserved`	Must be zero. Reserved for future use.
`transparentcolor`	Contains the red, green, and blue components of the transparent color.

Compatibility `BitmapDirectInfoType` is only defined if 4.0 New Feature Set is present.

BitmapFlagsType

The `BitmapFlagsType` bit field defines the `flags` field of `BitmapType`. It specifies the bitmap's attributes.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `BitmapFlagsType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct BitmapFlagsType {
  UInt16 compressed:1;
  UInt16 hasColorTable:1;
  UInt16 hasTransparency:1;
  UInt16 indirect:1;
  UInt16 forScreen:1;
  UInt16 directColor:1;
  UInt16 reserved:10;
} BitmapFlagsType;
```

Field Descriptions

`compressed`	If `true`, the bitmap is compressed and the `compressionType` field specifies the compression used. If `false`, the bitmap is uncompressed. The `BmpCompress` function sets this field.
`hasColorTable`	If `true`, the bitmap has its own color table. If `false`, the bitmap uses the system color table. You specify whether the bitmap has its own color table when you create the bitmap.
`hasTransparency`	If `true`, the OS will not draw pixels that have a value equal to the `transparentIndex`. If `false`, the bitmap has no transparency value. You specify the transparent color when you create the bitmap using Constructor.

`indirect`	If `true`, the address to the bitmap's data is stored where the bitmap itself would normally be stored. The actual bitmap data is stored elsewhere. If `false`, the bitmap data is stored directly following the bitmap header or directly following the bitmap's color table if it has one. Never set this flag. Only the display (screen) bitmap has the indirect bit set.
`forScreen`	If `true`, bitmap intended for the display (screen) window. Never set this flag.
`directColor`	If `true`, bitmap is a direct color (RGB) bitmap.
`reserved`	Reserved for future use.

Compatibility Flags `hasTransparency`, `indirect`, and `forScreen` are only defined if 3.5 New Feature Set is present. The `directColor` flag is only defined if 4.0 New Feature Set is present.

BitmapPtr

The `BitmapPtr` type defines a pointer to a `BitmapType` structure.

```
typedef BitmapType *BitmapPtr;
```

BitmapType

The `BitmapType` structure represents a bitmap. This structure defines both the bitmaps representing the window display and bitmap resources (`'Tbmp'` and `'tAIB'`) that you create using Constructor or some other application and load into your program.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `BitmapType` structure. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct BitmapType {
  Int16            width;
  Int16            height;
  UInt16           rowBytes;
  BitmapFlagsType  flags;
  UInt8            pixelSize;
  UInt8            version;
  UInt16           nextDepthOffset;
  UInt8            transparentIndex;
  UInt8            compressionType;
  UInt16           reserved;
} BitmapType;
```

Field Descriptions

`width`	The width of the bitmap in pixels. You specify this value when you create the bitmap. Use `BmpGetDimensions` to access this field.
`height`	The height of the bitmap in pixels. You specify this value when you create the bitmap. Use `BmpGetDimensions` to access this field.
`rowBytes`	The number of bytes stored for each row of the bitmap where `height` is the number of rows. Use `BmpGetDimensions` to access this field.
`flags`	The bitmap's attributes. See `BitmapFlagsType`.
`pixelSize`	The pixel depth. Currently supported pixel depths are 1, 2, 4, and 8-bit. You specify this value when you create the bitmap.
`version`	The version of bitmap encoding used. See Bitmap Constants.

`nextDepthOffset`	For bitmap families, this field specifies the start of the next bitmap in the family. The value it contains is the number of 4-byte words to the next `BitmapType` from the beginning of this one. If the bitmap is not part of a bitmap family or it is the last bitmap in the family, the `nextDepthOffset` is 0.
`transparentIndex`	The color index for the transparent color. Only used for version 2 bitmaps and only when the `hasTransparency` flag is set (see `BitmapFlagsType`). You specify this value when you create the bitmap using Constructor.
`compressionType`	The compression type used. Only used for version 2 bitmaps and only when the `compressed` flag is set (see `BitmapFlagsType`). See `BitmapCompressionType` for possible values. The `BmpCompress` function sets this field.
`reserved`	Reserved for future use. Must be set to 0.

Note the following about the `BitmapType` structure:

- None of these fields contains the actual bitmap data. Instead, the bitmap data is stored immediately following this `BitmapType` header structure. If the bitmap has its own color table, the color table is stored in between the header and the data. If the bitmap has a pixel size of 16, the `BitmapDirectInfoType` structure is stored between the header and the data. You can retrieve a bitmap's data by passing its `BitmapType` structure to `BmpGetBits`, and you can retrieve its color table with `BmpGetColortable`.
- Unlike most other user interface structures, the `BitmapType` does not store the bitmap's location on the screen. The `WindowType` or the `FormBitmapType` with which this bitmap is associated contains that information.

- A bitmap may be part of a **bitmap family**. A bitmap family is a group of bitmaps, each containing the same drawing but at a different pixel depth (see Figure 5.14 in the User Interface chapter of the *Palm OS Programmer's Companion*, vol. I). When requested to draw a bitmap family, the operating system chooses the version of the bitmap with the pixel depth closest to, but not greater than, the current pixel depth of the display. When `BitmapType` represents a bitmap family, the `nextDepthOffset` field contains the offset from the start of this bitmap to the next bitmap in the family.

Compatibility The `transparentIndex` and `compressionType` flags are defined only if 3.5 New Feature Set is present.

ColorTableType

The `ColorTableType` structure defines a color table. Bitmaps can have color tables attached to them; however, doing so is not recommended for performance reasons.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `ColorTableType` structure. Never access its structure members directly, or your code may break in future versions. Use `BmpGetColortable` to access this structure. Use the information below for debugging purposes only.

```
typedef struct ColorTableType {
  UInt16            numEntries;
  // RGBColorType   entry[];
} ColorTableType;
```

Field Descriptions

`numEntries` The number of entries in table. High bits (`numEntries` > 256) reserved.

The color table entries themselves are of type `RGBColorType`, and there is one per `numEntries`. Use the macro `ColorTableEntries` to retrieve these entries.

Care should be taken not to confuse a full color table (which includes the count) with an array of RGB color values. Some routines operate on entire color tables; others operate on lists of color entries.

Compatibility `ColorTableType` is defined only if 3.5 New Feature Set is present.

RGBColorType

The `RGBColorType` structure defines a color. It is used as an entry in the color table. `RGBColorTypes` can also be created manually and passed to several user interface functions.

```
typedef struct RGBColorType {
  UInt8 index;
  UInt8 r;
  UInt8 g;
  UInt8 b;
} RGBColorType;
```

Field Descriptions

`index` The index of this color in the color table. Not all functions that use `RGBColorType` use the `index` field.

Direct bitmaps support no more than 256 colors. The number of possible RGB colors greatly exceeds this amount. For this reason, some drawing functions use a color look up table (CLUT). If the CLUT is used, the `index` field contains the index of an available color that is the closest match to the color specified by the `r`, `g`, and `b` fields.

`r` Amount of red (0 to 255).

`g` Amount of green (0 to 255).

`b` Amount of blue (0 to 255).

Compatibility `RGBColorType` is defined only if 3.5 New Feature Set is present.

Bitmap Constants

Constant	Value	Description
`BitmapVersionZero`	0	Uses the version 0 encoding of a bitmap. Version 0 encoding is supported in Palm OS 1.0 and later.
`BitmapVersionOne`	1	Uses the version 1 encoding of a bitmap. Version 1 encoding is supported in Palm OS 3.0 and later. PalmRez automatically creates version 1 bitmaps unless you have specified a transparency index or a compressed type when creating the bitmap in Constructor.
`BitmapVersionTwo`	2	Uses the version 2 encoding of a bitmap. Palm OS 3.5 and later supports version 2 bitmaps. Version 2 bitmaps either use the transparency index or are compressed. If you programmatically create a bitmap using `BmpCreate`, a version 2 bitmap is created.

Bitmap Resources

You can create a bitmap resource and include it as part of your application's PRC file. Use the resource type `'Tbmp'` for most images and the resource type `'tAIB'` for application icons. Symbolically, these two resource types are `bitmapRsc` and `iconType`, respectively.

Note that if you are creating a bitmap or a bitmap family in Constructor, you create a `'tbmf'` resource (or `'taif'` resource for icons) and one or more 'PICT' images. The PalmRez post linker converts them into a single `'Tbmp'` or `'tAIB'` resource. Note that the PalmRez post linker takes PICT images even on the Microsoft Windows operating system.

Bitmap Functions

BmpBitsSize

Purpose Return the size of the bitmap's data.

Prototype
```
UInt16 BmpBitsSize (const BitmapType *bitmapP)
```

Parameters -> `bitmapP` Pointer to the bitmap. (See `BitmapType`.)

Result Returns the size in bytes of the bitmap's data, excluding the header and the color table.

Comments This function returns the bitmap's data size even if the bitmap's `indirect` flag is set. (See `BitmapFlagsType`.)

If the bitmap is compressed, this function returns the compressed size of the bitmap.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `BmpSize`, `BmpColortableSize`, `BmpGetBits`

BmpColortableSize

Purpose Return the size of the bitmap's color table.

Prototype
```
UInt16 BmpColortableSize
(const BitmapType *bitmapP)
```

Parameters -> `bitmapP` Pointer to the bitmap. (See `BitmapType`.)

Result Returns the size in bytes of the bitmap's color table or 0 if the bitmap does not use its own color table.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `BmpBitsSize`, `BmpSize`, `BmpGetColortable`

BmpCompress

Purpose Compress or uncompress a bitmap.

Prototype

```
Err BmpCompress (BitmapType *bitmapP,
BitmapCompressionType compType)
```

Parameters

-> `bitmapP`	Pointer to the bitmap to compress. (See `BitmapType`.)
-> `compType`	The type of compression to use. (See `BitmapCompressionType`.) If set to `BitmapCompressionTypeNone` and `bitmapP` is compressed, this function uncompresses the bitmap.

Result Returns one of the following values:

`errNone`	Success.
`sysErrParamErr`	Either the `compType` parameter does not specify a compression type or the bitmap is already compressed, is in the storage heap, or represents the screen.
`sysErrNoFreeResource`	There is not enough memory available to complete the operation.

Comments This function performs the specified compression and resizes the bitmap's allocated memory. The bitmap must be in the dynamic heap.

Compatibility Implemented only if 3.5 New Feature Set is present.

BmpCreate

Purpose Create a bitmap.

Prototype

```
BitmapType *BmpCreate (Coord width, Coord height,
UInt8 depth, ColorTableType *colortableP,
UInt16 *error)
```

Parameters

-> `width`	The width of the bitmap in pixels. Must not be 0.
-> `height`	The height of the bitmap in pixels. Must not be 0.
-> `depth`	The pixel depth of the bitmap. Must be 1, 2, 4, 8, or 16. This value is used as the `pixelSize` field of `BitmapType`.
-> `colortableP`	A pointer to the color table associated with the bitmap, or `NULL` if the bitmap should not include a color table. If specified, the number of colors in the color table must match the `depth` parameter. (2 for 1-bit, 4 for 2-bit, 16 for 4-bit, and 256 for 8-bit). 16-bit bitmaps do not use a color table.
<- `error`	Contains the error code if an error occurs.

Result Returns a pointer to the new bitmap structure (see `BitmapType`) or `NULL` if an error occurs. The parameter `error` contains one of the following:

`errNone`	Success.
`sysErrParamErr`	The `width`, `height`, `depth`, or `colorTableP` parameter is invalid. See the descriptions above for acceptable values.
`sysErrNoFreeResource`	There is not enough memory available to allocate the structure.

Comments

This function creates an uncompressed, non-transparent `BitmapVersionTwo` bitmap with the width, height, and depth that you specify.

If you pass a color table, the bitmap's `hasColorTable` flag is set. For performance reasons, attaching a custom color table to a bitmap is strongly discouraged. An alternative is to use the `WinPalette` command to change the color table as needed, draw the bitmap, and then undo your changes after you have finished displaying the bitmap.

`BmpCreate` allocates sufficient memory on the dynamic heap to hold the bitmap and initializes all of its pixels to white. To change the bitmap's contents, use the window drawing functions. First, you must use `WinCreateBitmapWindow` to create an off screen window wrapper around the bitmap, then draw to that window. For example:

```
BitmapType *bmpP;
WinHandle win;
Err error;
RectangleType onScreenRect;

bmpP = BmpCreate(10, 10, 8, NULL, &error);
if (bmpP) {
  win = WinCreateBitmapWindow(bmpP, &error);
  if (win) {
    WinSetDrawWindow(win);
    WinDrawLines(win, ...);
    /* etc */
    WinSetWindowBounds(win, onScreenRect);
  }
}
```

You cannot use this function to create a bitmap written directly to a database; that is, you must create the bitmap on the dynamic heap first, then write it to the storage heap.

It is not necessary to use `BmpCreate` to load a bitmap stored in a resource. Instead, you simply load the resource and lock its handle. The returned pointer is a pointer to a `BitmapType`. For example:

```
MemHandle resH =
  DmGetResource (bitmapRsc, rscID);
```

```
BitmapType *bitmap = MemHandleLock (resH);
```

Bitmaps 64 Kb and greater are now supported with Palm OS 4.0.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also BmpDelete

BmpDelete

Purpose Delete a bitmap structure.

Prototype

```
Err BmpDelete (BitmapType *bitmapP)
```

Parameters

-> bitmapP	Pointer to the structure of the bitmap to be deleted. (See BitmapType.)

Result Returns errNone upon success, sysErrParamErr if the bitmap's forScreen flag is set or the bitmap resides in the storage heap. Returns one of the memory errors if the freeing pointer fails.

Comments Only delete bitmaps that have been created using BmpCreate.

You cannot use this function on a bitmap located in a database. To delete a bitmap from a database, use the standard data manager calls.

Compatibility Implemented only if 3.5 New Feature Set is present.

BmpGetBits

Purpose Retrieve the bitmap's data.

Prototype

```
void *BmpGetBits (BitmapType *bitmapP)
```

Parameters

-> bitmapP	Pointer to the bitmap's structure. (See BitmapType.)

Result Returns a pointer to the bitmap's data.

Comments This function returns the bitmap's data even if the bitmap's `indirect` flag is set. (See `BitmapFlagsType`.)

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `BmpBitsSize`



New BmpGetBitDepth

Purpose Retrieve the depth of a bitmap.

Prototype
```
UInt8 BmpGetBitDepth (const BitmapType* bitmapP)
```

Parameters -> `bitmapP` Pointer to a bitmap. See `BitmapType`.

Result This function returns the bit depth of the bitmap, as represented by the `pixelSize` field in `BitmapType`. For debug ROMs, this function reports an error and returns 0 if `bitmapP` is `NULL`.

Compatibility Implemented only if 4.0 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `BmpGlueGetBitDepth`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `BmpGetDimensions`, `BmpGetNextBitmap`, `BmpGetSizes`

BmpGetColortable

Purpose Retrieve the bitmap's color table.

Prototype
```
ColorTableType *BmpGetColortable
(BitmapType *bitmapP)
```

Parameters

-> `bitmapP`	A pointer to the bitmap. See `BitmapType`.

Result Returns a pointer to the color table or `NULL` if the bitmap uses the system color table.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `BmpColortableSize`



New BmpGetDimensions

Purpose Retrieve the width, height and number of data bytes per row of a bitmap.

Prototype
```
void BmpGetDimensions (const BitmapType *bitmapP,
Coord *widthP, Coord *heightP, UInt16 *rowBytesP)
```

Parameters

`-> bitmapP`	Pointer to the bitmap. See `BitmapType`.
`<- widthP`	Pointer to bitmap's width in pixels. Use `NULL` if this information is not wanted.
`<- heightP`	Pointer to bitmap's height in pixels. Use `NULL` if this information is not wanted. Use `NULL` if this information is not wanted.
`<- rowBytesP`	Pointer to number of bytes per row of bitmap. Use `NULL` if this information is not wanted.

Result This function returns the width in pixels of the bitmap in `widthP`, the height in pixels of the bitmap in `heightP`, and the number of

bytes of data per row of the bitmap in `rowBytesP`. This function reports an error on debug ROMs if `bitmapP` is `NULL`.

Compatibility Implemented only if 4.0 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `BmpGlueGetDimensions`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `BmpGetBitDepth`, `BmpGetNextBitmap`, `BmpGetSizes`



New BmpGetNextBitmap

Purpose Retrieve the next bitmap type in a bitmap family.

Prototype
```
BitmapType *BmpGetNextBitmap(BitmapType *bitmapP)
```

Parameters

-> `bitmapP` Pointer to a bitmap. See `BitmapType`.

Result This function returns a pointer to the next `BitmapType` in a bitmap family. It returns `NULL` if `bitmapP` is the last bitmap. For debug ROMs, this function reports an error and returns 0 if `bitmapP` is `NULL`.

Compatibility Implemented only if 4.0 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `BmpGlueGetNextBitmap`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `BmpGetBitDepth`, `BmpGetDimensions`, `BmpGetSizes`



New BmpGetSizes

Purpose

Retrieve the size of a bitmap and its header structure.

Prototype

```
void BmpGetSizes (const BitmapType *bitmapP,
UInt32 *dataSizeP, UInt32 *headerSizeP)
```

Parameters

`->bitmapP`	Pointer to the bitmap. See `BitmapType`.
`<-dataSizeP`	Pointer to size of bitmap data, not including structures. Use `NULL` if this information is not wanted.
`<-headerSizeP`	Pointer to size of bitmap's structures, not including data. Use `NULL` if this information is not wanted.

Result

Returns the size of the bitmap and the size of the bitmap's structures. This function will report an error on debug ROMs if `bitmapP` is `NULL`.

Comments

This function returns the size in bytes of the bitmap data in `dataSizeP`. The size does not include the data structures (`BitmapType`, `BitmapDirectInfoType`, or color table) that are associated with a bitmap. The size of the structures (in bytes) are returned in `headerSizeP`, which includes the size of the `BitmapType`, `BitmapDirectInfoType` (if any), the color table (if any), and the size of the pointer for indirect bitmaps (described in `BitmapFlagsType`).

This function should be used when working with bitmaps that may be 64 Kb or greater. Do not use `BmpSize` or `BmpBitsSize` when working with bitmaps that may be greater than or equal to 64Kb.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`BmpGetBitDepth`, `BmpGetDimensions`, `BmpGetNextBitmap`

BmpSize

Purpose Return the size of the bitmap.

Prototype
```
UInt16 BmpSize (const BitmapType *bitmapP)
```

Parameters -> `bitmapP` A pointer to the bitmap. See `BitmapType`.

Result Returns the size in bytes of the bitmap, including its header, color table (if any), and `sizeof(BitmapDirectInfoType)` if one exists.

Comments If the bitmap has its `indirect` flag set (see `BitmapFlagsType`), the bitmap data is not included in the size returned by this function.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `BmpBitsSize`, `BmpColortableSize`

ColorTableEntries

Purpose Macro that returns the color table.

Prototype
```
ColorTableEntries (ctP)
```

Parameters -> `ctP` A pointer to a `ColorTableType` structure.

Result Returns an array of `RGBColorType` structures, one for each entry in the color table.

Comments You can use this macro to retrieve the RGB values in use by a bitmap. For example:

```
BitmapType *bmpP;
RGBColorType *tableP =
  ColorTableEntries(BmpGetColorTable(bmpP));
```

If you want to retrieve the RGB values in use by the system color table, you can simply use the `WinPalette` function instead of this macro:

```
RGBColorType table[256];
Err e;

/* allocate space for table */
e = WinPalette(winPaletteGet, 0, 256, tableP);
```

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `BmpGetColortable`

27

Character Attributes

This chapter provides reference material for character attributes functions defined in `CharAttr.h`.

Character Attribute Functions

ChrHorizEllipsis

Purpose Macro that returns the appropriate character code for the horizontal ellipsis.

Prototype

```
ChrHorizEllipsis (chP)
```

Parameters

<- `chP`	Pointer to a variable in which to return the horizontal ellipsis character code.

Result Returns nothing. Upon return, the variable pointed to by `chP` contains the horizontal ellipsis character.

Comments Version 3.1 of the Palm OS® uses different character codes for the horizontal ellipsis character and the numeric space character than earlier versions did. Use this macro to return the appropriate code for horizontal ellipsis regardless of which version of Palm OS your application is run on.

ChrIsHardKey

Purpose Macro that returns true if the character is one of the hard keys on the device.

Prototype
```
ChrIsHardKey (ch)
```

Parameters

-> `ch`	The character from the `keyDownEvent`.

Result `true` if the character is one of the four built-in hard keys on the device, `false` otherwise.

Compatibility This macro is obsolete and replaced by `TxtCharIsHardKey` if the International Feature Set is present.

ChrNumericSpace

Purpose Macro that returns the appropriate character code for the numeric space.

Prototype
```
ChrNumericSpace (chP)
```

Parameters

<- `chP`	Pointer to a variable in which to return the numeric space character code.

Result Returns nothing. Upon return, the variable pointed to by `chP` contains the numeric space character.

Comments Version 3.1 of the Palm OS uses different character codes for the horizontal ellipsis character and the numeric space character than earlier versions did. Use this macro to return the appropriate code for numeric space regardless of which version of Palm OS your application is run on.

GetCharAttr

Purpose Return a pointer to the character attribute array. This array is used by the character classification and character conversion macros (such as `isalpha`).

Prototype

```
UInt16* GetCharAttr (void)
```

Parameters None

Result A pointer to the attributes array. This is an array of 256 `UInt16` values, one for each possible character code. See `CharAttr.h` for an explanation of the attributes.

Compatibility This function is **not** implemented if International Feature Set is present.

NOTE: This function is provided for backwards compatibility only. Use Text Manager functions instead on systems that support the text manager.

See Also `TxtCharAttr`, `TxtCharXAttr`

GetCharCaselessValue

Purpose Return a pointer to an array that maps all characters to an assigned caseless and accentless value. Use this function for finding text.

Prototype

```
UInt8* GetCharCaselessValue (void)
```

Parameters None.

Result Returns a pointer to the sort array, which is an array of 256 bytes.

Comment The `GetCharCaselessValue` conversion table converts each character into a numeric value that is caseless and sorted according to Microsoft Windows sorting rules:

- Punctuation characters have the lowest values,
- followed by numbers,
- followed by alpha characters.

  All forms of each alpha character have equivalent values, so that e = E = e-grave = e-circumflex, etc.

This conversion table is used by all the Palm OS sorting and comparison routines to yield caseless searches and caseless sorts in the almost same order as Windows-based programs, except that Palm OS routines produce the same sorting for all locales.

Compatibility This function is **not** implemented if International Feature Set is present.

NOTE: This function is provided for backwards compatibility only. Use Text Manager functions instead on systems that support the text manager.

GetCharSortValue

Purpose Return a pointer to an array that maps all characters to an assigned sorting value. Use this function for ordering (sorting) text.

Prototype

```
UInt8* GetCharSortValue (void)
```

Parameters None.

Result Returns a pointer to the attributes array. This is an array of 256 `UInt8` values, one for each possible character code.

Compatibility This function is **not** implemented if International Feature Set is present.

NOTE: This function is provided for backwards compatibility only. Use Text Manager functions instead on systems that support the text manager.

28

Data and Resource Manager

This chapter describes the data manager and the resource manager API declared in the header file `DataMgr.h`. It discusses the following topics:

- Data Manager Data Structures
- Data Manager Constants
- Data Manager Functions
- Application-Defined Functions

For more information on the data and resource managers, see the chapter "Files and Databases" in the *Palm OS Programmer's Companion*, vol. I.

Data Manager Data Structures

DmOpenRef

The `DmOpenRef` type defines a pointer to an open database. The database pointer is created and returned by `DmOpenDatabase`. It is used in any function that requires access to an open database.

```
typedef void *DmOpenRef
```

DmResID

The `DmResID` type defines a resource identifier. You assign each resource an ID at creation time. Note that resource IDs greater than or equal to 10000 are reserved for system use.

```
typedef UInt16 DmResID;
```

DmResType

The `DmResType` type defines the type of a resource. The resource type is a four-character code such as `'Tbmp'` for bitmap resources.

```
typedef UInt32 DmResType;
```

SortRecordInfoType

The `SortRecordInfoType` structure specifies information that might be used to sort a record. It is used in the database sorting functions. To create this structure, you can call `DmRecordInfo`, which returns these values for a given record.

```
typedef struct {
  UInt8 attributes;
  UInt8 uniqueID[3];
} SortRecordInfoType;

typedef SortRecordInfoType *SortRecordInfoPtr;
```

Field Descriptions

`attributes` The record's attributes. See "Record Attribute Constants."

`uniqueID` The unique identifier for the record.

Data Manager Constants

Category Constants

The following constants are used to specify information about categories:

Constant	Value	Description
`dmAllCategories`	0xFF	A mask used to represent all categories.
`dmCategoryLength`	16	The length of a category name. Currently, this is 16 bytes, which includes the null terminator.

Constant	Value	Description
dmRecAttrCategoryMask	0x0F	A mask used to retrieve the category information from the record's attributes field.
dmRecNumCategories	16	The number of categories allowed. Currently, this is 16, which includes the "Unfiled" category.
dmUnfiledCategory	0	A mask used to indicate the Unfiled category.

Record Attribute Constants

The following constants specify a database record's attributes.

Constant	Value	Description
dmMaxRecordIndex	0xFFFF	Indicates the highest record index allowed.
dmAllRecAttrs	0xF0	A mask used to specify all record attributes.
dmRecAttrBusy	0x20	Busy (the application has locked access to this record). A call to DmGetRecord fails on a record that has this bit set, otherwise it sets this bit. Call DmReleaseRecord to release the record and clear this bit. The DmSetRecordInfo function cannot be used to alter the state of dmRecAttrBusy.
dmRecAttrDelete	0x80	Deleted
dmRecAttrDirty	0x40	Dirty (has been modified since last sync)
dmRecAttrSecret	0x10	Private
dmSysOnlyRecAttrs	0x20	A mask used to specify record attributes that only the system can change. (In other words, the busy attribute.)

Database Attribute Constants

The following constants define a database's attributes:

Constant	Description
`dmAllHdrAttrs`	A mask used to specify all header attributes.
`dmDBNameLength`	Maximum length of a database's name. Currently, this is 32 bytes, which include the null terminator. Note that database names must use only 7-bit ASCII characters (0x20 through 0x7E).
`dmHdrAttrAppInfoDirty`	The application info block is dirty (has been modified since the last sync).
`dmHdrAttrBackup`	The database should be backed up to the desktop computer if no application-specific conduit is available.
`dmHdrAttrBundle`	The database is bundled with its application during a beam. That is, if the user chooses to beam the application from the Launcher, the Launcher beams this database along with the application's resource database and overlay database. This attribute applies to Palm OS® 4.0 and higher. Note that overlay databases are automatically beamed with the application database on Palm OS 4.0 and higher. You do not need to set this bit in overlay databases.
`dmHdrAttrCopyPrevention`	Prevents the database from being copied by methods such as IR beaming.

Constant	Description
dmHdrAttrHidden	This database should be hidden from view. For example, this attribute is set to hide some applications in the launcher's main view. You can set it on record databases to have the launcher disregard the database's records when showing a count of records. This attribute applies to Palm OS version 3.2 and higher.
dmHdrAttrLaunchableData	This database is a data database but it can be "launched" from the launcher. For example, this attribute is set in Palm Query Applications (PQAs) launched by the Web Clipping Application Viewer application.
dmHdrAttrOpen	The database is open.
dmHdrAttrOKToInstallNewer	The backup conduit can install a newer version of this database with a different name if the current database is open. This mechanism is used to update the Graffiti® Shortcuts database, for example.
dmHdrAttrReadOnly	The database is a read-only database.
dmHdrAttrRecyclable	The database is recyclable. Recyclable databases are deleted when they are closed or upon a system reset. This attribute applies to Palm OS 4.0 and higher.
dmHdrAttrResDB	The database is a resource database.
dmHdrAttrResetAfterInstall	The device must be reset after this database is installed. That is, the HotSync® application forces a reset after installing this database.

Constant	Description
dmHdrAttrStream	The database is a file stream.
dmSysOnlyHdrAttrs	A mask specifying the attributes that only the system can change (open and resource database).

Error Codes

The following constants define error codes that are returned by the data manager and resource manager functions. Several functions return a failure value such as NULL or 0 instead of an error code. In many cases, you can call DmGetLastErr upon receiving this value and receive a more descriptive error code.

Also, note that on releases prior to Palm OS release 3.5, many data manager functions display a fatal error message using the ErrFatalDisplayIf macro if certain error conditions are true. Because the Palm OS ROMs are usually shipped with error checking set to partial, you receive the fatal error message. If a ROM is built with error checking set to none, the function returns one of the error codes listed here. (Note that Palm™ has never released a ROM with error checking set to none and has no plans to do so.)

Constant	Description
dmErrAlreadyExists	Another database with the same name already exists in RAM store.
dmErrAlreadyOpenForWrites	The database is already open with write access.
dmErrCantFind	The specified resource can't be found.
dmErrCantOpen	The database cannot be opened.
dmErrCorruptDatabase	The database is corrupted.

Constant	Description
dmErrDatabaseOpen	The function cannot be performed on an open database, and the database is open.
dmErrDatabaseNotProtected	DmDatabaseProtect failed to protect the specified database.
dmErrIndexOutOfRange	The specified index is out of range.
dmErrInvalidDatabaseName	The name you've specified for the database is invalid.
dmErrInvalidParam	The function received an invalid parameter.
dmErrMemError	A memory error occurred.
dmErrNoOpenDatabase	The function is to search all open databases, but there are none.
dmErrNotRecordDB	You've attempted to perform a record function on a resource database.
dmErrNotResourceDB	You've attempted to perform a resource manager function on a record database.
dmErrNotValidRecord	The record handle is invalid.
dmErrOpenedByAnotherTask	You've attempted to open a database that another task already has open.
dmErrReadOnly	You've attempted to write to or modify a database that is in read-only mode.

Constant	Description
dmErrRecordArchived	The function requires that the record not be archived, but it is.
dmErrRecordBusy	The function requires that the record not be busy, but it is.
dmErrRecordDeleted	The record has been deleted.
dmErrRecordInWrongCard	You've attempted to attach a record to a database when the record and database reside on different memory cards.
dmErrResourceNotFound	The resource can't be found.
dmErrROMBased	You've attempted to delete or modify a ROM-based database.
dmErrSeekFailed	The operation of seeking the next record in the category failed.
dmErrUniqueIDNotFound	A record with the specified unique ID can't be found.
dmErrWriteOutOfBounds	A write operation exceeded the bounds of the record.
memErrCardNotPresent	The specified card can't be found.
memErrChunkLocked	The associated memory chunk is locked.
memErrInvalidParam memErrNotEnoughSpace	A memory error occurred.

Constant	Description
`memErrInvalidStoreHeader` `memErrRAMOnlyCard`	The specified card has no storage RAM.
`omErrBaseRequiresOverlay`	An attempt was made to open a stripped resource database, but no associated overlay could be found.
`omErrUnknownLocale`	An attempt was made to open a resource database with overlays using an unknown locale.

Open Mode Constants

The following constants define the mode in which a database can be opened. You pass one or more of these as a parameter to `DmOpenDatabase`, `DmOpenDatabaseByTypeCreator`, or `DmOpenDBNoOverlay`:

Constant	Description
`dmModeReadWrite`	Read-write access.
`dmModeReadOnly`	Read-only access.
`dmModeWrite`	Write-only access.
`dmModeLeaveOpen`	Leave database open even after application quits.
`dmModeExclusive`	Don't let anyone else open this database.
`dmModeShowSecret`	Show records marked private.

Data Manager Functions

DmArchiveRecord

Purpose Mark a record as archived by leaving the record's chunk intact and setting the delete bit for the next sync.

Prototype

```
Err DmArchiveRecord (DmOpenRef dbP, UInt16 index)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `index`	Which record to archive.

Result Returns `errNone` if no error, or one of the following if an error occurs:

- `dmErrReadOnly`
- `dmErrIndexOutOfRange`
- `dmErrRecordArchived`
- `dmErrRecordDeleted`
- `memErrInvalidParam`

Some releases may display a fatal error message instead of returning the error code.

Comments When a record is archived, the deleted bit is set but the chunk is not freed and the local ID is preserved. This way, the next time the user synchronizes with the desktop system, the desktop can save the record data on the PC before it permanently removes the record entry and data from the Palm Powered™ device.

Based on the assumption that a call to `DmArchiveRecord` indicates that you are finished with the record and aren't going to refer to it again, this function sets the chunk's lock count to zero.

See Also `DmRemoveRecord`, `DmDetachRecord`, `DmNewRecord`, `DmDeleteRecord`

DmAttachRecord

Purpose Attach an existing chunk ID handle to a database as a record.

Prototype

```
Err DmAttachRecord (DmOpenRef dbP, UInt16 *atP,
MemHandle newH, MemHandle *oldHP)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
<-> `atP`	Pointer to the index where the new record should be placed. Specify the value `dmMaxRecordIndex` to add the record to the end of the database.
-> `newH`	Handle of the new record.
<-> `oldHP`	If non-`NULL` upon entry, indicates that the record at `*atP` should be replaced. Upon return, contains the handle to the replaced record.

Result Returns `errNone` if no error, or one of the following if an error occurs:

- `dmErrMemError`
- `memErrChunkLocked`
- `memErrInvalidParam`
- `memErrNotEnoughSpace`
- `dmErrReadOnly`
- `dmErrNotRecordDB`
- `dmErrRecordInWrongCard`
- `dmErrIndexOutOfRange`

Some releases may display a fatal error message instead of returning some of these error codes.

Comments Given the handle of an existing chunk, this routine makes that chunk a new record in a database and sets the dirty bit. The parameter `atP` points to an index variable. If `oldHP` is `NULL`, the new record is inserted at index `*atP` and all record indices that

follow are shifted down. If *atP is greater than the number of records currently in the database, the new record is appended to the end and its index is returned in *atP. If oldHP is not NULL, the new record replaces an existing record at index *atP and the handle of the old record is returned in *oldHP so that the application can free it or attach it to another database.

This function is useful for cutting and pasting between databases.

See Also DmDetachRecord, DmNewRecord, DmNewHandle, DmFindSortPosition

DmAttachResource

Purpose Attach an existing chunk ID to a resource database as a new resource.

Prototype

```
Err DmAttachResource (DmOpenRef dbP,
MemHandle newH, DmResType resType, DmResID resID)
```

Parameters

-> dbP	DmOpenRef to open database.
-> newH	Handle of new resource's data.
-> resType	Type of the new resource.
-> resID	ID of the new resource.

Result Returns errNone if no error, or one of the following if an error occurs:

- dmErrMemError
- memErrChunkLocked
- memErrInvalidParam
- memErrNotEnoughSpace
- dmErrReadOnly
- dmErrRecordInWrongCard

Some releases may display a fatal error message instead of returning some of these error codes. All releases may display a fatal error message if the database is not a resource database.

Comments Given the handle of an existing chunk with resource data in it, this routine makes that chunk a new resource in a resource database. The new resource will have the given type and ID.

See Also DmDetachResource, DmRemoveResource, DmNewHandle, DmNewResource

DmCloseDatabase

Purpose Close a database.

Prototype

```
Err DmCloseDatabase (DmOpenRef dbP)
```

Parameters -> dbP Database access pointer.

Result Returns errNone if no error, or dmErrInvalidParam if an error occurs. Some releases may display a fatal error message instead of returning the error code.

Comments This routine doesn't unlock any records that were left locked. Records and resources should not be left locked. If a record/resource is left locked, you should not use its reference because the record can disappear during a HotSync operation or if the database is deleted by the user. To prevent the database from being deleted, you can use DmDatabaseProtect before closing.

If there is an overlay associated with the database passed in, DmCloseDatabase closes the overlay as well.

If the database has the recyclable bit set (dmHdrAttrRecyclable), DmCloseDatabase calls DmDeleteDatabase to delete it.

Compatibility Starting with Palm OS 2.0, DmCloseDatabase updates the database's modification date.

- On Palm OS 2.0, the modification date is updated if the database was opened with write access.
- On Palm OS 3.0 and higher, the modification date is updated only if a change has been made and the database was opened with write access. Changes that trigger an update include

adding, deleting, archiving, rearranging, or resizing records, setting a record's dirty bit in DmReleaseRecord, rearranging or deleting categories, or updating the database header fields using DmSetDatabaseInfo.

Under Palm OS 1.0, the modification date was never updated.

If you need to ensure that the modification date is updated the same way regardless of the operating system version, use DmSetDatabaseInfo to set the modification date explicitly.

See Also DmOpenDatabase, DmDeleteDatabase, DmOpenDatabaseByTypeCreator

DmCreateDatabase

Purpose Create a new database on the specified card with the given name, creator, and type.

Prototype

```
Err DmCreateDatabase (UInt16 cardNo,
const Char *nameP, UInt32 creator, UInt32 type,
Boolean resDB)
```

Parameters

-> cardNo	The card number to create the database on.
-> nameP	Name of new database, up to 32 ASCII bytes long, including the null terminator (as specified by dmDBNameLength). Database names must use only 7-bit ASCII characters (0x20 through 0x7E).
-> creator	Creator of the database.
-> type	Type of the database.
-> resDB	If true, create a resource database.

Result Returns errNone if no error, or one of the following if an error occurs:

- dmErrInvalidDatabaseName
- dmErrAlreadyExists
- memErrCardNotPresent

- dmErrMemError
- memErrChunkLocked
- memErrInvalidParam
- memErrInvalidStoreHeader
- memErrNotEnoughSpace
- memErrRAMOnlyCard

May display a fatal error message if the master database list cannot be found.

Comments Call this routine to create a new database on a specific card. If another database with the same name already exists in RAM store, this routine returns a dmErrAlreadyExists error code. Once created, the database ID can be retrieved by calling DmFindDatabase. The database can be opened using the database ID. To create a resource database instead of a record-based database, set the `resDB` Boolean to `true`.

After you create a database, it's recommended that you call DmSetDatabaseInfo to set the version number. Databases default to version 0 if the version isn't explicitly set.

See Also DmCreateDatabaseFromImage, DmOpenDatabase, DmDeleteDatabase

DmCreateDatabaseFromImage

Purpose Create an entire database from a single resource that contains an image of the database.

Prototype

```
Err DmCreateDatabaseFromImage (MemPtr bufferP)
```

Parameters

-> `bufferP`	Pointer to locked resource containing database image.

Result Returns `errNone` if no error.

Comments An image is the same as a desktop file representation of a prc or pdb file.

This function is intended for applications in the ROM to install default databases after a hard reset. RAM-based applications that want to install a default database should install a pdb file separately to save storage heap space.

See Also DmCreateDatabase, DmOpenDatabase

DmDatabaseInfo

Purpose Retrieve information about a database.

Prototype

```
Err DmDatabaseInfo (UInt16 cardNo, LocalID dbID,
Char *nameP, UInt16 *attributesP,
UInt16 *versionP, UInt32 *crDateP,
UInt32 *modDateP, UInt32 *bckUpDateP,
UInt32 *modNumP, LocalID *appInfoIDP,
LocalID *sortInfoIDP, UInt32 *typeP,
UInt32 *creatorP)
```

Parameters

-> `cardNo`	Number of the card the database resides on.
-> `dbID`	Database ID of the database.
<- `nameP`	The database's name. Pass a pointer to 32-byte character array for this parameter, or `NULL` if you don't care about the name.
<- `attributesP`	The database's attribute flags. The section "Database Attribute Constants" lists constants you can use to query the values returned in this parameter. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `versionP`	The application-specific version number. The default version number is 0. Pass `NULL` for this parameter if you don't want to retrieve it.

<- `crDateP`	The date the database was created, expressed as the number of seconds since the first instant of Jan. 1, 1904. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `modDateP`	The date the database was last modified, expressed as the number of seconds since the first instant of Jan. 1, 1904. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `bckUpDateP`	The date the database was backed up, expressed as the number of seconds since the first instant of Jan. 1, 1904. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `modNumP`	The modification number, which is incremented every time a record in the database is added, modified, or deleted. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `appInfoIDP`	The local ID of the application info block, or `NULL`. The application info block is an optional field that the database may use to store application-specific information about the database. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `sortInfoIDP`	The local ID of the database's sort table. This is an optional field in the database header. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `typeP`	The database's type, specified when it is created. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `creatorP`	The database's creator, specified when it is created. Pass `NULL` for this parameter if you don't want to retrieve it.

Result Returns `errNone` if no error, or `dmErrInvalidParam` if an error occurs.

Compatibility Updating of the modification date differs based on which version of the OS your application is running on.

- Under Palm OS 1.0, the modification date is never updated.
- Under Palm OS 2.0, the modification date is updated every time a database opened with write access is closed.
- Beginning with Palm OS 3.0, the modification date is updated only if a change has been made to the database opened with write access. (The update still occurs upon closing the database.) Changes that trigger an update include adding, deleting, archiving, rearranging, or resizing records, setting a record's dirty bit in DmReleaseRecord, rearranging or deleting categories, or updating the database header fields using DmSetDatabaseInfo.

If you need to ensure that the modification date is updated the same way regardless of the operating system version, use DmSetDatabaseInfo to set the modification date explicitly.

See Also DmSetDatabaseInfo, DmDatabaseSize, DmOpenDatabaseInfo, DmFindDatabase, DmGetNextDatabaseByTypeCreator, TimSecondsToDateTime

DmDatabaseProtect

Purpose Increment or decrement the database's protection count.

Prototype

```
Err DmDatabaseProtect (UInt16 cardNo,
LocalID dbID, Boolean protect)
```

Parameters

-> `cardNo`	Card number of database to protect/unprotect.
-> `dbID`	Local ID of database to protect/unprotect.
-> `protect`	If `true`, `protect` count will be incremented. If `false`, `protect` count will be decremented.

Result Returns `errNone` if no error, or one of the following if an error occurs:

- memErrCardNotPresent
- dmErrROMBased

- dmErrCantFind
- memErrNotEnoughSpace
- dmErrDatabaseNotProtected

Comments This routine can be used to prevent a database from being deleted (by passing `true` for the `protect` parameter). It increments the protect count if `protect` is `true` and decrements it if `protect` is `false`. All `true` calls should be balanced by `false` calls before the application terminates.

Use this function if you want to keep a particular record or resource in a database locked down but don't want to keep the database open. This information is kept in the dynamic heap, so all databases are "unprotected" at system reset.

If the database is a resource database that has an overlay associated with it for the current locale, the overlay is also protected or unprotected by this call.

Compatibility Implemented only if 2.0 New Feature Set is present. Overlay support is only available if 3.5 New Feature Set is present.

DmDatabaseSize

Purpose Retrieve size information on a database.

Prototype

```
Err DmDatabaseSize (UInt16 cardNo, LocalID dbID,
UInt32 *numRecordsP, UInt32 *totalBytesP,
UInt32 *dataBytesP)
```

Parameters

-> `cardNo`	Card number the database resides on.
-> `dbID`	Database ID of the database.
<- `numRecordsP`	The total number of records in the database. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `totalBytesP`	The total number of bytes used by the database including the overhead. Pass `NULL` for this parameter if you don't want to retrieve it.

	<- dataBytesP	The total number of bytes used to store just each record's data, not including overhead. Pass NULL for this parameter if you don't want to retrieve it.

Result Returns errNone if no error, or dmErrMemError if an error occurs.

See Also DmDatabaseInfo, DmOpenDatabaseInfo, DmFindDatabase, DmGetNextDatabaseByTypeCreator

DmDeleteCategory

Purpose Delete all records in a category. The category name is not changed.

Prototype

```
Err DmDeleteCategory (DmOpenRef dbR,
UInt16 categoryNum)
```

Parameters

-> dbR	Database access pointer.
-> categoryNum	Category of records to delete. Category masks such as dmAllCategories are invalid.

Result Returns errNone if no error, or one of the following if an error occurs:

- dmErrReadOnly
- memErrInvalidParam

Some releases may display a fatal error message instead of returning the error code.

Comments This function deletes all records in a category, but does not delete the category itself (note that it deletes the record data and header info, and doesn't just set the deleted bit). For each record in the category, DmDeleteCategory marks the delete bit in the database header for the record and disposes of the record's data chunk. The record entry in the database header remains, but its localChunkID is set to NULL.

If the category contains no records, this function does nothing and returns errNone to indicate success. The categoryNum parameter

is assumed to represent a single category. If you pass a category mask to specify more than one category, this function interprets that value as a single category, finds no records to delete in that category, and returns `errNone`.

You can use the `DmRecordInfo` call to obtain a category index from a given record. For example:

```
DmOpenRef myDB;
UInt16 record, attr, category, total;

DmRecordInfo(myDB, record, &attr, NULL, NULL);
category = attr & dmRecAttrCategoryMask;
err = DmDeleteCategory(myDB, category);
```

Compatibility Implemented only if 2.0 New Feature Set is present.

DmDeleteDatabase

Purpose Delete a database and all its records.

Prototype

```
Err DmDeleteDatabase (UInt16 cardNo, LocalID dbID)
```

Parameters

-> `cardNo`	Card number the database resides on.
-> `dbID`	Database ID.

Result Returns `errNone` if no error, or one of the following if an error occurs:

- `dmErrCantFind`
- `dmErrCantOpen`
- `memErrChunkLocked`
- `dmErrDatabaseOpen`
- `dmErrROMBased`
- `memErrInvalidParam`
- `memErrNotEnoughSpace`

Comments Call this routine to delete a database. This routine deletes the database, the application info block, the sort info block, and any other overhead information that is associated with this database. After deleting the database, this function enqueues a deferred `sysNotifyDBDeletedEvent` notification, which will be broadcast at the top of the event loop.

If the database has an overlay associated with it, this function does **not** delete the overlay. You can delete the overlay with a separate call to `DmDeleteDatabase`.

This routine accepts a database ID as a parameter. To determine the database ID, call either `DmFindDatabase` or `DmGetDatabase` with a database index.

Compatibility The `sysNotifyDBDeletedEvent` notification is only broadcast if the 4.0 New Feature Set is present.

See Also `DmDeleteRecord`, `DmRemoveRecord`, `DmRemoveResource`, `DmCreateDatabase`, `DmGetNextDatabaseByTypeCreator`, `DmFindDatabase`

DmDeleteRecord

Purpose Delete a record's chunk from a database but leave the record entry in the header and set the `delete` bit for the next sync.

Prototype
```
Err DmDeleteRecord (DmOpenRef dbP, UInt16 index)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `index`	Which record to delete.

Result Returns `errNone` if no error, or one of the following if an error occurs:

- `dmErrReadOnly`
- `dmErrIndexOutOfRange`
- `dmErrRecordArchived`
- `dmErrRecordDeleted`
- `memErrInvalidParam`

Some releases may display a fatal error message instead of returning the error code.

Comments Marks the `delete` bit in the database header for the record and disposes of the record's data chunk. Does not remove the record entry from the database header, but simply sets the `localChunkID` of the record entry to `NULL`.

See Also DmDetachRecord, DmRemoveRecord, DmArchiveRecord, DmNewRecord

DmDetachRecord

Purpose Detach and orphan a record from a database but don't delete the record's chunk.

Prototype

```
Err DmDetachRecord (DmOpenRef dbP, UInt16 index,
MemHandle *oldHP)
```

Parameters

-> `dbP`	`DmOpenRef` to open.
-> `index`	Index of the record to detach.
<-> `oldHP`	Pointer to return handle of the detached record.

Result Returns `errNone` if no error, or one of the following if an error occurs:

- dmErrReadOnly
- dmErrIndexOutOfRange
- dmErrNotRecordDB
- memErrChunkLocked
- memErrInvalidParam

Some releases may display a fatal error message instead of returning the error code.

Comments This routine detaches a record from a database by removing its entry from the database header and returns the handle of the record's data chunk in `*oldHP`. Unlike DmDeleteRecord, this

routine removes its entry in the database header but it does not delete the actual record.

See Also DmAttachRecord, DmRemoveRecord, DmArchiveRecord, DmDeleteRecord

DmDetachResource

Purpose Detach a resource from a database and return the handle of the resource's data.

Prototype

```
Err DmDetachResource (DmOpenRef dbP, UInt16 index,
MemHandle *oldHP)
```

Parameters

-> dbP	DmOpenRef to open database.
-> index	Index of resource to detach.
<-> oldHP	Pointer to return handle of the detached record.

Result Returns errNone if no error, or one of the following if an error occurs:

- dmErrReadOnly
- dmErrIndexOutOfRange
- dmErrCorruptDatabase
- memErrChunkLocked
- memErrInvalidParam

Some releases may display a fatal error message instead of returning the error code. All releases may display a fatal error message if the database is not a resource database.

Comments This routine detaches a resource from a database by removing its entry from the database header and returns the handle of the resource's data chunk in *oldHP.

See Also DmAttachResource, DmRemoveResource

DmFindDatabase

Purpose Return the database ID of a database by card number and name.

Prototype
```
LocalID DmFindDatabase (UInt16 cardNo,
const Char *nameP)
```

Parameters

-> `cardNo`	Number of card to search.
-> `nameP`	Name of the database to look for.

Result Returns the database ID. If the database can't be found, this function returns 0, and `DmGetLastErr` returns an error code indicating the reason for failure.

See Also `DmGetNextDatabaseByTypeCreator`, `DmDatabaseInfo`, `DmOpenDatabase`

DmFindRecordByID

Purpose Return the index of the record with the given unique ID.

Prototype
```
Err DmFindRecordByID (DmOpenRef dbP,
UInt32 uniqueID, UInt16 *indexP)
```

Parameters

-> `dbP`	Database access pointer.
-> `uniqueID`	Unique ID to search for.
<- `indexP`	Return index.

Result Returns 0 if found, otherwise `dmErrUniqueIDNotFound`. May display a fatal error message if the unique ID is invalid.

See Also `DmQueryRecord`, `DmGetRecord`, `DmRecordInfo`

DmFindResource

Purpose Search the given database for a resource by type and ID, or by pointer if it is non-NULL.

Prototype

```
UInt16 DmFindResource (DmOpenRef dbP,
DmResType resType, DmResID resID, MemHandle resH)
```

Parameters

-> dbP	Open resource database access pointer.
-> resType	Type of resource to search for.
-> resID	ID of resource to search for.
->resH	Pointer to locked resource, or NULL.

Result Returns index of resource in resource database, or -1 if not found.

May display a fatal error message if the database is not a resource database.

Comments Use this routine to find a resource in a particular resource database by type and ID or by pointer. It is particularly useful when you want to search only one database for a resource and that database is not the topmost one.

IMPORTANT: This function searches for the resource only in the database you specify. If you pass a pointer to a base resource database, its overlay is **not** searched. To search both a base database and its overlay for a localized resource, use DmGet1Resource instead of this function.

If resH is NULL, the resource is searched for by type and ID.

If resH is not NULL, resType and resID are ignored and the index of the given locked resource is returned.

Once the index of a resource is determined, it can be locked down and accessed by calling DmGetResourceIndex.

See Also DmGetResource, DmSearchResource, DmResourceInfo, DmGetResourceIndex, DmFindResourceType

DmFindResourceType

Purpose Search the given database for a resource by type and type index.

Prototype

```
UInt16 DmFindResourceType (DmOpenRef dbP,
DmResType resType, UInt16 typeIndex)
```

Parameters

-> `dbP`	Open resource database access pointer.
-> `resType`	Type of resource to search for.
-> `typeIndex`	Index of given resource type.

Result Index of resource in resource database, or -1 if not found.

May display a fatal error message if the database is not a resource database.

Comments Use this routine to retrieve all the resources of a given type in a resource database. By starting at `typeIndex` 0 and incrementing until an error is returned, the total number of resources of a given type and the index of each of these resources can be determined. Once the index of a resource is determined, it can be locked down and accessed by calling `DmGetResourceIndex`.

IMPORTANT: This function searches for resources only in the database you specify. If you pass a pointer to a base resource database, its overlay is **not** searched. To search both a base database and its overlay for a localized resource, use `DmGet1Resource` instead of this function.

See Also `DmGetResource`, `DmSearchResource`, `DmResourceInfo`, `DmGetResourceIndex`, `DmFindResource`

DmFindSortPosition

Purpose Returns where in a sorted list of records a given record would be located. Useful to find where to insert a record with DmAttachRecord. Uses a binary search.

Prototype

```
UInt16 DmFindSortPosition (DmOpenRef dbP,
void *newRecord, SortRecordInfoPtr newRecordInfo,
DmComparF *compar, Int16 other)
```

Parameters

-> dbP	Database access pointer.
-> newRecord	Pointer to the new record.
-> newRecordInfo	Sort information about the new record. See SortRecordInfoType.
-> compar	Pointer to comparison function. See DmComparF.
-> other	Any value the application wants to pass to the comparison function. This parameter is often used to indicate a sort direction (ascending or descending).

Result The position where the record should be inserted.

The position should be viewed as between the record returned and the record before it. Note that the return value may be one greater than the number of records.

Comments If newRecord has the same key as another record in the database, DmFindSortPosition assumes that newRecord should be inserted after that record. If there are several records with the same key, newRecord is inserted after all of them. For this reason, if you use DmFindSortPosition to search for the location of a record that you know is already in the database, you must subtract 1 from the result. (Be sure to check that the value is not 0.)

If there are deleted records in the database, DmFindSortPosition only works if those records are at the end of the database.

DmFindSortPosition always assumes that a deleted record is greater than or equal to any other record.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also DmFindSortPositionV10

DmFindSortPositionV10

Purpose Return where a record should be. Useful to find where to insert a record with DmAttachRecord. Uses a binary search.

Prototype

```
UInt16 DmFindSortPositionV10 (DmOpenRef dbP,
void *newRecord, DmComparF *compar, Int16 other)
```

Parameters

-> dbP	Database access pointer.
-> newRecord	Pointer to the new record.
-> compar	Pointer to comparison function. See DmComparF.
-> other	Any value the application wants to pass to the comparison function.

Result Returns the position where the record should be inserted. The position should be viewed as between the record returned and the record before it. Note that the return value may be one greater than the number of records.

Comments If there are deleted records in the database, DmFindSortPositionV10 only works if those records are at the end of the database. DmFindSortPositionV10 always assumes that a deleted record is greater than or equal to any other record.

Compatibility This function corresponds to the 1.0 version of DmFindSortPosition.

See Also DmFindSortPosition, DmQuickSort, DmInsertionSort

DmGetAppInfoID

Purpose Return the local ID of the application info block.

Prototype
```
LocalID DmGetAppInfoID (DmOpenRef dbP).
```

Parameters

-> `dbP`	Database access pointer.

Result Returns local ID of the application info block. The application info block is an optional field that the database may use to store application-specific information about the database; if the database doesn't have an application info block, `DmGetAppInfoID` returns zero.

See Also `DmDatabaseInfo`, `DmOpenDatabase`

DmGetDatabase

Purpose Return the database header ID of a database by index and card number.

Prototype
```
LocalID DmGetDatabase (UInt16 cardNo,
UInt16 index)
```

Parameters

-> `cardNo`	Card number of database.
-> `index`	Index of database.

Result Returns the database ID, or 0 if an invalid parameter is passed.

Comments Call this routine to retrieve the database ID of a database by index. The index should range from 0 to `DmNumDatabases`-1.

This routine is useful for getting a directory of all databases on a card. The databases returned may reside in either the ROM or the RAM. The order in which databases are returned is not fixed; therefore, you should not rely on receiving a list of databases in a particular order.

See Also `DmOpenDatabase`, `DmNumDatabases`, `DmDatabaseInfo`, `DmDatabaseSize`

DmGetDatabaseLockState

Purpose

Return information about the number of locked and busy records in a database.

Prototype

```
void DmGetDatabaseLockState (DmOpenRef dbR,
UInt8 *highest, UInt32 *count, UInt32 *busy)
```

Parameters

-> `dbR`	Database access pointer.
<- `highest`	The highest lock count found for all of the records in the database. If a database has two records, one has a lock count of 2 and one has a lock count of 1, the highest lock count is 2. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `count`	The number of records that have the lock count that is returned in the `highest` parameter. Pass `NULL` for this parameter if you don't want to retrieve it.
<- `busy`	The number of records that have the busy bit set. Pass `NULL` for this parameter if you don't want to retrieve it.

Result

No return value. Returns all information in the parameters you pass.

Comments

This function is intended to be used for debugging purposes. You can use it to obtain information about how many records are busy and how much locking occurs.

Compatibility

Implemented only if 3.2 New Feature Set is present.

DmGetLastErr

Purpose Return error code from last data manager call.

Prototype

```
Err DmGetLastErr (void)
```

Parameters None.

Result Error code from last unsuccessful data manager call.

Comments Use this routine to determine why a data manager call failed. In particular, calls like DmGetRecord return 0 if unsuccessful, so calling DmGetLastErr is the only way to determine why they failed.

Note that DmGetLastErr does not always reflect the error status of the last data manager call. Rather, it reflects the error status of data manager calls that don't return an error code. For some of those calls, the saved error code value is not set to 0 when the call is successful.

For example, if a call to DmOpenDatabaseByTypeCreator returns NULL for database reference (that is, it fails), DmGetLastErr returns something meaningful; otherwise, it returns the error value of some previous data manager call.

Only the following data manager functions currently affect the value returned by DmGetLastErr:

DmFindDatabase	DmOpenDatabaseByTypeCreator
DmOpenDatabase	DmNewRecord
DmQueryRecord	DmGetRecord
DmQueryNextInCategory	DmPositionInCategory
DmSeekRecordInCategory	DmResizeRecord
DmGetResource	DmGet1Resource
DmNewResource	DmGetResourceIndex
DmNewHandle	DmOpenDBNoOverlay
DmResizeResource	

DmGetNextDatabaseByTypeCreator

Purpose

Return a database header ID and card number given the type and/or creator. This routine searches all memory cards for a match.

Prototype

```
Err DmGetNextDatabaseByTypeCreator
(Boolean newSearch, DmSearchStatePtr stateInfoP,
UInt32 type, UInt32 creator,
Boolean onlyLatestVers, UInt16 *cardNoP,
LocalID *dbIDP)
```

Parameters

-> `newSearch`	`true` if starting a new search.
<-> `stateInfoP`	If `newSearch` is `false`, this must point to the same data used for the previous invocation.
-> `type`	Type of database to search for, pass 0 as a wildcard.
-> `creator`	Creator of database to search for, pass 0 as a wildcard.
-> `onlyLatestVers`	If `true`, only the latest version of a database with a given type and creator is returned.
<- `cardNoP`	On exit, the card number of the found database.
<- `dbIDP`	Database local ID of the found database.

Result

Returns `errNone` if no error, or `dmErrCantFind` if no matches were found.

Comments

You may need to call this function successively to discover all databases having a specified type/creator pair.

To start the search, pass `true` for `newSearch`. Allocate a `DmSearchStateType` structure and pass it as the `stateInfoP` parameter. `DmGetNextDatabaseByTypeCreator` stores private information in `stateInfoP` and uses it if the search is continued.

To continue a search where the previous one left off, pass `false` for `newSearch` and pass the same `stateInfoP` that you used during the previous call to this function.

You can pass `NULL` as a wildcard operator for the `type` or `creator` parameter to conduct searches of wider scope. If the `type` parameter is `NULL`, this routine can be called successively to return all databases of the given creator. If the `creator` parameter is `NULL`, this routine can be called successively to return all databases of the given type. You can also pass `NULL` as the value for both of these parameters to return all available databases without regard to type or creator.

Because databases are scattered freely throughout memory space, they are not returned in any particular order—any database matching the specified type/creator criteria can be returned.Thus, if the value of the `onlyLatestVers` parameter is `false`, this function may return a database which is not the most recent version matching the specified type/creator pair. To obtain only the latest version of a database matching the search criteria, set the value of the `onlyLatestVers` parameter to `true`.

When determining which is the latest version of the database, RAM databases are considered newer than ROM databases that have the same version number. Because of this, you can replace any ROM-based application with your own version of it. Also, a RAM database on card 1 is considered newer than a RAM database on card 0 if the version numbers are identical.

WARNING! Don't create or delete a database while using `DmGetNextDatabaseByTypeCreator` to iterate through the existing databases. This could cause databases to be skipped, or it could result in a given database being returned more than once.

Compatibility

In Palm OS version 3.1 and higher, if `onlyLatestVers` is `true`, you only receive one matching database for each type/creator pair. In version 3.0 and earlier, you could receive multiple matching databases if `onlyLatestVers` was `true`.

Note that the behavior is different only when you have specified a value for both `type` and `creator` and `onlyLatestVers` is `true`.

For example, suppose your application maintains three databases that all have the same type, creator, and version number and you write this code to process them in some way:

```
DmSearchStateType state;
Boolean latestVer;
UInt16 card;
LocalID currentDB = 0;

theErr = DmGetNextDatabaseByTypeCreator(true,
  &state, myType, myCreator, latestVer, &card,
  &currentDB);
while (!theErr && currentDB) {
  /* do something with currentDB */
  /* now get the next DB */
  theErr = DmGetNextDatabaseByTypeCreator(
    false, &state, myType, myCreator,
  vlatestVer, &card, &currentDB);
}
```

If `latestVer` is `false`, then your code will work the same on all versions of Palm OS and will return all three databases whose type and creator match those specified. If `latestVer` is `true`, this code returns all three databases on Palm OS version 3.0 and earlier, but only returns one database on version 3.1 and higher. (Exactly which database it returns is unspecified.)

If you expect multiple databases to match your search criteria, make sure you call `DmGetNextDatabaseByTypeCreator` in one of the following ways to ensure that your code operates the same on all Palm OS versions:

- Set `onlyLatestVers` to `false` if you specify both a `type` and `creator`.
- Specify `NULL` for either the `type` or `creator` parameter (or both).

See Also DmGetDatabase, DmFindDatabase, DmDatabaseInfo, DmOpenDatabaseByTypeCreator, DmDatabaseSize

DmGetRecord

Purpose Return a handle to a record by index and mark the record busy.

Prototype
```
MemHandle DmGetRecord (DmOpenRef dbP,
UInt16 index)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `index`	Which record to retrieve.

Result Returns a handle to record data. If another call to `DmGetRecord` for the same record is attempted before the record is released, `NULL` is returned and `DmGetLastErr` returns an error code indicating the reason for failure.

Comments Returns a handle to given record and sets the `busy` bit for the record.

If the record is ROM-based (pointer accessed), this routine makes a fake handle to it and stores this handle in the `DmAccessType` structure.

`DmReleaseRecord` should be called as soon as the caller finishes viewing or editing the record.

See Also `DmSearchRecord`, `DmFindRecordByID`, `DmRecordInfo`, `DmReleaseRecord`, `DmQueryRecord`

DmGetResource

Purpose Search all open resource databases and return a handle to a resource, given the resource type and ID.

Prototype
```
MemHandle DmGetResource (DmResType type,
DmResID resID)
```

Parameters

-> `type`	The resource type.
->`resID`	The resource ID.

Result Handle to resource data. If the specified resource cannot be found, this function returns `NULL` and `DmGetLastErr` returns an error code indicating the reason for failure.

Comments Searches all open resource databases starting with the most recently opened one for a resource of the given type and ID. If found, the resource handle is returned. The application should call `DmReleaseResource` as soon as it finishes accessing the resource data. The resource handle is not locked by this function.

This function always returns the resource located in the overlay if any open overlay has a resource matching that type and ID. If there is no overlay version of the resource, this function returns the resource from the base database.

See Also `DmGet1Resource`, `DmReleaseResource`, `ResLoadConstant`

DmGetResourceIndex

Purpose Return a handle to a resource by index.

Prototype

```
MemHandle DmGetResourceIndex (DmOpenRef dbP,
UInt16 index)
```

Parameters

-> `dbP`	Access pointer to open database.
-> `index`	Index of the resource whose handle you want.

Result Handle to resource data. If the specified index is out of range, this function returns `NULL` and `DmGetLastErr` returns an error code indicating the reason for failure.

May display a fatal error message if the database is not a resource database.

IMPORTANT: This function accesses the resource only in the database you specify. If you pass a pointer to a base resource database, its overlay is **not** accessed. Therefore, you should use care when using this function to access a potentially localized resource. You can use DmSearchResource to obtain a pointer to the overlay database if the resource is localized; however, it's more convenient to use DmGetResource or DmGet1Resource.

See Also DmFindResource, DmFindResourceType, DmSearchResource

DmGet1Resource

Purpose Search the most recently opened resource database and return a handle to a resource given the resource type and ID.

Prototype
```
MemHandle DmGet1Resource (DmResType type,
DmResID resID)
```

Parameters

-> `type`	The resource type.
-> `resID`	The resource ID.

Result Handle to resource data. If unsuccessful, this function returns `NULL` and DmGetLastErr returns an error code indicating the reason for failure.

Comments Searches the most recently opened resource database for a resource of the given type and ID. If the database has an overlay associated with it, the overlay is searched first, and then the base database is searched if the overlay does not contain the resource. If found, the resource handle is returned. The application should call DmReleaseResource as soon as it finishes accessing the resource data. The resource handle is not locked by this function.

See Also DmGetResource, DmReleaseResource, ResLoadConstant

DmInsertionSort

Purpose Sort records in a database.

Prototype

```
Err DmInsertionSort (DmOpenRef dbR,
DmComparF *compar, Int16 other)
```

Parameters

-> `dbR`	Database access pointer.
-> `compar`	Comparison function. See `DmComparF`.
-> `other`	Any value the application wants to pass to the comparison function. This parameter is often used to indicate a sort direction (ascending or descending).

Result Returns `errNone` if no error, or one of the following if an error occurs:

- `dmErrReadOnly`
- `dmErrNotRecordDB`

Some releases may display a fatal error message instead of returning the error code.

Comments Deleted records are placed last in any order. All others are sorted according to the passed comparison function. Only records which are out of order move. Moved records are moved to the end of the range of equal records. If a large number of records are being sorted, try to use the quick sort.

The following insertion-sort algorithm is used: Starting with the second record, each record is compared to the preceding record. Each record not greater than the last is inserted into sorted position within those already sorted. A binary insertion is performed. A moved record is inserted after any other equal records.

See Also `DmQuickSort`

DmMoveCategory

Purpose Move all records in a category to another category.

Prototype

```
Err DmMoveCategory (DmOpenRef dbP,
UInt16 toCategory, UInt16 fromCategory,
Boolean dirty)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
->`toCategory`	Category to which the records should be added.
-> `fromCategory`	Category from which to remove records.
-> `dirty`	If `true`, set the dirty bit.

Result Returns 0 if successful, or `dmErrReadOnly` if the database is in read-only mode. Some releases may display a fatal error message instead of returning the error code.

Comments If `dirty` is `true`, the moved records are marked as dirty.

The `toCategory` and `fromCategory` parameters hold category index values. You can learn which category a record is in with the `DmRecordInfo` call and use that value in this function. For example, the following code, ensures that the records `rec1` and `rec2` are in the same category:

```
DmOpenRef myDB;
UInt16 rec1, rec2;
UInt16 rec1Attr, rec2Attr;
UInt16 category1, category2;

DmRecordInfo (myDB, rec1, &rec1Attr, NULL,
  NULL);
category1 = rec1Attr & dmRecAttrCategoryMask;
DmRecordInfo(myDB, rec2, &rec2Attr, NULL,
  NULL);
category2 = rec2Attr & dmRecAttrCategoryMask;
if (category1 != category2)
  DmMoveCategory(myDB, category1, category2,
    true);
```

DmMoveRecord

Purpose Move a record from one index to another.

Prototype

```
Err DmMoveRecord (DmOpenRef dbP, UInt16 from,
UInt16 to)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `from`	Index of record to move.
-> `to`	Where to move the record.

Result Returns `errNone` if no error, or one of the following if an error occurs:

- `dmErrReadOnly`
- `dmErrIndexOutOfRange`
- `dmErrNotRecordDB`
- `dmErrMemError`
- `memErrInvalidParam`
- `memErrChunkLocked`

Some releases may display a fatal error message instead of returning the error code.

Comments Insert the record at the `to` index and move other records down. The `to` position should be viewed as an insertion position. This value may be one greater than the index of the last record in the database. In cases where `to` is greater than `from`, the new index of the record becomes `to`–1 after the move is complete.

DmNewHandle

Purpose Attempt to allocate a new chunk in the same data heap or card as the database header of the passed database access pointer. If there is not enough space in that data heap, try other heaps.

Prototype
```
MemHandle DmNewHandle (DmOpenRef dbP, UInt32 size)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `size`	Size of new handle.

Result Returns the `chunkID` of new chunk. If an error occurs, returns 0, and `DmGetLastErr` returns an error code indicating the reason for failure.

Comments Allocates a new handle of the given size. Ensures that the new handle is in the same memory card as the given database. This guarantees that you can attach the handle to the database as a record to obtain and save its `LocalID` in the `appInfoID` or `sortInfoID` fields of the header.

The handle should be attached to a database as soon as possible. If it is not attached to a database and the application crashes, the memory used by the new handle is unavailable until the next soft reset.

DmNewRecord

Purpose Return a handle to a new record in the database and mark the record busy.

Prototype
```
MemHandle DmNewRecord (DmOpenRef dbP, UInt16 *atP,
UInt32 size)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
<-> `atP`	Pointer to index where new record should be placed. Specify the value `dmMaxRecordIndex` to add the record to the end of the database.

-> `size`	Size of new record.

Result Handle to record data. If an error occurs, this function returns 0 and `DmGetLastErr` returns an error code indicating the reason for failure.

Some releases may display a fatal error message if the database is opened in read-only mode or it is a resource database.

Comments Allocates a new record of the given size, and returns a handle to the record data. The parameter `atP` points to an index variable. The new record is inserted at index `*atP` and all record indices that follow are shifted down. If `*atP` is greater than the number of records currently in the database, the new record is appended to the end and its index is returned in `*atP`.

Both the `busy` and `dirty` bits are set for the new record and a unique ID is automatically created.

`DmReleaseRecord` should be called as soon as the caller finishes viewing or editing the record.

See Also `DmAttachRecord`, `DmRemoveRecord`, `DmDeleteRecord`

DmNewResource

Purpose Allocate and add a new resource to a resource database.

Prototype

```
MemHandle DmNewResource (DmOpenRef dbP,
DmResType resType, DmResID resID, UInt32 size)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `resType`	Type of the new resource.
-> `resID`	ID of the new resource.
-> `size`	Desired size of the new resource.

Result Returns a handle to the new resource. If an error occurs, this function returns `NULL` and `DmGetLastErr` returns an error code indicating the reason for failure.

May display a fatal error message if the database is not a resource database.

Comments Allocates a memory chunk for a new resource and adds it to the given resource database. The new resource has the given type and ID. If successful, the application should call DmReleaseResource as soon as it finishes initializing the resource.

See Also DmAttachResource, DmRemoveResource

DmNextOpenDatabase

Purpose Return DmOpenRef to next open database for the current task.

Prototype
```
DmOpenRef DmNextOpenDatabase (DmOpenRef currentP)
```

Parameters -> currentP Current database access pointer or NULL.

Result DmOpenRef to next open database, or NULL if there are no more.

Comments Call this routine successively to get the DmOpenRefs of all open databases. Pass NULL for currentP to get the first one. Applications don't usually call this function, but is useful for system information.

See Also DmOpenDatabaseInfo, DmDatabaseInfo

DmNextOpenResDatabase

Purpose Return access pointer to next open resource database in the search chain.

Prototype
```
DmOpenRef DmNextOpenResDatabase (DmOpenRef dbP)
```

Parameters -> dbP Database reference, or 0 to start search from the top.

Result Pointer to next open resource database.

Comments Returns pointer to next open resource database. To get a pointer to the first one in the search chain, pass `NULL` for `dbP`. This is the database that is searched when `DmGet1Resource` is called.

If you use this function to access a resource database that might have an overlay associated with it, be careful how you use the result. The `DmOpenRef` returned by this function is a pointer to the overlay database, not the base database. If you subsequently pass this pointer to `DmFindResource`, you'll receive a handle to the overlaid resource. If you're searching for a resource that is found only in the base, you won't find it. Instead, always use `DmGetResource` or `DmGet1Resource` to obtain a resource. Both of those functions search both the overlay databases and their associated base databases.

DmNumDatabases

Purpose Determine how many databases reside on a memory card.

Prototype
```
UInt16 DmNumDatabases (UInt16 cardNo)
```

Parameters -> `cardNo` Number of the card to check.

Result The number of databases found.

Comments This routine is helpful for getting a directory of all databases on a card. The routine `DmGetDatabase` accepts an index from 0 to `DmNumDatabases` -1 and returns a database ID by index.

See Also `DmGetDatabase`

DmNumRecords

Purpose Return the number of records in a database.

Prototype
```
UInt16 DmNumRecords (DmOpenRef dbP)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.

Result The number of records in a database.

Comments Records that have that have the `deleted` bit set (that is, records that will be deleted during the next synchronization because the user has marked them deleted) are included in the count. If you want to exclude these records from your count, use `DmNumRecordsInCategory` and pass `dmAllCategories` as the category.

See Also `DmNumRecordsInCategory`, `DmRecordInfo`, `DmSetRecordInfo`

DmNumRecordsInCategory

Purpose Return the number of records of a specified category in a database.

Prototype
```
UInt16 DmNumRecordsInCategory (DmOpenRef dbP,
UInt16 category)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `category`	Category index.

Result The number of records in the category.

Comments Because this function must examine all records in the database, it can be slow to return, especially when called on a large database.

Records that have the `deleted` bit set are not counted, and if the user has specified to hide or mask private records, private records are not counted either.

You can use the DmRecordInfo call to obtain a category index from a given record. For example:

```
DmOpenRef myDB;
UInt16 record, attr, category, total;

DmRecordInfo(myDB, record, &attr, NULL, NULL);
category = attr & dmRecAttrCategoryMask;
total = DmNumRecordsInCategory(myDB, category);
```

See Also DmNumRecords, DmQueryNextInCategory, DmPositionInCategory, DmSeekRecordInCategory, DmMoveCategory

DmNumResources

Purpose Return the total number of resources in a given resource database.

Prototype

```
UInt16 DmNumResources (DmOpenRef dbP)
```

Parameters -> `dbP` `DmOpenRef` to open database.

Result The total number of resources in the given database.

May display a fatal error message if the database is not a resource database.

Comments `DmNumResources` only counts the resources in the database indicated by the `DmOpenRef` parameter. If the database is a resource database that has an overlay associated with it, this function only returns the number of resources in the base database, not in the overlay.

DmOpenDatabase

Purpose

Open a database and return a reference to it. If the database is a resource database, also open its overlay for the current locale.

Prototype

```
DmOpenRef DmOpenDatabase (UInt16 cardNo,
LocalID dbID, UInt16 mode)
```

Parameters

-> `cardNo`	Card number database resides on.
-> `dbID`	The database ID of the database.
-> `mode`	Which mode to open database in (see "Open Mode Constants").

Result

Returns `DmOpenRef` to open database. May display a fatal error message if the database parameter is `NULL`. On all other errors, this function returns 0 and `DmGetLastErr` returns an error code indicating the reason for failure.

Comments

Call this routine to open a database for reading or writing.

This routine returns a `DmOpenRef` which must be used to access particular records in a database. If unsuccessful, 0 is returned and the cause of the error can be determined by calling `DmGetLastErr`.

When you use this routine to open a resource database in read-only mode, it also opens the overlay associated with this database for the current locale, if it exists. (The function `OmGetCurrentLocale` returns the current locale.) Overlays are resource databases typically used to localize applications, shared libraries, and panels. They have the same creator as the base database, a type of `'ovly'` (symbolically named `omOverlayDBType`), and contain resources with the same IDs and types as the resources in the base database. When you request a resource from the database using `DmGetResource` or `DmGet1Resource`, the overlay is searched first. If the overlay contains a resource for the given ID, it is returned. If not, the resource from the base database is returned.

The `DmOpenRef` returned by this function is the pointer to the base database, not to the overlay database, so care should be taken when

passing this pointer to functions such as DmFindResource because this circumvents the overlay.

It's possible to create a "stripped" base resource database, one that does not contain any user interface resources. DmOpenDatabase only opens a stripped database if its corresponding overlay exists. If the overlay does not exist or if the overlay doesn't match the resource database, DmOpenDatabase returns NULL and DmGetLastErr returns the error code omErrBaseRequiresOverlay.

If you open a resource database in a writable mode, the associated overlay is not opened. If you make changes to the resource database, the overlay database is invalidated if those changes affect any resources that are also in the overlay. This means that on future occasions where you open the resource database in read-only mode, the overlay will not be opened because Palm OS considers it to be invalid.

TIP: If you want to prevent your resource database from being overlaid, include an 'xprf' resource (symbolically named sysResTExtPrefs) in the database with the ID 0 (sysResIDExtPrefs) and set its disableOverlays flag. This resource is defined in UIResources.r.

Compatibility

Overlay support is only available if 3.5 New Feature Set is present. On earlier releases, this function opens resource databases without looking for an associated overlay.

If 4.0 New Feature Set is present, when DmOpenDatabase attempts to open a stripped resource database and cannot find an overlay for it, it searches for an overlay matching the default locale if the system locale is different from the default locale.

See Also

DmCloseDatabase, DmCreateDatabase, DmFindDatabase, DmOpenDatabaseByTypeCreator, DmDeleteDatabase, DmOpenDBNoOverlay

DmOpenDatabaseByTypeCreator

Purpose

Open the most recent revision of a database with the given type and creator. If the database is a resource database, also open its overlay for the current locale.

Prototype

```
DmOpenRef DmOpenDatabaseByTypeCreator
(UInt32 type, UInt32 creator, UInt16 mode)
```

Parameters

-> `type`	Type of database.
-> `creator`	Creator of database.
-> `mode`	Which mode to open database in (see "Open Mode Constants").

Result

`DmOpenRef` to open database. If the database couldn't be found, this function returns 0 and `DmGetLastErr` returns an error code indicating the reason for failure.

Comments

If you use this routine to open a resource database in read-only mode, it also opens the overlay associated with this database for the current locale. See `DmOpenDatabase` for more information on overlays and resource databases.

Compatibility

Overlay support is only available if 3.5 New Feature Set is present. On earlier releases, this function opens resource databases without looking for an associated overlay.

See Also

`DmCreateDatabase`, `DmOpenDatabase`, `DmOpenDatabaseInfo`, `DmCloseDatabase`, `DmOpenDBNoOverlay`

DmOpenDatabaseInfo

Purpose Retrieve information about an open database.

Prototype

```
Err DmOpenDatabaseInfo (DmOpenRef dbP,
LocalID *dbIDP, UInt16 *openCountP, UInt16 *modeP,
UInt16 *cardNoP, Boolean *resDBP)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
<- `dbIDP`	The ID of the database. Pass `NULL` for this parameter if you don't want to retrieve this information.
<- `openCountP`	The number of applications that have this database open. Pass `NULL` for this parameter if you don't want to retrieve this information.
<- `modeP`	The mode used to open the database (see "Open Mode Constants"). Pass `NULL` for this parameter if you don't want to retrieve this information.
<- `cardNoP`	The number of the card on which this database resides. Pass `NULL` for this parameter if you don't want to retrieve this information.
<- `resDBP`	If `true` upon return, the database is a resource database, `false` otherwise. Pass `NULL` for this parameter if you don't want to retrieve this information.

Result Returns `errNone` if no error.

See Also `DmDatabaseInfo`

DmOpenDBNoOverlay

Purpose Open a database and return a reference to it.

Prototype
```
DmOpenRef DmOpenDBNoOverlay (UInt16 cardNo,
LocalID dbID, UInt16 mode)
```

Parameters

-> `cardNo`	Card number database resides on.
-> `dbID`	The database ID of the database.
-> `mode`	Which mode to open database in (see "Open Mode Constants").

Result `DmOpenRef` to open database. May display a fatal error message if the database parameter is `NULL`. On all other errors, this function returns 0 and `DmGetLastErr` returns an error code indicating the reason for failure.

Comments Call this routine to open a database for reading or writing, while ignoring any overlay databases that might be associated with it.

This routine returns a `DmOpenRef` which must be used to access particular records in a database. If unsuccessful, 0 is returned and the cause of the error can be determined by calling `DmGetLastErr`.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `DmCloseDatabase`, `DmCreateDatabase`, `DmFindDatabase`, `DmOpenDatabaseByTypeCreator`, `DmDeleteDatabase`, `DmOpenDatabase`

DmPositionInCategory

Purpose Return a position of a record within the specified category.

Prototype
```
UInt16 DmPositionInCategory (DmOpenRef dbP,
UInt16 index, UInt16 category)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.

-> `index`	Index of the record.
-> `category`	Index of category to search.

Result Returns the position (zero-based). If the specified index is out of range, this function returns 0 and `DmGetLastErr` returns an error code indicating the reason for failure. Note that this means a 0 return value might indicate either success or failure. If this function returns 0 and `DmGetLastErr` returns `errNone`, the return value indicates that this is the first record in the category.

Comments Because this function must examine all records up to the current record, it can be slow to return, especially when called on a large database.

If the record is ROM-based (pointer accessed) this routine makes a fake handle to it and stores this handle in the `DmAccessType` structure.

To learn which category a record is in, use `DmRecordInfo`.

See Also `DmQueryNextInCategory`, `DmSeekRecordInCategory`, `DmMoveCategory`

DmQueryNextInCategory

Purpose Return a handle to the next record in the specified category for reading only (does not set the `busy` bit).

Prototype

```
MemHandle DmQueryNextInCategory (DmOpenRef dbP,
UInt16 *indexP, UInt16 category)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
<-> `indexP`	Index of a known record (often retrieved with `DmPositionInCategory`). If a "next" record is found, this index is updated to indicate that record.

-> `category`	Index of category to query, or `dmAllCategories` to find the next record in any category.

Result Returns a handle to the record following a known record, along with the index of that record. If a record couldn't be found, this function returns `NULL`, and `DmGetLastErr` returns an error code indicating the reason for failure.

Comments This function begins searching the database from the record at `*indexP` for a record that is in the specified category. If the record at `*indexP` belongs to that category, then a handle to it is returned. If not, the function continues searching until it finds a record in the category.

Thus, if you want to find the next record in the category after the one you have an index for, increment the index value before calling this function. For example:

```
DmOpenRef myDB;
UInt16 record, attr, category, pos;
MemHandle newRecH;

DmRecordInfo(myDB, record, &attr, NULL, NULL);
category = attr & dmRecAttrCategoryMask;
pos = DmPositionInCategory(myDB, record,
  category);
pos++;
newRecH = DmQueryNextInCategory(myDB, &pos,
category);
```

See Also `DmNumRecordsInCategory`, `DmPositionInCategory`, `DmSeekRecordInCategory`

DmQueryRecord

Purpose Return a handle to a record for reading only (does not set the `busy` bit).

Prototype
```
MemHandle DmQueryRecord (DmOpenRef dbP,
UInt16 index)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `index`	Which record to retrieve.

Result Returns a record handle. If an error occurs, this function returns `NULL`, and `DmGetLastErr` returns an error code indicating the reason for failure.

Some releases may display a fatal error message if the specified index is out of range.

Comments Returns a handle to the given record. Use this routine only when viewing the record. This routine successfully returns a handle to the record even if the record is busy.

If the record is ROM-based (pointer accessed) this routine returns the fake handle to it.

DmQuickSort

Purpose Sort records in a database.

Prototype
```
Err DmQuickSort (DmOpenRef dbP, DmComparF *compar,
Int16 other)
```

Parameters

-> `dbP`	Database access pointer.
-> `compar`	Comparison function. See `DmComparF`.
-> `other`	Any value the application wants to pass to the comparison function. This parameter is often used to indicate a sort direction (ascending or descending).

Result Returns `errNone` if no error, or one of the following if an error occurs:

- `dmErrReadOnly`
- `dmErrNotRecordDB`

Some releases may display a fatal error message instead of returning the error code.

Comments Deleted records are placed last in any order. All others are sorted according to the passed comparison function.

After `DmQuickSort` returns, equal database records do not have a consistent order. That is, if `DmQuickSort` is passed two equal records, their resulting order is unpredictable. To prevent records that contain the same data from being rearranged in an unpredictable order, pass the record's unique ID to the comparison function (using the `SortRecordInfoType` structure).

See Also `DmFindSortPositionV10`, `DmInsertionSort`

DmRecordInfo

Purpose Retrieve the record information as stored in the database header.

Prototype

```
Err DmRecordInfo (DmOpenRef dbP, UInt16 index,
UInt16 *attrP, UInt32 *uniqueIDP,
LocalID *chunkIDP)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `index`	Index of the record.
<- `attrP`	The record's attributes. See "Record Attribute Constants." Pass `NULL` for this parameter if you don't want to retrieve this value.
<- `uniqueIDP`	The record's unique ID. Pass `NULL` for this parameter if you don't want to retrieve this value.

	<- chunkIDP	The record's local ID. Pass NULL for this parameter if you don't want to retrieve this value.

Result Returns errNone if no error or dmErrIndexOutOfRange if the specified record can't be found. Some releases may display a fatal error message instead of returning the error code.

See Also DmNumRecords, DmSetRecordInfo, DmQueryNextInCategory

DmReleaseRecord

Purpose Clear the busy bit for the given record and set the dirty bit if dirty is true.

Prototype

```
Err DmReleaseRecord (DmOpenRef dbP, UInt16 index,
Boolean dirty)
```

Parameters

	-> dbP	DmOpenRef to open database.
	-> index	The record to unlock.
	-> dirty	If true, set the dirty bit.

Result Returns errNone if no error, or dmErrIndexOutOfRange if the specified index is out of range. Some releases may display a fatal error message instead of returning the error code.

Comments Call this routine when you finish modifying or reading a record that you've called DmGetRecord on or created using DmNewRecord.

See Also DmGetRecord

DmReleaseResource

Purpose Release a resource acquired with DmGetResource.

Prototype
```
Err DmReleaseResource (MemHandle resourceH)
```

Parameters

-> `resourceH` Handle to resource.

Result Returns `errNone` if no error.

Comments Marks a resource as being no longer needed by the application.

See Also DmGet1Resource, DmGetResource

DmRemoveRecord

Purpose Remove a record from a database and dispose of its data chunk.

Prototype
```
Err DmRemoveRecord (DmOpenRef dbP, UInt16 index)
```

Parameters

-> `dbP` `DmOpenRef` to open database.

-> `index` Index of the record to remove.

Result Returns `errNone` if no error, or one of the following if an error occurs:

- dmErrReadOnly
- dmErrIndexOutOfRange
- dmErrNotRecordDB
- memErrChunkLocked
- memErrInvalidParam

Some releases may display a fatal error message instead of returning the error code.

Comments Disposes of a the record's data chunk and removes the record's entry from the database header.

See Also DmDetachRecord, DmDeleteRecord, DmArchiveRecord, DmNewRecord

DmRemoveResource

Purpose Delete a resource from a resource database.

Prototype

```
Err DmRemoveResource (DmOpenRef dbP, UInt16 index)
```

Parameters

-> dbP	DmOpenRef to open database.
-> index	Index of resource to delete.

Result Returns errNone if no error, or one of the following if an error occurs:

- dmErrCorruptDatabase
- dmErrIndexOutOfRange
- dmErrReadOnly
- memErrChunkLocked
- memErrInvalidParam
- memErrNotEnoughSpace

May display a fatal error message if the database is not a resource database.

Comments This routine disposes of the memory manager chunk that holds the given resource and removes its entry from the database header.

See Also DmDetachResource, DmRemoveResource, DmAttachResource

DmRemoveSecretRecords

Purpose Remove all secret records.

Prototype
```
Err DmRemoveSecretRecords (DmOpenRef dbP)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.

Result Returns `errNone` if no error, or one of the following if an error occurs:

- `dmErrReadOnly`
- `dmErrNotRecordDB`

Some releases may display a fatal error message instead of returning the error code.

See Also `DmRemoveRecord`, `DmRecordInfo`, `DmSetRecordInfo`

DmResizeRecord

Purpose Resize a record by index.

Prototype
```
MemHandle DmResizeRecord (DmOpenRef dbP,
UInt16 index, UInt32 newSize)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `index`	Which record to retrieve.
-> `newSize`	New size of record.

Result Handle to resized record. Returns `NULL` if there is not enough space to resize the record, and `DmGetLastErr` returns an error code indicating the reason for failure. Some releases may display a fatal error message instead of returning the error code.

Comments This routine reallocates the record in another heap of the same memory card if the current heap is not big enough. If this happens, the handle changes, so be sure to use the returned handle to access the resized record.

DmResizeResource

Purpose Resize a resource and return the new handle.

Prototype
```
MemHandle DmResizeResource (MemHandle resourceH,
UInt32 newSize)
```

Parameters

-> `resourceH`	Handle to resource.
-> `newSize`	Desired new size of resource.

Result Returns a handle to newly sized resource. Returns `NULL` if there is not enough space to resize the resource, and `DmGetLastErr` returns an error code indicating the reason for failure. Some releases may display a fatal error message instead of returning the error code.

Comments Resizes the resource and returns a new handle. If necessary in order to grow the resource, this routine will reallocate it in another heap on the same memory card that it is currently in.

The handle may change if the resource had to be reallocated in a different data heap because there was not enough space in its present data heap.

DmResourceInfo

Purpose Retrieve information on a given resource.

Prototype
```
Err DmResourceInfo (DmOpenRef dbP, UInt16 index,
DmResType *resTypeP, DmResID *resIDP,
LocalID *chunkLocalIDP)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `index`	Index of resource to get info on.
<- `resTypeP`	The resource type. Pass `NULL` if you don't want to retrieve this information.

<- `resIDP`	The resource ID. Pass `NULL` if you don't want to retrieve this information.
<- `chunkLocalIDP`	The memory manager local ID of the resource data. Pass `NULL` if you don't want to retrieve this information.

Result Returns `errNone` if no error or `dmErrIndexOutOfRange` if an error occurred. May display a fatal error message if the database is not a resource database.

Comments If `dbP` is a pointer to a base resource database, the information returned is about the resource from that database alone; this function ignores any associated overlay.

See Also `DmGetResource`, `DmGet1Resource`, `DmSetResourceInfo`, `DmFindResource`, `DmFindResourceType`

DmSearchRecord

Purpose Search all open record databases for a record with the handle passed.

Prototype

```
UInt16 DmSearchRecord (MemHandle recH,
DmOpenRef *dbPP)
```

Parameters

-> `recH`	Record handle.
<- `dbPP`	The database that contains the record `recH`.

Result Returns the index of the record and database access pointer; if not found, returns -1 and `*dbPP` is 0.

See Also `DmGetRecord`, `DmFindRecordByID`, `DmRecordInfo`

DmSearchResource

Purpose Search all open resource databases for a resource by type and ID, or by pointer if it is non-NULL.

Prototype

```
UInt16 DmSearchResource (DmResType resType,
DmResID resID, MemHandle resH, DmOpenRef *dbPP)
```

Parameters

-> resType	Type of resource to search for.
-> resID	ID of resource to search for.
-> resH	Pointer to locked resource, or NULL.
<- dbPP	The resource database that contains the specified resource.

Result Returns the index of the resource, stores DmOpenRef in dbPP.

Comments This routine can be used to find a resource in all open resource databases by type and ID or by pointer. If resH is NULL, the resource is searched for by type and ID. If resH is not NULL, resType and resID is ignored and the index of the resource handle is returned. On return, *dbPP contains the access pointer of the resource database that the resource was eventually found in. Once the index of a resource is determined, it can be locked down and accessed by calling DmGetResourceIndex.

If any of the open databases are overlaid, this function finds and returns the localized version of the resource when searching by type and creator. In this case, the dbPP return value is a pointer to the overlay database, not the base resource database.

See Also DmGetResource, DmFindResourceType, DmResourceInfo, DmFindResource

DmSeekRecordInCategory

Purpose

Return the index of the record nearest the offset from the passed record index whose category matches the passed category. (The `offset` parameter indicates the number of records to move forward or backward.)

Prototype

```
Err DmSeekRecordInCategory (DmOpenRef dbP,
UInt16 *indexP, UInt16 offset, Int16 direction,
UInt16 category)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
<-> `index`	The index to start the search at. Upon return, contains the index of the record at `offset` from the index that you passed in.
-> `offset`	Offset of the passed record index. This must be a positive number; use `dmSeekBackward` for the `direction` parameter to search backwards.
-> `direction`	Must be either `dmSeekForward` or `dmSeekBackward`.
-> `category`	Category index.

Result

Returns `errNone` if no error; returns `dmErrIndexOutOfRange` or `dmErrSeekFailed` if an error occurred.

Comments

`DmSeekRecordInCategory` searches for a record in the specified category. The search begins with the record at `index`. When it finds a record in the specified category, it decrements the `offset` parameter and continues searching until a match is found and `offset` is 0.

Because of this, if you use `DmSeekRecordInCategory` to find the nearest matching record in a particular category, you must pass different `offset` parameters if the starting record is in the category than if it isn't. If the record at `index` is in the category, then you must pass an `offset` of 1 to find the next record in the category because the comparison is performed before the `index` value

changes. If the record at `index` isn't in the category, you must pass an `offset` of 0 to find the next record in the category. In this case, an `offset` of 1 skips the first matching record.

See Also DmNumRecordsInCategory, DmQueryNextInCategory, DmPositionInCategory, DmMoveCategory

DmSet

Purpose Write a specified value into a section of a record. This function also checks the validity of the pointer for the record and makes sure the writing of the record information doesn't exceed the bounds of the record.

Prototype

```
Err DmSet (void *recordP, UInt32 offset,
UInt32 bytes, UInt8 value)
```

Parameters

-> `recordP`	Pointer to locked data record (chunk pointer).
-> `offset`	Offset within record to start writing.
-> `bytes`	Number of bytes to write.
-> `value`	Byte value to write.

Result Returns `errNone` if no error. May display a fatal error message if the record pointer is invalid or the function overwrites the record.

Comments Must be used to write to data manager records because the data storage area is write-protected.

See Also DmWrite

DmSetDatabaseInfo

Purpose Set information about a database.

Prototype

```
Err DmSetDatabaseInfo (UInt16 cardNo,
LocalID dbID, const Char *nameP,
UInt16 *attributesP, UInt16 *versionP,
UInt32 *crDateP, UInt32 *modDateP,
UInt32 *bckUpDateP, UInt32 *modNumP,
LocalID *appInfoIDP, LocalID *sortInfoIDP,
UInt32 *typeP, UInt32 *creatorP)
```

Parameters

-> `cardNo`	Card number the database resides on.
-> `dbID`	Database ID of the database.
-> `nameP`	Pointer to 32-byte character array for new name, or `NULL`.
-> `attributesP`	Pointer to new attributes variable, or `NULL`. See "Database Attribute Constants" for a list of possible values.
-> `versionP`	Pointer to new version, or `NULL`.
-> `crDateP`	Pointer to new creation date variable, or `NULL`. Specify the value as a number of seconds since Jan. 1, 1904.
-> `modDateP`	Pointer to new modification date variable, or `NULL`. Specify the value as a number of seconds since Jan. 1, 1904.
-> `bckUpDateP`	Pointer to new backup date variable, or `NULL`. Specify the value as a number of seconds since Jan. 1, 1904.
-> `modNumP`	Pointer to new modification number variable, or `NULL`.
-> `appInfoIDP`	Pointer to new `appInfoID` variable, or `NULL`.
-> `sortInfoIDP`	Pointer to new `sortInfoID` variable, or `NULL`.
-> `typeP`	Pointer to new `type` variable, or `NULL`.

-> `creatorP` Pointer to new `creator` variable, or `NULL`.

Result Returns `errNone` if no error or one of the following if an error occurred:

- `dmErrInvalidDatabaseName`
- `dmErrAlreadyExists`
- `dmErrInvalidParam`

Comments When this call changes `appInfoID` or `sortInfoID`, the old chunk ID (if any) is marked as an orphaned chunk[1] and the new chunk ID is unorphaned. Consequently, you shouldn't replace an existing `appInfoID` or `sortInfoID` if that chunk has already been attached to another database.

Call this routine to set any or all information about a database except for the card number and database ID. This routine sets the new value for any non-`NULL` parameter.

See Also `DmDatabaseInfo`, `DmOpenDatabaseInfo`, `DmFindDatabase`, `DmGetNextDatabaseByTypeCreator`, `TimDateTimeToSeconds`

1. An "orphaned chunk" is one that is allocated in the storage heap, but to which nothing refers. If the orphaned chunk is not put into a database as a record, an appInfo block, or the like, and if the application doesn't keep track of it—in a global variable, perhaps—it could get lost. If the application doesn't get around to freeing the chunk before it quits or crashes, or before the device is reset, that storage will be forever unusable: the user can't delete it since the user only deletes databases.

   During a soft reset, the OS walks through the storage heap and frees any orphaned chunks that it finds. Since most users reset only rarely, however, you shouldn't rely on this happening.

DmSetRecordInfo

Purpose Set record information stored in the database header.

Prototype

```
Err DmSetRecordInfo (DmOpenRef dbP, UInt16 index,
UInt16 *attrP, UInt32 *uniqueIDP)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `index`	Index of record.
-> `attrP`	Pointer to new attribute variable, or `NULL`. See "Record Attribute Constants" for a list of possible values.
-> `uniqueIDP`	Pointer to new unique ID variable, or `NULL`.

Result Returns `errNone` if no error, or one of the following if an error occurred:

- `dmErrReadOnly`
- `dmErrNotRecordDB`
- `dmErrIndexOutOfRange`

Some releases may display a fatal error message instead of returning the error code.

Comments Sets information about a record. This routine cannot be used to set the `dmRecAttrBusy` bit; instead, use `DmGetRecord` to set the bit and `DmReleaseRecord` to clear it.

Normally, the unique ID for a record is automatically created by the data manager when a record is created using `DmNewRecord`, so an application would not typically change the unique ID.

See Also `DmNumRecords`, `DmRecordInfo`

DmSetResourceInfo

Purpose Set information on a given resource.

Prototype

```
Err DmSetResourceInfo (DmOpenRef dbP,
UInt16 index, DmResType *resTypeP,
DmResID *resIDP)
```

Parameters

-> `dbP`	`DmOpenRef` to open database.
-> `index`	Index of resource to set info for.
-> `resTypeP`	Pointer to new `resType` (resource type), or `NULL`.
-> `resIDP`	Pointer to new resource ID, or `NULL`.

Result Returns `errNone` if no error, or one of the following if an error occurred:

- `dmErrIndexOutOfRange`
- `dmErrReadOnly`

May display a fatal error message if the database is not a resource database.

Comments Use this routine to set all or a portion of the information on a particular resource. Any or all of the new info pointers can be `NULL`. If not `NULL`, the type and ID of the resource are changed to `*resTypeP` and `*resIDP`.

DmStrCopy

Purpose Copies a string to a record within a database that is open for writing.

Prototype

```
Err DmStrCopy (void *recordP, UInt32 offset,
const Char *srcP)
```

Parameters

<-> `recordP`	Pointer to data record (chunk pointer).
-> `offset`	Offset within record to start writing.
-> `srcP`	Pointer to null-terminated string.

Result Returns `errNone` if no error. May display a fatal error message if the record pointer is invalid or the function overwrites the record.

Comments This is one of the routines that must be used to write to Data Manager records; because the data storage area is write-protected, you cannot write to it directly. This routine checks the validity of the chunk pointer for the record to insure that writing the record will not exceed the chunk bounds. DmStrCopy is a convenience method that determines the size of the supplied string and then simply calls `DmWrite`.

See Also `DmSet`

DmWrite

Purpose Copies a specified number of bytes to a record within a database that is open for writing.

Prototype

```
Err DmWrite (void *recordP, UInt32 offset,
const void *srcP, UInt32 bytes)
```

Parameters

<-> `recordP`	Pointer to locked data record (chunk pointer).
-> `offset`	Offset within record to start writing.
-> `srcP`	Pointer to data to copy into record.
-> `bytes`	Number of bytes to write.

Result Returns `errNone` if no error. May display a fatal error message if the record pointer is invalid or the function overwrites the record.

Comments This is one of the routines that must be used to write to Data Manager records; because the data storage area is write-protected, you cannot write to it directly. This routine checks the validity of the

chunk pointer for the record to insure that writing the record will not exceed the chunk bounds.

See Also DmStrCopy, DmSet

DmWriteCheck

Purpose Check the parameters of a write operation to a data storage chunk before actually performing the write.

Prototype
```
Err DmWriteCheck (void *recordP, UInt32 offset,
UInt32 bytes)
```

Parameters

-> recordP	Locked pointer to recordH.
-> offset	Offset into record to start writing.
-> bytes	Number of bytes to write.

Result Returns errNone if no error; returns dmErrNotValidRecord or dmErrWriteOutOfBounds if an error occurred.

Application-Defined Functions

DmComparF

Purpose Compares two records in a database.

Prototype
```
typedef Int16 DmComparF (void *rec1, void *rec2,
Int16 other, SortRecordInfoPtr rec1SortInfo,
SortRecordInfoPtr rec2SortInfo,
MemHandle appInfoH)
```

Parameters

-> rec1, rec2	Pointers to the two records to compare.
-> other	Any other custom information you want passed to the comparison function. This parameter is often used to indicate a sort direction (ascending or descending).

-> `rec1SortInfo`, `rec2SortInfo`
Pointers to `SortRecordInfoType` structures that specify unique sorting information for the two records.

-> `appInfoH` A handle to the database's application info block.

Result Returns:

- 0 if `rec1` = `rec2`.
- < 0 if `rec1` < `rec2`.
- > 0 if `rec1` > `rec2`.

Comments This function is used to sort the records in a database. It is specifically called by `DmFindSortPosition`, `DmInsertionSort`, and `DmQuickSort`.

Compatibility The `DmComparF` prototype changed in Palm OS version 2.0. Previously, the prototype only defined the first three parameters.

As a rule, the change in the number of arguments from three to six doesn't cause problems when a 1.0 application is run on a 2.0 device because the system only pulls the arguments from the stack that are there.

Keep in mind, however, that some optimized applications built with tools other than Metrowerks CodeWarrior for Palm OS may have problems as a result of the change in arguments when running on a 2.0 or later device.

29

Error Manager

This chapter provides reference material for the error manager. The error manager API is declared in the header files `ErrorMgr.h` and `ErrorBase.h`. This chapter covers:

- ERROR_CHECK_LEVEL Define
- Error Manager Data Structures
- Error Manager Functions

For more information on the error manager, see the chapter "Debugging Strategies" in the *Palm OS Programmer's Companion*, vol. I.

ERROR_CHECK_LEVEL Define

The error manager uses the compiler define `ERROR_CHECK_LEVEL` to control the level of error messages displayed. You can set the value of the compiler define to control which level of error checking and display is compiled into the application. Three levels of error checking are supported: none, partial, and full.

If you set `ERR_CHECK_LEVEL` to...	**The compiler...**
`ERROR_CHECK_NONE (0)`	Doesn't compile in any error calls.
`ERROR_CHECK_PARTIAL (1)`	Compiles in only `ErrDisplay` and `ErrFatalDisplayIf` calls.
`ERROR_CHECK_FULL (2)`	Compiles in all three calls.

Error Manager Data Structures

ErrExceptionType

An `ErrExceptionType` structure is created for each `ErrTry` and `ErrCatch` block. At any point in the program, there is a linked list of these structures. An `ErrExceptionType` structure stores information about the state of the machine (register values) at the start of the `ErrTry` block.

```
typedef struct ErrExceptionType {
  struct ErrExceptionType *nextP;
  ErrJumpBuf state;
  Int32 err;
} ErrExceptionType;
typedef ErrExceptionType *ErrExceptionPtr;
```

Field Descriptions

`nextP` Next `ErrExceptionType` structure in the linked list.

`state` Storage for setjmp/longjmp.

`err` Error code.

Error Manager Functions

ErrAlert

Purpose Macro that displays an alert dialog for runtime errors.

Prototype `ErrAlert (err)`

Parameters -> `err` An error code (as type `Err`).

Result Returns 0, which indicates that the OK button has been clicked to dismiss the dialog.

Comments This macro is intended for use by applications that are likely to receive runtime errors when the application itself is not at fault. For

example, a networking application might use it to display an alert if the remote server cannot be found.

The error message displayed on the dialog is stored in a 'tSTL' resource. A 'tSTL' resource contains a list of strings that can be looked up by index. The `err` parameter is used as the index into this list.

To use application-defined error codes in `ErrAlert`, make sure that all of your error codes are greater than or equal to `appErrorClass`. This way, the error manager looks up the code in the application's 'tSTL' resource number 0. All other error codes are taken from 'tSTL' resource stored in the system.

Compatibility Implemented only if 3.2 New Feature Set is present.

ErrCatch

Purpose Macro that marks the end of an `ErrTry` block and the beginning of an `ErrCatch` block.

Prototype

```
ErrCatch (theErr)
```

Parameters

`-> theErr`	An exception code identifying the reason for the failure. This is the value supplied to the `ErrThrow` call that caused the jump to this `ErrCatch` block.

Result Returns nothing.

Comments `ErrCatch` can only be used in conjunction with `ErrTry` and `ErrEndCatch`. See the comments under `ErrTry` for usage instructions.

`ErrTry`, `ErrCatch` and `ErrThrow` are based on setjmp and longjmp. At the beginning of an `ErrTry` block, setjmp saves the machine registers. `ErrThrow` calls longjmp, which restores the registers and jumps to the beginning of the `ErrCatch` block. Therefore, any changes in the `ErrTry` block to variables stored in registers aren't retained when entering the `ErrCatch` block.

The solution is to declare variables that you want to use in both the `ErrTry` and `ErrCatch` blocks as "volatile". For example:

```
volatile long x = 1;  // Declare volatile local variable
ErrTry {
   x = 100;    // Set local variable in Try
   ErrThrow(-1);
}
ErrCatch(inErr) {
   if (x > 1) {    // Use local variable in Catch
      SysBeep(1);
   }
} ErrEndCatch
```

If you have many local variables after the `ErrCatch` you may want to put the `ErrTry` and `ErrCatch` in a separate enclosing function.

ErrDisplay

Purpose Macro that displays an error alert if error checking is set to partial or full.

Prototype

```
ErrDisplay (msg)
```

Parameters

-> `msg` Error message text as a string.

Result No return value.

Comments Call this macro to display an error message, source code filename, and line number. This macro is compiled into the code only if the compiler define `ERROR_CHECK_LEVEL` is set to 1 or 2 (`ERROR_CHECK_PARTIAL` or `ERROR_CHECK_FULL`).

See Also `ErrFatalDisplayIf`, `ErrNonFatalDisplayIf`

ErrDisplayFileLineMsg

Purpose Display a dialog with an error message. Do not allow the user to exit the dialog or continue.

Prototype

```
void ErrDisplayFileLineMsg
(const Char *const filename, UInt16 lineNo,
const Char *const msg)
```

Parameters

-> `filename`	Source code filename.
-> `lineno`	Line number in the source code file.
-> `msg`	Message to display.

Result Never returns.

Comment Called by `ErrFatalDisplayIf` and `ErrNonFatalDisplayIf`. This function is useful when the application is already on the device and being tested by users.

On Japanese systems, the system displays a generic message indicating that an error has occurred instead of displaying the English message.

See Also `ErrFatalDisplayIf`, `ErrNonFatalDisplayIf`, `ErrDisplay`

ErrEndCatch

Purpose Macro that marks the end of an `ErrCatch` block.

Prototype

```
ErrEndCatch
```

Parameters None.

Result Returns nothing.

Comments `ErrEndCatch` can only be used in conjunction with `ErrTry` and `ErrCatch`. See the comments under `ErrTry` for usage instructions.

ErrExceptionList

Purpose Return the address of the pointer to the first `ErrExceptionType` structure in the linked list of exception structures.

Prototype
```
MemPtr *ErrExceptionList (void)
```

Parameters None.

Result Returns the address of the pointer to the first `ErrExceptionType` structure linked into the exception list.

Comments This function is used by the ErrTry and ErrCatch macros as well as the `ErrThrow` function in order to find the exception list.

When called from an application, this routine returns the application's exception list.

ErrFatalDisplayIf

Purpose Macro that displays an error alert dialog if `condition` is `true` and error checking is set to partial or full.

Prototype
```
ErrFatalDisplayIf (condition, msg)
```

Parameters

-> `condition`	A boolean value. If `true`, display the error.
-> `msg`	Error message text as a string.

Result No return value.

Comments Call this macro to display a fatal error message, source code filename, and line number. The alert is displayed only if `condition` is `true`. The dialog is cleared only when the user resets the system by responding to the dialog.

This macro is compiled into the code if the compiler define `ERROR_CHECK_LEVEL` is set to 1 or 2 (`ERROR_CHECK_PARTIAL` or `ERROR_CHECK_FULL`).

See Also `ErrNonFatalDisplayIf`, `ErrDisplay`,

ErrNonFatalDisplayIf

Purpose Macro that displays an error alert dialog if condition is true and error checking is set to full.

Prototype
```
ErrNonFatalDisplayIf (condition, msg)
```

Parameters

-> `condition`	A boolean value. If `true`, display the error.
-> `msg`	Error message text as a string.

Result No return value.

Comments Call this macro to display a nonfatal error message, source code filename, and line number. The alert is displayed only if `condition` is `true`. The alert dialog is cleared when the user selects to continue (or resets the system).

This macro is compiled into the code only if the compiler define `ERROR_CHECK_LEVEL` is set to 2 (`ERROR_CHECK_FULL`).

See Also `ErrFatalDisplayIf`, `ErrDisplay`

ErrThrow

Purpose Cause a jump to the nearest Catch block.

Prototype
```
void ErrThrow (Int32 err)
```

Parameters

-> `err`	Error code.

Result Never returns.

Comments Use the macros ErrTry, ErrCatch, and ErrEndCatch in conjunction with this function.

See Also `ErrFatalDisplayIf`, `ErrNonFatalDisplayIf`, `ErrDisplay`

ErrTry

Purpose Macro that marks the beginning of a try/catch block.

Prototype `ErrTry`

Parameters None.

Result Returns nothing.

Comments An exception raised by a call to `ErrThrow`—even from within a nested subroutine—causes program execution to switch to the beginning of the `ErrCatch` block. If the end of the block enclosed by `ErrTry` is encountered without a call to `ErrThrow`, execution jumps to the line of code following the `ErrEndCatch` macro. See "The Try-and-Catch Mechanism" on page 343 of the *Palm OS Programmer's Companion*, vol. I for a more thorough description of how this mechanism works.

Example You must structure your code exactly as shown here. You can't use `ErrTry` without `ErrCatch` and `ErrEndCatch`, or vice versa.

```
ErrTry {
   // Do something which may fail. Call ErrThrow to signal
   // failure and force a jump to the following ErrCatch
   // block.
}
ErrCatch(inErr) {
   // Recover or cleanup after a failure in the above ErrTry
   // block. "inErr" is an exception code identifying the
   // reason for the failure.

   // Call ErrThrow if you want to jump out to the next
   // ErrCatch block.

   // The code in this block doen't execute if the above
   // ErrTry block completes without a call to ErrThrow.
} ErrEndCatch
```

30

Expansion Manager

This chapter provides the following information about the Expansion Manager:

- Expansion Manager Data Structures
- Expansion Manager Constants
- Expansion Manager Functions

The header file `ExpansionMgr.h` declares the Expansion Manager API. For more information on the Expansion Manager, see Chapter 8, "Expansion," in *Palm OS Programmer's Companion*, vol. I.

Note that the Expansion Manager is an optional system extension; the functions described in this chapter are implemented only if the Expansion Manager Feature Set is present.

Expansion Manager Data Structures

ExpCardInfoType

The `ExpCardInfoType` declaration defines a structure that is passed to `ExpCardInfo`. This structure is used to determine the characteristics of the card loaded in the slot. It is initialized by the underlying slot driver with the following information.

```
typedef struct ExpCardInfoTag {
  UInt32 capabilityFlags;
  Char
    manufacturerStr[expCardInfoStringMaxLen+1];
  Char productStr[expCardInfoStringMaxLen+1];
  Char
    deviceClassStr[expCardInfoStringMaxLen+1];
  Char
  deviceUniqueIDStr[expCardInfoStringMaxLen+1];
} ExpCardInfoType, *ExpCardInfoPtr;
```

Field Descriptions

`capabilityFlags`	Describes the capabilities of the card. The following flags are currently supported: • `expCapabilityHasStorage` indicates that the card supports reading and writing. • `expCapabilityReadOnly` indicates that the card is read only. • `expCapabilitySerial` indicates that the card supports a simple serial interface.
`manufacturerStr`	Names the manufacturer of the card. For example "Palm" or "Motorola".
`productStr`	Name of the product. For example "SafeBackup 32MB".
`deviceClassStr`	Describes the type of card, for example, "Backup" or "Ethernet".
`deviceUniqueIDStr`	Unique identifier for the product. A serial number for example. This value is set to the empty string if no identifier exists.

Expansion Manager Constants

Error Codes

The Expansion Manager defines the following error codes:

Constant	Description
`expErrUnsupportedOperation`	The operation is unsupported or undefined.
`expErrNotEnoughPower`	The required power is not available.
`expErrCardNotPresent`	There is no card present in the given slot.

Constant	Description
`expErrInvalidSlotRefNum`	The slot reference number is not valid.
`expErrSlotDeallocated`	The slot reference number is within the valid range, but the slot has been deallocated.
`expErrCardNoSectorReadWrite`	The card does not support the slot driver block read/write API.
`expErrCardReadOnly`	The card supports the slot driver block read/write API but the card is read only.
`expErrCardBadSector`	The card supports the slot driver block read/write API but the sector is bad.
`expErrCardProtectedSector`	The card supports the slot driver block read/write API but the sector is protected.
`expErrNotOpen`	The slot driver library has not been opened.
`expErrStillOpen`	The slot driver library is still open; it may have been opened more than once.
`expErrUnimplemented`	The call is unimplemented.
`expErrEnumerationEmpty`	There are no values remaining to enumerate.
`expErrIncompatibleAPIVer`	The API version of the underlying slot driver is not supported by this version of Expansion Manager.

Defined Media Types

The following media types are defined by the Expansion Manager. These media types are used with the function `VFSVolumeInfo` in the `VolumeInfoType.mediaType` field. The media type is also passed as a parameter to the `VFSRegisterDefaultDirectory` and `VFSUnregisterDefaultDirectory` functions.

Constant	Value	Description
`expMediaType_Any`	'wild'	Matches all media types when looking up a default directory
`expMediaType_MemoryStick`	'mstk'	Memory stick
`expMediaType_CompactFlash`	'cfsh'	Compact Flash
`expMediaType_SecureDigital`	'sdig'	Secure Digital
`expMediaType_MultiMediaCard`	'mmcd'	MultiMedia Card
`expMediaType_SmartMedia`	'smed'	SmartMedia
`expMediaType_RAMDisk`	'ramd'	A RAM disk based media
`expMediaType_PoserHost`	'pose'	Host file system emulated by the Palm OS® Emulator
`expMediaType_MacSim`	'PSim'	Host file system emulated by the Mac Simulator

Expansion Manager Functions



New ExpCardGetSerialPort

Purpose Get a card's serial port creator ID for use in serial access.

Prototype
```
Err ExpCardGetSerialPort(UInt16 slotRefNum, UInt32
*portP)
```

Parameters -> `slotRefNum` Slot number of slot to check.

	`<- portP`	Pointer to `UInt32` into which the serial port creator ID is stored.

Result Returns the following result codes:

`errNone`	No error
`expErrInvalidSlotRefNum`	The specified slot number is invalid.
`expErrSlotDeallocated`	The specified slot number is within the valid range but has been deallocated.

Compatibility Implemented only if the Expansion Manager Feature Set is present.

See Also `ExpCardInfo`



New ExpCardInfo

Purpose Obtains information about a card in a given slot.

Prototype

```
Err ExpCardInfo(UInt16 slotRefNum, ExpCardInfoType
*infoP)
```

Parameters

`-> slotRefNum`	Slot number.
`<- infoP`	Pointer to `ExpCardInfoType` structure.

Result Returns one of the following result codes:

`errNone`	No error
`expErrCardNotPresent`	There is no card present in the specified slot.
`expErrInvalidSlotRefNum`	The slot number is invalid.
`expErrSlotDeallocated`	The slot number is within the valid range but has been deallocated.

Comments This function returns information about a card, including whether the card supports secondary storage or is strictly read-only, by filling in the `ExpCardInfoType` structure's fields.

Compatibility Implemented only if the Expansion Manager Feature Set is present.

See Also `ExpCardGetSerialPort`, `ExpCardPresent`, `ExpSlotEnumerate`



New ExpCardPresent

Purpose Determines if a card is present in the given slot.

Prototype

```
Err ExpCardPresent(UInt16 slotRefNum)
```

Parameters

`-> slotRefNum` Slot number.

Result Returns the following result codes:

`errNone` A card is present in the specified slot.

`expErrCardNotPresent`
There is no card present in the specified slot.

`expErrInvalidSlotRefNum`
The specified slot number is not valid.

`expErrSlotDeallocated`
The specified slot number is within the valid range but has been deallocated.

Comments The Expansion Manager passes the call through to the appropriate slot driver.

Compatibility Implemented only if the Expansion Manager Feature Set is present.

See Also `ExpCardInfo`, `ExpSlotEnumerate`



New ExpSlotDriverInstall

Purpose

Installs and initializes a slot driver shared library into the system table.

Prototype

```
Err ExpSlotDriverInstall (UInt32 dbCreator,
UInt16 *slotLibRefNumP)
```

Parameters

`-> dbCreator` Database creator code of the slot driver library to be installed.

`-> slotLibRefNumP` Pointer to variable for returning the library reference number (on failure, `sysInvalidRefNum` is returned in this variable).

Result

Returns `errNone` if the slot driver is installed correctly. Because this function uses `SysLibInstall` to install the slot driver shared library, `ExpSlotDriverInstall` may return any of the error codes that `SysLibInstall` returns, including `sysErrLibNotFound`, `sysErrNoFreeRAM`, and `sysErrNoFreeLibSlots`. It may also return any error code returned by `SlotOpen`, the implementation of which is specific to a given device manufacturer.

Comments

This function is not typically called by applications but can be used to load additional slot drivers after the device has booted. It is called internally by the Expansion Manager to install a slot driver shared library into the library table and initialize it for use. Once installed, the `slotLibRefNum` can be used by other functions to refer to the library.

Compatibility

Implemented only if the Expansion Manager Feature Set is present.

See Also

`ExpSlotDriverRemove`, `ExpSlotLibFind`



New ExpSlotDriverRemove

Purpose Closes and remove a slot driver shared library from the system table.

Prototype
```
Err ExpSlotDriverRemove (UInt16 slotLibRefNum)
```

Parameters
`-> slotLibRefNum`
Slot driver shared library reference number.

Result Returns `errNone`.

Comments This function is not typically called by applications but can be used to unload slot drivers associated with external slots. It is called internally by the Expansion Manager to remove the shared library from the system table, and, if appropriate, release the slot allocated to the given `slotLibRefNum`. Prior to removing the slot driver, it unmounts any volumes associated with the slot.

Compatibility Implemented only if the Expansion Manager Feature Set is present.

See Also `ExpSlotDriverInstall`, `ExpSlotLibFind`, `SysLibRemove`



New ExpSlotEnumerate

Purpose Iterates through valid slot numbers.

Prototype
```
Err ExpSlotEnumerate(UInt16 *slotRefNumP,
UInt32 *slotIteratorP)
```

Parameters
`<- slotRefNumP` Reference number of the currently-enumerated slot.

`-> slotIteratorP`
Pointer to the index of the last entry enumerated. For the first iteration, initialize this parameter to the constant `expIteratorStart`. Upon return this references the next entry in the directory. If this is the last entry, this parameter is set to `expIteratorStop`.

Result Returns one of the following result codes:

`errNone` The slot reference number indicated by `slotRefNumP` is valid.

`expErrEnumerationEmpty`
There are no slots left to enumerate. `slotRefNumP` is set to the slot number of the currently enumerated slot or `invalidSlotRefNum` if there are no slots.

Comments This function iterates through the device's slots. The first time this function is called, set `*slotIteratorP` to `expIteratorStart` to find the initial slot. Once set this value is changed with each subsequent call to this function until it reaches the maximum number of slots, at which point `ExpSlotEnumerate` sets `*slotIteratorP` to `expIteratorStop`.

Example The following is an example of how `ExpSlotEnumerate` should be used:

```
UInt16 slotRefNum;
UInt32 slotIterator = expIteratorStart;
while (slotIterator != expIteratorStop) {
   if ((err = ExpSlotEnumerate(&slotRefNum,
            &slotIterator)) == errNone) {
      // do something with the slotRefNum
   else {
      // handle error (by breaking out of the
      // loop, most likely
   }
}
```

Compatibility Implemented only if the Expansion Manager Feature Set is present.

See Also ExpCardInfo, ExpCardPresent, ExpSlotDriverRemove



ExpSlotLibFind

Purpose Retrieves the slot driver library reference number for the driver that controls the specified slot.

Prototype

```
Err ExpSlotLibFind(UInt16 slotRefNum,
UInt16 *slotLibRefNum)
```

Parameters

`-> slotRefNum` Slot number.

`<- slotLibRefNum`
Pointer to the reference number for the slot driver library allocated to the given slot.

Result Returns the following result codes:

`errNone` No error.

`expErrInvalidSlotRefNum`
The slot number is invalid.

`expErrSlotDeallocated`
The slot number is within the valid range but has been deallocated.

Comments This function returns the reference number to the slot driver library loaded for the given `slotRefNum`. This function is used when making calls directly to the slot driver library.

Compatibility Implemented only if the Expansion Manager Feature Set is present.

See Also ExpSlotDriverInstall, ExpSlotDriverRemove

31

Feature Manager

This chapter provides reference material for the feature manager. The feature manager API is declared in the header file `FeatureMgr.h`.

For more information on the feature manager, see the section "Features" in the *Palm OS Programmer's Companion*, vol. I.

To learn how to use the predefined Palm OS® features to test for the existence of certain OS features, see the "Compatibility Guide" appendix.

Feature Manager Functions

FtrGet

Purpose Get a feature.

Prototype

```
Err FtrGet (UInt32 creator, UInt16 featureNum,
UInt32 *valueP)
```

Parameters

`-> creator`	Creator ID, which must be registered with Palm, Inc. This is usually the same as the creator ID for the application that owns this feature.
`-> featureNum`	Feature number of the feature.
`<- valueP`	Value of the feature is returned here.

Result Returns 0 if no error, or `ftrErrNoSuchFeature` if the specified feature number doesn't exist for the specified creator.

Comments The value of the feature is application-dependent.

See Also `FtrSet`

FtrGetByIndex

Purpose Get a feature by index.

Prototype
```
Err FtrGetByIndex (UInt16 index, Boolean romTable,
UInt32 *creatorP, UInt16 *numP, UInt32 *valueP)
```

Parameters

`-> index`	Index of feature.
`-> romTable`	If `true`, index into ROM table; otherwise, index into RAM table.
`<- creatorP`	Feature creator is returned here.
`<- numP`	Feature number is returned here.
`<- valueP`	Feature value is returned here.

Result Returns 0 if no error, or `ftrErrNoSuchFeature` if the index is out of range.

Comments This function is intended for system use only. It is used by shell commands. Most applications don't need it.

Until the caller gets back `ftrErrNoSuchFeature`, it should pass indices for each table (ROM, RAM) starting at 0 and incrementing. Note that in Palm OS 3.1 and higher, the RAM feature table serves the entire system. At system startup, the values in the ROM feature table are copied into the RAM feature table.

FtrPtrFree

Purpose Release memory previous allocated with `FtrPtrNew`.

Prototype
```
Err FtrPtrFree (UInt32 creator, UInt16 featureNum)
```

Parameters

`-> creator`	The creator ID for the feature.
`-> featureNum`	Feature number of the feature.

Result Returns 0 if no error, or `ftrErrNoSuchFeature` if an error occurs.

Comments This function unregisters the feature before freeing the memory associated with it.

Compatibility Implemented only if 3.1 New Feature Set is present.

FtrPtrNew

Purpose Allocate feature memory.

Prototype

```
Err FtrPtrNew (UInt32 creator, UInt16 featureNum,
UInt32 size, void **newPtrP)
```

Parameters

`-> creator`	Creator ID, which must be registered with Palm, Inc. This is usually the same as the creator ID for the application that owns this feature.
`-> featureNum`	Feature number of the feature.
`-> size`	Size in bytes of the temporary memory to allocate. The maximum chunk size is 64K.
`<- newPtrP`	Pointer to the memory chunk is returned here.

Result Returns 0 if no error, `memErrInvalidParam` if the value of `size` is 0, or `memErrNotEnoughSpace` if there is not enough space to allocate a chunk of the specified size.

Comments This function allocates a chunk of memory and stores a pointer to that chunk in the feature table. The same pointer is returned in `newPtrP`. The memory chunk remains allocated and locked until the next system reset or until you free the chunk with `FtrPtrFree`.

`FtrPtrNew` is useful if you want quick, efficient access to data that persists from one invocation of the application to the next. `FtrPtrNew` stores values on the storage heap rather than the dynamic heap, where free space is often extremely limited. The disadvantage to using feature memory is that writing to storage memory is slower than writing to dynamic memory.

NOTE: Starting with `Palm OS 3.5 FtrPtrNew` allows allocating chunks larger than 64k. Do keep in mind standard issues with allocating large chunks of memory: there might not be enough contiguous space, and it can impact system performance.

You can obtain the pointer to the chunk using `FtrGet`. To write to the chunk, you **must** use `DmWrite` because the chunk is in the storage heap, not the dynamic heap.

For example, if you allocate a memory chunk in this way:

```
FtrPtrNew(appCreator,
    myFtrMemFtr, 32, &ftrMem);
```

You can later access that memory and write to it using the following:

```
void* data;
if (!FtrGet(appCreator,
    myFtrMemFtr, (UInt32*)&data))
  DmWrite(data, 0, &someVal, sizeof(someVal));
```

Compatibility Implemented only if 3.1 New Feature Set is present.

See Also `FtrPtrResize`

FtrPtrResize

Purpose Resize feature memory.

Prototype

```
Err FtrPtrResize (UInt32 creator,
UInt16 featureNum, UInt32 newSize, void **newPtrP)
```

Parameters

`-> creator`	The creator ID for the feature.
`-> featureNum`	Feature number of the feature.
`-> newSize`	New size in bytes for the chunk.
`<- newPtrP`	Pointer to the memory chunk is returned here.

Result Returns 0 if no error, or `ftrErrNoSuchFeature` if the specified feature number doesn't exist for the specified creator,

`memErrInvalidParam` if `newSize` is 0, or `memErrNotEnoughSpace` if there's not enough free space available to allocate a chunk of that size.

Comments Use this function to resize a chunk of memory previously allocated by `FtrPtrNew`.

This function may move the chunk to a new location in order to resize it, so it is important to use the pointer returned by this function when accessing the memory chunk. The pointer in the feature table is automatically updated to be the same as the pointer returned by this function.

If this function fails, the old memory pointer still exists and its data is unchanged.

Compatibility Implemented only if 3.1 New Feature Set is present.

See Also `MemHandleResize`

FtrSet

Purpose Set a feature.

Prototype

```
Err FtrSet (UInt32 creator, UInt16 featureNum,
UInt32 newValue)
```

Parameters

`-> creator`	Creator ID, which must be registered with Palm, Inc. This is usually the same as the creator ID for the application that owns this feature.
`-> featureNum`	Feature number for this feature.
`-> newValue`	New value.

Result Returns 0 if no error, or `memErrNotEnoughSpace` if the feature table must be resized to add a new feature and no space is available.

Comments The value of the feature is application-dependent.

A feature that you define in this manner remains defined until the next system reset or until you explicitly undefine the feature with `FtrUnregister`.

See Also `FtrGet`, `FtrPtrNew`

FtrUnregister

Purpose Unregister a feature.

Prototype

```
Err FtrUnregister (UInt32 creator,
UInt16 featureNum)
```

Parameters

`-> creator` Creator ID for the feature.

`-> featureNum` Feature number of the feature.

Result Returns 0 if no error, or `ftrErrNoSuchFeature` if the specified feature number doesn't exist for the specified creator.

32

File Streaming

This chapter provides reference material for the file streaming API.

- File Streaming Constants
- File Streaming Functions
- File Streaming Error Codes

The header file `FileStream.h` declares the API that this chapter describes. For more information on file streaming, see the chapter "Files and Databases" in the *Palm OS Programmer's Companion*, vol. I.

File Streaming Constants

Primary Open Mode Constants

This section lists constants passed in the `openMode` parameter to the `FileOpen` function. These constants specify the mode in which a file stream is opened.

For each file stream, you must pass to the `FileOpen` function only one of the primary mode selectors listed.

Constant Values

`fileModeReadOnly`	Open for read-only access
`fileModeReadWrite`	Open/create for read/write access, discarding any previous version of stream
`fileModeUpdate`	Open/create for read/write, preserving previous version of stream if it exists
`fileModeAppend`	Open/create for read/write, always writing to the end of the stream

Secondary Open Mode Constants

You can use the | operator (bitwise inclusive OR) to append to a primary mode selector one or more of the secondary mode selectors listed below.

`fileModeDontOverwrite`	Prevents `fileModeReadWrite` from discarding an existing stream having the same name; may only be specified together with `fileModeReadWrite`
`fileModeLeaveOpen`	Leave stream open when application quits. Most applications should not use this option.
`fileModeExclusive`	No other application can open the stream until the application that opened it in this mode closes it.
`fileModeAnyTypeCreator`	Accept any type/creator when opening or replacing an existing stream. Normally, the `FileOpen` function opens only streams having the specified creator and type. Setting this option enables the `FileOpen` function to open streams having a type or creator other than those specified.
`fileModeTemporary`	Delete the stream automatically when it is closed. For more information, see Comment section of `FileOpen` function description.

File Streaming Functions

FileClearerr

Purpose Clear I/O error status, end of file error status, and last error.

Prototype
```
Err FileClearerr (FileHand stream)
```

Parameters --> `stream` Handle to open stream.

Result 0 if no error, or a `fileErr` code if an error occurs. See the section "File Streaming Error Codes" for more information.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FileGetLastError`, `FileRewind`

FileClose

Purpose Close the file stream and destroy its handle. If the stream was opened with `fileModeTemporary`, it is deleted upon closing.

Prototype
```
Err FileClose (FileHand stream)
```

Parameters --> `stream` Handle to open stream.

Result 0 if no error, or a `fileErr` code if an error occurs. See the section "File Streaming Error Codes" for more information.

Compatibility Implemented only if 3.0 New Feature Set is present.

FileControl

Purpose Perform the operation specified by the `op` parameter on the `stream` file stream.

Prototype

```
Err FileControl (FileOpEnum op, FileHand stream,
void *valueP, Int32 *valueLenP)
```

Parameters

`op`	The operation to perform, and its associated formal parameters. See the Comments section for a list of possible values.
--> `stream`	Open stream handle if required for file stream operation.
<--> `valueP`	Pointer to value or buffer, as required. This parameter is defined by the selector passed as the value of the `op` parameter. For details, see the Comments section.
<--> `valueLenP`	Pointer to value or buffer, as required. This parameter is defined by the selector passed as the value of the `op` parameter. For details, see the Comments section.

Result Returns either a value defined by the selector passed as the argument to the `op` parameter, or an error code resulting from the requested operation. For details, see the Comments section.

Comments Normally, you do not call the `FileControl` function yourself; it is called for you by most of the other file streaming functions and macros to perform common file streaming operations. You can call `FileControl` yourself to enable specialized read modes.

`fileOpNone` — No-op.

`fileOpDestructiveReadMode` — Enter destructive read mode, and rewind stream to its beginning. Once in this mode, there is no turning back: stream's contents after closing (or crash) are undefined.

Destructive read mode deletes blocks as data are read, thus freeing storage automatically. Once in destructive read mode, you cannot re-use the file stream—the contents of the stream are undefined after it is closed or after a crash.

Writing to files opened without write access or those that are in destructive read state is not allowed; thus, you cannot call the `FileWrite`, `FileSeek`, or `FileTruncate` functions on a stream that is in destructive read mode. One exception to this rule applies to streams that were opened in "write + append" mode and then switched into destructive read state. In this case, the `FileWrite` function can append data to the stream, but it also preserves the current stream position so that subsequent reads pick up where they left off (you can think of this as a pseudo-pipe).

ARGUMENTS:
`stream` = open stream handle
`valueP` = `NULL`
`valueLenP` = `NULL`

RETURNS:
zero on success;
`fileErr`... on error

`fileOpGetEOFStatus` — Get end-of-file status (like C runtime's `feof`) (err = `fileErrEOF`). Indicates end of file condition. Use `FileClearerr` to clear this error status.

ARGUMENTS:
`stream` = open stream handle
`valueP` = `NULL`
`valueLenP` = `NULL`

	RETURNS: zero if not end of file; non-zero if end of file

`fileOpGetLastError`	Get error code from last operation on stream, and clear the last error code value. Doesn't change status of EOF or I/O errors —use `FileClearerr` to reset all error codes.
	
	ARGUMENTS:
	`stream` = open stream handle
	`valueP` = `NULL`
	`valueLenP` = `NULL`
	
	RETURNS:
	Error code from last file stream operation

`fileOpClearError`	Clear I/O and EOF error status and last error.
	
	ARGUMENTS:
	`stream` = open stream handle
	`valueP` = `NULL`
	`valueLenP` = `NULL`
	
	RETURNS:
	zero on success; `fileErr`... on error

`fileOpGetIOErrorStatus`	Get I/O error status (like C runtime's `ferror`). Use `FileClearerr` to clear this error status.
	
	ARGUMENTS:
	`stream` = open stream handle
	`valueP` = `NULL`
	`valueLenP` = `NULL`
	
	RETURNS:
	zero if not I/O error;
	non-zero if I/O error is pending.

`fileOpGetCreatedStatus`	Find out whether file was created by `FileOpen` function

ARGUMENTS:
`stream` = open stream handle
`valueP` = Pointer to Boolean
`valueLenP` = Pointer to `Int32` variable set to sizeof(Boolean)

RETURNS:
zero on success; `fileErr`... on error. The Boolean variable will be set to non-zero if the file was created.

`fileOpGetOpenDbRef` — Get the open database reference (handle) of the underlying database that implements the stream (`NULL` if none); this is needed for performing Palm OS-specific operations on the underlying database, such as changing or getting creator and type, version, backup/reset bits, and so on.

ARGUMENTS:
`stream` = open stream handle
`valueP` = Pointer to `DmOpenRef` variable
`valueLenP` = Pointer to Int32 variable set to sizeof(DmOpenRef)

RETURNS:
zero on success; `fileErr`... on error. The `DmOpenRef` variable will be set to the file's open db reference that may be passed to Data Manager calls;

WARNING: Do not make any changes to the data of the underlying database -- doing so will corrupt the file stream.

`fileOpFlush` — Flush any cached data to storage.

ARGUMENTS:
`stream` = open stream handle
`valueP` = `NULL`
`valueLenP` = `NULL`

RETURNS:
zero on success; `fileErr`... on error;

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also FileClearerr, FileEOF, FileError, FileFlush, FileGetLastError, FileRewind

FileDelete

Purpose Deletes the specified file stream from the specified card. Only a closed stream may be passed to this function.

Prototype

```
Err FileDelete (UInt16 cardNo, const Char *nameP)
```

Parameters

`cardNo`	Card on which the file stream to delete resides. Currently, no Palm OS® devices support multiple cards, so this value must be 0.
`nameP`	String that is the name of the stream to delete.

Result 0 if no error, or a `fileErr` code if an error occurs. See the section "File Streaming Error Codes" for more information.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also FileOpen

FileDmRead

Purpose A macro that reads data from a file stream into a chunk, record, or resource residing in a database.

Prototype

```
Int32 FileDmRead (FileHand stream,
void *startOfDmChunkP, Int32 destOffset,
Int32 objSize, Int32 numObj, Err *errP)
```

Parameters

--> `stream`	Handle to open stream.
--> `startOfDmChunkP`	Pointer to beginning of chunk, record or resource residing in a database.

`destOffset`	Offset from `startOfDmChunkP` (base pointer) to the destination area (must be >= 0).
`objSize`	Size of each stream object to read.
`numObj`	Number of stream objects to read.
<--> `errP`	Pointer to variable that is to hold the error code returned by this function. Pass `NULL` to ignore. See the section "File Streaming Error Codes" for more information.

Result The number of whole objects that were read—note that the number of objects actually read may be less than the number requested.

Comments When the number of objects actually read is less than the number requested, you may be able to determine the cause of this result by examining the return value of the `errP` parameter or by calling the `FileGetLastError` function. If the cause is insufficient data in the stream to satisfy the full request, the current stream position is at end-of-file and the "end of file" indicator is set. If a non-`NULL` pointer was passed as the value of the `errP` parameter when the `FileDmRead` function was called and an error was encountered, `*errP` holds a non-zero error code when the function returns. In addition, the `FileError` and `FileEOF` functions may be used to check for I/O errors.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FileRead`, `FileError`, `FileEOF`

FileEOF

Purpose Get end-of-file status (err = `fileErrEOF` indicates end of file condition).

Prototype
```
Err FileEOF (FileHand stream)
```

Parameters --> `stream` Handle to open stream.

Result 0 if *not* end of file; non-zero if end of file. See the section "File Streaming Error Codes" for more information.

Comments This function's behavior is similar to that of the `feof` function provided by the C programming language runtime library.

Use `FileClearerr` to clear the I/O error status.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FileClearerr`, `FileGetLastError`, `FileRewind`

FileError

Purpose Get I/O error status.

Prototype
```
Err FileError (FileHand stream)
```

Parameters --> `stream` Handle to open stream.

Result 0 if no error, and non-zero if an I/O error indicator has been set for this stream. See the section "File Streaming Error Codes" for more information.

Comments This function's behavior is similar to that of the C programming language's `ferror` runtime function.

Use `FileClearerr` to clear the I/O error status.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also FileClearerr, FileGetLastError, FileRewind

FileFlush

Purpose Flush cached data to storage.

Prototype
```
Err FileFlush (FileHand stream)
```

Parameters --> `stream` Handle to open stream.

Result 0 if no error, or a `fileErr` code if an error occurs. See the section "File Streaming Error Codes" for more information.

Comments It is not always necessary to call this function explicitly—certain operations flush the contents of a stream automatically; for example, streams are flushed when they are closed. Because this function's behavior is similar to that of the `fflush` function provided by the C programming language runtime library, you only need to call it explicitly under circumstances similar to those in which you would call `fflush` explicitly.

Compatibility Implemented only if 3.0 New Feature Set is present.

FileGetLastError

Purpose Get error code from last operation on file stream, and clear the last error code value (will not change end of file or I/O error status -- use FileClearerr to reset all error codes)

Prototype
```
Err FileGetLastError (FileHand stream)
```

Parameters --> `stream` Handle to open stream.

Result Error code returned by the last file stream operation. See the section "File Streaming Error Codes" for more information.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also FileClearerr, FileEOF, FileError

FileOpen

Purpose Open existing file stream or create an open file stream for I/O in the mode specified by the `openMode` parameter.

Prototype

```
FileHand FileOpen (UInt16 cardNo,
const Char *nameP, UInt32 type, UInt32 creator,
UInt32 openMode, Err *errP)
```

Parameters

`cardNo`	Card on which the file stream to open resides. Currently, no Palm Powered™ devices support multiple cards, so this value must be 0.
--> `nameP`	Pointer to text string that is the name of the file stream to open or create. This value must be a valid name—no wildcards allowed, must not be `NULL`.
`type`	File type of stream to open or create. Pass `0` for wildcard, in which case `sysFileTFileStream` is used if the stream needs to be created and `fileModeTemporary` is not specified. If type is `0` and `fileModeTemporary` is specified, then `sysFileTTemp` is used for the file type of the stream this function creates.
`creator`	Creator of stream to open or create. Pass `0` for wildcard, in which case the current application's creator ID is used for the creator of the stream this function creates.
`openMode`	Mode in which to open the file stream. You must specify only one primary mode selector. Additionally, you can use the | operator (bitwise inclusive OR) to append one or more secondary mode selectors to the primary mode selector. See "Primary Open Mode Constants" and "Secondary Open Mode Constants" for the list of possible values.

<--> `errP`	Pointer to variable that is to hold the error code returned by this function. Pass `NULL` to ignore. See the section "File Streaming Error Codes" for a list of error codes.

Result If successful, returns a handle to an open file stream; otherwise, returns `0`.

In some cases, on some platforms, `FileOpen` returns a non-zero value when it has failed to open a file; thus, it is always a good idea to check the `errP` parameter value to determine if an error has occurred.

Comments The `fileModeReadOnly`, `fileModeReadWrite`, `fileModeUpdate`, and `fileModeAppend` modes are mutually exclusive—pass only one of them to the `FileOpen` function!

When the `fileModeTemporary` open mode is used and the file type passed to `FileOpen` is `0`, the `FileOpen` function uses `sysFileTTemp` (defined in `SystemMgr.rh`) for the file type, as recommended. In future versions of Palm OS, this configuration will enable the automatic cleanup of undeleted temporary files after a system crash. Automatic post-crash cleanup is not implemented in current versions of Palm OS.

To open a file stream even if it has a different type and creator than specified, pass the `fileModeAnyTypeCreator` selector as a flag in the `openMode` parameter to the `FileOpen` function.

The `fileModeLeaveOpen` mode is an esoteric option that most applications should not use. It may be useful for a library that needs to open a stream from the current application's context and keep it open even after the current application quits. By default, Palm OS automatically closes all databases that were opened in a particular application's context when that application quits. The `fileModeLeaveOpen` option overrides this default behavior.

Compatibility Implemented only if 3.0 New Feature Set is present.

FileRead

Purpose

A macro that reads data from a stream into a buffer. Do not use this macro to read data into a chunk, record or resource residing in a database—you must use the `FileDmRead` macro for such operations.

Prototype

```
Int32 FileRead (FileHand stream, void *bufP,
Int32 objSize, Int32 numObj, Err *errP)
```

Parameters

--> `stream`	Handle to open stream.
--> `bufP`	Pointer to beginning of buffer into which data is read
`objSize`	Size of each stream object to read.
`numObj`	Number of stream objects to read.
<--> `errP`	Pointer to variable that is to hold the error code returned by this function. Pass `NULL` to ignore. See the section "File Streaming Error Codes" for a list of error codes.

Result

The number of whole objects that were read—note that the number of objects actually read may be less than the number requested.

Comments

Do not use this macro to read data into a chunk, record or resource residing in a database—you must use the `FileDmRead` macro for such operations.

When the number of objects actually read is fewer than the number requested, you may be able to determine the cause of this result by examining the return value of the `errP` parameter or by calling the `FileGetLastError` function. If the cause is insufficient data in the stream to satisfy the full request, the current stream position is at end-of-file and the "end of file" indicator is set. If a non-`NULL` pointer was passed as the value of the `errP` parameter when the `FileRead` function was called and an error was encountered, `*errP` holds a non-zero error code when the function returns. In addition, the `FileError` and `FileEOF` functions may be used to check for I/O errors.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also FileDmRead

FileRewind

Purpose Reset position marker to beginning of stream and clear all error codes.

Prototype

```
Err FileRewind (FileHand stream)
```

Parameters

--> `stream` Handle to open stream.

Result 0 if no error, or a `fileErr` code if an error occurs. See the section "File Streaming Error Codes" for more information.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also FileSeek, FileTell, FileClearerr, FileEOF, FileError, FileGetLastError

FileSeek

Purpose Set current position within a file stream, extending the stream as necessary if it was opened with write access.

Prototype

```
Err FileSeek (FileHand stream, Int32 offset,
FileOriginEnum origin)
```

Parameters

--> `stream` Handle to open stream.

`offset` Position to set, expressed as the number of bytes from origin. This value may be positive, negative, or `0`.

`origin` Describes the origin of the position change. Possible values are:

`fileOriginBeginning`
From the beginning (first data byte of file).

`fileOriginCurrent`
From the current position.

`fileOriginEnd`
From the end of file (one position beyond last data byte).

Result 0 if no error, or a `fileErr` code if an error occurs. See the section "File Streaming Error Codes" for more information.

Comments Attempting to seek beyond end-of-file in a read-only stream results in an I/O error.

This function's behavior is similar to that of the `fseek` function provided by the C programming language runtime library.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FileRewind`, `FileTell`

FileTell

Purpose Retrieves the current position and, optionally, file size, of a stream.

Prototype

```
Int32 FileTell (FileHand stream, Int32 *fileSizeP,
Err *errP)
```

Parameters

--> `stream` Handle to open stream.

<-> `fileSizeP` Pointer to variable that holds value describing size of stream in bytes when this function returns. Pass `NULL` to ignore.

<--> `errP` Pointer to variable that is to hold the error code returned by this function. Pass `NULL` to ignore. See the section "File Streaming Error Codes" for a list of possible error codes.

Result If successful, returns current position, expressed as an offset in bytes from the beginning of the stream. If an error was encountered, returns `-1` as a signed long integer.

Comments The `FileTell` function can return the size of the input stream; as such, it provides some of the functionality of the standard C library `stat` function. Note, however, that unlike the `stat` function, `FileTell` requires that the file be open.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FileRewind`, `FileSeek`

FileTruncate

Purpose Truncate the file stream to a specified size; not allowed on streams open in destructive read mode or read-only mode.

Prototype

```
Err FileTruncate (FileHand stream, Int32 newSize)
```

Parameters

--> `stream`	Handle of open stream.
`newSize`	New size; must not exceed current stream size.

Result 0 if no error, or a `fileErr` code if an error occurs. See the section "File Streaming Error Codes" for a list of possible error codes.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FileTell`

FileWrite

Purpose Write data to a stream.

Prototype

```
Int32 FileWrite (FileHand stream,
const void *dataP, Int32 objSize, Int32 numObj,
Err *errP)
```

Parameters

--> `stream`	Handle to open stream.
--> `dataP`	Pointer to buffer holding data to write.

`objSize`	Size of each stream object to write; must be ≥ `0`.
`numObj`	Number of stream objects to write.
`<--> errP`	Optional pointer to variable that holds the error code returned by this function. Pass `NULL` to ignore. See the section "File Streaming Error Codes" for a list of possible error codes.

Result The number of whole objects that were written—note that the number of objects actually written may be less than the number requested. Should available storage be insufficient to satisfy the entire request, as much of the requested data as possible is written to the stream, which may result in the last object in the stream being incomplete.

Comments Writing to files opened without write access or those that are in destructive read state is not allowed; thus, you cannot call the `FileWrite`, `FileSeek`, or `FileTruncate` functions on a stream that is in destructive read mode. One exception to this rule applies to streams that were opened in "write + append" mode and then switched into destructive read state. In this case, the `FileWrite` function can append data to the stream, but it also preserves the current stream position so that subsequent reads pick up where they left off (you can think of this as a pseudo-pipe).

Compatibility Implemented only if 3.0 New Feature Set is present.

File Streaming Error Codes

This section lists all error codes returned by the file streaming functions.

Error Code	Value	Meaning
`fileErrMemErr`	(`fileErrorClass`|1)	Out of memory error
`fileErrInvalidParam`	(`fileErrorClass`|2)	Invalid parameter value passed
`fileErrCorruptFile`	(`fileErrorClass`|3)	Alleged stream is corrupted, invalid, or not a stream
`fileErrNotFound`	(`fileErrorClass`|4)	Couldn't find the stream
`fileErrTypeCreatorMismatch`	(`fileErrorClass`|5)	Type and/or creator not what was specified
`fileErrReplaceError`	(`fileErrorClass`|6)	Couldn't replace existing stream
`fileErrCreateError`	(`fileErrorClass`|7)	Couldn't create new stream
`fileErrOpenError`	(`fileErrorClass`|8)	Generic open error
`fileErrInUse`	(`fileErrorClass`|9)	Stream couldn't be opened or deleted because it is in use
`fileErrReadOnly`	(`fileErrorClass`|10)	Couldn't open in write mode because existing stream is read-only
`fileErrInvalidDescriptor`	(`fileErrorClass`|11)	Invalid file descriptor `(FileHandle)`

Error Code	Value	Meaning
`fileErrCloseError`	(`fileErrorClass`|12)	Error closing the stream
`fileErr`OutOfBounds	(`fileErrorClass`|13)	Attempted operation went out of bounds of the stream
`fileErrPermissionDenied`	(`fileErrorClass`|14)	Couldn't write to a stream open for read-only access
`fileErrIOError`	(`fileErrorClass`|15)	Generic I/O error
`fileErrEOF`	(`fileErrorClass`|16)	End-of-File error
`fileErrNotStream`	(`fileErrorClass`|17)	Attempted to open an entity that is not a stream

33

Float Manager

This chapter provides reference material for the Float Manager API as follows:

- Float Manager Data Structures
- Float Manager Functions

The Float Manager API is declared in the header file `FloatMgr.h`. For more information on the Float Manager, see the section "Floating-Point" in the *Palm OS Programmer's Companion*, vol. I.

Float Manager Data Structures

FlpCompDouble

Float Manager functions accept and require values of type `FlpDouble`. The `FlpCompDouble` union allows you to declare values that can be interpreted either as a double or as an `FlpDouble`. As well, this union contains fields that provide easy access to the component parts of the double-precision floating-point number.

```
typedef union {
  double d;
  FlpDouble fd;
  UInt32 ul[2];
  FlpDoubleBits fdb;
} FlpCompDouble
```

Field Descriptions

`d`	Provides access to the value as a `double`.
`fd`	Provides access to the value as a `FlpDouble`, which can be passed to or received from many Float Manager functions.
`ul`	Provides access to the value as two long integers.
`fdb`	Provides access to specific fields.

FlpDoubleBits

This structure provides direct access to the component parts of an IEEE-754 double-precision floating-point number. Use the `FlpCompDouble` union to convert numbers of type `double` to and from `FlpDoubleBits`.

```
typedef struct {
  UInt32 sign : 1;
  Int32 exp : 11;
  UInt32 manH : 20;
  UInt32 manL;
} FlpDoubleBits
```

Field Descriptions

`sign`	The sign bit. You can also use the `FlpGetSign` macro to obtain the sign bit, and the `FlpNegate`, `FlpSetNegative`, and `FlpSetPositive` macros to set the sign bit.
`exp`	The bits that make up the exponent. You can also use the `FlpGetExponent` macro to obtain the exponent value.
`manH`	The most-significant 20 bits of the mantissa.
`manL`	The least-significant 32 bits of the mantissa.

Float Manager Functions

FlpAToF

Purpose Convert a null-terminated ASCII string to a 64-bit floating-point number. The string must have the format:

```
[+|-][digits][.][digits][e|E[+|-][digits]]
```

Prototype

```
FlpDouble FlpAToF (const Char *s)
```

Parameters

`-> s` Pointer to the string to be converted.

Result Returns the value of the string as a floating-point number.

Comment The mantissa of the number is limited to 32 bits.

This function is close to ANSI C library compatible. ANSI requires the form:

```
[+|-]digits[.][digits][(e|E)[+|-]digits]
```

In order to maintain backward compatibility with the Palm OS 1.0, this function considers **all** of the "digits" sections to be optional. Here's a table showing the ANSI and Palm OS behavior with some sample strings:

String	ANSI	> Palm OS 1.0	Palm OS 1.0[a]	Notes
"+"	invalid	+0	+0	ANSI requires at least one digit.
".3"	invalid	0.3	0.3	ANSI requires a leading digit.
"0.3e123"	0.3e123	0.3e123	0.3e12	Palm OS 1.0 only allows 1 or 2 digit exponent.
"+1"	1	1	+0	Palm OS 1.0 doesn't allow a leading '+' sign.
"1e+2"	1e2	1e2	1	Palm OS 1.0 doesn't allow a '+' sign in the exponent.

String	ANSI	> Palm OS 1.0	Palm OS 1.0[a]	Notes
"0.3E3"	0.3e3	0.3e3	0.3	Palm OS 1.0 doesn't allow a capital 'E' to mark the exponent.
"4294967297"	4294967297	4294967297	1	Palm OS 1.0 uses unsigned long and wraps around.

a. Using the old Float Manager documented in Appendix C, "1.0 Float Manager." on page 1803.

Compatibility Implemented only if 2.0 New Feature Set is present. GCC users must use `FlpBufferAToF` instead of this function.

See Also `FlpFToA`

FlpBase10Info

Purpose Extract detailed information on the base 10 form of a floating-point number: the base 10 mantissa, exponent, and sign.

Prototype

```
Err FlpBase10Info (FlpDouble a, UInt32 *mantissaP,
Int16 *exponentP, Int16 *signP)
```

Parameters

`-> a`	The floating-point number.
`<- mantissaP`	The base 10 mantissa.
`<- exponentP`	The base 10 exponent.
`<- signP`	The sign: 1 if the number is negative, 0 otherwise.

Result Returns 0 if no error, or `flpErrOutOfRange` if the supplied floating-point number is either not a number (NaN) or is infinite.

Comments The mantissa is normalized so it contains at least 8 significant digits when printed as an integer value.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also FlpGetExponent, FlpGetSign

FlpBufferAToF

Purpose Convert a null-terminated ASCII string to a floating-point number. The string must be in the format: `[-]x[.]yyyyyyyy[e[-]zz]`

Prototype

```
void FlpBufferAToF (FlpDouble *result,
const Char *s)
```

Parameters

`<- result`	Pointer to the structure into which the return value is placed.
`-> s`	Pointer to the null-terminated ASCII string to be converted.

Result Returns the value of the string as a floating-point number.

Comments See FlpAToF for a complete description of this function.

Compatibility Implemented only if 2.0 New Feature Set is present. GCC users must use this function instead of FlpAToF due to incompatibilities in the way GCC handles structure return values. CodeWarrior users can use either function; they are binary compatible.

FlpBufferCorrectedAdd

Purpose Adds two floating-point numbers and corrects for least-significant-bit errors when the result should be zero but is instead very close to zero.

Prototype

```
void FlpBufferCorrectedAdd (FlpDouble *result,
FlpDouble firstOperand, FlpDouble secondOperand,
Int16 howAccurate)
```

Parameters

`<- result`	Pointer to the structure into which the return value is placed.

`-> firstOperand`
The first of the two numbers to be added.

`-> secondOperand`
The second of the two numbers to be added.

`-> howAccurate` The smallest difference in exponents that won't force the result to zero. The value returned from this function is forced to zero if the difference between exponents in the smaller of the two operands and the result exceeds this value. Supply a value of zero for this parameter to obtain the default level of accuracy (which is equivalent to a `howAccurate` value of 48).

Result Returns the calculated result.

Comments See `FlpCorrectedAdd` for a complete description of this function.

Compatibility Implemented only if 2.0 New Feature Set is present. GCC users must use this function instead of `FlpCorrectedAdd` due to incompatibilities in the way GCC handles structure return values. CodeWarrior users can use either function; they are binary compatible.

FlpBufferCorrectedSub

Purpose Subtracts two floating-point numbers and corrects for least-significant-bit errors when the result should be zero but is instead very close to zero.

Prototype
```
void FlpBufferCorrectedSub (FlpDouble *result,
FlpDouble firstOperand, FlpDouble secondOperand,
Int16 howAccurate)
```

Parameters `<- result` Pointer to the structure into which the return value is placed.

`-> firstOperand`
The value from which `secondOperand` is to be subtracted.

`-> secondOperand`
The value to subtract from `firstOperand`.

`-> howAccurate` The smallest difference in exponents that won't force the result to zero. The value returned from this function is forced to zero if the difference between exponents in the smaller of the two operands and the result exceeds this value. Supply a value of zero for this parameter to obtain the default level of accuracy (which is equivalent to a `howAccurate` value of 48).

Result Returns the calculated result.

Comments See `FlpCorrectedSub` for a complete description of this function.

Compatibility Implemented only if 2.0 New Feature Set is present. GCC users must use this function instead of `FlpCorrectedSub` due to incompatibilities in the way GCC handles structure return values. CodeWarrior users can use either function; they are binary compatible.

FlpCorrectedAdd

Purpose Adds two floating-point numbers and corrects for least-significant-bit errors when the result should be zero but is instead very close to zero.

Prototype
```
FlpDouble FlpCorrectedAdd (FlpDouble firstOperand,
FlpDouble secondOperand, Int16 howAccurate)
```

Parameters

`-> firstOperand`
The first of the two numbers to be added.

`-> secondOperand`
The second of the two numbers to be added.

`-> howAccurate` The smallest difference in exponents that won't force the result to zero. The value returned from `FlpCorrectedAdd` is forced to zero if, when the exponent of the result of the addition is subtracted from the exponent of the smaller of the two operands, the difference exceeds the value specified for `howAccurate`. Supply a value of zero for this parameter to obtain the default level of accuracy (which is equivalent to a `howAccurate` value of 48).

Result Returns the calculated result.

Comments Adding or subtracting a large number and a small number produces a result similar in magnitude to the larger number. Adding or subtracting two numbers that are similar in magnitude can, depending on their signs, produce a result with a very small exponent (that is, a negative exponent that is large in magnitude). If the difference between the result's exponent and that of the operands is close to the number of significant bits expressible by the mantissa, it is quite possible that the result should in fact be zero.

There also exist cases where it may be useful to retain accuracy in the low-order bits of the mantissa. For instance: 99999999 + 0.00000001 - 99999999. However, unless the fractional part is an exact (negative) power of two, it is doubtful that what few bits of mantissa that are available will be enough to properly represent the fractional value. In this example, the 99999999 requires 26 bits, leaving 26 bits for the .00000001; this guarantees inaccuracy after the subtraction.

The problem arises from the difficulty in representing decimal fractions such as 0.1 in binary. After about three successive additions or subtractions, errors begin to appear in the least significant bits of the mantissa. If the value represented by the most significant bits of the mantissa is then subtracted away, the least significant bit error is normalized and becomes the actual result—when in fact the result should be zero.

This problem is only an issue for addition and subtraction.

Compatibility Implemented only if 2.0 New Feature Set is present. GCC users must use `FlpBufferCorrectedAdd` instead of this function.

See Also `FlpCorrectedSub`

FlpCorrectedSub

Purpose Subtracts two floating-point numbers and corrects for least-significant-bit errors when the result should be zero but is instead very close to zero.

Prototype

```
FlpDouble FlpCorrectedSub (FlpDouble firstOperand,
FlpDouble secondOperand, Int16 howAccurate)
```

Parameters

`-> firstOperand`
The value from which `secondOperand` is to be subtracted.

`-> secondOperand`
The value to subtract from `firstOperand`.

`-> howAccurate` The smallest difference in exponents that won't force the result to zero.The value returned from `FlpCorrectedSub` is forced to zero if, when the exponent of the result of the subtraction is subtracted from the exponent of the smaller of the two operands, the difference exceeds the value specified for `howAccurate`. Supply a value of zero for this parameter to obtain the default level of accuracy (which is equivalent to a `howAccurate` value of 48).

Result Returns the calculated result.

Comments See the comments for `FlpCorrectedAdd`.

Compatibility Implemented only if 2.0 New Feature Set is present. GCC users must use `FlpBufferCorrectedSub` instead of this function.

FlpFToA

Purpose Convert a floating-point number to a null-terminated ASCII string in exponential format: `[-]x.yyyyyyye[-]zz`

Prototype

```
Err FlpFToA (FlpDouble a, Char *s)
```

Parameters

`-> a`	Floating-point number.
`<-s`	Pointer to buffer to contain the ASCII string.

Result Returns 0 if no error, or `flpErrOutOfRange` if the supplied value is infinite or is not a number. In this case, the buffer is set to the string "INF", "-INF", or "NaN" as appropriate.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `FlpAToF`

FlpGetExponent

Purpose Macro that returns the exponent of a 64-bit floating-point value. The returned value has the bias applied, so it ranges from -1023 to +1024.

Prototype

```
FlpGetExponent (x)
```

Parameters

`-> x`	The value from which the exponent is to be extracted.

Result Returns a `UInt32` containing the exponent of the specified value.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `FlpBase10Info`, `FlpGetSign`

FlpGetSign

Purpose Macro that returns the sign of a 64-bit floating-point value.

Prototype
```
FlpGetSign (x)
```

Parameters

`-> x`	The value from which the sign bit is to be extracted.

Result Returns a `UInt32` with a nonzero value if the specified value is negative, and with a zero value if it is positive.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `FlpBase10Info`, `FlpGetExponent`, `FlpNegate`, `FlpSetNegative`, `FlpSetPositive`

FlpIsZero

Purpose Macro that returns whether the specified 64-bit floating-point value is zero.

Prototype
```
FlpIsZero (x)
```

Parameters

`-> x`	The value for which the sign bit is desired.

Result Returns a `UInt32` with a nonzero value if the specified value is zero, and with a zero value if the specified value is other than zero.

Compatibility Implemented only if 2.0 New Feature Set is present.

FlpNegate

Purpose Macro that changes the sign bit of a 64-bit floating-point number.

Prototype
```
FlpNegate (x)
```

Parameters

`-> x`	The value in which the sign bit is to be changed.

Result Returns a 64-bit floating-point value which is the negative of the value specified by `x`.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `FlpGetSign`, `FlpSetNegative`, `FlpSetPositive`

FlpSetNegative

Purpose Macro that ensures that a 64-bit floating-point number is negative.

Prototype
```
FlpSetNegative (x)
```

Parameters

`-> x`	The value that is to be forced negative.

Result If the supplied 64-bit floating-point value is negative, that value is returned unchanged. If the supplied value is positive, the negative of that value is returned.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `FlpGetSign`, `FlpNegate`, `FlpSetPositive`

FlpSetPositive

Purpose Macro that ensures that a 64-bit floating-point number is positive.

Prototype
```
FlpSetPositive (x)
```

Parameters

`-> x`	The value that is to be forced positive.

Result If the supplied 64-bit floating-point value is positive, that value is returned unchanged. If the supplied value is negative, its absolute value is returned.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `FlpGetSign`, `FlpNegate`, `FlpSetNegative`

FlpVersion

Purpose Returns the version number of the Float Manager.

Prototype
```
UInt32 FlpVersion (void)
```

Parameters None.

Result Returns the version number of the Float Manager. The current version is represented by the constant `flpVersion`, which is defined in `FloatMgr.h`.

Compatibility Implemented only if 2.0 New Feature Set is present.

34

Fonts

This chapter provides reference material for font support. The API that this chapter describes is declared in the header files `Font.h` and `FontSelect.h`. For more information on fonts, see the "Text" section in the *Palm OS Programmer's Companion*, vol. I.

Font Functions

FntAverageCharWidth

Purpose Return the average character width in the current font.

Prototype

```
Int16 FntAverageCharWidth (void)
```

Parameters None.

Result Returns the average character width (in pixels).

FntBaseLine

Purpose Return the distance from the top of character cell to the baseline for the current font.

Prototype

```
Int16 FntBaseLine (void)
```

Parameters None.

Result Returns the baseline of the font (in pixels).

FntCharHeight

Purpose Return the character height in the current font including accents and descenders.

Prototype

```
Int16 FntCharHeight (void)
```

Parameters None

Result Height of the characters in the current font, expressed in pixels.

FntCharsInWidth

Purpose Find the length in bytes of the characters from a specified string that fit within a passed width.

Prototype

```
void FntCharsInWidth (Char const *string,
Int16 *stringWidthP, Int16 *stringLengthP,
Boolean *fitWithinWidth)
```

Parameters

-> `string` A pointer to the character string.

<-> `stringWidthP`
The maximum width to allow (in pixels). Upon return, contains the actual width allowed. Note that this value does not include any trailing spaces or tabs, which are stripped by this function.

<-> `stringLengthP`
The maximum length of text to allow, in bytes (assumes current font). Upon return, contains the number of bytes of text that can appear within the width. Note that this value does not include any trailing space or tabs, which are stripped by this function.

`<- fitWithinWidth`
Upon return, `false` if the string is considered truncated, `true` if it isn't.

Result Returns nothing.

Comments Spaces and tabs at the end of a string are ignored and removed. Characters after a carriage return are ignored, and the string is considered truncated.

See Also `FntWidthToOffset`

FntCharsWidth

Purpose Return the width of the specified character string. The **Missing Character Symbol** (an open rectangle) is substituted for any character that does not exist in the current font.

Prototype
```
Int16 FntCharsWidth (Char const *chars, Int16 len)
```

Parameters
-> `chars` Pointer to a string of characters.
-> `len` Length in bytes of the string.

Result Returns the width of the string, in pixels.

FntCharWidth

Purpose Return the width of the specified character. If the specified character does not exist within the current font, the Missing Character Symbol is substituted.

Prototype
```
Int16 FntCharWidth (Char ch)
```

Parameters
-> `ch` Character whose width is needed.

Result Returns the width of the specified character (in pixels).

Comments `FntCharWidth` works with single-byte characters only. To determine the pixel width of a single-byte character or a multi-byte character, use `FntWCharWidth` instead of this function on Palm OS® 4.0 and higher. Alternatively, for compatibility with earlier versions of Palm OS, link with the PalmOSGlue library and call `FntGlueWCharWidth`. For more information, see Chapter 76, "PalmOSGlue Library.".

FntDefineFont

Purpose Makes a custom font available to your application. The custom font is available only when the application that called this function is running; when the application quits, the custom font is uninstalled automatically.

Prototype

```
Err FntDefineFont (FontID font, FontPtr fontP)
```

Parameters

-> `font` An application-defined single-byte value greater than `fntAppFontCustomBase` (currently 128) that identifies the custom font to the system. Although this value is local to the application that called the `FntDefineFont` function, it must be greater than `fntAppCustomBase` because values less than that are reserved for system use. Note that font IDs are 8-bit unsigned values, and so must be less than 256.

-> `fontP` Pointer to the custom font resource to be used by this function. This resource must remain locked until the calling application undefines the custom font or quits.

Result

`errNone` no error

`memErrNotEnoughSpace` Insufficient dynamic heap space

Comments The font this function specifies is not available at build time; as a result, some UI elements—labels, for example—cannot determine

their bounds automatically as they do when using the built-in fonts. This mechanism and its associated tools may be augmented in the near future; for more information, stay in contact with Palm™.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FontSelect`, `FntSetFont`

FntDescenderHeight

Purpose Return the height of a character's descender in the current font. The height of a descender is the distance between the base line an the bottom of the character cell.

Prototype

```
Int16 FntDescenderHeight (void)
```

Parameters None.

Result Returns the height of a descender, expressed in pixels.

FntGetFont

Purpose Return the font ID of the current font.

Prototype

```
FontID FntGetFont (void)
```

Parameters None.

Result Returns the font ID of the current font.

FntGetFontPtr

Purpose Return a pointer to the current font.

Prototype

```
FontPtr FntGetFontPtr (void)
```

Parameters None.

Result Returns the `FontPtr` of the current font.

FntGetScrollValues

Purpose Return the values needed to update a scroll bar based on a specified string and the position within the string.

Prototype

```
void FntGetScrollValues (Char const *chars,
UInt16 width, UInt16 scrollPos, UInt16 *linesP,
UInt16 *topLine)
```

Parameters

-> `chars`	A null-terminated string.
-> `width`	The width of a line of text in the display, given in pixels.
-> `scrollPos`	The byte offset of the first character displayed on the topmost line.
<- `linesP`	Number of lines required to display the string.
<- `topLine`	The line of text that is the topmost visible line. Line numbering starts with 0.

Result Returns nothing. Stores the number of lines of text in `linesP` and the top visible line in `topLine`.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `FldGetScrollValues`

FntLineHeight

Purpose Return the height of a line in the current font. The height of a line is the height of the character cell plus the space between lines (the external leading).

Prototype
```
Int16 FntLineHeight (void)
```

Parameters None.

Result Returns the height of a line in the current font.

FntLineWidth

Purpose Return the width of the specified line of text, taking tab characters in to account. The function assumes that the characters passed are left-aligned and that the first character in the string is the first character drawn on a line. In other words, this routine doesn't work for characters that don't start at the beginning of a line.

Prototype
```
Int16 FntLineWidth (Char const *pChars,
UInt16 length)
```

Parameters

-> `pChars`	Pointer to a string of characters.
-> `length`	Length in bytes of the string.

Result Returns the line width (in pixels).

FntSetFont

Purpose Set the current font.

Prototype
```
FontID FntSetFont (FontID font)
```

Parameters

-> `font`	ID of the font to make the active font.

Result Returns the ID of the current font before the change.



New FntWCharWidth

Purpose Returns the width of the specified character. If the specified character does not exist within the current font, the Missing Character Symbol is substituted.

Prototype
```
Int16 FntWCharWidth (WChar iChar)
```

Parameters

-> `iChar`	Character whose width is needed.

Result Returns the width of the specified character (in pixels).

Comments `FntWCharWidth` works with both single-byte characters and multi-byte characters. However, you should always pass a `WChar` variable to this function rather than a `Char` to avoid sign extension problems on values 0x80 and higher.

Compatibility Implemented only if 4.0 New Feature Set is present. If you want to use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `FntGlueWCharWidth`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `FntCharWidth`

FntWidthToOffset

Purpose Given a pixel position, return the offset of the character displayed at that location.

Prototype
```
Int16 FntWidthToOffset (Char const *pChars,
UInt16 length, Int16 pixelWidth,
Boolean *leadingEdge, Int16 *truncWidth)
```

Parameters

-> `pChars`	Pointer to the character string.
-> `length`	The length in bytes of `pChars`.

`-> pixelWidth`	A horizontal pixel offset from the beginning of the string.
`<- leadingEdge`	Set to `true` if the pixel position `pixelWidth` falls on the left side of the character. Pass `NULL` for this parameter if you don't need this information.
`<- truncWidth`	The width of the text (in pixels) up to the returned offset. Pass `NULL` for this parameter if you don't need this information.

Result Returns the byte offset into `pChars` of the character that contains the pixel offset `pixelWidth`.

Compatibility Implemented only if 3.1 New Feature Set is present.

FntWordWrap

Purpose Given a string, determine how many bytes of text can be displayed within the specified width.

Prototype

```
UInt16 FntWordWrap (Char const *chars,
UInt16 maxWidth)
```

Parameters

-> `chars`	A pointer to a null-terminated string.
-> `maxWidth`	The maximum line width in pixels.

Result Returns the length of the line, in bytes. If the entire string cannot be displayed within `maxWidth`, the value that this function returns specifies the offset where the line should be broken, which is typical following a space, tab, or line-feed character.

Compatibility Implemented only if 2.0 New Feature Set is present.

FntWordWrapReverseNLines

Purpose Word wrap a text string backwards by the number of lines specified. The character position of the start of the first line and the number of lines that are actually word wrapped are returned.

Prototype

```
void FntWordWrapReverseNLines
(Char const *const chars, UInt16 maxWidth,
UInt16 *linesToScrollP, UInt16 *scrollPosP)
```

Parameters

-> `chars` A pointer to a null-terminated string.

-> `maxWidth` The maximum line width in pixels.

<-> `linesToScrollP` The number of lines to scroll. Upon return, contains the number of lines that were scrolled.

<-> `scrollPosP` The byte offset of the first character displayed on the topmost line. Upon return, contains the first character after wrapping.

Result Returns nothing. Stores the first character after wrapping and the number of lines scrolled in `scrollPosP` and `linesToScrollP`.

Compatibility Implemented only if 2.0 New Feature Set is present.

FontSelect

Purpose Displays a dialog box in which the user can choose one of three system-supplied fonts, and returns a `FontID` value representing the user's choice.

Prototype

```
FontID FontSelect (FontID fontID)
```

Parameters

-> `fontID` — A `fontID` value specifying the font to be highlighted as the default choice in the dialog box that this function displays. This value must be one of the following system-supplied constants:

`stdFont`
: Standard plain text font.

`boldFont`
: Bold version of `stdFont`.

`largeFont`
: A large plain text font (Japanese devices only).

`largeBoldFont`
: Larger version of `boldFont`.

Result

Returns a `fontID` value representing the font that the user chose in the dialog box that this function displays.

Comments

When your application starts up for the first time, it should use the features `sysFtrDefaultFont` and `sysFtrDefaultBoldFont` to determine the default font for the application. For example:

```
FtrGet(sysFtrCreator, sysFtrDefaultFont,
    &fntID)
```

After this call returns, `fntID` contains an ID compatible with the `FontSelect` function.

If you link with the PalmOSGlue library for backwards compatibility, you should instead call `FntGlueGetDefaultFontID` to determine the correct default font. For example:

```
fntID =
FntGlueGetDefaultFontID(defaultSystemFont);
```

Compatibility

Implemented only if 3.0 New Feature Set is present. The `sysFtrDefaultFont` and `sysFtrDefaultBoldFont` features are implemented only if 3.1 New Feature Set is present.

See Also

`FntGetFont`, `FntSetFont`

35

Graffiti Manager

This chapter provides reference material for the Graffiti® manager. The Graffiti manager API is declared in the header file `Graffiti.h`.

For more information on the Graffiti manager, see the section "Receiving User Input" in the *Palm OS Programmer's Companion*, vol. I.

Graffiti Manager Functions

GrfAddMacro

Purpose Add a macro to the macro list.

Prototype

```
Err GrfAddMacro (const Char *nameP,
UInt8 *macroDataP, UInt16 dataLen)
```

Parameters

`nameP`	Name of macro.
`macroDataP`	Data of macro.
`dataLen`	Size of macro data in bytes.

Result Returns 0 if no error; returns `grfErrNoMacros`, `grfErrMacroPtrTooSmall`, `dmErrNotValidRecord`, `dmErrWriteOutOfBounds` if an error occurs.

See Also `GrfGetMacro`, `GrfGetMacroName`, `GrfDeleteMacro`

GrfAddPoint

Purpose Add a point to the Graffiti point buffer.

Prototype
```
Err GrfAddPoint (PointType *pt)
```

Parameters

`pt`	Pointer to point buffer.

Result Returns 0 if no error; returns `grfErrPointBufferFull` if an error occurs.

See Also `GrfFlushPoints`

GrfCleanState

Purpose Remove any temporary shifts from the dictionary state.

Prototype
```
Err GrfCleanState (void)
```

Parameters None

Result Returns 0 if no error, or `grfErrNoDictionary` if an error occurs.

See Also `GrfInitState`

GrfDeleteMacro

Purpose Delete a macro from the macro list.

Prototype
```
Err GrfDeleteMacro (UInt16 index)
```

Parameters

`index`	Index of the macro to delete.

Result Returns 0 if no error, or `grfErrNoMacros`, `grfErrMacroNotFound` if an error occurs.

See Also `GrfAddMacro`

GrfFilterPoints

Purpose Filter the points in the Graffiti point buffer.

Prototype
```
Err GrfFilterPoints (void)
```

Parameters None.

Result Always returns 0.

See Also GrfMatch

GrfFindBranch

Purpose Locate a branch in the Graffiti dictionary by flags.

Prototype
```
Err GrfFindBranch (UInt16 flags)
```

Parameters

`flags`	Flags of the branch you're searching for.

Result Returns 0 if no error, or `grfErrNoDictionary` or `grfErrBranchNotFound` if an error occurs.

See Also GrfCleanState, GrfInitState

GrfFlushPoints

Purpose Dispose of all points in the Graffiti point buffer.

Prototype
```
Err GrfFlushPoints (void)
```

Parameters None.

Result Always returns 0.

See Also GrfAddPoint

GrfGetAndExpandMacro

Purpose Look up and expand a macro in the current macros.

Prototype
```
Err GrfGetAndExpandMacro (Char *nameP,
UInt8 *macroDataP, UInt16 *dataLenP)
```

Parameters

`nameP`	Name of macro to look up.
`macroDataP`	Macro contents returned here.
`dataLenP`	On entry, size of `macroDataP` buffer; on exit, number of bytes in macro data.

Result Returns 0 if no error, or `grfErrNoMacros` or `grfErrMacroNotFound` if an error occurs.

See Also `GrfAddMacro`, `GrfGetMacro`

GrfGetGlyphMapping

Purpose Look up a glyph in the dictionary and return the text.

Prototype
```
Err GrfGetGlyphMapping (UInt16 glyphID,
UInt16 *flagsP, void *dataPtrP, UInt16 *dataLenP,
UInt16 *uncertainLenP)
```

Parameters

`glyphID`	Glyph ID to look up.
`flagsP`	Returned dictionary flags.
`dataPtrP`	Where returned text goes.
`dataLenP`	On entry, size of `dataPtrP`; on exit, number of bytes returned.
`uncertainLenP`	Return number of uncertain characters in text.

Result Returns 0 if no error, or `grfErrNoDictionary` or `grfErrNoMapping` if an error occurs.

See Also `GrfMatch`

GrfGetMacro

Purpose Look up a macro in the current macros.

Prototype
```
Err GrfGetMacro (Char *nameP, UInt8 *macroDataP,
UInt16 *dataLenP)
```

Parameters

`nameP`	Name of macro to lookup.
`macroDataP`	Macro contents returned here.
`dataLenP`	On entry: size of `macroDataP` buffer. On exit: number of bytes in macro data.

Result Returns 0 if no error or `grfErrNoMacros`, `grfErrMacroNotFound`.

See Also `GrfAddMacro`

GrfGetMacroName

Purpose Look up a macro name by index.

Prototype
```
Err GrfGetMacroName (UInt16 index, Char *nameP)
```

Parameters

`index`	Index of macro.
`nameP`	Name returned here.

Result Returns 0 if no error, or `grfErrNoMacros` or `grfErrMacroNotFound` if an error occurs.

See Also `GrfAddMacro`, `GrfGetMacro`

GrfGetNumPoints

Purpose Return the number of points in the point buffer.

Prototype
```
Err GrfGetNumPoints (UInt16 *numPtsP)
```

Parameters

`numPtsP`	Returned number of points.

Result Always returns 0.

See Also `GrfAddPoint`

GrfGetPoint

Purpose Return a point out of the Graffiti point buffer.

Prototype
```
Err GrfGetPoint (UInt16 index, PointType *pointP)
```

Parameters

`index`	Index of the point to get.
`pointP`	Returned point.

Result Returns 0 if no error, or `grfErrBadParam` if an error occurs.

See Also `GrfAddPoint`, `GrfGetNumPoints`

GrfGetState

Purpose Return the current Graffiti shift state.

Prototype
```
Err GrfGetState (Boolean *capsLockP,
Boolean *numLockP, UInt16 *tempShiftP,
Boolean *autoShiftedP)
```

Parameters

`capsLockP`	Returns `true` if caps lock on.
`numLockP`	Returns `true` if num lock on.
`tempShiftP`	Current temporary shift.

`autoShiftedP`	Returns `TRUE` if shift not set by the user but by the system, for example, at the beginning of a line.

Result Always returns 0.

Compatibility Palm OS® 2.0 and later has more user-friendly auto shifting. It uses an upper case letter under these conditions:

- after an empty field
- after a period or other sentence terminator (such as ? or !).
- after two spaces

See Also `GrfSetState`

GrfInitState

Purpose Reinitialize the Graffiti dictionary state.

Prototype
```
Err GrfInitState (void)
```

Parameters None.

Result Always returns 0.

See Also `GrfGetState`, `GrfSetState`

GrfMatch

Purpose Recognize the current stroke in the Graffiti point buffer and return with the recognized text.

Prototype
```
Err GrfMatch (UInt16 *flagsP, void *dataPtrP,
UInt16 *dataLenP, UInt16 *uncertainLenP,
GrfMatchInfoPtr matchInfoP)
```

Parameters

`flagsP`	Glyph flags are returned here.
`dataPtrP`	Return text is placed here.
`dataLenP`	Size of `dataPtrP` on exit; number of characters returned on exit.
`uncertainLenP`	Return number of uncertain characters.
`matchInfoP`	Array of `grfMaxMatches`, or nil.

Result Returns 0 if no error, or `grfErrNoGlyphTable`, `grfErrNoDictionary`, or `grfErrNoMapping` if an error occurs.

See Also `GrfAddPoint`, `GrfFlushPoints`

GrfMatchGlyph

Purpose Recognize the current stroke as a glyph.

Prototype

```
Err GrfMatchGlyph (GrfMatchInfoPtr matchInfoP,
Int16 maxUnCertainty, UInt16 maxMatches)
```

Parameters

`matchInfoP`	Pointer to array of matches to fill in.
`maxUnCertainty`	Maximum number of errors to tolerate.
`maxMatches`	Size of `matchInfoP` array.

Result Returns 0 if no error, or `grfErrNoGlyphTable` if an error occurs.

See Also `GrfMatch`

GrfProcessStroke

Purpose Translate a stroke to keyboard events using Graffiti.

Prototype

```
Err GrfProcessStroke (PointType *startPtP,
PointType *endPtP, Boolean upShift)
```

Parameters

`startPtP`	Start point of stroke.
`endPtP`	End point of stroke.
`upShift`	Set to `true` to feed an artificial upshift into the engine.

Result Returns 0 if recognized.

Comments Called by `SysHandleEvent` when a `penUpEvent` is detected in the writing area. This routine recognizes the stroke and sends the recognized characters into the key queue. It also flushes the stroke out of the pen queue after recognition.

See Also `SysHandleEvent`

GrfSetState

Purpose Set the current shift state of Graffiti.

Prototype

```
Err GrfSetState (Boolean capsLock,
Boolean numLock, Boolean upperShift)
```

Parameters

`capsLock`	Set to `true` to turn on caps lock.
`numLock`	Set to `true` to turn on num lock.
`upperShift`	Set to `true` to put into upper shift.

Result Always returns 0.

See Also `GrfGetState`

36

Helper API

This chapter describes the Helper API declared in the header files `Helper.h` and `HelperServiceClass.h`. The Helper API is used when an application broadcasts a `sysNotifyHelperEvent` to all interested parties. The broadcaster of the notification and the notification clients (called **helpers**) use the Helper APIs to communicate with each other. The chapter discusses the following topics:

- Helper Data Structures
- Helper Constants

For more information on using the Helper API, see the section "Helper Notifications" on page 53 of the *Palm OS Programmer's Companion*, vol. I.

Helper Data Structures



New

HelperNotifyEnumerateListType

The `HelperNotifyEnumerateListType` provides the broadcaster of the helper notification with information about the services that the helper can provide. This structure is used as the `data` field of the `HelperNotifyEventType` structure when the action code is `kHelperNotifyActionCodeEnumerate`.

```
typedef struct HelperNotifyEnumerateListTypeTag
{
  struct HelperNotifyEnumerateListTypeTag
    *nextP;
  Char helperAppName[kHelperAppMaxNameSize];
  Char actionName[kHelperAppMaxActionNameSize];
  UInt32 helperAppID;
  UInt32 serviceClassID;
} HelperNotifyEnumerateListType;
```

Note that the helper allocates this structure and then adds it to the linked list of structures pointed to by `notifyDetailsP->data.enumerateP` in the `SysNotifyParamType` that is sent to the helper. The helper should allocate one structure per supported service.

Even though the helper allocates this structure, the helper is not responsible for freeing the structure. Instead, the application that broadcast the notification must free the structure.

Field Descriptions

`nextP`	A pointer to the next element in the list or `NULL` to signal the end of the list.
`helperAppName`	A null-terminated string containing the name of the helper application, suitable for display in the user interface. If more than one application can perform the same service, this name is displayed as one of the choices in a pop-up list.
`actionName`	A null-terminated string containing the name of the service that can be performed, suitable for display in the user interface. The action name should be short enough to display on a button.
`helperAppID`	The helper's creator ID or any other ID that uniquely identifies the helper.
`serviceClassID`	The ID of the service that the helper performs. See Helper Service Class IDs.

Compatibility Implemented only if 4.0 New Feature Set is present.



New HelperNotifyEventType

The `HelperNotifyEventType` structure contains all data associated with a helper notification (`sysNotifyHelperEvent`). A pointer to this structure is passed as the `notifyDetailsP` field in the `SysNotifyParamType` for that notification.

```
typedef struct HelperNotifyEventTypeTag {
  UInt16 version;
  HelperNotifyActionCodeType actionCode;
  union {
    struct HelperNotifyEnumerateListTypeTag
      *enumerateP;
    struct HelperNotifyValidateTypeTag
      *validateP;
    struct HelperNotifyExecuteTypeTag
      *executeP;
  } data;
} HelperNotifyEventType;
```

Field Descriptions

`version`	The version number for this structure. The current version is 1.
`actionCode`	The action that the helper application should perform. See Table 36.1.
`data`	Data specific to the action code. See Table 36.1.

The `HelperNotifyEventType` structure specifies which action is to be performed and contains data necessary for that action. All actions have some common data. Actions also have data specific to that action. The specific data uses a union that is part of the `HelperNotifyEventType` structure.

Table 36.1 Helper action codes

Action Code	data Type	Description
`kHelperNotifyActionCodeEnumerate`	`HelperNotifyEnumerateListType`	Send a list of available services.
`kHelperNotifyActionCodeValidate`	`HelperNotifyValidateType`	Validate the input data for the specified service.
`kHelperNotifyActionCodeExecute`	`HelperNotifyExecuteType`	Perform the specified service.

Compatibility Implemented only if 4.0 New Feature Set is present.



HelperNotifyExecuteType

The `HelperNotifyExecuteType` structure identifies the service to perform and contains the data necessary to perform that service. This structure is used as the `data` field of the `HelperNotifyEventType` structure when the action code is `kHelperNotifyActionCodeExecute`.

```
typedef struct HelperNotifyExecuteTypeTag {
  UInt32  serviceClassID;
  UInt32 helperAppID;
  Char *dataP;
  Char *displayedName;
  void *detailsP;
  Err err;
} HelperNotifyExecuteType;
```

Field Descriptions

`serviceClassID`	The ID of the service to be performed. See Helper Service Class IDs.
`helperAppID`	The unique ID of the helper; a value of 0 indicates that any available helper for the specified service class should perform the service.
`dataP`	A null-terminated string specific to this service, such as a phone number for the dial service or an email address for the email service. See Table 36.2. Multiple fields must be separated by semicolons (;).
`displayedName`	A null-terminated string containing an optional, human-readable description of the string in `dataP`. For example, if `dataP` contains a phone number, this field might contain the name of the person at that number.
`detailsP`	A pointer to a data structure containing extra information that this service requires. See Table 36.2. If the service does not require extra information, this field is `NULL`.
`err`	An error code that indicates if the service was performed successfully or not. If the service was successful, this field contains `errNone`, and the `handled` field in the notification data structure should be set to `true`.

The following table lists the Palm OS-defined values for the service class ID and for each service, shows what value `dataP` contains and what type of structure `detailsP` points to.

Table 36.2 HelperNotifyExecuteType values

Service Class ID	dataP Value	detailsP Value
`kHelperServiceClassIDVoiceDial`	The telephone number to dial	`NULL`
`kHelperServiceClassIDEMail`	The email address to which the message is to be sent	`HelperServiceEMailDetailsType`
`kHelperServiceClassIDSMS`	The SMS mailbox number to which the message is to be sent	`HelperServiceSMSDetailsType`
`kHelperServiceClassIDFax`	The fax number to which the fax is to be sent	`NULL`

Compatibility Implemented only if 4.0 New Feature Set is present.



New HelperNotifyValidateType

The `HelperNotifyValidateType` structure identifies a service that should be validated and the helper that should validate it. This structure is used as the `data` field of the `HelperNotifyEventType` structure when the action code is `kHelperNotifyActionCodeValidate`.

```
typedef struct HelperNotifyValidateTypeTag {
  UInt32 serviceClassID;
  UInt32 helperAppID;
} HelperNotifyValidateType;
```

Field Descriptions

`serviceClassID` The ID of the service to be validated. See Helper Service Class IDs.

`helperAppID` The creator ID of the helper application. 0 indicates that any available helper for the specified service should respond. If nonzero, only the helper with the matching creator ID should respond.

The helper returns `true` in the `handled` field of the `SysNotifyParamType` structure to indicate that the service can be performed or `false` to indicate that the service cannot be performed.

Compatibility Implemented only if 4.0 New Feature Set is present.



New HelperServiceEMailDetailsType

The `HelperServiceEMailDetailsType` structure provides additional data for the email service. It is used as the `detailsP` field in the `HelperNotifyExecuteType` when the service class ID is `kHelperServiceClassIDEMail`.

```
typedef struct _HelperServiceEMailDetailsType
{
  UInt16 version;
  Char *cc;
  Char *subject;
  Char *message;
} HelperServiceEMailDetailsType;
```

Field Descriptions

`version` The version number for this structure. The current version is 1.

`cc` A null-terminated string containing an email address that should be sent a carbon copy of the message. Multiple addresses are separated by a semi-colon (;). May be `NULL` if there are no email addresses to carbon copy.

`subject` A null-terminated string containing the subject line. May be `NULL`.

`message` Initial message body string or `NULL`.

Compatibility Implemented only if 4.0 New Feature Set is present.



New HelperServiceSMSDetailsType

The `HelperServiceSMSDetailsType` structure provides the SMS message to be sent. It is used as the `detailsP` field in the `HelperNotifyExecuteType` when the service class ID is `kHelperServiceClassIDSMS`.

```
typedef struct _HelperServiceSMSDetailsType {
  UInt16 version;
  Char *message;
} HelperServiceSMSDetailsType;
```

Field Descriptions

`version` The version number for this structure. The current version is 1.

`message` A null-terminated string containing the body of the message to be sent, or `NULL`.

Compatibility Implemented only if 4.0 New Feature Set is present.

Helper Constants

Helper Service Class IDs

The header file `HelperServiceClass.h` defines the constants listed in Table 36.3 as service class IDs. Third party developers may define their own service classes. To do so, you must register a 32-bit identifier with Palm, Inc. on this web site:

`http://www.palmos.com/dev/tech/palmos/creatorid/`

Alternatively, you can use a creator ID that you already own.

Table 36.3 Service class ID constants

Constant	Value	Description
`kHelperServiceClassIDVoiceDial`	'voic'	Dial a phone number for a voice telephone call.
`kHelperServiceClassIDEMail`	'mail'	Send an email message.
`kHelperServiceClassIDSMS`	'sms_'	Send an SMS message.
`kHelperServiceClassIDFax`	'fax_'	Send a fax.

Compatibility Implemented only if 4.0 New Feature Set is present.

37

Key Manager

This chapter provides reference material for the key manager. The key manager API is declared in the header file `KeyMgr.h`.

For more information on the key manager, see the section "Receiving User Input" in the *Palm OS Programmer's Companion*, vol. I.

Key Manager Functions

KeyCurrentState

Purpose Return bit field with bits set for each key that is currently depressed.

Prototype

```
UInt32 KeyCurrentState (void)
```

Parameters None.

Result Returns a `UInt32` with bits set for keys that are depressed. See `keyBitPower`, `keyBitPageUp`, `keyBitPageDown`, etc., in `KeyMgr.h`.

Comments Called by applications that need to poll the keys.

See Also `KeyRates`

KeyRates

Purpose Get or set the key repeat rates.

Prototype

```
Err KeyRates (Boolean set, UInt16* initDelayP,
UInt16* periodP, UInt16* doubleTapDelayP,
Boolean* queueAheadP)
```

Parameters

`set`	If `true`, settings are changed; if `false`, current settings are returned.
`initDelayP`	Initial delay in ticks for a auto-repeat event.
`periodP`	Auto-repeat rate specified as period in ticks.
`doubleTapDelayP`	Maximum double-tap delay, in ticks.
`queueAheadP`	If `true`, auto-repeating keeps queueing up key events if the queue has keys in it. If `false`, auto-repeat doesn't enqueue keys unless the queue is already empty.

Result Returns 0 if no error.

See Also `KeyCurrentState`

KeySetMask

Purpose Specify which keys generate `keyDownEvents`.

You can specify this either by using this function or by using the `poweredOnKeyMask` modifier.

Prototype

```
UInt32 KeySetMask (UInt32 keyMask)
```

Parameters

`keyMask`	Mask with bits set for those keys to generate `keyDownEvents` for.

Result Returns the old `key Mask`.

Compatibility Implemented only if 2.0 New Feature Set is present.

38

Locale Manager

This chapter describes the Locale Manager API as described in the header files `LocaleMgr.h`, `Localize.h`, and `PalmLocale.h`. It discusses the following topics:

- Locale Manager Data Types
- Locale Manager Constants
- Locale Manager Functions

For more information on the Locale Manager, see the chapter "Localized Applications" on page 313 of the *Palm OS Programmer's Companion*, vol. I.

Locale Manager Data Types

CountryType

The `CountryType` defines a country code. The Country Constants in `PalmLocale.h` define the possible values for `CountryType` variables.

```
typedef UInt8 CountryType;
```

Compatibility Prior to version 4.0, `CountryType` was an enum in `Preferences.h` that defined only 33 country codes. The Palm OS® 4.0 definition of `CountryType` is compatible with the previous definition.

LanguageType

The `LanguageType` defines a language code. The Language Constants in `PalmLocale.h` define the possible values for `LanguageType` variables.

```
typedef UInt8 LanguageType;
```

Compatibility The `LanguageType` definition was added in Palm OS 3.5. Prior to version 4.0, `LanguageType` was an enum in `Preferences.h` that defined only eight language codes. The Palm OS 4.0 definition of `LanguageType` is compatible with the previous definition.



LmLocaleType

The `LmLocaleType` struct defines the country and language used in a locale.

```
struct _LmLocaleType {
  UInt16          language;
  UInt16          country;
};
typedef struct _LmLocaleType LmLocaleType;
```

Field Descriptions

`language` One of the Language Constants. This value identifies the language spoken in the current locale.

`country` One of the Country Constants. This value identifies the locale's country, which helps to identify the language dialect. For example, a language of `lEnglish` specifies a different dialect if the country is `cUnitedKingdom` than if it is `cUnitedStates`.

Note that the `language` and `country` fields are type `UInt16` instead of `LanguageType` and `CountryType`.

Compatibility The `LmLocaleType` is defined only if 4.0 New Feature Set is present. It supersedes the `OmLocaleType` that was introduced with Palm OS 3.5. `LmLocaleType` is bit-compatible with `OmLocaleType`.

NumberFormatType

The `NumberFormatType` enum specifies how numbers are formatted. You can pass a `NumberFormatType` value to

LocGetNumberSeparators and receive the appropriate separator characters for thousands and decimals.

```
typedef enum {
  nfCommaPeriod,
  nfPeriodComma,
  nfSpaceComma,
  nfApostrophePeriod,
  nfApostropheComma
} NumberFormatType;
```

Value Descriptions

`nfCommaPeriod`	Uses a comma (,) as the thousands separator and a period (.) as the decimal separator.
`nfPeriodComma`	Uses a period as the thousands separator and a comma as the decimal separator.
`nfSpaceComma`	Uses a space ( ) as the thousands separator and a comma as the decimal separator.
`nfApostrophePeriod`	Uses an apostrophe (‘) as the thousands separator and a period as the decimal separator.
`nfApostropheComma`	Uses an apostrophe as the thousands separator and a comma as the decimal separator.

Locale Manager Constants

Character Encoding Constants

The `PalmLocale.h` file defines several character encoding constants that are used as values of `CharEncodingType` variables. The character encoding constants generally follow the format:

`charEncoding`*`Name`*

where *Name* is the name of the character encoding.

The following table shows examples of the character encoding constants. For a complete list, see the `PalmLocale.h` file.

Constant	Description
`charEncodingUnknown`	Unknown to this version of Palm OS®
`charEncodingAscii`	ISO 646-1991
`charEncodingISO8859_1`	ISO 8859 Part 1 (also known as ISO Latin 1). This encoding is commonly used for the Roman alphabet
`charEncodingPalmLatin`	Palm OS version of Microsoft Windows code page 1252. This encoding is identical to code page 1252 with Palm-specific characters added in the control range.
`charEncodingShiftJIS`	Encoding for 0208-1990 with single-byte Japanese Katakana. This encoding is commonly used for Japanese alphabets.
`charEncodingPalmSJIS`	Palm OS version of Microsoft Windows code page 932. This encoding is identical to code page 932, with Palm-specific characters added in the control range and with a Yen symbol instead of the Reverse Solidus at location 0x5c.
`charEncodingCP1252`	Microsoft Windows extensions to ISO 8859 Part 1
`charEncodingCP932`	Microsoft Windows extensions to Shift JIS
`charEncodingUTF8`	Eight-bit safe encoding for Unicode

Country Constants

The `PalmLocale.h` file defines several country constants that are used as values of `CountryType` variables. The country type constants have the following format:

`c`*CountryName*

where *CountryName* is the name of the country. There is one constant for each country identified in the ISO 3166 standard, which currently defines 239 countries.

The following table shows examples of the country type constants. For a complete list, see the `PalmLocale.h` file.

Constant	Description
`cAustralia`	Australia
`cAustria`	Austria
`cBelgium`	Belgium

Language Constants

The `PalmLocale.h` file defines several language constants that are used as values of `LanguageType` variables. The language type constants have the following format:

`l`*LanguageName*

where *LanguageName* is the name of the language. There is one constant for each language specified in the ISO 639 standard, which currently defines 137 languages.

The following table shows examples of the language type constants. For a complete list, see the `PalmLocale.h` file.

Constant	Description
`lEnglish`	English
`lFrench`	French
`lGerman`	German

Locale Manager Size Constants

You can use the Locale Manager size constants to determine the size of strings to allocate for some of the locale settings.

NOTE: The variables in the table below do not count the terminating null character. Therefore, you need to allocate a string of size `kMaxCountryNameLen+1` to hold a country name, for example.

Constant	Value	Description
`kMaxCountryNameLen`	19	The maximum length of a country name string.
`kMaxCurrencyNameLen`	19	The maximum length of a currency name string.
`kMaxCurrencySymbolLen`	5	The maximum length of a currency symbol string.

Locale Manager Functions



New LmGetLocaleSetting

Purpose Return the requested setting for a given locale.

Prototype

```
Err LmGetLocaleSetting (UInt16 iLocaleIndex,
LmLocaleSettingChoice iChoice, void *oValue,
UInt16 iValueSize)
```

Parameters

-> `iLocaleIndex` Index of the locale whose settings you want to retrieve.

-> `iChoice` The setting you want to retrieve. This is a constant in the form `lmChoice`*SettingName*. See Table 38.1 for a list of possible values.

<- `oValue`	The value of the `iChoice` setting. The size of this buffer depends on the value of `iChoice`, as shown in Table 38.1.
-> `iValueSize`	The size of the `oValue` buffer.

Result Returns one of the following values:

`errNone`	Success.
`lmErrBadLocaleIndex`	`iLocaleIndex` is out of range.
`lmErrSettingDataOverflow`	The `oValue` buffer is too small to hold the setting's value.
`lmErrBadLocaleSettingChoice`	The `iChoice` parameter contains an unknown or unsupported value.

Comments This function accesses the private locale system resource and returns the requested information in the `oValue` parameter. The size and type of the `oValue` parameter depend on which setting you want to retrieve. Table 38.1 lists and describes the possible settings and the type of data returned in `oValue` for each setting. For fixed-size values, make sure that `oValue` is no larger than the returned value.

This function returns the default settings for the locale. Users can override many of the locale settings using the Preferences application. Applications should always honor the user's preferences rather than the locale defaults. For this reason, it's recommended that if a corresponding system preference is available, you should check it instead (using `PrefGetPreference`). Use `LmGetLocaleSetting` only if you want to retrieve values that the user cannot override (such as the country name or currency symbol) or if you want to retrieve information about a locale other than the current locale.

Table 38.1 LmGetLocaleSetting choices and sizes

lmChoice...	oValue Data Type	Description
`CountryName`	String buffer of size `kMaxCountryNameLen+1` bytes	The name of the locale's country.
`CurrencyName`	String buffer of size `kMaxCurrencyNameLen+1` bytes	The name of the currency used in this locale.
`CurrencySymbol`	String buffer of size `kMaxCurrencySymbolLen+1` bytes	The symbol used to denote monetary values in this locale.
`CurrencyDecimalPlaces`	`UInt16`	The number of decimal places that monetary values are typically given.
`DateFormat`	`DateFormatType`	The short date format used in this locale. For example: 95/12/31
`Locale`	`LmLocaleType`	A structure containing the locale's language and country codes.
`LongDateFormat`	`DateFormatType`	The long date format used in this locale. For example: 31 Dec 1995
`MeasurementSystem`	MeasurementSystemType	The measurement system (metric system or English system) used in this locale.
`NumberFormat`	`NumberFormatType`	The format used for numbers, with regards to the thousands separator and the decimal point, in this locale.

Table 38.1 LmGetLocaleSetting choices and sizes *(continued)*

lmChoice...	oValue Data Type	Description
`TimeFormat`	`TimeFormatType`	The format used for time values in this locale.
`TimeZone`	`Int16`	The locale's default time zone given as the number of minutes east of Greenwich Mean Time (GMT).
`UniqueCurrency Symbol`	String buffer of size `kMaxCurrencySymbolLen+1` bytes	A unique symbol for monetary values. For example, the symbol `$` is used both for US dollars and Portuguese escudos. The unique currency symbol for US dollars is `US$`.
`WeekStartDay`	`UInt16`	The first day of the week (Sunday or Monday) in this locale. Days of the week are numbered from 0 to 6 starting with Sunday = 0.

Compatibility Implemented only if 4.0 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `LmGlueGetLocaleSetting`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `LmGetNumLocales`, `LmLocaleToIndex`



New LmGetNumLocales

Purpose Return the number of known locales.

Prototype

```
UInt16 LmGetNumLocales (void)
```

Parameters None.

Result Returns the number of locales that the locale system resource defines.

Comments Use this function to obtain the range of possible values that you can pass as an index to `LmGetLocaleSetting`. If `LmGetNumLocales` returns 3, then `LmGetLocaleSetting` accepts indexes in the range of 0 to 2.

This function returns only the number of locales for which the ROM has locale information. It does not return the number of locales that could possibly be defined. For example, the system resource currently contains no locale whose language is `lHebrew` and country is `cIsrael`, even though that is a valid locale.

Compatibility Implemented only if 4.0 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `LmGlueGetNumLocales`. For more information, see Chapter 76, "PalmOSGlue Library."

New LmLocaleToIndex

Purpose

Convert an `LmLocaleType` to an index.

Prototype

```
Err LmLocaleToIndex (const LmLocaleType *iLocale,
UInt16 *oLocaleIndex)
```

Parameters

-> `iLocale` The locale to convert.

<- `oLocaleIndex` The index of `iLocale` upon return.

Result

Returns `errNone` upon success or `lmErrUnknownLocale` if the locale could not be found.

Comments

You can use this function to obtain a valid index to pass to `LmGetLocaleSetting`. For example, you might use the Overlay Manager routine `OmGetSystemLocale` to return the locale used on the current system and then pass that locale to this function to obtain its index.

```
LmLocaleType locale;
Char oValue[kMaxCurrencySymbolLen+1];
UInt16 index;

OmGetSystemLocale(&locale);
LmLocaleToIndex(&locale, &index);
LmGetLocaleSetting(index,
  lmChoiceCurrencySymbol, oValue,
  sizeof(oValue));
```

The `LmLocaleType` that is passed in `iLocale` can use the constants `lmAnyCountry` or `lmAnyLanguage` as wildcards. For example, if the country is `lmAnyCountry`, `LmLocaleToIndex` returns the index of the first locale that matches the language.

Compatibility

Implemented only if 4.0 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `LmGlueLocaleToIndex`. For more information, see Chapter 76, "PalmOSGlue Library."

39

Memory Manager

This chapter provides reference information for the memory manager. The memory manager API is declared in the header file `MemoryMgr.h`.

For more information on the memory manager, see the chapter "Memory" in the *Palm OS Programmer's Companion*, vol. I.

Memory Manager Functions

MemCardInfo

Purpose Return information about a memory card.

Prototype

```
Err MemCardInfo (UInt16 cardNo, Char* cardNameP,
Char* manufNameP, UInt16* versionP,
UInt32* crDateP, UInt32* romSizeP,
UInt32* ramSizeP, UInt32* freeBytesP)
```

Parameters

`cardNo`	Card number.
`cardNameP`	Pointer to character array (32 bytes), or 0.
`manufNameP`	Pointer to character array (32 bytes), or 0.
`versionP`	Pointer to version variable, or 0.
`crDateP`	Pointer to creation date variable, or 0.
`romSizeP`	Pointer to ROM size variable, or 0.
`ramSizeP`	Pointer to RAM size variable, or 0.
`freeBytesP`	Pointer to free byte-count variable, or 0.

Result Returns 0 if no error.

Comments Pass 0 for those variables that you don't want returned.

MemCmp

Purpose Compare two blocks of memory.

NOTE: Blocks are compared as unsigned bytes.

Prototype

```
Int16 MemCmp (const void* s1, const void* s2,
Int32 numBytes)
```

Parameters

`s1, s2`	Pointers to block of memory.
`numBytes`	Number of bytes to compare.

Result Zero if they match, non-zero if not:

+ if `s1` > `s2`

- if `s1` < `s2`

Compatibility Implemented only if 2.0 New Feature Set is present.

`MemCmp` can be used to test the equality of blocks in memory on all versions that support `MemCmp`; however, testing the sort ordering of blocks in memory works reliably only on Palm OS® versions 3.5 and higher. On versions earlier than 3.2, `MemCmp` always returns a positive value if the blocks are unequal. On versions 3.2 and 3.3, `MemCmp` reliably returns positive to indicate `s1` > `s2` and negative to indicate `s1` < `s2` **only** if the characters that differ are less than 128 apart. If the difference is greater than that, `MemCmp` may return positive when it should return negative and vice versa.

MemDebugMode

Purpose Return the current debugging mode of the memory manager.

Prototype

```
UInt16 MemDebugMode (void)
```

Parameters No parameters.

Result Returns debug flags as described for `MemSetDebugMode`.

MemHandleCardNo

Purpose Return the card number a chunk resides in.

Prototype
```
UInt16 MemHandleCardNo (MemHandle h)
```

Parameters -> `h` Chunk handle.

Result Returns the card number.

Comments Call this routine to retrieve the card number (0 or 1) a movable chunk resides on.

See Also `MemPtrCardNo`

MemHandleDataStorage

Purpose Return `true` if the given handle is part of a data storage heap. If not, it's a handle in the dynamic heap.

Prototype
```
Boolean MemHandleDataStorage (MemHandle h)
```

Parameters -> `h` Chunk handle.

Result Returns `true` if the handle is part of a data storage heap.

Comments Called by Fields package routines to determine if they need to worry about data storage write-protection when editing a text field.

See Also `MemPtrDataStorage`

MemHandleFree

Purpose Dispose of a movable chunk.

Prototype
```
Err MemHandleFree (MemHandle h)
```

Parameters -> `h` Chunk handle.

Result Returns 0 if no error, or `memErrInvalidParam` if an error occurs.

Comments Call this routine to dispose of a movable chunk.

See Also `MemHandleNew`

MemHandleHeapID

Purpose Return the heap ID of a chunk.

Prototype
```
UInt16 MemHandleHeapID (MemHandle h)
```

Parameters -> `h` Chunk handle.

Result Returns the heap ID of a chunk.

Comments Call this routine to get the heap ID of the heap a chunk resides in.

See Also `MemPtrHeapID`

MemHandleLock

Purpose Lock a chunk and obtain a pointer to the chunk's data.

Prototype
```
MemPtr MemHandleLock (MemHandle h)
```

Parameters -> `h` Chunk handle.

Result Returns a pointer to the chunk.

Comments Call this routine to lock a chunk and obtain a pointer to the chunk.

`MemHandleLock` and `MemHandleUnlock` should be used in pairs.

See Also `MemHandleNew`, `MemHandleUnlock`

MemHandleNew

Purpose Allocate a new movable chunk in the dynamic heap and returns a handle to it.

Prototype
```
MemHandle MemHandleNew (UInt32 size)
```

Parameters

-> `size`	The desired size of the chunk.

Result Returns a handle to the new chunk, or 0 if unsuccessful.

Comments Use this call to allocate dynamic memory. Before you can write data to the memory chunk that `MemHandleNew` allocates, you must call `MemHandleLock` to lock the chunk and get a pointer to it.

See Also `MemPtrFree`, `MemPtrNew`, `MemHandleFree`, `MemHandleLock`

MemHandleResize

Purpose Resize a chunk.

Prototype
```
Err MemHandleResize (MemHandle h, UInt32 newSize)
```

Parameters

-> `h`	Chunk handle.
-> `newSize`	The new desired size.

Result

0	No error.
`memErrInvalidParam`	Invalid parameter passed.
`memErrNotEnoughSpace`	Not enough free space in heap to grow chunk.

`memErrChunkLocked`
Can't grow chunk because it's locked.

Comments Call this routine to resize a chunk. This routine is always successful when shrinking the size of a chunk, even if the chunk is locked. When growing a chunk, it first attempts to grab free space immediately following the chunk so that the chunk does not have to move. If the chunk has to move to another free area of the heap to grow, it must be movable and have a lock count of 0.

On devices running version 2.0 or earlier of Palm OS, the `MemHandleResize` function tries to resize the chunk only within the same heap, whereas `DmResizeRecord` will look for space in other data heaps if it can't find enough space in the original heap.

See Also `MemHandleNew`, `MemHandleSize`

MemHandleSetOwner

Purpose Set the owner ID of a chunk.

Prototype
```
Err MemHandleSetOwner (MemHandle h, UInt16 owner)
```

Parameters

-> `h` Chunk handle.

-> `owner` New owner ID of the chunk. Specify 0 to set the owner to the operating system.

Result Returns 0 if no error, or `memErrInvalidParam` if an error occurs.

Comments When you allocate a parameter block to pass to `SysUIAppSwitch` or `SysAppLaunch`, you must call `MemPtrSetOwner` to grant ownership of the parameter block chunk to the OS (your application is originally set as the owner). If the parameter block structure references any chunks by handle, you also must call `MemHandleSetOwner` to grant ownership of those blocks to the OS. If you don't change the ownership of these chunks, they will get freed before the application you're launching has a chance to use them.

MemHandleSize

Purpose Return the requested size of a chunk.

Prototype
```
UInt32 MemHandleSize (MemHandle h)
```

Parameters

-> `h`	Chunk handle.

Result Returns the requested size of the chunk.

Comments Call this routine to get the size originally requested for a chunk.

See Also `MemHandleResize`

MemHandleToLocalID

Purpose Convert a handle into a local chunk ID which is card relative.

Prototype
```
LocalID MemHandleToLocalID (MemHandle h)
```

Parameters

-> `h`	Chunk handle.

Result Returns local ID, or NULL (0) if unsuccessful.

Comments Call this routine to convert a chunk handle to a local ID.

See Also `MemLocalIDToGlobal`, `MemLocalIDToLockedPtr`

MemHandleUnlock

Purpose Unlock a chunk given a chunk handle.

Prototype
```
Err MemHandleUnlock (MemHandle h)
```

Parameters

-> `h`	The chunk handle.

Result

0	No error.

`memErrInvalidParam`
Invalid parameter passed.

Comments Call this routine to decrement the lock count for a chunk.

`MemHandleLock` and `MemHandleUnlock` should be used in pairs.

See Also `MemHandleLock`

MemHeapCheck

Purpose Check validity of a given heap.

Prototype
```
Err MemHeapCheck (UInt16 heapID)
```

Parameters `heapID` ID of heap to check.

Result Returns 0 if no error.

See Also `MemDebugMode`, `MemSetDebugMode`

MemHeapCompact

Purpose Compact a heap.

Prototype
```
Err MemHeapCompact (UInt16 heapID)
```

Parameters -> `heapID` ID of the heap to compact.

Result Always returns 0.

Comments Most applications never need to call this function explicitly. The system software calls this function at various times; for example, during memory allocation (if sufficient free space is not available) and during system reboot.

Call this routine to compact a heap and merge all free space. This routine attempts to move all movable chunks to the start of the heap and merge all free space in the center of the heap.

MemHeapDynamic

Purpose Return `true` if the given heap is a dynamic heap.

Prototype
```
Boolean MemHeapDynamic (UInt16 heapID)
```

Parameters `heapID` ID of the heap to be tested.

Result Returns `true` if dynamic, `false` if not.

Comments Dynamic heaps are used for volatile storage, application stacks, globals, and dynamically allocated memory.

NOTE: In Palm OS 3.5, the dynamic heap is sized based on the amount of memory available, and is generally larger than before.

See Also `MemNumHeaps`, `MemHeapID`

MemHeapFlags

Purpose Return the heap flags for a heap.

Prototype
```
UInt16 MemHeapFlags (UInt16 heapID)
```

Parameters -> `heapID` ID of heap.

Result Returns the heap flags.

Comments Call this routine to retrieve the heap flags for a heap. The flags can be examined to determine if the heap is ROM based or not. ROM-based heaps have the `memHeapFlagReadOnly` bit set (the `memHeapFlagReadOnly` mask has a value of `0x0001`).

See Also `MemNumHeaps`, `MemHeapID`

MemHeapFreeBytes

Purpose Return the total number of free bytes in a heap and the size of the largest free chunk in the heap.

Prototype
```
Err MemHeapFreeBytes (UInt16 heapID,
UInt32* freeP, UInt32* maxP)
```

Parameters

-> `heapID`	ID of heap.
<-> `freeP`	Pointer to a variable of type `UInt32` for free bytes.
<-> `maxP`	Pointer to a variable of type `UInt32` for max free chunk size. Do not pass `NULL` for this argument.

Result Always returns 0.

Comments This routine doesn't compact the heap but may be used to determine in advance whether an allocation request will succeed. Before allocating memory, call this function and test the value returned in `maxP` to determine whether enough free space to fulfill your allocation request exists. If not, you may make more space available by calling the `MemHeapCompact` function. Note that both `MemPtrNew` and `MemHandleNew` automatically compact the heap if necessary.

See Also `MemHeapSize`, `MemHeapID`, `MemHeapCompact`

MemHeapID

Purpose Return the heap ID for a heap, given its index and the card number.

Prototype
```
UInt16 MemHeapID (UInt16 cardNo, UInt16 heapIndex)
```

Parameters

-> `cardNo`	The card number, either 0 or 1.
-> `heapIndex`	The heap index, anywhere from 0 to `MemNumHeaps` - 1.

Result Returns the heap ID.

Comments Call this routine to retrieve the heap ID of a heap, given the heap index and the card number. A heap ID must be used to obtain information on a heap such as its size, free bytes, etc., and is also passed to any routines which manipulate heaps.

See Also MemNumHeaps

MemHeapScramble

Purpose Scramble the specified heap.

Prototype
```
Err MemHeapScramble (UInt16 heapID)
```

Parameters `heapID` ID of heap to scramble.

Comments The system attempts to move each movable chunk.

Useful for debugging.

Result Always returns 0.

See Also MemDebugMode, MemSetDebugMode

MemHeapSize

Purpose Return the total size of a heap including the heap header.

Prototype
```
UInt32 MemHeapSize (UInt16 heapID)
```

Parameters -> `heapID` ID of heap.

Result Returns the total size of the heap.

See Also MemHeapFreeBytes, MemHeapID

MemLocalIDKind

Purpose Return whether or not a local ID references a handle or a pointer.

Prototype
```
LocalIDKind MemLocalIDKind (LocalID local)
```

Parameters

-> `local`	Local ID to query

Result Returns `LocalIDKind`, or a `memIDHandle` or `memIDPtr` (see `MemoryMgr.h`).

Comments This routine determines if the given local ID is to a nonmovable (`memIDPtr`) or movable (`memIDHandle`) chunk.

MemLocalIDToGlobal

Purpose Convert a local ID, which is card relative, into a global pointer in the designated card.

Prototype
```
MemPtr MemLocalIDToGlobal (LocalID local,
UInt16 cardNo)
```

Parameters

-> `local`	The local ID to convert.
-> `cardNo`	Memory card the chunk resides in.

Result Returns pointer or handle to chunk.

See Also `MemLocalIDKind`, `MemLocalIDToLockedPtr`

MemLocalIDToLockedPtr

Purpose Return a pointer to a chunk given its local ID and card number.

If the local ID references a movable chunk handle, this routine automatically locks the chunk before returning.

Prototype

```
MemPtr MemLocalIDToLockedPtr (LocalID local,
UInt16 cardNo)
```

Parameters

`local`	Local chunk ID.
`cardNo`	Card number.

Result Returns pointer to chunk, or 0 if an error occurs.

See Also `MemLocalIDToGlobal`, `MemLocalIDToPtr`, `MemLocalIDKind`, `MemPtrToLocalID`, `MemHandleToLocalID`

MemLocalIDToPtr

Purpose Return pointer to chunk, given the local ID and card number.

Prototype

```
MemPtr MemLocalIDToPtr (LocalID local,
UInt16 cardNo)
```

Parameters

-> `local`	Local ID to query.
-> `cardNo`	Card number the chunk resides in.

Result Returns a pointer to the chunk, or 0 if error.

Comments If the local ID references a movable chunk and that chunk is **not** locked, this function returns 0 to indicate an error.

See Also `MemLocalIDToGlobal`, `MemLocalIDToLockedPtr`

MemMove

Purpose Move a range of memory to another range in the dynamic heap.

Prototype
```
Err MemMove (void* dstP, const void* sP,
Int32 numBytes)
```

Parameters

`dstP`	Pointer to destination.
`sP`	Pointer to source.
`numBytes`	Number of bytes to move.

Result Always returns 0.

Comments Handles overlapping ranges.

For operations where the destination is in a data heap, see `DmSet`, `DmWrite`, and related functions.

MemNumCards

Purpose Return the number of memory card slots in the system. Not all slots need to be populated.

Prototype
```
UInt16 MemNumCards (void)
```

Parameters None.

Result Returns number of slots in the system.

MemNumHeaps

Purpose Return the number of heaps available on a particular card.

Prototype
```
UInt16 MemNumHeaps (UInt16 cardNo)
```

Parameters -> `cardNo` The card number; either 0 or 1.

Result Number of heaps available, including ROM- and RAM-based heaps.

Comments Call this routine to retrieve the total number of heaps on a memory card. The information can be obtained by calling `MemHeapSize`, `MemHeapFreeBytes`, and `MemHeapFlags` on each heap using its heap ID. The heap ID is obtained by calling `MemHeapID` with the card number and the heap index, which can be any value from 0 to `MemNumHeaps`.

MemNumRAMHeaps

Purpose Return the number of RAM heaps in the given card.

Prototype
```
UInt16 MemNumRAMHeaps (UInt16 cardNo)
```

Parameters `cardNo` The card number.

Result Returns the number of RAM heaps.

See Also `MemNumCards`

MemPtrCardNo

Purpose Return the card number (0 or 1) a nonmovable chunk resides on.

Prototype
```
UInt16 MemPtrCardNo (MemPtr p)
```

Parameters -> `p` Pointer to the chunk.

Result Returns the card number.

See Also `MemHandleCardNo`

MemPtrDataStorage

Purpose Return `true` if the given pointer is part of a data storage heap; if not, it is a pointer in the dynamic heap.

Prototype
```
Boolean MemPtrDataStorage (MemPtr p)
```

Parameters `p` Pointer to a chunk.

Result Returns `true` if the chunk is part of a data storage heap.

Comments Called by Fields package to determine if it needs to worry about data storage write-protection when editing a text field.

See Also `MemHeapDynamic`

MemPtrFree

Purpose Macro to dispose of a chunk.

Prototype
```
Err MemPtrFree (MemPtr p)
```

Parameters

-> `p`	Pointer to a chunk.

Result

0	If no error or `memErrInvalidParam` (invalid parameter).

Comments Call this routine to dispose of a nonmovable chunk.

MemPtrHeapID

Purpose Return the heap ID of a chunk.

Prototype
```
UInt16 MemPtrHeapID (MemPtr p)
```

Parameters

-> `p`	Pointer to the chunk.

Result Returns the heap ID of a chunk.

Comments Call this routine to get the heap ID of the heap a chunk resides in.

MemPtrNew

Purpose Allocate a new nonmovable chunk in the dynamic heap.

Prototype
```
MemPtr MemPtrNew (UInt32 size)
```

Parameters

-> `size`	The desired size of the chunk.

Result Returns pointer to the new chunk, or 0 if unsuccessful.

Comments This routine allocates a nonmovable chunk in the dynamic heap and returns a pointer to the chunk. Applications can use it when allocating dynamic memory.

In Palm OS 3.5, the dynamic heap is sized based on the amount of memory available, and is generally larger than before.

NOTE: You cannot allocate a zero-size reference block.

MemPtrRecoverHandle

Purpose Recover the handle of a movable chunk, given a pointer to its data.

Prototype
```
MemHandle MemPtrRecoverHandle (MemPtr p)
```

Parameters

-> `p`	Pointer to the chunk.

Result Returns the handle of the chunk, or 0 if unsuccessful.

Comments Don't call this function for pointers in ROM or nonmovable data chunks.

MemPtrResize

Purpose Resize a chunk.

Prototype
```
Err MemPtrResize (MemPtr p, UInt32 newSize)
```

Parameters

-> `p`	Pointer to the chunk.
-> `newSize`	The new desired size.

Result Returns 0 if no error, or `memErrNotEnoughSpace` `memErrInvalidParam`, or `memErrChunkLocked` if an error occurs.

Comments Call this routine to resize a locked chunk. This routine is always successful when shrinking the size of a chunk. When growing a chunk, it attempts to use free space immediately following the chunk.

See Also `MemPtrSize`, `MemHandleResize`

MemPtrSetOwner

Purpose Set the owner ID of a chunk.

Prototype
```
Err MemPtrSetOwner (MemPtr p, UInt16 owner)
```

Parameters

-> `p`	Pointer to the chunk.
-> `owner`	New owner ID of the chunk. Specify 0 to set the owner to the operating system.

Result Returns 0 if no error, or `memErrInvalidParam` if an error occurs.

Comments When you allocate a parameter block to pass to `SysUIAppSwitch` or `SysAppLaunch`, you must call `MemPtrSetOwner` or `MemHandleSetOwner` to grant ownership of the parameter block chunk, and any other chunks referenced in it, to the OS (your application is originally set as the owner). If you don't change the ownership of the parameter block, it will get freed before the application you're launching has a chance to use it.

MemPtrSize

Purpose Return the size of a chunk.

Prototype
```
UInt32 MemPtrSize (MemPtr p)
```

Parameters

-> `p`	Pointer to the chunk.

Result The requested size of the chunk.

Comments Call this routine to get the original requested size of a chunk.

MemPtrToLocalID

Purpose Convert a pointer into a card-relative local chunk ID.

Prototype
```
LocalID MemPtrToLocalID (MemPtr p)
```

Parameters -> `p` Pointer to a chunk.

Result Returns the local ID of the chunk.

Comments Call this routine to convert a chunk pointer to a local ID.

See Also `MemLocalIDToPtr`

MemPtrUnlock

Purpose Unlock a chunk, given a pointer to the chunk.

Prototype
```
Err MemPtrUnlock (MemPtr p)
```

Parameters `p` Pointer to a chunk.

Result 0 if no error, or `memErrInvalidParam` if an error occurs.

Comments A chunk must **not** be unlocked more times than it was locked.

See Also `MemHandleLock`

MemSet

Purpose Set a memory range in a dynamic heap to a specific value.

Prototype
```
Err MemSet (void* dstP, Int32 numBytes,
UInt8 value)
```

Parameters

`dstP`	Pointer to the destination.
`numBytes`	Number of bytes to set.
`value`	Value to set.

Result Always returns 0.

Comments For operations where the destination is in a data heap, see `DmSet`, `DmWrite`, and related functions.

MemSetDebugMode

Purpose Set the debugging mode of the memory manager.

Prototype

```
Err MemSetDebugMode (UInt16 flags)
```

Parameters

`flags`	Debug flags.

Comments Use the logical OR operator (|) to provide any combination of one, more, or none of the following flags:

```
memDebugModeCheckOnChange
memDebugModeCheckOnAll
memDebugModeScrambleOnChange
memDebugModeScrambleOnAll
memDebugModeFillFree
memDebugModeAllHeaps
memDebugModeRecordMinDynHeapFree
```

Result Returns 0 if no error, or -1 if an error occurs.

MemStoreInfo

Purpose Return information on either the RAM store or the ROM store for a memory card.

Prototype

```
Err MemStoreInfo (UInt16 cardNo,
UInt16 storeNumber, UInt16* versionP,
UInt16* flagsP, Char* nameP, UInt32* crDateP,
UInt32* bckUpDateP, UInt32* heapListOffsetP,
UInt32* initCodeOffset1P,
UInt32* initCodeOffset2P, LocalID* databaseDirIDP)
```

Parameters

-> `cardNo`	Card number, either 0 or 1.
-> `storeNumber`	Store number; 0 for ROM, 1 for RAM.
<-> `versionP`	Pointer to version variable, or 0.
<-> `flagsP`	Pointer to flags variable, or 0.
<-> `nameP`	Pointer to character array (32 bytes), or 0.
<-> `crDateP`	Pointer to creation date variable, or 0.
<-> `bckUpDateP`	Pointer to backup date variable, or 0.
<-> `heapListOffsetP`	Pointer to `heapListOffset` variable, or 0.
<-> `initCodeOffset1P`	Pointer to `initCodeOffset1` variable, or 0.
<-> `initCodeOffset2P`	Pointer to `initCodeOffset2` variable, or 0.
<-> `databaseDirIDP`	Pointer to database directory chunk ID variable, or 0.

Result Returns 0 if no error, or `memErrCardNotPresent`, `memErrRAMOnlyCard`, or `memErrInvalidStoreHeader` if an error occurs.

Comments Call this routine to retrieve any or all information on either the RAM store or the ROM store for a card. Pass 0 for variables that you don't wish returned.

40

Notification Manager

This chapter describes the Notification Manager API as declared in the header file `NotifyMgr.h`. It discusses the following topics:

- Notification Constants
- Notification Functions
- Application-Defined Functions

The chapter "Notifications" in this book lists the possible notifications and describes the data sent with each. Also see the section "Notifications" on page 46 in the "Application Startup and Stop" chapter of the *Palm OS Programmer's Companion*, vol. I for a description of how to use notifications.

Notification Constants

Miscellaneous Constants

This following miscellaneous constants are used in the Notification Manager. For other notification constants, see the "Notifications" chapter in this book.

Constant	Value	Description
`sysNotifyBroadcasterCode`	`'psys'`	The value passed as the creator ID of the broadcaster for notifications broadcast by the system.
`sysNotifyDefaultQueueSize`	30	The maximum number of nested broadcasts allowed.
`sysNotifyNoDatabaseID`	0xFFFFFFFF	The database local ID used by the system when it registers for notifications.

Constant	Value	Description
`sysNotifyNormalPriority`	0	Typical priority value used when registering for notifications.
`sysNotifyVersionNum`	1	Current Notification Manager version. This number is stored in the system feature `sysFtrNumNotifyMgrVersion`.

Notification Functions

SysNotifyBroadcast

Purpose Synchronously send a notification to all applications registered for it.

Prototype

```
Err SysNotifyBroadcast
(SysNotifyParamType *notify)
```

Parameters

<-> `notify` Identifies the notification to be broadcast. See `SysNotifyParamType`.

Result Returns one of the following error codes:

`errNone` No error.

`sysNotifyErrBroadcastBusy`
The broadcast stack limit has already been reached.

`sysErrParamErr`
The background thread is broadcasting the notification and the `notify` parameter is `NULL`.

`sysNotifyErrNoStackSpace`
There is not enough space on the stack for the notification.

Comments

When you call this function, the notification you specify is broadcast to all applications, shared libraries, and other code resources that have registered to receive that notification. The broadcast is performed synchronously, meaning that the system broadcasts the notification immediately and waits for each notification client to perform its notification handler and return before the `SysNotifyBroadcast` call returns. This notification occurs in priority order.

The system allows nested notifications; that is, the recipient of a notification might broadcast a new notification, whose recipient might broadcast another new notification and so on. The constant `sysNotifyDefaultQueueSize` specifies how many levels of nested notification are allowed. If you reach this limit, the error `sysNotifyErrBroadcastBusy` is returned and your notification is not broadcast. To avoid reaching the limit, use `SysNotifyBroadcastDeferred` instead of `SysNotifyBroadcast` in your notification handlers. This ensures that the notification will not be broadcast until the top of the event loop.

WARNING! Do not call `SysNotifyBroadcast` from code that might be called from a background task (such as a trap patch) with the memory semaphore reserved. Use `SysNotifyBroadcastDeferred` instead.

Compatibility

Implemented only if Notification Feature Set is present.

SysNotifyBroadcastDeferred

Purpose

Enqueue a notification for later broadcast.

Prototype

```
Err SysNotifyBroadcastDeferred
(SysNotifyParamType *notify, Int16 paramSize)
```

Parameters

<-> `notify` The notification to enqueue. See `SysNotifyParamType`.

-> `paramSize`	Size of the data pointed to by the field `notify->notifyDetailsP`.

Result Returns one of the following error codes:

`errNone`	No error.
`memErrNotEnoughSpace`	There is not enough memory to allocate a new notification entry in the queue.
`sysErrParamErr`	`paramSize` is a negative number.
`sysNotifyErrQueueFull`	The queue has reached its maximum number of entries.

Comments This function is similar to `SysNotifyBroadcast` except that the broadcast does not take place until the top of the event loop (specifically, the next time `EvtGetEvent` is called). The system copies the `notify` structure to a new memory chunk, which is disposed of upon completion of the broadcast. (The `paramSize` value is used when copying the `notifyDetailsP` portion of the `notify` structure.)

Compatibility Implemented only if Notification Feature Set is present.



New SysNotifyBroadcastFromInterrupt

Purpose Allows interrupt handlers to enqueue a notification for later broadcast.

Prototype

```
Err SysNotifyBroadcastFromInterrupt
(UInt32 notifyType, UInt32 broadcaster,
void *notifyDetailsP)
```

Parameters

-> `notifyType` The type of event that occurred. See the chapter Notifications for a complete list of the notifications that Palm OS® broadcasts.

-> `broadcaster` The creator ID of the device or application that broadcast the notification.

-> `notifyDetailsP`
Pointer to data specific to this notification. See the Notifications chapter for the specific instances where this parameter is used.

Result

Returns one of the following error codes:

`errNone` No error.

`sysNotifyErrQueueFull`
The queue has reached its maximum number of entries.

Comments

Like `SysNotifyBroadcastDeferred`, this function enqueues a notification to be broadcast at the top of the event loop (specifically, the next time `EvtGetEvent` is called). It differs from `SysNotifyBroadcastDeferred` in that it is interrupt-safe and intended to be called from interrupt handlers.

This function is intended to be used by device drivers and other low-level software to generate a notification about a hardware change. For example, the Expansion Manager uses `SysNotifyBroadcastFromInterrupt` to broadcast `sysNotifyCardInsertedEvent` and `sysNotifyCardRemovedEvent` when a card is inserted into or removed from the expansion slot. `SysNotifyBroadcastFromInterrupt` is not intended to be used by general third party applications. Patching `SysNotifyBroadcastFromInterrupt` will cause the system to hang.

Compatibility

Implemented only if 4.0 New Feature Set is present.

SysNotifyRegister

Purpose Register to receive a notification.

Prototype

```
Err SysNotifyRegister (UInt16 cardNo,
LocalID dbID, UInt32 notifyType,
SysNotifyProcPtr callbackP, Int8 priority,
void *userDataP)
```

Parameters

-> `cardNo`	Number of the storage card on which the application or code resource resides.
-> `dbID`	Local ID of the application or code resource.
-> `notifyType`	The notification that the application wants to receive. See the chapter Notifications.
-> `callbackP`	Set to `NULL` to receive the notification as an application launch code. If your code does not have a `PilotMain` function (for example, if it is a shared library), pass a pointer to a function that should be called when the notification is broadcast. See `SysNotifyProcPtr`.
-> `priority`	The priority with which the application should receive the event. Most applications and other code resources should always use `sysNotifyNormalPriority`. In rare circumstances, you may need to ensure that your code is notified toward the beginning or toward the end of the notification sequence. If so, be sure to leave some space so that your code won't collide with the system's handling of notifications or with another application's handling of notifications. In general, Palm™ recommends using a value whose least significant bits are 0 (such as 32, 64, 96, and so on). The smaller the priority, the earlier your code is notified.
-> `userDataP`	Caller-defined data to pass to the notification handler.

Result Returns one of the following error codes:

`errNone`	No error.
`sysErrParamErr`	The database ID is `NULL`.
`sysNotifyErrDuplicateEntry`	This application is already registered to receive this notification.

Comments Call this function when your code should receive a notification that a specific event has occurred or is about to occur. See the Notifications chapter for a list of the possible notifications. Once you register for a notification, you remain registered to receive it until a system reset occurs or until you explicitly unregister using `SysNotifyUnregister`.

If you're writing an application, you should pass `NULL` as the `callbackP` parameter. In this case, the system notifies your application by sending it the `sysAppLaunchCmdNotify` launch code. This launch code's parameter block points to a `SysNotifyParamType` structure containing details about the notification.

If your code is not in an application, for example, it is a shared library or a separately loaded code resource, then receiving a launch code is not possible. In this case, pass a pointer to a callback function in `callbackP`. This callback should follow the prototype shown in `SysNotifyProcPtr`. Note that you should always supply a card number and database ID to `SysNotifyRegister`, even if you specify a callback function.

IMPORTANT: Because the `callbackP` pointer is used to directly call the function, the pointer must remain valid from the time `SysNotifyRegister` is called to the time the notification is broadcast. If the function is in a shared library, you must keep the library open. If the function is in a separately loaded code resource, the resource must remain locked while registered for the notification. When you close a library or unlock a resource, you must first unregister for any notifications. If you don't, the system will crash when the notification is broadcast.

Whether the notification handler is responding to `sysAppLaunchCmdNotify` or uses the callback function, the notification handler may perform any processing necessary. As with most launch codes, it's not possible to access global variables. If the handler needs access to any particular value to respond to the notification, pass a pointer to that value in the `userDataP` parameter. The system passes this pointer back to your application or callback function in the launch code's parameter block.

The notification handler may unregister for this notification or register for other notifications. It may also broadcast another notifications; however, it's recommended that you use `SysNotifyBroadcastDeferred` to do this so as not to overflow the broadcast stack.

You may display a user interface in your notification handler; however, you should be careful when you do so. Many of the notifications are broadcast during `SysHandleEvent`, which means your application event loop might not have progressed to the point where it is possible for you to display a user interface, or you might overflow the stack by displaying a user interface at this stage. See the "Notifications" chapter to learn which notifications are broadcast during `SysHandleEvent`.

Compatibility Implemented only if Notification Feature Set is present.

SysNotifyUnregister

Purpose Cancel notification of the given event.

Prototype

```
Err SysNotifyUnregister(UInt16 cardNo,
LocalID dbID, UInt32 notifyType, Int8 priority)
```

Parameters

-> `cardNo`	Number of the storage card on which the application or code resource resides.
-> `dbID`	Local ID of the application or code resource.
-> `notifyType`	The notification for which to unregister. See Notifications.

-> `priority` The priority value you passed as part of `SysNotifyRegister`.

Result Returns one of the following error codes:

`errNone` No error.

`sysNotifyErrEntryNotFound` The application wasn't registered to receive this notification.

Comments Use this function to remove your code from the list of those that receive notifications about a particular event. This function is particularly necessary if you are writing a shared library or a separately loaded code resource that receives notifications. When the resource is unloaded, it must unregister for all of its notifications, or the system will crash when the notification is broadcast.

Compatibility Implemented only if Notification Feature Set is present.

Application-Defined Functions

SysNotifyProcPtr

Purpose Handle a notification.

Prototype

```
Err (*SysNotifyProcPtr)
(SysNotifyParamType *notifyParamsP)
```

Parameters

-> `notifyParamsP` Pointer to a structure that contains the notification event that occurred and any other information about it. See `SysNotifyParamType`.

Result Always return 0.

Comments You pass this function as a parameter to `SysNotifyRegister` when registering code that does not have a `PilotMain` for a notification. See the description of `SysNotifyRegister` for advice on writing a notification handler.

IMPORTANT: Because the `callbackP` pointer is used to directly call the function, the pointer must remain valid from the time `SysNotifyRegister` is called to the time the notification is broadcast. If the function is in a shared library, you must keep the library open. If the function is in a separately loaded code resource, the resource must remain locked while registered for the notification. When you close a library or unlock a resource, you must first unregister for any notifications. If you don't, the system will crash when the notification is broadcast.

41

Overlay Manager

This chapter describes the overlay manager API as declared in the header file `OverlayMgr.h`. It discusses the following topics:

- Overlay Manager Data Structures
- Overlay Manager Constants
- Overlay Manager Functions

For more information on the overlay manager, see the section "Maintaining Backward Compatibility" on page 315 in the "Localized Applications" chapter of the *Palm OS Programmer's Companion*, vol. I.

Overlay Manager Data Structures

OmLocaleType

The `OmLocaleType` struct specifies a locale.

```
typedef struct {
  UInt16   language;
  UInt16   country;
} OmLocaleType;
```

Field Descriptions

`language` The language spoken in the locale. This value is one of the `LanguageType` constants.

`country` The country or region where the language is spoken. This value is one of the `CountryType` constants.

Compatibility If Palm OS® 4.0 New Feature Set is present, the `LmLocaleType` replaces `OmLocaleType`. For backward compatibility, `OmLocaleType` is mapped to `LmLocaleType`.

Overlay Manager Constants

Constant	Value	Description
`omOverlayRscType`	`'ovly'`	Symbolic name of an overlay resource that is contained in both the base database and the overlay database.
`omOverlayRscID`	1000	Resource ID of the overlay resource in both the base database and the overlay database.
`omFtrCreator`	`'ovly'`	Creator value used for the `omFtrShowErrorsFlag` feature.
`omFtrDefaultLocale`	1	Feature that specifies the default locale stored in the ROM. The default locale is used in cases where the system is attempting to open a "stripped" database (one that requires an overlay) and an overlay matching the current locale cannot be found. In this case, the system then looks for an overlay matching the default locale. Use `FtrGet` and `FtrSet` to retrieve and set this value.
`omFtrShowErrorsFlag`	0	Feature that controls the number of error messages displayed by the overlay manager. If this feature is set to `true`, the overlay manager may display several more error messages when validating an overlay against its base database. This feature only takes effect when the error checking level is set to full (common on debug ROMs, not on release ROMs). Use `FtrGet` and `FtrSet` to retrieve and set this value.

Overlay Manager Functions

OmGetCurrentLocale

Purpose Return the current locale.

Prototype

```
void OmGetCurrentLocale
(LmLocaleType *currentLocale)
```

Parameters

<- `currentLocale`
Points to an `LmLocaleType` struct that identifies the current locale.

Result Returns nothing.

Comments This function returns the current locale. The current locale controls which overlays are used for resource databases. For example, suppose you have one application and two associated overlays installed, one for US English and one for British English. In this case, if the country specified in the locale returned by this function is `cUnitedKingdom`, the British English overlay is used for the application. If the country returned is `cUnitedStates`, the US English overlay is used.

Compatibility Implemented only if 3.5 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `OmGlueGetCurrentLocale`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `OmGetSystemLocale`

OmGetIndexedLocale

Purpose Return a system locale by index.

Prototype
```
Err OmGetIndexedLocale (UInt16 localeIndex,
LmLocaleType *theLocale)
```

Parameters

-> `localeIndex`	Zero-based index of the locale to return.
<- `theLocale`	Points to an `LmLocaleType` struct that identifies the locale at that index.

Result Returns `errNone` upon success, or `omErrInvalidLocaleIndex` if the index is out of bounds.

Comments `OmGetIndexedLocale` is used in a loop to discover how many system overlays are installed for system resources.

If the 4.0 New Feature Set is present, use `OmGetNextSystemLocale` instead of this function. `OmGetIndexedLocale` can be slow on ROMs that contain many valid system locales.

Compatibility Implemented only if 3.5 New Feature Set is present.

In Palm OS 3.5, this function would not return a system overlay that was located in RAM. The Palm OS 4.0 version of this function does return system overlays located in RAM.

See Also `OmGetSystemLocale`, `OmGetNextSystemLocale`



New OmGetNextSystemLocale

Purpose

Return a system locale.

Prototype

```
Err OmGetNextSystemLocale (Boolean iNewSearch,
OmSearchStateType *ioStateInfoP,
LmLocaleType *oLocaleP)
```

Parameters

-> `iNewSearch` `true` if this function call is starting a new search, or `false` if this function call is a continuation of a search.

<-> `ioStateInfoP` If `iNewSearch` is `false`, this must point to the same data used for the previous invocation.

<- `oLocaleP` The found locale.

Result

Returns `errNone` if no error or `omErrNoNextSystemLocale` if no matches were found.

Comments

You can call this function successively to discover how many system overlays are installed for system resources. Each system overlay found determines a separate valid system locale. Any locale returned by this function can be sent to `OmSetSystemLocale` to change the system locale.

To start the search, pass `true` for `iNewSearch`. Allocate an `OmSearchStateType` structure and pass its address as `ioStateInfoP`. `OmGetNextSystemLocale` stores private information in `ioStateInfoP` and uses it if the search is continued.

To continue a search where the previous one left off, pass `false` for `iNewSearch` and pass the same `ioStateInfoP` that you used during the previous call to this function.

When called successively, this function eventually returns all system overlays that are in ROM or RAM.

Compatibility

Implemented only if 4.0 New Feature Set is present.

OmGetRoutineAddress

Purpose Return the address of an overlay manager function.

Prototype
```
void *OmGetRoutineAddress (OmSelector inSelector)
```

Parameters

-> `inSelector` One of the routine selectors defined in `OverlayMgr.h`.

Result Returns the address of the corresponding function. Returns `NULL` if an invalid routine selector is passed.

Comments You typically use this function to determine whether an overlay manager function exists on the device. As future releases of Palm OS add new functions, older devices with earlier versions of the overlay manager will not implement these newer functions. If `OmGetRoutineAddress` returns `NULL`, the function is unavailable.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `IntlGetRoutineAddress`, `SysGetTrapAddress`

OmGetSystemLocale

Purpose Return the system locale.

Prototype
```
void OmGetSystemLocale
(LmLocaleType *systemLocale)
```

Parameters

<- `systemLocale` Points to an `LmLocaleType` struct that identifies the system locale.

Result Returns nothing.

Comments You typically don't use this function. Instead, use `OmGetCurrentLocale`, which returns the locale that determines which overlays are used.

The system locale is saved in the storage heap header and persists across soft resets. When the device is reset, the system locale is used to set the current locale.

Compatibility Implemented only if 3.5 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `OmGlueGetSystemLocale`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `OmGetCurrentLocale`

OmLocaleToOverlayDBName

Purpose Return the overlay database's name given the base database name and the locale.

Prototype

```
Err OmLocaleToOverlayDBName
(const Char *baseDBName,
const LmLocaleType *targetLocale,
Char *overlayDBName)
```

Parameters

-> `baseDBName` The name of the base resource database associated with the overlay.

-> `targetLocale` The locale to which this overlay applies. See `LmLocaleType`. Pass `NULL` to use the current locale.

<- `overlayDBName` The overlay database name given the base database name and the target locale. This buffer must be at least `dmDBNameLength` bytes.

Result Returns `errNone` upon success, or `omErrUnknownLocale` if the `targetLocale` parameter is invalid.

Comments The appropriate overlay database name is currently:

`baseDBName_llCC`

where:

baseDBName	The name of the base database as you passed it in.
ll	A two-character code identifying the language.
CC	A two-character code identifying the country.

The base database name is truncated if necessary to allow for this suffix.

For example, the base database "MemoPad" might have an overlay for US English named "MemoPad_enUS".

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also OmOverlayDBNameToLocale

OmOverlayDBNameToLocale

Purpose Return an overlay database's locale given its name.

Prototype

```
Err OmOverlayDBNameToLocale
(const Char *overlayDBName,
LmLocaleType *overlayLocale)
```

Parameters

-> `overlayDBName`
The name of the overlay database.

<- `overlayLocale`
Points to an `LmLocaleType` structure identifying the overlay's locale.

Result Returns `errNone` upon success, `omErrBadOverlayDBName` if the string `overlayDBName` is not long enough to have a locale suffix, or `omErrUnknownLocale` if the locale cannot be determined.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also OmLocaleToOverlayDBName

OmSetSystemLocale

Purpose Set the system locale and reset the device.

Prototype

```
Err OmSetSystemLocale
(const LmLocaleType *systemLocale)
```

Parameters

-> `systemLocale` An `LmLocaleType` structure specifying the locale to switch the system to.

Result Returns `errNone` upon success, or one of the following if an error occurs:

`omErrUnknownLocale`
There is no system overlay for `systemLocale`.

`omErrInvalidLocale`
The system overlay for `systemLocale` has been found but is invalid.

`dmErrInvalidParam`
An error occurred while opening the overlay.

`dmErrMemError` A memory error occurred while opening the overlay.

`dmErrDatabaseOpen`
The system overlay was already open.

Comments This function changes the system locale to the specified locale if it exists. It first determines that the system overlay exists for the requested locale and that it matches the base system database. If so, it updates the system locale information saved in the storage heap header and resets the device. After the device is reset, the current locale is set to the system locale.

A Palm Powered™ device has a default locale hard-coded into the ROM. This locale is used to set the system locale after a hard reset or any time that the storage heap header is invalid. The storage heap header is typically only invalid when the device is turned on for the first time.

Compatibility Implemented only if 3.5 New Feature Set is present.

In Palm OS 3.5, this function would not switch to a system overlay that was located in the RAM. The Palm OS 4.0 version of this function does return system overlays located in the RAM.

See Also `OmGetSystemLocale`

42

Password

This chapter provides reference material for the password API. The password API is declared in the header file `Password.h`.

Password Functions

PwdExists

Purpose Return `true` if the system password is set.

Prototype
```
Boolean PwdExists()
```

Parameters None

Result Returns `true` if the system password is set.

PwdRemove

Purpose Remove the encrypted password string and recover data hidden in databases.

Prototype
```
void PwdRemove (void)
```

Parameters None

Result Returns nothing.

PwdSet

Purpose Use a passed string as the new password. The password is stored in an encrypted form.

Prototype

```
void PwdSet (Char* oldPassword, Char* newPassword)
```

Parameters

`oldPassword`	The old password must be successfully verified or the new password isn't accepted
`newPassword`	`Char*` to a string to use as the password. `NULL` means no password.

Result Returns nothing.

PwdVerify

Purpose Verify that the string passed matches the system password.

Prototype

```
Boolean PwdVerify (Char* string)
```

Parameters

`string`	String to compare to the system password. `NULL` means no current password. The allocated length of string must be at least `pwdLength` characters long.

Result Returns `true` if the string matches the system password.

43

Pen Manager

This chapter provides reference material for the pen manager. The pen manager API is declared in the header file `PenMgr.h`.

For more information on the pen manager, see the section "Receiving User Input" in the *Palm OS Programmer's Companion*, vol. I.

Pen Manager Functions

PenCalibrate

Purpose Set the calibration of the pen.

Prototype

```
Err PenCalibrate (PointType* digTopLeftP,
PointType* digBotRightP, PointType* scrTopLeftP,
PointType* scrBotRightP)
```

Parameters

`digTopLeftP`	Digitizer output from top-left coordinate.
`digBotRightP`	Digitizer output from bottom-right coordinate.
`scrTopLeftP`	Screen coordinate near top-left corner.
`scrBotRightP`	Screen coordinate near bottom-right corner.

Result Returns 0 if no error.

Comments Called by Preferences application when calibrating pen.

See Also `PenResetCalibration`

PenResetCalibration

Purpose Reset the calibration in preparation for calibrating the pen again.

Prototype

```
Err PenResetCalibration (void)
```

Parameters None.

Result Always returns 0.

Comments Called by Preferences application before capturing points when calibrating the pen.

See Also PenCalibrate

WARNING! The digitizer is off after calling this routine and must be calibrated again!

44

Preferences

This chapter describes the preferences API as declared in the header file `Preferences.h`. It discusses the following topics:

- Preferences Data Types
- Preferences Constants
- Preferences Functions

For more information on preferences, see the section "Preferences" on page 276 of the *Palm OS Programmer's Companion*, vol. I.

Preferences Data Types

MeasurementSystemType

The `MeasurementSystemType` enum specifies the preference for units of measure.

```
typedef enum {
  unitsEnglish = 0,
  unitsMetric
} MeasurementSystemType;
```

Value Descriptions

`unitsEnglish`	The English measurement system (feet, inches, and so on).
`unitsMetric`	The Metric system (meters, centimeters, and so on).



New SecurityAutoLockType

The `SecurityAutoLockType` enum specifies the values for the auto-locking preference. The auto-locking preference specifies when the device will shut down and lock itself.

```
typedef enum {
  never = 0,
  uponPowerOff,
  atPresetTime,
  afterPresetDelay
} SecurityAutoLockType;
```

Value Descriptions

`never`	The auto-lock feature is disabled.
`uponPowerOff`	The device locks itself each time it is powered off.
`atPresetTime`	The device locks itself at a certain time every day.
`afterPresetDelay`	The device locks itself after a certain amount of idle time.

SoundLevelTypeV20

The `SoundLevelTypeV20` enum specifies whether certain sounds are enabled or disabled. This enum is used as the values for several of the system sound preferences.

```
typedef enum {
  slOn = 0,
  slOff = 1
} SoundLevelTypeV20;
```

Value Descriptions

`slOn`	Enabled
`slOff`	Disabled

Compatibility This enumerated type is obsolete in Palm OS® versions 3.0 and higher. The preferences that use this enum are replaced by preferences that indicate sound volume as well as whether the sound is on or off.

SystemPreferencesChoice

The `SystemPreferencesChoice` enum defines values that you can pass to `PrefGetPreference` and `PrefSetPreference` to retrieve or set a system preference value. `SystemPreferencesChoice` defines one constant for each field in the `SystemPreferencesType` structure, which should be considered a private structure.

Table 44.1 lists and describes the `SystemPreferencesChoice` constants. For each constant, it shows what type of data is returned by `PrefGetPreference` for that constant and which version of the system preferences structure contains this preference.

The system preferences structure keeps track of its own version information because it has been updated several times. Each Palm OS release that updates the system preferences structure increments the structure's version number. Table 44.1 on page 738 specifies which version of the system preferences structure introduced that preference. For each preferences version, there is a constant representing that version. See the section "Preferences Constants" on page 747 for information on which preferences version corresponds with which Palm OS release. You should check the preference's version number before attempting to obtain the value of any preference introduced after version 2. For example:

```
LmLocaleType currentLocale;
if (PrefGetPreference(prefVersion) >=
    preferenceDataVer9) {
  currentLocale = (LmLocaleType)
    PrefGetPreference(prefLocale);
}
```

Most of the system preferences can be set in the Preferences and Security applications. Table 44.2 on page 746 specifies which system preference is set by each user interface field in these two applications.

Table 44.1 SystemPreferencesChoice Constants

Constant	Type	Version	Description
`prefVersion`	`UInt16`	2	The preferences version number.
`prefCountry`	`CountryType`	2	The country for which the device was built.
`prefDateFormat`	`DateFormatType`	2	The short format used to display dates. For example: 95/12/31
`prefLongDateFormat`	`DateFormatType`	2	The long format used to display dates. For example: 31 Dec 1995
`prefWeekStartDay`	`Int8`	2	The first day of the week (Sunday or Monday). Days of the week are numbered from 0 to 6 starting with Sunday = 0.
`prefTimeFormat`	`TimeFormatType`	2	The format used to display time values.
`prefNumberFormat`	`NumberFormatType`	2	The format used for numbers, with regards to the thousands separator and the decimal point.
`prefAutoOffDuration`	`UInt8`	2	Minutes of user idle time before the device powers off. The default value for this preference is specified by the `defaultAutoOffDuration` constant.

Table 44.1 SystemPreferencesChoice Constants *(continued)*

Constant	Type	Version	Description
			`prefAutoOffDuration` is replaced by `prefAutoOffDurationSecs` in version 8 of the preferences structure.
`prefSysSoundLevelV20`	`SoundLevelTypeV20`	2	Specifies whether system sounds are enabled or disabled.
`prefGameSoundLevelV20`	`SoundLevelTypeV20`	2	Specifies whether game sound effects are on or off.
`prefAlarmSoundLevelV20`	`SoundLevelTypeV20`	2	Specifies whether sound alarms are on or off.
`prefHidePrivateRecordsV33`	`Boolean`	2	If `true`, applications should not display database records that have the secret attribute bit set.
`prefDeviceLocked`	`Boolean`	2	If `true`, the device is locked. When the device is locked, it remains so until the user enters the password.
`prefLocalSyncRequiresPassword`	`Boolean`	2	If `true`, the user must enter a password before a HotSync® operation can be performed.
`prefRemoteSyncRequiresPassword`	`Boolean`	2	If `true`, the user must enter a password on the desktop computer before a HotSync operation can be performed.
`prefSysBatteryKind`	`SysBatteryKind`	2	The type of batteries installed. Use `SysBatteryInfo` to retrieve the battery type instead of this preference.

Table 44.1 SystemPreferencesChoice Constants *(continued)*

Constant	Type	Version	Description
`prefMinutesWestOfGMT`	`UInt32`	2	The time zone given as minutes **east** of Greenwich Mean Time (GMT). For preferences version 9 and higher, use `prefTimeZone` instead.
`prefDaylightSavings`	`DaylightSavingsTypes`	2	The type of daylight savings correction. For preferences version 9 and higher, use `prefDaylightSavingAdjustment` instead.
`prefRonamaticChar`	`UInt16`	2	The virtual character generated when the user enters the ronamatic stroke. The ronamatic stroke is dragging the pen from the Graffiti® area to the top of the screen.
`prefHard1CharAppCreator`	`UInt32`	2	The creator ID of the application to be launched by the left-most hard key (the Date Book button by default).
`prefHard2CharAppCreator`	`UInt32`	2	The creator ID of the application to be launched by the second hard key from the left (the Address button by default).
`prefHard3CharAppCreator`	`UInt32`	2	The creator ID of the application to be launched by the second hard key from the right (the To Do List button by default).

Table 44.1 SystemPreferencesChoice Constants *(continued)*

Constant	Type	Version	Description
`prefHard4CharAppCreator`	`UInt32`	2	The creator ID of the application to be launched by the right-most hard key (the Memo Pad button by default).
`prefCalcCharAppCreator`	`UInt32`	2	The creator ID of the application to be launched by the Calculator silkscreen button.
`prefHardCradleCharAppCreator`	`UInt32`	2	The creator ID of the application to be launched by the hard key on the HotSync cradle.
`prefLauncherAppCreator`	`UInt32`	2	The creator ID of the application to be launched by the Applications silkscreen button.
`prefHardCradle2CharAppCreator`	`UInt32`	2	The creator ID of the application to be launched by the HotSync button on the modem.
`prefAnimationLevel`	`AnimationLevelType`	2	Reserved for future use.
`prefSysSoundVolume`	`UInt16`	3	The sound level for system sounds, such as taps and beeps. This is a value from 0 to `sndMaxAmp`.
`prefGameSoundVolume`	`UInt16`	3	The sound level for game sounds. This is a value from 0 to `sndMaxAmp`.

Table 44.1 SystemPreferencesChoice Constants *(continued)*

Constant	Type	Version	Description
`prefAlarmSoundVolume`	`UInt16`	3	The sound level for alarms. This is a value from 0 to `sndMaxAmp`.
`prefBeamReceive`	`Boolean`	3	If `true`, the device can receive beams from other devices. If `false`, the device cannot receive beams but can still send them. This preference is not currently used. Instead, use the `ExgControl` function with one of the constants described in "IR Control Constants."
`prefCalibrateDigitizerAtReset`	`Boolean`	3	If `true`, the user must recalibrate the digitizer after a soft reset. The default is `false`.
`prefSystemKeyboardID`	`UInt16`	4	The resource ID of the keyboard panel.
`prefDefSerialPlugIn`	`UInt32`	4	The creator ID of the default serial plug-in database.
`prefStayOnWhenPluggedIn`	`Boolean`	5	If `true`, the device stays powered on when it is in the cradle.
`prefStayLitWhenPluggedIn`	`Boolean`	5	If `true` and `prefStayOnWhenPluggedIn` is `true`, the device stays lit when it is in its cradle.
`prefAntennaCharAppCreator`	`UInt32`	6	The creator ID of the application to launch when the antenna is raised (used only for devices with built-in antennas).

Table 44.1 SystemPreferencesChoice Constants *(continued)*

Constant	Type	Version	Description
`prefMeasurementSystem`	`MeasurementSystemType`	7	The system of measurement to use.
`prefShowPrivateRecords`	`privateRecordViewEnum`	8	Specifies whether the private records should be displayed, masked, or completely hidden.
`prefAutoOffDurationSecs`	`UInt16`	8	Seconds of user idle time before the device powers off. The default value for this preference is specified by the `defaultAutoOffDurationSecs` constant.
`prefTimeZone`	`Int16`	9	The time zone given as minutes east of Greenwich Mean Time (GMT). **IMPORTANT:** Changing the value of this preference does not update the current time.
`prefDaylightSavingAdjustment`	`Int16`	9	The number of minutes to add to the current time for daylight savings time. **IMPORTANT:** Changing the value of this preference does not update the current time.
`prefTimeZoneCountry`	`CountryType`	9	The country selected to specify what the time zone is.

Table 44.1 SystemPreferencesChoice Constants *(continued)*

Constant	Type	Version	Description
`prefAutoLockType`	`SecurityAutoLockType`	9	Specifies when the auto-locking feature should take effect. Possibilities are upon power off, at a preset time, or after a certain number of seconds.
`prefAutoLockTime`	`UInt32`	9	The time value for the auto-locking feature if the system should lock itself after a delay or at a preset time. Depending on the value of `prefAutoLockType`, this value is either an absolute date and time given as the number of seconds since January 1, 1904 or a timeout value given as a number of seconds from the current time.
`prefAutoLock TimeFlag`	`Boolean`	9	If `true`, `prefAutoLockTime` is given in minutes. If `false`, the time is given in hours.
`prefLanguage`	`LanguageType`	9	The language that the device should use.
`prefAttention Flags`	`AttnFlagsType`	9	The user's preferences for receiving attention signals. The returned value is a bit mask that should be tested (using the `&` operator) with one of the following values:

Table 44.1 SystemPreferencesChoice Constants *(continued)*

Constant	Type	Version	Description

			`kAttnFlagsUserWantsLED`
			`kAttnFlagsUserWants Sound`
			`kAttnFlagsUserWants Vibrate`
			`kAttnFlagsUserWants CustomEffect`
			
			Note that you can override the values in `prefAttentionFlags` when you make Attention Manager function calls. See the section "Getting the User's Attention" on page 239 of the *Palm OS Programmer's Companion*, vol. I for more information.

`prefDefault AppCreator`	`UInt32`	9	Creator ID of the application that is launched after a reset. If 0, the system default application is launched.
`prefLocale`	`LmLocaleType`	9	The device's current locale, which specifies the country and language.

Preferences in the User Interface

Table 44.2 shows the `SystemPreferencesChoice` constants and how they correspond to the values that users can set in the Preferences and Security applications. For further information about each preference, see Table 44.1.

Table 44.2 Preferences set in standard apps

Application/Panel	Field	SystemPreferencesChoice Constant
Preferences application General panel	Auto-off After	`prefAutoOffDuration`, `prefAutoOffDurationSecs` (Palm OS 3.5 and higher)
	Stay on in Cradle	`prefStayOnWhenPluggedIn`
	System Sound	`prefSysSoundLevelV20`, `prefSysSoundVolume`
	Alarm Sound	`prefAlarmSoundLevelV20`, `prefAlarmSoundVolume`
	Alarm Vibrate[a]	`prefAttentionFlags`
	Alarm LED[a]	`prefAttentionFlags`
	Game Sound	`prefGameSoundLevelV20`, `prefGameSoundVolume`
	Beam Receive field	`prefBeamReceive`
Preferences application Date & Time panel	Set Time Zone field	`prefTimeZone`
	Daylight Saving	`prefDaylightSaving Adjustment`
Preferences application Formats panel	Preset to	`prefCountry`
	Time	`prefTimeFormat`
	Date	`prefDateFormat`, `prefLongDateFormat`
	Week starts	`prefWeekStartDay`
	Numbers	`prefNumberFormat`

Table 44.2 Preferences set in standard apps *(continued)*

Application/Panel	Field	SystemPreferencesChoice Constant
Preferences application Buttons panel	Buttons on main panel	`prefHard1CharAppCreator`, `prefHard2CharAppCreator`, `prefHard3CharAppCreator`, `prefHard4CharAppCreator`, `prefCalcCharAppCreator`, `prefLauncherAppCreator`
	Pen button	`prefRonamaticChar`
	HotSync button	`prefHardCradleCharAppCreator` `prefHardCradle2CharAppCreator`
Security application	Current Privacy	`prefHidePrivateRecordsV33`, `prefShowPrivateRecords`
Security application	Lock & Turn Off button	`prefDeviceLocked`, `prefAutoLockType`, `prefAutoLockTime`, `prefAutoLockTimeFlag`

a. The Alarm Vibrate and Alarm LED preferences only appear on handhelds running Palm OS 4.0 and later that have the appropriate hardware capabilities. If you install Palm OS 4.0 on an older device, these preferences do not display.

Preferences Constants

The following table describes the constants defined in the `Preferences.h` header file.

Constant	Value	Description
`defaultAutoLockTime`	0	Initial setting for the `prefAutoLockTime` preference.
`defaultAutoLockTimeFlag`	0	Initial setting for the `prefAutoLockTimeFlag` preference.
`defaultAutoLockType`	never	Initial setting for the `prefAutoLockType` preference.
`defaultAutoOffDuration`	2	Initial setting for the `prefAutoOffDuration` preference, given in minutes.
`defaultAutoOffDuration Secs`	120	Initial setting for the `prefAutoOffDurationSecs` preference, given in seconds. The value is 2 times the number of seconds in a minute, or 120 seconds.
`defaultAlarmSoundLevel`	`slOn`	Initial setting for the `prefAlarmSoundLevelV20` preference.
`defaultAlarmSoundVolume`	`sndMaxAmp`	Initial setting for the `prefAlarmSoundVolume` preference.
`defaultGameSoundLevel`	`slOn`	Initial setting for the `prefGameSoundLevelV20` preference.
`defaultGameSoundVolume`	`sndMaxAmp`	Initial setting for the `prefGameSoundVolume` preference.
`defaultSysSoundLevel`	`slOn`	Initial setting for the `prefSysSoundLevelV20` preference.
`defaultSysSoundVolume`	`sndMaxAmp`	Initial setting for the `prefSysSoundVolume` preference.

Constant	Value	Description
noPreferenceFound	-1	The value that PrefGetAppPreferences returns to indicate that preferences couldn't be found.
preferenceDataVer2	2	Version 2 of the system preferences structure, used for Palm OS 2.0.
preferenceDataVer3	3	Version 3 of the system preferences structure, used for Palm OS 3.0.
preferenceDataVer4	4	Version 4 of the system preferences structure, used for Palm OS 3.1.
preferenceDataVer5	5	Version 5 of the system preferences structure, used for Palm OS 3.2.
preferenceDataVer6	6	Version 6 of the system preferences structure, used for Palm OS 3.3.
preferenceDataVer8	8	Version 8 of the system preferences structure, used for Palm OS 3.5.
preferenceDataVer9	9	Version 9 of the system preferences structure, used for Palm OS 4.0.
preferenceDataVerLatest	preferenceDataVer9	The latest version of the system preferences structure.

Preferences Functions

PrefGetAppPreferences

Purpose Return a copy of an application's preferences resource.

Prototype

```
Int16 PrefGetAppPreferences (UInt32 creator,
UInt16 id, void *prefs, UInt16 *prefsSize,
Boolean saved)
```

Parameters

-> `creator`	Creator ID of the application that owns the preferences.
-> `id`	ID number of the preferences resource to retrieve. The IDs `0x8000` through `0xFFFF` are reserved for system use.
<- `prefs`	Pointer to a buffer to hold the preferences.
<-> `prefsSize`	Pointer to the size of the `prefs` buffer passed in. Upon return, contains the number of bytes actually written or the number of bytes needed for the `prefs` structure. You can determine the required size for the `prefs` structure by passing `NULL` for `prefs`. Upon return, the `prefsSize` parameter contains the required size. Always compare the value returned in this parameter with the value you passed in. If the two values differ, you need to resize the `prefs` structure and call this function again.

-> `saved`	If `true`, retrieve the preferences from the "saved" preferences database, which is backed up during a HotSync operation. If `false`, retrieve the preferences from the "unsaved" preferences database, which is usually not backed up during a HotSync operation.

Result Returns the version number of the retrieved preferences resource, or returns the constant `noPreferenceFound` if the preferences resource wasn't found. The returned version number is the same version number that was passed to the `PrefSetAppPreferences` function.

Comments Use this function to retrieve the preferences that you previously set with the `PrefSetAppPreferences` function. You typically call this function in your `StartApplication` function upon a normal launch. The values of the `id` and `saved` parameters should be the same as you specified when calling `PrefSetAppPreferences`, and the `prefs` parameter should be a structure of the same type as you passed to `PrefSetAppPreferences`. Most applications store all preferences in a single preferences resource retrieved by a single call to `PrefGetAppPreferences`, but this is not required. You can use multiple preferences resources if you wish.

The version number returned by this function allows you to handle the case where a new version of the application is being run for the first time. You can compare the value returned by this function with the current version number to determine if you need to set default values for any preferences created by the current release. For more information, see the section "Updating Preferences Upon a New Release" on page 285 of the *Palm OS Programmer's Companion*, vol. I.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `PrefSetPreferences`, `PrefGetAppPreferencesV10`

PrefGetAppPreferencesV10

Purpose Return a copy of an application's preferences.

Prototype
```
Boolean PrefGetAppPreferencesV10 (UInt32 type,
Int16 version, void *prefs, UInt16 prefsSize)
```

Parameters

-> `type`	Creator ID of the application that owns the preferences.
-> `version`	Version number of the application's preferences.
<- `prefs`	Pointer to a buffer to hold preferences.
-> `prefsSize`	Size of the buffer passed.

Result Returns `false` if the preference resource was not found or the preference resource contains the wrong version number.

Comments The content and format of an application preference is application-dependent.

Compatibility This function corresponds to the 1.0 version of `PrefGetAppPreferences`.

See Also `PrefSetPreferences`, `PrefGetAppPreferences`

PrefGetPreference

Purpose Return a system preference. Use this function instead of `PrefGetPreferences`.

Prototype
```
UInt32 PrefGetPreference
(SystemPreferencesChoice choice)
```

Parameters

-> `choice`	A constant that specifies what preference to retrieve. See `SystemPreferencesChoice`.

Result Returns the system preference or 0 if the preference could not be found. On debug ROMs, a non-fatal error message is also displayed if the specified preference cannot be found.

Comments This function replaces the 1.0 function `PrefGetPreferences`. While `PrefGetPreferences` only lets you retrieve the whole system preferences structure, this function lets you specify which preference to retrieve.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `PrefSetPreference`, `PrefGetAppPreferences`, `PrefGetAppPreferencesV10`

PrefGetPreferences

Purpose Return a copy of the system preferences.

Prototype
```
void PrefGetPreferences (SystemPreferencesPtr p)
```

Parameters <- `p` Pointer to system preferences.

Result Returns nothing. Stores the system preferences in `p`.

Comments The `p` parameter points to a memory block allocated by the caller that is filled in by this function.

This function is often called in `StartApplication` to get localized settings.

NOTE: This function can only be used to retrieve preferences that were in the 1.0 version of the preferences structure.

See Also `PrefSetPreferences`

PrefOpenPreferenceDB

Purpose Return a handle to either the saved or unsaved preference database.

Prototype
```
DmOpenRef PrefOpenPreferenceDB (Boolean saved)
```

Parameters

-> `saved`	If `true`, open the "saved" preferences database, which is backed up during a HotSync operation. If `false`, open the "unsaved" preferences database, which usually is not backed up during a HotSync operation.

Result Returns the handle, or 0 if an error results.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `PrefGetPreference`, `PrefSetPreference`, `PrefOpenPreferenceDBV10`

PrefOpenPreferenceDBV10

Purpose Return a handle to the system preference database.

Prototype
```
DmOpenRef PrefOpenPreferenceDBV10 (void)
```

Parameters None.

Result Returns the handle, or 0 if an error results.

Compatibility This function corresponds to the 1.0 version of `PrefOpenPreferenceDB`.

See Also `PrefGetPreferences`, `PrefSetPreferences`

PrefSetAppPreferences

Purpose Set an application's preferences in the specified preferences database.

Prototype

```
void PrefSetAppPreferences (UInt32 creator,
UInt16 id, Int16 version, const void *prefs,
UInt16 prefsSize, Boolean saved)
```

Parameters

-> `creator`	Creator ID of the application that owns this preference.
-> `id`	ID number of the preference to set. An application can have multiple preferences. The IDs `0x8000` through `0xFFFF` are reserved for system use.
-> `version`	Version number of the application's preferences.
-> `prefs`	Pointer to a buffer that holds the current value of the preferences structure. Pass `NULL` if you want to delete the preferences.
-> `prefsSize`	Size of the buffer passed. Pass 0 if you want to delete the preferences structure.
-> `saved`	If `true`, saves the preferences in the "saved" preferences database. If not, saves the preferences in the "unsaved" preferences database.

Result Returns nothing.

Comments You typically call this function in your `StopApplication` function to save the current state of the application or if the user has changed an application preference during the current session.

The "saved" preferences database is backed up when a user performs the HotSync operation. The "unsaved" preferences database is not backed up by default. (The user can use a third-party tool to set the backup bit in the "unsaved" preferences database, which would cause it to be backed up.) Both the "saved" and the

"unsaved" preferences reside in the storage heap and thus persist across soft resets. The only way that preferences are lost is if a hard reset is performed. "Which Preferences Database to Use" on page 283 of the *Palm OS Programmer's Companion*, vol. I describes how to choose between the "saved" and "unsaved" preferences databases.

The version number that you pass as the `version` parameter is returned when the preferences are retrieved by `PrefGetAppPreferences`. You can use this version number to determine if a new release of the application is being run for the first time. For more information, see the section "Updating Preferences Upon a New Release" on page 285 of the *Palm OS Programmer's Companion*, vol. I.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `PrefSetAppPreferencesV10`

PrefSetAppPreferencesV10

Purpose Save an application's preferences in the preferences database.

Prototype

```
void PrefSetAppPreferencesV10 (UInt32 creator,
Int16 version, void *prefs, UInt16 prefsSize)
```

Parameters

-> `creator`	Creator ID of the application that owns this preference.
-> `version`	Version number of the application.
-> `prefs`	Pointer to a buffer holding preferences.
-> `prefsSize`	Size of the buffer passed.

Result Returns nothing.

Comments The content and format of an application preference is application-dependent.

Compatibility This function corresponds to the 1.0 version of `PrefSetAppPreferences`.

See Also PrefSetAppPreferences, PrefGetPreferences

PrefSetPreference

Purpose Set a system preference. Using this function instead of `PrefSetPreferences` allows you to set selected preferences without having to access the whole structure.

Prototype

```
void PrefSetPreference
(SystemPreferencesChoice choice, UInt32 value)
```

Parameters

-> `choice`	A SystemPreferencesChoice constant specifying the preference to be set.
-> `value`	Value to assign to the preference.

Result Returns nothing. If the specified preference cannot be found, displays a non-fatal error message on debug ROMs. On release ROMs, this function fails silently.

Compatibility Implemented only if 2.0 New Feature Set is present.

PrefSetPreferences

Purpose Set the system preferences.

Prototype

```
void PrefSetPreferences (SystemPreferencesPtr p)
```

Parameters

-> `p`	Pointer to system preferences.

Result Returns nothing.

Comments Unless there's a reason for you to access the whole preferences structure, call PrefSetPreference instead.

NOTE: This function can only be used to set preferences that were in the 1.0 version of the preferences structure.

See Also PrefGetPreferences

45

Rectangles

This chapter provides reference material for the rectangles API, declared in the header file `Rect.h`. It is divided into the following sections:

- Rectangle Data Structures
- Rectangle Functions

Rectangle Data Structures

PointType

The `PointType` structure defines a point within a window or on the screen.

```
typedef struct PointType {
  Coord x;
  Coord y;
} PointType;
```

Field Descriptions

`x`	Horizontal coordinate.
`y`	Vertical coordinate.

RectangleType

The `RectangleType` structure defines a rectangular portion of a window or of the screen.

```
typedef struct RectangleType {
  PointType  topLeft;
  PointType  extent;
} RectangleType;
typedef RectangleType *RectanglePtr;
```

Field Descriptions

`topLeft`	Coordinates of the upper-left corner of the rectangle relative to the window or screen in which the rectangle resides.
`extent`	Width (`extent.x`) and height (`extent.y`) of the rectangle.

Rectangle Functions

RctCopyRectangle

Purpose Copy the source rectangle to the destination rectangle.

Prototype

```
void RctCopyRectangle
(const RectangleType* srcRectP,
RectangleType* dstRectP)
```

Parameters

`srcRectP`	A pointer to the rectangle to be copied.
`dstRectP`	A pointer to the destination rectangle.

See Also `RctSetRectangle`

RctGetIntersection

Purpose Determine the intersection of two rectangles.

Prototype

```
void RctGetIntersection (const RectangleType* r1P,
const RectangleType* r2P, RectangleType* r3P)
```

Parameters

`r1P`	A pointer to a source rectangle.
`r2P`	A pointer to the other source rectangle.
`r3P`	Upon return, points to a rectangle representing the intersection of `r1` and `r2`.

Comments The rectangle type `RectangleType`, which is pointed to by `RectanglePtr`, stores the coordinates for the top-left corner of the rectangle plus the rectangle's width and height. This function returns in the `r3` parameter a pointer to the rectangle that represents the intersection of the first two rectangles.

If the rectangles `r1` and `r2` do not intersect, `r3` contains a rectangle whose top-left coordinate is the maximum of `r1` and `r2`'s top-left coordinates and whose extent varies based on the location of the two rectangles.

Compatibility On releases prior to Palm OS® 3.5, if rectangles `r1` and `r2` don't intersect, `r3` contains a rectangle that begins at coordinates (0,0 and has 0 width and 0 height. On Palm OS 3.5 and later, if the two rectangles don't intersect then `r3` contains a rectangle in which one or both of the extent coordinates is zero.

RctInsetRectangle

Purpose Move all of the boundaries of a rectangle by a specified offset.

Prototype
```
void RctInsetRectangle (RectangleType* rP,
Coord insetAmt)
```

Parameters

`rP`	A pointer to the rectangle.
`insetAmt`	Number of pixels to move the boundaries. This can be a negative number.

Comments The rectangle type `RectangleType`, which is pointed to by `RectanglePtr`, stores the coordinates for the top-left corner of the rectangle plus the rectangle's width and height. This function adds `insetAmt` to the x and y values of the top-left coordinate and then adjusts the width and the height accordingly so that all of the sides of the rectangle are contracted or expanded by the same amount.

A positive `insetAmt` creates a smaller rectangle that is contained inside the old rectangle's boundaries. A negative `insetAmt` creates a larger rectangle that surrounds the old rectangle.

See Also `RctOffsetRectangle`

RctOffsetRectangle

Purpose Move the top and left boundaries of a rectangle by the specified values.

Prototype
```
void RctOffsetRectangle (RectangleType* rP,
Coord deltaX, Coord deltaY)
```

Parameters

`rP`	A pointer to the rectangle.
`deltaX`	Number of pixels to move the left boundary. This can be a negative number.
`deltaY`	Number of pixels to move the top boundary. This can be a negative number.

Comments The rectangle type `RectangleType`, which is pointed to by `RectanglePtr`, stores the coordinates for the top-left corner of the rectangle plus the rectangle's width and height. This function adds `deltaX` to the x value of the top-left coordinate and `deltaY` to the y value. The width and height are unchanged. Thus, this function shifts the position of the rectangle by the `deltaX` and `deltaY` amounts.

See Also `RctInsetRectangle`

RctPtInRectangle

Purpose Determine if a point lies within a rectangle's boundaries.

Prototype
```
Boolean RctPtInRectangle (Coord x, Coord y,
const RectangleType* rP)
```

Parameters

`x`	The x coordinate of the point.
`y`	The y coordinate of the point.
`rP`	The rectangle.

Result Returns `true` if the point (`x`, `y`) lies within the boundaries of rectangle `r`, `false` otherwise.

RctSetRectangle

Purpose Sets a rectangle's values.

Prototype

```
void RctSetRectangle (RectangleType* rP,
Coord left, Coord top, Coord width, Coord height)
```

Parameters

`rP`	A pointer to the rectangle to be set.
`left`	The x value for the top-left coordinate of the rectangle.
`top`	The y value for the top-left coordinate of the rectangle.
`width`	The rectangle's width.
`height`	The rectangle's height.

See Also `RctCopyRectangle`

46

Sound Manager

This chapter provides reference material for the sound manager.

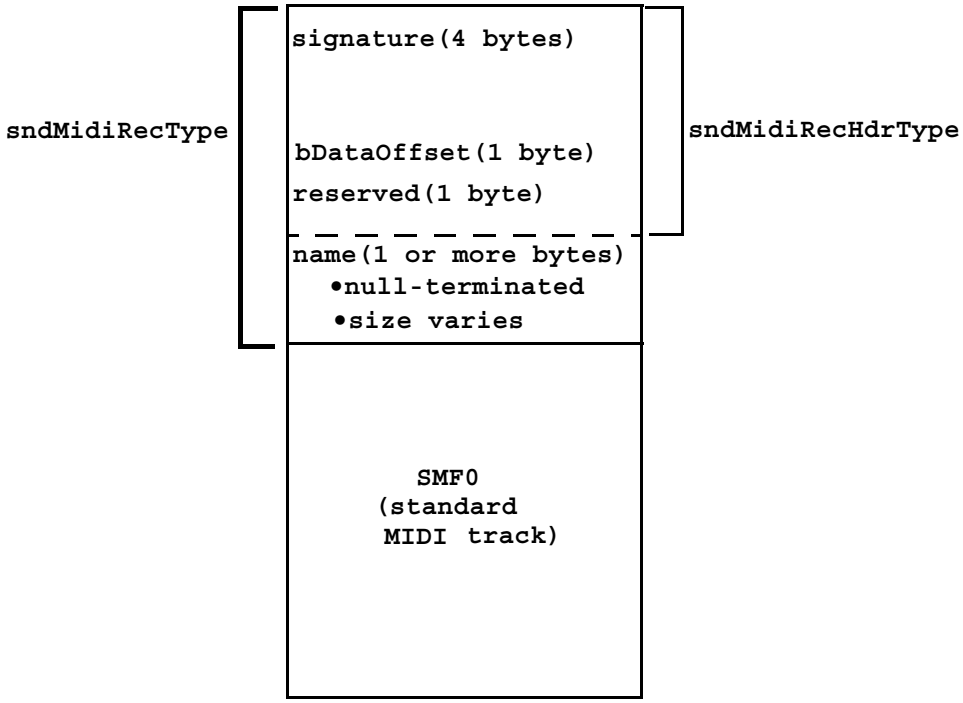
- Sound Manager Data Structures
- Application-Defined Functions
- Sound Manager Functions

The header file `SoundMgr.h` declares the API that this chapter describes. For more information on the sound manager, see the section "Sound" in the *Palm OS Programmer's Companion*, vol. I.

Sound Manager Data Structures

This section describes the data structures that define the MIDI records and parameter blocks used by sound manager functions. Figure 46.1 depicts a Palm OS® MIDI record graphically.

Figure 46.1 Palm OS Midi Record



SndCallbackInfoType

The `SndCallbackInfoType` structure wraps the sound manager callback functions that you implement. See "Application-Defined Functions" for more information.

```
typedef struct SndCallbackInfoType {
  MemPtr   funcP;
  UInt32   dwUserData;
} SndCallbackInfoType;
```

Field Descriptions

`funcP`	Pointer to the callback function (`NULL` = no function).
`dwUserData`	Value to pass in `dwUserData` parameter of callback function.

SndCmdIDType

The `SndCmdIDType` enumerated type defines the commands that may be specified in the `cmd` field of the `SndCommandType`. Each command defines its own specific use of the `param1`, `param2`, and `param3` fields.

```
typedef enum SndCmdIDType {
  sndCmdFreqDurationAmp = 1,
  sndCmdNoteOn,
  sndCmdFrqOn,
  sndCmdQuiet
} SndCmdIDType;
```

Value Descriptions

`sndCmdFreqDurationAmp`	Play a sound, blocking for the entire duration (except for zero amplitude). `param1` = frequency in Hz `param2` = duration in milliseconds `param3` = amplitude (0 to `sndMaxAmp`) If value of `param3` is 0, returns immediately.
`sndCmdNoteOn`	Play sound at specified MIDI key index with max duration and velocity; return immediately, without waiting for playback to complete. Any other sound play request made before this one completes will interrupt it. `param1` = MIDI key index (0-127) `param2` = maximum duration in milliseconds `param3` = velocity (0 to 127) to be interpolated as amplitude.

`sndCmdFrqOn`	Similar to `sndCmdNoteOn` except note to play is specified as frequency in Hz. `param1` = frequency in Hz `param2` = maximum duration in milliseconds `param3` = amplitude (0 - sndMaxAmp)
`sndCmdQuiet`	Stop playback of current sound. `param1` = 0 `param2` = 0 `param3` = 0

IMPORTANT: `SndDoCmd` in versions of Palm OS before 3.0 will generate a fatal error on anything other than `sndCmdFreqDurationAmp`. For this reason, applications wishing to take advantage of these new commands while staying compatible with the earlier version of the OS, **must** avoid using these commands when running on OS versions less than v3.0. Beginning with v3.0, `SndDoCmd` returns `sndErrBadParam` when an unknown command is passed.

SndCommandType

The `SndCommandType` structure is passed as the value of the `cmdP` parameter to the `SndDoCmd` function. Its parameters are defined by the `SndCmdIDType`.

```
typedef struct SndCommandType {
  SndCmdIDType    cmd;
  UInt8           reserved;
  Int32           param1;
  UInt16          param2;
  UInt16          param3;
} SndCommandType;
```

Field Descriptions

`cmd`	Command ID.
`reserved`	Reserved for future use.
`param1, param2, param3`	Use varies according to value of `cmd`.

SndMidiListItemType

When the `SndCreateMidiList` function returns `true`, its `entHP` parameter holds a handle to a memory chunk containing an array of `SndMidiListItemType` structures.

```
typedef struct SndMidiListItemType{
  Char    name[sndMidiNameLength];
  UInt32  uniqueRecID;
  LocalID dbID;
  UInt16   cardNo;
} SndMidiListItemType;
```

Field Descriptions

`name`	MIDI name including null terminator.
`uniqueRecID`	Unique ID of the record containing the MIDI file.
`dbID`	Database (file) ID.
`cardNo`	Number of the memory card on which the MIDI file resides.

SndMidiRecHdrType

The `SndMidiRecHdrType` structure defines the fixed-size portion of a Palm OS MIDI record. (See `SndCallbackInfoType`.)

```
typedef struct SndMidiRecHdrType {
  UInt32  signature;
  UInt8   bDataOffset;
  UInt8   reserved;
} SndMidiRecHdrType;
```

Field Descriptions

`signature` Set to `sndMidiRecSignature`.

`bDataOffset` Offset from the beginning of the record to the Standard MIDI File data stream.

`reserved` Set to zero.

SndMidiRecType

The `SndMidiRecType` structure defines a variable-length header precedes the actual MIDI data in a Palm OS MIDI record. It consists of a fixed-size MIDI record header followed by the name of the MIDI track.

```
typedef struct SndMidiRecType {
  SndMidiRecHdrType  hdr;
  Char               name[1];
} SndMidiRecType;
```

Field Descriptions

`hdr` Fixed-size portion of the Palm OS MIDI record header. See `SndMidiRecHdrType`.

`name` Track name: 1 or more characters including null terminator. The length of name, including null terminator, must not be greater than `sndMidiNameLength`. The null character must always be provided, even for tracks that have no name.

SndSmfCallbacksType

The `SndSmfCallbacksType` structure is passed as the value of the `callbacksP` parameter to the `SndPlaySmf` function.

```
typedef struct SndSmfCallbacksType {
  SndCallbackInfoType  completion;
  SndCallbackInfoType  blocking;
  SndCallbackInfoType  reserved;
} SndSmfCallbacksType;
```

Field Descriptions

`completion` Completion callback function (see `SndComplFuncType`).

`blocking` Blocking hook callback function (see `SndBlockingFuncType`).

`reserved` Reserved. Set to 0 before passing.

SndSmfChanRangeType

This `SndSmfChanRangeType` structure is passed as the value of the `chanRangeP` parameter to the `SndPlaySmf` function. It specifies a range of enabled channels. Events for channels outside this range are ignored.

If this structure is not passed, all channels in the track are ignored.

```
typedef struct SndSmfChanRangeType {
  UInt8   bFirstChan;
  UInt8   bLastChan;
} SndSmfChanRangeType;
```

Field Descriptions

`bFirstChan` First MIDI channel (0-15 decimal).

`bLastChan` Last MIDI channel (0-15 decimal).

SndSmfOptionsType

The `SndSmfOptionsType` structure is passed as the value of the `selP` parameter to the `SndPlaySmf` function.

```
typedef struct SndSmfOptionsType {
  UInt32       dwStartMilliSec;
  UInt32       dwEndMilliSec;
  UInt16      amplitude;
  Boolean     interruptible;
  UInt8       reserved1;
  UInt32      reserved;
} SndSmfOptionsType;
```

Field Descriptions

`dwStartMilliSec`	Position at which to begin playback, expressed as number of milliseconds from beginning of the track. 0 means start from the beginning of the track. This field is used as input for `sndSmfCmdPlay` and output for `sndSmfCmdDuration`.
`dwEndMilliSec`	Position at which to stop playback, expressed as number of milliseconds from beginning of track. `sndSmfPlayAllMilliSec` means play the entire track; the default is to play the entire track if this structure is not passed in. This field is used as input for `sndSmfCmdPlay` and output for `sndSmfCmdDuration`.
`amplitude`	Used only for `sndSmfCmdPlay`. Specifies relative volume. Possible values range from 0 to `sndMaxAmp`, inclusively. The default is `sndMaxAmp` if this structure is not passed in. If 0, the sound is not played and the call returns immediately.
`interruptible`	Used only for `sndSmfCmdPlay`. If `true`, sound play will be interrupted if user interacts with the controls (digitizer, buttons, etc.) even if the interaction does not generate a sound command. If `false`, playback is not interrupted. The default behavior is `true` if this structure is not passed in.
`reserved1`	Reserved for future use.
`reserved`	Reserved. Set to 0 before passing.

Sound Manager Functions

SndCreateMidiList

Purpose Generate a list of MIDI records having a specified creator.

Prototype

```
Boolean SndCreateMidiList (UInt32 creator,
Boolean multipleDBs, UInt16* wCountP,
MemHandle *entHP)
```

Parameters

`->creator`	Creator of database in which to find MIDI records. Pass `0` for wildcard.
`->multipleDBs`	Pass `true` to search multiple databases for MIDI records. Pass `false` to search only in the first database found that meets search criteria.
`<->wCountP`	When the function returns, contains the number of MIDI records found.
`<->entHP`	When the function returns, this handle holds a a memory chunk containing an array of `SndMidiListItemType` if MIDI records were found.

Result Returns `false` if no MIDI records were found, `true` if MIDI records were found. When this function returns `true`, it updates the `wCountP` parameter to hold the number of MIDI records found and updates the `entHP` parameter to hold a handle to an array of `SndMidiListItemType` structures. Each record of this type holds the name, record ID, database ID, and card number of a MIDI record. The order of entries in `entHP` is not specified and should not be relied upon.

Comments This function is useful for displaying lists of sounds residing on the Palm™ device as MIDI records.

Compatibility Implemented only if `3.0 New Feature Set` is present.

See Also `DmFindRecordByID`, `DmOpenDatabase`, `DmQueryRecord`, `DmOpenDatabaseByTypeCreator`,"Rock Music" sample code

SndDoCmd

Purpose Send a sound manager command to a specified sound channel.

Prototype

```
Err SndDoCmd (void* channelP, SndCommandPtr cmdP,
Boolean noWait)
```

Parameters

-> `channelP`	Pointer to sound channel. Present implementation doesn't support multiple channels. Must be `NULL`. Passing `NULL` for the channel pointer causes the command to be sent to the shared sound channel; currently, this is the only option.
-> `cmdP`	Pointer to a `SndCommandType` holding a parameter block that specifies the note to play, its duration, and amplitude.
-> `noWait`	Because asynchronous mode is not yet supported for all commands, you must pass `0` for this value. In the future, pass `0` to await completion (synchronous) and pass a nonzero value to specify immediate return (asynchronous).

Result

0	No error.
`sndErrBadParam`	Invalid parameter.
`sndErrBadChannel`	Invalid channel pointer.
`sndErrQFull`	Sound queue is full.

Comments This function is useful for simple sound playback applications, such as playing a single note to provide user feedback. In addition to providing the same behavior it did in versions 1.0 and 2.0 of Palm OS, (specify the frequency, duration, and amplitude of a single note to be played) new command selectors provided in Palm OS 3.0 and higher allow you to use MIDI values to specify pitch, duration, and amplitude of the note to play, and to stop the note currently being played.

Compatibility Commands (see SndCmdIDType) other than sndCmdFreqDurationAmp are implemented only if 3.0 New Feature Set is present.

See Also SndPlaySmf

SndGetDefaultVolume

Purpose Return default sound volume levels cached by sound manager.

Prototype

```
void SndGetDefaultVolume (UInt16* alarmAmpP,
UInt16* sysAmpP, UInt16* masterAmpP)
```

Parameters

<-> alarmAmpP	Pointer to storage for alarm amplitude.
<-> sysAmpP	Pointer to storage for system sound amplitude.
<-> masterAmpP	Pointer to storage for master amplitude.

Result Returns nothing.

Comments Any pointer arguments may be passed as NULL. In that case, the corresponding setting is not returned.

New SndInterruptSmfIrregardless

Purpose Interrupts any currently playing SMF file, even if the interruptible flag is false.

Prototype

```
Err SndInterruptSmfIrregardless (void)
```

Parameters None.

Result Returns errNone.

Comments This function is similar to the SndPlaySmfIrregardless and SndPlaySmfResourceIrregardless functions except that it doesn't play a sound after interrupting the current sound.

Implemented only if 4.0 New Feature Set is present.

See Also SndPlaySmf, SndPlaySmfResourceIrregardless

SndPlaySmf

Purpose Performs the operation specified by the `cmd` parameter: play the specified standard MIDI file (SMF) or return the number of milliseconds required to play the entire SMF.

Prototype

```
Err SndPlaySmf (void* chanP, SndSmfCmdEnum cmd,
UInt8* smfP, SndSmfOptionsType* selP,
SndSmfChanRangeType* chanRangeP,
SndSmfCallbacksType* callbacksP, Boolean bNoWait)
```

Parameters

`chanP` The sound channel used to play the sound. This value must always be `NULL` because current versions of Palm OS provide only one sound channel that all applications share.

`cmd` The operation to perform, as specified by one of the following selectors:

`sndSmfCmdPlay`
Play the selection synchronously.

`sndSmfCmdDuration`
Return the duration of the entire SMF, expressed in milliseconds.

`-> smfP` Pointer to the SMF data in memory. This pointer can reference a valid SndCallbackInfoType followed by MIDI data, or it can point directly to the beginning of the SMF data.

-> selP	NULL or a pointer to a SndSmfOptionsType specifying options for playback volume, position in the SMF from which to begin playback, and whether playback can be interrupted by user interaction with the display. See the SndSmfOptionsType for the default behavior specified by a NULL value.
-> chanRangeP	NULL or a pointer to a SndSmfChanRangeType specifying a continuous range of MIDI channels 0 -15 to use for playback. If this value is NULL, all tracks are played.
-> callbacksP	NULL or a pointer to a SndSmfCallbacksType that holds your callback functions. Functions of type SndBlockingFuncType execute periodically while a note is playing, and functions of type SndComplFuncType execute after playback of the SMF completes. For more information, see the "Application-Defined Functions" section.
bNoWait	This value is ignored. This function always finishes playing the SMF selection before returning; however, you can execute a callback function while the SMF is playing.

Result Returns 0 if no error. When an error occurs, this function returns one of the following values; for more information see the SoundMgr.h file included with the Palm OS SDK:

sndErrBadParam	Bogus value passed to this function.
sndErrBadChannel	Invalid sound channel.
sndErrMemory	Insufficient memory.
sndErrOpen	Tried to open channel that's already open.
sndErrQFull	Can't accept more notes.
sndErrQEmpty	Internal use; never returned to applications.

`sndErrFormat`	Unsupported data format.
`sndErrBadStream`	Invalid data stream.
`sndErrInterrupted`	Play was interrupted.

Comments Although this call is synchronous, a callback function can be called while a note is playing. If the callback does not return before the number of system ticks required to play the current sound have elapsed, the next note in the SMF will not start on time.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `SndInterruptSmfIrregardless`, `SndPlaySmfIrregardless`, `SndPlaySmfResource`, `SndDoCmd`, `SndCreateMidiList`



New SndPlaySmfIrregardless

Purpose Performs the operation specified by the `cmd` parameter: play the specified standard MIDI file (SMF) or return the number of milliseconds required to play the entire SMF. Unlike `SndPlaySmf`, this function interrupts any currently playing SMF file to play the new sound, even if the interruptible flag was `false` when the currently playing SMF file was started.

Prototype

```
Err SndPlaySmfIrregardless (void *chanP,
SndSmfCmdEnum cmd, UInt8 *smfP,
SndSmfOptionsType *selP,
SndSmfChanRangeType *chanRangeP,
SndSmfCallbacksType *callbacksP, Boolean bNoWait)
```

Parameters

`chanP`	The sound channel used to play the sound. This value must always be `NULL` because current versions of Palm OS provide only one sound channel that all applications share.

`cmd`	The operation to perform, as specified by one of the following selectors:
	`sndSmfCmdPlay`
	Play the selection synchronously.
	`sndSmfCmdDuration`
	Return the duration of the entire SMF, expressed in milliseconds.

`-> smfP`	Pointer to the SMF data in memory. This pointer can reference a valid `SndCallbackInfoType` followed by MIDI data, or it can point directly to the beginning of the SMF data.
`-> selP`	`NULL` or a pointer to a `SndSmfOptionsType` specifying options for playback volume, position in the SMF from which to begin playback, and whether playback can be interrupted by user interaction with the display. See the `SndSmfOptionsType` for the default behavior specified by a `NULL` value.
`-> chanRangeP`	`NULL` or a pointer to a `SndSmfChanRangeType` specifying a continuous range of MIDI channels 0 -15 to use for playback. If this value is `NULL`, all tracks are played.
`-> callbacksP`	`NULL` or a pointer to a `SndSmfCallbacksType` that holds your callback functions. Functions of type `SndBlockingFuncType` execute periodically while a note is playing, and functions of type `SndComplFuncType` execute after playback of the SMF completes. For more information, see the "Application-Defined Functions" section.

`bNoWait`	This value is ignored. This function always finishes playing the SMF selection before returning; however, you can execute a callback function while the SMF is playing.

Result Returns `0` if no error. When an error occurs, this function returns one of the following values; for more information see the `SoundMgr.h` file included with the Palm OS SDK:

`sndErrBadParam`	Bogus value passed to this function.
`sndErrBadChannel`	Invalid sound channel.
`sndErrMemory`	Insufficient memory.
`sndErrOpen`	Tried to open channel that's already open.
`sndErrQFull`	Can't accept more notes.
`sndErrQEmpty`	Internal use; never returned to applications.
`sndErrFormat`	Unsupported data format.
`sndErrBadStream`	Invalid data stream.
`sndErrInterrupted`	Play was interrupted.

Comments Although this call is synchronous, a callback function can be called while a note is playing. If the callback does not return before the number of system ticks required to play the current sound have elapsed, the next note in the SMF will not start on time.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `SndInterruptSmfIrregardless`, `SndPlaySmf`, `SndPlaySmfResourceIrregardless`, `SndDoCmd`, `SndCreateMidiList`

SndPlaySmfResource

Purpose Plays a MIDI sound read out of an open resource database.

Prototype

```
Err SndPlaySmfResource (UInt32 resType,
Int16 resID,
SystemPreferencesChoice volumeSelector)
```

Parameters

-> `resType` SMF resource type.

-> `resID` SMF resource ID.

-> `volumeSelector` Volume setting to use. The volume is read out of the system preferences. Possible values for this parameter are:

```
prefSysSoundVolume
prefGameSoundVolume
prefAlarmSoundVolume
```

Result Returns 0 upon success, otherwise one of the following:

`sndErrBadParam`	The `volumeSelector` parameter has an invalid value or the SMF resource has invalid data.
`dmErrCantFind`	The specified resource does not exist on the specified card.

or any error code returned by `SndPlaySmf` .

Comments This is a convenience function to be used in place of `SndPlaySmf`. It plays an SMF sound stored in a resource database. This function plays the entire sound on all MIDI channels at the volume specified and allows the sound to be interrupted by a key down event or a digitizer event. No callbacks are specified.

Compatibility Implemented only if 3.2 New Feature Set is present.

See Also `SndPlaySmfResourceIrregardless`



New SndPlaySmfResourceIrregardless

Purpose

Plays a MIDI sound read out of an open resource database. Unlike SndPlaySmfResource, this function interrupts any currently playing SMF file to play the new sound, even if the interruptible flag was `false` when the currently playing SMF file was started.

Prototype

```
Err SndPlaySmfResourceIrregardless
(UInt32 resType, Int16 resID,
SystemPreferencesChoice volumeSelector)
```

Parameters

-> `resType` SMF resource type.

-> `resID` SMF resource ID.

-> `volumeSelector`
Volume setting to use. The volume is read out of the system preferences. Possible values for this parameter are:

```
prefSysSoundVolume
prefGameSoundVolume
prefAlarmSoundVolume
```

Result

Returns 0 upon success, otherwise one of the following:

`sndErrBadParam`	The `volumeSelector` parameter has an invalid value or the SMF resource has invalid data.
`dmErrCantFind`	The specified resource does not exist on the specified card.

or any error code returned by SndPlaySmf .

Comments

This is a convenience function to be used in place of SndPlaySmfIrregardless. It plays an SMF sound stored in a resource database. This function plays the entire sound on all MIDI channels at the volume specified and allows the sound to be interrupted by a key down event or a digitizer event. No callbacks are specified.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also SndInterruptSmfIrregardless, SndPlaySmfResource

SndPlaySystemSound

Purpose Play a standard system sound.

Prototype
```
void SndPlaySystemSound (SndSysBeepType beepID)
```

Parameters -> `beepID` System sound to play.

Result Returns nothing.

Comments The `SndSysBeepType` enum is defined in `SoundMgr.h` as follows:

```
typedef enum SndSysBeepType {
  sndInfo = 1,
  sndWarning,
  sndError,
  sndStartUp,
  sndAlarm,
  sndConfirmation,
  sndClick
  } SndSysBeepType;
```

Note that in versions of Palm OS prior to 3.0, all of these sounds were synchronous and blocking. In Palm OS 3.0 and higher, `sndAlarm` still blocks, but the rest of these system sounds are implemented asynchronously.

The `SndPlaySystemSound` function uses the system volume setting when playing the sound.

SndSetDefaultVolume

Purpose Sets the default sound volume levels cached by sound manager.

Prototype
```
void SndSetDefaultVolume (UInt16* alarmAmpP,
UInt16* sysAmpP, UInt16* defAmpP)
```

Parameters

<-> `alarmAmpP`	Pointer to storage for alarm amplitude.
<-> `sysAmpP`	Pointer to storage for system sound amplitude.
<-> `defAmpP`	Pointer to storage for default amplitude.

Result Returns nothing.

Comments Any pointer arguments may be passed as `NULL`. In that case, the corresponding setting is not altered.

Application-Defined Functions

This section describes callback functions to be executed by the `SndPlaySmf` function.

SndComplFuncType

Purpose Executed after playback of the SMF completes.

Prototype
```
void SndComplFuncType (void* chanP,
UInt32 dwUserData)
```

Parameters

-> `chanP`	The sound channel used to play the sound. This value must always be `NULL` because current versions of Palm OS provide only one sound channel that all applications share.
-> `dwUserData`	Application-defined data that this function needs to access, or `NULL`.

Result Returns nothing.

See Also `SndSmfCallbacksType`

SndBlockingFuncType

Purpose A non-blocking callback function that is executed periodically during playback of the SMF.

Prototype

```
Boolean SndBlockingFuncType (void* chanP,
UInt32 dwUserData, Int32 sysTicksAvailable)
```

Parameters

-> `chanP` The sound channel used to play the sound. This value must always be `NULL` because current versions of Palm OS provide only one sound channel that all applications share.

-> `dwUserData` Application-defined data that this function needs to access.

-> `sysTicksAvailable` The maximum amount of time available for completion of this function, or `NULL`.

Result Returns `true` to continue playback, or `false` to cancel playback.

Comments Suggested uses for this function include updating the user interface or checking for user input.

See Also `SndSmfCallbacksType`, `SndPlaySmf`

47

Standard IO

This chapter provides reference material for the standard IO API:

- Standard IO Functions
- Standard IO Provider Functions
- Application-Defined Function

The header files `StdIOPalm.h` and `StdIOProvider.h` declare the standard IO API. For more information on using the standard IO API, see the chapter "Standard IO Applications"in the *Palm OS Programmer's Companion*, vol. I.

Standard IO Functions

The macros and functions in this section enable standard IO.

fgetc

Purpose Macro that calls `Siofgetc` to return the next character from the input stream.

Prototype

```
fgetc (fs)
```

Parameters

-> `fs`	An input stream from which to read the next character. You can specify only the value `stdin` for this parameter; alternate streams are not currently implemented.

Result The next character from the input stream. The return value `EOF` indicates an error occurred.

fgets

Purpose Macro that calls `Siofgets` to return a string from the input stream.

Prototype

```
fgets (strP, maxChars, fs)
```

Parameters

<- `strP`	A pointer to the returned string.
-> `maxChars`	The number of characters to read from the input stream, plus one for the null terminator.
-> `fs`	An input stream from which to read the next character. You can specify only the value `stdin` for this parameter; alternate streams are not currently implemented.

Result A pointer to the string read from the input stream. If an error or EOF occurs before any characters are read, returns `NULL`.

Comments The returned string is always terminated by a null character.

fprintf

Purpose Macro that calls `Siofprintf` to write formatted output to an output stream.

Prototype

```
fprintf (fs, formatP, ...)
```

Parameters

-> `fs`	An output stream to which to write the formatted output. You can specify only the value `stdout` for this parameter; alternate streams are not currently implemented.
-> `formatP`	A pointer to a format string that controls how subsequent arguments are converted for output.
-> `...`	Zero or more parameters to be formatted as specified by the `formatP` string.

Result Returns the number of characters written out (not including the null terminator used to end output strings). Returns a negative number if there is an error.

Comments This function internally calls `StrVPrintF` to do the formatting. See that function for details on which format specifications are supported.

fputc

Purpose Macro that calls `Siofputc` to write a character to the output stream.

Prototype

```
fputc (c, fs)
```

Parameters

-> `c`	A character to write to the output stream.
-> `fs`	An output stream to which to write the character. You can specify only the value `stdout` for this parameter; alternate streams are not currently implemented.

Result The character that was written. If an error occurs, the value `EOF` is returned.

fputs

Purpose Macro that calls `Siofputs` to write a string to the output stream.

Prototype

```
fputs (strP, fs)
```

Parameters

-> `strP`	A pointer to the string to write.
-> `fs`	An output stream to which to write the string. You can specify only the value `stdout` for this parameter; alternate streams are not currently implemented.

Result Returns 0 on success and the value `EOF` on error.

getchar

Purpose Macro that calls `Siofgetc` to read the next character from the `stdin` input stream.

Prototype

```
getchar ()
```

Result The next character from the input stream. The return value `EOF` indicates an error occurred.

gets

Purpose Macro that calls `Siogets` to read a string from the `stdin` input stream.

Prototype

```
gets (strP)
```

Parameters

<- `strP`	A pointer to the returned string.

Result A pointer to the string read from the input stream. If an error or EOF occurs before any characters are read, returns `NULL`.

Comments The returned string does not include a null terminator. You must ensure that the input line, if any, is sufficiently short to fit in the string.

printf

Purpose Macro that calls `Sioprintf` to write formatted output to the stdout output stream.

Prototype

```
printf (formatP, ...)
```

Parameters

-> `formatP`	A pointer to a format string that controls how subsequent arguments are converted for output.
-> ...	Zero or more parameters to be formatted as specified by the `formatP` string.

Result Returns the number of characters written out (not including the null terminator used to end output strings).

Comments This function internally calls `StrVPrintF` to do the formatting. See that function for details on which format specifications are supported. Returns a negative number if there is an error.

putc

Purpose Macro that calls `Siofputc` to write a character to the output stream.

Prototype

```
putc (c, fs)
```

Parameters

-> `c`	A character to write to the output stream.
-> `fs`	An output stream to which to write the character. You can specify only the value `stdout` for this parameter; alternate streams are not currently implemented.

Result The character that was written. If an error occurs, the value `EOF` is returned.

putchar

Purpose Macro that calls `Siofputc` to write a character to the `stdout` output stream.

Prototype

```
putchar (c)
```

Parameters

-> `c`	A character to write to the `stdout` output stream.

Result The character that was written. If an error occurs, the value `EOF` is returned.

puts

Purpose Macro that calls `Sioputs` to write a string to the output stream `stdout`.

Prototype
```
puts (strP)
```

Parameters

-> `strP`	A pointer to the string to write to `stdout`.

Result Returns a nonnegative value on success and the value `EOF` on error.

SioAddCommand

Purpose Adds a built-in command that is supplied by the standard IO provider application.

Prototype
```
void SioAddCommand (const Char *cmdStr,
SioMainProcPtr cmdProcP)
```

Parameters

-> `cmdStr`	Pointer to a string that is the command name.
-> `cmdProcP`	Pointer to the command entry point function (the `SioMain` function).

Result Returns nothing.

Comments This routine is useful for registering a command that is inside the standard IO provider application instead of in its own database.

This routine must be used to test commands under the Simulator since it can't launch application databases.

Siofgetc

Return the next character from the input stream.

Prototype

```
Int16 Siofgetc (FILE *fs)
```

Parameters

-> `fs`	An input stream from which to read the next character. You can specify only the value `stdin` for this parameter; alternate streams are not currently implemented.

Result The next character from the input stream. The return value `EOF` indicates an error occurred.

See Also `fgetc`

Siofgets

Purpose Return a string from the input stream.

Prototype

```
Char *Siofgets (Char *strP, UInt16 maxChars,
FILE *fs)
```

Parameters

<- `strP`	A pointer to the returned string.
-> `maxChars`	The number of characters to read from the input stream, plus one for the null terminator.
-> `fs`	An input stream from which to read the next character. You can specify only the value `stdin` for this parameter; alternate streams are not currently implemented.

Result A pointer to the string read from the input stream. If an error or EOF occurs before any characters are read, returns `NULL`.

Comments The returned string is always terminated by a null character.

See Also `fgets`

Siofprintf

Purpose Write formatted output to an output stream.

Prototype
```
Int16 Siofprintf (FILE *fs, const Char *formatP,
...)
```

Parameters

-> `fs`	An output stream to which to write the formatted output. You can specify only the value `stdout` for this parameter; alternate streams are not currently implemented.
-> `formatP`	A pointer to a format string that controls how subsequent arguments are converted for output.
-> ...	Zero or more parameters to be formatted as specified by the `formatP` string.

Result Returns the number of characters written out (not including the null terminator used to end output strings). Returns a negative number if there is an error.

Comments This function internally calls `StrVPrintF` to do the formatting. See that function for details on which format specifications are supported.

See Also `fprintf`

Siofputc

Purpose Write a character to the output stream.

Prototype
```
Int16 Siofputc (Int16 c, FILE *fs)
```

Parameters

-> `c`	A character to write to the output stream.

	-> `fs`	An output stream to which to write the character. You can specify only the value `stdout` for this parameter; alternate streams are not currently implemented.

Result The character that was written. If an error occurs, the value `EOF` is returned.

See Also `fputc`

Siofputs

Purpose Write a string to the output stream.

Prototype

```
Int16 Siofputs (const Char *strP, FILE *fs)
```

Parameters

	-> `strP`	A pointer to the string to write.
	-> `fs`	An output stream to which to write the string. You can specify only the value `stdout` for this parameter; alternate streams are not currently implemented.

Result Returns 0 on success and the value `EOF` on error.

See Also `fputs`

Siogets

Purpose Read a string from the `stdin` input stream.

Prototype

```
Char *Siogets (Char *strP)
```

Parameters

	<- `strP`	A pointer to the returned string.

Result A pointer to the string read from the input stream. If an error or EOF occurs before any characters are read, returns `NULL`.

Comments The returned string does not include a null terminator. You must ensure that the input line, if any, is sufficiently short to fit in the string.

See Also gets

Sioprintf

Purpose Write formatted output to the stdout output stream.

Prototype
```
Int16 Sioprintf (const Char *formatP, ...)
```

Parameters

-> formatP	A pointer to a format string that controls how subsequent arguments are converted for output.
-> ...	Zero or more parameters to be formatted as specified by the formatP string.

Result Returns the number of characters written out (not including the null terminator used to end output strings).

Comments This function internally calls StrVPrintF to do the formatting. See that function for details on which format specifications are supported. Returns a negative number if there is an error.

See Also printf

Sioputs

Purpose Write a string to the output stream stdout.

Prototype
```
Int16 Sioputs (const Char *strP)
```

Parameters

-> strP	A pointer to the string to write to stdout.

Result Returns a nonnegative value on success and the value EOF on error.

See Also puts

Siosystem

Purpose Execute another Stdio command.

Prototype
```
Int16 Siosystem (const Char *cmdStrP)
```

Parameters

-> `cmdStrP`	A pointer to a string containing the command line to execute.

Result Returns a value >= 0 on success or < 0 on failure.

Comments This function first looks for a built-in command with the specified name. If none is found, it looks for a Stdio application database with the name "Cmd-*cmdname*" where *cmdname* is the first word in the command string `cmdStrP`.

See Also `SioExecCommand`, `system`

Siovfprintf

Purpose Write formatted output to the `stdout` output stream.

Prototype
```
Int16 Siovfprintf (FILE *fs, const Char *formatP,
_Palm_va_list args)
```

Parameters

-> `fs`	An output stream to which to write the formatted output. You can specify only the value `stdout` for this parameter; alternate streams are not currently implemented.
-> `formatP`	A pointer to a format string that controls how subsequent arguments are converted for output.
-> `args`	A pointer to a list of zero or more parameters to be formatted as specified by the `formatP` string.

Result Returns the number of characters written out (not including the null terminator used to end output strings). Returns a negative number if there is an error.

Comments This function internally calls `StrVPrintF` to do the formatting. See that function for details on which format specifications are supported.

See Also `vfprintf`

sprintf

Purpose Macro that calls `StrPrintF` to write formatted output to the stdout output stream.

Prototype

```
sprintf (formatP, ...)
```

Parameters

-> `formatP`	A pointer to a format string that controls how subsequent arguments are converted for output.
-> ...	Zero or more parameters to be formatted as specified by the `formatP` string.

Result Returns the number of characters written out (not including the null terminator used to end output strings).

Comments See `StrVPrintF` for details on which format specifications are supported. Returns a negative number if there is an error.

system

Purpose Macro that calls `Siosystem` to execute another Stdio command.

Prototype

```
system (cmdStrP)
```

Parameters

-> `cmdStrP`	A pointer to a string containing the command line to execute.

Result Returns a value >= 0 on success or < 0 on failure.

Comments This function first looks for a built-in command with the specified name. If none is found, it looks for a Stdio application database with the name "Cmd-*cmdname*" where *cmdname* is the first word in the command string `cmdStrP`.

See Also `SioExecCommand`

vfprintf

Purpose Macro that calls `Siovfprintf` to write formatted output to the stdout output stream.

Prototype

```
vfprintf (fs, formatP, args)
```

Parameters

-> `fs`	An output stream to which to write the formatted output. You can specify only the value `stdout` for this parameter; alternate streams are not currently implemented.
-> `formatP`	A pointer to a format string that controls how subsequent arguments are converted for output.
-> `args`	A pointer to a list of zero or more parameters to be formatted as specified by the `formatP` string.

Result Returns the number of characters written out (not including the null terminator used to end output strings). Returns a negative number if there is an error.

Comments This function internally calls `StrVPrintF` to do the formatting. See that function for details on which format specifications are supported.

vsprintf

Purpose Macro that calls StrVPrintF to write formatted output to the stdout output stream.

Prototype
```
vsprintf (fs, formatP, args)
```

Parameters

-> `fs`	An output stream to which to write the formatted output. You can specify only the value `stdout` for this parameter; alternate streams are not currently implemented.
-> `formatP`	A pointer to a format string that controls how subsequent arguments are converted for output.
-> `args`	A pointer to a list of zero or more parameters to be formatted as specified by the `y` string.

Result Returns the number of characters written out (not including the null terminator used to end output strings). Returns a negative number if there is an error.

Comments See StrVPrintF for details on which format specifications are supported.

Standard IO Provider Functions

These functions are used by a standard IO provider application.

SioClearScreen

Purpose Clears the entire standard IO output field.

Prototype
```
void SioClearScreen (void)
```

Parameters None.

Result Returns nothing.

SioExecCommand

Purpose Executes a command line.

Prototype

```
Int16 SioExecCommand (const Char *cmd)
```

Parameters

-> `cmd`	A pointer to a string containing the command line to execute.

Result Returns a value >= 0 on success or < 0 on failure.

Comments This function first looks for a built-in command with the specified name. If none is found, it looks for a Stdio application database with the name "Cmd-*cmdname*" where *cmdname* is the first word in the command string `cmd`.

If you pass the string “help” or “?” for the `cmd` parameter, `SioExecCommand` causes a help string to be printed for each built-in command. It actually executes each built-in command, passing the string “?” as `argv[1]`. Each command should handle this argument by printing a help line.

The `SioExecCommand` function is faster than calling `system` to execute a command. However, `SioExecCommand` can be called only by the standard IO provider application, not the standard IO application.

SioFree

Purpose Closes down the standard IO manager.

Prototype

```
Err SioFree (void)
```

Parameters None.

Result Returns 0 on success.

SioHandleEvent

Purpose Handles an event in the form that contains the standard IO output field and scroll arrows if the event belongs to the text field or scroll arrows.

Prototype
```
Boolean SioHandleEvent (SysEventType *event)
```

Parameters

-> `event`	Pointer to an `EventType` structure.

Result Returns `true` if the event was handled and should not be processed by the application's own form event handler; returns `false` otherwise.

Comments This function must be called from the form event handler before it does its own processing with any of the objects unrelated to standard IO in the form.

SioInit

Purpose Initializes the standard IO manager.

Prototype
```
Err SioInit (UInt16 formID, UInt16 fieldID,
UInt16 scrollerID)
```

Parameters

-> `formID`	The ID of the form that contains the input/output field.
-> `fieldID`	The ID of the field to be used for input/output.
-> `scrollerID`	The ID of the scroller associated with the input/output form.

Result Returns 0 on success.

Application-Defined Function

You must supply this function in your stdio application.

SioMain

Purpose The main entry point for the stdio application.

Prototype

```
Int16 SioMain (UInt16 argc, const Char *argv[])
```

Parameters

-> `argc`	The number of parameters passed on the command line.
-> `argv`	An array of character pointers, one for each parameter passed on the command line.

Result The return value from this routine is passed back to the system call that invoked it. Return 0 for no error.

48

String Manager

This chapter provides reference material for the string manager. The string manager API is declared in the header file `StringMgr.h`.

For more information, see the section "Text" on page 137 in the *Palm OS Programmer's Companion*, vol. I.

String Manager Functions

StrAToI

Purpose Convert a string to an integer.

Prototype
```
Int32 StrAToI (const Char *str)
```

Parameters

-> `str`	Pointer to a string to convert.

Result Returns the integer.

Comments Use this function instead of the standard `atoi` routine.

StrCaselessCompare

Purpose Compare two strings with case and accent insensitivity.

Prototype
```
Int16 StrCaselessCompare (const Char *s1,
const Char *s2)
```

Parameters

-> `s1`	Pointer to a string.
-> `s2`	Pointer to a string.

Result Returns 0 if the strings match.

Returns a positive number if `s1` > `s2`.

Returns a negative number if `s1` < `s2`.

Comments Use this function instead of the standard `stricmp` routine.

To support systems that use multi-byte character encodings, consider using `TxtCaselessCompare` instead of this function (or `TxtCompare` for a case-sensitive comparison). Both functions can match single-byte characters with their multi-byte equivalents, but `TxtCaselessCompare` can also return the length of the matching text.

See Also `StrNCaselessCompare`, `TxtCaselessCompare`, `StrCompare`, `StrNCompare`

StrCat

Purpose Concatenate one null-terminated string to another.

Prototype

```
Char *StrCat (Char *dst, const Char *src)
```

Parameters

-> `dst`	Pointer to the null-terminated destination string.
-> `src`	Pointer to the null-terminated source string.

Result Returns a pointer to the destination string.

Comments Use this function instead of the standard `strcat` routine.

StrChr

Purpose Look for a character within a string.

Prototype

```
Char *StrChr (const Char *str, WChar chr)
```

Parameters

-> `str`	Pointer to the string to be searched.
-> `chr`	Character to search for.

Result Returns a pointer to the first occurrence of character in `str`. Returns `NULL` if the character is not found.

Comments Use this function instead of the standard `strchr` routine.

This function can handle both single-byte characters and multi-byte characters correctly. However, you should make sure that you pass a `WChar` variable to `StrChr` instead of a `Char`. If you pass a `Char` variable, the function sign-extends the variable to a `WChar`, which causes problems if the value is 0x80 or higher.

Compatibility This routine does not correctly find a '\0' character on Palm OS® version 1.0.

See Also `StrStr`

StrCompare

Purpose Compare two strings.

Prototype

```
Int16 StrCompare (const Char *s1, const Char *s2)
```

Parameters

-> `s1`	Pointer to a string.
-> `s2`	Pointer to a string.

Result Returns 0 if the strings match.

Returns a positive number if `s1` sorts after `s2` alphabetically.

Returns a negative number if `s1` sorts before `s2` alphabetically.

Comments Use this function or `StrCompareAscii` instead of the standard `strcmp` routine. This function is case sensitive.

To support systems that use multi-byte character encodings, consider using `TxtCompare` instead of this function. Both functions can match single-byte characters with their multi-byte equivalents, but `TxtCompare` can also return the length of the matching text.

Compatibility Prior to Palm OS 4.0, `StrCompare` and `TxtCompare` only performed one level of comparison and returned as soon as they found two unequal characters. For example, if you compared the string "celery" with the string "Cauliflower," both functions returned a value indicating that "celery" should appear before "Cauliflower" because they sorted "c" before "C."

In Palm OS 4.0, `StrCompare` calls `TxtCompare`, and `TxtCompare` performs a comparison using up to six comparison tables for sorting with increasing precision. As a result, in Palm OS 4.0 and higher, `StrCompare` sorts "Cauliflower" before "celery."

See Also `StrNCompare`, `StrNCaselessCompare`, `TxtCaselessCompare`



New StrCompareAscii

Purpose Compare two ASCII strings.

Prototype

```
Int16 StrCompareAscii (const Char *s1,
const Char *s2)
```

Parameters

-> `s1`	Pointer to a string.
-> `s2`	Pointer to a string.

Result Returns 0 if the strings match.

Returns a positive number if `s1` sorts after `s2` alphabetically.

Returns a negative number if `s1` sorts before `s2` alphabetically.

Comments Use this function instead of the standard `strcmp` routine. Use it to do case-sensitive comparisons on strings that are guaranteed to be 7-bit ASCII strings. This function performs a fast, simple byte-to-byte comparison that is guaranteed never to change.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also StrCompare, StrNCompare, TxtCompare, StrCaselessCompare, StrNCaselessCompare, TxtCaselessCompare, StrNCompareAscii

StrCopy

Purpose Copy one string to another.

Prototype
```
Char *StrCopy (Char *dst, const Char *src)
```

Parameters

-> `dst` Pointer to the destination string.

-> `src` Pointer to the source string.

Result Returns a pointer to the destination string.

Comments Use this function instead of the standard `strcpy` routine.

This function does not work properly with overlapping strings.

StrDelocalizeNumber

Purpose Delocalize a number passed in as a string. Convert the number from any localized notation to US notation (decimal point and thousandth comma). The current thousand and decimal separators have to be passed in.

Prototype
```
Char *StrDelocalizeNumber (Char *s,
Char thousandSeparator, Char decimalSeparator)
```

Parameters

<-> `s` Pointer to the number as an ASCII string.

-> `thousandSeparator` Current thousand separator.

-> `decimalSeparator` Current decimal separator.

Result Returns a pointer to the changed number and modifies the string in `s`.

Comments The current `thousandSeparator` and `decimalSeparator` can be determined by obtaining the value of the `prefNumberFormat` preference using `PrefGetPreference` and then passing the returned `NumberFormatType` to `LocGetNumberSeparators`. For example:

```
Char *localizedNum;
NumberFormatType numFormat;
Char thousandsSeparator, decimalSeparator;

numFormat = (NumberFormatType)
  PrefGetPreference(prefNumberFormat);
LocGetNumberSeparators(numFormat,
  &thousandsSeparator, &decimalSeparator);
StrDelocalizeNumber(localizedNum,
  thousandsSeparator, decimalSeparator);
```

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `StrLocalizeNumber`, `LocGetNumberSeparators`

StrIToA

Purpose Convert an integer to ASCII.

Prototype

```
Char *StrIToA (Char *s, Int32 i)
```

Parameters

<- `s`	Pointer to a string of size `maxStrIToALen` in which to store the results.
-> `i`	Integer to convert.

Result Returns a pointer to the result string.

See Also `StrAToI`, `StrIToH`

StrIToH

Purpose Convert an integer to hexadecimal ASCII.

Prototype
```
Char *StrIToH (Char *s, UInt32 i)
```

Parameters

<- `s`	Pointer to a string in which to store the results.
-> `i`	Integer to convert.

Result Returns the string pointer `s`.

See Also `StrIToA`

StrLen

Purpose Compute the length of a string.

Prototype
```
Int16 StrLen (const Char *src)
```

Parameters

-> `src`	Pointer to a string.

Result Returns the length of the string in bytes.

Comments Use this function instead of the standard `strlen` routine.

This function returns the length of the string in bytes. On systems that support multi-byte characters, the number returned does not always equal the number of characters.

StrLocalizeNumber

Purpose Convert a number (passed in as a string) to localized format, using a specified thousands separator and decimal separator.

Prototype
```
Char *StrLocalizeNumber (Char *s,
Char thousandSeparator, Char decimalSeparator)
```

Parameters

<-> `s` Numeric ASCII string to localize.

-> `thousandSeparator`
Localized thousand separator.

-> `decimalSeparator`
Localized decimal separator.

Result Returns a pointer to the changed number. Converts the number string in `s` by replacing all occurrences of "," with `thousandSeparator` and all occurrences of "." with `decimalSeparator`.

Comments The current `thousandSeparator` and `decimalSeparator` can be determined by obtaining the value of the `prefNumberFormat` preference using `PrefGetPreference` and then passing the returned `NumberFormatType` to `LocGetNumberSeparators`. For example:

```
Char *localizedNum;
NumberFormatType numFormat;
Char thousandsSeparator, decimalSeparator;

numFormat = (NumberFormatType)
  PrefGetPreference(prefNumberFormat);
LocGetNumberSeparators(numFormat,
  &thousandsSeparator, &decimalSeparator);
StrLocalizeNumber(localizedNum,
  thousandsSeparator, decimalSeparator);
```

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `StrDelocalizeNumber`

StrNCaselessCompare

Purpose Compares two strings out to *n* characters with case and accent insensitivity.

Prototype

```
Int16 StrNCaselessCompare (const Char *s1,
const Char *s2, Int32 n)
```

Parameters

-> `s1`	Pointer to the first string.
-> `s2`	Pointer to the second string.
-> `n`	Length in bytes of the text to compare.

Result Returns 0 if the strings match.

Returns a positive number if `s1` > `s2`.

Returns a negative number if `s1` < `s2`.

Comments To support systems that use multi-byte character encodings, consider using `TxtCaselessCompare` instead of this function (or `TxtCompare` for a case-sensitive comparison). Both functions can match single-byte characters with their multi-byte equivalents, but `TxtCaselessCompare` can also return the length of the matching text.

Compatibility Implemented only if 2.0 New Feature Set is present. As of Palm OS 4.0, both `s1` and `s2` must be null-terminated strings.

See Also `StrNCompare`, `StrCaselessCompare`, `TxtCaselessCompare`, `StrCompare`

StrNCat

Purpose Concatenates one string to another clipping the destination string to a maximum of *n* bytes (including the null character at the end).

IMPORTANT: The Palm OS implementation of `StrNCat` differs from the implementation in the standard C library. See the Comments section for details.

Prototype

```
Char *StrNCat (Char *dst, const Char *src,
Int16 n)
```

Parameters

-> `dst`	Pointer to the null-terminated destination string.
-> `src`	Pointer to the source string.
-> `n`	Maximum length in bytes for `dst`, including the terminating null character.

Result Returns a pointer to the destination string.

Comment This function differs from the standard C `strncat` function in these ways:

- `StrNCat` treats the parameter `n` as the maximum length in bytes for `dst`. That means it will copy at most `n - StrLen(dst) - 1` bytes from `src`. The standard C function always copies `n` bytes from `src` into `dst`. (It copies the entire `src` into `dst` if the length of `src` is less than `n`).
- If the length of the destination string reaches `n - 1`, `StrNCat` stops copying bytes from `src` and appends the terminating null character to `dst`. If the length of the destination string is already greater than or equal to `n - 1` before the copying begins, `StrNCat` does not copy any data from `src`.
- In the standard C function, if `src` is less than `n`, the entire `src` string is copied into `dst` and then the remaining space is filled with null characters. `StrNCat` does not fill the remaining space with null characters in released ROMs. In debug ROMs, `StrNCat` fills the remaining bytes with the value `0xFE`.

On systems with multi-byte character encodings, this function makes sure that it does not copy part of a multi-byte character. If the last byte copied from `src` contains the high-order or middle byte of a multi-byte character, `StrNCat` backs up in `dst` until the byte after the end of the previous character, and replaces that byte with a null character.

Compatibility Implemented only if 2.0 New Feature Set is present.

StrNCompare

Purpose Compare two strings out to *n* bytes. This function is case and accent sensitive.

Prototype

```
Int16 StrNCompare (const Char *s1, const Char *s2,
Int32 n)
```

Parameters

-> `s1`	Pointer to a string.
-> `s2`	Pointer to a string.
-> `n`	Length in bytes of text to compare.

Result Returns 0 if the strings match.

Returns a positive number if `s1` > `s2`.

Returns a negative number if `s1` < `s2`.

Comments To support systems that use multi-byte character encodings, consider using `TxtCompare` instead of this function. Both functions can match single-byte characters with their multi-byte equivalents, but `TxtCompare` can also return the length of the matching text.

Compatibility Implemented only if 2.0 New Feature Set is present. As of Palm OS 4.0, both `s1` and `s2` must be null-terminated strings.

See Also `StrCompare`, `StrNCaselessCompare`, `StrCaselessCompare`, `TxtCaselessCompare`, `StrNCompareAscii`



New StrNCompareAscii

Purpose Compare two ASCII strings out to *n* bytes.

Prototype
```
Int16 StrNCompareAscii (const Char *s1,
const Char *s2, Int32 n)
```

Parameters

-> `s1`	Pointer to a string.
-> `s2`	Pointer to a string.
-> `n`	Length in bytes of text to compare.

Result Returns 0 if the strings match.

Returns a positive number if `s1` sorts after `s2` alphabetically.

Returns a negative number if `s1` sorts before `s2` alphabetically.

Comments Use this function instead of the standard `strncmp` routine. Use it to do case-sensitive comparisons on strings that are guaranteed to be 7-bit ASCII strings. This function performs a fast, simple byte-to-byte comparison that is guaranteed never to change.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `StrCompare`, `StrNCompare`, `TxtCompare`, `StrCaselessCompare`, `StrNCaselessCompare`, `TxtCaselessCompare`, `StrCompareAscii`

StrNCopy

Purpose Copies up to *n* bytes from a source string to the destination string. Terminates `dst` string at index *n*-1 if the source string length was *n*-1 or less.

Prototype
```
Char *StrNCopy (Char *dst, const Char *src,
Int16 n)
```

Parameters

-> `dst`	Pointer to the destination string.
-> `src`	Pointer to the source string.
-> `n`	Maximum number of bytes to copy from `src` string.

Result Returns nothing.

Comments On systems with multi-byte character encodings, this function makes sure that it does not copy part of a multi-byte character. If the `n`th byte of `src` contains the high-order or middle byte of a multi-byte character, `StrNCopy` backs up in `dst` until the byte after the end of the previous character, and replaces the remaining bytes (up to `n`-1) with nulls.

Be aware that the `n`th byte of `dst` upon return may contain the last byte of a multi-byte character. If you plan to terminate the string by setting its last character to NULL, you must not pass the entire length of the string to `StrNCopy`. If you do, your code may overwrite the final byte of the last character.

```
// WRONG! You may overwrite part of multi-byte
// character.
Char dst[n];
StrNCopy(dst, src, n);
dst[n-1] = chrNull;
```

Instead, if you write to the last byte of the destination string, pass one less than the size of the string to `StrNCopy`.

```
// RIGHT. Instead pass n-1 to StrNCopy.
Char dst[n];
StrNCopy(dst, src, n-1);
dst[n-1] = chrNull;
```

Compatibility Implemented only if 2.0 New Feature Set is present.

StrPrintF

Purpose Implements a subset of the ANSI C `sprintf` call, which writes formatted output to a string.

Prototype

```
Int16 StrPrintF (Char *s,
const Char *formatStr, ...)
```

Parameters

-> `s`	Pointer to a string into which the results are written.
-> `formatStr`	Pointer to the format specification string.
...	Zero or more arguments to be formatted as specified by `formatStr`.

Result Number of characters written to destination string. Returns a negative number if there is an error.

Comments This function internally calls `StrVPrintF` to do the formatting. See that function for details on which format specifications are supported.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `StrVPrintF`

StrStr

Purpose Look for a substring within a string.

Prototype

```
Char *StrStr (const Char *str, const Char *token)
```

Parameters

-> `str`	Pointer to the string to be searched.
-> `token`	Pointer to the string to search for.

Result Returns a pointer to the first occurrence of `token` in `str` or `NULL` if not found.

Comments Use this function instead of the standard `strstr` routine.

On systems with multi-byte character encodings, this function makes sure that it does not match only part of a multi-byte character. If the matching strings begins at an inter-character boundary, then this function returns `NULL`.

NOTE: If the value of the `token` parameter is the empty string, this function returns `NULL`. This is different than the standard `strstr` function, which returns `str` when `token` is the empty string.

See Also `StrChr`

StrToLower

Purpose Convert all the characters in a string to lowercase.

Prototype
```
Char *StrToLower (Char *dst, const Char *src)
```

Parameters

-> `dst`	Pointer to a string.
-> `src`	Pointer to a null-terminated string.

Result Returns a pointer to the destination string.

Compatibility Prior to Palm OS version 3.5, this function only converted accented characters on Japanese devices. On Palm OS version 3.5 and higher, all characters are appropriately lowercased, including accented characters on Latin devices.

StrVPrintF

Purpose Implements a subset of the ANSI C `vsprintf` call, which writes formatted output to a string.

Prototype

```
Int16 StrVPrintF (Char *s, const Char *formatStr,
va_list arg)
```

Parameters

<- `s`	Pointer to a string into which the results are written. This string is always terminated by a null terminator.
-> `formatStr`	Pointer to the format specification string.
-> `arg`	Pointer to a list of zero or more parameters to be formatted as specified by the `formatStr` string.

Result Number of characters written to destination string, not including the null terminator. Returns a negative number if there is an error.

Comments Like the C `vsprintf` function, this function is designed to be called by your own function that takes a variable number of arguments and passes them to this function. For details on how to use it, see "Using the StrVPrintF Function" on page 138 in *Palm OS Programmer's Companion*, vol. I, or refer to `vsprintf` in a standard C reference book.

Currently, only the conversion specifications `%d`, `%i`, `%u`, `%x`, `%s`, and `%c` are implemented by `StrVPrintF` (and related functions). Optional modifiers that are supported include: `+`, `-`, <space>, `*`, <digits>, `h` and `l` (long). Following is a brief description of how these format specifications work (see a C book for more details).

Each conversion specification begins with the % character. Following the % character, there may be one or more of the characters list in Table 48.1, in sequence.

Table 48.1 StrVPrintF Format Specification

Character	Description

+	Specifies that a sign always be placed before a number produced by a signed conversion. A + overrides a space if both are used. Example:
	`StrPrintF(s,"%+d  %+d",4,-5);`
	Output to `s`:
	`+4  -5`

-	Specifies that the printed value is left justified within the field width allowed for it. Example:
	`StrPrintF(s,"%5d%-5d%d",6,9,8);`
	Output to `s`:
	`    69    8`

<space>	Specifies that a minus sign always be placed before a negative number and a space before a positive number. Example:
	`StrPrintF(s,"% d  % d",4,-5);`
	Output to `s`:
	` 4  -5`

*	Indicates that the next argument (must be an integer) in the list specifies the field width. In this case, the argument following that one is used for the value of this field. Example:
	`StrPrintF(s,"%*d%d",4,8,5);`
	Output to `s`:
	`8   5`

<number>	Specifies a minimum field width. If the converted value has fewer characters than the field width, it will be padded with spaces on the left (or right, if the left justified flag is also specified) to fill out the field width. Example:
	`StrPrintF(s,"%d%5d",4,3);`
	Output to `s`:
	`4    3`

Table 48.1 StrVPrintF Format Specification *(continued)*

Character		Description

h		Specifies that the following d, i, u, or x conversion corresponds to a `short` or `unsigned short` argument. Example:
		`StrPrintF(s,"%hd",401);`
		Output to `s`:
		`401`

l or L		Specifies that the following d, i, u, x, or c conversion corresponds to a `long` or `unsigned long`
		`StrPrintF(s,"%ld",999999999);`
		Output to `s`:
		`999999999`

<character>		A character that indicates the type of conversion to be performed. The supported conversion characters include:

	d or i	A signed integer argument is converted to decimal notation. Example:
		`StrPrintF(s,"%d  %d",4,-4);`
		Output to `s`:
		`4  -4`

	u	An unsigned integer argument is converted to decimal notation. Example:
		`StrPrintF(s,"%u  %u",4,-4);`
		Output to `s`:
		`4  65532`

	x	An integer argument is converted to hexadecimal notation. Example:
		`StrPrintF(s,"%x",125);`
		Output to `s`:
		`0000007D`

Table 48.1 StrVPrintF Format Specification *(continued)*

Character	Description

`s`	A string (`char *`) argument is copied to the destination string. Example:
	`StrPrintF(s,"ABC%s","DEF");`
	Output to `s`:
	`ABCDEF`

`c` or `C`	A single character argument is copied to the destination string. If `C` is used or if the `l` modifier is used, the argument must be a `WChar`. Example:
	`StrPrintF(s,"Telephone%c",'s');`
	Output to `s`:
	`Telephones`

`%`	A % character is copied to the destination string. Example:
	`StrPrintF(s,"%%");`
	Output to `s`:
	`%`

Example Here's an example of how to use this call:

```
#include <unix_stdarg.h>
void MyPrintF(Char *s, Char *formatStr, ...)
{
  va_list args;
  Char text[0x100];
  va_start(args, formatStr);
  StrVPrintF(text, formatStr, args);
  va_end(args);
  MyPutS(text);
}
```

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `StrPrintF`, Using the StrVPrintF Function

49

System Event Manager

This chapter describes functions available in the system event manager. The system event manager API is declared in the header files `Event.h` and `SysEvtMgr.h`.

For more information on the system event manager, see the chapter "Event Loop" in the *Palm OS Programmer's Companion*, vol. I. The reference for specific events sent by the system are documented in "Palm OS Events".

System Event Manager Data Structures

The following system event manager data structures are documented in the "Palm OS Events" chapter:

- `eventsEnum`
- `EventType`
- `EventPtr`

System Event Manager Functions

EvtAddEventToQueue

Purpose Add an event to the event queue.

Prototype

```
void EvtAddEventToQueue (const EventType *event)
```

Parameters

-> `event` Pointer to the structure that contains the event.

Result Returns nothing.

Comments This function makes a copy of the structure that you pass in and adds it to the event queue.

EvtAddUniqueEventToQueue

Purpose Add an event to the event queue, replacing one of the same type if it is found.

Prototype

```
void EvtAddUniqueEventToQueue
(const EventType *eventP, UInt32 id,
Boolean inPlace)
```

Parameters

-> `eventP`	Pointer to the structure that contains the event.
-> `id`	ID of the event. 0 means match only on the type.
-> `inPlace`	If `true`, any existing event is replaced. If `false`, the existing event is deleted and a new event is added to end of queue.

Result Returns nothing.

Comments This function looks for an event in the event queue of the same event type and ID (if specified). The routine replaces it with the new event, if found.

If no existing event is found, the new event is copied to the queue.

If an existing event is found, the routine proceeds as follows:

- If `inPlace` is `true`, the existing event is replaced with a copy of the new event.
- If `inPlace` is `false`, the existing event is removed and the new event is added to the end of the queue.

Compatibility Implemented only if 2.0 New Feature Set is present.

EvtCopyEvent

Purpose Copy an event.

Prototype
```
void EvtCopyEvent (const EventType *source,
EventType *dest)
```

Parameters

-> `source`	Pointer to the structure containing the event to copy.
<- `dest`	Pointer to the structure to copy the event to.

Result Returns nothing.

Comments Use this function only if you want to create an event that has the same type as the `source` event. The `data` field in an `EventType` structure is specific to events of a given type. If you were to use this function to copy a `keyDownEvent` and then change it to a `frmLoadEvent`, the resulting `frmLoadEvent` would not have the proper `data` field.

If you want to create an event of a different type, do not use `EvtCopyEvent`. First clear the `EventType` structure using `MemSet` and then change the event type:

```
MemSet(&event, sizeof(EventType), 0);
event.eType = frmLoadEvent;
event.data.frmLoad.formID = formID;
EvtAddEventToQueue(&event);
```

EvtDequeuePenPoint

Purpose Get the next pen point out of the pen queue. This function is called by recognizers.

Prototype
```
Err EvtDequeuePenPoint (PointType *retP)
```

Parameters

<- `retP`	Return point.

Result Always returns 0.

Comments Called by a recognizer that wishes to extract the points of a stroke. Returns the point (-1, -1) at the end of a stroke.

Before calling this routine, you must call EvtDequeuePenStrokeInfo.

EvtDequeuePenStrokeInfo

Purpose Initiate the extraction of a stroke from the pen queue.

Prototype
```
Err EvtDequeuePenStrokeInfo (PointType *startPtP,
PointType *endPtP)
```

Parameters

<- startPtP	Start point returned here.
<- endPtP	End point returned here.

Result Always returns 0.

Comments Called by the system function EvtGetSysEvent when a penUpEvent is being generated. This routine must be called before EvtDequeuePenPoint is called.

Subsequent calls to EvtDequeuePenPoint return points at the starting point in the stroke and including the end point. After the end point is returned, the next call to EvtDequeuePenPoint returns the point -1, -1.

See Also EvtDequeuePenPoint

EvtEnableGraffiti

Purpose Set Graffiti® enabled or disabled.

Prototype
```
void EvtEnableGraffiti (Boolean enable)
```

Parameters

-> enable	true to enable Graffiti, false to disable Graffiti.

Result Returns nothing.

EvtEnqueueKey

Purpose Place keys into the key queue.

Prototype

```
Err EvtEnqueueKey (WChar ascii, UInt16 keycode,
UInt16 modifiers)
```

Parameters

-> `ascii`	Character code for the key.
-> `keycode`	Virtual key code of key. This is the `keyCode` field of the `keyDownEvent` and is currently unused.
-> `modifiers`	Modifiers for `keyDownEvent`.

Result Returns 0 if successful, or `evtErrParamErr` if an error occurs.

Comments This function disables interrupts while the queue header is being modified because both interrupt- and non-interrupt-level code can post keys into the queue.

Entries in the key queue only take 1 byte if the `ascii` parameter has a value less than 256 and the `keycode` and `modifiers` parameters are both zero. Otherwise an entry can take up to 7 bytes.

IMPORTANT: Make sure you pass a `WChar` as the `ascii` parameter, not a `Char`. If you pass a high-ASCII `Char`, the compiler sign-extends it to be a 16-bit value, resulting in the wrong character being added to the key queue.

EvtEventAvail

Purpose Return `true` if an event is available.

Prototype

```
Boolean EvtEventAvail (void)
```

Parameters None.

Result Returns `true` if an event is available, `false` otherwise.

Compatibility Implemented only if 2.0 New Feature Set is present.

EvtFlushKeyQueue

Purpose Flush all keys out of the key queue.

Prototype
```
Err EvtFlushKeyQueue (void)
```

Parameters None.

Result Always returns 0.

Comments Called by the system function `EvtSetPenQueuePtr`.

EvtFlushNextPenStroke

Purpose Flush the next stroke out of the pen queue.

Prototype
```
Err EvtFlushNextPenStroke ()
```

Parameters None.

Result Always returns 0.

Comments Called by recognizers that need only the start and end points of a stroke. If a stroke has already been partially dequeued (by `EvtDequeuePenStrokeInfo`) this routine finishes the stroke dequeueing. Otherwise, this routine flushes the next stroke in the queue.

See Also `EvtDequeuePenPoint`

EvtFlushPenQueue

Purpose Flush all points out of the pen queue.

Prototype
```
Err EvtFlushPenQueue (void)
```

Parameters None

Result Always returns 0.

Comments Called by the system function `EvtSetKeyQueuePtr`.

See Also `EvtPenQueueSize`

EvtGetEvent

Purpose Return the next available event.

Prototype
```
void EvtGetEvent (EventType *event, Int32 timeout)
```

Parameters

<- `event`	Pointer to the structure to hold the event returned.
-> `timeout`	Maximum number of ticks to wait before an event is returned (`evtWaitForever` means wait indefinitely).

Comments Pass `evtWaitForever` as the timeout in most instances. When running on the device, this makes the CPU go into doze mode until the user provides input. For applications that do animation, pass a `timeout` value greater than or equal to zero.

Note that a timeout value greater than or equal to zero is simply the *maximum* number of ticks which can elapse before `EvtGetEvent` returns an event. If any other event—including a `nilEvent`—occurs before this time has elapsed, `EvtGetEvent` will return that event. Otherwise, once the specified time has elapsed `EvtGetEvent` generates and returns a `nilEvent`. If you supply a value of zero for the timeout parameter, `EvtGetEvent` returns the

event currently in the queue, or, if there aren't any events in the queue, it immediately generates and returns a `nilEvent`.

Result Returns nothing.

EvtGetPen

Purpose Return the current status of the pen.

Prototype
```
void EvtGetPen (Int16 *pScreenX, Int16 *pScreenY,
Boolean *pPenDown)
```

Parameters

<-	`pScreenX`	x location relative to display.
<-	`pScreenY`	y location relative to display.
<-	`pPenDown`	`true` or `false`.

Result Returns nothing.

Comments Called by various UI routines.

See Also `KeyCurrentState`

EvtGetPenBtnList

Purpose Return a pointer to the silk-screen button array.

Prototype
```
const PenBtnInfoType *EvtGetPenBtnList
(UInt16 *numButtons)
```

Parameters

<-	`numButtons`	The number of elements in the returned array.

Result Returns a pointer to an array of the silk-screen buttons.

Comments This function returns an array of `PenBtnInfoType` structures:

```
typedef struct PenBtnInfoType {
  RectangleType  boundsR;
  WChar          asciiCode;
  UInt16         keyCode;
  UInt16         modifiers;
} PenBtnInfoType;
```

The fields in the `PenBtnInfoType` contain the following information:

`boundsR`	The button's bounding rectangle.
`asciiCode`	The character code generated when the button is tapped. This is typically a virtual character.
`keyCode`	Currently unused.
`modifiers`	Modifiers for the key down event. (See the description of the modifiers field for `keyDownEvent`.)

The number of buttons is device-dependent. Even devices with the same Palm OS® version may have differing numbers of silk-screen buttons. For example, Japanese devices typically have four extra silk-screen buttons used to transliterate characters into different alphabets.

See Also `EvtProcessSoftKeyStroke`

EvtGetSilkscreenAreaList

Purpose Returns a pointer to the silk-screen area array. This array contains the bounds of each silk-screen area.

Prototype

```
const SilkscreenAreaType *EvtGetSilkscreenAreaList
(UInt16 *numAreas)
```

Parameters

`<- numAreas`	The number of elements in the returned array.

Result Returns a pointer to an array containing the bounds of each silk-screen area.

Comments This function returns an array of the `SilkscreenAreaType` structures:

```
typedef struct SilkscreenAreaType {
  RectangleType bounds;
  UInt32    areaType;
  UInt16    index;
  } SilkscreenAreaType;
```

The fields in this structure provide the following information.

`bounds` The area's bounds.

`areaType` The area type, can be one of the following:

`silkscreenRectGraffiti`
The Graffiti area.

`silkscreenRectScreen`
The entire silkscreen area.

Palm Powered™ devices not manufactured by Palm, Inc. may have other area types.

`index` If the area type is `silkscreenRectGraffiti`, this field is either `alphaGraffitiSilkscreenArea` to represent the portion where letters are entered or `numericGraffitiSilkscreenArea` to represent the portion where numbers are entered.

Compatibility Implemented only if 3.5 New Feature Set is present.

EvtKeydownIsVirtual

Purpose Macro that indicates if `eventP` is a pointer to a virtual character key down event.

Prototype

```
EvtKeydownIsVirtual (eventP)
```

Parameters -> `eventP` Pointer to an `EventType` structure.

Result Returns `true` if the character is a letter in an alphabet or a numeric digit, `false` otherwise.

Comments The macro assumes that the caller has already determined the event is a `keyDownEvent`.

This macro is intended for use by the system. Applications should use `TxtGlueCharIsVirtual`, contained in the PalmOSGlue Library.

Compatibility Implemented in the Palm OS 3.5 SDK.

See Also `TxtGlueCharIsVirtual`

EvtKeyQueueEmpty

Purpose Return `true` if the key queue is currently empty.

Prototype

```
Boolean EvtKeyQueueEmpty (void)
```

Parameters None.

Result Returns `true` if the key queue is currently empty, otherwise returns `false`.

Comments Usually called by the key manager to determine if it should enqueue auto-repeat keys.

EvtKeyQueueSize

Purpose Return the size of the current key queue in bytes.

Prototype

```
UInt32 EvtKeyQueueSize (void)
```

Parameters None.

Result Returns size of queue in bytes.

Comments Called by applications that wish to see how large the current key queue is.

EvtPenQueueSize

Purpose Return the size of the current pen queue in bytes.

Prototype
```
UInt32 EvtPenQueueSize (void)
```

Parameters None.

Result Returns size of queue in bytes.

Comments Call this function to see how large the current pen queue is.

EvtProcessSoftKeyStroke

Purpose Translate a stroke in the system area of the digitizer and enqueue the appropriate key events in to the key queue.

Prototype
```
Err EvtProcessSoftKeyStroke (PointType *startPtP,
PointType *endPtP)
```

Parameters

-> `startPtP`	Start point of stroke.
-> `endPtP`	End point of stroke.

Result Returns 0 if recognized, -1 if not recognized.

See Also `EvtGetPenBtnList`, `GrfProcessStroke`

EvtResetAutoOffTimer

Purpose Reset the auto-off timer.

Prototype
```
Err EvtResetAutoOffTimer (void)
```

Parameters None.

Result Always returns 0.

Comments Called by the serial link manager; can be called periodically by other managers.

`EvtResetAutoOffTimer` resets the auto-off timer so that the device does not turn off until at least the default amount of idle time has passed. You can use this function to ensure that the device doesn't automatically power off during a long operation without user input (for example, when there is a lot of serial port activity).

If you need more control over the auto-off timer and the 3.5 New Feature Set is present, consider using `EvtSetAutoOffTimer` instead of this function.

See Also `SysSetAutoOffTime`

EvtSetAutoOffTimer

Purpose Set the auto-off timer.

Prototype

```
Err EvtSetAutoOffTimer (EvtSetAutoOffCmd cmd,
UInt16 timeout)
```

Parameters

-> `cmd`	One of the defined commands.
-> `timeout`	A new timeout value in seconds. If `cmd` is `ResetTimer`, this parameter is ignored.

Result Always returns `errNone`.

Comments Use `EvtSetAutoOffTimer` to ensure that the device doesn't automatically power off during a long operation that has no user input (for example, when there is a lot of serial port activity).

The `cmd` parameter specifies the operation you want to perform. It takes one of the following `EvtSetAutoOffCmd` constants:

`SetAtLeast`	Make sure that the device won't turn off until `timeout` seconds of idle time has passed. (This operation only changes the current value if it's less than the value you specify.)
`SetExactly`	Set the timer to turn off in `timeout` seconds

`SetAtMost`	Make sure the device will turn before `timeout` seconds has passed. (This operation only changes the current value if it's greater than the value you specify.)
`SetDefault`	Change the default auto-off timeout to `timeout` seconds.
`ResetTimer`	Reset the auto-off timer so that the device does not turn off until at least the default seconds of idle time has passed.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `EvtResetAutoOffTimer`, `SysSetAutoOffTime`

EvtSetNullEventTick

Purpose Make sure a `nilEvent` occurs in at least the specified number of ticks.

Prototype

```
Boolean EvtSetNullEventTick(UInt32 tick)
```

Parameters

-> `tick`	Maximum number of system ticks that should elapse before a `nilEvent` is added to the queue.

Result Returns `true` if timeout value changed, or `false` if it did not change.

Compatibility In versions prior to Palm OS 3.5, this function was implemented as a macro.

EvtSysEventAvail

Purpose Return `true` if a low-level system event (such as a pen or key event) is available.

Prototype
```
Boolean EvtSysEventAvail (Boolean ignorePenUps)
```

Parameters

`ignorePenUps`	If `true`, this routine ignores pen-up events when determining if there are any system events available.

Result Returns `true` if a system event is available.

Comment Call `EvtEventAvail` to determine whether high-level software events are available.

Compatibility Implemented only if 2.0 New Feature Set is present.

EvtWakeup

Purpose Force the event manager to wake up and send a `nilEvent` to the current application.

Prototype
```
Err EvtWakeup (void)
```

Parameters None.

Result Always returns 0.

Comments Called by interrupt routines, like the sound manager and alarm manager.

See Also `EvtWakeupWithoutNilEvent`



New EvtWakeupWithoutNilEvent

Purpose Force the event manager to wake up without sending a `nilEvent` to the current application.

Prototype

```
Err EvtWakeupWithoutNilEvent()
```

Parameters None.

Result Always returns 0.

Comments Called by interrupt routines.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `EvtWakeup`

50

System Manager

This chapter provides reference material for the system manager. The system manager API is declared in the header files `SystemMgr.h` and `SysUtils.h`.

For more information on the system manager, see the chapters "Application Startup and Stop" and "Palm System Support" in the *Palm OS Programmer's Companion*, vol. I.

System Manager Data Structures

SysDBListItemType

The `SysDBListItemType` structure describes a single database or panel. The `SysCreateDataBaseList` and `SysCreatePanelList` functions each create and return an array of `SysDBListItemType` structures.

```
typedef struct {
  Char          name[dmDBNameLength];
  UInt32        creator;
  UInt32        type;
  UInt16        version;
  LocalID       dbID;
  UInt16        cardNo;
  BitmapPtr     iconP;
} SysDBListItemType;
```

System Functions

SysAppLaunch

Purpose

Launch a specified application as a subroutine of the caller.

Prototype

```
Err SysAppLaunch (UInt16 cardNo, LocalID dbID,
UInt16 launchFlags, UInt16 cmd, MemPtr cmdPBP,
UInt32 *resultP)
```

Parameters

-> `cardNo`, `dbID`	The card number and ID of the resource database of the application to launch.
-> `launchFlags`	Set to 0.
-> `cmd`	Launch code.
-> `cmdPBP`	Launch code parameter block.
<- `resultP`	The value returned from the application's `PilotMain` routine.

Result

Returns 0 if no error, or one of `sysErrParamErr`, `memErrNotEnoughSpace`, or `sysErrOutOfOwnerIDs`.

Comments

Applications can use `SysAppLaunch` to send a specific launch code to another application and have control return to the calling application when finished. This function in effect makes the specified application a subroutine of the caller. If you want to actually close your application and call another application, use `SysUIAppSwitch` instead of this function. `SysUIAppSwitch` sends the current application an `appStopEvent` and then starts the specified application.

Do not use this function to open the system-supplied Application Launcher application. If another application has replaced the default launcher with one of its own, this function will open the custom launcher instead of the system-supplied one. To open the system-supplied launcher reliably, enqueue a `keyDownEvent` that contains a `launchChr`, as shown in the section "Application Launcher" of the user interface chapter in the *Palm OS Programmer's Companion*, vol. I.

NOTE: For important information regarding the correct use of this function, see the "Application Startup and Stop" chapter in the *Palm OS Programmer's Companion*, vol. I.

See Also SysBroadcastActionCode, SysUIAppSwitch, SysCurAppDatabase

SysBatteryInfo

Purpose Retrieve settings for the batteries. Set `set` to `false` to retrieve battery settings. (Applications should **not** change any of the settings).

WARNING! Use this function only to **retrieve** settings!

Prototype

```
UInt16 SysBatteryInfo (Boolean set,
UInt16 *warnThresholdP,
UInt16 *criticalThresholdP, Int16 *maxTicksP,
SysBatteryKind *kindP, Boolean *pluggedIn,
UInt8 *percentP)
```

Parameters

`set`	If `false`, parameters with non-`NULL` pointers are retrieved. Never set this parameter to `true`.
`warnThresholdP`	Pointer to battery voltage warning threshold in volts*100, or `NULL`.
`criticalThresholdP`	Pointer to the battery voltage critical threshold in volts*100, or `NULL`.
`maxTicksP`	Pointer to the battery timeout, or `NULL`.
`kindP`	Pointer to the battery kind, or `NULL`.
`pluggedIn`	Pointer to `pluggedIn` return value, or `NULL`.
`percentP`	Percentage of power remaining in the battery.

Result Returns the current battery voltage in volts*100.

Comments Call this function to make sure an upcoming activity won't be interrupted by a low battery warning.

`warnThresholdP` and `maxTicksP` are the battery-warning voltage threshold and time out. If the battery voltage falls below the threshold, or the timeout expires, a `lowBatteryChr` key event is put on the queue. Normally, applications call `SysHandleEvent` which calls `SysBatteryDialog` in response to this event.

`criticalThresholdP` is the battery voltage threshold. If battery voltage falls below this level, the system turns itself off without warning and doesn't turn on until battery voltage is above it again.

Compatibility This function was revised for Palm OS® 3.0. In Palm OS 3.0, the `percentP` parameter was added. This enhancement is implemented only if 3.0 New Feature Set is present.

See Also `SysBatteryInfoV20`

SysBatteryInfoV20

Purpose Retrieve settings for the batteries. Set to `false` to retrieve battery settings. (Applications should **not** change any of the settings).

WARNING! Use this function only to **retrieve** settings!

Prototype

```
UInt16 SysBatteryInfoV20 (Boolean set,
UInt16 *warnThresholdP,
UInt16 *criticalThresholdP, Int16 *maxTicksP,
SysBatteryKind *kindP, Boolean *pluggedIn)
```

Parameters

`set` If `false`, parameters with non-`NULL` pointers are retrieved. Never set this parameter to `true`.

`warnThresholdP` Pointer to battery voltage warning threshold in volts*100, or `NULL`.

`criticalThresholdP` Pointer to the battery voltage critical threshold in volts*100, or `NULL`.

`maxTicksP`	Pointer to the battery timeout, or `NULL`.
`kindP`	Pointer to the battery kind, or `NULL`.
`pluggedIn`	Pointer to `pluggedIn` return value, or `NULL`.

Result Returns the current battery voltage in volts*100.

Comments Call this function to make sure an upcoming activity won't be interrupted by a low battery warning.

`warnThresholdP` and `maxTicksP` are the battery-warning voltage threshold and time out. If the battery voltage falls below the threshold, or the timeout expires, a `lowBatteryChr` key event is put on the queue. Normally, applications call `SysHandleEvent` which calls `SysBatteryDialog` in response to this event.

`criticalThresholdP` is the battery voltage threshold. If battery voltage falls below this level, the system turns itself off without warning and doesn't turn on until battery voltage is above it again.

Compatibility This function corresponds to the Palm OS 2.0 version of `SysBatteryInfo`. Implemented only if 3.0 New Feature Set is present.

See Also `SysBatteryInfo`

SysBinarySearch

Purpose Search elements in a sorted array for the specified data according to the specified comparison function.

Prototype

```
Boolean SysBinarySearch (void const *baseP,
Int16 numOfElements, Int16 width,
SearchFuncPtr searchF, void const *searchData,
Int32 other, Int32 *position, Boolean findFirst)
```

Parameters

`baseP`	Base pointer to an array of elements
`numOfElements`	Number of elements to search. Must be greater than 0.
`width`	Width of each array element.
`searchF`	Search function.
`searchData`	Data to search for. This data is passed to the `searchF` function.
`other`	Data to be passed as the third parameter (the `other` parameter) to the comparison function.
`position`	Pointer to the position result.
`findFirst`	If set to `true`, the first matching element is returned. Use this parameter if the array contains duplicate entries to ensure that the first such entry will be the one returned.

Result

Returns `true` if an exact match was found. In this case, `position` points to the element number where the data was found.

Returns `false` if an exact match was not found. If `false` is returned, `position` points to the element number where the data should be inserted if it was to be added to the array in sorted order.

Comments

The array must be sorted in ascending order prior to the search. Use `SysInsertionSort` or `SysQSort` to sort the array.

The search starts at element 0 and ends at element (`numOfElements` - 1).

The search function's (`searchF`) prototype is:

```
Int16 _searchF (void const *searchData,
void const *arrayData, Int32 other);
```

The first parameter is the data for which to search, the second parameter is a pointer to an element in the array, and the third parameter is any other necessary data.

The function returns:

- > 0 if the search data is greater than the element
- < 0 if the search data is less than the element
- 0 if the search data is the same as the element

Compatibility Implemented only if 2.0 New Feature Set is present.

SysBroadcastActionCode

Purpose Send the specified action code (launch code) and parameter block to the latest version of every UI application.

Prototype

```
Err SysBroadcastActionCode (UInt16 cmd,
MemPtr cmdPBP)
```

Parameters

`cmd`	Action code to send.
`cmdPBP`	Action code parameter block to send.

Result Returns 0 if no error, or one of the following errors: `sysErrParamErr`, `memErrNotEnoughSpace`, or `sysErrOutOfOwnerIDs`.

See Also `SysAppLaunch`, Chapter 3, "Application Startup and Stop." of the *Palm OS Programmer's Companion*, vol. I

SysCopyStringResource

Purpose Copy a resource string to a passed string.

Prototype

```
void SysCopyStringResource (Char *string,
Int16 theID)
```

Parameters

`string`	String to copy the resource string to.
`theID`	Resource string ID.

Result Stores a copy of the resource string in `string`.

SysCreateDataBaseList

Purpose

Generate a list of databases found on the memory cards matching a specific type and return the result. If `lookupName` is `true` then a name in a `tAIN` resource is used instead of the database's name and the list is sorted. Only the last version of a database is returned. Databases with multiple versions are listed only once.

Prototype

```
Boolean SysCreateDataBaseList (UInt32 type,
UInt32 creator, UInt16 *dbCount, MemHandle *dbIDs,
Boolean lookupName)
```

Parameters

`type`	The type of database to find. Use `0` to find all databases.
`creator`	The creator ID of the database to find. Use `0` to find any creator ID.
`dbCount`	A pointer to an integer value that is updated by this function to the number of matching databases.
`dbIDs`	A pointer to a handle that gets allocated to contain the database list. Upon return, this references an array of `SysDBListItemType` structures. See the Comments section below for more information.
`lookupName`	If `true`, `SysCreateDatabaseList` uses `tAIN` names and sorts the list.

Result

Returns `false` if no databases were found, and `true` if any databases were found. The value of `dbCount` is updated to reflect the number of databases that were found. If at least one database is found, `dbIDs` is updated to reference a array of `SysDBListItemType` structures; this array contains `dbCount` items.

Comments

If this function returns `true` and the value of `dbCount` is greater then 0, than you can iterate through the list of database items, as shown in Listing 50.1

Listing 50.1 Using the SysCreateDatabaseList function

```
SysDBListItemType          *dbListIDsP;
MemHandle                  dbListIDsH;
UInt16                     dbCount = 0;
Boolean                    status;
UInt16                     counter;
SysDBListItemType          theItem;

status = SysCreateDatabaseList(sysFileTpqa, 0,
                               &dbCount, &dbListIDsH, false);

if (status == true && dbCount > 0)
   {
   dbListIDsP = MemHandleLock (dbListIDsH);
   for (counter = 0; counter < dbCount; counter++)
      if StrCompare(dbListIDsP[counter].name,
                                                "MINE") == 0
      // we found my database
   ...
...
   MemPtrFree (dbListIDsP);
   }
```

NOTE: It is your responsibility to free the memory allocated by this function for the list of databases.

Compatibility Implemented only if 2.0 New Feature Set is present.

SysCreatePanelList

Purpose Generate a list of panels found on the memory cards and return the result. Multiple versions of a panel are listed once.

Prototype

```
Boolean SysCreatePanelList (UInt16 *panelCount,
MemHandle *panelIDs)
```

Parameters

`panelCount` Pointer to set to the number of panels.

	`panelIDs`	A pointer to a handle that gets allocated to contain the panel list. Upon return, this references an array of `SysDBListItemType` structures.

Result Returns `false` if no panels were found, and `true` if any panels were found. The value of `panelCount` is updated to reflect the number of panels that were found. If at least one panel is found, `panelIDs` is updated to reference a array of `SysDBListItemType` structures; this array contains `panelCount` items.

Comments If this function returns `true` and the value of `panelCount` is greater than 0, than you can iterate through the list of panel items, as shown in Listing 50.1. It is your responsibility to free the memory allocated for the panel list.

Compatibility Implemented only if 2.0 New Feature Set is present.

SysCurAppDatabase

Purpose Return the card number and database ID of the current application's resource database.

Prototype

```
Err SysCurAppDatabase (UInt16 *cardNoP,
LocalID *dbIDP)
```

Parameters

`cardNoP`	Pointer to the card number; 0 or 1.
`dbIDP`	Pointer to the database ID.

Result Returns 0 if no error, or `SysErrParamErr` if an error occurs.

See Also `SysAppLaunch`, `SysUIAppSwitch`

SysErrString

Purpose Returns text to describe an error number. This routine looks up the textual description of a system error number in the appropriate List resource and creates a string that can be used to display that error.

The actual string will be of the form: "<error message> (XXXX)" where XXXX is the hexadecimal error number.

This routine looks for a resource of type 'tstl' and resource ID of (err>>8). It then grabs the string at index (err & 0x00FF) out of that resource.

The first string in the resource is called index #1 by Constructor, NOT #0. For example, an error code of 0x0101 will fetch the first string in the resource.

Prototype

```
Char *SysErrString (Err err, Char *strP,
UInt16 maxLen)
```

Parameters

`err`	Error number
`strP`	Pointer to space to form the string
`maxLen`	Size of `strP` buffer.

Result Stores the error number string.

Compatibility Implemented only if 2.0 New Feature Set is present.

SysFormPointerArrayToStrings

Purpose Form an array of pointers to strings in a block. Useful for setting the items of a list.

Prototype

```
MemHandle SysFormPointerArrayToStrings (Char *c,
Int16 stringCount)
```

Parameters

`c`	Pointer to packed block of strings, each terminated by `NULL`.
`stringCount`	Count of strings in block.

Result Unlocked handle to allocated array of pointers to the strings in the passed block. The returned array points to the strings in the passed packed block. Note that you'll need to free the returned handle when you no longer need it.

SysGetOSVersionString

Purpose Return the version number of the Palm™ operating system.

Prototype

```
Char *SysGetOSVersionString()
```

Parameters None.

Result Returns a string such as "v. 3.0."

Comments You must free the returned string using the `MemPtrFree` function.

Compatibility Implemented only if 3.0 New Feature Set is present.

SysGetROMToken

Purpose Return from ROM a value specified by token.

Prototype

```
Err SysGetROMToken (UInt16 cardNo, UInt32 token,
UInt8 **dataP, UInt16 *sizeP)
```

Parameters

-> `cardNo` The card on which the ROM to be queried resides. Currently, no Palm hardware provides multiple cards, so this value must be 0.

-> `token` The value to retrieve, as specified by one of the following tokens:

`sysROMTokenSnum`
: The serial number of the ROM, expressed as a text string with no null terminator.

<- `dataP` Pointer to a text buffer that holds the requested value when the function returns.

<- `sizeP` The number of bytes in the `dataP` buffer.

Result Returns the requested value if no error, or an error code if an error occurs. If this function returns an error, or if the returned pointer to the buffer is `NULL`, or if the first byte of the text buffer is `0xFF`, then no serial number is available.

Comments The serial number is shown to the user in the Application Launcher, along with a checksum digit you can use to validate input when your users read the ID from their device and type it in or tell it to someone else.

Compatibility Implemented only if 3.0 New Feature Set is present. Serial numbers are available only on flash ROM-based units.

See Also "Retrieving the ROM Serial Number" section in the *Palm OS Programmer's Companion*, vol. I shows how to retrieve the ROM serial number and calculate its associated checksum.

SysGetStackInfo

Purpose Return the start and end of the current thread's stack.

Prototype

```
Boolean SysGetStackInfo (MemPtr *startPP,
MemPtr *endPP)
```

Parameters

`startPP`	Upon return, points to the start of the stack.
`endPP`	Upon return, points to the end of the stack.

Result Returns `true` if the stack has not overflowed, that is, the value of the stack overflow address has not been changed. Returns `false` if the stack overflow value has been overwritten, meaning that a stack overflow has occurred.

Compatibility Implemented only if 3.0 New Feature Set is present.

SysGetTrapAddress

Purpose Return the address of a function given its system trap.

Prototype

```
void *SysGetTrapAddress (UInt16 trapNum)
```

Parameters

-> `trapNum` One of the routine selectors defined in `SysTraps.h` (`sysTrap...`) or `CoreTraps.h` on Palm OS version 3.5 and higher.

Result Returns the address of the corresponding function. Returns the address of the `sysTrapSysUnimplemented` routine if an invalid routine selector is passed; compare the results of this function to `SysGetTrapAddress(sysTrapSysUnimplemented)` if you need to check for an invalid routine selector.

Comments Use this function for performance reasons. You can then use the address it returns to call the function without having to go through the trap dispatch table. This function is mostly useful for optimizing the performance of functions called in a tight loop.

The Palm OS trap dispatch mechanism allows the trap table entries to be modified at any time, either as the result of a system update or a hack. For this reason, it's important to call this function **immediately before** entering the tight loop. If the trap address changes in between when you call `SysGetTrapAddress` and you use the address, the wrong function will be called.

Compatibility On Palm OS 3.0 and earlier, this function contains a bug that causes it to return a garbage value if an invalid routine selector is passed. To prevent this bug from affecting your application, use `SysGlueGetTrapAddress` in the PalmOSGlue library instead of calling this function directly. For more information, see Chapter 76, "PalmOSGlue Library."

SysGremlins

Purpose Query the Gremlins facility. You pass a selector for a function and parameters for that function. Gremlins performs the function call and returns the result.

Prototype

```
UInt32 SysGremlins (GremlinFunctionType selector,
GremlinParamsType *params)
```

Parameters

`selector`	The selector for a function to pass to Gremlins.
`params`	Pointer to a parameter block used to pass parameters to the function specified by `selector`.

Result Returns the result of the function performed in Gremlins.

Comments Currently, only one selector is defined, `GremlinIsOn`, which takes no parameters. `GremlinIsOn` returns 0 if Gremlins is not running, non-zero if it is running.

Currently, non-zero values are returned only from the version of Gremlins in the Palm OS emulator. The Gremlins running in the Simulator on a Macintosh and over the serial line via the Palm Debugger return zero for `GremlinIsOn`.

Use this function if you need to alter the application's behavior when Gremlins is running. For example, the debug 3.0 ROM refuses to bring up the digitizer panel when Gremlins is running under the emulator.

Compatibility Implemented only if 3.0 New Feature Set is present.

In Palm OS 3.2 and later, `SysGremlins` is replaced by the functions defined in the file `HostControl.h`. Specifically, the one selector supported by `SysGremlins` is replaced with the function `HostGremlinIsRunning`. For backward compatibility, `SysGremlins` is mapped to `HostGremlinIsRunning`.

SysHandleEvent

Purpose Handle defaults for system events such as hard and soft key presses.

Prototype
```
Boolean SysHandleEvent (EventPtr eventP)
```

Parameters

`eventP`	Pointer to an event.

Result Returns `true` if the system handled the event.

Comments Applications should call this routine immediately after calling `EvtGetEvent` unless they want to override the default system behavior. However, overriding the default system behavior is almost never appropriate for an application.

See Also `EvtProcessSoftKeyStroke`, `KeyRates`

SysInsertionSort

Purpose Sort elements in an array according to the passed comparison function.

Prototype
```
void SysInsertionSort (void *baseP,
Int16 numOfElements, Int16 width,
CmpFuncPtr comparF, Int32 other)
```

Parameters

`baseP`	Base pointer to an array of elements.
`numOfElements`	Number of elements to sort (must be at least 2).
`width`	Width of an element.
`comparF`	Comparison function. See Comments, below.
`other`	Other data passed to the comparison function.

Result Returns nothing.

Comments

Only elements which are out of order move. Moved elements are moved to the end of the range of equal elements. If a large amount of elements are being sorted, try to use the quick sort (see SysQSort).

This is the insertion sort algorithm: Starting with the second element, each element is compared to the preceding element. Each element not greater than the last is inserted into sorted position within those already sorted. A binary search for the insertion point is performed. A moved element is inserted after any other equal elements.

In order to use `SysInsertionSort` you must write a comparison function with the following prototype:

```
Int16 comparF (void *p1, void *p2, Int32 other)
```

Your comparison function must return zero if `p1` equals `p2`, a positive integer value if `p1` is greater than `p2`, and a negative integer value if `p1` is less than `p2`. Note that the value of the parameter named `other` is passed through from the `SysInsertionSort` call and can be used to control the behavior of the `comparF` function if appropriate for your application.

Compatibility

The Palm OS 2.0 comparison function (`comparF`) has this prototype:

```
Int comparF (VoidPtr, VoidPtr, Long other);
```

The Palm OS 1.0 comparison function (`comparF`) has this prototype:

```
Int comparF (BytePtr A, BytePtr B, Long other);
```

See Also

SysQSort

SysKeyboardDialog

Purpose Pop up the system keyboard if there is a field object with the focus. The field object's text chunk is edited directly.

Prototype
```
void SysKeyboardDialog (KeyboardType kbd)
```

Parameters

`kbd`	The keyboard type. See `Keyboard.h`.

Result Returns nothing. Changes the field's text chunk.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `SysKeyboardDialogV10`, `FrmSetFocus`

SysKeyboardDialogV10

Purpose Pop up the system keyboard if there is a field object with the focus. The field object's text chunk is edited directly.

Prototype
```
void SysKeyboardDialogV10 ()
```

Parameters None.

Result Returns nothing. The field's text chunk is changed.

Compatibility Corresponds to the 1.0 implementation of `SysKeyboardDialog`.

See Also `SysKeyboardDialog`, `FrmSetFocus`

SysLibFind

Purpose Return a reference number for a library that is already loaded, given its name.

Prototype
```
Err SysLibFind (const Char *nameP,
UInt16 *refNumP)
```

Parameters

`nameP`	Pointer to the name of a loaded library.
`refNumP`	Pointer to a variable for returning the library reference number (on failure, this variable is undefined)

Result 0 if no error; otherwise: `sysErrLibNotFound` (if the library is not yet loaded), or another error returned from the library's install entry point.

Comments Most built-in libraries (NetLib, serial, IR) are preloaded automatically when the system is reset. Third-party libraries must be loaded before this call can succeed (use `SysLibLoad`). You can check if a library is already loaded by calling `SysLibFind` and checking for a 0 error return value (it will return a non-zero value if the library is not loaded).

SysLibInstall

Purpose Installs a library into the Library table and calls the library's install entry point.

Prototype
```
Err SysLibInstall (SysLibEntryProcPtr libraryP,
UInt16 *refNumP)
```

Parameters

-> libraryP	Pointer to the library being installed.
<- refNumP	Pointer to the variable in which the installed library reference number is stored upon return.

Result Returns 0 if no error, `memErrNotEnoughSpace` if an error occurs while allocating memory in the library table, or, if an error is returned by the call to the library's install entry point, that error is returned and the reference number is set to `sysInvalidRefNum`.

Comments This routine is largely used by the Palm OS, but can also be called by applications that need to install their own libraries.

Compatibility Implemented only if 2.0 New Feature Set is present.

SysLibLoad

Purpose Load a library given its database creator and type.

Prototype

```
Err SysLibLoad (UInt32 libType, UInt32 libCreator,
UInt16 *refNumP)
```

Parameters

`libType`	Type of library database.
`libCreator`	Creator of library database.
`refNumP`	Pointer to variable for returning the library reference number (on failure, `sysInvalidRefNum` is returned in this variable)

Result 0 if no error; otherwise: `sysErrLibNotFound`, `sysErrNoFreeRAM`, `sysErrNoFreeLibSlots`, or other error returned from the library's install entry point.

Comments Presently, the "load" functionality is **not** supported when you use the Palm OS Simulator.

When an application no longer needs a library that it **successfully** loaded via `SysLibLoad`, it is responsible for unloading the library by calling `SysLibRemove` and passing it the library reference number returned by `SysLibLoad`. More information is available in the white paper on shared libraries, which you can find on the Palm developer support web site.

Compatibility Implemented only if 2.0 New Feature Set is present.

SysLibRemove

Purpose Unload a library previously loaded with SysLibLoad.

Prototype
```
Err SysLibRemove (UInt16 refNum)
```

Parameters

-> refNum	The library reference number.

Result 0 if no error; otherwise sysErrParamErr if the refNum is not a valid library reference number.

Comments SysLibRemove releases the dynamic memory allocated to the shared library's dispatch table, resources, and global variables.

SysQSort

Purpose Sort elements in an array according to the supplied comparison function.

Prototype
```
void SysQSort (void *baseP, Int16 numOfElements,
Int16 width, CmpFuncPtr comparF, Int32 other)
```

Parameters

baseP	Base pointer to an array of elements.
numOfElements	Number of elements to sort (must be at least 2).
width	Width of an element.
comparF	Comparison function. See Comments, below.
other	Other data passed to the comparison function.

Result Returns nothing.

Comments Equal records can be in any position relative to each other because a quick sort tends to scramble the ordering of records. As a result, calling SysQSort multiple times can result in a different order if the records are not completely unique. If you don't want this behavior, use the insertion sort instead (see SysInsertionSort).

To pick the pivot point, the quick sort algorithm picks the middle of three records picked from around the middle of all records. That way, the algorithm can take advantage of partially sorted data.

These optimizations are built in:

- The routine contains its own stack to limit uncontrolled recursion. When the stack is full, an insertion sort is used because it doesn't require more stack space.
- An insertion sort is also used when the number of records is low. This avoids the overhead of a quick sort which is noticeable for small numbers of records.
- If the records seem mostly sorted, an insertion sort is performed to move only those few records that need to be moved.

In order to use `SysQSort` you must write a comparison function with the following prototype:

```
Int16 comparF (void *p1, void *p2, Int32 other)
```

Your comparison function must return zero if `p1` equals `p2`, a positive integer value if `p1` is greater than `p2`, and a negative integer value if `p1` is less than `p2`. Note that the value of the parameter named `other` is passed through from the `SysQSort` call and can be used to control the behavior of the `comparF` function if appropriate for your application.

Compatibility

The Palm OS 2.0 comparison function (`comparF`) has this prototype:

```
Int comparF (VoidPtr, VoidPtr, Long other);
```

The Palm OS 1.0 comparison function (`comparF`) has this prototype:

```
Int comparF (BytePtr A, BytePtr B, Long other);
```

See Also `SysInsertionSort`

SysRandom

Purpose Return a random number anywhere from 0 to `sysRandomMax`.

Prototype
```
Int16 SysRandom (Int32 newSeed)
```

Parameters

`newSeed`	New seed value, or 0 to use existing seed.

Result Returns a random number.

SysReset

Purpose Perform a soft reset and reinitialize the globals and the dynamic memory heap.

Prototype
```
void SysReset (void)
```

Parameters None.

Result No return value.

Comments This routine resets the system, reinitializes the globals area and all system managers, and reinitializes the dynamic heap. All database information is preserved. This routine is called when the user presses the hidden reset switch on the device.

When running an application using the simulator, this routine looks for two data files that represent the memory of card 0 and card 1. If these are found, the Palm OS memory image is created using them. If they are not found, they are created.

When running an application on the device, this routine simply looks for the memory cards at fixed locations.

SysSetAutoOffTime

Purpose Set the time out value in seconds for auto-power-off. Zero means never power off.

Prototype
```
UInt16 SysSetAutoOffTime (UInt16 seconds)
```

Parameters

`seconds`	Time out in seconds, or 0 for no time out.

Result Returns previous value of time out in seconds.

SysSetTrapAddress

Purpose Set the address of the function corresponding to a system trap.

Prototype
```
Err SysSetTrapAddress (UInt16 trapNum,
void *procP)
```

Parameters

-> `trapNum`	One of the routine selectors defined in `SysTraps.h` (`sysTrap...`) or `CoreTraps.h` on Palm OS version 3.5 and higher.
-> `procP`	Pointer to a function that the trap identified by `trapNum` is to point to.

Result Returns 0 if no error, or `SysErrParamErr` if `trapNum` is greater than the number of traps in the trap table.

Comments This function is useful for patching a system trap, in combination with `SysGetTrapAddress`. To patch a system trap in your application, first call `SysGetTrapAddress` to get the trap address and then save this value somewhere. Next use `SysSetTrapAddress` to set the trap address to point to your function. Before your application exits, remove the patch by calling `SysSetTrapAddress` and passing in the original trap address you saved.

WARNING! If your application patches a system trap using this function, you **must** remove the patch before your application exits. Do **not** use this mechanism to permanently patch system traps as it may cause unpredictable results for the system and other applications.

SysStringByIndex

Purpose

Copy a string out of a string list resource by index. String list resources are of type `'tSTL'` and contain a list of strings and a prefix string.

ResEdit always displays the items in the list as starting at 1, not 0. Consider this when creating your string list.

Prototype

```
Char *SysStringByIndex (UInt16 resID,
UInt16 index, Char *strP, UInt16 maxLen)
```

Parameters

`resID`	Resource ID of the string list.
`index`	String to get out of the list.
`strP`	Pointer to space to form the string.
`maxLen`	Size of `strP` buffer.

Result

Returns a pointer to the copied string. The string returned from this call will be the prefix string appended with the designated index string. Indices are 0-based; index 0 is the first string in the resource.

Compatibility

Implemented only if 2.0 New Feature Set is present.

SysTaskDelay

Purpose Put the processor into doze mode for the specified number of ticks.

Prototype
```
Err SysTaskDelay (Int32 delay)
```

Parameters

`delay`	Number of ticks to wait (see `SysTicksPerSecond`)

Result Returns 0 if no error.

See Also `EvtGetEvent`

SysTicksPerSecond

Purpose Return the number of ticks per second. This routine allows applications to be tolerant of changes to the ticks per second rate in the system.

Prototype
```
UInt16 SysTicksPerSecond (void)
```

Parameters None

Result Returns the number of ticks per second.

Compatibility Implemented only if 2.0 New Feature Set is present.

SysUIAppSwitch

Purpose Try to make the current UI application quit and then launch the UI application specified by card number and database ID.

Prototype
```
Err SysUIAppSwitch (UInt16 cardNo, LocalID dbID,
UInt16 cmd, MemPtr cmdPBP)
```

Parameters

-> `cardNo`	Card number for the new application; currently only card 0 is valid.
-> `dbID`	ID of the new application's resource database.
-> `cmd`	Action code (launch code).
-> `cmdPBP`	Action code (launch code) parameter block.

Result

Returns 0 if no error.

May display a fatal error message if the `cardNo` parameter is invalid. On debug ROMs, displays a fatal error message if there is no currently running application.

Comments

Do not use this function to open the system-supplied Application Launcher application. If another application has replaced the default launcher with one of its own, this function will open the custom launcher instead of the system-supplied one. To open the system-supplied launcher reliably, enqueue a `keyDownEvent` that contains a `launchChr`, as shown in the section "Application Launcher" of the user interface chapter in the *Palm OS Programmer's Companion*, vol. I.

If you are passing a parameter block (the `cmdPBP` parameter), you must set the owner of the parameter block chunk to the operating system. To do this, and for more information, see `MemPtrSetOwner`. If the parameter block structure contains references by pointer or handle to any other chunks, you also must set the owner of those chunks by using `MemHandleSetOwner` or `MemPtrSetOwner`. If you set the owner of this parameter block properly, the system maintains the parameter block and frees it when the second application quits. If you don't set the owner of the parameter block, the system frees the parameter block as soon as the calling application quits, causing unpredictable results.

See Also

`SysAppLaunch`, Chapter 3, "Application Startup and Stop." in the *Palm OS Programmer's Companion*, vol. I.

Application-Defined Functions

PilotMain

Purpose The entry point for all Palm OS applications, this function's sole purpose is to receive and respond to launch codes.

Prototype

```
UInt32 PilotMain(UInt16 cmd, void *cmdPBP,
UInt16 launchFlags)
```

Parameters

`-> cmd`	The launch code to which your application is to respond. See Chapter 1, "Application Launch Codes," on page 3 for a list of predefined launch codes. You may create additional launch codes; see "Creating Your Own Launch Codes" on page 44 of the *Palm OS Programmer's Companion*, vol. I.
`-> cmdPBP`	A pointer to a structure containing any launch-command-specific parameters, or `NULL` if the launch code has none. See the description of each launch code for a description of the parameter structure that accompanies it, if any.
`-> launchFlags`	Flags that indicate whether your application's global variables are available, whether your application is now the active application, whether it already was the active application, and so on. See "Launch Flags" on page 34 for a list of launch flags.

Result Return `errNone` if your application processed the launch code successfully, or an appropriate error code if there was a problem. When another application invokes your application using `SysAppLaunch`, this value is returned to the caller.

Comments See Chapter 3, "Application Startup and Stop," on page 35 of the *Palm OS Programmer's Companion*, vol. I for a discussion on how applications receive and handle launch codes, with examples.

51

Text Manager

This chapter provides information about the text manager API declared in `TextMgr.h` by discussing these topics:

- Text Manager Data Structures
- Text Manager Functions

For more information on the text manager, see the chapter "Localized Applications" on page 313 in the *Palm OS Programmer's Companion*, vol. I.

Text Manager Data Structures

CharEncodingType

The `CharEncodingType` enum specifies possible character encodings. The Character Encoding Constants define the possible values for `CharEncodingType` variables.

```
UInt8 CharEncodingType;
```

A given device supports a single character encoding. The currently available devices support either the Palm™ version of Windows code page 1252 (an extension of ISO Latin 1) or the Palm version of Windows code page 932 (an extension of Shift JIS). These encodings are identical to their Windows counterparts with some additional characters added in the control range.

Compatibility Prior to version 4.0, `CharEncodingType` was an enum that defined only eight character encodings. The Palm OS® 4.0 definition of `CharEncodingType` is compatible with the previous definition.

Text Manager Functions

TxtByteAttr

Purpose Return the possible locations of a given byte within a multi-byte character.

Prototype

```
UInt8 TxtByteAttr (UInt8 inByte)
```

Parameters

`-> inByte`	A byte representing all or part of a valid character.

Result Returns a byte with one or more of the following bits set:

`byteAttrFirst`	First byte of multi-byte character.
`byteAttrLast`	Last byte of multi-byte character.
`byteAttrMiddle`	Middle byte of multi-byte character.
`byteAttrSingle`	Single-byte character.

Comments If `inByte` is valid in more than one location of a character, multiple return bits are set. For example, 0x40 in the Shift JIS character encoding is valid as a single-byte character and as the low-order byte of a double-byte character. Thus, the return value for `TxtByteAttr(0x40)` on a Shift JIS system has both the `byteAttrSingle` and `byteAttrLast` bits set.

Text manager functions that need to determine the byte positioning of a character use `TxtByteAttr` to do so. You rarely need to use this function yourself.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueByteAttr`. For more information, see Chapter 76, "PalmOSGlue Library."

TxtCaselessCompare

Purpose Perform a case-insensitive comparison of two text buffers.

Prototype

```
Int16 TxtCaselessCompare (const Char* s1,
UInt16 s1Len, UInt16* s1MatchLen, const Char* s2,
UInt16 s2Len, UInt16* s2MatchLen)
```

Parameters

`-> s1`	Pointer to the first text buffer to compare.
`-> s1Len`	Length in bytes of the text pointed to by `s1`.
`<- s1MatchLen`	Points to the offset of the first character in `s1` that determines the sort order. Pass `NULL` for this parameter if you don't need to know this number.
`-> s2`	Pointer to the second text buffer to compare.
`-> s2Len`	Length in bytes of the text pointed to by `s2`.
`<- s2MatchLen`	Points to the offset of the first character in `s2` that determines the sort order. Pass `NULL` for this parameter if you don't need to know this number.

Result Returns one of the following values:

< 0	If `s1` occurs before `s2` in alphabetical order.
> 0	If `s1` occurs after `s2` in alphabetical order.
0	If the two substrings that were compared are equal.

Comments In certain character encodings (such as Shift JIS), one character may be accurately represented as either a single-byte character or a multi-byte character. `TxtCaselessCompare` accurately matches a single-byte character with its multi-byte equivalent. For this reason, the values returned in `s1MatchLen` and `s2MatchLen` are not always equal.

You must make sure that the parameters `s1` and `s2` point to a the start of a valid character. That is, they must point to the first byte of

a multi-byte character or they must point to a single-byte character; if they don't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCaselessCompare`. For more information, see Chapter 76, "PalmOSGlue Library."

In Palm OS 4.0, the `TxtCaselessCompare` function terminates when it finds a null byte in the string. In earlier releases, it terminated only when it reached the ending byte specified by the length parameters.

See Also `StrCaselessCompare`, `TxtCompare`, `StrCompare`

TxtCharAttr

Purpose Return a character's attributes.

Prototype
```
UInt16 TxtCharAttr (WChar inChar)
```

Parameters

`-> inChar`	Any valid character.

Result Returns a 16-bit unsigned value with any of the following bits set:

`charAttrPrint`	Printable
`charAttrSpace`	Blank space, tab, or newline
`charAttrAlNum`	Alphanumeric
`charAttrAlpha`	Alphabetic
`charAttrCntrl`	Control character
`charAttrGraph`	Character that appears on the screen; that is, is not whitespace, a control character, or a virtual character.
`charAttrDelim`	Word delimiter (whitespace or punctuation).

Comments The character passed to this function must be a valid character given the system encoding.

This function is used in the text manager's character attribute macros (`TxtCharIsAlNum`, `TxtCharIsCntrl`, and so on). The macros perform operations analogous to the standard C functions `isPunct`, `isPrintable`, and so on. Usually, you'd use one of these macros instead of calling `TxtCharAttr` directly.

To obtain attributes specific to a given character encoding, use `TxtCharXAttr`.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharAttr`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharIsValid`

TxtCharBounds

Purpose Return the boundaries of a character containing the byte at a specified offset in a string.

Prototype

```
WChar TxtCharBounds (const Char* inText,
UInt32 inOffset, UInt32* outStart, UInt32* outEnd)
```

Parameters

`-> inText`	Pointer to the text buffer to search.
`-> inOffset`	A valid offset into the buffer `inText`. This location may contain a byte in any position (start, middle, or end) of a multi-byte character.
`<- outStart`	Points to the starting offset of the character containing the byte at `inOffset`.
`<- outEnd`	Points to the ending offset of the character containing the byte at `inOffset`.

Result Returns the character located between the offsets `outStart` and `outEnd`.

Comments Use this function to determine the boundaries of a character in a string or text buffer.

If the byte at `inOffset` is valid in more than one location of a character, the function must search back toward the beginning of the text buffer until it finds an unambiguous byte to determine the appropriate boundaries. For this reason, `TxtCharBounds` is often slow and should be used only where needed.

You must make sure that the parameter `inText` points to the beginning of the string. That is, if the string begins with a multi-byte character, `inText` must point to the first byte of that character; if it doesn't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharBounds`. For more information, see Chapter 76, "PalmOSGlue Library."

TxtCharEncoding

Purpose Return the minimum encoding required to represent a character.

Prototype
```
CharEncodingType TxtCharEncoding (WChar inChar)
```

Parameters `-> inChar` A valid character.

Result A `CharEncodingType` value that indicates the minimum encoding required to represent `inChar`. If the character isn't recognizable, `charEncodingUnknown` is returned.

Comments The minimum encoding is the encoding that takes the lowest number of bytes to represent the character. For example, if the character is a blank or a tab character, the minimum encoding is `charEncodingAscii` because these characters can be represented in single-byte ASCII. If the character is a ü, the minimum encoding is `charEncodingISO8859_1`.

Because Palm OS only supports a single character encoding at a time, the result of this function is always logically equal to or less than the encoding used on the current system. That is, you'll only

receive a return value of `charEncodingISO8859_1` if you're running on a US or European system and you pass a non-ASCII character.

Use this function for informational purposes only. Your code should not assume that the character encoding returned by this function is the Palm OS system character encoding. (Instead use `FtrGet` as shown in the `TxtCharXAttr` function description.)

Use `TxtMaxEncoding` to determine the order of encodings.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharEncoding`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtStrEncoding`, `TxtMaxEncoding`

TxtCharIsAlNum

Purpose Macro that indicates if the character is alphanumeric.

Prototype
```
TxtCharIsAlNum (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is a letter in an alphabet or a numeric digit, `false` otherwise.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsAlNum`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharIsDigit`, `TxtCharIsAlpha`

TxtCharIsAlpha

Purpose Macro that indicates if a character is a letter in an alphabet.

Prototype
```
TxtCharIsAlpha (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is a letter in an alphabet, `false` otherwise.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsAlpha`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharIsAlNum`, `TxtCharIsLower`, `TxtCharIsUpper`

TxtCharIsCntrl

Purpose Macro that indicates if a character is a control character.

Prototype
```
TxtCharIsCntrl (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is a non-printable character, such as the bell character or a carriage return; `false` otherwise.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsCntrl`. For more information, see Chapter 76, "PalmOSGlue Library."

TxtCharIsDelim

Purpose Macro that indicates if a character is a delimiter.

Prototype
```
TxtCharIsDelim (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is a word delimiter (whitespace or punctuation), `false` otherwise.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsDelim`. For more information, see Chapter 76, "PalmOSGlue Library."

TxtCharIsDigit

Purpose Macro that indicates if the character is a decimal digit.

Prototype
```
TxtCharIsDigit (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is 0 through 9, `false` otherwise.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsDigit`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharIsAlNum`, `TxtCharIsHex`

TxtCharIsGraph

Purpose Macro that indicates if a character is a graphic character.

Prototype
```
TxtCharIsGraph (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is a graphic character, `false` otherwise.

Comments A graphic character is any character visible on the screen, in other words, letters, digits, and punctuation marks. A blank space is not a graphic character because it is not visible.

This macro differs from `TxtCharIsPrint` in that it returns `false` if the character is whitespace. `TxtCharIsPrint` returns `true` if the character is whitespace.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsGraph`. For more information, see Chapter 76, "PalmOSGlue Library."

TxtCharIsHardKey

Purpose Macro that returns true if the character is one of the hard keys on the device.

Prototype
```
TxtCharIsHardKey (m, ch)
```

Parameters

`-> m`	The modifier keys from the `keyDownEvent`.
`-> ch`	The character from the `keyDownEvent`.

Result `true` if the character is one of the built-in hard keys on the device, `false` otherwise.

Compatibility Valid only if International Feature Set is present.

See Also `ChrIsHardKey`

TxtCharIsHex

Purpose Macro that indicates if a character is a hexadecimal digit.

Prototype
```
TxtCharIsHex (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is a hexadecimal digit from 0 to F, `false` otherwise.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsHex`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharIsDigit`

TxtCharIsLower

Purpose Macro that indicates if a character is a lowercase letter.

Prototype
```
TxtCharIsLower (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is a lowercase letter, `false` otherwise.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsLower`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharIsAlpha`, `TxtCharIsUpper`

TxtCharIsPrint

Purpose Macro that indicates if a character is printable.

Prototype
```
TxtCharIsPrint (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is not a control character, `false` otherwise.

Comments This macro differs from `TxtCharIsGraph` in that it returns `true` if the character is whitespace. `TxtCharIsGraph` returns `false` if the character is whitespace.

If you are using a debug ROM and you pass a virtual character to this macro, a fatal alert is generated.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsPrint`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharIsValid`

TxtCharIsPunct

Purpose Macro that indicates if a character is a punctuation mark.

Prototype
```
TxtCharIsPunct (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is a punctuation mark, `false` otherwise.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsPunct`. For more information, see Chapter 76, "PalmOSGlue Library."

TxtCharIsSpace

Purpose Macro that indicates if a character is a whitespace character.

Prototype
```
TxtCharIsSpace (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is whitespace such as a blank space, tab, or newline; `false` otherwise.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsSpace`. For more information, see Chapter 76, "PalmOSGlue Library."

TxtCharIsUpper

Purpose Macro that indicates if a character is an uppercase letter.

Prototype
```
TxtCharIsUpper (ch)
```

Parameters

`-> ch`	A valid character.

Result Returns `true` if the character is an uppercase letter, `false` otherwise.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsUpper`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharIsAlpha`, `TxtCharIsLower`

TxtCharIsValid

Purpose Determine whether a character is valid character given the Palm OS character encoding.

Prototype
```
Boolean TxtCharIsValid (WChar inChar)
```

Parameters `-> inChar` A character.

Result Returns `true` if `inChar` is a valid character; `false` if `inChar` is not a valid character.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharIsValid`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharAttr`, `TxtCharIsPrint`

TxtCharSize

Purpose Return the number of bytes required to store the character in a string.

Prototype
```
UInt16 TxtCharSize (WChar inChar)
```

Parameters `-> inChar` A valid character.

Result The number of bytes required to store the character in a string.

Comments Although character variables are always two-byte long `WChar` values, in some character encodings such as Shift-JIS, characters in strings are represented by a mix of one or more bytes per character. If the character can be represented by a single byte (its high-order byte is 0), it is stored in a string as a single-byte character.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS,

link with the PalmOSGlue library and call `TxtGlueCharSize`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharBounds`

TxtCharWidth

Purpose Return the width required to display the specified character in the current font. If the specified character does not exist within the current font, the missing character symbol is substituted.

Prototype
```
Int16 TxtCharWidth (WChar inChar)
```

Parameters `-> inChar` A valid character.

Result Returns the width of the specified character (in pixels).

Comments Use `FntWCharWidth` or `FntGlueWCharWidth` instead of this routine.

Compatibility Implemented only if International Feature Set is present.

TxtCharXAttr

Purpose Return the extended attribute bits for a character.

Prototype
```
UInt16 TxtCharXAttr (WChar inChar)
```

Parameters `-> inChar` A valid character.

Result Returns an unsigned 16-bit value with one or more extended attribute bits set. For specific return values, look in the header files that are specific to certain character encodings (`CharLatin.h` or `CharShiftJIS.h`).

Comments To interpret the results, you must know the character encoding being used. Use `FtrGet` with `sysFtrNumEncoding` as the feature

number to determine the character encoding. This returns one of the CharEncodingType values. For example:

```
WChar ch;
UInt16 attr;
UInt32 encoding;
...
attr = TxtCharXAttr(ch);
if (FtrGet(sysFtrCreator, sysFtrNumEncoding,
  &encoding) != errNone)
  encoding = charEncodingPalmLatin;
if (encoding == charEncodingUTF8) {
}
```

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueCharXAttr`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also TxtCharAttr

TxtCompare

Purpose Performs a case-sensitive comparison of all or part of two text buffers.

Prototype

```
Int16 TxtCompare (const Char* s1, UInt16 s1Len,
UInt16* s1MatchLen, const Char* s2, UInt16 s2Len,
UInt16* s2MatchLen)
```

Parameters

`-> s1`	Pointer to the first text buffer to compare.
`-> s1Len`	The length in bytes of the text pointed to by `s1`.
`<- s1MatchLen`	Points to the offset of the first character in `s1` that determines the sort order. Pass `NULL` for this parameter if you don't need to know this number.
`-> s2`	Pointer to the second text buffer to compare.
`-> s2Len`	The length in bytes of the text pointed to by `s2`.

`<- s2MatchLen`	Points to the offset of the first character in `s2` that determines the sort order. Pass `NULL` for this parameter if you don't need to know this number.

Result Returns one of the following values:

< 0	If `s1` occurs before `s2` in alphabetical order.
> 0	If `s1` occurs after `s2` in alphabetical order.
0	If the two substrings that were compared are equal.

Comments This function performs a case-sensitive comparison. If you want to perform a case-insensitive comparison, use `TxtCaselessCompare`.

The `s1MatchLen` and `s2MatchLen` parameters are not as useful for the `TxtCompare` function as they are for the `TxtCaselessCompare` function because `TxtCompare` implements a multi-pass sort algorithm. (See the Compatibility section below for further details.) For example, comparing the string "celery" with the string "Cauliflower" returns a positive value to indicate that "celery" sorts after "Cauliflower," and it returns a match length of 1 to indicate that the second letter determines the sort order ("e" comes after "a"). However, because `TxtCompare` ultimately does a case-sensitive comparison, comparing the string "c" to the string "C" produces a negative result and a match length of 0.

In certain character encodings (such as Shift JIS), one character may be accurately represented as either a single-byte character or a multi-byte character. `TxtCompare` accurately matches a single-byte character with its multi-byte equivalent. For this reason, the values returned in `s1MatchLen` and `s2MatchLen` are not always equal.

You must make sure that the parameters `s1` and `s2` point to the start of a a valid character. That is, they must point to the first byte of a multi-byte character or they must point to a single-byte character; if they don't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS,

link with the PalmOSGlue library and call `TxtGlueCompare`. For more information, see Chapter 76, "PalmOSGlue Library."

Prior to Palm OS 4.0, `TxtCompare` and `StrCompare` only performed one level of comparison and returned as soon as they found two unequal characters. For example, if you compared the string "celery" with the string "Cauliflower," both functions returned a value indicating that "celery" should appear before "Cauliflower" because they sorted "c" before "C."

In Palm OS 4.0, `StrCompare` calls `TxtCompare`, and `TxtCompare` performs a comparison using up to six comparison tables for sorting with increasing precision. As a result, in Palm OS 4.0 and higher, `TxtCompare` sorts "Cauliflower" before "celery." The `TxtGlueCompare` function uses a two-pass sort on pre-4.0 devices, which will also sort "Cauliflower" before "celery."

Palm OS 4.0 sorting of Shift-JIS characters attempts to duplicate the sorting algorithm described by the JIS standard.

In Palm OS 4.0, the `TxtCompare` function terminates when it finds a null byte in the string. In earlier releases, it terminated only when it reached the ending byte specified by the length parameters.

See Also `StrCompare`, `TxtFindString`



New TxtConvertEncoding

Purpose Convert a text buffer from one character encoding to another.

Prototype

```
Err TxtConvertEncoding (Boolean newConversion,
TxtConvertStateType *ioStateP,
const Char *srcTextP, UInt16 *ioSrcBytes,
CharEncodingType srcEncoding, Char *dstTextP,
UInt16 *ioDstBytes, CharEncodingType dstEncoding,
const Char *substitutionStr,
UInt16 substitutionLen)
```

Parameters

-> `newConversion`
`true` if this function call is starting a new conversion, or `false` if this function call is a continuation of a previous conversion.

<-> `ioStateP`
If `newConversion` is `false`, this parameter must point to the same data used for the previous invocation. If `newConversion` is `true` and no subsequent calls are planned, this parameter can be `NULL`.

-> `srcTextP`
A pointer to a text buffer. If `newConversion` is `true`, this must point to the start of a text buffer. If `newConversion` is `false`, it may point to a location in the middle of a text buffer. In either case, it must point to an inter-character boundary.

<-> `ioSrcBytes`
A pointer to the length in bytes of the text in `srcTextP` that needs to be converted. Upon return, contains the number of bytes successfully processed.

-> `srcEncoding`
The character encoding that `srcTextP` currently uses. This should be one of the Character Encoding Constants.

<- `dstTextP`
A pointer to the destination text buffer or `NULL`. This should always point to the location where `TxtConvertEncoding` can begin writing.

<-> `ioDstBytes`
A pointer to the length in bytes of `dstTextP`. Upon return, contains the number of bytes required to represent `srcTextP` in the new encoding.

-> `dstEncoding`
The character encoding to which to convert `srcTextP`. This should be one of the Character Encoding Constants.

-> `substitutionStr`
: A string that should be used to substitute any invalid or inconvertible characters that occur in `srcTextP`. This string must already be in the destination encoding. If `NULL`, the function immediately returns when an invalid character is encountered.

-> `substitutionLen`
: The number of bytes in `substitutionStr`, not including the terminating null byte.

Result Returns `errNone` upon success or one of the following if an error occurs:

`txtErrConvertOverflow`
: The destination buffer is not large enough to contain the converted text.

`txtErrConvertUnderflow`
: The end of the source buffer contains a partial character.

`txtErrNoCharMapping`
: The device does not contain a mapping between the source and destination encodings for at least one of the characters in `srcTextP`.

`txtErrUnknownEncoding`
: One of the specified encodings is not valid. Currently, both the source and destination encodings must match either the device's encoding or one of the Unicode character encodings.

Comments This function converts `ioSrcBytes` of text in `srcTextP` from the `srcEncoding` to the `dstEncoding` character encoding and returns the result in `dstTextP`.

Currently, the focus of `TxtConvertEncoding` is to convert between Unicode-encoded text and the device's character encoding. For this reason, `TxtConvertEncoding` can only handle conversions between the device's encoding and one of UTF-8, UCS-

2, UTF-16LE, or UTF-16BE. If you specify any other character encoding for either the source or the destination buffer, the error code `txtErrUnknownEncoding` is returned.

You can retrieve the device's encoding using the following function:

```
FtrGet(sysFtrCreator, sysFtrNumEncoding,
  &encoding)
```

If you're converting text that was received from the Internet, the encoding's name is passed along with the text data. Use the `TxtNameToEncoding` function to convert the name to a `CharEncodingType` value.

The `dstTextP` buffer must be large enough to hold the result of converting `srcTextP` to the specified encoding. You can pass `NULL` for the `dstTextP` parameter to determine the required length of the buffer before actually doing the conversion. (The required length is returned in `ioDstBytes`.)

If the function encounters an inconvertible character in the source text, it puts `substitutionStr` in the destination buffer in that character's place and continues the conversion. When the conversion is complete, it returns `txtErrNoCharMapping` to indicate that an error occurred. If `substitutionStr` is `NULL`, the function stops the conversion and immediately returns `txtErrNoCharMapping`. `ioSrcBytes` is set to the offset of the inconvertible character, `dstTextP` contains the converted string up to that point, and `ioDstBytes` contains the size of the converted text. You can examine the character at `ioSrcBytes` and choose to move past it and continue the conversion. Follow the rules for making repeated calls to `TxtConvertEncoding` as described in the next paragraph.

You can make repeated calls to `TxtConvertEncoding` in a loop if you only want to convert part of the input buffer at a time. When you make repeated calls to this function, the first call should use `true` for `newConversion`, and `srcTextP` should point to the start of the text buffer. All subsequent calls should use the following values:

`newConversion`	`false`.
`ioStateP`	The same data that was returned by the previous invocation.

`srcTextP`	The location where this call should begin converting. Typically, this would be the previous `srcTextP` plus the number of bytes returned in `ioSrcBytes`. If you are skipping over an inconvertible character, `srcTextP` must point to the character after that location.
<-> `ioSrcBytes`	The number of bytes that this function call should convert.
`dstTextP`	A pointer to a location where this function can begin writing the converted string. You might choose to have each function call write to a different destination buffer. To have successive calls write to the same buffer, pass the previous `dstTextP` plus the number of bytes returned in `ioDstBytes` each time.
`ioDstBytes`	The number of bytes available for output in the `dstTextP` buffer. In other words, the number of bytes remaining.

Compatibility Implemented only if 4.0 New Feature Set is present.

TxtEncodingName

Purpose Obtain a character encoding's name.

Prototype

```
const Char* TxtEncodingName
(CharEncodingType inEncoding)
```

Parameters

-> `inEncoding`	One of the values from `CharEncodingType`, indicating a character encoding.

Result A constant string containing the name of the encoding. The possible return values are defined in `PalmLocale.h`. They are:

`encodingNameAscii`	us ascii
`encodingNameISO8859_1`	ISO-8859-1

`encodingNameCP1252`	ISO-8859-1-Windows-3.1-Latin-1
`encodingNameShiftJIS`	Shift_JIS
`encodingNameCP932`	Windows-31J
`encodingNameUTF8`	UTF-8
`""`	The encoding is not known

Comments Use this function to obtain the official name of the character encoding, suitable to pass to an Internet application or any other application that requires the character encoding's name to be passed along with the data.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueEncodingName`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtNameToEncoding`

TxtFindString

Purpose Perform a case-insensitive search for a string in another string.

Prototype

```
Boolean TxtFindString (const Char* inSourceStr,
const Char* inTargetStr, UInt32* outPos,
UInt16* outLength)
```

Parameters

`-> inSourceStr`	Pointer to the string to be searched.
`-> inTargetStr`	Prepared version of the string to be found. This string should either be passed directly from the `strToFind` field in the `sysAppLaunchCmdFind` launch code's parameter block or it should be prepared using the PalmOSGlue function `TxtGluePrepFindString`.

`<- outPos`	Pointer to the offset of the match in `inSourceStr`.
`<- outLength`	Pointer to the length in bytes of the matching text.

Result Returns `true` if the function finds `inTargetStr` within `inSourceStr`; `false` otherwise.

If found, the values pointed to by the `outPos` and `outLength` parameters are set to the starting offset and the length of the matching text. If not found, the values pointed to by `outPos` and `outLength` are set to 0.

The search that `TxtFindString` performs is locale-dependent. On most ROMs with Latin-based encodings, `TxtFindString` returns `true` only if the string is at the beginning of a word. On Shift-JIS encoded ROMs, `TxtFindString` returns `true` if the string is located anywhere in the word.

Comments Use this function instead of `FindStrInStr` to support the global system find facility. This function contains an extra parameter, `outLength`, to specify the length of the text that matched. Pass this value to `FindSaveMatch` in the `appCustom` parameter. Then when your application receives `sysAppLaunchCmdGoTo`, the `matchCustom` field contains the length of the matching text. You use the length of matching text to highlight the match within the selected record.

You must make sure that the parameters `inSourceStr` and `inTargetStr` point to the start of a valid character. That is, they must point to the first byte of a multi-byte character, or they must point to a single-byte character; if they don't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueFindString`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCaselessCompare`

TxtGetChar

Purpose Retrieve the character starting at the specified offset within a text buffer.

Prototype
```
WChar TxtGetChar (const Char* inText,
UInt32 inOffset)
```

Parameters

`-> inText`	Pointer to the text buffer to be searched.
`-> inOffset`	A valid offset into the buffer `inText`. This offset must point to an inter-character boundary.

Result Returns the character at `inOffset` in `inText`.

Comments You must make sure that the parameter `inText` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueGetChar`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtGetNextChar`, `TxtSetNextChar`

TxtGetNextChar

Purpose Retrieve the character starting at the specified offset within a text buffer.

Prototype
```
UInt16 TxtGetNextChar (const Char* inText,
UInt32 inOffset, WChar* outChar)
```

Parameters

`-> inText`	Pointer to the text buffer to be searched.

`-> inOffset`	A valid offset into the buffer `inText`. This offset must point to an inter-character boundary.
`<- outChar`	The character at `inOffset` in `inText`. Pass `NULL` for this parameter if you don't need the character returned.

Result Returns the size in bytes of the character at `inOffset`. If `outChar` is not `NULL` upon entry, it points to the character at `inOffset` upon return.

Comments You can use this function to iterate through a text buffer character-by-character in this way:

```
UInt16 i = 0;
WChar ch;
while (i < bufferLength) {
    i += TxtGetNextChar(buffer, i, &ch);
    //do something with ch.
}
```

You must make sure that the parameter `inText` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueGetNextChar`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtGetChar`, `TxtGetPreviousChar`, `TxtSetNextChar`

TxtGetPreviousChar

Purpose Retrieve the character before the specified offset within a text buffer.

Prototype

```
UInt16 TxtGetPreviousChar (const Char* inText,
UInt32 inOffset, WChar* outChar)
```

Parameters

`-> inText`	Pointer to the text buffer to be searched.
`-> inOffset`	A valid offset into the buffer `inText`. This offset must point to an inter-character boundary.
`<- outChar`	The character immediately preceding `inOffset` in `inText`. Pass `NULL` for this parameter if you don't need the character returned.

Result Returns the size in bytes of the character preceding `inOffset` in `inText`. If `outChar` is not `NULL` upon entry, then it points to the character preceding `inOffset` upon return. Returns 0 if `inOffset` is at the start of the buffer (that is, `inOffset` is 0).

Comments You can use this function to iterate through a text buffer character-by-character in this way:

```
WChar ch;
/* Find the start of the character containing
the last byte. */
TxtCharBounds (buffer, bufferLength - 1,
&start, &end);
i = start;
while (i > 0) {
    i -= TxtGetPreviousChar(buffer, i, &ch);
    //do something with ch.
}
```

This function is often slower to use than `TxtGetNextChar` because it must determine the appropriate character boundaries if the byte immediately before the offset is valid in more than one location (start, middle, or end) of a multi-byte character. To do this, it must

work backwards toward the beginning of the string until it finds an unambiguous byte.

You must make sure that the parameter `inText` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueGetPreviousChar`. For more information, see Chapter 76, "PalmOSGlue Library."

TxtGetTruncationOffset

Purpose Return the appropriate byte position for truncating a text buffer such that it is at most a specified number of bytes long.

Prototype

```
UInt32 TxtGetTruncationOffset (const Char* inText,
UInt32 inOffset)
```

Parameters

`-> inText`	Pointer to a text buffer.
`-> inOffset`	An offset into the buffer `inText`.

Result Returns the appropriate byte offset for truncating `inText` at a valid inter-character boundary. The return value may be less than or equal to `inOffset`.

Comments You must make sure that the parameter `inText` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueGetTruncationOffset`. For more information, see Chapter 76, "PalmOSGlue Library."



New TxtGetWordWrapOffset

Purpose Locate an appropriate place for a line break in a text buffer.

Prototype

```
UInt32 TxtGetWordWrapOffset (const Char *iTextP,
UInt32 iOffset)
```

Parameters

-> `iTextP`	Pointer to a text buffer.
-> `iOffset`	Pointer to the offset where the search should begin. The search is performed backward starting from this offset.

Result Returns the offset of a character that can begin on a new line (typically, the beginning of the word that contains `iOffset` or last word before `iOffset`). If an appropriate break could not be found, returns `iOffset`.

Comments The `FntWordWrap` and `FntWordWrapReverseNLines` functions call `TxtGetWordWrapOffset` to locate an appropriate place to break the text. The returned offset points to the character that should begin the next line.

This function starts at `iOffset` and works backward until it finds a character that typically occurs between words (for example, white space or punctuation). Then it moves forward until it locates the character that begins a word (typically, a letter or number). Note that this function may return an offset value that is greater than the one passed in if the offset passed in occurs immediately before white space or in the middle of white space.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TxtWordBounds`

TxtMaxEncoding

Purpose Return the higher of two encodings.

Prototype
```
CharEncodingType TxtMaxEncoding
(CharEncodingType a, CharEncodingType b)
```

Parameters

`-> a`	A character encoding to compare.
`-> b`	Another character encoding to compare.

Result Returns the higher of `a` or `b`. One character encoding is higher than another if it is more specific. For example code page 1252 is "higher" than ISO 8859-1 because it represents more characters than ISO 8859-1.

Comments This function is used by `TxtStrEncoding` to determine the encoding required for a string.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueMaxEncoding`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharEncoding`, `CharEncodingType`

New TxtNameToEncoding

Purpose Return an encoding's constant given its name.

Prototype
```
CharEncodingType TxtNameToEncoding
(const Char *iEncodingName)
```

Parameters

`-> iEncodingName`
One of the string constants containing the official name of an encoding. See `TxtEncodingName` for a list.

Result Returns one of the Character Encoding Constants. Returns `charEncodingUnknown` if the specified encoding could not be found.

Comments Use this function to convert a character encoding's name as received from an Internet application into the character encoding constant that some text manager functions require.

This function properly converts aliases for a character encoding. For example, passing the strings "us-ascii", "ASCII", "cp367", and "IBM367" all return `charEncodingAscii`.

The known character encodings are device-dependent. For example, a device with the Shift-JIS encoding will not know all of the aliases for Latin character encodings; however, it will know all of the aliases for Shift-JIS.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TxtEncodingName`

TxtNextCharSize

Purpose Macro that returns the size of the character starting at the specified offset within a text buffer.

Prototype

```
TxtNextCharSize (inText, inOffset)
```

Parameters

`-> inText`	Pointer to the text buffer to be searched.
`-> inOffset`	A valid offset into the buffer `inText`. This offset must point to an inter-character boundary.

Result Returns (as a `UInt16`) the size in bytes of the character at `inOffset`.

Comments You must make sure that the parameter `inText` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueNextCharSize`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtGetNextChar`

TxtParamString

Purpose Replace substrings within a string with the specified values.

Prototype

```
Char* TxtParamString (const Char* inTemplate,
const Char* param0, const Char* param1,
const Char* param2, const Char* param3)
```

Parameters

-> `inTemplate`	The string containing the substrings to replace.
-> `param0`	String to replace ^0 with or `NULL`.
-> `param1`	String to replace ^1 with or `NULL`.
-> `param2`	String to replace ^2 with or `NULL`.
-> `param3`	String to replace ^3 with or `NULL`.

Result Returns a pointer to a locked relocatable chunk in the dynamic heap that contains the appropriate substitutions.

Comments This function searches `inTemplate` for occurrences of the sequences ^0, ^1, ^2, and ^3. When it finds these, it replaces them with the corresponding string passed to this function. Multiple instances of each sequence will be replaced.

The replacement strings can also contain the substitution strings, provided they refer to a later parameter. That is, the `param0` string contain have references to ^1, ^2, and ^3, the `param1` string can have references to ^2 and ^3, and the `param2` string can have references to ^3. Any other occurrences of the substitution strings in the replacement strings are ignored. For example, if `param3` is the string "^0", any occurrences of ^3 in `inTemplate` are replaced with the string "^0".

You must make sure that the parameter `inTemplate` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

`TxtParamString` allocates space for the returned string in the dynamic heap through a call to `MemHandleNew`, and then returns the result of calling `MemHandleLock` with this handle. Your code is responsible for freeing this memory when it is no longer needed.

Compatibility Implemented if 3.5 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueParamString`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtReplaceStr`, `FrmCustomAlert`

TxtPreviousCharSize

Purpose Macro that returns the size of the character before the specified offset within a text buffer.

Prototype

```
TxtPreviousCharSize (inText, inOffset)
```

Parameters

`-> inText`	Pointer to the text buffer.
`-> inOffset`	A valid offset into the buffer `inText`. This offset must point to an inter-character boundary.

Result Returns (as a `UInt16`) the size in bytes of the character preceding `inOffset` in `inText`. Returns 0 if `inOffset` is at the start of the buffer (that is, `inOffset` is 0).

Comments You must make sure that the parameter `inText` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

This macro is often slower to use than `TxtNextCharSize` because it must determine the appropriate character boundaries if the byte

immediately before the offset is valid in more than one location (start, middle, or end) of a multi-byte character. To do this, it must work backwards toward the beginning of the string until it finds an unambiguous byte.

Compatibility Valid only if International Feature Set is present. To use this macro in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGluePreviousCharSize`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtGetPreviousChar`

TxtReplaceStr

Purpose Replace a substring of a given format with another string.

Prototype

```
UInt16 TxtReplaceStr (Char* ioStr,
UInt16 inMaxLen, const Char* inParamStr,
UInt16 inParamNum)
```

Parameters

`<-> ioStr`	The string in which to perform the replacing.
`-> inMaxLen`	The maximum length in bytes that `ioStr` can become.
`-> inParamStr`	The string that ^*inParamNum* should be replaced with. If `NULL`, no changes are made.
`-> inParamNum`	A single-digit number (0 to 9).

Result Returns the number of occurrences found and replaced.

Returns a fatal error message if `inParamNum` is greater than 9.

Comments This function searches `ioStr` for occurrences of the string ^*inParamNum*, where *inParamNum* is any digit from 0 to 9. When it finds the string, it replaces it with `inParamStr`. Multiple instances will be replaced as long as the resulting string doesn't contain more than `inMaxLen` bytes, not counting the terminating null.

You can set the `inParamStr` parameter to `NULL` to determine the required length of `ioStr` before actually doing the replacing. `TxtReplaceStr` returns the number of occurrences it finds of

^*inParamNum*. Multiply this value by the length of the `inParamStr` you intend to use to determine the appropriate length of `ioStr`.

You must make sure that the parameter `ioStr` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueReplaceStr`. For more information, see Chapter 76, "PalmOSGlue Library."

TxtSetNextChar

Purpose Set a character within a text buffer.

Prototype

```
UInt16 TxtSetNextChar (Char* ioText,
UInt32 inOffset, WChar inChar)
```

Parameters

`<-> ioText`	Pointer to a text buffer.
`-> inOffset`	A valid offset into the buffer `inText`. This offset must point to an inter-character boundary.
`-> inChar`	The character to replace the character at `inOffset` with. Must not be a virtual character.

Result Returns the size of `inChar`.

Comments This function replaces the character in `ioText` at the location `inOffset` with the character `inChar`. Note that there must be enough space at `inOffset` to write the character.

You can use `TxtCharSize` to determine the size of `inChar`.

You must make sure that the parameter `ioText` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueSetNextChar`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtGetNextChar`

TxtStrEncoding

Purpose Return the encoding required to represent a string.

Prototype

```
CharEncodingType TxtStrEncoding
(const Char* inStr)
```

Parameters `-> inStr` A string.

Result A `CharEncodingType` value that indicates the encoding required to represent `inChar`. If any character in the string isn't recognizable, then `charEncodingUnknown` is returned.

Comments The encoding for the string is the maximum encoding of any character in that string. For example, if a two-character string contains a blank space and a ü, the appropriate encoding is `charEncodingISO8859_1`. The blank space's minimum encoding is ASCII. The minimum encoding for the ü is ISO 8859-1. The maximum of these two encodings is ISO 8859-1.

Because Palm OS only supports a single character encoding at a time, the results of this function is always logically equal to or less than the encoding used on the current system. That is, you'll only receive a return value of `charEncodingISO8859_1` if you're running on a Latin-based system.

Use this function for informational purposes only. Your code should not assume that the character encoding returned by this function is the Palm OS system's character encoding. (Instead use `FtrGet` as shown in the `TxtCharXAttr` function description.)

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS,

link with the PalmOSGlue library and call `TxtGlueStrEncoding`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `TxtCharEncoding`, `TxtMaxEncoding`

TxtTransliterate

Purpose Converts the specified number of bytes in a text buffer using the specified operation.

Prototype

```
Err TxtTransliterate (const Char* inSrcText,
UInt16 inSrcLength, Char* outDstText,
UInt16* ioDstLength, TranslitOpType inOp)
```

Parameters

`-> inSrcText` Pointer to a text buffer.

`-> inSrcLength` The length in bytes of `inSrcText`.

`<- outDstText` The output buffer containing the converted characters.

`<->ioDstLength` Upon entry, the maximum length of `outDstText`. Upon return, the actual length of `outDstText`.

`-> inOp` A 16-bit unsigned value that specifies which transliteration operation is to be performed. The values possible for this field are specific to the character encoding used on a particular device. These operations are universally available:

`translitOpUpperCase`
: Converts the character to uppercase letters.

`translitOpLowerCase`
: Converts the characters to lowercase letters.

`translitOpPreprocess`
: Don't actually perform the operation. Instead, return in `ioDstLength` the amount of space required for the output text.

Result Returns one of the following values:

`errNone` Success

`txtErrUknownTranslitOp`
`inOp`'s value is not recognized

`txtErrTranslitOverrun`
`inSrcText` and `outDstText` point to the same memory location and the operation has caused the function to overwrite unprocessed data in the input buffer.

`txtErrTranslitOverflow`
`outDstText` is not large enough to contain the converted string.

`txtErrTranslitUnderflow`
The end of the source buffer contains a partial character.

Comments `inSrcText` and `outDstText` may point to the same location if you want to perform the operation in place. However, you should be careful that the space required for `outDstText` is not larger than `inSrcText` so that you don't generate a `txtErrTranslitOverrun` error.

For example, suppose on a Shift JIS encoded system, you want to convert a series of single-byte Japanese Katakana symbols to double-byte Katakana symbols. You cannot perform this operation in place because it replaces a single-byte character with a multi-byte character. When the first converted character is written to the buffer, it overwrites the second input character. Thus, a text overrun has occurred.

You can ensure that you have enough space for the output by OR-ing your chosen operation with `translitOpPreprocess`. For example, to convert a string to uppercase letters, do the following:

```
outSize = buf2Len;
error = TxtTransliterate(buf1, buf1len, &buf2,
&outSize,
translitOpUpperCase|translitOpPreprocess);
```

```
if (outSize > buf2len)
    /* allocate more memory for buf2 */
error = TxtTransliterate(buf1, buf1Len, &buf2,
&outSize, translitOpUpperCase);
```

You must make sure that the parameter `inSrcText` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueTransliterate`. For more information, see Chapter 76, "PalmOSGlue Library."

In Palm OS 4.0, the `TxtTransliterate` function terminates when it finds a null byte in the source string. In earlier releases, it terminated only when it reached the ending byte specified by the length parameter.

TxtWordBounds

Purpose Find the boundaries of a word of text that contains the character starting at the specified offset.

Prototype

```
Boolean TxtWordBounds (const Char* inText,
UInt32 inLength, UInt32 inOffset,
UInt32* outStart, UInt32* outEnd)
```

Parameters

`-> inText`	Pointer to a text buffer.
`-> inLength`	The length in bytes of the text pointed to by `inText`.
`-> inOffset`	A valid offset into the text buffer `inText`. This offset must point to the beginning of a character.
`<- outStart`	The starting offset of the text word.
`<- outEnd`	The ending offset of the text word.

Result Returns `true` if a word is found. Returns `false` if the word doesn't exist or is punctuation or whitespace.

Comments Assuming the ASCII encoding, if the text buffer contains the string "Hi! How are you?" and you pass 5 as the offset, `TxtWordBounds` returns the start and end of the word containing the character at offset 5, which is the character "o". Thus, `outStart` and `outEnd` would point to the start and end of the word "How".

You must make sure that the parameter `inText` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character; if it doesn't, results are unpredictable.

Compatibility Implemented only if International Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `TxtGlueWordBounds`. For more information, see Chapter 76, "PalmOSGlue Library."

In Palm OS 4.0, the `TxtWordBounds` function terminates when it finds a null byte in the string. In earlier releases, it terminated only when it reached the ending byte specified by the length parameter.

See Also `TxtCharBounds`, `TxtCharIsDelim`, `TxtGetWordWrapOffset`

52

Text Services Manager

This chapter provides information about the Text Services Manager API as declared in `TextServicesMgr.h`. The Text Services Manager provides the caller with an API for interacting with various text services, including front-end processors (FEPs), which are sometimes known as input methods. This chapter discusses the following topics:

- Text Services Manager Data Structures
- Text Services Manager Functions

Text Services Manager Data Structures

TsmFepModeType

The `TsmFepModeType` type specifies the input modes used by the functions `TsmGetFepMode` and `TsmSetFepMode`.

```
typedef UInt16 TsmFepModeType;
```

`TsmFepModeType` can be set to one of the defined modes listed in the following table.

Constant	Value	Description
`tsmFepModeDefault`	0	The default input mode for the FEP. For example, with the Japanese FEP, the default mode is Hiragana.

Constant	Value	Description
`tsmFepModeOff`	1	Indicates that there is no active FEP input mode (the FEP is off).
`tsmFepModeCustom`	128	A custom FEP input mode. You can have more than one custom mode; the starting value is 128. Katagana is an example of a custom input mode for the Japanese FEP.

Text Services Manager Functions

TsmGetFepMode

Purpose Return the current input mode for the active front-end processor (FEP).

Prototype

```
TsmFepModeType TsmGetFepMode (void *nullParam)
```

Parameters

`-> nullParam` An unused status pointer that must be set to `NULL`.

Result If there is an active FEP, returns the current mode for the active FEP. If there is no active FEP, returns `tsmFepModeOff`.

Comments The most common use for this function is to save the current FEP mode. You could then call `TsmSetFepMode` to set the current mode to "off" and again to restore the saved mode once the application has finished using a special text field.

Compatibility Implemented only if 3.5 New Feature Set is present. In Palm OS 3.5, the `nullParam` parameter takes a non-`NULL` value, allowing the caller to maintain its own status record. In Palm OS 4.0, this parameter is unused and must be set to `NULL`. Any other value generates a non-fatal alert.

See Also `TsmSetFepMode`

TsmSetFepMode

Purpose Set the input mode for the active front-end processor (FEP) to be the mode defined by the parameter `inNewMode`.

Prototype

```
TsmFepModeType TsmSetFepMode (void *nullParam,
TsmFepModeType inNewMode)
```

Parameters

`-> nullParam`	An unused status pointer that must be set to `NULL`.
`-> inNewMode`	The new FEP input mode.

Result Returns the previous input mode. If there is no active FEP, returns `tsmFepModeOff`.

Comments The most common use for this function is to set the FEP mode to "off" while the application is using a special text field, and then to restore the previous mode. See `TsmGetFepMode` for more information on saving and restoring the FEP mode.

One common reason for explicitly disabling the FEP in code is when a text field will only contain 7-bit ASCII (numeric fields automatically turn off the FEP). For example, if the application has a password field and the contents of that field will always be 7-bit ASCII, the application should turn off the FEP to help prevent the user from entering invalid characters into the field.

Another common case occurs when the application has a numeric field, but cannot just rely on the numeric field attribute. For example, if you want the user to be able to enter the minus ("-") sign, you cannot use a numeric field because the field code prevents the user from entering this character: its not a digit or a period. In this case, you should make it a regular field and have the application screen the characters. The application should disable the FEP when such a pseudo-numeric field is active.

IMPORTANT: A mode change is currently enqueued as a keyDown event so that the field and FEP can remain properly synchronized and so that the mode change doesn't affect any pending keyDown events. The mode change does not happen until the enqueued keyDown event is passed to `FrmHandleEvent`; if you call `TsmGetFepMode` immediately after calling `TsmSetFepMode`, you won't see a mode change.

There are also some current limitations with changing the mode: there must be an active form; and if there is an active field in the form, it must not be a numeric field.

Compatibility Implemented only if 3.5 New Feature Set is present. In Palm OS 3.5, the `nullParam` parameter takes a non-`NULL` value, allowing the caller to maintain its own status record. In Palm OS 4.0, this parameter is unused and must be set to `NULL`. Any other value generates a non-fatal alert.

See Also `TsmGetFepMode`, `EvtEnqueueKey`, `FrmHandleEvent`

53

Time Manager

This chapter provides reference material for the time manager.

- Time Manager Data Structures
- Time Manager Functions

The header file `DateTime.h` declares the API that this chapter describes. For more information on the time manager, see the section "Time" in the *Palm OS Programmer's Companion*, vol. I.

Time Manager Data Structures

The time manager uses these structures to store information.

DateFormatType

The `DateFormatType` enum specifies the different display formats for date values.

```
typedef enum {
    dfMDYWithSlashes,
    dfDMYWithSlashes,
    dfDMYWithDots,
    dfDMYWithDashes,
    dfYMDWithSlashes,
    dfYMDWithDots,
    dfYMDWithDashes,
    dfMDYLongWithComma,
    dfDMYLong,
    dfDMYLongWithDot,
    dfDMYLongNoDay,
    dfDMYLongWithComma,
    dfYMDLongWithDot,
    dfYMDLongWithSpace,
    dfMYMed,
    dfMYMedNoPost,
    dfMDYWithDashes
} DateFormatType;
```

Value Descriptions

`dfMDYWithSlashes`	The month, day, and year numbers separated by slashes. For example, `12/31/95`. This is considered a short format.
`dfDMYWithSlashes`	The day, month, and year numbers separated by slashes. For example, `31/12/95`. This is considered a short format.
`dfDMYWithDots`	The day, month, and year numbers separated by dots. For example, `31.12.95`. This is considered a short format.
`dfDMYWithDashes`	The day, month, and year numbers separated by dashes. For example, `31-12-95`. This is considered a short format.
`dfYMDWithSlashes`	The year, month, and day numbers separated by slashes. For example, `95/12/31`. This is considered a short format.
`dfYMDWithDots`	The year, month, and day numbers separated by dots. For example, `95.12.31`. This is considered a short format.
`dfYMDWithDashes`	The year, month, and day numbers separated by dashes. For example, `95-12-31`. This is considered a short format.

`dfMDYLongWithComma`	The month, day, and year in long format, with a comma. For example, `Dec 31, 1995.` This is considered a long format.
`dfDMYLong`	The month, day, and year in long format. For example, `31 Dec 1995.` This is considered a long format.
`dfDMYLongWithDot`	The month, day, and year in long format, with a dot. For example, `31. Dec 1995.` This is considered a long format.
`dfDMYLongNoDay`	The month and year in long format. For example, `Dec 1995.` This is considered a long format.
`dfDMYLongWithComma`	The day, month, and year in long format, with a comma. For example, `31 Dec, 1995.` This is considered a long format.
`dfYMDLongWithDot`	The year, month, and day in long format with dot separators. For example, `1995.12.31.` This is considered a long format.
`dfYMDLongWithSpace`	The year, month, and day in long format with space separators. For example, `1995 Dec 31.` This is considered a long format.

`dfMYMed`	The month in long format with the two-digit year, preceded by an apostrophe. For example, `Dec '95`. This is considered a medium format.
`dfMYMedNoPost`	The month in long format with the two-digit year. For example, `Dec 95`. This is considered a medium format.
`dfMDYWithDashes`	The month, day, and year numbers separated by dashes. For example, `12-31-95`. This is considered a short format.

Compatibility The `dfMDYWithDashes` constant is defined in Palm OS® 4.0 and higher.

DateTimeType

The `DateTimeType` structure represents a date and time value.

```
typedef struct{
  Int16    second;
  Int16    minute;
  Int16    hour;
  Int16    day;
  Int16    month;
  Int16    year;
  Int16    weekDay;
} DateTimeType
typedef DateTimeType *DateTimePtr;
```

Field Descriptions

second The number of seconds. This is a value between 0 and 59.

`minute` The number of minutes. This is a value between 0 and 59.

`hour` The number of hours. This is a value between 0 and 23.

`day` The day number. This is a value between 1 and 31.

`month` The month number. This is a value between 1 and 12.

`year` The year number.

weekDay The day number. This represents the number of days since Sunday and is thus a value between 0 and 6.

DateType

The `DateType` structure represents a date value.

```
typedef struct{
  UInt16 year   :7;
  UInt16 month  :4;
  UInt16 day    :5;
} DateType;

typedef DateType *DatePtr;
```

Field Descriptions

`year` The number of years since 1904.

Note that this is the format used on Macintosh computers.

`month` The month number. This is a value between 1 and 12.

`day` The day number. This is a value between 1 and 31.

DaylightSavingsTypes

The `DaylightSavingsTypes` enum specifies the different forms of daylight savings times that you can specify for date and time values.

Note that the table uses "DST" to represent daylight savings time.

```
typedef enum {
    dsNone,
    dsUSA,
    dsAustralia,
    dsWesternEuropean,
    dsMiddleEuropean,
    dsEasternEuropean,
    dsGreatBritain,
    dsRumania,
    dsTurkey,
    dsAustraliaShifted
} DaylightSavingsTypes;
```

Value Descriptions

`dsNone`	No DST (daylight savings time)
`dsUSA`	U.S.A. DST
`dsAustralia`	Australian DST
`dsWesternEuropean`	Western European DST
`dsMiddleEuropean`	Middle European DST
`dsEasternEuropean`	Eastern European DST
`dsGreatBritain`	Great Britain and Eire DST
`dsRumania`	Rumanian DST
`dsTurkey`	Turkish DST
`dsAustraliaShifted`	Australian DST, with the 1986 shift

Compatibility

If 4.0 New Feature Set is present, this data type is obsolete. In versions 4.0 and higher, Palm OS represents daylight savings time as an integer value that gives the number of minutes to add to the current time for daylight savings time.

DayOfMonthType

The `DayOfMonth` enum specifies the different day-of-the-week numeric values that are returned by the `DayOfMonth` function. These values are used to represent repeating appointments that occur on specific days of the month; for example, the first Friday or the third Tuesday of each month.

```
typedef enum {
    dom1stSun, dom1stMon, dom1stTue, dom1stWen,
      dom1stThu, dom1stFri, dom1stSat,
    dom2ndSun, dom2ndMon, dom2ndTue, dom2ndWen,
      dom2ndThu, dom2ndFri, dom2ndSat,
    dom3rdSun, dom3rdMon, dom3rdTue, dom3rdWen,
      dom3rdThu, dom3rdFri, dom3rdSat,
    dom4thSun, dom4thMon, dom4thTue, dom4thWen,
      dom4thThu, dom4thFri, dom4thSat,
    domLastSun, domLastMon, domLastTue,
      domLastWen, domLastThu, domLastFri,
      domLastSat
} DayOfWeekType;
```

Value Descriptions

`dom1stSun`	The first Sunday of the month.
`dom1stMon`	The first Monday of the month.
`dom1stTue`	The first Tuesday of the month.
`dom1stWen`	The first Wednesday of the month.
`dom1stThu`	The first Thursday of the month.
`dom1stFri`	The first Friday of the month.
`dom1stSat`	The first Saturday of the month.
`dom2ndSun`	The second Sunday of the month.
`dom2ndMon`	The second Monday of the month.
`dom2ndTue`	The second Tuesday of the month.
`dom2ndWen`	The second Wednesday of the month.

`dom2ndThu`	The second Thursday of the month.
`dom2ndFri`	The second Friday of the month.
`dom2ndSat`	The second Saturday of the month.
`dom3rdSun`	The third Sunday of the month.
`dom3rdMon`	The third Monday of the month.
`dom3rdTue`	The third Tuesday of the month.
`dom3rdWen`	The third Wednesday of the month.
`dom3rdThu`	The third Thursday of the month.
`dom3rdFri`	The third Friday of the month.
`dom3rdSat`	The third Saturday of the month.
`dom4thSun`	The fourth Sunday of the month.
`dom4thMon`	The fourth Monday of the month.
`dom4thTue`	The fourth Tuesday of the month.
`dom4thWen`	The fourth Wednesday of the month.
`dom4thThu`	The fourth Thursday of the month.
`dom4thFri`	The fourth Friday of the month.
`dom4thSat`	The fourth Saturday of the month.
`domLastSun`	The last Sunday of the month.
`domLastMon`	The last Monday of the month.
`domLastTue`	The last Tuesday of the month.
`domLastWen`	The last Wednesday of the month.
`domLastThu`	The last Thursday of the month.
`domLastFri`	The last Friday of the month.
`domLastSat`	The last Saturday of the month.

Compatibility On Palm OS versions earlier than 4.0, this type was named `DayOfWeekType`.

TimeFormatType

The `TimeFormatType` enum specifies the different display formats for time values.

```
typedef enum
  {
      tfColon,
      tfColonAMPM,
      tfColon24h,
      tfDot,
      tfDotAMPM,
      tfDot24h,
      tfHoursAMPM,
      tfHours24h,
      tfComma24h,
  } TimeFormatType;

typedef TimeFormatType *TimeFormatPtr;
```

Value Descriptions

`tfColon`	The hour and minutes separated by a colon character. For example, `1:00`.
`tfColonAMPM`	The hour and minutes separated by a colon and followed by an AM/PM indication. For example, `1:00 pm`.
`tfColon24h`	The 24-hour time with the hour and minutes separated by a colon character. For example, `13:00`.
`tfDot`	The hour and minutes separated by a dot character. For example, `1.00`.
`tfDotAMPM`	The hour and minutes separated by a period and followed by an AM/PM indication. For example, `1.00 pm`.

`tfDot24h`	The 24-hour time with the hour and minutes separated by a dot character. For example, `13.00`.
`tfHoursAMPM`	The hour value followed by an AM/PM indication. For example, `1 pm`.
`tfHours24h`	The 24-hour value, followed by an AM/PM indication. For example, `13`.
`tfComma24h`	The 24-hour time with the hour and minutes separated by a comma character. For example, `13,00`.

TimeType

The `TimeType` structure represents a time value.

```
typedef struct {
  UInt8   hours;
  UInt8   minutes;
} TimeType;
typedef TimeType *TimePtr;
```

Field Descriptions

`hours`	The number of hours. This is a value between 0 and 23.
`minutes`	The number of minutes. This is value between 0 and 59.

Time Manager Constants

The following table shows the constants that represent the maximum lengths of strings returned by the date and time formatting routines `DateToAscii`, `DateToDOWDMFormat`, and `TimeToAscii`.

Constant	Value	Description
`dateStringLength`	9	Maximum length of the string returned by `DateToAscii` for short date formats.
`longDateStrLength`	15	Maximum length of the string returned by `DateToAscii` for medium and long date formats.
`timeStringLength`	9	Maximum length of the string returned by `TimeToAscii`.
`dowDateStringLength`	19	Maximum length of the string returned by `DateToDOWDMFormat` for short date formats.
`dowLongDateStrLength`	25	Maximum length of the string returned by `DateToDOWDMFormat` for both medium and long date formats.

Time Manager Functions

DateAdjust

Purpose Return a new date +/- the days adjustment.

Prototype
```
void DateAdjust (DatePtr dateP, Int32 adjustment)
```

Parameters

<-> `dateP`	A pointer to a `DateType` structure with the date to be adjusted.
`-> adjustment`	The number of days by which to adjust the date.

Result Returns nothing. Upon return, `dateP` contains the adjusted date.

Comments This function advances the date and manages month and year wrapping conditions.

See Also `TimAdjust`

DateDaysToDate

Purpose Converts a date specified as the number of days since January 1, 1904 to a DateType structure.

Prototype
```
void DateDaysToDate (UInt32 days, DatePtr date)
```

Parameters

`-> days`	The number of days since 1/1/1904.
`<-> date`	A pointer to a DateType structure that is updated with the computed date values.

Result Returns nothing. Upon return, the date information is returned in the structure referenced by the `date` parameter.

See Also DateSecondsToDate, DateToDays

DateSecondsToDate

Purpose Converts a date specified as the number of seconds since January 1, 1904 to a DateType structure.

Prototype
```
void DateSecondsToDate (UInt32 seconds,
DatePtr date)
```

Parameters

`-> seconds`	The number of seconds since 1/1/1904.
`<- date`	A pointer to a DateType structure that is updated with the computed date values.

Result Returns nothing. The structure referenced by the `date` parameter is updated with the date information.

See Also DateDaysToDate, DateToDays

DateTemplateToAscii

Purpose Convert the specified date values into a string that is formatted according to a formatting template specification.

Prototype

```
UInt16 DateTemplateToAscii (const Char *templateP,
UInt8 months, UInt8 days, UInt16 years,
Char *stringP, Int16 stringLen)
```

Parameters

`-> templateP`	A pointer to the template string used to format the date. The maximum length of this string is `maxDateTemplateLen` bytes, excluding the null terminator character. See the Comments section below for details on how to specify date formatting in this template string.
`-> months`	The month number, which must be a value between `1` and `12`.
`-> days`	The day number, which must be a value between `1` and `31`.
`-> years`	The four-digit year number. For example, `1995`.
`<- stringP`	A pointer to a string that is updated with the result. If `stringP` is `NULL`, this function does not write an output string; however, it does return the length required for the output string. If `stringP` is not `NULL`, this function writes the formatted string to `stringP`, writing up to `stringSize` bytes into `stringP`.
`-> stringLen`	The size of the `stringP` buffer.

Result The length of the formatted string, without the terminating `NULL` byte.

The `DateTemplateToAscii` returns the required length of the formatted string even if the `stringP` parameter is `NULL`; this allows you to determine the buffer size at runtime.

Comments

This function is intended as a replacement for the `DateToAscii` and `DateToDOWDMFormat` functions.

This function uses the formatting template referenced by `templateP` to create a formatted string from the date values that you pass in.

You specify a series of formatting substrings in `templateP`. Each substring has the form:

```
^<valueType><formatModifier>
```

Each substring has three components:

- The ^ character begins a substring.
- The `<valueType>` component is a single-digit value that specifies the value type.
- The `<formatModifier>` component is a single-letter value that specifies how you want that value formatted.

The following is an example of a template specification with three substrings:

```
^0z ^2l ^4r
```

Table 53.1 shows the values you can specify for the `<valueType>` component. Note that the formatted result depends on the `<modifier>` value.

Table 53.1 Template value types for the `DateTemplateToAscii` function

Value	Value type	Formatted examples
0	Day number	1, 01, 23, 31
1	Day name	Tue, Tuesday
2	Month name	May, Aug, August
3	Month number	4, 04, 11
4	Year number	97, 1997

Table 53.2 shows the values you can specify for the <modifier> component of each template substring.

Table 53.2 Template modifier types for the `DateTemplateToAscii` function

Modifier	Description
s	Formats the value in short form
r	Formats the value in regular form
l	Formats the value in long form
z	Adds a leading zero to the formatted numeric value

Finally, Table 53.3 shows examples of each value type formatted with each modifier type.

Table 53.3 Examples of formatted values

Value type	Raw value	`s` (Short format)	`r` (Regular format)	`l` (Long format)	`z` (Zero format)
`0` **(Day number)**	`2`	2	2	2	02
`1` **(Day name)**	`2`	T	Tue	Tuesday	n/a
`2` **(Month name)**	`11`	N	Nov	November	n/a
`3` **(Month number)**	`11`	11	11	11	11
`4` **(Year number)**	`2000`	00	2000	2000	n/a

For example, calling `DateTemplateToAscii` as follows:

```
DateTemplateToAscii("^0z ^2l ^4r", 2, 7,
  2000, myStr, 20)
```

Produces the following formatted string:

```
07 February 2000
```

Compatibility Implemented only if 3.5 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `DateGlueTemplateToAscii`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `DateToAscii`, `DateToDOWDMFormat`

DateToAscii

Purpose Convert the passed date to a string using the format specified by the `dateFormat` parameter.

Prototype

```
void DateToAscii (UInt8 months, UInt8 days,
UInt16 years, DateFormatType dateFormat,
Char *pString)
```

Parameters

`-> months`	The month number, which must be a value between `1` and `12`.
`-> days`	The day number, which must be a value between `1` and `31`.
`-> years`	The four-digit year number. For example, `1995`.
`-> dateFormat`	Any `DateFormatType` format.
`<- pString`	A pointer to string that is updated with the result. This string must be of length `dateStringLength` for short formats or `longDateStrLength` for medium or long formats. Note that these lengths do include the terminating `NULL` byte. For more information about required string lengths, see Time Manager Constants.

Result Returns nothing. The string reference by `pString` is updated with the formatted string.

Comments If you are using a debug ROM, the string buffer is filled with either `dateStringLength` or `longStrLength` debugging bytes, depending on the value of the `dateFormat` parameter.

It is important to allocate enough space for your string buffer. Finding buffer overflow errors can be difficult when using a debug ROM. One common situation is when you pass a buffer that is too small from a form, for an element such as a label or title. Then, the buffer overflow can clobber objects that follow the form in memory. When a form element's location information is corrupted, it disappears from the display.

Note that you can use the `DateTemplateToAscii` function instead of this function if the 3.5 feature set is present. You can call the `DateTemplateToAscii` function with a NULL string buffer to predetermine the required size for your buffer.

See Also `TimeToAscii`, `DateToDOWDMFormat`, `DateTemplateToAscii`

DateToDays

Purpose Convert the `DateType` structure to the number of days elapsed from January 1, 1904.

Prototype
```
UInt32 DateToDays (DateType date)
```

Parameters

`-> date`	A `DateType` structure.

Result Returns the number of days elapsed from January 1, 1904 to the specified `date`.

See Also `DateDaysToDate`

DateToDOWDMFormat

Purpose

Convert a date to a formatted string using the format specified by the `dateFormat` parameter. The resultant string includes the name of the day of the week.

Prototype

```
void DateToDOWDMFormat (UInt8 months, UInt8 days,
UInt16 years, DateFormatType dateFormat,
Char *pString)
```

Parameters

`-> months`	The month number, which must be a value between `1` and `12`.
`-> days`	The day number, which must be a value between `1` and `31`.
`-> years`	The four-digit year number. For example, `1995`.
`-> dateFormat`	Any `DateFormatType` format.
`<- pString`	A pointer to a string that is updated with the result. The string must be of length `dowDateStringLength` for short formats or `dowLongDateStrLength` for medium or long date formats. See Time Manager Constants for string buffer lengths.

Result

Returns nothing. The string referenced by `pString` is updated with the formatted string.

Comments

The values of some of the Time Manager Constants that specify the required string buffer lengths do change from time to time. You should always use the constants or verify the required lengths by checking the `datetime.h` file.

It is important to allocate enough space for your string buffer. Finding buffer overflow errors can be difficult when using a debug ROM. One common situation is when you pass a buffer that is too small from a form, for an element such as a label or title. Then, the buffer overflow can clobber objects that follow the form in memory.

When a form element's location information is corrupted, it disappears from the display.

Note that you can use the DateTemplateToAscii function instead of this function if the 3.5 feature set is present. You can call the DateTemplateToAscii function with a NULL string buffer to predetermine the required size for your buffer.

Compatibility On Palm OS 3.1 Japanese ROMs, this function contains a bug that prevented it from properly displaying 4-byte long day names. To prevent this bug from affecting your application, use DateGlueToDOWDMFormat in the PalmOSGlue library instead of calling this function directly. For more information, see Chapter 76, "PalmOSGlue Library."

See Also DateToAscii, DateTemplateToAscii

DayOfMonth

Purpose Return a value that represents the day of a month on which the specified date occurs. The value represents a quantity such as "First Monday" or "Third Friday" as is used for repeating appointments in the Datebook.

Prototype

```
Int16 DayOfMonth (Int16 month, Int16 day,
Int16 year)
```

Parameters

-> month	The month number, which must be a value between 1 and 12.
-> day	The day number, which must be a value between 1 and 31.
-> year	The four-digit year number. For example, 1995.

Result Returns a value that represents day of the month. This value is one of the DayOfMonthType values.

Comments The returns value can be used to specify on which day of the month an appointment repeats.

DayOfWeek

Purpose Return the day of the week value for a specified date.

Prototype
```
Int16 DayOfWeek (Int16 month, Int16 day,
Int16 year)
```

Parameters

`-> month`	The month number, which must be a value between `1` and `12`.
`-> day`	The day number, which must be a value between `1` and `31`.
`-> year`	The four-digit year number. For example, `1995`.

Result Returns one of the following values for the day of the week of the specified date, as shown in the following table:

Day name	Returned day value
Sunday	`0`
Monday	`1`
Tuesday	`2`
Wednesday	`3`
Thursday	`4`
Friday	`5`
Saturday	`6`

DaysInMonth

Purpose Return the number of days in the month.

Prototype
```
Int16 DaysInMonth (Int16 month, Int16 year)
```

Parameters

`-> month`	The month number, which must be a value between `1` and `12`.
`-> year`	The four-digit year number. For example, `1995`.

Result Returns the number of days in the month for the specified year.

TimAdjust

Purpose Return a new date, with the time adjusted by the specified number of seconds.

Prototype

```
void TimAdjust (DateTimePtr dateTimeP,
Int32 adjustment)
```

Parameters

`<-> dateTimeP`	A pointer to a `DateType` structure.
`-> adjustment`	The number of seconds by which to adjust the time.

Result Returns nothing. The structure referenced by `dateTimeP` is modified to contain the updated date and time.

Comments This function advances the time by the specified number of seconds and takes care of any wraparound conditions.

See Also `DateAdjust`

TimDateTimeToSeconds

Purpose Return the number of seconds elapsed from 12:00 A.M. on January 1, 1904 to the specified date and time.

Prototype

```
UInt32 TimDateTimeToSeconds
(DateTimePtr dateTimeP)
```

Parameters

`-> dateTimeP`	A pointer to a `DateTimeType` structure.

Result The number of seconds elapsed from 12:00 A.M. on January 1, 1904 to the date referenced by `dateTimeP`.

See Also `TimSecondsToDateTime`

TimGetSeconds

Purpose Return the current date and time of the device in seconds since 12:00 A.M. on January 1, 1904.

Prototype

```
UInt32 TimGetSeconds (void)
```

Parameters None.

Result The number of seconds elapsed from 12:00 A.M. on January 1, 1904 to the current date and time on the device.

See Also `TimSetSeconds`

TimGetTicks

Purpose Return the tick count since the last reset. The tick count does not advance while the device is in sleep mode.

Prototype

```
UInt32 TimGetTicks (void)
```

Parameters None.

Result Returns the tick count.

Comments You can call the `SysTicksPerSecond` routine to determine the number of ticks per second.

See Also `SysTicksPerSecond`

TimSecondsToDateTime

Purpose Converts a date specified as the number of seconds since January 1, 1904 to a `DateTimeType` structure.

Prototype

```
void TimSecondsToDateTime (UInt32 seconds,
DateTimePtr dateTimeP)
```

Parameters

`-> seconds`	A date specified as the number of seconds elapsed from 12:00 A.M. on January 1, 1904 to the date
`<- dateTimeP`	A pointer to a `DateTimeType` structure that is updated with the date and time values.

Result Returns nothing. The structure referenced by `dateTimeP` is updated with the date and time computed for the number of seconds since 12:00 A.M. on January 1, 1904 .

See Also `TimDateTimeToSeconds`

TimSetSeconds

Purpose Set the clock of the device to the date and time passed as the number of seconds since 12:00 A.M. on January 1, 1904.

Prototype

```
void TimSetSeconds (UInt32 seconds)
```

Parameters

`-> seconds`	The number of seconds since 12:00 A.M. on January 1, 1904.

Result Returns nothing.

Comments On systems with the Notification Feature Set is present, this function broadcasts the `sysNotifyTimeChangeEvent` to all interested parties. See Chapter 40, "Notification Manager." for more information.

See Also `TimGetSeconds`

TimeToAscii

Purpose Convert the time to a string that is formatted according to the specified `timeFormat`.

Prototype

```
void TimeToAscii (UInt8 hours, UInt8 minutes,
TimeFormatType timeFormat, Char *pString)
```

Parameters

`-> hours`	The number of hours. This must be a value between `0` and `23`.
`-> minutes`	The number of minutes. This must be a value between `0` and `59`.
`-> timeFormat`	The time format for the resultant string. This must be one of the `TimeFormatType` values.
`<- pString`	A pointer to a string that is updated with the resultant string. This string must be of length `timeStringLength`. See Time Manager Constants for information on string buffer lengths.

Result Returns nothing. The string referenced by `pString` is updated with the formatted string.

Comments If you are using a debug ROM in Palm OS 3.5, the string buffer is filled with `timeStringLength` debugging bytes.

It is important to allocate enough space for your string buffer. Finding buffer overflow errors can be difficult when using a debug ROM. One common situation is when you pass a buffer that is too small from a form, for an element such as a label or title. Then, the buffer overflow can clobber objects that follow the form in memory. When a form element's location information is corrupted, it disappears from the display.

See Also `DateToAscii`



New TimeZoneToAscii

Purpose Convert a time zone to a string.

Prototype

```
void TimeZoneToAscii (Int16 timeZone,
const LmLocaleType *localeP, Char *string)
```

Parameters

-> `timeZone`	A pointer to the time zone, given as minutes east of Greenwich Mean Time (GMT).
-> `localeP`	A pointer to a locale (see `LmLocaleType`) that identifies the time zone country. You can use the constant `lmAnyLanguage` as the value for the language field of the structure pointed to by this parameter.
<- `string`	A pointer to a string in which to return the result. This string must be of length `timeZoneStringLength`.

Result Returns nothing.

Comments This function returns a descriptive string for the specified time zone. This string identifies the time zone first by its country, such as "USA (Mountain)" or "Canada (Eastern)." If the function cannot find a time zone that matches the specified GMT offset and country, it returns a string containing the time zone as a numeric offset from the GMT (for example, "GMT+9:00").

Compatibility Implemented only if 4.0 New Feature Set is present.



New TimTimeZoneToUTC

Purpose Converts a date and time from a given time zone to Universal Coordinated Time (UTC). UTC is also known as Greenwich Mean Time (GMT).

Prototype

```
UInt32 TimTimeZoneToUTC (UInt32 seconds,
Int16 timeZone, Int16 daylightSavingAdjustment)
```

Parameters

-> `seconds` The number of seconds since 12:00 A.M. on January 1, 1904.

-> `timeZone` The time zone, given as the number of minutes east of UTC. For time zones west of UTC but before the international dateline, this is a negative number.

-> `daylightSavingAdjustment` The number of minutes to add to the current time for daylight savings time in this time zone.

Result Returns the same time as `seconds` but in the Universal Coordinated Time. The value is still given as the number of seconds since 12:00 A.M. on January 1, 1904.

Comments The returned value is not necessarily the time in Greenwich because Greenwich may be observing daylight saving time.

You can use this function to convert the local time to UTC. The time zone and the daylight savings adjustment are system preferences that can be retrieved using `PrefGetPreference`. For example, the following code converts the current local time to UTC:

```
Int16 timeZone =
  PrefGetPreference(prefTimeZone);
Int16 daylightSavingAdjustment =
  PrefGetPreference(
  prefDaylightSavingAdjustment);
UInt32 utcTime =
  TimTimeZoneToUTC(TimGetSeconds(), timeZone,
  daylightSavingAdjustment);
```

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also TimUTCToTimeZone



TimUTCToTimeZone

Purpose Converts a date and time from Universal Coordinated Time (UTC) to the specified time zone. UTC is also known as Greenwich Mean Time (GMT).

Prototype

```
UInt32 TimUTCToTimeZone (UInt32 seconds,
Int16 timeZone, Int16 daylightSavingAdjustment)
```

Parameters

-> `seconds` The number of seconds since 12:00 A.M. on January 1, 1904 in UTC.

-> `timeZone` The time zone, given as the number of minutes east of UTC. For time zones west of UTC before the international dateline, this is a negative number.

-> `daylightSavingAdjustment` The number of minutes to add to the current time for daylight savings time in this time zone.

Result Returns the same time as `seconds` but in the specified time zone. The value is still given as the number of seconds since 12:00 A.M. on January 1, 1904.

Comments The `seconds` value is not necessarily the time in Greenwich because Greenwich may be observing daylight saving time.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also TimTimeZoneToUTC

54

Virtual File System Manager

The Virtual File System (VFS) Manager is a layer of software that manages all installed file system libraries. It provides a unified API to application developers while allowing them to seamlessly access many different types of file systems —such as VFAT, HFS, and NFS—on many different types of media, including Compact Flash, Memory Stick, and SmartMedia.This chapter provides reference material for the VFS Manager API as follows:

- VFS Manager Data Structures
- VFS Manager Constants
- VFS Manager Functions
- Application-Defined Functions

The header file `VFSMgr.h` declares the VFS Manager API. For more information on the VFS Manager, see Chapter 8, "Expansion," in *Palm OS Programmer's Companion*, vol. I.

Note that the VFS Manager is an optional system extension; the functions described in this chapter are implemented only if the VFS Manager Feature Set is present.

VFS Manager Data Structures

FileInfoType

The `FileInfoType` structure contains information about a specified file or directory. This information is returned as a

parameter to VFSDirEntryEnumerate. The structure is defined as follows:

```
typedef struct FileInfoTag {
  UInt32 attributes;
  Char *nameP;
  UInt16 nameBufLen;
} FileInfoType, *FileInfoPtr;
```

Field Descriptions

attributes	Characteristics of the file or directory. See File and Directory Attributes for the bits that make up this field.
nameP	Pointer to the buffer that receives the full name of the file or directory. Initialize this parameter to NULL if you don't want to receive the name.
nameBufLen	Size of the nameP buffer, in bytes.

FileRef

The FileRef type is used to encode references to files and directories.

```
typedef UInt32 FileRef;
```

VFSAnyMountParamType

The VFSAnyMountParamType structure is a base structure for VFSSlotMountParamType, VFSPOSEMountParamType, and other similar structures that may be defined in the future. Use one or the other according to how you set the mountClass parameter.

```
typedef struct VFSAnyMountParamTag {
  UInt16 volRefNum;
  UInt16 reserved;
  UInt32 mountClass;
} VFSAnyMountParamType;

typedef VFSAnyMountParamType
*VFSAnyMountParamPtr;
```

Field Descriptions

`volRefNum`	The volume reference number. This is initially obtained when you successfully mount a volume. It can then be used to format a volume with `VFSVolumeFormat` or unmount a volume with `VFSVolumeUnmount`.
`reserved`	Reserved for future use.
`mountClass`	Defines the type of mount to use with the specified volume. See Volume Mount Classes for a list of mount types.

VFSSlotMountParamType

The `VFSSlotMountParamType` structure is used when you are mounting a card located in an Expansion Manager slot. The `vfsMountParam->mountClass` field must be set to `VFSMountClass_SlotDriver`.

```
typedef struct VFSSlotMountParamTag {
  VFSAnyMountParamType vfsMountParam;
  UInt16 slotLibRefNum;
  UInt16 slotRefNum;
} VFSSlotMountParamType;
```

Field Descriptions

`vfsMountParam`	See the description of `VFSAnyMountParamType` for an explanation of the fields in this structure. Set `vfsMountParam->mountClass` to `VFSMountClass_SlotDriver` to mount an Expansion Manager slot.
`slotLibRefNum`	Reference number for the slot driver library allocated to the given slot number. Obtain this field by calling `ExpSlotLibFind`.
`slotRefNum`	Number of the slot to be mounted.

VFSPOSEMountParamType

The `VFSPOSEMountParamType` structure is used when you are mounting a volume through the Palm OS® Emulator. The `vfsMountParam->mountClass` must be set to `VFSMountClass_POSE`. Note that ordinary applications and file systems shouldn't use `VFSPOSEMountParamType`.

```
typedef struct VFSPOSEMountParamTag {
  VFSAnyMountParamType vfsMountParam;
  UInt8 poseSlotNum;
} VFSPOSEMountParamType
```

Field Descriptions

`vfsMountParam`	See the description of `VFSAnyMountParamType` for an explanation of the fields in this structure. Set `vfsMountParam->mountClass` to `VFSMountClass_POSE` to mount a virtual slot.
`poseSlotNum`	Number of the virtual slot number to be mounted by the Palm OS Emulator.

VolumeInfoType

The `VolumeInfoType` structure defines information that is returned to `VFSVolumeInfo` and used throughout the VFS functions.

```
typedef struct VolumeInfoTag {
  UInt32 attributes;
  UInt32 fsType;
  UInt32 fsCreator;
  UInt32 mountClass;
  UInt16 slotLibRefNum;
  UInt16 slotRefNum;
  UInt32 mediaType;
  UInt32 reserved;
} VolumeInfoType, *VolumeInfoPtr;
```

Field Descriptions

`attributes`	Characteristics of the volume. See Volume Attributes for the bits that make up this field.
`fsType`	File system type for this volume. See Defined File Systems for a list of the supported file systems.
`fsCreator`	Creator ID of this volume's file system driver. This information is used with `VFSCustomControl`.
`mountClass`	Mount class that mounted this volume. The supported mount classes are listed under Volume Mount Classes.
`slotLibRefNum`	Reference to the slot driver library with which the volume is mounted. This field is only valid when the mount class is `vfsMountClass_SlotDriver`.
`slotRefNum`	Slot number where the card containing the volume is loaded. This field is only valid when the mount class is `vfsMountClass_SlotDriver`.
`mediaType`	Type of card media. See Defined Media Types in the Expansion Manager chapter for the list of values. This field is only valid when the mount class is `vfsMountClass_SlotDriver`.
`reserved`	Reserved for future use.

VFS Manager Constants

Defined File Systems

The following file systems are currently defined by the VFS Manager. These values are used with `VFSVolumeInfo` in the `VolumeInfoType.fsType` parameter.

Constant	Value	Description
vfsFilesystemType_AFS	'afsu'	Unix Andrew file system
vfsFilesystemType_EXT2	'ext2'	Linux file system
vfsFilesystemType_FAT	'fats'	FAT12 and FAT16, which only handles 8.3 filenames
vfsFilesystemType_FFS	'ffsb'	Unix Berkeley block based file system
vfsFilesystemType_HFS	'hfss'	Macintosh standard hierarchical file system
vfsFilesystemType_HFSPlus	'hfse'	Macintosh extended hierarchical file system
vfsFilesystemType_HPFS	'hpfs'	OS2 High Performance file system
vfsFilesystemType_MFS	'mfso'	Macintosh original file system
vfsFilesystemType_NFS	'nfsu'	Unix Networked file system
vfsFilesystemType_Novell	'novl'	Novell file system
vfsFilesystemType_NTFS	'ntfs'	Windows NT file system
vfsFilesystemType_VFAT	'vfat'	FAT12 and FAT16 extended to handle long filenames

Open Mode Constants

This section describes constants that are used for the `openMode` parameter to the `VFSFileOpen` function. These constants specify the mode in which a file or directory is opened.

Constant	Value	Description
`vfsModeExclusive`	`0x0001U`	Open and lock the file or directory. This mode excludes anyone else from using the file or directory until it is closed.
`vfsModeRead`	`0x0002U`	Open for read access.
`vfsModeWrite`	`0x0004U | vfsModeExclusive`	Open for write access.
`vfsModeReadWrite`	`vfsModeWrite | vfsModeRead`	Open for read/write access.
`vfsModeCreate`	`0x0008U`	Create the file if it doesn't already exist. This open mode is implemented in the VFS layer, rather than in the file system library.
`vfsModeTruncate`	`0x0010U`	Truncate the file to zero (0) bytes after opening, removing all existing data. This open mode is implemented in the VFS layer, rather than in the file system library.
`vfsModeVFSLayerOnly`	`vfsModeCreate | vfsModeTruncate`	Mask used to isolate those flags that are only used by the VFS layer. These flags are not passed to the file system layer.
`vfsModeLeaveOpen`	`0x0020U`	Leave the file open even after the application exits.

File and Directory Attributes

The constants in the following table define bits that can be used individually or in combination when setting or interpreting the file attributes for a given file or directory. See `VFSFileGetAttributes`, `VFSFileSetAttributes`, and the FileInfoType data structure for specific use.

Constant	Value	Description
`vfsFileAttrReadOnly`	`0x00000001UL`	Read-only file or directory
`vfsFileAttrHidden`	`0x00000002UL`	Hidden file or directory
`vfsFileAttrSystem`	`0x00000004UL`	System file or directory
`vfsFileAttrVolumeLabel`	`0x00000008UL`	Volume label
`vfsFileAttrDirectory`	`0x00000010UL`	Directory
`vfsFileAttrArchive`	`0x00000020UL`	Archived file or directory
`vfsFileAttrLink`	`0x00000040UL`	Link to another file or directory

Volume Attributes

The constants in the following table define bits that can be used individually or in combination to make up the attributes field in the `VolumeInfoType` structure.

Constant	Value	Description
`vfsVolumeAttrHidden`	`0x00000004UL`	The volume should not be visible to the user.
`vfsVolumeAttrReadOnly`	`0x00000002UL`	The volume is read only.
`vfsVolumeAttrSlotBased`	`0x00000001UL`	Reserved. Check the mount class to determine how a volume is mounted.

Volume Mount Classes

The following constants define how a given volume is mounted. The `mountClass` field in the `VFSAnyMountParamType` and `VolumeInfoType` structures takes on one of these values.

Constant	Value	Description
`vfsMountClass_POSE`	`'pose'`	Mount the volume through the Palm OS Emulator. This is used for testing.
`vfsMountClass_Simulator`	`sysFileTSimulator`	Mount the volume through the simulator. This is used for testing.
`vfsMountClass_SlotDriver`	`sysFileTSlotDriver`	Mount the volume with a slot driver shared library.

Error Codes

The VFS Manager defines the following error codes:

Constant	Description
`vfsErrBadData`	The operation could not be completed because of invalid data.
`vfsErrBadName`	Invalid filename, path, or volume label.
`vfsErrBufferOverflow`	The supplied buffer is too small.
`vfsErrDirectoryNotFound`	Returned when the path leading up to the file does not exist.
`vfsErrDirNotEmpty`	The directory is not empty and therefore cannot be deleted.
`vfsErrFileAlreadyExists`	A file with this name exists already in this location.

Constant	Description
`vfsErrFileBadRef`	The file reference is invalid: it has been closed or was not obtained from `VFSFileOpen`.
`vfsErrFileEOF`	The file pointer is at the end of the file.
`vfsErrFileGeneric`	Generic file error.
`vfsErrFileNotFound`	The file was not found at the specified location.
`vfsErrFilePermissionDenied`	The requested permissions could not be granted.
`vfsErrFileStillOpen`	Returned from the underlying file system's delete function if the file is still open.
`vfsErrIsADirectory`	This operation can only be performed on a regular file, not a directory.
`vfsErrNameShortened`	A volume name or filename was automatically shortened to conform to the file system specification.
`vfsErrNoFileSystem`	None of the installed file systems support this operation.
`vfsErrNotADirectory`	This operation can only performed on a directory.
`vfsErrVolumeBadRef`	The volume reference number is invalid.
`vfsErrVolumeFull`	There is insufficient space left on the volume.
`vfsErrVolumeStillMounted`	Returned from the underlying file system's format function if the volume is still mounted.

VFS Manager Functions



New VFSCustomControl

Purpose Make a custom API call to a particular file system, given its creator ID. You can use `VFSVolumeInfo` to determine the creator ID of the file system for a given volume.

Prototype

```
Err VFSCustomControl(UInt32 fsCreator,
UInt32 apiCreator, UInt16 apiSelector,
void *valueP, UInt16 *valueLenP)
```

Parameters

`-> fsCreator`	Creator of the file system to call. A value of zero (0) tells the VFS Manager to check each registered file system, looking for one which supports the call.
`-> apiCreator`	Registered creator ID.
`-> apiSelector`	Custom operation to perform.
`<-> valueP`	A pointer to a buffer containing data specific to the operation. On exit, depending on the function of the particular custom call and on the value of `valueLenP`, the contents of this buffer may have been updated.
`<-> valueLenP`	On entry, points to the size of the `valueP` buffer. On exit, this value reflects the size of the data written to the `valueP` buffer. If `valueLenP` is `NULL`, `valueP` is passed to the file system but is not updated on exit.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.

`expErrUnsupportedOperation`
The specified opcode and/or creator is unsupported or undefined.

`sysErrParamErr` The `valueP` buffer is too small.

`vfsErrNoFileSystem`
VFS Manager cannot find an appropriate file system to handle the request.

Comments The driver identifies the call and its API by a registered creator ID and a selector. This allows file system developers to extend the API by defining selectors for their creator IDs. It also allows file system developers to support selectors (and custom calls) defined by other file system developers.

This function must return `expErrUnsupportedOperation` for all unsupported or undefined opcodes and/or creators.

Compatibility Implemented only if the VFS Manager Feature Set is present.



New VFSDirCreate

Purpose Create a new directory.

Prototype

```
Err VFSDirCreate(UInt16 volRefNum,
const Char *dirNameP)
```

Parameters

`-> volRefNum` Volume reference number returned from `VFSVolumeEnumerate`.

`-> dirNameP` Pointer to the full path of the directory to be created.

Result Returns one of the following error codes:

`errNone` No error

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrBadName`	Some or all of the path, up to but not including the last component specified in the `dirNameP` parameter, does not exist.
`vfsErrFileAlreadyExists`	A file with this name already exists in this location.
`vfsErrNoFileSystem`	The VFS Manager cannot find an appropriate file system to handle the request.
`vfsErrVolumeBadRef`	The volume has not been mounted.
`vfsErrVolumeFull`	There is not enough space left on the volume.

Comments

All parts of the path except the last component must already exist. The `vfsFileAttrDirectory` attribute is set with this function.

`VFSDirCreate` does not open the directory. Any operations you want to perform on this directory require a reference, which is obtained through a call to `VFSFileOpen`.

Compatibility

Implemented only if the VFS Manager Feature Set is present.



New VFSDirEntryEnumerate

Purpose

Enumerate the entries in a given directory. Entries can include files, links, and other directories.

Prototype

```
Err VFSDirEntryEnumerate (FileRef dirRef,
UInt32 *dirEntryIteratorP, FileInfoType *infoP)
```

Parameters

`-> dirRef`	Directory reference returned from `VFSFileOpen`.

`<-> dirEntryIteratorP`
: Pointer to the index of the last entry enumerated. For the first iteration, initialize this parameter to the constant `vfsIteratorStart`. Upon return, this references the next entry in the directory. If `infoP` is the last entry, this parameter is set to `vfsIteratorStop`.

`<-> infoP`
: Pointer to the `FileInfoType` data structure that contains information about the given directory entry. The `nameP` and `nameBufLen` fields in this structure must be initialized prior to calling `VFSDirEntryEnumerate`.

Result Returns one of the following error codes:

`errNone`
: No error.

`expErrEnumerationEmpty`
: There are no directory entries left to enumerate.

`expErrNotOpen`
: The file system library necessary for this call has not been installed or has not been opened.

`sysErrParamErr`
: The `dirEntryIteratorP` is not valid.

`vfsErrFileBadRef`
: The specified file reference is invalid.

`vfsErrIsNotADirectory`
: The specified file reference is valid, but does not point to a directory.

`vfsErrNoFileSystem`
: The VFS Manager cannot find an appropriate file system to handle the request.

Comments The directory to be enumerated must first be opened with `VFSFileOpen` in order to obtain a file reference. In order to obtain information on all entries in a directory you must make repeated calls to `VFSDirEntryEnumerate` inside a loop. Boundaries on the iteration are the defined constants `vfsIteratorStart` and `vfsIteratorStop`. Before the first call to `VFSDirEntryEnumerate`, `dirEntryIteratorP` should be

initialized to `vfsIteratorStart`. Each iteration then changes the value pointed to by `dirEntryIteratorP`. When information on the last entry in the directory is returned, `dirEntryIteratorP` is set to `vfsIteratorStop`.

WARNING! Creating, renaming, or deleting any file or directory invalidates the enumeration. After any such operation, the enumeration will need to be restarted.

Example The following code excerpt illustrates how to use `VFSDirEntryEnumerate`.

```
FileInfoType info;
FileRef dirRef;
UInt32 dirIterator;
Char *fileName = MemPtrNew(256);  // should check for err

// open the directory first, to get the directory reference
// volRefNum must have already been defined
err = VFSFileOpen(volRefNum, "/", vfsModeRead, &dirRef);
if(err == errNone) {

   info.nameP = fileName;    // point to local buffer
   info.nameBufLen = sizeof(fileName);
   dirIterator = vfsIteratorStart
   while (dirIterator != vfsIteratorStop) {
      // Get the next file
      err = VFSDirEntryEnumerate (dirRef, &dirIterator,
         &info);
      if (err == errNone) {
         // Do something with the directory entry information
         // Pull the attributes from info.attributes
         // The file name is in fileName
      } else {
         // handle error, possibly by breaking out of the
loop
      }
   } else {
      // handle directory open error here
   }
   MemPtrFree(fileName);
}
```

Compatibility Implemented only if the VFS Manager Feature Set is present.



New VFSExportDatabaseToFile

Purpose Save the specified database to a `.pdb` or `.prc` file on an external storage card.

Prototype

```
Err VFSExportDatabaseToFile (UInt16 volRefNum,
const Char *pathNameP, UInt16 cardNo,
LocalID dbID)
```

Parameters

`-> volRefNum`	Volume on which the destination file should be created.
`-> pathNameP`	Pointer to the complete path and name of the destination file to be created.
`-> cardNo`	Card number on which the `.pdb` or `.prc` being exported resides.
`-> dbID`	ID of the database being exported.

Result Returns one of the following error codes:

`errNone`	No error
`expErrNotEnoughPower`	There is insufficient battery power to perform the database export operation.
`vfsErrBadName`	The path name specified in `pathNameP` is not valid.

Comments This utility function exports a database from main memory to a `.pdb` or `.prc` file on an external storage card. This function is the opposite of `VFSImportDatabaseFromFile`. It first creates the file specified in the `pathNameP` parameter with `VFSFileCreate`. After opening the file the Exchange Manager function `ExgDBWrite` is called with an internal callback function for exporting the file from the Data Manager. The Exchange Manager makes repeated calls to this callback function, which receives the data back in blocks. Once all the data has been exported, VFS Manager closes the file.

This function is used, for example, to copy applications from main memory to a storage card.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also VFSExportDatabaseToFileCustom, VFSFileWrite, VFSImportDatabaseFromFile



New VFSExportDatabaseToFileCustom

Purpose Saves the specified database to a .pdb or .prc file on an external storage card. This function differs from VFSExportDatabaseToFile in that it allows you to track the progress of the export operation.

Prototype

```
Err VFSExportDatabaseToFileCustom
(UInt16 volRefNum, const Char *pathNameP,
UInt16 cardNo, LocalID dbID,
VFSExportProcPtr exportProcP, void *userDataP)
```

Parameters

-> volRefNum	Volume on which the destination file should be created.
-> pathNameP	Pointer to the complete path and name of the destination file to be created.
-> cardNo	Card number on which the .pdb or .prc being exported resides.
-> dbID	ID of the database being exported.
-> exportProcP	User-defined callback function that tracks the progress of the export. This function should allow the user to cancel the export. Pass NULL if you don't have a progress callback function. See VFSExportProcPtr for the requirements of this function.

`-> userDataP`	Pointer to any data you want to pass to the callback function specified in `exportProcP`. This information is not used internally by the VFS Manager. Pass `NULL` if you don't have a progress callback function or if that function doesn't need any such data.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotEnoughPower`	There is insufficient battery power to perform the database export operation.
`vfsErrBadName`	The path name specified in `pathNameP` is not valid.

This function can also return any error code other than `errNone` produced by your callback function.

Comments This function is similar to `VFSExportDatabaseToFile` in that it exports a database from main memory to a `.pdb` or `.prc` file on an external storage card. It extends the functionality by allowing you to specify a callback function that tracks the progress of the export. It first creates the file specified in the `pathNameP` parameter with `VFSFileCreate`. After opening the file, the Exchange Manager function `ExgDBWrite` is called with an internal callback function for exporting the file from the Data Manager. Exchange Manager makes repeated calls to this function, which receives the data back in blocks. The progress tracker, if one has been specified, is also called every time a new chunk of data is passed back. Once all the data has been exported, the VFS Manager closes the file.

This function is used, for example, to copy applications from main memory to a storage card.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSExportDatabaseToFile`, `VFSFileWrite`, `VFSImportDatabaseFromFileCustom`

New VFSFileClose

Purpose Closes a file or directory that has been opened with `VFSFileOpen`.

Prototype

```
Err VFSFileClose (FileRef fileRef)
```

Parameters

`-> fileRef`	File reference number returned from `VFSFileOpen`.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrFileBadRef`	The specified file reference is invalid.

Compatibility Implemented only if the VFS Manager Feature Set is present.

New VFSFileCreate

Purpose Create a file. This function cannot be used to create a directory; use `VFSDirCreate` instead.

Prototype

```
Err VFSFileCreate(UInt16 volRefNum,
const Char *pathNameP)
```

Parameters

`-> volRefNum`	Reference number of the volume on which to create the file. This volume reference number is returned from `VFSVolumeEnumerate`.
`-> pathNameP`	Pointer to the full path of the file to be created. All parts of the path, excluding the filename, must already exist.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrBadName`	The `pathNameP` is invalid.
`vfsErrFileAlreadyExists`	A file with this name already exists in this location.
`vfsErrNoFileSystem`	The VFS Manager cannot find an appropriate file system to handle the request.
`vfsErrVolumeBadRef`	The volume has not been mounted.
`vfsErrVolumeFull`	There is not enough space left on the volume.

Comments It is the responsibility of the file system library to ensure that all filenames are translated into a format that is compatible with the native format of the file system, such as the 8.3 convention for a FAT file system without long filename support. See Naming Files in the Expansion chapter of the *Palm OS Programmer's Companion*, vol. I for a description of how to construct a valid path.

This function does not open the file. Use `VFSFileOpen` to open the file.

This function should not be used to create a directory. To create a directory use `VFSDirCreate`.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileDelete`

New VFSFileDBGetRecord

Purpose

Load a record from an opened `.pdb` file on an external card into the storage heap.

Prototype

```
Err VFSFileDBGetRecord (FileRef ref,
UInt16 recIndex, MemHandle *recHP,
UInt8 *recAttrP, UInt32 *uniqueIDP)
```

Parameters

`-> ref`	The file reference returned from `VFSFileOpen`. Note that the open file must be a `.pdb` file.
`-> recIndex`	The index of the record to load.
`<- recHP`	Pointer to the record data's handle in the storage heap. If `NULL` is returned in this parameter there is either no data in this field or an error occurred reading this data from the file. If the handle is not `NULL`, you must dispose of the allocated handle using `MemHandleFree`.
`<- recAttrP`	Pointer to the attributes of the record. The values returned are identical to the atttributes returned from `DmRecordInfo`. See Record Attribute Constants in the Data and Resource Manager chapter for a description of each attribute. Pass `NULL` for this parameter if you do not want to retrieve this information.
`<- uniqueIDP`	Pointer to the unique identifier for this record. Pass `NULL` for this parameter if you do not want to retrieve this information.

Result

Returns one of the following error codes:

`errNone`	No error.
`dmErrIndexOutOfRange`	The `recIndex` is out of range.

`dmErrNotRecordDB`	The file referenced by `ref` is not a record database.
`memErrNotEnoughSpace`	There is not enough space in memory for the requested record entry.
`sysErrParamErr`	A `NULL` value was passed in for the `recHP`, `recAttrP`, and `uniqueIDP` parameters.
`vfsErrBadData`	The local offsets (`localChunkID`) from the top of the `.pdb` to the start of the raw record data for this entry are out of order.

Comments This function is analogous to `DmGetRecord` but works with files on an external card rather than databases in main memory. This function allocates a handle of the appropriate size from the storage heap and returns it in the `recHP` parameter. The caller is responsible for freeing this memory, using `MemHandleFree`, when it is no longer needed.

NOTE: This function is not efficient for multiple accesses and should be used sparingly.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileReadData`



New VFSFileDBGetResource

Purpose Load a resource into the storage heap from an opened `.prc` file.

Prototype

```
Err VFSFileDBGetResource (FileRef ref,
DmResType type, DmResID resID, MemHandle *resHP)
```

Parameters

`-> ref`	The file reference returned from `VFSFileOpen`. Note that the open file must be a `.prc` file.
`-> type`	The type of resource to load. See the Data and Resource Manager chapter for more information on resources.
`-> resID`	The ID of resource to load.
`<- resHP`	Pointer to the resource data handle that was loaded into memory.

Result

Returns one of the following error codes:

`errNone`	No error.
`dmErrNotResourceDB`	The file referenced by `ref` is not a resource database.
`dmErrResourceNotFound`	The requested resource was not found.
`memErrNotEnoughSpace`	There is not enough space in memory for the requested resource entries.
`sysErrParamErr`	`resHP` is `NULL`.

Comments

This function locates the specified resource in the open `.prc` file. See the *Palm OS File Format Specification* for more information on the layout of `.prc` files.

Once the resource is found, `VFSFileDBGetResource` allocates a handle of the appropriate size in the storage heap and reads it into memory. The handle to this memory location is returned through the `resHP` parameter. The caller is responsible for freeing this memory, using `MemHandleFree`, when it is no longer needed.

NOTE: This function is not efficient for multiple accesses and should be used sparingly.

Compatibility

Implemented only if the VFS Manager Feature Set is present.



New VFSFileDBInfo

Purpose

Get information about a database represented by an open `.prc` or `.pdb` file.

Prototype

```
Err VFSFileDBInfo (FileRef ref, Char *nameP,
UInt16 *attributesP, UInt16 *versionP,
UInt32 *crDateP, UInt32 *modDateP,
UInt32 *bckUpDateP, UInt32 *modNumP,
MemHandle *appInfoHP, MemHandle *sortInfoHP,
UInt32 *typeP, UInt32 *creatorP,
UInt16 *numRecordsP)
```

Parameters

`-> ref` The file reference returned from `VFSFileOpen`. Note that the open file must be a `.prc` or `.pdb` file.

`<- nameP` Pointer to a 32-byte character array in which the database name is returned. Pass `NULL` for this parameter if you do not want to retrieve the database name.

`<- attributesP` Pointer to the database attributes stored in the file. The values returned are identical to the atttributes returned from `DmDatabaseInfo`. See the Database Attribute Constants section for a description of each attribute. Pass `NULL` for this parameter if you do not want to retrieve the database's attributes.

`<- versionP` Pointer to the application-specific version number of the database. The default version number is zero (0). Pass `NULL` for this parameter if you do not want to retrieve the version number.

`<- crDateP` Pointer to the date the database was created, expressed in seconds since midnight (00:00:00) January 1, 1904. Pass `NULL` for this parameter if you do not want to retrieve the creation date.

`<- modDateP`	Pointer to the date the database was last modified, expressed in seconds since midnight (00:00:00) January 1, 1904. A database's modification date is updated only if a change has been made to the database when it is opened with write access. Pass `NULL` for this parameter if you do not want to retrieve the database's modification date.
`<- bckUpDateP`	Pointer to the date the database was last backed up, expressed in seconds since midnight (00:00:00) January 1, 1904. Pass `NULL` for this parameter if you do not want to retrieve the database's backup date.
`<- modNumP`	Pointer to the number of times the database was modified. This number is updated every time a record is added, modified, or deleted. Pass `NULL` for this parameter if you do not want to retrieve the modification count.
`<- appInfoHP`	Pointer to the application info block handle. If `NULL` is returned in this parameter, either there is no data in this field or an error occurred reading this data from the file. If a value other than `NULL` is returned, you must dispose of the allocated handle using `MemHandleFree`. If you do not want to retrieve the application info block, pass `NULL` for this parameter
`<- sortInfoHP`	Pointer to the sort info block handle. If `NULL` is returned in this parameter, either there is no data in this field or an error occurred reading this data from the file. If a value other than `NULL` is returned, you must dispose of the allocated handle using `MemHandleFree`. Pass `NULL` for this parameter if you do not want to retrieve the sort info block handle.

`<- typeP` Pointer to the type of database as it was created. This may be a user-defined database type or a database type defined by the Palm OS. Some of the more common database types returned here are:

Type	Description
`'appl'`	Standard Palm™ application (resource database)
`'libr'`	Standard shared library
`'libf'`	File system shared library
`'libs'`	Slot driver shared library
`'data'`	Standard Palm data file (record database)

Pass `NULL` for this parameter if you do not want to retrieve the database's type.

`<- creatorP` Pointer to the database's creator. Pass NULL for this parameter if you do not want to retrieve this information.

`<- numRecordsP` Pointer to the number of records in the database. Pass `NULL` for this parameter if you do not want to retrieve this information.

Result Returns one of the following error codes:

`errNone` No error

`memErrNotEnoughSpace`
There is not enough space in memory for the database header.

`vfsErrBadData` The file referenced by the `ref` parameter is too small to contain a database header, or the database header is corrupted.

Comments This function is analogous to DmDatabaseInfo, but it works with files on an external card rather than with databases in main memory. See the *Palm OS File Format Specification* for a description of the header block in `.prc` and `.pdb` files.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also VFSFileGetAttributes, VFSFileGetDate



VFSFileDelete

Purpose Deletes a closed file or directory.

Prototype

```
Err VFSFileDelete(UInt16 volRefNum,
const Char *pathNameP)
```

Parameters

`-> volRefNum`	Volume reference number returned from VFSVolumeEnumerate.
`-> pathNameP`	Pointer to the full path of the file or directory to be deleted.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrBadName`	The path name specified in `pathNameP` is not valid.
`vfsErrDirNotEmpty`	The directory being deleted is not empty.
`vfsErrFileStillOpen`	The file is still open.
`vfsErrFileNotFound`	The file could not be found.

`vfsErrFilePermissionDenied`
The requested permissions could not be granted.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

`vfsErrVolumeBadRef`
The volume has not been mounted.

Compatibility Implemented only if the VFS Manager Feature Set is present.



New VFSFileEOF

Purpose Get end-of-file status for an open file. This function only operates on files and cannot be used with directories.

Prototype

```
Err VFSFileEOF (FileRef fileRef)
```

Parameters

`-> fileRef` File reference returned from `VFSFileOpen`.

Result Returns one of the following error codes:

`errNone` No error. File pointer is not at end of the file.

`vfsErrFileEOF` The file pointer is at the end of file.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrFileBadRef`
The specified file reference is invalid.

`vfsErrIsADirectory`
The specified file reference points to a directory instead of a file. This is an invalid operation on a directory.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

Compatibility Implemented only if the VFS Manager Feature Set is present.



New VFSFileGetAttributes

Purpose Obtain the attributes of an open file or directory.

Prototype

```
Err VFSFileGetAttributes (FileRef fileRef,
UInt32 *attributesP)
```

Parameters

`-> fileRef`	File reference returned from `VFSFileOpen`.
`<- attributesP`	Pointer to the attributes associated with the file or directory. See File and Directory Attributes for a list of values that can be returned through this parameter.

Result Returns one of the following error codes:

`errNone`	No error
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrFileBadRef`	The specified file reference is invalid.
`vfsErrNoFileSystem`	The VFS Manager cannot find an appropriate file system to handle the request.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileDBInfo`, `VFSFileGetDate`, `VFSFileSetAttributes`



New VFSFileGetDate

Purpose Obtain the dates on an open file or directory.

Prototype

```
Err VFSFileGetDate (FileRef fileRef,
UInt16 whichDate, UInt32 *dateP)
```

Parameters

`-> fileRef`	File reference returned from `VFSFileOpen`.

`-> whichDate`	Specifies which date—creation, modification, or last access—you want. Supply one of the following values:
	`vfsFileDateCreated`
	`vfsFileDateModified`
	`vfsFileDateAccessed`
	Note that not all file systems are required to support the above dates. If the supplied date type is not supported by the file system, `VFSFileGetDate` returns `expErrUnsupportedOperation`.

`<- dateP`	Pointer to the requested date. This field is expressed in the standard Palm OS date format — the number of seconds since midnight (00:00:00) January 1, 1904.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`expErrUnsupportedOperation`	The specified date type is not supported by the underlying file system.

`vfsErrFileBadRef`	The specified file reference is invalid.
`sysErrParamErr`	The `whichDate` parameter is not one of the defined constants.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileDBInfo`, `VFSFileGetAttributes`, `VFSFileSetDate`



VFSFileOpen

Purpose Opens a file or directory and returns a reference for it.

Prototype

```
Err VFSFileOpen (UInt16 volRefNum,
const Char *pathNameP, UInt16 openMode,
FileRef *fileRefP)
```

Parameters

`-> volRefNum`	The volume reference number returned from `VFSVolumeEnumerate`.
`-> pathNameP`	Pointer to the full path of the file or directory to be opened. This must be a valid path. It cannot be empty and can not contain null characters. The format of the pathname should match what the underlying file system supports. See "Naming Files" in Chapter 8, "Expansion," of the *Palm OS Programmer's Companion,* vol. I for a description of how to construct a valid path.
`-> openMode`	Mode to use when opening the file. See the Open Mode Constants section for a list of accepted modes.
`<- fileRefP`	Pointer to the opened file or directory reference which is supplied to various other `VFSFile...` operations. This value is filled in on return.

Result Returns one of the following error codes:

`errNone` No error.

`expErrCardReadOnly`
The open mode requested includes write access but the file is read-only.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrBadName` The `pathNameP` parameter is invalid.

`vfsErrFileNotFound`
The specified file or directory could not be found.

`vfsErrFilePermissionDenied`
The file cannot be opened in the requested open mode, or it has already been opened with `vfsModeExclusive`.

`vfsErrVolumeBadRef`
The specified volume has not been mounted.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileClose`, `VFSDirEntryEnumerate`



New VFSFileRead

Purpose Read data from a file into the dynamic heap. This function only operates on files and cannot be used with directories; use `VFSDirEntryEnumerate` to explore the contents of a directory.

Prototype
```
Err VFSFileRead (FileRef fileRef, UInt32 numBytes,
void *bufP, UInt32 *numBytesReadP)
```

Parameters

`-> fileRef` File reference returned from `VFSFileOpen`.

`-> numBytes` Number of bytes to read.

`<- bufP` Pointer to the destination chunk where the data is to be stored. This can be a pointer to any writable memory.

`<- numBytesReadP`
Pointer to an unsigned integer that reflects the number of bytes actually read. This value is set on return and does not need to be initialized. If no bytes are read the value is set to zero. Pass `NULL` for this parameter if you do not need to know how many bytes were read.

Result Returns one of the following error codes:

`errNone` No error.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrFileBadRef`
The specified file reference is invalid.

`vfsErrFileEOF` The end of the file has been reached.

`vfsErrFilePermissionDenied`
Read permission is not enabled for this file.

`vfsErrIsADirectory`
The specified file reference is for a directory instead of a file. This is an invalid operation on a directory.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

Comments The file system does not use `DmWrite` and cannot be used to read data into the storage heap.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileReadData`, `VFSFileWrite`, `VFSImportDatabaseFromFile`



New VFSFileReadData

Purpose

Read data from a file into a chunk of memory in the storage heap. This function only operates on files and cannot be used with directories; use `VFSDirEntryEnumerate` to explore the contents of a directory.

Prototype

```
Err VFSFileReadData (FileRef fileRef,
UInt32 numBytes, void *bufBaseP, UInt32 offset,
UInt32 *numBytesReadP)
```

Parameters

`-> fileRef` File reference returned in `VFSFileOpen`.

`-> numBytes` Number of bytes to read.

`<- bufBaseP` Pointer to the destination chunk in the storage heap where the data is to be stored. This pointer must be obtained through the appropriate call to the Memory Manager API.

`-> offset` Offset, in bytes, within the `bufBaseP` chunk where the data is to be written.

`<- numBytesReadP` Pointer to an unsigned integer that reflects the number of bytes actually read. This value is set on return and does not need to be initialized. If no bytes are read, the value is set to zero. Pass `NULL` for this parameter if you do not need to know how many bytes were read.

Result

Returns one of the following error codes:

`errNone` No error.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrFileBadRef` The specified file reference is invalid.

`vfsErrFileEOF` The end of the file has been reached.

`vfsErrFilePermissionDenied`
Read permission is not enabled for this file.

`vfsErrIsADirectory`
The specified file reference is for a directory instead of a file. This is an invalid operation on a directory.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

Comments When data is read from an external card with `VFSFileReadData`, it is copied into a chunk of memory in the storage heap. This chunk **must** be allocated by the application before the call to `VFSFileReadData`. This function calls `DmWrite` to put the data in the storage heap.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileRead`, `VFSFileWrite`



VFSFileRename

Purpose Rename a closed file or directory. This function cannot be used to move a file to another directory within the file system.

Prototype

```
Err VFSFileRename (UInt16 volRefNum,
const Char *pathNameP, const Char *newNameP)
```

Parameters

`-> volRefNum` Volume reference number returned from `VFSVolumeEnumerate`.

`-> pathNameP` Pointer to the full path of the file or directory to be renamed.

`-> newNameP` Pointer to the new filename. Note that this is the name of the file only and does not include the path to the file.

Result Returns one of the following error codes:

`errNone` No error.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrBadName` The name provided in either `pathNameP` or `newNameP` is invalid. This is also returned if the string pointed to by `newNameP` is a path, rather than a filename.

`vfsErrFileAlreadyExists`
A file with the new name already exists in this location.

`vfsErrFileNotFound`
The source file could not be found.

`vfsErrFilePermissionDenied`
Write permission is not enabled for this file.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

`vfsErrVolumeBadRef`
The volume has not been mounted.

`vfsErrVolumeFull`
There is not enough space left on the volume.

Comments **WARNING!** This function invalidates directory enumeration. You cannot continue enumerating files after renaming one of them with this function. If you need to operate on additional files in the directory, you must first restart the enumeration.

Example Below is an example of how to use `VFSFileRename`. Note that the renamed file remains in the `/PALM/Programs` directory; `VFSFileRename` can't be used to move files from one directory to another.

```
// volRefNum must have been previously defined; most likely,
// it was returned by VFSVolumeEnumerate

err = VFSFileRename(volRefNum, "/PALM/Programs/foo.prc",
   "bar.prc");
if (err != errNone) {
      // handle error...
}
```

Compatibility Implemented only if the VFS Manager Feature Set is present.

New VFSFileResize

Purpose Change the size of an open file. This function only operates on files and cannot be used with directories.

Prototype

```
Err VFSFileResize (FileRef fileRef,
UInt32 newSize)
```

Parameters

`-> fileRef`	File reference returned from `VFSFileOpen`.
`-> newSize`	The desired new size of the file. This can be larger or smaller then the current file size.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrFileBadRef`	The specified file reference is invalid.
`vfsErrIsADirectory`	The specified file reference points to a directory instead of a file. This is an invalid operation on a directory.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

`vfsErrVolumeFull`
There is not enough space left on the volume.

Comments The location of the file pointer is undefined after a call to this function.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileSize`



New VFSFileSeek

Purpose Set the position within an open file from which to read or write. This function only operates on files and cannot be used with directories.

Prototype

```
Err VFSFileSeek (FileRef fileRef,
FileOrigin origin, Int32 offset)
```

Parameters

`-> fileRef` File reference returned from `VFSFileOpen`.

`-> origin` Origin to use when calculating the new position. The `offset` parameter indicates the desired new position relative to this origin, which can be one of the following:

`vfsOriginBeginning`
The beginning of the file.

`vfsOriginCurrent`
The current position within the file.

`vfsOriginEnd`
The end of the file. Only negative offsets are allowed when `origin` is set to `vfsOriginEnd`.

`-> offset`	Offset, either positive or negative, from the origin to which the current position should be set. A value of zero (0) positions you at the specified origin.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrFileBadRef`	The specified file reference is invalid.
`vfsErrFileEOF`	The file pointer is at the end of file.
`vfsErrIsADirectory`	The specified file reference points to a directory instead of a file. This is an invalid operation on a directory.
`sysErrParamErr`	The specified origin is not one of the defined constants.

Comments During a call to this function, if the resulting position would be beyond the end of the file, it sets the position to the end of the file.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileSize`, `VFSFileTell`



New VFSFileSetAttributes

Purpose Change the attributes of an open file or directory.

Prototype

```
Err VFSFileSetAttributes (FileRef fileRef,
UInt32 attributes)
```

Parameters

`-> fileRef`	File reference returned from `VFSFileOpen`.
`-> attributes`	Attributes to associate with the file or directory. See File and Directory Attributes for a list of values you can use when setting this parameter:

Result

Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`sysErrParamErr`	One of the parameters is invalid.
`vfsErrFileBadRef`	The specified file reference is invalid.
`vfsErrNoFileSystem`	The VFS Manager cannot find an appropriate file system to handle the request.

Comments

NOTE: You cannot use this function to set the `vfsFileAttrDirectory` or `vfsFileAttrVolumeLabel` attributes. The `vfsFileAttrDirectory` is set when you call `VFSDirCreate`. The `vfsFileAttrVolumeLabel` is set when you call `VFSVolumeSetLabel`. This function may fail when setting other attributes, depending on the underlying file system.

Compatibility

Implemented only if the VFS Manager Feature Set is present.

See Also

`VFSFileGetAttributes`, `VFSFileSetDate`



New VFSFileSetDate

Purpose

Changes the dates on an open file or directory.

Prototype

```
Err VFSFileSetDate (FileRef fileRef,
UInt16 whichDate, UInt32 date)
```

Parameters

`-> fileRef` File reference returned in `VFSFileOpen`.

`-> whichDate` Specifies which date—creation, modification, or last access—to modify. Supply one of the following values:

`vfsFileDateCreated`

`vfsFileDateModified`

`vfsFileDateAccessed`

Note that not all file systems are required to support the above dates. If the supplied date type is not supported by the file system, `VFSFileGetDate` returns `expErrUnsupportedOperation`.

`-> date` The new date. This field should be expressed in the standard Palm OS date format — the number of seconds since midnight (00:00:00) January 1, 1904.

Result

Returns one of the following error codes:

`errNone` No error.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`expErrUnsupportedOperation`
The specified date type is not supported by the underlying file system.

`sysErrParamErr` The `whichDate` parameter is not one of the defined constants.

`vfsErrFileBadRef`
The specified file reference is invalid.

`vfsErrFilePermissionDenied`
Write permission is not enabled for this file.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileGetDate`, `VFSFileSetAttributes`



VFSFileSize

Purpose Obtain the size of an open file. This function only operates on files and cannot be used with directories.

Prototype

```
Err VFSFileSize (FileRef fileRef,
UInt32 *fileSizeP)
```

Parameters

`-> fileRef` File reference returned from `VFSFileOpen`.

`<- fileSizeP` Pointer to the size of the open file.

Result Returns one of the following error codes:

`errNone` No error.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrFileBadRef`
The specified file reference is invalid.

`vfsErrIsADirectory`
The specified file reference points to a directory instead of a file. This is an invalid operation on a directory.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileResize`, `VFSFileTell`, `VFSVolumeSize`



VFSFileTell

Purpose Get the current position of the file pointer within an open file. This function only operates on files and cannot be used with directories.

Prototype

```
Err VFSFileTell (FileRef fileRef,
UInt32 *filePosP)
```

Parameters

`-> fileRef` File reference returned from `VFSFileOpen`.

`<- filePosP` Pointer to the current file position.

Result Returns one of the following error codes:

`errNone` No error.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrFileBadRef`
The specified file reference is invalid.

`vfsErrIsADirectory`
The specified file reference points to a directory instead of a file. This is an invalid operation on a directory.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSFileSeek`, `VFSFileSize`



New VFSFileWrite

Purpose Write data to an open file. This function only operates on files and cannot be used with directories.

Prototype

```
Err VFSFileWrite (FileRef fileRef,
UInt32 numBytes, const void *dataP,
UInt32 *numBytesWrittenP)
```

Parameters

`-> fileRef` File reference returned from `VFSFileOpen`.

`-> numBytes` The number of bytes to write.

`-> dataP` Pointer to the data that is to be written.

`<- numBytesWrittenP`
Pointer to an unsigned integer that reflects the number of bytes actually written. This value is set on return and does not need to be initialized. If no bytes are written the value is set to zero. Pass `NULL` for this parameter if you do not need to know how many bytes were written.

Result Returns one of the following error codes:

`errNone` No error.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrFileBadRef`
The specified file reference is invalid.

`vfsErrFilePermissionDenied`
Write permission is not enabled for this file.

`vfsErrIsADirectory`
The specified file reference points to a directory instead of a file. This is an invalid operation on a directory.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

`vfsErrVolumeFull`
There is not enough space left on the volume.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSExportDatabaseToFile`, `VFSExportDatabaseToFileCustom`, `VFSFileRead`, `VFSFileReadData`



New VFSGetDefaultDirectory

Purpose Determine the default location on the given volume for files of a particular type.

Prototype

```
Err VFSGetDefaultDirectory (UInt16 volRefNum,
const Char *fileTypeStr, Char *pathStr,
UInt16 *bufLenP)
```

Parameters

`-> volRefNum` Volume reference number returned from `VFSVolumeEnumerate`.

`-> fileTypeStr` Pointer to the requested file type, as a null-terminated string. The file type may either be a MIME media type/subtype pair, such as "image/jpeg", “text/plain”, or “audio/basic”; or a file extension, such as “ . `jpeg`.”

`<- pathStr` Pointer to the buffer which receives the default directory path for the requested file type.

`<-> bufLenP` Pointer to the size of the path. Set this to the size of `pathStr` buffer on input. Reflects the number of bytes copied to `pathStr` on output.

Result Returns one of the following error codes:

`errNone`	No error.
`vfsErrBadName`	There is no default directory registered for the requested file type.
`vfsErrBufferOverflow`	A match was found, but the `pathStr` buffer is too small to hold the resulting path string. A partial path is returned in `pathStr`.
`vfsErrFileNotFound`	No match was found for the specified volume. The error could have occurred with either the media type specified for this volume or the file type requested.

Comments This function returns the complete path to the default directory registered for the specified file type. A default directory can be registered for each type of media supported. The directory should be registered under media and file type. Note that this directory is typically a "root" directory for the file type; any subdirectories under this root directory should also be searched for files of the appropriate type.

This function can be used by an image viewer application, for example, to find the directory containing images without having to know what type of media the volume was on. This could be "/DCIM", "/images", or something else depending on the type of media.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSDirEntryEnumerate`, `VFSRegisterDefaultDirectory`, `VFSUnregisterDefaultDirectory`



New VFSImportDatabaseFromFile

Purpose Creates a database from a `.pdb` or `.prc` file on an external storage card.

Prototype

```
Err VFSImportDatabaseFromFile (UInt16 volRefNum,
const Char *pathNameP, UInt16 *cardNoP,
LocalID *dbIDP)
```

Parameters

`-> volRefNum`	Volume on which the source file resides.
`-> pathNameP`	Pointer to the full path and name of the source file.
`<- cardNoP`	Pointer to a variable that receives the card number of the newly-created database. If the database already resides in the storage heap, the card number of the existing database is returned along with the error `dmErrAlreadyExists`.
`<- dbIDP`	Pointer to a variable that receives the database ID of the new database. If the database already resides in the storage heap, the database ID of the existing database is returned along with the error `dmErrAlreadyExists`.

Result Returns one of the following error codes:

`errNone`	No error.
`dmErrAlreadyExists`	The `.prc` or `.pdb` file already exists in the storage heap. In this case the `cardNoP` and `dbIDP` are set to point to the existing file.
`expErrNotEnoughPower`	There is insufficient battery power to complete the requested operation.
`vfsErrBadName`	The path name specified in `pathNameP` is not valid.

Comments This utility function imports a `.pdb` or `.prc` file resident on an external storage card into a new database in the storage heap. It first calls `VFSFileOpen` to open the file specified in `pathNameP`. Assuming that a corresponding `.prc` or `.pdb` does not already exist in the storage heap, `VFSImportDatabaseFromFile` calls the Exchange Manager function `ExgDBRead` with an internal callback function for importing a file to the Data Manager. The Exchange Manager makes repeated calls to this function, which passes the data back in blocks. Once the file has been successfully imported, the owner (the imported file, if it's an executable, or the associated application if it is not) is sent a `sysAppLaunchCmdSyncNotify` launch code to make it aware of the new database.

This function doesn't provide any progress indication to the user. If you need to provide feedback to the user as the file import progresses, use `VFSImportDatabaseFromFileCustom` instead.

This function is used, for example, to copy applications from a storage card to main memory.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSExportDatabaseToFile`, `VFSFileRead`



New VFSImportDatabaseFromFileCustom

Purpose Create a database from the specified `.pdb` or `.prc` file on an external storage card. This function differs from `VFSImportDatabaseFromFile` in that it allows you to track the progress of the import operation.

Prototype

```
Err VFSImportDatabaseFromFileCustom
(UInt16 volRefNum, const Char *pathNameP,
UInt16 *cardNoP, LocalID *dbIDP,
VFSImportProcPtr importProcP, void *userDataP)
```

Parameters

`-> volRefNum` Volume on which the source file resides.

`-> pathNameP`	Pointer to the full path and name of the source file.
`<- cardNoP`	Pointer to the variable that receives the card number of the newly-created database. If the database already resides in the storage heap, the card number of the existing database is returned along with the error `dmErrAlreadyExists`.
`<- dbIDP`	Pointer to the variable that receives the database ID of the new database. If the database already resides in the storage heap, the database ID of the existing database is returned along with the error `dmErrAlreadyExists`.
`-> importProcP`	User-defined callback function that tracks the progress of the import. This function should allow the user to cancel the import. Pass `NULL` if you don't have a progress callback function. See `VFSImportProcPtr` for the requirements of this function.
`-> userDataP`	Pointer to any data you want to pass to the callback function specified in `importProcP`. This information is not used internally by the VFS Manager. Pass `NULL` if you don't have a progress callback function, or if that function doesn't need any such data.

Result Returns one of the following error codes:

`errNone`	No error
`vfsErrBadName`	The path name specified in `pathNameP` is not valid.
`expErrNotEnoughPower`	The power required to import a database is not available.
`dmErrAlreadyExists`	The `.prc` or `.pdb` file already exists in main memory. In this case the `cardNoP` and `dbIDP` are set to point to the existing file.

Comments This function is similar to VFSImportDatabaseFromFile in that it imports a .pdb or .prc file on an external storage card into a new database on the storage heap. It extends the functionality by allowing you to specify a callback function that tracks the progress of the export. It first calls VFSFileOpen to open the file specified in pathNameP. If a corresponding .prc or .pdb does not already exist in main memory, it calls the Exchange Manager function ExgDBRead with an internal callback function for importing the file from the Data Manager. The Exchange Manager makes repeated calls to this function, which receives the data back in blocks. The progress tracker, if one has been specified, is also called every time a new chunk of data is passed back. Once the file has been successfully imported, the owner (the imported file, if it's an executable, or the associated application if it is not) is sent a sysAppLaunchCmdSyncNotify launch code to make it aware of the new database.

This function is used, for example, to copy applications from a storage card to main memory.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also VFSFileRead, VFSExportDatabaseToFileCustom



New VFSInstallFSLib

Purpose Install a file system library so that the VFS Manager can use it.

Prototype

```
Err VFSInstallFSLib (UInt32 creator,
UInt16 *fsLibRefNumP)
```

Parameters

`-> creator` Creator ID of the database containing the file system library to be installed.

`<- fsLibRefNumP` Pointer to the reference number for the newly installed file system library. Supply NULL for this parameter if you don't need the library reference number.

Result If the file system library was loaded and installed without error, `errNone` is returned. Any error generated by the underlying file system while opening the file system library or determining its type will be returned from `VFSInstallFSLib`. Other errors that can be generated during the file system library installation process include:

`expErrIncompatibleAPIVer`
The file system library has an incompatible API version.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`memErrNotEnoughSpace`, `memErrChunkNotLocked`, or `memErrChunkLocked`
A memory problem occurred while inserting the library reference into the list of installed libraries.

`sysErrLibNotFound`, `sysErrNoFreeRAM`, `sysErrNoFreeLibSlots` (or some other error returned from the library's install entry point)
An error occurred while loading the library.

Comments This function calls `SysLibLoad` to load the file system library into the library table. Once loaded the appropriate file system is asked to open the library. At boot time `VFSInstallFSLib` is called internally by the Expansion Manager to load all installed file system libraries and initialize them for use.

`VFSInstallFSLib` is not normally called by applications.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSRemoveFSLib`



New VFSRegisterDefaultDirectory

Purpose

Registers a specific directory as the default location for files of a given type on a particular kind of external storage card. This function is generally called by a slot driver for files and media types that are supported by that slot driver.

Prototype

```
Err VFSRegisterDefaultDirectory
(const Char *fileTypeStr, UInt32 mediaType,
const Char *pathStr)
```

Parameters

`-> fileTypeStr` Pointer to the file type to register. This is a null-terminated string that can either be a MIME media type/subtype pair, such as "image/jpeg", "text/plain", or "audio/basic"; or a file extension, such as "`.jpeg`".

`-> mediaType` Type of card media for which the default directory is being associated. See Defined Media Types in the Expansion Manager chapter for the list of accepted values.

`-> pathStr` Pointer to the default directory path to be associated with the specified file type. This string must be null-terminated, and must be the full path to the directory.

Result

Returns one of the following error codes:

`errNone` No error.

`sysErrParamErr` Either the `fileTypeStr` parameter is `NULL` or the `pathStr` parameter is `NULL`.

`vfsErrFileAlreadyExists`
A default directory has already been registered for this file type on the specified card media type.

Comments This function first verifies that a default directory has not already been registered for the specified combination of file type and media type, and returns `vfsErrFileAlreadyExists` if one has been registered. To change an existing entry in the registry, you must first remove the existing entry with a call to `VFSUnregisterDefaultDirectory` before re-registering it with `VFSRegisterDefaultDirectory`.

The specified directory registered for a given file type is intended to be the "root" default directory. If a given default directory has one or more subdirectories, applications should also search those subdirectories for files of the appropriate type.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSGetDefaultDirectory`



New VFSRemoveFSLib

Purpose Remove a file system library from the library table, so that the VFS Manager can no longer use it.

Prototype

```
Err VFSRemoveFSLib (UInt16 fsLibRefNum)
```

Parameters

`-> fsLibRefNum` Library reference number of the file system library to remove from the library table.

Result Returns one of the following error codes:

`errNone` No error.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrNoFileSystem` VFS Manager can not find the file system specified in `fsLibRefNum`.

Comments This function is not normally called by applications. It unmounts any volumes that the specified file system may have mounted. It then closes the library and removes it from the library table with SysLibRemove.

Compatibility Implemented only if the VFS Manager Feature Set is present.



New VFSUnregisterDefaultDirectory

Purpose Sever the association between a particular file type and a default directory for a given type of card media.

Prototype

```
Err VFSUnregisterDefaultDirectory
(const Char *fileTypeStr, UInt32 mediaType)
```

Parameters

`-> fileTypeStr` Pointer to the file type with which the default directory is associated. This is a null-terminated string that can either be a MIME media type/subtype pair, such as "image/jpeg", "text/plain", or "audio/basic"; or a file extension, such as "`.jpeg`".

`-> mediaType` Type of card media for which the default directory is associated. See Defined Media Types in the Expansion Manager chapter for the list of accepted values.

Result Returns one of the following error codes:

`errNone` No error.

`sysErrParamErr` The `fileTypeStr` parameter is `NULL`.

`vfsErrFileNotFound` A default directory could not be found in the registry for the specified file and media type.

Comments

NOTE: Caution is advised when using this function, since you may remove another application's registration, causing data to mysteriously disappear from those applications.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also VFSGetDefaultDirectory, VFSRegisterDefaultDirectory



VFSVolumeEnumerate

Purpose Enumerate the mounted volumes.

Prototype

```
Err VFSVolumeEnumerate (UInt16 *volRefNumP,
UInt32 *volIteratorP)
```

Parameters

`<- volRefNumP` Pointer to the reference number for the volume represented by the current enumeration, or `vfsInvalidVolRef` if there are no more volumes to be enumerated or an error occurred.

`<-> volIteratorP` Pointer to a variable that holds the index of the current enumeration. Set the variable to `vfsIteratorStart` prior to the first iteration. Each call to `VFSVolumeEnumerate` updates the variable to the index of the next volume. When the last volume is reached, the variable pointed to by `volIteratorP` is set to `vfsIteratorStop`.

Result Returns one of the following error codes:

`errNone` No error

`expErrEnumerationEmpty` There are no volumes to enumerate.

`sysErrParamErr` The value pointed to by `volIteratorP` is not valid. This error is also returned when `volIteratorP` is `vfsIteratorStop`.

Comments This function returns a pointer to the volume reference number in the `volRefNumP` parameter. In order to traverse all volumes you must make repeated calls to `VFSVolumeEnumerate` inside a loop. Before the first call to `VFSVolumeEnumerate`, the variable pointed

to by `volIteratorP` should be initialized to `vfsIteratorStart`. Each iteration then increments `volIteratorP` to the next entry after updating `volRefNumP`. When the last volume is reached, `*volIteratorP` is set to `vfsIteratorStop`. If there are no volumes to enumerate, `VFSVolumeEnumerate` returns `expErrEnumerationEmpty` when first called.

Example Below is an example of how to use `VFSVolumeEnumerate`.

```
UInt16 volRefNum;
UInt32 volIterator = vfsIteratorStart;

while (volIterator != vfsIteratorStop) {
   err = VFSVolumeEnumerate(&volRefNum, &volIterator);
   if (err == errNone) {
      // Do something with the volRefNum
   } else {
      // handle error... possibly by
      // breaking out of the loop
   }
}
```

Compatibility Implemented only if the VFS Manager Feature Set is present.



New VFSVolumeFormat

Purpose Format and mount the volume installed in a given slot.

Prototype

```
Err VFSVolumeFormat (UInt8 flags,
UInt16 fsLibRefNum,
VFSAnyMountParamPtr vfsMountParamP)
```

Parameters

`-> flags` Flags that control how the volume should be formatted. Currently, the only flag not reserved is `vfsMountFlagsUseThisFileSystem`. Pass this flag to cause the volume to be formatted using the file system specified by `fsLibRefNum`. Pass zero (0) to have the VFS Manager attempt to format the volume using a file system appropriate to the slot.

`-> fsLibRefNum` Reference number of the file system library for which the volume should be formatted. This number is obtained through a call to `SysLibFind` with the name of the library you want to use. If the `flags` field is not set to `vfsMountFlagsUseThisFileSystem`, this parameter is ignored.

`<->vfsMountParamP` Parameters to be used when formatting the volume and when mounting the volume after it has been formatted. Supply a pointer to either a `VFSSlotMountParamType` or a `VFSPOSEMountParamType` structure. Note that you'll need to cast your structure pointer to a `VFSAnyMountParamPtr`. Set the `mountClass` field to the appropriate value: if you are mounting to an Expansion Manager slot, set `mountClass` to `VFSMountClass_SlotDriver` and initialize `slotLibRefNum` and `slotRefNum` to the appropriate values. See the descriptions of `VFSAnyMountParamType`, `VFSSlotMountParamType`, and `VFSPOSEMountParamType` for information on the fields that make up these data structures.

Result Returns one of the following error codes:

`errNone` No error.

`expErrNotEnoughPower`
There is insufficient battery power to format and/or mount a volume.

`expErrNotOpen` The file system library necessary for this call has not been installed or has not been opened.

`vfsErrNoFileSystem`
The VFS Manager cannot find an appropriate file system to handle the request.

Comments The slot driver currently only supports one volume per slot. If the volume is successfully formatted and mounted, the reference number of the mounted volume is returned in `vfsMountParamP->volRefNum`. If the format is unsuccessful or cancelled, `vfsMountParamP->volRefNum` is set to `vfsInvalidVolRef`.

If `vfsMountFlagsUseThisFileSystem` is passed as a flag, `VFSVolumeFormat` attempts to format the volume using the file system library specified by `fsLibRefNum`. Typically the flag parameter is not set. In this case `VFSVolumeFormat` tries to find a compatible library to format the volume, as follows:

1. Check to see if the default file system library feature is set. If it is, and if that file system is installed, it is used to format the volume. You can set the default file system using `FtrSet`; supply `sysFileCVFSMgr` for the feature creator, and `vfsFtrIDDefaultFS` for the feature number.
2. Check to see if any of the installed file systems are natively supported for the slot on which the VFS Manager is trying to format. If one of them is, it is used to format the volume.
3. If none of the installed file systems can perform the format using the slot's native type, a dialog displays warning the user that their media may become incompatible with other devices if they continue with the format. The user may continue or cancel the format. If the user chooses to continue, `VFSVolumeFormat` formats the volume using the first file system library that was installed.

When calling `VFSVolumeFormat`, the volume can either be mounted or unmounted. The underlying file system library call requires the volume to be unmounted. `VFSVolumeFormat` checks to see if the volume is currently mounted and unmounts it, if

necessary, using VFSVolumeUnmount before making the file system call. If the file system successfully formats the volume, VFSVolumeFormat mounts it and posts a sysNotifyVolumeMountedEvent notification.

Example The following code excerpt formats a volume on an Expansion Manager slot using a compatible file system.

```
VFSSlotMountParamType slotParam;
UInt32 slotIterator = expIteratorStart;

slotParam.vfsMountParamP.mountClass =
   VFSMountClass_SlotDriver;
err = ExpSlotEnumerate(&slotParam.slotRefNum,
   &slotIterator);
err = ExpSlotLibFind(slotParam.slotRefNum,
   &slotParam.slotLibRefNum);

err = VFSVolumeFormat(NULL, NULL,
   (VFSAnyMountParamPtr) & slotParam);
```

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also VFSVolumeMount



New VFSVolumeGetLabel

Purpose Determine the volume label for a particular volume.

Prototype

```
Err VFSVolumeGetLabel(UInt16 volRefNum,
Char *labelP, UInt16 bufLen)
```

Parameters

`-> volRefNum`	Volume reference number returned from VFSVolumeEnumerate.
`<- labelP`	Pointer to a character buffer into which the volume name is placed.
`-> bufLen`	Length, in bytes, of the `labelP` buffer.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrNoFileSystem`	The VFS Manager cannot find an appropriate file system to handle the request.
`vfsErrVolumeBadRef`	The specified volume has not been mounted.
`vfsErrBufferOverflow`	The value specified in `bufLen` is not big enough to receive the full volume label.
`vfsErrNameShortened`	There was an error reading the full volume name. A shortened version is being returned.

Comments Volume reference numbers can change each time you mount a given volume. To keep track of a particular volume, save the volume's label rather than its reference number. Volume labels can be up to 255 characters long. They can contain any normal character, including spaces and lower case characters, in any character set as well as the following special characters: $ % ' - _ @ ~ ` ! ( ) ^ # & + , ; = [ ].

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSVolumeSetLabel`



New VFSVolumeInfo

Purpose Get information about the specified volume.

Prototype

```
Err VFSVolumeInfo(UInt16 volRefNum, VolumeInfoType
*volInfoP)
```

Parameters

`-> volRefNum`	Volume reference number returned from `VFSVolumeEnumerate`.
`<- volInfoP`	Pointer to the structure that receives the volume information for the specified volume. See `VolumeInfoType` for more information on the fields in this data structure.

Result

Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrNoFileSystem`	The VFS Manager cannot find an appropriate file system to handle the request.
`vfsErrVolumeBadRef`	The specified volume reference number is invalid.

Compatibility

Implemented only if the VFS Manager Feature Set is present.

See Also

`VFSVolumeGetLabel`, `VFSVolumeSize`

New VFSVolumeMount

Purpose

Mount the card's volume on the specified slot.

Prototype

```
Err VFSVolumeMount(UInt8 flags,
UInt16 fsLibRefNum,
VFSAnyMountParamPtr vfsMountParamP)
```

Parameters

`-> flags` Flags that control how the volume should be mounted. Currently, the only flag not reserved is `vfsMountFlagsUseThisFileSystem`. Pass this flag to cause the volume to be mounted using the file system specified by `fsLibRefNum`. Pass zero (0) to have the VFS Manager attempt to mount the volume using a file system appropriate for the slot.

`-> fsLibRefNum` Reference number of the file system library for which the volume should be mounted. This number is obtained through a call to `SysLibFind` with the name of the library you want to use. If the `flags` field is not set to `vfsMountFlagsUseThisFileSystem`, this parameter is ignored.

`<->vfsMountParamP`
Parameters to be used when mounting the volume after it has been formatted. Supply a pointer to either a `VFSSlotMountParamType` or a `VFSPOSEMountParamType` structure. Note that you'll need to cast your structure pointer to a `VFSAnyMountParamPtr`. Set the `mountClass` field to the appropriate value: if you are mounting to an Expansion Manager slot, set `mountClass` to `VFSMountClass_SlotDriver` and initialize `slotLibRefNum` and `slotRefNum` to the appropriate values. See the descriptions of `VFSAnyMountParamType`, `VFSSlotMountParamType`, and `VFSPOSEMountParamType` for information on the fields that make up these data structures.

Result Returns one of the following error codes:

`errNone` No error.

`expErrNotEnoughPower`
There is insufficient battery power to mount a volume.

`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`sysErrParamErr`	`vfsMountParamP` was initialized to `NULL`.
`vfsErrNoFileSystem`	The VFS Manager cannot find an appropriate file system to handle the request.
`vfsErrVolumeStillMounted`	The volume is already mounted with a different file system than was specified in `fsLibRefNum`.

Comments

The slot driver only supports one volume per slot. The reference number of the mounted volume is returned in `vfsMountParamP->volRefNum`. If `vfsMountFlagsUseThisFileSystem` is passed as a flag, `VFSVolumeMount` attempts to mount the volume using the file system library specified by `fsLibRefNum`. Otherwise `VFSVolumeMount` tries to find a file system library which is able to mount the volume. If none of the installed file system libraries is able to mount the volume, `VFSVolumeMount` attempts to re-format the volume (using `VFSVolumeFormat`) and then mount it. If `VFSVolumeMount` manages to successfully mount the volume, it ends by posting a `sysNotifyVolumeMountedEvent` notification.

After VFSVolumeMount successfully mounts a volume, it broadcasts `sysNotifyVolumeMountedEvent`. The VFS Manager, upon being notified of this event, searches the newly-mounted volume for `/PALM/start.prc`. If `start.prc` is found in the `/PALM` directory, the VFS Manager copies it to main memory and launches it. If start.prc is not found, the VFS Manager switches to the Launcher instead. This behavior can be overridden; see Card Insertion and Removal in Chapter 8, "Expansion," *Palm OS Programmer's Companion*, vol. I.

When `VFSVolumeMount` is called, if the volume is already mounted with a different file system than was specified in `fsLibRefNum`, a `vfsErrVolumeStillMounted` error is returned. If the volume is already mounted with the same file system that is specified in `fsLibRefNum`, or if `vfsMountFlagsUseThisFileSystem` is not set, `VFSVolumeMount` returns `errNone` and sets `volRefNumP` to the reference number of the currently mounted volume.

Example The following code excerpt mounts a volume on an Expansion Manager slot using a compatible file system.

```
VFSSlotMountParamType slotParam ;
UInt32 slotIterator = expIteratorStart;

slotParam.vfsMountParamP.mountClass =
   VFSMountClass_SlotDriver;
err = ExpSlotEnumerate(&slotParam.slotRefNum,
   &slotIterator);
err = ExpSlotLibFind(slotParam.slotRefNum,
   &slotParam.slotLibRefNum);

err = VFSVolumeMount(NULL, NULL,
   (VFSAnyMountParamPtr) & slotParam);
```

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also VFSVolumeFormat, VFSVolumeUnmount



VFSVolumeSetLabel

Purpose Changes the volume label for a mounted volume.

Prototype

```
Err VFSVolumeSetLabel(UInt16 volRefNum,
const Char *labelP)
```

Parameters

`-> volRefNum`	Volume reference number returned from VFSVolumeEnumerate.
`-> labelP`	Pointer to the label to be applied to the specified volume. This string must be null-terminated.

Result Returns one of the following error codes:

`errNone`	No error
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrBadName`	The supplied label is invalid.

`vfsErrNameShortened`
: Indicates that the label name was too long. A shortened version of the label name was used instead.

`vfsErrVolumeBadRef`
: The specified volume has not been mounted.

Comments Volume labels can be up to 255 characters long. They can contain any normal character, including spaces and lower case characters, in any character set as well as the following special characters: $ % ' - _ @ ~ ` ! ( ) ^ # & + , ; = [ ]. See Naming Volumes in Chapter 8, "Expansion," *Palm OS Programmer's Companion*, vol. I for guidelines on naming.

NOTE: Most clients should not need to call this function. This function may create or delete a file in the root directory, which would invalidate any current calls to `VFSDirEntryEnumerate`.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSVolumeGetLabel`



New VFSVolumeSize

Purpose Determine the total amount of space on a volume, as well as the amount that is currently being used.

Prototype

```
Err VFSVolumeSize(UInt16 volRefNum,
UInt32 *volumeUsedP, UInt32 *volumeTotalP)
```

Parameters

`-> volRefNum` Volume reference number returned from `VFSVolumeEnumerate`.

`<- volumeUsedP` Pointer to a variable that receives the amount of space, in bytes, in use on the volume.

`<- volumeTotalP`
: Pointer to a variable that receives the total amount of space on the volume, in bytes.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrNoFileSystem`	The VFS Manager cannot find an appropriate file system to handle the request.
`vfsErrVolumeBadRef`	The specified volume has not been mounted.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSVolumeInfo`



New VFSVolumeUnmount

Purpose Unmount the given volume.

Prototype

```
Err VFSVolumeUnmount(UInt16 volRefNum)
```

Parameters

`-> volRefNum`	Volume reference number returned from `VFSVolumeEnumerate`.

Result Returns one of the following error codes:

`errNone`	No error.
`expErrNotOpen`	The file system library necessary for this call has not been installed or has not been opened.
`vfsErrNoFileSystem`	The VFS Manager cannot find an appropriate file system to handle the request.
`vfsErrVolumeBadRef`	The specified volume has not been mounted.

Comments This function closes any opened files and posts a `sysNotifyVolumeUnmountedEvent` notification once the file system is successfully unmounted.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also `VFSVolumeMount`

Application-Defined Functions



New VFSExportProcPtr

Purpose User-defined callback function supplied to `VFSExportDatabaseToFileCustom` that tracks the progress of the export.

Prototype

```
typedef Err (*VFSExportProcPtr)
(UInt32 totalBytes, UInt32 offset,
void *userDataP)
```

Parameters

`totalBytes`	The total number of bytes being exported.
`offset`	Undefined.
`userDataP`	Pointer to any application-specific data passed to the callback function. This pointer may be `NULL` if your callback doesn't need any such data.

Result Your progress tracker should allow the user to abort the export. Return `errNone` if the export should continue, or any other value to abort the export process. If you return a value other than `errNone`, that value will be returned by `VFSExportDatabaseToFileCustom`.

Comments See the Progress Dialogs section in the *Palm OS Programmer's Companion*, vol. I for more information on writing a progress tracker.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also VFSImportProcPtr



VFSImportProcPtr

Purpose User-defined callback function supplied to VFSImportDatabaseFromFileCustom that tracks the progress of the import.

Prototype

```
typedef Err (*VFSImportProcPtr)
(UInt32 totalBytes, UInt32 offset,
void *userDataP)
```

Parameters

`totalBytes`	The total number of bytes being imported.
`offset`	The number of bytes that have already been imported. This value, along with the total number of bytes being imported, allows you to inform the user how far along the import is.
`userDataP`	Pointer to NY application-specific data passed to the callback function. This pointer may be `NULL` if your callback doesn't need any such data.

Result Your progress tracker should allow the user to abort the import. Return `errNone` if the import should continue, or any other value to abort the import process. If you return a value other than `errNone`, that value will be returned by VFSImportDatabaseFromFileCustom.

Comments See the Progress Dialogs section in the *Palm OS Programmer's Companion*, vol. I for more information on writing a progress tracker.

Compatibility Implemented only if the VFS Manager Feature Set is present.

See Also VFSExportProcPtr

55

Windows

This chapter provides information about windows by discussing these topics:

- Window Data Structures
- Window Functions

No resources are associated with window objects.

The header file `Window.h` declares the API that this chapter describes. For more information on windows, see the section "Forms, Windows, and Dialogs" in the *Palm OS Programmer's Companion*, vol. I.

Window Data Structures

CustomPatternType

The `CustomPatternType` type holds an 8-by-8 bit pattern that is one bit deep. Each byte specifies a row of the pattern. When drawing, a pattern is tiled to fill a specified region. This pattern is used by `WinFillLine` and `WinFillRectangle`.

The `PatternType` specifies the name of the current pattern.

```
typedef UInt8 CustomPatternType [8];
```

Compatibility In pre-3.5 systems, the `CustomPatternType` is an array of 4 16-bit words. Note the size of this data type has not changed.

DrawStateType

The `DrawStateType` structure defines the current drawing state, which is the Palm OS® implementation of a **pen**. This drawing state is saved with `WinPushDrawState` and restored with `WinPopDrawState`.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `DrawStateType` structure. Access it only through the functions described below. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct DrawStateType {
  WinDrawOperation   transferMode;
  PatternType        pattern;
  UnderlineModeType  underlineMode;
  FontID             fontId;
  FontPtr            font;
  CustomPatternType  patternData;
  IndexedColorType   foreColor;
  IndexedColorType   backColor;
  IndexedColorType   textColor;
  RGBColorType       foreColorRGB;
  RGBColorType       backColorRGB;
  RGBColorType       textColorRGB;
} DrawStateType;
```

Field Description

`transferMode`	The current transfer mode for color drawing. See `WinDrawOperation`. Use `WinSetDrawMode` to set this value.
`pattern`	The name of the current pattern. See `PatternType`. If set to `customPattern`, the `patternData` field contains the actual pattern. Use `WinGetPatternType` and `WinSetPatternType` to retrieve and set this value.
`underlineMode`	The current underline mode. See `UnderlineModeType`. Use `WinSetUnderlineMode` to set this value.
`fontId`	The ID of the current font. Use `FntSetFont` to set this value.

`font`	A pointer to the current font. Use `FntSetFont` to set this value.
`patternData`	The current pattern being used by the `WinFill` functions if `pattern` is `customPattern`. See `CustomPatternType`. Use `WinGetPattern` and `WinSetPattern` to retrieve and set this value.
`foreColor`	Index of the current color used for the foreground. Use `WinSetForeColor` to set this value.
`backColor`	Index of the current color used for the background. Use `WinSetBackColor` to set this value.
`textColor`	Index of the current color used for text. Use `WinSetTextColor` to set this value.
`foreColorRGB`	RGB value of the current color used for the foreground. Use `WinSetForeColorRGB` to set this value. Only valid for Palm OS 4.0.
`backColorRGB`	RGB value of the current color used for the background. Use `WinSetBackColorRGB` to set this value. Only valid for Palm OS 4.0.
`textColorRGB`	RGB value of the current color used for text. Use `WinSetTextColorRGB` to set this value. Only valid for Palm OS 4.0.

Compatibility This type is implemented only if 3.5 New Feature Set is present.

FrameBitsType

The `FrameBitsType` structure specifies attributes of a window's frame.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `FrameBitsType` bit field. Never access its bit field members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef union FrameBitsType {
  struct {
    UInt16 cornerDiam  : 8;
    UInt16 reserved_3  : 3;
    UInt16 threeD      : 1;
    UInt16 shadowWidth : 2;
    UInt16 width       : 2;
  } bits;
  UInt16 word;
} FrameBitsType;
```

Field Descriptions

`cornerDiam`	Corner radius of frame; maximum is 38.
`reserved_3`	Reserved.
`threeD`	Set this bit to draw a 3D button. This feature is not currently supported.
`shadowWidth`	Width of shadow.
`width`	Frame width.
`word`	Reserved.

FrameType

The `FrameType` type specifies a window frame style.

```
typedef UInt16 FrameType;
```

The `FrameType` can be set to one of the defined frame types shown in the table below, or a custom frame type as defined by a `FrameBitsType` structure.

Constant	Value	Description
noFrame	0	No frame
simpleFrame	1	Plain rectangular frame
rectangleFrame	1	Plain rectangular frame
simple3DFrame	0x0012	3D frame with width of 2. This frame type is not supported.
roundFrame	0x0401	Round frame with width of 1.
boldRoundFrame	0x0702	Round frame with width of 2.
popupFrame	0x0205	Popup frame style with slight corner roundness, width of 1 and shadow of 1.
dialogFrame	0x0302	Dialog frame style with slight corner roundness and width of 2.
menuFrame	popupFrame	Same as popupFrame.

IndexedColorType

The IndexedColorType type is used to specify a color by its index value; that is, by its location in a color table. Color tables are defined by the ColorTableType structure, which is declared in Bitmap.h. The IndexedColorType can hold a 1, 2, 4, or 8-bit index.

```
typedef UInt8 IndexedColorType;
```

Compatibility This type is implemented only if 3.5 New Feature Set is present.

PatternType

The PatternType enumerated type specifies a pattern for drawing. This type is returned by WinGetPatternType and is used as a parameter to the WinSetPatternType function.

```
typedef enum { blackPattern, whitePattern,
grayPattern, customPattern } PatternType;
```

Value Descriptions

`blackPattern`	Pattern with all bits on.
`whitePattern`	Pattern with all bits off.
`grayPattern`	Pattern with alternating on and off bits.
`customPattern`	Custom pattern specified by `CustomPatternType`.

These patterns all operate with current foreground and background color instead of black and white. In effect, `blackPattern` is only black if the current foreground color is black. `whitePattern` uses the current background color. `grayPattern` and `customPattern` uses a combination of background and foreground colors.

UnderlineModeType

The `UnderlineModeType` enumerated type specifies possible values for the underline mode stored in `DrawStateType`.

```
typedef enum { noUnderline, grayUnderline,
solidUnderline, colorUnderline }
UnderlineModeType;
```

Value Descriptions

`noUnderline`	No underline.
`grayUnderline`	Underline is drawn using a dotted line in the current foreground color.
`solidUnderline`	Underline is drawn using a solid line in the foreground color.
`colorUnderline`	Underline is drawn using a solid line in the foreground color.

Compatibility The `solidUnderline` and `colorUnderline` options are only available in Palm OS 3.1 and higher.

WindowFlagsType

The `WindowFlagsType` specifies different window attributes.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `WindowFlagsType` bit field. Access it only through the functions described below. Never access its bit field members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct WindowFlagsType {
  UInt16 format:1;
  UInt16 offscreen:1;
  UInt16 modal:1;
  UInt16 focusable:1;
  UInt16 enabled:1;
  UInt16 visible:1;
  UInt16 dialog:1;
  UInt16 freeBitmap:1;
  UInt16 reserved :8;
} WindowFlagsType;
```

Field Descriptions

`format` If set, use the `genericFormat`. If 0, use `screenFormat`.
Screen format is the native format of the video system; windows in this format can be copied to the display faster. The generic format is device-independent. A window cannot be enabled (that is, accept pen input) unless it uses screen format.

`offscreen` If set, the window is offscreen. If 0, the window is onscreen.

`modal` If set, the window is modal. If 0, the window is not modal. You set this value when you create the window. This value is returned by `WinModal`.

`focusable`	If set, the window can accept the focus. If 0, the window does not accept the focus. You set this value when you create the window.
`enabled`	If set, the window is enabled. If 0, the window is disabled.
`visible`	If set, the window is visible if it is onscreen. If 0, the window is not visible.
`dialog`	If set, the window is a form. If 0, the window is not a form. The `FrmInitForm` function sets this value.
`freeBitmap`	If set, free the bitmap when the window is freed. If 0, retain the bitmap after the window is freed.
`reserved`	Reserved for future use. Must be 0.

Compatibility

In OS versions previous to 3.5, the `freeBitmap` flag was not present. Instead, a `compressed` flag was present, where 0 specified uncompressed and 1 specified compressed. This compressed flag is now part of the `BitmapType`.

WindowType

The `WindowType` structure represents a window.

WARNING! Palm, Inc. does not support or provide backward compatibility for the `WindowType` structure. Access it only through the functions described below. Never access its structure members directly, or your code may break in future versions. Use the information below for debugging purposes only.

```
typedef struct WindowType {
  Coord              displayWidthV20;
  Coord              displayHeightV20;
  void               *displayAddrV20;
  WindowFlagsType    windowFlags;
  RectangleType      windowBounds;
  AbsRectType        clippingBounds;
  BitmapPtr          bitmapP;
```

```
    FrameBitsType       frameType;
    DrawStateType       *drawStateP;
    struct WindowType      *nextWindow;
  } WindowType;
```

Field Descriptions

`displayWidthV20` Width of the window in pre OS 3.5 devices. In OS 3.5, use `WinGetBounds` to return the window width.

`displayHeightV20` Height of the window in pre OS 3.5 devices. In OS 3.5, use `WinGetBounds` to return the window height.

`displayAddrV20` Pointer to the window display memory buffer in pre OS 3.5 devices. In OS 3.5 or later, call `WinGetBitmap` and then `BmpGetBits` to obtain the window's memory buffer.

WARNING! Writing directly to screen memory will not be supported in all devices.

`windowFlags` Window attributes (see `WindowFlagsType`).

`windowBounds` Display-relative bounds of the window. Use `WinGetBounds` and `WinSetBounds` to retrieve and set this value.

`clippingBounds` Bounds for clipping any drawing within the window. Use `WinGetClip` and `WinSetClip` to retrieve and set this value.

`bitmapP` Pointer to the window bitmap, which holds the window's contents. Use `WinGetBitmap` to retrieve this value.

`frameType` Frame attributes; see `FrameBitsType`.

`drawStateP`	Pointer to a state of the current transfer mode, pattern mode, font, underline mode, and colors. See `DrawStateType`. Only one drawing state exists in the system. Each window points to the same structure.
`nextWindow`	Pointer to the next window in a linked list of windows. This linked list of windows is called the active window list. Do not change this field.

Compatibility In OS versions previous to 3.5, this structure is slightly different. Specifically, the `bitmapP` field is instead a `viewOrigin` field of type `PointType` and specified the window origin point on the display. The `drawStateP` was named `gstate` and was of type `GraphicStatePtr`. The complete definition is shown below:

```
typedef struct WinTypeStruct {
  Word                  displayWidth;
  Word                  displayHeight;
  VoidPtr               displayAddr;
  WindowFlagsType       windowFlags;
  RectangleType         windowBounds;
  AbsRectType           clippingBounds;
  PointType             viewOrigin;
  FrameBitsType         frameType;
  GraphicStatePtr       gstate;
  struct WinTypeStruct    *nextWindow;
} WindowType;
```

WinDrawOperation

The `WinDrawOperation` enumerated type specifies the transfer mode for color drawing. This type is used as a parameter to the `WinCopyRectangle` and `WinSetDrawMode` functions.

```
typedef enum {winPaint, winErase, winMask,
winInvert, winOverlay, winPaintInverse,
winSwap} WinDrawOperation;
```

Value Descriptions

`winPaint`	Destination replaced with source pixels (copy mode).
`winErase`	Destination cleared where source pixels are off (AND mode).
`winMask`	Destination cleared where source pixels are on (AND NOT mode).
`winInvert`	Destination inverted where source pixels are on (XOR mode).
`winOverlay`	Destination set only where source pixels are on (OR mode).
`winPaintInverse`	Destination replaced with inverted source (copy NOT mode).
`winSwap`	Destination foreground and background colors are swapped, leaving any other colors unchanged (color invert operation).

For `winErase`, `winMask`, `winInvert`, and `winOverlay`, *on* means a pixel with an non-zero index (or for direct bitmaps, a non-zero RGB value), while *off* means a pixel with an index of 0 (or for direct bitmaps, an RGB value of 0).

When drawing graphics data, the OS first converts the source data to the depth of the destination. For 1-bit bitmaps, rectangles, lines, text, and pixels, this operation uses the foreground color for *on* pixels and the background color for *off* pixels. For bitmaps with a depth greater than 1, the source bitmap's pixel values are used rather than the foreground and background colors.

The OS then applies the logical operation described in this section. For `winMask` (AND NOT) and `winPaintInverse` (COPY NOT), the depth-converted source data is inverted before it is AND'ed with or COPY'ed to the destination data.

`winSwap` is not a color invert operation, although a pair of `winSwap` operations will restore the original graphics data. This mode is used by the OS to select and deselect areas of the screen. It changes destination pixels matching the foreground color to the

background color, and changes destination pixels matching the background color to the foreground color. It is a mode available for rectangles, lines, and pixels, but not text or bitmaps. This mode ignores the current pattern.

Compatibility This type is implemented only if 3.5 New Feature Set is present. In earlier releases, this type is named `ScrOperation` and its values begin with the prefix `scr` rather than `win`. `WinDrawOperation` is fully compatible with `ScrOperation`.

WinHandle

The `WinHandle` type is a pointer to a `WindowType` structure. Note that this may change.

```
typedef WindowType *WinHandle;
```

WinLineType

The `WinLineType` structure defines a line.

```
typedef struct WinLineType {
  Coord x1;
  Coord y1;
  Coord x2;
  Coord y2;
} WinLineType;
```

Field Descriptions

`x1`	X coordinate of the first endpoint of the line.
`y1`	Y coordinate of the first endpoint of the line.
`x2`	X coordinate of the second endpoint of the line.
`y2`	Y coordinate of the second endpoint of the line.

Compatibility This type is implemented only if 3.5 New Feature Set is present.

WinPtr

The `WinPtr` type is a pointer to a `WindowType` structure.

```
typedef WindowType *WinPtr;
```

Window Functions

WinClipRectangle

Purpose Shrink the rectangle to make it fit within the clipping region of the current draw window.

Prototype

```
void WinClipRectangle (RectangleType *rP)
```

Parameters

`<-> rP`	Pointer to a structure holding the rectangle to clip. The rectangle returned is the intersection of the rectangle passed and the clipping bounds of the draw window.

Result Returns nothing.

Comments This function does not change the clipping rectangle of the window. To modify the window's clipping rectangle, use the `WinSetClip` and `WinResetClip` functions.

The draw window is the window to which all drawing functions send their output. It is returned by `WinGetDrawWindow`.

See Also `WinCopyRectangle`, `WinDrawRectangle`, `WinEraseRectangle`, `WinGetClip`

WinCopyRectangle

Purpose Copy a rectangular region from one place to another (either between windows or within a single window).

Prototype

```
void WinCopyRectangle (WinHandle srcWin,
WinHandle dstWin, const RectangleType *srcRect,
Coord destX, Coord destY, WinDrawOperation mode)
```

Parameters

`-> srcWin`	Window from which the rectangle is copied. If `NULL`, use the draw window.
`-> dstWin`	Window to which the rectangle is copied. If `NULL`, use the draw window.
`-> srcRect`	Bounds of the region to copy.
`-> destX`	Top bound of the rectangle in destination window.
`-> destY`	Left bound of the rectangle in destination window.
`-> mode`	The method of transfer from the source to the destination window (see `WinDrawOperation`).

Result Returns nothing.

Comments Copies the bits of the window inside the rectangle region.

If the destination bitmap is compressed, the mode parameter must be `winPaint`, and the destination coordinates must be (0,0). If the width of the destination rectangle is less than 16 pixels or if the destination coordinates are not (0,0), then this function turns off compression for the destination bitmap. Normally, you do not copy to a compressed bitmap. Instead, you copy to an uncompressed bitmap and compress it afterwards.

Compatibility In OS versions before 3.5, the `mode` parameter was defined as type `ScrOperation`. It is defined as type `WinDrawOperation` only if 3.5 New Feature Set is present. `ScrOperation` and `WinDrawOperation` are fully compatible with each other.

In OS versions before 3.5, it was common practice to render a bitmap in an offscreen window and then use `WinCopyRectangle` to draw it on the screen. In version 3.5 and higher, the preferred method of doing this is to use `WinDrawBitmap` or `WinPaintBitmap`.

See Also `WinDrawBitmap`

WinCreateBitmapWindow

Purpose Create a new offscreen window.

Prototype

```
WinHandle WinCreateBitmapWindow
(BitmapType *bitmapP, UInt16 *error)
```

Parameters

`-> bitmapP`	Pointer to a bitmap to associate with the window. (See `BitmapType`.)
`<- error`	Pointer to any error this function encounters.

Result Returns the handle of the new window upon success, or `NULL` if an error occurs. The `error` parameter contains one of the following:

`errNone`	No error.
`sysErrParamErr`	The `bitmapP` parameter is invalid. The bitmap must be uncompressed and it must have a valid pixel size (1, 2, 4, or 8). It must not be the screen bitmap.
`sysErrNoFreeResource`	There is not enough memory to allocate a new window structure.

Comments Use `WinCreateBitmapWindow` if you want to draw into a previously created bitmap, such as a bitmap created using `BmpCreate`.

This function generates a window wrapper for the specified bitmap. The newly created window is offscreen, uses the generic format (for device independence), and is added to the active window list. Use

WinSetDrawWindow to make it the draw window, and then use the window drawing functions to modify the bitmap.

When you use this function to create a window and then delete the window with WinDeleteWindow, the bitmap is **not** freed when the window is freed.

WinCreateOffscreenWindow uses this function to create its offscreen window. If you call `WinCreateOffscreenWindow` instead of using this function, the bitmap is freed when `WinDeleteWindow` is called.

The bitmap data will not be blitted properly if the depth of the screen is changed using WinScreenMode and the new window uses a bitmap that does not define the bitmap's color table. See WinScreenMode for information on how to work around this limitation.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also WinCreateWindow, WinCreateOffscreenWindow

WinCreateOffscreenWindow

Purpose Create a new offscreen window and add it to the window list.

Prototype

```
WinHandle WinCreateOffscreenWindow (Coord width,
Coord height, WindowFormatType format,
UInt16 *error)
```

Parameters

`-> width`	Width of the window in pixels.
`-> height`	Height of the window in pixels.
`-> format`	Either `screenFormat` or `genericFormat`. In general, you should use `genericFormat` for offscreen windows.
`<- error`	Pointer to any error this function encounters.

Result Returns the handle of the new window upon success, or `NULL` if an error occurs. The `error` parameter contains one of the following:

`errNone`	No error.

`sysErrParamErr` The `width` or `height` parameter is `NULL` or the current color table is invalid.

`sysErrNoFreeResource`
There is not enough memory to complete the function.

The debug ROM gives a warning if you try to draw to a bad window address.

Comments

Windows created with this routine draw to a memory buffer instead of the display. Use this function for temporary drawing operations such as double-buffering or save-behind operations.

The memory buffer has two formats: screen format and generic format. Screen format is the native format of the video system; windows in this format can be copied to the display faster. The generic format is device-independent. A window cannot be enabled (that is, accept pen input) unless it uses screen format.

This function differs from `WinCreateBitmapWindow` in the following ways:

- `WinCreateOffscreenWindow` creates a new bitmap in the same depth as the current screen. `WinCreateBitmapWindow` uses the bitmap you pass in, which may or may not be in the same depth as the current screen.
- `WinCreateOffscreenWindow` uses the screen format you specify. `WinCreateBitmapWindow` always uses `genericFormat` for the format argument.
- When you delete the window created with `WinCreateOffscreenWindow`, its bitmap is freed along with the window. The bitmap used in the `WinCreateBitmapWindow` is not freed when the window is freed.

The bitmap data will not be blitted properly if the depth of the screen is changed using `WinScreenMode` and the new window uses a bitmap that does not define the bitmap's color table. See `WinScreenMode` for information on how to work around this limitation.

See Also

`WinCreateWindow`, `WinScreenMode`,

WinCreateWindow

Purpose Create a new window and add it to the window list.

Prototype

```
WinHandle WinCreateWindow
(const RectangleType *bounds, FrameType frame,
Boolean modal, Boolean focusable, UInt16 *error)
```

Parameters

`-> bounds`	Display-relative bounds of the window.
`-> frame`	Type of frame around the window (see `FrameType`).
`-> modal`	`true` if the window is modal.
`-> focusable`	`true` if the window can be the active window.
`<- error`	Pointer to any error encountered by this function.

Result Returns the handle of the new window upon success, or `NULL` if an error occurs. The `error` parameter contains one of the following:

`errNone`	No error.
`sysErrNoFreeResource`	There is not enough memory to complete the operation.

Comments Windows created by this routine draw to the display. See `WinCreateOffscreenWindow` for information on drawing off screen.

You typically don't call this function directly. Instead, you use `FrmInitForm` to create form windows from a resource. Forms are much more flexible and have better system support. All forms are windows, but not all windows are forms.

The window is created with the bounds and frame type that you specify and uses the bitmap and drawing state of the current draw window. Its clipping region is reset according to the bounds you specify.

All window flags are set to 0 except for the modal and focusable flags, which you pass as a parameter to this function. Specifically, newly created windows are disabled and invisible. You must specifically enable the window before the window can accept input. You can do so with `WinSetActiveWindow`.

See Also `WinDeleteWindow`

WinDeleteWindow

Purpose Remove a window from the window list and free the memory used by the window.

Prototype

```
void WinDeleteWindow (WinHandle winHandle,
Boolean eraseIt)
```

Parameters

`-> winHandle`	Handle of window to delete.
`-> eraseIt`	If `true`, the window is erased before it is deleted. If `false`, the window is not erased.

Result Returns nothing.

Comments This function frees all memory associated with the window. Windows created using `WinCreateOffscreenWindow` have their bitmaps freed; windows created using `WinCreateWindow` or `WinCreateBitmapWindow` do not.

The `eraseIt` parameter affects onscreen windows only; offscreen windows are never erased. As a performance optimization, you might set `eraseIt` to `false` for an onscreen window if you know that you are going to immediately redraw the area anyway. For example, when the form manager closes a form dialog, it restores the area with the save-behind bits it had stored for that form. For this reason, when the form manager deletes the dialog window, it passes `false` for `eraseIt` because the entire area will be redrawn.

WinDisplayToWindowPt

Purpose Convert a display-relative coordinate to a window-relative coordinate. The coordinate returned is relative to the display window.

Prototype
```
void WinDisplayToWindowPt (Coord *extentX,
Coord *extentY)
```

Parameters

`<-> extentX`	Pointer to x coordinate to convert.
`<-> extentY`	Pointer to y coordinate to convert.

Result Returns nothing.

See Also WinWindowToDisplayPt

WinDrawBitmap

Purpose Draw a bitmap at the given coordinates in `winPaint` mode (see WinDrawOperation for mode details).

Prototype
```
void WinDrawBitmap (BitmapPtr bitmapP, Coord x,
Coord y)
```

Parameters

`-> bitmapP`	Pointer to a bitmap.
`-> x`	The x coordinate of the top-left corner.
`-> y`	The y coordinate of the top-left corner.

Result Returns nothing.

Comments If the bitmap has multiple depths (is a bitmap family), the closest match less than or equal to the current draw window depth is used. If such a bitmap does not exist, the bitmap with the closest match greater than the draw window depth is used.

If the bitmap has its own color table, color conversion to the draw window color table will be applied (on OS 3.5 or later). This color conversion is slow and not recommended. Instead of including a

color table in the bitmap, consider using WinPalette to change the system color table, draw the bitmap, and then change the system color table back when the bitmap is no longer visible.

This function differs from WinPaintBitmap in that this function always uses winPaint mode (copy mode) as the transfer mode. WinPaintBitmap uses the current drawing state transfer mode.

See Also WinEraseRectangle

WinDrawChar

Purpose Draw the specified character in the draw window.

Prototype

```
void WinDrawChar (WChar theChar, Coord x, Coord y)
```

Parameters

-> theChar	The character to draw. This may be either a single-byte character or a multi-byte character.
-> x	x coordinate of the location where the character is to be drawn (left bound).
-> y	y coordinate of the location where the character is to be drawn (top bound).

Result Returns nothing.

Comments Before calling this function, call WinSetUnderlineMode and FntSetFont to set the desired underline and font to draw the characters.

This function differs from WinPaintChar in that this function always uses winPaint mode (see WinDrawOperation). This means the on bits are drawn in the text color, the off bits are in the background color, and underlines are in the foreground color. WinPaintChar uses the current drawing state transfer mode instead of winPaint.

Compatibility Implemented only if 3.1 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS,

link with the PalmOSGlue library and call `WinGlueDrawChar`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `WinDrawChars`, `WinDrawInvertedChars`, `WinDrawTruncChars`, `WinEraseChars`, `WinInvertChars`, `WinPaintChars`

WinDrawChars

Purpose Draw the specified characters in the draw window.

Prototype

```
void WinDrawChars (const Char *chars, Int16 len,
Coord x, Coord y)
```

Parameters

`-> chars`	Pointer to the characters to draw.
`-> len`	Length in bytes of the characters to draw.
`-> x`	x coordinate of the first character to draw (left bound).
`-> y`	y coordinate of the first character to draw (top bound).

Result Returns nothing.

Comments This function is useful for printing non-editable status or warning messages on the screen.

Before calling this function, call `WinSetUnderlineMode` and `FntSetFont` to set the desired underline and font to draw the characters.

This function differs from `WinPaintChars` in that this function always uses `winPaint` mode (see `WinDrawOperation`). This means the on bits are drawn in the text color, the off bits are in the background color, and underlines are in the foreground color. `WinPaintChar` uses the current drawing state transfer mode instead of `winPaint`.

See Also `WinDrawChar`, `WinDrawInvertedChars`, `WinDrawTruncChars`, `WinEraseChars`, `WinInvertChars`, `WinPaintChar`

WinDrawGrayLine

Purpose Draw a dashed line in the draw window.

Prototype
```
void WinDrawGrayLine (Coord x1, Coord y1,
Coord x2, Coord y2)
```

Parameters

`-> x1`	x coordinate of line start point.
`-> y1`	y coordinate of line start point.
`-> x2`	x coordinate of line endpoint.
`-> y2`	y coordinate of line endpoint.

Result Returns nothing.

Comments This routine does not draw in the gray color; it draws with alternating foreground and background pixels. That is, it uses the `grayPattern` pattern type.

See Also `WinDrawLine`, `WinEraseLine`, `WinFillLine`, `WinInvertLine`, `WinPaintLine`, `WinPaintLines`

WinDrawGrayRectangleFrame

Purpose Draw a gray rectangular frame in the draw window.

Prototype
```
void WinDrawGrayRectangleFrame (FrameType frame,
const RectangleType *rP)
```

Parameters

`-> frame`	Type of frame to draw (see `FrameType`).
`-> rP`	Pointer to the rectangle to frame.

Result Returns nothing.

Comments This routine does not draw in the gray color; it draws with alternating foreground and background pixels. The standard gray

pattern is not used by this routine; rather, the frame is drawn so that the top-left pixel of the frame is always on.

See Also WinDrawRectangleFrame, WinEraseRectangleFrame, WinGetFramesRectangle, WinInvertRectangleFrame, WinPaintRectangleFrame

WinDrawInvertedChars

Purpose Draw the specified characters inverted (background color) in the draw window.

Prototype

```
void WinDrawInvertedChars (const Char *chars,
Int16 len, Coord x, Coord y)
```

Parameters

`-> chars`	Pointer to the characters to draw.
`-> len`	Length in bytes of the characters to draw.
`-> x`	x coordinate of the first character to draw (left bound).
`-> y`	y coordinate of the first character to draw (top bound).

Result Returns nothing.

Comments This routine draws the **on** bits and any underline in the background color and the **off** bits in the text color. (Black and white uses copy NOT mode.) This is the standard function for drawing inverted text.

Before calling this function, consider calling WinSetUnderlineMode and FntSetFont.

See Also WinDrawChar, WinDrawChars, WinDrawTruncChars, WinEraseChars, WinInvertChars, WinPaintChar, WinPaintChars

WinDrawLine

Purpose Draw a line in the draw window using the current foreground color.

Prototype
```
void WinDrawLine (Coord x1, Coord y1, Coord x2,
Coord y2)
```

Parameters

`-> x1`	x coordinate of line start point.
`-> y1`	y coordinate of line start point.
`-> x2`	x coordinate of line endpoint.
`-> y2`	y coordinate of line endpoint.

Result Returns nothing.

Comments This function differs from `WinPaintLine` in that it always uses `winPaint` mode (see `WinDrawOperation`). `WinPaintLine` uses the current drawing state transfer mode instead of `winPaint`.

See Also `WinDrawGrayLine`, `WinEraseLine`, `WinFillLine`, `WinInvertLine`, `WinPaintLine`, `WinPaintLines`

WinDrawPixel

Purpose Draw a pixel in the draw window using the current foreground color.

Prototype
```
void WinDrawPixel (Coord x, Coord y)
```

Parameters

`-> x`	Pointer to the x coordinate of a pixel.
`-> y`	Pointer to the y coordinate of a pixel.

Result Returns nothing. May display a fatal error message if the draw window's bitmap is compressed.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinErasePixel`, `WinInvertPixel`, `WinPaintPixel`, `WinPaintPixels`

WinDrawRectangle

Purpose Draw a rectangle in the draw window using the current foreground color.

Prototype
```
void WinDrawRectangle (const RectangleType *rP,
UInt16 cornerDiam)
```

Parameters

`-> rP`	Pointer to the rectangle to draw.
`-> cornerDiam`	Radius of rounded corners. Specify zero for square corners.

Result Returns nothing.

Comments The `cornerDiam` parameter specifies the radius of four imaginary circles used to form the rounded corners. An imaginary circle is placed within each corner tangent to the rectangle on two sides.

This function differs from `WinPaintRectangle` in that it always uses `winPaint` mode (see `WinDrawOperation`). `WinPaintRectangle` uses the current drawing state transfer mode instead of `winPaint`.

See Also `WinEraseRectangle`, `WinFillRectangle`, `WinInvertRectangle`

WinDrawRectangleFrame

Purpose Draw a rectangular frame in the draw window using the current foreground color.

Prototype
```
void WinDrawRectangleFrame (FrameType frame,
const RectangleType *rP)
```

Parameters

`-> frame`	Type of frame to draw (see `FrameType`).
`-> rP`	Pointer to the rectangle to frame.

Result Returns nothing.

Comments The frame is drawn outside the specified rectangle.

This function differs from `WinPaintRectangleFrame` in that it always uses `winPaint` mode (see `WinDrawOperation`). `WinPaintRectangleFrame` uses the current drawing state transfer mode instead of `winPaint`.

See Also `WinDrawGrayRectangleFrame`, `WinEraseRectangleFrame`, `WinGetFramesRectangle`, `WinInvertRectangleFrame`

WinDrawTruncChars

Purpose Draw the specified characters in the draw window, truncating the characters to the specified width.

Prototype

```
void WinDrawTruncChars (const Char *chars,
Int16 len, Coord x, Coord y, Coord maxWidth)
```

Parameters

`-> chars`	Pointer to the characters to draw.
`-> len`	Length in bytes of the characters to draw.
`-> x`	x coordinate of the first character to draw (left bound).
`-> y`	y coordinate of the first character to draw (top bound).
`-> maxWidth`	Maximum width in pixels of the characters that are to be drawn.

Result Returns nothing.

Comments Before calling this function, consider calling `WinSetUnderlineMode` and `FntSetFont`.

If drawing all of the specified characters requires more space than `maxWidth` allows, `WinDrawTruncChars` draws one less than the number of characters that can fit in `maxWidth` and then draws an ellipsis (...) in the remaining space. (If the boundary characters are narrower than the ellipsis, more than one character may be dropped to make room.) If `maxWidth` is narrower than the width of an ellipsis, nothing is drawn.

Use this function to truncate text that may contain multi-byte characters.

Compatibility Implemented only if 3.1 New Feature Set is present. To use this function in code intended to be run on earlier versions of Palm OS, link with the PalmOSGlue library and call `WinGlueDrawTruncChars`. For more information, see Chapter 76, "PalmOSGlue Library."

See Also `WinDrawChar`, `WinDrawChars`, `WinDrawInvertedChars`, `WinEraseChars`, `WinInvertChars`, `WinPaintChar`, `WinPaintChars`

WinEraseChars

Purpose Erase the specified characters in the draw window.

Prototype

```
void WinEraseChars (const Char *chars, Int16 len,
Coord x, Coord y)
```

Parameters

`-> chars`	Pointer to the characters to erase.
`-> len`	Length in bytes of the characters to erase.
`-> x`	x coordinate of the first character to erase (left bound).
`-> y`	y coordinate of the first character to erase (top bound).

Result Returns nothing.

Comments The `winMask` transfer mode is used to erase the characters. See `WinDrawOperation` for more information. This has the effect of erasing only the on bits for the characters rather than the entire text rectangle. This function only works if the foreground color is black and the background color is white.

See Also `WinDrawChar`, `WinDrawChars`, `WinDrawInvertedChars`, `WinDrawTruncChars`, `WinInvertChars`, `WinPaintChar`, `WinPaintChars`

WinEraseLine

Purpose Draw a line in the draw window using the current background color.

Prototype
```
void WinEraseLine (Coord x1, Coord y1, Coord x2,
Coord y2)
```

Parameters

`-> x1`	x coordinate of line start point.
`-> y1`	y coordinate of line start point.
`-> x2`	x coordinate of line endpoint.
`-> y2`	y coordinate of line endpoint.

Result Returns nothing.

See Also WinDrawGrayLine, WinDrawLine, WinFillLine, WinInvertLine, WinPaintLine, WinPaintLines

WinErasePixel

Purpose Draw a pixel in the draw window using the current background color.

Prototype
```
void WinErasePixel (Coord x, Coord y)
```

Parameters

`-> x`	Pointer to the x coordinate of a pixel.
`-> y`	Pointer to the y coordinate of a pixel.

Result Returns nothing.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also WinDrawPixel, WinInvertPixel, WinPaintPixel, WinPaintPixels

WinEraseRectangle

Purpose Draw a rectangle in the draw window using the current background color.

Prototype
```
void WinEraseRectangle (const RectangleType *rP,
UInt16 cornerDiam)
```

Parameters

`-> rP`	Pointer to the rectangle to erase.
`-> cornerDiam`	Radius of rounded corners. Specify zero for square corners.

Result Returns nothing.

Comments The `cornerDiam` parameter specifies the radius of four imaginary circles used to form the rounded corners. An imaginary circle is placed within each corner tangent to the rectangle on two sides.

See Also `WinDrawRectangle`, `WinFillRectangle`, `WinInvertRectangle`, `WinPaintRectangle`

WinEraseRectangleFrame

Purpose Draw a rectangular frame in the draw window using the current background color.

Prototype
```
void WinEraseRectangleFrame (FrameType frame,
const RectangleType *rP)
```

Parameters

`-> frame`	Type of frame to draw (see `FrameType`).
`-> rP`	Pointer to the rectangle to frame.

Result Returns nothing.

See Also `WinDrawGrayRectangleFrame`, `WinDrawRectangleFrame`, `WinGetFramesRectangle`, `WinInvertRectangleFrame`, `WinPaintRectangleFrame`

WinEraseWindow

Purpose Erase the contents of the draw window.

Prototype
```
void WinEraseWindow (void)
```

Parameters None.

Result Returns nothing.

Comments `WinEraseRectangle` is used to erase the window. This routine doesn't erase the frame around the draw window. See `WinEraseRectangleFrame` and `WinGetWindowFrameRect`.

WinFillLine

Purpose Fill a line in the draw window with the current pattern.

Prototype
```
void WinFillLine (Coord x1, Coord y1, Coord x2,
Coord y2)
```

Parameters

`-> x1`	x coordinate of line start point.
`-> y1`	y coordinate of line start point.
`-> x2`	x coordinate of line endpoint.
`-> y2`	y coordinate of line endpoint.

Result Returns nothing.

Comments You can set the current pattern with `WinSetPattern`.

See Also `WinDrawGrayLine`, `WinDrawLine`, `WinEraseLine`, `WinInvertLine`, `WinPaintLine`, `WinPaintLines`

WinFillRectangle

Purpose Draw a rectangle in the draw window with current pattern.

Prototype
```
void WinFillRectangle (const RectangleType *rP,
UInt16 cornerDiam)
```

Parameters

`-> rP`	Pointer to the rectangle to draw.
`-> cornerDiam`	Radius of rounded corners. Specify zero for square corners.

Result Returns nothing.

Comments You can set the current pattern with `WinSetPattern`.

The `cornerDiam` parameter specifies the radius of four imaginary circles used to form the rounded corners. An imaginary circle is placed within each corner tangent to the rectangle on two sides.

See Also `WinDrawRectangle`, `WinEraseRectangle`, `WinInvertRectangle`, `WinPaintRectangle`

WinGetActiveWindow

Purpose Return the window handle of the active window.

Prototype
```
WinHandle WinGetActiveWindow (void)
```

Parameters None.

Result Returns the handle of the active window. All user input is directed to the active window.

See Also `WinSetActiveWindow`, `WinGetDisplayWindow`, `WinGetFirstWindow`, `WinGetDrawWindow`

WinGetBitmap

Purpose Return a pointer to a window's bitmap, which holds the window contents.

Prototype
```
BitmapType *WinGetBitmap (WinHandle winHandle)
```

Parameters

`-> winHandle`	Handle of window from which to get the bitmap.

Result Returns a pointer to the bitmap or `NULL` if `winHandle` is invalid.

Comments For onscreen windows, the bitmap returned always represents the whole screen. Thus, the top-left corner of the returned bitmap may not be the top-left corner of the window.

Compatibility Implemented only if 3.5 New Feature Set is present.

WinGetBounds

Purpose Return the bounds of the current draw window in display-relative coordinates.

Prototype
```
void WinGetBounds (WinHandle winH,
RectangleType *rP)
```

Parameters

`-> winH`	Handle to a window.
`<- rP`	Pointer to a rectangle.

Result Returns nothing.

Comments This function returns in `rP` the bounds of the window represented by `winH`. This corresponds to the convention used by `WinSetBounds`, because it takes a window handle as an argument.

Prior to Palm OS 4.0, `WinGetBounds` returned the bounds of the draw window, and did not take a window handle as an argument. If an application needed to determine the bounds of an arbitrary

window, the application would call WinSetDrawWindow to temporarily set the draw window to the desired window, then WinGetBounds would be called to get the bounds of the draw window, then WinSetDrawWindow would be called again to restore the draw window. This is no longer necessary.

Compatibility Implemented only if 4.0 New Feature Set is **not** present. As of Palm OS 4.0, applications should use WinGetDrawWindowBounds.

See Also WinGetWindowExtent, WinGetDrawWindowBounds

WinGetClip

Purpose Return the clipping rectangle of the draw window.

Prototype
```
void WinGetClip (RectangleType *rP)
```

Parameters

`<- rP`	Pointer to a structure to hold the clipping bounds.

Result Returns nothing.

See Also WinSetClip

WinGetDisplayExtent

Purpose Return the width and height of the display (the screen).

Prototype
```
void WinGetDisplayExtent (Coord *extentX,
Coord *extentY)
```

Parameters

`<- extentX`	Pointer to the width of the display in pixels.
`<- extentY`	Pointer to the height of the display in pixels.

Result Returns nothing.

WinGetDisplayWindow

Purpose Return the window handle of the display (screen) window.

Prototype
```
WinHandle WinGetDisplayWindow (void)
```

Parameters None.

Result Returns the handle of display window.

Comments The display window is created by the system at start-up; it has the same size as the Palm OS drawable area of the physical display (screen).

See Also WinGetDisplayExtent, WinGetActiveWindow, WinGetDrawWindow

WinGetDrawWindow

Purpose Return the window handle of the current draw window.

Prototype
```
WinHandle WinGetDrawWindow (void)
```

Parameters None.

Result Returns handle of draw window.

See Also WinGetDisplayWindow, WinGetActiveWindow, WinSetDrawWindow



New WinGetDrawWindowBounds

Purpose Return the bounds of the draw window.

Prototype
```
void WinGetDrawWindowBounds (RectangleType *rP)
```

Parameters <- `rP` Pointer to the window bounds.

Result Returns nothing.

Comments A pointer to the bounds of the draw window is returned. This function is equivalent to `WinGetBounds` that was in Palm OS prior to 4.0.

See Also `WinGetBounds`

WinGetFirstWindow

Purpose Return a pointer to the first window in the linked list of windows.

Prototype
```
WinHandle WinGetFirstWindow (void)
```

Parameters None.

Result Returns handle of first window.

Comments This function is usually used by the system only.

See Also `WinGetActiveWindow`

WinGetFramesRectangle

Purpose Return the rectangle that includes a rectangle together with the specified frame around it.

Prototype
```
void WinGetFramesRectangle (FrameType frame,
const RectangleType *rP,
RectangleType *obscuredRect)
```

Parameters

`-> frame`	Type of rectangle frame (see `FrameType`).
`-> rP`	Pointer to the rectangle to frame.
`<- obscuredRect`	Pointer to the rectangle that includes both the specified rectangle and its frame.

Result Returns nothing.

Comments Frames are always drawn around (outside) a rectangle.

See Also `WinGetWindowFrameRect`, `WinGetBounds`

WinGetPattern

Purpose Return the current fill pattern.

Prototype
```
void WinGetPattern (CustomPatternType *patternP)
```

Parameters

`<- patternP`	Buffer where the current pattern is returned (see `CustomPatternType`).

Result Returns nothing.

Comments The fill pattern is used by `WinFillLine` and `WinFillRectangle`.

This function returns the value of `patternData` in the current drawing state. (See `DrawStateType`.) The `patternData` field is only set if the pattern field is `customPattern`. Therefore, it's a

good idea to use WinGetPatternType instead of this function on systems that support WinGetPatternType.

See Also WinSetPattern

WinGetPatternType

Purpose Return the current pattern type.

Prototype
```
PatternType WinGetPatternType (void)
```

Parameters None.

Result Returns the current draw window pattern type (see PatternType). If the return value is `customPattern`, you can retrieve the pattern with WinGetPattern.

Comments The fill pattern is used by WinFillLine and WinFillRectangle.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also WinSetPatternType

WinGetPixel

Purpose Return the color value of a pixel in the current draw window.

Prototype
```
IndexedColorType WinGetPixel (Coord x, Coord y)
```

Parameters

`-> x`	Pointer to the x coordinate of a pixel.
`-> y`	Pointer to the y coordinate of a pixel.

Result Returns the indexed color value of the pixel. See IndexedColorType. A return value of 0 means either that the coordinates do not lie in the current draw window or that they do and the color of that pixel is index 0 (typically white).

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also WinIndexToRGB



WinGetPixelRGB

Purpose Return the RGB color values of a pixel in the current draw window.

Prototype

```
Err WinGetPixelRGB (Coord x, Coord y,
RGBColorType *rgbP)
```

Parameters

`-> x`	Pointer to the x coordinate of a pixel.
`-> y`	Pointer to the y coordinate of a pixel.
`<- rgbP`	RGB color components of the pixel.

Result Returns `errNone` or `sysErrParamErr`. `sysErrParamErr` is returned when the `x` or `y` arguments are < 0 or when they are outside the bounds of the draw window.

Comments The RGB color values of the pixel are returned as an `RGBColorType`. This function can be used with both indexed or direct color modes. A return value of `sysErrParamErr` means that the coordinates do not lie in the current draw window.

Compatibility Implemented only if 4.0 New Feature Set is present.

WinGetWindowExtent

Purpose Return the width and height of the current draw window.

Prototype

```
void WinGetWindowExtent (Coord *extentX,
Coord *extentY)
```

Parameters

`<- extentX`	Pointer to the width in pixels of the draw window.

	`<- extentY`	Pointer to the height in pixels of the draw window.

Result Returns nothing.

See Also `WinGetBounds`, `WinGetWindowFrameRect`,

WinGetWindowFrameRect

Purpose Return a rectangle, in display-relative coordinates, that defines the size and location of a window and its frame.

Prototype

```
void WinGetWindowFrameRect (WinHandle winHandle,
RectangleType *r)
```

Parameters

	`-> winHandle`	Handle of window whose coordinates are desired.
	`<- r`	Pointer to the coordinates of the window.

Result Returns nothing.

See Also `WinGetBounds`

WinIndexToRGB

Purpose Convert an index in the currently active color table to an RGB value.

Prototype

```
void WinIndexToRGB (IndexedColorType i,
RGBColorType *rgbP)
```

Parameters

	`-> i`	A color index value. See `IndexedColorType`.
	`<- rgbP`	Pointer to an RGB color value corresponding to the index value `i`. See `RGBColorType`.

Result Returns nothing.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also WinRGBToIndex

WinInvertChars

Purpose Invert the specified characters in the draw window.

Prototype

```
void WinInvertChars (const Char *chars, Int16 len,
Coord x, Coord y)
```

Parameters

`-> chars`	Pointer to the characters to invert.
`-> len`	Length in bytes of the characters to invert.
`-> x`	x coordinate of the first character to invert (left bound).
`-> y`	y coordinate of the first character to invert (top bound).

Result Returns nothing.

Comments This function applies the `winInvert` operation of WinDrawOperation to the characters in the draw window.

To perform color inverting, use WinSetTextColor and WinSetBackColor to choose the desired colors, and call WinPaintChar.

See Also WinDrawChar, WinDrawChars, WinDrawInvertedChars, WinDrawTruncChars, WinEraseChars, WinPaintChar, WinPaintChars

WinInvertLine

Purpose Invert a line in the draw window (using the `WinDrawOperation` `winInvert`).

Prototype
```
void WinInvertLine (Coord x1, Coord y1, Coord x2,
Coord y2)
```

Parameters

`-> x1`	x coordinate of line start point.
`-> y1`	y coordinate of line start point.
`-> x2`	x coordinate of line endpoint.
`-> y2`	y coordinate of line endpoint.

Result Returns nothing.

See Also `WinDrawGrayLine`, `WinDrawLine`, `WinEraseLine`, `WinFillLine`, `WinPaintLine`, `WinPaintLines`

WinInvertPixel

Purpose Invert a pixel in the draw window (using the `WinDrawOperation` `winInvert`).

Prototype
```
void WinInvertPixel (Coord x, Coord y)
```

Parameters

`-> x`	Pointer to the x coordinate of a pixel.
`-> y`	Pointer to the y coordinate of a pixel.

Result Returns nothing.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinDrawPixel`, `WinErasePixel`, `WinPaintPixel`, `WinPaintPixels`

WinInvertRectangle

Purpose Invert a rectangle in the draw window (using the `WinDrawOperation` `winInvert`).

Prototype
```
void WinInvertRectangle (const RectangleType *rP,
UInt16 cornerDiam)
```

Parameters

`-> rP`	Pointer to the rectangle to invert.
`-> cornerDiam`	Radius of rounded corners. Specify zero for square corners.

Result Returns nothing.

Comments The `cornerDiam` parameter specifies the radius of four imaginary circles used to form the rounded corners. An imaginary circle is placed within each corner tangent to the rectangle on two sides.

The operating system itself does not use the inverting routines. Instead, it uses the `winSwap` transfer mode, or it changes the color selection and uses the `WinPaint...` routines.

See Also `WinDrawRectangle`, `WinEraseRectangle`, `WinFillRectangle`, `WinPaintRectangle`

WinInvertRectangleFrame

Purpose Invert a rectangular frame in the draw window (using the `WinDrawOperation` `winInvert`).

Prototype
```
void WinInvertRectangleFrame (FrameType frame,
const RectangleType *rP)
```

Parameters

`-> frame`	Type of frame to draw (see `FrameType`).
`-> rP`	Pointer to the rectangle to frame.

Result Returns nothing.

See Also WinDrawGrayRectangleFrame, WinDrawRectangleFrame, WinEraseRectangleFrame, WinGetFramesRectangle, WinPaintRectangleFrame

WinModal

Purpose Return `true` if the specified window is modal.

Prototype
```
Boolean WinModal (WinHandle winHandle)
```

Parameters

`-> winHandle`	Handle of a window.

Result Returns `true` if the window is modal, otherwise `false`.

Comments A window is modal if it cannot lose the focus.

See Also FrmAlert, FrmCustomAlert, FrmDoDialog

WinPaintBitmap

Purpose Draw a bitmap in the current draw window at the specified coordinates with the current draw mode.

Prototype
```
void WinPaintBitmap (BitmapType *bitmapP, Coord x,
Coord y)
```

Parameters

`-> bitmapP`	Pointer to a bitmap.
`-> x`	The x coordinate of the top-left corner.
`-> y`	The y coordinate of the top-left corner.

Result Returns nothing.

Comments If the bitmap has multiple depths (is a bitmap family), the closest match less than or equal to the current draw window depth is used. If such a bitmap does not exist, the bitmap with the closest match greater than the draw window depth is used.

Using `WinPaintBitmap` is now recommended instead of the previous practice of rendering bitmaps into an offscreen window and then using `WinCopyRectangle` to draw them on screen.

The current draw mode is set by `WinSetDrawMode`.

If the bitmap has its own color table, color conversion to the draw window color table will be applied (on OS 3.5 or later). This color conversion is slow and not recommended. Instead of including a color table in the bitmap, consider using `WinPalette` to change the system color table, draw the bitmap, and then change the system color table back when the bitmap is no longer visible.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinDrawBitmap`, `WinEraseRectangle`

WinPaintChar

Purpose Draw a character in the draw window using the current drawing state.

Prototype

```
void WinPaintChar (WChar theChar, Coord x,
Coord y)
```

Parameters

`-> theChar`	The character to draw. This may be either a single-byte character or a multi-byte character.
`-> x`	x coordinate of the location where the character is to be drawn (left bound).
`-> y`	y coordinate of the location where the character is to be drawn (top bound).

Result Returns nothing.

See Also `WinPaintChar` draws the **on** bits in the text color and the **off** bits in the background color, with underlines (if any) drawn in the foreground color using the current drawing mode.

This function uses the current drawing state, which is stored in a `DrawStateType` structure. See the description of that structure to

learn the functions you can call to set the drawing state to the values you want.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also WinDrawChar, WinDrawChars, WinDrawInvertedChars, WinDrawTruncChars, WinEraseChars, WinInvertChars, WinPaintChars

WinPaintChars

Purpose Draw the specified characters in the draw window with the current draw state.

Prototype

```
void WinPaintChars (const Char *chars, Int16 len,
Coord x, Coord y)
```

Parameters

`-> chars`	Pointer to the characters to draw.
`-> len`	Length in bytes of the characters to draw.
`-> x`	x coordinate of the first character to draw (left bound).
`-> y`	y coordinate of the first character to draw (top bound).

Result Returns nothing.

Comments `WinPaintChars` draws the **on** bits in the text color and the **off** bits in the background color, with underlines (if any) drawn in the foreground color using the current drawing mode.

This function uses the current drawing state, which is stored in a `DrawStateType` structure. See the description of that structure to learn the functions you can call to set the drawing state to the state you want.

Before calling this function, consider calling `WinSetUnderlineMode` and `FntSetFont`.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also WinDrawChar, WinDrawChars, WinDrawInvertedChars, WinDrawTruncChars, WinEraseChars, WinInvertChars, WinPaintChar

WinPaintLine

Purpose Draw a line in the draw window using the current drawing state.

Prototype

```
void WinPaintLine (Coord x1, Coord y1, Coord x2,
Coord y2)
```

Parameters

`-> x1`	x coordinate of line beginning point.
`-> y1`	y coordinate of line beginning point.
`-> x2`	x coordinate of line endpoint.
`-> y2`	y coordinate of line endpoint.

Result Returns nothing.

Comments This function uses the current drawing state, which is stored in a DrawStateType structure. See the description of that structure to learn the functions you can call to set the drawing state to the state you want.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also WinDrawLine, WinDrawGrayLine, WinEraseLine, WinFillLine, WinInvertLine, WinPaintLines

WinPaintLines

Purpose Draw several lines in the draw window using the current drawing state.

Prototype
```
void WinPaintLines (UInt16 numLines,
WinLineType lines[])
```

Parameters

`-> numLines`	Number of lines to paint.
`-> lines`	Array of lines. See `WinLineType`.

Result Returns nothing.

Comments This function uses the current drawing state, which is stored in a `DrawStateType` structure. See the description of that structure to learn the functions you can call to set the drawing state to the state you want.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinDrawLine`, `WinDrawGrayLine`, `WinEraseLine`, `WinFillLine`, `WinInvertLine`, `WinPaintLine`

WinPaintPixel

Purpose Render a pixel in the draw window using the current drawing state.

Purpose
```
void WinPaintPixel (Coord x, Coord y)
```

Parameters

`-> x`	Pointer to the x coordinate of a pixel.
`-> y`	Pointer to the y coordinate of a pixel.

Result Returns nothing.

Comments This function uses the current drawing state, which is stored in a `DrawStateType` structure. See the description of that structure to

learn the functions you can call to set the drawing state to the state you want.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinDrawPixel`, `WinErasePixel`, `WinInvertPixel`, `WinPaintPixels`

WinPaintPixels

Purpose Render several pixels in the draw window using the current drawing state.

Prototype

```
void WinPaintPixels (UInt16 numPoints,
PointType pts[])
```

Parameters

`-> numPoints`	Number of pixels to paint.
`-> pts`	Array of pixels.

Result Returns nothing.

Comments This function uses the current drawing state, which is stored in a `DrawStateType` structure. See the description of that structure to learn the functions you can call to set the drawing state to the state you want.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinDrawPixel`, `WinErasePixel`, `WinInvertPixel`, `WinPaintPixel`

WinPaintRectangle

Purpose Draw a rectangle in the draw window using the current drawing state.

Prototype
```
void WinPaintRectangle (const RectangleType *rP,
UInt16 cornerDiam)
```

Parameters

`-> rP`	Pointer to the rectangle to draw.
`-> cornerDiam`	Radius of rounded corners. Specify zero for square corners.

Result Returns nothing.

Comments The `cornerDiam` parameter specifies the radius of four imaginary circles used to form the rounded corners. An imaginary circle is placed within each corner tangent to the rectangle on two sides.

This function uses the current drawing state, which is stored in a `DrawStateType` structure. See the description of that structure to learn the functions you can call to set the drawing state to the state you want.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinDrawRectangle`, `WinEraseRectangle`, `WinFillRectangle`, `WinInvertRectangle`

WinPaintRectangleFrame

Purpose Draw a rectangular frame in the draw window using the current drawing state.

Prototype
```
void WinPaintRectangleFrame (FrameType frame,
const RectangleType *rP)
```

Parameters

`-> frame`	Type of frame to draw (see `FrameType`).
`-> rP`	Pointer to the rectangle to frame.

Result Returns nothing.

Comments The frame is drawn outside the specified rectangle.

This function uses the current drawing state, which is stored in a `DrawStateType` structure. See the description of that structure to learn the functions you can call to set the drawing state to the state you want.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinDrawGrayRectangleFrame`, `WinDrawRectangleFrame`, `WinEraseRectangleFrame`, `WinGetFramesRectangle`, `WinInvertRectangleFrame`

WinPalette

Purpose Set or retrieve the palette for the draw window.

Prototype

```
Err WinPalette (UInt8 operation, Int16 startIndex,
UInt16 paletteEntries, RGBColorType *tableP)
```

Parameters

`-> operation` Specify one of the following values:

`winPaletteGet`
: Retrieve the palette. Entries are read from the palette beginning at `startIndex` and placed into `tableP` beginning at index 0.

`winPaletteSet`
: Set the palette. Entries from `tableP` (beginning at index 0) are set into the palette beginning at `startIndex` in the palette.

`winPaletteSetToDefault`
: Set the palette to the default system palette.

`-> startIndex`	Identifies where in the palette to start reading or writing. Specify `WinUseTableIndexes` to indicate that the entries are not to be set or read sequentially; instead, the `index` value in each `RGBColorType` entry in `tableP` determines which slot in the palette is to be set or read. You can use this technique to get or set several discontiguous palette entries with a single function call.
`-> paletteEntries`	Number of palette entries to get or set.
`<-> tableP`	A pointer to a buffer of `RGBColorType` entries that is either read from or written to, depending on the `operation` parameter; the table entries from 0 to `paletteEntries` – 1 are affected by this routine.

Result Returns one of the following values:

`errNone`	Success.
`winErrPalette`	The current draw window does not have a color table, a set operation has overflowed the color table, or one of the entries in `tableP` has an invalid index value
`sysErrParamErr`	The `startIndex` value is invalid.

Comments Here are some examples of how this routine works:

- If `startIndex` is 0 and `paletteEntries` is 10, the first 10 elements of the palette will be set from `tableP` or will be copied into `tableP`.
- If `startIndex` is 10 and `paletteEntries` is 5, then entries 10, 11, 12, 13, and 14 in the palette will be set from or copied to elements 0, 1, 2, 3, and 4 in `tableP`.
- If `startIndex` is `WinUseTableIndexes` and `paletteEntries` is 1, then the index value in the `RGBColorType` of element 0 of `tableP` will be read from or copied to `tableP`; in this case, the `index` field of the `RGBColorType` will not change.

During a set operation, this function broadcasts the `sysNotifyDisplayChangeEvent` to notify any interested observer that the color palette has changed.

One use for this function is if you need to display a bitmap that uses a color table other than the one in use by the system. You can attach a custom color table to a bitmap, and if you do, the bitmap is drawn using that color table. However, this is a performance drain. As an optimization, you can use `WinPalette` to change the system color table to match that used by the bitmap, display the bitmap, and use `WinPalette` to reset the color table when the bitmap is no longer visible.

Compatibility Implemented only if 3.5 New Feature Set is present.

WinPopDrawState

Purpose Restore the draw state values to the last saved set on the stack.

Prototype

```
void WinPopDrawState (void)
```

Parameters None.

Result Returns nothing.

Comments Use this routine to restore the draw state saved by the previous call to `WinPushDrawState`.

After you call this function, the current draw window's `drawStateP` field points to the restored drawing state.

Compatibility Implemented only if 3.5 New Feature Set is present.

WinPushDrawState

Purpose Save the current draw state values onto the draw state stack.

Prototype
```
void WinPushDrawState (void)
```

Parameters None.

Result Returns nothing.

Comments Use this routine to save the current draw state before making changes to it using the functions listed in the `DrawStateType` structure's description. Call `WinPopDrawState` to restore the saved settings.

Compatibility Implemented only if 3.5 New Feature Set is present.

WinResetClip

Purpose Reset the clipping rectangle of the draw window to the portion of the draw window that is within the bounds of the display.

Prototype
```
void WinResetClip (void)
```

Parameters None.

Result Returns nothing.

See Also `WinSetClip`

WinRestoreBits

Purpose Copy the contents of the specified window to the draw window and delete the passed window.

Prototype
```
void WinRestoreBits (WinHandle winHandle,
Coord destX, Coord destY)
```

Parameters

`-> winHandle`	Handle of window to copy and delete.
`-> destX`	x coordinate in the draw window to copy to.
`-> destY`	y coordinate in the draw window to copy to.

Result Returns nothing.

Comments This routine is generally used to restore a region of the display that was saved with `WinSaveBits`.

See Also `WinSaveBits`

WinRGBToIndex

Purpose Convert an RGB value to the index of the closest color in the currently active color lookup table (CLUT).

Prototype

```
IndexedColorType WinRGBToIndex
(const RGBColorType *rgbP)
```

Parameters

`-> rgbP`	Pointer to an RGB color value.

Result Returns the index of the closest matching color in the CLUT.

Comments Palm OS 3.5 supports a maximum of 256 colors. The number of possible RGB colors greatly exceeds this amount. For this reason, an exact match may not be available for `rgbP`. If there is no exact RGB match, then a luminance best-fit is used if the color lookup table is entirely gray scale (red, green, and blue values for each entry are identical), or a shortest-distance fit in RGB space is used if the palette contains colors. RGB shortest distance may not always produce the actual closest perceptible color, but it's relatively fast and works for the system palette.

`WinRGBToIndex` uses the draw window's color table to return the appropriate color table index. If the draw window does not have a color table, the default color table of the current screen is used.

If the draw window does not have a color table, and if the depth of the draw window and the depth of the screen are different, this

function will return an inappropriate index. If this situation exists, the application should either define a color table for the draw window, or use WinScreenMode to set the screen depth to the same depth as the draw window before calling WinRGBToIndex.

NOTE: The bitmap data will not be blitted properly if the depth of the screen is changed using WinScreenMode and the new window uses a bitmap that does not define the bitmap's color table. See WinScreenMode for information on how to work around this limitation.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also WinIndexToRGB, WinScreenMode

WinSaveBits

Purpose Create an offscreen window and copy the specified region from the draw window to the offscreen window.

Prototype

```
WinHandle WinSaveBits
(const RectangleType *source, UInt16 *error)
```

Parameters

`-> source`	Pointer to the bounds of the region to save, relative to the display.
`<- error`	Pointer to any error encountered by this function.

Result Returns the handle of the window containing the saved image, or zero if an error occurred.

Comments The offscreen window is the same size as the region to copy.

This function tries to copy the window's bitmap using compressed format if possible. It may display a fatal error message if an error occurs when it tries to shrink the pointer for the compressed bits.

See Also WinRestoreBits

WinScreenLock

Purpose "Lock" the current screen by switching the UI concept of the screen base address to an area that is not reflected on the display.

Prototype

```
UInt8 *WinScreenLock (WinLockInitType initMode)
```

Parameters

`-> initMode` Indicates how to initialize the new screen area. Specify one of the following values:

`winLockCopy`
: Copy old screen to new.

`winLockErase`
: Erase new screen to white.

`winLockDontCare`
: Don't do anything

Result Returns a pointer to the new screen base address, or `NULL` if this routine fails.

Comments This routine can be used to "freeze" the display while doing lengthy drawing operations to avoid a flickering effect. Call `WinScreenUnlock` to unlock the display and cause it to be updated with any changes. The screen must be unlocked as many times as it is locked to actually update the display.

Because this function copies the screen, using it is a relatively expensive operation.

Compatibility Implemented only if 3.5 New Feature Set is present.

WinScreenMode

Purpose Sets or returns display parameters, including display geometry, bit depth, and color support.

Prototype

```
Err WinScreenMode
(WinScreenModeOperation operation, UInt32 *widthP,
UInt32 *heightP, UInt32 *depthP,
Boolean *enableColorP)
```

Parameters The `widthP`, `heightP`, `depthP`, and `enableColorP` parameters are used in different ways for different operations. See Comments at the end of this description for details.

`-> operation` The work this function is to perform, as specified by one of the following selectors:

`winScreenModeGet`
: Return the current settings for the display.

`winScreenModeGetDefaults`
: Return the default settings for the display.

`winScreenModeGetSupportedDepths`
: Return in `depthP` a hexadecimal value indicating the supported screen depths. The binary representation of this value defines a bitfield in which the value 1 indicates support for a particular display depth. The position representing a particular bit depth corresponds to the value $2^{(bitDepth-1)}$. See the Example at the end of this function description for more information.

`winScreenModeGetSupportsColor`
: Return `true` as the value of the `enableColorP` parameter when color mode can be enabled.

`winScreenModeSet`
: Change display settings to the values specified by the other arguments to the `WinScreenMode` function.

`winScreenModeSetToDefaults`
: Change display settings to default values.

`<-> widthP` Pointer to new/old screen width.

`<-> heightP` Pointer to new/old screen height.

`<-> depthP` Pointer to new/old/available screen depth.

`<-> enableColorP`
: Pass `true` to enable color drawing mode.

Result If no error, returns values as specified by the `operation` argument. Various invalid arguments may cause this function to return a `sysErrParamErr` result code. In rare cases, a failed allocation can cause this function to return a `memErrNotEnoughSpace` error.

Comments The `widthP`, `heightP`, `depthP`, and `enableColorP` parameters are used in different ways for different operations. All "get" operations overwrite these values with a result when the function returns. The `winScreenModeSet` operation changes current display parameters when passed valid argument values that are not `NULL` pointers. The `winScreenModeSetToDefaults` operation ignores values passed for all of these parameters.

Table 55.1 summarizes parameter usage for each operation this function performs.

Table 55.1 Use of parameters to WinScreenMode function

Operation `winScreenMode...`	widthP	heightP	depthP	enableColorP
`...Get`	returned	returned	returned	returned
`...GetDefaults`	returned	returned	returned	returned
`...GetSupportedDepths`	pass in	pass in	returned	pass in
`...GetSupportsColor`	pass in	pass in	pass in	returned
`...Set`	pass in	pass in	pass in	pass in
`...SetToDefaults`	ignored	ignored	ignored	ignored

This function ignores `NULL` pointer arguments to the `widthP`, `heightP`, `depthP`, and `enableColorP` parameters; thus, you can pass a `NULL` pointer for any of these values to leave the current value unchanged. Similarly, when getting values, this function does not return a value for any `NULL` pointer argument.

If you change the display depth, it is recommended that you restore it to its previous state when your application closes, even though the system sets display parameters back to their default values when launching an application.

Note that none of the other operations interprets the depth parameter the same way that `winScreenModeGetSupportedDepths` does. For example, to set the display depth to 8-bit mode, you use 8 (decimal) for the display depth, not 0x80 (128 decimal).

When a window is created, and if the window's associated bitmap does not have its own color table, the window will use the system's default color translation tables when a blitting operation occurs to that window. When the system's bit depth changes, the system's default color translation tables are recalculated based on the new screen depth. When the blit occurs at the new screen depth to the offscreen window, the color translation tables are out of sync.

To workaround this system limitation, developers should either:

- allocate offscreen windows after changing the depth, or
- use `WinCreateBitmapWindow` so that it uses a bitmap with a defined color table.

The latter workaround causes the system to perform color matching when blitting, so the first workaround may be preferred.

Compatibility

Implemented only if 3.5 New Feature Set is present. In OS versions prior to 3.5, this function is called `ScrDisplayMode`. The prototype for `ScrDisplayMode` is similar to `WinScreenMode`:

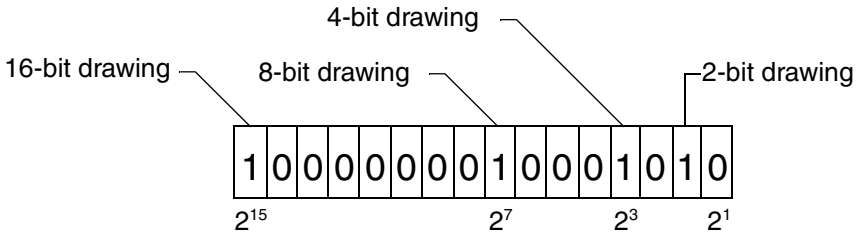
```
Err ScrDisplayMode (
ScrDisplayModeOperation operation,
DWordPtr widthP, DWordPtr heightP,
DWordPtr depthP, BooleanPtr enableColorP)
```

The only other difference between `ScrDisplayMode` and `WinScreenMode` is that the `ScrDisplayModeOperation` constants begin with the prefix `scrDisplayMode` rather than `winScreenMode`.

Example

Here are some additional examples of return values provided by the `winScreenModeGetSupportedDepths` mode of the `WinScreenMode` function.

This function indicates support for 4-bit drawing by returning a value of `0x08`, or 2^3, which corresponds to a binary value of `1000`. Support for bit depths of 2 and 1 is indicated by a return value of `0x03`. Support for bit depths of 4, 2, and 1 is indicated by `0x0B`, which is a binary value of `1011`. Support for bit depths of 16, 8, 4 and 2 is indicated by `0x808A`. The figure immediately following depicts this final example graphically.



WinScreenUnlock

Purpose Unlock the screen and update the display.

Prototype
```
void WinScreenUnlock (void)
```

Parameters None.

Result Returns nothing.

Comments The screen must be unlocked as many times as it is locked to actually update the display.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinScreenLock`

WinScrollRectangle

Purpose Scroll a rectangle in the draw window.

Prototype
```
void WinScrollRectangle (const RectangleType *rP,
WinDirectionType direction, Coord distance,
RectangleType *vacatedP)
```

Parameters

`-> rP`	Pointer to the rectangle to scroll.
`-> direction`	Direction to scroll (`winUp`, `winDown`, `winLeft`, or `winRight`).
`-> distance`	Distance to scroll in pixels.
`<- vacatedP`	Pointer to the rectangle that needs to be redrawn because it has been vacated as a result of the scroll.

Result Returns nothing.

Comments The rectangle scrolls within its own bounds. Any portion of the rectangle that is scrolled outside its bounds is clipped.

WinSetActiveWindow

Purpose Make a window the active window.

Prototype
```
void WinSetActiveWindow (WinHandle winHandle)
```

Parameters
`-> winHandle` Handle of a window.

Result Returns nothing.

Comments The active window is not actually set in this routine; flags are set to indicate that a window is being exited and another window is being entered. The routine `EvtGetEvent` sends a `winExitEvent` and a `winEnterEvent` when it detects these flags. The active window is set by `EvtGetEvent` when it sends the `winEnterEvent`. The draw window is also set to the new active window when the active window is changed.

The window is enabled before it is made active.

All user input is directed to the active window.

See Also `WinGetActiveWindow`, `EvtGetEvent`

WinSetBackColor

Purpose Set the background color to use in subsequent draw operations.

Prototype
```
IndexedColorType WinSetBackColor
(IndexedColorType backColor)
```

Parameters
`-> backColor` Color to set; specify a value of type `IndexedColorType`.

Result Returns the previous background color index.

Comments This function changes the current drawing state. If necessary, use `WinPushDrawState` to preserve the current drawing state before you set this function and use `WinPopDrawState` to restore it later.

To set the foreground color to a predefined UI color default, use UIColorGetTableEntryIndex as an input to this function. For example:

```
curColor = WinSetBackColor
(UIColorGetTableEntryIndex(UIFieldBackground));
```

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also WinSetForeColor, WinSetTextColor



New WinSetBackColorRGB

Purpose Set the background color to use in subsequent draw operations.

Prototype

```
void WinSetBackColorRGB
(const RGBColorType *newRgbP,
RGBColorType *prevRgbP)
```

Parameters

`-> newRgbP`	Color to set; specify a value of type RGBColorType.
`<- prevRgbP`	Previous color; specify a value of type RGBColorType.

Result Returns nothing

Comments This function takes new and previous RGBColorType arguments. It is okay to set `newRgbP` or `prevRgbP` to `NULL`. If an application only wants to get the current color, the `newRgbP` argument is set to `NULL`. If the application does not care about the previous color, `prevRgbP` can be set to `NULL`.

This function sets the `backColorRGB` field of the DrawStateType structure to the value specified by `newRgbP`. It then sets the index field of `newRgbP` to the 8 bit system palette entry that most closely matches the RGB components. Finally, it sets the `backColor` index field of DrawStateType to this index value.

This function changes the current drawing state. If necessary, use `WinPushDrawState` to preserve the current drawing state before you set this function and use `WinPopDrawState` to restore it later.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `WinSetForeColorRGB`, `WinSetTextColorRGB`

WinSetBounds

Purpose Set the bounds of the window to display-relative coordinates.

Prototype

```
void WinSetBounds (WinHandle winHandle,
const RectangleType *rP)
```

Parameters

`-> winHandle`	Handle for the window for which to set the bounds.
`-> rP`	Pointer to a rectangle to use for bounds.

Result Returns nothing.

Comments A visible window cannot have its bounds modified.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `WinGetBounds`

WinSetClip

Purpose Set the clipping rectangle of the draw window.

Prototype

```
void WinSetClip (const RectangleType *rP)
```

Parameters

`-> rP`	Pointer to a structure holding the clipping bounds.

Result Returns nothing.

See Also `WinClipRectangle`, `WinSetClip`, `WinGetClip`

WinSetDrawMode

Purpose Set the transfer mode to use in subsequent draw operations.

Prototype
```
WinDrawOperation WinSetDrawMode
(WinDrawOperation newMode)
```

Parameters

`-> newMode`	Transfer mode to set; specify one of the `WinDrawOperation` values.

Result Returns the previous transfer mode.

Comments This function changes the current drawing state. If necessary, use `WinPushDrawState` to preserve the current drawing state before you set this function and use `WinPopDrawState` to restore it later.

Compatibility Implemented only if 3.5 New Feature Set is present.

WinSetDrawWindow

Purpose Set the draw window. (All drawing operations are relative to the draw window.)

Prototype
```
WinHandle WinSetDrawWindow (WinHandle winHandle)
```

Parameters

`-> winHandle`	Handle of a window.

Result Returns the previous draw window.

Compatibility OS versions before 3.5 allowed you to use `NULL` as a parameter to this function to set the draw window to the display window (or screen window). In version 3.5 and higher, this practice is discouraged. If `winHandle` is `NULL`, the debug ROM sets the draw window to `badDrawWindowValue`, and you are warned if you try to draw to it.

See Also `WinGetDrawWindow`, `WinSetActiveWindow`

WinSetForeColor

Purpose Set the foreground color to use in subsequent draw operations.

Prototype
```
IndexedColorType WinSetForeColor
(IndexedColorType foreColor)
```

Parameters

`-> foreColor`	Color to set; specify a value of type `IndexedColorType`.

Result Returns the previous foreground color index.

Comments This function changes the current drawing state. If necessary, use `WinPushDrawState` to preserve the current drawing state before you set this function and use `WinPopDrawState` to restore it later.

To set the foreground color to a predefined UI color default, use `UIColorGetTableEntryIndex` as an input to this function. For example:

```
curColor = WinSetForeColor
  (UIColorGetTableEntryIndex
  (UIObjectForeground));
```

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinSetBackColor`, `WinSetTextColor`



New WinSetForeColorRGB

Purpose Set the foreground color to use in subsequent draw operations.

Prototype
```
void WinSetForeColorRGB
(const RGBColorType *newRgbP,
RGBColorType *prevRgbP)
```

Parameters

`-> newRgbP`	Color to set; specify a value of type `RGBColorType`.

	`<- prevRgbP`	Previous color; specify a value of type `RGBColorType`.

Result Returns nothing.

Comments This function takes new and previous `RGBColorType` arguments. It is okay to set `newRgbP` or `prevRgbP` to `NULL`. If an application only wants to get the current color, the `newRgbP` argument is set to `NULL`. If the application does not care about the previous color, `prevRgbP` can be set to `NULL`.

This function sets the `foreColorRGB` field of the `DrawStateType` structure to the value specified by `newRgbP`. It then sets the index field of `newRgbP` to the 8 bit system palette entry that most closely matches the RGB components. Finally, it sets the `foreColor` index field of `DrawStateType` to this index value.

This function changes the current drawing state. If necessary, use `WinPushDrawState` to preserve the current drawing state before you set this function and use `WinPopDrawState` to restore it later.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `WinSetBackColorRGB`, `WinSetTextColorRGB`

WinSetPattern

Purpose Set the current fill pattern.

Prototype

```
void WinSetPattern
(const CustomPatternType *patternP)
```

Parameters	`-> patternP`	Pattern to set (see `CustomPatternType`).

Result Returns nothing.

Comments The fill pattern is used by `WinFillLine` and `WinFillRectangle`.

This function changes the current drawing state. If necessary, use `WinPushDrawState` to preserve the current drawing state before you set this function and use `WinPopDrawState` to restore it later.

See Also `WinGetPattern`

WinSetPatternType

Purpose Set the current pattern type.

Prototype
```
void WinSetPatternType (PatternType newPattern)
```

Parameters

`-> newPattern`	Pattern type to set for the draw window (see `PatternType`).

Result Returns nothing.

Comments This function sets the `pattern` field of the drawing state to `newPattern` and sets the `patternData` field to `NULL`. To set `patternData` to a custom pattern use `WinSetPattern`.

The fill pattern is used by `WinFillLine` and `WinFillRectangle`.

This function changes the current drawing state. If necessary, use `WinPushDrawState` to preserve the current drawing state before you set this function and use `WinPopDrawState` to restore it later.

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinGetPatternType`

WinSetTextColor

Purpose Set the color to use for drawing characters in subsequent draw operations.

Prototype
```
IndexedColorType WinSetTextColor
(IndexedColorType textColor)
```

Parameters

`-> textColor`	Color to set; specify a value of type `IndexedColorType`.

Result Returns the previous text color index.

Comments This function changes the current drawing state. If necessary, use `WinPushDrawState` to preserve the current drawing state before you set this function and use `WinPopDrawState` to restore it later.

To set the foreground color to a predefined UI color default, use `UIColorGetTableEntryIndex` as an input to this function. For example:

```
curColor = WinSetTextColor
  (UIColorGetTableEntryIndex(UIFieldText));
```

Compatibility Implemented only if 3.5 New Feature Set is present.

See Also `WinSetBackColor`, `WinSetForeColor`



New WinSetTextColorRGB

Purpose Set the color to use for drawing characters in subsequent draw operations.

Prototype

```
void WinSetTextColorRGB
(const RGBColorType *newRgbP,
RGBColorType *prevRgbP)
```

Parameters

`-> newRgbP`	Color to set; specify a value of type `RGBColorType`.
`<- prevRgbP`	Previous color; specify a value of type `RGBColorType`.

Result Returns nothing.

Comments This function takes new and previous `RGBColorType` arguments. It is okay to set `newRgbP` or `prevRgbP` to `NULL`. If an application only

wants to get the current color, the `newRgbP` argument is set to `NULL`. If the application does not care about the previous color, `prevRgbP` can be set to `NULL`.

This function sets the `textColorRGB` field of the `DrawStateType` structure to the value specified by `newRgbP`. It then sets the index field of `newRgbP` to the 8 bit system palette entry that most closely matches the RGB components. Finally, it sets the `textColor` index field of `DrawStateType` to this index value.

This function changes the current drawing state. If necessary, use `WinPushDrawState` to preserve the current drawing state before you set this function and use `WinPopDrawState` to restore it later.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `WinSetBackColorRGB`, `WinSetForeColorRGB`

WinSetUnderlineMode

Purpose Set the graphic state to enable or disable the underlining of characters.

Prototype

```
UnderlineModeType WinSetUnderlineMode
(UnderlineModeType mode)
```

Parameters

`<-> mode`	New underline mode type; see `UnderlineModeType`.

Result Returns the previous underline mode type.

Comments This function changes the current drawing state. If necessary, use `WinPushDrawState` to preserve the current drawing state before you set this function and use `WinPopDrawState` to restore it later.

See Also `WinDrawChars`

WinValidateHandle

Purpose Return `true` if the specified handle references a valid window object.

Prototype
```
Boolean WinValidateHandle (WinHandle winHandle)
```

Parameters

`-> winHandle` The handle to be tested.

Result Returns `true` if the specified handle references a non-`NULL` pointer to a window in the active window list, `false` if the handle references a window whose values are out of sync with the current system state.

Comments For debugging purposes only. Do not include this function in commercial applications.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `FrmValidatePtr`, `FrmRemoveObject`

WinWindowToDisplayPt

Purpose Convert a window-relative coordinate to a display-relative coordinate.

Prototype
```
void WinWindowToDisplayPt (Coord *extentX,
Coord *extentY)
```

Parameters

`<-> extentX` Pointer to x coordinate to convert.

`<-> extentY` Pointer to y coordinate to convert.

Result Returns nothing.

Comments The coordinate passed is assumed to be relative to the draw window.

See Also `WinDisplayToWindowPt`

56

Miscellaneous System Functions

This chapter describes miscellaneous system functions. The functions in this chapter are declared in the header files `Crc.h`, `DLServer.h`, `IntlMgr.h`, and `Localize.h`.

Crc16CalcBlock

Purpose Calculate the 16-bit CRC of a data block using the table lookup method.

Prototype

```
UInt16 Crc16CalcBlock (const void *bufP,
UInt16 count, UInt16 crc)
```

Parameters

`bufP`	Pointer to the data buffer.
`count`	Number of bytes in the buffer.
`crc`	Seed crc value.

Result A 16-bit CRC for the data buffer.

DlkGetSyncInfo

Purpose Get the sync info managed by Desktop Link. This function is often used to obtain the user name on the handheld.

Prototype

```
Err DlkGetSyncInfo (UInt32 *succSyncDateP,
UInt32 *lastSyncDateP,
DlkSyncStateType *syncStateP, Char *nameBufP,
Char *logBufP, Int32 *logLenP)
```

Parameters

`<- succSyncDateP`
Pointer to the location where the date of the last successful sync is stored. Supply `NULL` for this parameter if this date isn't needed.

`<- lastSyncDateP`
Pointer to the location where the date of the last sync, successful or otherwise, is stored. Supply `NULL` for this parameter if this date isn't needed.

`<- syncStateP` Pointer to a `DlkSyncStateType` enum into which the state of the last sync is stored. Supply `NULL` for this parameter if the state information isn't needed. See the Comments, below, for a description of this enum.

`<- nameBufP` Pointer to a string buffer into which the null-terminated handheld user name is stored. This string buffer must have been preallocated to be at least `dlkUserNameBufSize` bytes in length. Supply `NULL` for this parameter if the user name isn't needed.

`<- logBufP` Pointer to a string buffer into which the sync log text, null-terminated, is stored. Supply `NULL` for this parameter if the log text isn't needed. If you supply a valid pointer for this parameter, you must specify the preallocated buffer length using the `logLenP` parameter; the returned log text will be truncated, if necessary, to fit within the buffer.

`<-> logLenP` Pointer to the log buffer size. If `logBufP` is not `NULL`, on entry you must set this value to the size of the `logBufP` buffer. When this function returns, this value indicates the actual length of the log text, not counting the null terminator.

Result Returns `errNone` if no error, or `dlkErrMemory` if the Desktop Link preferences resource couldn't be locked.

Comments The state information returned through `syncStateP` has one of the values defined by the `DlkSyncStateType` enum:

```
typedef enum DlkSyncStateType {
  dlkSyncStateNeverSynced = 0,
  dlkSyncStateInProgress,
  dlkSyncStateLostConnection,
  dlkSyncStateLocalCan,
  dlkSyncStateRemoteCan,
  dlkSyncStateLowMemoryOnTD,
  dlkSyncStateAborted,
  dlkSyncStateCompleted,
  dlkSyncStateIncompatibleProducts,
  dlkSyncStateNPOD
} DlkSyncStateType;
```

Value	Description
`dlkSyncStateNeverSynced`	The handheld has never been synced.
`dlkSyncStateInProgress`	A sync is currently in progress.
`dlkSyncStateLostConnection`	The connection was lost during sync.
`dlkSyncStateLocalCan`	Sync was cancelled by the user on the handheld.
`dlkSyncStateRemoteCan`	Sync was cancelled by the user from the desktop.

Value	Description
dlkSyncStateLowMemoryOnTD	Sync ended due to a low memory condition on the handheld.
dlkSyncStateAborted	Sync was aborted for some other reason.
dlkSyncStateCompleted	Sync completed normally.
dlkSyncStateIncompatibleProducts	Sync ended because the desktop HotSync product is incompatible with this version of the handheld HotSync.
dlkSyncStateNPOD	The sync could not take place because the handheld has a 4.0-style password but the desktop hasn't yet been updated to a compatible version.

Example This function is most often used to obtain the handheld user name. The following code excerpt shows how to do this (for clarity, error-checking has been omitted):

```
MemHandle nameH;
char *nameP;

// Allocate a buffer for the user name
nameH = MemHandleNew(dlkUserNameBufSize);
nameP = MemHandleLock(nameH);

// Obtain the user's name
DlkGetSyncInfo(NULL, NULL, NULL, nameP, NULL, NULL);

// ... Do something with the user name here ...

// Now that we're done with the user name, free the buffer
MemPtrUnlock(nameP);
```

Compatibility The `dlkSyncStateIncompatibleProducts` enum value was added in Palm OS 3.0. The `dlkSyncStateNPOD` enum value was added in Palm OS 4.0.

IntlGetRoutineAddress

Purpose Return the address of an international manager or text manager function.

Prototype

```
void *IntlGetRoutineAddress
(IntlSelector inSelector)
```

Parameters

`-> inSelector` One of the routine selectors defined in `IntlMgr.h`.

Result Returns the address of the corresponding function. Returns `NULL` if an invalid routine selector is passed.

Comments Use this function for performance reasons. It returns the address of an international manager or text manager function. You can then use this address to call the function without having to go through the international manager's trap dispatch table. This function is mostly useful for optimizing the performance of text manager routines that are called in a tight loop.

You might also use this function to check for the presence of newer international manager and text manager functions. If the result is `NULL`, the function is not implemented on this device.

Compatibility Implemented only if International Feature Set is present.

See Also `IntlSetRoutineAddress`, `SysGetTrapAddress`



IntlSetRoutineAddress

Purpose

Set the address of the function corresponding to an international manager or text manager function.

Prototype

```
Err IntlSetRoutineAddress (IntlSelector iSelector,
void *iProcPtr)
```

Parameters

-> `iSelector` One of the routine selectors defined in `IntlMgr.h`.

-> `iProcPtr` Pointer to a function that the routine identified by `iSelector` should point to.

Result

Returns `errNone` if no error, or `intlErrInvalidSelector` if `iSelector` does not refer to a valid international manager or text manager routine.

Comments

This function is useful for patching an international or text manager function. Normally only a locale module would need to patch one of these functions.

WARNING! If your application patches an international manager function using this function, you **must** remove the patch before your application exits. Do **not** use this mechanism to permanently patch international manager functions as it may cause unpredictable results for the system and other applications.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`IntlGetRoutineAddress`, `SysSetTrapAddress`

LocGetNumberSeparators

Purpose Get localized number separators.

Prototype

```
void LocGetNumberSeparators
(NumberFormatType numberFormat,
Char *thousandSeparator, Char *decimalSeparator)
```

Parameters

-> `numberFormat` The format to use (see `NumberFormatType`).

<- `thousandSeparator`
The character used for the thousands separator.

<- `decimalSeparator`
The character used for the decimal separator.

Result Returns nothing.

Comments The format to use is stored in the system preferences. You can obtain it by passing `prefNumberFormat` to `PrefGetPreference`.

Compatibility Implemented only if 2.0 New Feature Set is present.

See Also `StrLocalizeNumber`, `StrDelocalizeNumber`, "Localized Applications" in the *Palm OS Programmer's Companion*, vol. I



Part III: Communications

57
Connection Manager

The Connection Manager allows other applications to access, add, and delete connection profiles contained in the Connection panel.

This chapter provides reference material for the Connection Manager API declared in the header file `ConnectionMgr.h`:

- Connection Manager Constants
- Connection Manager Functions

For more information on the Connection Manager, see the chapter "Serial Communication" on page 89 in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

Connection Manager Data Types



CncProfileID

The `CncProfileID` type uniquely identifies a connection profile within the Connection Manager profile database. You pass this ID as a parameter to several of the Connection Manager functions. You can obtain a connection's profile ID using `CncProfileGetIDFromName` or `CncProfileGetIDFromIndex`.

```
typedef UInt32 CncProfileID
```

Compatibility Defined only if Connection Manager Feature Set is present.

Connection Manager Constants

Profile Parameter Constants

The Connection Manager defines the following constants to represent individual parameters in the preinstalled connection profiles. Not all parameters work for all types of profiles.

Parameter Name	Parameter Type	Description
`kCncParamBaud`	`UInt32`	The baud rate to use for the connection.
`kCncParamBluetoothDeviceAddr`	buffer	The 48-bit address (`BD_ADDR`) of the device that the handheld is connected to through the Bluetooth port. This parameter is only valid if `kCncParamPort` specifies the Bluetooth port.
`kCncParamBluetoothDeviceName`	string	The name of the device that the handheld is connected to through the Bluetooth port. The device name uses the UTF-8 character encoding. This parameter is only valid if `kCncParamPort` specifies the Bluetooth port.
`kCncParamCountryIndex`	`UInt16`	The index into the list of strings returned by `kCncParamIntlModemCountryStringList` and `kCncParamIntlModemResetStringList` that provides the name of the country and the reset commands for this profile.

Parameter Name	Parameter Type	Description
`kCncParamDeviceKind`	`UInt16`	The type of connection being made (general serial connection, connection to a modem, connection to a phone, and so on). This value is one of the Device Kind Constants.
`kCncParamDialingMode`	`UInt8`	For modem profiles, the dialing mode. 1 for Pulse dialing, 0 for TouchTone.
`kCncParamFlowControl`	`UInt16`	The flow control setting (hardware handshaking). 0 indicates that the flow control setting is enabled and disabled depending on the baud rate. If you choose this setting, flow control is on if the baud rate is greater than 2400. 1 indicates that flow control is always on, and 2 indicates that it is always off. This parameter does not apply to phone profiles.
`kCncParamInitString`	string	For modem and phone profiles, a null-terminated string containing the commands sent to initialize the connection. The Connection panel removes any carriage returns or other control characters from the initialization string before saving it to the profile.

Parameter Name	Parameter Type	Description
`kCncParamIntlModemCountryStringList`	buffer	For modem profiles, a data buffer containing a list of possible countries. This list contains the countries in which the modem can operate and is shown in the Connection panel Details form for this profile. The selected country controls the dialing prefix and the modem reset string. The `kCncParamCountryIndex` parameter specifies the country selected from this list. The data buffer contains a packed block of null-terminated strings, each containing a country name. The first 16 bits of the data buffer (a `UInt16` value) tells how many strings are contained in the block. You can use `SysFormPointerArrayToStrings` to convert the data buffer into an array of strings. Because this parameter value has a variable size, you must first call `CncProfileSettingGet` with a `NULL` data pointer to obtain the correct size.
`kCncParamIntlModemResetStringList`	buffer	For modem profiles, a data buffer containing possible reset strings. The `kCncParamCountryIndex` parameter specifies which reset string from this list is to be used. The actual string used is itself stored in the `kCncParamResetString` parameter.

Parameter Name	Parameter Type	Description
		The data buffer contains a packed block of null-terminated strings, each containing a reset string. The first 16 bits of the data buffer (a `UInt16` value) tells how many strings are contained in the block. You can use `SysFormPointerArrayToStrings` to convert the data buffer into an array of strings. Because this parameter value has a variable size, you must first call `CncProfileSettingGet` with a `NULL` data pointer to obtain the correct size.
`kCncParamInvisible`	system flag	If `true`, the profile is hidden from the user. If `false`, the profile is visible. This parameter is not currently used.
`kCncParamLocked`	system flag	If `true`, the profile is locked so that the user cannot edit it. If `false`, the profile can be edited. This parameter can be set by profiles, such as phone profiles, created by third party utilities.
`kCncParamName`	string	The name of the profile.
`kCncParamNoDetails`	system flag	If `true`, the profile details should not be displayed. If `false`, they can be displayed. The profile details are the parameters that appear in the Details form of the Connection panel.

Parameter Name	Parameter Type	Description
`kCncParamNonEditable`	system flag	If `true`, the Connection panel's Edit form should be suppressed for this profile. If `false`, the Edit form can be displayed. This parameter differs from `kCncParamLocked` and `kCncParamReadOnly` in that it causes an alert to be displayed if the user taps the Edit button. The other parameters allow the Edit form to be displayed but do not allow changes to be made. Also, a non-editable profile can be deleted, but a read-only or locked profile cannot.
`kCncParamPort`	`UInt32`	The logical, physical, or virtual port identifier. See "Port Constants" on page 1351 in the "Serial Manager" chapter for more information.
`kCncParam_PSDCreator`	`UInt32`	For phone profiles, the creator ID of the phone driver.
`kCncParam_PSDName`	string	For phone profiles, the name of the phone driver.
`kCncParam_PSDParameterBuffer`	buffer	For phone profiles, a data buffer containing any necessary data that the phone driver needs to store. This parameter typically holds data that is set using the Details form of the Connection panel.
`kCncParam_PSDType`	`UInt32`	For phone profiles, the database type for the phone driver.

Parameter Name	Parameter Type	Description
`kCncParamReadOnly`	system flag	If `true`, the profile is read-only and cannot be edited. If `false`, the profile can be edited. This parameter is only intended to be used by profiles pre-installed in the Palm OS.
`kCncParamReceiveTimeOut`	`UInt32`	For phone profiles, the number of milliseconds to wait for a response from the phone. This time-out value is used by telephony functions that don't need to access the network (for example, the function `TelNwkGetSelectedNetwork`).
`kCncParamResetString`	string	For modem and phone profiles, the reset string. For modem profiles, this is one of the strings in `kCncParamIntlModemResetStringList`.
`kCncParamSerialPortFlags`	`UInt32`	For phone profiles, bit flags that correspond to various serial port hardware settings. See "Serial Settings Constants" on page 1354 for more information.
`kCncParamSystemFlags`	`UInt32`	A bit flag representing all system flags. Currently, only bits 0 through 4 have a meaning. These correspond to the read-only bit, the invisible bit, the noneditable bit, the no details bit, and the locked bit, respectively.
`kCncParamTimeOut`	`UInt32`	The amount of time in milliseconds to wait for a response when CTS is unasserted and hardware flow control is on.

Parameter Name	Parameter Type	Description
kCncParamTTCreator	UInt32	For phone profiles, the creator ID of the telephony task used by the phone driver.
kCncParamTTType	UInt32	For phone profiles, the database type for the telephony task used by the phone driver.
kCncParamVersion	UInt8	The version of the Connection Manager API under which this profile was created. The current version number is kCncProfileVersion.
kCncParamVolume	UInt16	For modem profiles, the modem volume.

Compatibility Defined only if Connection Manager Feature Set is present.

Profile Parameter Size Constants

The size constants specify the size of each of the predefined parameters described in "Profile Parameter Constants." The following table lists the parameter name, the size of its value, and the size constant that gives this size. These size constants are suitable for passing to CncProfileSettingGet and CncProfileSettingSet.

Profile Parameter Name	Size	Size Constant
kCncParamBaud	32	kCncParamBaudSize
kCncParamBluetoothDeviceAddr	8	kCncParamBluetoothDeviceAddrSize
kCncParamBluetoothDeviceName	249	kCncParamBluetoothDeviceNameMaxSize
kCncParamCountryIndex	16	kCncParamCountryIndexSize

Profile Parameter Name	Size	Size Constant
kCncParamDeviceKind	16	kCncParamDeviceKindSize
kCncParamDialingMode	8	kCncParamDialingModeSize
kCncParamFlowControl	16	kCncParamFlowControlSize
kCncParamInitString	81	kCncParamInitStringMaxSize
	81	kCncProfileUsualInitStringSize
kCncParamInvisible	8	kCncParamInvisibleSize
kCncParamLocked	8	kCncParamLockedSize
kCncParamName	22	kCncParamNameMaxSize
	22	kCncProfileNameSize
kCncParamNoDetails	8	kCncParamNoDetailsSize
kCncParamNonEditable	8	kCncParamNonEditableSize
kCncParamPort	32	kCncParamPortSize
kCncParam_PSDCreator	32	kCncParam_PSDCreatorSize
kCncParam_PSDName	32	kCncParam_PSDNameSize
kCncParam_PSDType	32	kCncParam_PSDTypeSize
kCncParamReadOnly	8	kCncParamReadOnlySize
kCncParamReceiveTimeOut	32	kCncParamReceiveTimeOutSize
kCncParamResetString	81	kCncParamResetStringMaxSize
	8	kCncProfileClassicResetStringSize
	81	kCncProfileUsualResetStringSize
kCncParamSystemFlags	32	kCncParamSystemFlagsSize
kCncParamTimeOut	32	kCncParamTimeOutSize

Profile Parameter Name	Size	Size Constant
kCncParamTTCreator	32	kCncParamTTCreatorSize
kCncParamTTType	32	kCncParamTTTypeSize
kCncParamVersion	8	kCncParamVersionSize
kCncParamVolume	16	kCncParamVolumeSize

Compatibility Defined only if Connection Manager Feature Set is present.

Device Kind Constants

The device kind constants specify the type of connection being made. You specify the type of connection by defining a value for the kCncParamDeviceKind parameter using CncProfileSettingSet.

Constant	Value	Description
kCncDeviceKindSerial	0	The connection is through the serial port.
kCncDeviceKindModem	1	The connection is to a modem.
kCncDeviceKindPhone	2	The connection is to a phone.
kCncDeviceKindLocalNetwork	3	The connection is to a LAN.

Compatibility Defined only if Connection Manager Feature Set is present.

Profile Parameter Types

The parameter type constants specify the type of value stored for a parameter in a connection profile. If you define a new parameter using CncDefineParamID, you must use one of these constants to specify the type of data the parameter stores. The macro CncGetParamType can return this information for any parameter in the profile.

Constant	Value	Description
kCncParamSystemFlag	0x00	A system flag parameter. The Connection Manager can store up to 32 system flags. Flags are stored in a single bit and returned as a UInt8 value. The entire system flags word can be returned if you pass kCncParamSystemFlags to CncProfileSettingGet.
kCncParamUInt8	0x01	A UInt8 parameter.
kCncParamUInt16	0x02	A UInt16 parameter.
kCncParamUInt32	0x03	A UInt32 parameter.
kCncParamString	0x04	A string parameter.
kCncParamBuffer	0x05	A generic block of data.

Compatibility Defined only if Connection Manager Feature Set is present.

Connection Manager Functions

CncAddProfile

Purpose Adds a profile to the Connection Manager.

Prototype

```
Err CncAddProfile (Char *name, UInt32 port,
UInt32 baud, UInt16 volume, UInt16 handShake,
const Char *initString, const Char *resetString,
Boolean isModem, Boolean isPulse)
```

Parameters

<-> name — Pointer to the profile name to be added. If the name is already taken in the Connection panel then a duplication number is appended to it. The name added is returned here.

-> `port`	The port identification used by the profile. See "Specifying the Port" on page 100 of the *Palm OS Programmer's Companion*, vol. II, *Communications* for more information.
-> `baud`	The baud rate used by the profile.
-> `volume`	The volume setting for the device (for Modem only).
-> `handShake`	Flow control setting (hardware handshaking). 0 specifies automatic (on at speeds > 2400 baud), 1 specifies always on, and 2 specifies always off.
-> `initString`	Pointer to the initialization string used by a modem (for Modem only).
-> `resetString`	Pointer to the reset string used by a modem (for Modem only).
-> `isModem`	`true` if Modem, `false` if Direct.
-> `isPulse`	true if Pulse dial, `false` if TouchTone.

Result

`errNone`	No error.
`cncErrAddProfileFailed`	The add operation failed.
`cncErrProfileListFull`	The add operation failed because the profile list is full.
`cncErrConDBNotFound`	The connection database is missing.

Comments

All profiles within the Connection Manager must have a unique name. The Connection Manager tries to append a duplication number to the end of the name if you specify a name that is already taken.

There is a maximum limit to the number of profiles that can be maintained by the Connection Manager. If the limit is passed, an error is returned and that profile will not be added.

Profiles that do not need certain fields may pass `NULL` in the place of a value.

Compatibility Implemented only if New Serial Manager Feature Set is present.

If Connection Manager Feature Set is present, use `CncProfileCreate` instead of using this function. `CncAddProfile` is still supported for backward compatibility. In Palm OS 4.0 and higher, the maximum number of profiles that can be defined has greatly increased.

Example

```
AddMyProfile()
{
   Char *myConNameP;
   Err err;

   myConNameP = MemPtrNew(cncProfileNameSize);

   StrCopy(myConNameP, "Foobar");

   err = CncAddProfile(myConNameP, 'u328', 57600, 0, 0,
      "AT&FX4", 0,  true, false);

   MemPtrFree(myConNameP);
}
```



New CncDefineParamID

Purpose Macro that creates and returns a parameter ID.

Prototype

```
CncDefineParamID (parameterRange, parameterType,
parameterID)
```

Parameters

-> `parameterRange`

Use `kCncParamThirdPartiesRange` to specify that this parameter is not defined by the OS.

-> `parameterType`

The type of value stored for the parameter. See "Profile Parameter Types" for a list of possible values.

-> `parameterID`	A unique value between 0 to 1023. The value must be unique within the profile for which you are defining the parameter. If you are using a `parameterType` of `kCncParamSystemFlag`, specify a value from 0 to 31 to identify which of the system flag bit is to be set.

Result Returns the parameter ID as a `UInt16` value.

Comments You use this macro only if you are defining your own connection profile and have a parameter that you need to define within that profile. The parameter ID immediately precedes its parameter value in the Connection Manager profile database. Because of how the database is formatted, the parameter ID must tell the Connection Manager how to interpret the next series of bytes. For this reason, the high order bits of the parameter ID include information about the type of value and whether the value is defined by the system or a third party.

Compatibility Parameter IDs of this format are only used if 4.0 New Feature Set is present.

See Also `CncProfileSettingSet`, `CncProfileSettingGet`

CncDeleteProfile

Purpose Removes a profile from the Connection Manager.

Prototype

```
Err CncDeleteProfile (const Char *name)
```

Parameters

-> `name`	Pointer to the name of the profile to be deleted.

Result

`errNone`	No error.
`cncErrProfileReadOnly`	The profile could not be deleted because it is read only.

`cncErrProfileNotFound`
The profile could not be found.

`cncErrConDBNotFound`
The connection database is missing.

Comments The profiles that come preinstalled on the unit are read only and cannot be deleted.

Compatibility Implemented only if New Serial Manager Feature Set is present.

If Connection Manager Feature Set is present, use `CncProfileDelete` instead of using this function.



New CncGetParamType

Purpose Macro that returns the parameter type portion of the parameter ID.

Prototype
```
CncGetParamType (parameterID)
```

Parameters -> `parameterID` A `UInt16` that contains the parameter ID.

Result Returns a `UInt16` value where bits 11 through 14 contain one of the values in "Profile Parameter Types" and the other bits are clear.

Compatibility Parameter IDs of this format are only used if 4.0 New Feature Set is present.

See Also `CncProfileSettingSet`, `CncProfileSettingGet`

CncGetProfileInfo

Purpose Returns the settings for a profile.

Prototype

```
Err CncGetProfileInfo (Char *name, UInt32 *port,
UInt32 *baud, UInt16 *volume, UInt16 *handShake,
Char *initString, Char *resetString,
Boolean *isModem, Boolean * isPulse)
```

Parameters

-> `name`	Pointer to the name of the profile to be returned. Passing in `NULL` causes this function to return the settings for the profile currently selected in the Connection panel.
<- `port`	Pointer to the port identifier that the profile uses.
<- `baud`	Pointer to the baud rate that has been set for this profile.
<- `volume`	Pointer to the volume of the device (applies only to modems).
<- `handShake`	Pointer to the flow control setting (hardware handshaking). 0 indicates automatic (on at speeds > 2400 baud), 1 indicates always on, and 2 indicates always off.
<- `initString`	Pointer to the initialization string for the device (applies only to modems).
<- `resetString`	Pointer to the reset string for the device (applies only to modems).
<- `isModem`	Pointer to a Boolean value: `true` for Modem, `false` for Direct.
<- `isPulse`	Pointer to a Boolean value: `true` for Pulse dial, `false` for TouchTone.

Result

`errNone`	No error.
`cncErrGetProfileFailed`	The get profile operation failed. The profile may or may not be there.

`cncErrProfileNotFound`
The profile could not be found

`cncErrConDBNotFound`
The connection database is missing.

Comments One or more of the parameters may be set to `NULL` if that information is not desired.

Compatibility Implemented only if New Serial Manager Feature Set is present.

If Connection Manager Feature Set is present, use `CncProfileSettingGet` with one of the "Profile Parameter Constants" instead of using this function.

Example

```
{
     UInt32 portID, baud;
     UInt16 openPort;
     // get port id
     err = CncGetProfileInfo("Direct Serial", &portID, &baud,
        0, 0, 0, 0, 0, 0);
     if(!err)
     { // open the port
        SrmOpen(portID, baud, &openPort);
     }
}
```

CncGetProfileList

Purpose Returns a list of available profiles that are available through the Connection Manager.

Prototype

```
Err CncGetProfileList (Char ***nameListPPP,
UInt16 *countP)
```

Parameters

<- `nameListPPP`	Pointer to a pointer to a list of profile names.
<- `countP`	Pointer to the number of profile names.

Result

`errNone` No error.

`cncErrGetProfileListFailed`
The profile list could not be found.

`cncErrConDBNotFound`
The connection database is missing.

Comments

Allocation of the list is handled by the Connection Manager; deallocation is the responsibility of the calling application. Appended to the end of the list will be "-Current-", which represents the profile currently selected in the Connection panel.

Compatibility

Implemented only if New Serial Manager Feature Set is present.

Example

```
//Declared globally
Char ** globalProfileList;
ListType *listP;
UInt16 globalProfileCount;

void SetConnectionList()
{
   //Get the list from the Connection Manager
   err = CncGetProfileList(&globalProfileList,
      &globalProfileCount);
   //Set the UI list
   LstSetListChoices(listP, globalProfileList,
      globalProfileCount);
}

void StopApplication()
{
   UInt16 i;

   //Deallocate the connection list
   For(i = 0; i < globalProfileCount;  i++)
      MemPtrFree(globalProfileList[ i ]);
   MemPtrFree(globalProfileList);
}
```



New CncGetSystemFlagBitnum

Purpose Macro that returns the number uniquely identifying a system flag parameter.

Prototype
```
CncGetSystemFlagBitnum (parameterID)
```

Parameters -> `parameterID` The `UInt16` containing the parameter ID.

Result Returns the ID of the system flag parameter, which is a value from 0 to 31.

Compatibility Parameter IDs of this format are only used if Connection Manager Feature Set is present.

See Also `CncProfileSettingSet`, `CncProfileSettingGet`



New CncGetTrueParamID

Purpose Macro that returns the portion of the parameter ID that uniquely identifies the parameter.

Prototype
```
CncGetTrueParamID (parameterID)
```

Parameters -> `parameterID` A `UInt16` containing the parameter ID.

Result Returns a `UInt16` containing just the parameter ID. The high-order bits, which specify the parameter type and address space, are clear.

Compatibility Parameter IDs of this format are only used if Connection Manager Feature Set is present.

See Also `CncProfileSettingSet`, `CncProfileSettingGet`



New CncIsFixedLengthParamType

Purpose Macro that specifies whether the parameter value is fixed length or variable length.

Prototype
```
CncIsFixedLengthParamType (parameterID)
```

Parameters -> `parameterID` A `UInt16` containing the parameter ID.

Result Returns `true` if the parameter is a fixed length parameter type such as `UInt32`, or `false` if it is a variable length type.

Compatibility Parameter IDs of this format are only used if Connection Manager Feature Set is present.

See Also `CncProfileSettingSet`, `CncProfileSettingGet`



New CncIsSystemFlags

Purpose Macro that returns whether the parameter value is a system flag.

Prototype
```
CncIsSystemFlags (parameterID)
```

Parameters -> `parameterID` The `UInt16` containing the parameter ID.

Result Returns `true` if the parameter type is a system flag. Returns `false` otherwise.

Compatibility Parameter IDs of this format are only used if Connection Manager Feature Set is present.

See Also `CncProfileSettingSet`, `CncProfileSettingGet`



New CncIsSystemRange

Purpose Macro that specifies whether the parameter is in the system range or in the third-party range.

Prototype
```
CncIsSystemRange (parameterID)
```

Parameters -> `parameterID` A `UInt16` containing the parameter ID.

Result Returns `true` if the parameter ID is defined by Palm OS, or `false` if it is defined by a third party.

Compatibility Parameter IDs of this format are only used if Connection Manager Feature Set is present.

See Also `CncProfileSettingSet`, `CncProfileSettingGet`



New CncIsThirdPartiesRange

Purpose Macro that specifies whether the parameter is defined by a third party.

Prototype
```
CncIsThirdPartiesRange (parameterID)
```

Parameters -> `parameterID` A `UInt16` containing the parameter ID.

Result Returns `true` if the parameter is a third-party parameter, or `false` if it is a system parameter.

Compatibility Parameter IDs of this format are only used if Connection Manager Feature Set is present.

See Also `CncProfileSettingSet`, `CncProfileSettingGet`



New CncIsVariableLengthParamType

Purpose Macro that returns whether the parameter value is a variable-length type.

Prototype
```
CncIsVariableLengthParamType (parameterID)
```

Parameters -> `parameterID` A `UInt16` containing the parameter ID.

Result Returns `true` if the parameter is a variable-length string or a buffer or `false` if it holds a fixed-length type such as an integer.

Compatibility Parameter IDs of this format are only used if Connection Manager Feature Set is present.

See Also `CncProfileSettingSet`, `CncProfileSettingGet`



New CncProfileCloseDB

Purpose Closes the Connection Manager profile database.

Prototype
```
Err CncProfileCloseDB (void)
```

Parameters None.

Result

`errNone` No error.

`kCncErrDBAccessFailed` The database could not be closed or this is a reference counting error.

Comments Use `CncProfileOpenDB` and `CncProfileCloseDB` as an optimization if you make several Connection Manager calls in succession. All Connection Manager calls open the profile database when they begin and close the database when they are finished. The

Connection Manager maintains a reference count that tells it whether the database is open. If you call `CncProfileOpenDB` before making another Connection Manager call, the next call does not open or close the database. This saves your application the overhead of opening and closing the database each time a call is made.

Compatibility Implemented only if Connection Manager Feature Set is present.



New CncProfileCount

Purpose Returns the number of connection profiles currently defined in the Connection Manager profile database.

Prototype

```
Err CncProfileCount (UInt16 *profilesCountP)
```

Parameters

<- `profilesCountP`
The number of profiles.

Result

`errNone` No error.

`kCncErrDBAccessFailed`
The profile database could not be opened.

Compatibility Implemented only if Connection Manager Feature Set is present.

See Also `CncGetProfileList`



New CncProfileCreate

Purpose Adds a profile record to the Connection Manager profile database.

Prototype

```
Err CncProfileCreate (CncProfileID *profileIdP)
```

Parameters

<- `profileIdP` Upon return, the unique ID of the new profile.

Result

`errNone` No error.

`kCncErrDBAccessFailed`
The profile database could not be opened.

a Data Manager error
The new record could not be created.

Comments This function creates a new empty record in the Connection Manager profile database. To populate the profile, use `CncProfileSettingSet` to set parameter values, including the profile name. Use `CncDefineParamID` if you need to store information unique to your profile.

Compatibility Implemented only if Connection Manager Feature Set is present.

See Also `CncAddProfile`, `CncProfileDelete`



New CncProfileDelete

Purpose Deletes a profile.

Prototype

```
Err CncProfileDelete (CncProfileID profileId)
```

Parameters

`profileId` The ID of the profile to delete.

Result

`errNone` No error.

`kCncErrDBAccessFailed`
The profile database could not be opened, or the record could not be deleted.

`kCncErrProfileParamNotFound`
The database does not contain a profile with the specified ID.

Comments The profiles that come preinstalled on the unit are read only and cannot be deleted.

Compatibility Implemented only if Connection Manager Feature Set is present.

See Also CncDeleteProfile, CncProfileCreate



CncProfileGetCurrent

Purpose Returns the ID of the currently selected profile in the Connection panel.

Prototype

```
Err CncProfileGetCurrent
(CncProfileID *profileIdP)
```

Parameters

<- profileIdP	The ID of the current profile.

Result

errNone	No error.
kCncErrDBAccessFailed	The profile database could not be opened.

Compatibility Implemented only if Connection Manager Feature Set is present.

See Also CncProfileGetIDFromIndex, CncProfileGetIDFromName, CncProfileGetIndex, CncProfileSetCurrent



CncProfileGetIDFromIndex

Purpose Returns the profile ID given its index into the Connection Manager profile database.

Prototype

```
Err CncProfileGetIDFromIndex (UInt16 index,
CncProfileID *profileIdP)
```

Parameters

-> index	The index of the Connection Manager profile.
<- profileIdP	The ID of the Connection Manager profile.

Result

`errNone`	No error.
`kCncErrDBAccessFailed`	The profile database could not be opened.
`kCncErrProfileParamNotFound`	No profile at that index.

Compatibility Implemented only if Connection Manager Feature Set is present.

See Also `CncProfileGetIDFromName`, `CncProfileGetCurrent`, `CncProfileGetIndex`



New CncProfileGetIDFromName

Purpose Returns the profile ID given its name.

Prototype

```
Err CncProfileGetIDFromName
(const Char *profileNameP,
CncProfileID *profileIdP)
```

Parameters

-> `profileNameP`	The name of the profile. The name is displayed in a pop-up list in the Connection panel. If you pass the string "-Current-", this function returns the ID of the current profile.
<- `profileIdP`	The profile ID.

Result

`errNone`	No error.
`kCncErrDBAccessFailed`	The profile database could not be opened.
`kCncErrProfileParamNotFound`	No profile with the specified name.

Compatibility Implemented only if Connection Manager Feature Set is present.

See Also `CncProfileGetCurrent`, `CncProfileGetIDFromIndex`, `CncProfileGetIndex`



New CncProfileGetIndex

Purpose Returns the index of the profile given its ID.

Prototype
```
Err CncProfileGetIndex (CncProfileID profileId,
UInt16 *indexP)
```

Parameters

-> `profileId`	The profile ID.
<- `indexP`	The index of the profile's record in the Connection Manager profile database.

Result

`errNone`	No error.
`kCncErrDBAccessFailed`	The profile database could not be opened.
`kCncErrProfileParamNotFound`	No profile with the specified ID.

Compatibility Implemented only if Connection Manager Feature Set is present.

See Also `CncProfileGetIDFromIndex`



New CncProfileOpenDB

Purpose Opens the Connection Manager profile database.

Prototype
```
Err CncProfileOpenDB (void)
```

Parameters None

Result

`errNone`	No error.
`kCncErrDBAccessFailed`	The profile database could not be opened.

Comments Use `CncProfileOpenDB` and `CncProfileCloseDB` as an optimization if you make several Connection Manager calls in succession. All Connection Manager calls open the profile database when they begin and close the database when they are finished. The Connection Manager maintains a reference count that tells it whether the database is open. If you call `CncProfileOpenDB` before making another Connection Manager call, the next call does not open or close the database. This saves your application the overhead of opening and closing the database each time a call is made.

The Connection Manager profile database is created if it does not exist.

Compatibility Implemented only if Connection Manager Feature Set is present.



New CncProfileSetCurrent

Purpose Sets the current profile.

Prototype
```
Err CncProfileSetCurrent (CncProfileID profileId)
```

Parameters

-> `profileId` The ID of the profile to be made current.

Result

`errNone` No error.

`kCncErrDBAccessFailed` The profile database could not be opened.

Comments The current profile is the profile that is used for the next network connection attempt.

Compatibility Implemented only if Connection Manager Feature Set is present.

See Also `CncProfileGetCurrent`



New CncProfileSettingGet

Purpose

Obtains a value stored in one of the Connection Manager profiles.

Prototype

```
Err CncProfileSettingGet (CncProfileID profileId,
UInt16 paramId, void *paramBufferP,
UInt16 *ioParamSizeP)
```

Parameters

-> `profileId` The ID of the profile from which to obtain a parameter value.

-> `paramId` The ID of the parameter to obtain. See "Profile Parameter Constants" for a list of the parameters used in the profiles that come preinstalled on the device.

<- `paramBufferP` A pointer to a buffer into which to write the parameter value. If the parameter stores a variable-sized value, you can determine the necessary size by passing `NULL` for `paramBufferP`. Upon return, `paramSize` contains the required size.

<-> `ioParamSizeP`
On input, a pointer to the size of the buffer into which to write the parameter. On output, points to the number of bytes written to `paramBufferP`.

Result

`errNone` No error.

`kCncErrDBAccessFailed`
The profile database could not be opened.

`kCncErrProfileGetParamFailed`
The Connection Manager failed to obtain the value of the parameter.

`kCncErrProfileBadSystemFlagBitnum`
An attempt was made to obtain the value of a system flag that is undefined.

`kCncErrProfileBadParamSize`
The `paramBufferP` buffer is too small. `ioParamSizeP` contains the correct size for the buffer.

`kCncErrProfileParamNotFound`
The specified parameter is not defined in the profile.

Compatibility Implemented only if Connection Manager Feature Set is present.

See Also `CncProfileSettingSet`, `CncGetProfileInfo`



New CncProfileSettingSet

Purpose Sets a parameter value in the specified profile.

Prototype

```
Err CncProfileSettingSet (CncProfileID iProfileId,
UInt16 paramId, const void *paramBufferP,
UInt16 paramSize)
```

Parameters

`-> iProfileId` The ID of the profile.

`-> paramId` The ID of the parameter. See "Profile Parameter Constants" for a list of the parameters defined in the preinstalled connection profiles.

`-> paramBufferP`
A pointer to the value to set for this parameter.

`-> paramSize` The size of the buffer passed in `paramBufferP`. See "Profile Parameter Size Constants."

Result

`errNone` No error.

`kCncErrDBAccessFailed`
The profile database could not be opened.

`kCncErrProfileParamNotFound`
No profile with the specified ID.

`kCncErrProfileSetParamFailed`
: The Connection Manager failed to set the value of the parameter.

`kCncErrProfileBadParamSize`
: The `paramBufferP` buffer is too small. Most likely, the passed in size does not allow space for a string parameter's null terminator.

`kCncErrProfileParamNameHasChange`
: An attempt was made to set the profile name to a name that is already used. The Connection Manager appends a duplication number to the end of the name and returns this error. You should use `CncProfileSettingGet` to find out the new name.

Compatibility Implemented only if Connection Manager Feature Set is present.

See Also `CncDefineParamID`, `CncProfileSettingGet`

58

Exchange Manager

This chapter describes the Exchange Manager API declared in the header file `ExgMgr.h` and the Exchange Local Library API declared in the header file `ExgLocalLib.h`. It discusses the following topics:

- Exchange Manager Data Structures
- Exchange Manager Constants
- Exchange Manager Functions
- Application-Defined Functions

For more information on the Exchange Manager, see the chapter "Object Exchange" on page 1 of *Palm OS Programmer's Companion*, vol. II, *Communications*.

Exchange Manager Data Structures

ExgAskResultType

The `ExgAskResultType` enum defines possible values for the `result` field of the `sysAppLaunchCmdExgAskUser` launch code parameter block.

```
typedef enum {
  exgAskDialog,
  exgAskOk,
  exgAskCancel }
ExgAskResultType;
```

Value Descriptions

`exgAskDialog`	The Exchange Manager should display a dialog that prompts the user to confirm the receipt of data. See `ExgDoDialog`.
`exgAskOk`	Accept the data.
`exgAskCancel`	Reject the data.

ExgGoToType

The `ExgGoToType` structure defines the `goToParams` field of the `ExgSocketType` structure. Applications that want to be launched after the data is received place their creator IDs in the `goToCreator` field and define the `goToParams` field. The values in this structure are copied to the `sysAppLaunchCmdGoto` launch code's parameter block.

```
typedef struct {
  UInt16 dbCardNo;
  LocalID dbID;
  UInt16 recordNum;
  UInt32 uniqueID;
  UInt32 matchCustom;
} ExgGoToType;
```

Field Descriptions

`dbCardNo`	The card number of the database that contains the added record.
`dbID`	The local ID of the database that contains the added record.
`recordNum`	The index of the record that was added.
`uniqueID`	The unique ID of the record that was added. This field is not used.
`matchCustom`	Application-specific information.



ExgLocalSocketInfoType

The `ExgLocalSocketInfoType` structure identifies information specific to the Local Exchange Library. The `socketRef` field of the `ExgSocketType` structure is set to this structure when you send and receive data using the Local Exchange Library. The Local Exchange Library creates this structure if it does not already exist. You only need to create it if you want to supply non-default values for the `noAsk` or `previewInfoP` fields.

```
typedef struct {
  Boolean freeOnDisconnect;
  Boolean noAsk;
  ExgPreviewInfoType *previewInfoP;
  FileHand tempFileH;
  Err err;
  ExgLocalOpType op;
} ExgLocalSocketInfoType;
```

Field Descriptions

`freeOnDisconnect` Whether the structure is freed when the `ExgDisconnect` call is made. The default is `true`. In general, code that allocates a structure should be responsible for freeing that structure. Therefore, if you have allocated `ExgLocalSocketInfoType`, you should set this field to `false` and explicitly free the structure when you are finished with it.

`noAsk` Set to `true` to disable the display of the exchange dialog. If you want to, for example, create a vCalendar object and send it to the Datebook application in response to a user command, you probably want to set `noAsk` to `true` so that the user does not have to confirm the receipt of the data they just requested you to send.

`previewInfoP` A pointer to an `ExgPreviewInfoType` structure, used to display a preview of the data. If you wanted to simply use another application to help display data, you would create and initialize this structure.

`tempFileH` A temporary buffer that the Local Exchange Library uses. Do not set this field directly; the Local Exchange Library should set it.

`err` The error code returned from the Local Exchange Library. Do not set this field directly; the Local Exchange Library should set it.

`op` The operation in progress. Do not set this field directly. The Local Exchange Library sets this field to one of the following:

`exgLocalOpNone`	No operation in progress.
`exgLocalOpPut`	A send is in progress.
`exgLocalOpAccept`	A receive is in progress.
`exgLocalOpGet`	A get is in progress.
`exgLocalOpGetSender`	The library is receiving information from the sending application during a get operation.



ExgPreviewInfoType

The `ExgPreviewInfoType` structure provides information to the `ExgNotifyPreview` function. The `ExgNotifyPreview` function uses this information to have the application display a preview of the data to be received in the exchange dialog.

```
typedef struct {
  UInt16          version;
  ExgSocketType   *socketP;
  UInt16          op;
  Char            *string;
  UInt32          size;
  RectangleType   bounds;
  UInt16          types;
  Err             error;
} ExgPreviewInfoType;
```

Field Descriptions

-> `version` Set this field to 0 to specify version 0 of this structure.

-> `socketP` A pointer to the socket structure (see `ExgSocketType`). The `libraryRef` field must identify the exchange library from which preview data should be received, and the `target`, `type`, or `name` field should be defined as well.

-> `op` One of the following constants:

`exgPreviewDialog`
Display a modal dialog containing the preview. This constant is only used in situations where one application launches another to display data.

`exgPreviewDraw`
The preview is a graphic.

`exgPreviewLongString`
The preview is a long string.

`exgPreviewQuery`
Ask the application which preview operations it supports. The answer is returned in the `types` field. If the application does not support any preview modes, the `error` field contains `exgErrNotSupported`.

`exgPreviewShortString`
The preview is a short string.

<- `string`	A buffer into which the application places the string preview if `exgPreviewLongString` or `exgPreviewShortString` is specified.
-> `size`	The allocated size of the `string` field.
-> `bounds`	The bounds of the rectangle in which the application draws the graphic if `exgPreviewDraw` is specified.
<- `types`	Upon return from an `exgPreviewQuery` operation, a bit field identifying the types of previews the application supports.
<- `error`	The error code returned from the application. If `errNone`, the preview operation was successful.

Applications can define and use their own constants for the preview operation. Operations specific to an application are numbered starting at `exgPreviewFirstUser` and should be no greater than `exgPreviewLastUser`.

Compatibility This structure is only defined if 4.0 New Feature Set is present.

ExgSocketType

The `ExgSocketType` structure defines an Exchange Manager socket, which is passed to most Exchange Manager functions. The `ExgSocketPtr` type points to a `ExgSocketType` structure.

```
typedef struct ExgSocketType {
  UInt16 libraryRef;
  UInt32 socketRef;
  UInt32 target;
  UInt32 count;
  UInt32 length;
  UInt32 time;
  UInt32 appData;
  UInt32 goToCreator;
  ExgGoToType goToParams;
  UInt16 localMode:1;
  UInt16 packetMode:1;
  UInt16 noGoTo:1;
  UInt16 noStatus:1;
  UInt16 preview:1;
  UInt16 reserved:11;
  Char *description;
  Char *type;
  Char *name;
} ExgSocketType;

typedef ExgSocketType* ExgSocketPtr;
```

Note that when data is received, some of the fields in this structure may not have values. When you are sending data, it is recommended that you provide values for all of these fields, but you should not rely on receiving values for the fields marked optional.

Field Descriptions

`libraryRef` The exchange library in use. When an application or library receives a socket, this field is already assigned.

When sending data, applications may identify the exchange library they want to connect with by providing a URL in the `name` field. If so, they should use 0 for the `libraryRef` field. The Exchange Manager then determines which library corresponds to the URL and assigns the `libraryRef` field. See the Comments in the `ExgPut` function description for more information.

`socketRef` The connection identifier. This value is supplied by the exchange library when a connection is established. It contains any necessary library-specific data.

`target` The creator ID of the application that should receive the message.

`count` The number of objects in this connection, usually 1 (optional).

`length` The total byte count for all objects being sent (optional).

`time` The last modified time of object (optional).

`appData` Application-specific information (optional).

`goToCreator` The creator ID of the application to launch using the `sysAppLaunchCmdGoto` launch code after the item is received if `noGoTo` is 0. The value is assigned by the application that receives the object. See the Comments section in `ExgDisconnect` for more information.

`goToParams` If `goToCreator` is specified, then this field contains data that is copied into the launch code's parameter block. See `ExgGoToType`.

`localMode`	Set to 1 to exchange with local device only. A `localMode` of 1 is equivalent to specifying a URL with the `exgLocalPrefix`. Set to 0 to enable an exchange with a remote machine. The default is 0.
`packetMode`	Set to 1 to use connectionless packet mode (Ultra). The default is 0. Ultra mode is not currently supported.
`noGoTo`	Set to 1 to disable launching the application with `sysAppLaunchCmdGoto`. The default is 0.
`noStatus`	If `true`, the exchange library should not display a progress dialog. If `false`, the library can display a progress dialog. The default is `false`. The Exchange Manager sets and clears this bit at various times while data is received. Applications may also want to set this bit if they use the Local Exchange Library and want to prevent the progress dialog from being displayed during a send.
`preview`	If `true`, a preview is in progress. The `ExgNotifyPreview` function sets this bit while the preview takes place and clears it when the preview is finished. Exchange libraries should not discard any data while a preview is in progress because the full data must be sent later if the receiving user accepts it.
`reserved`	Reserved system flags.
`description`	A pointer to the text description of the object (optional).
`type`	A pointer to the MIME type of the object (optional).
`name`	The name of the object being sent. This can be a URL whose scheme identifies the exchange library to connect with. If the name has a colon, it is treated as a URL.

Compatibility The `noGoTo` and `noStatus` flags are only defined if 3.5 New Feature Set is present, and the `noStatus` flag has no effect unless 4.0 New Feature Set is present. The `preview` flag is only defined if 4.0 New Feature Set is present.

Exchange Manager Constants

Registry ID Constants

The registry ID constants are used in the Exchange Manager registry. Exchange libraries register for the URL prefixes they handle, and applications register for the types of data they receive. The registry ID constants specify which type of data is being registered for.

Constant	Value	Description
`exgRegCreatorID`	0xfffb	Register for a creator ID. The `target` field of the `ExgSocketType` contains the creator ID of the application that should receive the data. Typically, the application with the matching creator ID receives the data, but it is possible for one application to register for another's creator ID and receive data in its place.
`exgRegSchemeID`	0xfffc	Register for a URL scheme. Typically, only exchange libraries register for URL schemes. Applications can register for URL schemes, but they only receive the URL when `ExgRequest` is called. If the `name` field of the `ExgSocketType` contains a colon (:), the portion of the URL before the colon is the URL scheme. The default library registered for URLs with that scheme will handle the message.

Constant	Value	Description
`exgRegExtensionID`	0xfffd	Register for a filename extension. If the `name` field of the `ExgSocketType` contains a period (.), the portion of the name after the last period is the filename extension. The application registered to handle files of that extension will handle the message.
`exgRegTypeID`	0xfffe	Register for a MIME type. If the `type` field of the `ExgSocketType` contains a value, the application registered to receive that MIME type handles the message.

Compatibility The `exgRegCreatorID` and `exgRegSchemeID` constants are only defined if 4.0 New Feature Set is present.

Predefined URL Schemes

The Exchange Manager provides these predefined URL schemes, for which exchange libraries can register.

Constant	Value	Description
`exgBeamScheme`	"_beam"	The URL scheme for Beam commands. By default, the IR Library handles this scheme.
`exgSendScheme`	"_send"	The URL scheme for Send commands. The purpose of the Send command is to provide a choice of transport mechanisms to the user; therefore, any exchange library that sends data should register for this scheme.
`exgLocalScheme`	"_local"	The URL scheme for the Local Exchange Library.

Compatibility These constants are only defined if 4.0 New Feature Set is present.

Predefined URL Prefixes

The Exchange Manager provides the following prefixes, which can be used to construct URLs appropriate for the `name` field of the `ExgSocketType` structure. When sending data, applications provide a URL to identify the exchange library that should transport the data.

Constant	Value	Description
`exgBeamPrefix`	(`exgBeamScheme` ":")	The URL to beam data.
`exgSendPrefix`	("?" `exgSendScheme` ":")	A URL for the general Send command. Because this URL begins with a question mark (?), the Exchange Manager displays a dialog with a list of exchange libraries registered for the `exgSendScheme`. The user then chooses the desired exchange library.
`exgSendBeamPrefix`	("?" `exgSendScheme` ";" `exgBeamScheme` ":")	A URL for the general Send command. The Exchange Manager displays a dialog with a list of exchange libraries registered for either the `exgSendScheme` or the `exgBeamScheme`.
`exgLocalPrefix`	(`exgLocalScheme` ":")	The URL for using the Local Exchange Library.

Compatibility These constants are only defined if 4.0 New Feature Set is present.

Exchange Manager Functions

ExgAccept

Purpose Accepts a connection from a remote device.

Prototype
```
Err ExgAccept (ExgSocketType *socketP)
```

Parameters

-> `socketP`	A pointer to the socket structure (see `ExgSocketType`).

Result Returns one of the following error codes:

`errNone`	Success
`exgErrBadLibrary`	Couldn't find default exchange library
`exgErrNotSupported`	A preview is in progress, and the exchange library identified by `libraryRef` doesn't support preview mode

Other error codes depend on the exchange library.

Displays a fatal error message if `socketP` does not have a `libraryRef` specified.

Comments Applications call this function when launched with the `sysAppLaunchCmdExgReceiveData` or `sysAppLaunchCmdExgPreview` launch code. The launch code contains `socketP` in its parameter block. Applications should pass this socket to `ExgAccept` to accept the connection, then call `ExgReceive` one or more times to receive the data, and then call `ExgDisconnect` to disconnect.

NOTE: Don't create the socket on the receiving side of an exchange. The socket is passed to you in the command parameter block of the `sysAppLaunchCmdExgReceiveData` or `sysAppLaunchCmdExgPreview` launch code.

Compatibility Implemented only if 3.0 New Feature Set is present. Preview mode is supported only if 4.0 New Feature Set is present.

See Also ExgConnect, ExgPut, ExgGet

ExgConnect

Purpose Establishes a connection with a remote socket.

Prototype

```
Err ExgConnect (ExgSocketType *socketP)
```

Parameters

-> `socketP`	A pointer to the socket structure (see ExgSocketType). Specify either a value for the `libraryRef` field or a URL in the `name` field. `libraryRef` should be 0 if the `name` field contains a URL.

Result Returns one of the following error codes:

`errNone`	Success
`exgErrBadLibrary`	Couldn't find exchange library
`exgErrNotSupported`	The library doesn't support the operation specified in `socketP`
`exgErrUserCancel`	The user cancelled the connection operation
`exgMemError`	There isn't enough free memory to respond to the request
`exgErrNotEnoughPower`	The battery does not have enough power to perform the operation

Other error codes depend on the exchange library.

Comments Applications can call this function to initiate a connection for sending multiple objects or for performing two-way communications. Some exchange libraries support sending multiple objects but do not support this call. See "Sending Multiple Objects" on page 16 of *Palm OS Programmer's Companion*, vol. II, *Communications* for more information.

Before calling this function, the application must initialize the `socketP` parameter. The socket should identify the exchange library to connect with by providing either a library reference number in the `libraryRef` field or a URL in the `name` field. The default exchange library registered for that type of URL handles the connection.

To provide users with a choice of transport mechanisms, specify a URL that begins with a question mark (?). The Exchange Manager displays a dialog with a list of all exchange libraries that respond to URLs of the specified type. If only one exchange library is registered for this URL scheme, no dialog is displayed.

For example, many applications on Palm OS® 4.0 or higher support a Send command. This command generates a URL with the prefix `exgSendPrefix` (see Predefined URL Prefixes). The Exchange Manager displays a dialog containing a list of libraries registered for that URL scheme. The user selects an exchange library, and that library's `ExgLibConnect` function is called.

If the library is not specified by either URL or library reference number (in the `libraryRef` field), the Exchange Manager by default uses the IR Library; however, if the `localMode` flag is set, the Local Exchange Library is used instead.

In addition to specifying the library, you can set the `count` field in `socketP` before making this call to indicate the number of objects that are going to be sent. Use a `count` of 0 if the number of objects isn't known in advance.

If no error is returned from `ExgConnect`, applications can follow this call either by sending multiple objects or requesting data from the remote device or both. To send an object, call `ExgPut` at the beginning of each object and call `ExgSend` one or more times per object to send the data. To request data from the remote device, use `ExgGet` (and then use `ExgReceive` to receive the requested data). You can use these calls in combination with each other to support two-way communications. After all of the objects have been sent and received, call `ExgDisconnect` to disconnect.

IMPORTANT: Not all exchange libraries support the sending of multiple objects or using `ExgGet` to request data.

Compatibility Implemented only if 3.0 New Feature Set is present. `ExgConnect` was for system use only until the release of Palm OS 4.0. Multiple object sending and identifying exchange libraries by URL are supported only if 4.0 New Feature Set is present. On earlier releases, this function is an alias for the `ExgPut` function.

See Also `ExgPut`, `ExgAccept`, `ExgGet`



New ExgControl

Purpose Requests that an exchange library perform an operation.

Prototype

```
Err ExgControl (ExgSocketType *socketP, UInt16 op,
void *valueP, UInt16 *valueLenP)
```

Parameters

-> `socketP`	A pointer to the socket structure (see `ExgSocketType`). Specify either a value for the `libraryRef` field or a URL in the `name` field. `libraryRef` should be 0 if the `name` field contains a URL.
-> `op`	A constant identifying the operation that the exchange library should perform. See the Comments section for more information.
<-> `valueP`	Upon entry, a parameter that the exchange library requires to perform the operation, if any. Most operations do not require an input parameter. Upon return, contains the result of the operation.
<-> `valueLenP`	The size of the `valueP` buffer. The size is updated upon return to show the actual length of the content returned.

Result Returns one of the following error codes:

`errNone`	Success
`exgErrBadLibrary`	Couldn't find the requested exchange library
`exgErrNotSupported`	The exchange library does not support the requested operation

Other error codes depend on the exchange library.

Comments The Exchange Manager uses this function to request information from the exchange library. Applications may also call this function.

The Exchange Manager defines and uses a set of operation constants that it might send to any exchange library. These constants begin with the prefix `exgLibCtlGet`. The type of the variable pointed to by `valueP` depends on the type of operation to be performed. Table 58.1 lists and describes the predefined Exchange Manager operations.

Table 58.1 ExgControl operations for all exchange libraries

Operation exgLibCtlGet...	value Data Type	Description
`Preview`	Boolean. Output only.	Returns `true` if the exchange library supports preview mode or `false` if not. If the exchange library does not respond to this operation, it is assumed to support preview mode.
`Title`	String buffer of size `exgTitleBufferSize` bytes. Output only.	Returns the name of the exchange library as it should appear in the Send dialog. All exchange libraries must respond to this operation.

Table 58.1 ExgControl operations for all exchange libraries

Operation exgLibCtlGet...	value Data Type	Description
`Version`	`UInt16`. Output only.	Returns the version of the exchange library API that this library implements. The constant `exgLibAPIVersion` defines the current version number. If the exchange library does not respond to this operation, the library supports the version of the Exchange Library API defined in Palm OS 4.0.

An exchange library may also define its own operations. For example, the IR Library supports operations to enable or disable beaming, to set the baud rates, or to use the serial port (see "IR Control Constants" on page 1200). The SMS Library supports operations that allow you to set the SMS preferences for sending messages or to manipulate multipart messages (see "SMS Control Constants" on page 1701). Operations specific to an exchange library are numbered starting at `exgLibCtlSpecificOp`.

The `socketP` passed to this function must identify an exchange library either using the `libraryRef` field or using a URL in the `name` field. The Comments section in `ExgConnect` describes how an application should identify the exchange library.

Compatibility Implemented only if 4.0 New Feature Set is present.

ExgDBRead

Purpose Converts a Palm OS database from its internal format and writes it to storage RAM.

Prototype

```
Err ExgDBRead (ExgDBReadProcPtr readProcP,
ExgDBDeleteProcPtr deleteProcP, void* userDataP,
LocalID* dbIDP, UInt16 cardNo,
Boolean* needResetP, Boolean keepDates)
```

Parameters

-> `readProcP`	A pointer to a function that reads in the database and passes it to `ExgDBRead`. See `ReadProc` for details.
-> `deleteProcP`	A pointer to a function that is called if a database with an identical name already exists on the device. See `DeleteProc` for details.
-> `userDataP`	A pointer to any data you want to pass to either the `readProcP` or `deleteProcP` functions. Often, this parameter is used to pass the `ExgSocketType` that is required by many Exchange Manager functions.
<- `dbIDP`	The ID of the database that `ExgDBRead` created on the local device.
<- `cardNo`	The number of the card on which the database was stored by `ExgDBRead`.
<- `needResetP`	Set to `true` by `ExgDBRead` if the `dmHdrAttrResetAfterInstall` attribute bit is set in the received database.
-> `keepDates`	Specify `true` to retain the creation, modification, and last backup dates as set in the received database header. Specify `false` to reset these dates to the current date.

Result

Returns `errNone` if successful; otherwise, returns one of the data manager error codes (`dmErr...`) or a callback-specific error code. (If the `readProcP` function returns an error, it is also returned by `ExgDBRead`.)

Comments

This function converts data received from an exchange library or from any other transport mechanism into a Palm OS database and stores that database in the storage heap. It is not required that you use this function in conjunction with Exchange Manager calls. That is, it's possible to use this function to perform other operations, such as converting a database created on the desktop computer to a Palm OS formatted database in the storage heap.

The primary use of this function, however, is to receive a database that has been beamed onto the device. In this case, call `ExgDBRead` in response to the launch code `sysAppLaunchCmdExgReceiveData` after calling `ExgAccept` to accept the connection. Place the call to `ExgReceive` in the read callback function you passed as the `readProcP` parameter. Pass the `ExgSocketType` structure returned from `ExgAccept` in the `userDataP` parameter so that you have access to it in the read callback function.

The read callback function performs the actual reading of data. `ExgDBRead` calls the read callback function multiple times. Each time, the `sizeP` parameter contains the number of bytes `ExgDBRead` expects the data returned in `dataP` to contain. It's important for the read callback function to set the number of bytes (in `sizeP`) that it actually placed in `dataP` if it's not the same as what `ExgDBRead` expected. `ExgDBRead` stops calling the read callback function after 0 is returned in `sizeP`.

The callback function you pass in `deleteProcP` handles the case where the database being read already exists on the device. It is called only in that circumstance. The callback function may want to close the database if it is open, change the existing database's name, or delete the existing database to allow an overwrite. See `DeleteProc` for more information.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `ExgDBWrite`

ExgDBWrite

Purpose Converts a given Palm OS database from its internal format on the local device and writes it using a function you supply.

Prototype

```
Err ExgDBWrite (ExgDBWriteProcPtr writeProcP,
void* userDataP, const char* nameP, LocalID dbID,
UInt16 cardNo)
```

Parameters

-> `writeProcP`	A pointer to a function that writes out the database identified by `dbID`. See `WriteProc` for details.
-> `userDataP`	A pointer to any data you want to pass to the `writeProcP` function. Often, this parameter is used to pass the `ExgSocketType` that is required by many Exchange Manager functions.
-> `nameP`	A pointer to the name of the database that you want `ExgDBWrite` to write. This database is passed to `writeProcP`.
-> `dbID`	The ID of the database that you want `ExgDBWrite` to pass to `writeProcP`. If you don't supply an ID, then `nameP` is used to search for the database by name.
-> `cardNo`	The number of the card on which to look for the database identified by `nameP`.

Result

Returns `errNone` if successful; otherwise, returns one of the data manager error codes (`dmErr...`) or a callback-specific error code. (If the `writeProcP` function returns an error, it is also returned by `ExgDBWrite`.)

Comments

This function converts a Palm OS formatted database on the storage heap into a stream of bytes that can be sent over the Internet or over any other transport mechanism. It is not required that you use this function in conjunction with Exchange Manager calls.

The primary use of this function, however, is to write a database that is going to be beamed onto another device. In this case, call `ExgDBWrite` after establishing the connection with `ExgPut`. Place the call to `ExgSend` in the write callback function you passed as the `writeProcP` parameter. Pass the `ExgSocketType` structure returned from `ExgSend` in the `userDataP` parameter so that you have access to it in the write callback function.

The write callback function performs the actual writing of data. `ExgDBWrite` calls the write callback function multiple times. Each time, the `sizeP` parameter contains the number of bytes of `dataP`

that are to be written. If the write callback function didn't handle it all, it's important that it set in `sizeP` the number of bytes that it did handle successfully. `ExgDBWrite` stops calling the write callback function after 0 is returned in `sizeP`.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `ExgDBRead`

ExgDisconnect

Purpose Terminates an Exchange Manager transfer and disconnects.

Prototype

```
Err ExgDisconnect (ExgSocketType *socketP,
Err error)
```

Parameters

-> `socketP`	A pointer to the socket structure (see `ExgSocketType`) identifying the connection to terminate.
-> `error`	Any error that occurred. This parameter tells the exchange library why the connection is being broken. Normally the error code from `ExgSend` or `ExgReceive` is passed in here.

Result Returns one of the following error codes:

`errNone`	Success
`exgErrBadLibrary`	Couldn't find default exchange library
`exgMemError`	Couldn't read data to send
`exgErrUserCancel`	User cancelled transfer

Other error codes depend on the exchange library.

May display a fatal error message if `socketP` doesn't contain a `libraryRef` value.

Comments

Applications must call this function when finished sending data or receiving data. It terminates the connection made with ExgConnect, ExgAccept, ExgPut, or ExgGet.

In the error parameter, pass any error that occurs during the application loop, including errors returned from other Exchange Manager functions. This ensures that the connection is shut down knowing that it failed rather than succeeded.

It's especially important to check the result code from this function, since this will tell you if the transfer was successful. An errNone return value means that the item was delivered to the destination successfully. It does not mean that the user on the other end actually kept the data.

ExgDisconnect is used after sending and receiving. When receiving, the application can insert its creator ID into the goToCreator field in the socket structure and add other goto information in the goToParams field. After the application returns from the sysAppLaunchCmdExgReceiveData launch code, the exchange library may call ExgNotifyGoto, which launches the goToCreator application with the standard launch code sysAppLaunchCmdGoto.

IMPORTANT: Placing your creator ID in the goToCreator field is no longer a guarantee that you receive this launch code starting in Palm OS 4.0 because Palm OS 4.0 supports the sending of multiple objects at once. Thus, another application might overwrite the goToCreator field after your application has disconnected, making that application the recipient of the launch code.

Note that some exchange libraries wait to establish a connection until ExgDisconnect is called. The IR Library, for example, buffers the data that it receives and then waits until ExgDisconnect to actually send this data unless ExgConnect is called to establish a multi-object send connection.

Compatibility

Implemented only if 3.0 New Feature Set is present.

Prior to Palm OS release 4.0, the Exchange Manager always launched the `goToCreator` application, if one was provided, upon return from this function. If 4.0 New Feature Set is present, the Exchange Manager does not launch the `goToCreator` application. Exchange libraries that want the previous behavior must explicitly call `ExgNotifyGoto`.

See Also `ExgReceive`, `ExgSend`

ExgDoDialog

Purpose Displays a dialog that allows users to accept or reject the receipt of data.

Prototype

```
Boolean ExgDoDialog (ExgSocketType *socketP,
ExgDialogInfoType *infoP, Err *errP)
```

Parameters

-> `socketP`	A pointer to the socket structure (see `ExgSocketType`) identifying the connection. A value must be provided for the `libraryRef` field. Applications can obtain the socket structure from the `sysAppLaunchCmdExgAskUser` launch code parameter block.
<-> `infoP`	A pointer to an `ExgDialogInfoType` structure (see the Comments section below).
<- `errP`	`errNone` if no error, or the error code if an error occurred. Currently, no errors are returned.

Result Returns `true` if the user clicks the OK button on the dialog, or `false` otherwise.

Comments This function displays the exchange dialog, which prompts the user to accept or reject incoming data.

By default, the Exchange Manager calls this function if the receiving application doesn't handle the `sysAppLaunchCmdExgAskUser`

launch code or if it returns `exgAskDialog` from the launch code handler. When the Exchange Manager calls `ExgDoDialog`, the dialog displays a message similar to "Do you want to accept 'John Doe' into Address Book?" and allows the user to accept or reject the data. If the user clicks OK, the data should be received as an unfiled record.

The Exchange Manager attempts to display a preview of the data in the exchange dialog to provide users with enough information to determine if they want to accept or reject the data. To display the preview data, it calls `ExgNotifyPreview`. Applications wishing to support preview mode should respond to the launch code `sysAppLaunchCmdExgPreview`. See the `ExgNotifyPreview` function's description for more information.

Applications may also want to allow users to select a category in which to accept the incoming data. To do so, handle `sysAppLaunchCmdExgAskUser` to call `ExgDoDialog` directly and pass it a pointer to an `ExgDialogInfoType` structure. The `ExgDialogInfoType` structure is defined as follows:

```
typedef struct {
  UInt16     version;
  DmOpenRef  db;
  UInt16     categoryIndex;
} ExgDialogInfoType;
```

-> `version` Set this field to 0 to specify version 0 of this structure.

-> `db` A pointer to an open database that defines the categories the dialog should display.

<- `categoryIndex` The index of the category in which the user wants to file the incoming data.

If `db` is valid, the function extracts the category information from the specified database and displays it in a pop-up list. Upon return, the `categoryIndex` field contains the index of the category the user selected, or `dmUnfiledCategory` if the user did not select a category.

If the call to `ExgDoDialog` is successful, your application is responsible for retaining the value returned in `categoryIndex` and using it to file the incoming data as a record in that category.

One way to do this is to store the `categoryIndex` in the socket's `appData` field (see `ExgSocketType`) and then extract it from the socket in your response to the launch code `sysAppLaunchCmdExgReceiveData`. For example:

```
if (cmd == sysAppLaunchCmdExgReceiveData) {
  UInt16 category =
    (ExgSocketPtr)cmdPBP->appData;
  /* other declarations */

/* Receive the data, and create a new record
  using the received data. indexNew is the
  index of this record. */
  if (category !- dmUnfiledCategory) {
    UInt16 attr;
    Err err;
    err = DmRecordInfo(dbP, indexNew, &attr,
      NULL, NULL);

    // Set the category to the one the user
    // specified, and mark the record dirty.
    if ((attr & dmRecAttrCategoryMask) !=
      category) {
      attr &= ~dmRecAttrCategoryMask;
      attr |= category | dmRecAttrDirty;
      err = DmSetRecordInfo(dbP, indexNew,
        &attr, NULL);
    }
  }
}
```

Some of the Palm OS built-in applications (Address Book, Memo, and ToDo) use this method of setting the category on data received through beaming. Refer to the example code for these applications provided in the SDK for a more complete example of how to use `ExgDoDialog`.

When you explicitly call `ExgDoDialog`, you must set the `result` field of the `sysAppLaunchCmdExgAskUser` launch code's parameter block to either `exgAskOk` (upon success) or `exgAskCancel` (upon failure) to prevent the system from displaying the dialog a second time.

Compatibility Implemented only if 3.5 New Feature Set is present.

Preview mode display in the exchange dialog is implemented only if 4.0 New Feature Set is present.

ExgGet

Purpose Establishes a connection and requests an object from a remote device.

Prototype

```
Err ExgGet (ExgSocketType *socketP)
```

Parameters

-> `socketP`	A pointer to the socket structure (see `ExgSocketType`). Specify either a value for the `libraryRef` field or a URL in the `name` field. `libraryRef` should be 0 if the `name` field contains a URL. The `target`, `type`, or `name` fields should identify the data being requested.

Result Returns one of the following error codes:

`errNone`	Success
`exgErrBadLibrary`	Couldn't find default exchange library
`exgErrUserCancel`	The user cancelled the operation
`exgMemError`	There is not enough free memory to perform the operation

Other error codes depend on the exchange library.

Comments Applications use this function to request data (initiate a send) from a remote device. Not all exchange libraries support this operation.

Before calling this function, the application must initialize the `socketP` parameter. The socket should identify the exchange library to connect with by providing either a library reference number in the `libraryRef` field or a URL in the `name` field. The default exchange library registered for the URL's scheme handles the connection. The socket should also specify what data it is

requesting by providing values for at least one of the `target`, `name`, and `type` fields. Specifying the data in the `name` field is the most common method.

To provide users with a choice of transport mechanisms, the application can provide a URL that begins with a question mark (?). The Exchange Manager displays a dialog with a list of all exchange libraries that respond to URLs of the specified type. If only one exchange library is registered for this URL scheme, no dialog is displayed.

If the library is not specified by either URL or library reference number, the Exchange Manager by default uses the IR Library; however, if the `localMode` flag is set, the Local Exchange Library is used instead.

Applications can use `ExgGet` to initiate a send from the Local Exchange Library. For more information, see "Sending and Receiving Locally" on page 32 of the *Palm OS Programmer's Companion*, vol. II, *Communications*.

If no error is returned, applications should follow this call with one or more calls to `ExgReceive`, to receive the data, or `ExgDisconnect`, to disconnect.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `ExgPut`, `ExgConnect`



New ExgGetDefaultApplication

Purpose Retrieves the default application for the specified type of data or the default exchange library for URLs with the specified scheme.

Prototype

```
Err ExgGetDefaultApplication (UInt32 *creatorIDP,
UInt16 id, const Char *dataTypeP)
```

Parameters

<-	`creatorIDP`	A pointer to the creator ID of the default application or default exchange library.

-> `id`	The registry ID constant identifying the type of data in `dataTypeP`. See Registry ID Constants.
-> `dataTypeP`	A pointer to a string that contains the type of data for which to retrieve the default application or library. If `dataTypeP` is a file extension, do not include the period (.). If it is a URL, do not include the colon (:).

Result Returns `errNone` if a match was found or `exgErrNoKnownTarget` if there is no default application or library for this type of data.

Comments You might use this function to see which application on this device will receive a particular type of data or to see which library on this device handles URLs of a particular scheme.

For example, to find out which application receives TXT files on this device, do the following:

```
UInt32 creatorID;
Err error;
error = ExgGetDefaultApplication(&creatorID,
  exgRegExtensionID, "TXT");
if (!error) {
    //creatorID contains default application.
```

To find out which exchange library handles URLs that use the beam prefix, do the following:

```
UInt32 creatorID;
Err error;
error = ExgGetDefaultApplication(&creatorID,
  exgRegSchemeID, exgBeamScheme);
if (!error) {
    //creatorID contains default library.
```

It's possible to have several applications registered to receive the same type of data, but none of them is the default. When the Exchange Manager receives an object of that type, it selects an application to receive the data, and it selects that same application every time. The selected application effectively becomes the default for the data type even though it is not explicitly set as the default. If

this is the case, the `ExgGetDefaultApplication` function returns the creator ID of this de-facto default application.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `ExgGetRegisteredApplications`, `ExgGetRegisteredTypes`, `ExgRegisterDatatype`, `ExgSetDefaultApplication`



New ExgGetRegisteredApplications

Purpose Retrieves a list of all applications registered to receive data of a specified type.

Prototype

```
Err ExgGetRegisteredApplications
(UInt32 **creatorIDsP, UInt32 *numAppsP,
Char **namesP, Char **descriptionsP, UInt16 id,
const Char *dataTypeP)
```

Parameters

<- `creatorIDsP`	An array of the creator IDs of the applications registered to receive objects of this type. Pass `NULL` for this parameter if you only want to know how many applications are registered for this type.
<- `numAppsP`	The number of applications registered to receive objects of this type. This is the number of elements in the `creatorIDsP` array, the `namesP` array, and the `descriptionsP` array.
<- `namesP`	A packed list of strings, suitable for passing to `SysFormPointerArrayToStrings`, containing the names of the applications or libraries. Each string is no more than `exgMaxTitleLen` characters. Pass `NULL` for this parameter if you don't want to retrieve it.

<- descriptionsP
: A packed list of strings, suitable for passing to SysFormPointerArrayToStrings, containing the descriptions of the applications or libraries. Descriptions are specified when the applications or libraries register for data. Each string is no more than exgMaxDescriptionLength characters. Pass NULL for this parameter if you don't want to retrieve it.

-> id
: The registry ID constant identifying the type of data in dataTypeP. See Registry ID Constants.

-> dataTypesP
: A pointer to a tab-delimited, null-terminated string listing the items to register. (Use \t for the tab character.) Each item in the string must be no more than exgMaxTypeLength characters. There can be no more than 16 types total.

Result Returns errNone upon success or exgMemError if the function cannot allocate space for the creator IDs, names, or descriptions.

IMPORTANT: This function allocates enough space for the creatorIDsP, namesP, and descriptionsP arrays as long as you do not pass NULL for the parameters. You are still responsible for freeing these arrays.

Comments You might use this function to see which applications on this device can receive a particular type of data or to see which libraries on this device handle URLs of a particular scheme. You can also use it to built a list of choices from which the user can select a default application or default exchange library for a particular data type or URL scheme. For example, iMessenger uses this function to build a list of mailto handlers so that the user can choose one of them to be the default.

The Exchange Manager itself uses ExgGetRegisteredApplications to find exchange libraries

when it is given a URL that begins with a question mark (?). It displays the returned list to the user in the Send With dialog.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also ExgGetDefaultApplication, ExgGetRegisteredTypes, ExgRegisterDatatype, ExgSetDefaultApplication



New ExgGetRegisteredTypes

Purpose Retrieve a list of all data types for which a registration exists.

Prototype

```
Err ExgGetRegisteredTypes (Char **dataTypesP,
UInt32 *sizeP, UInt16 id)
```

Parameters

<- `dataTypesP`	A packed list of strings, suitable for passing to SysFormPointerArrayToStrings, containing a sorted list of data types for which a registration exists. Each string is no more than `exgMaxTypeLength` characters.
<- `sizeP`	The number of elements in the `dataTypesP` array.
-> `id`	The type of data to search for. For example, you can search for all registered creator IDs, all registered MIME types, and so on.

Result Returns `errNone` upon success or `exgMemError` if the function cannot allocate space for the data types array.

IMPORTANT: This function allocates enough space for the `dataTypesP` array as long as you do not pass `NULL` for the parameter. You are still responsible for freeing this array.

Comments This function could be used to create an application that allows users to choose the default application for each data type.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also ExgGetDefaultApplication, ExgGetRegisteredApplications, ExgRegisterDatatype, ExgSetDefaultApplication



New ExgGetTargetApplication

Purpose Retrieves the application that should receive a specific message. This function does not search for libraries.

Prototype

```
Err ExgGetTargetApplication
(ExgSocketType *socketP, Boolean unwrap,
UInt32 *creatorIDP, Char *descriptionP,
UInt32 descriptionSize)
```

Parameters

-> `socketP` A pointer to the socket structure (see ExgSocketType). The structure should contain values for the `target`, `type`, or `name` fields.

-> `unwrap` If `true`, only an application that registered to receive the data type with the `exgUnwrap` flag set should be the target application. If `false`, the target application should be an application that registered with the `exgUnwrap` flag clear.

<- `creatorIDP` The creator ID of the application that should receive this object.

<-> `descriptionP` The application's description from the registry, if any.

-> `descriptionSize` The size of the `descriptionP` buffer.

Result Returns one of the following error codes:

`errNone`	Success
`exgErrTargetMissing`	The `target` field contains a creator ID, but the application with that creator ID does not exist
`exgErrNoKnownTarget`	No application is registered to receive the data type

Comments The Exchange Manager uses this function to determine which application should be launched to receive incoming data. Applications and libraries may call this function as well.

`ExgGetTargetApplication` determines the target application by doing the following:

- If the `socketP->target` field contains a creator ID, the Exchange Manager searches the registry to see if an application is registered for that creator ID as the default application. If the registry does not contain an entry for the creator ID, it checks to see if the application identified by the creator ID is installed on this device. If an application is found for the `target`, that is the application returned.
- If the `socketP->type` field contains a MIME type, the Exchange Manager searches the registry for an application registered to receive objects of that type. If one is found, that is the application returned.
- If the `socketP->name` field contains a period (.), the portion after the last period is taken to be the file extension. The Exchange Manager searches the registry for an application registered to receive a file with the specified extension. If one is found, that is the application returned. If not, `exgErrNoKnownTarget` is returned.

If more than one application is registered for the target, type, or file extension, this function returns the one that is registered as the default. If no application is registered as the default, then a specific application is chosen. The Exchange Manager chooses this same application each time. That is, each time a file with a TXT extension is sent with no target or MIME type specified, the

`ExgGetTargetApplication` returns the same application to handle the receipt.

Set the `unwrap` parameter to `true` if the object was sent as part of another object, such as a vStock object that was sent as an attachment to an e-mail message. In this case, the Exchange Manager searches for an application that registered to receive the target, the type, or the file extension of the vStock object with the `exgUnwrap` flag set. If an application is found, the vStock object is delivered, and the exchange library should discard the object that contained it (the e-mail message). If there is no application registered to receive the data with the `exgUnwrap` flag set, this function returns `exgErrNoKnownTarget`. In this case, the exchange library should call `ExgNotifyReceive` again passing the entire e-mail message instead of just the vStock attachment.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `ExgSetDefaultApplication`, `ExgNotifyPreview`, `ExgNotifyReceive`, `ExgRegisterDatatype`

New ExgNotifyGoto

Purpose Launches the target application using `sysAppLaunchCmdGoto`.

Prototype

```
Err ExgNotifyGoto (ExgSocketType *socketP,
UInt16 flags)
```

Parameters

-> `socketP`	A socket identifying the object to deliver (see `ExgSocketType`). The `goToCreator` field contains the application to be launched, and the `goToParams` field contains data for the launch code's parameter block.
-> `flags`	Not currently used. Pass 0 for this parameter.

Result Returns one of the following error codes:

`errNone`	Success or the `goToCreator` field is empty
`dmErr...` (one of the data manager error codes)	The specified application could not be found
`memErrNotEnoughSpace`	Not enough memory available to create the launch code's parameter block

Comments Exchange libraries call this function if they want to support immediate display of the received object. Applications do not call this function.

Most exchange libraries should call `ExgNotifyGoto` after the return from `ExgNotifyReceive` so that the user can inspect the newly received data. If the exchange library is most often used by a single application that does not require the launch code, this call to `ExgNotifyGoto` can be skipped. For example, the SMS Library does not call `ExgNotifyGoto`. SMS messages are received by the SMS Messenger application, which does not launch upon receiving data.

`ExgNotifyGoto` only launches an application if one is specified in the `goToCreator` field and the `noGoTo` parameter is `false`. If a `goToCreator` is not specified, it is not considered an error. This gives the application a way to override the default behavior.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `ExgNotifyReceive`, `ExgDisconnect`



New ExgNotifyPreview

Purpose Displays a preview in the exchange dialog of the data to be received.

Prototype

```
Err ExgNotifyPreview (ExgPreviewInfoType *infoP)
```

Parameters

<-> `infoP`	An `ExgPreviewInfoType` structure containing information about the preview operation.

Result Returns one of the following error codes:

`errNone`	Success
`exgErrNotSupported`	The exchange library doesn't support preview mode
`exgErrNoKnownTarget`	There is no application registered to receive the type of object

Other error codes depend on the application.

Comments This function performs the preview operation specified in the `op` field of the `infoP` parameter. The `ExgDoDialog` function calls this function to show a data preview in the exchange dialog. Exchange libraries might want to call this function in certain circumstances. An application rarely calls this function, but it may do so if it displays its own dialog in response to the launch code `sysAppLaunchCmdExgAskUser`.

`ExgNotifyPreview` uses `ExgGetTargetApplication` to determine the appropriate target application for this data and then launches that application with the launch code `sysAppLaunchCmdExgPreview`, passing `infoP` as the parameter block. The application responds to this launch code by accepting the connection, receiving the data from the exchange library, and depending on the operation requested, drawing the data into the `infoP->bounds` rectangle or returning it in the `infoP->string` field, and then disconnecting. The `ExgDoDialog` function uses the returned information to draw the preview portion of the dialog.

If the preview data is a string, the `ExgNotifyPreview` provides a series of fallback strings that are used if the exchange library doesn't support preview or the application doesn't respond to the launch code. If the application fails to return a string, this function provides one of the following:

- the data's description from `socketP->description`
- the filename in `socketP->name`
- the target application's description as stored in the exchange registry
- the MIME type in `socketP->type`
- the file extension in `socketP->name`

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `ExgNotifyReceive`, `ExgDisconnect`

ExgNotifyReceive

Purpose Delivers an object to the appropriate application using the registry.

Prototype

```
Err ExgNotifyReceive (ExgSocketType *socketP,
UInt16 flags)
```

Parameters

`<-> socketP` A pointer to the socket structure (see `ExgSocketType`).

`-> flags` A bit field. Pass 0 or a combination of the following constants (OR the constants together to specify more than one):

`exgUnwrap`
: The object being delivered should only be handled by an application that registered to receive it with the `exgUnwrap` flag set.

`exgNoAsk`
: Do not ask the user to confirm receipt of data. If this constant is passed, the target application does not receive the `sysAppLaunchCmdExgAskUser` launch code, and the Exchange Manager does not call `ExgDoDialog` to display the user confirmation dialog.

`exgGet`
: Specifies that this is a request for the application to send data rather than to receive data.

Result Returns one of the following error codes:

`errNone`	Success
`exgErrTargetMissing`	The `target` field contains a creator ID, but the application with that creator ID does not exist
`exgErrNoKnownTarget`	No application is registered to receive the data type
`exgErrUserCancel`	The user cancelled the operation

Other error codes depend on the application that is launched.

Comments Exchange libraries call this function to initiate a receive operation on the receiving device. Applications do not call this function.

The `ExgNotifyReceive` function uses `ExgGetTargetApplication` to determine which application should receive the data, then sends that application the appropriate launch codes.

If the `flags` parameter is 0, a receive operation is assumed. The `ExgNotifyReceive` function does the following:

1. It sends the application the `sysAppLaunchCmdExgAskUser` launch code.

2. If the application returns `exgAskDialog` or does not respond to the launch code, it calls `ExgDoDialog`, which sends the application the `sysAppLaunchCmdExgPreview` launch code to have the application receive preview data for the dialog.
3. It sends the application the `sysAppLaunchCmdExgReceiveData` launch code to tell the application to receive the data.

If the `flags` field contains the `exgNoAsk` flag, the first and second steps are skipped.

If the `flags` field contains `exgGet`, this function is a request for data to send to the remote device, not a request to receive data from the remote device. In this case, `ExgNotifyReceive` launches the target application with the `sysAppLaunchCmdExgGetData` launch code.

If the `flags` field has the `exgUnwrap` bit set, it means that the object to be received was sent as part of another object, and it should only be sent to an application that registered to receive it with the `exgUnwrap` flag set. For example, if the exchange library receives an e-mail message with an attached vStock object, the exchange library may call `ExgNotifyReceive` with the `exgUnwrap` flag set and a socket that describes the vStock data type to see if there is an application that registered to receive it directly. If no application is registered to receive vStock objects with the `exgUnwrap` flag set, `ExgNotifyReceive` returns `exgErrNoKnownTarget`. The exchange library should then call `ExgNotifyReceive` again, but this time without the `exgUnwrap` flag and with a socket that describes the e-mail message data type. This second call sends the object to the application registered to receive the e-mail message rather than its vStock attachment. That application may extract the vStock attachment from the message and use the Local Exchange Library to send it to an application registered to receive vStock objects normally (without the `exgUnwrap` flag).

Compatibility Implemented only if 3.0 New Feature Set is present. `ExgNotifyReceive` was a system use only function until the release of Palm OS 4.0.

If the 4.0 New Feature Set is **not** present, the `flags` parameter is not supported, so libraries cannot suppress the exchange dialog, send objects with attachments, or perform a get operation. These features are all added in the 4.0 New Feature Set. Also, if the 4.0 new feature set is not present, this function performs the equivalent of `ExgNotifyGoto` after the application has returned from receiving data. Exchange libraries wishing to support this functionality should call `ExgNotifyGoto` immediately after calling `ExgNotifyReceive`.

See Also `ExgNotifyPreview`

ExgPut

Purpose Initiates the transfer of data to the destination device.

Prototype

```
Err ExgPut (ExgSocketType *socketP)
```

Parameters

-> `socketP` Pointer to the socket structure (see `ExgSocketType`). Specify either a value for the `libraryRef` field or a URL in the `name` field. `libraryRef` should be 0 if the `name` field contains a URL. The structure should also contain a value for the `target`, `type`, or `name` field.

Result Returns one of the following error codes:

`errNone`	Success
`exgErrBadLibrary`	Couldn't find default exchange library
`exgMemError`	Not enough memory to initialize transfer
`exgErrNotEnoughPower`	The battery does not have enough power to perform the operation

Other error codes depend on the exchange library.

Comments

Applications call this function to start a send operation.

If the connection does not already exist, this function establishes one. You must create and initialize an `ExgSocketType` structure containing information about the data to send and the destination application. All unused fields in the structure **must** be set to 0.

If no error is returned, this call **must** be followed by `ExgSend`, to begin sending data, or `ExgDisconnect`, to disconnect. You may need to call `ExgSend` multiple times to send all the data.

The socket's `libraryRef` field or the `name` field must identify the library that performs the transfer. The `libraryRef` field identifies the exchange library by its library reference number. The `name` field identifies the library by URL. The socket should also specify what data is being sent by providing values for at least one of the `target`, `name`, and `type` fields. Use of the `name` field is the most common method.

To provide users with a choice of transport mechanisms, the application can provide a URL that begins with a question mark (?). The Exchange Manager displays a dialog with a list of all exchange libraries that respond to URLs of the specified type. If only one exchange library is registered for this URL scheme, no dialog is displayed.

For example, many applications on Palm OS 4.0 or higher support a Send command. This command generates a URL with the prefix `exgSendPrefix` (see Predefined URL Prefixes). The Exchange Manager displays a dialog containing a list of libraries registered for that URL scheme. The user selects an exchange library, and that library's `ExgLibSend` function is called.

If the library is not specified by either URL or library reference number, the Exchange Manager by default uses the IR Library; however, if the `localMode` flag is set, the Local Exchange Library is used instead.

Compatibility

Implemented only if 3.0 New Feature Set is present.

Support for identifying exchange libraries by URL is implemented only if 4.0 New Feature Set is present.

See Also

`ExgDisconnect`, `ExgSend`, `ExgConnect`

ExgReceive

Purpose Receives data from a remote device.

Prototype

```
UInt32 ExgReceive (ExgSocketType *socketP,
void *bufP, UInt32 bufLen, Err *err)
```

Parameters

-> `socketP`	A pointer to the socket structure (see `ExgSocketType`).
<- `bufP`	A pointer to the buffer in which to receive the data.
-> `bufLen`	The number of bytes to receive.
<- `err`	A pointer to an error code result.

Result Returns the number of bytes actually received. A zero result indicates the end of the transmission.

An error code is returned in the address indicated by `err`. The error code `exgErrUserCancel` is returned if the user cancels the operation. The error code `exgErrNotSupported` is returned if the application calls this function during a preview and the exchange library does not have any more data available or does not support preview.

May display a fatal error message if the library reference number is not provided in `socketP`.

Comments Applications call this function in the following circumstances:

- In response to the `sysAppLaunchCmdExgReceiveData` launch code, following a successful call to `ExgAccept`.
- In response to the `sysAppLaunchCmdExgPreview` launch code, following a successful call to `ExgAccept`.
- To receive requested data following a successful call to `ExgGet`.

After receiving the data, applications call `ExgDisconnect` to terminate the connection.

This function blocks the application until the end of the transmission or until the requested number of bytes has been received. However, exchange libraries can provide their own user interface that is shown during this call, is updated as necessary, and allows the user to cancel the operation in progress.

Compatibility Implemented only if 3.0 New Feature Set is present. Preview mode and `ExgGet` are only supported if 4.0 New Feature Set is present.

See Also `ExgNotifyReceive`



New ExgRegisterDatatype

Purpose Registers an application to receive a specific type of data, or registers an exchange library to handle specific URL schemes.

Prototype

```
Err ExgRegisterDatatype (UInt32 creatorID,
UInt16 id, const Char *dataTypesP,
const Char *descriptionsP, UInt16 flags)
```

Parameters

-> `creatorID`	The creator ID of the registering application or exchange library.
-> `id`	A registry ID constant identifying the type of the items being registered. See Registry ID Constants.
-> `dataTypesP`	Pointer to a tab-delimited, null-terminated string listing the items to register. (Use `"\t"` for the tab character.) To unregister, pass a `NULL` value. Each item in the string must be no more than `exgMaxTypeLength` characters. There can be no more than 16 types total.

NOTE: If specifying file extensions, do not include the period (.) that precedes the extension. If specifying URL prefixes, do not include the colon (:) at the end of the prefix.

-> `descriptionsP`

Pointer to a tab-delimited, null-terminated string that lists descriptions for the items in the `dataTypesP` parameter. (Use `"\t"` for the tab character.) Each description must be no longer than `exgMaxDescriptionLength`. Pass `NULL` to leave out the descriptions.

There must either be one description for all types or the number of descriptions must match the number of types.

The descriptions are used in dialogs displayed by Exchange Manager to identify applications or libraries.

-> `flags`

A bit field specifying registration options. Currently, only one bit is used: the unwrap bit. Pass the `exgUnwrap` constant to specify that the application is registering to receive objects of this type directly if the object is sent as part of another object. For example, if a vStock object is sent as an attachment to an email message, the Exchange Manager should send the vStock object to this application directly rather than sending the message to the email application.

Result

Returns `errNone` if successful, `exgMemError` if there is not enough memory to save the registration info, or one of the data manager error codes (`dmErr`...).

Comments

Both applications and exchange libraries use this function to register with the Exchange Manager to receive certain types of data.

Applications must register with the Exchange Manager to receive data objects that do not specifically target that application using the creator ID in the `target` field.

Exchange libraries register to receive data with certain URL schemes. If an exchange library is not registered to receive URLs, it only handles the receipt and sending of data if its library reference number is explicitly specified in the `ExgSocketType` structure.

Otherwise, the IR Library handles all incoming data for which a library could not be found.

Both applications and libraries should register to receive data as soon as possible after they are installed and as soon as possible after a hard reset. For example, applications can call `ExgRegisterDatatype` in response to the `sysAppLaunchCmdSyncNotify` launch code, which they receive immediately after install. Exchange libraries implemented as applications can also use this strategy. Exchange libraries implemented as shared libraries should call `ExgRegisterDatatype` in their startup functions.

Make only one call to `ExgRegisterDatatype` per registry type. If you want to register to receive multiple items, use a tab character (`\t`) to separate the items. If you were to, for example, make one call to register for the DOC file extension and one call to register for the TXT extension, the second call overwrites the first. However, if you want to register with the `exgUnwrap` flag set, make one call without the `exgUnwrap` flag and one call with the `exgUnwrap` flag set. The application registered with the `exgUnwrap` flag set is stored in a different part of the registry.

Specify `exgRegExtensionID` to register to receive data that has a filename with a particular extension. For example, if your application wants to receive files with a TXT extension, it could register like this:

```
ExgRegisterDatatype(myCreator,
  exgRegExtensionID, "TXT", NULL, 0);
```

If the application wants to receive files with a TXT extension or with a DOC extension, it could register like this:

```
ExgRegisterDatatype(myCreator,
  exgRegExtensionID, "TXT\tDOC", NULL, 0);
```

Specify `exgRegTypeID` to register to receive data with a specific MIME type. For example, if your application wants to receive "setext" text files, it could register like this:

```
ExgRegisterDatatype(myCreator, exgRegTypeID,
"text/x-setext", NULL, 0);
```

Specify `exgRegCreatorID` to register to receive data targeted for a particular creator ID. For example, if your application wants to handle all data intended for the ToDo application, it could register like this:

```
Char toDoCreatorStr[5];
MemMove(toDoCreatorStr, sysFileCToDo, 4);
toDoCreatorStr[4] = chrNull;
ExgRegisterDatatype(myCreator, exgRegCreatorID,
  toDoCreatorStr, NULL, 0);
```

NOTE: To override one application's receipt of data, you need to also set your application as the default for this creator ID. See `ExgSetDefaultApplication`.

Most exchange libraries will want to register for a unique URL scheme that identifies only that library, plus they should register for a more general scheme, such as the send scheme (`exgSendScheme`), which causes the library to be listed in the Send With dialog when the user performs the Send command. The registry ID constant for URL prefixes is `exgRegSchemeID`.

```
ExgRegisterDatatype(myLibCreator,
  exgRegSchemeID, myScheme "\t" exgSendScheme,
  NULL, 0);
```

Registrations are active until a hard reset or until the application or library is removed. The registration information is preserved across a soft reset. When an application is removed, its registry information is also automatically removed from the registry, so there is not normally a need to unregister. If you want to unregister, you can call `ExgRegisterDatatype` with a `NULL` value for the `dataTypesP` parameter.

Multiple applications can be registered to receive the same type of data. If this is the case, the application that is registered as the default (using `ExgSetDefaultApplication`) is the one that receives the data unless the exchange socket explicitly specifies another application should receive it. If there is no default specified, the Exchange Manager determines a default.

Multiple libraries may also be registered to receive the same type of URL. In this case, if the URL begins with a question mark (?), the Exchange Manager displays a dialog so that the user can select which exchange library to use. If the URL does not begin with a question mark, the exchange library registered as the default is used. If there is no default specified, the Exchange Manager determines a default.

Compatibility Implemented only if 4.0 New Feature Set is present. `ExgRegisterDatatype` replaces the `ExgRegisterData` function.

See Also `ExgRegisterData`, `ExgGetTargetApplication`, `ExgPut`, `ExgGetDefaultApplication`, `ExgGetRegisteredApplications`, `ExgGetRegisteredTypes`

ExgRegisterData

Purpose Registers an application to receive a specific type of data. This function is deprecated and replaced with `ExgRegisterDatatype`.

Prototype

```
Err ExgRegisterData (UInt32 creatorID, UInt16 id,
const Char *dataTypesP)
```

Parameters

-> `creatorID`	Creator ID of the registering application.
-> `id`	Registry ID identifying the type of the items being registered. Specify `exgRegExtensionID` or `exgRegTypeID`.
-> `dataTypesP`	Pointer to a tab-delimited, null-terminated string listing the items to register. (Use `\t` for the tab character.) These include file extensions or MIME types. To unregister, pass a `NULL` value.

Result Returns `errNone` if successful, otherwise, one of the data manager error codes (`dmErr...`).

Comments Applications that wish to receive data from anything other than another Palm Powered™ handheld running the same application must use this function to register for the kinds of data they can receive. Call this function when your application is loaded on the device.

Compatibility This function corresponds to the Palm OS 3.5 version of `ExgRegisterDatatype`. It is implemented only if 3.0 New Feature Set is present.



New ExgRequest

Purpose Requests some data from an exchange library or an application using a URL.

Prototype

```
Err ExgRequest (ExgSocketType *socketP)
```

Parameters

-> `socketP`	Pointer to the socket structure (see `ExgSocketType`). Specify a URL in the `name` field and a `libraryRef` of 0.

Result Returns one of the following error codes:

`errNone`	Success
`exgErrBadLibrary`	Couldn't find default exchange library
`exgErrNotEnoughPower`	The device does not have enough power to perform the operation
`sysErrLibNotFound`	Couldn't find library or application to respond to URL

Other error codes depend on the exchange library or application.

Comments The `ExgRequest` function is similar to `ExgGet` in that both are used to request data. The difference is that the application that calls `ExgGet` is always the application that receives the data. When you

call `ExgRequest`, the application that receives the data is the application that is registered to receive it. For example, using `ExgRequest`, it is possible for one application to use the Exchange Manager to retrieve a vCard using any supported transport mechanism and have that data sent directly to the Address Book application instead of to the calling application.

The `socketP` passed to this function identifies the exchange library using a URL in the `name` field. The application must know beforehand the proper URL prefix for the exchange library with which it wants to connect. See Predefined URL Prefixes for a list of URL prefixes that the Exchange Manager provides.

If the provided URL begins with a question mark (?) and there are several exchange libraries registered for the specified URL scheme, the Exchange Manager displays a dialog from which the user selects the appropriate transport mechanism.

If the Exchange Manager cannot find a library that is registered for the specified URL, it assumes that an application is registered to receive the URL, and it launches that application with the `sysAppLaunchCmdGoToURL` launch code.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `ExgGet`, `ExgNotifyReceive`

ExgSend

Purpose Sends data to the destination device.

Prototype

```
UInt32 ExgSend (ExgSocketType *socketP,
const void *bufP, UInt32 bufLen, Err * err)
```

Parameters

-> `socketP` A pointer to the socket structure (see `ExgSocketType`). A value must be provided for the `libraryRef` field. The structure should also contain values for the `target`, `type`, or `name` fields.

-> `bufP` A pointer to the data to send.

-> `bufLen`	The number of bytes to send.
<- `err`	A pointer to an error code result.

Result Returns the number of bytes actually sent, normally the same number as specified in `bufLen`. An error code is returned in the address indicated by `err`. The error code `exgErrUserCancel` is returned if the user cancels the operation.

May display a fatal error message if the `socketP` parameter does not contain a value for the `libraryRef` field.

Comments Call this function one or more times to send all the data, following a successful call to `ExgPut`. After sending the data, call `ExgDisconnect` to terminate the connection.

The exchange library may break large amounts of data into multiple packets or assemble small send commands together into larger packets, but the application will not be aware of these transport level details.

This function blocks the application until all the data is sent. However, the exchange library may provide its own user interface that is updated as necessary and allows the user to cancel the operation in progress.

Compatibility Implemented only if 3.0 New Feature Set is present.

See Also `ExgReceive`, `ExgGet`

New ExgSetDefaultApplication

Purpose Sets the application that receives a specified type of data by default. This function also sets the default exchange library that handles particular URL schemes.

Prototype

```
Err ExgSetDefaultApplication (UInt32 creatorID,
UInt16 id, const Char *dataTypeP)
```

Parameters

-> `creatorID`	The creator ID of the application or library that should become the default for this type of data.
-> `id`	A registry ID constant identifying the type of data in `dataTypeP`. See Registry ID Constants.
-> `dataTypesP`	A pointer to a null-terminated string containing the desired type of data.

NOTE: If specifying a file extension, do not include the period (.) that precedes the extension. If specifying a URL prefix, do not include the colon (:) at the end of the prefix.

Result

Returns `errNone` upon success or `exgErrNoKnownTarget` if the specified application is not registered to receive the specified data type.

Comments

This function sets the default application that receives data of a certain type when no target is specified and the default exchange library that handles URLs with a certain prefix if no library reference number is specified.

Palm™ strongly recommends that applications allow the user to determine which application should become the default recipient for a data type. To do so, an application can use `ExgGetRegisteredApplications` to get the list of applications registered for the same type of data as it is, and then display a dialog listing those applications and allow the user to select it. Then it should call `ExgSetDefaultApplication` with the user-specified default.

If you call `ExgSetDefaultApplication` with an application or library that is already the default, this function has no effect.

An application can become the default for its own creator ID even if it has not specifically registered to receive its own creator ID. That is, suppose several applications are registered to receive objects targeted for the ToDo application's creator ID. The ToDo application itself is not registered for its own creator ID, as it is not necessary to do so. However, an application can use code like the following to set the ToDo application as the default for its own creator ID.

```
Char toDoCreatorStr[5];
MemMove(toDoCreatorStr, sysFileCToDo, 4);
toDoCreatorStr[4] = chrNull;
ExgSetDefaultApplication(sysFileCToDo,
  exgRegCreatorID, toDoCreatorStr);
```

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also ExgGetDefaultApplication, ExgRegisterDatatype

Application-Defined Functions

DeleteProc

Purpose Handles the case where a database with an identical name already exists on the device.

Prototype

```
Boolean DeleteProc (const char* nameP,
UInt16 version, UInt16 cardNo, LocalID dbID,
void* userDataP)
```

Parameters

-> nameP	A pointer to the name of the identical database.
-> version	The version of the identical database.
-> cardNo	The card number of the identical database.
-> dbID	The database ID of the identical database.
-> userDataP	The userDataP parameter you passed to ExgDBRead. If used, this parameter contains any application-specific data you find necessary. If the ReadProc function is implemented using Exchange Manager calls, this often contains the ExgSocketType structure.

Result Return true to have the ExgDBRead function continue to read the database. Use this return value if you have deleted or moved the existing database or if you want the database to be overwritten.

Return `false` to have `ExgDBRead` exit without reading the database.

Comments This function is called if the Data Manager can't create the incoming database because a database with the same name already exists. You should delete the existing database or take some other action, such as changing the database name. It is appropriate to prompt the user before choosing to delete or move the database.

ReadProc

Purpose Reads in the database and pass it to `ExgDBRead`.

Prototype

```
Err ReadProc (void* dataP, UInt32* sizeP,
void* userDataP)
```

Parameters

<- `dataP`	A pointer to a buffer where this function should place the database data. This buffer is allocated in the dynamic heap by `ExgDBRead`; you don't need to use `DmWrite` when filling it.
<-> `sizeP`	The size of `dataP`. This value is set by `ExgDBRead` to the number of bytes it expects to receive in `dataP`. You must set this value to the number of bytes you return in `dataP` (if it's not the same).
-> `userDataP`	The `userDataP` parameter you passed to `ExgDBRead`. Pass the `ExgSocketType` structure if you implement this function using Exchange Manager calls.

Result Return an error number, or `errNone` if there is no error. If this function returns an error, `ExgDBRead` deletes the database it was creating, cleans up any memory it allocated, then exits, returning the error passed back from this function.

Comments `ExgDBRead` is commonly used to receive a database from a beam or from some other transport mechanism. In this case, an appropriate

implementation of this callback function is to call ExgReceive as shown here:

```
Err MyReadDBProc (void *dataP, UInt32 *sizeP,
  void *userDataP)
{
  Err err = errNone;

  //userDataP contains ExgSocketType pointer.
  *sizeP =
    ExgReceive((ExgSocketType *)userDataP,
      dataP, *sizeP, &err);
  return err;
}
```

WriteProc

Purpose Writes out the database.

Prototype

```
Err WriteProc (const void* dataP, UInt32* sizeP,
void* userDataP)
```

Parameters

-> `dataP`	A pointer to a buffer containing the database data, placed there by ExgDBWrite.
<-> `sizeP`	The number of bytes placed in `dataP` by `ExgDBWrite`. If you were unable to write out or send all of the data in this chunk, on exit, set `sizeP` to the number of bytes you did write.
-> `userDataP`	The `userDataP` parameter you passed to `ExgDBWrite`. You can use it for application-specific data. Pass the ExgSocketType structure if you implement this function using Exchange Manager calls.

Result Return an error number, or `errNone` if there is no error. If this function returns an error, `ExgDBWrite` closes the database it was reading, cleans up any memory it allocated, then exits, returning the error passed back from this function.

Comments `ExgDBWrite` is commonly used to write a database that is going to be beamed to another device (or sent through some other transport mechanism). In this case, an appropriate implementation of this callback function is to call `ExgSend` as shown here:

```
Err MyWriteDBProc (void *dataP, UInt32 *sizeP,
  void *userDataP)
{
  Err err = errNone;

  //userDataP contains ExgSocketType pointer.
  *sizeP =
    ExgSend((ExgSocketType *)userDataP,
      dataP, *sizeP, &err);
  return err;
}
```

59

Exchange Library

The Exchange Library API described in this chapter and declared in `ExgLib.h` specifies a minimal set of functions that all exchange libraries must implement. This chapter is directed towards developers who use or create exchange libraries. Developers creating an exchange library should also read the Exchange Libraries chapter of the *Palm OS Programmer's Companion*, vol. II, *Communications*.

Exchange Library Functions



New

ExgLibAccept

Purpose Accept an incoming connection.

Prototype

```
Err ExgLibAccept(UInt16 libRefnum,
ExgSocketType *exgSocketP)
```

Parameters

-> `libRefnum` Reference number of this exchange library.

-> `exgSocketP` A pointer to the socket structure (see `ExgSocketType`).

Result Returns `errNone` if no error. `exgErrNotSupported` is returned if a preview is in progress and the exchange library does not support preview. Other error codes are defined by each exchange library.

Comments The Exchange Manager's `ExgAccept` function simply calls `ExgLibAccept` in the exchange library identified by the

ExgSocketType structure passed to ExgAccept. An application calls the Exchange Manager's ExgAccept function when:

- The application wants to initiate a connection to receive data, which it does in response to sysAppLaunchCmdExgReceiveData.
- The application wants to initiate a connection to receive a preview of the data, which it does in response to sysAppLaunchCmdExgAskUser.

Any implementation of ExgLibAccept should update any progress dialogs to indicate that data is being accepted (or received) into an application.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also ExgLibPut

New ExgLibClose

Purpose Library-specific. Although this function is not called by the Exchange Manager, all shared libraries normally implement it.

Prototype

```
Err ExgLibClose (UInt16 libRefnum)
```

Parameters -> libRefnum Reference number of this exchange library.

Comments Exchange libraries are free to implement this function for internal or external use. The Exchange Manager does not call it.

New ExgLibConnect

Purpose Open a connection in preparation for sending or receiving objects.

Prototype

```
Err ExgLibConnect(UInt16 libRefNum,
ExgSocketType *exgSocketP)
```

Parameters

-> `libRefNum`	Reference number of this exchange library.
<-> `exgSocketP`	A pointer to an `ExgSocketType` structure identifying the socket through which objects will be sent or received.

Result

Returns `errNone` if no error. If `ExgLibConnect` is not supported by this library, this function returns `exgErrNotSupported`; if its use is optional, `errNone` is returned. Other error codes are defined by each exchange library.

Comments

The Exchange Manager may call this function to initiate a connection for sending multiple objects or for performing two-way communications. Some exchange libraries support sending multiple objects but do not support this call. See "Sending Multiple Objects" on page 16 of *Palm OS Programmer's Companion*, vol. II, *Communications* for more information.

Not all exchange libraries support this operation. In this case, the first call to `ExgLibPut` must clean up after itself before returning an error. The exchange library should not expect to get an `ExgLibDisconnect` call.

If `ExgLibConnect` is supported and an application calls `ExgConnect`, the exchange library should delay any cleanup until `ExgLibDisconnect`, unless `ExgLibConnect` returns an error, in which case it should clean up after itself. The library can expect to get an `ExgLibDisconnect` call if it returns `errNone` from `ExgLibConnect`. If the application does not call `ExgConnect`, the first call to `ExgLibPut` must clean up after itself before returning an error and the exchange library should not expect to get an `ExgLibDisconnect` call.

The Exchange Manager's `ExgConnect` function calls `ExgLibConnect` in the exchange library identified by the `ExgSocketType` structure passed to `ExgConnect`. If `ExgLibConnect` is implemented to return `exgErrNotEnoughPower`, the Exchange Manager puts up an alert, so there is no need for the exchange library to do so. Other error codes are not treated specially by the Exchange Manager; the exchange library must put up its own alerts when appropriate.

The exchange library may prompt the user for addressing information.

Compatibility Implemented only if 4.0 New Feature Set is present.



New ExgLibControl

Purpose Supply information about the exchange library.

Prototype

```
Err ExgLibControl(UInt16 libRefNum, UInt16 op,
void *valueP, UInt16 *valueLenP)
```

Parameters

-> `libRefNum`	Reference number of this exchange library.
-> `op`	The operation to perform.
<-> `valueP`	An operation specific parameter. See comments below.
<-> `valueLenP`	An operation specific parameter. See comments below.

Result Returns `errNone` if no error. `exgErrNotSupported` is returned if the specified operation is not supported by the exchange library. `exgErrBadParam` is returned if the parameters (`valueP` and `valueLenP`) are not appropriate for the operation. Additional error codes are defined by each exchange library.

Comments `ExgLibControl` is a general purpose function that performs various minor operations based upon a selector. `ExgMgr.h` defines three selectors: `exgLibCtlGetTitle`, which should be supported by all exchange libraries, `exgLibCtlGetVersion`, and `exgLibCtlGetPreview`. Additional library-specific selectors should be numbered starting at `exgLibCtlSpecificOp` (0x8000).

Upon receiving `exgLibCtlGetTitle`, the exchange library must return a library title suitable for use in Exchange Manager dialogs. Exchange libraries that are built as applications should generally return their `'tAIN'` resource as their title. Be sure that the title

returned through `valueP` honors the maximum length specified in `valueLenP`.

An exchange library that implements the Palm OS 4.0 version of the Exchange Library API needn't do anything special upon receiving `exgLibCtlGetVersion`; it should simply return `exgErrNotSupported`. Otherwise, set `valueP` to a two-byte value indicating the API version number and return `errNone`. Note that a version number of zero corresponds to the Palm OS 4.0 version of the Exchange Library API.

The `exgLibCtlGetPreview` operation is used by the Exchange Manager to determine whether a given exchange library supports preview. The Exchange Manager assumes that an exchange library supports preview if the exchange library doesn't implement this operation. To indicate that a library does not support preview, set `valueP` to `false` and return `errNone`.

The Exchange Manager's `ExgControl` function simply calls `ExgLibControl` in the exchange library identified by the `ExgSocketType` structure passed to `ExgControl`.

Compatibility Implemented only if 4.0 New Feature Set is present.



New ExgLibDisconnect

Purpose Disconnect a connection made with `ExgLibConnect`, `ExgLibAccept`, `ExgLibPut`, or `ExgLibGet`.

Prototype

```
Err ExgLibDisconnect(UInt16 libRefnum,
ExgSocketType *exgSocketP, Err error)
```

Parameters

-> `libRefnum`	Reference number of this exchange library.
-> `exgSocketP`	A pointer to an `ExgSocketType` structure identifying the socket connection to be disconnected.
-> `error`	The current error state. Used to indicate why the connection is being broken, for example, user cancel or out of memory.

Result Typically the same error code passed in. However, this function may return an error even if `errNone` is passed in.

Comments `ExgLibDisconnect` may be used to finish reading the data during a preview; in this case, the connection, if any, is not shut down.

Applications call `ExgLibDisconnect` when all data has been sent or the application wants to stop the send process. If the send data process is not completed, the caller should pass an error parameter indicating why the operation was stopped. `ExgLibDisconnect` is responsible for completing the operation and closing any communication ports if necessary. If data was buffered for sending in `ExgLibSend`, then the disconnect process may actually perform the entire transmit operation. It is important to note that the `ExgLibDisconnect` function can be called for an `ExgLibPut`, `ExgLibAccept` or `ExgLibGet` function. So it is equally important to keep track of the current operation in the `ExgSocketType`. If dialogs are displayed, this function must update them as appropriate. If there are errors, `ExgLibDisconnect` should display them (if allowed by the application).

If `ExgLibConnect` is not supported, the first call to `ExgLibPut` must clean up after itself before returning an error and the exchange library should not expect to get an `ExgLibDisconnect` call.

If `ExgLibConnect` is supported and an application calls `ExgConnect`, the exchange library should delay any cleanup until `ExgLibDisconnect`, unless `ExgLibConnect` returns an error, in which case it should clean up after itself. The library can expect to get an `ExgLibDisconnect` call if it returns `errNone` from `ExgLibConnect`. If the application does not call `ExgConnect`, the first call to `ExgLibPut` must clean up after itself before returning an error and the exchange library should not expect to get an `ExgLibDisconnect` call.

The Exchange Manager's `ExgDisconnect` function simply calls `ExgLibDisconnect` in the exchange library identified by the `ExgSocketType` structure passed to `ExgDisconnect`.

Compatibility Implemented only if 4.0 New Feature Set is present.



New ExgLibGet

Purpose

Establish a connection and request data from a remote device.

Prototype

```
Err ExgLibGet(UInt16 libRefNum,
ExgSocketType *exgSocketP)
```

Parameters

-> `libRefNum` Reference number of this exchange library.

<-> `exgSocketP` A pointer to the socket structure (see `ExgSocketType`).

Result

Returns `errNone` if no error. `exgErrNotSupported` is returned if this operation is not supported by this library. Other error codes are defined by each exchange library.

Comments

`ExgLibGet` informs the library that it should make a connection to the remote device and request information from it. When an exchange library's `ExgLibGet` function is called, it should fetch the requested data and prepare to deliver it when the application calls `ExgReceive`. After `ExgLibReceive` is called, possibly more than once, `ExgLibDisconnect` follows.

The Exchange Manager's `ExgGet` function calls `ExgLibGet` in the exchange library identified in the `ExgSocketType` structure passed to `ExgGet`. If an exchange library's implementation of `ExgLibGet` returns `exgErrNotEnoughPower`, the Exchange Manager puts up an alert, so there is no need for the exchange library to do so. Other error codes are not treated specially by the Exchange Manager; an exchange library must put up its own alerts when appropriate.

Compatibility

Implemented only if 4.0 New Feature Set is present.



New ExgLibHandleEvent

Purpose Handle exchange-library-specific events.

Prototype
```
Boolean ExgLibHandleEvent(UInt16 libRefnum,
void *eventP)
```

Parameters

-> `libRefnum`	Reference number of this exchange library.
-> `eventP`	The event to handle.

Result Returns `true` if the event was handled.

Comments Exchange libraries are free to implement this function for internal or external use. The `Exchange Manager` does not call it.



New ExgLibOpen

Purpose Library-specific. Although this function is not called by the Exchange Manager, all shared libraries normally implement it.

Prototype
```
Err ExgLibOpen (UInt16 libRefnum)
```

Parameters

-> `libRefnum`	Reference number of this exchange library.

Comments Exchange libraries are free to implement this function for internal or external use. The `ExgLibDisconnect` does not call it.



New ExgLibPut

Purpose

Signals the start of an object to be transferred to the destination device.

Prototype

```
Err ExgLibPut(UInt16 libRefnum,
ExgSocketType *exgSocketP)
```

Parameters

-> `libRefnum` Reference number of this exchange library.

<- `exgSocketP` Pointer to the socket structure (see `ExgSocketType`).

Result

Returns `errNone` if no error. Error codes are defined by each exchange library.

Comments

Opens a connection if necessary. The actual data should be sent using `ExgLibSend` after which the connection should be shut down with `ExgLibDisconnect`. The exchange library may prompt the user for addressing information.

The first time this library is called, it may be necessary to allocate global variables and perform other initialization steps. It is usually a good idea to keep any state information about the open connection in the `socketRef` field of the `exgSocketP` structure. This data can then be passed to subsequent operations on that socket.

`ExgLibPut` should then check if the `socketRef` field of `exgSocketP` has been initialized. In some cases, an application may already have filled `socketRef` with addressing information. The use of `socketRef` is entirely up to the exchange library. If `socketRef` is empty, the exchange library needs to fill in any addressing information. In order to do this, the exchange library needs to open its own dialog asking the user for whatever addressing information would be appropriate.

Exchange libraries are responsible for any validation of data entered in the addressing dialog and may use Address Book lookup or other system features to improve the user experience.

Once the user has completed addressing and confirmed the dialog, the exchange library should, in general, call the Progress Manager to open a progress dialog that remains open during the entire put operation. The progress dialog should not be opened if the `noStatus` option was passed or if the transaction is in asynchronous mode.

If the exchange library is displaying dialogs, it must also look for events and pass them to the Progress Manager.

Other operations within `ExgLibPut` depend on the exchange library. The exchange library may open communications ports and establish remote links at this time. Or it may just open a stream for buffering data until a later operation. If any errors occur in this process, the exchange library is responsible for removing any progress dialogs and returning an error. If displaying progress, the exchange library may also need to display an error using the Progress Manager before returning.

NOTE: The progress dialog is converted to an error dialog and waits until the user dismisses it. This may occur in `ExgLibPut` or later in `ExgLibDisconnect`. It depends on whether the connection was made in `ExgLibPut` or whether it used an existing connection made by a previous call to `ExgLibConnect` or `ExgLibPut`.

The first call to `ExgLibPut` must clean up after itself before returning an error if one of the following conditions exists:

- If `ExgLibConnect` is not supported.
- If `ExgLibConnect` is supported and an application does not call `ExgLibConnect`.

The Exchange Manager's `ExgPut` function calls `ExgLibPut` in the exchange library identified by the `ExgSocketType` structure passed to `ExgPut`. If `ExgLibPut` returns `exgErrNotEnoughPower`, the Exchange Manager puts up an alert, so there is no need for the exchange library to do so. Other error codes are not treated specially by the Exchange Manager; an exchange library must put up its own alerts when appropriate.

Compatibility Implemented only if 4.0 New Feature Set is present.



New ExgLibReceive

Purpose

Receive data from a remote device.

Prototype

```
UInt32 ExgLibReceive(UInt16 libRefNum,
ExgSocketType *exgSocketP, void *bufP,
UInt32 bufSize, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of this exchange library.
<-> `exgSocketP`	A pointer to the socket structure (see `ExgSocketType`).
-> `bufP`	A pointer to a buffer into which the data is put.
-> `bufSize`	The size of the buffer in bytes.
<- `errP`	The error code result: `errNone` if no error. `exgErrNotSupported` is returned if used during a preview and the exchange library does not support preview or if there appears to be more data available but `ExgLibReceive` cannot obtain it.Error codes are defined by each exchange library.

Result

The number of bytes received. Returns 0 if the object is complete. `ExgLibReceive` blocks until at least one byte is available or the object is complete.

Comments

Use after `ExgLibGet` or `ExgLibAccept` to receive the contents of the object. May be used after `ExgLibAccept` to examine the contents during a preview.

`ExgLibReceive` must update any progress dialogs to indicate that data is being received. The `ExgLibReceive` should fill the buffer passed as much as possible and return the actual number of bytes that were stored in the buffer. `ExgLibReceive` must block for at least one byte if the stream is not complete. Returning zero bytes indicates the end of the data.

The Exchange Manager's ExgReceive function simply calls ExgLibReceive in the exchange library identified by the ExgSocketType structure passed to ExgReceive.

Compatibility Implemented only if 4.0 New Feature Set is present.



New ExgLibRequest

Purpose Requests an object using a URL, then has the Exchange Manager send it to the default application registered for the object's type.

Prototype

```
Err ExgLibRequest(UInt16 libRefNum,
ExgSocketType *socketP)
```

Parameters

-> libRefNum — Reference number of this exchange library.

<-> socketP — Pointer to the socket structure (see ExgSocketType). The name field contains the URL of the object being requested.

Result Returns errNone if no error. exgErrNotSupported is returned if the exchange library does not support this operation. Other error codes are defined by each exchange library.

Comments The exchange library may prompt the user for addressing information. The socket's name field may contain a URL with any of the schemes for which the exchange library registered. Not all exchange libraries support this operation.

NOTE: This function is often used to tickle an exchange library to make it check to see if there are new messages. These messages are then delivered as usual. So there may or may not be a specific object being requested.

The Exchange Manager's ExgRequest function calls ExgLibRequest in the exchange library identified by the ExgSocketType structure passed to ExgRequest. If

`ExgLibRequest` returns `exgErrNotEnoughPower`, the Exchange Manager puts up an alert, so there is no need for the exchange library to do so. Other error codes are not treated specially by the Exchange Manager; an exchange library must put up its own alerts when appropriate.

See Also `ExgRegisterDatatype`



New ExgLibSend

Purpose Send data for an object to a destination device.

Prototype

```
UInt32 ExgLibSend(UInt16 libRefNum,
ExgSocketType *exgSocketP, const void *bufP,
UInt32 bufLen, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of this exchange library.
<- `exgSocketP`	A pointer to the socket structure (see `ExgSocketType`).
-> `bufP`	A pointer to a buffer containing the data to send.
-> `bufLen`	The number of bytes to send.
<- `errP`	The error code result. Error codes are defined by each exchange library

Result Returns the number of bytes sent.

Comments Applications call `ExgSend` after `ExgPut` and in response to `sysAppLaunchCmdExgGetData`.

This function blocks until all bytes to be sent are actually sent, or until an error occurs (such as device full).

`ExgLibSend` may be called any number of times with varying size buffers to transmit information. If dialogs are being displayed, `ExgLibSend` must keep them updated (perhaps with animation or

progress information). `ExgLibSend` must also check for events and let the Progress Manager handle them.

The Exchange Manager's `ExgSend` function simply calls `ExgLibSend` in the exchange library identified by the `ExgSocketType` structure passed to `ExgSend`.

Compatibility Implemented only if 4.0 New Feature Set is present.



New ExgLibSleep

Purpose The device is going to sleep.

Prototype

```
Err ExgLibSleep(UInt16 libRefnum)
```

Parameters -> `libRefnum` Reference number of this exchange library.

Result Returns `errNone` if no error. Error codes are defined by each exchange library.

Comments The device goes into sleep mode when the user turns the device off, the auto-off timer expires, or power is low. All shared libraries must implement `ExgLibSleep`; however, no processing is required and simply returning `errNone` is enough.

IMPORTANT: Libraries must return from this function quickly to allow sufficient time for emergency shutdown situations (removal of batteries, for example).



New ExgLibWake

Purpose The device is waking up.

Prototype
```
Err ExgLibWake(UInt16 libRefnum)
```

Parameters -> `libRefnum` Reference number of this exchange library.

Result Returns `errNone` if no error. Error codes are defined by each exchange library.

Comments The device wakes up when the user turns the device on. All shared libraries must implement `ExgLibWake`, although no processing is required; it is enough to simply return `errNone`.

60

IR Library

The IR (InfraRed) library is a shared library that provides a direct interface to the IR communications capabilities of the Palm OS®. This chapter provides reference material for the IR library API:

- IR Library Data Structures
- IR Library Constants
- IR Stack Callback Events
- IR Library Functions
- IAS Functions
- Application-Defined Functions

The header file `irlib.h` declares the IR library API. For more information on the IR library, see the chapter "Beaming (Infrared Communication)" in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

IR Library Data Structures

This section lists some of the more important data types used by the IR library functions.

IrConnect

The `IrConnect` structure is used to manage an IrLMP or Tiny TP connection.

```
typedef struct _hconnect {
  UInt8 lLsap;
  UInt8 rLsap;
  UInt8 flags;
  UInt8 reserved;
  IrCallBack callBack;
  IrPacket packet;
  ListEntry packets;
```

```
  UInt16 sendCredit;
  UInt8 availCredit;
  UInt8 dataOff;
} _hconnect;
```

Field Descriptions

`lLsap`	The local LSAP on which this connection listens.
`rLsap`	The LSAP assigned to the remote side.
`flags`	For system use only.
`reserved`	Reserved for future use.
`callBack`	Pointer to callback function. For system use only.
`packet`	For system use only.
`packets`	List of packets to send.
`sendCredit`	Amount of credit from peer.
`availCredit`	Amount of credit to give to peer.
`dataOff`	Amount of data less than IrLAP size.

IrPacket

The `IrPacket` structure is used for sending IrDA packets.

```
typedef struct _IrPacket {
  ListEntry node;
  UInt8 *buff;
  UInt16 len;
  IrConnect* origin;
  UInt8 headerLen;
  UInt8 header[14];
  UInt8 reserved;
} IrPacket;
```

Field Descriptions

`node`	For system use only.
`buff`	Pointer to the send data buffer.

`len`	Number of bytes in data buffer.
`origin`	Pointer to connection that owns the packet. For system use only.
`headerLen`	Number of bytes contained in the header. For system use only.
`header`	Storage for header. For system use only.
`reserved`	Reserved for future use.

IMPORTANT: The `node` field must be the first field in the structure. It is used internally by the stack.

IrIASObject

The `IrIASObject` structure is used as storage for an IAS object managed by the local IAS server. An object of this type is passed as the `obj` parameter to the `IrIAS_Add` function.

```
typedef struct _IrIasObject {
  UInt8 *name;
  UInt8 len;
  UInt8 nAttribs;
  IrIasAttribute* attribs;
} IrIasObject;
```

Field Descriptions

`name`	Pointer to name of object.
`len`	Length of object name.
`nAttribs`	Number of attributes.
`attribs`	Pointer to an array of attributes.

IrIasQuery

The `IrIasQuery` structure is used to perform IAS queries. The `IrIasQuery` object is passed as the `token` parameter to functions such as `IrIAS_Query` and IrIAS_Next.

```
typedef struct _IrIasQuery {
  UInt8 queryLen;
  UInt8 reserved;
  UInt8 *queryBuf;
  UInt16 resultBufSize;
  UInt16 resultLen;
  UInt16 listLen;
  UInt16 offset;
  UInt8 retCode;
  UInt8 overFlow;
  UInt8 *result;
  IrIasQueryCallBack callBack;
} _IrIasQuery;
```

Field Descriptions

`queryLen`	Total length of the query.
`reserved`	Reserved for future use.
`queryBuf`	Pointer to buffer containing the query.
`resultBufSize`	Size of the result buffer.
`resultLen`	Actual number of bytes in the result buffer.
`listLen`	Number of items in the result list.
`offset`	Offset into the results buffer.
`retCode`	Return code of operation.
`overFlow`	Set to `true` if result exceeded result buffer size.
`result`	Pointer to buffer containing result.
`callBack`	Pointer to query callback function.

IrCallbackParms

The `IrCallbackParms` structure is used to pass information from the stack to the upper layer of the stack (application). Not all fields are valid at any given time. The type of event determines which fields are valid. The `IrCallbackParms` object is passed as the second parameter to the `IrCallback` function.

```
typedef struct {
  IrEvent event;
  UInt8 reserved1;
  UInt8 *rxBuff;
  UInt16 rxLen;
  IrPacket* packet;
  IrDeviceList* deviceList;
  IrStatus status;
  UInt8 reserved2;
} IrCallBackParms;
```

Field Descriptions

`event`	Event causing the callback.
`reserved1`	Reserved for future use.
`rxBuff`	Received data buffer.
`rxLen`	Length of data in received buffer.
`packet`	Pointer to packet being returned.
`deviceList`	Pointer to discovery device list.
`status`	Status of stack.
`reserved2`	Reserved for future use.

IrStatsType

The `IrStatsType` structure defines performance statistics for the IR Library. Use the `ExgControl` function with an `irGetStatistics` operation to retrieve these statistics. See IR Control Constants for more information.

```
typedef struct {
  UInt16 recLineErrors;
  UInt16 crcErrors;
} IrStatsType;
```

Field Descriptions

`recLineErrors` The number of serial errors since the library opened.

`crcErrors` The number of CRC errors since the library opened.

IR Library Constants

IR Control Constants

The IR control constants define operations that the IR Exchange Library can perform. You pass these constants as the operation parameter to `ExgControl`. The following table lists the operation constants, the data that should be passed as the `valueP` parameter to `ExgControl`, and what operation is performed in response.

Operation Constant	value Data Type	Description
`irGetScanningMode`	`Boolean`. Output only.	Returns `true` in `*valueP` if beaming is enabled or `false` if beaming is disabled.
`irGetStatistics`	`IrStatsType`. Output only.	Returns performance statistics.
`irRestoreScanning`	None	Re-enables beaming after an `irSuppressScanning` operation. This operation keeps track of the number of requests that beaming be disabled and re-enables beaming only when the count reaches 0.

Operation Constant	value Data Type	Description
		This operation differs from `irSetScanningMode` in that it does not update the saved preferences.
`irSetScanningMode`	`Boolean`. Input only.	Enables or disables beaming. This operation modifies the saved preferences database, which is back up during a HotSync® operation. Because of this, beaming may remain disabled after a reset if you use this operation to disable it. If you want to temporarily disable beaming use `irSuppressScanning` and `irRestoreScanning` instead.
`irSetBaudMask`	`UInt16` containing a mask of the `irOpenOptSpeed...` constants defined in `IrLib.h`. Input only.	Sets the possible baud rates that the IR Library will use to those specified in `*valueP`. OR the `irOpenOptSpeed...` constants together to specify more than one. The default rate is 0, which causes the baud rate to be determined by the hardware. This operation is sometimes useful for debugging connections. Generally, you should set all bits up to the fastest rate you want to allow. To reset, use this operation again and pass 0 in `*valueP`. If you change the baud rate, your changes are until the device is reset or you perform this operation again.

Operation Constant	value Data Type	Description
`irSetSerialMode`	`Boolean`. Input only.	If the specified value is `true`, the IR Library uses the serial port instead of the infrared port until the device is reset. This option is useful for debugging. You can run your application in POSE and use the IR Library to communicate with a device connected in the cradle.
`irSetSupported`	`Boolean`. Input only.	If `true`, IR is supported on this device. If `false`, IR is not supported. You can use this constant to disable the unsupported dialog that normally displays when a beam is attempted and no IR support is available.
`irSuppressScanning`	None	Temporarily disables beam receive. This operation keeps track of the number of requests that beaming be disabled and re-enables beaming (through `irRestoreScanning`) only when the count reaches 0. This operation differs from `irSetScanningMode` in that it does not update the saved preferences.

IR Stack Callback Events

The IR stack calls the application by way of a callback function stored in each `IrConnect` structure. The callback function is called with a pointer to the `IrConnect` structure and a pointer to a parameter structure. The parameter structure contains an `event` field, which indicates the reason the callback is called, and other parameters, which have meaning based on the event.

The meaning of the events is described in the following sections.

LEVENT_DATA_IND

Data has been received. The received data is accessed using fields `rxBuff` and `rxLen`.

LEVENT_DISCOVERY_CNF

Indicates the completion of a discovery operation. The field `deviceList` points to the discovery list.

LEVENT_LAP_CON_CNF

The requested IrLAP connection has been made successfully. The callback function of all bound `IrConnect` structures is called.

LEVENT_LAP_CON_IND

Indicates that the IrLAP connection has come up. The callback of all bound `IrConnect` structures is called.

LEVENT_LAP_DISCON_IND

Indicates that the IrLAP connection has gone down. This means that all IrLMP connections are also down. A callback with event `LEVENT_LM_CON_IND` is not given. The callback function of all bound `IrConnect` structures is called.

LEVENT_LM_CON_CNF

The requested IrLMP/Tiny TP connection has been made successfully. Connection data from the other side is found using fields `rxBuff` and `rxLen`.

LEVENT_LM_CON_IND

Other device has initiated a connection. `IrConnectRsp` should be called to accept the connection. Any data associated with the connection request can be found using fields `rxBuff` and `rxLen`, data pointer and length, respectively.

LEVENT_LM_DISCON_IND

The IrLMP/Tiny TP connection has been disconnected. Any data associated with the disconnect indication can be found using fields `rxBuff` and `rxLen`, data pointer and length, respectively.

LEVENT_PACKET_HANDLED

A packet is being returned. A pointer to the packet exists in field `packet`.

LEVENT_STATUS_IND

Indicates that a status event from the stack has occurred. The `status` field indicates the status generating the event. Possible status values are as follows:

- `IR_STATUS_NO_PROGRESS` which means that IrLAP has no progress for 3 seconds threshold time (for example, the beam is blocked).
- `IR_STATUS_LINK_OK` which indicates that the no progress condition has cleared.
- `IR_STATUS_MEDIA_NOT_BUSY` which indicates that the IR media has transitioned from busy to not busy.

LEVENT_TEST_CNF

Indicates that a TEST command has completed. The `status` field indicates if the test was successful.

- `IR_STATUS_SUCCESS` indicates that the operation was successful and the data in the test response can be found by using the `rxBuff` and `rxLen` fields.
- `IR_STATUS_FAILED` indicates that no TEST response was received. The packet passed to perform the test command is passed back in the `packet` field and is now available (no separate packet handled event occurs).

LEVENT_TEST_IND

Indicates that a TEST command frame has been received. A pointer to the received data is in `rxBuff` and `rxLen`. A pointer to the

packet that is sent in response to the test command is in the `packet` field. The packet is currently set up to respond with the same data sent in the command TEST frame. If different data is desired as a response, then you need to modify the packet structure. This event is sent to the callback function in all bound `IrConnect` structures. The IAS connections ignore this event.

IR Library Functions

IrAdvanceCredit

Purpose Advances the credit to the other side of the connection.

Prototype
```
void IrAdvanceCredit (IrConnect* con,
UInt8 credit)
```

Parameters

--> `con`	Pointer to `IrConnect` structure representing connection to which credit is advanced.
--> `credit`	Amount of credit to advance.

Result Returns nothing.

Comments The credit passed by this function is added to the existing available credit, which must not exceed 127. This function only makes sense for a Tiny TP connection.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrBind

Purpose Obtains a local LSAP selector and registers the connection with the protocol stack.

Prototype
```
IrStatus IrBind (UInt16 refNum, IrConnect* con,
IrCallBack callBack)
```

Parameters

--> `refnum`	IR library refNum.
<--> `con`	Pointer to `IrConnect` structure.
--> `callBack`	Pointer to a callBack function that handles the indications and confirmation from the protocol stack.

Result

`IR_STATUS_SUCCESS` means the operation completed successfully. The assigned LSAP can be found in `con->lLsap`.

`IR_STATUS_FAILED` means the operation failed for one of the following reasons:

- `con` is already bound to the stack.
- There is no room in the connection table.

Comments

The `IrConnect` structure is re-initialized. Any values stored in the structure are lost. The assigned LSAP is returned in the `lLsap` field of `con`. The type of the connection is set to IrLMP. The `IrConnect` must be bound to the stack before it can be used.

Compatibility

Implemented only if 3.0 New Feature Set is present.

IrClose

Purpose

Closes the IR library. This releases the global memory for the IR stack and any system resources it uses. This must be called when an application is done with the IR library.

Prototype

```
Err IrClose (UInt16 refnum)
```

Parameters

--> `refnum`	IR library refNum.

Result

Returns 0 if successful.

Comments

Do not call this function unless the call to `IrOpen` was successful.

Compatibility

Implemented only if 3.0 New Feature Set is present.

IrConnectIrLap

Purpose Starts an IrLAP connection.

Prototype
```
IrStatus IrConnectIrLap (UInt16 refNum,
IrDeviceAddr deviceAddr)
```

Parameters

--> `refnum`	IR library refNum.
--> `deviceAddr`	32-bit address of device to which connection should be made.

Result `IR_STATUS_PENDING` means the operation started successfully; the result is returned by way of a callback.

`IR_STATUS_MEDIA_BUSY` means the operation failed because the media is busy. Media busy is caused by one of the following reasons:

- Other devices are using the IR medium.
- An IrLAP connection already exists.
- A discovery process is in progress.

Comments The result is signaled to all bound `IrConnect` structures by way of the callback function. The callback event is `LEVENT_LAP_CON_CNF` if successful or `LEVENT_LAP_DISCON_IND` if unsuccessful.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrConnectReq

Purpose Requests an IrLMP or Tiny TP connection.

Prototype
```
IrStatus IrConnectReq (UInt16 refNum,
IrConnect* con, IrPacket* packet, UInt8 credit)
```

Parameters

--> `refnum`	IR library refNum.

--> `con`	Pointer to `IrConnect` structure for handling the connection. The `rLsap` field must contain the LSAP selector for the peer on the other device. Also the type of the connection must be set. Use `IR_SetConTypeLMP` to set the type to an IrLMP connection or `IR_SetConTypeTTP` to set the type to a Tiny TP connection.
--> `packet`	Pointer to a packet that contains connection data. Even if no connection data is needed, the packet must point to a valid `IrPacket` structure. The packet is returned by way of the callback function with the `LEVENT_PACKET_HANDLED` event if no errors occur. The maximum size of the packet is `IR_MAX_CON_PACKET` for an IrLMP connection or `IR_MAX_TTP_CON_PACKET` for a Tiny TP connection.
--> `credit`	Initial amount of credit advanced to the other side. Must be less than 127. It is ANDed with 0x7f, so if it is greater than 127, unexpected results occur. This parameter is ignored if the connection is an IrLMP connection.

Result `IR_STATUS_PENDING` means the operation has been started successfully and the result is returned by way of the callback function with the event `LEVENT_LM_CON_CNF` if the connection is made or `LEVENT_LM_DISCON_IND` if connection fails. The packet is returned by way of the callback with the event `LEVENT_PACKET_HANDLED`.

`IR_STATUS_FAILED` means the operation failed because of one of the following reasons. Note that the packet is available immediately.

- The connection is busy (already involved in a connection).
- The `IrConnect` structure is not bound to the stack.
- The packet size exceeds maximum allowed.

`IR_STATUS_NO_IRLAP` means the operation failed because there is no IrLAP connection (the packet is available immediately).

Comments The result is signaled by way of the callback specified in the `IrConnect` structure. The callback event `LEVENT_LM_CON_CNF` indicates that the connection is up and `LEVENT_LM_DISCON_IND` indicates that the connection failed. Before calling this function the fields in the `con` structure must be properly set.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrConnectRsp

Purpose Accepts an incoming connection that has been signaled by way of the callback with the event `LEVENT_LM_CON_IND`.

Prototype

```
IrStatus IrConnectRsp (UInt16 refNum,
IrConnect* con, IrPacket* packet, UInt8 credit)
```

Parameters

--> `refnum`	IR library refNum.
--> `con`	Pointer to `IrConnect` structure.
--> `packet`	Pointer to a packet that contains connection data. Even if no connection data is needed, the packet must point to a valid `IrPacket` structure. The packet is returned by way of the callback with the `LEVENT_PACKET_HANDLED` event if no errors occur. The maximum size of the packet is `IR_MAX_CON_PACKET` for an IrLMP connection or `IR_MAX_TTP_CON_PACKET` for a Tiny TP connection.
--> `credit`	Initial amount of credit advanced to the other side. Must be less than 127. It is ANDed with 0x7f, so if it is greater than 127, unexpected results occur. This parameter is ignored if the connection is an IrLMP connection.

Result `IR_STATUS_PENDING` means the operation has been started successfully and the packet is returned by way of the callback function with the event `LEVENT_PACKET_HANDLED`.

IR_STATUS_FAILED means the operation failed because of one of the following reasons. Note that the packet is available immediately.

- The connection is not in the proper state to require a response.
- The IrConnect structure is not bound to the stack.
- The packet size exceeds the maximum allowed.

IR_STATUS_NO_IRLAP means the operation failed because there is no IrLAP connection (the packet is available immediately).

Comments IrConnectRsp can be called during the callback or later to accept the connection. The type of the connection must already have been set to IrLMP or Tiny TP before the LEVENT_LM_CON_IND event.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrDataReq

Purpose Sends a data packet.

Prototype

```
IrStatus IrDataReq (UInt16 refNum, IrConnect* con,
IrPacket* packet)
```

Parameters

--> refnum	IR library refNum.
--> con	Pointer to IrConnect structure that specifies the connection over which the packet should be sent.
--> packet	Pointer to a valid IrPacket structure that contains data to send. The packet should not exceed the maximum size found with IrMaxTxSize.

Result IR_STATUS_PENDING means the packet has been queued by the stack. The packet is returned by way of the callback with event LEVENT_PACKET_HANDLED.

`IR_STATUS_FAILED` means the operation failed because of one of the following reasons. Note that the packet is available immediately.

- The `IrConnect` structure is not bound to the stack.
- The packet size exceeds the maximum allowed.
- The `IrConnect` structure does not represent an active connection.

Comments The packet is owned by the stack until it is returned by way of the callback with event `LEVENT_PACKET_HANDLED`. The largest packet that can be sent is found by calling `IrMaxTxSize`.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrDisconnectIrLap

Purpose Disconnects an IrLAP connection.

Prototype

```
IrStatus IrDisconnectIrLap (UInt16 refNum)
```

Parameters --> `refnum` IR library refNum.

Result `IR_STATUS_PENDING` means the operation started successfully and all bound `IrConnect` structures are called back when complete.

`IR_STATUS_NO_IRLAP` means the operation failed because no IrLAP connection exists.

Comments When the IrLAP connection goes down, the callback of all bound `IrConnect` structures is called with event `LEVENT_LAP_DISCON_IND`.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrDiscoverReq

Purpose Starts an IrLMP discovery process.

Prototype
```
IrStatus IrDiscoverReq (UInt16 refNum,
IrConnect* con)
```

Parameters

--> `refnum`	IR library refNum.
--> `con`	Pointer to a bound `IrConnect` structure.

Result `IR_STATUS_PENDING` means the operation is started successfully; the result is returned by way of callback.

`IR_STATUS_MEDIA_BUSY` means the operation failed because the media is busy. Media busy is caused by one of the following reasons:

- Other devices are using the IR medium.
- A discovery process is already in progress.
- An IrLAP connection exists.

`IR_STATUS_FAILED` means the operation failed because the `IrConnect` structure is not bound to the stack.

Comments The result is signaled by way of the callback function specified in the `IrConnect` structure with the event `LEVENT_DISCOVERY_CNF`. Only one discovery can be invoked at a time.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIsIrLapConnected

Purpose Determines if an IrLAP connection exists.

Prototype
```
BOOL IrIsIrLapConnected (UInt16 refNum)
```

Parameters

--> `refnum`	IR library refNum.

Result `true` if IrLAP is connected, `false` otherwise.

Comments Only available if `IR_IS_LAP_FUNCS` is defined.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIsMediaBusy

Purpose Determines if the IR media is busy.

Prototype
```
BOOL IrIsMediaBusy (UInt16 refNum)
```

Parameters --> `refnum` IR library refNum.

Result `true` if IR media is busy, `false` otherwise.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIsNoProgress

Purpose Determines if IrLAP is not making progress.

Prototype
```
BOOL IrIsNoProgress (UInt16 refNum)
```

Parameters --> `refnum` IR library refNum.

Result `ture` if IrLAP is not making progress, `false` otherwise.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIsRemoteBusy

Purpose Determines if IrLAP of the other device is busy.

Prototype
```
BOOL IrIsRemoteBusy (UInt16 refNum)
```

Parameters --> `refnum` IR library refNum.

Result `ture` if IrLAP of the other device is busy, `false` otherwise.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrLocalBusy

Purpose Sets the IrLAP local busy flag.

Prototype
```
void IrLocalBusy (UInt16 refNum, BOOL flag)
```

Parameters

--> `refnum`	IR library refNum.
--> `flag`	Value (`true` or `false`) to set for local busy flag of IrLAP.

Result Returns nothing.

Comments If local busy is set to `true`, then the local IrLAP layer sends RNR (Receive Not Ready) frames to the other side indicating it cannot receive any more data. If the local busy is set to false, IrLAP is ready to receive frames.

The setting takes effect the next time IrLAP sends an RR (Receive Ready) frame. If IrLAP has data to send, the data is sent first, so it should be used carefully.

This function should not be used when using Tiny TP or when multiple connections exist.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrMaxRxSize

Purpose Returns the maximum size buffer that can be sent by the other device.

Prototype
```
UInt16 IrMaxRxSize (UInt16 refNum, IrConnect* con)
```

Parameters

--> `refnum`	IR library refNum.
--> `con`	Pointer to `IrConnect` structure that represents an active connection.

Result Returns the maximum size buffer that can be sent by the other device (maximum bytes that can be received). The value returned is only valid for active connections. The maximum size varies for each connection and is based on the negotiated IrLAP parameters and the type of the connection.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrMaxTxSize

Purpose Returns the maximum size allowed for a transmit packet.

Prototype
```
UInt16 IrMaxTxSize (UInt16 refNum, IrConnect* con)
```

Parameters

--> `refnum`	IR library refNum.
--> `con`	Pointer to `IrConnect` structure that represents an active connection.

Result Returns the maximum size allowed for a transmit packet. The value returned is only valid for active connections. The maximum size varies for each connection and is based on the negotiated IrLAP parameters and the type of the connection.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrOpen

Purpose Opens the IR library. This allocates the global memory for the IR stack and reserves the system resources it requires. This must be done before any other IR library calls are made.

Prototype
```
Err IrOpen (UInt16 refnum, UInt32 options)
```

Parameters

--> `refnum`	IR library refNum. This value is returned from the function `SysLibFind`, which you must call first to load the IR library.
--> `options`	Open options flags. See the Comments section for details.

Result Returns 0 if successful.

Comments The following flags can be specified for the `options` parameter to set the speed of the connection:

`irOpenOptSpeed115200`	Set to maximum negotiated baud rate.
`irOpenOptSpeed57600`	Set to 57600 bps (default if no flags given).
`irOpenOptSpeed9600`	Set to 9600 bps.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrSetConTypeLMP

Purpose Sets the type of the connection to IrLMP. This function must be called after the `IrConnect` structure is bound to the stack.

Prototype
```
void IrSetConTypeLMP (IrConnect* con)
```

Parameters --> `con` Pointer to `IrConnect` structure.

Result Returns nothing.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrSetConTypeTTP

Purpose Sets the type of the connection to Tiny TP. This function must be called after the `IrConnect` structure is bound to the stack.

Prototype
```
void IrSetConTypeTTP (IrConnect* con)
```

Parameters --> `con` Pointer to `IrConnect` structure.

Result Returns nothing.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrSetDeviceInfo

Purpose Sets the XID info string used during discovery to the given string and length.

Prototype
```
IrStatus IrSetDeviceInfo (UInt16 refNum,
UInt8 *info, UInt8 len)
```

Parameters

--> `refnum`	IR library refNum.
--> `info`	Pointer to array of bytes.
--> `len`	Number of bytes pointed to by `info`.

Result `IR_STATUS_SUCCESS` means the operation is successful.

`IR_STATUS_FAILED` means the operation failed because `info` is too big.

Comments The XID `info` string contains hints and the nickname of the device. The size cannot exceed `IR_MAX_DEVICE_INFO` bytes.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrTestReq

Purpose Requests a TEST command frame be sent in the NDM (Normal Disconnect Mode) state.

Prototype
```
IrStatus IrTestReq (UInt16 refNum,
IrDeviceAddr devAddr, IrConnect* con,
IrPacket* packet)
```

Parameters

--> `refnum`	IR library refNum.
--> `devAddr`	Address of device where TEST is sent. This address is not checked so it can be the broadcast address or 0.
--> `con`	Pointer to `IrConnect` structure specifying the callback function to call to report the result.

	--> `packet`	Pointer to an `IrPacket` structure that contains the data to send in the TEST command packet. The maximum size data that can be sent is `IR_MAX_TEST_PACKET`. Even if no data is to be sent, a valid packet must be passed.

Result

`IR_STATUS_PENDING` means the operation has been started successfully and the result is returned by way of the callback function with the event `LEVENT_TEST_CNF`. This is also the indication returning the packet.

`IR_STATUS_FAILED` means the operation failed because of one of the following reasons. Note that the packet is available immediately.

- The `IrConnect` structure is not bound to the stack.
- The packet size exceeds the maximum allowed.

`IR_STATUS_MEDIA_BUSY` means the operation failed because the media is busy or the stack is not in the NDM state (the packet is available immediately).

Comments

The result is signaled by way of the callback specified in the `IrConnect` structure. The callback event is `LEVENT_TEST_CNF` and the status field indicates the result of the operation. `IR_STATUS_SUCCESS` indicates success and `IR_STATUS_FAILED` indicates no response was received. A packet must be passed containing the data to send in the TEST frame. The packet is returned when the `LEVENT_TEST_CNF` event is given.

Compatibility

Implemented only if 3.0 New Feature Set is present.

IrUnbind

Purpose

Unbinds the `IrConnect` structure from the protocol stack, freeing its LSAP selector.

Prototype

```
IrStatus IrUnbind (UInt16 refNum, IrConnect* con)
```

Parameters

	--> `refnum`	IR library refNum.
	--> `con`	Pointer to `IrConnect` structure to unbind.

Result `IR_STATUS_SUCCESS` means the operation completed successfully.

`IR_STATUS_FAILED` means the operation failed for one of the following reasons:

- The `IrConnect` structure was not bound.
- The `lLsap` field contained an invalid number.

Compatibility Implemented only if 3.0 New Feature Set is present.

IAS Functions

This section describes the following functions and macros related to IAS database:

- `IrIAS_Add`
- `IrIAS_GetInteger`
- `IrIAS_GetIntLsap`
- `IrIAS_GetObjectID`
- `IrIAS_GetOctetString`
- `IrIAS_GetOctetStringLen`
- `IrIAS_GetType`
- `IrIAS_GetUserString`
- `IrIAS_GetUserStringCharSet`
- `IrIAS_GetUserStringLen`
- `IrIAS_Next`
- `IrIAS_Query`
- `IrIAS_SetDeviceName`
- `IrIAS_StartResult`

IrIAS_Add

Purpose Adds an IAS object to the IAS Database.

Prototype

```
IrStatus IrIAS_Add (UInt16 refNum,
IrIasObject* obj)
```

Parameters

--> `refnum`	IR library refNum.
--> `obj`	Pointer to an `IrIASObject` structure.

Result

`IR_STATUS_SUCCESS` means the operation is successful.

IR_STATUS_FAILED means the operation failed for one of the following reasons:

- There is no space in the database.
- An entry with the same class name already exists.
- The attributes of the object violate the IrDA Lite rules (attribute name exceeds `IR_MAX_IAS_NAME`, or attribute value exceeds `IR_MAX_IAS_ATTR_SIZE`).
- The class name exceeds `IR_MAX_IAS_NAME`.

Comments

The object is not copied, so the memory for the object must exist for as long as the object is in the database. The IAS database is designed to allow only objects with unique class names, and it checks for this. Class names and attributes names must not exceed `IR_MAX_IAS_NAME`. Also, attribute values must not exceed `IR_MAX_IAS_ATTR_SIZE`.

Compatibility

Implemented only if 3.0 New Feature Set is present.

IrIAS_GetInteger

Purpose

Macro to return an integer value, assuming that the current result item is of type `IAS_ATTRIB_INTEGER`. (Call `IrIAS_GetType` to determine the type of the current result item.)

Prototype

```
IrIAS_GetInteger (t)
```

Parameters

--> `t`	Pointer to an `IrIasQuery` structure.

Result

Integer value returned as a `UInt32`.

Compatibility

Implemented only if 3.0 New Feature Set is present.

IrIAS_GetIntLsap

Purpose Macro to return an integer value that represents an LSAP, assuming that the current result item is of type `IAS_ATTRIB_INTEGER`. (Call `IrIAS_GetType` to determine the type of the current result item.) Usually integer values returned in a query are LSAP selectors.

Prototype
```
IrIAS_GetIntLsap (t)
```

Parameters --> `t` Pointer to an `IrIasQuery` structure.

Result Integer value returned as a `UInt8`.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIAS_GetObjectID

Purpose Macro to return the unique object ID of the current result item.

Prototype
```
IrIAS_GetObjectID (t)
```

Parameters --> `t` Pointer to an `IrIasQuery` structure.

Result Returns the object ID as a `UInt16` type.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIAS_GetOctetString

Purpose Macro to return a pointer to an octet string, assuming that the current result item is of type IAS_ATTRIB_OCTET_STRING. (Call `IrIAS_GetType` to determine the type of the current result item.)

Prototype
```
IrIAS_GetOctetString (t)
```

Parameters --> `t` Pointer to an `IrIasQuery` structure.

Result Pointer to octet string of type `UInt8`.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIAS_GetOctetStringLen

Purpose Macro to return the length of an octet string, assuming that the current result item is of type `IAS_ATTRIB_OCTET_STRING`. (Call `IrIAS_GetType` to determine the type of the current result item.)

Prototype

```
IrIAS_GetOctetStringLen (t)
```

Parameters --> `t` Pointer to an `IrIasQuery` structure.

Result Length of octet string returned as a `UInt16`.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIAS_GetType

Purpose Macro to return the type of the current result item.

Prototype

```
IrIAS_GetType (t)
```

Parameters --> `t` Pointer to an `IrIasQuery` structure.

Result Type of result item, such as `IAS_ATTRIB_INTEGER`, `IAS_ATTRIB_OCTET_STRING` or `IAS_ATTRIB_USER_STRING`. The return value is of type `UInt8`.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIAS_GetUserString

Purpose Macro to return a pointer to a user string, assuming that the current result item is of type `IAS_ATTRIB_USER_STRING`. (Call `IrIAS_GetType` to determine the type of the current result item.)

Prototype
```
IrIAS_GetUserString (t)
```

Parameters

--> `t`	Pointer to an `IrIasQuery` structure.

Result Pointer to result string of type `UInt8`.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIAS_GetUserStringCharSet

Purpose Macro to return the character set of the user string, assuming that the current result item is of type `IAS_ATTRIB_USER_STRING`. (Call `IrIAS_GetType` to determine the type of the current result item.)

Prototype
```
IrIAS_GetUserStringCharSet (t)
```

Parameters

--> `t`	Pointer to an `IrIasQuery` structure.

Result Character set returned as an `IrCharSet` value.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIAS_GetUserStringLen

Purpose Macro to return the length of a user string, assuming that the current result item is of type `IAS_ATTRIB_USER_STRING`. (Call `IrIAS_GetType` to determine the type of the current result item.)

Prototype
```
IrIAS_GetUserStringLen (t)
```

Parameters

--> `t`	Pointer to an `IrIasQuery` structure.

Result Length of user string returned as a `UInt8` value.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIAS_Next

Purpose Moves the internal pointer to the next result item.

Prototype
```
UInt8* IrIAS_Next (UInt16 refNum,
IrIasQuery* token)
```

Parameters

--> `refnum`	IR library refNum.
--> `token`	Pointer to an `IrIasQuery` structure.

Result Pointer to the next result item, or 0 if there are no more items.

Comments This function returns a pointer to the start of the next result item. If the pointer is 0, then there are no more result items.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIAS_Query

Purpose Makes an IAS query of the IAS database of another device.

Prototype
```
IrStatus IrIAS_Query (UInt16 refNum,
IrIasQuery* token)
```

Parameters

--> `refnum`	IR library refNum.
--> `token`	Pointer to an `IrIasQuery` structure initialized as described in the Comments section.

Result `IR_STATUS_SUCCESS` means the operation is started successfully and the result is signaled by way of the callback function.

`IR_STATUS_FAILED` means the operation failed for one of the following reasons:

- The query exceeds `IR_MAX_QUERY_LEN`.
- The `result` field of `token` is 0.
- The `resultBufSize` field of `token` is 0.
- The `callback` field of `token` is 0.
- A query is already in progress.

`IR_STATUS_NO_IRLAP` means the operation failed because there is no IrLAP connection.

Comments

An IrLAP connection must exist to the other device. The IAS query token must be initialized as described below. The result is signaled by calling the callback function whose pointer exists in the `IrIasQuery` structure. Only one query can be made at a time.

The `IrIasQuery` structure passed in the `token` parameter must be initialized as follows:

- Assign a pointer to a callback function in which the result is signaled.
- Set `result` to point to a buffer large enough to hold the result of the query.
- Set `resultBufSize` to the size of the result buffer.
- Set `queryBuf` to point to a valid query.
- Set `queryLen` to the number of bytes in `queryBuf`. The length must not exceed `IR_MAX_QUERY_LEN`.

Compatibility

Implemented only if 3.0 New Feature Set is present.

IrIAS_SetDeviceName

Purpose

Sets the `value` field of the device name attribute of the "Device" object in the IAS database.

Prototype

```
IrStatus IrIAS_SetDeviceName (UInt16 refNum,
UInt8 *name, UInt8 len)
```

Parameters

--> `refnum`	IR library refNum.
--> `name`	Pointer to an IAS `value` field for the device name attribute of the device object. It includes the attribute type, character set and device name. This `value` field should be a constant and the pointer must remain valid until `IrIAS_SetDeviceName` is called with another pointer.
--> `len`	Total length of the `value` field. Maximum size allowed is `IR_MAX_IAS_ATTR_SIZE`.

Result `IR_STATUS_SUCCESS` means the operation is successful.

`IR_STATUS_FAILED` means `len` is too big, or the `value` field is not a valid user string.

Compatibility Implemented only if 3.0 New Feature Set is present.

IrIAS_StartResult

Purpose Macro to put the internal pointer to the start of the result buffer.

Prototype

```
IrIAS_StartResult (t)
```

Parameters

--> `t`	Pointer to an `IrIasQuery` structure.

Result Returns nothing.

Compatibility Implemented only if 3.0 New Feature Set is present.

Application-Defined Functions

The functions in this section are supplied by the developer and can be named anything.

IrIasQueryCallBack

Purpose Signals the result of IAS query. The result of IAS queries is signaled by calling this callback function which is pointed to by the `callBack` field of the `IrIasQuery` structure.

Prototype

```
void IrIasQueryCallBack (IrStatus status)
```

Parameters

`--> status` The status of the query operation. The following values can be passed:

`IR_STATUS_SUCCESS` means the query operation finished successfully and the results can be parsed.

`IR_STATUS_DISCONNECT` means the link or IrLMP connection was disconnected during the query, so the results are not valid.

Result Returns nothing.

61

Modem Manager

This chapter provides reference material for the modem manager API. The header file `ModemMgr.h` declares the modem manager API.

Modem Manager Functions

MdmDial

Purpose Initialize the modem, dial the phone number and wait for result.

Prototype

```
Err MdmDial (MdmInfoPtr modemP, Char *okDialP,
Char *userInitP, Char *phoneNumP)
```

Parameters

`modemP`	Pointer to modem info structure (filled in by caller)
`okDialP`	(NOT IMPLEMENTED) Pointer to string of chars allowed in dial string
`userInitP`	Pointer to modem setup string without the AT prefix.
`phoneNumP`	Pointer to phone number string

Result 0 if successful; otherwise `mdmErrNoTone`, `mdmErrNoDCD`, `mdmErrBusy`, `mdmErrUserCan`, `mdmErrCmdError`

Comments When executing this function, the system performs these steps:

- Switch to the requested initial baud rate.
- If HW hand-shake is requested, enable CTS/RTS hand-shaking; otherwise, disable it.
- Reset the modem.

- Execute the setup string (if any).
- Configure the modem with required settings.
- Dial the phone number.
- Wait for CONNECT XXXXX or other response.
- If auto-baud is requested, switch to the connected baud rate.

MdmHangUp

Purpose Hang up the modem.

Prototype

```
Err MdmHangUp (MdmInfoPtr modemP)
```

Parameters `modemP` Pointer to modem info structure (filled in by caller)

Result 0 if successful.

WARNING! This function alters configuration of the serial port (without restoring it).

62

Net Library

This chapter describes the API available in the net library and its Berkeley sockets equivalents. The header file `NetMgr.h` declares the net library API. The chapter covers:

- Net Library Data Structures
- Net Library Constants
- Net Library Functions

For more information on the net library, see the chapter "Network Communication" in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

IMPORTANT: Applications cannot directly use the net library to make wireless connections. Use the INetLib for wireless connections.

Net Library Data Structures

NetHostInfoBufType

The `NetHostInfoBufType` struct contains information about a host. The `NetHostInfoType` struct, which maps to the `hostent` struct, points to fields in this struct for its information.

```
typedef struct {
  NetHostInfoType  hostInfo;
  Char          name[netDNSMaxDomainName+1];
  Char          *aliasList[netDNSMaxAliases+1];
  Char         aliases[netDNSMaxAliases]
                  [netDNSMaxAliases+1];
  NetIPAddr     *addressList[netDNSMaxAddresses];
  NetIPAddr     address[netDNSMaxAddresses];
} NetHostInfoBufType, *NetHostInfoBufPtr;
```

Field Descriptions

`hostInfo`	A `NetHostInfoType` struct, which maps to the Berkeley UNIX sockets `hostent` structure.
`name`	Official host name.
`aliasList` `aliases`	An array of aliases for the host name.
`addressList` `address`	An array of pointers to 32-bit IP addresses in host byte order.

NetHostInfoType

The `NetHostInfoType` structure maps to the Berkeley UNIX sockets `hostent` structure. It is defined as follows:

```
typedef struct {
  Char       *nameP;
  Char       **nameAliasesP;
  UInt16     addrType;
  UInt16     addrLen;
  UInt8      **addrListP;
} NetHostInfoType, *NetHostInfoPtr;
```

Field Descriptions

`nameP`	Official host name.
`nameAliasesP`	An array of aliases for the host name.
`addrType`	The type of the return addresses. See `NetSocketAddrEnum`.
`addrLen`	The length in bytes of the return addresses.
`addrListP`	An array of pointers to addresses in host byte order.

NetServInfoBufType

The `NetServInfoBufType` struct contains information about a service. The `NetServInfoType` struct, which maps to the `servent` struct, points to fields in this struct for much of its information.

```
struct {
  NetServInfoType servInfo;
  Char            name[netServMaxName+1];
  Char          *aliasList[netServMaxAliases+1];
  Char            aliases[netServMaxAliases]
[netServMaxName];
  Char            protoName[netProtoMaxName+1];
  UInt8           reserved;
} NetServInfoBufType, *NetServInfoBufPtr;
```

Field Descriptions

`servInfo`	A `NetServInfoType` struct, which maps to the Berkeley UNIX sockets `servent` structure.
`name`	Official name of the service
`aliasList` `aliases`	Array of aliases for the service name.
`protoName`	Name of the protocol to use.
`reserved`	Reserved for future use.

NetServInfoType

The `NetServInfoType` structure maps to the `servent` structure in Berkeley UNIX sockets API. It contains information about a service.

```
struct {
  Char       *nameP;
  Char       **nameAliasesP;
  UInt16     port;
  Char       *protoP;
} NetServInfoType, *NetServInfoPtr;
```

Field Descriptions

`nameP`	Official name of the service
`nameAliasesP`	Array of aliases for the service name.
`port`	Port number for the service.
`protoP`	Name of the protocol to use.

NetSocketAddrEnum

The `NetSocketAddrEnum` enum specifies the address types supported by the net library.

```
typedef enum {
  netSocketAddrRaw = 0,
  netSocketAddrINET = 2
} NetSocketAddrEnum
```

Value Descriptions

`netSocketAddrRaw`	Raw address. Supported in Palm OS® version 3.0 and higher.
`netSocketAddrINET`	IP address.

NetSocketAddrINType

The `NetSocketAddrINType` struct holds an internet socket address, that is, a socket that uses one of the internet protocols. This structure directly maps to the BSD UNIX `sockaddr_in` structure.

```
typedef struct NetSocketAddrINType {
  Int16      family;
  UInt16     port;
  NetIPAddr  addr;
} NetSocketAddrINType;
```

Field Descriptions

`family` Address family in host byte order. This is either `netSocketAddrINET` or `netSocketAddrRaw`.

`port` The port in network byte order.

`addr` The IP address in network byte order.

NetSocketAddrRawType

The `NetSocketAddrRawType` structure holds a raw socket address.

```
typedef struct NetSocketAddrRawType {
  Int16 family;
  UInt16 ifInstance;
  UInt32 ifCreator;
} NetSocketAddrRawType;
```

Field Descriptions

`family`	Address family in host byte order. This is either `netSocketAddrINET` or `netSocketAddrRaw`.
`ifInstance`	The instance number of the interface that the socket uses to send and receive data.
`ifCreator`	The creator of the interface that the socket uses.

Compatibility Raw sockets are supported in Palm OS version 3.0 and higher.

NetSocketAddrType

The `NetSocketAddrType` structure holds a generic socket address. This struct can hold any type of address including Internet addresses. It directly maps to the BSD UNIX `sockaddr` structure.

Note that this structure is the same size as `NetSocketAddrINType` and `NetSocketAddrRawType`. This means that one of those two structures can be used for parameters declared to be `NetSocketAddrType`.

```
typedef struct NetSocketAddrType {
  Int16 family;
  UInt8 data[14];
} NetSocketAddrType;
```

NetSocketRef

The `NetSocketRef` defines a socket descriptor. The socket descriptor is created and returned by `NetLibSocketOpen`. It is used in any function that requires access to a socket.

```
typedef Int16 NetSocketRef
```

NetSocketTypeEnum

The `NetSocketTypeEnum` enum specifies the available socket types.

```
typedef enum {
  netSocketTypeStream=1,
  netSocketTypeDatagram=2,
  netSocketTypeRaw=3,
  netSocketTypeReliableMsg=4
} NetSocketTypeEnum
```

Value Descriptions

`netSocketTypeStream`	Streams protocol over wireline.
`netSocketTypeDatagram`	UDP protocol.
`netSocketTypeRaw`	Raw mode.

Net Library Constants

I/O Flags

The I/O flags specify special handling instructions to functions that send and receive data. You can OR these values together to specify more than one.

`netIOFlagOutOfBand`	Process out-of-band data. Available for send calls only.
`netIOFlagPeek`	Peek at incoming message without dequeuing it.
`netIOFlagDontRoute`	Send without using routing. This constant is currently ignored.

Tracing Bits

The tracing bits are used to set the level of event tracing. An application can get a list of events in the trace buffer using the `NetLibMaster` call.

You can set the tracing for each network interface using `NetLibIFSettingSet` and for the net library in general with `NetLibSettingSet`.

`netTracingErrors`	Record run-time errors. This is the default.
`netTracingMsgs`	Record application trace messages.
`netTracingPkts`	Record packets I/O.
`netTracingFuncs`	Record function flow.
`netTracingAppMsgs`	Record application messages sent using `NetLibTracePrintF` and `NetLibTracePutS`.

Net Library Functions

NetHToNL

Purpose Macro that converts a 32-bit value from host to network byte order.

Prototype
```
NetHToNL (x)
```

Parameters

-> `x`	32-bit value to convert.

Result Returns `x` in network byte order.

Sockets Equivalent
```
htonl()
```

See Also NetNToHS, NetNToHL, NetHToNS

NetHToNS

Purpose Macro that converts a 16-bit value from host to network byte order.

Prototype
```
NetHToNS (x)
```

Parameters

-> `x`	16-bit value to convert.

Result Returns `x` in network byte order.

Sockets Equivalent
```
htons()
```

See Also NetNToHS, NetNToHL, NetHToNL

NetLibAddrAToIN

Purpose Converts an ASCII string representing a dotted decimal IP address into a 32-bit IP address in network byte order.

Prototype
```
NetIPAddr NetLibAddrAToIN (UInt16 libRefnum,
const Char *a)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `a`	Pointer to ASCII dotted decimal string.

Result Returns a 32-bit network byte order IP address or -1 if `a` doesn't represent a dotted decimal IP address

Sockets Equivalent
```
UInt32 inet_addr (char *cp)
```

See Also NetLibAddrINToA

NetLibAddrINToA

Purpose Converts an IP address from 32-bit network byte order into a dotted decimal ASCII string.

Prototype
```
Char *NetLibAddrINToA (UInt16 libRefnum,
NetIPAddr inet, Char *spaceP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `inet`	32-bit IP address in network byte order.
<- `spaceP`	Buffer used to hold the return value.

Result Returns in `spaceP` the dotted decimal ASCII string representation of the IP address.

Sockets Equivalent
```
char *inet_ntoa (struct in_addr in)
```

See Also NetLibAddrAToIN

NetLibClose

Purpose Closes the net library.

Prototype

```
Err NetLibClose (UInt16 libRefnum,
UInt16 immediate)
```

Parameters

-> `libRefnum`	Reference number of the net library.
-> `immediate`	If `true`, library will shut down immediately. If `false`, library will shut down only if close timer expires before another `NetLibOpen` is issued.

Result Returns one of the following values:

0	Success.
`netErrNotOpen`	Library was not open.
`netErrStillOpen`	Not really an error; returned if library is still in use by another application.

Sockets Equivalent None.

Comments Applications must call this function when they no longer need the net library. If the net library open count is greater than 1 before this call is made, the count is decremented and `netErrStillOpen` is returned. If the open count was 1, the library takes the following action:

- If `immediate` is `true`, the library shuts down immediately. All network interfaces are brought down, the net protocol stack task is terminated, and all memory used by the net library is freed.
- If `immediate` is `false`, a close timer is created and this call returns immediately without actually bringing the net library down. Instead it leaves it up and running but marks it as in the "close-wait" state. It remains in this state until either the timer expires or another `NetLibOpen` is issued. If the timer

expires, the library is shut down. If another `NetLibOpen` call is issued before the timer expires (possibly by another application), the timer is cancelled and the library is marked as fully open.

In most cases, you should pass `false` for `immediate`. This allows the user to quit one Internet application and launch another within a short period of time without having to wait through the process of closing down and then re-establishing dial-up network connections.

See Also `NetLibOpen`, `NetLibOpenCount`

NetLibConnectionRefresh

Purpose This routine is a convenience call for applications. It checks the status of all connections and optionally tries to open any that were closed.

Prototype

```
Err NetLibConnectionRefresh (UInt16 refNum,
Boolean refresh, UInt8 *allInterfacesUpP,
UInt16 *netIFErrP)
```

Parameters

`-> refnum`	Reference number of the net library.
`-> refresh`	If `true`, any connections that aren't currently open are opened.
`<- allInterfacesUpP`	Set to `true` if all connections are open.
`<- netIFErrP`	First error encountered when reopening connections that were closed. (See `NetLibIFUp` for a list of possible values.)

Result Returns one of the following values:

0	Success.
`netErrBufTooSmall`	
`netErrOutOfCmdBlocks`	
`netErrNoInterfaces`	

Sockets Equivalent None.

Comments This function determines whether a connection is up based on the internal status of the TCP/IP stack. To test the presence of a "physical connection" (phone line, modem, serial cable), a command should be sent once it's been determined that the logical connection is up. If the physical connection is broken, nothing returns and a timeout error eventually occurs.

NetLibDmReceive

Purpose Receive data from a socket directly into a database record.

Prototype

```
Int16 NetLibDmReceive (UInt16 libRefNum,
NetSocketRef socket, void *recordP,
UInt32 recordOffset, UInt16 rcvLen, UInt16 flags,
void *fromAddrP, UInt16 *fromLenP, Int32 timeout,
Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.
<- `recordP`	Pointer to beginning of record to receive data into. Must be locked for use.
-> `recordOffset`	Offset from beginning of record to read data into.
-> `rcvLen`	Maximum number of bytes to read.
-> `flags`	One or more `netIOFlagxxx` flags. See "I/O Flags."
<- `fromAddrP`	Pointer to buffer to hold address of sender (a `NetSocketAddrType` struct). Pass `NULL` if you don't need sender information.
<-> `fromLenP`	On entry, size of `fromAddrP` buffer. On exit, actual size of returned address in `fromAddrP`. Pass `NULL` if you don't need sender information.

-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns the number of bytes successfully received. If the return value is 0, the socket has been shut down by the remote host. If the return value is -1, an error has occurred and `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrWouldBlock`	
`netErrUserCancel`	
`netErrOutOfMemory`	

Comments This call behaves similarly to `NetLibReceive` but reads the data directly into a database record, which is normally write-protected. The caller must pass a pointer to the start of the record and an offset into the record of where to start the read.

NetLibFinishCloseWait

Purpose Forces the net library to do a complete close if it's currently in the close-wait state.

Prototype
```
Err NetLibFinishCloseWait (UInt16 libRefnum)
```

Parameters

-> `libRefnum`	Reference number of the net library.

Result Returns one of the following values:

0	Success.
`netErrTimeout`	

Sockets Equivalent None.

Comments This call checks the current open state of the net library. If it's in the close-wait state (see NetLibClose), it forces the library to perform an immediate, complete close operation.

NetLibGetHostByAddr

Purpose Looks up a host name given its IP address.

Prototype

```
NetHostInfoPtr NetLibGetHostByAddr
(UInt16 libRefNum, UInt8 *addrP, UInt16 len,
UInt16 type, NetHostInfoBufPtr bufP,
Int32 timeout, Err *errP)
```

Parameters

-> libRefNum	Reference number of the net library.
-> addrP	IP address of host to lookup.
-> len	Length, in bytes, of *addrP.
-> type	Type of addrP. See NetSocketAddrEnum.
<- bufP	Pointer to a NetHostInfoBufType struct in which to store the results of the lookup.
-> timeout	Maximum timeout in system ticks; -1 means wait forever.
<- errP	Contains an error code if the return value is 0.

Result Returns a pointer to the NetHostInfoType portion of bufP that contains results of the lookup. If the return value is 0, an error has occurred, and errP contains one of the following values:

0	No error
netErrTimeout	Call timed out.
netErrNotOpen	The referenced net library has not been opened yet.
netErrDNSNameTooLong	
netErrDNSBadName	

```
netErrDNSLabelTooLong
netErrDNSAllocationFailure
netErrDNSTimeout
netErrDNSUnreachable
netErrDNSFormat
netErrDNSServerFailure
netErrDNSNonexistantName
netErrDNSNIY
netErrDNSRefused
netErrDNSImpossible
netErrDNSNoRRS
netErrDNSAborted
netErrDNSBadProtocol
netErrDNSTruncated
netErrDNSNoRecursion
netErrDNSIrrelevant
netErrDNSNotInLocalCache
netErrDNSNoPort
```

Sockets Equivalent

```
struct hostent *gethostbyaddr (char *addr,
int len, int type);
```

Comments

This call queries the domain name server(s) to look up a host name given its IP address.

See Also

NetLibGetHostByName

NetLibGetHostByName

Purpose Looks up a host IP address given a host name.

Prototype

```
NetHostInfoPtr NetLibGetHostByName
(UInt16 libRefNum, const Char *nameP,
NetHostInfoBufPtr bufP, Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `nameP`	Name of host to look up.
<- `bufP`	Pointer to a `NetHostInfoBufType` struct in which to store the results of the lookup.
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is 0.

Result Returns a pointer to the `NetHostInfoType` portion of `bufP`, which contains results of the lookup. If the return value is 0, an error has occurred and `errP` contains one of the following values:

0	No error
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrDNSNameTooLong`	
`netErrDNSBadName`	
`netErrDNSLabelTooLong`	
`netErrDNSAllocationFailure`	
`netErrDNSTimeout`	
`netErrDNSUnreachable`	
`netErrDNSFormat`	
`netErrDNSServerFailure`	
`netErrDNSNonexistantName`	

```
netErrDNSNIY
netErrDNSRefused
netErrDNSImpossible
netErrDNSNoRRS
netErrDNSAborted
netErrDNSBadProtocol
netErrDNSTruncated
netErrDNSNoRecursion
netErrDNSIrrelevant
netErrDNSNotInLocalCache
netErrDNSNoPort
```

Sockets Equivalent

```
struct hostent *gethostbyname (char *name);
```

Comments This call first checks the local name -> IP address host table in the net library preferences. If the entry is not found, it then queries the domain name server(s).

See Also NetLibGetHostByAddr, NetLibGetMailExchangeByName

NetLibGetMailExchangeByName

Purpose Looks up the name of a host to use for a given mail exchange.

Prototype

```
Int16 NetLibGetMailExchangeByName
(UInt16 libRefNum, Char *mailNameP,
UInt16 maxEntries, Char hostNames[][255+1],
UInt16 priorities[], Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `mailNameP`	Name of the mail exchange to look up.
-> `maxEntries`	Maximum number of host names to return.

<- hostNames	Array of character strings of length 255+1. The host name results are stored in this array. This array must be able to hold at least maxEntries host names.
<- priorities	Array of Words. The priorities of each host name found are stored in this array. This array must be at least maxEntries in length.
-> timeout	Maximum timeout in system ticks; -1 means wait forever.
<- errP	Contains an error code if the return value is less than 0.

Result Returns the number of entries successfully found. If the return value is a negative number, an error has occurred, and errP contains one of the following values:

0	No error
netErrTimeout	Call timed out.
netErrNotOpen	The referenced net library has not been opened yet.

netErrDNSNameTooLong

netErrDNSBadName

netErrDNSLabelTooLong

netErrDNSAllocationFailure

netErrDNSTimeout

netErrDNSUnreachable

netErrDNSFormat

netErrDNSServerFailure

netErrDNSNonexistantName

netErrDNSNIY

netErrDNSRefused

netErrDNSImpossible

netErrDNSNoRRS

`netErrDNSAborted`

`netErrDNSBadProtocol`

`netErrDNSTruncated`

`netErrDNSNoRecursion`

`netErrDNSIrrelevant`

`netErrDNSNotInLocalCache`

`netErrDNSNoPort`

Sockets Equivalent None

Comments This call looks up the name(s) of host(s) to use for sending an e-mail. The caller passes the name of the mail exchange in `mailNameP` and gets back a list of host names to which the mail message can be sent.

See Also NetLibGetHostByAddr, NetLibGetHostByName

NetLibGetServByName

Purpose Looks up the port number for a standard TCP/IP service, given the desired protocol.

Prototype

```
NetServInfoPtr NetLibGetServByName
(UInt16 libRefNum, const Char *servNameP,
const Char *protoNameP, NetServInfoBufPtr bufP,
Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `servNameP`	Name of the service to look up. Possible services are “echo”, “discard”, “daytime”, “qotd”, “chargen”, “ftp-data”, “ftp”, “telnet”, “smtp”, “time”, “name”, “finger”, “pop2”, “pop3”, “nntp”, “imap2”.
-> `protoNameP`	Desired protocol to use, either “udp” or “tcp”.

<- `bufP`	Pointer to a `NetServInfoBufType` struct in which to store the results of the lookup.
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is 0.

Result Returns a pointer to the `NetServInfoType` portion of `bufP` that contains results of the lookup. If the return value is 0, and error has occurred and `errP` contains one of the following values:

0	No error
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrUnknownProtocol`	
`netErrUnknownService`	

Sockets Equivalent

```
struct servent *getservbyname (char *addr,
char *proto);
```

Comments This call is a convenience call for looking up a standard port number given the name of a service and the protocol to use.

See Also `NetLibGetHostByName`

NetLibIFAttach

Purpose Attach a new network interface.

Prototype

```
Err NetLibIFAttach (UInt16 libRefNum,
UInt32 ifCreator, UInt16 ifInstance,
Int32 timeout)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `ifCreator`	Creator of interface to attach.

-> `ifInstance`	Instance number of interface to attach. The instance number is one of the values returned by `NetLibIFGet`.
-> `timeout`	Timeout in ticks; -1 means infinite timeout.

Result Returns one of the following values:

0	Success.
`netErrInterfaceNotFound`	
`netErrTooManyInterfaces`	

Sockets Equivalent None

Comments This call can be used to attach a new network interface to the net library. Network interfaces are self-contained databases of type `'neti'`. The `ifCreator` parameter to this function is used to locate the network interface database of the given creator.

If the net library is already open when this call is made, the network interface's database will be located and then called to initialize itself and attach itself to the protocol stack in real time. If the net library is not open when this call is made, the creator and instance number of the interface are stored in the net library's preferences database and the interface is initialized and attached to the stack the next time the net library is opened.

See Also `NetLibIFGet`, `NetLibIFDetach`

NetLibIFDetach

Purpose Detach a network interface from the protocol stack.

Prototype

```
Err NetLibIFDetach (UInt16 libRefNum,
UInt32 ifCreator, UInt16 ifInstance,
Int32 timeout)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `ifCreator`	Creator of interface to detach.
-> `ifInstance`	Instance number of interface to detach.
-> `timeout`	Timeout in ticks; -1 means infinite timeout.

Result

Returns one of the following values:

0	Success.
`netErrInterfaceNotFound`	

Sockets Equivalent

None

Comments

If the net library is already open when this call is made, the interface is brought down and detached from the protocol stack in real time. If the net library is not open when this call is made, the creator and instance number of the interface are removed in the net library's preferences database and the interface is not attached the next time the library is opened.

See Also

`NetLibIFGet`, `NetLibIFAttach`

NetLibIFDown

Purpose

Bring an interface down and hang up a connection.

Prototype

```
Err NetLibIFDown (UInt16 libRefNum,
UInt32 ifCreator, UInt16 ifInstance,
Int32 timeout)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `ifCreator`	Creator of interface to attach.
-> `ifInstance`	Instance number of interface to attach.
-> `timeout`	Timeout in ticks; -1 means wait forever.

Result Returns one of the following values:

0	Success.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrInterfaceNotFound`	

Sockets Equivalent None

Comments The net library must be open before this call can be made. For dial-up interfaces, this call terminates a connection and hangs up the modem if necessary.

`NetLibClose` automatically brings down any attached interfaces, so this routine doesn't normally have to be called.

If the interface is already down, this routine returns immediately with no error.

See Also `NetLibIFGet`, `NetLibIFAttach`, `NetLibIFDetach`, `NetLibIFUp`

NetLibIFGet

Purpose Get the creator and instance number of an installed interface by index.

Prototype

```
Err NetLibIFGet (UInt16 libRefNum, UInt16 index,
UInt32 *ifCreatorP, UInt16 *ifInstanceP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `index`	Index of the interface to get. Indices start at 0.
<- `ifCreatorP`	The interface's creator.
<- `ifInstanceP`	The interface's instance number.

Result Returns one of the following values:

0	Success.
`netErrInvalidInterface`	Index too high
`netErrPrefNotFound`	No current value for setting.

Sockets Equivalent None

Comments To get a list of all installed interfaces, call this function with successively increasing indices starting from 0 until the error `netErrInvalidInterface` is returned.

The `ifCreator` and `ifInstance` values returned from this call can then be used with the `NetLibSettingGet` call to get more information about that particular interface.

See Also `NetLibIFAttach`, `NetLibIFDetach`, "Settings for Interface Selection" in the *Palm OS Programmer's Companion*, vol. II, *Communications*

NetLibIFSettingGet

Purpose Retrieves a network interface specific setting.

Prototype

```
Err NetLibIFSettingGet (UInt16 libRefNum,
UInt32 ifCreator, UInt16 ifInstance,
UInt16 setting, void *valueP, UInt16 *valueLenP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `ifCreator`	Creator of the network interface.
-> `ifInstance`	Instance number of the network interface.
-> `setting`	Setting to retrieve; one of the `NetIFSettingEnum` constants.

<- `valueP`	Space for return value of setting.
<-> `valueLenP`	On entry, size of `valueP`. On exit, actual size of setting.

Result Returns one of the following values:

0	Success.
`netErrUnknownSetting`	Invalid setting constant.
`netErrPrefNotFound`	No current value for setting.
`netErrBufTooSmall`	`valueP` was too small to hold entire setting. Setting value was truncated to fit in `valueP`.
`netErrUnimplemented`	
`netErrInterfaceNotFound`	
`netErrBufWrongSize`	

Sockets Equivalent None

Comments This call can be used to retrieve the current value of any network interface setting. The caller must pass a pointer to a buffer to hold the return value (`valueP`), the size of the buffer (`*valueLenP`), and the setting ID (`setting`). The setting ID is one of the constants in the `NetIFSettingEnum` type.

Some settings, such as the login script, are variable size. For these types of settings, you can obtain the actual size required for the buffer by passing 0 for `*valueLenP`. The required size is returned in `valueLenP`.

Table 62.1 lists the network interface settings and the size of each setting. Some are only applicable to certain types of interfaces. Settings not applicable to a specific interface can be safely ignored and not set to any particular value.

Table 62.1 Network Interface Settings

netIFSetting...	Type	Description
`ResetAll`	`void`	Use with `NetLibIFSettingSet` only. This clears all other settings for the interface to their default values.
`Up`	`UInt8`	Read-only. `true` if interface is currently up.
`Name`	`Char[32]`	Read-only. Name of this interface.
`ReqIPAddr`	`UInt32`	IP address of interface.
`SubnetMask`	`UInt32`	Subnet mask for interface. Doesn't need to be specified for PPP or SLIP type connections.
`Broadcast`	`UInt32`	Broadcast address for interface. Doesn't need to be specified for PPP or SLIP type connections.
`Username`	`Char[32]`	User name. Only required if the login script uses the user name substitution escape sequence in it. Call `NetLibIFSettingSet` with a `valueLen` of 0 to remove this setting.
`Password`	`Char[32]`	Password. Only required if the login script uses the password substitution escape sequence in it. Call `NetLibIFSettingSet` with a `valueLen` of 0 to remove this setting. If the login script uses password substitution and no password setting is set, the user will be prompted for a password at connect time.
`AuthUsername`	`Char[32]`	Authentication user name. Only required if the authentication protocol uses a different user name than the what's in the `netIFSettingUsername` setting. If this setting is empty (`valueLen` of 0), the Username setting will be used instead. Call `NetLibIFSettingSet` with a `valueLen` of 0 to remove this setting.

Table 62.1 Network Interface Settings *(continued)*

netIFSetting...	Type	Description
`AuthPassword`	`Char[32]`	Authentication password. If "$" then the user will be prompted for the authentication password at connect time. Else, if 0 length, then the `netIFSettingPassword` setting or the result of its prompt will be used instead. Call `NetLibIFSettingSet` with a `valueLen` of 0 to remove this setting.
`ServiceName`	`Char[]`	Service name. Used for display purposes while showing the connection progress dialog box. Call `NetLibIFSettingSet` with a `valueLen` of 0 to remove this setting.
`LoginScript`	`Char[]`	Login script. Only required if the particular service requires a login sequence. Call `NetLibIFSettingSet` with a `valueLen` of 0 to remove this setting. See below for a description of the login script format.
`ConnectLog`	`Char[]`	Connect log. Generally, this setting is just retrieved, not set. It contains a log of events from the most recent login. To clear this setting, call `NetLibIFSettingSet` with a `valueLen` of 0.
`InactivityTimeout`	`UInt16`	Maximum number of seconds of inactivity allowed. Set to 0 to ignore.
`EstablishmentTimeout`	`UInt16`	Maximum delay, in seconds, allowed between each stage of connection establishment or login script line. Must be non-zero.
`DynamicIP`	`UInt8`	If non-zero, negotiate for an IP address. If `false`, the IP address specified in the `netIFSettingReqIPAddr` setting will be used. Default is `false`.
`VJCompEnable`	`UInt8`	If non-zero, enable VJ header compression. Default is `true` for PPP, `false` for SLIP, and `true` for CSLIP.

Table 62.1 Network Interface Settings *(continued)*

netIFSetting...	Type	Description
`VJCompSlots`	`UInt8`	Number of slots to use for VJ compression. Default is 4 for PPP and 16 for SLIP and CSLIP. More slots require more memory so it is best to keep this number to a minimum.
`MTU`	`UInt16`	Maximum transmission unit in octets. Currently not implemented in SLIP or PPP.
`AsyncCtlMap`	`UInt32`	Bit mask of characters to escape for PPP. Default is 0.
`PortNum`	`UInt16`	Which serial communication port to use. Port 0 is the only port available on the device.
`BaudRate`	`UInt32`	Serial port baud rate to use in bits per second.
`FlowControl`	`UInt8`	If bit 0 is 1, use hardware handshaking on the serial port. Default is no hardware handshaking.
`StopBits`	`UInt8`	Number of stop bits. Default is 1.
`ParityOn`	`UInt8`	`true` if parity detection enabled. Default is `false`.
`ParityEven`	`UInt8`	`true` for even parity detection. Default is `true`.
`UseModem`	`UInt8`	If `true`, dial-up through modem. If `false`, go direct over serial port
`PulseDial`	`UInt8`	If `true`, pulse dial modem. Else, tone dial. Default is tone dial.
`ModemInit`	`Char[]`	Zero-terminated modem initialization string, not including the "AT". If not specified (`valueLen` of 0), the modem initialization string from system preferences are used.
`ModemPhone`	`Char[]`	Zero-terminated modem phone number string. Only required if `netIFSettingUseModem` is true.

Table 62.1 Network Interface Settings *(continued)*

netIFSetting...	Type	Description
`RedialCount`	`UInt16`	Number of times to re-dial modem when trying to establish a connection. Only required if `netIFSettingUseModem` is true.
`DNSQuery`	`UInt8`	`true` if PPP queries for DNS address. The default is `true`.
`TraceBits`	`UInt32`	A bitfield of various trace bits. See "Tracing Bits." An application can get a list of events in the trace buffer using the `NetLibMaster` call. Each interface has its own trace bits setting so that trace event recording in each interface can be selectively enabled or disabled.
`ActualIPAddr`	`UInt32`	Read-only. The actual IP address that the interface ends up using. The login script execution engine stores the result of the "g" (get IP address) command here as does the PPP negotiation logic.
`ServerIPAddr`	`UInt32`	Read-only. The IP address of the PPP server we're connected to.
`BringDownOnPowerDown`	`UInt8`	`true` if the interface is brought down when the Palm OS device is turned off.
`RawMode`	`UInt32`	Specifies if the interface is in raw mode. The net library places an interface in raw mode when it is bound to a raw socket in the raw domain. Raw sockets are available in Palm OS version 3.0 and higher.

See Also `NetLibIFSettingSet`, `NetLibSettingGet`, `NetLibSettingSet`, "Interface Specific Settings" in the *Palm OS Programmer's Companion*, vol. II, *Communications*

NetLibIFSettingSet

Purpose Sets a network interface specific setting.

Prototype

```
Err NetLibIFSettingSet (UInt16 libRefNum,
UInt32 ifCreator, UInt16 ifInstance,
UInt16 setting, void *valueP, UInt16 valueLen)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `ifCreator`	Creator of the network interface.
-> `ifInstance`	Instance number of the network interface.
-> `setting`	The setting to set, one of the `NetIFSettingEnum` constants. See Table 62.1.
-> `valueP`	Space new value of setting.
-> `valueLen`	Size of new setting.

Result Returns one of the following values:

0	Success.
`netErrUnknownSetting`	Invalid setting constant.
`netErrPrefNotFound`	No current value for setting.
`netErrBufTooSmall`	`valueP` was too small to hold entire setting. Setting value was truncated to fit in `valueP`.
`netErrUnimplemented`	
`netErrInterfaceNotFound`	
`netErrBufWrongSize`	
`netErrReadOnlySetting`	

Sockets Equivalent None

Comments This call can be used to set the current value of any network interface setting. The caller must pass a pointer to a buffer which holds the new value (`valueP`), the size of the buffer (`valueLen`), and the setting ID (`setting`).

See `NetLibIFSettingGet` for an explanation of each of the settings.

Of particular interest is the `netIFSettingResetAll` setting, which, if used, resets all settings for the interface to their default values. When using this setting, `valueP` and `valueLen` are ignored.

See Also `NetLibIFSettingGet`, `NetLibSettingGet`, `NetLibSettingSet`, "Interface Specific Settings" in the *Palm OS Programmer's Companion*, vol. II, *Communications*

NetLibIFUp

Purpose Bring an interface up and establish a connection.

Prototype

```
Err NetLibIFUp (UInt16 libRefNum,
UInt32 ifCreator, UInt16 ifInstance)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `ifCreator`	Creator of interface to attach.
-> `ifInstance`	Instance number of interface to attach.

Result Returns one of the following values:

0	Success.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrInterfaceNotFound`	
`netErrUserCancel`	
`netErrBadScript`	
`netErrPPPTimeout`	

`netErrAuthFailure`

`netErrPPPAddressRefused`

Sockets Equivalent None

Comments The net library must be open before this call can be made. For dial-up interfaces, this call will dial up the modem if necessary and run through the connect script to establish the connection.

If the interface is already up, this routine returns immediately with no error. This call doesn't take a timeout parameter because it relies on each interface to have its own established timeout setting.

See Also `NetLibIFGet`, `NetLibIFAttach`, `NetLibIFDetach`, `NetLibIFDown`

NetLibMaster

Purpose Retrieves the network statistics, interface statistics, and the contents of the trace buffer.

Prototype

```
Err NetLibMaster (UInt16 libRefNum, UInt16 cmd,
NetMasterPBPtr pbP, Int32 timeout)
```

Parameters

-> `libRefNum`	Reference number of the net library.

-> `cmd`	Function to perform (`NetMasterEnum` type). The following commands are supported:
	`netMasterInterfaceInfo`
	`netMasterInterfaceStats`
	`netMasterIPStats`
	`netMasterICMPStats`
	`netMasterUDPStats`
	`netMasterTCPStats`
	`netMasterTraceEventGet`

<-> `pbP`	Command parameter block.
-> `timeout`	Timeout in ticks; -1 means wait forever.

Result Returns one of the following values:

0	No error
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrUnimplemented`	

Sockets Equivalent None

Comments This call allows applications to get detailed information about the net library. This information is usually helpful in debugging network configuration problems.

This function takes a command word (`cmd`) and parameter block pointer (`pbP`) as arguments and returns its results in the parameter block on exit. Which values you must specify in the parameter block and which values are returned are specific to the command you specify.

netMasterInterfaceInfo

The `pbP->interfaceInfo` struct specifies interface information.

`->`	`index`	Index of interface to fetch info about.
`<-`	`creator`	Creator of interface.
`<-`	`instance`	Instance of interface.
`<-`	`netIFP`	Private interface info pointer.
`<-`	`drvrName`	Driver type that interface uses (“PPP”, “SLIP”, etc.).
`<-`	`hwName`	Hardware driver name (“Serial Library”, etc.).
`<-`	`localNetHdrLen`	Number of bytes in local net header.
`<-`	`localNetTrailerLen`	Number of bytes in local net trailer.

<-	localNetMaxFrame	Local net maximum frame size.
<-	ifName	Interface name with instance number concatenated.
<-	driverUp	true if interface driver is up.
<-	ifUp	true if interface media layer is up.
<-	hwAddrLen	Length of interface's hardware address.
<-	hwAddr	Interface's hardware address.
<-	mtu	Maximum transfer unit of interface.
<-	speed	Speed in bits per second.
<-	lastStateChange	Time in milliseconds of last state change.
<-	ipAddr	IP address of interface.
<-	subnetMask	Subnet mask of local network.
<-	broadcast	Broadcast address of local network.

netMasterInterfaceStats

The pbP->interfaceStats structure specifies interface statistics.

->	index	Index of interface to fetch info about.
<-	inOctets	Number of octets received.
<-	inUcastPkts	Number of packets received.
<-	inNUcastPkts	Number of broadcast packets received.
<-	inDiscards	Number of incoming packets that were discarded.
<-	inErrors	Number of packet errors encountered.
<-	inUnknownProtos	Number of unknown protocols encountered.

<-	outOctets	Number octets sent.
<-	outUcastPkts	Number of packets sent.
<-	outNUcastPkts	Number of broadcast packets sent.
<-	outDiscards	Number of packets discarded.
<-	outErrors	Number of outbound packet errors.

netMasterIPStats

The pbP->ipStats structure contains statistics about the IP protocol. See NetMgr.h for a complete list of statistics returned.

netMasterICMPStats

The pbP->icmpStats structure contains statistics about the ICMP protocol. See NetMgr.h for a complete list of statistics returned.

netMasterUDPStats

The pbP->udpStats structure contains statistics about the UDP protocol. See NetMgr.h for a complete list of statistics returned.

netMasterTCPStats

The pbP->tcpStats structure contains statistics about the TCP protocol. See NetMgr.h for a complete list of statistics returned.

netMasterTraceEventGet

The pbP->traceEventGet structure contains a trace event.

->	index	Index of event to fetch.
<-	textP	Pointer to text string to return event in. Should be at least 256 bytes long.

See Also NetLibSettingSet

NetLibOpen

Purpose Opens and initializes the net library.

Prototype

```
Err NetLibOpen (UInt16 libRefnum,
UInt16 *netIFErrsP)
```

Parameters

-> `libRefnum` Reference number of the net library.

<- `netIFErrsP` First error encountered when bringing up network interfaces. (See `NetLibIFUp` for a list of possible values.)

Result Returns one of the following values:

0 No error.

`netErrAlreadyOpen`
Not really an error; returned if library was already open and the open count was simply incremented.

`netErrOutOfMemory`
Not enough memory available to open the library.

`netErrNoInterfaces`
Incorrect setup.

`netErrPrefNotFound`
Incorrect setup.

Comments Applications must call this function before using the net library. If the net library was already open, `NetLibOpen` increments its open count. Otherwise, it opens the library, initializes it, starts up the net protocol stack component of the library as a separate task, and brings up all attached network interfaces.

`NetLibOpen` uses settings saved in the net library's preferences database during initialization. These settings include the interfaces to attach, the IP addresses, etc. It's assumed that these settings have been previously set up by a preference panel or equivalent so an

application doesn't normally have to set them up before calling `NetLibOpen`.

If any of the attached interfaces fails to come up, *`netIFErrsP` will contain the error number of the first interface that encountered a problem.

See Also `SysLibFind`, `NetLibClose`, `NetLibOpenCount`

NetLibOpenCount

Purpose Retrieves the open count of the net library.

Prototype

```
Err NetLibOpenCount (UInt16 refNum,
UInt16 *countP)
```

Parameters

-> `refNum`	Reference number of the net library.
<- `countP`	Contains the open count of the net library upon return.

Result Always returns 0.

Sockets Equivalent None.

Comments This call will most likely only be used by the Network preferences panel. Most applications will simply call `NetLibOpen` unconditionally during startup and `NetLibClose` when they exit.

NetLibReceive

Purpose Receive data from a socket into a single buffer.

Prototype

```
Int16 NetLibReceive (UInt16 libRefNum,
NetSocketRef socket, void *bufP, UInt16 bufLen,
UInt16 flags, void *fromAddrP, UInt16 *fromLenP,
Int32 timeout, Err *errP);
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.
<- `bufP`	Pointer to buffer to hold received data.
-> `bufLen`	Length of `bufP` buffer.
-> `flags`	One or more `netIOFlagxxx` flags. See "I/O Flags."
<- `fromAddrP`	Pointer to buffer to hold address of sender (a `NetSocketAddrType`).
<-> `fromLenP`	On entry, size of `fromAddrP` buffer. On exit, actual size of returned address in `fromAddrP`.
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result

Returns the number of bytes successfully received. If the return value is 0, the socket has been shut down by the remote host. If the return value is -1, an error has occurred, and `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrWouldBlock`	
`netErrUserCancel`	

Sockets Equivalent

```
int recvfrom (int socket, const void *bufP,
int bufLen, int flags, const void *fromAddrP,
int *fromLenP);

int recv (int socket, const void *bufP,
int bufLen, int flags);

int read (int socket, const void *bufP,
int bufLen);
```

Comments For stream-based sockets, this call reads whatever bytes are available and returns the number of bytes actually read into the caller's buffer. If there is no data available, this call will block until at least one byte arrives, until the socket is shut down by the remote host, or until a timeout occurs.

For datagram-based sockets, this call reads a complete datagram and returns the number of bytes in the datagram. If the caller's buffer is not large enough to hold the entire datagram, the end of the datagram is discarded. If a datagram is not available, this call will block until one arrives, or until the call times out.

The data is read into a single buffer pointed to by `bufP`.

See Also `NetLibReceive`, `NetLibDmReceive`, `NetUReadN`, `NetLibSend`, `NetLibSendPB`

NetLibReceivePB

Purpose Receive data from a socket into a multi-buffer gather-read array.

Prototype

```
Int16 NetLibReceivePB (UInt16 libRefNum,
NetSocketRef socket, NetIOParamType *pbP,
UInt16 flags, Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.
-> `pbP`	Pointer to parameter block containing buffer info.
-> `flags`	One or more `netIOFlagxxx` flags. See "I/O Flags."
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns the number of bytes successfully received. Returns 0 if the socket has been shut down by the remote host. If the return value is

-1, an error has occurred, and `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrWouldBlock`	

Sockets Equivalent

```
int recvmsg (int socket, const struct msghdr *pbP,
int flags);
```

Comments

The `pbP` parameter is a pointer to a `NetIOParamType` structure. `NetIOParamType` is defined as follows:

```
typedef struct {
  UInt8        *addrP;
  UInt16       addrLen;
  NetIOVecPtr  iov;
  UInt16       iovLen;
  UInt8        *accessRights;
  UInt16       accessRightsLen;
} NetIOParamType, *NetIOParamPtr;
```

You provide the following information in this struct:

`addrP`	Address of sender, set by `NetLibReceivePB`. Set to 0 if you don't require this field.
`addrLen`	Length of `*addrP`.
`iov`	Array of buffers into which the data should be received. `NetIOVecPtr` is a pointer to a `NetIOVecType` structure, which has two fields:

`bufP`	Pointer to a buffer.
`bufLen`	Length of `bufP`.

`iovLen`	Length of the `iov` array.
`accessRights`	Access rights. This field currently isn't used and should be set to 0.
`accessRightsLen`	Length of the `*accessRights`. This field currently isn't used and should be set to 0.

For stream-based sockets, this call reads whatever bytes are available and returns the number of bytes actually read into the caller's buffer. If no data is available, this call will block until at least one byte arrives, until the socket is shut down by the remote host, or until a timeout occurs.

For datagram-based sockets, this call reads a complete datagram and returns the number of bytes in the datagram. If the caller's buffer is not large enough to hold the entire datagram, the end of the datagram is discarded. If a datagram is not available, this call will block until one arrives, or until the call times out.

The data is read into the gather-read array specified by the `pbP->iov` array.

See Also `NetLibReceive`, `NetLibDmReceive`, `NetLibSend`, `NetLibSendPB`

NetLibSelect

Purpose Blocks until I/O is ready on one or more descriptors, where a descriptor can represent socket input, socket output, or a user input event like a pen tap or key press.

Prototype

```
Int16 NetLibSelect (UInt16 libRefNum,
UInt16 width, NetFDSetType *readFDs,
NetFDSetType *writeFDs, NetFDSetType *exceptFDs,
Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `width`	Number of descriptor bits to check in the `readFDs`, `writeFDs`, and `exceptFDs` descriptor sets.

<-> `readFDs`	Pointer to 32-bit `NetFDSetType` containing set of bits representing descriptors to check for input.
<-> `writeFDs`	Pointer to 32-bit `NetFDSetType` containing set of bits representing descriptors to check for output.
<-> `exceptFDs`	Pointer to 32-bit `NetFDSetType` containing set of bits representing descriptors to check for exception conditions. This parameter is ignored. Upon return, its bits are always cleared.
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result

Returns the sum total number of ready file descriptors in `*readFDs`, `*writeFDs`, and `*exceptFDs`. Returns 0 upon timeout. If the return value is -1, an error has occurred, and `errP` contains one of the following values:

0	No error
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.

Sockets Equivalent

```
int select (int width, fd_set *readfds,
fd_set *writefds, fd_set *exceptfds,
struct timeval *timeout);
```

Comments

This call blocks until one or more descriptors are ready for I/O. In the Palm OS environment, a descriptor is either a `NetSocketRef` or the "stdin" descriptor, `sysFileDescStdIn`. The `sysFileDescStdIn` descriptor will be ready for input whenever a user event is available like a pen tap or key press.

The caller should set which bits in each descriptor set need to be checked by using the `netFDZero` and `netFDSet` macros. After this

call returns, the macro `netFDIsSet` can be used to determine which descriptors in each set are actually ready.

On exit, the total number of ready descriptors is returned and each descriptor set is updated with the appropriate bits set for each ready descriptor in that set.

The following example illustrates how to use this call to check for input on a socket or a user event:

```
Err    err;
NetSocketRef    socket;
NetFDSetType    readFDs,writeFDs,exceptFDs;
Int16       numFDs;
UInt16       width;

// Create the descriptor sets
netFDZero(&readFDs);
netFDZero(&writeFDs);
netFDZero(&exceptFDs);
netFDSet(sysFileDescStdIn, &readFDs);
netFDSet(socket, &readFDs);

// Calculate the max descriptor number and
// use that +1 as the max width.
// Alternatively, we could simply use the
// constant netFDSetSize as the width which
// is simpler but makes the NetLibSelect call
// slightly slower.
width = sysFileDescStdIn;
if (socket > width) width = socket;

// Wait for any one of the descriptors to be
// ready.
numFDs = NetLibSelect(AppNetRefnum, width+1,
  &readFDs, &writeFDs, &exceptFDs,
  AppNetTimeout, &err);
```

Also see the `NetSample` example application in the `Palm OS Examples` folder. The function `CmdTelnet` in the file

`CmdTelnet.c` shows how to use the Berkeley sockets `select` function and how to interpret the results.

See Also `NetLibSocketOptionSet`

NetLibSend

Purpose Send data to a socket from a single buffer.

Prototype

```
Int16 NetLibSend (UInt16 libRefNum,
NetSocketRef socket, void *bufP, UInt16 bufLen,
UInt16 flags, void *toAddrP, UInt16 toLen,
Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.
-> `bufP`	Pointer to data to write.
-> `bufLen`	Length of data to write
-> `flags`	One or more `netIOFlagxxx` flags. See "I/O Flags."
-> `toAddrP`	Address to send to (a pointer to a `NetSocketAddrType`), or 0.
-> `toLen`	Size of `toAddrP` buffer.
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns the number of bytes successfully sent. Returns 0 if the socket has been shut down by the remote host. If the return value is -1, an error has occurred, and `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.

`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrMessageTooBig`	
`netErrSocketNotConnected`	
`netErrSocketClosedByRemote`	
`netErrIPCantFragment`	
`netErrIPNoRoute`	
`netErrIPNoSrc`	
`netErrIPNoDst`	
`netErrIPktOverflow`	
`netErrOutOfCmdBlocks`	
`netErrOutOfPackets`	
`netErrInterfaceNotFound`	
`netErrInterfaceDown`	
`netErrUnreachableDest`	
`netErrNoMultiPktAddr`	
`netErrWouldBlock`	

Sockets Equivalent

```
int sendto (int socket, const void *bufP,
int bufLen, int flags, const void *toAddrP,
int toLen);
```

```
int send (int socket, const void *bufP,
int bufLen, int flags);
```

```
int write (int socket, const void *bufP,
int bufLen,);
```

Comments

This call attempts to write data to the specified socket and returns the number of bytes actually sent, which may be less than or equal to the requested number of bytes. The data is passed in a single buffer that `bufP` points to.

For datagram sockets, you must only send a single packet at a time. If the data is too large to fit in a single UDP packet (1536 bytes), no data is sent and -1 is returned.

The `toAddrP` field applies only to datagram sockets without an existing connection. An error is returned if the datagram socket was previously connected and `toAddrP` is specified. Stream-based sockets, by definition, must have a connection established with a remote host before data can be written. Raw sockets (supported in Palm OS version 3.0 and higher) must construct the entire IP header, including the destination address, before data can be sent; thus, the address is taken from the data to be sent.

If there isn't enough buffer space to send any data, this call will block until there is enough buffer space, or until a timeout.

NOTE: For stream-based sockets, this call may write only a portion of the desired data. It always returns the number of bytes actually written. Consequently, the caller should be prepared to call this routine repeatedly until the desired number of bytes have been written, or until it returns 0 or -1.

See Also NetLibSendPB, NetUWriteN, NetLibReceive, NetLibReceivePB, NetLibDmReceive

NetLibSendPB

Purpose Send data to a socket from a scatter-write array.

Prototype

```
Int16 NetLibSendPB (UInt16 libRefNum,
NetSocketRef socket, NetIOParamType *pbP,
UInt16 flags, Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.
-> `pbP`	Pointer to parameter block containing buffer info. See the description in NetLibReceivePB.

-> `flags`	One or more `netIOFlagxxx` flags. See "I/O Flags."
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns the number of bytes successfully sent. Returns 0 if the socket has been shut down by the remote host. If the return value is -1, an error has occurred, and `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.

`netErrParamErr`

`netErrSocketNotOpen`

`netErrMessageTooBig`

`netErrSocketNotConnected`

`netErrSocketClosedByRemote`

`netErrIPCantFragment`

`netErrIPNoRoute`

`netErrIPNoSrc`

`netErrIPNoDst`

`netErrIPktOverflow`

`netErrOutOfCmdBlocks`

`netErrOutOfPackets`

`netErrInterfaceNotFound`

`netErrInterfaceDown`

`netErrUnreachableDest`

`netErrNoMultiPktAddr`

`netErrWouldBlock`

Sockets Equivalent

```
int sendmsg (int socket, const struct msghdr *pbP,
int flags);
```

Comments

This call attempts to write data to the given socket and returns the number of bytes actually sent, which may be less than or equal to the requested number of bytes. The data is passed in the scatter-write array specified in the `pbP` parameter block.

For datagram sockets, you must only send a single packet at a time. If the data is too large to fit in a single UDP packet, no data is sent and -1 is returned.

The `toAddrP` field applies only to datagram sockets without an existing connection. An error is returned if the datagram socket was previously connected and `toAddrP` is specified. Stream-based sockets, by definition, must have a connection established with a remote host before data can be written. Raw sockets (supported in Palm OS version 3.0 and higher) must construct the entire IP header, including the destination address, before data can be sent; thus, the address is taken from the data to be sent.

If there isn't enough buffer space to send any data, this call will block until there is space, or until a timeout.

NOTE: For stream-based sockets, this call may write only a portion of the desired data. It always returns the number of bytes actually written. Consequently, the caller should be prepared to call this routine repeatedly until the desired number of bytes have been written, or until it returns 0 or -1.

See Also

NetLibSend, NetLibReceive, NetLibReceivePB, NetLibDmReceive

NetLibSettingGet

Purpose Retrieves a general setting.

Prototype

```
Err NetLibSettingGet (UInt16 libRefNum,
UInt16 setting, void *valueP, UInt16 *valueLenP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `setting`	Setting to retrieve, one of the `NetSettingEnum` constants.
<- `valueP`	Space for return value of setting.
<-> `valueLenP`	On entry, size of `valueP`. On exit, actual size of setting.

Result Returns one of the following values:

0	Success.
`netErrUnknownSetting`	Invalid setting constant
`netErrPrefNotFound`	No current value for setting
`netErrBufTooSmall`	`valueP` was too small to hold entire setting. Setting value was truncated to fit in `valueP`.
`netErrBufWrongSize`	

Sockets Equivalent None

Comments This call retrieves the current value of any general setting. The caller must pass a pointer to a buffer to hold the return value (`valueP`), the size of the buffer (*`valueLenP`), and the setting ID (`setting`). The setting ID is one of the `NetSettingEnum` constants in the `netSettingEnum` type.

Some settings, such as the host table, are variable size. For these types of settings, you can obtain the actual size required for the

buffer by passing 0 for `*valueLenP`. The required size is returned in `valueLenP`.

Table 62.2 lists the general settings and the type of each setting.

Table 62.2 Net Library General Settings

netSetting...	Type	Description
`ResetAll`	`void`	Used for `NetLibSettingSet` only. This will clear all other settings to their default values.
`PrimaryDNS`	`UInt32`	IP address of primary DNS server. This setting must be set to a non-zero IP address in order to support any of the name lookup calls.
`SecondaryDNS`	`UInt32`	IP address of secondary DNS server. Set to 0 to have stack ignore this setting.
`DefaultRouter`	`UInt32`	IP address of default router. Default value is 0 which is appropriate for most implementations with only one attached interface (besides loopback). Packets with destination IP addresses that don't lie in the subnet of an attached interface will be sent to this router through the default interface specified by the `netSettingDefaultIFCreator/ netSettingDefaultIFInstance` pair.
`DefaultIFCreator`	`UInt32`	Creator of the default network interface. Default value is 0, which is appropriate for most implementations. Packets with destination IP addresses that don't lie in the subnet of a directly attached interface are sent through this interface. If this setting is 0, the stack automatically makes the first non-loopback interface the default interface.
`DefaultIFInstance`	`UInt16`	Instance number of the default network interface. Packets with destination IP addresses that don't lie in the subnet of an attached interface are sent through the default interface. Default value is 0.

Table 62.2 Net Library General Settings *(continued)*

netSetting...	Type	Description
`HostName`	`Char[]`	A zero-terminated character string of 64 bytes or less containing the host name of this machine. This setting is not actually used by the stack. It's present mainly for informative purposes and to support the `gethostname/sethostname` sockets API calls. To clear the host name, call `NetLibIFSettingSet` with a valueLen of 0.
`DomainName`	`Char[]`	A zero-terminated character string of 256 bytes or less containing the default domain. This default domain name is appended to all host names before name lookups are performed. If the name is not found, the host name is looked up again without appending the domain name to it. To have the stack not use the domain name, call `NetLibIFSettingSet` with a `valueLen` of 0.
`HostTbl`	`Char[]`	A null-terminated character string containing the host table. This table is consulted first before sending a DNS query to the DNS server(s). To have the stack not use a host table, call `NetLibIFSettingSet` with a `valueLen` of 0. The format of a host table is a series of lines separated by '\n' in the following format: `host.company.com A 111.222.333.444`
`CloseWaitTime`	`UInt32`	The close-wait time in milliseconds. This setting must be specified. See the discussion of the `NetLibClose` call for an explanation of the close-wait time.
`TraceBits`	`UInt32`	A bitfield of various trace bits. See "Tracing Bits." Default value is (`netTracingErrors | netTracingAppMsgs`). An application can get a list of events in the trace buffer using the `NetLibMaster` call.

Table 62.2 Net Library General Settings *(continued)*

netSetting...	Type	Description
`TraceSize`	`UInt32`	Maximum trace buffer size in bytes. Setting this setting always clears the existing trace buffer. Default is 2 KB.
`TraceRoll`	`UInt8`	Boolean value, default is `true` (non-zero). If `true`, trace buffer will roll over when it fills. If `false`, tracing will stop as soon as trace buffer fills.

See Also `NetLibSettingSet`, `NetLibIFSettingSet`, `NetLibIFSettingGet`, `NetLibMaster`

NetLibSettingSet

Purpose Sets a general setting.

Prototype

```
Err NetLibSettingSet (UInt16 libRefNum,
UInt16 setting, void *valueP, UInt16 valueLen)
```

Parameters

-> `libRefNum` Reference number of the net library.

-> `setting` Setting to set; one of the `NetSettingEnum` constants. See Table 62.2.

-> `valueP` New value for the setting.

-> `valueLen` Size of new setting.

Result Returns one of the following values:

0 Success.

`netErrUnknownSetting` Invalid setting constant.

`netErrInvalidSettingSize` `valueLen` was invalid for the given setting.

`netErrBufTooSmall` `valueP` was too small to hold entire setting. Setting value was truncated to fit in `valueP`.

`netErrBufWrongSize`

`netErrReadOnlySetting`

Sockets Equivalent None

Comments This call can be used to set the current value of any general setting. The caller must pass a pointer to a buffer which holds the new value (`valueP`), the size of the buffer (`valueLen`), and the setting ID (`setting`). The setting ID is one of the `netSettingXXX` constants in the `NetSettingEnum` type.

See `NetLibSettingGet` for an explanation of each of the settings.

Of particular interest is the `netSettingResetAll` setting, which, if used, will reset all general settings to their default values. When using this setting, `valueP` and `valueLen` are ignored.

See Also `NetLibSettingGet`, `NetLibSettingSet`, `NetLibIFSettingSet`, `NetLibMaster`

NetLibSocketAccept

Purpose Accept a connection from a stream-based socket.

Prototype

```
Int16 NetLibSocketAccept (UInt16 libRefnum,
NetSocketRef socket, NetSocketAddrType *sockAddrP,
Int16 *addrLenP, Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.
<- `sockAddrP`	Address of remote host is returned here.
<->`addrLenP`	On entry, length of `sockAddrP` buffer in bytes. On exit, length of returned address stored in `*sockAddrP`.
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns the `NetSocketRef` of the new socket. If the return value is -1, an error has occurred, and `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrSocketNotConnected`	
`netErrSocketClosedByRemote`	
`netErrWrongSocketType`	
`netErrSocketNotListening`	
`netErrUnimplemented`	

Sockets Equivalent

```
int accept (int socket, void *sockAddrP,
int *addrLenP);
```

Comments Accepts the next connection request from a remote client. This call is only applicable to stream-based sockets. Before calling `NetLibSocketAccept` on a socket, a server application needs to:

- Open the socket (`NetLibSocketOpen`).
- Bind the socket to a local address (`NetLibSocketBind`).
- Set the maximum pending connection-request queue length (`NetLibSocketListen`).

`NetLibSocketAccept` will block until a successful connection request is obtained from a remote client. After a successful connection is made, this call returns with the address of the remote host in `*sockAddrP` and the socket descriptor of a **new** socket as the return value. You then use the new socket to send and receive data.

See Also `NetLibSocketBind`, `NetLibSocketListen`

NetLibSocketAddr

Purpose Returns the local and remote addresses currently associated with a socket.

Prototype

```
Int16 NetLibSocketAddr (UInt16 libRefnum,
NetSocketRef socketRef,
NetSocketAddrType *locAddrP, Int16 *locAddrLenP,
NetSocketAddrType *remAddrP, Int16 *remAddrLenP,
Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socketRef`	Descriptor for the open socket.
<- `locAddrP`	Local address of socket is returned here.
<->`locAddrLenP`	On entry, length of `locAddrP` buffer in bytes. On exit, length of returned address stored in `*locAddrP`.
<- `remAddrP`	Address of remote host is returned here.
<->`remAddrLenP`	On entry, length of `remAddrP` buffer in bytes. On exit, length of returned address stored in `*remAddrP`.
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns 0 upon success and -1 if an error occurred. If the return value is -1, `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrSocketClosedByRemote`	
`netErrOutOfCmdBlocks`	

Sockets Equivalent

```
int getpeername (int s, struct sockaddr *name,
int *namelen);

int getsockname (int s, struct sockaddr *name,
int *namelen);
```

Comments

This call is mainly useful for stream-based sockets. It allows the caller to find out what address was bound to a connected socket and the address of the remote host that it's connected to.

In Palm OS version 3.0 and higher, if you pass a raw socket to this function, it returns the instance number and creator of the interface to which the socket is bound.

See Also

NetLibSocketBind, NetLibSocketConnect, NetLibSocketAccept

NetLibSocketBind

Purpose

Assign a local address to a socket.

Prototype

```
Int16 NetLibSocketBind (UInt16 libRefnum,
NetSocketRef socket, NetSocketAddrType *sockAddrP,
Int16 addrLen, Int32 timeout, Err *errP)
```

Parameters

-> libRefNum	Reference number of the net library.
-> socket	Descriptor for the open socket.
-> sockAddrP	Pointer to the address to give to the socket. This can be a NetSocketAddrINType or a NetSocketAddrRawType.
-> addrLen	Length of address in *sockAddrP.
-> timeout	Maximum timeout in system ticks; -1 means wait forever.
<- errP	Contains an error code if the return value is -1.

Result

Returns 0 upon success and -1 if an error occurred. If an error occurred, errP contains one of the following values:

0	No error.

`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrSocketAlreadyConnected`	
`netErrSocketClosedByRemote`	
`netErrOutOfCmdBlocks`	

Sockets Equivalent

```
int bind (int socket, const void *sockAddrP,
int addrLen);
```

Comments

Applications that want to wait for an incoming connection request from a remote host must call this function. After calling `NetLibSocketBind`, applications can call `NetLibSocketListen` and then `NetLibSocketAccept` to make the socket ready to accept connection requests.

Compatibility

Raw sockets are only supported in Palm OS version 3.0 and higher. See `NetLibSocketOpen` for instructions on how to bind raw sockets.

See Also

`NetLibSocketConnect`, `NetLibSocketListen`, `NetLibSocketAccept`

NetLibSocketClose

Purpose

Close a socket.

Prototype

```
Int16 NetLibSocketClose (UInt16 libRefnum,
NetSocketRef socket, Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.

	-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
	<- `errP`	Contains an error code if the return value is -1.

Result Returns 0 upon success and -1 if an error occurred. If an error occurred, `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrOutOfCmdBlocks`	

Sockets Equivalent

```
int close (int socket);
```

Comments Closes down a socket and frees all memory associated with it.

See Also NetLibSocketOpen, NetLibSocketShutdown

NetLibSocketConnect

Purpose Assign a destination address to a socket and initiate three-way handshake if it's stream based.

Prototype

```
Int16 NetLibSocketConnect (UInt16 libRefnum,
NetSocketRef socket, NetSocketAddrType *sockAddrP,
Int16 addrLen, Int32 timeout, Err *errP)
```

Parameters

	-> `libRefNum`	Reference number of the net library.
	-> `socket`	Descriptor for the open socket.
	-> `sockAddrP`	Pointer to address to connect to.
	-> `addrLen`	Length of address in `*sockAddrP`.

-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns 0 upon success and -1 if an error occurred. If an error occurred, `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrSocketBusy`	
`netErrNoInterfaces`	Incorrect setup.
`netErrPortInUse`	
`netErrQuietTimeNotElapsed`	
`netErrInternal`	
`netErrSocketAlreadyConnected`	
`netErrSocketClosedByRemote`	
`netErrTooManyTCPConnections`	
`netErrWouldBlock`	
`netErrWrongSocketType`	
`netErrOutOfCmdBlocks`	

Sockets Equivalent

```
int connect (int socket, const void *sockAddrP,
int addrLen);
```

See Also NetLibSocketBind, NetUTCPOpen

NetLibSocketListen

Purpose Put a stream-based socket into passive listen mode.

Prototype

```
Int16 NetLibSocketListen (UInt16 libRefnum,
NetSocketRef socket, UInt16 queueLen,
Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.
-> `queueLen`	Maximum number of pending connections allowed.
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns 0 upon success and -1 if an error occurred. If an error occurred, `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrOutOfResources`	
`netErrSocketNotOpen`	
`netErrSocketBusy`	
`netErrNoInterfaces`	Incorrect setup.
`netErrPortInUse`	
`netErrInternal`	
`netErrSocketAlreadyConnected`	
`netErrSocketClosedByRemote`	

`netErrWrongSocketType`

`netErrQuietTimeNotElapsed`

`netErrOutOfCmdBlocks`

Sockets Equivalent

```
int listen (int socket, int queueLen);
```

Comments

Sets the maximum allowable length of the queue for pending connections. This call is only applicable to stream-based (TCP/IP) sockets.

After a socket is created and bound to a local address using `NetLibSocketBind`, a server application can call `NetLibSocketListen` and then `NetLibSocketAccept` to accept connections from remote clients.

The `queueLen` is currently quietly limited to 1 (higher values are ignored).

See Also

`NetLibSocketBind`, `NetLibSocketAccept`

NetLibSocketOpen

Purpose

Open a new socket.

Prototype

```
NetSocketRef NetLibSocketOpen (UInt16 libRefnum,
NetSocketAddrEnum domain, NetSocketTypeEnum type,
Int16 protocol, Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `domain`	Address domain. See `NetSocketAddrEnum`.
-> `type`	Desired type of connection. See `NetSocketTypeEnum`.
-> `protocol`	Protocol to use. This parameter is currently ignored.

	For raw sockets in the `netSocketAddrINET` domain, specify one of the following:
	`netSocketProtoIPTCP`
	`netSocketProtoIPUDP`
	`netSocketProtoIPRAW`
	For all other socket types or for raw sockets in the raw domain, this parameter is ignored.

-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns the `NetSocketRef` of the opened socket or -1 if an error occurred. If an error occurred, `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrNoMoreSockets`	
`netErrOutOfCmdBlocks`	
`netErrOutOfMemory`	

Sockets Equivalent

```
int socket (int domain, int type, int protocol);
```

Comments Allocates memory for a new socket and opens it.

Raw sockets are supported in Palm OS version 3.0 and higher. Two types of raw sockets are supported:

- Raw sockets in the `netSocketAddrINET` domain

  In this case, you must bind the socket to an IP address using `NetLibSocketBind`, passing a `NetSocketAddrINType` structure for the socket address. The port field is ignored.

For applications that use raw sockets in the INET domain, the net library checks the destination IP address of all incoming packets to see if it matches any of those raw sockets. If it does, the packet is enqueued directly into the matching socket and is **not** passed to the protocol stack.

When an application sends data through raw sockets in the IP domain, the net library packages the data into a packet and passes it directly to the interface's send routine. You are responsible for forming the entire IP header, including any necessary checksums, source and destination IP address, and so on.

- Raw sockets in the `netSocketAddrRaw` domain with no protocol

  In this case, you must bind the socket to an interface using `NetLibSocketBind`, passing a `NetSocketAddrRawType` structure for the socket address. The instance and creator specify which interface the caller wants to receive raw packets from.

  When an interface is bound to a raw socket with no protocol, the net library places that interface into raw mode. In raw mode, the interface passes all incoming packets, no matter what the link layer protocol, to its raw receive function.

  When an application sends data through a raw socket with no protocol, the net library packages the data into a packet and passes it directly to the interface's send routine.

  The interface remains in raw mode until the raw socket is closed.

Compatibility Raw sockets supported only in Palm OS version 3.0 and higher.

See Also `NetLibSocketClose`, `NetUTCPOpen`

NetLibSocketOptionGet

Purpose Retrieves the current value of a socket option.

Prototype

```
Int16 NetLibSocketOptionGet (UInt16 libRefnum,
NetSocketRef socket, UInt16 level, UInt16 option,
void *optValueP, UInt16 *optValueLenP,
Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.
-> `level`	Level of the option, one of the `NetSocketOptLevelEnum` constants. See `NetLibSocketOptionSet`.
-> `option`	One of the `NetSocketOptEnum` constants. See `NetLibSocketOptionSet`.
<- `optValueP`	Pointer to variable holding new value of option.
<-> `optValueLenP`	Size of variable pointed to by `optValueP` on entry. Actual size of return value on exit.
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns 0 upon success and -1 if an error occurred. If an error occurred, `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrUnimplemented`	

`netErrWrongSocketType`

`netErrInvalidSettingSize`

Sockets Equivalent

```
int getsockopt (int socket, int level, int option,
const void *optValueP, int *optValueLenP);
```

Comments

Returns the current value of a socket option. The caller passes a pointer to a variable to hold the returned value (in `optValueP`) and the size of this variable (in `*optValueLenP`). On exit, `*optValueP` is updated with the actual size of the return value.

For all of the fixed size options (every option except `netSocketOptIPOptions`), `*optValueLenP` is unmodified on exit and this call does its best to return the value in the caller's desired type size.

For compatibility with existing Internet applications, this call is quite flexible on the `*optValueLenP` parameter. If the desired type for an option is `FLAG`, this call supports an `*optValueLenP` of 1, 2, or 4. If the desired type for an option is `int`, it supports an `*optValueLenP` of 2 or 4.

See `NetLibSocketOptionSet` for a list of available options.

See Also `NetLibSocketOptionSet`

NetLibSocketOptionSet

Purpose

Set a socket option.

Prototype

```
Int16 NetLibSocketOptionSet (UInt16 libRefnum,
NetSocketRef socket, UInt16 level, UInt16 option,
void *optValueP, UInt16 optValueLen,
Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.

-> `level`	Level of the option, one of the `NetSocketOptLevelEnum` constants. See the comments section.
-> `option`	One of the `NetSocketOptEnum` constants. See the comments section.
-> `optValueP`	Pointer to the variable holding the new value of the option.
-> `optValueLen`	Size of variable pointed to by `optValueP`.
-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result

Returns 0 upon success and -1 if an error occurred. If an error occurred, `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.
`netErrParamErr`	
`netErrSocketNotOpen`	
`netErrUnimplemented`	
`netErrWrongSocketType`	
`netErrInvalidSettingSize`	

Sockets Equivalent

```
int setsockopt (int socketRef, int level,
int option, const void *optValueP,
int optValueLen);
```

Comments

Sets various options associated with a socket. The caller passes a pointer to the new option value in `optValueP` and the size of the option in `optValueLen`.

Table 62.3 lists the available options.

- The Level column specifies the option level, which is one of the `netSocketOptLevelXXX` constants.
- The Option column lists the option, which is one of the `netSocketOptXXX` constants.
- The G/S column lists whether this option can be fetched with the `NetLibSocketOptionGet` call (G) and/or set (S) with this call.
- The type column lists the data type of the option.
- The I column specifies whether or not this option is currently implemented.

Table 62.3 Net Library Socket Options

netSocket OptLevel...	netSocketOpt...	G/S	Type	I	Description
IP	IPOptions	GS	`UInt8[]`	N	Options in IP Header
TCP	TCPNoDelay	GS	`FLAG`	Y	Don't delay send to coalesce packets
TCP	TCPMaxSeg	G	`int`	Y	Get TCP maximum segment size
Socket	SockDebug	GS	`FLAG`	N	Turn on recording of debug info
Socket	SockAcceptConn	G	`FLAG`	N	Socket has had listen
Socket	SockReuseAddr	GS	`FLAG`	N	Allow local address reuse
Socket	SockKeepAlive	GS	`FLAG`	Y	Keep connections alive
Socket	SockDontRoute	GS	`FLAG`	N	Just use interface addresses
Socket	SockBroadcast	GS	`FLAG`	N	Permit sending of broadcast messages
Socket	SockUseLoopback	GS	`FLAG`	N	Bypass hardware when possible

Table 62.3 Net Library Socket Options *(continued)*

netSocket OptLevel...	netSocketOpt...	G/S	Type	I	Description
Socket	SockLinger	GS	`NetSocketLingerType`	Y	Linger on close if data present `NetSocketLingerType` is a structure with two fields: `onOff` (`true` or `false`) and `time` (linger time in seconds).
Socket	SockOOBInLine	GS	`FLAG`	N	Leave received OOB data in-line
Socket	SockSndBufSize	GS	`int`	N	Send buffer size
Socket	SockRcvBufSize	GS	`int`	N	Receive buffer size
Socket	SockSndLowWater	GS	`int`	N	Send low-water mark
Socket	SockRcvLowWater	GS	`int`	N	Receive low-water mark
Socket	SockSndTimeout	GS	`int`	N	Send timeout
Socket	SockRcvTimeout	GS	`int`	N	Receive timeout
Socket	SockErrorStatus	G	`int`	Y	Get error status and clear
Socket	SockSocketType	G	`int`	Y	Get socket type
Socket	SockNonBlocking	GS	`FLAG`	Y	Set non-blocking mode on/off

For compatibility with existing Internet applications, this call is quite flexible on the `optValueLen` parameter. If the desired type for an option is `FLAG`, this call accepts an `optValueLen` of 1, 2, or 4. If the desired type for an option is `int`, it accepts an `optValueLen` of 2 or 4.

Except for the `netSocketOptSockNonBlocking` option, all options listed above have equivalents in the sockets API. The `netSocketOptSockNonBlocking` option was added to this call in the net library in order to implement the functionality of the UNIX `fcntl()` control call, which can be used to turn nonblocking mode on and off for sockets.

See Also NetLibSocketOptionGet

NetLibSocketShutdown

Purpose Shut down a socket in one or both directions.

Prototype

```
Int16 NetLibSocketShutdown (UInt16 libRefnum,
NetSocketRef socket, Int16 direction,
Int32 timeout, Err *errP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `socket`	Descriptor for the open socket.

-> `direction`	Direction to shut down. One of the `NetSocketDirEnum` constants. Specifically:
	`netSocketDirInput`
	`netSocketDirOutput`
	`netSocketDirBoth`

-> `timeout`	Maximum timeout in system ticks; -1 means wait forever.
<- `errP`	Contains an error code if the return value is -1.

Result Returns 0 upon success and -1 if an error occurred. If an error occurred, `errP` contains one of the following values:

0	No error.
`netErrTimeout`	Call timed out.
`netErrNotOpen`	The referenced net library has not been opened yet.

`netErrParamErr`

`netErrSocketNotOpen`

`netErrNoMultiPktAddr`

`netErrOutOfCmdBlocks`

Sockets Equivalent

```
int shutdown (int socket, int direction);
```

Comments

Shuts down communication in one or both directions on a socket.

If direction is `netSocketDirInput`, the socket is marked as down in the receive direction and further read operations from it return a `netErrSocketInputShutdown` error.

NetLibTracePrintF

Purpose

Store debugging information in the net library's trace buffer.

Prototype

```
Err NetLibTracePrintF (UInt16 libRefNum,
const Char *formatStr, ...)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `formatStr`	A `printf` style format string.
-> ...	Arguments to the format string.

Result

Returns 0 upon success or `netErrNotOpen` if the net library has not been opened.

Sockets Equivalent

None

Comments

This call is a convenient debugging tool for developing Internet applications. It stores a message into the net library's trace buffer, which can later be dumped using the `NetLibMaster` call. The net library's trace buffer is used to store run-time errors that the net library encounters as well as errors and messages from network interfaces and from applications that use this call.

The `formatStr` parameter is a `printf` style format string which supports the following format specifiers:

%d, %i, %u, %x, %s, %c

but it does **not** support field widths, leading 0's etc.

Note that the `netTracingAppMsgs` bit of the `netSettingTraceBits` setting must be set using the call `NetLibSettingSet(...netSettingTraceBits...)`. Otherwise, this routine will do nothing.

See Also `NetLibTracePutS`, `NetLibMaster`, `NetLibSettingSet`

NetLibTracePutS

Purpose Store debugging information in the net library's trace buffer.

Prototype

```
Err NetLibTracePutS (UInt16 libRefNum, Char *strP)
```

Parameters

-> `libRefNum`	Reference number of the net library.
-> `strP`	String to store in the trace buffer.

Result Returns 0 upon success or `netErrNotOpen` if the net library has not been opened.

Sockets Equivalent None

Comments This call is a convenient debugging tool for developing Internet applications. It will store a message into the net library's trace buffer which can later be dumped using the `NetLibMaster` call. The net library's trace buffer is used to store run-time errors that the net library encounters as well as errors and messages from network interfaces and from applications that use this call.

Note the `netTracingAppMsgs` bit of the `netSettingTraceBits` setting must be set using the `NetLibSettingSet(...netSettingTraceBits...)` call or this routine will do nothing.

See Also `NetLibTracePrintF`, `NetLibMaster`, `NetLibSettingSet`

NetNToHL

Purpose Macro that converts a 32-bit value from network to host byte order.

Prototype
```
NetNToHL (x)
```

Parameters -> `x` 32-bit value to convert.

Result Returns `x` in host byte order.

Errors none

Sockets Equivalent
```
ntohl()
```

See Also NetNToHS, NetHToNL, NetHToNS

NetNToHS

Purpose Macro that converts a 16-bit value from network to host byte order.

Prototype
```
NetNToHS (x)
```

Parameters -> `x` 16-bit value to convert.

Result Returns `x` in host byte order.

Errors None

Sockets Equivalent
```
ntohs()
```

See Also NetHToNL, NetNToHL, NetHToNS

63

Network Utilities

This chapter describes network utilities provided in the module `NetSocket.c`. These utilities are convenience functions that you can use in place of net library functions in applications that use the net library. You can find `NetSocket.c` in the folder `Libraries\Net\Src`. (On Palm OS® 3.5, `NetSocket.c` is in the folder `CodeWarrior Libraries\Comms\NetSocket\Src`.)

The include file for the functions described in this chapter is `<unix/sys_socket.h>`. This header file is not included by any other Palm™ header file; you must explicitly include it in your code.

For more information on `NetSocket.c` and `sys_socket.h`, see the chapter "Network Communication" in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

Network Utility Functions

NetUReadN

Purpose Reads a specified number of bytes from a socket.

Prototype

```
Int32 NetUReadN (NetSocketRef fd, UInt8* bufP,
UInt32 numBytes)
```

Parameters

`-> fd`	Descriptor for the open socket.
`<- bufP`	Pointer to buffer to hold received data.
`-> numBytes`	Number of bytes to read.

Result Returns the number of bytes actually read. If the return value is less than 0, an error occurred.

Comments This function repeatedly calls NetLibReceive until numBytes have been read or until NetLibReceive returns an error.

See Also NetUWriteN

NetUTCPOpen

Purpose Opens a TCP (streams-based) socket and connects it to a server.

Prototype

```
NetSocketRef NetUTCPOpen (Char* hostName,
Char* serviceName, Int16 port)
```

Parameters

-> hostName	Remote host, given either by name or by dotted decimal address.
-> serviceName	The name of a network service. Possible services are “echo”, “discard”, “daytime”, “qotd”, “chargen”, “ftp-data”, “ftp”, “telnet”, “smtp”, “time”, “name”, “finger”, “pop2”, “pop3”, “nntp”, “imap2”. The value of this parameter is ignored if the port parameter is greater than zero.
-> port	The number of the port to connect to on the remote host. Set port to zero to use serviceName instead.

Result Returns the socket descriptor of the socket that was connected, or -1 if an error occurred.

Comments If serviceName is given and port is less than or equal to zero, this function looks up the port number for that service on the remote host and uses it for the connection.

This function is the equivalent of calling NetLibSocketOpen and NetLibSocketConnect (or socket and connect).

NOTE: This function does not return specific reasons for failure if there is a failure. This function is not production-quality code. It is provided as a quick and dirty way of creating a connection and as sample code that can be used as a reference.

NetUWriteN

Purpose Writes the specified number of bytes to a socket.

Prototype

```
Int32 NetUWriteN (NetSocketRef fd, UInt8* bufP,
UInt32 numBytes)
```

Parameters

`-> fd`	Descriptor for the open socket.
`-> bufP`	Pointer to buffer to write.
`-> numBytes`	Number of bytes to write.

Result Returns the number of bytes actually sent. If the return value is less than 0, an error occurred.

Comments This function repeatedly calls `NetLibSend` until `numBytes` have been written or until `NetLibSend` returns an error.

See Also `NetUReadN`

64

Script Plugin

This chapter describes the login script plugin support. You write a plugin to add to the list of available login script commands in the Network preferences panel. This chapter covers:

- Script Plugin Data Types
- Script Plugin Constants

The header file `ScriptPlugin.h` declares the API described in this chapter.

For more information on the script plugin, see the section "Extending the Network Login Script Support" on page 151 in the "Network Communication" chapter of the *Palm OS Programmer's Companion*, vol. II, *Communications*.

Script Plugin Data Types

PluginCallbackProcType

The `PluginCallbackProcType` defines the `procP` field in `PluginExecCmdType`.

```
typedef struct {
  ScriptPluginSelectorProcPtr selectorProcP;
} PluginCallbackProcType,
*PluginCallbackProcPtr;
```

Field Descriptions

`selectorProcP` The address of a selector-based callback function for accessing the functionality of the network interface. Each network interface provides it own `ScriptPluginSelectorProc` function. See `ScriptPluginSelectorProc`.

PluginCmdPtr

The `PluginCmdPtr` type defines a pointer to a `PluginCmdType` structure.

```
typedef PluginCmdType * PluginCmdPtr;
```

PluginCmdType

The `PluginCmdType` structure specifies the name of a command.

```
typedef struct {
  Char    commandName[pluginMaxCmdNameLen + 1];
  Boolean hasTxtStringArg;
  UInt8   reserved;
} PluginCmdType;
```

Field Descriptions

`commandName`	The name of the command. This string appears in the pull-down list in the Network preferences panel's script view. The pull-down list contains all available commands from all plugins. Make sure that your command name is unique and as short as possible.
`hasTxtStringArg`	`true` if the command takes an argument. In this case when the user selects this command, the Network preferences panel displays a field next to the command name where the user should enter the argument. This argument is passed in the `txtStringArg` field in `PluginExecCmdType` when the command is to be executed.
`reserved`	Reserved for future use.

PluginExecCmdType

The `PluginExecCmdType` structure defines the parameter block for the `scptLaunchCmdExecuteCmd` launch code. This structure specifies which command is to be executed and provides any necessary arguments for the command. Your plugin should respond by executing the command.

```
typedef struct {
  Char    commandName[pluginMaxCmdNameLen + 1];
  Char    txtStringArg
                [pluginMaxLenTxtStringArg + 1];
  PluginCallbackProcPtr procP;
  void *  handle;
} PluginExecCmdType, *PluginExecCmdPtr;
```

Field Descriptions

`commandName`	The command's name. This is the string that appears in the pull-down list in the script view of the Network preferences panel.
`txtStringArg`	If the command takes an argument, this field provides the argument as a string. A `NULL` value means either that the user did not provide a value, or that you didn't specify that the command takes an argument.
`procP`	A pointer to a `PluginCallbackProcType` structure, which identifies the network interface function that the plugin can use to execute the command.
`handle`	Handle to information specific to a particular connection. You must pass this value when you call the function pointed to by `procP`.

PluginInfoPtr

The `PluginInfoPtr` type defines a pointer to a `PluginInfoType` structure.

```
typedef PluginInfoType * PluginInfoPtr;
```

PluginInfoType

The `PluginInfoType` structure is the parameter block for the `scptLaunchCmdListCmds` launch code. When your plugin receives the launch code, the `PluginInfoType` structure is empty. Your plugin should fill in the `PluginInfoType` and return it. The system uses the information returned to construct the pull-down list of available script commands and build a table of which plugin will execute which script command.

```
typedef struct {
  Char  pluginName[pluginMaxModuleNameLen + 1];
  UInt16  numOfCommands;
  PluginCmdType  command[pluginMaxNumOfCmds];
} PluginInfoType;
```

Field Descriptions

`pluginName`	A name that the system can use to identify your plugin. This is typically the same name you give the PRC file.
`numOfCommands`	The number of commands that your plugin defines. The maximum allowed is `pluginMaxNumOfCmds`.
`command`	An array of `PluginCmdType` structures that provide information about the commands that your plugin defines.

ScriptPluginLaunchCodesEnum

The `ScriptPluginLaunchCodesEnum` defines the launch codes for the script plugin. Your script plugin's `PilotMain` function should respond to the launch codes defined in this enum.

```
typedef enum {
  scptLaunchCmdDoNothing =
      sysAppLaunchCmdCustomBase,
  scptLaunchCmdListCmds,
  scptLaunchCmdExecuteCmd
} ScriptPluginLaunchCodesEnum;
```

Value Descriptions

`scptLaunchCmdDoNothing`	This launch code is a no-op supplied only to provide a beginning value for the script plugin launch codes. It is not necessary to respond to this launch code.
`scptLaunchCmdListCmds`	Provide information about the commands that your plugin executes. See `PluginInfoType`.
`scptLaunchCmdExecuteCmd`	Execute the specified command. This launch code is received when the system is executing a user's login script during a network connection attempt. Your plugin should respond by executing the command provided in the `PluginExecCmdType` parameter block.

Script Plugin Constants

Command Constants

The following constants identify the available commands that the network interface can perform for you. These commands are building blocks that you use to create your own script commands. To perform one of these tasks, pass the constant value as an argument to the network interface's callback function (ScriptPluginSelectorProc).

Constant	Value	Description
pluginNetLibDoNothing	0	For debugging purposes.
pluginNetLibReadBytes	1	Read the specified number of bytes from the open connection.
pluginNetLibWriteBytes	2	Write the specified number of bytes to the open connection.
pluginNetLibGetUserName	3	Get the user name from the network service profile.
pluginNetLibGetUserPwd	4	Get the user's password from the network service profile.
pluginNetLibCheckCancelStatus	5	Check to see if the user canceled the connection.
pluginNetLibPromptUser	6	Prompt the user for input.
pluginNetLibConnLog	7	Write a string to the network service's connection log.
pluginNetLibCallUIProc	8	Have the network interface call a function in your plugin that displays UI.

Constant	Value	Description
		Use this command if you need to display a more complicated user interface than the simple user prompt that the network interface provides.
`pluginNetLibGetSerLibRefNum`	9	Obtain the serial library's reference number. You need the reference number to perform any serial library commands, which you might need to perform more complex work with the connection port.

Size Constants

The following table lists constants that control the size of strings in your plugin and the size of the plugin itself.

Constant	Value	Description
`pluginMaxCmdNameLen`	15	The maximum length for the command's name, not including the terminating `NULL` character. This is the string displayed to the user in the pull-down menu.
`pluginMaxModuleNameLen`	15	The maximum length for the plugin's name (not including the terminating `NULL` character), which is typically the name of the PRC file as well.
`pluginMaxNumOfCmds`	10	The maximum number of commands that your plugin can define.
`pluginMaxLenTxtStringArg`	63	The maximum length of the argument that each command can take, not including the terminating `NULL` character.

Script Plugin Functions

ScriptPluginSelectorProc

Purpose

A function provided by the network interface for the purpose of performing script commands.

Prototype

```
Err (*ScriptPluginSelectorProcPtr) (void *handle,
UInt16 command, void *dataBufferP, UInt16 *sizeP,
UInt16 *dataTimeoutP, void *procAddrP);
```

Parameters

-> `handle`	Handle to information specific to a particular connection.
-> `command`	The command to be executed. See "Command Constants" for a list of possible values. The rest of the parameters to this callback function are interpreted differently based on the value of the `command` parameter. See the table in the "Comments" section for specifics.
<-> `dataBufferP`	A pointer to arguments to pass to the command or a pointer to data returned by the command. See the "Comments" section below.
<-> `sizeP`	The size of `dataBufferP`.
-> `dataTimeoutP`	Number of seconds to wait for the command to execute. 0 means wait forever. Applies only to commands that request information from the network.
-> `procAddrP`	Pointer to a user interface callback function that the network interface should call to complete the function. Used only by `pluginNetLibCallUIProc`. This function should take one argument of the same type that you pass to `dataBufferP` and should return `void`.

Result Returns 0 upon success, or an error condition upon failure. If an error condition is returned, your plugin should stop processing and return the error condition from its `PilotMain`.

Comments When your plugin receives the `scptLaunchCmdExecuteCmd` launch code, the parameter block contains the command's name, its text string argument (if any), and a pointer to the network interface's callback function. You should use this callback function any time you need to communicate with the network library, the user, or the host computer during execution of your command.

The callback function takes as arguments the handle to information about this connection (which is also passed in the launch code's parameter block), and the command that the service should execute. The rest of the parameters are interpreted differently based on what the value the command argument is. See the table below.

pluginNetLib	dataBufferP	sizeP	dataTimeOutP	procAddrP
`DoNothing`	N/A	N/A	N/A	N/A
`ReadBytes`	On return, contains the bytes that were read.	On input, contains the number of bytes to read. On return, contains the number of bytes actually read.	Number of seconds to wait before timing out the operation.	N/A
`WriteBytes`	On input, contains the data to send.	On input, contains the number of bytes to send.	Number of seconds to wait for a response before canceling.	N/A

pluginNetLib	dataBufferP	sizeP	dataTimeOutP	procAddrP
		On return, contains the number of bytes actually sent.		
`UserName`	On return, contains the user's name	On return, contains the size of the string pointed to by `dataBufferP`.	N/A	N/A
`UserPwd`	On return, contains the user's password.	On return, contains the size of the string pointed to by `dataBufferP`.	N/A	N/A
`CheckCancelStatus`	On return, the Boolean value `true` if the user canceled the command, `false` otherwise.	Size of Boolean.	N/A	N/A
`PromptUser`	On input, the prompt to display. On return, the text that the user entered.	On input and on return, the size of the string pointed to by `dataBufferP`.	N/A	N/A
`ConnLog`	The string that should be written to the log.	N/A	N/A	N/A

pluginNetLib	dataBufferP	sizeP	dataTimeOutP	procAddrP
`CallUIProc`	A pointer to a structure to pass to your callback function as a parameter. This structure should contain a handle to the form to be displayed, plus any other necessary information.	N/A	N/A	A pointer to a function in your plugin that displays the form.
`GetSerLib RefNum`	On return, contains the serial library's reference number.	N/A	N/A	N/A

65

Virtual Drivers

This chapter provides reference material for the Serial Manager virtual device driver API:

- Driver Data Structures
- Driver Constants
- Virtual Driver-Defined Functions
- Serial Manager Queue Functions

The header files `SerialVdrv.h`and `SerialDrvr.h` declare the virtual driver API. For more information on writing device drivers for the Serial Manager, see section "Writing a Virtual Device Driver" on page 114 in the "Serial Communication" chapter of *Palm OS Programmer's Companion*, vol. II, *Communications*.

Driver Data Structures

DrvrInfoType

The `DrvrInfoType` structure defines information about the serial hardware. It is passed to and filled in by the `DrvEntryPoint` for a virtual driver.

```
typedef struct {
  UInt32 drvrID;
  UInt32 drvrVersion;
  UInt32 maxBaudRate;
  UInt32 handshakeThreshold;
  UInt32 portFlags;
  const Char *portDesc;
  DrvrIRQEnum irqType;
  UInt8 multipleEnumerations;
  UInt32 dbCreator;
} DrvrInfoType;
```

Value Descriptions

`drvrID`	4-character creator type, such as 'u328'.
`drvrVersion`	Version of code that works for this hardware. See Driver Version Constants.
`maxBaudRate`	Maximum baud rate supported by this hardware.
`handshakeThreshold`	Baud rate at which the use of hardware handshaking is necessary.
`portFlags`	Bit flags denoting features of this hardware. The flags are described in Port Feature Constants.
`portDesc`	Pointer to null-terminated string describing this hardware. This string appears in the Connection panel to describe the port to the user (only if the `portCncMgrVisible` bit in `portFlags` is set). Can be `NULL` if the driver contains a resource (of type 'tSTR' and id `kPortDescStrID`) that supplies this string.
`irqType`	IRQ line being used for this hardware. For a virtual driver, specify `drvrIRQNone`.
`multipleEnumerations`	The number of entries in the driver table required for this driver. If 0, the driver has a single entry.
`dbCreator`	Creator ID of the database containing this driver.

Compatibility

The `multipleEnumerations` and `dbCreator` fields are only defined if New Serial Manager Feature Set Version 2 is present.

DrvrRcvQType

The `DrvrRcvQType` structure defines the virtual driver receive buffer and function pointers to functions that access and save data to the buffer. A pointer to this structure is passed to the `VdrvOpenProcPtr` function. The `DrvrHWRcvQPtr` type defines a pointer to a `DrvrRcvQType` structure.

```
typedef struct DrvrRcvQType {
  void *rcvQ;
  WriteByteProcPtr qWriteByte;
  WriteBlockProcPtr qWriteBlock;
  GetSizeProcPtr qGetSize;
  GetSpaceProcPtr qGetSpace;
  SignalCheckPtr qSignalCheck;
} DrvrRcvQType;

typedef DrvrRcvQType *DrvrHWRcvQPtr;
```

Value Descriptions

`rcvQ`	Pointer to the receive buffer.
`qWriteByte`	Function pointer to a function that the virtual driver can use to write one byte to the Serial Manager's receive queue. See the `WriteByteProcPtr` function.
`qWriteBlock`	Function pointer to a function that the virtual driver can use to write a block of bytes to the Serial Manager's receive queue. See the `WriteBlockProcPtr` function.
`qGetSize`	Function pointer to a function that the virtual driver can use to get the total size of the Serial Manager's receive queue. See the `GetSizeProcPtr` function.
`qGetSpace`	Function pointer to a function that the virtual driver can use to get the available space in the Serial Manager's receive queue. See the `GetSpaceProcPtr` function.

`qSignalCheck`	Function pointer to a function that the virtual driver can use to perform a signal check for the Serial Manager's receive queue. See the `SignalCheckPtr` function.

Compatibility The `qSignalCheck` field is only defined if New Serial Manager Feature Set Version 2 is present.

DrvrStatusEnum

The `DrvrStatusEnum` enumerated type specifies serial status bit flags. Return these enumerated types from the `VdrvStatusProcPtr` call.

```
typedef enum DrvrStatusEnum {
  drvrStatusCtsOn = 0x0001,
  drvrStatusRtsOn = 0x0002,
  drvrStatusDsrOn = 0x0004,
  drvrStatusTxFifoFull = 0x0008,
  drvrStatusTxFifoEmpty = 0x0010,
  drvrStatusBreakAsserted = 0x0020,
  drvrStatusDataReady = 0x0040,
  drvrStatusLineErr = 0x0080
} DrvrStatusEnum;
```

Value Descriptions

`drvrStatusCtsOn`	Set if CTS line is active.
`drvrStatusRtsOn`	Set if RTS line is active.
`drvrStatusDsrOn`	Set if DSR is on.
`drvrStatusTxFifoFull`	Set if transmit FIFO is full; cleared if FIFO has space.
`drvrStatusTxFifoEmpty`	Set if transmit FIFO is empty.
`drvrStatusBreakAsserted`	Set if sending break characters is enabled.
`drvrStatusDataReady`	Used by debugger only.
`drvrStatusLineErr`	Used by debugger only.

SrmRcvQType

The `SrmRcvQType` structure defines the Serial Manager receive queue. This queue is passed as a parameter to the virtual driver.

```
typedef struct SrmRcvQType {
  UInt32 qStart;
  UInt32 qEnd;
  UInt32 qSize;
  UInt8 *qData;
  void *qPort;
} SrmRcvQType;
```

Field Descriptions

`qStart`	The start of the queue.
`qEnd`	The end of the queue.
`qSize`	The size of the queue.
`qData`	The data currently in the queue.
`qPort`	A pointer to the current foreground port.

Compatibility The `SrmRcvQType` structure was previously a private structure. It is declared publicly if New Serial Manager Feature Set Version 2 is present.

VdrvAPIType

The `VdrvAPIType` structure defines function pointers to the required virtual driver functions. When passed a pointer to this structure in the `DrvEntryPoint` function, that function must fill in the pointers to the virtual driver functions appropriately.

```
typedef struct {
  VdrvOpenProcPtr drvOpen;
  VdrvCloseProcPtr drvClose;
  VdrvControlProcPtr drvControl;
  VdrvStatusProcPtr drvStatus;
  VdrvReadProcPtr drvRead;
  VdrvWriteProcPtr drvWrite;
  VdrvOpenProcV4Ptr drvOpenV4;
  VdrvControlCustomProcPtr drvControlCustom;
} VdrvAPIType;
```

Value Descriptions

`drvOpen`	Pointer to the driver open function.
`drvClose`	Pointer to the driver close function.
`drvControl`	Pointer to the driver control function.
`drvStatus`	Pointer to the driver status function.
`drvRead`	Pointer to the driver read function.
`drvWrite`	Pointer to the driver write function.
`drvOpenV4`	Pointer to the driver open function for New Serial Manager Feature Set Version 2.
`drvControlCustom`	Pointer to the driver custom control function.

Compatibility `drvOpenV4` and `drvControlCustom` are declared if both New Serial Manager Feature Set Version 2 and 4.0 New Feature Set are present.



VdrvConfigType

The `VdrvConfigType` structure specifies parameters for opening a serial port. This structure is passed as a parameter to `VdrvOpenProcV4Ptr`.

```
typedef struct VdrvConfigType {
  UInt32 baud;
  UInt32 drvrId;
  UInt32 function;
  MemPtr drvrDataP;
  UInt16 drvrDataSize;
  UInt32 sysReserved1;
  UInt32 sysReserved2;
} VdrvConfigType;
```

Field Descriptions

`baud` Baud rate at which to open the connection. Serial drivers that do not require baud rates ignore this field.

`drvrId` Creator ID of the application or library that is using the Serial Manager.

`function` The reason why the port was opened. Specify the creator ID of the application that is opening the port or one of the following values:

`serFncUndefined`
Undefined function. This is the default value for this field.

`serFncPPPSession`
The connection is to be used for the PPP interface.

`serFncSLIPSession`
The connection is to be used for the SLIP session.

`serFncDebugger` The connection is to be used for a debugging session.

`serFncHotSync` The connection is to be used for a HotSync operation.

`serFncConsole` The connection is to the debugging console.

`serFncTelephony`
The connection is to the telephony library.

The `function` field is used by protocols such as USB and Bluetooth that perform different setup tasks based on which type of application is using them. RS-232 drivers ignore this parameter.

`drvrDataP` Pointer to a driver-specific data block.

`drvrDataSize`	The size of the data block pointed to by `drvrDataP`.
`sysReserved1`	Reserved for future use.
`sysReserved2`	Reserved for future use.

Compatibility This structure is only defined if both New Serial Manager Feature Set Version 2 and 4.0 New Feature Set are present.

VdrvCtlOpCodeEnum

The `VdrvCtlOpCodeEnum` enumerated type specifies a serial control operation. You should handle each of these constants when passed for the `controlCode` parameter to the `VdrvControlProcPtr` call.

```
typedef enum VdrvCtlOpCodeEnum {
  vdrvOpCodeNoOp = 0,
  vdrvOpCodeSetBaudRate = 0x1000,
  vdrvOpCodeSetSettingsFlags,
  vdrvOpCodeSetCtsTimeout,
  vdrvOpCodeClearErr,
  vdrvOpCodeSetSleepMode,
  vdrvOpCodeSetWakeupMode,
  vdrvOpCodeFIFOCount,
  vdrvOpCodeStartBreak,
  vdrvOpCodeStopBreak,
  vdrvOpCodeStartLoopback,
  vdrvOpCodeStopLoopback,
  vdrvOpCodeFlushTxFIFO,
  vdrvOpCodeFlushRxFIFO,
  vdrvOpCodeSendBufferedData,
  vdrvOpCodeRcvCheckIdle,
  vdrvOpCodeEmuSetBlockingHook,
  vdrvOpCodeGetOptTransmitSize,
  vdrvOpCodeGetMaxRcvBlockSize,
  vdrvOpCodeNotifyBytesReadFromQ,
  vdrvOpCodeSetDTRAsserted,
  vdrvOpCodeGetDTRAsserted,
  vdrvOpCodeWaitForConfiguration,
```

```
    vdrvOpCodeGetUSBDeviceDescriptor,
    vdrvOpCodeGetUSBConfigDescriptor,
    vdrvOpCodeEnableIRDA,
    vdrvOpCodeDisableIRDA,
    vdrvOpCodeEnableUART,
    vdrvOpCodeDisableUART,
    vdrvOpCodeRxEnable,
    vdrvOpCodeRxDisable,
    vdrvOpCodeLineEnable,
    vdrvOpCodeEnableUARTInterrupts,
    vdrvOpCodeDisableUARTInterrupts,
    vdrvOpCodeSetReceiveQueue,
    vdrvOpCodeSaveState,
    vdrvOpCodeRestoreState,
    vdrvOpCodeSetYieldPortCallback,
    vdrvOpCodesetYieldPortRefCon,
    vdrvOpCodeUserDef = 0x2000,
    vdrvOpCodeSystem = 0x7000,
    vdrvOpCodeCustom = 0x8000
} VdrvCtlOpCodeEnum;
```

Value Descriptions

`vdvrOpCodeSetBaudRate`	Sets the baud rate.
`vdvrOpCodeSetSettingsFlags`	Sets the data transmission options. The bit flags are described in Serial Settings Constants.
`vdvrOpCodeSetCtsTimeout`	Hardware handshake timeout.
`vdvrOpCodeClearErr`	Clears the hardware error state.
`vdvrOpCodeSetSleepMode`	Puts the port in sleep mode (not typically used for virtual drivers).
`vdvrOpCodeSetWakeupMode`	Wakes up the port from sleep mode (not typically used for virtual drivers).
`vdvrOpCodeFIFOCount`	Returns the number of bytes currently in the FIFO (or best estimate).

`vdvrOpCodeStartBreak`	Sends a break character or enables the sending of break characters.
`vdvrOpCodeStopBreak`	Stops sending break characters.
`vdvrOpCodeStartLoopback`	Starts loopback mode (not typically used for virtual drivers).
`vdvrOpCodeStopLoopback`	Stops loopback mode (not typically used for virtual drivers).
`vdrvOpCodeFlushTxFIFO`	Flushes the contents of the transmit FIFO.
`vdrvOpCodeFlushRxFIFO`	Flushes the contents of the receive FIFO.
`vdrvOpCodeSendBufferedData`	Notifies virtual device to send any buffered data it has not emptied from its internal buffers.
`vdrvOpCodeRcvCheckIdle`	Called periodically to allow the virtual device time to check if there is data to be received. Because virtual devices execute in the same thread as applications, they can be prevented from handling notifications of received data.
`vdrvOpCodeEmuSetBlockingHook`	Special op code for the Simulator.
`vdrvOpCodeGetOptTransmitSize`	Returns the optimum buffer size for sending data or returns 0 to specify any buffer size is acceptable.
`vdrvOpCodeGetMaxRcvBlockSize`	Returns the maximum receive block size that the Serial Manager should request from the virtual device. Can be used to implement flow control.
`vdrvOpCodeNotifyBytesReadFromQ`	Tells the virtual device that some number of bytes have been read from the receive queue by the client application. Can be used to implement flow control.
`vdrvOpCodeSetDTRAsserted`	Asserts or de-asserts the DTR signal.
`vdrvOpCodeGetDTRAsserted`	Gets the status of the DTR signal.

`vdrvOpCodeWaitForConfiguration`	Waits for USB enumeration to complete. Called from the send and receive functions of the Serial Manager. The driver should have a timeout for how long it waits for enumeration to complete. The driver should return with no error if enumeration has already occurred or has occurred within the driver's timeout. If the enumeration has not occurred within the driver's timeout, the driver should return `serErrTimeOut`.
`vdrvOpCodeGetUSBDeviceDescriptor`	Retrieves the device descriptor of a USB driver. Used to gather information about the device's capabilities. Implementation of this op code is optional. If the driver chooses to implement this op code, then the driver should return a pointer to the device descriptor. A driver that chooses not to implement this op code should return `serErrNotSupported`.
`vdrvOpCodeGetUSBConfigDescriptor`	Retrieves the configuration descriptor of a USB driver. Used to gather information about the device's capabilities. Implementation of this op code is optional. If the driver chooses to implement this op code, then the driver should return a pointer to the device descriptor. A driver that chooses not to implement this op code should return `serErrNotSupported`.
`vdrvOpCodeEnableIRDA`	Enable the IrDA mode and power up the IR line drivers.
`vdrvOpCodeDisableIRDA`	Disable the IrDA mode and disable the IR line drivers.
`vdrvOpCodeEnableUART`	Powers up the UART and the line drivers.
`vdrvOpCodeDisableUART`	Powers down the UART and the line drivers.

`vdrvOpCodeRxEnable`	Enables the receive FIFO, enables UART interrupts, and does whatever else is necessary to allow the UART to receive data.
`vdrvOpCodeRxDisable`	Disables the receive FIFO and UART interrupts and does whatever is needed to prevent the UART from receiving data.
`vdrvOpCodeLineEnable`	Enables the main serial line driver for the UART.
`vdrvOpCodeEnableUARTInterrupts`	Enables the appropriate UART receive interrupts.
`vdrvOpCodeDisableUARTInterrupts`	Disables all UART interrupts.
`vdrvOpCodeSetReceiveQueue`	This op code is used by the Serial Manager to set the driver's receive queue. This control code is called when a driver that has previously been opened as a background port is opened as a fully open bidirectional port.
`vdrvOpCodeSaveState`	Invoked when this port is yielded. This is a hook for the driver to save any current state.
`vdrvOpCodeRestoreState`	Invoked when the foreground port is closed and this port can become the foreground port.
`vdrvOpCodeSetYieldPortCallback`	Set the function to be called if the Serial Manager attempts to open another port when this one is open. This op code is for system use only.
`vdrvOpCodeSetYieldPortRefCon`	Data to pass to the yield port callback function. System use only.
`vdvrOpCodeUserDef`	User defined function invoked through `SrmControl`.

`vdrvOpCodeSystem`	Reserves op codes between 0x7000 and 0x8000 for system use.
`vdrvOpCodeCustom`	Reserves op codes greater than 0x8000 for driver-specific use.

Compatibility The op codes starting at `vdrvOpCodeWaitForConfiguration` are defined only if New Serial Manager Feature Set Version 2 is present. The op codes for yieldable ports and custom operations are defined only if both 4.0 New Feature Set is present as well.

Driver Constants

Driver Version Constants

The driver version constants specify which version of the driver API is implemented by this driver. The `DrvEntryPoint` function passes this value back to the Serial Manager in the `drvrVersion` field of the `DrvrInfoType` function.

Constant	Value	Description
`kDrvrVersion`	4	The latest version of the API.
`kDrvrVersion3`	3	The version of the driver API that corresponds to New Serial Manager Feature Set Version 1 (which ships with roughly Palm OS® 3.3 up to Palm OS 4.0).
`kDrvrVersion4`	4	The version of the driver API that corresponds to New Serial Manager Feature Set Version 2 (which ships with roughly Palm OS 4.0 and higher).

Port Feature Constants

The port feature constants are flags that describe serial hardware capabilities.

Constant	Value	Description
`portPhysicalPort`	0x00000001	Should be set for a physical port, unset for a virtual port.
`portBkgndModeSupported`	0x00000002	Denotes that this port can be used for background ports. This flag is only applicable to virtual drivers. Background mode support is implied on physical drivers.
`portRS232Capable`	0x00000004	Set if this hardware has an RS-232 port.
`portIRDACapable`	0x00000008	Set if this hardware has an IR port and supports IrDA mode.
`portCradlePort`	0x00000010	Set if this hardware controls the cradle port.
`portExternalPort`	0x00000020	Set if this hardware port is external or on a memory card.
`portModemPort`	0x00000040	Set if this hardware communicates with a modem.
`portCncMgrVisible`	0x00000080	Set if this serial port's name is to be displayed in the Connection panel.
`portConsolePort`	0x00000100	Denotes this hardware controls the console port.
`portUSBCapable`	0x00000200	Set if this hardware has a USB port.
`portPrivateUse`	0x00001000	Set if this driver is for special software and not general applications.

Compatibility USB support is only available if New Serial Manager Feature Set Version 2 is present.

Virtual Driver-Defined Functions

The functions in this section must be defined by your virtual driver.

DrvEntryPoint

Purpose Entry point for the virtual driver.

Prototype

```
Err DrvEntryPoint (DrvrEntryOpCodeEnum opCode,
void *uartData)
```

Parameters

-> `opCode`	Entry function code.
<-> `uartData`	Pointer to data specific to `opCode`.

Result

`errNone`	No error.
-1	The op code is invalid or the hardware could not be found.

Comments This function's purpose is based on the value of the `opCode` parameter. The three possible codes are `drvrEntryGetUartFeatures`, `drvrEntryGetDrvrFuncts`, and `drvrEntryGetUartFtrsNEntries`.

`DrvEntryPoint` is called with the `drvrEntryGetUartFeatures` code when the Serial Manager is installed into the system at boot time and is looking for all installed drivers. When this op code is set, the `uartData` pointer points to a `DrvrInfoType` structure. This function does not allocate the structure, it just fills in the fields with information.

This function should check to make sure the associated serial device can operate under the current OS and system settings. If the hardware cannot be found, the function should leave the `DrvrInfoType` struct untouched and return a -1 error.

The driver needs to supply a string that describes the port it manages. This string is displayed to the user in the Connection panel and is returned by the `SrmGetDeviceInfo` function. To set this string, copy it into the `portDesc` field of the `DrvrInfoType`

structure. Alternatively, you can supply this string in a driver resource of type 'tSTR' and id `kPortDescStrID`.

If the `DrvrInfoType` structure has a positive value in the `multipleEnumerations` field upon return, the Serial Manager defines one port for each entry in the driver table. The `DrvEntryPoint` function is called again, this time with the `drvrEntryGetUartFtrsNEntries` code. The `uartData` pointer points to a new `DrvrInfoType` structure whose `multipleEnumerations` field indicates which port is to be defined. The function should supply all information specific to this port.

`DrvEntryPoint` is called with the `drvrEntryGetDrvrFuncts` code when a virtual port is opened. The `uartData` pointer points to a `VdrvAPIType` structure and `DrvEntryPoint` must fill in the fields of this structure with appropriate function pointers.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

The `drvrEntryGetUartFtrsNEntries` is only supported if New Serial Manager Feature Set Version 2 is present. This function is fully backwards compatible. Passing 0 for the `multipleEnumerations` field defines a single port for the driver.

VdrvCloseProcPtr

Purpose Handles all activities needed to close the virtual device.

Prototype

```
Err (*VdrvCloseProcPtr) (VdrvDataPtr drvrData)
```

Parameters

-> `drvrData` Pointer to the driver's private global area.

Result

`errNone` No error.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

VdrvControlProcPtr

Purpose

Extends the `SrmControl` function to the level of the virtual device.

Prototype

```
Err (*VdrvControlProcPtr) (VdrvDataPtr drvrData,
VdrvCtlOpCodeEnum controlCode, void *controlData,
UInt16 *controlDataLen)
```

Parameters

-> `drvrData`	Pointer to the driver's private global area.
-> `controlCode`	Control function op code. One of the op codes listed in the `VdrvCtlOpCodeEnum` type.
<-> `controlData`	Pointer to data for the specified control function.
<-> `controlDataLen`	Pointer to length of control data being passed in or out.

Result

`errNone`	No error.
`serErrNotSupported`	`controlCode` not supported.
`serErrBadParam`	`controlData` or `controlDataLen` is bad.

Comments

This function should support the op codes listed in the `VdrvCtlOpCodeEnum` type. If this function does not support an op code, it must return the `serErrNotSupported` error code for that op code.

Table 65.1 shows what is passed for the `controlData` and `controlDataLen` parameters for each of the control codes that use them. Control codes not listed do not use these parameters.

Table 65.1 VDrvControlProcPtr Parameters

`vdvrOpCodeSetBaudRate`	-> `controlData` = Pointer to `Int32` (baud rate), -> `controlDataLen` = Pointer to `sizeof(Int32)`.
`vdvrOpCodeSetSettingsFlags`	-> `controlData` = Pointer to `UInt32` (bitfield; see Serial Settings Constants) -> `controlDataLen` = Pointer to `sizeof(UInt32)`
`vdvrOpCodeFIFOCount`	-> `controlData` = Pointer to `Int16`, which contains the number of bytes in the FIFO. -> `controlDataLen` = Pointer to `sizeof(Int16)`.
`vdrvOpCodeGetOptTransmitSize`	<- `controlData` = Pointer to `Int32` (buffer size), <- `controlDataLen` = Pointer to `sizeof(Int32)`. Return the optimum buffer size for sending data, or 0 to specify any buffer size is acceptable.
`vdrvOpCodeGetMaxRcvBlockSize`	<- `controlData` = Pointer to `Int32` (block size), <- `controlDataLen` = Pointer to `sizeof(Int32)`. Return the maximum block size that the Serial Manager should request from the virtual device.
`vdrvOpCodeNotifyBytes ReadFromQ`	-> `controlData` = Pointer to `Int32` (number of bytes read), -> `controlDataLen` = Pointer to `sizeof(Int32)`.
`vdrvOpCodeSetDTRAsserted`	-> `controlData` = Pointer to Boolean indicating whether to enable or disable DTR. -> `controlDataLen` = Pointer to `sizeof(Boolean)`

Table 65.1 VDrvControlProcPtr Parameters *(continued)*

`vdrvOpCodeGetDTRAsserted`	<- `controlData` = Pointer to Boolean indicating whether DTR is enabled. <- `controlDataLen` = Pointer to `sizeof(Boolean)`
`vdvrOpCodeUserDef`	<-> `controlData` = Pointer from `SrmControl` (user-defined data), <-> `controlDataLen` = Pointer to `sizeof(Int32).`

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.



New VdrvControlCustomProcPtr

Purpose Extends the `SrmCustomControl` function to the level of the virtual device.

Prototype

```
Err (*VdrvControlCustomProcPtr)
(VdrvDataPtr drvrData, UInt16 opCode,
UInt32 creator, void *controlData,
void *controlDataLenP)
```

Parameters

-> `drvrData`	Pointer to the driver's private global area.
-> `controlCode`	Control function op code.
-> `creator`	Creator ID of the driver that defines the op code. The combination of creator ID and op code uniquely identifies the operation to be performed.
<-> `controlData`	Pointer to data for the specified control function.
<-> `controlDataLen`	Pointer to length of control data being passed in or out.

Result

`errNone`	No error.
`serErrNotSupported`	`controlCode` not supported.
`serErrBadParam`	`controlData` or `controlDataLen` is bad.

Comments This function is a mechanism for a virtual driver to create control codes specific to that driver, allowing for the support of new technologies that have interfaces through the Serial Manager.

Compatibility Implemented only if both New Serial Manager Feature Set Version 2 and 4.0 New Feature Set are present.

VdrvOpenProcPtr

Purpose Initializes the virtual device to begin communication.

Prototype

```
Err (*VdrvOpenProcPtr) (VdrvDataPtr *drvrData,
UInt32 baudRate, DrvrHWRcvQPtr rcvQP)
```

Parameters

<-> `drvrData`	Pointer to a pointer to the driver's private global area (allocated by this function). A pointer to this private global area is passed to the other virtual driver functions.
-> `baudRate`	Initial baud rate setting.
-> `rcvQP`	Pointer to the driver's receive queue buffer structure. For details on the fields, see `DrvrRcvQType`.

Result

`errNone`	No error.

Comments This function must allocate and initialize any global variables (and pass back a pointer to a pointer to them in `drvrDataP`), do any set-up necessary for communicating with other software, and save the `rcvQP` pointer since it will need the functions and pointers to structures enclosed within to be able to save received data into the Serial Manager's receive queue.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.



New VdrvOpenProcV4Ptr

Purpose

Initializes the virtual device to begin communication.

Prototype

```
Err (*VdrvOpenProcV4Ptr) (VdrvDataPtr *drvrData,
VdrvConfigPtr configP, DrvrHWRcvQPtr rcvQP)
```

Parameters

<-> `drvrData`	Pointer to a pointer to the driver's private global area (allocated by this function). A pointer to this private global area is passed to the other virtual driver functions.
-> `configP`	Pointer to the configuration structure specifying the port's properties. See `VdrvConfigType`.
-> `rcvQP`	Pointer to the driver's receive queue buffer structure. For details on the fields, see `DrvrRcvQType`.

Result

`errNone`	No error.

Comments

This function must allocate and initialize any global variables (and pass back a pointer to a pointer to them in `drvrDataP`), do any set-up necessary for communicating with other software, and save the `rcvQP` pointer since it will need the functions and pointers to structures enclosed within to be able to save received data into the Serial Manager's receive queue.

Compatibility

Implemented only if both New Serial Manager Feature Set Version 2 and 4.0 New Feature Set are present.

VdrvStatusProcPtr

Purpose Returns virtual device status.

Prototype
```
UInt16 (*VDrvStatusProcPtr) (VdrvDataPtr drvrData)
```

Parameters

-> `drvrData`	Pointer to the driver's private global area.

Result An unsigned long bitfield denoting the status of the virtual device, but only if the virtual device is emulating hardware. The individual bit flags are described in the `DrvrStatusEnum` type.

Comments Generally, status is returned only to the client application using the virtual device. The Serial Manager does not use status information from virtual devices.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

VdrvWriteProcPtr

Purpose Writes a block of bytes.

Prototype
```
UInt32 (*VdrvWriteProcPtr) (VdrvDataPtr drvrDataP,
void *bufP, UInt32 size, Err *errP)
```

Parameters

-> `drvrDataP`	Pointer to the driver's private global area.
-> `bufP`	Pointer to buffer containing the data to be written to the virtual device.
-> `size`	Number of bytes in the buffer `bufP`.
<- `errP`	Pointer to an error code resulting from the operation. Zero is returned if there is no error.

Result Returns the actual number of bytes written.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

Serial Manager Queue Functions

The functions in this section are supplied by the Serial Manager to the virtual driver through the `DrvrRcvQType` passed to the `VdrvOpenProcPtr` function.

GetSizeProcPtr

Purpose Returns the total size of the Serial Manager's receive queue.

Prototype
```
typedef UInt32 (*GetSizeProcPtr) (void *theQ)
```

Parameters -> `theQ` Pointer to the receive queue.

Result Size in bytes of the Serial Manager's receive queue.

Comments This function is useful for implementing flow control.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

GetSpaceProcPtr

Purpose Returns the available space in the Serial Manager's receive queue.

Prototype
```
typedef UInt32 (*GetSpaceProcPtr) (void *theQ)
```

Parameters -> `theQ` Pointer to the receive queue.

Result Size in bytes of the available space in the Serial Manager's receive queue.

Comments This function is useful for implementing flow control.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.



New SignalCheckPtr

Purpose Check the queue to see if the semaphore needs to be signalled.

Prototype

```
typedef void (*SignalCheckPtr) (void *theQ,
UInt16 lineErrsP)
```

Parameters

-> `theQ`	Pointer to the receive queue.
-> `lineErrsP`	Any serial line errors received should be reported here.

Result Returns nothing.

Comments This function signals that there is data to be received without writing anything to the receive queue. The `WriteByteProcPtr` and `WriteBlockProcPtr` functions also signal that there is data to be received after they have written the data to the queue.

Compatibility Implemented only if New Serial Manager Feature Set Version 2 is present.

WriteBlockProcPtr

Purpose Writes a block of bytes to the Serial Manager's receive queue.

Prototype

```
typedef Err (*WriteBlockProcPtr) (void *theQ,
UInt8 *bufP, UInt16 size, UInt16 lineErrs)
```

Parameters

-> `theQ`	Pointer to the receive queue.
-> `bufP`	Pointer to the buffer holding bytes to be written.
-> `size`	Size of `bufP`.
-> `lineErrs`	Any serial line errors received should be reported here.

Result	`errNone`	No error.
	`serErrLineErr`	There was a software overrun line error.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

WriteByteProcPtr

Purpose Writes one byte to the Serial Manager's receive queue.

Prototype

```
typedef Err (*WriteByteProcPtr) (void *theQ,
UInt8 theByte, UInt16 lineErrs)
```

Parameters	-> `theQ`	Pointer to the receive queue.
	-> `theByte`	The byte to be written to the queue.
	-> `lineErrs`	Any serial line errors received should be reported here.
Result	`errNone`	No error.
	`serErrLineErr`	There was a software overrun line error.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

66

Serial Manager

This chapter provides reference material for the Serial Manager API:

- Serial Manager Data Structures
- Serial Manager Constants
- Serial Manager Functions
- Serial Manager Application-Defined Functions

The header file `SerialMgr.h` declares the Serial Manager API. The file `SystemResources.h` defines some serial port constants. For more information on the Serial Manager, see the chapter "Serial Communication" on page 89 of the *Palm OS Programmer's Companion*, vol. II, *Communications*.

Serial Manager Data Structures

DeviceInfoType

The `DeviceInfoType` structure defines information about a serial device. This structure is returned by the `SrmGetDeviceInfo` function.

```
typedef struct DeviceInfoType {
  UInt32 serDevCreator;
  UInt32 serDevFtrInfo;
  UInt32 serDevMaxBaudRate;
  UInt32 serDevHandshakeBaud;
  Char *serDevPortInfoStr;
  UInt8 reserved[8];
} DeviceInfoType;
typedef DeviceInfoType *DeviceInfoPtr;
```

Value Descriptions

`serDevCreator`	Four-character creator ID for serial driver.
`serDevFtrInfo`	Flags defining features of this serial hardware. See Serial Capabilities Constants for a description of these flags.
`serDevMaxBaudRate`	Maximum baud rate for this device.
`serDevHandshakeBaud`	Hardware handshaking is recommended for baud rates over this rate.
`serDevPortInfoStr`	Description of serial hardware device or virtual device.

SrmCtlEnum

The `SrmCtlEnum` enumerated type specifies a serial control operation. Specify one of these enumerated types for the `op` parameter to the `SrmControl` call.

```
typedef enum SrmCtlEnum {
  srmCtlFirstReserved = 0,
  srmCtlSetBaudRate,
  srmCtlGetBaudRate,
  srmCtlSetFlags,
  srmCtlGetFlags,
  srmCtlSetCtsTimeout,
  srmCtlGetCtsTimeout,
  srmCtlStartBreak,
  srmCtlStopBreak,
  srmCtlStartLocalLoopback,
  srmCtlStopLocalLoopback,
  srmCtlIrDAEnable,
  srmCtlIrDADisable,
  srmCtlRxEnable,
  srmCtlRxDisable,
  srmCtlEmuSetBlockingHook,
  srmCtlUserDef,
```

```
    srmCtlGetOptimalTransmitSize,
    srmCtlSetDTRAsserted,
    srmCtlGetDTRAsserted,
    srmCtlSetYieldPortCallback,
    srmCtlSetYieldPortRefCon,
    srmCtlSystemReserved = 0x7000
    srmCtlCustom = 0x8000,
    srmCtlLAST
} SrmCtlEnum;
```

Value Descriptions

`srmCtlSetBaudRate`	Sets the current baud rate for the serial hardware.
`srmCtlGetBaudRate`	Gets the current baud rate for the serial hardware.
`srmCtlSetFlags`	Sets the current flag settings for the serial hardware. Specify flags from the set described in Serial Settings Constants.
`srmCtlGetFlags`	Gets the current flag settings for the serial hardware.
`srmCtlSetCtsTimeout`	Sets the current CTS timeout value for hardware handshaking.
`srmCtlGetCtsTimeout`	Gets the current CTS timeout value for hardware handshaking.
`srmCtlStartBreak`	Turn RS-232 break signal on. Caller is responsible for turning this signal on and off and insuring it is on long enough to generate a viable break.
`srmCtlStopBreak`	Turn RS-232 break signal off.
`srmCtlStartLocalLoopback`	Start local loopback test.
`srmCtlStopLocalLoopback`	Stop local loopback test.

`srmCtlIrDAEnable`	Enable IrDA connection on this serial port. **NOTE:** You cannot enable an IrDA connection on a VZ processor.
`srmCtlIrDADisable`	Disable IrDA connection on this serial port.
`srmCtlRxEnable`	Enable receiver (for IrDA).
`srmCtlRxDisable`	Disable receiver (for IrDA).
`srmCtlEmuSetBlockingHook`	Set a blocking hook routine for emulation mode only. Not supported on the actual device.
`srmCtlUserDef`	This is a user-defined function that third-party hardware developers can use to set or retrieve hardware-specific information from the serial driver. This op code invokes the driver's corresponding control function with its user-defined op code and the parameters are passed directly through to the serial driver. A serial driver that does not handle this function returns a `serErrBadParam` error. The `srmCtlUserDef` op code is superseded by defining a custom op code if New Serial Manager Feature Set Version 2 is present.
`srmCtlGetOptimalTransmitSize`	Ask the port for the most efficient buffer size for transmitting data packets. This op code returns an error (buffering not necessary), 0 (buffering requested, but application can choose buffer size), or a number greater than 0 (recommended buffer size).
`srmCtlSetDTRAsserted`	Enabled or disable the DTR signal. This is not supported by all hardware.
`srmCtlGetDTRAsserted`	Ask the port whether the DTR signal is enabled or disabled.

`srmCtlSetYieldPortCallback`	Set the function to be called if the Serial Manager attempts to open another port when this one is open. This op code is for system use only.
`srmCtlSetYieldPortRefCon`	Data to pass to the yield port callback function. System use only.
`srmCtlSystemReserved`	Reserves op codes between 0x7000 and 0x8000 for system use.
`srmCtlCustom`	Reserves op codes greater than 0x8000 for driver-specific use.

Compatibility Custom control op codes are only supported if both New Serial Manager Feature Set Version 2 and 4.0 New Feature Set are present.



New SrmOpenConfigType

The `SrmOpenConfigType` structure specifies parameters for opening a serial port. This structure is passed as a parameter to `SrmExtOpen`.

```
typedef struct SrmOpenConfigType {
  UInt32 baud;
  UInt32 function;
  MemPtr drvrDataP;
  UInt16 drvrDataSize;
  UInt32 sysReserved1;
  UInt32 sysReserved2;
} SrmOpenConfigType;
```

Field Descriptions

`baud` Baud rate at which to open the connection. Serial drivers that do not require baud rates ignore this field.

`function` The reason why the port was opened. Specify the creator ID of the application that is opening the port or one of the following values:

`serFncUndefined`
Undefined function. This is the default value for this field.

`serFncPPPSession`
The connection is to be used for the PPP interface.

`serFncSLIPSession`
The connection is to be used for the SLIP session.

`serFncDebugger` The connection is to be used for a debugging session.

`serFncHotSync` The connection is to be used for a HotSync operation.

`serFncConsole` The connection is to the debugging console.

`serFncTelephony`
The connection is to the telephony library.

The `function` field is used by protocols such as USB and Bluetooth that perform different setup tasks based on which type of application is using them. RS-232 drivers ignore this parameter.

`drvrDataP` Pointer to a driver-specific data block.

`drvrDataSize` The size of the data block pointed to by `drvrDataP`.

`sysReserved1`	Reserved for future use.
`sysReserved2`	Reserved for future use.

Compatibility This structure is only defined if both New Serial Manager Feature Set Version 2 and 4.0 New Feature Set are present.

Serial Manager Constants

Port Constants

When you specify the port to open in the `SrmOpen`, `SrmOpenBackground`, `SrmExtOpen`, or `SrmExtOpenBackground` call, you can use either a logical port constant, physical port constant, or a virtual port constant, but it is highly recommended that you use a logical port constant wherever possible.

Logical Serial Port Constants

These constants specify the logical port names.

Constant	Value	Description
`serPortLocalHotSync`	0x8000	The physical HotSync port. The Serial Manager automatically detects whether this port is USB or RS-232.
`serPortCradlePort`	0x8000	Cradle port. The Serial Manager automatically detects whether this port is USB or RS-232. Most applications should specify this as the port.
`serPortIrPort`	0x8001	The IR port. This is a raw IrDA port with no protocol support.
`serPortConsolePort`	0x8002	The debug console port, either USB or RS-232. USB is preferred where both are available.

Constant	Value	Description
`serPortCradleRS232Port`	0x8003	Port for the RS-232 cradle. Specify this port if you want to ensure that your application uses RS-232 communications only.
`serPortCradleUSBPort`	0x8004	Port for the USB cradle. Specify this port if you want to ensure that your application uses USB communications only.

Compatibility USB ports are only supported if New Serial Manager Feature Set Version 2 is present.

Physical Serial Port Constants

The physical port constants specify 4-character constants that reference the physical hardware of the device. Doing so is **not** recommended because the hardware they reference may not exist on a particular device.

Physical port	Value	Description
`sysFileCUart328`	'u328'	Cradle port using the 68328 UART. This port can be switched between RS232 and IrDA mode using the `SrmControl` call.
`sysFileCUart328EZ`	'u8EZ'	Cradle port using the 68328EZ UART. This port can also be switched between RS232 and IrDA mode.
`sysFileCUart650`	'u650'	Specifies the IR port on the upgrade card for Palm Personal or Palm Professional devices. This gives you a raw IR port like calling `SrmControl` does, but it only exists on devices that have the upgrade card.

Virtual Serial Port Constants

The virtual port constants specify 4-character constants that identify virtual ports, simulating a hardware interface. Virtual ports are not tied to specific hardware.

Physical port	Value	Description
`sysFileCVirtIrComm`	'ircm'	A virtual serial cable over an IrDA link using the IRComm protocol. It can only be used to talk to another IRComm device.
`sysFileCVirtRfComm`	'rfcm'	RFCOMM (Bluetooth) virtual port plug-in.
`sysFileCBtConnectPanelHelper`	'btcp'	Bluetooth Connection Panel helper application.

Compatibility All virtual port constants other than `sysFileCVirtIrComm` are only defined if both New Serial Manager Feature Set Version 2 and 4.0 New Feature Set are present.

Serial Capabilities Constants

The serial capabilities constant flags describe serial hardware capabilities. These flags are set in the `serDevFtrInfo` field of the `DeviceInfoType` structure.

Constant	Value	Description
`serDevCradlePort`	0x00000001	Serial hardware controls RS-232 serial from cradle connector of Palm device.
`serDevRS232Serial`	0x00000002	Serial hardware has RS-232 line drivers.
`serDevIRDACapable`	0x00000004	Serial hardware has IR line drivers and generates IrDA mode serial signals.
`serDevModemPort`	0x00000008	Serial hardware drives modem connection.
`serDevCncMgrVisible`	0x00000010	Serial device port name string is to be displayed in the Connection panel.

Constant	Value	Description
`serDevConsolePort`	0x00000020	Serial device is the default console port.
`serDevUSBCapable`	0x00000040	Serial hardware controls USB serial from cradle connector of Palm device.

Compatibility USB ports are only supported if New Serial Manager Feature Set Version 2 is present.

Serial Settings Constants

The serial settings constants identify bit flags that correspond to various serial hardware settings. Use `SrmControl` with the op code `srmCtlSetFlags` to control which settings are used.

Constant	Value	Description
`srmSettingsFlagStopBitsM`	0x00000001	Mask for stop bits field
`srmSettingsFlagStopBits1`	0x00000000	1 stop bit (default)
`srmSettingsFlagStopBits2`	0x00000001	2 stop bits
`srmSettingsFlagParityOnM`	0x00000002	Mask for parity on
`srmSettingsFlagParityEvenM`	0x00000004	Mask for parity even
`srmSettingsFlagXonXoffM`	0x00000008	Mask for Xon/Xoff flow control (not implemented)
`srmSettingsFlagRTSAutoM`	0x00000010	Mask for RTS receive flow control. This is the default.
`srmSettingsFlagCTSAutoM`	0x00000020	Mask for CTS transmit flow control
`srmSettingsFlagBitsPerCharM`	0x000000C0	Mask for bits per character
`srmSettingsFlagBitsPerChar5`	0x00000000	5 bits per character
`srmSettingsFlagBitsPerChar6`	0x00000040	6 bits per character

Constant	Value	Description
`srmSettingsFlagBitsPerChar7`	0x00000080	7 bits per character
`srmSettingsFlagBitsPerChar8`	0x000000C0	8 bits per character (default)
`srmSettingsFlagFlowControlIn`	0x00000100	Protect the receive buffer from software overruns. When this flag and `srmSettingsFlagRTSAutoM` are set, which is the default case, it causes the Serial Manager to assert RTS to prevent the transmitting device from continuing to send data when the receive buffer is full. Once the application receives data from the buffer, RTS is de-asserted to allow data reception to resume. Note that this feature effectively prevents software overrun line errors but may also cause CTS timeouts on the transmitting device if the RTS line is asserted longer than the defined CTS timeout value.
`srmSettingsFlagRTSInactive`	0x00000200	If this flag is set and `srmSettingsFlagRTSAutoM` is not set, RTS is held in the inactive (flow off) state forever.

Status Constants

The status constants identify bit flags that correspond to the status of serial signals. They can be returned by the `SrmGetStatus` function.

Constant	Value	Description
`srmStatusCtsOn`	0x00000001	CTS line is active.
`srmStatusRtsOn`	0x00000002	RTS line is active.
`srmStatusDsrOn`	0x00000004	DSR line is active.
`srmStatusBreakSigOn`	0x00000008	Break signal is active.

Line Error Constants

The line error constants identify bit flags that correspond to the line errors that may occur on the port. They can be returned by the `SrmGetStatus` function.

Constant	Value	Description
`serLineErrorParity`	0x0001	Parity error
`serLineErrorHWOverrun`	0x0002	Hardware overrun
`serLineErrorFraming`	0x0004	Framing error
`serLineErrorBreak`	0x0008	Break signal asserted
`serLineErrorHShake`	0x0010	Line handshake error
`serLineErrorSWOverrun`	0x0020	Software overrun
`serLineErrorCarrierLost`	0x0040	Carrier detect signal dropped

Serial Manager Functions

SrmClearErr

Purpose Clears the port of any line errors.

Prototype
```
Err SrmClearErr (UInt16 portId)
```

Parameters

-> `portID`	Port ID returned from `SrmOpen` or `SrmExtOpen`.

Result This function returns the following error codes:

`errNone`	No error.
`serErrNotSupported`	The port is not the foreground port.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

SrmClose

Purpose Closes a serial port and makes it available to other applications, regardless of whether the port is a foreground or background port.

Prototype
```
Err SrmClose (UInt16 portId)
```

Parameters

-> `portId`	Port ID for port to be closed.

Result This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The serial port is not open.
`serErrNoDevicesAvail`	No serial devices could be found.

Comments If a foreground port is being closed and a background port exists, the background will have access to the port as long as another foreground port is not opened.

If a foreground port is being closed and a yielded port exists, the yielded port will have access to the port as long as it does not yield to the opening of another foreground port. If there are both a yielded port and a background port for the foreground port being closed, the yielded port takes precedence over the background port.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also SrmOpen, SrmOpenBackground

SrmControl

Purpose Performs a serial control function.

Prototype

```
Err SrmControl (UInt16 portId, UInt16 op,
void *valueP, UInt16 *valueLenP)
```

Parameters

-> portID	Port ID returned from SrmOpen or SrmExtOpen.
-> op	Control operation to perform. Specify one of the SrmCtlEnum enumerated types.
<-> valueP	Pointer to a value to use for the operation. See Comments for details.
<-> valueLenP	Pointer to the size of *valueP. See Comments for details.

Result This function returns the following error codes:

errNone	No error.
serErrBadParam	An invalid op code was specified.
serErrBadPort	This port doesn't exist.
serErrNotOpen	The serial port is not open.

`serErrNoDevicesAvail`
No serial devices could be found.

`serErrNotSupported`
The specified op code is not supported in the current configuration.

Comments Table 66.1 shows what to pass for the `valueP` and `valueLenP` parameters for each of the operation codes. Control codes not listed do not use these parameters. See `SrmCtlEnum` for a complete list of control codes.

Table 66.1 SrmControl Parameters

Operation Code	Parameters
`srmCtlSetBaudRate`	-> `valueP` = Pointer to `Int32` (baud rate) -> `valueLenP` = Pointer to `sizeof(Int32)`
`srmCtlGetBaudRate`	<- `valueP` = Pointer to `Int32` (baud rate) <- `valueLenP` = Pointer to `Int16`
`srmCtlSetFlags`	-> `valueP` = Pointer to `Uint32` (bitfield; see Serial Settings Constants) -> `valueLenP` = Pointer to `sizeof(UInt32)`
`srmCtlGetFlags`	<- `valueP` = Pointer to `UInt32` (bitfield) <- `valueLenP` = Pointer to `Int16`
`srmCtlSetCtsTimeout`	-> `valueP` = Pointer to `Int32` (timeout value) -> `valueLenP` = Pointer to `sizeof(Int32)`
`srmCtlGetCtsTimeout`	<- `valueP` = Pointer to `Int32` (timeout value) <- `valueLenP` = Pointer to `Int16`
`srmCtlUserDef`	<-> `valueP` = Pointer passed to the serial or virtual driver <-> `valueLenP` = Pointer to `sizeof(Int32)` For a serial driver, these pointers are passed to the driver's control function and they contain that functions return values (if any) upon return.

Table 66.1 SrmControl Parameters *(continued)*

Operation Code	Parameters
`srmCtlGetOptimalTransmitSize`	<- `valueP` = Pointer to `Int32` <- `valueLenP` = Pointer to `sizeof(Int32)` If an error is returned by `SrmControl`, no buffering should be done. If `valueP` points to zero, buffering is requested, but the transmitting application cannot determine the buffer size. If `valueP` points to a number > 0, then try to send data in blocks of this number of bytes, as this is the most efficient block size for this particular device.
`srmCtlSetDTRAsserted`	-> `valueP` = Pointer to Boolean indicating whether to enable or disable DTR. -> `valueLenP` = Pointer to `sizeof(Boolean)`
`srmCtlGetDTRAsserted`	<- `valueP` = Pointer to Boolean indicating whether DTR is enabled. <- `valueLenP` = Pointer to `Int16`

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmCustomControl`

New SrmCustomControl

Purpose Performs a custom serial control function.

Prototype

```
Err SrmCustomControl (UInt16 portId,
UInt16 opCode, UInt32 creator, void *valueP,
UInt16 *valueLenP)
```

Parameters

-> `portID`	Port ID returned from `SrmOpen` or `SrmExtOpen`.
-> `opCode`	Control operation to perform. The op code must be greater than `srmCtlCustom`.
-> `creator`	Creator ID of the driver that defines the op code. The combination of creator ID and op code uniquely identifies the operation to be performed.
<-> `valueP`	Pointer to a value to use for the operation.
<-> `valueLenP`	Pointer to the size of `*valueP`.

Result

This function returns the following error codes:

`errNone`	No error.
`serErrNotSupported`	The port is not the foreground port.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The serial port is not open.
`serErrNoDevicesAvail`	No serial devices could be found.

Comments

This function is a mechanism for a virtual driver to create control codes specific to that driver, allowing for the support of new technologies that have interfaces through the Serial Manager.

This function simply forwards the `opCode` and any `valueP` parameter to the virtual driver for the port. The virtual driver may return its own error code if the `opCode` or the input in `valueP` is invalid.

Compatibility

Implemented only if both New Serial Manager Feature Set Version 2 and 4.0 New Feature Set are present.

See Also

`SrmControl`



New SrmExtOpen

Purpose

Opens a foreground port connection with the specified configuration.

Prototype

```
Err SrmExtOpen (UInt32 port,
SrmOpenConfigType *configP, UInt16 configSize,
UInt16 *newPortIdP)
```

Parameters

-> `port`	The four-character port name (such as 'ircm' or 'u328') or logical port number to be opened. (See Port Constants.)
-> `configP`	Pointer to the configuration structure specifying the serial port's properties. See `SrmOpenConfigType`.
-> `configSize`	The size of the configuration structure pointed to by `configP`.
<- `newPortIdP`	Contains the port ID to be passed to other Serial Manager functions.

Result

This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	The `port` parameter does not specify a valid port.
`serErrBadParam`	The `configP` parameter is `NULL`.
`serErrAlreadyOpen`	The Serial Manager already has a port open.
`memErrNotEnoughSpace`	There was not enough memory available to open the port.

Comments

Do not keep the port open any longer than necessary. An open serial port consumes more energy from the device's batteries.

The values specified in the `configP` parameter depend on the type of connection being made. For RS-232 connections, you specify the baud rate but not a purpose. For USB connections, you specify a purpose but not a baud rate.

A newly opened port has its line errors cleared, the default CTS timeout set (specified by the constant `srmDefaultCTSTimeout`), a 512-byte receive queue allocated, 1 stop bit, 8 bits per character, RTS enabled, and flow control enabled. To increase the receive queue size, use `SrmSetReceiveBuffer`. To change the other serial port settings, use `SrmControl`.

Compatibility Implemented only if both New Serial Manager Feature Set Version 2 and 4.0 New Feature Set are present. The `SrmExtOpen` function replaces the `SrmOpen` function.

See Also `SrmOpen`, `SrmExtOpenBackground`

New SrmExtOpenBackground

Purpose Opens a port with the specified configuration in the background. Background ports relinquish control when another task opens the port with the `SrmOpen` or `SrmExtOpen` call.

Prototype

```
Err SrmExtOpenBackground (UInt32 port,
SrmOpenConfigType *configP, UInt16 configSize,
UInt16 *newPortIdP)
```

Parameters

-> `port`	Physical or logical port number to be opened. See Port Constants for more information.
-> `configP`	Pointer to the configuration structure specifying the serial port's properties. See `SrmOpenConfigType`.
-> `configSize`	The size of the configuration structure pointed to by `configP`.
<- `newPortIdP`	Contains the port ID to be passed to other Serial Manager functions.

Result This function returns the following error codes:

`errNone` No error.

`serErrAlreadyOpen`
This port already has an installed background owner.

`serErrBadPort` This port doesn't exist.

`serErrNotSupported`
This type of port cannot be opened in the background.

`serErrBadParam` The `configP` parameter is `NULL`.

`memErrNotEnoughSpace`
There was not enough memory available to open the port.

Comments This function is provided to support tasks that want to use a serial device to receive data only when no other task is using the port.

If a background port is forced to surrender control of the hardware as a result of another task opening a foreground connection, all buffers for the background port are flushed. After this active task closes the port, active control of the port is returned to the background task. Only one task can have background ownership of the port.

Note that background ports have limited functionality: they can only receive data and notify owning clients of what data has been received.

The values specified in the `configP` parameter depend on the type of connection being made. For RS-232 connections, you specify the baud rate but not a purpose. For USB connections, you specify a purpose but not a baud rate.

Compatibility Implemented only if both New Serial Manager Feature Set Version 2 and 4.0 New Feature Set are present. The `SrmExtOpenBackground` function replaces the `SrmOpenBackground` function.

See Also `SrmOpen`, `SrmExtOpen`

SrmGetDeviceCount

Purpose Returns the number of available serial devices.

Prototype
```
Err SrmGetDeviceCount (UInt16 *numOfDevicesP)
```

Parameters

<- `numOfDevicesP` Pointer to address where the number of serial devices is returned.

Result

`errNone` No error.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmGetDeviceInfo`

SrmGetDeviceInfo

Purpose Returns information about a serial device.

Prototype
```
Err SrmGetDeviceInfo (UInt32 deviceID,
DeviceInfoType *deviceInfoP)
```

Parameters

-> `deviceID` ID of serial device to get information for. You can pass a zero-based index (0, 1, 2, ...), a valid port ID returned from `SrmOpen` or `SrmExtOpen`, or a 4-character port name (such as 'u328', 'u650', or 'ircm'). See Port Constants.

<- `deviceInfoP` Pointer to a `DeviceInfoType` structure where information about the device is returned.

Result This function returns the following error codes:

`errNone` No error.

`serErrBadPort` This port doesn't exist.

`serErrNoDevicesAvail` The Serial Manager cannot find any serial devices.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also SrmGetDeviceCount

SrmGetStatus

Purpose Returns status information about the serial hardware.

Prototype

```
Err SrmGetStatus (UInt16 portId,
UInt32 *statusFieldP, UInt16 *lineErrsP)
```

Parameters

-> portID	Port ID returned from SrmOpen or SrmExtOpen.
<- statusFieldP	Pointer to address where hardware status information for the port is returned. This is a 32-bit field using the flags described in Status Constants.
<- lineErrsP	Pointer to address where the number of line errors for the port is returned. The line error flags are described in Line Error Constants.

Result This function returns the following error codes:

errNone	No error.
serErrBadPort	This port doesn't exist.
serErrNotSupported	The port is a yielded port.
serErrNoDevicesAvail	No serial devices could be found.

Comments Typically, SrmGetStatus is called to retrieve the line errors for the port if some of the send and receive functions return a serErrLineErr error code.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

SrmOpen

Purpose Opens a foreground port connection with the specified port name or logical port number.

Prototype

```
Err SrmOpen (UInt32 port, UInt32 baud,
UInt16 *newPortIdP)
```

Parameters

-> `port`	The four-character port name or logical port number to be opened. See Port Constants for more information.
-> `baud`	Initial baud rate of port.
<- `newPortIdP`	Contains the port ID to be passed to other Serial Manager functions.

Result This function returns the following error codes:

`errNone`	No error.
`serErrAlreadyOpen`	This port already has an installed foreground owner.
`serErrBadPort`	This port doesn't exist.
`memErrNotEnoughSpace`	There was not enough memory available to open the port.

Comments Only one application or task may have access to a particular serial port at any time.

Do not keep the port open any longer than necessary. An open serial port consumes more energy from the device's batteries.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

If New Serial Manager Feature Set Version 2 is present, the `SrmOpen` function is replaced by `SrmExtOpen`. `SrmOpen` is supported for backward compatibility.

See Also `SrmOpenBackground`

SrmOpenBackground

Purpose

Allows a task to open, initialize, and use the port, but always relinquishes control of the port when another task opens the port with the `SrmOpen` call.

Prototype

```
Err SrmOpenBackground (UInt32 port, UInt32 baud,
UInt16 *newPortIdP)
```

Parameters

-> `port` The four-character port name or logical port number to be opened. See Port Constants for more information.

-> `baud` Initial baud rate of port.

<- `newPortIdP` Contains the port ID to be passed to other Serial Manager functions.

Result

This function returns the following error codes:

`errNone` No error.

`serErrAlreadyOpen` This port already has an installed background owner.

`serErrBadPort` This port doesn't exist.

`memErrNotEnoughSpace` There was not enough memory available to open the port.

Comments

This function is provided to support tasks that want to use a serial device to receive data only when no other task is using the port.

If a background port is forced to surrender control of the hardware as a result of another task opening a foreground connection, all buffers for the background port are flushed. After this active task closes the port, active control of the port is returned to the background task. Only one task can have background ownership of the port.

Note that background ports have limited functionality: they can only receive data and notify owning clients of what data has been received.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

If New Serial Manager Feature Set Version 2 is present, the `SrmOpenBackground` function is replaced by `SrmExtOpenBackground`. `SrmOpenBackground` is supported for backward compatibility.

See Also `SrmOpen`

SrmPrimeWakeupHandler

Purpose Sets the number of received bytes that triggers a call to the wakeup handler function.

Prototype

```
Err SrmPrimeWakeupHandler (UInt16 portId,
UInt16 minBytes)
```

Parameters

-> `portId`	Port ID returned from `SrmOpen` or `SrmExtOpen`.
-> `minBytes`	Number of bytes that must be received before wakeup handler is called. Typically, this is set to 1.

Result This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.
`serErrNoDevicesAvail`	No serial devices could be found.

Comments This function primes a wakeup handler installed by `SrmSetWakeupHandler`.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmSetWakeupHandler`, `WakeupHandlerProcPtr`

SrmReceive

Purpose Receives a specified number of bytes.

Prototype

```
UInt32 SrmReceive (UInt16 portId, void *rcvBufP,
UInt32 count, Int32 timeout, Err *errP)
```

Parameters

-> `portID`	Port ID returned from `SrmOpen` or `SrmExtOpen`.
<- `rcvBufP`	Pointer to buffer where received data is to be returned.
-> `count`	Length of data buffer (in bytes). This specifies the number of bytes to receive.
-> `timeout`	The amount of time (in ticks) that the Serial Manager waits to receive the requested block of data. At the end of the timeout, data received up to that time is returned.
<- `errP`	Error code.

Result Number of bytes of data actually received.

Comments The following error codes can be returned in `errP`:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.
`serErrTimeOut`	Unable to receive data within the specified timeout period.
`serErrConfigurationFailed`	The port needs time to configure, and the configuration has failed.
`serErrNotSupported`	The port is not the foreground port.
`serErrConfigurationFailed`	The port could not configure itself.

`serErrLineErr`	A line error occurred during the receipt of data. Use `SrmGetStatus` to obtain the exact line error.
`serErrNoDevicesAvail`	No serial devices could be found.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmReceiveCheck`, `SrmReceiveFlush`, `SrmReceiveWait`

SrmReceiveCheck

Purpose Checks the receive FIFO and returns the number of bytes in the serial receive queue.

Prototype

```
Err SrmReceiveCheck (UInt16 portId,
UInt32 *numBytesP)
```

Parameters

-> `portID`	Port ID returned from `SrmOpen` or `SrmExtOpen`.
<- `numBytesP`	Number of bytes in the receive queue.

Result This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.
`serErrLineErr`	A line error has occurred. Use `SrmGetStatus` to obtain the exact line error.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmReceive`, `SrmReceiveFlush`, `SrmReceiveWait`

SrmReceiveFlush

Purpose Flushes the receive FIFOs.

Prototype

```
Err SrmReceiveFlush (UInt16 portId, Int32 timeout)
```

Parameters

-> `portId`	Port ID returned from `SrmOpen` or `SrmExtOpen`.
-> `timeout`	Timeout value, in ticks.

Result This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.
`serErrNotSupported`	The port is not the foreground port.
`serErrNoDevicesAvail`	No serial devices could be found.

Comments The `timeout` value forces this function to wait a period of ticks after flushing the port to see if more data shows up to be flushed. If more data arrives within the timeout period, the port is flushed again and the timeout counter is reset and waits again. The function only exits after no more bytes are received by the port for the full timeout period since the last flush of the port. To avoid this waiting behavior, specify 0 for the timeout period.

Any errors on the line are cleared before this function returns.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmReceive`, `SrmReceiveCheck`, `SrmReceiveWait`

SrmReceiveWait

Purpose

Waits until some number of bytes of data have arrived into the serial receive queue, then returns.

Prototype

```
Err SrmReceiveWait (UInt16 portId, UInt32 bytes,
Int32 timeout)
```

Parameters

-> `portID`	Port ID returned from `SrmOpen` or `SrmExtOpen`.
-> `bytes`	Number of bytes to wait for.
-> `timeout`	Timeout value, in ticks.

Result

This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.
`serErrTimeOut`	Unable to receive data within the specified timeout period.
`serErrNotSupported`	The port is not the foreground port.
`serErrBadParam`	The bytes parameter exceeds the size of the receive queue. Use `SrmSetReceiveBuffer` to increase the size of the receive queue.
`serErrLineErr`	A line error occurred during the receipt of data. Use `SrmGetStatus` to obtain the exact line error.
`serErrNoDevicesAvail`	No serial devices could be found.

Comments

If this function returns no error, the application can either check the number of bytes currently in the receive queue (using `SrmReceiveCheck`) or it can just specify a buffer and receive the data by calling `SrmReceive`.

Do not call `SerReceiveWait` from within a wakeup handler. If you do, the `serErrTimeOut` error is returned.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmReceive`, `SrmReceiveCheck`, `SrmReceiveFlush`

SrmReceiveWindowClose

Purpose Closes direct access to the Serial Manager's receive queue.

Prototype

```
Err SrmReceiveWindowClose (UInt16 portId,
UInt32 bytesPulled)
```

Parameters

-> `portId` Port ID returned from `SrmOpen` or `SrmExtOpen`.

-> `bytesPulled` Number of bytes the application read from the receive queue.

Result This function returns the following error codes:

`errNone` No error.

`serErrBadPort` This port doesn't exist.

`serErrNotOpen` The port is not open.

`serErrNotSupported` The port is not the foreground port.

`serErrNoDevicesAvail` No serial devices could be found.

Comments Call this function when the application has read as many bytes as it needs out of the receive queue or it has read all the available bytes.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmReceiveWindowOpen`

SrmReceiveWindowOpen

Purpose Provides direct access to the Serial Manager's receive queue.

Prototype

```
Err SrmReceiveWindowOpen (UInt16 portId,
UInt8 **bufPP, UInt32 *sizeP)
```

Parameters

-> `portId`	Port ID returned from `SrmOpen` or `SrmExtOpen`.
<- `bufPP`	Pointer to a pointer to the receive buffer.
<- `sizeP`	Available bytes in buffer.

Result This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.
`serErrNotSupported`	The port is not the foreground port.
`serErrLineErr`	The data in the queue contains line errors.
`serErrNoDevicesAvail`	No serial devices could be found.

Comments This function lets applications directly access the Serial Manager's receive queue to eliminate buffer copying by the Serial Manager. This access is a "back door" route to the received data. After retrieving data from the buffer, the application must call `SrmReceiveWindowClose`.

Applications that want to empty the receive buffer entirely should call the `SrmReceiveWindowOpen` and `SrmReceiveWindowClose` functions repeatedly until the buffer size returned is 0.

IMPORTANT: Once an application calls `SrmReceiveWindowOpen`, it should not attempt to receive data via the normal method of calling `SrmReceive` or `SrmReceiveWait`, as these functions interfere with direct access to the receive queue.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmReceiveWindowClose`

SrmSend

Purpose Sends a block of data out the specified port.

Prototype

```
UInt32 SrmSend (UInt16 portId, const void *bufP,
UInt32 count, Err *errP)
```

Parameters

-> `portID`	Port ID returned from `SrmOpen` or `SrmExtOpen`.
-> `bufp`	Pointer to data to send.
-> `count`	Length of data buffer, in bytes.
<- `errP`	Error code. See the Comments section for details.

Result Number of bytes of data actually sent.

Comments When `SrmSend` returns, you should check the value returned in the `errP` parameter. If `errNone`, then the entire data buffer was sent. If not `errNone`, then the result equals the number of bytes sent before the error occurred. The possible error values are:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.

`serErrTimeOut`	Unable to send data within the specified CTS timeout period.
`serErrNoDevicesAvail`	No serial devices could be found.
`serErrConfigurationFailed`	The port configuration has failed.
`serErrNotSupported`	The specified port is not the foreground port.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmSendCheck`, `SrmSendFlush`, `SrmSendWait`

SrmSendCheck

Purpose Checks the transmit FIFO and returns the number of bytes left to be sent.

Prototype

```
Err SrmSendCheck (UInt16 portId,
UInt32 *numBytesP)
```

Parameters

-> `portID`	Port ID returned from `SrmOpen` or `SrmExtOpen`.
<- `numBytesP`	Number of bytes left in the FIFO queue.

Result This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.
`serErrNotSupported`	This feature not supported by the hardware.
`serErrNoDevicesAvail`	No serial devices could be found.

Comments Not all serial devices support this feature.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also SrmSend, SrmSendFlush, SrmSendWait

SrmSendFlush

Purpose Flushes the transmit FIFO.

Prototype

```
Err SrmSendFlush (UInt16 portId)
```

Parameters

-> `portID` Port ID returned from SrmOpen or SrmExtOpen.

Result This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.
`serErrNotSupported`	The port is not the foreground port.
`serErrNoDevicesAvail`	No serial devices could be found.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also SrmSend, SrmSendCheck, SrmSendWait

SrmSendWait

Purpose Waits until all previous data has been sent from the transmit FIFO, then returns.

Prototype

```
Err SrmSendWait (UInt16 portId)
```

Parameters

-> `portID`	Port ID returned from `SrmOpen` or `SrmExtOpen`.

Result

This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.
`serErrTimeOut`	Unable to send data within the CTS timeout period.
`serErrNotSupported`	The port is not the foreground port.
`serErrNoDevicesAvail`	No serial devices could be found.

Comments

Consider calling this function if your software needs to detect when all data has been transmitted by `SrmSend`. The `SrmSend` function blocks until all data has been transmitted or a timeout occurs. A subsequent call to `SrmSendWait` blocks until all data queued up for transmission has been transmitted or until another CTS timeout occurs (if CTS handshaking is enabled).

Compatibility

Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also

`SrmSend`, `SrmSendCheck`, `SrmSendFlush`

SrmSetReceiveBuffer

Purpose

Installs a new buffer into the Serial Manager's receive queue.

Prototype

```
Err SrmSetReceiveBuffer (UInt16 portId,
void *bufP, UInt16 bufSize)
```

Parameters

-> `portID`	Port ID returned from `SrmOpen` or `SrmExtOpen`.

-> `bufP`	Pointer to new receive buffer. Ignored if `bufSize` is `NULL`.
-> `bufSize`	Size of new receive buffer in bytes. To remove this buffer and allocate a new default buffer (512 bytes), specify `NULL`.

Result This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	This port is not open.
`memErrNotEnoughSpace`	Not enough memory to allocate default buffer.
`serErrNoDevicesAvail`	No serial devices could be found.

Comments The buffer that you pass to this function must remain allocated while you have the serial port open. Before you close the serial port, you must restore the default queue by calling `SrmSetReceiveBuffer` with `NULL` as the `bufP` and `bufSize` arguments.

IMPORTANT: Applications must install the default buffer before closing the port (or disposing of the new receive queue).

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

SrmSetWakeupHandler

Purpose Installs a wakeup handler.

Prototype

```
Err SrmSetWakeupHandler (UInt16 portId,
WakeupHandlerProcPtr procP, UInt32 refCon)
```

Parameters

-> `portID`	Port ID returned from `SrmOpen` or `SrmExtOpen`.
-> `procP`	Pointer to a `WakeupHandlerProcPtr` function. Specify `NULL` to remove a handler.
-> `refCon`	User-defined data that is passed to the wakeup handler function. This can be a pointer or not.

Result

This function returns the following error codes:

`errNone`	No error.
`serErrBadPort`	This port doesn't exist.
`serErrNotOpen`	The port is not open.
`serErrNoDevicesAvail`	No serial devices could be found.

Comments

The wakeup handler is a function in your application that you want to be called whenever there is data ready to be received on the specified port.

The wakeup handler function will not become active until it is primed with a number of bytes that is greater than 0, by the `SrmPrimeWakeupHandler` function. Every time a wakeup handler is called, it must be re-primed (using `SrmPrimeWakeupHandler`) in order to be called again.

Compatibility

Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also

`SrmPrimeWakeupHandler`, `WakeupHandlerProcPtr`

Serial Manager Application-Defined Functions

WakeupHandlerProcPtr

Purpose Called after some number of bytes are received by the Serial Manager's interrupt function.

Prototype

```
void (*WakeupHandlerProcPtr)(UInt32 refCon)
```

Parameters

`->refCon`	User-defined data passed from the `SrmSetWakeupHandler` function.

Result Returns nothing.

Comments This handler function is installed by calling `SrmSetWakeupHandler`. The number of bytes after which it is called is specified by `SrmPrimeWakeupHandler`.

IMPORTANT: Because wakeup handlers are called during interrupt time, they cannot call **any** Palm OS® system functions, including `SrmReceive`, that may block the system in any way. Wakeup handlers should also be very short so as to reduce interrupt latency.

Two common implementations of wakeup handlers include:

- Calling `EvtWakeup`, which causes any pending `EvtGetEvent` call to return and then sends a `nilEvent` to the current application.
- Using `SrmReceiveWindowOpen` and `SrmReceiveWindowClose` to gain direct access to the receive queue without blocking.

Compatibility Implemented only if New Serial Manager Feature Set Version 1 is present.

See Also `SrmPrimeWakeupHandler`, `SrmSetWakeupHandler`

67

Old Serial Manager

This chapter provides reference material for the serial manager API:

- Serial Manager Data Structures
- Serial Manager Functions

The header file `SerialMgrOld.h` declares the serial manager API. For more information on the serial manager, see the chapter "Serial Communication" in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

NOTE: The API described in this chapter is obsolete if the New Serial Manager Feature Set is present. The API is still supported for backward compatibility; however, the Serial Manager APIs are preferred.

Serial Manager Data Structures

SerCtlEnum

To perform a control function, applications call `SerControl`, which performs one of the control operations specified by `SerCtlEnum`, which has the following elements:

Element	Description
`serCtlFirstReserved = 0`	Reserve 0
`serCtlStartBreak`	Turn RS232 break signal on. Applications have to make sure that the break is set long enough to generate a value BREAK! `valueP = 0; valueLenP = 0`
`serCtlStopBreak`	Turn RS232 break signal off: `valueP = 0; valueLenP = 0`

Element	Description
`serCtlBreakStatus`	Get RS232 break signal status (on or off): `valueP` = ptr to Word for returning status (0 = off, !0 = on) `*valueLenP = sizeof(Word)`
`serCtlStartLocalLoopback`	Start local loopback test; `valueP = 0, valueLenP = 0`
`serCtlStopLocalLoopback`	Stop local loopback test `valueP = 0, valueLenP = 0`
`serCtlMaxBaud`	`valueP = ptr` to `DWord` for returned baud *valueLenP `= sizeof(DWord)`
`serCtlHandshakeThreshold`	Retrieve HW handshake threshold; this is the maximum baud rate that does not require hardware handshaking `valueP` = `ptr` to DWord for returned baud `*valueLenP = sizeof(DWord)`
`serCtlEmuSetBlockingHook`	Set a blocking hook routine. **WARNING!** WARNING: For use with the Simulator on Mac OS only: NOT SUPPORTED ON THE PALM DEVICE. `valueP = ptr` to SerCallbackEntryType `*valueLenP=sizeof(SerCallbackEntryType)` Returns the old settings in the first argument.
`serCtlLAST`	Add new address entries before this one.

SerSettingsType

The `SerSettingsType` structure defines serial port attributes; it is used by the calls `SerGetSettings` and `SerSetSettings`. The `SerSettingsPtr` type points to a `SerSettingsType` structure.

```
typedef struct SerSettingsType {
  UInt32 baudRate;
```

```
    UInt32 flags;
    Int32 ctsTimeout;
    } SerSettingsType;

  typedef SerSettingsType* SerSettingsPtr;
```

Field Descriptions

`baudRate`	Baud rate
`flags`	Miscellaneous settings
`ctsTimeout`	Maximum number of ticks to wait for CTS to become asserted before transmitting; used only when configured with the `serSettingsFlagCTSAutoM` flag.

Serial Manager Functions

SerClearErr

Purpose Reset the serial port's line error status.

Prototype

```
Err SerClearErr (UInt16 refNum)
```

Parameters

->	`refNum`	The serial library reference number.

Result

0	No error.

Comments Call `SerClearErr` only after a serial manager function (`SerReceive`, `SerReceiveCheck`, `SerSend`, etc.) returns with the error code `serErrLineErr`.

The reason for this is that `SerClearErr` resets the serial port. So, if `SerClearErr` is called unconditionally while a byte is coming into the serial port, that byte is guaranteed to become corrupted.

The right strategy is to always check the error code returned by a serial manager function. If it 's `serErrLineErr`, call

`SerClearErr` immediately. However, don't make unsolicited calls to `SerClearErr`.

When you get `serErrLineErr`, consider flushing the receive queue for a fraction of a second by calling `SerReceiveFlush`. `SerReceiveFlush` calls `SerClearErr` for you.

SerClose

Purpose Release the serial port previously acquired by `SerOpen`.

Prototype

```
Err SerClose (UInt16 refNum)
```

Parameters

-> `refNum`	Serial library reference number.

Result

0	No error.
`serErrNotOpen`	Port wasn't open.
`serErrStillOpen`	Port still held open by another process.

Comments Releases the serial port and shuts down serial port hardware if the open count has reached 0. Open serial ports consume more energy from the device's batteries; it's therefore essential to keep a port open only as long as necessary.

Caveat Don't call `SerClose` unless the return value from `SerOpen` was 0 (zero) or `serErrAlreadyOpen`.

See Also `SerOpen`

SerControl

Purpose Perform a control function.

Prototype

```
Err SerControl (UInt16 refNum, UInt16 op,
void *valueP, UInt16 *valueLenP)
```

Parameters

-> `refNum`	Reference number of library.
-> `op`	Control operation to perform (`SerCtlEnum`).
<-> `valueP`	Pointer to value for operation.
<-> `valueLenP`	Pointer to size of value.

Result

0	No error.
`serErrBadParam`	Invalid parameter (unknown).
`serErrNotOpen`	Library not open.

Comments This function provides extensible control features for the serial manager. You can

- Turn on/off the RS232 break signal and check its status.
- Perform a local loopback test.
- Get the maximum supported baud rate.
- Get the hardware handshake threshold baud rate.

Compatibility Implemented only if 2.0 New Feature Set is present.

SerGetSettings

Purpose Fill in the `SerSettingsType` structure with current serial port attributes.

Prototype

```
Err SerGetSettings (UInt16 refNum,
SerSettingsPtr settingsP)
```

Parameters

-> `refNum`	Serial library reference number.
<-> `settingsP`	Pointer to `SerSettingsType` structure to be filled in.

Result

0	No error.
`serErrNotOpen`	The port wasn't open.

Comments The information returned by this call includes the current baud rate, CTS timeout, handshaking options, and data format options.

See the `SerSettingsType` structure for more details.

See Also `SerSend`

SerGetStatus

Purpose Return the pending line error status for errors that have been detected since the last time `SerClearErr` was called.

Prototype

```
UInt16 SerGetStatus (UInt16 refNum,
Boolean *ctsOnP, Boolean *dsrOnP)
```

Parameters

-> `refNum`	Serial library reference number.
-> `ctsOnP`	Pointer to location for storing a Boolean value.
-> `dsrOnP`	Pointer to location for storing a Boolean value.

Result Returns any combination of the following constants, bitwise OR'ed together:

`serLineErrorParity`
Parity error.

`serLineErrorHWOverrun`
Hardware overrun.

`serLineErrorFraming`
Framing error.

`serLineErrorBreak`
Break signal detected.

`serLineErrorHShake`
Line handshake error.

`serLineErrorSWOverrun`
Software overrun.

Comments When another serial manager function returns an error code of `serErrLineErr`, `SerGetStatus` can be used to find out the specific nature of the line error(s).

The values returned via `ctsOnP` and `dsrOnP` are not meaningful in the present version of the software

See Also `SerClearErr`

SerOpen

Purpose Acquire and open a serial port with given baud rate and default settings.

Prototype

```
Err SerOpen (UInt16 refNum, UInt16 port,
UInt32 baud)
```

Parameters

-> `refNum`	Serial library reference number.
-> `port`	Port number.
-> `baud`	Baud rate.

Result

0	No error.
`serErrAlreadyOpen`	Port was open. Enables port sharing by "friendly" clients (not recommended).
`serErrBadParam`	Invalid parameter.
`memErrNotEnoughSpace`	Insufficient memory.

Comments Acquires the serial port, powers it up, and prepares it for operation. To obtain the serial library reference number, call `SysLibFind` with "Serial Library" as the library name. This reference number must be passed as a parameter to all serial manager functions. The device currently contains only one serial port with port number 0 (zero).

The baud rate is an integral baud value (for example - 300, 1200, 2400, 9600, 19200, 38400, 57600, etc.). The Palm OS® device has been tested at the standard baud rates in the range of 300 - 57600 baud.

Baud rates through 1 Mbit are theoretically possible. Use CTS handshaking at baud rates above 19200 (see SerSetSettings).

An error code of 0 (zero) or serErrAlreadyOpen indicates that the port was successfully opened. If the port is already open when SerOpen is called, the port's open count is incremented and an error code of serErrAlreadyOpen is returned. This ability to open the serial port multiple times allows cooperating tasks to share the serial port. Other tasks must refrain from using the port if serErrAlreadyOpen is returned and close it by calling SerClose.

SerReceive

Purpose Receives size bytes worth of data or returns with error if a line error or timeout is encountered.

Prototype

```
UInt32 SerReceive (UInt16 refNum, void *bufP,
UInt32 count, Int32 timeout, Err* errP)
```

Parameters

refNum	Serial library reference number.
<-> bufP	Buffer for receiving data.
-> count	Number of bytes to receive.
-> timeout	Interbyte timeout in ticks, 0 for none, -1 forever.
<-> errP	For returning error code.

Result Number of bytes received:

*errP = 0	No error.
serErrLineErr	RS232 line error.
serErrTimeOut	Interbyte timeout.

Compatibility Implemented only if 2.0 New Feature Set is present.

NOTE: The old versions of SerSend and SerReceive are still available as SerSend10 and SerReceive10 (not V10).

See Also SerReceive10

SerReceive10

Purpose Receive a stream of bytes.

Prototype

```
Err SerReceive10 (UInt16 refNum, void *bufP,
UInt32 bytes, Int32 timeout)
```

Parameters

-> `refNum`	The serial library reference number.
-> `bufP`	Pointer to the buffer for receiving data.
-> `bytes`	Number of bytes desired.
-> `timeout`	Interbyte time out in system ticks (-1 = forever).

Result

0	No error. Requested number of bytes was received.
`serErrTimeOut`	Interbyte time out exceeded while waiting for the next byte to arrive.
`serErrLineErr`	Line error occurred (see `SerClearErr` and `SerGetStatus`).

Comments `SerReceive` blocks until all the requested data has been received or an error occurs. Because this call returns immediately without any data if line errors are pending, it is important to acknowledge the detection of line errors by calling `SerClearErr`. If you just need to retrieve all or some of the bytes which are already in the receive queue, call `SerReceiveCheck` first to get the count of bytes presently in the receive queue.

Compatibility This function corresponds to the 1.0 version of `SerReceive`.

SerReceiveCheck

Purpose Return the count of bytes presently in the receive queue.

Prototype
```
Err SerReceiveCheck (UInt16 refNum,
UInt32 *numBytesP)
```

Parameters

-> `refNum`	Serial library reference number.
<-> `numBytesP`	Pointer to location for returning the byte count.

Result

0	No error.
`serErrLineErr`	Line error pending (see `SerClearErr` and `SerGetStatus`).

Comments Because this call does not return the byte count if line errors are pending, it is important to acknowledge the detection of line errors by calling `SerClearErr`.

See Also `SerReceiveWait`

SerReceiveFlush

Purpose Discard all data presently in the receive queue and flush bytes coming into the serial port. Clear the saved error status.

Prototype
```
void SerReceiveFlush (UInt16 refNum,
Int32 timeout)
```

Parameters

-> `refNum`	Serial library reference number.
-> `timeout`	Interbyte time out in system ticks (-1 = forever).

Result Returns nothing.

Comments `SerReceiveFlush` blocks until a timeout occurs while waiting for the next byte to arrive.

SerReceiveWait

Purpose

Wait for at least `bytes` bytes of data to accumulate in the receive queue.

Prototype

```
Err SerReceiveWait (UInt16 refNum, UInt32 bytes,
Int32 timeout)
```

Parameters

-> `refNum`	Serial library reference number.
-> `bytes`	Number of bytes desired.
-> `timeout`	Interbyte timeout in system ticks (-1 = forever).

Result

0	No error.
`serErrTimeOut`	Interbyte timeout exceeded while waiting for next byte to arrive.
`serErrLineErr`	Line error occurred (see `SerClearErr` and `SerGetStatus`).

Comments

This is the preferred method of waiting for serial input, since it blocks the current task and allows switching the processor into a more energy-efficient state.

`SerReceiveWait` blocks until the desired number of bytes accumulate in the receive queue or an error occurs. The desired number of bytes must be less than the current receive queue size. The default queue size is 512 bytes. Because this call returns immediately if line errors are pending, it is important to acknowledge the detection of line errors by calling `SerClearErr`.

See Also

`SerReceiveCheck`, `SerSetReceiveBuffer`

SerSend

Purpose Send one or more bytes of data over the serial port.

Prototype

```
UInt32 SerSend (UInt16 refNum, void *bufP,
UInt32 count, Err *errP
```

Parameters

-> `refNum`	Serial library reference number.
-> `bufP`	Pointer to data to send.
-> `count`	Number of bytes to send.
<-> `errP`	For returning error code.

Result Returns the number of bytes transferred.

Stores in `errP`:

0	No error.
`serErrTimeOut`	Handshake timeout.

The old calls worked, but they did not return enough info when they failed. The new calls (available in Palm OS v2.0 and greater) add more parameters to solve this problem and make serial communications programming simpler.

Don't call the new functions when running on Palm OS 1.0.

Compatibility Implemented only if 2.0 New Feature Set is present.

NOTE: The old versions of `SerSend` and `SerReceive` are still available as `SerSend10` and `SerReceive10` (not V10).

See Also `SerSend10`, `SerSendWait`

SerSend10

Purpose Send a stream of bytes to the serial port.

Prototype

```
Err SerSend10 (UInt16 refNum, void *bufP,
UInt32 size)
```

Parameters

-> `refNum`	Serial library reference number.
-> `bufP`	Pointer to the data to send.
-> `size`	Size (in number of bytes) of the data to send.

Result

0	No error.
`serErrTimeOut`	Handshake timeout (such as waiting for CTS to become asserted).

Comments In the present implementation, `SerSend10` blocks until all data is transferred to the UART or a timeout error (if CTS handshaking is enabled) occurs. Future implementations may queue up the request and return immediately, performing transmission in the background. If your software needs to detect when all data has been transmitted, see `SerSendWait`.

This routine observes the current CTS time out setting if CTS handshaking is enabled (see `SerGetSettings` and `SerSend`).

Compatibility This function corresponds to the 1.0 version of `SerSend`.

See Also `SerSend`, `SerSendWait`

SerSendFlush

Purpose Discard all data presently in the transmit queue.

Prototype
```
Err SerSendFlush (UInt16 refNum)
```

Parameters

-> `refNum`	Serial library reference number.

Result

0	No error.

See Also `SerSend`, `SerSendWait`

SerSendWait

Purpose Wait until the serial transmit buffer empties.

Prototype
```
Err SerSendWait (UInt16 refNum, Int32 timeout)
```

Parameters

-> `refNum`	Serial library reference number.
-> `timeout`	Reserved for future enhancements. Set to (-1) for compatibility.

Result

0	No error.
`serErrTimeOut`	Handshake timeout (such as waiting for CTS to become asserted).

Comments `SerSendWait` blocks until all data is transferred or a timeout error (if CTS handshaking is enabled) occurs. This routine observes the current CTS timeout setting if CTS handshaking is enabled (see `SerGetSettings` and `SerSend`).

See Also `SerSend`

SerSetReceiveBuffer

Purpose Replace the default receive queue. To restore the original buffer, pass `bufSize` = 0.

Prototype

```
Err SerSetReceiveBuffer (UInt16 refNum,
void *bufP, UInt16 bufSize)
```

Parameters

-> `refNum`	Serial library reference number.
-> `bufP`	Pointer to buffer to be used as the new receive queue.
-> `bufSize`	Size of buffer, or 0 to restore the default receive queue.

Result Returns 0 if successful.

Comments The specified buffer needs to contain 32 extra bytes for serial manager overhead (its size should be your application's requirement plus 32 bytes). The default receive queue must be restored before the serial port is closed. To restore the default receive queue, call `SerSetReceiveBuffer` passing 0 (zero) for the buffer size. The serial manager does not free the custom receive queue.

SerSetSettings

Purpose Set the serial port settings; that is, change its attributes.

Prototype

```
Err SerSetSettings (UInt16 refNum,
SerSettingsPtr settingsP)
```

Parameters

-> `refNum`	Serial library reference number.
<-> `settingsP`	Pointer to the filled in `SerSettingsType` structure.

Result

0	No error.
`serErrNotOpen`	The port wasn't open.
`serErrBadParam`	Invalid parameter.

Comments The attributes set by this call include the current baud rate, CTS timeout, handshaking options, and data format options. See the definition of the SerSettingsType structure for more details.

To do 7E1 transmission, OR together:

```
serSettingsFlagBitsPerChar7 |
serSettingsFlagParityOnM |
serSettingsFlagParityEvenM |
serSettingsFlagStopBits1
```

If you're trying to communicate at speeds greater than 19.2 Kbps, you need to use hardware handshaking: `serSettingsFlagRTSAutoM | serSettingsFlagCTSAutoM`.

See Also SerGetSettings

68

Serial Link Manager

This chapter provides reference material for the serial link manager API. The header file `SerialLinkMgr.h` declares the serial link manager API. For more information on the serial link manager, see the chapter "Serial Communication" in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

Serial Link Manager Functions

SlkClose

Purpose Close down the serial link manager.

Prototype

```
Err SlkClose (void)
```

Parameters None.

Result

0	No error.
`slkErrNotOpen`	The serial link manager was not open.

Comments When the open count reaches zero, this routine frees resources allocated by serial link manager.

SlkCloseSocket

Purpose Closes a socket previously opened with `SlkOpenSocket`.

The caller is responsible for closing the communications library used by this socket, if necessary.

Prototype

```
Err SlkCloseSocket (UInt16 socket)
```

Parameters

`socket`	The socket ID to close.

Result

0	No error.
`slkErrSocketNotOpen`	The socket was not open.

Comments `SlkCloseSocket` frees system resources the serial link manager allocated for the socket. It does not free resources allocated and passed by the client, such as the buffers passed to `SlkSetSocketListener`; this is the client's responsibility. The caller is also responsible for closing the communications library used by this socket.

See Also `SlkOpenSocket`

SlkFlushSocket

Purpose Flush the receive queue of the communications library associated with the given socket.

Prototype

```
Err SlkFlushSocket (UInt16 socket, Int32 timeout)
```

Parameters

-> `socket`	Socket ID.
-> `timeout`	Interbyte timeout in system ticks.

Result

0	No error.
`slkErrSocketNotOpen`	The socket wasn't open.

SlkOpen

Purpose Initialize the serial link manager.

Prototype

```
Err SlkOpen (void)
```

Parameters None.

Result

0	No error.
`slkErrAlreadyOpen`	No error.

Comments Initializes the serial link manager, allocating necessary resources. Return codes of 0 (zero) and `slkErrAlreadyOpen` both indicate success. Any other return code indicates failure. The `slkErrAlreadyOpen` function informs the client that someone else is also using the serial link manager. If the serial link manager was successfully opened by the client, the client needs to call `SlkClose` when it finishes using the serial link manager.

SlkOpenSocket

Purpose Open a serial link socket and associate it with a communications library. The socket may be a known static socket or a dynamically assigned socket.

Prototype

```
Err SlkOpenSocket (UInt16 portID,
UInt16 *socketP, Boolean staticSocket)
```

Parameters

`portID`	Comm library reference number for socket.
`socketP`	Pointer to location for returning the socket ID.
`staticSocket`	If `TRUE`, `*socketP` contains the desired static socket number to open. If `FALSE`, any free socket number is assigned dynamically and opened.

Result

0	No error.
`slkErrOutOfSockets`	No more sockets can be opened.

Comments The communications library must already be initialized and opened (see `SerOpen`). When finished using the socket, the caller must call `SlkCloseSocket` to free system resources allocated for the socket. For information about well-known static socket IDs, see The Serial Link Protocol.

SlkReceivePacket

Purpose Receive and validate a packet for a particular socket or for any socket. Check for format and checksum errors.

Prototype

```
Err SlkReceivePacket (UInt16 socket,
Boolean andOtherSockets, SlkPktHeaderPtr headerP,
void* bodyP, UInt16 bodySize, Int32 timeout)
```

Parameters

-> `socket`	The socket ID.
-> `andOtherSockets`	If `TRUE`, ignore destination in packet header.
<-> `headerP`	Pointer to the packet header buffer (size of `SlkPktHeaderType`).
<-> `bodyP`	Pointer to the packet client data buffer.
-> `bodySize`	Size of the client data buffer (maximum client data size which can be accommodated).
-> `timeout`	Maximum number of system ticks to wait for beginning of a packet; -1 means wait forever.

Result

0	No error.
`slkErrSocketNotOpen`	The socket was not open.
`slkErrTimeOut`	Timed out waiting for a packet.

`slkErrWrongDestSocket`
The packet being received had an unexpected destination.

`slkErrChecksum` Invalid header checksum or packet CRC-16.

`slkErrBuffer` Client data buffer was too small for packet's client data.

If `andOtherSockets` is `FALSE`, this routine returns with an error code unless it gets a packet for the specific socket.

If `andOtherSockets` is `TRUE`, this routine returns successfully if it sees any incoming packet from the communications library used by `socket`.

Comments You may request to receive a packet for the passed socket ID only, or for any open socket which does not have a socket listener. The parameters also specify buffers for the packet header and client data, and a timeout. The timeout indicates how long the receiver should wait for a packet to begin arriving before timing out. If a packet is received for a socket with a registered socket listener, it will be dispatched via its socket listener procedure. On success, the packet header buffer and packet client data buffer is filled in with the actual size of the packet's client data in the packet header's `bodySize` field.

SlkSendPacket

Purpose Send a serial link packet via the serial output driver.

Prototype
```
Err SlkSendPacket (SlkPktHeaderPtr headerP,
SlkWriteDataPtr writeList)
```

Parameters

`<->` `headerP` Pointer to the packet header structure with client information filled in (see Comments).

`->` `writeList` List of packet client data blocks (see Comments).

Result

0	No error.
`slkErrSocketNotOpen`	The socket was not open.
`slkErrTimeOut`	Handshake timeout.

Comments `SlkSendPacket` stuffs the signature, client data size, and the checksum fields of the packet header. The caller must fill in all other packet header fields. If the transaction ID field is set to 0 (zero), the serial link manager automatically generates and stuffs a new non-zero transaction ID. The array of `SlkWriteDataType` structures enables the caller to specify the client data part of the packet as a list of noncontiguous blocks. The end of list is indicated by an array element with the `size` field set to 0 (zero). This call blocks until the entire packet is sent out or until an error occurs.

SlkSetSocketListener

Purpose Register a socket listener for a particular socket.

Prototype

```
Err SlkSetSocketListener (UInt16 socket,
SlkSocketListenPtr socketP)
```

Parameters

-> `socket`	Socket ID.
-> `socketP`	Pointer to a `SlkSocketListenType` structure.

Result

0	No error.
`slkErrBadParam`	Invalid parameter.
`slkErrSocketNotOpen`	The socket was not open.

Comments Called by applications to set up a socket listener.

Since the serial link manager does not make a copy of the `SlkSocketListenType` structure, but instead saves the passed pointer to it, the structure

- must **not** be an automatic variable (that is, local variable allocated on the stack)
- may be a global variable in an application
- may be a locked chunk allocated from the dynamic heap

The `SlkSocketListenType` structure specifies pointers to the socket listener procedure and the data buffers for dispatching packets destined for this socket. Pointers to two buffers must be specified: the packet header buffer (size of `SlkPktHeaderType`), and the packet body (client data) buffer. The packet body buffer must be large enough for the largest expected client data size. Both buffers may be application global variables or locked chunks allocated from the dynamic heap.

The socket listener procedure is called when a valid packet is received for the socket. Pointers to the packet header buffer and the packet body buffer are passed as parameters to the socket listener procedure.

NOTE: The application is responsible for freeing the `SlkSocketListenType` structure or the allocated buffers when the socket is closed. The serial link manager doesn't do it.

SlkSocketPortID

Purpose Get the port ID associated with a particular socket; for use with the new serial manager.

Prototype

```
ErrSlkSocketPortID (UInt16 socket,
UInt16 * portIDP)
```

Parameters

-> `socket`	The socket ID.
<-> `portIDP`	Pointer to location for returning the port ID.

Result

0	No error.
`slkErrSocketNotOpen`	The socket was not open.

Compatibility Implemented only if New Serial Manager Feature Set is present.

SlkSocketSetTimeout

Purpose Set the interbyte packet receive-timeout for a particular socket.

Prototype

```
Err SlkSocketSetTimeout (UInt16 socket,
Int32 timeout)
```

Parameters

-> `socket`	Socket ID.
-> `timeout`	Interbyte packet receive-timeout in system ticks.

Result

0	No error.
`slkErrSocketNotOpen`	The socket was not open.

69

Telephony Basic Services

This chapter provides reference material for the Telephony API, which you can use to interface with telephone systems and equipment. This chapter discusses the following topics:

- Telephony Data Structures
- Telephony Constants
- Telephony Functions
- Feature Support Functions

The header file `TelephonyMgr.h` declares the telephony manager API. The header file `TelephonyMgrType.h` declares the data structures that you use with the telephony manager API.

For more information about using the telephony manager, see Chapter 8, "Telephony Manager" in the *Palm OS Programmer's Companion*.

Telephony Service Types

The telephony API organizes functions within sets called **service sets**. Each service set contains a related set of functions that may or may not be available on a particular mobile device or network. You can use the `TelIs<ServiceSet>Available` function to determine if a service set is supported in the current environment, and you can use the `TelIs<FunctionName>Supported` to determine if a specific function is supported in the current environment.

The telephony API documentation has been split into several chapters. Each chapter covers one or more of the service sets, as shown in Table 69.1

Table 69.1 Telephony service types

Service prefix	Functionality	Chapter	Description
`Tel`	Basic	Chapter 69, "Telephony Basic Services."	Basic functions that are always available.
`TelCfg`	Configuration	Chapter 70, "Telephony Security and Configuration."	Allows configuration of the phone, including Short Message Services (SMS) configuration.
`TelDtc`	Data calls	Chapter 72, "Telephony Calls."	Functions for handling data calls.
`TelEmc`	Emergency calls	Chapter 72, "Telephony Calls."	Functions for handling emergency calls.
`TelInf`	Information	Chapter 69, "Telephony Basic Services."	Functions for retrieving information about the phone.
`TelNwk`	Network	Chapter 71, "Telephony Network."	Provides network oriented services, including authorization, signal level, search mode, and related operations.
`TelOem`	OEM	Chapter 69, "Telephony Basic Services."	Provides OEMs with the ability to incorporate custom functionality.
`TelPhb`	Phone book	Chapter 74, "Telephony Phone Book."	Functions for managing the phone book.

Table 69.1 Telephony service types *(continued)*

Service prefix	Functionality	Chapter	Description
`TelPow`	Power	Chapter 69, "Telephony Basic Services."	Provides access to power supply level.
`TelSms`	Short Message Service	Chapter 73, "Telephony SMS."	Addresses the SMS, including functions for reading, sending, replying to, and deleting short messages.
`TelSnd`	Sound	Chapter 69, "Telephony Basic Services."	Functions for playing key tones on and muting the phone.
`TelSpc`	Speech calls	Chapter 72, "Telephony Calls."	Function for handling voice calls, including dual tone modulated frequency (DTMF) sounds.
`TelSty`	Security	Chapter 70, "Telephony Security and Configuration."	Supports PIN code management for access to phone and Subscriber Identity Module (SIM) security-related features.

Telephony Data Structures

This section describes the data structures used with the basic services portion of the telephony API.



TelEventType

The `TelGetEvent` and `TelGetTelephonyEvent` functions both return a `TelEventType` structure to provide information about a telephony-related event.

You call the `TelGetEvent` function to retrieve telephony and other events.

You call the TelGetTelephonyEvent function to retrieve only telephony events. This function does not consume non-telephony events.

```
typedef struct _TelEventType
{
  eventsEnum   eType;
  Boolean      penDown;
  UInt8        tapCount;
  Int16        screenX;
  Int16        screenY;
  UInt16       functionId;
  UInt16       transId;
  MemPtr       *paramP;
  Err          returnCode;
} TelEventType;
```

Field Descriptions

`eType`	One of the eventsEnum constants. Specifies the type of the event.
`penDown`	`true` if the pen was down at the time of the event, and `false` if the pen was up. Note that this field is not filled in for telephony events.
`tapCount`	The number of taps received at this location. This value is used mainly by text fields. When the user taps in a text field, two taps selects a word, and three taps selects the entire line. Note that this field is not filled in for telephony events.
`screenX`	Window-relative position of the pen in pixels (number of pixels from the left bound of the window). Note that this field is not filled in for telephony events.

`screenY`	Window-relative position of the pen in pixels (number of pixels from the top left of the window). Note that this field is not filled in for telephony events.
`functionId`	The ID of the message associated with the function call, which specifies the telephony manager function that generated this event.
`transId`	The transaction ID that was associated with this asynchronous function call.
`paramP`	A pointer to a parameter structure that was passed when an asynchronous call was made.
`returnCode`	The return code of the asynchronously called function. The value of this field is `errNone` if the function succeeded, or an error code if the function failed.

Note that the first five fields of the `TelEventType` structure are the same as the first five fields of the `EventType` structure, which is described in Chapter 3, "Palm OS Events."



TelCallStateType

The `TelGetCallState` function uses the `TelCallState` structure to retrieve information about the current state of the connected phone.

```
typedef struct _TelGetCallStateType
{
  UInt8    state;
  UInt8    callType;
  UInt8    callServiceType;
  UInt8    numberSize;
  Char     *number;
} TelGetCallStateType;
```

Field Descriptions

<-	state	Filled in with one of the Telephone Call State Constants, which indicates the current state of the telephone call.
<-	callType	Filled in with one of the Telephone Call Type Constants, which indicates the call type of the current telephone call.
<-	callServiceType	Filled in with one of the Telephone Call Service Type Constants, which indicates the call service type of the current telephone call.
<->	numberSize	The size of the `number` string buffer. When the structure is used as an input parameter, this is the allocated size, in bytes, of the buffer. Upon return, this is the actual size of the string, including the null terminator character. If the `number` buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the data, and the function using this structure generates a `telErrBufferSize` error.
<-	number	A buffer into which the telephone number string is stored. When the structure describes an incoming telephone call, this is the incoming telephone number. When the structure describes an outgoing telephone call, this is the telephone number that has been called. Note that if this buffer is too small to contain the entire retrieved string, the end of the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBufferSize` error.



TelInfGetformationType

The `TelInfGetInformation` function uses the `TelInfGetInformationType` structure to retrieve information about the phone with which you are communicating.

```
typedef struct _TelInfGetInformationType
{
  UInt8      infoType;
  UInt8      size;
  UInt8      *value;
} TelInfGetInformationType;
```

Field Descriptions

->	`infoType`	The type of information to retrieve. This must be one of the Information Type Constants.
<->	`size`	The size of the `value` buffer. When the structure is used as an input parameter, this is the allocated size, in bytes, of the buffer. Upon return, this is the actual size of the buffer. If the `value` buffer is too small to contain all of the retrieved information, this field is assigned the entire length of the data, and the function using this structure generates a `telErrBufferSize` error.
<-	`value`	A buffer into which the information is stored.



TelOemCallType

You use the `TelOemCallType` to specify a `TelOemCall` function.

```
typedef struct _TelOemCallType
{
  UInt32  OemID;
  UInt8   funcID;
  void    *paramP;
} TelOemCallType;
```

Field Descriptions

->	`OemID`	The unique ID of the OEM function set.
->	`funcID`	The ID of the function within the OEM function set.
<->	`paramP`	A pointer to a parameter block that is passed to the OEM function. The OEM function might modify some of the fields in the parameter block.



New TelSendCommandStringType

The `TelSendCommandString` function uses the `TelSendCommandStringType` structure to send a command string.

```
typedef struct _TelSendCommandStringType
{
  Char    *commandString;
  Char    *resultString;
  UInt16  resultSize;
  UInt32  timeOut;
} TelSendCommandStringType;
```

Field Descriptions

->	`commandString`	The command string to send.
<-	`resultString`	The result string.
<->	`resultSize`	The size of the `resultString` string buffer. When the structure is used as an input parameter, this is the allocated size, in bytes, of the buffer. Upon return, this is the actual size of the string, including the null terminator character. If the `resultString` buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the data, and the function using this structure generates a `telErrBufferSize` error.
->	`timeOut`	The number of milliseconds before timing out.

New TelSndPlayKeyToneType

The `TelSndPlayKeyTone` function uses the `TelSndPlayKeyToneType` structure to specify a key tone.

```
typedef struct _TelSndPlayKeyToneType
{
  UInt8    keycode;
  UInt8    type;
} TelSndPlayKeyToneType;
```

Field Descriptions

->	`keycode`	The keycode of the key tone to play. This must be one of the Keycode Constants.
->	`type`	The tone type. This must be one of the Key Sound Type Constants.

Telephony Constants

This section describes the data structures used with the basic services portion of the telephony API, which include the following constant types:

- Battery Status Constants
- Telephone Call State Constants
- Telephone Call Type Constants
- Telephone Call Service Type Constants
- Error Code Constants
- Information Type Constants
- Keycode Constants
- Key Sound Type Constants
- Message Identifier Constants
- Service Set Constants

Battery Status Constants

The battery status constants provide information about the phone's battery.

Constant	Value	Description
`kTelPowBatteryPowered`	`0`	The phone is powered by a battery.
`kTelPowBatteryNotPowered`	`1`	The phone has a battery connected to it, but is not using that battery.
`kTelPowNobattery`	`2`	The phone does not have a battery connected to it.
`kTelPowBatteryFault`	`3`	The phone has a recognized power fault; calls are currently inhibited.

Telephone Call State Constants

The `TelCallStateType` structure uses the telephone call state constants to encode the current state of the connected telephone call.

Constant	Value	Description
`kTelCallIdle`	`0x00`	The connection is idle.
`kTelCallConnecting`	`0x01`	A telephone call is currently connecting.
`kTelCallConnected`	`0x02`	A telephone call is currently connected.
`kTelCallRedial`	`0x03`	A telephone call is being re-dialed.
`kTelCallIncoming`	`0x04`	A telephone call is currently incoming.
`kTelCallIncomingAck`	`0x05`	An incoming telephone call is currently being acknowledged.
`kTelCallDisconnecting`	`0x06`	A telephone call is being disconnected.

Telephone Call Type Constants

The TelCallStateType structure uses the telephone call type constants to encode the type of the current telephone call.

Constant	Value	Description
kTelCallTypeOutgoing	0x00	An outgoing telephone call.
kTelCallTypeIncoming	0x01	An incoming telephone call.

Telephone Call Service Type Constants

The TelCallStateType structure uses the telephone call service type constants to encode the service type of the current telephone call.

Constant	Value	Description
kTelCallServiceVoice	0x00	A voice telephone call.
kTelCallServiceData	0x01	A data telephone call.

Error Code Constants

The telephony manager functions return the error code constants shown in the following table to indicate their status.

Constant	Description
telErrBufferSize	One of the buffers used to retrieve data is too small.
telErrCodingScheme	The coding scheme specified for the short message is not valid.
telErrCommandFailed	The specified command could not be performed by the phone. Check the phone driver.
telErrEntryNotFound	The specified entry was not found.
telErrFeatureNotSupported	The specified feature is not supported by the phone or network.

Constant	Description
`telErrGenericDrvNotFound`	The generic driver could not be found.
`telErrInvalidAppId`	The specified application ID is not valid.
`telErrInvalidDial`	The dial string contains an invalid character.
`telErrInvalidIndex`	The index specified for accessing a value in storage is incorrect.
`telErrInvalidParameter`	A parameter is not valid.
`telErrInvalidString`	The text string contains an invalid character.
`telErrLibStillInUse`	The shared lib is currently being used by another application. Do not unload it!
`telErrMemAllocation`	A memory allocation error occurred.
`telErrMsgAllocation`	The telephony messages pool is empty; a message could not be allocated.
`telErrNetworkTimeOut`	The network did not reply within the standard time delay amount.
`telErrNoNetwork`	There is no network available.
`telErrNoSIMInserted`	The SIM card is not inserted.
`telErrNoSpecificDrv`	A specific driver was not specified.
`telErrNotInstalled`	The shared library could not be installed.
`telErrPassword`	The password is not correct.
`telErrPhoneCodeRequired`	A phone code is required.
`telErrPhoneComm`	The communication link with the phone is down.
`telErrPhoneMemAllocation`	The phone's memory is full.
`telErrPhoneMemFailure`	The phone encountered a memory error.

Constant	Description
`telErrPhoneNumber`	One of the following errors has occurred: the phone number is wrong, the SMS center is not valid, or the receiver phone number is wrong for the SMS.
`telErrPhoneReply`	The phone reply syntax is incorrect. Check the phone driver.
`telErrPhoneToSIMPINRequired`	A phone 2 SIM PIN code is required
`telErrPIN2Required`	A PIN2 code is required.
`telErrPINRequired`	A PIN code is required.
`telErrPUK2Required`	A PUK2 code is required.
`telErrPUKRequired`	A PUK code is required.
`telErrResultBusyResource`	A resource is busy.
`telErrResultTimeOut`	A time-out was reached.
`telErrResultUserCancel`	The user cancelled the action.
`telErrSecurity`	Access to the phone was not granted.
`telErrSettings`	The telephony settings are not valid; this is due to 1) the Phone Panel preferences do not exist, or 2) the Telephony Profile is not correctly set.
`telErrSIMBusy`	The SIM could not reply.
`telErrSIMFailure`	The SIM is not working properly.
`telErrSIMWrong`	The SIM is not accepted by the phone.
`telErrSpcCallError`	The voice telephone call encountered an error.
`telErrSpcLineIsBusy`	The voice telephone call failed.
`telErrSpcLineIsReleased`	The voice telephone call has been released.
`telErrSpecificDrvNotFound`	The specified driver could not be found.
`telErrTooManyApps`	The applications table is full.

Constant	Description
`telErrTTaskNotFound`	The Telephony Task could not be found.
`telErrTTaskNotRunning`	The Telephony Task is not running.
`telErrUnavailableValue`	The requested value can not be retrieved at the specified time. This is usually due to a `TelSpcGetCallerNumber` request when there is no active line.
`telErrUnknown`	An unknown telephony manager error occurred.
`telErrValidityPeriod`	The validity period specified for the short message is not valid.
`telErrValueStale`	The information could not be retrieved; a copy of the most recently retrieved value has been returned instead.
`telErrVersion`	The shared library version does not match the version associated with the application.

Information Type Constants

The `TelInfGetformationType` structure uses the information type constants to encode the type of information to retrieve about the phone.

Constant	Value	Description
`kTelInfPhoneBrand`	`0`	The brand name of the phone.
`kTelInfPhoneModel`	`1`	The model number of the phone.
`kTelInfPhoneRevision`	`2`	The revision number of the phone.

Keycode Constants

The `TelSndPlayKeyToneType` structure uses the keycode constants to specify the key tone to play.

Constant	Value	Description
kTel0Key	0x00	The 0 key on the phone keypad.
kTel1Key	0x01	The 1 key on the phone keypad.
kTel2Key	0x02	The 2 key on the phone keypad.
kTel3Key	0x03	The 3 key on the phone keypad.
kTel4Key	0x04	The 4 key on the phone keypad.
kTel5Key	0x05	The 5 key on the phone keypad.
kTel6Key	0x06	The 6 key on the phone keypad.
kTel7Key	0x07	The 7 key on the phone keypad.
kTel8Key	0x08	The 8 key on the phone keypad.
kTel9Key	0x09	The 9 key on the phone keypad.
kTelPoundKey	0x23	The POUND(#) key on the phone keypad.
kTelStarKey	0x2A	The STAR(*) key on the phone keypad.
kTelSendKey	0x45	The SEND key on the phone keypad.
kTelEndKey	0x46	The END key on the phone keypad.
kTelClrKey	0x47	The CLEAR key on the phone keypad.
kTelSaveKey	0x48	The SAVE key on the phone keypad.

Key Sound Type Constants

The TelSndPlayKeyToneType structure uses the key sound type constants to specify how the tone is played.

Constant	Value	Description
kTelSndSingleTone	0x00	Play the key sound as a single tone.
kTelSndMultiTones	0x01	Play the key sound as a multiple tones.

Message Identifier Constants

The message identifier constants are used with asynchronous calls to identify which telephony function is being or has been called. The `TelMessages` enumeration defines a constant for each function name.

Each message identifier constant has the form:

`k`*`functionName`*`Message`

where *functionName* is replaced by a function name.

The following table shows examples of message identifier constants. For a complete list, see the `TelephonyMgr.h` file.

Constant	Function
`kTelGetCallStateMessage`	`TelGetCallState`
`kTelNwkSelectNetworkMessage`	`TelNwkSelectNetwork`
`kTelSmsReadMessageMessage`	`TelSmsReadMessage`

Service Set Constants

The service set constants specify a set of API services.

Constant	Value	Description
`kTelNwkServiceId`	`0`	The network service set.
`kTelStyServiceId`	`1`	The security service set.
`kTelPowServiceId`	`2`	The power service set.
`kTelCfgServiceId`	`3`	The configuration service set.
`kTelSmsServiceId`	`4`	The short message service set.
`kTelEmcServiceId`	`5`	The emergency telephone call service set.
`kTelSpcServiceId`	`6`	The speech telephone call service set.
`kTelDtcServiceId`	`7`	The data telephone call service set.
`kTelPhbServiceId`	`8`	The phone book service set.

Constant	Value	Description
kTelOemServiceId	9	The OEM service set.
kTelSndServiceId	10	The sound service set.
kTelInfServiceId	11	The information service set.

Telephony Functions

This section describes the functions used with the basic services portion of the telephony API.



New TelCancel

Purpose

Cancels an asynchronous function call.

Prototype

```
Err TelCancel(UInt16 iRefnum, TelAppID iAppId,
UInt16 iTransId, UInt16 *ioTransIdP);
```

Parameters

-> iRefnum	The telephony manager library reference number.
-> iAppId	The telephone application attachment identifier for your application.
-> iTransId	The transaction ID associated with the function that you are cancelling.
<-> ioTransIdP	Set the value of this parameter to NULL to cause the function to execute synchronously. If this parameter is not NULL, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns errNone if the function call was successfully cancelled. Returns the telErrCommandFailed error code if the function call could not be cancelled.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`functionId`	`kTelUrqCancelMessage`
`paramId`	Points to the unsigned integer value passed to this function in the `iTransId` parameter.

Comments

This function cancels a pending asynchronous function call. You can cancel any asynchronous call except for an asynchronous call to the `TelCancel` function.

The function call that is cancelled returns the `telErrUserCancel` error code.

Compatibility

Implemented only if 4.0 New Feature Set is present.



New TelClose

Purpose

Close the shared library.

Prototype

```
Err TelClose(UInt16 iRefnum, TelAppID iAppId);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.

Result

Returns an error code, or error none if the library was successfully closed. If the library is currently being used by another application, this function returns the `telErrLibStillInUse` error code.

Comments

Call this function when you are done with the telephony manager. You can only use this function synchronously.

If no other application is using the telephony manager, this function stops the Telephony task and releases any resources used by the telephony manager.

See Also TelOpen



New TelClosePhoneConnection

Purpose Closes down communications with the connected phone.

Prototype

```
Err TelClosePhoneConnection(UInt16 iRefnum,
TelAppID iAppId, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelUrqClosePhoneConnectionMessage`

Comments Call this function when you have finished communications with the phone and are ready to disconnect from it.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also TelOpenPhoneConnection



New TelGetCallState

Purpose Retrieves the current telephone call state information.

Prototype

```
Err TelGetCallState(UInt16 iRefnum,
TelAppID iAppId, TelGetCallStateType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelCallStateType` structure that describes the state of the current telephone call. On input, the `numberSize` field of this structure specifies the allocated size of the `number` buffer. Upon return, the `numberSize` field specifies the actual size of the telephone number, even if it was truncated to fit into the buffer.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelCallStateType` structure passed to this function in the `ioCallState` parameter.
`functionId`	`kTelGetCallStateMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

This function retrieves information about the current telephone call state of the connection with the phone, and stores that information into the supplied `TelCallStateType` structure.

The current incoming or outgoing telephone call number is stored into the `number` field of the `TelCallStateType` structure referenced by `ioCallStateP`. If the `number` field buffer is too small to contain the complete telephone number, the string is truncated (and ends with the null terminator character) and this function returns the `telErrBufferSize` error. The `numberSize` field of the structure is always updated to contain the actual size of the complete telephone number.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelSpcAcceptCall`, `TelSpcCallNumber`



New TelGetEvent

Purpose Retrieves events for applications that use the telephony manager.

Prototype

```
void TelGetEvent(UInt16 iRefnum, TelAppID iAppId,
EventPtr oEventP, Int32 iTimeOut);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oEventP`	A pointer to a `TelEventType` structure. Upon return, the structure contains the event information, which you should use as described in the Comments section.
`-> iTimeout`	Maximum number of ticks to wait before an event is returned (`evtWaitForever` means wait indefinitely).

Result Returns nothing.

Comments This function retrieves both telephony and standard Palm OS® events. You must call this function to retrieve events in any application that is running in the UI task and using the telephony manager.

Upon return from this function, you need to test the type of the event by examining the `oEventP->type` field. If the event type is a telephony event, then you need to cast the pointer as follows to access the fields:

```
TelEventType *telEventP =
                        (TelEventType *)oEventP;
```

This function calls both the `EvtGetEvent` and `TelGetTelephonyEvent` functions to retrieve the next event for your application.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also EvtGetEvent, TelGetTelephonyEvent



TelGetTelephonyEvent

Purpose Retrieves telephony events only.

Prototype

```
void TelGetTelephonyEvent(UInt16 iRefnum,
TelAppID iAppId, EventPtr oEventP,
Int32 iTimeOut);
```

Parameters

-> iRefnum	The telephony manager library reference number.
-> iAppId	The telephone application attachment identifier for your application.
<- oEventP	A pointer to a TelEventType structure. Upon return, the structure contains the event information.
-> iTimeout	Maximum number of ticks to wait before an event is returned (evtWaitForever means wait indefinitely).

Result Returns nothing.

Comments Use this function instead of the TelGetEvent function when you only want to process telephony events.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also EvtGetEvent, TelGetEvent



New TelInfGetInformation

Purpose

Retrieve brand, model, and revision information for the phone.

Prototype

```
Err TelInfGetInformation(UInt16 iRefnum,
TelAppID iAppId,
TelInfGetInformationType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelInfGetformationType` structure. On input, the `infoType` field of the structure contains the type of information that you want retrieved. The `size` field of this structure specifies the allocated size of the `value` buffer. Upon return, the `size` field specifies the actual size of the information that was retrieved, even if it was truncated to fit into the buffer.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelInfGetformationType` structure passed to this function in the `ioInfoP` parameter.
`functionId`	`kTelInfGetInformationMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

Call this function to retrieve information about the currently connected phone.

The retrieved information is stored into the `value` field of the `TelInfGetformationType` referenced by `ioInfoP` structure. If the `value` field buffer is too small to contain the complete information, the value is truncated and this function returns the `telErrBufferSize` error. The `size` field of the structure is always updated to contain the actual size of the retrieved information.

Before using this function, you should verify that it is available by calling the `TelIsInfServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.



New TelIsCfgServiceAvailable

Purpose

A macro that determines if the configuration service set is available in the current environment.

Prototype

```
TelIsCfgServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelCfgServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments You need to call this macro before calling any function in the configuration service set, which is the family of functions that begin with the `TelCfg` prefix.

The configuration service set functions are documented in Chapter 73, "Telephony SMS."

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelIs<FunctionName>Supported`



New TelIsDtcServiceAvailable

Purpose A macro that determines if the data calls service set is available in the current environment.

Prototype

```
TelIsDtcServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelDtcServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

CommentsI

You need to call this macro before calling any function in the data calls service set, which is the family of functions that begin with the `TelDtc` prefix.

The data calls service set functions are documented in Chapter 72, "Telephony Calls."

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelIs<FunctionName>Supported`



TelIsEmcServiceAvailable

Purpose

A macro that determines if the emergency calls service set is available in the current environment.

Prototype

```
TelIsEmcServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.

`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelEmcServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments

You need to call this macro before calling any function in the emergency calls service set, which is the family of functions that begin with the `TelEmc` prefix.

The emergency calls service set functions are documented in Chapter 72, "Telephony Calls."

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelIs<FunctionName>Supported`



New TelIsInfServiceAvailable

Purpose

A macro that determines if the information service set is available in the current environment.

Prototype

```
TelIsInfServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelInfServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments

You need to call this macro before calling any function in the information service set, which is the family of functions that begin with the `TelInf` prefix.

The information service set functions are documented in this chapter.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelIs<FunctionName>Supported`



TelIsNwkServiceAvailable

Purpose A macro that determines if the network service set is available in the current environment.

Prototype

```
TelIsNwkServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelNwkServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments You need to call this macro before calling any function in the network service set, which is the family of functions that begin with the `TelNwk` prefix.

The network service set functions are documented in Chapter 71, "Telephony Network."

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelIs<FunctionName>Supported`



New TelIsOemServiceAvailable

Purpose A macro that determines if the OEM service set is available in the current environment.

Prototype

```
TelIsOemServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelOemServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments

You need to call this macro before calling any function in the OEM service set, which is the family of functions that begin with the `TelOem` prefix.

The OEM service set functions are documented in this chapter.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelIs<FunctionName>Supported`

New TelIsPhbServiceAvailable

Purpose

A macro that determines if the phone book service set is available in the current environment.

Prototype

```
TelIsPhbServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.

`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelPhbServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments

You need to call this macro before calling any function in the phone book service set, which is the family of functions that begin with the `TelPhb` prefix.

The phone book service set functions are documented in Chapter 74, "Telephony Phone Book."

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelIs<FunctionName>Supported`



New TelIsPhoneConnected

Purpose

Determines if a phone is connected.

Prototype

```
Err TelIsPhoneConnected(UInt16 iRefnum,
TelAppID iAppId, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelUrqIsPhoneConnectedMessage`

Comments

Call this function to determine if there is currently a phone connected.

Compatibility

Implemented only if 4.0 New Feature Set is present.



New TelIsPowServiceAvailable

Purpose

A macro that determines if the power services set is available in the current environment.

Prototype

```
TelIsPowServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelPowServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments

You need to call this macro before calling any function in the power service set, which is the family of functions that begin with the `TelPow` prefix.

The power service set functions are documented in this chapter.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelIs<FunctionName>Supported`



TelIsSmsServiceAvailable

Purpose A macro that determines if the Short Message Service (SMS) service set is available in the current environment.

Prototype

```
TelIsSmsServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelSmsServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments You need to call this macro before calling any function in the SMS service set, which is the family of functions that begin with the `TelSms` prefix.

The SMS service set functions are documented in Chapter 73, "Telephony SMS."

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelIs<FunctionName>Supported`



New TelIsSndServiceAvailable

Purpose A macro that determines if the sound service set is available in the current environment.

Prototype

```
TelIsSndServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelSndServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments

You need to call this macro before calling any function in the sound service set, which is the family of functions that begin with the `TelSnd` prefix.

The sound service set functions are documented in this chapter.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelIs<FunctionName>Supported`

New TelIsSpcServiceAvailable

Purpose

A macro that determines if the speech telephone call service set is available in the current environment.

Prototype

```
TelIsSpcServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelSpcServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments You need to call this macro before calling any function in the speech telephone call service set, which is the family of functions that begin with the `TelSpc` prefix.

The speech telephone call service set functions are documented in Chapter 72, "Telephony Calls."

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelIs<FunctionName>Supported`



New TelIsStyServiceAvailable

Purpose

A macro that determines if the security service set is available in the current environment.

Prototype

```
TelIsStyServiceAvailable (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the service set is available, or an error code if not.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	`kTelStyServiceId`
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments

You need to call this macro before calling any function in the security service set, which is the family of functions that begin with the `TelSty` prefix.

The security service set functions are documented in Chapter 71, "Telephony Network."

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelIs<FunctionName>Supported`



New TelMatchPhoneDriver

Purpose Determines if the currently selected driver matches the connected phone.

Prototype

```
Err TelMatchPhoneDriver(UInt16 iRefnum,
TelAppID iAppId, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelUrqMatchPhoneDriverMessage`

Compatibility

Implemented only if 4.0 New Feature Set is present.



TelOemCall

Purpose

Pass a call to an OEM function.

Prototype

```
Err TelOemCall(UInt16 iRefnum, TelAppID iAppId,
TelOemCallType *ioParamP, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelOemCallType` structure that contains information about the OEM function call.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transID`	matches the output value of the `ioTransIdP` parameter
`functionId`	matches the function ID in the `TelOemCallType` structure passed to this function in the `ioParamP` parameter
`paramId`	points to the `TelOemCallType` structure passed to this function in the `ioParamP` parameter

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

Call this function to send a request to an OEM function. The calling function and the OEM function are responsible for coordinating the parameter block that is passed in the `TelOemCallType` structure.

Before using this function, you should verify that it is available by calling the `TelIsOemServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.



New

TelOpen

Purpose

Open the telephony manager API to initialize telephony services and launch the telephony task.

Prototype

```
Err TelOpen(UInt16 iRefnum, UInt32 iVersnum,
TelAppID *oAppIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iVersnum`	The version number of the shared library for which your application was developed.
`<- oAppIdP`	A pointer to an application ID value. Upon return, this is the application ID that you supply as a parameter to the any other telephony functions that you call.

Result

Returns `errNone` if the function was successful or returns an error code if not successful. The following errors can occur:

- the telephony task could not be found (`telErrTTaskNotFound`)
- the telephony task could not be launched (`telErrTTaskNotRunning`)
- the phone driver could not be found
- the shared library version is not valid

Comments

You can only call this function synchronously. You must call this function before calling any other telephony manager functions.

You can specify the current version of the shared library by using the `kTelMgrVersion` constant as the value of the `iVersnum` parameter.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelClose`



New TelOpenPhoneConnection

Purpose Open communications with the connected phone.

Prototype

```
Err TelOpenPhoneConnection(UInt16 iRefnum,
TelAppID iAppId, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelUrqOpenPhoneConnectionMessage`

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelClosePhoneConnection`

New TelPowGetBatteryStatus

Purpose

Retrieves the status of the phone's battery.

Prototype

```
Err TelPowGetBatteryStatus(UInt16 iRefnum,
TelAppID iAppId, UInt8 *oStatusP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oStatusP`	A pointer to an unsigned byte value. Upon return, this is the battery status value, which is one of the Battery Status Constants.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oStatusP` parameter.
`functionId`	`kTelPowBatteryStatusMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oStatusP` remains in memory until the asynchronous call completes.

Comments Before using this function, you should verify that it is available by calling the `TelIsPowServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelPowGetPowerLevel`



New TelPowGetPowerLevel

Purpose Retrieve the current level of the phone battery, as a percentage value.

Prototype

```
Err TelPowGetPowerLevel(UInt16 iRefnum,
TelAppID iAppId, UInt8 *oPowerP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oPowerP`	A pointer to an unsigned byte value. Upon return, this is the battery percentage value.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oPowerP` parameter.
`functionId`	`kTelPowGetPowerLevelMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oPowerP` remains in memory until the asynchronous call completes.

Comments

The returned percentage value is an integer value between 0 and 100.

Before using this function, you should verify that it is available by calling the `TelIsPowServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelPowGetBatteryStatus`



New TelPowSetPhonePower

Purpose Turns the phone on or off.

Prototype

```
Err TelPowSetPhonePower(UInt16 iRefnum,
TelAppID iAppId, Boolean iPowerOn);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iPowerOn`	Set to `true` to turn the phone on, and set to `false` to turn the phone off.

Result Returns `errNone` if the function was successful and an error code if not.

Comments This function can only be called synchronously.

Before using this function, you should verify that it is available by calling the `TelIsPowServiceAvailable` macro.

This function corresponds to the `kTelPowSetPhonePowerMessage` function ID value.

Compatibility Implemented only if 4.0 New Feature Set is present.



New TelSendCommandString

Purpose Sends a command string to the phone or to the network.

Prototype

```
Err TelSendCommandString(UInt16 iRefnum,
TelAppID iAppId,
TelSendCommandStringType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a command string structure of type `TelSendCommandStringType`.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSendCommandStringType` structure passed to this function in the `ioParam` parameter.
`functionId`	Matches the function ID in the `TelOemCallType` referenced by the `ioParamP` structure.

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Compatibility

Implemented only if 4.0 New Feature Set is present.



New TelSndMute

Purpose

Mute or un-mute an active telephone call.

Prototype

```
Err TelSndMute(UInt16 iRefnum, TelAppID iAppId,
Boolean iMuteOn, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iMuteOn`	Set to `true` to mute the telephone call, or set to `false` to unmute the telephone call.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the Boolean value passed to this function in the `iMuteOn` parameter.
`functionId`	`kTelSndMuteMessage`

Comments Before using this function, you should verify that it is available by calling the `TelIsSndServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSndPlayKeyTone`



New TelSndPlayKeyTone

Purpose Play a keytone sound on the phone.

Prototype

```
Err TelSndPlayKeyTone(UInt16 iRefnum,
TelAppID iAppId, TelSndPlayKeyToneType *iParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iParamP`	A pointer to a `TelSndPlayKeyToneType` structure that describes the tone to play.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSndPlayKeyToneType` structure passed to this function in the `iKeyToneP` parameter.
`functionId`	`kTelSndPlayKeyTone`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iParamP` remains in memory until the asynchronous call completes.

Comments Before using this function, you should verify that it is available by calling the `TelIsSndServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSndStopKeyTone`



New TelSndStopKeyTone

Purpose Stop the playing of a keytone sound on the phone.

Prototype

```
Err TelSndStopKeyTone(UInt16 iRefnum,
TelAppID iAppId, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.

`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelSndStopKeyToneMessage`

Comments Before using this function, you should verify that it is available by calling the `TelIsSndServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSndPlayKeyTone`

Feature Support Functions

This section describes the functions that you can call to determine if a specific feature or function is supported in the current operating environment.



New TelIs<FunctionName>Supported

Purpose

Determines if the specified function is supported.

Prototype

```
TelIs<FunctionName>Supported (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the specified function is supported.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the specified function is supported, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	The function ID of the function for which you are testing. For example, if you call `TelIsCgfGetSmsCenterSupported`, the value of this field is `kTelCfgGetSmsCenterMessage`. The function ID value for each function is described in the documentation for the function.
`functionId`	`kTelUrqIsFunctionSupportedMessage`

Comments This is a family of synchronous macros that test if a specific function is available in the current environment.

To use the macro, substitute a function name for the *<FunctionName>* portion of the macro name. You can substitute any Telephony Manager function name; for a complete list of the Telephony Manager functions, see "Summary of Telephony Manager" on page 197 in *Palm OS Programmer's Companion*, vol. II, *Communications*.

For example, to determine if the `TelNwkGetSignalLevel` function is available in the current environment, call the `TelIsNwkGetSignalLevelSupported` macro.

NOTE: A service set can be available without all of its functions being available. Thus, if the `TelIs<ServiceSet>Available` macro returns `true` for a specific service set, you know that the service set is available, but you need to call `TelIs<FunctionName>Supported` to determine if a specific function is available.

This macro corresponds to the `kTelUrqIsFunctionSupportedMessage` function ID value.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelIs<ServiceSet>Available`

New TelIs<ServiceSet>Available

Purpose Determines if the specified service set is available.

Prototype

```
TelIs<ServiceSet>Available (iRefnum, iAppId,
ioTransIdP);
```

Parameters

`-> iRefnum` The telephony manager library reference number.

`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the service set is available.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` if the service set is available, or an error code if not.
`transId`	The transaction ID of the operation.
`paramP`	The service ID of the service set for which you are testing. For example, if you call `TelIsCfgServiceAvailable`, the value of this field is `kTelCfgServiceId`. The service IDs are described in Service Set Constants.
`functionId`	`kTelUrqIsServiceAvailableMessage`

Comments

This is a family of synchronous macros that test if a specific service set is available. You must call the appropriate set availability function before calling a function in the set.

NOTE: A service set can be available without all of its functions being available. You can use this macro to determine the availability of a specific service set, which you might use to determine the configuration of your applications' user interface. To test if a specific function is supported, use the `TelIs<FunctionName>Supported` macro.

You can call these specific macros to determine if the service set is available:

- `TelIsCfgServiceAvailable` to determine if the configuration service set is available.
- `TelIsDtcServiceAvailable` to determine if the data calls service set is available.
- `TelIsEmcServiceAvailable` to determine if the emergency calls service set is available.
- `TelIsInfServiceAvailable` to determine if the information service set is available.
- `TelIsNwkServiceAvailable` to determine if the network service set is available.
- `TelIsOemServiceAvailable` to determine if the OEM service set is available.
- `TelIsPhbServiceAvailable` to determine if the phone book service set is available.
- `TelIsPowServiceAvailable` to determine if the power service set is available.
- `TelIsSmsServiceAvailable` to determine if the SMS service set is available.
- `TelIsSndServiceAvailable` to determine if the sound service set is available.
- `TelIsSpcServiceAvailable` to determine if the speech calls service set is available.
- `TelIsStyServiceAvailable` to determine if the security service set is available.

Each of these macros corresponds to the `kTelUrqIsServiceSupportedMessage` function ID value.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelIs<FunctionName>Supported`

70

Telephony Security and Configuration

This chapter describes the telephony security and configuration service sets of the telephony API.

For more information about the telephony manager basic services and the different service sets, see Chapter 69, "Telephony Basic Services."

This chapter describes:

- Telephony Security and Configuration Data Structures
- Telephony Security and Configuration Constants
- Telephony Security and Configuration Functions

For more information about using the telephony manager, see Chapter 8, "Telephony Manager" in the *Palm OS Programmer's Companion*.

Telephony Security and Configuration Data Structures

This section describes the data structures used with the telephony security and configuration service sets portion of the telephony API.



TelCfgGetPhoneNumberType

The `TelCgfGetPhoneNumber` function uses a `TelCfgGetPhoneNumberType` structure to retrieve the connected phone dial number.

```
typedef struct _TelCfgGetPhoneNumberType
  UInt8    size;
  Char*    value;
} TelCfgGetPhoneNumberType;
```

Field Descriptions

`<->` `size` — The size of the `value` buffer.

When the structure is used as an input parameter, this is the allocated size, in bytes, of the `value` buffer.

Upon return, this is the actual size of the string, including the null terminator character. If the `value` buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.

`<-` `value` — A buffer into which the dial number is stored.

Note that if this buffer is too small to contain the entire retrieved string, the end of the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBufferSize` error.



New TelCfgGetSmsCenterType

The `TelCfgGetSmsCenter` function uses a `TelCfgGetSmsCenterType` structure to retrieve the SMS service center dial number.

```
typedef struct _TelCfgGetSmsCenterType
  UInt8   size;
  Char*   value;
} TelCfgGetSmsCenterType;
```

Field Descriptions

`<->` `size` — The size of the `value` buffer.

When the structure is used as an input parameter, this is the allocated size, in bytes, of the `value` buffer.

Upon return, this is the actual size of the string, including the null terminator character. If the `value` buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.

`<-` `value` — A buffer into which the dial number is stored.

Note that if this buffer is too small to contain the entire retrieved string, the end of the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBufferSize` error.



New TelStyChangeAuthenticationType

You use the `TelStyChangeAuthenticationType` to change an authentication code with the `TelStyChangeAuthenticationCode` function.

```
typedef struct _TelStyChangeAuthenticationType
{
  UInt    codeId;
  Char*   oldCode;
  Char*   newCode;
} TelStyChangeAuthenticationType;
```

Field Descriptions

`->` `codeId` — The ID of the authentication code to change.

`->` `oldCode` — The previous value of the code.

`->` `newCode` — The new value of the code.

Telephony Security and Configuration Constants

This section describes the constants used with the telephony security and configuration service sets of the telephony API.

Authentication State Constants

The authentication state constants describe the current authentication state of the mobile unit connection.

Constant	Value	Description
`kTelStyReady`	`0`	No additional security information is expected.
`kTelStyPin1CodeId`	`1`	The PIN1 code is expected.
`kTelStyPin2CodeId`	`2`	The PIN2 code is expected.
`kTelStyPuk1CodeId`	`3`	The PUK1 code is expected.
`kTelStyPuk2CodeId`	`4`	The PUK2 code is expected.
`kTelStyPhoneToSimCodeId`	`5`	The phone-to-SIM code is expected.
`kTelStyFirstOemCodeId`	`6`	An OEM code is expected. The constant `kTelStyFirstOemCodeId` specifies the first OEM authentication code. You can specify additional OEM codes by incrementing this value. For example, to specify the third OEM authentication code, use the following: `kTelStyFirstOemCodeId+2`

Telephony Security and Configuration Functions

This section describes the data structures used with the telephony security and configuration service sets portion of the telephony API.



New TelCgfGetPhoneNumber

Purpose

Retrieve the connected telephone number.

Prototype

```
Err TelCfgGetPhoneNumber(UInt16 iRefnum,
TelAppID iAppId,
TelCfgGetPhoneNumberType* ioParamP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelCfgGetPhoneNumberType` structure that is filled in with the dial telephone number.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelCfgGetPhoneNumberType` structure passed to this function in the `ioParamP` parameter.
`functionId`	`kTelCfgGetPhoneNumberMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

The connected dial telephone number is stored into the `value` field of the `TelCfgGetPhoneNumberType` structure referenced by `ioParamP`. If the `value` field buffer is too small to contain the complete telephone number, the string is truncated (and ends with the null terminator character) and this function returns the `telErrBufferSize` error. The `size` field of the structure is always updated to contain the actual size of the complete telephone number.

Before using this function, you should verify that it is available by calling the `TelIsCfgServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelCfgSetSmsCenter`, `TelSmsSendMessage`



New TelCfgGetSmsCenter

Purpose

Retrieve the SMS service center dial telephone number.

Prototype

```
Err TelCfgGetSmsCenter(UInt16 iRefnum,
TelAppID iAppId, TelCfgGetSmsCenterType* ioParamP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelCfgGetSmsCenterType` structure that is filled in with the dial telephone number.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelCfgGetSmsCenterType` structure passed to this function in the `ioParamP` parameter.
`functionId`	`kTelCfgGetSmsCenterMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments The service center dial telephone number is stored into the `value` field of the `TelCfgGetSmsCenterType` structure referenced by `ioParamP`. If the `value` field buffer is too small to contain the complete telephone number, the string is truncated (and ends with the null terminator character) and this function returns the `telErrBufferSize` error. The `size` field of the structure is always updated to contain the actual size of the complete telephone number.

Before using this function, you should verify that it is available by calling the `TelIsCfgServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelCfgSetSmsCenter`, `TelSmsSendMessage`



TelCfgSetSmsCenter

Purpose Set the SMS service center dial telephone number.

Prototype

```
Err TelCfgSetSmsCenter(UInt16 iRefnum,
TelAppID iAppId, const Char* iDialNumberP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum` The telephony manager library reference number.

`-> iAppId` The telephone application attachment identifier for your application.

`-> iDialNumberP` A pointer to the null-terminated dial telephone number string for the SMS service center.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the string passed to this function in the `iDialNumberP` parameter.
`functionId`	`kTelCfgSetSmsCenterMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `iDialNumberP` remains in memory until the asynchronous call completes.

Comments

Before using this function, you should verify that it is available by calling the `TelIsCfgServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelCfgGetSmsCenter`, `TelSmsSendMessage`



New TelStyChangeAuthenticationCode

Purpose

Change the value of an authentication code. Note that you can only use this function with GSM networks.

Prototype

```
Err TelStyChangeAuthenticationCode(UInt16 iRefnum,
TelAppID iAppId,
TelStyChangeAuthenticationType* iParamP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iParamP`	A pointer to a `TelStyChangeAuthenticationType` structure that contains the old and new authentication code values.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.

`paramP`	Points to the `TelStyChangeAuthenticationType` structure passed to this function in the `iParamP` parameter.
`functionId`	`kTelStyChangeAuthenticationCodeMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iParamP` remains in memory until the asynchronous call completes.

Comments Before using this function, you should verify that it is available by calling the `TelIsStyServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelStyEnterAuthenticationCode`



TelStyEnterAuthenticationCode

Purpose Enter the authentication code for which the phone is currently waiting. Note that you can only use this function with GSM networks.

Prototype

```
Err TelStyEnterAuthenticationCode(UInt16 iRefnum,
TelAppID iAppId, const Char* iCodeP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iCodeP`	A pointer to the null-terminated authentication code string to send to the phone.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the string passed to this function in the `iCodeP` parameter.
`functionId`	`kTelStyEnterAuthenticationCodeMessage`

WARNING! When using this function asynchronously, you must ensure that the string referenced by `iCodeP` remains in memory until the asynchronous call completes.

Comments Before using this function, you should verify that it is available by calling the `TelIsStyServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelStyChangeAuthenticationCode`, `TelStyGetAuthenticationState`



New TelStyGetAuthenticationState

Purpose

Returns the current state of user authentication.

Prototype

```
Err TelStyGetAuthenticationState(UInt16 iRefnum,
TelAppID iAppId, UInt8* oStateP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oStateP`	A pointer to an unsigned byte value. Upon return, this is the authentication state, which is one of the Authentication State Constants.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oStateP` parameter.
`functionId`	`kTelStyGetAuthenticationStateMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oStateP` remains in memory until the asynchronous call completes.

Comments Before using this function, you should verify that it is available by calling the `TelIsStyServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelStyEnterAuthenticationCode`

71

Telephony Network

This chapter describes the telephony network service set of the telephony API.

For more information about the telephony manager basic services and the different service sets, see Chapter 69, "Telephony Basic Services."

This chapter describes:

- Telephony Network Data Structures
- Telephony Network Constants
- Telephony Network Functions

For more information about using the telephony manager, see Chapter 8, "Telephony Manager" in the *Palm OS Programmer's Companion*.

Telephony Network Data Structures

This section describes the data structures used with the telephony network service set portion of the telephony API.



New

TelNwkGetLocationType

You use the `TelNwkGetLocationType` structure with the `TelNwkGetLocation` function to retrieve information about the location of the phone.

```
typedef struct _TelNwkGetLocationType
{
  Char*   value;
  UInt16  size;
} TelNwkGetLocationType;
```

Field Descriptions

<-	value	A buffer into which the current location string is stored. The format of this string is network-dependent. Note that if this buffer is too small to contain the entire retrieved string, the end of the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBufferSize` error.
<->	size	The size of the `value` string. When the structure is used as an input parameter, this is the allocated size, in bytes, of the `value` buffer. Upon return, this is the actual size of the string, including the null terminator character. If the `value` buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.



New TelNwkGetNetworkNameType

You use the `TelNwkGetNetworkNameType` structure with the `TelNwkGetNetworkName` function to retrieve the name of the specified, registered network.

```
typedef struct _TelNwkGetNetworkNameType
{
  UInt32   id;
  Char*    value;
  UInt16   size;
} TelNwkGetNetworkNameType;
```

Field Descriptions

->	`id`	The network ID.
<-	`value`	A null-terminated string buffer into which the network name is stored. Note that if this buffer is too small to contain the entire retrieved string, the end of the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBufferSize` error.
<->	`size`	The size of the `value` string. When the structure is used as an input parameter, this is the allocated length, in bytes, of the `value` buffer. Upon return, this is the actual size of the string, including the null terminator character. If the `value` buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error



TelNwkGetNetworksType

You use the `TelNwkGetNetworksType` structure with the `TelNwkGetNetworks` function to retrieve the number of registered networks.

```
typedef struct _TelNwkGetNetworksType
{
  UInt32*  networkIdP;
  UInt8    size;
} TelNwkGetNetworksType;
```

Field Descriptions

<-	networkIdP	An array into which the retrieved network IDs are stored. Note that if this buffer is too small to contain all of the available IDs, the data is truncated and the function using this structure generates a `telErrBufferSize` error.
<->	size	When the structure is use as an input value, this is the allocated size, in elements, of the `networkIdP` array. When the structure is used as a return value, this is the number of network IDs that are available. If the the `networkP` buffer is too small to contain all of the retrieved IDs, this field is assigned the entire number of available IDs, and the function using this structure generates a `telErrBufferSize` error

Telephony Network Constants

This section describes the constants used with the telephony network service set of the telephony API, which include the following constant types:

- Network Type Constants
- Network Search Mode Constants

Network Type Constants

The network type constants describe the type of network connection.

Constant	Value	Description
`kTelNwkCDMA`	`0`	A CDMA network.
`kTelNwkGSM`	`1`	A GSM network.

Constant	Value	Description
kTelNwkTDMA	2	A TDMA network.
kTelNwkPDC	3	A PDC network.

Network Search Mode Constants

The network search mode constants describe the search mode used to find a network.

Constant	Value	Description
kTelNwkManualSearch	0	Manual network searching.
kTelNwkAutomaticSearch	1	Automatic network searching.

Telephony Network Functions

This section describes the data structures used with the telephony network service set portion of the telephony API.



New TelNwkGetLocation

Purpose Retrieve information about the location of the mobile unit.

Prototype

```
Err TelNwkGetLocation(UInt16 iRefnum,
TelAppID iAppId, TelNwkGetLocationType* ioParamP,
UInt16* ioTransIdP);
```

Parameters

-> iRefnum	The telephony manager library reference number.
-> iAppId	The telephone application attachment identifier for your application.

`<-> ioParamP`	A pointer to a `TelNwkGetLocationType` structure. On input, the `size` field of this structure specifies the allocated size of the `value` buffer. Upon return, the `size` field specifies the actual size of the location string, even if it was truncated to fit into the buffer.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelNwkGetLocationType` passed to this function in the `ioParamP` parameter.
`functionId`	`kTelNwkGetLocationMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

The location information string is stored into the `value` field of the `TelNwkGetLocationType` structure referenced by `ioParamP`. If the `value` buffer is too small to contain the complete string, the string is truncated (and ends with the null terminator character) and this function returns the `telErrBufferSize` error. The `size` field

of the structure is always updated to contain the actual size of the complete string.

Before using this function, you should verify that it is available by calling the `TelIsNwkServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.



TelNwkGetNetworkName

Purpose Returns the name of a registered network.

Prototype

```
Err TelNwkGetNetworkName(UInt16 iRefnum,
TelAppID iAppId,
TelNwkGetNetworkNameType* ioParamP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelNwkGetNetworkNameType` structure that stores the network name. On input, the `size` field of this structure specifies the allocated size of the `value` buffer. Upon return, the `size` field specifies the actual size of the name string, even if it was truncated to fit into the buffer.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelNwkGetNetworkNameType` passed to this function in the `ioParamP` parameter.
`functionId`	`kTelNwkGetNetworkNameMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

The network name string is stored into the `value` field of the structure. If the `value` field buffer in the `TelNwkGetNetworkNameType` structure is too small to contain the complete string, the string is truncated (and ends with the null terminator character) and this function returns the `telErrBufferSize` error. The `size` field is always updated to contain the actual size of the complete string.

The string that is returned in the `value` field of the structure referenced by `ioParamP` is network dependent.

On a GSM network, the result string is compliant with the AT 07.07 European Telecommunications Standards Institute (ETSI) standard for `COPS` and `CREG` commands. The result string contains the following elements:

- The network type, as returned by the `TelNwkGetNetworkType` function, and followed by a semicolon (`';'`) character.

- The network operator, using the following syntax:
  `<area code> ';' <network registration>`

  The `<area code>` value is the value retrieved by issuing the `AT+CREG?` command.

  The `<network registration>` value is the value retrieved by issuing the `AT+CREG?` command.

Before using this function, you should verify that it is available by calling the `TelIsNwkServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelNwkGetNetworks`, `TelNwkGetSelectedNetwork`



New TelNwkGetNetworks

Purpose Retrieves information about the registered networks.

Prototype

```
Err TelNwkGetNetworks(UInt16 iRefnum,
TelAppID iAppId, TelNwkGetNetworksType* ioParamP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelNwkGetNetworksType` structure that stores the network IDs. On input, the `size` field of this structure contains the size, in elements, of the `networkIdP` array field. Upon return, the `networkIdP` array contains the IDs of the registered networks, and the `size` field contains the number of IDs in the array.

`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelNwkGetNetworksType` passed to this function in the `ioParamP` parameter.
`functionId`	`kTelNwkGetNetworkCountMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

Before using this function, you should verify that it is available by calling the `TelIsNwkServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelNwkGetNetworkName`, `TelNwkGetNetworks`



New TelNwkGetNetworkType

Purpose

Retrieve the type of the selected network.

Prototype

```
Err TelNwkGetNetworkType(UInt16 iRefnum,
TelAppID iAppId, UInt8* oTypeP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oTypeP`	A pointer to an unsigned byte value. Upon return, this is the network type. This is one of Network Type Constants.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oTypeP` parameter.
`functionId`	`kTelNwkGetNetworkTypeMessage`

WARNING! When using this function asynchronously, you must ensure that the value you pass for the `oTypeP` parameter remains in memory until the asynchronous call completes.

Comments Before using this function, you should verify that it is available by calling the `TelIsNwkServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.



New TelNwkGetSearchMode

Purpose Returns the current network search mode.

Prototype

```
Err TelNwkGetSearchMode(UInt16 iRefnum,
TelAppID iAppId, UInt8* oModeP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oModeP`	A pointer to an unsigned byte value. Upon return, this is the type of search mode that is currently being used. This is one of the Network Search Mode Constants.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oModeP` parameter.
`functionId`	`kTelNwkGetSearchModeMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oModeP` remains in memory until the asynchronous call completes.

Comments

Before using this function, you should verify that it is available by calling the `TelIsNwkServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelNwkSetSearchMode`

New

TelNwkGetSelectedNetwork

Purpose

Retrieve the network identifier of the currently selected network.

Prototype

```
Err TelNwkGetSelectedNetwork(UInt16 iRefnum,
TelAppID iAppId, UInt32* oNetworkIdP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.

`<- oNetworkIdP`	A pointer to an unsigned integer value. Upon return, this is the identifier of the currently selected network.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oNetworkIdP` parameter.
`functionId`	`kTelNwkGetSelectedNetworkMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oNetworkP` remains in memory until the asynchronous call completes.

Comments

Before using this function, you should verify that it is available by calling the `TelIsNwkServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelNwkGetNetworkName`, `TelNwkGetNetworks`, `TelNwkSelectNetwork`



New TelNwkGetSignalLevel

Purpose

Retrieve the selected network carrier signal level.

Prototype

```
Err TelNwkGetSignalLevel(UInt16 iRefnum,
TelAppID iAppId, UInt8* oSignalP,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oSignalP`	A pointer to an unsigned byte value. Upon return, this is an indication of the signal level in decibels per milliwatt (dBm). The values are explained in the Comments section.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oSignalP` parameter.
`functionId`	`kTelNwkGetSignalLevelMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oSignalP` remains in memory until the asynchronous call completes.

Comments This function sets the value of the variable referenced by `oSignalP` to an integer value that indicates the signal strength in dBm.

The following table describes the signal strength values.

Signal level value	dBm value
0	≤ 113 dBm
1	111 dBm
2 to 30	109 dBm to 53 dBm
31	≥ 51 dBm
99	unknown or undetectable

Before using this function, you should verify that it is available by calling the `TelIsNwkServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

New TelNwkSelectNetwork

Purpose Select a network to use from among the set of registered networks.

Prototype

```
Err TelNwkSelectNetwork(UInt16 iRefnum,
TelAppID iAppId, UInt32 iNetworkId,
UInt16* ioTransIdP);
```

Parameters

`-> iRefnum` The telephony manager library reference number.

`-> iAppId` The telephone application attachment identifier for your application.

`-> iNetworkId`	The identifier of the network to select.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the network unsigned integer value passed to this function in the `iNetworkId` parameter.
`functionId`	`kTelNwkSelectNetworkMessage`

Comments Before using this function, you should verify that it is available by calling the `TelIsNwkServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelNwkGetNetworkName`, `TelNwkGetNetworks`, `TelNwkGetSelectedNetwork`



New TelNwkSetSearchMode

Purpose Sets the search mode used to find a network.

Prototype

```
Err TelNwkSetSearchMode(UInt16 iRefnum,
TelAppID iAppId, UInt8 iMode, UInt16* ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iMode`	The search mode to use. This must be one of the Network Search Mode Constants.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `iMode` parameter.
`functionId`	`kTelNwkSetSearchModeMessage`

Comments

Before using this function, you should verify that it is available by calling the `TelIsNwkServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelNwkGetSearchMode`

72

Telephony Calls

This chapter describes the telephony calls service sets of the telephony API, which include:

- The data calls service set
- The emergency calls service set
- The speech (voice) calls service set

For more information about the telephony manager basic services and the different service sets, see Chapter 69, "Telephony Basic Services."

This chapter describes:

- Telephony Calls Data Structures
- Telephony Calls Functions

For more information about using the telephony manager, see Chapter 8, "Telephony Manager" in the *Palm OS Programmer's Companion*.

Telephony Calls Data Structures

This section describes the data structures used with the telephony calls service sets portion of the telephony manager API.



TelDtcCallNumberType

The `TelDtcCallNumber` function uses the `TelDataCallNumberType` structure to specify information about the telephone number to call.

```
typedef struct _TelDtcCallNumberType
{
  char    *dialNumberP;
  UInt8   lineId;
} TelDtcCallNumberType;
```

Field Descriptions

->	dialNumberP	The telephone number to dial.
<-	lineId	Upon return from the TelDtcCallNumber function, this is the ID of the data line that was established for the telephone call.



New TelDtcReceiveDataType

The TelDtcReceiveData function uses a TelDtcReceiveDataType structure to receive data from an open data line.

```
typedef struct _TelDtcReceiveDataType
{
  UInt8      *data;
  UInt32     size;
  UInt32     timeout;
} TelDtcReceiveDataType;
```

Field Descriptions

<-	data	A buffer into which the data is stored. Note that if this buffer is too small to contain all of the available data, the end of the data is truncated and the function using this structure generates a telErrBufferSize error.
<->	size	When the structure is used as an input parameter, this is the allocated size, in bytes, of the data buffer. Upon return, this is the actual number of bytes of data that was retrieved. If the data buffer is too small to contain all of the retrieved data, the function using this structure generates a telErrBufferSize error.
->	timeout	The number of milliseconds to wait before timing out.



TelDtcSendDataType

The `TelDtcReceiveData` function uses a `TelDtcSendDataType` structure to send data to an open data line.

```
typedef struct _TelDtcSendDataType
{
  UInt8     *data;
  UInt32    size;
} TelDtcSendDataType;
```

Field Descriptions

-> `data` A pointer to the data to send.

-> `size` The number of bytes of `data` in the data buffer.



TelEmcGetNumberType

The `TelEmcGetNumber` function uses a `TelEmcGetNumberType` structure to retrieve an emergency dial telephone number.

```
typedef struct _TelEmcGetNumberType
{
  UInt8     index;
  UInt8     size;
  Char      *value;
} TelEmcGetNumberType;
```

Field Descriptions

->	index	The index of the telephone number. This is a zero-based index.
<->	size	When the structure is used as an input parameter, this is the allocated size, in bytes, of the `value` buffer. Upon return, this is the actual size of the string, including the null terminator character. If the `value` buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.
<-	value	A null-terminated string buffer into which the emergency dial telephone number is stored. Note that if this buffer is too small to contain all of the available data, the end of the data is truncated and the function using this structure generates a `telErrBufferSize` error.



New TelEmcSetNumberType

The `TelEmcSetNumber` function uses a `TelEmcNumberType` structure to set an emergency dial telephone number.

```
typedef struct _TelEmcSetNumberType
{
  UInt8  index;
  Char   *value;
} TelEmcSetNumberType;
```

Field Descriptions

->	index	The index of the telephone number. This is a zero-based index.
->	value	The string value of the number to store as the `index`th entry.



TelSpcGetCallerNumberType

The TelSpcGetCallerNumber function uses a TelSpcGetCallerNumberType structure to retrieve an incoming telephone number.

```
typedef struct _TelSpcGetCallerNumberType
{
  Char    *value;
  UInt8   size;
} TelSpcGetCallerNumberType;
```

Field Descriptions

`<-` `value` — A null-terminated string buffer into which the caller telephone number is stored.

Note that if this buffer is too small to contain all of the available data, the end of the data is truncated and the function using this structure generates a `telErrBufferSize` error.

`<->` `size` — When the structure is used as an input parameter, this is the allocated size, in bytes, of the `value` buffer.

Upon return, this is the actual size of the caller dial telephone number, including the null terminator character. If the `value` buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.



TelSpcPlayDTMFType

The TelSpcPlayDTMF function uses a TelSpcPlayDTMFType structure to specify the qualities of the DTMF (dual-tone, multi-frequency) sound sent by the phone to the network or remote, connected equipment.

```
typedef struct _TelSpcPlayDTMFType
  UInt8   keyTone;
  UInt32  duration;
} TelSpcPlayDTMFType;
```

Field Descriptions

-> `keyTone` The keycode of the key tone to play. This must be one of the Keycode Constants.

-> `duration` The duration of the tone, specified as a multiple of ten milliseconds.

Telephony Calls Functions

This section describes the functions used with the telephony calls service sets portion of the telephony API.



TelDtcCallNumber

Purpose Initiate a data telephone call.

Prototype

```
Err TelDtcCallNumber(UInt16 iRefnum,
TelAppID iAppId, TelDtcCallNumberType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

-> `iRefnum` The telephony manager library reference number.

-> `iAppId` The telephone application attachment identifier for your application.

<-> `ioParamP` A pointer to a `TelDtcCallNumberType` structure that specifies information about the telephone call.

<-> `ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelDtcCallNumberType` structure passed to this function in the `ioDataCallParamP` parameter.
`functionId`	`kTelDtcCallNumberMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioDataCallParamP` remains in memory until the asynchronous call completes.

Comments

Call this function to start a data telephone call.

Before using this function, you should verify that it is available by calling the `TelIsDtcServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelDtcCloseLine`



New TelDtcCloseLine

Purpose

Hang up a data line.

Prototype

```
Err TelDtcCloseLine(UInt16 iRefnum,
TelAppID iAppId, UInt8 iLineId,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iLineId`	The ID of the line to hang up. This is the ID returned by a previous call to the `TelDtcCallNumber` function.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `iLineId` parameter.
`functionId`	`kTelDtcCloseLineMessage`

Comments Call this function to end a data telephone call.

Before using this function, you should verify that it is available by calling the `TelIsDtcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelDtcCallNumber`



New TelDtcReceiveData

Purpose Receive data on an opened data communications line.

Prototype

```
Err TelDtcReceiveData(UInt16 iRefnum,
TelAppID iAppId, TelDtcReceiveDataType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelDtcReceiveDataType` structure.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelDtcReceiveDataType` structure passed to this function in the `ioRcvDataP` parameter.
`functionId`	`kTelDtcReceiveDataMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments Call this function to receive data during an active data telephone call.

Before using this function, you should verify that it is available by calling the `TelIsDtcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelDtcCallNumber`, `TelDtcCloseLine`, `TelDtcSendData`



TelDtcSendData

Purpose Send data on an opened data line.

Prototype

```
Err TelDtcSendData(UInt16 iRefnum,
TelAppID iAppId, TelDtcSendDataType *iParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.

`-> iAppId`	The telephone application attachment identifier for your application.
`-> iParamP`	A pointer to a `TelDtcSendDataType` structure that specifies the data to send.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelDtcSendDataType` structure passed to this function in the `iSendDataP` parameter.
`functionId`	`kTelDtcSendDataMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iParamP` remains in memory until the asynchronous call completes.

Comments

Call this function to send data during an active data telephone call.

Before using this function, you should verify that it is available by calling the `TelIsDtcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelDtcCallNumber`, `TelDtcCloseLine`, `TelDtcReceiveData`



TelEmcCall

Purpose Call the currently selected emergency service.

Prototype

```
Err TelEmcCall(UInt16 iRefnum, TelAppID iAppId,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelEmcCallMessage`

Comments This function calls the telephone number specified in a previous call to the `TelEmcSelectNumber` function. In synchronous mode, this function returns after the dial command has been sent to the phone.

After calling this function, sub-launched applications can receive notifications when the following telephony events occur. Note that these notifications can be raised after both synchronous and asynchronous calls to this function.

Event	Description
`sysTelSpcLaunchCmdCallReleased`	Warns that the telephone call has been released.
`sysTelSpcLaunchCmdCallBusy`	Warns that the called equipment is busy.
`sysTelSpcLaunchCmdCallConnect`	Warns that the line is open. The ID of the open line is stored in the `UInt32` value of the parameter block passed to the application.
`sysTelSpcLaunchCmdCallError`	Warns that an error occurred while attempting to complete the telephone call.

Before using this function, you should verify that it is available by calling the `TelIsEmcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelEmcCloseLine`,



New TelEmcCloseLine

Purpose Close the line that is currently opened for an emergency telephone call.

Prototype

```
Err TelEmcCloseLine(UInt16 iRefnum,
TelAppID iAppId, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelDtcCloseLineMessage`

Comments

Call this function to end an emergency telephone call.

Before using this function, you should verify that it is available by calling the `TelIsEmcServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelEmcCall`



New TelEmcGetNumber

Purpose

Retrieve an emergency dial telephone number.

Prototype

```
Err TelEmcGetNumber(UInt16 iRefnum,
TelAppID iAppId, TelEmcGetNumberType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelEmcGetNumberType` structure in which you assign the index of the telephone number that you want to retrieve. On input, the `size` field of this structure specifies the allocated size of the `value` buffer. Upon return, the `size` field specifies the actual size of the telephone number, even if it was truncated to fit into the buffer.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelEmcGetNumberType` structure passed to this function in the `ioGetNumberP` parameter.
`functionId`	`kTelEmcGetNumberMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments The emergency call telephone number is stored into the `value` field of the `TelEmcGetNumberType` structure referenced by `ioGetNumberP`. If the `value` buffer is too small to contain the complete string, the string is truncated (and ends with the null terminator character) and this function returns the `telErrBufferSize` error. The `size` field of the structure is always updated to contain the actual size of the complete string.

Before using this function, you should verify that it is available by calling the `TelIsEmcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelEmcGetNumberCount`, `TelEmcSetNumber`, `TelEmcSelectNumber`



TelEmcGetNumberCount

Purpose Retrieve the count of emergency dial telephone numbers.

Prototype

```
Err TelEmcGetNumberCount(UInt16 iRefnum,
TelAppID iAppId, UInt8 *oCountP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oCountP`	Upon return, the total number of emergency call numbers available.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to unsigned integer passed to this function in the `oCountP` parameter.
`functionId`	`kTelEmcGetNumberMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oCountP` remains in memory until the asynchronous call completes.

Comments

The emergency telephone call number is stored into the `value` field of the `TelEmcGetNumberType` structure referenced by `ioGetNumberP`. If the `value` buffer is too small to contain the complete string, the string is truncated (and ends with the null terminator character) and this function returns the `telErrBufferSize` error. The `size` field of the structure is always updated to contain the actual size of the complete string.

Before using this function, you should verify that it is available by calling the `TelIsEmcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also TelEmcGetNumber, TelEmcSetNumber, TelEmcSelectNumber



TelEmcSelectNumber

Purpose Select the current emergency telephone number. This is the telephone number that gets dialed when you call the TelEmcCall function.

Prototype

```
Err TelEmcSelectNumber(UInt16 iRefnum,
TelAppID iAppId, UInt8 iIndex,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iIndex`	The zero-based index of the emergency telephone number that you want selected.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the TelEventType event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.

`paramP`	Points to the unsigned integer value passed to this function in the `iIndex` parameter.
`functionId`	`kTelEmcSelectNumberMessage`

Comments Before using this function, you should verify that it is available by calling the `TelIsEmcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelEmcCall`, `TelEmcGetNumber`, `TelEmcGetNumberCount`, `TelEmcSetNumber`



New TelEmcSetNumber

Purpose Set the telephone number for the specified emergency dial number.

Prototype

```
Err TelEmcSetNumber(UInt16 iRefnum,
TelAppID iAppId, TelEmcSetNumberType *iParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iParamP`	A pointer to a `TelEmcSetNumberType` structure that specifies the telephone number.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelEmcSetNumberType` structure passed to this function in the `iNumberP` parameter.
`functionId`	`kTelEmcSetNumberMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iParamP` remains in memory until the asynchronous call completes.

Comments

Call this function to associate a new telephone number with the emergency dial number that has the specified `iIndex`.

Before using this function, you should verify that it is available by calling the `TelIsEmcServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelEmcGetNumber`,



New

TelSpcAcceptCall

Purpose

Accept an incoming voice telephone call.

Prototype

```
Err TelSpcAcceptCall(UInt16 iRefnum,
TelAppID iAppId, UInt8 *oLineIdP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oLineIdP`	A pointer to an unsigned byte value. Upon return, this is the ID of the voice line assigned to the telephone call.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer passed to this function in the `oLineIdP` parameter.
`functionId`	`kTelSpcAcceptCallMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oLineIdP` remains in memory until the asynchronous call completes.

Comments

If another line was active prior to the execution of this function, that line is put on hold. Note that there can only be one line active at any given time, and there can only be one line on hold at any given time.

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSpcGetCallerNumber`, `TelSpcRejectCall`



New TelSpcCallNumber

Purpose Initiate a voice telephone call.

Prototype

```
Err TelSpcCallNumber(UInt16 iRefnum,
TelAppID iAppId, const Char *iDialNumberP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iDialNumberP`	A pointer to the telephone number to call.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the string passed to this function in the `iDialNumberP` parameter.
`functionId`	`kTelSpcCallNumberMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `iDialNumberP` remains in memory until the asynchronous call completes.

Comments

A successful return from a synchronous call or receipt of a successful notification from an asynchronous call does not mean that the telephone call has been connected; instead, it indicates that the dial command was sent to the phone. Successful connection of the telephone call is indicated with a sub-launch.

The dial number is formatted according to the following syntax:

```
DialNumber ::=  <Phone_Number> | <Code_String>
              | <Phone_Number> <Code_String>

Phone_Number ::=  <IntlPrefix><NatlNumber>
                | <NatlNumber>

IntlPrefix ::=  '+' <country code>

NatlNumber ::=  {{Pause<Pause>}{<Digit>}}

Code_String ::=  <Symbol>{<Symbol>}

Symbol ::=       <Digit> | '#' | '*'

Digit ::=       '0' | '1' | '2' | '3' | '4'
              | '5' | '6' | '7' | '8' | '9'

Pause ::=         ','
```

After calling this function, sub-launched applications can receive notifications when the following telephony events occur. Note that these notifications can be raised after both synchronous and asynchronous calls to this function.

Event	Description
`sysTelSpcLaunchCmdCallReleased`	This is passed to a sub-launched application to warn that the telephone call has been released.
`sysTelSpcLaunchCmdCallBusy`	This is passed to a sub-launched application to warn that the called equipment is busy.
`sysTelSpcLaunchCmdCallConnect`	This is passed to a sub-launched application to warn that the line is open. The ID of the open line is stored in the `UInt32` value of the parameter block passed to the application.
`sysTelSpcLaunchCmdCallError`	This is passed to a sub-launched application to warn that the telephone call encountered an error.

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSpcCloseLine`



New TelSpcCloseLine

Purpose Ends a voice telephone call.

Prototype

```
Err TelSpcCloseLine(UInt16 iRefnum,
TelAppID iAppId, UInt8 iLineId,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iLineId`	The ID of the voice line that you want to close. This is the ID returned by a previous call to the `TelSpcAcceptCall` function.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `iLineId` parameter.
`functionId`	`kTelSpcCloseLineMessage`

Comments

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelSpcCallNumber`



New TelSpcConference

Purpose

Initiate a conference telephone call by merging the active line and the held line.

Prototype

```
Err TelSpcConference(UInt16 iRefnum,
TelAppID iAppId, UInt8 *oLineIdP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oLineIdP`	A pointer to an unsigned byte value. Upon return, this is the ID of the voice line assigned to the telephone call.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oLineIdP` parameter.
`functionId`	`kTelSpcConferenceMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oLineIdP` remains in memory until the asynchronous call completes.

Comments Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSpcCallNumber`, `TelSpcCloseLine`, `TelSpcHoldLine`, `TelSpcRetrieveHeldLine`, `TelSpcSelectLine`



New TelSpcGetCallerNumber

Purpose Retrieve the telephone number of the caller on an incoming telephone call.

Prototype

```
Err TelSpcGetCallerNumber(UInt16 iRefnum,
TelAppID iAppId,
TelSpcGetCallerNumberType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelSpcGetCallerNumberType` structure that is used to retrieve the caller's telephone number. On input, the `size` field of this structure specifies the allocated size of the `value` buffer. Upon return, the `size` field specifies the actual size of the telephone number, even if it was truncated to fit into the buffer.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful. If there is no active incoming telephone call, this function returns the `telErrUnavailableValue` error.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSpcGetCallerNumberType` structure passed to this function in the `ioParamP` parameter.
`functionId`	`kTelSpcGetCallerNumberMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

The emergency telephone call number is stored into the `value` field of the `TelSpcGetCallerNumberType` structure referenced by `ioParamP`. If the `value` buffer is too small to contain the complete string, the string is truncated (and ends with the null terminator character) and this function returns the `telErrBufferSize` error. The `size` field of the structure is always updated to contain the actual size of the complete string.

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSpcAcceptCall`



TelSpcHoldLine

Purpose Put the currently active voice line on hold.

Prototype

```
Err TelSpcHoldLine(UInt16 iRefnum,
TelAppID iAppId, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelSpcHoldLineMessage`

Comments Note that there can only be one line active at any given time, and there can only be one line on hold at any given time.

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSpcRetrieveHeldLine`



New TelSpcPlayDTMF

Purpose Play a dual-tone multi-frequency sound to the network system for a specified duration. Note that you can only play a DTMF while a voice telephone call is active.

Prototype

```
Err TelSpcPlayDTMF(UInt16 iRefnum,
TelAppID iAppId, TelSpcPlayDTMFType *iParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iParamP`	A pointer to a `TelSpcPlayDTMFType` structure that specifies the tone to play and its duration.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSpcPlayDTMFType` structure passed to this function in the `iParamP` parameter
`functionId`	`kTelSpcPlayDTMFMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iParamP` remains in memory until the asynchronous call completes.

Comments

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelSpcSendBurstDTMF`, `TelSpcStartContinuousDTMF`, `TelSpcStopContinuousDTMF`

New

TelSpcRejectCall

Purpose

Reject an incoming voice telephone call.

Prototype

```
Err TelSpcRejectCall(UInt16 iRefnum,
TelAppID iAppId, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.

`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelSpcRejectCallMessage`

Comments Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSpcAcceptCall`, `TelSpcGetCallerNumber`



New TelSpcRetrieveHeldLine

Purpose Reconnect the voice line that is currently on hold, making it the active voice line.

Prototype

```
Err TelSpcRetrieveHeldLine(UInt16 iRefnum,
TelAppID iAppId, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelSpcRetrieveHeldLineMessage`

Comments

Note that there can only be one line active at any given time, and there can only be one line on hold at any given time.

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelSpcHoldLine`



New TelSpcSelectLine

Purpose

Select the specified line ID as the newly active voice line.

Prototype

```
Err TelSpcSelectLine(UInt16 iRefnum,
TelAppID iAppId, UInt8 iLineId,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iLineId`	The ID of the voice line that you want to activate. This is the ID returned by a previous call to the `TelSpcAcceptCall` function.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `iLineId` parameter.
`functionId`	`kTelSpcSelectLineMessage`

Comments If a line was active previous to completion of this function, that line is put on hold. Note that there can only be one line active at any given time, and there can only be one line on hold at any given time.

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSpcConference`



New TelSpcSendBurstDTMF

Purpose Send a string of dual-tone, multi-frequency sounds to the network system. Note that you can only play a DTMF while a voice telephone call is active.

Prototype

```
Err TelSpcSendBurstDTMF(UInt16 iRefnum,
TelAppID iAppId, const Char *iDTMFStringP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum` The telephony manager library reference number.

`-> iAppId` The telephone application attachment identifier for your application.

`-> iDTMFStringP` A null-terminated string of keytone values. Each byte of the string specifies one of the Keycode Constants.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the string passed to this function in the `iDTMFStringP` parameter.
`functionId`	`kTelSpcSendBurstDTMFMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iDTMFStringP` remains in memory until the asynchronous call completes.

Comments

This function sends a burst string of keytones to the network. Each key tone is played for the default duration defined by the network.

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelSpcPlayDTMF`, `TelSpcStartContinuousDTMF`, `TelSpcStopContinuousDTMF`



New TelSpcStartContinuousDTMF

Purpose

Send a continuous dual-tone, multi-frequency sound to the network system. Note that you can only play a DTMF while a voice telephone call is active.

Prototype

```
Err TelSpcStartContinuousDTMF(UInt16 iRefnum,
TelAppID iAppId, UInt8 iKeyCode,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iKeyCode`	The keycode to send to the network. This must be one of the Keycode Constants.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `iKeyCode` parameter.
`functionId`	`kTelSpcStartContinuousDTMFMessage`

Comments This function sends a key tone to the network system that is played continuously until the `TelSpcStopContinuousDTMF` function executes.

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSpcPlayDTMF`, `TelSpcSendBurstDTMF`, `TelSpcStopContinuousDTMF`



New TelSpcStopContinuousDTMF

Purpose Stop the continuous playing of a tone that was started by calling the `TelSpcStartContinuousDTMF` function.

Prototype

```
Err TelSpcStopContinuousDTMF(UInt16 iRefnum,
TelAppID iAppId, UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	A `NULL` pointer.
`functionId`	`kTelSpcStopContinuousDTMFMessage`

Comments

This function stops the continuous playing of the tone that was previously initiated by calling the `TelSpcStartContinuousDTMF` function.

Before using this function, you should verify that it is available by calling the `TelIsSpcServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelSpcPlayDTMF`, `TelSpcSendBurstDTMF`, `TelSpcStartContinuousDTMF`

73

Telephony SMS

This chapter describes the telephony SMS service set of the telephony API.

For more information about the telephony manager basic services and the different service sets, see Chapter 69, "Telephony Basic Services."

This chapter describes:

- Telephony SMS Data Structures
- Telephony SMS Constants
- Telephony SMS Functions

For more information about using the telephony manager, see Chapter 8, "Telephony Manager" in the *Palm OS Programmer's Companion*.

Telephony SMS Data Structures

This section describes the data structures used with the SMS service set of the telephony API.



TelSmsDateTimeType

Several of the other data structures used with the Telephony SMS functions include an `TelSmsDateTimeType` structure to store a date and time value.

```
typedef _TelSmsDateTimeType
  Boolean    absolute;
  UInt32     dateTime;
} TelSmsDateTimeType
```

Field Descriptions

`->` `absolute` — If `true`, the `dateTime` value is a Palm™ absolute time value, which is the number of seconds since 1/1/1904. If `false`, the `dateTime` value is relative to the current date and time.

`->` `dateTime` — The date and time value.

If the `absolute` field is `true`, this is expressed as the number of seconds elapsed since 12:00 A.M. on January 1, 1904. This is the format returned by the `TimGetSeconds` function.

If the `absolute` field is `false`, this is expressed as the number of seconds elapsed from the current time.



New TelSmsDeleteMessageType

The `TelSmsDeleteMessage` function uses a `TelSmsDeleteMessageType` structure to specify the message to be deleted.

```
typedef struct _TelSmsDeleteMessageType
  UInt8    messageType;
  UInt16   index;
} TelSmsDeleteMessageType;
```

Field Descriptions

`->` `messageType` — The message type. This is one of the SMS Message Type Constants.

`->` `index` — The index of the SMS message in the phone's storage that is to be deleted. Note that the message is deleted from the storage area selected by a call to the `TelSmsSelectStorage` function.

Note that the index is zero-based.



TelSmsDeliveryAdvancedCDMAType

The TelSmsDeliveryMessageType structure includes a TelSmsDeliveryAdvancedCDMAType structure for CDMA messages.

```
typedef struct _TelSmsDeliveryAdvancedCDMAType
{
  UInt8               messageType;
  TelSmsDateTimeType  validityPeriod;
  UInt8               priority;
  UInt8               privacy;
  Boolean             alertOnDeliveryRequest;
  Boolean             manualAckRequest;
  UInt8               voiceMessageNumber;
  UInt8               callbackNumberSize;
  Char                *callbackNumberAddress;
  UInt8               languageIndicator;
} TelSmsDeliveryAdvancedCDMAType;
```

Field Descriptions

`messageType`	The type of the message. This must be one of the SMS Message Type Constants.
`validityPeriod`	An TelSmsDateTimeType structure that specifies the amount of time for which the message is valid.
`priority`	The message priority. This must be one of the SMS Message Urgency Constants.
`privacy`	The privacy type of the message. This must be one of the SMS Message Privacy Constants.
`alertOnDeliveryRequest`	`true` if the user is to be alerted upon delivery of this message, and `false` if not.
`manualAckRequest`	`true` if a reply is requested from the user, and `false` if not.
`voiceMessageNumber`	

`callbackNumberSize`	When the structure is used as an input parameter, this is the allocated size, in bytes, of the `callbackNumberAddress` string. Upon return, this is the actual size of the string, including the null terminator character. If the buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.
`callbackNumberAddress`	A buffer into which the callback number address string is stored. Note that if this buffer is too small to contain the entire retrieved string, the end of the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBufferSize` error.
`languageIndicator`	



New TelSmsDeliveryAdvancedGSMType

The `TelSmsDeliveryMessageType` structure includes a `TelSmsDeliveryAdvancedGSMType` structure for GSM messages.

```
typedef struct _TelSmsDeliveryAdvancedGSMType
{
  UInt16     protocolId;
  Boolean    replyPath;
  Char       *serviceCenterNumber;
  UInt8      serviceCenterNumberSize;
} TelSmsDeliveryAdvancedGSMType
```

Field Descriptions

<-	protocolId	The protocol used for this message. This is one of the SMS Message Transport Protocol Constants.
<-	replyPath	If this value is set, then you use the serviceCenterNumber to reply to this message. If this value is not set, you use the default service center provided by your network operator.
<->	serviceCenterNumber	A buffer into which the service center number string is stored. Note that if this buffer is too small to contain the entire retrieved string, the end of the string is truncated (and ends with the null terminator character) and the function using this structure generates a telErrBufferSize error.
<->	seviceCenterNumberSize	When the structure is used as an input parameter, this is the allocated size, in bytes, of the serviceCenterNumber string. Upon return, this is the actual size of the string, including the null terminator character. If the buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a telErrBufferSize error.



TelSmsDeliveryAdvancedTDMAType

The TelSmsDeliveryMessageType structure includes a TelSmsDeliveryAdvancedTDMAType structure for TDMA messages.

```
typedef struct _TelSmsDeliveryAdvancedTDMAType
{
  UInt8                 messageType;
  TelSmsDateTimeType    validityPeriod;
```

```
  UInt8                 priority;
  UInt8                 privacy;
  Boolean               alertOnDeliveryRequest;
  Boolean               manualAckRequest;
  UInt8                 voiceMessageNumber;
  UInt8                 callbackNumberSize;
  Char                  *callbackNumberAddress;
  UInt8                 languageIndicator;
} TelSmsDeliveryAdvancedTDMAType;
```

Field Descriptions

`messageType`	The type of the message. This must be one of the SMS Message Type Constants.
`validityPeriod`	An `TelSmsDateTimeType` structure that specifies the amount of time for which the message is valid.
`priority`	The message priority. This must be one of the SMS Message Urgency Constants.
`privacy`	The privacy type of the message. This must be one of the SMS Message Privacy Constants.
`alertOnDeliveryRequest`	`true` if the user is to be alerted upon delivery of this message, and `false` if not.
`manualAckRequest`	`true` if a reply is requested from the user, and `false` if not.
`voiceMessageNumber`	
`callbackNumberSize`	When the structure is used as an input parameter, this is the allocated size, in bytes, of the `callbackNumberAddress` string. Upon return, this is the actual size of the string, including the null terminator character. If the buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.

`callbackNumberAddress` A buffer into which the callback telephone number address string is stored.

Note that if this buffer is too small to contain the entire retrieved string, the end of the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBufferSize` error.

`languageIndicator`



TelSmsDeliveryMessageType

The `TelSmsReadMessage` function uses a `TelSmsDeliveryMessageType` structure to retrieve information about a delivered message.

```
typedef struct _TelSmsDeliveryMessageType
{
  UInt16                version;
  UInt16                index;
  UInt32                messageIdentifier;
  TelSmsDateTimeType    timeStamp;
  UInt16                dataSize;
  UInt8                 *data;
  UInt8                 dataCodingScheme;
  UInt8                 originatingAddressSize;
  Char                  *originatingAddress;
  Boolean               otherToReceive;
  Boolean               reportDeliveryIndicator;
  UInt8                 standardType;

  union
  {
    TelSmsDeliveryAdvancedGSMType
                                    advancedGSM;
    TelSmsDeliveryAdvancedCDMAType
                                     advancedCDMA;
```

```
    TelSmsDeliveryAdvancedTDMAType
                                        advancedTDMA;
  } advancedParams;
  UInt8                 extensionsCount;
  TelSmsExtensionType   *extensionsP;
} TelSmsDeliveryMessageType;
```

Field Descriptions

`->  version`	The version of the SMS API associated with this message.
`<-> index`	Upon return, the SMS index of the message on the phone. This is a 0-based index. This value is used for input only when calling the `TelSmsReadMessage` function to read one SMS at a time.
`<-  messageIdentifier`	The message identifier.
`<-  timeStamp`	The message time stamp. This is a `TelSmsDateTimeType` structure.
`<-> dataSize`	The size of the `data` buffer. When the structure is used as an input parameter, this is the allocated size of the `data` buffer. Upon return, this is the actual size of the message data. If the buffer is too small to contain the entire message, this field is assigned the entire length of the message, and the function using this structure generates a `telErrBufferSize` error.
`<-> data`	A buffer into which the retrieved data is stored. Note that if this buffer is too small to contain the entire retrieved message, the end of the message is truncated and the function using this structure generates a `telErrBufferSize` error.

<-	`dataCodingScheme`	The coding scheme used for the data. This is one of the SMS Data Coding Scheme Constants.
<->	`originatingAddressSize`	The size of the `originatingAddress` buffer. When the structure is used as an input parameter, this is the allocated size of the `originatingAddress` buffer. Upon return, this is the actual size of the string, including the null terminator character. If the buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.
<->	`originatingAddress`	A buffer into which the originating address string is stored. Note that if this buffer is too small to contain the entire string, the end of the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBufferSize` error.
<-	`otherToReceive`	Indicates whether there are more messages waiting to be received from the service center to the mobile device.
<-	`reportDeliveryIndicator`	If `true`, indicates that the originating user has asked the network to send a delivery report.
<-	`standardType`	Indicates which field of the `advancedParams` union contains the message information. This is one of the Network Type Constants described in Chapter 71, "Telephony Network." If this value is `kTelNwkCDMA`, then the `advancedParams` union contains a `TelSmsDeliveryAdvancedCDMAType` structure, and similarly for the other values.

`advancedParams`	Advanced message information for GSM, CDMA, or TDMA messages. This is one of the following structure types:
	`TelSmsDeliveryAdvancedGSMType`
	`TelSmsDeliveryAdvancedCDMAType`
	`TelSmsDeliveryAdvancedTDMAType`

`<-> extensionsCount`	On input, this is the number of extension structures allocated for this message. You must allocate at least one structure to specify the multipart information. Upon return, this is the number of extensions in the SMS header. If the SMS header contains more extensions than you have allocated, the available extension structures are filled, and this function generates a `telErrBufferSize` error.
`<-> extensionsP`	A pointer to an array of `TelSmsExtensionType` structures that you have allocated for this message. You must allocate this array before using this structure.



New TelSmsExtensionType

The `TelSmsExtensionType` structure specifies multipart information about a message.

```
typedef struct _TelSmsExtensionType
  UInt8    extensionTypeId;
  union
  {
    TelSmsMultiPartExtensionType    mp;
    TelSmsNbsExtensionType          nbs;
    TelSmsUserExtensionType         user;
  } extension;
} TelSmsExtensionType;
```

Field Descriptions

`<-> extensionTypeId` Identifies the type of SMS extension structure found in the `extension` union. This is one of the SMS Extension Type Constants.

If the value of this field is `kTelSmsMultipartExtensionTypeId`, then the extension union contains a `TelSmsMultiPartExtensionType` structure.

If the value of this field is `kTelSmsNbsExtensionTypeId` or `kTelSmsNbs2ExtensionTypeId`, then the union contains a `TelSmsNbsExtensionType` structure. The difference between these two is that `kTelSmsNbs2ExtensionTypeId` indicates that the port value is a long instead of a short.

If this field contains any other value, then the union contains a `TelSmsUserExtensionType` structure.

`<- extension` The extension information, which is one of the following structure types:
`TelSmsMultiPartExtensionType`
`TelSmsNbsExtensionType`
`TelSmsUserExtensionType`



New TelSmsGetAvailableStorageType

The `TelSmsGetAvailableStorage` function uses a `TelSmsGetAvailableStorageType` structure to retrieve information about the storage available on the phone.

```
typedef struct _TelSmsGetAvailableStorageType
  UInt16   count;
  UInt8    *storagesP;
} TelSmsGetAvailableStorageType;
```

Field Descriptions

`<-> count` The size of the `storagesP` buffer.

When the structure is used as an input parameter, this is the allocated number of values in the buffer. Upon return, this is the total number of storage areas in the phone.

`<- storagesP` A buffer into which the retrieved storage IDs are stored. Each value stored into the buffer is one of the SMS Storage ID Constants.



New TelSmsGetMessageCountType

The `TelSmsGetMessageCount` function uses a `TelSmsGetMessageCountType` structure to retrieve information about messages in the currently selected storage.

```
typedef struct _TelSmsGetMessageCountType
{
  UInt8    messageType;
  UInt16   slots;
  UInt16   count;
} TelSmsGetMessageCountType;
```

Field Descriptions

`-> messageType` The type of message for which you want to retrieve the count. This must be one of the SMS Message Type Constants.

You must fill this in on input to the `TelSmsGetMessageCount` function.

`<- slots` The total number of message slots available in the currently selected storage area (for all message types).

`<- count` The number of filled slots for the specified `messageType` in the currently selected storage area.



New TelSmsManualAckType

The `TelSmsSendManualAcknowledge` function uses a `TelSmsManualAckType` structure to specify the information used to send a message acknowledgment.

```
typedef struct _TelSmsManualAckType
{
  UInt16     version;
  Char       *destinationAddress;
  UInt32     messageId;
  UInt16     dataSize;
  UInt8      *data;
  UInt8      dataCodingScheme;
  UInt8      responseCode;
} TelSmsManualAckType;
```

Field Descriptions

`version` The version of the SMS API used for this acknowledgment.

`destinationAddress` The destination address. For GSM, the length of this address is 12 bytes. For CDMA, the length is up to 128 bytes.

`messageId` Upon return, the ID of the acknowledgment.

`dataSize` The size of the `data` buffer.

When the structure is used as an input parameter, this is the allocated size of the `data` buffer.

Upon return, this is the actual size of the data. If the buffer is too small to contain all of the data, this field is assigned the entire length of the data, and the function using this structure generates a `telErrBufferSize` error.

`data`	A buffer into which the retrieved data is stored. Note that if this buffer is too small to contain the all of the retrieved data, the data is truncated and the function using this structure generates a `telErrBufferSize` error.
`dataCodingScheme`	The coding scheme used for the data. This must be one of the SMS Data Coding Scheme Constants.
`responseCode`	The response code. The value of this field depends on the network being used.



New TelSmsMultiPartExtensionType

The `TelSmsExtensionType` structure uses a `TelSmsMultiPartExtensionType` structure to describe information about a multipart message.

```
typedef struct _TelSmsMultiPartExtensionType
  UInt16  bytesSent;
  UInt16  partCurrent;
  UInt16  partCount;
  UInt16  partId;
} TelSmsMultiPartExtensionType;
```

Field Descriptions

`<-> bytesSent`	On input, set this value to 0. Upon return, this is the current count of message bytes that have been sent.
`<-> partCurrent`	On input, set this value to 0. Upon return, this is the part number of the current message part.

`<-> partCount` — On input, set this value to 0.

Upon return, this is the number of message parts required to send the data.

`<-> partId` — The ID of the current SMS message. This ID is unique and is the same for all parts of the message. This information is required to reassemble a multi-part SMS.

On input, set this value to 0.



TelSmsNbsExtensionType

The `TelSmsExtensionType` structure uses a `TelSmsNbsExtensionType` structure to describe information about a NBS message.

```
typedef struct _TelSmsNbsExtensionType
  UInt16  destPort;
  UInt16  srcPort;
} TelSmsNbsExtensionType;
```

Field Descriptions

`<-> destPort` — When the structure is used for input, this is the NBS port number used to encode the data.

Upon return, this is the NBS port number that was used for the data.

`<-> srcPort` — This is currently the same as the `destPort`.



TelSmsReadMessagesType

The `TelSmsReadMessages` function uses a `TelSmsReadMessagesType` structure to retrieve messages from the currently selected storage area.

```
typedef struct _TelSmsReadMessagesType
  UInt16                      first;
  UInt16                      count;
  TelSmsDeliveryMessageType  *messagesP;
} TelSmsReadMessagesType;
```

Field Descriptions

`->  first` — The index of the first message to retrieve. Message indexes are zero-based.

`<-> count` — The size of the `messagesP` buffer.

When the structure is used as an input parameter, this is the allocated number of pointers in the `messagesP` buffer.

Upon return, this is the number of messages that could be retrieved. If the `messagesP` buffer is too small to contain all of the messages, this field is assigned the entire count, and the function using this structure generates a `telErrBufferSize` error.

`<-> messagesP` — An array of pointers to `TelSmsDeliveryMessageType` structures, each of which is filled in with a retrieved message, if available.



New TelSmsReadReportsType

The `TelSmsReadReports` function uses a `TelSmsReadReportsType` structure to retrieve reports from the currently selected storage area.

```
typedef struct _TelSmsReadReportsType
  UInt16             first;
  UInt16             count;
  TelSmsReportType   *reportsP;
} TelSmsReadReportsType;
```

Field Descriptions

`->  first` — The index of the first report to retrieve. Report indexes are zero-based.

`<->  count` — The size of the `reportsP` buffer.

When the structure is used as an input parameter, this is the allocated number of pointers in the `reportsP` buffer.

Upon return, this is the actual number of reports that could be read. If the buffer is too small to contain all of the reports, this field is assigned the entire count, and the function using this structure generates a `telErrBufferSize` error.

`<->  reportsP` — An array of pointers to `TelSmsReportType` structures, each of which is filled in with a retrieved message, if available.

Note that if this buffer is too small to contain all of the retrieved reports, the function using this structure generates a `telErrBufferSize` error.



TelSmsReadSubmittedMessagesType

The `TelSmsReadSubmittedMessages` function uses a `TelSmsReadSubmittedMessagesType` structure to retrieve submitted messages from the currently selected storage area.

```
typedef struct _TelSmsReadMessagesType
  UInt16                        first;
  UInt16                        count;
  TelSmsDeliveryMessageType   *submittedsP;
} TelSmsReadSubmittedMessagesType;
```

Field Descriptions

->	first	The index of the first message to retrieve. Message indexes are zero-based.
<->	count	The size of the `submittedsP` buffer. When the structure is used as an input parameter, this is the allocated number of pointers in the `submittedsP` buffer. Upon return, this is the number of messages that were actually read.
<->	submittedsP	An array of pointers to `TelSmsSubmittedMessageType` structures, each of which is filled in with a retrieved message, if available.



TelSmsReportType

The `TelSmsReadReport` function uses a `TelSmsReportType` structure to retrieve a report from the report storage area.

```
typedef struct _TelSmsReportType
{
  UInt16               version;
  UInt16               index;
  UInt8                reportType;
  UInt32               messageId;
  UInt16               dataSize;
  UInt8                *data;
  UInt8                dataCodingScheme;
  Char                 *originatingAddress;
  UInt8                originatingAddressSize;
  UInt8                report;
  TelSmsDateTimeType   timeStamp;
} TelSmsReportType;
```

Field Descriptions

->	version	
<-	index	The index of the report in the phone storage area.

`<-`	`reportType`	The delivery report type.
`<-`	`messageId`	The message ID.
`<->`	`dataSize`	The size of the `data` buffer. When the structure is used as an input parameter, this is the allocated size of the `data` buffer. Upon return, this is the actual size of the data. If the buffer is too small to contain all of the data, this field is assigned the entire length of the data, and the function using this structure generates a `telErrBufferSize` error.
`<->`	`data`	A buffer into which the retrieved data is stored. Note that if this buffer is too small to contain the all of the retrieved data, the data is truncated and the function using this structure generates a `telErrBufferSize` error.
`<-`	`dataCodingScheme`	The encoding scheme used for the report data. This must be one of the SMS Data Coding Scheme Constants.
`<->`	`originatingAddress`	A buffer into which the originating address is stored. Note that if this buffer is too small to contain the entire retrieved string, the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBuffrSize` error.

<->	originatingAddressSize	The size of the originatingAddress buffer. Upon return, this is the actual size of the string, including the null terminator character. If the buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a telErrBufferSize error.
<-	report	The ID of the delivery confirmation report associated with the message.
<-	timeStamp	An TelSmsDateTimeType structure that stores the time at which the message corresponding to the report was received.



New TelSmsSendMessageType

```
typedef struct _TelSmsSendMessageType
{
  UInt32                    messageId;
  TelSmsSubmitMessageType   message;
} TelSmsSendMessageType;
```

Field Descriptions

<-	messageId	The SMS ID that was assigned by the telephone to the outgoing message. For a multi-part message, each part has its own message ID.
->	message	A structure of type TelSmsSubmitMessageType that contains the message data and parameters.



TelSmsSubmitAdvancedCDMAType

The TelSmsSubmitMessageType structure includes a TelSmsSubmitAdvancedCDMAType structure for CDMA messages.

```
typedef struct _TelSmsSubmitAdvancedCDMAType
{
  Boolean               manualAckRequest;
  UInt8                 messageType;
  TelSmsDateTimeType    deferredDate;
  UInt8                 priority;
  UInt8                 privacy;
  Boolean               alertOnDeliveryRequest;
  Char                  *callbackNumber;
  UInt8                 callbackNumberSize;
} TelSmsSubmitAdvancedCDMAType;
```

Field Descriptions

manualAckRequest	true if a reply is requested from the user, and false if not.
messageType	The type of the message. This must be one of the SMS Message Type Constants.
deferredDate	
priority	
privacy	The privacy type of the message. This must be one of the SMS Message Privacy Constants.
alertOnDeliveryRequest	true if the user is to be alerted upon delivery of this message, and false if not.

`callbackNumber`	A buffer into which the retrieved callback telephone number string is stored. Note that if this buffer is too small to contain the entire retrieved string, the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBufferSize` error.
`callbackNumberSize`	When the structure is used as an input parameter, this is the allocated size, in bytes, of the `callbackNumber` string. Upon return, this is the actual size of the string, including the null terminator character. If the buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.



New TelSmsSubmitAdvancedGSMType

The `TelSmsSubmitMessageType` structure includes a `TelSmsSubmitAdvancedGSMType` structure for GSM messages.

```
typedef struct _TelSmsSubmitAdvancedGSMType
{
  UInt16       protocolId;
  Boolean      rejectDuplicatedRequest;
  Boolean      replyPath;
  Char         *serviceCenterNumber;
  UInt8        serviceCenterNumberSize;
} TelSmsSubmitAdvancedGSMType;
```

Field Descriptions

->	`protocolId`	Specifies gateway information for routing a message to another transport. Some service centers provide a gateway between SMS and other transports such as mail and FAX. Service centers may reject messages with `protocolId` values that are reserved or unsupported. The mobile device does not interpret reserved or unsupported values, but does store them as received.
->	`rejectDuplicatedRequest`	A Boolean value that specifies if the service center should accept a submit message for a submit message that is still held in the service center when that message has the same identifier and destination address as a previously submitted message from the same originating address.
->	`replyPath`	The path that the service center can use to deliver a reply to the originating message. The reply path is requested by the originating mobile device by setting the `replyPath` parameter in the original submit message. If the service center supports reply path requests from the mobile device, the service center sets the `replyPath` parameter in the response.
->	`serviceCenterNumber`	A buffer containing the service telephone number string.
->	`serviceCenterNumberSize`	The allocated size, in bytes, of the `serviceCenterNumber` string.



New TelSmsSubmitAdvancedTDMAType

The `TelSmsSubmitMessageType` structure includes a `TelSmsSubmitAdvancedTDMAType` structure for TDMA messages.

```
typedef struct _TelSmsSubmitAdvancedTDMAType
{
  Boolean              manualAckRequest;
  UInt8                messageType;
  TelSmsDateTimeType   deferredDate;
  UInt8                priority;
  UInt8                privacy;
  Boolean              alertOnDeliveryRequest;
  Char                 *callbackNumber;
  UInt8                callbackNumberSize;
} TelSmsSubmitAdvancedTDMAType;
```

Field Descriptions

`manualAckRequest`	`true` if a reply is requested from the user, and `false` if not.
`messageType`	The type of the message. This must be one of the SMS Message Type Constants.
`deferredDate`	
`priority`	
`privacy`	The privacy type of the message. This must be one of the SMS Message Privacy Constants.
`alertOnDeliveryRequest`	`true` if the user is to be alerted upon delivery of this message, and `false` if not.

`callbackNumber`	A buffer into which the retrieved callback telephone number string is stored. Note that if this buffer is too small to contain the entire retrieved string, the string is truncated (and ends with the null terminator character) and the function using this structure generates a `telErrBufferSize` error.
`callbackNumberSize`	When the structure is used as an input parameter, this is the allocated size, in bytes, of the `callbackNumber` string. Upon return, this is the actual size of the string, including the null terminator character. If the buffer is too small to contain the entire retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.



New TelSmsSubmitMessageType

The `TelSmsReadReports` and `TelSmsSendMessageType` structures use a `TelSmsSubmitMessageType` structure to stored reports retrieved from the currently selected storage area.

```
typedef struct _TelSmsSubmitMessageType
{
  UInt16                version;
  Boolean               networkDeliveryRequest;
  Char                  *destinationAddress;
  UInt8                 destinationAddressSize;
  UInt16                dataSize;
  UInt8                 *data;
  UInt8                 dataCodingScheme;
  TelSmsDateTimeType    validityPeriod;
  UInt8                 standardType;
  union
  {
```

```
    TelSmsSubmitAdvancedGSMType   advancedGSM;
    TelSmsSubmitAdvancedCDMAType  advancedCDMA;
    TelSmsSubmitAdvancedTDMAType  advancedTDMA;
  } advancedParams;
  UInt8                   extensionsCount;
  TelSmsExtensionType     *extensionsP;
} TelSmsSubmitMessageType;
```

Field Descriptions

`-> version`	The version of the SMS API associated with this message.
`-> networkDeliveryRequest`	If this value is true, the service center accepts the submit message.
`<-> destinationAddress`	A buffer that contains the phone number of the message recipient.
`<-> destinationAddressSize`	The size of the destination address string.
`<-> dataSize`	The size of the `data` buffer.
`<-> data`	A buffer into which the retrieved message data is stored.
`-> dataCodingScheme`	The coding scheme used for the data. This is one of the SMS Data Coding Scheme Constants.
`-> validityPeriod`	An `TelSmsDateTimeType` structure that specifies the amount of time for which the message is valid.
`-> standardType`	Indicates which field of the `advancedParams` union contains the message information. This must be one of the Network Type Constants described in Chapter 71, "Telephony Network." If this value is `kTelNwkCDMA`, then the `advancedParams` union contains a `TelSmsDeliveryAdvancedCDMAType` structure, and similarly for the other values.

-> advancedParams	Advanced message information for GSM, CDMA, or TDMA messages. This is a pointer to one of the following structure types: TelSmsSubmitAdvancedCDMAType TelSmsDeliveryAdvancedGSMType TelSmsDeliveryAdvancedTDMAType
<-> extensionsCount	The number of extension structures allocated for this message. You must allocate at least one structure to specify the multipart information.
<-> extensionsP	A pointer to an array of TelSmsExtensionType structures that you have allocated for this message.



TelSmsSubmittedMessageType

The TelSmsReadSubmittedMessage function uses a TelSmsSubmittedMessageType structure to retrieve reports from the currently selected storage area.

```
typedef struct _TelSmsSubmittedMessageType
{
  UInt16                    index;
  TelSmsSubmitMessageType   message;
} TelSmsSubmittedMessageType;
```

Field Descriptions

-> index	The index of the message on the phone.
<- message	A TelSmsSubmitMessageType structure that represents the message.



TelSmsUserExtensionType

The TelSmsExtensionType structure uses a TelSmsUserExtensionType structure to describe a user-defined extended message header.

```
typedef struct _TelSmsUserExtensionType
  UInt8   *extHeader;
  UInt8   extHeaderSize;
} TelSmsUserExtensionType;
```

Field Descriptions

`<-> extHeader` On input, this field must be set to 0.

Upon return, this is a pointer to the user-defined header content.

`<-> extHeaderSize` On input, this field must be set to 0.

Upon return, this is the size of the user-defined header content.

Telephony SMS Constants

This section describes the constants used with the SMS service set of the telephony API.

SMS Extension Type Constants

The SMS extension type constants describe the type of extension used to represent part of a message.

Constant	Value	Description
`kTelSmsMultiPartExtensionTypeId`	`0x00`	A multipart short message.
`kTelSmsNbsExtensionTypeId`	`0x04`	An NBS message with short port number value.
`kTelSmsNbs2ExtensionTypeId`	`0x05`	An NBS message with long port number value.
`(any other value)`		A user-defined extension type.

SMS Message Type Constants

The SMS message type constants describe the delivery type of a message.

Constant	Value	Description
`kTelSmsMessageTypeDelivered`	`0`	A delivered message.
`kTelSmsMessageTypeReport`	`1`	A report message.
`kTelSmsMessageTypeSubmitted`	`2`	A submitted message.
`kTelSmsMessageTypeManualAck`	`3`	A manual acknowledgement message.
`kTelSmsMessageTypeAllTypes`	4	All messages.

SMS Message Transport Protocol Constants

The SMS message transport protocol constants describe the protocol used to deliver a message.

Constant	Value	Description
`ktelSmsDefaultProtocol`	`0`	The default message transport protocol.
`kTelSmsFaxProtocol`	`1`	A FAX message.
`kTelSmsX400Protocol`	`2`	An X.400 message.
`kTelSmsPagingProtocol`	`3`	A paged message.
`kTelSmsEmailProtocol`	`4`	An email message.
`kTelSmsErmesProtocol`	`5`	An Ermes message.
`kTelSmsVoiceProtocol`	`6`	A voice message.

SMS Storage ID Constants

The SMS storage ID constants describe the storage location of a message.

Constant	Value	Description
`kTelSmsStorageSIM`	`0`	Stored in the SIM.
`kTelSmsStoragePhone`	`1`	Stored in the phone.
`kTelSmsStorageAdaptor`	`2`	Stored in the telephone adaptor.
`kTelSmsStorageFirstOem`	`3`	Storage managed by the OEM. This constant specifies the first OEM storage area. You can specify additional OEM storage areas by incrementing this value. For example, to specify the third OEM storage area, use the following: `kTelSmsStorageFirstOem+2`

SMS Data Coding Scheme Constants

The SMS data coding scheme constants describe the encoding used for SMS data.

Constant	Value	Description
`kTelSms8BitsEncoding`	`0`	8-bit encoding.
`kTelSmsBitsASCIIEncoding`	`1`	ANSI X3.4 encoding.
`kTelSmsIA5Encoding`	`2`	CCITT T.50 encoding.
`kTelSmsIS91Encoding`	`3`	TIA/EIA/IS-91 section 3.7.1 encoding.
`kTelSmsUCS2Encoding`	`4`	UCS2 encoding; used with GSM only.
`kTelSmsDefaultGSMEncoding`	`5`	Default encoding for GSM only.

SMS Message Urgency Constants

The SMS message urgency constants describe the priority level of a message in a `TelSmsDeliveryAdvancedCDMAType` or `TelSmsDeliveryAdvancedTDMAType` structure.

Constant	Value	Description
`kTelSmsUrgencyNormal`	`0`	Normal urgency.
`kTelSmsUrgencyUrgent`	`1`	An urgent message.
`kTelSmsUrgencyEmergency`	`2`	An emergency message.

SMS Message Privacy Constants

The SMS message privacy constants describe the privacy type of a message in a CDMA or TDMA advanced parameters.

Constant	Value	Description
`kTelSmsPrivacyNotRestricted`	`0`	Privacy level 0.
`kTelSmsPrivacyRestricted`	`1`	Privacy level 1.
`kTelSmsPrivacyConfidential`	`2`	Privacy level 2.
`kTelSmsPrivacySecret`	`3`	Privacy level 3.

Telephony SMS Functions

This section describes the functions used with the SMS service set of the telephony API.



New TelSmsDeleteMessage

Purpose Delete an SMS report, delivered message, or submitted message.

Prototype

```
Err TelSmsDeleteMessage(UInt16 iRefnum,
TelAppID iAppId,
TelSmsDeleteMessageType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelSmsDeleteMessageType` structure that specifies the index and type of the message to delete.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSmsDeleteMessageType` structure passed to this function in the `iDelInfoP` parameter.
`functionId`	`kTelSmsDeleteMessageMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

If the deleted message has been delivered, the deletion is performed in the current storage, which you can set with the `TelSmsSelectStorage` function.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSmsSelectStorage`



New TelSmsGetAvailableStorage

Purpose Retrieve the list of all available storage on the phone.

Prototype

```
Err TelSmsGetAvailableStorage(UInt16 iRefnum,
TelAppID iAppId,
TelSmsGetAvailableStorageType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelSmsGetAvailableStorageType` structure that is filled in with information about the storage areas available on the phone.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSmsGetAvailableStorageType` structure passed to this function in the `ioParamP` parameter.
`functionId`	`kTelSmsGetAvailableStorageMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

The count of storage areas available on the phone is stored into the `count` field of the `TelSmsGetAvailableStorageType` structure referenced by `ioParamP`, and the storage ID of each available room is stored into the buffer referenced by the `storagesP` field. If the `storagesP` buffer is too small to contain all of the storage IDs, the buffer is truncated and this function returns the `telErrBufferSize` error. The `count` field of the structure is always updated to contain the total number of available storage areas on the phone.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelSmsSelectStorage`



New TelSmsGetDataMaxSize

Purpose

Returns the maximum length, in bytes, of a message on the current network.

Prototype

```
Err TelSmsGetDataMaxSize(UInt16 iRefnum,
TelAppID iAppId, UInt16 *oSizeP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oSizeP`	A pointer to an unsigned integer value that is updated witht he maximum length of an SMS message on the current network.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oSizeP` parameter.
`functionId`	`kTelGetDataMaxSizeMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oSizeP` remains in memory until the asynchronous call completes.

Comments You can use this function to determine the maximum size you need to allocate to read a message.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSmsReadMessage`, `TelSmsReadMessages`



New TelSmsGetMessageCount

Purpose Retrieve the total number of message slots, and the number of filled slots for the specified message type.

Prototype

```
Err TelSmsGetMessageCount(UInt16 iRefnum,
TelAppID iAppId,
TelSmsGetMessageCountType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelSmsGetMessageCountType` structure that specifies the message type and is filled in with the count information.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSmsGetMessageCountType` structure passed to this function in the `ioParamP` parameter.
`functionId`	`kTelSmsGetMessageCountMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

The currently selected storage area pertains only to delivered messages; other message types are stored on the phone, but not in a specific storage area. If you specify delivered messages, this function retrieves information about the messages in the currently selected SMS storage area on the phone. If you specify a different message type, this function retrieves information about the messages in the phone.

You specify the message type by assigning one of the SMS Message Type Constants to the `messageType` field of the `TelSmsGetMessageCountType` structure before calling this function.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSmsSelectStorage`



New TelSmsGetSelectedStorage

Purpose Retrieve the ID of the currently selected storage area on the phone.

Prototype

```
Err TelSmsGetSelectedStorage(UInt16 iRefnum,
TelAppID iAppId, UInt8 *oStorageIdP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oStorageIdP`	A pointer to an unsigned byte value that is assigned the ID of the currently selected storage area on the phone. The assigned ID value is one of the SMS Storage ID Constants.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oStorageIdP` parameter.
`functionId`	`kTelGetSelectedStorageMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oStorageIdP` remains in memory until the asynchronous call completes.

Comments

The currently selected storage area pertains only to delivered messages; other message types are stored on the phone, but not in a specific storage area.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelSmsSelectStorage`

New TelSmsGetUniquePartId

Purpose

Return a unique part identifier to assign to the `partId` field of a submit message.

Prototype

```
Err TelSmsGetUniquePartId(UInt16 iRefnum,
TelAppID iAppId, UInt16 *oUniqueIdP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oUniqueIdP`	A pointer to a unsigned integer value. Upon return, this is the unique part ID value.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oUniqueIdP` parameter.
`functionId`	`kTelGetUniquePartIdMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oUniquePartIdP` remains in memory until the asynchronous call completes.

Comments

This function corresponds to the `kTelUrqSmsGetUniquePartIdMessage` value.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also TelSmsSendMessage



TelSmsReadMessage

Purpose Retrieve a delivered message from the currently selected storage area.

Prototype

```
Err TelSmsReadMessage(UInt16 iRefnum,
TelAppID iAppId,
TelSmsDeliveryMessageType *ioMessageP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioMessageP`	A pointer to a `TelSmsDeliveryMessageType` structure that is filled in with the message. On input, the `index` field of this structure contains the index of the message that you want retrieved. Message indexes are zero-based. On input, the `extensionsCount` field specifies the number of extensions allocated in the `extensionsP` array. You must allocate at least one multi-part extension, even for a single-part message.

On input, the `dataSize` field specifies the allocated size of the `data` buffer, and the `originatingAddressSize` field specifies the allocated size of the `originatingAddress` string. Upon return, each size field specifies the complete size of the data that was stored into the buffer, even if the data had to be truncated to fit.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode` `errNone` upon success or an error code upon failure.

`transId` The transaction ID of the operation.

`paramP` Points to the `TelSmsDeliveryMessageType` structure passed to this function in the `ioMessageP` parameter.

`functionId` `kTelSmsReadMessageMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioMessageP` remains in memory until the asynchronous call completes.

Comments

The message data is stored into the `data` field of the `TelSmsDeliveryMessageType` structure referenced by `ioMessageP`. If the complete message data is too large to fit into the

allocated size of the `data` field, the message data is truncated and this function returns the `telErrBufferSize` error. The `dataSize` field of the structure is always updated to contain the actual size, in bytes, of the message data.

The originating address string is stored into the `originatingData` field of the `TelSmsDeliveryMessageType` structure referenced by `ioMessageP`. If the complete string is too large to fit into the allocated size of the `originatingData` field, the string is truncated (and ends with the null terminator character) and this function returns the `telErrBufferSize` error. The `originatingAddressSize` field of the structure is always updated to contain the actual size of the string.

Before calling this function, you need to allocate a number of fields and structures in and related to the `TelSmsDeliveryMessage` structure:

- Allocate each address field with a size of at least `kTelMaxPhoneNumberLen + 1`.

  For example, for a GSM message, you must allocate the `originatingAddress` and `serviceCenterNumber` fields in the `TelSmsDeliveryAdvancedGSMType` structure in the `TelSmsDeliveryMessage` structure.

- Allocate the message field data to have the maximum size of a message on the current network. You can determine this value by calling the `TelSmsGetDataMaxSize` function.

- Allocate at least one `TelSmsExtensionType` structure in the `TelSmsDeliveryMessage` structure. You must have at least one extension structure, even if your message has only a single part. If you do not allocate enough extensions for the message, `TelSmsReadMessage` returns an error. Palm recommends allocating between 3 and 5 extensions for a message.

- You should not allocate a pointer for user extension data. If you receive user extension data, the user extension pointer will reference a block in the message data. Do not deallocate the user extension data when you release the structure.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also TelSmsSelectStorage



TelSmsReadMessages

Purpose Retrieve a range of delivered messages from the currently selected storage area.

Prototype

```
Err TelSmsReadMessages(UInt16 iRefnum,
TelAppID iAppId, TelSmsReadMessagesType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum` The telephony manager library reference number.

`-> iAppId` The telephone application attachment identifier for your application.

`<-> ioParamP` A pointer to a `TelSmsReadMessagesType` structure.

On input, the `first` field of this structure specifies the index of the first message to retrieve. Message indexes are zero-based.

On input the `count` field of this structure specifies the allocated size of the `messagesP` buffer. Upon return, the `count` field specifies the actual number of messages that were available, even if that many could not fit into the buffer.

The `messagesP` buffer must contain pointers to `TelSmsDeliveryMessageType` structures that have been allocated. Each of these structures must be initialized as described for the `TelSmsReadMessage` function.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSmsReadMessagesType` structure passed to this function in the `ioParamP` parameter.
`functionId`	`kTelSmsReadMessagesMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` and all of the structures referenced by it remain in memory until the asynchronous call completes.

Comments

If the message data or originating address string data for any of the retrieved messages is larger than the allocated size of its corresponding buffer, the data is truncated into the buffer, and this function returns the `telErrBufferSize` error.

For more information about using this function and allocating structures for its use, see the Comments description for the `TelSmsReadMessage` function.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also TelSmsSelectStorage



TelSmsReadReport

Purpose Read a report from the currently selected storage area.

Prototype

```
Err TelSmsReadReport(UInt16 iRefnum,
TelAppID iAppId, TelSmsReportType *ioReportP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioReportP`	A pointer to a `TelSmsReportType` structure. On input the `index` field of this structure contains the index of the message that you want retrieved. Message indexes are zero-based. On input, the `dataSize` field specifies the allocated size of the `data` buffer, and the `originatingAddressSize` field specifies the allocated size of the `originatingAddress` string. Upon return, each size field specifies the complete size of the data that was stored into the buffer, even the data had to be truncated to fit.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSmsReportType` structure passed to this function in the `ioReportP` parameter.
`functionId`	`kTelSmsReadReportMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iEntryP` remains in memory until the asynchronous call completes.

Comments

The report data is stored into the `data` field of the `TelSmsReportType` structure referenced by `ioReportP`. If the complete message data is too large to fit into the allocated size of the `data` field, the report data is truncated and this function returns the `telErrBufferSize` error. The `dataSize` field of the structure is always updated to contain the actual size, in bytes, of the report data.

The originating address string is stored into the `originatingData` field of the `TelSmsReportType` structure referenced by `ioReportP`. If the complete string is too large to fit into the allocated size of the `originatingData` field, the string is truncated (and ends with the null terminator character) and this function returns the `telErrBufferSize` error. The `originatingAddressSize` field of the structure is always updated to contain the actual size of the string.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also TelSmsSelectStorage, TelSmsSendMessage



TelSmsReadReports

Purpose Retrieve a range of reports from the currently selected storage.

Prototype

```
Err TelSmsReadReports(UInt16 iRefnum,
TelAppID iAppId, TelSmsReadReportsType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum` The telephony manager library reference number.

`-> iAppId` The telephone application attachment identifier for your application.

`<-> ioParamP` A pointer to a TelSmsReadReportsType structure.

On input, the `first` field of this structure specifies the index of the first report to retrieve. Report indexes are zero-based.

On input the `count` field of this structure specifies the allocated size of the `reportsP` buffer. Upon return, the `count` field specifies the actual number of reports that were available, even if that many could not fit into the buffer.

The `reportsP` buffer must contain pointers to TelSmsReportType structures that have been allocated. Each of these structures must be initialized as described for the TelSmsReadReport function.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSmsReadReportsType` structure passed to this function in the `ioParamP` parameter.
`functionId`	`kTelSmsReadReportsMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` and all of the structured referenced by it remain in memory until the asynchronous call completes.

Comments

If the report data or originating address string data for any of the retrieved messages is larger than the allocated size of its corresponding buffer, the data is truncated into the buffer, and this function returns the `telErrBufferSize` error. For more information, see the Comments description for the `TelSmsReadReport` function.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSmsSelectStorage`, `TelSmsSendMessage`



New TelSmsReadSubmittedMessage

Purpose Read a previously submitted message that was kept on the phone after being sent.

Prototype

```
Err TelSmsReadSubmittedMessage(UInt16 iRefnum,
TelAppID iAppId,
TelSmsSubmittedMessageType *ioMessageP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum` The telephony manager library reference number.

`-> iAppId` The telephone application attachment identifier for your application.

`<-> ioMessageP` A pointer to a `TelSmsSubmitMessageType` structure that is filled in with the message.

On input, the `index` field contains the index of the message that you want retrieved. Message indexes are zero-based.

Upon return, the `message` field structure is filled in with the message information. You must initialize the buffer size fields of this structure prior to calling this function, including the `dataSize`, `callbackNumberSize`, `serviceCenterNumberSize`, and `destinationAddressSize` fields. Each of these fields is initialized with the allocated size of its corresponding buffer.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSmsSubmitMessageType` structure passed to this function in the `ioMessageP` parameter.
`functionId`	`kTelSmsSubmittedMessageMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iEntryP` remains in memory until the asynchronous call completes.

Comments

The message data is stored into the `data` field of the `TelSmsSubmitMessageType` structure referenced by `ioMessageP`. If the complete message data is too large to fit into the allocated size of the `data` field, the end of the message data is truncated and this function returns the `telErrBufferSize` error. The `dataSize` field of the structure is always updated to contain the actual size, in bytes, of the message data.

The same strategy applies to the `callbackNumber` buffer and `callbackNumberSize` fields, the `destinationAddress` buffer and `destinationAddressSize` fields, and the `serviceCenterNumber` buffer and `serviceCenterNumberSize` fields. If the size of the data for any

of the buffer fields exceeds the allocated length of the buffer, the end of the data is truncated and this function returns the `telErrBufferSize` error. Each of the size fields is always updated to contain the complete size of the data intended for the buffer.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSmsSelectStorage`, `TelSmsSendMessage`



New TelSmsReadSubmittedMessages

Purpose Retrieve a range of submitted messages from the currently selected storage area.

Prototype

```
Err TelSmsReadSubmittedMessages(UInt16 iRefnum,
TelAppID iAppId,
TelSmsReadSubmittedMessagesType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelSmsReadMessagesType` structure. On input, the `first` field of this structure specifies the index of the first message to retrieve. Message indexes are zero-based.

On input, the `count` field of this structure specifies the allocated size of the `submittedsP` buffer. Upon return, the `count` field specifies the actual number of messages that were available, even if that many could not fit into the buffer.

The `submittedsP` buffer must contain pointers to `TelSmsSubmittedMessageType` structures that have been allocated. Each of these structures must be initialized as described for the `TelSmsReadSubmittedMessage` function.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSmsReadSubmittedMessagesType` structure passed to this function in the `ioParamP` parameter.
`functionId`	`kTelSmsReadSubmittedMessagesMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` and all of the structured referenced by it remain in memory until the asynchronous call completes.

Comments If the message data or any of the other variable-length data for any of the retrieved messages is larger than the allocated size of its corresponding buffer, the end of the data is truncated, and this function returns the `telErrBufferSize` error. For more information, see the Comments description for the `TelSmsReadSubmittedMessage` function.

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSmsSelectStorage`, `TelSmsSendMessage`

New TelSmsSelectStorage

Purpose Select a storage area on the phone as the current storage area.

Prototype

```
Err TelSmsSelectStorage(UInt16 iRefnum,
TelAppID iAppId, UInt8 iStorageId,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> ioStorageId`	The ID of the storage area. This must be one of the SMS Storage ID Constants.

`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `ioStorageId` parameter.
`functionId`	`kTelSmsSelectStorageMessage`

Comments

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelSmsGetAvailableStorage`, `TelSmsGetSelectedStorage`



New TelSmsSendManualAcknowledge

Purpose

Send a manual acknowledgment of a previously received message. Note that this function is not supported on GSM networks.

Prototype

```
Err TelSmsSendManualAcknowledge(UInt16 iRefnum,
TelAppID iAppId, TelSmsManualAckType *ioAckP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioAckP`	A pointer to a structure of type `TelSmsManualAckType`. The fields of this structure specify information about the message being acknowledged. Upon return, the `messageId` field is filled in with the ID of the acknowledgment.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSmsManualAckType` structure passed to this function in the `ioAckP` parameter.
`functionId`	`kTelSmsSendManualAcknowledgeMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iEntryP` remains in memory until the asynchronous call completes.

Comments Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.



TelSmsSendMessage

Purpose Send an SMS message.

Prototype

```
Err TelSmsSendMessage(UInt16 iRefnum,
TelAppID iAppId,
TelSmsSendMessageType *ioMessageP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioMessageP`	A pointer to a structure of type `TelSmsSendMessageType`. On input, the `message` field of this structure contains a `TelSmsSubmitMessageType` with the message to send. You must also allocate and zero at least one `TelSmsExtensionType` structure for the multi-part information. On output, the `messageId` field of this structure is filled in with the ID that was assigned to the sent message.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelSmsSendMessageType` structure passed to this function in the `ioMessageP` parameter.
`functionId`	`kTelSmsSendMessageMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioMessageP` remains in memory until the asynchronous call completes.

Comments

You need to make multiple calls to the `TelSmsSendMessage` function to send your message:

- The first call to `TelSmsSendMessage` does not actually send the message. It computes the number of parts and fills in the multi-part extension structures in the `TelSmsSendMessageType` structure. Note that you must allocate at least one extension structure, even for single-part messages.
- Subsequent calls to `TelSmsSendMessage` actually send the data.
- To send an entire message, you need to call `TelSmsSendMessage` in a loop. Terminate the loop when an error occurs, or when the `byteSend` field of the first `TelSmsExtensionType` structure has the same value as the

`dataSize` field of the `TelSmsSendMessageType` structure that represents the message. For example:

```
while (!TelSmsSendMessage(...) &&
        msg.extensionP[0].extension.mp.byteSend != dataSend);
```

Before using this function, you should verify that it is available by calling the `TelIsSmsServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelSmsSelectStorage`, `TelCfgSetSmsCenter`

74

Telephony Phone Book

This chapter describes the phone book service set of the telephony API.

For more information about the telephony manager basic services and the different service sets, see Chapter 69, "Telephony Basic Services."

This chapter describes:

- Telephony Phone Book Data Structures
- Telephony Phone Book Constants
- Telephony Phone Book Functions

For more information about using the telephony manager, see Chapter 8, "Telephony Manager" in the *Palm OS Programmer's Companion*.

Telephony Phone Book Data Structures

This chapter describes the data structures used with the phone book service set of the telephony API.



New

TelPhbEntryType

The `TelPhbEntryType` structure describes a single entry in a phone book.

```
typedef struct _TelPhbEntryType
{
  UInt16   phoneIndex;
  Char     *fullName;
  UInt8    fullNameSize;
```

```
  Char     *dialNumber;
  UInt8    dialNumberSize;
} TelPhbEntryType;
```

Field Descriptions

->	`phoneIndex`	The index of the entry in the phone's phone book. This index is always zero-based. The telephony manager is responsible for converting this index into the physical index in the phone, if required.
<-	`fullName`	A buffer into which the retrieved full name of the entry is stored. This string is stored using the local character set of the Palm™ handheld device. The telephony manager is responsible for converting the character set, if required. Note that if this buffer is too small to contain the entire retrieved string, the string is truncated and the function using this structure generates a `telErrBufferSize` error.
<->	`fullNameSize`	The size of the `fullName string`. When the structure is used as an input parameter, this is the allocated size, in bytes, of the `fullName` buffer. Upon return, this is the actual size of the string, including the null terminator character. If the `fullName` buffer is too small to contain all of the retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.

<-	`dialNumber`	A buffer into which the retrieved telephone number of the entry is stored. Note that if this buffer is too small to contain the entire retrieved string, the string is truncated, and the function using this structure generates a `telErrBufferSize` error.
<->	`dialNumberSize`	The size of the `dialNumber string`. When the structure is used as an input parameter, this is the allocated size, in bytes, of the `dialNumber` buffer. Upon return, this is the actual size of the string, including the null terminator character. If the `dialNumber` buffer is too small to contain all of the retrieved string, this field is assigned the entire length of the string, and the function using this structure generates a `telErrBufferSize` error.



New TelPhbGetAvailablePhonebooksType

The `TelPhbGetAvailablePhonebooks` functions uses the `TelPhbGetAvailablePhonebooksType` structure to return a list of the phone books available on the phone.

```
typedef struct
_TelPhbGetAvailablePhonebooksType
{
  UInt16   count;
  UInt8    *phonebooksP;
} TelPhbGetAvailablePhonebooksType;
```

Field Descriptions

`<-> count` The number of entries in the array referenced by `phonebooksP`.

When the structure is used as an input parameter, this is the allocated size, in bytes, of the `phonebooksP` buffer. Upon return, this is the actual size of the retrieved data.

Upon return, this is the actual number of phone book IDs that could be retrieved. If the `phonebooksP` buffer is too small to contain all of the IDs, this field is assigned the actual count, and the function using this structure generates a `telErrBufferSize` error.

`<- phonebooksP` An array of retrieved phone book IDs. Each ID is one of the Phone Book Type Constants.



New TelPhbGetEntriesType

The `TelPhbGetEntries` function uses the `TelPhbGetEntriesType` structure to return a list of phone entries.

```
typedef struct _TelPhbGetEntriesType
{
  UInt16             first;
  UInt16             count;
  TelPhbEntryType    *entriesP;
} TelPhbGetEntriesType;
```

Field Descriptions

`-> first` The index of the first entry in the array referenced by `entriesP`.

`<-> count`	When the structure is used as an input parameter, this is number of entries that you want retrieved. Upon return, this is the actual number of entries that were retrieved.
`<- entriesP`	An array of pointers to retrieved `TelPhbEntryType` structures.



TelPhbGetEntryCountType

The `TelPhbGetEntryCount` function uses the `TelPhbGetEntryCountType` structure to return information about the entries in the currently selected phone book.

```
typedef struct _TelPhbGetEntryCountType
{
  UInt16          slots;
  UInt16          count;
} TelPhbGetEntryCountType;
```

Field Descriptions

`<- slots`	The total number of entry slots available in the phone book.
`c count`	The number of filled slots in the phone book.



TelPhbGetEntryMaxSizesType

The `TelPhbGetEntryMaxSizes` function uses the `TelPhbGetEntryMaxSizesType` structure to return size information about the entries in the currently selected phone book.

```
typedef struct _TelPhbGetEntryMaxSizeType
{
  UInt8           fullNameMaxSize;
  UInt8           dialNumberMaxSize;
} TelPhbGetEntryMaxSizesType;
```

Field Descriptions

`<- fullNameMaxSize` The largest size of any `fullName` field in the phone book.

`<- dialNumberMaxSize` The largest size of any `dialNumber` field in the phone book.

Telephony Phone Book Constants

This section describes the constants used with the phone book service set of the telephony API.

Phone Book Type Constants

The phone book type constants specify the type of phone book that is currently selected.

Constant	Value	Description
`kTelPhbFixedPhonebook`	`0`	The phone book stored on the phone.
`kTelPhbSimPhonebook`	`1`	The phone book stored on the SIM card.
`kTelPhbPhonePhonebook`	`2`	The phone book stored on the phone.
`kTelPhbLastDialedPhonebook`	`3`	The phone book from which a telephone number was most recently dialed.
`kTelPhbSimAndPhonePhonebook`	`4`	The combined phone and SIM card phone books.
`kTelPhbAdaptorPhonebook`	`5`	The phone book stored on the telephone adaptor.
`kTelPhbFirstOemPhonebook`	`6`	The ID of the first OEM phone book. You can specify additional OEM phone books by incrementing this value; for example, the second OEM phone book is specified as: `kTelPhbFirstOemPhonebook+1`

Telephony Phone Book Functions

This section describes the functions used with the phone book service set of the telephony API.



TelPhbAddEntry

Purpose

Add or replace an entry in the currently selected phone book.

Prototype

```
Err TelPhbAddEntry(UInt16 iRefnum,
TelAppID iAppId, TelPhbEntryType *iEntryP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`-> iEntryP`	A pointer to a `TelPhbEntryType` structure that contains the new entry information.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.

`paramP`	Points to the `TelPhbEntryType` structure passed to this function in the `iEntryP` parameter.
`functionId`	`kTelPhbAddEntryMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iEntryP` remains in memory until the asynchronous call completes.

Comments The `phoneIndex` field of the `TelPhbEntryType` structure referenced by `iEntryP` specifies the index at which to write the entry.

Before using this function, you should verify that it is available by calling the `TelIsPhbServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelPhbDeleteEntry`, `TelPhbSelectPhonebook`

New TelPhbDeleteEntry

Purpose Deletes an entry from the currently selected phone book.

Prototype

```
Err TelPhbDeleteEntry(UInt16 iRefnum,
TelAppID iAppId, UInt16 iEntryIndex,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.

`-> iEntryIndex`	The zero-based, logical index of the entry that you want deleted. The Telephony Manager computes the physical index.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `iEntryIndex` parameter.
`functionId`	`kTelPhbDeleteEntryMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `iEntryP` remains in memory until the asynchronous call completes.

Comments

Before using this function, you should verify that it is available by calling the `TelIsPhbServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelPhbAddEntry`, `TelPhbSelectPhonebook`



New TelPhbGetAvailablePhonebooks

Purpose

Retrieve the list of all phone books available on the phone.

Prototype

```
Err TelPhbGetAvailablePhonebooks(UInt16 iRefnum,
TelAppID iAppId,
TelPhbGetAvailablePhonebooksType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioParamP`	A pointer to a `TelPhbGetAvailablePhonebooksType` structure that lists the available phone books. On input, the `count` field of this structure specifies the allocated size of the `phonebookP` buffer. Upon return, the `count` field specifies the actual number of entries retrieved, even if they were truncated to fit into the buffer.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelPhbGetAvailablePhonebooksType` structure passed to this function in the `iEntryP` parameter.
`functionId`	`kTelPhbGetAvailablePhonebooksMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` remains in memory until the asynchronous call completes.

Comments

The phone book IDs are stored into the `phonebookP` field of the `TelPhbGetAvailablePhonebooksType` structure referenced by `ioParamP`. If the `phonebookP` buffer is too small to contain all of the IDs, the information is truncated and this function returns the `telErrBufferSize` error. The `count` field of the structure is always updated to contain the actual number of entries that were retrieved.

Before using this function, you should verify that it is available by calling the `TelIsPhbServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelPhbSelectPhonebook`



New TelPhbGetEntries

Purpose Retrieve a range of entries from the currently selected phone book.

Prototype

```
Err TelPhbGetEntries(UInt16 iRefnum,
TelAppID iAppId, TelPhbGetEntriesType *ioParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum` The telephony manager library reference number.

`-> iAppId` The telephone application attachment identifier for your application.

`<-> ioParamP` A pointer to a `TelPhbGetEntriesType` structure that is updated with the phone book entry information. The first entry retrieved is specified in the `first` field of this structure, which is zero-based; the number of entries retrieved is specified by the `count` field. Thus, the last entry retrieved is specified by:

```
ioParamP->first +
    ioParamP->count-1
```

Upon return, the `count` field of the structure is the number of entries that were actually retrieved.

The `entriesP` field of this structure is a buffer that you allocate to contain the required number of pointers. Each pointer references a `TelPhbEntryType` structure that you must also preallocate.

On input, the `fullNameSize` and `dialNumberSize` fields of this structure specify the allocated sizes of the `fullName` and `dialNumber` buffers. Upon return, the `fullNameSize` and `dialNumberSize` fields specify the actual sizes of the buffers, even if a string was truncated to fit into the buffer.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelPhbGetEntriesType` structure passed to this function in the `ioEntriesP` parameter.
`functionId`	`kTelPhbGetEntriesMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `ioParamP` and any structures that it references remain in memory until the asynchronous call completes.

Comments

The phone book information is stored into the `TelPhbEntryType` structures that you preallocate and refer to in the `entriesP` field of the `TelPhbGetEntriesType` referenced by the `ioParamP` parameter.

If any buffer in any of the `TelPhbEntryType` structures is too small, the string intended for that buffer is truncated, and this function returns the `telErrBufferSize` error. In any case, the `fullNameSize` and `dialNumberSize` fields of each `TelPhbEntryType` structure contain the actual size of their respective strings.

If any entries in the specified range are empty, the entry is not retrieved, and the `count` value in the structure is updated.

Before using this function, you should verify that it is available by calling the `TelIsPhbServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelPhbGetEntry`, `TelPhbSelectPhonebook`



New TelPhbGetEntry

Purpose Retrieve one entry from the currently selected phone book.

Prototype

```
Err TelPhbGetEntry(UInt16 iRefnum,
TelAppID iAppId, TelPhbEntryType *ioEntryP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<-> ioEntryP`	A pointer to a `TelPhbEntryType` structure that is updated with the phone book entry information. On input, the `fullNameSize` and `dialNumberSize` fields of this structure specify the allocated sizes of the `fullName` and `dialNumber` buffers. Upon return, the `fullNameSize` and `dialNumberSize` fields specify the actual sizes of the buffers, even if a string was truncated to fit into the buffer.

`<-> ioTransIdP` Set the value of this parameter to `NULL` to cause the function to execute synchronously.

If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelPhbEntryType` structure passed to this function in the `ioEntryP` parameter.
`functionId`	`kTelPhbGetEntryMessage`

Comments

The phone book information is stored into the `TelPhbEntryType` that you preallocate. If either buffer in the structure is too small, the string intended for that buffer is truncated, and this function returns the `telErrBufferSize` error. In any case, the `fullNameSize` and `dialNumberSize` fields of the structure contain the actual size of their respective strings.

Before using this function, you should verify that it is available by calling the `TelIsPhbServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelPhbGetEntries`, `TelPhbSelectPhonebook`



New TelPhbGetEntryCount

Purpose

Retrieve the total number of entries, and the number of filled entries in the currently selected phone book.

Prototype

```
Err TelPhbGetEntryCount(UInt16 iRefnum,
TelAppID iAppId, TelPhbGetEntryCountType *oParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oParamP`	A pointer to a `TelPhbGetEntryCountType` structure that is updated with information about the number of entries in the phone book.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.

`paramP`	Points to the `TelPhbGetEntryCountType` structure passed to this function in the `oParamP` parameter.
`functionId`	`kTelPhbGetEntryCountMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `oParamP` remains in memory until the asynchronous call completes.

Comments The total number of slots and the number of filled slots in the currently selected phone book are stored in the `TelPhbGetEntryCountType` structure referenced by `oParamP`.

Before using this function, you should verify that it is available by calling the `TelIsPhbServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelPhbSelectPhonebook`



TelPhbGetEntryMaxSizes

Purpose Retrieves the maximum buffer sizes of any entries in the currently selected phone book.

Prototype

```
Err TelPhbGetEntryMaxSizes(UInt16 iRefnum,
TelAppID iAppId,
TelPhbGetEntryMaxSizesType *oParamP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.

`<- oParamP`	A pointer to a `TelPhbGetEntryMaxSizesType` structure that is updated with information about the maximum buffer sizes of entries in the phone book.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the `TelPhbGetEntryMaxSizesType` structure passed to this function in the `oParamP` parameter.
`functionId`	`kTelPhbGetEntryMaxSizesMessage`

WARNING! When using this function asynchronously, you must ensure that the structure referenced by `oParamP` remains in memory until the asynchronous call completes.

Comments

The maximum size of any full name entry and the maximum size of any telephone number entry in the currently selected phone book are stored in the `TelPhbGetEntryMaxSizesType` structure referenced by `oParamP`.

Before using this function, you should verify that it is available by calling the `TelIsPhbServiceAvailable` macro.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `TelPhbGetEntries`, `TelPhbGetEntry`



New TelPhbGetSelectedPhonebook

Purpose Retrieve the ID of the currently selected phone book.

Prototype

```
Err TelPhbGetSelectedPhonebook(UInt16 iRefnum,
TelAppID iAppId, UInt8 *oPhbIdP,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.
`<- oPhbIdP`	A pointer to an unsigned byte value. Upon return, this is filled in with the identifier of the currently selected phone book. The identifier is one of the Phone Book Type Constants.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `oPhIdP` parameter.
`functionId`	`kTelPhbGetSelectedPhonebookMessage`

WARNING! When using this function asynchronously, you must ensure that the value referenced by `oPhbIdP` remains in memory until the asynchronous call completes.

Comments

Before using this function, you should verify that it is available by calling the `TelIsPhbServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelPhbSelectPhonebook`

New TelPhbSelectPhonebook

Purpose

Make the specified phone book the currently selected phone book.

Prototype

```
Err TelPhbSelectPhonebook(UInt16 iRefnum,
TelAppID iAppId, UInt8 iPhbId,
UInt16 *ioTransIdP);
```

Parameters

`-> iRefnum`	The telephony manager library reference number.
`-> iAppId`	The telephone application attachment identifier for your application.

`-> iPhbId`	The identifier of the phone book that you want selected as the current phone book. This must be one of the Phone Book Type Constants.
`<-> ioTransIdP`	Set the value of this parameter to `NULL` to cause the function to execute synchronously. If this parameter is not `NULL`, the call executes asynchronously. Upon return from this function, this points to the transaction identifier associated with the asynchronous operation.

Synchronous Result

Returns `errNone` if the function was successful or returns an error code if not successful.

Asynchronous Result

The following fields are updated in the `TelEventType` event that is sent when the operation completes:

`returnCode`	`errNone` upon success or an error code upon failure.
`transId`	The transaction ID of the operation.
`paramP`	Points to the unsigned integer value passed to this function in the `iPhbId` parameter.
`functionId`	`kTelPhbSelectPhonebookMessage`

Comments

Before using this function, you should verify that it is available by calling the `TelIsPhbServiceAvailable` macro.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`TelPhbGetAvailablePhonebooks`, `TelPhbGetSelectedPhonebook`



Part IV: Libraries

75

Internet Library

This chapter provides reference material for the Internet library API:

- Internet Library Data Structures
- Internet Library Constants
- Internet Library Functions

The header file `INetMgr.h` declares the Internet library API. For more information on the Internet library, see the chapter "Network Communication" in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

NOTE: The information in this chapter applies only to version 3.2 or later of the Palm OS® on Palm VII® devices. These features are implemented only if the Wireless Internet Feature Set is present.

WARNING! In future OS versions, Palm, Inc. does not intend to support or provide backward compatibility for the Internet library API documented in this chapter.

Internet Library Data Structures

INetCompressionTypeEnum

The `INetCompressionTypeEnum` enum indicates the type of compression used for data exchanged via a socket. One of these enumerated types is set as the value of the `inetSockSettingCompressionTypeID` socket setting (a read-only setting).

```
typedef enum {
inetCompressionTypeNone = 0,
inetCompressionTypeBitPacked,
inetCompressionTypeLZ77
} INetCompressionTypeEnum;
```

Value Descriptions

`inetCompressionTypeNone`	No compression.
`inetCompressionTypeBitPacked`	Custom 5-bit compression scheme. This is typically used for data sent from the Palm Web Clipping Proxy server.
`inetCompressionTypeLZ77`	Not used; reserved for future use.

INetConfigNameType

The `INetConfigNameType` structure holds the name of an Internet library network **configuration**. A configuration is a set of specific values for the Internet library settings. The Internet library defines a set of built-in configuration aliases for common network setups. These aliases point to configurations instead of holding the actual values themselves. You can use an alias anywhere in the API you would use a configuration. System-defined configuration aliases are listed in "Configuration Aliases" on page 1634.

This structure is used in the functions `INetLibConfigIndexFromName`, `INetLibConfigRename`, and `INetLibConfigSaveAs`.

```
#define inetConfigNameSize 32;

typedef struct {
Char name[inetConfigNameSize]; // name of configuration
} INetConfigNameType, *INetConfigNamePtr;
```

Field Description

`name`	A configuration name (up to 32 characters).

INetContentTypeEnum

The `INetContentTypeEnum` enum specifies the type of content to be exchanged via a socket. One of these enumerated types is set as the value of the `inetSockSettingContentTypeID` socket setting (a read-only setting).

```
typedef enum {
inetContentTypeTextPlain = 0,
inetContentTypeTextHTML,
inetContentTypeImageGIF,
inetContentTypeImageJPEG,
inetContentTypeApplicationCML,
inetContentTypeImagePalmOS,
inetContentTypeOther
} INetContentTypeEnum;
```

Value Descriptions

`inetContentTypeTextPlain`	Not used
`inetContentTypeTextHTML`	Not used
`inetContentTypeImageGIF`	Not used
`inetContentTypeImageJPEG`	Not used
`inetContentTypeApplicationCML`	Compressed HTML content (format used by the Palm Web Clipping Proxy server and PQAs)
`inetContentTypeImagePalmOS`	Palm OS® bitmap
`inetContentTypeOther`	Some undefined content type

INetHTTPAttrEnum

The `INetHTTPAttrEnum` enum specifies HTTP request and response attributes that are set by `INetLibSockHTTPAttrSet` and returned by `INetLibSockHTTPAttrGet`.

```
typedef enum {

//-------------------------------------------------------------
// Request only attributes
//-------------------------------------------------------------
// The following are ignored unless going through a CTP proxy
inetHTTPAttrWhichPart, // (W) UInt32 (0 -> N)
inetHTTPAttrIncHTTP, // (W) UInt32 (Boolean) only applicable
       // when inetHTTPAttrConvAlgorithm set to ctpConvNone
inetHTTPAttrCheckMailHi, // (W) UInt32
inetHTTPAttrCheckMailLo, // (W) UInt32
inetHTTPAttrReqContentVersion, // (W) UInt32, desired content
           // version. Represented as 2 low bytes. Lowest
byte is
          // minor version, next higher byte is major
version.
//-------------------------------------------------------------
---
// Response only attributes
//-------------------------------------------------------------
---
// Server response info
inetHTTPAttrRspSize, // (R) UInt32, entire HTTP Response size
           // including header and data
inetHTTPAttrResult, // (R) UInt32 (ctpErrXXX when using CTP
Proxy)
inetHTTPAttrErrDetail, // (R) UInt32 (server/proxy err code
when
           // using CTP Proxy)
inetHTTPAttrReason, // (R) Char[]
// Returned entity attributes
inetHTTPAttrContentLength, // (R) UInt32
inetHTTPAttrContentLengthUncompressed, // (R) UInt32 (in
bytes)
inetHTTPAttrContentLengthUntruncated, //(R) UInt32
inetHTTPAttrContentVersion, // (R) UInt32, actual content
version.
          // Represented as 2 low bytes. Lowest byte is minor
          // version, next higher byte is major version.
inetHTTPAttrContentCacheID, // (R) UInt32, cacheID for this
item
inetHTTPAttrReqSize // (R) UInt32 size of request sent
} INetHTTPAttrEnum;
```

Value Descriptions

`inetHTTPAttrWhichPart`	An index to the part of the response data desired, if the response data is partitioned into chunks. Write-only.
`inetHTTPAttrIncHTTP`	A Boolean that, if set, causes HTTP header data to be included as part of the content when retrieving raw data. Applicable only when `inetSettingConvAlgorithm` is set to `ctpConvNone`. Write-only.
`inetHTTPAttrCheckMailHi`	High-order byte of ID for checking mail. Write-only.
`inetHTTPAttrCheckMailLo`	Low-order byte of ID for checking mail. Write-only.
`inetHTTPAttrReqContentVersion`	Desired content version. Represented as 2 low bytes. Lowest byte is minor version, next higher byte is major version. Write-only.
`inetHTTPAttrRspSize`	Size of entire HTTP (header and data). Read-only.
`inetHTTPAttrResult`	Transport protocol error code. Read-only.
`inetHTTPAttrErrDetail`	Server/proxy error code when using the Palm Web Clipping Proxy server. Read-only.
`inetHTTPAttrReason`	Transport protocol error message. Read-only.
`inetHTTPAttrContentLength`	Size of response data. Read-only.
`inetHTTPAttrContentLengthUncompressed`	Size of uncompressed response data. Read-only.
`inetHTTPAttrContentLengthUntruncated`	Total size of response data (it may have been truncated to less than this). Read-only.

`inetHTTPAttrContentVersion`	Actual content version. Represented as 2 low bytes. Lowest byte is minor version, next higher byte is major version. Read-only.
`inetHTTPAttrContentCacheID`	Cache ID for this item. Read-only.
`inetHTTPAttrReqSize`	Size of request sent. Read-only.

INetSchemeEnum

The `INetSchemeEnum` enum specifies a protocol (http, https, etc.) used by a socket. Specify one of these enumerated types for the `INetSockSettingScheme` socket setting and for the `scheme` parameter to the `INetLibSockOpen` call.

```
typedef enum {
inetSchemeUnknown = -1,
inetSchemeDefault = 0,   // not used

inetSchemeHTTP, // http:
inetSchemeHTTPS, // https:
inetSchemeFTP, // ftp:
inetSchemeGopher, // gopher:
inetSchemeFile, // file:
inetSchemeNews, // news:
inetSchemeMailTo, // mailto:
inetSchemePalm, // palm:
inetSchemePalmCall, // palmcall:
inetSchemeMail, // not applicable to URLs, but used
                    //  for the INetLibSockOpen call when
                    //  creating a socket for mail IO
inetSchemeMac, // mac: - Mac file system HTML

inetSchemeFirst = inetSchemeHTTP, // first one
inetSchemeLast = inetSchemeMail // last one
} INetSchemeEnum;
```

Value Descriptions

`inetSchemeHTTP`	Use the HTTP protocol.
`inetSchemeHTTPS`	Use the HTTPS protocol (for a secure connection).

`inetSchemeFTP`	Use the FTP protocol. Not implemented.
`inetSchemeGopher`	Use the Gopher protocol. Not implemented.
`inetSchemeFile`	Launch local PQA file
`inetSchemeNews`	Use the News protocol. Not implemented.
`inetSchemeMailTo`	Launch the local messaging application, passing a "to" address.
`inetSchemePalm`	Launches a local application database. The URL is expected to be in the form cccc.tttt, where cccc is a four character creator name and tttt is a four character database type. This pair of strings is used to identify an application database to receive the launch message via a call to `SysUIAppSwitch`.
`inetSchemePalmCall`	Launches a local application database. The URL is expected to be in the form cccc.tttt, where cccc is a four character creator name and tttt is a four character database type. This pair of strings is used to identify an application database to receive the launch message via a call to `SysAppLaunch`.
`inetSchemeMail`	Creates a socket for mail I/O.
`inetSchemeMac`	Handles opening Mac OS file system HTML URLs. For use by the Simulator only.

INetSettingEnum

The `INetSettingEnum` enum specifies a setting to be returned or set by the `INetLibSettingGet` or `INetLibSettingSet` calls.

```
typedef enum {
inetSettingCacheSize, // (RW) UInt32, max size of cache
inetSettingCacheRef, // (R) DmOpenRef, ref of cache DB
inetSettingNetLibConfig, // (RW) UInt32, NetLib config to use
inetSettingRadioID, // (R) UInt32[2], the 64-bit radio ID
inetSettingBaseStationID, // (R)  UInt32, the radio base
station Id
inetSettingMaxRspSize, // (W) UInt32 (in bytes)
inetSettingConvAlgorithm, // (W) UInt32 (CTPConvEnum)
inetSettingContentWidth, // (W) UInt32 (in pixels)
```

```
inetSettingContentVersion, // (W) UInt32, content version
(encoder
                                        // version)
inetSettingNoPersonalInfo, // (RW) UInt32, send no deviceID/
zipcode
inetSettingUserName,
inetSettingLast
} INetSettingEnum;
```

Value Descriptions

`inetSettingCacheSize`	Maximum size of cache (in bytes).
`inetSettingCacheRef`	`DmOpenRef`, reference to cache database. Read-only.
`inetSettingNetLibConfig`	The index of the net library network configuration to use. This value is saved as part of the preferences for each Internet library configuration. A value of 0 means to use the current configuration.
`inetSettingRadioID`	64-bit radio ID. Read-only. Used for wireless connections only.
`inetSettingBaseStationID`	Radio base station ID. Read-only. Used for wireless connections only.
`inetSettingMaxRspSize`	Maximum response size, in bytes. The default is 1024 bytes. Write-only.
`inetSettingConvAlgorithm`	Content conversion desired. Write-only. Possible values include: `ctpConvCML` (use 5-bit compression scheme), `ctpConvCML8Bit` (use 5-bit compression scheme, but in 8-bit form for debugging), `ctpConvCMLLZ77` (use LZ77 compression scheme), `ctpConvNone` (no conversion; data is returned in native format)
`inetSettingContentWidth`	Width of the display for content. The default setting is 160 (pixels). Write-only.

`inetSettingContentVersion`	Content version (encoder version). Write-only. This setting is used to let the server know what encoder version it should use to encode content sent to the Palm client. Normally you don't need to set this value as it is initialized by `INetLibOpen`. The default encoder version is 0x8001.
`inetSettingNoPersonalInfo`	Send no device ID or zipcode information to the proxy server. This value is saved as part of the preferences for each Internet library configuration.
`inetSettingUserName`	Not applicable.

INetSockSettingEnum

The `INetSockSettingEnum` enum specifies a socket setting to be returned or set by the `INetLibSockSettingGet` or `INetLibSockSettingSet` calls.

```
typedef enum {
inetSockSettingScheme, // (R) UInt32, INetSchemeEnum
inetSockSettingSockContext, // (RW) UInt32,
inetSockSettingCompressionType, // (R) Char[]
inetSockSettingCompressionTypeID, // (R) UInt32
                                                    //
(INetCompressionTypeEnum)
inetSockSettingContentType, // (R) Char[]
inetSockSettingContentTypeID, // (R) UInt32
(INetContentTypeEnum)
inetSockSettingData, // (R) UInt32, pointer to data
inetSockSettingDataHandle,// (R) UInt32, handle to data
inetSockSettingDataOffset,// (R) UInt32, offset to data from
handle
inetSockSettingTitle, // (W) Char[]
inetSockSettingURL, // (R) Char[]
inetSockSettingIndexURL, // (RW) Char[]
inetSockSettingFlags, // (RW) UInt16, one or more of
                                  // inetOpenURLFlagXXX flags
inetSockSettingReadTimeout, // (RW) UInt32, read timeout in
ticks
inetSockSettingContentVersion,// (R) UInt32, content version
number
inetSockSettingLast
} INetSockSettingEnum;
```

Value Descriptions

`inetSockSettingScheme`	Requested scheme; one of the `INetSchemeEnum` values. Read-only.
`inetSockSettingSockContext`	Not used.
`inetSockSettingCompressionType`	Name of requested compression type. Read-only.
`inetSockSettingCompressionTypeID`	Requested compression type; one of the `INetCompressionTypeEnum` values. Read-only.
`inetSockSettingContentType`	String containing the MIME type of the content. Used only on received raw data. Read-only.
`inetSockSettingContentTypeID`	Content type of socket data; one of the `INetContentTypeEnum` values. Read-only.
`inetSockSettingData`	Pointer to socket data. Read-only.
`inetSockSettingDataHandle`	Handle to socket data. Read-only.
`inetSockSettingDataOffset`	Offset to socket data from handle. Read-only.
`inetSockSettingTitle`	Web page title. This value is written to the cache (and the Web Clipping Application Viewer uses it later in a history list of cache entries). Write-only.
`inetSockSettingURL`	URL of requested data. Read-only.
`inetSockSettingIndexURL`	Index (or master) URL of requested data (for cache indexing). This is the topmost web page in a group of hierarchical pages; it serves to group the pages together and to filter cache list results. The Web Clipping Application Viewer sets this to the URL of the active PQA, for all pages linked from the PQA.

`inetSockSettingFlags`	URL request flags; one or more of `inetOpenURLFlag...` flags (see URL Open Constants).
`inetSockSettingReadTimeout`	The default timeout value for reads when the application uses the event mechanism. The time since last receiving data from a socket is monitored and a timeout error status event is returned from `INetLibGetEvent` if the timeout is exceeded.
`inetSockSettingContentVersion`	Content version number. Read-only.

INetStatusEnum

The `INetStatusEnum` enum specifies the status of the socket. The status is returned in the `inetSockStatusChangeEvent` event structure and by the call `INetLibSockStatus`.

```
typedef enum {
inetStatusNew, // just opened
inetStatusResolvingName, // looking up host address
inetStatusNameResolved, // found host address
inetStatusConnecting, // connecting to host
inetStatusConnected, // connected to host
inetStatusSendingRequest, // sending request
inetStatusWaitingForResponse, // waiting for response
inetStatusReceivingResponse, // receiving response
inetStatusResponseReceived, // response received
inetStatusClosingConnection, // closing connection
inetStatusClosed, // closed
inetStatusAcquiringNetwork, // network temporarily
                                    // unreachable; socket on
hold
inetStatusPrvInvalid = 30 // internal value, not returned by
                                    // INetMgr. Should be
last.
} INetStatusEnum;
```

Value	Descriptions
`inetStatusNew`	Just opened
`inetStatusResolvingName`	Looking up host address
`inetStatusNameResolved`	Found host address
`inetStatusConnecting`	Connecting to host
`inetStatusConnected`	Connected to host
`inetStatusSendingRequest`	Sending request
`inetStatusWaitingForResponse`	Waiting for response
`inetStatusReceivingResponse`	Receiving response
`inetStatusResponseReceived`	Response received
`inetStatusClosingConnection`	Closing connection
`inetStatusClosed`	Connection closed
`inetStatusAcquiringNetwork`	Network temporarily unreachable; socket on hold
`inetStatusPrvInvalid`	Not used

Internet Library Constants

Configuration Aliases

The constants listed here specify Internet library network configuration alias names. Most of the Internet library API requires a configuration index rather than a name. Use `INetLibConfigIndexFromName` to obtain the alias's index from the name. For more information, see `INetConfigNameType`.

The following aliases are defined for configuration names:

Alias	Name string	Description
`inetCfgNameDefault`	".Default"	Initially points to a generic configuration with no proxy. This uses the configuration set by the user in the Network preferences panel.
`inetCfgNameDefWireline`	".DefWireline"	Initially points to a generic configuration with no proxy. This uses the configuration set by the user in the Network preferences panel.
`inetCfgNameDefWireless`	".DefWireless"	Initially points to a generic configuration with no proxy. This uses the configuration set by the user in the Network preferences panel.
`inetCfgNameCTPDefault`	".CTPDefault"	Initially points to either ".CTPWireless" (on Palm VII® units) or ".CTPWireline" (on all other units). On the Palm VII unit, the Web Clipping Application Viewer application uses this configuration.
`inetCfgNameCTPWireline`	".CTPWireline"	Initially points to a wireline configuration that uses the Palm Web Clipping Proxy server.
`inetCfgNameCTPWireless`	".CTPWireless"	Initially points to a wireless configuration that uses the Palm.Net™ wireless system and the Palm Web Clipping Proxy server.

URL Info Constants

The `inetURLInfoFlag...` constants convey information about a URL and are returned by the function `INetLibURLGetInfo`.

Constant	Value	Description
inetURLInfoFlagIsSecure	0x0001	URL was encrypted.
inetURLInfoFlagIsRemote	0x0002	URL was retrieved from the net.
inetURLInfoFlagIsInCache	0x0004	URL is stored in the cache.

URL Open Constants

The inetOpenURLFlag... constants control how the INetLibURLOpen call operates with respect to caching and encryption. These flags are also used for the inetSockSettingFlags socket setting.

Constant	Value	Description
inetOpenURLFlagLookInCache	0x0001	Read data from the cache, if available.
inetOpenURLFlagKeepInCache	0x0002	Store the item in the cache, overwriting any other entries with an equivalent URL.
inetOpenURLFlagForceEncOn	0x0008	Use encryption even if scheme does not desire it.
inetOpenURLFlagForceEncOff	0x0010	Do not use encryption even if scheme desires it.

Internet Library Functions

INetLibCacheGetObject

Purpose Returns information about an entry in the cache database, including a handle to the record. Either the URL or the unique ID can be used to find the cache entry.

Prototype

```
Err INetLibCacheGetObject (UInt16 libRefnum,
MemHandle clientParamH, UInt8 *urlTextP,
UInt32 uniqueID, INetCacheInfoPtr cacheInfoP)
```

Parameters

`-> libRefnum` Refnum of the Internet library.

`-> clientParamH` Inet handle allocated by `INetLibOpen`.

`-> urlTextP` Pointer to URL text string to find. If this parameter is `NULL`, then `uniqueID` is used to find the entry.

`-> uniqueID` Unique ID of the cache entry to find. This value can be obtained by calling `INetLibCacheList`. This parameter is ignored if `urlTextP` is specified.

`<- INetCacheInfoPtr` Pointer to a structure where information about the cache entry is returned. See the Comments section for details.

Result

0	No error
`inetErrParamsInvalid`	One or more of the parameters are invalid.

Comments

The `INetCacheInfoPtr` type returned from this function is defined as a pointer to an `INetCacheInfoType` structure, which has the following definition:

```
typedef struct {
MemHandle recordH; // handle to the cache
record
INetContentTypeEnum contentType;
INetCompressionTypeEnum encodingType;
UInt32 uncompressedDataSize;
UInt8 flags; // unused
UInt8 reserved;
UInt16 dataOffset; // offset to content
UInt16 dataLength; // size of content
```

```
UInt16 urlOffset; // offset to URL
UInt32 viewTime; // time last viewed
UInt32 createTime; // time entry was created
UInt16 murlOffset; // offset to master URL
} INetCacheInfoType, *INetCacheInfoPtr;
```

Compatibility Implemented only if Wireless Internet Feature Set is present.

INetLibCacheList

Purpose Returns an item from the cache list, based on its URL and index in the list.

Prototype

```
Err INetLibCacheList (UInt16 libRefnum,
MemHandle inetH, UInt8 *cacheIndexURLP,
UInt16 *indexP, UInt32 *uidP,
INetCacheEntryP cacheP)
```

Parameters

`-> libRefnum` Refnum of the Internet library.

`-> inetH` Inet handle allocated by `INetLibOpen`.

`-> cacheIndexURLP` Pointer to a master URL string. Cache entries indexed with this master URL are returned. The Web Clipping Application Viewer sets the master URL of a cache page to the URL of the active PQA, so all pages linked from the PQA have the same master URL.

`<-> indexP` Pointer to the index of the entry. Specify an index to find entries at this index or higher in the list. Specify `NULL` to search from the beginning, the first time you call this function. The index of the entry following the one found is returned on exit.

`<- uidP` Pointer to a long value where the unique ID of the found cache entry is returned.

`<- cacheP` Pointer to a structure where information about the found cache entry is returned. See the Comments section for details.

Result

0	No error
`inetErrTypeNotCached`	Cache entry under requested index not found
`inetErrParamsInvalid`	The `cacheIndexURLP` parameter is `NULL`
`inetErrCacheInvalid`	The cache database doesn't exist

Comments

This function first sorts the list of cache entries by URL. Then it returns in `uidP` the unique ID of the first cache entry with an index equal to or greater than `indexP`. The `indexP` value is updated to point to the next entry upon return.

To generate a complete list of cache entries having the same master URL (as for a history list), call this function repeatedly, always specifying the updated `index`, until it returns the error `inetErrTypeNotCached`.

Note that a URL can exist multiple times in the Web Clipping Application Viewer cache database, thus the need for the `uidP` value.

The `INetCacheEntryP` type returned from this function is defined as a pointer to an `INetCacheEntryType` structure, which has the following definition:

```
typedef struct {
UInt8 *urlP; // ptr to URL string
UInt16 urlLen; // length of URL string
UInt8 *titleP; // ptr to title string
UInt16 titleLen; // length of title string
UInt32 lastViewed; // time last viewed
                   // seconds since 1/1/1904
UInt32 firstViewed; // time first viewed
                   // seconds since 1/1/1904
} INetCacheEntryType, *INetCacheEntryP;
```

Compatibility

Implemented only if Wireless Internet Feature Set is present.

INetLibCheckAntennaState

Purpose Checks the antenna state and displays a dialog asking the user to raise it if it is down.

Prototype
```
Err INetLibCheckAntennaState(UInt16 refNum)
```

Parameters

`-> refNum`	Refnum of the Internet library.

Result

0	The user raised the antenna.
`netErrUserCancel`	The user closed the dialog by tapping Cancel.

This call can also return data manager errors if it fails internally.

Comments Applications don't need to check the antenna state by using this call. If an application opens the Internet library, the Internet library checks the antenna state when needed and displays the dialog to prompt the user to raise the antenna.

Compatibility Implemented only if Wireless Internet Feature Set is present.

INetLibClose

Purpose Closes up and frees an inet handle. Closes or decrements the open count of the net library.

Prototype
```
Err INetLibClose (UInt16 libRefnum,
MemHandle inetH)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> inetH`	Inet handle allocated by `INetLibOpen`.

Result

0	No error

Comments This call must be made by an application when it's done with the Internet library. It closes any Internet sockets open by the application, disposes the memory referenced by the given inet handle, and calls `NetLibClose`, if necessary, to close the net Library or decrement its open count.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibOpen`

INetLibConfigAliasGet

Purpose Determines to which configuration a built-in alias points.

Prototype

```
Err INetLibConfigAliasGet (UInt16 refNum,
UInt16 aliasIndex, UInt16 *indexP,
Boolean *isAnotherAliasP)
```

Parameters

`-> libRefnum` Refnum of the Internet library.

`-> aliasIndex` Index of alias configuration to query. This is the index of the configuration in the internal array of configurations stored by the system. This is the same as the index of the item in the array returned by `INetLibConfigList`, or the index returned by `INetLibConfigIndexFromName`.

`<- indexP` Pointer where the index of the configuration pointed to by `aliasIndex` is returned. 0 is returned if `aliasIndex` does not point to another configuration.

`<- isAnotherAliasP` If `*indexP` is the index of another alias configuration, this Boolean is set to `true`.

Result

0	No error
`inetErrParamsInvalid`	`aliasIndex` is not valid
`inetErrConfigNotAlias`	`aliasIndex` is not an alias configuration

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibConfigAliasSet`

INetLibConfigAliasSet

Purpose Points any of the built-in aliases (".DefWireline", ".DefWireless", etc.) to a given defined configuration.

Prototype

```
Err INetLibConfigAliasSet (UInt16 refNum,
UInt16 configIndex, UInt16 aliasToIndex)
```

Parameters

`-> libRefnum` Refnum of the Internet library.

`-> configIndex` Index of configuration to set. This is the index of the configuration in the internal array of configurations stored by the system. This is the same as the index of the item in the array returned by `INetLibConfigList`, or the index returned by `INetLibConfigIndexFromName`.

`-> aliasToIndex` Index of configuration that the alias identified by `configIndex` is to point to. Specify 0 to remove an existing alias assignment.

Result

0	No error
`inetErrConfigNotAlias`	`configIndex` is not an alias configuration

`inetErrParamsInvalid`	`configIndex` or `aliasToIndex` is not valid
`inetErrConfigCantPointToAlias`	Alias doesn't point to a real entry

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibConfigAliasGet`

INetLibConfigDelete

Purpose Deletes a configuration.

Prototype

```
Err INetLibConfigDelete (UInt16 refNum,
UInt16 index)
```

Parameters

`-> refnum`	Refnum of the Internet library.
`-> index`	Index of configuration to delete. This is the index of the configuration in the internal array of configurations stored by the system. This is the same as the index of the item in the array returned by `INetLibConfigList`, or the index returned by `INetLibConfigIndexFromName`.

Result

0	No error
`inetErrParamsInvalid`	Index not valid
`inetErrConfigCantDelete`	Attempted to delete an alias configuration

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibConfigIndexFromName`, `INetLibConfigList`

INetLibConfigIndexFromName

Purpose Returns the index of a configuration given it's name.

Prototype

```
Err INetLibConfigIndexFromName (UInt16 refNum,
INetConfigNamePtr nameP, UInt16 *indexP)
```

Parameters

`-> refnum`	Refnum of the Internet library.
`-> nameP`	Pointer to an `INetConfigNameType` structure that names the configuration whose index you want to get.
`<- indexP`	Pointer where the index of the configuration identified in `nameP` is returned.

Result

0	No error
`inetErrConfigNotFound`	Could not find requested configuration name

Comments If you name an alias, this routine returns the index of the alias entry, not the configuration the alias points to. This way the alias can be pointed to a different configuration.

Applications should store the index of the configuration they're using, rather than the name, so that they won't be confused if the user edits the name of the configuration from the Preferences panel.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibConfigList`

INetLibConfigList

Purpose

Returns an array containing a list of the available Internet library network configurations.

Prototype

```
Err INetLibConfigList (UInt16 refNum,
INetConfigNameType nameArray[],
UInt16 *arrayEntriesP)
```

Parameters

`-> refnum` Refnum of the Internet library.

`-> nameArray` Pointer to an array of `INetConfigNameType` structs that is to be filled in by this routine.

`<-> arrayEntriesP` On entry, a pointer to the number of entries available in `nameArray`; on exit, a pointer to the total number of entries in the system (which could exceed the size of the array you pass in).

Result

0	No error

Comments

This routine can be used to obtain a list of available configurations for selection by the user.

Note that the built-in alias configurations, which start with a period, should not be displayed to the user as selectable choices. They are designed for internal use by applications that need a predetermined type of service (like ".CTPWireless" for PQA applications.) Also, any configurations that start with an underscore, like "_CTPRAM", should not be displayed. These typically are configurations created by the Internet library for internal use and cannot be edited using the Network preferences panel.

Compatibility

Implemented only if Wireless Internet Feature Set is present.

See Also

`INetLibConfigMakeActive`

INetLibConfigMakeActive

Purpose Makes the given configuration active without having to close and reopen the Internet library by using `INetLibOpen`.

Prototype

```
Err INetLibConfigMakeActive (UInt16 refNum,
MemHandle inetH, UInt16 configIndex)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> inetH`	Inet handle allocated by `INetLibOpen`.
`-> configIndex`	Index of configuration to activate. This is the index of the configuration in the internal array of configurations stored by the system. This is the same as the index of the item in the array returned by `INetLibConfigList`, or the index returned by `INetLibConfigIndexFromName`.

Result

0	No error

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibConfigSaveAs`, `INetLibConfigList`, `INetLibConfigIndexFromName`

INetLibConfigRename

Purpose Renames a configuration.

Prototype

```
Err INetLibConfigRename (UInt16 refNum,
UInt16 index, INetConfigNamePtr newNameP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.

`-> index`	Index of configuration to rename. This is the index of the configuration in the internal array of configurations stored by the system. This is the same as the index of the item in the array returned by `INetLibConfigList`, or the index returned by `INetLibConfigIndexFromName`.
`-> newNameP`	Pointer to an `INetConfigNameType` structure holding the new name of the configuration. The name cannot start with a period or an underscore.

Result

0	No error
`inetErrConfigBadName`	Trying to save as an alias (beginning with ".") or as a built-in configuration (beginning with "_").
`inetErrParamsInvalid`	Invalid index
`inetErrConfigCantDelete`	Configuration to be renamed is either an alias or a built-in configuration

Comments After renaming, the configuration index stays the same so that applications that are set up to use that configuration will still work correctly. Note that built-in configuration aliases (ones that start with a period) cannot be renamed.

Compatibility Implemented only if Wireless Internet Feature Set is present.

INetLibConfigSaveAs

Purpose Saves the current network configuration settings under the given name.

Prototype
```
Err INetLibConfigSaveAs (UInt16 refNum,
MemHandle inetH, INetConfigNamePtr nameP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> inetH`	Inet handle allocated by `INetLibOpen`.
`-> nameP`	Pointer to an `INetConfigNameType` structure holding the name of the configuration. The name cannot start with a period or an underscore.

Result

0	No error
`inetErrConfigBadName`	Trying to save as an alias (beginning with ".") or as a built-in configuration (beginning with "_").
`inetErrConfigTooMany`	The internal configurations table is full. No more entries can be stored.

Comments If the configuration name specified already exists, it is replaced with the new settings.

Compatibility Implemented only if Wireless Internet Feature Set is present.

INetLibGetEvent

Purpose A replacement for `EvtGetEvent` that informs an application of status changes to Internet sockets as well as user interface events.

Prototype
```
void INetLibGetEvent (UInt16  libRefnum,
MemHandle inetH, INetEventType *eventP,
Int32 timeout)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> inetH`	Inet handle allocated by `INetLibOpen`, or `NULL`.
`<-> eventP`	The event structure is returned via this pointer.
`-> timeout`	Timeout in ticks. Specify `evtWaitForever` to wait forever.

Result

0	No error

Comments This call is designed to replace `EvtGetEvent` in applications which use the Internet library. For convenience, if `inetH` is `NULL`, `INetLibGetEvent` is equivalent to `EvtGetEvent`.

`INetLibGetEvent` returns two additional events besides those returned by `EvtGetEvent`: `inetSockReadyEvent` and `inetSockStatusChangeEvent`.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibSockStatus`, `INetLibURLOpen`, `INetLibSockOpen`, `INetLibSockRead`

INetLibOpen

Purpose Creates a new application inet handle structure. Opens or increments the open count of the net library.

Prototype

```
Err INetLibOpen (UInt16 libRefnum, UInt16 config,
UInt32 flags, DmOpenRef cacheRef,
UInt32 cacheSize, MemHandle *inetHP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library. Pass the value "INet.lib" to `SysLibFind` to return this refnum.

`-> config`	Indicates the type of network service desired by the application. Returned by `INetLibConfigIndexFromName`.
`-> flags`	Currently unused; set to 0.
`-> cacheRef`	Document cache database reference. Obtain this by using one of the `DmOpenDatabase`... calls. Pass `NULL` if you don't want to use a cache.
`-> cacheSize`	Maximum size of the document cache (in bytes). This is ignored if you pass `NULL` for `cacheRef`.
`<- inetHP`	Pointer to a handle variable.

Result

0	No error
`inetErrTooManyClients`	Too many clients opened already
`inetErrIncompatibleInterface`	The net library is already open with an incompatible interface

Comments This call must be made by an application before it can use any other Internet library calls. This call opens the Internet library and returns a pointer to an inet handle, which is then passed to subsequent calls to the Internet library. Every application that opens the Internet library gets its own unique inet handle.

When an application is done using the Internet library, it must call `INetLibClose`, which closes both the Internet library and the net library, if necessary.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibClose`, `INetLibConfigIndexFromName`

INetLibSettingGet

Purpose Retrieves current settings for an inet handle.

Prototype

```
Err INetLibSettingGet (UInt16 libRefnum,
MemHandle inetH, UInt16 setting, void *bufP,
UInt16 *bufLenP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> inetH`	Inet handle allocated by `INetLibOpen`.
`-> setting`	The setting to get. Specify one of the `INetSettingEnum` enumerated types.
`<- bufP`	Pointer to buffer where the return value is to be put.
`<-> bufLenP`	Size of `bufP` on entry. Size of setting value on exit.

Result

0	No error
`inetErrParamsInvalid`	Invalid setting requested
`inetErrSettingSizeInvalid`	`*bufLenP` is the incorrect size for the requested setting

Comments This call can be used to retrieve the current settings of an inet handle. Some settings have default values that are stored in the system preferences database; see `INetSettingEnum` for details.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibOpen`, `INetLibSettingSet`, `INetSettingEnum`

INetLibSettingSet

Purpose Changes a setting for an inet handle.

Prototype

```
Err INetLibSettingSet (UInt16 libRefnum,
MemHandle inetH, UInt16 setting, void *bufP,
UInt16 bufLen)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> inetH`	Inet handle allocated by `INetLibOpen`.
`-> setting`	The setting to set. Specify one of the `INetSettingEnum` enumerated types.
`-> bufP`	Pointer to the new setting value.
`-> bufLen`	Size of the value in `bufP`.

Result

0	No error
`inetErrParamsInvalid`	Invalid setting specified
`inetErrSettingSizeInvalid`	`bufLen` is the incorrect size for the specified setting

Comments Any changes made to the settings last only as long as the `inetH` is around (until `INetLibClose` is called) and do not affect other applications that might be using the Internet library.

An important note is that settings made through this call essentially change the default values for any sockets subsequently created through `INetLibURLOpen` or `INetLibSockOpen`.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibSettingGet`, `INetSettingEnum`

INetLibSockClose

Purpose Closes an inet socket handle.

Prototype
```
Err INetLibSockClose (UInt16 libRefnum,
MemHandle socketH)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> socketH`	Handle of the socket to close.

Result

0	No error

Comments This call closes down and releases all memory associated with a socket created by `INetLibSockOpen` or `INetLibURLOpen`.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibOpen`, `INetLibSockOpen`, `INetLibURLOpen`

INetLibSockConnect

Purpose Establishes a connection with a remote host.

Prototype
```
Err INetLibSockConnect (UInt16 libRefnum,
MemHandle sockH, UInt8 *hostnameP, UInt16 port,
Int32 timeou)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> sockH`	Handle (allocated by `INetLibSockOpen` or `INetLibURLOpen`) of the socket to connect.
`-> hostnameP`	Pointer to host name string; can be dotted decimal text string.
`-> port`	Port number, or 0 for default port.
`-> timeou`	Timeout in ticks. Specify `evtWaitForever` to wait forever.

Result

0	No error

Comments This call associates a remote host name and port number with a socket and, depending on the socket protocol, initiates a connection with that remote host.

This call may return immediately before actually finishing the connect. The application can simply go ahead and submit additional calls such as `INetLibSockRead`, or it may wait for the connect to complete by either polling `INetLibSockStatus` until the socket status is `inetStatusConnected` (not recommended), or by waiting for an `inetSockStatusChangeEvent` event from `INetLibGetEvent` and checking the status then (preferred).

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibSockOpen`, `INetLibSockStatus`, `INetLibGetEvent`

INetLibSockHTTPAttrGet

Purpose Queries HTTP request header formed by the local host, or the response header information returned by a remote host.

Prototype

```
Err INetLibSockHTTPAttrGet (UInt16 libRefnum,
MemHandle sockH, UInt16 attr, UInt16 attrIndex,
void *bufP, UInt32 *bufLenP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> sockH`	Handle (allocated by `INetLibSockOpen` or `INetLibURLOpen`) of the socket.
`-> attr`	The attribute to get. Specify one of the `INetHTTPAttrEnum` values.
`-> attrIndex`	The attribute index (if any). Currently unused.
`<- bufP`	Pointer to the address where the result is returned.
`<-> bufLenP`	Pointer to the size of `bufP` on entry; size of returned value on exit.

Result

0	No error
`inetErrSettingNotImplemented`	Invalid setting specified
`inetErrSettingSizeInvalid`	`bufLen` is the incorrect size for the specified setting

Comments This call queries either the request header formed by `INetLibSockHTTPReqCreate` and `INetLibSockHTTPAttrSet`, or the response header returned by the remote host.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibSockHTTPReqCreate`

INetLibSockHTTPAttrSet

Purpose Adds additional HTTP request headers to an HTTP request in a socket.

Prototype

```
Err INetLibSockHTTPAttrSet (UInt16 libRefnum,
MemHandle sockH, UInt16 attr, UInt16 attrIndex,
UInt8 *bufP, UInt16 bufLen, UInt16 flags)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> sockH`	Handle (allocated by `INetLibSockOpen` or `INetLibURLOpen`) of the socket.
`-> attr`	The attribute to set. Specify one of the `INetHTTPAttrEnum` values.
`-> attrIndex`	The attribute index (if any). Currently unused.
`-> bufP`	Pointer to additional header text to add.
`-> bufLen`	Length of `bufP`.
`-> flags`	Flags that control the addition of new headers. Currently unused.

Result

0	No error
`inetErrSettingNotImplemented`	Invalid setting specified
`inetErrSettingSizeInvalid`	`bufLen` is the incorrect size for the specified setting

Comments This call modifies attributes of an HTTP request formed by `INetLibSockHTTPReqCreate`. Generally, attributes are set only before calling `INetLibSockHTTPReqSend`.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibSockHTTPReqCreate`, `INetLibSockHTTPReqSend`

INetLibSockHTTPReqCreate

Purpose Forms an HTTP request for the socket.

Prototype

```
Err INetLibSockHTTPReqCreate (UInt16 libRefnum,
MemHandle sockH, UInt8 *verbP, UInt8 *resNameP,
UInt8 *refererP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> sockH`	Handle (allocated by `INetLibSockOpen` or `INetLibURLOpen`) of the socket.
`-> verbP`	Reserved for future use.
`-> resNameP`	Pointer to a string holding the name of the resource to get or put.
`-> refererP`	Pointer to a string holding the name of the referring URL, or `NULL` if none.

Result

0	No error
`inetErrParamsInvalid`	Not an HTTP socket

Comments This call forms an HTTP request for the socket. The request is not actually sent to the remote host until `INetLibSockHTTPReqSend` is called. The HTTP verb used in the request is determined by the value of the `writeP` parameter passed to `INetLibSockHTTPReqSend`: if this parameter is `NULL`, "GET" is used. Otherwise, "POST" is used.

After a call to `INetLibSockHTTPReqCreate` but before a call to `INetLibSockHTTPReqSend`, the application can add additional HTTP request headers using `INetLibSockHTTPAttrSet`.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibSockHTTPAttrSet`, `INetLibSockHTTPReqSend`

INetLibSockHTTPReqSend

Purpose Sends an HTTP request to the remote host or looks for data in the cache.

Prototype

```
Err INetLibSockHTTPReqSend (UInt16 libRefnum,
MemHandle sockH, void *writeP, UInt32 writeLen,
Int32 timeout)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> sockH`	Handle (allocated by `INetLibSockOpen` or `INetLibURLOpen`) of the socket.
`-> writeP`	Pointer to additional data to send after the request headers. Usually used for `POST` and `PUT` operations.
`-> writeLen`	Number of bytes in `writeP`.
`-> timeout`	Timeout in ticks.

Result

0	No error
`inetErrRequestTooLong`	Request too big
`inetErrEncryptionNotAvail`	Encryption requested but not available

Comments

This call sends an HTTP request created by `INetLibSockHTTPReqCreate` and `INetLibSockHTTPAttrSet` to the remote host. If this is an `POST` or `PUT` operation, the data to write can be specified in `writeP`.

`INetLibSockHTTPReqSend` doesn't always do network I/O. If the proper socket flag is set, it checks first to see if the requested data is already in the cache. If it is, then a pointer to the cached data is stored in the socket and the socket status is updated to show that data is ready to be read. This will trigger an `inetSockReadyEvent` event.

The socket flag (`inetOpenURLFlagLookInCache`) that causes the cache to be checked first can be set via the `flags` parameter to `INetLibURLOpen` or by calling `INetLibSockSettingSet` with the appropriate setting (`inetSockSettingFlags`).

After sending the request, the application can wait for the response to arrive by either polling `INetLibSockStatus` until the `inputReady` boolean is set (not recommended), or by waiting for an `inetSockReadyEvent` event from `INetLibGetEvent` (preferred).

Compatibility

Implemented only if Wireless Internet Feature Set is present.

See Also

`INetLibSockHTTPReqCreate`, `INetLibSockHTTPAttrSet`, `INetLibGetEvent`

INetLibSockOpen

Purpose Creates and returns a new inet socket handle.

Prototype

```
Err INetLibSockOpen (UInt16 libRefnum,
MemHandle inetH, UInt16 scheme, MemHandle *sockHP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> inetH`	Inet handle allocated by `INetLibOpen`.
`-> scheme`	The protocol scheme to use. Specify one of the `INetSchemeEnum` types.
`<- sockHP`	Pointer to the address where the socket handle is returned.

Result

0	No error
`inetErrTooManySockets`	Too many sockets open
`inetErrSchemeNotSupported`	Requested URL scheme not supported

Comments This call creates a new socket for the given scheme. No network I/O is performed. This is a relatively low level call that can be used in place of `INetLibURLOpen` when finer control over the socket settings is required.

Using `INetLibURLOpen`, an HTTP request can be handled with the simple sequence: `INetLibURLOpen`, `INetLibSockRead`, and `INetLibSockClose`. When using `INetLibSockOpen`, the same HTTP request would be handled by replacing the `INetLibURLOpen` call with the sequence: `INetLibSockOpen`, `INetLibSockSettingSet` (optional), `INetLibSockConnect`, `INetLibSockHTTPReqCreate`, `INetLibSockHTTPAttrSet` (optional), and `INetLibSockHTTPReqSend`.

The use of `INetLibSockOpen` allows an application finer control over the socket settings (by calling `INetLibSockSettingSet`) and the HTTP request headers (by calling `INetLibSockHTTPAttrSet`).

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also INetLibOpen, INetLibURLOpen, INetLibSockRead, INetLibSockClose, INetLibSockSettingSet, INetLibSockHTTPAttrSet

INetLibSockRead

Purpose Reads data from a socket.

Prototype

```
Err INetLibSockRead (UInt16 libRefnum,
MemHandle sockH, void *bufP, UInt32 reqBytes,
UInt32 *actBytesP, Int32 timeout)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> sockH`	Inet handle allocated by `INetLibOpen`.
`-> bufP`	Pointer to buffer where the data is placed.
`-> reqBytes`	Requested number of bytes.
`<- actBytesP`	Pointer to the actual number of bytes read.
`-> timeout`	Timeout in ticks. Specify `evtWaitForever` to wait forever.

Result

0	No error

Comments This call attempts to read `reqBytes` bytes from the given socket. It returns the actual number of bytes read in `*actBytesP`. If the connection with the remote host has been closed, `*actBytesP` contains 0 on exit.

Note that it is normal for the actual bytes read to be less than the requested number of bytes. The application should be prepared to call this routine repeatedly until the desired number of bytes have been read or until `*actBytesP` contains 0, indicating the connection has been closed, or until an error is returned.

This call returns as much data as possible without blocking, however, if no data is available to be read, it does block until at least one byte is available.

Normally, applications will wait for an `inetSockReadyEvent` from `INetLibGetEvent` before calling `INetLibSockRead`. Alternatively, the application could call `INetLibSockStatus` to determine if the socket has any data ready (not recommended), or could simply rely on `INetLibSockRead` to block until at least one byte is available to read. If no data is available before the timeout expires, `inetErrReadTimeout` error is returned.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibURLOpen`, `INetLibSockOpen`, `INetLibSockStatus`, `INetLibSockClose`, `INetLibGetEvent`

INetLibSockSettingGet

Purpose Retrieves a socket setting.

Prototype

```
Err INetLibSockSettingGet (UInt16 libRefnum,
MemHandle socketH, UInt16 setting, void *bufP,
UInt16 *bufLenP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> socketH`	Handle (allocated by `INetLibSockOpen` or `INetLibURLOpen`) of the socket to get a setting from.
`-> setting`	The setting to get. Specify one of the `INetSockSettingEnum` values.
`<- bufP`	Pointer to buffer where the setting value is to be placed.
`<-> bufLenP`	Size of `bufP` on entry; size of returned value on exit.

Result

0	No error
`inetErrParamsInvalid`	Invalid setting requested
`inetErrSettingSizeInvalid`	`*bufLenP` is the incorrect size for the requested setting

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also `INetLibSockSettingSet`

INetLibSockSettingSet

Purpose Changes a setting of a socket.

Prototype

```
Err INetLibSockSettingSet (UInt16 libRefnum,
MemHandle socketH, UInt16 setting, void *bufP,
UInt16 bufLen)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> socketH`	Handle (allocated by `INetLibSockOpen` or `INetLibURLOpen`) of the socket to set.
`-> setting`	The setting to set. Specify one of the `INetSockSettingEnum` values.
`-> bufP`	Pointer to buffer containing the new setting value.
`-> bufLen`	Size of new setting in `bufP`.

Result

0	No error
`inetErrSettingNotImplemented`	Invalid setting specified
`inetErrSettingSizeInvalid`	`bufLen` is the incorrect size for the setting

Comments This call can be use to override a general setting for a particular socket.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also INetLibSockSettingGet

INetLibSockStatus

Purpose Retrieves the current status of a socket.

Prototype

```
Err INetLibSockStatus (UInt16 libRefnum,
MemHandle socketH, UInt16 *statusP, Err *sockErrP,
Boolean *inputReadyP, Boolean *outputReadyP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> socketH`	Handle (allocated by `INetLibSockOpen` or `INetLibURLOpen`) of the socket to get status on.
`<- statusP`	Pointer to the address where the status is returned. The status will be one of the `INetStatusEnum` values.
`<- sockErrP`	Currently unused.
`<- inputReadyP`	Pointer to a Boolean; `true` is returned if the socket has data available to read.
`<- outputReadyP`	Pointer to a Boolean; `true` is returned if the socket can accept data for writing.

Result

0	No error

Comments Most applications that use `INetLibGetEvent` will rarely need to use this call since socket status changes are returned in the event structure.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also INetLibURLOpen, INetLibSockOpen, INetLibSockRead, INetLibGetEvent

INetLibURLCrack

Purpose Cracks a URL text string into its components.

Prototype

```
Err INetLibURLCrack (UInt16 libRefnum,
UInt8 *urlTextP, INetURLType *urlP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> urlTextP`	Pointer to URL text string.
`<-> urlP`	Pointer to address where the URL information block is to be returned.

Result

0	No error
`inetErrParamsInvalid`	`urlTextP` is `NULL` or empty, or `urlP` is `NULL`
`inetErrURLVersionInvalid`	`urlP` is wrong version

Comments If a pointer member of `urlP` is set to `NULL` on entry, then on exit it will point to the start of that component within the original `urlTextP` string; the associated member length is set to the length of that URL component. If a pointer member of `urlP` is not `NULL` on entry, then it must point to a buffer of sufficient size to hold the member data, and on exit the component string will be copied into this buffer and the associated member length will be updated with the actual size. Note that the returned strings are not null-terminated, so the length values are important.

It's easiest to initialize the `InetURLType` block to zeros and let this function fill in all the information about the URL components.

The `InetURLType` block returned from this function has the following structure:

```
typedef struct {
UInt16 version; // 0, for future compatibility
UInt8 *schemeP; // ptr to scheme portion
UInt16 schemeLen; // size of scheme portion
UInt16 schemeEnum; // INetSchemeEnum; the
scheme
UInt8 *usernameP; // ptr to username portion
UInt16 usernameLen; // size of username
UInt8 *passwordP; // ptr to password portion
UInt16 passwordLen; // size of password
UInt8 *hostnameP; // ptr to host name portion
UInt16 hostnameLen; // size of host name
UInt16 port; // port number
UInt8 *pathP; // ptr to path portion
UInt16 pathLen; // size of path
UInt8 *paramP; // ptr to param (;param)
UInt16 paramLen; // size of param
UInt8 *queryP; // ptr to query (?query)
UInt16 queryLen; // size of query
UInt8 *fragP; // ptr to fragment (#frag)
UInt16 fragLen; // size of fragment
} INetURLType
```

Compatibility Implemented only if Wireless Internet Feature Set is present.

INetLibURLGetInfo

Purpose Returns information about a URL.

Prototype

```
Err INetLibURLGetInfo (UInt16 libRefnum,
MemHandle inetH, UInt8 *urlTextP,
INetURLInfoType *urlInfoP)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> inetH`	Inet handle allocated by `INetLibOpen`.

`-> urlTextP`	Pointer to URL text string.
`<-> urlInfoP`	Pointer to address where the URL information structure is to be returned.

Result

0	No error
`inetErrParamsInvalid`	`urlInfoP` is `NULL` or incorrect version

Comments The `InetURLInfo` block returned from this function has the following structure:

```
typedef struct {
UInt16 version; // 0, for future compatibility
UInt16 flags; // flags word
UInt32 undefined; // reserved for future use
} INetURLInfo
```

The `flags` word can consist of some combination of these values:

```
inetURLInfoFlagIsSecure // URL was encrypted
inetURLInfoFlagIsRemote // URL was retrieved
from the net
inetURLInfoFlagIsInCache // URL is stored in
the cache
```

Compatibility Implemented only if Wireless Internet Feature Set is present.

INetLibURLOpen

Purpose Accesses a URL on the Internet or in the cache.

Prototype

```
Err INetLibURLOpen (UInt16 libRefnum,
MemHandle inetH, UInt8 *urlP,
UInt8 *cacheIndexURLP, MemHandle *sockHP,
Int32 timeout, UInt16 flags)
```

Parameters

`-> libRefnum`	Refnum of the Internet library.
`-> inetH`	Inet handle allocated by `INetLibOpen`.

`-> urlP`	Pointer to string containing the URL to access.
`-> cacheIndexURLP`	Pointer to URL string under which the requested URL should be indexed in the cache. Specify `NULL` if you don't need to index the cache. If you are using the Web Clipping Application Viewer cache (not recommended), you must follow the Viewer convention, which is to pass the URL of the active PQA.
`<- sockHP`	Pointer to address where the socket handle is returned.
`-> timeout`	Timeout in ticks. Specify `evtWaitForever` to wait forever.
`-> flags`	Flags indicating caching and encryption options desired. Specify zero, one, or more of the URL open flags (see URL Open Constants).

Result

0	No error
`inetErrParamsInvalid`	`urlP` is `NULL`

Comments

This call sets up a connection to a resource on the Internet addressed by `urlP` and returns a socket handle. Note that if you specify that the cache should be searched first, and if the data is found in the cache, no network I/O occurs. The application can then read that socket resource through the `INetLibSockRead` call.

This call is a convenience routine that internally makes the following calls for http URLs: `INetLibSockOpen`, `INetLibSockConnect`, `INetLibSockHTTPReqCreate`, and `INetLibSockHTTPReqSend`.

This routine returns immediately before performing any required network I/O. It is then up to the caller to either block on `INetLibSockRead`, or to use `INetLibGetEvent` to model asynchronous operation. Using `INetLibGetEvent` is the preferred way of performing network I/O since it maximizes battery life and user-interface responsiveness.

Compatibility Implemented only if Wireless Internet Feature Set is present.

See Also INetLibSockOpen, INetLibSockConnect, INetLibSockRead, INetLibSockClose

INetLibURLsAdd

Purpose Concatenates two URLs, resulting in one absolute URL.

Prototype

```
Err INetLibURLsAdd (UInt16 libRefnum,
Char *baseURLStr, Char *embeddedURLStr,
Char *resultURLStr, UInt16 *resultLenP)
```

Parameters

`-> libRefnum` Refnum of the Internet library.

`-> baseURLStr` Pointer to base URL string.

`-> embeddedURLStr` Pointer to URL text string to append.

`<-> resultURLStr` Pointer to resulting URL string.

`<-> resultLenP` Pointer to size of `resultURLStr` buffer on entry. On exit, pointer to resulting URL length (including null terminator).

Result

0	No error

Comments Used to append a URL fragment to a base URL, resulting in an absolute URL string that can be passed to INetLibURLOpen or other functions. This routine ensures that the resulting string conforms to the URL format.

Compatibility Implemented only if Wireless Internet Feature Set is present.

INetLibWiCmd

Purpose Invokes a command that operates on the wireless indicator.

Prototype

```
Boolean INetLibWiCmd (UInt16 refNum, UInt16 cmd,
int enableOrX, int y)
```

Parameters

`-> refNum`	Refnum of the Internet library.
`-> cmd`	The command to invoke. Specify one of the `WiCmdEnum` values (see Comments section).
`-> enableOrX`	If `cmd` is `wiCmdSetEnabled`, specify 1 to enable the wireless indicator or 0 to disable it. If `cmd` is `wiCmdSetLocation`, this specifies the x coordinate of the location.
`-> y`	The y coordinate of the location. Used only if `cmd` is `wiCmdSetLocation`.

Result If `cmd` is `wiCmdEnabled`, this function returns `true` if the wireless indicator is enabled or `false` if it is not. For other command types, the return value is undefined.

Comments The wireless indicator is a 19x13 pixel image on the screen to indicate the current wireless signal strength. This shows as 0 - 5 bars. If the application is in a non-modal window with a title bar, the preferred location for the indicator is at (140,1).

It automatically updates itself as long as you are calling `INetLibGetEvent`. It should be shown on screen while a wireless transaction is in progress. It may also be shown when the user has nothing useful to do next but initiate a wireless transaction, and there isn't much other useful information being displayed.

The `WiCmdEnum` enum specifies a command that operates on the wireless indicator in the user interface. The definition of this type is found in `WirelessIndicator.h` and is as follows:

```
typedef enum {
wiCmdInit =0,
wiCmdClear,
wiCmdSetEnabled,
```

```
wiCmdDraw,
wiCmdEnabled,
wiCmdSetLocation,
wiCmdErase
} WiCmdEnum;
```

Value Descriptions

`wiCmdInit`	Initializes the wireless indicator. You must invoke this command first, before using any of the others.
`wiCmdClear`	Applications shouldn't use this command. To erase the indicator, disable it by using `wiCmdSetEnabled` and passing 0 for `enableOrX`.
`wiCmdSetEnabled`	Enables or disables the wireless indicator.
`wiCmdDraw`	Redraws the wireless indicator using the latest data. Applications don't need to use this command since the indicator is redrawn automatically by `INetLibGetEvent`.
`wiCmdEnabled`	Returns a Boolean indicating if the wireless indicator is enabled.
`wiCmdSetLocation`	Sets the location for the wireless indicator on the screen.
`wiCmdErase`	Erases the wireless indicator. Applications shouldn't use this command. To erase the indicator, disable it by using `wiCmdSetEnabled` and passing 0 for `enableOrX`.

Compatibility Implemented only if Wireless Internet Feature Set is present.

76

PalmOSGlue Library

This chapter describes the API provided in the link library PalmOSGlue (`PalmOSGlue.lib` or `libPalmOSGlue.a`).

You use PalmOSGlue if you want to use the international and text manager features described in the chapter "Localized Applications" on page 313 in the *Palm OS Programmer's Companion*, vol. I and you want to maintain backward compatibility with earlier releases of Palm OS®. If you link with PalmOSGlue, include the headers in the folder `Incs\Libraries\PalmOSGlue`, and make calls as they are listed in this chapter, then your code will run regardless of whether the device's version of the operating system implements international support. The code in PalmOSGlue either uses the text manager or international manager on the ROM or, if the managers don't exist, executes a simple Latin equivalent of the function.

NOTE: PalmOSGlue is a link library, not a shared library. Linking with this library increases your application's code size. The amount by which your code size increases varies depending on the number of library functions you call. Use PalmOSGlue only on versions 2.0 and later of Palm OS.

In addition to covering the text and international manager API, PalmOSGlue adds some functions that are not included in any version of the Palm OS. This chapter describes the functions that are unique to PalmOSGlue and provides a mapping of PalmOSGlue calls to calls that exist in later versions of Palm OS.

PalmOSGlue Functions

The following table shows the mapping between the functions declared in the glue headers and the international functions and macros. To learn more about a glue function, click the link in the right column.

This table lists only those functions that map to a function that exists in newer versions of the OS. The functions that are exclusive to PalmOSGlue are not listed. They are described following this table.

Table 76.1 PalmOSGlue function mappings

This PalmOSGlue function...	...is identical to...
BmpGlueGetBitDepth	BmpGetBitDepth
BmpGlueGetDimensions	BmpGetDimensions
BmpGlueGetNextBitmap	BmpGetNextBitmap
DateGlueTemplateToAscii	DateTemplateToAscii
DateGlueToDOWDMFormat	DateToDOWDMFormat
FntGlueWCharWidth	FntWCharWidth
FntGlueWidthToOffset	FntWidthToOffset
FrmGlueGetActiveField	
LmGlueGetLocaleSetting	LmGetLocaleSetting
LmGlueGetNumLocales	LmGetNumLocales
LmGlueLocaleToIndex	LmLocaleToIndex
LstGlueGetTopItem	LstGetTopItem
OmGlueGetCurrentLocale	OmGetCurrentLocale
OmGlueGetSystemLocale	OmGetSystemLocale
ResGlueLoadConstant	ResLoadConstant
SysGlueGetTrapAddress	SysGetTrapAddress
TsmGlueGetFepMode	TsmGetFepMode

Table 76.1 PalmOSGlue function mappings *(continued)*

This PalmOSGlue function...	...is identical to...
TsmGlueSetFepMode	TsmSetFepMode
TblGlueGetNumberOf Columns	TblGetNumberOfColumns
TblGlueGetTopRow	TblGetTopRow
TblGlueSetSelection	TblSetSelection
TxtGlueByteAttr	TxtByteAttr
TxtGlueCaselessCompare	TxtCaselessCompare
TxtGlueCharAttr	TxtCharAttr
TxtGlueCharBounds	TxtCharBounds
TxtGlueCharEncoding	TxtCharEncoding
TxtGlueCharIsAlNum	TxtCharIsAlNum
TxtGlueCharIsAlpha	TxtCharIsAlpha
TxtGlueCharIsCntrl	TxtCharIsCntrl
TxtGlueCharIsDelim	TxtCharIsDelim
TxtGlueCharIsDigit	TxtCharIsDigit
TxtGlueCharIsGraph	TxtCharIsGraph
TxtGlueCharIsHex	TxtCharIsHex
TxtGlueCharIsLower	TxtCharIsLower
TxtGlueCharIsPrint	TxtCharIsPrint
TxtGlueCharIsPunct	TxtCharIsPunct
TxtGlueCharIsSpace	TxtCharIsSpace
TxtGlueCharIsUpper	TxtCharIsUpper
TxtGlueCharIsValid	TxtCharIsValid
TxtGlueCharSize	TxtCharSize

Table 76.1 PalmOSGlue function mappings *(continued)*

This PalmOSGlue function...	...is identical to...
TxtGlueCharWidth	FntWCharWidth
TxtGlueCharXAttr	TxtCharXAttr
TxtGlueCompare	TxtCompare
TxtGlueEncodingName	TxtEncodingName
TxtGlueFindString	TxtFindString
TxtGlueGetChar	TxtGetChar
TxtGlueGetNextChar	TxtGetNextChar
TxtGlueGetPreviousChar	TxtGetPreviousChar
TxtGlueGetTruncation Offset	TxtGetTruncationOffset
TxtGlueMaxEncoding	TxtMaxEncoding
TxtGlueNextCharSize	TxtNextCharSize
TxtGlueParamString	TxtParamString
TxtGluePreviousCharSize	TxtPreviousCharSize
TxtGlueReplaceStr	TxtReplaceStr
TxtGlueSetNextChar	TxtSetNextChar
TxtGlueStrEncoding	TxtStrEncoding
TxtGlueTransliterate	TxtTransliterate
TxtGlueWordBounds	TxtWordBounds
WinGlueDrawChar	WinDrawChar
WinGlueDrawTruncChars	WinDrawTruncChars



New CtlGlueGetControlStyle

Purpose Return the type of the control, such as button, slider, and so on.

Prototype
```
ControlStyleType CtlGlueGetControlStyle
(const ControlType *ctlP)
```

Parameters

-> `ctlP`	A pointer to a `ControlType` structure.

Result Returns one of the `ControlStyleType` constants.

Compatibility Implemented only in the PalmOSGlue library.



New FldGlueGetLineInfo

Purpose Retrieve the word-wrapping information for a visible line within a field.

Prototype
```
Boolean FldGlueGetLineInfo (const FieldType *fldP,
UInt16 lineNum, UInt16 *startP, UInt16 *lengthP)
```

Parameters

-> `fldP`	A pointer to a `FieldType` structure.
-> `lineNum`	The number of the visible line in the field about which you want to retrieve information. Lines are numbered starting at 0.
<- `startP`	The byte offset into the `FieldType`'s `text` field of the first character displayed by this line. If the line is blank, `start` is equal to the length of the field's text string.
<- `lengthP`	The length in bytes of the portion of the string displayed on this line. If the line is blank, the length is 0.

Result Returns `true` if `startP` and `endP` contain valid values, or `false` if the field is a single-line field or does not contain a line numbered `lineNum`.

Compatibility Implemented only in the PalmOSGlue library.

FntGlueGetDefaultFontID

Purpose Return the font ID of a default font.

Prototype

```
FontID FntGlueGetDefaultFontID (
FontDefaultType inFontType)
```

Parameters

-> `inFontType` A `FontDefaultType` constant specifying one of the system default fonts. This value can be one of the following:

`defaultSystemFont`
: The default font for the system.

`defaultLargeFont`
: The default large font.

`defaultSmallFont`
: The default small font.

`defaultBoldFont`
: The default bold font.

Result Returns the ID of `inFontType`.

Comments Use this function whenever you need to obtain a font ID for one of the system default fonts. The default fonts (and thus, the IDs for the default fonts) vary depending on the system's locale. For example, Japanese systems have a different set of default fonts than systems using the Latin character encoding.

Use this function in place of the constants that specify the IDs of default fonts, as shown in the following table.

In place of this...	...use FntGlueGetDefaultFontID with this constant...
`stdFont`	`defaultSystemFont` (best for displaying text) or: `defaultSmallFont` (if you want a smaller font)
`largeFont`	`defaultLargeFont`
`largeBoldFont`	`defaultLargeFont`
`boldFont`	`defaultBoldFont`

Note that `defaultSystemFont` and `defaultSmallFont` might return the same font ID or different font IDs, depending on the system locale.

Compatibility Implemented only in the PalmOSGlue library.

See Also `FontSelect`, `FntGetFont`, `FntSetFont`



New FrmGlueGetObjectUsable

Purpose Returns whether an object in a form has been hidden.

Prototype

```
Boolean
FrmGlueGetObjectUsable (const FormType *formP,
UInt16 objIndex)
```

Parameters

-> `formP` A pointer to a `FormType` structure.

-> `objIndex` The index of the object on the form.

Result Returns `true` if the object is usable, meaning that it is considered part of the user interface. Returns `false` if the object is not usable. Objects that are not usable never appear on the screen. The function `FrmHideObject` clears an object's usable bit to hide that the object.

Comments Implemented only in the PalmOSGlue library.



New IntlGlueGetRoutineAddress

Purpose Return the address of a Text Manager function or of its PalmOSGlue equivalent.

Prototype

```
void *IntlGlueGetRoutineAddress
(IntlSelector selector, const void *latinSymbol)
```

Parameters

-> `selector` One of the routine selectors defined in `IntlMgr.h`.

-> `latinSymbol` The corresponding `TxtLatin`*`func`* symbol defined in `IntlGlue.h`.

Result Returns the address of the native Palm OS function if it is defined. If the function is not defined, it returns the address of the corresponding PalmOSGlue function.

Comments Use `IntlGlueGetRoutineAddress` for performance reasons. You can use the address that it returns to call the function at that address without having to go through the International Manager's trap dispatch table. `IntlGlueGetRoutineAddress` is mostly useful for optimizing the performance of Text Manager functions that are called in a tight loop.

To call `IntlGlueGetRoutineAddress`, you must pass both the international trap for the function and the corresponding symbol. For example, to obtain the address of the `TxtGetNextChar` function or of its PalmOSGlue equivalent, you would make this call:

```
myTxtGetNextChar =
  IntlGlueGetRoutineAddress(intlTxtGetNextChar,
    TxtLatinGetNextChar);
```

The returned address is only valid while your application stays locked in memory. For this reason, you should only use the returned address up to the point where your application terminates. When the application starts up again, you should call `IntlGlueGetRoutineAddress` again.

Compatibility Implemented only in the PalmOSGlue library.

See Also IntlGetRoutineAddress



SysGlueTrapExists

Purpose Macro that indicates if a given trap exists on the current system.

Prototype
```
SysGlueTrapExists (trapNum)
```

Parameters

-> `trapNum`	One of the system trap constants.

Result Returns `true` if the current operating system defines the system trap `trapNum`, or `false` if the trap does not exist on that version of the operating system.

Compatibility Implemented only in the PalmOSGlue library.

TxtGlueCharIsVirtual

Purpose Return whether a character is a virtual character or not.

Prototype
```
Boolean TxtGlueCharIsVirtual(UInt16 inModifiers,
WChar inChar)
```

Parameters

-> `inModifiers`	The value passed in the `modifiers` field of the keyDownEvent.
-> `inChar`	A character.

Result Returns `true` if the character `inChar` is a virtual character, `false` otherwise.

Comments Virtual characters are nondisplayable characters that trigger special events in the operating system, such as displaying low battery warnings or displaying the keyboard dialog. Virtual characters

should never occur in any data and should never appear on the screen.

Starting in Palm OS 3.1, the command modifier bit is always set in the `keyDownEvent` for a virtual character because the range for virtual characters overlaps the range for "real" characters that should appear on the screen. Earlier releases of the operating system did not always set the command modifier for virtual characters.

You can use this function to test whether a character is virtual or not. Pass the `chr` and `modifiers` fields exactly as you received them in the `keyDownEvent`, and this function performs the appropriate check based on the operating system version.

Compatibility Implemented only in the PalmOSGlue library.

TxtGlueGetHorizEllipsisChar

Purpose Return the horizontal ellipsis character.

Prototype

```
WChar TxtGlueGetHorizEllipsisChar (void)
```

Parameters None.

Result Returns the character code for horizontal ellipsis.

Comments Versions 3.1 and higher of the Palm OS use different character codes for the horizontal ellipsis character and the numeric space character than earlier versions did. Use `TxtGlueGetHorizEllipsisChar` to return the appropriate code for horizontal ellipsis regardless of which version of Palm OS your application is run on.

Compatibility Implemented only in the PalmOSGlue library.

See Also `ChrHorizEllipsis`, `TxtGlueGetNumericSpaceChar`

TxtGlueGetNumericSpaceChar

Purpose Return the numeric space character.

Prototype
```
WChar TxtGlueGetNumericSpaceChar (void)
```

Parameters None.

Result Returns the character code for numeric space.

Comments Versions 3.1 and higher of the Palm OS use different character codes for the horizontal ellipsis character and the numeric space character than earlier versions did. Use `TxtGlueGetNumericSpaceChar` to return the appropriate code for numeric space regardless of which version of Palm OS your application is run on.

Compatibility Implemented only in the PalmOSGlue library.

See Also `ChrNumericSpace`, `TxtGlueGetHorizEllipsisChar`

TxtGlueLowerChar

Purpose Convert a character to lowercase.

Prototype
```
WChar TxtGlueLowerChar (WChar inChar)
```

Parameters -> `inChar` A character.

Result Returns the character as a lowercase letter.

Comments This function does not handle the case in which the lowercase version of a character is represented by two or more characters. If you need to handle this situation, call the `TxtGlueLowerStr` function instead of this one.

Compatibility Implemented only in the PalmOSGlue library.

See Also `TxtGlueUpperChar`, `TxtGlueLowerStr`, `TxtGlueUpperStr`, `TxtGlueTransliterate`, `TxtTransliterate`, `StrToLower`

TxtGlueLowerStr

Purpose Convert a string to all lowercase letters.

Prototype
```
void TxtGlueLowerStr (Char* ioString,
UInt16 inMaxLength)
```

Parameters

<-> `ioString`	The string to be converted.
-> `inMaxLength`	The size of the buffer that contains the string, excluding the terminating `NULL` character.

Result Returns in `ioString` the input string with its letters converted to lowercase.

Comments Converting a string from uppercase to lowercase letters or vice versa may change the size of the string. For this reason, you should always check the size of the `ioString` after this call returns.

You must make sure that the parameter `ioString` points to the start of a a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character. If it doesn't, results are unpredictable.

This function can only handle characters in the ISO Latin 1 character encoding unless the International Feature Set is present.

Compatibility Implemented only in the PalmOSGlue library.

See Also `TxtGlueUpperStr`, `TxtGlueLowerChar`, `TxtGlueUpperChar`, `StrToLower` `TxtGlueTransliterate`, `TxtTransliterate`

TxtGluePrepFindString

Purpose Set up for `TxtFindString` or `FindStrInStr`.

Prototype
```
void TxtGluePrepFindString (const Char* inSource,
CharPtr outDest, UInt16 inDstSize)
```

Parameters

`-> inSource`	Pointer to the text to be searched for. Must not be `NULL`.
`<- outDest`	The same text as in `inSource` but converted to a suitable format for searching. `outDest` must not be the same address as `inSource`.
`-> inDstSize`	The length in bytes of the area pointed to by `outDest`.

Result Returns in `outDest` an appropriately converted string.

Comments Use this function to normalize the string to search for before using `TxtGlueFindString`, `TxtFindString`, or `FindStrInStr` to perform a search that is internal to your application. If you use any of these three search routines in response to the `sysAppLaunchCmdFind` launch code, the string that the launch code passes in is already properly normalized for the search.

This function normalizes the string to be searched for. The method by which a search string is normalized varies depending on the version of Palm OS and the character encoding supported by the device.

Only `inDstSize` bytes of `inSource` are written to `outDest`. If necessary to prevent overflow of the destination buffer, not all of `inSource` is converted.

You must make sure that the parameter `inSource` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character. If it doesn't, results are unpredictable.

Compatibility Implemented only in the PalmOSGlue library.

TxtGlueStripSpaces

Purpose Strip trailing and/or leading spaces from a string.

Prototype

```
Char* TxtGlueStripSpaces (Char* ioStr,
Boolean leading, Boolean trailing)
```

Parameters

<-> `ioStr`	Any string.
-> `leading`	If `true`, strip the leading spaces from the string.
-> `trailing`	If `true`, strip the trailing spaces from the string.

Result Returns `ioStr` with the specified spaces stripped from it. Note that this function both changes the `ioStr` buffer parameter and returns a pointer to it.

Comments You must make sure that the parameter `ioStr` points to the start of a a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character. If it doesn't, results are unpredictable.

Compatibility Implemented only in the PalmOSGlue library.



New TxtGlueTruncateString

Purpose Determine if a string can be displayed in a given number of pixels. If not, truncate the string.

Prototype

```
Boolean TxtGlueTruncateString (Char *ioStringP,
UInt16 inMaxWidth)
```

Parameters

<-> `ioStringP`	A null-terminated string. Upon return, the string is truncated if necessary so that it can be displayed in `inMaxWidth` pixels.
-> `inMaxWidth`	The maximum width in pixels.

Result Returns `true` if the string was truncated, or `false` if the string can fit without truncation.

Comments This function determines whether `ioStringP` can be displayed in the specified width without being truncated. If it can, `TxtGlueTruncateString` returns `false`. If the string must be truncated, this function truncates the string to one less than the

number of characters that can fit in `inMaxWidth` and then appends an ellipsis (...) character. (If the boundary characters are narrower than the ellipsis, more than one character may be dropped to make room). If `inMaxWidth` is narrower than the width of an ellipsis, the string is set to the empty string.

Compatibility Implemented only in the PalmOSGlue library.

See Also `FntWidthToOffset`, `WinDrawTruncChars`, `TxtGetTruncationOffset`

TxtGlueUpperChar

Purpose Convert a character to uppercase.

Prototype
```
WChar TxtGlueUpperChar (WChar inChar)
```

Parameters -> `inChar` Any character.

Result Returns the character as an uppercase letter.

Comments This function does not handle the case in which the uppercase version of a character is represented by two or more characters. If you need to handle this situation, call the `TxtGlueUpperStr` function instead of this one.

Compatibility Implemented only in the PalmOSGlue library.

See Also `TxtGlueLowerChar`, `TxtGlueUpperStr` `TxtGlueLowerStr`, `TxtGlueTransliterate`, `TxtTransliterate` `StrToLower`

TxtGlueUpperStr

Purpose Convert a string to all uppercase letters.

Prototype

```
void TxtGlueUpperStr (Char* ioString,
UInt16 inMaxLength)
```

Parameters

<-> `ioString`	The string to be converted.
-> `inMaxLength`	The size of the buffer that contains the string, excluding the terminating `NULL` character.

Result Returns in `ioString` the input string with its letters converted to uppercase.

Comments Converting a string from uppercase to lowercase letters or vice versa may change the size of the string. For this reason, you should always check the size of the `ioString` after this call returns.

You must make sure that the parameter `ioString` points to the start of a valid character. That is, it must point to the first byte of a multi-byte character or it must point to a single-byte character. If it doesn't, results are unpredictable.

This function can only handle characters in the ISO Latin 1 character encoding unless the International Feature Set is present.

Compatibility Implemented only in the PalmOSGlue library.

See Also `TxtGlueLowerStr`, `TxtGlueUpperChar`, `TxtGlueLowerChar`, `TxtGlueTransliterate`, `TxtTransliterate` `StrToLower`

77

SMS Exchange Library

This chapter describes the SMS Exchange Library API declared in the header file `SmsLib.h`. It discusses the following topics:

- SMS Exchange Library Data Structures
- SMS Exchange Library Constants

You interact with the SMS Exchange Library using the Exchange Manager APIs described in Chapter 58, "Exchange Manager," on page 1123 of this book. For further information on using Exchange Manager, see "Object Exchange" on page 1 of the of the *Palm OS Programmer's Companion*, vol. II, *Communications*.

SMS Exchange Library Data Structures



SmsParamsType

The `SmsParamsType` structure identifies information specific to the SMS Exchange Library. The `socketRef` field of the `ExgSocketType` structure is set to this structure when you send or receive data using the SMS Exchange Library. You only need to create this structure and assign it to the `socketRef` field if you have an SMS message to send and want to use non-default values for some of the fields; otherwise, the SMS Exchange Library creates this structure for you and provides default values.

```
typedef struct SmsParamsTag
{
  UInt32 creator;
  UInt16 smsID;
  Char *extension;
```

```
    Char *mimeTypes;
    UInt32 appCreator;
    UInt8 dataCodingScheme;
    UInt8 networkType;
    UInt8 dataType;
    UInt16 nbsDestPort;
    UInt16 nbsSrcPort;
    union
    {
      SmsSendParamsType send;
      SmsReceiveParamsType receive;
      SmsReportParamsType report;

    } data;

  } SmsParamsType, *SmsParamsPtr;
```

Field Descriptions

`creator`	Creator ID of the SMS Exchange Library. Always set this to `sysFileCSmsLib`.
`smsID`	The ID of the message that was sent. Do not set this field directly; the SMS Exchange Library should set it.
`extension`	If the SMS message has an attachment, this field specifies the attachment name. Do not set this field directly; the SMS Exchange Library sets it if necessary. See the `appCreator` field description for details.
`mimeTypes`	If the SMS message has an attachment, this field specifies the MIME type of the attachment. Do not set this field directly; the SMS Exchange Library sets it if necessary. See the `appCreator` field description for details.

`appCreator`	The creator ID of the target application for the attachment to the SMS message. Do not set this field directly; the SMS Exchange Library sets it if necessary. When the SMS Exchange Library receives a message with an attachment, it unwraps the message and attempts to deliver the attachment directly to an application that is registered to receive it. If no application is registered to receive unwrapped attachments of that type, the SMS Exchange Library sends the entire SMS message, and it sets the `extension`, `mimeTypes`, and `appCreator` fields in this structure. The SMS application can use this information to have the Exchange Manager deliver the attachment to the appropriate application using the Local Exchange Library.
`dataCodingScheme`	The data encoding scheme that the message uses. See SMS Data Coding Scheme Constants.
`networkType`	Indicates the type of advanced parameters. See SMS Network Type Constants.
`dataType`	Identifies the type of message being received, such as multipart or return receipt. See SMS Message Type Constants.
`nbsDestPort`	The Narrow Band Socket (NBS) port on which you want the data sent. The SMS Exchange Library sets this for you if you leave it blank. When data is being received, this field is set to the NBS port on which the data was received.

`nbsSrcPort`	The NBS port on which you want the data sent. You should set this field to the same value as `nbsDestPort`. If you leave it blank, the SMS Exchange Library provides a value for you.
`data`	The data being sent or received. See `SmsReceiveParamsType`, `SmsReportParamsType`, and `SMSSendParamsType`.

Compatibility This structure is only defined if 4.0 New Feature Set is present.



New SmsPrefType

The `SmsPrefType` structure defines the SMS Exchange Library preferences for sending and receiving SMS messages. Applications can use the `ExgControl` function to get, set, or display these preferences to the user.

```
typedef struct SmsPrefTag
{
  UInt32 validity;
  UInt16 warnOver;
  Boolean leave;
  Boolean report;
  Boolean autoSMSC;
  Char smscNumber[kSmsMaxPhoneSize];
} SmsPrefType, *SmsPrefPtr;
```

Field Descriptions

`validity`	The number of seconds before the message expires. If the message cannot be delivered to the recipient, the service center repeatedly attempts to deliver the message until it expires. The default is one hour.
`warnOver`	The number of parts a user can send without confirmation. If the user attempts to send a message with more than this number of parts, an alert is displayed, and the user can choose to send the message anyway. The default is 3 parts. (If the user attempts to send a message with more than 3 parts, an alert is displayed.)
`leave`	If `true`, any incoming messages retrieved from a phone remain on the phone as well. If `false`, the messages are deleted from the phone's inbox.
`report`	If `true`, the user receives confirmation that an SMS message was delivered.
`autoSMSC`	If `true`, don't use the value stored in the `smscNumber` field.
`smscNumber`	The message center to be used. If `NULL` or the empty string, the SMS message center set by the phone is used.

Compatibility This structure is only defined if 4.0 New Feature Set is present.



New SmsReceiveCDMAParamsType

The `SmsReceiveParamsType` includes an `SmsReceiveCDMAParamsType` structure for CDMA messages.

```
typedef struct SmsReceiveCDMAParamsTag
{
  UInt8 messageType;
  TelSmsDateTimeType validityPeriod;
```

```
    UInt8 priority;
    UInt8    privacy;
    Boolean alertOnDeliveryRequest;
    Boolean manualAckRequest;
    UInt8    voiceMessageNumber;
    UInt8    languageIndicator;
    Char *   callbackNumberAddress;
  } SmsReceiveCDMAParamsType,
  *SmsReceiveCDMAParamsPtr;
```

Field Descriptions

`messageType`	The type of the message. This is one of the SMS Message Type Constants constants defined in `TelephonyMgr.h`.
`validityPeriod`	The amount of time for which the message is valid. See `TelSmsDateTimeType`. The default is set according to the SMS preferences.
`priority`	The message priority. This must be one of the SMS Message Urgency Constants defined in `TelephonyMgr.h`.
`privacy`	The privacy type of the message. This must be one of the SMS Message Privacy Constants defined in `TelephonyMgr.h`.
`alertOnDeliveryRequest`	`true` if the user is to be alerted upon delivery of this message, and `false` if not.
`manualAckRequest`	`true` if a confirmation is requested from the recipient, and `false` if not.
`voiceMessageNumber`	The number of new messages in your voice mail.

`languageIndicator`	Reserved for future use.
`callbackNumberAddress`	The callback number to which confirmations are to be sent.

Compatibility This structure is only defined if 4.0 New Feature Set is present.



SmsReceiveGSMParamsType

The `SmsReceiveParamsType` includes an `SmsReceiveGSMParamsType` structure for GSM messages.

```
typedef struct SmsReceiveGSMParamsTag
{
  UInt16 protocolId;
  Char *serviceCenterNumber;
  Boolean replyPath;
} SmsReceiveGSMParamsType,
*SmsReceiveGSMParamsPtr;
```

Field Descriptions

`protocolId`	Reserved for future use.
`serviceCenterNumber`	The SMS service center that must be used to send a reply. If `NULL`, the service center specified in the preferences is used.
`replyPath`	If `true`, replies must be made through the SMS service center specified by `serviceCenterNumber`.

Compatibility This structure is only defined if 4.0 New Feature Set is present.



New SmsReceiveParamsType

The `SmsReceiveParamsType` structure is used as the `data` field for the `SmsParamsType` structure when the SMS Exchange Library has received data. The SMS Exchange Library always supplies the values for these fields.

```
typedef struct SmsReceiveParamsTag
{
  UInt32 timeStamp;
  Char *originatingAddress;
  UInt8 leaveOnPhone:1;
  UInt8 forceSlotMode:1;
  UInt8 reserved:6;
  UInt16 index;
  Boolean otherToReceive;
  Boolean reportDeliveryIndicator;
  union
  {
    SmsReceiveGSMParamsType gsm;
    SmsReceiveCDMAParamsType cdma;
    SmsReceiveTDMAParamsType tdma;
  } protocol;
} SmsReceiveParamsType, *SmsReceiveParamsPtr;
```

Field Descriptions

`timeStamp`	The time at which the message was delivered, given as the number of seconds since January 1, 1904.
`originatingAddress`	The number from which the message was received.
`leaveOnPhone`	If `true`, messages received on the phone are not deleted from the phone's inbox. If not specified, this is set according to the system preferences.

`forceSlotMode`	If `true`, use slot mode parsing. If `false`, use block mode parsing. The default is block mode.
`reserved`	Reserved for future use.
`index`	Location where the message is stored on the mobile phone.
`otherToReceive`	If `true`, there are more messages to be received from the service center.
`reportDeliveryIndicator`	If `true`, the sender has requested confirmation. The recipient of the message does not send the confirmation; the SMS service center does.
`protocol`	Values specific to the protocol used to send the message. Currently, only GSM is supported.

Compatibility This structure is only defined if 4.0 New Feature Set is present.



SmsReceiveTDMAParamsType

The `SmsReceiveParamsType` includes an `SmsReceiveTDMAParamsType` structure for TDMA messages. This structure is currently the same as the `SmsReceiveCDMAParamsType` structure.

```
typedef SmsReceiveCDMAParamsType
  SmsReceiveTDMAParamsType,
  *SmsReceiveTDMAParamsPtr;
```

Compatibility This structure is only defined if 4.0 New Feature Set is present.



New SmsReportParamsType

The `SMSReportParamsType` structure is used as the `data` field for the `SmsParamsType` structure when the SMS Exchange Library has received a delivery confirmation. The SMS Exchange Library always sets the values for these fields.

```
typedef struct SmsReportParamsTag
{
  UInt32 timeStamp;
  UInt16 index;
  UInt8 reportType;
  UInt8 report;
  Char* originatingAddress;
} SmsReportParamsType, *SmsReportParamsPtr;
```

Field Descriptions

`timeStamp`	The date and time at which the message was delivered, given as the number of seconds since January 1, 1904.
`index`	Location where the message is stored on the mobile phone.
`reportType`	One of the Delivery Report Type constants defined in `TelephonyMgr.h`.
`report`	One of the Delivery Status Report constants defined in `TelephonyMgr.h`.
`originatingAddress`	Phone number to which the message was sent.

Compatibility This structure is only defined if 4.0 New Feature Set is present.



New SmsSendCDMAParamsType

The `SMSSendParamsType` includes an `SmsSendCDMAParamsType` structure for CDMA messages.

```
typedef struct SmsSendCDMAParamsTag
{
  UInt8   messageType;
  TelSmsDateTimeType deferredDate;
  UInt8   priority;
  UInt8   privacy;
  UInt8   alertOnDelivery:1;
  UInt8   manualAckRequest:1;
  UInt8   reserved:6;
  Char*   callbackNumber;
} SmsSendCDMAParamsType, *SmsSendCDMAParamsPtr;
```

Field Descriptions

`messageType`	The type of the message. This is one of the SMS Message Type Constants constants defined in `TelephonyMgr.h`.
`deferredDate`	Not used.
`priority`	The message priority. This must be one of the SMS Message Urgency Constants.
`privacy`	The privacy type of the message. This must be one of the SMS Message Privacy Constants.
`alertOnDelivery`	`true` if the user is to be alerted upon delivery of this message, and `false` if not.
`manualAckRequest`	`true` if a confirmation is requested from the recipient, and `false` if not.
`reserved`	Reserved for future use.
`callbackNumber`	Number to which the confirmation should be sent.

Compatibility This structure is only defined if 4.0 New Feature Set is present.



SmsSendGSMParamsType

The SMSSendParamsType includes an SmsSendGSMParamsType structure for GSM messages.

```
typedef struct SmsSendGSMParamsTag
{
  UInt16 protocolId;
  Char *serviceCenterNumber;
  Boolean rejectDuplicated;
  Boolean replyPath;
} SmsSendGSMParamsType, *SmsSendGSMParamsPtr;
```

Field Descriptions

protocolId	Reserved for future use.
serviceCenterNumber	The message center to be used. If not specified, the service center is set according to the system preferences.
rejectDuplicated	If true, the service center rejects messages that have the same message ID, destination address, and originating address as a previously submitted message.
replyPath	If true, the service center that delivers the message is requested to provide information about itself to the recipient so that replies are made through the same service center.

Compatibility This structure is only defined if 4.0 New Feature Set is present.



SMSSendParamsType

The `SMSSendParamsType` structure is used as the `data` field for the `SmsParamsType` structure when the SMS Exchange Library is sending data.

```
typedef struct SmsSendParamsTag
{
  TelSmsDateTimeType validityPeriod;
  Char *destinationAddress;
  UInt8 networkDeliveryRequested:1;
  UInt8 ignoreDefaultValue:1;
  UInt8 reserved:6;
  UInt16 partCount;
  UInt16 lastPart;
  UInt8 converter;
  union
  {
    SmsSendGSMParamsType gsm;
    SmsSendCDMAParamsType cdma;
    SmsSendTDMAParamsType tdma;
  } protocol;
} SmsSendParamsType, *SmsSendParamsPtr;
```

Field Descriptions

`validityPeriod`	The amount of time for which the message is valid. See `TelSmsDateTimeType`. The default is set according to the SMS preferences.
`destinationAddress`	A buffer that contains the phone number of the message recipient. If no phone number is supplied, the user is prompted for the phone number.

`networkDeliveryRequested`	If `true`, the SMS service center sends a delivery confirmation. The default is set according to the SMS preferences. The SMS Exchange Library disables this field for multipart messages.
`ignoreDefaultValue`	If `false`, the validity period, network delivery requested, and SMS center specified in the preferences are used regardless of the values supplied in this structure. If `true`, the values supplied in this structure are used.
`reserved`	Reserved for future use.
`partCount`	The number of parts in the message. 0 means that the message is not a multipart message. If NBS is used to send the message, it determines the number of parts.
`lastPart`	The last part of a multipart message that was successfully sent.
`converter`	The header added to the data to specify how it is converted.
`protocol`	Data specific to the protocol used to send the message. Currently, only GSM is supported.

Compatibility This structure is only defined if 4.0 New Feature Set is present.

New SmsSendTDMAParamsType

The SMSSendParamsType includes an SmsSendTDMAParamsType structure for TDMA messages. This structure is currently the same as the SmsSendCDMAParamsType structure.

```
typedef SmsSendCDMAParamsType
  SmsSendTDMAParamsType,
  *SmsSendTDMAParamsPtr;
```

Compatibility This structure is only defined if 4.0 New Feature Set is present.

SMS Exchange Library Constants

SMS Control Constants

The SMS control constants are passed as the operation parameter to the ExgControl function. The ExgControl function is a way to communicate directly with the SMS Exchange Library. The following table lists the operation constant, the type of data that should be passed as the valueP parameter to ExgControl, and what the SMS Exchange Library does in response.

Table 77.1 ExgControl operations for SMS library

Operation	value Data Type	Description
exgLibSmsPrefGetOp	SmsPrefType	Returns a pointer to the SMS Exchange Libraries preferences in valueP, creating the preferences and setting them to the default values if they do not exist.
exgLibSmsPrefGet DefaultOp	SmsPrefType	Returns the default values for the SMS Exchange Library preferences.

Table 77.1 ExgControl operations for SMS library *(continued)*

Operation	value Data Type	Description
`exgLibSmsPrefSetOp`	`SmsPrefType`	Sets the SMS Exchange Library preferences to the values passed in `valueP`.
`exgLibSmsPref DisplayOp`	`kSmsNetworkAuto` or `kSmsNetworkGSM`. Input only.	Display a form that allows the user to set the SMS preferences.
`exgLibSms IncompleteGetCountOp`	`UInt16`. Output only.	Get the number of incomplete messages currently stored in the SMS Exchange Library. The library stores message parts as it receives them. When it has received all of the parts, it reassembles the message and delivers it. This operation tells how many messages are currently under assembly.
`exgLibSms IncompleteDeleteOp`	`UInt16`. Input only.	Delete the incomplete message with the ID passed in `valueP`. Pass -1 to delete all incomplete messages.

Compatibility These constants are only defined if 4.0 New Feature Set is present.

SMS Data Coding Scheme Constants

The SMS data coding scheme constants describe the coding scheme used for SMS data. These values are used as the `dataCodingScheme` parameter of the `SmsParamsType` structure.

Constant	Value	Description
`kSmsRowDataEncoding`	0	8-bit encoding scheme. This is the default.
`kSmsTextEncoding`	1	7-bit encoding scheme.

Compatibility These constants are only defined if 4.0 New Feature Set is present.

SMS Network Type Constants

The SMS network type constants identify the type of network being used for SMS messages. Currently, only the GSM network is supported.

Constant	Value	Description
`kSmsNetworkAuto`	`-1`	The network is set by the phone. This is the default.
`kSmsNetworkCDMA`	`kTelNwkCDMA`	A CDMA network. Currently not supported.
`kSmsNetworkGSM`	`kTelNwkGSM`	A GSM network.
`kSmsNetworkTDMA`	`kTelNwkTDMA`	A TDMA network. Currently not supported.
`kSmsNetworkPDC`	`kTelNwkPDC`	A PDC network. Currently not supported.

Compatibility These constants are only defined if 4.0 New Feature Set is present.

SMS Message Type Constants

The SMS message type constants identify the type of message being sent. They are used as the `dataType` field of the `SmsParamsType` structure.

Constant	Value	Description
`kSmsMessageType`	0	Standard SMS message of no more than 160 bytes. This is the default.
`kSmsIncompleteType`	1	A part of a multipart SMS message.
`kSmsReportType`	2	A confirmation, indicating that an SMS message was successfully sent.

Compatibility These constants are only defined if 4.0 New Feature Set is present.

SMS Converter Constants

The SMS converter constants identify the header information added to an SMS message.

Constant	Value	Description
`kSmsNBSConverter`	0	An NBS header is added to the message. This is the default.
`kSmsNoConverter`	1	No header is added to the message.

Compatibility These constants are only defined if 4.0 New Feature Set is present.

78

Personal Data Interchange Library

This chapter provides reference material for the Personal Data Interchange (PDI) library, which provides tools for reading and writing vObjects, including vCards and vCals. This chapter discusses the following topics:

- PDI Library Data Structures
- PDI Library Constants
- PDI Library Functions

The header file `PdiLib.h` declares the Personal Data Interchange library API. The header file `PdiConst.h` declares the constants that you use with the PDI library.

For information about how to use the functions and constants described in this chapter, see Chapter 3, "Personal Data Interchange" in the *Palm OS Programmer's Companion*.

PDI Library Data Structures

This section describes the data structures used with the PDI library functions.



PdiDictionary

The `PdiDictionary` type is a simple typedef that represents an internal, binary object.

```
typedef UInt8 PdiDictionary;
```



New PdiReaderType

The `PdiReaderType` data structure represents a PDI reader object, which you use to read data from an input stream.

```
typedef struct _PdiReader {
  Err                error;
  UInt8              encoding;
  CharEncodingType   charset;
  UInt16             written;
  UInt8              fieldNum;
  UInt16             property;
  UInt16             propertyValueType;
  UInt16             parameter;
  UInt32  parameterPairs[kPDI_ENTRIES_NUMBER];
  UInt16             customFieldNumber;
  void               *appData;
  UInt16             pdiRefNum;
  UInt16             events;
  Char               *groupName;
  Char               *propertyName;
  Char               *parameterName;
  Char               *parameterValue;
  Char               *propertyValue;
  PdiDictionary      *dictionary[2];
} PdiReaderType
```

Field Descriptions

`error`	The most recent error.
`encoding`	The type of encoding for the property value.
`charset`	The character set of the property value.
`written`	The number of characters that have currently been written to the buffer.
`fieldNum`	The current field number.
`property`	The ID of the current property.

`propertyValueType`	The value type of the current property value.
`parameter`	The ID of the most recently parsed parameter name.
`parameterPairs`	An integer array with bits set for each parameter value that has been parsed for the current property value. **NOTE:** You must use the `PdiParameterPairTest` macro to access this field.
`customFieldNumber`	The number of the custom field parsed by the reader for the current property. Custom fields are used in the Palm™ Address Book.
`appData`	Application-dependent data field.
`pdiRefNum`	The library reference number associated with this reader.
`events`	The mask of events handled by the reader in its most recent operation. This is a combination of some number of the event constants described in Reader Event Constants.
`groupName`	The group name for the current property.
`propertyName`	The name of the current property.
`parameterName`	The name of the current parameter.
`parameterValue`	The value of the current parameter.

`propertyValue`	The current property value string.
`dictionary`	An array of two dictionary pointers. The `dictionary[0]` entry is the main dictionary built into the PDI library, and the `dictionary[1]` entry is an optional, custom dictionary associated with the reader object with a call to `PdiDefineReaderDictionary`.



PdiWriterType

The `PdiWriterType` data structure represents a PDI writer object, which you use to write data to an output stream.

```
typedef struct _PdiWriter {
  void               *appData;
  UInt16             pdiRefNum;
  UInt16             encoding;
  CharEncodingType   charset;
  Err                error;
  PdiDictionary      *dictionary[2];
} PdiWriterType
```

Field Descriptions

`appData`	Application-dependent data field.
`pdiRefNum`	The library reference number associated with this reader.
`encoding`	The type of encoding for the property value.
`charset`	The character set of the property value.

`error`	The most recent error.
`dictionary`	An array of two dictionary pointers. The `dictionary[0]` entry is the main dictionary built into the PDI library, and the `dictionary[1]` entry is an optional, custom dictionary associated with the reader object with a call to `PdiDefineReaderDictionary`.

PDI Library Constants

This section describes the constants used in the PDI library, which include the following constant types:

- Buffer Management Constants
- Encoding Type Constants
- Error Code Constants
- Parameter Name Constants
- Parameter Value Constants
- Property Name Constants
- Property Type Constants
- Property Value Field Constants
- Property Value Format Constants
- Reader and Writer Options Constants
- Reader Event Constants
- Value Type Constants

Buffer Management Constants

You use the buffer management constants to determine how buffers are managed in the PDI reader.

Constant	Value	Description
kPdiResizableBuffer	0xFFFF	Indicates that the buffer can be automatically resized by the PDI library.
kPdiDefaultBufferMaxSize	0x3FFF	The default maximum buffer size, in bytes. You can change the maximum size of a reader's buffer by calling the PdiDefineResizing function.
kPdiDefaultBufferDeltaSize	0x0010	The default number of bytes by which the input buffer is grown when the PDI library performs automatic resizing. You can change the delta amount of a reader's buffer by calling PdiDefineResizing function.

Encoding Type Constants

You use the encoding type constants to specify the type of encoding used in a vObject reader or writer.

Constant	Value	Description
kPdiASCIIEncoding	0	The vObject is not encoded.
kPdiQPEncoding	kPdiPAV_ENCODING_QUOTED_PRINTABLE	The vObject uses the quoted printable encoding.
kPdiB64Encoding	kPdiPAV_ENCODING_BASE64	The vObject uses Base 64 encoding. The writer outputs "ENCODING=BASE64."

Constant	Value	Description
kPdiBEncoding	kPdiPAV_ENCODING_B	The vObject uses Base 64 encoding. This is the same as the kPdiB64Encoding value, except that the PDI writer outputs "ENCODING=B." This encoding is used with the vCard 3.0 standard.
kPdiEscapeEncoding	0x8000	The vObject uses escapes for special characters.
kPdiNoEncoding	0x8001	The PDI writer does not encode the vObject value.

Error Code Constants

The PDI library functions return the error code constants shown in the following table to indicate their status.

Constant	Description
pdiErrRead	An error occurred while reading from the input stream.
pdiErrWrite	An error occurred while writing to the output stream.
pdiErrNoPropertyName	An attempt was made to write a property value before the property name was written.
pdiErrNoPropertyValue	The application did not write the last property value.
pdiErrMoreChars	The buffer is full. Superfluous characters have been discarded.
pdiErrNoMoreFields	There are no more property fields to read.

Constant	Description
pdiErrOpenFailed	The PDI library could not be opened.
pdiErrCloseFailed	The PDI library could not be closed. This can occur if another application is using the library.

Parameter Name Constants

The PdiConst.h file defines several parameter name constants that you can use to specify the name of a parameter in functions that use parameter names. The parameter name constants have the following format:

```
kPdiPAN_parameterName
```

where *parameterValue* is replaced by a parameter name.

The following table shows examples of parameter name constants. For a complete list, see the PdiConst.h file.

Constant	Description
kPdiPAN_TYPE	The TYPE parameter.
kPdiPAN_ENCODING	The ENCODING parameter.
kPdiPAN_STATUS	The STATUS parameter.

Parameter Value Constants

The PdiConst.h file defines several parameter value constants that you can use to specify the name and value of a parameter in functions that use name and value pairs. The parameter value constants have the following format:

```
kPdiPAV_parameterName_parameterValue
```

where *parameterName* is replaced by a parameter name and *parameterValue* is replaced by a parameter value.

The following table shows examples of parameter value constants. For a complete list, see the PdiConst.h file.

Constant	Description
kPdiPAV_TYPE_TEL	The parameter name is TYPE and the parameter value is TEL.
kPdiPAV_ENCODING_BASE64	The parameter name is ENCODING and parameter value is BASE64.
kPdiPAV_ENCODING_8BIT	The parameter name is ENCODING and the parameter value is 8BIT.
kPdiPAV_STATUS_ACCEPTED	The parameter name is STATUS and the parameter value is ACCEPTED.

Property Name Constants

The PdiConst.h file defines several property name constants that you can use to specify the name of a PDI property in functions that use property names. The property name constants have the following format:

```
kPdiPRN_propertyName
```

where *propertyName* is replaced by a property name.

The following table shows examples of property name constants. For a complete list, see the PdiConst.h file.

Constant	Description
kPdiPRN_ADR	The ADR property.
kPdiPRN_BDAY	The BDAY property.
kPdiPRN_BEGIN	The BEGIN property.
kPdiPRN_BEGIN_VCARD	The BEGIN:VCARD property.

Property Type Constants

You use the property type constants to specify the data type of a property.

Constant	Value	Description
kPdiType_URI	0	The data is a uniform resource identifier.
kPdiType_UTC_OFFSET	1	The data is an offset from UTC to local time.
kPdiType_RECUR	2	The data is a specification of a recurrence rule.
kPdiType_PERIOD	3	The data is a precise period of time.
kPdiType_CAL_ADDRESS	4	Calendar user data.
kPdiType_BINARY	5	Binary data.
kPdiType_TEXT	6	Text data.
kPdiType_FLOAT	7	Floating-point data.
kPdiType_DURATION	8	Time duration data.
kPdiType_DATE_TIME	9	Calendar date and time data.
kPdiType_DATE	10	Date data.
kPdiType_INTEGER	11	Signed integer data.
kPdiType_TIME	12	Time-of-day data.

Property Value Field Constants

The `PdiConst.h` file defines several property value field constants that you can use to specify the position of a PDI property value field in functions that use fields. The property value field constants have the following format:

`kPdiPVF_`*`propertyValueField`*

where *propertyValueField* is replaced by a property value field name.

The following table shows examples of property name constants. For a complete list, see the `PdiConst.h` file.

Constant	Description
kPdiPVF_ADR_POST_OFFICE	The property value field that stores the post office portion of the address.
kPdiPVF_ADR_EXTENDED	The property value field that stores the extended portion of the address.
kPdiPAN_ADR_COUNTRY	The property value field that stores the country portion of the address.

Property Value Format Constants

Some properties have **structured values**, which are values that contain multiple fields. These fields are typically separated by commas or semicolons in the vObject input or output stream. You use the property value format constants with the PdiReadPropertyField and PdiWritePropertyStr functions to specify how to handle fields in a structured value.

Constant	Value	Description
kPdiNoFields	0	There are no fields in the property value; PdiReadPropertyField reads the entire value, or PdiWritePropertyStr specifies that the entire value should be written.
kPdiCommaFields	1	Fields are separated with comma (“,“) characters; PdiReadPropertyField reads one field, or PdiWritePropertyStr specifies that one field should be written.

Constant	Value	Description
`kPdiSemicolonFields`	`2`	Fields are separated with semicolon (“;“) characters; `PdiReadPropertyField` reads one field, or `PdiWritePropertyStr` specifies that one field should be written.
`kPdiDefaultFields`	`4`	The parser decides the property value format, based on the property name.
`kPdiConvertComma`	`8`	Fields are separated with comma characters; `PdiReadPropertyField` reads the entire value and converts each comma into a newline (“`\n`“) character.
`kPdiConvertSemicolon`	`16`	Fields are separated with semicolon characters; `PdiReadPropertyField` reads the entire value and converts each semicolon into a newline (“`\n`“) character.

Reader and Writer Options Constants

You use the reader and writer option constants to control how the PDI reader (parser) reads values from the input stream or to control how the PDI writer (generator) writes values to the output stream.

Constant	Value	Description
`kPdiEnableFolding`	`1`	Enables folding of properties in the output stream. **Folding** is a mechanism for breaking long lines to allow them to be transmitted without change. If you specify this flag, the PDI library folds long lines. Note that folding is not compatible with earlier versions of the Palm OS®. Also note that other encoding formats, including quoted-printable and Base 64, define their own mechanisms for splitting long lines.
`kPdiEnableQuotedPrintable`	`2`	Enables quoted-printable encoding in the output stream and makes it the default encoding. This is an encoding format for non-ASCII values. You must have this enabled for compatibility with earlier versions of the Palm OS. If you do not specify this option, the default encoding is Base 64.

Constant	Value	Description
`kPdiEscapeMultiFieldValues`	`4`	For compatibility with earlier versions of the Palm OS. You must enable this for compatibility with earlier versions of the Palm OS. However, some non-Palm PDI software does not support this format. For more information about compatibility with earlier versions of the Palm OS, see Format Compatibility in the *Palm OS Programmer's Companion*, vol. II, *Communications*.
`kPdiPalmCompatibility`	`6`	This is a combination of `kPdiEnableQuotedPrintable | kPdiEscapeMultiFieldValues`. It forces the PDI writer to generate output that is compatible with earlier versions of the Palm OS.
`kPdiEnableB`	`8`	Enables base 64 encoding in the output stream, and tells the PDI writer to output "`ENCODING=B`" instead of "`ENCODING=BASE64`" when encoding a value with Base 64. Note: the vCard 3.0 standard has replaced the earlier `ENCODING=BASE64` with `ENCODING=B`. The meaning is the same.
`kPdiOpenParser`	`8`	Specifies that the PDI reader is open to all formats, including Palm and others.

Reader Event Constants

The PDI reader event constants specify the events that the reader has handled during the current read operation. The event values are combined together and stored in the `events` field of the PDI reader object. You can use them to test whether the reader handled a certain event.

Constant	Value	Description
`kPdiEOFEventMask`	`1`	End of file was reached.
`kPdiGroupNameEventMask`	`2`	A group name was found.
`kPdiPropertyNameEventMask`	`4`	A property name was found.
`kPdiParameterNameEventMask`	`8`	A parameter name was found.
`kPdiParameterValueEventMask`	`16`	A parameter value was found.
`kPdiPropertyDefinedEventMask`	`32`	A property definition was found; this implies that the ":" separator character was found.
`kPdiPropertyValueEventMask`	`64`	An entire property value was found
`kPdiPropertyValueFieldEventMask`	`128`	A value field was found; this implies that the ";" separator character was found.
`kPdiPropertyValueItemEventMask`	`256`	A value item was found; this implies that the "," separator character was found.

Constant	Value	Description
kPdiPropertyValueMoreCharsEventMask	512	The buffer that you provided was not large enough. The superfluous characters have been discarded.
kPdiBeginObjectEventMask	1024	The object begin indicator BEGIN was reached.
kPdiEndObjectEventMask	2048	The object end indicator END was reached.

Value Type Constants

You can use the following constants to specify data typing information for the PdiWritePropertyBinaryValue, PdiWritePropertyFields, and PdiWritePropertyValue functions.

Constant	Value	Description
kPdiWriteData	0	The value is data. The PDI writer does not compute a character set. You can use this for binary data or pure ASCII data.
kPdiWriteText	8	The value is text data. The PDI writer parses the data character by character to compute the correct charset and character encoding for the data.
kPdiWriteMultiline	16	Explicitly specifies that the value contains special characters, such as newlines, and must be encoded. If this flag is not specified, the encoding is determined by the applied character set.

PDI Library Functions



New PdiDefineReaderDictionary

Purpose Installs a new custom dictionary for use with a PDI reader object.

Prototype

```
PdiDictionary *PdiDefineReaderDictionary
(UInt16 libRefnum, PdiReaderType *ioReader,
PdiDictionary *dictionary,
Boolean disableMainDictionary)
```

Parameters

`-> libRefnum` The PDI library reference number.

`-> ioReader` The PDI reader object with which to associate the dictionary. This object must have previously been created by a call to the `PdiReaderNew` function.

`-> dictionary` A pointer to a dictionary object that was created by the . The dictionary object is an array of binary data.

`-> disableMainDictionary` If `true`, the main reader dictionary is disabled, and only this new dictionary is searched for terms; if `false`, the new dictionary supplements the main dictionary.

Result Returns a pointer to the previously installed custom dictionary, or `NULL` if there was not a previously installed custom dictionary.

Comments This function installs a dictionary for use with the `ioReader` object. The dictionary contains the syntax for extensions or replacements of the PDI properties about which the PDI reader knows. The reader knows about properties specified in one of the vObject standards, including the vCard or vCal standards.

You can uninstall the current custom dictionary by specifying `NULL` as the value of the `dictionary` parameter,

Note that the dictionary must have previously been compiled by the dictionary tool. For more information, review the PDI sample code, which you can find on the Palm Developer's Knowledge Base at http://www.palmos.com/dev/tech/kb.



New PdiDefineResizing

Purpose

Defines the sizing information to use when automatically resizing a buffer. PDI reader objects read information from the input stream into a buffer and automatically adjust the buffer size as required.

Prototype

```
Err PdiDefineResizing(UInt16 libRefnum,
PdiReaderType *ioReader, UInt16 deltaSize,
UInt16 maxSize)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioReader`	The PDI reader object, which was created by a previous call to the `PdiReaderNew` function.
`-> deltaSize`	The number of bytes by which to grow the buffer when it needs resizing.
`-> maxSize`	The maximum allowable size for the buffer.

Result

Returns `errNone` if successful, and an error code if not successful.

Comments

This function redefines the values to use when resizing a buffer. It does not perform any other actions.

The resizing values are used if your reader runs out of buffer space when storing input data during the processing of a property value. If possible, the reader resizes its internal buffer, using the values that you supply in this function.

The default resizing values apply if you do not call this function. The default values are:

```
kPdiDefaultBufferDeltaSize    0x0010
kPdiDefaultBufferMaxSize      0x3FFF
```

Compatibility

Implemented only if 4.0 New Feature Set is present.

New PdiDefineWriterDictionary

Purpose

Installs a new custom dictionary for use with a PDI writer object.

Prototype

```
PdiDictionary *PdiDefineWriterDictionary
(UInt16 libRefnum, PdiWriterType *ioWriter,
PdiDictionary *dictionary,
Boolean disableMainDictionary)
```

Parameters

`-> libRefnum` The PDI library reference number.

`-> ioWriter` The PDI writer object with which to associate the dictionary. This object must have previously been created by a call to the `PdiWriterNew` function.

`-> dictionary` A pointer to a dictionary object that was created by the . The dictionary object is an array of binary data.

`-> disableMainDictionary` If `true`, the main dictionary is disabled, and only this new dictionary is searched for terms; if `false`, the new dictionary supplements the main dictionary.

Result

Returns a pointer to the previously installed custom dictionary, or `NULL` if there was not a previously installed custom dictionary.

Comments

This function installs a dictionary for use with the `ioWriter` object. The dictionary contains the syntax for extensions or replacements of the PDI properties about which the PDI writer knows. The writer knows about properties specified in one of the vObject standards, including the vCard or vCal standards.

You can uninstall the current custom dictionary by specifying `NULL` as the value of the `dictionary` parameter,

Note that the dictionary must have previously been compiled by the dictionary tool. For more information, review the PDI sample code, which you can find on the Palm Developer's Knowledge Base at http://oasis.palm.com/dev/kb.



PdiEnterObject

Purpose Tells the PDI library to enter into a recursively-defined object.

Prototype

```
Err PdiEnterObject(UInt16 libRefnum,
PdiReaderType *ioReader)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioReader`	The PDI reader object, which was created by a previous call to the `PdiReaderNew` function.

Result Returns `errNone` if successful, and an error code if not successful.

Comments Some vObjects recursively define other vObjects. Your application can choose whether or not to enter and parse the recursively defined objects.

If you want to parse the nested object definition, you need to call this function; otherwise, all of the properties of the nested object are skipped when the next call is made to the `PdiReadProperty` or `PdiReadPropertyName` functions.

Call this function after a `BEGIN_VObject` statement of the nested object has been parsed.

Compatibility Implemented only if 4.0 New Feature Set is present.



PdiLibClose

Purpose Close the PDI library after your application has finished with it.

Prototype

```
Err PdiLibClose(UInt16 libRefnum)
```

Parameters

`-> libRefnum`	The PDI library reference number.

Result Returns 0 if no other application uses the library. You may need to call `SysLibRemove` to remove the PDI library from the system library table.

Returns `pdiErrCloseFailed` if the library could not be closed.

Comments You should call this function after your application has finished with the PDI library, to allow the resources to be recovered.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiLibOpen`



New PdiLibOpen

Purpose Opens the PDI library for use by your application.

Prototype

```
Err PdiLibOpen(UInt16 libRefnum)
```

Parameters `-> libRefnum` The PDI library reference number.

Result Returns `errNone` if successful, and an error code if not successful.

Comments You must call this function before calling any of the other PDI functions.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiLibClose`



New PdiParameterPairTest

Purpose A macro that determines if the reader has already parsed the specified parameter value or name-value pair.

Prototype

```
PdiParameterPairTest (reader, pair)
```

Parameters

`-> reader`	The PDI reader object, which was created by a previous call to the `PdiReaderNew` function.
`-> pair`	The ID of the parameter. This must be one of the Parameter Value Constants.

Result Returns `true` if the specified parameter name-value pair has been parsed for the current property, and `false` if not.

Comments Some vObject generators do not specify the parameter name if the name is considered evident from the context. This means that both of the following constructs are considered proper:

```
Name=Value
Value
```

The `PdiParameterPairTest` macro returns `true` if the value has been parsed in either format. For example,

```
PdiParameterPairTest(reader, kPdiPAV_TYPE_WORK)
```

returns true for either of the following:

```
Type=WORK
WORK
```

Compatibility Implemented only if 4.0 New Feature Set is present.



New PdiReaderDelete

Purpose Delete a PDI reader object that is associated with the specified library number.

Prototype

```
void PdiReaderDelete(UInt16 libRefnum,
PdiReaderType **ioReader)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`<-> ioReader`	A pointer to the PDI reader object, which was created by a previous call to the `PdiReaderNew` function.

Result Returns nothing.

Comments This function deletes the `UDAReader` object associated with the reader object and frees the memory that was allocated for the reader object. The `ioReader` parameter is set to `NULL`.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiReaderNew`



PdiReaderNew

Purpose Create and initialize a new PDI reader object for use with the specified PDI library number.

Prototype

```
PdiReaderType *PdiReaderNew(UInt16 libRefnum,
UDAReaderType *input, UInt16 version)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> input`	The Unified Data Access (UDA) input object associated with the reader.
`-> version`	Options to control the parsing behavior of the reader. You can use a combination of the Reader and Writer Options Constants.

Result Returns a pointer to the new PDI reader object. Returns `NULL` if the reader cannot be created.

Comments The current implementation of the `PdiReaderNew` function does not make use of the `optionFlags` settings because the reader knows how to adapt itself to all of the supported formats. The options will be used in future versions.

The `input` value is a UDA object for reading data from an input stream that can be connected to various data sources. For example, you can use a `UDAExchangeReader` to read data from the Exchange Manager, and you can use a `UDAStringReader` to read

data from a string. For more information about the UDA Manager, see Chapter 79, "Unified Data Access Manager."

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiReaderDelete`, `PdiWriterNew`



New PdiReadParameter

Purpose Read a single parameter name and its value from an input stream.

Prototype

```
Err PdiReadParameter(UInt16 libRefnum,
PdiReaderType *ioReader)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioReader`	The PDI reader object, which was created by a previous call to the `PdiReaderNew` function.

Result

`0`	The parameter and its value were read successfully.
`kPdiReadError`	The parameter and its value could not be successfully read from the input stream.

This function initializes the `parameterName` and `parameter` fields of the `ioReader` object.

This function sets the appropriate bits in the reader's `parameterValues` field if the parameter name is recognized.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiReaderNew`, `PdiReadPropertyField`, `PdiReadPropertyName`



New PdiReadProperty

Purpose Read the next property and its parameters from the input stream.

Prototype

```
Err PdiReadProperty(UInt16 libRefnum,
PdiReaderType *ioReader)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioReader`	The PDI reader object, which was created by a previous call to the `PdiReaderNew` function.

Result Returns `errNone` if successful. Returns `kPdiReadError` if an error occurs.

Comments The `PdiReadProperty` function reads a property name and its parameters, by reading until it encounters the PDI " : " separator character.

This function looks each name up in the properties dictionary, and sets the appropriate bit in the `ioReader` object structure to indicate that property-parameter pair has been read. The properties dictionary stores information about properties that are considered well known, as described in The PDI Library Properties Dictionary in Chapter 3, "Personal Data Interchange" in the *Palm OS Programmer's Companion*.

To read a property, you call `PdiReadProperty`, followed by a call or calls to the `PdiReadPropertyField` function to read the property value. For more information, see Reading Properties in Chapter 3, "Personal Data Interchange" in the *Palm OS Programmer's Companion*.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiReaderNew`, `PdiReadPropertyField`, `PdiReadPropertyName`, `PdiReadParameter`



New PdiReadPropertyField

Purpose

Read one field of a property value. The property value can be structured to contain multiple fields that are separated by commas or semicolons.

Prototype

```
Err PdiReadPropertyField(UInt16 libRefnum,
PdiReaderType *ioReader, Char **bufferPP,
UInt16 bufferSize, UInt16 readMode)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioReader`	The PDI reader object, which was created by a previous call to the `PdiReaderNew` function.
`<-> bufferPP`	A pointer to a pointer to the buffer into which the field characters are stored. Set this value to `NULL` to allow the PDI library to manage it. Note that the PDI library may need to resize the buffer; thus, the value of this parameter might change.
`-> bufferSize`	The size, in bytes, of the input buffer for reading the field. You can use the `PdiResizableBuffer` constant to specify that the PDI Library can automatically resize the buffer as required. If you do not specify the `PdiResizableBuffer` value, then the PDI library assumes that `buffer` cannot be moved, and that its size is fixed.
`-> readMode`	The format of the fields in the property value. Use one of the Property Value Format Constants.

Result

`0`	The field was read successfully.

`kPdiNoMoreFieldsError`
: There are no more fields to read because the entire value has already been read.

`kPdiMoreCharsError`
: The buffer is not large enough to store the entire field.

Comments

The value returned in the `buffer` is terminated with the "`\0`" character.

If the field is an empty string, the buffer is erased from memory, and the value of `buffer` is set to `NULL`.

If you specify `kPdiResizableBuffer` for the value of the `bufferSize` parameter, and the buffer needs more space, `PdiReadPropertyField` resizes the buffer for you, which may cause the value of `buffer` to be modified.

This function initializes the `propertyValue` and `fieldNum` fields of the `ioReader` object.

To read a property, you usually call the `PdiReadProperty` function, followed by a call or calls to `PdiReadPropertyField` to read the property value. For more information, see Reading Properties in Chapter 3, "Personal Data Interchange" in the *Palm OS Programmer's Companion*.

Compatibility

Implemented only if 4.0 New Feature Set is present.

See Also

`PdiReaderNew`, `PdiReadProperty`, `PdiReadPropertyName`, `PdiReadParameter`

New PdiReadPropertyName

Purpose

Read a property name from an input stream. Use this function when you want to parse and process each parameter individually.

Prototype

```
Err PdiReadPropertyName(UInt16 libRefnum,
PdiReaderType *ioReader)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioReader`	The PDI reader object, which was created by a previous call to the `PdiReaderNew` function.

Result Returns `errNone` if successful, and an error code if not successful.

Comments The `PdiReadProperty` function reads a property name only, reading until it encounters the PDI " : " separator character, or until it encounters the first parameter " , " separator character.

This function initializes the `property` and `propertyName` fields of the `ioReader` object.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiReaderNew`, `PdiReadPropertyField`, `PdiReadProperty`, `PdiReadParameter`

New PdiSetCharset

Purpose Force the character set of the next property value that is written by the specified PDI writer.

Prototype

```
Err PdiSetCharset(UInt16 libRefnum,
PdiWriterType *ioWriter, CharEncodingType charset)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioWriter`	The PDI writer object, which was created by a previous call to the `PdiWriterNew` function.

`-> charset`	The character set to use for the property value. This must be one of the following `CharEncodingType` values:
	`charEncodingAscii`
	`charEncodingISO8859_1`
	`charEncodingShiftJIS`
	`charEncodingISO2022Jp`

Result Returns `errNone` if successful, and an error code if not successful.

Comments This function tells `ioWriter` to use the specified `charSet` for the next property value that it writes, rather than computing a character set for that value.

You can determine the current character setting by examining the `charset` field of your PDI writer object.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiSetEncoding`



PdiSetEncoding

Purpose Force the encoding of the current property value.

Prototype

```
Err PdiSetEncoding(UInt16 libRefnum,
PdiWriterType *ioWriter, UInt16 encoding)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioReader`	The PDI writer object, which was created by a previous call to the `PdiWriterNew` function.
`-> encoding`	The encoding to apply to the property value. This must be one of the Encoding Type Constants.

Result Returns `errNone` if successful, and an error code if not successful.

Comments This function changes the encoding for the property value to the specified `encoding` value

You can determine the current encoding setting by examining the `encoding` field of your PDI writer object.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also PdiSetCharset



PdiWriteBeginObject

Purpose Writes a vObject begin tag to an output stream.

Prototype

```
Err PdiWriteBeginObject(UInt16 libRefnum,
PdiWriterType *ioWriter, UInt16 objectNameID)
```

Parameters

`-> libRefnum` The PDI library reference number.

`-> ioWriter` The PDI writer object, which was created by a previous call to the PdiWriterNew function.

`-> objectNameID` The object name ID. This must be one of the Property Name Constants that begins an object, including the following:

`kPdiPRN_BEGIN_VCAL`

`kPdiPRN_BEGIN_VCAL`

`kPdiPRN_BEGIN_VCARD`

`kPdiPRN_BEGIN_VEVENT`

`kPdiPRN_BEGIN_VFREEBUSY`

`kPdiPRN_BEGIN_VJOURNAL`

`kPdiPRN_BEGIN_VTIMEZONE`

`kPdiPRN_BEGIN_VTODO`

Result Returns `errNone` if successful, and an error code if not successful.

Comments Call this function to begin writing a vObject to the output stream. It writes a begin tag such as "`BEGIN:VCARD`" to the output stream.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also PdiWriteEndObject, PdiWriteProperty



PdiWriteEndObject

Purpose Writes a vObject end tag to an output stream.

Prototype

```
Err PdiWriteEndObject(UInt16 libRefnum,
PdiWriterType *ioWriter, UInt16 objectNameID)
```

Parameters

`-> libRefnum` The PDI library reference number.

`-> ioWriter` The PDI writer object, which was created by a previous call to the PdiWriterNew function.

`-> objectNameID` The object name ID. This must be one of the Property Name Constants that ends an object, including the following:

`kPdiPRN_END_VCAL`

`kPdiPRN_END_VCAL`

`kPdiPRN_END_VCARD`

`kPdiPRN_END_VEVENT`

`kPdiPRN_END_VFREEBUSY`

`kPdiPRN_END_VJOURNAL`

`kPdiPRN_END_VTIMEZONE`

`kPdiPRN_END_VTODO`

Result Returns `errNone` if successful, and an error code if not successful.

Comments Call this function to finish writing a vObject to the output stream. It writes a end tag such as "`END:VCARD`" to the output stream.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also PdiWriteBeginObject, PdiWriteProperty



PdiWriteParameter

Purpose Write a parameter, and optionally its name, to an output stream.

Prototype

```
Err PdiWriteParameter(UInt16 libRefnum,
PdiWriterType *ioWriter, UInt16 parameter,
Boolean parameterName)
```

Parameters

`-> libRefnum` The PDI library reference number.

`-> ioWriter` The PDI writer object, which was created by a previous call to the PdiWriterNew function.

`-> parameter` The ID of the parameter. This must be one of the Parameter Value Constants.

`-> parameterName`
If this is `true`, the parameter name, followed by the "=" symbol, followed by the parameter value is written to the output stream.

If this is `false`, only the parameter value is written to the output stream.

Result Returns `errNone` if successful, and an error code if not successful.

Comments Use this function to write a parameter to the output stream. To write a property, you usually call the PdiWriteProperty function, followed by calls to `PdiWriteParameter` to write any parameters, followed by a call to the PdiWritePropertyValue function to write the property value. For more information, see Writing Properties in Chapter 3, "Personal Data Interchange" in the *Palm OS Programmer's Companion*.

You can use the `parameterName` argument to specify that you want the parameter name written as well as the parameter value.

For example, the following table shows what is written if the value of `parameter` is `kPdiPAV_TYPE_HOME`.

Value of parameterName	Data written to output stream
`true`	`TYPE=HOME`
`false`	`HOME`

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiWriteProperty`, `PdiWritePropertyValue`, `PdiWritePropertyFields`, `PdiWritePropertyStr`, `PdiWriteParameterStr`

New PdiWriteParameterStr

Purpose Write a parameter name and the parameter value to an output stream.

Prototype

```
Err PdiWriteParameterStr(UInt16 libRefnum,
PdiWriterType *ioWriter,
const Char *parameterName,
const Char *parameterValue)
```

Parameters

`-> libRefnum` The PDI library reference number.

`-> ioWriter` The PDI writer object, which was created by a previous call to the `PdiWriterNew` function.

`-> parameterName` The name of the parameter. If the value of this is the empty string or `NULL`, only the parameter value is written.

`-> parameterValue` The parameter value string.

Result Returns `errNone` if successful, and an error code if not successful.

Comments This function writes the parameter name, followed by the "=" symbol, followed by the parameter value, to the output stream. If `parameterName` is `NULL`, or if its value is the empty string, just the parameter value is written.

This function is similar to the `PdiWriteParameter` function. The difference is that `PdiWriteParameterStr` takes the name and value of the parameter as strings, while `PdiWriteParameter` takes them as ID constants.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiWriteProperty`, `PdiWritePropertyValue`, `PdiWritePropertyFields`, `PdiWritePropertyStr`, `PdiWriteParameter`



New PdiWriteProperty

Purpose Writes a property name to an output stream.

Prototype

```
Err PdiWriteProperty(UInt16 libRefnum,
PdiWriterType *ioWriter, UInt16 propertyNameID)
```

Parameters

`-> libRefnum` The PDI library reference number.

`-> ioWriter` The PDI writer object, which was created by a previous call to the `PdiWriterNew` function.

`-> propertyNameID` The property name to write. This must be one of the Property Name Constants.

Result Returns `errNone` if successful, and an error code if not successful.

Comments To write a property, you usually call `PdiWriteProperty`, followed by calls to the `PdiWriteParameter` function to write any parameters, followed by a call to the `PdiWritePropertyValue` function to write the property value. For more information, see Writing Properties in Chapter 3, "Personal Data Interchange" in the *Palm OS Programmer's Companion*.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also PdiWritePropertyValue, PdiWritePropertyFields, PdiWritePropertyStr, PdiWriteParameter



New PdiWritePropertyBinaryValue

Purpose Write a binary property value to an output stream.

Prototype

```
Err PdiWritePropertyBinaryValue(UInt16 libRefnum,
PdiWriterType *ioWriter, const Char *buffer,
UInt16 size, UInt16 options)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioWriter`	The PDI writer object, which was created by a previous call to the `PdiWriterNew` function.
`-> buffer`	A buffer that contains the binary data.
`-> size`	The number of bytes of data to write from the buffer.
`-> options`	The data type. This must be a combination of one or more of the Value Type Constants.

Result Returns `errNone` if successful, and an error code if not successful.

Comments Use this function to write a binary data property value, such as a sound or an image.

This function encodes the data when it is written. The character set that gets applied to the data is not computed by this function; however, you can call the `PdiSetCharset` function to set the character set.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also PdiWriteProperty, PdiWritePropertyFields, PdiWritePropertyValue



New PdiWritePropertyFields

Purpose Write a structured property value with multiple fields to an output stream.

Prototype

```
Err PdiWritePropertyFields(UInt16 libRefnum,
PdiWriterType *ioWriter, Char *fields[],
UInt16 fieldNumber, UInt16 options)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioWriter`	The PDI writer object, which was created by a previous call to the `PdiWriterNew` function.
`-> fields`	An array of strings, each of which is a field of the property value. Individual fields can be `NULL`.
`-> fieldNumber`	The number of fields in the `Fields` array.
`-> options`	The data type. This must be a combination of one or more of the Value Type Constants.

Result Returns `errNone` if successful, and an error code if not successful.

Comments Use this function to write a property value that contains multiple fields.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiWritePropertyValue`, `PdiWritePropertyBinaryValue`, `PdiReadPropertyField`



New PdiWritePropertyStr

Purpose

Writes the name of a property to the output stream, and specifies the property value's structure for subsequent write operations.

Prototype

```
Err PdiWritePropertyStr(UInt16 libRefnum,
PdiWriterType *ioWriter, const Char *propertyName,
UInt8 writeMode, UInt8 requiredFields)
```

Parameters

`-> libRefnum` The PDI library reference number.

`-> ioWriter` The PDI writer object, which was created by a previous call to the `PdiWriterNew` function.

`-> propertyName` The name of the property to write.

`-> writeMode` The format of the fields in the property value. Use one of the following Property Value Format Constants:

`kPdiNoFields`

`kPdiCommaFields`

`kPdiSemicolonFields`

`-> requiredFields` The number of required fields for the value. This is usually set to `1`.

Result

Returns `errNone` if successful, and an error code if not successful.

Comments

Use this function when you are writing a property that is not in the dictionary, or when you are writing a property that uses value formatting that differs from the standard formatting stored in the dictionary for the property name.

This function writes the property name to the output stream, and then establishes the structure of the property's value, including the number of required fields and the separator character for those fields. After calling this function, the next call to the

PdiWritePropertyValue or PdiWritePropertyFields functions correctly writes the property value.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also PdiWriteProperty, PdiWritePropertyValue, PdiWritePropertyFields, PdiWriteParameter



New PdiWritePropertyValue

Purpose Write a string to the output stream as the entire value of a property.

Prototype

```
Err PdiWritePropertyValue(UInt16 libRefnum,
PdiWriterType *ioWriter, Char *buffer,
UInt16 options)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> ioWriter`	The PDI writer object, which was created by a previous call to the PdiWriterNew function.
`-> buffer`	The input buffer that contains the string to be written.
`-> options`	The data type. This must be a combination of one or more of the Value Type Constants.

Result Returns `errNone` if successful, and an error code if not successful.

Comments Use this function to write a property value that contains a single field.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also PdiWriteProperty, PdiWritePropertyFields, PdiWriteParameter, PdiWritePropertyBinaryValue



PdiWriterDelete

Purpose Delete a PDI output stream object.

Prototype

```
void PdiWriterDelete(UInt16 libRefnum,
PdiWriterType **ioWriter)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`<-> ioWriter`	A pointer to the PDI writer object, which was created by a previous call to the `PdiWriterNew` function.

Result Returns nothing.

Comments This function frees the memory that was allocated for the writer object. The `ioWriter` parameter is set to `NULL`.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiWriterNew`



PdiWriterNew

Purpose Initializes a new PDI writer object for use with the specified library number.

Prototype

```
PdiWriterType *PdiWriterNew(UInt16 libRefnum,
UDAWriterType *output, UInt16 version)
```

Parameters

`-> libRefnum`	The PDI library reference number.
`-> output`	The Unified Data Access (UDA) output object associated with the writer.

`-> version`	Options to control the behavior of the writer. You can use a combination of the Reader and Writer Options Constants.

Result Returns a pointer to the new PDI writer object. Returns `NULL` if the reader cannot be created.

Comments The `media` pointer is copied into a field in the writer object; thus, you do not need to retain your copy.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `PdiWriterDelete`, `PdiReaderNew`

79

Unified Data Access Manager

This chapter provides reference material for the Unified Data Access (UDA) Manager, which provides a mechanism for abstracting read and write access to different kinds of source and destination media, including memory and the Exchange Manager.

The Personal Data Interchange (PDI) reader and writer objects use UDA objects, and you must create UDA objects to use the PDI functions.

This chapter discusses the following topics:

- UDA Manager Data Structures
- UDA Manager Constants
- UDA Manager Functions
- UDA Object Creation Functions

The header file `UDAMgr.h` declares the Unified Data Access Manager API.

You use the UDA Manager in conjunction with the PDI library. For more information about the PDI library, see Chapter 78, "Personal Data Interchange Library."

Chapter 3, "Personal Data Interchange," in the *Palm OS Programmer's Companion*, vol. II, *Communications*, provides examples of using the UDA functions with the PDI library.

UDA Manager Data Structures



UDABufferSize

The `UDABufferSize` type is a simple typedef that defines the size of buffers used with UDA read functions.

```
typedef UInt16 UDABufferSize;
```



UDAObjectType

The `UDAObjectType` is the base class for all UDA objects, and defines the common properties of all of the objects.

```
typedef struct UDAObjectTag {
    UInt16                optionFlags;
    UDADeleteFunction     deleteF;
    UDAControlFunction    controlF;
} UDAObjectType
```

Field Descriptions

`optionFlags`	Options for the object. This is a combination of values described in Object Option Flags.
`deleteF`	The delete function associated with this UDA object.
`controlF`	The control function associated with this UDA object.



UDAFilterType

The `UDAFilterType` represents UDA Filters, which take input from a UDA Reader or UDA Filter, perform some encoding or decoding operation, and output the data to a memory buffer.

```
typedef struct UDAFilterTag {
    UInt16                optionFlags;
    UDADeleteFunction     deleteF;
    UDAControlFunction    controlF;
    UDAReadFunction       readF;
    UDAReaderType*        upperReader;
} UDAFilterType
```

Field Descriptions

`optionFlags`	Options for the object. This is a combination of values described in Object Option Flags.
`deleteF`	The delete function associated with this UDA object.
`controlF`	The control function associated with this UDA object.
`readF`	The read function associated with this UDA object.
`upperReader`	The `UDAReaderType` or `UDAFilterType` object that reads the data that this object outputs.



UDAReaderType

The `UDAReaderType` represents UDA Readers, which read input from a medium.

```
typedef struct UDAReaderTag {
    UInt16                optionFlags;
    UDADeleteFunction     deleteF;
    UDAControlFunction    controlF;
    UDAReadFunction       readF;
} UDAReaderType
```

Field Descriptions

`optionFlags`	Options for the object. This is a combination of values described in Object Option Flags.
`deleteF`	The delete function associated with this UDA object.
`controlF`	The control function associated with this UDA object.
`readF`	The read function associated with this UDA object.



New UDAWriterType

The `UDAWriterType` represents UDA Writers, which take data from a UDA Reader or UDA Filter and write the data to an output medium.

```
typedef struct UDAWriterTag {
    UInt16                optionFlags;
    UDADeleteFunction     deleteF;
    UDAControlFunction    controlF;
    UDAWriteFunction      initiateWriteF;
    UDAFlushFunction      flushF;
    UDAReaderType*        upperReader;
} UDAWriterType
```

Field Descriptions

`optionFlags`	Options for the object. This is a combination of values described in Object Option Flags.
`deleteF`	The delete function associated with this UDA object.
`controlF`	The control function associated with this UDA object.

`initiateWriteF`	The write function associated with this UDA object.
`flushF`	The flush function associated with this UDA object.
`upperReader`	The `UDAReaderType` object that reads the data that this object writes.

UDA Manager Constants

This section describes the constants used with the UDA Manager, which include the following constant types:

- Control Flags
- Error Constants
- Object Option Flags
- Miscellaneous Constants

Control Flags

Use the control flag constants to control UDA objects with the `UDAControl` macro.

Constant	Value	Description
`kUDAReinitialize`	`1`	Used with the `UDAControl` macro to reinitialize the UDA object.

Error Constants

At the time of this writing, there is only one error constant associated with the UDA object API.

Constant	Description
`udaErrControl`	Returned by the `UDAControl` macro when the control parameter is not valid for the UDA object.

Object Option Flags

You use the object option flag constants to determine information about the internal state of UDA objects. Note that the `UDAEndOfReader` and `UDAMoreData` macros provide you with a convenient means of accessing this same information.

Constant	Value	Description
`kUDAEndOfReader`	`1`	Indicates that the UDA reader has reached the end of its data.
`kUDAMoreData`	`2`	Indicates that the UDA reader needs more space in the read buffer to do its work.

Miscellaneous Constants

Constant	Value	Description
`kUDAZeroTerminatedBuffer`	`0xFFFF`	Indicates that the buffer is a null terminated string. Use this value when creating or reinitializing a `UDAMemoryReader` object.

UDA Manager Functions



New UDAControl

Purpose Applies controls to a UDA object.

Prototype

```
Err UDAControl (UDAObjectType* ioObjectP,
UInt16 parameter, va_args);
```

Parameters

`-> ioObjectP` A pointer to the `UDAObjectType` object that you want to control. This can be a `UDAReaderType`, a `UDAFilterType`, or a `UDAWriterType` object.

`-> parameter`	The control action that you want applied to the object.
`-> va_args`	Additional parameters, as required for the control and object type.

Result Returns `errNone` if no error, or `udaErrorClass` if the control parameter is not valid for the `ioObjectP`.

Comments The `UDAControl` function applies a control action to a UDA object. You may need to supply additional parameters, depending on the object type and control parameter values.

The only control action defined in Palm OS 4.0 is `kUDAReinitialize`. You can use it as shown in Table 79.1.

Table 79.1 UDAControl actions

Object Type	Usage	Action
`UDAExchangeReaderType`	`UDAControl(myExgRdr, kUDAReinitialize)`	Does nothing
`UDAExchangeWriterType`	`UDAControl(myExgWtr, kUDAReinitialize)`	Does nothing
`UDAMemoryReaderType`	`UDAControl(myMemRdr, kUDAReinitialize, bufferP, bufferSize)`	Reinstalls a new buffer for the memory reader. See `UDAMemoryReaderNew` for more information about the parameters.

Compatibility Implemented only if 4.0 New Feature Set is present.



UDADelete

Purpose Macro that deletes a UDA object.

Prototype

```
UDADelete (ioObjectP)
```

Parameters `-> ioObjectP` A pointer to the `UDAObjectType` object that you want to delete. This can be a `UDAReaderType`, a `UDAFilterType`, or a `UDAWriterType` object.

Result Returns nothing.

Comments The `ioObjectP` pointer is not valid after this macro completes.

Compatibility Implemented only if 4.0 New Feature Set is present.



New UDAEndOfReader

Purpose Macro that tests if the end of the reader has been reached.

Prototype `UDAEndOfReader (ioReaderP)`

Parameters `-> ioReaderP` A pointer to a `UDAReaderType` object.

Result Returns `true` if the end of the reader referenced by `ioReaderP` has been reached, and `false` if not.

Comments The end of the reader has been reached.

Compatibility Implemented only if 4.0 New Feature Set is present.



New UDAFilterJoin

Purpose Macro that joins a filter with a reader.

Prototype `UDAFilterJoin (ioFilterP, newReaderP)`

Parameters `--> ioFilterP` A pointer to a `UDAFilterType` object.

`--> newReaderP` A pointer to the `UDAReaderType` object with which you want the filter joined.

Result Returns nothing.

Comments Each `UDAFilterType` object receives its data from the `UDAReaderType` object to which it is joined; this relationship is established when you create the filter object. You can use this macro to change the reader with which the filter is joined. Upon completion, the filter referenced by `ioFilterP` is joined with the reader referenced by `newReaderP`.

Compatibility Implemented only if 4.0 New Feature Set is present.



New UDAInitiateWrite

Purpose Macro that causes the `UDAWriterType` object to read data and then write that data to output.

Prototype

```
UDAInitiateWrite (ioWriterP)
```

Parameters `-> ioWriterP` A pointer to a `UDAWriterType` object.

Result Returns `errNone` if successful, and an error code if not.

Comments When you use this macro, the `ioWriterP` reads data from the reader to which it is joined. It reads data until the reader is empty, and then writes the data to the output medium.

Compatibility Implemented only if 4.0 New Feature Set is present.



UDAMoreData

Purpose Macro that tests if there is more data available to read, but not enough room in the buffer to read it in.

Prototype
```
UDAMoreData (ioReaderP)
```

Parameters

`-> ioReaderP` A pointer to a `UDAReaderType` object.

Result Returns `true` if there is more data available for the reader and `false` if there is no more data available.

Comments You can use this macro with `UDAReaderType` objects to determine if there is more data waiting to read. This can happen when the reader's buffer is full.

Compatibility Implemented only if 4.0 New Feature Set is present.



UDARead

Purpose Macro that uses the specified `UDAReaderType` object to read data from the input source and place that data into the specified buffer.

Prototype
```
UDARead (ioReaderP, bufferToFillP, bufferSize,
errorP)
```

Parameters

`-> ioReaderP` A pointer to a `UDAReaderType` object that performs the read.

`-> bufferToFillP` A pointer to the buffer into which data is placed.

`-> bufferSize` The size of the buffer, in bytes.

`-> errorP`	A pointer to an `Err` value that represents the result of the read operation; if the operation is successful, the value is set to `errNone`.

Result Returns the number of bytes that were actually read. This value can be less than or equal to the value of `bufferSize`.

Comments The reader reads from the input source associated with the reader object and places the data into the specified buffer, reading no more than `bufferSize` bytes of data.

Compatibility Implemented only if 4.0 New Feature Set is present.



New UDAWriterFlush

Purpose Macro that flushes the buffer used by the `UDAWriterType` object.

Prototype
```
UDAWriterFlush (ioWriterP)
```

Parameters

`-> ioWriterP`	A pointer to a `_UDAWriterType` object.

Result Returns `errNone` if successful, and an error code if not.

Comments You can use this macro to flush any data remaining in the buffer of the writer object referenced by `ioWriterP`. This causes any data in the buffer to be sent to the output medium.

Compatibility Implemented only if 4.0 New Feature Set is present.



New UDAWriterJoin

Purpose Macro that joins a writer object to a different reader object.

Prototype
```
UDAWriterJoin (ioWriterP, newReaderP)
```

Parameters

`-> ioWriterP` A pointer to a `UDAWriterType` object.

`-> newReaderP` A pointer to the `UDAReaderType` object with which you want the writer joined.

Result Returns nothing.

Comments Each `UDAWriterType` object receives its data from the `UDAReaderType` object to which it is joined; this relationship is established when you create the writer object. You can use this macro to change the reader with which the writer is joined. Upon completion, the writer referenced by `ioWriterP` is joined with the reader referenced by `newReaderP`.

Compatibility Implemented only if 4.0 Feature Set is present.

UDA Object Creation Functions



New UDAExchangeReaderNew

Purpose Creates a new `UDAReaderType` object that you can use to read data from an Exchange Manager socket.

Prototype

```
UDAReaderType* UDAExchangeReaderNew
(ExgSocketPtr socket);
```

Parameters

`-> socket` A pointer to an `ExgSocketType` structure that describes the connection.

Result Returns a pointer to the newly created `UDAReaderType` object, or `NULL` if the reader could not be created.

Comments Use this function to create a UDA Reader object that takes input from an Exchange Manager socket.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `ExgSocketType`



UDAExchangeWriterNew

Purpose Creates a new `UDAWriterType` object that you can use to write data to an Exchange Manager socket.

Prototype

```
UDAWriterType* UDAExchangeWriterNew
(ExgSocketPtr socket, UDABufferSize bufferSize);
```

Parameters

`-> socket`	A pointer to an `ExgSocketType` structure that describes the connection.
`-> bufferSize`	The size, in bytes, of the buffer for the new writer object.

Result Returns a pointer to the newly created UDA Writer, or NULL if the writer could not be created.

Comments Use this function to create a UDA Writer object that sends output to an Exchange Manager socket.

Compatibility Implemented only if 4.0 New Feature Set is present.

See Also `ExgSocketType`



New UDAMemoryReaderNew

Purpose Creates a new `UDAReaderType` object that you can use to read data from a memory buffer.

Prototype

```
UDAReaderType* UDAMemoryReaderNew
(const UInt8* bufferP,
UDABufferSize bufferSizeInBytes);
```

Parameters

`-> bufferP`	A pointer to a buffer in memory from which data is read.
`-> bufferSize`	The size of the buffer, in bytes. If this value is equal to `kUDAZeroTerminatedBuffer`, `bufferP` must point to a null-terminated string buffer.

Result Returns a pointer to the newly created `UDAReaderType` object, or `NULL` if the reader could not be created.

Comments Use this function to create a reader that takes input from memory.

Compatibility Implemented only if 4.0 New Feature Set is present.



Part V: Appendixes

A

System Use Only Functions

This appendix lists functions that are purposely undocumented because they are for system use only.

WARNING! System Use Only.

```
AbtShowAbout
AlmAlarmCallback
AlmCancelAll
AlmDisplayAlarm
AlmEnableNotification
AlmInit
AlmTimeChange
Crc16CalcBigBlock
DmInit
DmResetRecordStates
ErrAlertCustom
EvtDequeueKeyEvent
EvtEnqueuePenPoint
EvtGetSysEvent
EvtInitialize
EvtSetKeyQueuePtr
EvtSetPenQueuePtr
EvtSysInit
ExgInit
ExgNotifyReceiveV35
ExpCardInserted
ExpCardRemoved
ExpInit
ExpSlotRegister
ExpSlotUnregister
FileReadLow
```

```
Find
FrmActiveState
FrmAddSpaceForObject
FrmGetUserModifiedState
FrmSetNotUserModified
FtrInit
GrfFieldChange
GrfFree
GrfInit
HostControl
INetLibSleep
INetLibSockMailAttrGet
INetLibSockMailAttrSet
INetLibSockMailQueryProgress
INetLibSockMailReqAdd
INetLibSockMailReqCreate
INetLibSockMailReqSend
INetLibWake
InsPtCheckBlink
InsPtInitialize
IntlInit
IrHandleEvent
IrWaitForEvent
KeyboardStatusNew
KeyboardStatusFree
KbdSetLayout
KbdGetLayout
KbdSetPosition
KbdGetPosiiton
KbdSetShiftState
KbdGetShiftState
KbdDraw
KbdErase
KbdHandleEvent
MemCardFormat
MemChunkFree
MemChunkNew
MemHandleFlags
MemHandleLockCount
MemHandleOwner
```

MemHandleResetLock
MemHeapFreeByOwnerID
MemHeapInit
MemInit
MemInitHeapTable
MemKernelInit
MemPtrFlags
MemPtrOwner
MemPtrResetLock
MemSemaphoreRelease
MemSemaphoreReserve
MemStoreSetInfo
NetLibConfigAliasGet
NetLibConfigAliasSet
NetLibConfigDelete
NetLibConfigIndexFromName
NetLibConfigList
NetLibConfigMakeActive
NetLibConfigRename
NetLibConfigSaveAs
NetLibHandlePowerOff
NetLibOpenConfig
NetLibOpenIfCloseWait
NetLibSleep
NetLibWake
PenClose
PenGetRawPen
PenOpen
PenRawToScreen
PenScreenToRaw
PenSleep
PenWake
ResLoadString
ScrCompressScanLine
ScrCopyRectangle
ScrDeCompressScanLine
ScrDrawChars
ScrDrawNotify
ScrInit
ScrLineRoutine

ScrRectangleRoutine
ScrScreenInfo
ScrSendUpdateArea
SerDbgAssureOpen
SerialMgrInstall
SerReceiveISP
SerReceiveWindowClose
SerReceiveWindowOpen
SerSetMapPort
SerSetWakeupHandler
SerSleep
SerWake
SlkProcessRPC
SlkSysPktDefaultResponse
SndInit
SndSetDefaultVolume
SrmSelectorErrPrv
SrmSleep
SrmWake
SysAppStartup
SysAppExit
SysBatteryDialog
SysCardImageDeleted
SysCardImageInfo
SysColdBoot
SysDisableInts
SysDoze
SysEvGroupCreate
SysEvGroupRead
SysEvGroupSignal
SysEvGroupWait
SysInit
SysKernelInfo
SysLaunchConsole
SysLCDBrightness
SysLCDContrast
SysLibClose
SysLibInstall
SysLibOpen
SysLibSleep

SysLibTblEntry
SysLibWake
SysMailboxCreate
SysMailboxDelete
SysMailboxFlush
SysMailboxSend
SysMailboxWait
SysNewOwnerID
SysPowerOn
SysResSemaphoreCreate
SysResSemaphoreDelete
SysResSemaphoreRelease
SysResSemaphoreReserve
SysRestoreStatus
SysSemaphoreCreate
SysSemaphoreDelete
SysSemaphoreSet
SysSemaphoreSignal
SysSemaphoreWait
SysSetA5
SysSetPerformance
SysSleep
SysTaskCreate
SysTaskDelete
SysTaskID
SysTaskResume
SysTaskSetTermProc
SysTaskSuspend
SysTaskSwitching
SysTaskTrigger
SysTaskWait
SysTaskWaitClr
SysTaskWake
SysTimerCreate
SysTimerDelete
SysTimerRead
SysTimerWrite
SysTranslateKernelErr
SysUIBusy
SysUILaunch

```
SysUnimplemented
SysWantEvent
TimInit
TxtPrepFindString
UIColorPopTable
UIColorPushTable
UIInitialize
UIReset
UIPopTable
UIPushTable
VFSInit
WinAddWindow
WinDrawWindowFrame
WinDisableWindow
WinEnableWindow
WinGetWindowPointer
WinInitializeWindow
WinMoveWindowAddr
WinRemoveWindow
WinScreenInit
WinSetColors
```

B

Compatibility Guide

This appendix lists groups of functions and other features (such as events and launch codes) that have been added to the Palm OS® after version 1.0.

Before you use any new functions or features in an application, you must check to ensure that they are implemented in the OS version your application is running on. Checking the OS version number is **not** a reliable indicator that a specific feature is present, since some later OS versions do not include features present in earlier versions. In order to ensure that your code is supported, you must check for the presence of individual features.

To make this check easier, this appendix lists new functions and features in groups such that all functions and features in a group are always implemented together in the ROM of a Palm™ device. This means that you can check for a single feature in that group and be assured that if that feature is present than all functions and features in that group are implemented.

Each group includes a recommended test to check if it is implemented. The following groups are described:

- 2.0 New Feature Set
- 3.0 New Feature Set
- 3.1 New Feature Set
- 3.2 New Feature Set
- International Feature Set
- Japanese Feature Set
- Wireless Internet Feature Set
- New Serial Manager Feature Set
- Connection Manager Feature Set
- 3.5 New Feature Set
- Notification Feature Set

- 4.0 New Feature Set
- Expansion Manager Feature Set
- VFS Manager Feature Set

2.0 New Feature Set

You can check that this feature set is implemented by checking that the system version is 2.0 or higher. Use this FtrGet call:

```
err = FtrGet(sysFtrCreator,
sysFtrNumROMVersion, &romversion);
```

The `romversion` parameter should be `0x02003000` or greater.

Launch Codes

This feature set adds the following launch codes:

sysAppLaunchCmdLookup
sysAppLaunchCmdPanelCalledFromApp
sysAppLaunchCmdReturnFromPanel
sysAppLaunchCmdSystemLock

Functions

This feature set adds the following functions:

CategoryInitialize
CategorySetName
DmDeleteCategory
DmDatabaseProtect
EvtAddUniqueEventToQueue
EvtEventAvail
EvtSysEventAvail
FldGetNumberOfBlankLines
FldGetScrollValues
FldSetInsertionPoint
FntGetScrollValues
FntWordWrap
FntWordWrapReverseNLines
FrmPointInTitle
FrmSetMenu
FrmSetObjectBounds

KeySetMask
LocGetNumberSeparators
LstScrollList
LstGetVisibleItems
MemCmp
MenuSetActiveMenuRscID
PhoneNumberLookup
PrefSetPreference
PrefGetPreference
SclDrawScrollBar
SclGetScrollBar
SclHandleEvent
SclSetScrollBar
SerControl
StrDelocalizeNumber
StrLocalizeNumber
StrNCaselessCompare
StrNCat
StrNCompare
StrNCopy
StrPrintF
StrVPrintF
SysBinarySearch
SysCreateDataBaseList
SysCreatePanelList
SysErrString
SysGraffitiReferenceDialog
SysLibLoad
SysStringByIndex
SysTicksPerSecond
TblHasScrollBar
TblSetBounds
TblSetColumnEditIndicator
TblSetRowStaticHeight
WinSetWindowBounds

Existing Functions that Changed

Several functions that existed in 1.0 were changed in 2.0:

CategoryCreateList (old function renamed CategoryCreateListV10)

CategoryEdit (old function renamed CategoryEditV10)

CategoryFreeList (old function renamed CategoryFreeListV10)

CategorySelect (old function renamed CategorySelectV10)

SelectDay (old function renamed SelectDayV10)

DmFindSortPosition (old function renamed DmFindSortPositionV10)

PrefGetAppPreferences (old function renamed PrefGetAppPreferencesV10)

PrefOpenPreferenceDB (old function renamed PrefOpenPreferenceDBV10)

PrefSetAppPreferences (old function renamed PrefSetAppPreferencesV10)

SerReceive (old function renamed SerReceive10)

SerSend (old function renamed SerSend10)

SysKeyboardDialog (old function renamed SysKeyboardDialogV10)

Other Changes

As a rule, all Palm OS applications developed with the 1.0 SDK should run error-free on the latest device. There are two possible pitfalls for 1.0 applications:

- **fldChangedEvent Change—**The operating system now correctly sends a `fldChangedEvent` whenever a field object is changed. Previously, the event was at times not sent, especially when a `FldSetText` operation was performed. If your application doesn't catch the events that are now sent, it may have problems.
- **Non-standard tools—**If your application was not developed with Metrowerks Code Warrior for the Palm OS, it may run into problems. One known problem can occur if the application:
  - was compiled with optimization turned on
  - uses system preferences

3.0 New Feature Set

You can check that this feature set is implemented by checking that the system version is 3.0 or higher. Use this FtrGet call:

```
err = FtrGet(sysFtrCreator,
sysFtrNumROMVersion, &romVersion);
```

The `romVersion` parameter should be greater than or equal to `0x03003000`, which can be constructed using the `sysMakeROMVersion` macro:

```
sysMakeROMVersion(3,0,0,sysROMStageRelease,0)
```

Launch Codes

This feature set adds the following launch codes:

sysAppLaunchCmdExgAskUser
sysAppLaunchCmdExgReceiveData

In addition, the launch code sysAppLaunchCmdGoto is now also sent by the exchange manager, in addition to its use by the global find operation.

Font

This feature set adds the following font:

`largeBoldFont`

Functions

This feature set adds the following functions:

Dynamic User Interface Functions

CtlNewControl
CtlValidatePointer
FldNewField
FrmNewBitmap
FrmNewForm
FrmNewGadget
FrmNewLabel
FrmRemoveObject
FrmValidatePtr
LstNewList
WinValidateHandle

For more information on creating and using dynamic user interface elements, see the section "Dynamic UI" on page 126 of the *Palm OS Programmer's Companion*, vol. I.

Font Functions

FontSelect
FntDefineFont

For more information on these functions and the support for custom fonts, see the section "Fonts in Palm OS 3.0 and Later" on page 139 of the *Palm OS Programmer's Companion*, vol. I.

Progress Manager Functions

PrgHandleEvent
PrgStartDialog
PrgStopDialog
PrgUpdateDialog
PrgUserCancel

For more information, see the section "Progress Dialogs" on page 86 of the *Palm OS Programmer's Companion*, vol. I.

File Streaming Functions

FileClearerr
FileClose
FileControl
FileDelete
FileDmRead
FileEOF
FileError
FileFlush
FileGetLastError
FileOpen
FileRead
FileReadLow (system use only)
FileRewind
FileSeek
FileTell
FileTruncate
FileWrite

For more information, see the section "File Streaming Application Program Interface" on page 191 of the *Palm OS Programmer's Companion*, vol. I.

Sound Functions

SndCreateMidiList
SndPlaySmf
SndDoCmd (enhanced in 3.0)

Exchange Manager Functions

ExgAccept
ExgDBRead
ExgDBWrite
ExgDisconnect
ExgPut
ExgReceive
ExgRegisterData
ExgSend

For more information, see the chapter Beaming (Infrared Communication) in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

IR Library Functions

IrAdvanceCredit
IrBind
IrClose
IrConnectIrLap
IrConnectReq
IrConnectRsp
IrDataReq
IrDisconnectIrLap
IrDiscoverReq
IrIAS_Add
IrIAS_GetInteger
IrIAS_GetIntLsap
IrIAS_GetObjectID
IrIAS_GetOctetString
IrIAS_GetOctetStringLen
IrIAS_GetType
IrIAS_GetUserString
IrIAS_GetUserStringCharSet
IrIAS_GetUserStringLen
IrIAS_Next
IrIAS_Query
IrIAS_SetDeviceName
IrIAS_StartResult
IrIsIrLapConnected
IrIsMediaBusy
IrIsNoProgress
IrIsRemoteBusy
IrLocalBusy
IrMaxRxSize
IrMaxTxSize
IrOpen
IrSetConTypeLMP
IrSetConTypeTTP
IrSetDeviceInfo
IrTestReq
IrUnbind

For more information, see the chapter Beaming (Infrared Communication) in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

Miscellaneous Functions

FrmRestoreActiveState	SysGetROMToken
FrmSaveActiveState	SysGetStackInfo
ScrDisplayMode	SysGremlins
SysGetAppInfo (system use only)	TblGetItemFont
SysGetOSVersionString	TblSetItemFont

Existing Functions that Changed

Two functions that existed in 2.0 were changed in 3.0:

CategoryEdit (old function renamed CategoryEditV20)

SysBatteryInfo (old function renamed SysBatteryInfoV20)

Other Changes

- The dynamic heap has been increased in size to 96 KB.
- Storage RAM is no longer subdivided into multiple storage heaps of 64 KB each. All storage RAM on a memory card is configured as a single storage heap.
- Each flash ROM-based Palm device holds a serial number that identifies it uniquely and can be retrieved via SysGetROMToken. For more information, see "Retrieving the ROM Serial Number" on page 304 of the *Palm OS Programmer's Companion*, vol. I.
- The Application Launcher (accessed via the silkscreen "Applications" button) is now an application, rather than a popup. The SysAppLauncherDialog function, which provides the API to the old popup launcher, is still present in Palm OS 3.0 for compatibility purposes, but has not been updated and generally should not be used. For more information, see "Application Launcher" on page 147 of the *Palm OS Programmer's Companion*, vol. I.
- The sound manager supports MIDI sound files, adding new sounds, asynchronous playback, and other features. There are also new selectors for setting the volume preferences. For more information, see the section "Sound" on page 288 of the *Palm OS Programmer's Companion*, vol. I.

The following functions existed in the system previously, but were not documented:

RctCopyRectangle
RctGetIntersection
RctInsetRectangle
RctOffsetRectangle
RctPtInRectangle
RctSetRectangle

The following event type existed in the system previously, but was not previously documented:

frmGotoEvent

3.1 New Feature Set

You can check that this feature set is implemented by checking that the system version is 3.1 or higher. Use this FtrGet call:

```
err = FtrGet(sysFtrCreator,
sysFtrNumROMVersion, &romVersion);
```

The romVersion parameter should be greater than or equal to 0x03103000, which can be constructed using the sysMakeROMVersion macro:

```
sysMakeROMVersion(3,1,0,sysROMStageRelease,0)
```

Functions

This feature set adds the following functions:

ChrHorizEllipsis
ChrNumericSpace
ContrastAdjust
FntWidthToOffset
FtrPtrNew
FtrPtrFree
FtrPtrResize
SelectOneTime
WinDrawChar
WinDrawTruncChars

NOTE: The `PalmOSGlue.lib` provides compatibility functions and macros for `ChrHorizEllipsis`, `ChrNumericSpace`, `WinDrawChar`, and `WinDrawTruncChars`. If you want to use these functions on systems that don't have the 3.1 feature set, you can link your application with `PalmOSGlue.lib`. See the chapter "PalmOSGlue Library" on page 1671 for more information.

Changes to the Character Encoding

Starting in Palm OS 3.1, the character encoding used on most systems is Microsoft Windows code page 1252. Versions prior to 3.1 used an encoding that was very similar to code page 1252 but did not follow it exactly. The following changes to the character set are introduced in Palm OS 3.1:

- Some of the special Palm OS glyphs in the high ASCII range (such as the shortcut stroke and the command stroke) have been moved down into the control code range, and other characters (such as the numeric space and horizontal ellipsis) have been copied into the control range so that they're guaranteed to exist in every encoding. For the numeric space and horizontal ellipsis, you can use the macros `ChrNumericSpace` and `ChrHorizEllipsis` to return the appropriate character regardless of the character map. In `PalmOSGlue.lib`, these two macros are named `TxtGlueGetNumericSpaceChar` and `TxtGlueGetHorizEllipsisChar`, respectively.
- The four playing-card characters have been moved from the high ASCII range in the standard four fonts to the 9-point Symbol font.

Other Changes in 3.1

- Palm OS 3.1 supports a new processor: the EZ Dragonball processor. This processor is compatible with the existing Dragonball processor, so your application should run

without changes as long as it doesn't access registers or system globals directly.

If your application needs to know if it is running on an EZ Dragonball, it can check using the following code:

```
DWord id, chip;
Word revision;
Err err;
err = FtrGet(sysFtrCreator,
      sysFtrNumProcessorID, &id);
if (!err) {
  chip = id & sysFtrNumProcessorMask;
  revision = id & 0x0ffff;
  if (chip==sysFtrNumProcessor328)
    // traditional Dragonball
  else if (chip==sysFtrNumProcessorEZ)
    // Dragonball EZ
}
```

- The constant `preferenceDataVersion` was removed and replaced with `preferenceDataVerLatest`.
- Character variables are now two bytes long. The type `WChar` defines a character variable.
- The `keyDownEvent` structure's `chr` field (which contains the input character) has been changed from a `Word` to a `WChar`.
- The string manager functions `StrChr` and `StrStr` now treat buffers as characters, not arbitrary byte arrays. If you previously used these functions to search data buffers, your code may no longer work.
- The string manager function `StrToLower` can now handle any type of characters, including accented characters.
- The underline attribute of `FieldAttrType` now has support for the value 2. Previously, the only underline modes available were no underline (0) and gray underline (1). In Palm OS 3.1 and higher, the value 2 is interpreted as solid underline. The `UnderlineModeType` enum defined in `Window.h` defines the possible values for the underline attribute.

- The use of the DmGetNextDatabaseByTypeCreator onlyLatestVers parameter changed in 3.1. If onlyLatestVers is true, you only receive one matching database for each type/creator pair. In version 3.0 and earlier, you could receive multiple matching databases if onlyLatestVers was true. See that function's description for a more detailed description.

3.2 New Feature Set

You can check that this feature set is implemented by checking that the system version is 3.2 or higher. Use this FtrGet call:

```
err = FtrGet(sysFtrCreator,
sysFtrNumROMVersion, &romVersion);
```

The romVersion parameter should be greater than or equal to 0x03203000, which can be constructed using the sysMakeROMVersion macro:

```
sysMakeROMVersion(3,2,0,sysROMStageRelease,0)
```

Functions

This feature set adds the following functions:

AlmGetProcAlarm
AlmSetProcAlarm
ClipboardAppendItem
DmGetDatabaseLockState
ErrAlert
SndPlaySmfResource

Existing Functions that Changed

Two functions that existed in 3.0 were changed in 3.2:

SysGremlins was removed and replaced with a SysGremlins macro that maps it to the function HostGremlinIsRunning. The prototype is slightly different, but you can still call SysGremlins in the same way you did before.

PrgStartDialog (old function renamed PrgStartDialogV31)

Other Changes in 3.2

- The prototype for the system use only function `AlmDisplayAlarm` changed from no return value to a Boolean return value. This change may affect system patches and extensions that intercept `AlmDisplayAlarm` calls.

International Feature Set

You can check that this feature set is implemented by checking for the existence of the international manager. You can check by calling `FtrGet` as follows:

```
err = FtrGet(sysFtrCreator, sysFtrNumIntlMgr,
&value);
```

If the international manager is installed, the `value` parameter will be non-zero and the returned error should also be zero (for no error).

You can learn more about the international manager by reading the chapter "Localized Applications" on page 313 in the *Palm OS Programmer's Companion*, vol. I.

NOTE: If you want to use international functions on systems that don't have the international feature, you can link your application with `PalmOSGlue.lib`. The functions in this library are the same as those listed below except that they use the prefix "TxtGlue" instead of "Txt." For more information, see the chapter "PalmOSGlue Library" on page 1671.

Functions

This feature set adds the following functions:

Text Manager Functions

TxtByteAttr
TxtCaselessCompare
TxtCharAttr
TxtCharBounds
TxtCharEncoding
TxtCharIsAlNum
TxtCharIsAlpha
TxtCharIsCntrl
TxtCharIsDigit
TxtCharIsGraph
TxtCharIsHardKey
TxtCharIsHex
TxtCharIsLower
TxtCharIsPrint
TxtCharIsPunct
TxtCharIsSpace
TxtCharIsUpper
TxtCharSize
TxtCharXAttr
TxtCompare
TxtEncodingName
TxtFindString
TxtGetChar
TxtGetNextChar
TxtGetPreviousChar
TxtCharIsValid
TxtMaxEncoding
TxtNextCharSize
TxtPreviousCharSize
TxtReplaceStr
TxtSetNextChar
TxtStrEncoding
TxtTransliterate
TxtGetTruncationOffset
TxtWordBounds

Other Functions

IntlGetRoutineAddress

Removed Functions and Macros

If the international feature set exists, then the following functions and macros are no longer available:

GetCharAttr
GetCharCaselessValue
GetCharSortValue
IsAscii
IsAlNum
IsAlpha
IsCntrl
IsDigit
IsGraph
IsLower
IsPrint
IsPunct
IsSpace
IsUpper
IsHex
IsDelim

Japanese Feature Set

You can check that the Japanese feature set is implemented by checking if the unit is Japanese. You can check by calling FtrGet as follows:

```
err = FtrGet(sysFtrCreator, sysFtrNumEncoding,
&value);
```

The unit has the Japanese OS if the `value` parameter is `charEncodingCP932`.

For further information about the Japanese implementation, see the section "Notes on the Japanese Implementation" in the *Palm OS Programmer's Companion*, vol. I.

Wireless Internet Feature Set

You can check that this feature set is implemented by checking for the existence of the Web Clipping Application Viewer (*Viewer*) and

iMessenger™ applications. Here's an example of how to check for Viewer:

```
DmSearchStateType searchState;
UInt cardNo;
LocalID dbID;
err = DmGetNextDatabaseByTypeCreator(true,
&searchState, sysFileTApplication,
sysFileCClipper, true, &cardNo, &dbID);
```

If Viewer is not present, the `DmGetNextDatabaseByTypeCreator` routine returns an error. To check for iMessenger, you can use the creator type `sysFileCMessaging`.

NOTE: The Viewer was formerly described as the Clipper.

You can learn more about the Palm.Net™ system for wireless Internet access and the programmatic interfaces to the Viewer and iMessenger applications by reading the chapter "Internet and Messaging Applications" in the *Palm OS Programmer's Companion*, vol. II, *Communications*. For a more complete description, see the *Web Clipping Developer's Guide*.

Launch Codes

This feature set adds the following launch codes:

`sysAppLaunchCmdAddRecord` (for iMessenger application; existed for Mail in 3.0)
`sysAppLaunchCmdGoToURL`
`sysAppLaunchCmdOpenDB`
`sysAppLaunchCmdURLParams`

Events

This feature set adds the following events:

`inetSockReadyEvent`
`inetSockStatusChangeEvent`

This feature set also adds the following `keyDownEvent` key codes:

```
vchrHardAntenna
vchrRadioCoverageOK
vchrRadioCoverageFail
```

These key codes are described in the section Wireless keyDownEvent Key Codes.

Functions

This feature set adds the following functions.

Internet Library Functions

For more information, see the chapter "Network Communication" in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

INetLibCacheGetObject	INetLibSockClose
INetLibCacheList	INetLibSockConnect
INetLibCheckAntennaState	INetLibSockHTTPAttrGet
INetLibClose	INetLibSockHTTPAttrSet
INetLibConfigAliasGet	INetLibSockHTTPReqCreate
INetLibConfigAliasSet	INetLibSockHTTPReqSend
INetLibConfigDelete	INetLibSockOpen
INetLibConfigIndexFromName	INetLibSockRead
INetLibConfigList	INetLibSockSettingGet
INetLibConfigMakeActive	INetLibSockSettingSet
INetLibConfigRename	INetLibSockStatus
INetLibConfigSaveAs	INetLibURLCrack
INetLibGetEvent	INetLibURLGetInfo
INetLibOpen	INetLibURLOpen
INetLibSettingGet	INetLibURLsAdd
INetLibSettingSet	INetLibWiCmd

New Serial Manager Feature Set

The New Serial Manager feature set has two different versions.

New Serial Manager Feature Set Version 1

You can check that this feature set is implemented by checking for the existence of the new Serial Manager. You can check by calling FtrGet as follows:

```
err = FtrGet(sysFileCSerialMgr,
sysFtrNewSerialPresent, &value);
```

If the new Serial Manager is installed, the `value` parameter will be non-zero and the returned error should also be zero (for no error).

You can learn more about the new Serial Manager and Connection Manager by reading the sections "The Serial Manager" on page 92 and "The Connection Manager" on page 116 in the *Palm OS Programmer's Companion*, vol. II, *Communications*.

This feature set adds the following functions.

Serial Manager Functions

SrmClearErr
SrmClose
SrmControl
SrmGetDeviceCount
SrmGetDeviceInfo
SrmGetStatus
SrmOpen
SrmOpenBackground
SrmPrimeWakeupHandler
SrmReceive
SrmReceiveCheck
SrmReceiveFlush
SrmReceiveWait
SrmReceiveWindowClose
SrmReceiveWindowOpen
SrmSend
SrmSendCheck
SrmSendFlush
SrmSendWait
SrmSetReceiveBuffer
SrmSetWakeupHandler
WakeupHandlerProcPtr

Virtual Driver Functions

DrvEntryPoint
GetSizeProcPtr
GetSpaceProcPtr
VdrvControlProcPtr
VdrvOpenProcPtr
VdrvStatusProcPtr
VdrvWriteProcPtr
WriteBlockProcPtr
WriteByteProcPtr

Connection Manager Functions

CncAddProfile
CncDeleteProfile
CncGetProfileInfo
CncGetProfileList

Serial Link Manager Function

SlkSocketPortID

New Serial Manager Feature Set Version 2

You can check that version 2 of the new Serial Manager feature set is implemented by checking the Serial Manager version number and the Palm OS version number. You can check by calling FtrGet as follows:

```
err = FtrGet(sysFileCSerialMgr,
  sysFtrNewSerialVersion, &value);
err = FtrGet(sysFtrCreator,
  sysFtrNumROMVersion, &romVersion);
```

The new Serial Manager is present if:

- Both calls to `FtrGet` return zero (for no error).
- The `value` parameter is 2.
- The `romVersion` parameter is `0x04003000`, which can be constructed using the `sysMakeROMVersion` macro:

  `sysMakeROMVersion(4,0,0,sysROMStageRelease,0)`

This feature set adds the following functions.

Serial Manager Functions

SrmCustomControl
SrmExtOpen
SrmExtOpenBackground

Virtual Driver Functions

`VdrvControlCustomProcPtr` `VdrvOpenProcV4Ptr` `SignalCheckPtr`

IMPORTANT: Some Handspring devices ship with Palm OS version 3.5 but have new Serial Manager feature set version 2. These devices support the `SignalCheckPtr` virtual driver function and have expanded functionality for USB support, but they do not support the other function calls in this feature set.

You can learn more about the new Serial Manager by reading the chapter "Serial Communication" on page 89 of *Palm OS Programmer's Companion*, vol. II, *Communications*.

Connection Manager Feature Set

You can check that the Connection Manager feature set is implemented by checking the value of the Connection Manager feature. You can check by calling `FtrGet` as follows:

```
err = FtrGet(kCncFtrCncMgrCreator,
kCncFtrCncMgrVersion, &version);
```

The `version` parameter should be greater than or equal to `0x00040001`. In the 4.0 Palm OS SDK, this value is represented by the `kCncMgrVersion` constant.

NOTE: An earlier version of the Connection Manager is available if New Serial Manager Feature Set Version 1 is present.

Functions

This feature set adds the following functions:

`CncProfileCloseDB`
`CncProfileCount`
`CncProfileCreate`
`CncProfileDelete`
`CncProfileGetCurrent`
`CncProfileGetIDFromIndex`
`CncProfileGetIDFromName`
`CncProfileGetIndex`
`CncProfileOpenDB`
`CncProfileSetCurrent`
`CncProfileSettingGet`
`CncProfileSettingSet`

3.5 New Feature Set

You can check that this feature set is implemented by checking that the system version is 3.5 or higher. Use this FtrGet call:

```
err = FtrGet(sysFtrCreator,
sysFtrNumROMVersion, &romVersion);
```

The romVersion parameter should be greater than or equal to 0x03503000, which can be constructed using the sysMakeROMVersion macro:

```
sysMakeROMVersion(3,5,0,sysROMStageRelease,0)
```

Launch Codes

This feature set adds the following launch codes:

sysAppLaunchCmdNotify

Events

This feature set adds the following events:

frmGadgetEnterEvent
frmGadgetMiscEvent
menuCmdBarOpenEvent
menuOpenEvent

Functions

This feature set adds the following functions.

Bitmaps

BmpBitsSize
BmpColortableSize
BmpCompress
BmpCreate
BmpDelete
BmpGetBits
BmpGetColortable
BmpSize
ColorTableEntries

For more information on creating and using bitmaps, see the section "Bitmaps" on page 118 of the *Palm OS Programmer's Companion*, vol. I.

Controls

CtlGetSliderValues
CtlNewGraphicControl
CtlNewSliderControl
CtlSetGraphics
CtlSetSliderValues

These functions add support for graphical buttons and slider controls. For more information, see the section "Controls" on page 88 of the *Palm OS Programmer's Companion*, vol. I.

Forms

FrmCustomResponseAlert
FrmNewGsi
FrmSetGadgetHandler

Among the changes to form functions is extended gadget support. For more information on gadgets and extended gadgets, see the section "Custom UI Objects" on page 124 of the *Palm OS Programmer's Companion*, vol. I.

Menus

MenuAddItem
MenuCmdBarDisplay
MenuHideItem
MenuCmdBarAddButton
MenuCmdBarGetButtonData
MenuShowItem

For more information on using menu functions, see the section "Menus" on page 101 of the *Palm OS Programmer's Companion*, vol. I.

Overlay Manager

OmGetCurrentLocale
OmGetIndexedLocale
OmGetRoutineAddress
OmGetSystemLocale
OmLocaleToOverlayDBName
OmOverlayDBNameToLocale
OmSetSystemLocale

For more information on using the overlay manager, see the section "Maintaining Backward Compatibility" on page 315 of the *Palm OS Programmer's Companion*, vol. I.

Private Records

SecSelectViewStatus
SecVerifyPW

Tables

TblGetItemPtr
TblRowMasked
TblSetColumnMasked
TblSetRowMasked

UI Colors

UIColorGetTableEntryIndex
UIColorGetTableEntryRGB
UIColorSetTableEntry

For more information on using the UI Colors API, see the section "Color and Grayscale Support" on page 128 of the *Palm OS Programmer's Companion*, vol. I.

UI Controls

UIBrightnessAdjust
UIPickColor

Windows

WinCreateBitmapWindow
WinDrawPixel
WinErasePixel
WinGetBitmap
WinGetPatternType
WinGetPixel
WinIndexToRGB
WinInvertPixel
WinPaintBitmap
WinPaintChar
WinPaintChars
WinPaintLine
WinPaintLines
WinPaintPixel
WinPaintPixels
WinPaintRectangle
WinPaintRectangleFrame
WinPalette
WinPopDrawState
WinPushDrawState
WinRGBToIndex
WinScreenLock
WinScreenMode
WinScreenUnlock
WinSetBackColor
WinSetDrawMode
WinSetForeColor
WinSetPatternType
WinSetTextColor

For more information on using the window functions, see the section "Drawing on the Palm OS Device" on page 81 of the *Palm OS Programmer's Companion*, vol. I.

Miscellaneous New Functions

DmOpenDBNoOverlay
ExgDoDialog
DateToAscii
ResLoadConstant
TxtParamString

Existing Functions that Changed

The following functions that existed prior to 3.5 have changed in release 3.5:

`ScrDisplayMode` was changed to `WinScreenMode`.

`ContrastAdjust` was changed to `UIContrastAdjust`.

`SelectTime` (old function renamed `SelectTimeV33`)

New Data Types

The data types `Byte`, `Word`, `DWord` and so on are now deprecated. It is recommend that you use the corresponding new data types. For example, use `Int16` instead of `SWord` and `UInt32` instead of `DWord`. In particular, the unfortunate distinction between `Handle`/`VoidHand` has been fixed; use `MemHandle` instead.

To learn in general how the type names changed, see the header file `PalmOSCompatibility.h`. This file provides a mapping from the old type name to the new type name. If you need to move forward without modifying your code, you can include this file in your project to provide declarations for the old type names.

Changes to Events

- The `tapCount` field has been added to the `EventType` structure. The `tapCount` field specifies the number of times the user tapped the pen at the current location; in fields, two taps selects a word, and three taps selects a line.

IMPORTANT: Because the `tapCount` field has been added to the `EventType` structure, it has become more critical that you clear the event structure before using it to add a new event to the queue. Otherwise, the `tapCount` will be incorrect for the new event.

- The structures for `ctlRepeatEvent` and `ctlSelectEvent` have a `value` field added to them. This new field is used only for sliders; it holds the current value of the slider.
- Form objects now handle the `frmTitleSelectEvent` by adding a `keyDownEvent` with the `vchrMenu` character to the event queue (which causes the form's menu to display).
- Some of the structure definitions for system-level events have moved from `Event.h` to `SysEvent.h`.
- The `winEnterEvent` is now not generated until `FrmDrawForm` is called. Make sure to draw your form in response to `frmOpenEvent`, not `winEnterEvent`.
- `EvtSetNullEventTick` is now a function. In previous releases it was a macro.

Other Changes

- `FrmDrawForm`

  On release 3.5, `FrmDrawForm` erases the window's rectangle before it draws, so you must perform custom drawing after the call to `FrmDrawForm`, not before. If you have drawn before the call to `FrmDrawForm`, your changes are lost. On debug ROMs, the window handle is invalid until `FrmDrawForm` is called so that draws before `FrmDrawForm` result in a bus error.

- Resource Manager

  The resource manager functions have been updated to work with overlay databases. See "Maintaining Backward Compatibility" on page 315 in the *Palm OS Programmer's Companion*, vol. I.

- DmGetDatabase

  The order in which this call returns databases has changed. Previously all of the databases from ROM were returned first, then all from RAM. Now they are intermingled. Developers should not rely on the order in which databases are returned from this call.

- StrToLower

  This function is different in 3.5 Latin ROMs. Previously it only changed A through Z. Now it also changes high ASCII characters.

- Time Manager

  If you are using a debug ROM, the string buffer is filled with `dateStringLength` or `longStrLength` debugging bytes, depending on the `dateFormat` parameter. For the routines that return the day-of-week name in addition to the date, the size of the buffers has been expanded, so developers need to check the max lengths defined in `DateTime.h`.

- The format of the storage heap header has changed, thus any existing saved Simulator card images are invalid and should be tossed.

- Category Data Structures

  The data structure `AppInfoType` has been documented.

  `CategoryCreateList` now has a "hide" function with two new constants; `categoryHideEditCategory, and categoryDefaultEditCategoryString`.

- FtrPtrNew

  `FtrPtrNew` now allows allocating chunks larger than 64KB.

- Dynamic heap

  The dynamic heap is now sized based on the amount of memory available to the system.

Table B.1 Dynamic heap size

Device RAM Size	Heap Size
$x < 2MB$	64KB
$2MB \leq x < 4MB$	128KB
$x \geq 4MB$	256KB

Notification Feature Set

You can check that this feature set is implemented by checking for the existence of the notification manager. You can check by calling FtrGet as follows:

```
err = FtrGet(sysFtrCreator,
sysFtrNumNotifyMgrVersion, &value);
```

If the notification manager is part of the system, the `value` parameter will be non-zero and the returned error should also be zero (for no error).

Notification Manager

SysNotifyBroadcast
SysNotifyBroadcastDeferred
SysNotifyRegister
SysNotifyUnregister

To learn more about the notification manager, see the section "Notifications" on page 46 of the *Palm OS Programmer's Companion*, vol. I.

4.0 New Feature Set

You can check that this feature set is implemented by checking that the system version is 4.0 or higher. Use this FtrGet call:

```
err = FtrGet(sysFtrCreator,
sysFtrNumROMVersion, &romVersion);
```

The `romVersion` parameter should be greater than or equal to `0x04003000`, which can be constructed using the `sysMakeROMVersion` macro:

```
sysMakeROMVersion(4,0,0,sysROMStageRelease,0)
```

Launch Codes

This feature set adds the following launch codes:

sysAppLaunchCmdAttention
sysAppLaunchCmdExgGetData
sysAppLaunchCmdExgPreview

Notifications

This feature set adds the following notifications:

cncNotifyProfileEvent
sysExternalConnectorAttachEvent
sysExternalConnectorDetachEvent
sysNotifyCardInsertedEvent
sysNotifyCardRemovedEvent
sysNotifyDBDeletedEvent
sysNotifyDeleteProtectedEvent
sysNotifyDeviceUnlocked
sysNotifyGotUsersAttention
sysNotifyHelperEvent
sysNotifyLocaleChangedEvent
sysNotifyNetLibIFMediaEvent
sysNotifyRetryEnqueueKey
sysNotifyVolumeMountedEvent
sysNotifyVolumeUnmountedEvent

Functions

This feature set adds the following functions:

Attention Manager

AttnDoSpecialEffects
AttnForgetIt
AttnGetAttention
AttnGetCounts
AttnIndicatorEnable
AttnIndicatorEnabled
AttnIterate
AttnListOpen
AttnUpdate

Date and Time Manager

TimeZoneToAscii
TimTimeZoneToUTC
TimUTCToTimeZone

Exchange Manager

ExgGetTargetApplication
ExgGetDefaultApplication
ExgGetRegisteredApplications
ExgSetDefaultApplication
ExgControl
ExgNotifyGoto
ExgNotifyPreview
ExgRequest
ExgGetRegisteredTypes

For more information on the Exchange Manager, see the chapter Chapter 1, "Object Exchange," on page 1 of the *Palm OS Programmer's Companion*, vol. II, *Communications*.

Exchange Library

ExgLibAccept
ExgLibClose
ExgLibConnect
ExgLibControl
ExgLibDisconnect
ExgLibGet
ExgLibHandleEvent
ExgLibOpen
ExgLibPut
ExgLibReceive
ExgLibRequest
ExgLibSend
ExgLibSleep
ExgLibWake

For more information on the Exchange Library API, see Chapter 59, "Exchange Library," on page 1179 of the *Palm OS Programmer's Companion*, vol. II, *Communications*.

Locale Manager

LmGetLocaleSetting
LmGetNumLocales
LmLocaleToIndex

Miscellaneous UI

PhoneNumberLookupCustom

Notification Manager

SysNotifyBroadcastFromInterrupt

PDI Library Functions

PdiDefineReaderDictionary
PdiDefineResizing
PdiEnterObject
PdiLibClose
PdiLibOpen
PdiParameterPairTest
PdiReaderDelete
PdiReaderNew
PdiReadParameter
PdiReadProperty
PdiReadPropertyField
PdiReadPropertyName
PdiSetEncoding
PdiWriteBeginObject
PdiWriteEndObject
PdiWriteParameter
PdiWriteParameterStr
PdiWriteProperty
PdiWritePropertyBinaryValue
PdiWritePropertyFields
PdiWritePropertyStr
PdiWritePropertyValue
PdiWriterDelete
PdiWriterNew

For more information, see the chapter Chapter 3, "Personal Data Interchange" in the *Palm OS Programmer's Companion*.

Sound Manager Functions

SndInterruptSmfIrregardless
SndPlaySmfIrregardless
SndPlaySmfResourceIrregardless

For more information, see the chapter Chapter 10, "Palm System Support" in the *Palm OS Programmer's Companion*.

Telephony Manager Functions

TelCancel
TelCfgGetPhoneNumber
TelCfgGetSmsCenter
TelCfgGetSmsCenter
TelClose
TelClosePhoneConnection
TelDtcCallNumber
TelDtcCloseLine
TelDtcReceiveData
TelDtcSendData
TelEmcCall
TelEmcCloseLine
TelEmcGetNumber
TelEmcGetNumberCount
TelEmcSelectNumber
TelEmcSetNumber
TelGetCallState
TelGetEvent
TelGetTelephonyEvent
TelInfGetInformation
TelIs<FunctionName>Supported
TelIs<ServiceSet>Available
TelIsCfgServiceAvailable
TelIsDtcServiceAvailable
TelIsEmcServiceAvailable
TelIsInfServiceAvailable
TelIsNwkServiceAvailable
TelIsOemServiceAvailable
TelIsPhbServiceAvailable
TelIsPhoneConnected
TelIsPowServiceAvailable
TelIsSmsServiceAvailable
TelIsSndServiceAvailable
TelIsSpcServiceAvailable
TelIsStyServiceAvailable
TelMatchPhoneDriver

TelNwkGetLocation
TelNwkGetNetworkName
TelNwkGetNetworks
TelNwkGetNetworkType
TelNwkGetSearchMode
TelNwkGetSelectedNetwork
TelNwkGetSignalLevel
TelNwkSelectNetwork
TelNwkSetSearchMode
TelOemCall
TelOpen
TelOpenPhoneConnection
TelPhbAddEntry
TelPhbDeleteEntry
TelPhbGetAvailablePhonebooks
TelPhbGetEntries
TelPhbGetEntry
TelPhbGetEntryCount
TelPhbGetEntryMaxSizes
TelPhbGetSelectedPhonebook
TelPhbSelectPhonebook
TelPowGetBatteryStatus
TelPowGetPowerLevel
TelPowSetPhonePower
TelSendCommandString

TelSmsDeleteMessage
TelSmsGetAvailableStorage
TelSmsGetDataMaxSize
TelSmsGetMessageCount
TelSmsGetSelectedStorage
TelSmsGetUniquePartId
TelSmsReadMessage
TelSmsReadMessages
TelSmsReadReport
TelSmsReadReports
TelSmsReadSubmittedMessage
TelSmsReadSubmittedMessages
TelSmsSelectStorage
TelSmsSendManualAcknowledge
TelSmsSendMessage
TelSndMute
TelSndPlayKeyTone
TelSndStopKeyTone
TelSpcAcceptCall
TelSpcCallNumber
TelSpcCloseLine
TelSpcConference
TelSpcGetCallerNumber
TelSpcHoldLine
TelSpcPlayDTMF
TelSpcRejectCall
TelSpcRetrieveHeldLine
TelSpcSelectLine
TelSpcSendBurstDTMF
TelSpcStartContinuousDTMF
TelSpcStopContinuousDTMF
TelStyChangeAuthenticationCode
TelStyEnterAuthenticationCode
TelStyGetAuthenticationState

For more information about the Telephony Manager, see Chapter 8, "Telephony Manager" in the *Palm OS Programmer's Companion*.

Windows Manager

WinGetPixelRGB
WinSetBackColorRGB
WinSetForeColorRGB
WinSetTextColorRGB

Internationalization Functions

The following functions have been added to aid with creating localized applications:

FntWCharWidth
OmGetNextSystemLocale
StrCompareAscii
TxtConvertEncoding
TxtGetWordWrapOffset
TxtNameToEncoding
IntlSetRoutineAddress

Existing Functions that Changed

The following functions that existed prior to 4.0 have changed in release 4.0:

`ExgRegisterData` has been deprecated. The preferred function is now `ExgRegisterDatatype`.

`ExgGet` was previously unimplemented. It has been implemented in release 4.0

`ExgNotifyReceive` was previously a private function. It is now a public function, intended to be used by exchange libraries.

`FldRecalculateField` now updates the word-wrapping information whenever `FldRecalculateField` is called, regardless of the value of the `redraw` parameter. Prior to Palm OS 4.0 it updated the word-wrapping information only if the `redraw` parameter was set to `true`.

`StrNCaselessCompare` now requires both of its string parameters to be null-terminated.

`StrNCompare` now requires both of its string parameters to be null-terminated.

Expansion Manager Feature Set

Because not every system has (or needs) Expansion Manager services, applications wishing to use these services should check to make sure they are present before calling them. This is accomplished by checking for the Expansion Manager's system feature with a call to `FtrGet`, supplying `sysFileCExpansionMgr` for the feature creator and `expFtrIDVersion` for the feature number.

The following code shows how to check for the presence and proper version of the Expansion Manger. Note that `expectedExpMgrVersionNum` should be replaced by the actual version number you expect.

```
UInt32 expMgrVersion;
Err err;

err = FtrGet(sysFileCExpansionMgr,
  expFtrIDVersion, &expMgrVersion);
if(err){
    // Expansion Manager not installed
} else {
    // check version number of Expansion Manager,
    // if necessary
    if(expMgrVersion == expectedExpMgrVersionNum)
        // everything is OK
}
```

You can learn more about the expansion manager by reading Chapter 30, "Expansion Manager," on page 593.

Functions

This feature set adds the following functions.

Expansion Manager Functions

ExpCardGetSerialPort
ExpCardInfo
ExpCardPresent
ExpSlotDriverInstall
ExpSlotDriverRemove
ExpSlotEnumerate
ExpSlotLibFind

VFS Manager Feature Set

Because not every system has (or needs) Virtual File System (VFS) Manager services, applications wishing to use these services should check to make sure they are present before calling them. This is accomplished by checking for the VFS Manager's system feature with a call to FtrGet, supplying `sysFileCVFSMgr` for the feature creator and `vfsFtrIDVersion` for the feature number.

The following code shows how to check for the presence and proper version of the VFS Manger. Note that `expectedVFSMgrVersionNum` should be replaced by the actual version number you expect.

```
UInt32 vfsMgrVersion;
Err err;

err = FtrGet(sysFileCVFSMgr,
  vfsFtrIDVersion, &vfsMgrVersion);
if(err){
    // VFS Manager not installed
} else {
    // check version number of VFS Manager,
    // if necessary
    if(vfsMgrVersion == expectedVFSMgrVersionNum)
        // everything is OK
}
```

You can learn more about the VFS manager by reading Chapter 54, "Virtual File System Manager," on page 941.

Functions

This feature set adds the following functions.

VFS Manager Functions

VFSCustomControl
VFSCustomControl
VFSDirEntryEnumerate
VFSExportDatabaseToFile
VFSExportDatabaseToFileCustom
VFSFileClose
VFSFileCreate
VFSFileDBGetRecord
VFSFileDBGetResource
VFSFileSize
VFSFileTell
VFSFileWrite
VFSGetDefaultDirectory
VFSImportDatabaseFromFileCustom
VFSImportDatabaseFromFile
VFSInstallFSLib
VFSRegisterDefaultDirectory
VFSRemoveFSLib

VFSFileDBInfo
VFSFileDelete
VFSFileEOF
VFSFileGetAttributes
VFSFileGetDate
VFSFileOpen
VFSFileRead
VFSFileReadData
VFSFileRename
VFSFileResize
VFSFileSetAttributes
VFSFileSetDate
VFSFileSeek
VFSUnregisterDefaultDirectory
VFSVolumeEnumerate
VFSVolumeFormat
VFSVolumeGetLabel
VFSVolumeInfo
VFSVolumeMount
VFSVolumeSetLabel
VFSVolumeSize
VFSVolumeUnmount

C

1.0 Float Manager

This section provides reference material for the Palm OS 1.0 float manager. In Palm OS 1.0, the float manager API is declared in the header file `FloatMgr.h`. In Palm OS 2.0, this file was renamed to `FloatMgrOld.h`. In Palm OS 3.5, this file was made private.

NOTE: New applications should no longer use the 1.0 Float Manager. See Chapter 33, "Float Manager," on page 629 for the functions now provided by the Float Manager.

Float Manager Functions

FplAdd

Purpose Add two floating-point numbers (returns a + b).

Prototype
```
FloatType FplAdd (FloatType a, FloatType b)
```

Parameters

`a, b`	The floating-point numbers.

Result Returns the normalized floating-point result of the addition.

Comments Under Palm OS® 2.0 and later, most applications will want to use the arithmetic symbols instead. See the "Floating-Point" section in the *Palm OS Programmer's Companion*, vol. I.

FplAToF

Purpose Convert a zero-terminated ASCII string to a floating-point number. The string must be in the format: [-]x[.]yyyyyyyy[e[-]zz]

Prototype
```
FloatType FplAToF (char* s)
```

Parameters

`s`	Pointer to the ASCII string.

Result Returns the floating-point number.

Comments The mantissa of the number is limited to 32 bits.

See Also `FplFToA`, `FplFree`, `FplInit`

FplBase10Info

Purpose Extract detailed information on the base 10 form of a floating-point number: the base 10 mantissa, exponent, and sign.

Prototype
```
Err FplBase10Info (FloatType a, ULong* mantissaP,
Int* exponentP, Int* signP)
```

Parameters

`a`	The floating-point number.
`mantissaP`	The base 10 mantissa (return value).
`exponentP`	The base 10 exponent (return value).
`signP`	The sign, 1 or -1 (return value).

Result Returns an error code, or 0 if no error.

Comments The mantissa is normalized so it contains at least `kMaxSignificantDigits` significant digits when printed as an integer value.

`FlpBase10Info` reports that zero is "negative"; that is, it returns a one for xSign. If this is a problem, a simple workaround is:

```
if (xMantissa == 0) {
    xSign = 0;
```

FplDiv

Purpose Divide two floating-point numbers (result = dividend/divisor).

Prototype

```
FloatType FplDiv (FloatType dividend,
FloatType divisor)
```

Parameters

`dividend`	Floating-point dividend.
`divisor`	Floating-point divisor.

Result Returns the normalized floating-point result of the division.

Under Palm OS 2.0 and later, most applications will want to use the arithmetic symbols instead. See the "Floating-Point" section in the *Palm OS Programmer's Companion*, vol. I.

FplFloatToLong

Purpose Convert a floating-point number to a long integer.

Prototype

```
Long FplFloatToLong (FloatType f)
```

Parameters

`f`	Floating-point number to be converted.

Result Returns the long integer.

See Also `FplLongToFloat`, `FplFloatToULong`

FplFloatToULong

Purpose Convert a floating-point number to an unsigned long integer.

Prototype
```
ULong FplFloatToULong (FloatType f)
```

Parameters

`f`	Floating-point number to be converted.

Result Returns an unsigned long integer.

See Also `FplLongToFloat`, `FplFloatToLong`

FplFree

Purpose Release all memory allocated by the floating-point initialization.

Prototype
```
void FplFree()
```

Parameters None.

Result Returns nothing.

Comments Applications must call this routine after they've called other functions that are part of the float manager.

See Also `FplInit`

FplFToA

Purpose Convert a floating-point number to a zero-terminated ASCII string in exponential format: `[-]x.yyyyyyye[-]zz`

Prototype
```
Err FplFToA (FloatType a, char* s)
```

Parameters

`a`	Floating-point number.
`s`	Pointer to buffer to contain the ASCII string.

Result Returns an error code, or 0 if no error.

See Also FplAToF, FplFree, FplInit

FplInit

Purpose Initialize the floating-point conversion routines.

Allocate space in the system heap for floating-point globals.

Initialize the `tenPowers` array in the globals area to the powers of 10 from -99 to +99 in floating-point format.

Prototype

```
Err FplInit()
```

Parameters None.

Result Returns an error code, or 0 if no error.

Comments Applications must call this routine before calling any other `Fpl` function.

See Also FplFree

FplLongToFloat

Purpose Convert a long integer to a floating-point number.

Prototype

```
FloatType FplLongToFloat (Long x)
```

Parameters

`x`	A long integer.

Result Returns the floating-point number.

FplMul

Purpose Multiply two floating-point numbers.

Prototype
```
FloatType FplMul (FloatType a, FloatType b)
```

Parameters

`a, b`	The floating-point numbers.

Result Returns the normalized floating-point result of the multiplication.

Comments Under Palm OS 2.0 and later, most applications will want to use the arithmetic symbols instead. See the "Floating-Point" section in the *Palm OS Programmer's Companion*, vol. I.

FplSub

Purpose Subtract two floating-point numbers (returns a - b).

Prototype
```
FloatType FplSub (FloatType a, FloatType b)
```

Parameters

`a, b`	The floating-point numbers.

Result Returns the normalized floating-point result of the subtraction.

Comments Under Palm OS 2.0 and later, most applications will want to use the arithmetic symbols instead. See the "Floating-Point" section in the *Palm OS Programmer's Companion*, vol. I.

Index

Symbols

_searchF 846

Numerics

2.0 feature set 1768
3.0 feature set 1771
3.1 feature set 1775
3.2 feature set 1778
3.5 feature set 1787, 1793

A

accented characters and StrToLower 819
active form 257
active window 95, 1026, 1071
adding event to event queue 825
AddrLookupParamsType 444
AlarmMgr.h 455
alarms 455–459
    and launch codes 16
    canceling 456
    procedure alarms 457
    setting 456
    sysAppLaunchCmdTimeChange 33
alert resource 39
alerts 248
    confirmation 40
    custom alert 251, 252
    error 40
    information 40
    SysFatalAlert 376
    warning 40
allocating chunks on dynamic heap 705
AlmAlarmProcPtr 459
almErrFull 457, 458
almErrMemory 457, 458
AlmGetAlarm 455
AlmGetProcAlarm 456
almProcCmdCustom 460
AlmProcCmdEnum 459
almProcCmdReschedule 459
almProcCmdTriggered 459
AlmSetAlarm 456
AlmSetProcAlarm 457
alphaGraffitiSilkscreenArea 834
appErrorClass 587
appEvtHookKeyMask 84
AppInfoPtr 121
appInfoStringsRsc 132
AppLaunchCmd.h 443
application preferences 750
applications
    Security 32
appStopEvent 71
archiving
    marking record as archived 522
atoi function substitute (StrAToI) 805
Attention Manager 109, 461
AttnCallbackProc 485
AttnCommandArgsType 463
AttnCommandType 461
AttnDoSpecialEffects 474
attnErrMemory 471
AttnFlagsType 467
AttnForgetIt 475
AttnGetAttention 476
AttnGetCounts 478
AttnIndicatorEnable 479
AttnIndicatorEnabled 480
AttnIterate 481
AttnLaunchCodeArgsType 469
AttnLevelType 470
AttnListOpen 482
AttnNotifyDetailsType 109
AttnUpdate 482
auto-off
    setting 864
    timer 836, 837
autoRepeatKeyMask 84
Auto-Shift (field) 47

B

badDrawWindowValue 1074
BarBeamBitmap 337
BarCopyBitmap 337
BarCutBitmap 337
BarDeleteBitmap 337

BarInfoBitmap 337
BarPasteBitmap 337
BarSecureBitmap 337
BarUndoBitmap 337
base 10 form of floating-point number 632, 1804
battery timeout 843, 845
battery voltage warning threshold 843, 844
bitmap label for button 43
Bitmap.h 487
BitmapCompressionType 487
BitmapDirectInfoType 488
BitmapFlagsType 489
BitmapPtr 491
bitmapRsc 496
bitmaps
    See Also form bitmap resource
    drawing 1028
BitmapType 491
BitmapVersionOne 496
BitmapVersionTwo 496
BitmapVersionZero 496
blank lines in field 197
BmpBitsSize 497
BmpColortableSize 497
BmpCompress 498
BmpCreate 499
BmpDelete 501
BmpGetBits 501
BmpGetColortable 503
BmpGlueGetBitDepth 502, 1672
BmpGlueGetDimensions 504, 1672
BmpGlueGetNextBitmap 504, 1672
BmpSize 506
boldFont 653, 1677
boot, and heap compacting 696
bound of next line for global find 228
busy bit 565
button resource 41
    bitmap label 43
    increment arrow 42
    label 42
ButtonFrameType 143
buttons (silk-screened buttons) 83
byteAttrFirst 870
byteAttrLast 870
byteAttrMiddle 870
byteAttrSingle 870

C

calibrating the pen 733
canceling alarms 456
capsLockMask 84
card number 691
catalog resources 37
categories, setting label 283
category
    DmSeekRecordInCategory 576
    moving records 552
Category Constants 121
Category Data Structures 121
Category Functions 121
CategoryCreateList 123, 129, 1769
CategoryCreateListV10 125
categoryDefaultEditCategoryString 122, 123, 124, 126, 133, 1792
categoryDefaultEditString 125, 134
CategoryEdit 126, 1769
CategoryEditV10 128
CategoryEditV20 127
CategoryFind 128
CategoryFreeList 129, 1769
CategoryFreeListV10 130
CategoryGetName 130
CategoryGetNext 131
categoryHideEditCategory 122, 123, 124, 125, 133, 134, 1792
CategoryInitialize 122, 132
CategorySelect 132, 1769
CategorySelectV10 134
CategorySetName 135
CategorySetTriggerLabel 136
CategoryTruncateName 137
cCountryName constants 681
character attribute functions 509–512
character encodings 874, 890, 898, 904
characters
    See Also multi-byte characters

attributes 872, 875, 876, 877, 878, 879, 880, 881, 883
converting 905
drawable 882
drawing in window 1030
erasing 1036
graphic 878
inverting 1049
printable 880
size 882
sorting text 512
valid 882
CharAttr.h 509
charAttrAlNum 872
charAttrAlpha 872
charAttrCntrl 872
charAttrDelim 872
charAttrGraph 872
charAttrPrint 872
charAttrSpace 872
charEncoding... constants 680
CharEncodingType 679, 869
charEncodingUnknown 899
check box resource 43
Group ID 44
toggle area 44
checkboxTableItem 382
ChrHorizEllipsis 509, 1776
ChrIsHardKey 510
ChrNumericSpace 510, 1776
chunks
card number 691
disposing of chunk 692
heap ID 692, 705
locking 693
size 695
unlocking 695, 708
clipboard 191, 209
Clipboard.h 139
ClipboardAddItem 140
ClipboardAppendItem 141
ClipboardFormatType 139
ClipboardGetItem 142
Clipper application 1781
clipping rectangle 1073
closing net library 1240, 1243
CncAddProfile 1101
CncDefineParamID 1103
CncDefineParamId 1100
CncDeleteProfile 1104
CncGetParamType 1100, 1105
CncGetProfileInfo 1106
CncGetProfileList 1107
CncGetSystemFlagBitnum 1109
CncGetTrueParamID 1109
CncIsFixedLengthParamType 1110
CncIsSystemFlags 1110
CncIsSystemRange 1111
CncIsThirdPartiesRange 1111
CncIsVariableLengthParamType 1112
cncNotifyProfileEvent 100
CncProfileCloseDB 1112
CncProfileCount 1113
CncProfileCreate 1103, 1113
CncProfileDelete 1114
CncProfileGetIDFromIndex 1091, 1115
CncProfileGetIDFromName 1091, 1116
CncProfileGetIndex 1117
CncProfileID 1091
CncProfileNotifyDetailsType 101
CncProfileOpenDB 1117
CncProfileSetCurrent 1118
CncProfileSettingGet 1098, 1101, 1119
CncProfileSettingSet 1098, 1120
code resource 35, 36
ColorTableEntries 506
ColorTableType 494
commandChr 335, 343, 346, 347, 350
commandKeyMask 84
compacting heaps 696
comparing memory blocks 690
compatibility 1767–1802
Confirmation alert 40
connect 1304
connection manager 1785
ConnectionMgr.h 1091
constantRscType 450
Constructor
catalog resources 37

ContrastAdjust 440, 1790
control objects
and pen tracking 89
drawing 153
erasing 154
selection in a group 258
structure 143
Control.h 143
ControlAttrType 144
controlKeyMask 84
ControlPtr 145
ControlStyleType 146, 1675
ControlType 72, 73, 147
coordinates, display-relative vs. window-relative 1028
CoreTraps.h 854, 864
CountryType 677, 721
Crc.h 1081
Crc16CalcBlock 1081
creating active window 1026
creating modal window 1026
creator ID 27
CtlDrawControl 145, 153
CtlEnabled 144, 154
ctlEnterEvent 72, 73, 157, 269
CtlEraseControl 145, 154
ctlExitEvent 72, 73, 157
CtlGetLabel 148, 155
CtlGetSliderValues 155
CtlGetValue 145, 152, 156
CtlHandleEvent 72, 73, 156
CtlHideControl 144, 157
CtlHitControl 158
CtlNewControl 158
CtlNewGraphicControl 159, 160
CtlNewSliderControl 159
ctlRepeatEvent 72, 157, 269, 1791
ctlSelectEvent 72, 73, 269, 1791
CtlSetEnabled 144, 163
CtlSetGraphics 149, 150, 164
CtlSetLabel 148, 165
CtlSetSliderValues 166
CtlSetUsable 144, 167
CtlSetValue 145, 152, 167
CtlShowControl 144, 168
CtlValidatePointer 169
current time 33
custom fill patterns, getting 1045
custom UI element 51
CustomPatternType 1009
customTableItem 382, 427

D

data manager error codes 518–521, 544
data resource 36
data storage heap 704
handles 691
database ID 537
databases
closing 525
creating 526
cutting and pasting 524
deleting. *See Also* DmDatabaseProtect
overlays 560
SysCreateDataBaseList 848
DataMgr.h 513
date 174
date system resource 171
DateAdjust 923
DateDaysToDate 924
DateGlueTemplateToAscii 928, 1672
DateGlueToDOWDMFormat 931, 1672
DatePtr 917
DateSecondsToDate 924
dateStringLength 928
dateTableItem 382
DateTime.h 913
DateTimePtr 916
DateTimeType 916
DateToAscii 928
DateToDays 929
DateToDOWDMFormat 930
DateType 917
day selector object 74
Day.h 171
DayDrawDays 172
DayDrawDaySelector 173
DayHandleEvent 173

DayOfMonth 919, 931
DayOfWeek 932
DayOfWeekType 921
daySelectEvent 74
DaysInMonth 932
debugging and MemHeapScramble 699
debugging mode 690, 709
defaultAlarmSoundLevel 748
defaultAlarmSoundVolume 748
defaultAutoLockTime 748
defaultAutoLockTimeFlag 748
defaultAutoLockType 748
defaultAutoOffDuration 738, 748
defaultAutoOffDurationSecs 743, 748
defaultBoldFont 1676
defaultGameSoundLevel 748
defaultGameSoundVolume 748
defaultLargeFont 1676
defaultSmallFont 1676
defaultSysSoundLevel 748
defaultSysSoundVolume 748
defaultSystemFont 1676
delete bit 532, 535
delete callback function 1175
DeleteProc 1141, 1142, 1175
deleting databases *See Also* DmDatabaseProtect
deleting records 534
DeviceInfoType 1365
DeviceInfoType structure 1345
dialogs
  command buttons 50
  Edit Categories 126
  placement 49
digitizer
  and PenResetCalibration function 734
  and penUpEvent 90
  EvtProcessSoftKeyStroke 836
DirectionType 211
DlkGetSyncInfo 1082
DLServer.h 1081
dmAllCategories 128, 514, 558
dmAllHdrAttrs 516
dmAllRecAttrs 515
DmArchiveRecord 522
DmAttachRecord 523
DmAttachResource 524
dmCategoryLength 121, 130, 514
DmCloseDatabase 525
DmComparF 540, 551, 567, 583
DmCreateDatabase 526
DmCreateDatabaseFromImage 527
DmDatabaseInfo 528
DmDatabaseProtect 525, 530
DmDatabaseSize 531
dmDBNameLength 516, 526
DmDeleteCategory 532
DmDeleteDatabase 525, 533
DmDeleteRecord 534
DmDetachRecord 535
DmDetachResource 536
dmErrDatabaseNotProtected 519
dmErrRecordArchived 520
DmFindDatabase 527, 534, 537
DmFindRecordByID 537
DmFindResource 538
DmFindResourceType 539
DmFindSortPosition 540, 1770
DmFindSortPositionV10 541
DmGet1Resource 550, 557
DmGetAppInfoID 542
DmGetDatabase 534, 542
DmGetDatabaseLockState 543
DmGetLastErr 544
DmGetNextDatabaseByTypeCreator 545
DmGetRecord 548
DmGetResource 548, 550
DmGetResourceIndex 549
dmHdrAttrAppInfoDirty 516
dmHdrAttrBackup 516
dmHdrAttrBundle 516
dmHdrAttrCopyPrevention 516
dmHdrAttrHidden 517
dmHdrAttrLaunchableData 517
dmHdrAttrOKToInstallNewer 517
dmHdrAttrOpen 517
dmHdrAttrReadOnly 517
dmHdrAttrRecyclable 525

dmHdrAttrResDB 517
dmHdrAttrResetAfterInstall 517
dmHdrAttrStream 518
DmInsertionSort 551
dmMaxRecordIndex 515, 523, 554
dmModeExclusive 521
dmModeLeaveOpen 521
dmModeReadOnly 521
dmModeReadWrite 521
dmModeShowSecret 521
dmModeWrite 521
DmMoveCategory 552
DmMoveRecord 553
DmNewHandle 554
DmNewRecord 554
DmNewResource 555
DmNextOpenDatabase 556
DmNextOpenResDatabase 556
DmNumDatabases 557
DmNumRecords 558
DmNumRecordsInCategory 558
DmNumResources 559
DmOpenDatabase 521, 560
DmOpenDatabaseByTypeCreator 562
DmOpenDatabaseInfo 563
DmOpenDBNoOverlay 521, 564
DmOpenRef 513
DmPositionInCategory 564
DmQueryNextInCategory 565
DmQueryRecord 567
DmQuickSort 567
dmRecAttrBusy 515
dmRecAttrCategoryMask 515, 552
dmRecAttrDelete 515
dmRecAttrDirty 515
dmRecAttrSecret 515
dmRecNumCategories 121, 515
DmRecordInfo 568
DmReleaseRecord 548, 555, 569
DmReleaseResource 556, 570
DmRemoveRecord 570
DmRemoveResource 571
DmRemoveSecretRecords 572
DmResID 513
DmResizeRecord 572
DmResizeResource 573
DmResourceInfo 573
DmResType 514
DmSearchRecord 574
DmSearchResource 550, 575
DmSearchStatePtr 545
DmSearchStateType 545
dmSeekBackward 576
dmseekForward 576
DmSeekRecordInCategory 576
DmSet 577
DmSetDatabaseInfo 578
DmSetRecordInfo 580
DmSetResourceInfo 581
DmStrCopy 581
dmSysOnlyHdrAttrs 518
dmSysOnlyRecAttrs 515
dmUnfiledCategory 515, 1147
DmWrite 582
DmWriteCheck 583
doubleTapKeyMask 84
dowDateStringLength 930
dowLongDateStrLength 930
doze mode
    SysTaskDelay 866
Dragonball EZ 1776
draw window 1074
drawable characters 882
drawDetail structure 464
drawing rectangular frame 1031, 1034, 1058
drawItemsCallback 309, 319
drawList structure 465
DrawStateType 1009
DrvEntryPoint
    for virtual driver 1333
DrvrInfoType structure 1319
DrvrRcvQType structure 1321
DrvrStatusEnum 1322
dynamic heap
    adding chunk 693
    allocating chunk 705
    moving memory 702

reinitializing 863
test 697
dynamic heap handles 691
dynamic scrolling 92

E

Edit Categories dialog 126
editingStrID 124, 133
enabling windows 1027
erasing characters 1036
erasing lines in window 1037
erasing rectangle 1038
ErrAlert 586
ErrCatch 587, 591
ErrDisplay 588
ErrDisplayFileLineMsg 589
ErrEndCatch 589, 591
ErrExceptionList 590
ErrExceptionType 586
ErrFatalDisplayIf 590
errNone 1146, 1376
ErrNonFatalDisplayIf 591
Error alert 40
error code from data manager call 544
error manager 585–591
ERROR_CHECK_FULL 585
ERROR_CHECK_LEVEL 585, 588, 590, 591
ERROR_CHECK_NONE 585
ERROR_CHECK_PARTIAL 585
ErrorBase.h 585
ErrorMgr.h 585
ErrThrow 591
ErrTry 591, 592
event queue, adding event 825
Event.H 825
Event.h 67, 1791
EventPtr 71
events 67, 96
eventsEnum 68
EventType 67–96, 1790
EvtAddEventToQueue 825
EvtAddUniqueEventToQueue 826
EvtCopyEvent 827
EvtDequeuePenPoint 827
EvtDequeuePenStrokeInfo 828
EvtEnableGraffiti 828
EvtEnqueueKey 829
EvtEventAvail 829
EvtFlushKeyQueue 830
EvtFlushNextPenStroke 830
EvtFlushPenQueue 831
EvtGetEvent 88, 714, 715
EvtGetPen 832
EvtGetPenBtnList 832
EvtKeyQueueEmpty 835
EvtKeyQueueSize 835
EvtPenQueueSize 836
EvtProcessSoftKeyStroke 836
EvtResetAutoOffTimer 836, 837
EvtSetAutoOffCmd 837
EvtSysEventAvail 839
evtWaitForever 88, 831, 1428, 1429
EvtWakeup 839, 1382
EvtWakeupWithoutNilEvent 840
exchange manager 1123, 1773
ExgAccept 20, 21, 1135, 1142, 1145, 1165
exgAskCancel 17
exgAskOk 17
ExgAskParamType 18
ExgAskResultType 1123
exgBeamPrefix 1134
exgBeamScheme 1133, 1134
ExgConnect 1136, 1145
ExgControl 1138
ExgDBDeleteProcPtr 599, 985, 1141
ExgDBRead 1140, 1176
ExgDBWrite 1142
ExgDBWriteProcPtr 600, 992, 1143
ExgDialogInfoType 1146, 1147
ExgDisconnect 18, 21, 1137, 1144, 1150, 1164, 1165, 1173
ExgDoDialog 17, 1124, 1146, 1159, 1161
exgErrNoKnownTarget 1151, 1156, 1157, 1159, 1161, 1162, 1174
exgErrNotSupported 1128, 1159
ExgGet 1137, 1145, 1149, 1150, 1165, 1166
exgGet 1161

ExgGetDefaultApplication 1150
ExgGetRegisteredApplications 1152
ExgGetRegisteredTypes 1154
ExgGetTargetApplication 1155, 1159, 1161
ExgGoToType 1124
ExgLibAccept 1179
exgLibAPIVersion 1140
ExgLibClose 1180
ExgLibConnect 1180
ExgLibControl 1182
exgLibCtlGetPreview 1139
exgLibCtlGetTitle 1139
exgLibCtlGetVersion 1140
exgLibCtlSpecificOp 1140
ExgLibDisconnect 1183
ExgLibGet 1185
ExgLibHandleEvent 1186
ExgLibOpen 1186
ExgLibPut 1187
ExgLibReceive 1189
ExgLibRequest 1190
ExgLibSend 1191
ExgLibSleep 1192
exgLibSmsIncompleteDeleteOp 1702
exgLibSmsIncompleteGetCountOp 1702
exgLibSmsPrefDisplayOp 1702
exgLibSmsPrefGetDefaultOp 1701
exgLibSmsPrefGetOp 1701
exgLibSmsPrefSetOp 1702
ExgLibWake 1193
ExgLocalLib.h 1123
exgLocalOpAccept 1126
exgLocalOpGet 1126
exgLocalOpGetSender 1126
exgLocalOpNone 1126
exgLocalOpPut 1126
ExgLocalOpType 1125
exgLocalPrefix 1131, 1134
exgLocalScheme 1133, 1134
ExgLocalSocketInfoType 1125
exgMaxDescriptionLength 1153, 1167
exgMaxTitleLen 1152
exgMaxTypeLength 1153, 1154, 1166
exgMemError 1154, 1167
ExgMgr.h 593, 1123
exgNoAsk 1161
ExgNotifyGoto 1145, 1146, 1157, 1163
ExgNotifyPreview 1127, 1147, 1159
ExgNotifyReceive 1157, 1158, 1160
exgPreviewDialog 19, 1127
exgPreviewDraw 20, 1127
exgPreviewFirstUser 20, 1128
ExgPreviewInfoType 1126, 1127, 1159
exgPreviewLastUser 20, 1128
exgPreviewLongString 20, 1128
exgPreviewQuery 20, 1128
exgPreviewShortString 1128
ExgPut 18, 1137, 1138, 1143, 1145, 1163, 1173
ExgReceive 20, 21, 1137, 1142, 1150, 1165, 1177
exgRegCreatorID 1132, 1133, 1169
exgRegExtensionID 1133, 1151
ExgRegisterData 27
ExgRegisterDatatype 1166
ExgRegisterDataV35 1170
exgRegSchemeID 1132, 1133, 1151, 1169
exgRegTypeID 1133, 1168
ExgRequest 27, 1171
ExgSend 18, 1137, 1143, 1164, 1172, 1178
exgSendBeamPrefix 1134
exgSendPrefix 1134
exgSendScheme 1133, 1134, 1169
ExgSetDefaultApplication 1169, 1173
ExgSocketType 18, 596, 1124, 1129, 1141, 1143
exgTitleBufferSize 1139
exgUnwrap 1160, 1162, 1167
Expansion Manager 103, 104, 593
expCapabilityHasStorage 594
expCapabilityReadOnly 594
expCapabilitySerial 594
ExpCardGetSerialPort 596
ExpCardInfo 597
ExpCardInfoType 593
ExpCardPresent 598
expErrCardBadSector 595
expErrCardNoSectorReadWrite 595
expErrCardNotPresent 594

expErrCardProtectedSector 595
expErrCardReadOnly 595
expErrEnumerationEmpty 595
expErrIncompatibleAPIVer 595
expErrInvalidSlotRefNum 595
expErrNotEnoughPower 594
expErrNotOpen 595
expErrSlotDeallocated 595
expErrStillOpen 595
expErrUnimplemented 595
expErrUnsupportedOperation 594
expHandledSound 104
expHandledVolume 103, 104
expMediaType_Any 596
expMediaType_CompactFlash 596
expMediaType_MacSim 596
expMediaType_MemoryStick 596
expMediaType_MultiMediaCard 596
expMediaType_PoserHost 596
expMediaType_RAMDisk 596
expMediaType_SecureDigital 596
expMediaType_SmartMedia 596
ExpSlotDriverInstall 599
ExpSlotDriverRemove 600
ExpSlotEnumerate 600
ExpSlotLibFind 602
extended gadget 234, 287, 295
EZ Dragonball 1776

F

fatal alert 376
FatalAlert.h 375
fcntl 1299
FeatureMgr.h 603
features *See* functions starting with Ftr
fgetc 787
fgets 788
field objects
    and text height 201
    modifying 193
    structure 184
field resource 45
    Auto-Shift 47
    Has Scrollbar 47
Field.h 181
FieldAttrType 181
FieldPtr 184
FieldType 184
file mode constants 609, 610
file streaming 1772
FileClearerr 611
FileClose 611
FileControl 612
FileDelete 616
FileDmRead 616
FileEOF 618
FileError 618
FileFlush 619
FileGetLastError 619
FileInfoType 941
FileOpen 620
FileOpEnum 613
FileOriginEnum 623
FileRead 622
FileRef 942
FileRewind 623
FileSeek 623
FileStream.h 609
FileTell 624
FileTruncate 625
FileWrite 625
fill patterns
    getting 1045
    setting 1076
Find (global find) 22, 25, 227–230
    saving data 31
Find (lookup) 28
Find icon 83
Find.h 227
FindDrawHeader 24, 227
FindGetLineBounds 228
FindParamsType 22
FindSaveMatch 23, 26, 228
FindStrInStr 24, 230
flags, launch flags 34
FldCalcFieldHeight 189
fldChangedEvent 75, 212, 1770
FldCompactText 190

FldCopy 191
FldCut 191
FldDelete 192
FldDirty 193
FldDrawField 193
fldEnterEvent 75, 204, 269
FldEraseField 194
FldFreeMemory 194
FldGetAttributes 195
FldGetBounds 195
FldGetFont 196
FldGetInsPtPosition 196
FldGetMaxChars 197
FldGetNumberOfBlankLines 197
FldGetScrollPosition 198
FldGetScrollValues 198
FldGetSelection 199
FldGetTextAllocatedSize 200
FldGetTextHandle 200
FldGetTextHeight 201
FldGetTextLength 202
FldGetTextPtr 202
FldGetVisibleLines 203
FldGrabFocus 203, 402
FldHandleEvent 75, 76, 204
fldHeightChangedEvent 76, 206, 213, 269
FldInsert 205
FldMakeFullyVisible 206
FldNewField 207
FldPaste 209
FldRecalculateField 209
FldReleaseFocus 210
FldScrollable 211
FldScrollField 211
FldSendChangeNotification 212
FldSendHeightChangeNotification 213
FldSetAttributes 213
FldSetBounds 214
FldSetDirty 215
FldSetFont 215
FldSetInsertionPoint 216
FldSetInsPtPosition 217
FldSetMaxChars 217
FldSetMaxVisibleLines 218
FldSetScrollPosition 219
FldSetSelection 219
FldSetText 220
FldSetTextAllocatedSize 222
FldSetTextHandle 222
FldSetTextPtr 224
FldSetUsable 225
FldUndo 225
FldWordWrap 226
FloatMgr.h 629
FloatMgr.h (Palm OS 1.0) 1803
FloatMgrOld.h 1803
flushing pen queue 831
FntAverageCharWidth 643
FntBaseLine 643
FntCharHeight 644
FntCharsInWidth 644
FntCharsWidth 645
FntCharWidth 645
FntDefineFont 646
FntDescenderHeight 647
FntGetFont 647
FntGetFontPtr 648
FntGetScrollValues 648
FntGlueGetDefaultFontID 653, 1676
FntGlueWCharWidth 646, 650, 1672
FntGlueWidthToOffset 1672
FntLineHeight 649
FntLineWidth 649
FntSetFont 649
FntWCharWidth 650
FntWidthToOffset 650
FntWordWrap 651
FntWordWrapReverseNLines 652
focus
    and modal window 1052
    FrmGetFocus 259
    FrmSetFocus 286
Font.h 643
FontDefaultType 1676
fonts
    and FldGetFont 196
    font ID 647

functions 643–652
FontSelect 652
FontSelect.h 643
form bitmap resource 50
form objects
FormType structure 244
functions 247–293
form resource 48
dialog command 50
modal 48
Save Behind 48
screen command buttons 50
title 49
form, active 257
Form.h 231
FormActiveStateType 281, 282
FormAttrType 231
FormBitmapType 233
FormCheckResponseFuncType 247, 252, 293
FormEventHandlerType 245, 294
FormFrameType 233
FormGadgetAttrType 234
formGadgetDeleteCmd 253, 255, 295
formGadgetDrawCmd 290, 295
formGadgetEraseCmd 272, 295
formGadgetHandleEventCmd 295
FormGadgetHandlerType 77, 253, 255, 272, 290
FormGadgetType 77, 78, 236, 295
FormLabelType 237
FormLineType 238
FormObjAttrType 238
FormObjectKind 239
FormObjectType 240
FormObjListType 242
FormPopupType 242
FormPtr 243
FormRectangleType 243
FormTitleType 244
FormType 244, 1677
FplAdd 1803
FplAToF 1804
FplBase10Info 1804
FplDiv 1805
FplFloatToLong 1805
FplFloatToULong 1806
FplFree 1806
FplFToA 1806
FplInit 1807
FplLongToFloat 1807
FplMul 1808
FplSub 1808
fprintf 788
fputc 789
fputs 789
frame type constants 1013
FrameBitsType 1011
frames
drawing in window 1031, 1034, 1058
FrameType 1012
FrmAlert 248
FrmCloseAllForms 76, 249
frmCloseEvent 76, 249, 268, 269
FrmCopyLabel 249
FrmCopyTitle 250
FrmCustomAlert 251
FrmCustomResponseAlert 252
FrmDeleteForm 253, 295
FrmDispatchEvent 254, 268
FrmDoDialog 254
FrmDrawForm 255, 270, 295
FrmEraseForm 81, 256
frmGadgetEnterEvent 77, 269, 296
frmGadgetMiscEvent 77, 269, 296
FrmGetActiveField 256
FrmGetActiveForm 257
FrmGetActiveFormID 257
FrmGetControlGroupSelection 258
FrmGetControlValue 258
FrmGetFirstForm 259
FrmGetFocus 259
FrmGetFormBounds 260
FrmGetFormId 260
FrmGetFormPtr 261
FrmGetGadgetData 236, 261
FrmGetLabel 262
FrmGetNumberOfObjects 262
FrmGetObjectBounds 148, 149, 151, 263

FrmGetObjectId 263
FrmGetObjectIndex 264
FrmGetObjectPosition 265
FrmGetObjectPtr 266
FrmGetObjectType 266
FrmGetTitle 267
FrmGetWindowHandle 267
FrmGlueGetActiveField 1672
frmGotoEvent 78
FrmGotoForm 76, 79, 268
FrmGraffitiStateType 246
FrmHandleEvent 76, 77, 80, 90, 268, 295
FrmHelp 271
FrmHideObject 154, 158, 234, 235, 272, 295
FrmInitForm 273
frmInvalidObjectId 263
frmLoadEvent 79, 268
FrmNewBitmap 273
FrmNewForm 274
FrmNewGadget 276
FrmNewGsi 277
FrmNewLabel 278
frmNoSelectedControl 247, 258
frmOpenEvent 78, 79, 245, 254, 255, 268, 279
FrmPointInTitle 279
FrmPopupForm 79, 279
frmRedrawUpdateCode 82, 247, 291
FrmRemoveObject 280
frmResponseCreate 247, 294
frmResponseQuit 247, 294
FrmRestoreActiveState 281, 282
FrmReturnToForm 281
FrmSaveActiveState 281, 282
FrmSaveAllForms 80, 282
frmSaveEvent 80, 282
FrmSetActiveForm 95, 283
FrmSetCategoryLabel 283
FrmSetControlGroupSelection 284
FrmSetControlValue 285
FrmSetEventHandler 285
FrmSetFocus 286, 402
FrmSetGadgetData 236, 287
FrmSetGadgetHandler 236, 287
FrmSetMenu 288, 341, 348
FrmSetObjectBounds 148, 149, 151, 288
FrmSetObjectPosition 289
FrmSetTitle 289
FrmShowObject 168, 234, 290, 295
frmTitleEnterEvent 80, 270
frmTitleSelectEvent 80, 270, 1791
frmUpdateEvent 81, 247, 255, 270, 291
FrmUpdateForm 81, 82, 291
FrmUpdateScrollers 292
FrmValidatePtr 292
FrmVisible 293
ftrErrNoSuchFeature 603, 604, 606, 607, 608
FtrGet 603
FtrGetByIndex 604
FtrPtrFree 604
FtrPtrNew 605
FtrPtrResize 606
FtrSet 607
FtrUnregister 608

G

gadget resource 51, 234, 287
    extended 234, 287, 295
getchar 790
GetCharAttr 511
GetCharCaselessValue 511
GetCharSortValue 512
gethostname 1281
gets 790
GetSize 1341
GetSpace 1341
global find 22, 25, 227–230, 891, 1682
    FindDrawHeader 227
    FindGetLineBounds 228
    saving data 31
gotIt structure 466
goto (global find) 25
GoToParamsType 26
Graffiti
    Command shortcuts 87
    enabling and disabling 828
Graffiti manager
    functions 655–663

Graffiti recognizer
EvtDequeuPenPoint 827
Graffiti Reference Dialog 376
Graffiti Shift
functions 297–299
Indicator resource 52
Graffiti.h 655
GraffitiReference.h 375
GraffitiShift.h 297
GraffitiUI.h 375
graphic characters 878
GraphicControlType 148, 153
GraphicStatePtr 1018
GrfAddMacro 655
GrfAddPoint 656
GrfCleanState 656
GrfDeleteMacro 656
GrfFilterPoints 657
GrfFindBranch 657
GrfFlushPoints 657
GrfGetAndExpandMacro 658
GrfGetGlyphMapping 658
GrfGetMacro 659
GrfGetMacroName 659
GrfGetNumPoints 660
GrfGetPoint 660
GrfGetState 660
GrfInitState 661
GrfMatch 661
GrfMatchGlyph 662
GrfProcessStroke 662
GrfSetState 663
Group ID 44
groups of controls 258
GsiEnable 297
GsiEnabled 297
GsiInitialize 298
GsiSetLocation 298
GsiSetShiftState 298

H

hard reset 32
Has Scrollbar (field) 47
header line for global find 227
heap ID 699, 705
of chunk 692
heap space required 36
heaps
compacting 696
free bytes 698
ROM based 697
height of text in field 201
Help ID 40
Helper API 665
Helper.h 665
HelperNotifyEnumerateListType 665
HelperNotifyEventType 665, 667, 670
HelperNotifyExecuteType 668, 671, 672
HelperNotifyValidateType 670
HelperServiceClass.h 665
HelperServiceEMailDetailsType 670, 671
HelperServiceSMSDetailsType 670
HelperServiceSMSDetailType 672
HostControl.h 855
hostent 1232
HostGremlinIsRunning 855
HotSync and sysAppLaunchCmdSyncNotify 31
HotSync operation 117

I

icons 83
iconType 496
ID
databases 537
heap 699
iMessenger application 1782
increment arrow 42
IndexedColorType 1013
INetCacheEntryType 1639
INetCacheInfoType 1637
inetCfgName... constants 1634–1635
inetCompressionType... constants 1624
INetCompressionTypeEnum 1623
INetConfigNameType 1624
inetContentType... constants 1625
INetContentTypeEnum 1625
inetHTTPAttr... constants 1627–1628

INetHTTPAttrEnum 1625
INetLibCacheGetObject 1636
INetLibCacheList 1638
INetLibCheckAntennaState 1640
INetLibClose 1640
INetLibConfigAliasGet 1641
INetLibConfigAliasSet 1642
INetLibConfigDelete 1643
INetLibConfigIndexFromName 1644
INetLibConfigList 1645
INetLibConfigMakeActive 1646
INetLibConfigRename 1646
INetLibConfigSaveAs 1648
INetLibGetEvent 1648
INetLibOpen 1649
INetLibSettingGet 1651
INetLibSettingSet 1652
INetLibSockClose 1653
INetLibSockConnect 1653
INetLibSockHTTPAttrGet 1654
INetLibSockHTTPAttrSet 1655
INetLibSockHTTPReqCreate 1656
INetLibSockHTTPReqSend 1657
INetLibSockOpen 1659
INetLibSockRead 1660
INetLibSockSettingGet 1661
INetLibSockSettingSet 1662
INetLibSockStatus 1663
INetLibURLCrack 1664
INetLibURLGetInfo 1665
INetLibURLOpen 1666
INetLibURLsAdd 1668
INetLibWiCmd 1669
INetMgr.h 67, 1623, 1705
inetOpenURLFlag... constants 1636
inetScheme... constants 1628–1629
INetSchemeEnum 1628
inetSetting... constants 1630–1631
INetSettingEnum 1629
inetSockReadyEvent 82
inetSockSetting... constants 1632–1633
INetSockSettingEnum 1631
inetSockStatusChangeEvent 83
inetStatus... constants 1634
INetStatusEnum 1633
InetURLInfo type 1666
inetURLInfoFlag... constants 1636
InetURLType 1665
information alert 40
initialization 28
insertion point functions 301–303
insertion points
  and FldGetInsPtPosition 196
  and FldGrabFocus 203
  and FldReleaseFocus 210
  and FldSetInsertionPoint 216
  displayed in field 193
insertion sort 857
InsPoint.h 301
InsPtEnable 301
InsPtEnabled 302
InsPtGetHeight 302
InsPtGetLocation 302
InsPtSetHeight 303
InsPtSetLocation 303
international manager 1779
Internet library 1623
intlErrInvalidSelector 1086
IntlGetRoutineAddress 1085, 1086
IntlGlue.h 1678
IntlGlueGetRoutineAddress 1678
IntlMgr.h 1081, 1085, 1086, 1678
IntlSetRoutineAddress 1086
inverting characters in draw window 1049
inverting line in draw window 1050
IR Library 1773
IR manager 1195
IrAdvanceCredit 1205
IrBind 1205
IrCallbackParms 1199
IrClose 1206
IrConnectIrLap 1207
IrConnectReq 1207
IrConnectRsp 1209
IrDataReq 1210
IrDisconnectIrLap 1211
IrDiscoverReq 1212

irGetScanningMode 1200
irGetStatistics 1200
IrIAS_Add 1219
IrIAS_GetInteger 1220
IrIAS_GetIntLsap 1221
IrIAS_GetObjectID 1221
IrIAS_GetOctetString 1221
IrIAS_GetOctetStringLen 1222
IrIAS_GetType 1222
IrIAS_GetUserString 1223
IrIAS_GetUserStringCharSet 1223
IrIAS_GetUserStringLen 1223
IrIAS_Next 1224
IrIAS_Query 1224
IrIAS_SetDeviceName 1225
IrIAS_StartResult 1226
IrIasQueryCallback 1227
IrIsIrLapConnected 1212
IrIsMediaBusy 1213
IrIsNoProgress 1213
IrIsRemoteBusy 1213
irlib.h 1195
IrLocalBusy 1214
IrMaxRxSize 1214
IrMaxTxSize 1215
IrOpen 1215
irRestoreScanning 1200
irSetBaudMask 1201
IrSetConTypeLMP 1216
IrSetConTypeTTP 1216
IrSetDeviceInfo 1217
irSetScanningMode 1201
irSetSerialMode 1202
irSetSupported 1202
irSuppressScanning 1202
IrTestReq 1217
IrUnbind 1218
ISO 639 681
iterate structure 466

J

Japanese feature set 1781

K

kAttnCommandCustomEffect 462
kAttnCommandDrawDetail 462
kAttnCommandDrawList 462
kAttnCommandGoThere 462
kAttnCommandGotIt 462
kAttnCommandIterate 463
kAttnCommandPlaySound 462
kAttnCommandSnooze 462
kAttnFlagsAllBits 467
kAttnFlagsAlwaysCustomEffect 468
kAttnFlagsAlwaysLED 468
kAttnFlagsAlwaysSound 468
kAttnFlagsAlwaysVibrate 468
kAttnFlagsCapabilitiesMask 472
kAttnFlagsCustomEffectBit 467
kAttnFlagsEverything 468
kAttnFlagsHasCustomEffect 473
kAttnFlagsHasLED 473
kAttnFlagsHasSound 473
kAttnFlagsHasVibrate 473
kAttnFlagsLEDBit 467
kAttnFlagsNoCustomEffect 468
kAttnFlagsNoLED 468
kAttnFlagsNoSound 468
kAttnFlagsNothing 468
kAttnFlagsNoVibrate 468
kAttnFlagsSoundBit 467
kAttnFlagsUserSettingsMask 472
kAttnFlagsUserWantsCustomEffect 473, 745
kAttnFlagsUserWantsLED 473, 745
kAttnFlagsUserWantsSound 473, 745
kAttnFlagsUserWantsVibrate 473, 745
kAttnFlagsUseUserSettings 467
kAttnFlagsVibrateBit 467
kAttnFtrCapabilities 472
kAttnFtrCreator 472
kAttnLevelInsistent 470
kAttnLevelSubtle 470
kCncDeviceKindLocalNetwork 1100
kCncDeviceKindModem 1100
kCncDeviceKindPhone 1100
kCncDeviceKindSerial 1100

kCncErrDBAccessFailed 1114, 1115, 1116, 1117, 1118, 1119, 1120
kCncErrProfileParamNotFound 1114, 1116, 1117, 1120
kCncFtrCncMgrVersion 1786
kCncMgrVersion 1786
kCncNotifyCreateRequest 101
kCncNotifyDeleteRequest 101
kCncNotifyModifyRequest 101
kCncNotifyUpdateListRequest 102
kCncParam_PSDCreator 1096, 1099
kCncParam_PSDName 1096, 1099
kCncParam_PSDParameterBuffer 1096
kCncParam_PSDType 1096, 1099
kCncParamBaud 1092, 1098
kCncParamBluetoothDeviceAddr 1092, 1098
kCncParamBluetoothDeviceName 1092, 1098
kCncParamBuffer 1101
kCncParamCountryIndex 1092, 1098
kCncParamDeviceKind 1093, 1099
kCncParamDialingMode 1093, 1099
kCncParamFlowControl 1093, 1099
kCncParamInitString 1093, 1099
kCncParamIntlModemCountryStringList 1094
kCncParamIntlModemResetStringList 1094, 1097
kCncParamInvisible 1095, 1099
kCncParamLocked 1095, 1099
kCncParamName 1095, 1099
kCncParamNoDetails 1095, 1099
kCncParamNonEditable 1096, 1099
kCncParamPort 1096, 1099
kCncParamReadOnly 1097, 1099
kCncParamReceiveTimeOut 1097, 1099
kCncParamResetString 1097, 1099
kCncParamSerialPortFlags 1097
kCncParamString 1101
kCncParamSystemFlag 1101, 1104
kCncParamSystemFlags 1097, 1099, 1101
kCncParamThirdPartiesRange 1103
kCncParamTimeOut 1097, 1099
kCncParamTTCreator 1098, 1100
kCncParamTTType 1098, 1100
kCncParamUInt16 1101
kCncParamUInt32 1101
kCncParamUInt8 1101
kCncParamVersion 1098, 1100
kCncParamVolume 1098, 1100
kCncProfileClassicResetStringSize 1099
kCncProfileNameSize 1099
kCncProfileUsualInitStringSize 1099
kCncProfileUsualResetStringSize 1099
kCncProfileVersion 1098
kDrvrVersion 1331
kDrvrVersion3 1331
kDrvrVersion4 1331
key manager functions 675–676
key queue
  size 835
keyBitPageDown 675
keyBitPageUp 675
keyBitPower 675
keyboard display 858
KeyCurrentState 675
keyDownEvent 83, 116, 204, 245, 270, 325, 338, 345, 346, 829, 1777
KeyMgr.h 675
KeyRates 676
KeySetMask 676
kHelperNotifyActionCodeEnumerate 665, 668
kHelperNotifyActionCodeExecute 668
kHelperNotifyActionCodeValidate 668, 670
kHelperServiceClassIDEMail 670, 671, 673
kHelperServiceClassIDFax 670, 673
kHelperServiceClassIDSMS 670, 672, 673
kHelperServiceClassIDVoiceDial 670, 673
kMaxCountryNameLen 682, 684
kMaxCurrencyNameLen 682, 684
kMaxCurrencySymbolLen 682, 684, 685

L

label (button) 42
label resource 53
  bitmap label for button 43
  wrapping text 53
labelTableItem 382
LanguageType 677, 721
largeBoldFont 653, 1677, 1771

largeFont 653, 1677
launch codes
summary 3, 97, 665
SysBroadcastActionCode 847
launch flags 34
Launcher.h 375
LEVENT_DATA_IND 1203
LEVENT_DISCOVERY_CNF 1203
LEVENT_LAP_CON_CNF 1203
LEVENT_LAP_CON_IND 1203
LEVENT_LAP_DISCON_IND 1203
LEVENT_LM_CON_CNF 1203
LEVENT_LM_CON_IND 1203
LEVENT_LM_DISCON_IND 1204
LEVENT_PACKET_HANDLED 1204
LEVENT_STATUS_IND 1204
LEVENT_TEST_CNF 1204
LEVENT_TEST_IND 1204
libEvtHookKeyMask 84
libPalmOSGlue.a 1671
LineInfoPtr 188
LineInfoType 188
lines
erasing 1037
inverting 1050
list objects
and pen tracking 89
creating category list 123
drawItemsCallback 309, 319
fields 307
functions 309–319
structure 306
list resource
and popup trigger 54
vs. menu resource 54
List.h 305
ListAttrType 305
ListDrawDataFuncType 319
ListPtr 309
lists
setting items 852
ListType structure 306
lLanguageName constants 681
lmAnyCountry 687
lmAnyLanguage 179, 687, 937
lmChoice... constants 684
lmErrBadLocaleIndex 683
lmErrBadLocaleSettingChoice 683
lmErrSettingDataOverflow 683
lmErrUnknownLocale 687
LmGetLocaleSetting 686, 687
LmGlueGetLocaleSetting 685, 1672
LmGlueGetNumLocales 686, 1672
LmGlueLocaleToIndex 687, 1672
LmLocaleType 678, 684, 687, 745
local ID 700, 708
from chunk handle 695
locale 745
LocaleMgr.h 677
Localize.h 677, 1081
LocGetNumberSeparators 16, 679, 1087
locking chunk 693
locking system 32
longDateStrLength 928
lookup 28
example 28
LstDrawList 309
lstEnterEvent 84, 85, 270, 312
LstEraseList 310
lstExitEvent 85
LstGetNumberOfItems 310
LstGetSelection 311
LstGetSelectionText 311
LstGetTopItem 312
LstGetVisibleItems 312
LstGlueGetTopItem 1672
LstHandleEvent 84, 85, 312
LstMakeItemVisible 313
LstNewList 314
LstPopupList 315
LstScrollList 315
lstSelectEvent 85
LstSetDrawFunction 316
LstSetHeight 316
LstSetListChoices 317
LstSetPosition 318
LstSetSelection 318
LstSetTopItem 319

M

maxCategoryWidthID 136
maxFieldLines 207
maxFieldTextLen 217
maxStrIToALen 810
MdmDial 1229
mdmErrBusy 1229
mdmErrCmdError 1229
mdmErrNoDCD 1229
mdmErrNoTone 1229
mdmErrUserCan 1229
MdmHangUp 1230
MeasurementSystemType 684
MemCardInfo 689
MemCmp 690
MemDebugMode 690
memErrChunkLocked 694
memErrInvalidParam 605, 607, 693, 696, 705
memErrNotEnoughSpace 141, 605, 607, 693, 842, 847, 1389
MemHandleCardNo 691
MemHandleDataStorage 691
MemHandleFree 692
MemHandleHeapID 692
MemHandleLock 692
MemHandleNew 693
MemHandleResize 693
MemHandleSetOwner 694
MemHandleSize 695
MemHandleSsetOwner 694
MemHandleToLocalID 695
MemHandleUnlock 695
MemHeapCheck 696
MemHeapCompact 696
MemHeapDynamic 697
memHeapFlagReadOnly 697
MemHeapFlags 697
MemHeapFreeBytes 698
MemHeapID 698
MemHeapScramble 699
MemHeapSize 699
MemLocalIDKind 700
MemLocalIDToGlobal 700
MemLocalIDToGlobalNear 700
MemLocalIDToLockedPtr 701
MemLocalIDToPtr 701
MemMove 702
MemNumCards 702
MemNumHeaps 698, 703
MemNumRAMHeaps 703
memory
    and FldCompactText 190
    and FldFreeMemory 194
    and FldSetText 221, 224
memory blocks, comparing 690
memory card information 689
memory manager
    debugging mode 690, 709
MemoryMgr.h 689
MemPtrCardNo 704
MemPtrDataStorage 704
MemPtrFree 705
MemPtrHeapID 705
MemPtrNew 705
MemPtrRecoverHandle 706
MemPtrResize 706
MemPtrSetOwner 707
MemPtrSize 707
MemPtrSsetOwner 707
MemPtrToLocalID 708
MemPtrUnlock 708
MemSet 708
MemSetDebugMode 709
MemStoreInfo 710
Menu Item Object fields 331
menu objects
    See Also menus 328
    fields 329
    structure 328
menu pulldown object 332
Menu.h 323
MenuAddItem 330, 334, 338
MenuBarAttrType 323
MenuBarPtr 328
MenuBarType 328, 335
menuButtonCause 88, 346
menuChr 335, 343, 346, 347, 350

menuCloseEvent 86
MenuCmdBarAddButton 86, 112, 325, 335, 336
MenuCmdBarButtonType 324, 326
MenuCmdBarDisplay 339
MenuCmdBarGetButtonData 339
menuCmdBarMaxTextLength 340
menuCmdBarOnLeft 335
menuCmdBarOnRight 335
menuCmdBarOpenEvent 86, 112, 204, 270, 336, 339, 340, 346
menuCmdBarResultMenuItem 338
MenuCmdBarResultType 326
MenuCmdBarType 324, 326, 330
menuCommandCause 88, 346
MenuDispose 340
MenuDrawMenu 324, 341
MenuEraseStatus 326, 340, 342
menuErrNoMenu 334
menuErrNotFound 335
menuErrOutOfMemory 336
menuErrSameId 334
menuErrTooManyItems 336
menuEvent 81, 87, 270, 325, 338, 346
MenuGetActiveMenu 343
MenuHandleEvent 86, 87, 112, 324, 338, 341, 345
MenuHideItem 347
MenuInit 347
MenuItemType 330, 335
menuOpenEvent 87, 335, 346, 347, 350
MenuPullDownPtr 332
MenuPullDownType 330, 332
menus
  active area 56
  FrmSetMenu 288
  functions 334–348
  user interaction 56
MenuSeparatorChar 331, 333, 334
MenuSetActiveMenu 347, 348
MenuSetActiveMenuRscID 349
MenuShowItem 349
Missing Character Symbol 645
modal form 48
modal window 315, 1026, 1052
modem 1229
ModemMgr.h 1229
modified field objects 193
multi-byte characters 870, 873, 893, 895, 899, 901
  attributes 883
  comparison 871, 884
  converting 905
  encodings support 869–908
  searching 891, 1682
  size 882
multiple preferences 755

N

narrowTextTableItem 384, 390, 391, 407
net library
  closing 1240, 1243
  open count 1267
  opening 1266
netErrAlreadyOpen 1266
netErrAuthFailure 1262
netErrBadScript 1261
netErrBufTooSmall 1241, 1255, 1260, 1279, 1282
netErrBufWrongSize 1255, 1260, 1279, 1283
netErrDNSAborted 1245, 1247, 1249
netErrDNSAllocationFailure 1245, 1246, 1248
netErrDNSBadName 1244, 1246, 1248
netErrDNSBadProtocol 1245, 1247, 1249
netErrDNSFormat 1245, 1246, 1248
netErrDNSImpossible 1245, 1247, 1248
netErrDNSIrrelevant 1245, 1247, 1249
netErrDNSLabelTooLong 1245, 1246, 1248
netErrDNSNameTooLong 1244, 1246, 1248
netErrDNSNIY 1245, 1247, 1248
netErrDNSNonexistantName 1245, 1246, 1248
netErrDNSNoPort 1245, 1247, 1249
netErrDNSNoRecursion 1245, 1247, 1249
netErrDNSNoRRS 1245, 1247, 1248
netErrDNSNotInLocalCache 1245, 1247, 1249
netErrDNSRefused 1245, 1247, 1248
netErrDNSServerFailure 1245, 1246, 1248
netErrDNSTimeout 1245, 1246, 1248
netErrDNSTruncated 1245, 1247, 1249
netErrDNSUnreachable 1245, 1246, 1248
netErrInterfaceDown 1275, 1277

netErrInterfaceNotFound 1251, 1252, 1253, 1255, 1260, 1261, 1275, 1277
netErrInternal 1289, 1290
netErrInvalidInterface 1254
netErrInvalidSettingSize 1282, 1295, 1296
netErrIPCantFragment 1275, 1277
netErrIPktOverflow 1275, 1277
netErrIPNoDst 1275, 1277
netErrIPNoRoute 1275, 1277
netErrIPNoSrc 1275, 1277
netErrMessageTooBig 1275, 1277
netErrNoInterfaces 1241, 1266, 1289, 1290
netErrNoMoreSockets 1292
netErrNoMultiPacketAddr 1300
netErrNoMultiPktAddr 1275, 1277
netErrNotOpen 1240, 1243, 1244, 1246, 1248, 1250, 1253, 1261, 1263, 1268, 1270, 1272, 1275, 1277, 1284, 1285, 1287, 1288, 1289, 1290, 1292, 1294, 1296, 1299, 1300, 1301
netErrOutOfCmdBlocks 1241, 1275, 1277, 1287, 1288, 1289, 1291, 1292, 1300
netErrOutOfMemory 1243, 1266, 1292
netErrOutOfPackets 1275, 1277
netErrOutOfResources 1290
netErrParamErr 1243, 1263, 1268, 1270, 1275, 1277, 1284, 1285, 1287, 1288, 1289, 1290, 1292, 1294, 1296, 1300
netErrPortInUse 1289, 1290
netErrPPPAddressRefused 1262
netErrPPPTimeout 1261
netErrPrefNotFound 1254, 1255, 1260, 1266, 1279
netErrQuietTimeNotElapsed 1289, 1291
netErrReadOnlySetting 1260, 1283
netErrSocketAlreadyConnected 1287, 1289, 1290
netErrSocketBusy 1289, 1290
netErrSocketClosedByRemote 1275, 1277, 1284, 1285, 1287, 1289, 1290
netErrSocketInputShutdown 1300
netErrSocketNotConnected 1275, 1277, 1284
netErrSocketNotListening 1284
netErrSocketNotOpen 1243, 1268, 1270, 1275, 1277, 1284, 1285, 1287, 1288, 1289, 1290, 1294, 1296, 1300
netErrStillOpen 1240
netErrTimeout 1243, 1244, 1246, 1248, 1250, 1268, 1270, 1274, 1277, 1284, 1285, 1287, 1288, 1289, 1290, 1292, 1294, 1296, 1299
netErrTooManyInterfaces 1251
netErrTooManyTCPConnections 1289
netErrUnimplemented 1255, 1260, 1263, 1284, 1294, 1296
netErrUnknownProtocol 1250
netErrUnknownService 1250
netErrUnknownSetting 1255, 1260, 1279, 1282
netErrUnreachableDest 1275, 1277
netErrUserCancel 1243, 1261, 1268
netErrWouldBlock 1243, 1268, 1270, 1275, 1277, 1289
netErrWrongSocketType 1284, 1289, 1291, 1295, 1296
netFDIsSet 1273
netFDSet 1272
netFDSetSize 1273
NetFDSetType 1272
netFDZero 1272
NetHostInfoBufType 1231
NetHostInfoType 1232
NetHToNL 1238
NetHToNS 1238
netIFMediaDown 113
netIFMediaUp 113
NetIFSettingEnum 1254, 1255, 1260
netIOFlagDontRoute 1237
netIOFlagOutOfBand 1237
netIOFlagPeek 1237
NetIOParamType 1270
NetIOVecPtr 1270
NetIOVecType 1270
NetIPAddr 1231, 1239
NetLibAddrAToIN 1239
NetLibAddrINToA 1239
NetLibClose 1240
NetLibConnectionRefresh 1241
NetLibDmReceive 1242
NetLibFinishCloseWait 1243
NetLibGetHostByAddr 1244
NetLibGetHostByName 1246
NetLibGetMailExchangeByName 1247
NetLibGetServByName 1249
NetLibIFAttach 1250
NetLibIFDetach 1251
NetLibIFDown 1252

NetLibIFGet 1253
NetLibIFSettingGet 1254
NetLibIFSettingSet 1260
NetLibIFUp 1261
NetLibMaster 1262
NetLibOpen 1266
NetLibOpenCount 1267
NetLibReceive 1267, 1304
NetLibReceivePB 1269
NetLibSelect 1271
NetLibSend 1274, 1305
NetLibSendPB 1276
NetLibSettingGet 1279
NetLibSettingSet 1282
NetLibSocketAccept 1283, 1284
NetLibSocketAddr 1285
NetLibSocketBind 1286
NetLibSocketClose 1287
NetLibSocketConnect 1288, 1304
NetLibSocketListen 1290, 1291
NetLibSocketOpen 1291, 1304
NetLibSocketOptionGet 1294
NetLibSocketOptionSet 1295
NetLibSocketShutdown 1299
NetLibTracePrintF 1300
NetLibTracePutS 1301
NetMasterEnum 1262
netMasterICMPStats command 1265
netMasterInterfaceInfo command 1263
netMasterInterfaceStats command 1264
netMasterIPStats command 1265
NetMasterPBPtr 1262
netMasterTCPStats command 1265
netMasterTraceEventGet command 1265
netMasterUDPStats command 1265
NetMgr.h 1231
NetNToHL 1302
NetNToHS 1302
NetServInfoBufType 1233
NetServInfoType 1233
NetSettingEnum 1279, 1282
NetSocket.c 1303
NetSocketAddrEnum 1234
netSocketAddrINET 1292
NetSocketAddrINType 1234
netSocketAddrRaw 1293
NetSocketAddrRawType 1235
NetSocketAddrType 1236
netSocketDirBoth 1299
NetSocketDirEnum 1299
netSocketDirInput 1299
netSocketDirOutput 1299
NetSocketLingerType 1298
NetSocketOptEnum 1294, 1296, 1297
NetSocketOptLevelEnum 1294, 1296, 1297
netSocketProtoIPRAW 1292
netSocketProtoIPTCP 1292
netSocketProtoIPUDP 1292
NetSocketRef 1291
NetSocketTypeEnum 1236
netTracingAppMsgs 1237
netTracingErrors 1237
netTracingFuncs 1237
netTracingMsgs 1237
netTracingPkts 1237
NetUReadN 1303
NetUTCPOpen 1304
NetUWriteN 1305
new serial manager 1784
nilEvent 88, 838, 839
noFocus 245, 247, 260, 286
noListSelection 311
noMenuItemSelection 330, 333
noMenuSelection 329, 333
noPreferenceFound 749, 751
noteTextTableItem 391
notification manager 1793
notifyDetailsP 111
NotifyMgr.h 97, 711
NumberFormatType 678, 684
numericGraffitiSilkscreenArea 834
numericTableItem 382
numLockMask 84
numUneditableCategories 124, 126, 133

O

off-screen windows 1024
omErrBadOverlayDBName 728
omErrBaseRequiresOverlay 521
omErrDatabaseRequiresOverlay 561
omErrInvalidLocaleIndex 724
omErrNoNextSystemLocale 725
omErrUnknownLocale 521, 727, 728
omFtrCreator 722
omFtrDefaultLocale 722
omFtrShowErrorsFlag 722
OmGetCurrentLocale 723
OmGetIndexedLocale 724
OmGetNextSystemLocale 725
OmGetRoutineAddress 726
OmGetSystemLocale 687, 726
OmGlueGetCurrentLocale 723, 1672
OmGlueGetSystemLocale 727, 1672
OmLocaleToOverlayDBName 727
OmLocaleType 678, 721
OmOverlayDBNameToLocale 728
omOverlayDBType 560
omOverlayRscID 722
omOverlayRscType 722
OmSelector 726
OmSetSystemLocale 729
open count of net library 1267
opening net library 1266
optionKeyMask 84
OverlayMgr.h 721, 726
overlays 560

P

Palm OS 2.0 feature set 1768
Palm OS 3.0 feature set 1771
Palm OS 3.1 feature set 1775
Palm OS 3.2 feature set 1778
Palm OS 3.5 feature set 1787, 1793
PalmLocale.h 677, 679, 681
PalmOSGlue.lib 1671, 1776, 1779
panel list (SysCreatePanelList) 849
password functions 731
Password.h 731
PatternType 1013
PdiDefineReaderDictionary 1721
PdiDefineResizing 1722
PdiDefineWriterDictionary 1723
PdiEnterObject 1724
PdiLibClose 1724
PdiLibOpen 1725
PdiParameterPairTest 1725
PdiReaderDelete 1726
PdiReaderNew 1727
PdiReaderType 1706
PdiReadParameter 1728
PdiReadProperty 1729
PdiReadPropertyField 1730
PdiReadPropertyName 1731
PdiSetCharset 1732
PdiSetEncoding 1733
PdiWriteBeginObject 1734
PdiWriteEndObject 1735
PdiWriteParameter 1736
PdiWriteParameterStr 1737
PdiWriteProperty 1738
PdiWritePropertyBinaryValue 1739
PdiWritePropertyFields 1740
PdiWritePropertyStr 1741
PdiWritePropertyValue 1742
PdiWriterDelete 1743
PdiWriterNew 1743
PdiWriterType 1708
pen
    current status 832
pen manager functions 733–734
pen queue
    flushing 831
    size 836
PenBtnInfoType 833
PenCalibrate 733
penDownEvent 72, 75, 77, 80, 84, 89, 93, 156, 204, 270, 312, 345, 370
PenMgr.h 733
penMoveEvent 89
PenResetCalibration 734
penUpEvent 90, 828
PhoneLookup.h 443

PhoneNumberLookup 447
PhoneNumberLookupCustom 448
PilotMain 868
PluginCallbackProcType 1307
PluginCmdPtr 1308
PluginCmdType 1308
PluginExecCmdType 1309, 1311
PluginInfoPtr 1310
PluginInfoType 1310, 1311
pluginMaxNumOfCmds 1310
pluginNetLibCallUIProc 1312, 1314, 1317
pluginNetLibCheckCancelStatus 1312, 1316
pluginNetLibConnLog 1312, 1316
pluginNetLibDoNothing 1312, 1315
pluginNetLibGetSerLibRefNum 1313, 1317
pluginNetLibGetUserName 1312, 1316
pluginNetLibGetUserPwd 1312, 1316
pluginNetLibPromptUser 1312, 1316
pluginNetLibReadBytes 1312, 1315
pluginNetLibWriteBytes 1312, 1315
PointType 759
popSelectEvent 90, 269, 271
popup list 315
popup trigger resource 57
    and list 54
popupTriggerTableItem 383
port... constants 1332
poweredOnKeyMask 84
pref resource 36
prefAlarmSoundVolume 781, 782
preferenceDataVer2 749
preferenceDataVer3 749
preferenceDataVer4 749
preferenceDataVer5 749
preferenceDataVer6 749
preferenceDataVer8 749
preferenceDataVer9 749
preferenceDataVerLatest 749, 1777
preferenceDataVersion 1777
preferences
    changing with launch codes 30
    multiple application preferences 755
Preferences.h 677, 678, 735
prefGameSoundVolume 781, 782
PrefGetAppPreferences 750, 1770
PrefGetAppPreferencesV10 752
PrefGetPreference 111, 752
PrefGetPreferences 753
PrefOpenPreferenceDB 754
PrefOpenPreferenceDBV10 754
PrefSetAppPreferences 755
PrefSetAppPreferencesV10 756
PrefSetPreference 757
PrefSetPreferences 757
prefShowPrivateRecords 352, 353
prefSysSoundVolume 781, 782
prefTimeZone 179
PrgCallbackData 360, 361
PrgCallbackFunc 360
PrgHandleEvent 355
PrgStartDialog 356
PrgStartDialogV31 357
PrgStopDialog 358
PrgUpdateDialog 359
PrgUserCancel 360
printable characters 880
printf 790
PrivateRecords.h 351
privateRecordViewEnum 351, 743
procedure alarms 457
progress manager 1772
Progress Manager callback function 360
Progress.h 355
push button resource 58
    creating row 59
putc 791
putchar 791
puts 792
PwdExists 731
PwdRemove 731
PwdSet 732
PwdVerify 732

Q

query callback function 1227

R

radio button *See* push button
RAM-based heaps 703
RctCopyRectangle 760
RctGetIntersection 760
RctInsetRectangle 761
RctOffsetRectangle 762
RctPtInRectangle 762
RctSetRectangle 763
read callback function 1176
ReadProc 1141, 1176
records
  deleting 534
  detaching 535
  ID 537
  retrieving information 568
Rect.h 759
RectanglePtr 761
rectangles
  copying 760
  erasing 1038
  intersecting 760
  moving 762
  resizing 761
  scrolling 1070
RectangleType 759, 761
reinitializing dynamic memory heap 863
repeat control object
  and ctlRepeatEvent 72
repeating button 72
repeating button resource 59
reset 32, 863
ResGlueLoadConstant 450, 1672
ResLoadConstant 450
ResLoadForm 451
ResLoadMenu 451
resource database (SysCurAppDatabase) 850
resource ID 513
resource type 514, 539
resources
  alert 39
  check box 43
  field 45
  form 48
  form bitmap 50
  gadget 51
  Graffiti Shift Indicator 52
  label 53
  popup trigger 57
  push button 58
  repeating button 59
  retrieving 548
  retrieving information 573
  scrollbar 61
  searching for 575
  selector trigger 62
  string 63
  table 64
resumeSleepChr 116, 117
RGBColorType 495
ROM-based heaps 697, 703
ROM-based records 565, 567

S

Save Behind 48
SclDrawScrollBar 369
sclEnterEvent 91, 271, 370
sclExitEvent 91, 371
SclGetScrollBar 369
SclHandleEvent 91, 92, 370
sclRepeatEvent 92, 271, 371
  and sclExitEvent 92
SclSetScrollBar 371
scptLaunchCmdDoNothing 1311
scptLaunchCmdExecuteCmd 3, 1309, 1311
scptLaunchCmdListCmds 3, 1310, 1311
ScrDisplayMode 1069, 1774, 1790
ScrDisplayModeOperation 1069
screen command buttons 50
ScriptPlugin.h 1307
ScriptPluginLaunchCodesEnum 1311
ScriptPluginSelectorProcPtr 1314
scroll arrows
  FrmUpdateScrollers 292
scroll position in field 198
scrollbar functions 369–373
scrollbar objects
  fields 367
  in tables 404
  structure 367

scrollbar resource 61
ScrollBar.h 365
ScrollBarAttrType 365
ScrollBarPtr 366
ScrollBarRegionType 366
ScrollBarType 367
scrolling rectangle in window 1070
ScrOperation 1020
searching for string 230
secret records, removing 572
SecSelectViewStatus 352
Security application 32
SecurityAutoLockType 736
SecVerifyPW 353
SelDay.h 171
SelectDay 174, 1769
selectDayByDay 174
selectDayByMonth 174
selectDayByWeek 174
SelectDayV10 175
selection in field 199
SelectOneTime 175
selector trigger resource 62
SelectTime 177
SelectTimeZone 178
SelTime.h 171
SelTimeZone.h 171
separatorItemSelection 333
SerClearErr 1385, 1388
SerClose 1386
SerControl 1386
serCtlBreakStatus (in SerCtlEnum) 1384
serCtlEmuSetBlockingHook (in SerCtlEnum) 1384
SerCtlEnum 1383
serCtlFirstReserved (in SerCtlEnum) 1383
serCtlHandshakeThreshold (in SerCtlEnum) 1384
serCtlLAST (in SerCtlEnum) 1384
serCtlMaxBaud (in SerCtlEnum) 1384
serCtlStartBreak (in SerCtlEnum) 1383
serCtlStartLocalLoopback (in SerCtlEnum) 1384
serCtlStopBreak (in SerCtlEnum) 1383
serCtlStopLocalLoopback (in SerCtlEnum) 1384
serDev... constants 1353
serErrAlreadyOpen 1386, 1389, 1390
serErrBadParam 1387, 1389, 1397
serErrBadPort 1376
serErrConfigurationFailed 1370, 1377
serErrLineErr 1385, 1386, 1389, 1390, 1391, 1392, 1393
serErrNotOpen 1376, 1377, 1378, 1379, 1386, 1387, 1397
serErrNotSupported 1377, 1378, 1379
serErrStillOpen 1386
serErrTimeOut 1377, 1390, 1391, 1393, 1394, 1395, 1396
serFncConsole 1325, 1350
serFncDebugger 1325, 1350
serFncHotSync 1325, 1350
serFncPPPSession 1325, 1350
serFncSLIPSession 1325, 1350
serFncTelephony 1325, 1350
serFncUndefined 1325, 1350
SerGetSettings 1387
SerGetStatus 1388
serial capabilities constants 1353
Serial Library 1389
serial port feature constants 1332
serial settings constants 1354
serial status constants 1356
SerialDrvr.h 1319
SerialLinkMgr.h 1399
SerialMgr.h 1345
SerialMgrOld.h 1383
SerialVdrv.h 1319
serLineError... constants 1388
SerOpen 1389
serPortConsolePort 1351
serPortCradlePort 1351
serPortCradleRS232Port 1352
serPortCradleUSBPort 1352
serPortIrPort 1351
serPortLocalHotSync 1351
SerReceive 1390
SerReceive10 1391
SerReceiveCheck 1392
SerReceiveFlush 1392
SerReceiveWait 1393

SerSend 1394
SerSend10 1395
SerSendFlush 1396
SerSendWait 1396
SerSetReceiveBuffer 1397
SerSetSettings 1397
SerSettingsType 1384
servent 1233
sethostname 1281
shiftKeyMask 84
silk-screen buttons
    EvtGetPenBtnList 832
SilkscreenAreaType 834
silkscreenRectGraffiti 834
silkscreenRectScreen 834
SioAddCommand 792
SioClearScreen 800
SioExecCommand 801
Siofgetc 787, 790, 793
Siofgets 788, 793
Siofprintf 788, 794
Siofputc 789, 791, 794
Siofputs 789, 795
SioFree 801
Siogets 790, 795
SioHandleEvent 802
SioInit 802
SioMain 803
Sioprintf 790, 796
Sioputs 792, 796
Siosystem 797, 798
Siovfprintf 797, 799
SleepEventParamType 116
SliderControlType 150
SlkClose 1399
SlkCloseSocket 1400
slkErrAlreadyOpen 1401
slkErrBadParam 1404
slkErrBuffer 1403
slkErrChecksum 1403
slkErrNotOpen 1399
slkErrOutOfSockets 1402
slkErrSocketNotOpen 1400, 1402, 1404, 1406
slkErrTimeOut 1402, 1404
slkErrWrongDestSocket 1403
SlkFlushSocket 1400
SlkOpen 1401
SlkOpenSocket 1401
SlkPktHeaderType 1405
SlkReceivePacket 1402
SlkSendPacket 1403
SlkSetSocketListener 1404
SlkSocketListenType 1405
SlkSocketPortID 1405
SlkSocketSetTimeout 1406
SlkWriteDataType 1404
SmsLib.h 1687
SmsParamsType 1687
SmsPrefType 1690
SmsReceiveCDMAParamsType 1691
SmsReceiveGSMParamsType 1693
SmsReceiveParamsType 1694
SmsReceiveTDMAParamsType 1695
SMSReportParamsType 1696
SmsSendCDMAParamsType 1696
SMSSendParamsType 1699
SmsSendTDMAParamsType 1701
SndBlockingFuncType 785
SndCallbackInfoType 766
SndCmdIDType 767
SndCommandType 768
SndComplFuncType 784
SndCreateMidiList 769, 773
SndDoCmd 768, 774
SndGetDefaultVolume 775
SndInterruptSmfIrregardless 775
sndMaxAmp 741, 772
SndMidiListItemType 769
sndMidiNameLength 770
SndMidiRecHdrType 769
sndMidiRecSignature 770
SndMidiRecType 770
SndPlaySMF 770, 771, 776
SndPlaySmfIrregardless 778
SndPlaySmfResource 781
SndPlaySmfResourceIrregardless 782

SndPlaySystemSound 783
SndSetDefaultVolume 784
SndSmfCallbacksType 770
SndSmfChanRangeType 771
sndSmfCmdDuration 772, 776, 779
SndSmfCmdEnum 776, 779
sndSmfCmdPlay 772, 776, 779
SndSmfOptionsType 771
sndSmfPlayAllMilliSec 772
SndSysBeepType 783
sockaddr 1236
sockaddr_in 1234
socket 1304
socket listener 1403
socket listener procedure 1403, 1405
soft reset 32, 863
sorting array elements 857
sorting text 512
SortRecordInfoType 514
sound manager 1772
sound manager error codes 777, 780
sound manager functions 773–783
SoundMgr.h 765
sprintf 798
sprintf (StrPrintF) 818
SrmClearErr 1357
SrmClose 1357
SrmControl 1346, 1352, 1354, 1358, 1363
srmControlCustom 1361
SrmCtlEnum 1346, 1358
SrmCustomControl 1360
srmDefaultCTSTimeout 1363
SrmExtOpen 1351, 1362, 1363, 1367
SrmExtOpenBackground 1351, 1363, 1369
SrmGetDeviceCount 1365
SrmGetDeviceInfo 1345, 1365
SrmGetStatus 1356, 1366, 1371, 1373
SrmOpen 1351, 1363, 1367
SrmOpenBackground 1351, 1368
SrmOpenConfigType 1349, 1362, 1363
SrmPrimeWakeupHandler 1369, 1381, 1382
SrmRcvQType 1323
SrmReceive 1370, 1373, 1376, 1382
SrmReceiveCheck 1371, 1373
SrmReceiveFlush 1372
SrmReceiveWait 1373, 1376
SrmReceiveWindowClose 1374, 1382
SrmReceiveWindowOpen 1375, 1382
SrmSend 1376, 1379
SrmSendCheck 1377
SrmSendFlush 1378
SrmSendWait 1378
SrmSetReceiveBuffer 1363, 1373, 1379
srmSettings... constants 1354
SrmSetWakeupHandler 1369, 1380, 1382
srmStatus... constants 1356
stack size 36
standard IO functions 787
StartApplication
    and PrefGetPreferences 753
startup code 36
stat 625
stdFont 653, 1677
StdIOPalm.h 787
StdIOProvider.h 787
StrAToI 805
StrCaselessCompare 805
StrCat 806
strcat function substitute (StrCat) 806
StrChr 806
strchr function substitute (StrChr) 807
strcmp function substitute (StrCompare) 807
StrCompare 807
StrCompareAscii 808
StrCopy 809
strcpy function substitute (StrCopy) 809
StrDelocalizeNumber 809
StrDelocalizeNumber, and launch code 16
stricmp function substitute
  (StrCaselessCompare) 806
string manager 805–823
string resource 63
    copying 847
string searching 230
StringMgr.h 805
StrIToA 810
StrIToH 811

StrLen 811
strlen function substitute (StrLen) 811
StrLocalizeNumber 811
launch code 16
StrNCaselessCompare 813
StrNCat 814
strncat function substitute (StrNCat) 814
strncmp 816
StrNCompare 815
StrNCompareAscii 816
StrNCopy 816
strokes, translating 836
StrPrintF 798, 818
StrStr 818
strstr function substitute (StrStr) 819
StrToLower 819
structure of field object 184
StrVPrintF 800, 820
summary of launch codes 3, 97, 665
sys_socket.h 1303
SysAlarmTriggeredParamType 9, 459
SysAppLaunch 842
sysAppLaunchCmdAddRecord 6
sysAppLaunchCmdAlarmTriggered 9, 16, 457
sysAppLaunchCmdCardLaunch 14
SysAppLaunchCmdCardType 15
sysAppLaunchCmdCountryChange 16
sysAppLaunchCmdDisplayAlarm 9, 16, 457
sysAppLaunchCmdExgAskUser 17, 19, 1123, 1146, 1159, 1161, 1771
sysAppLaunchCmdExgGetData 18
sysAppLaunchCmdExgPreview 19, 21, 1135, 1147, 1159, 1165
sysAppLaunchCmdExgReceiveData 20, 21, 1135, 1142, 1145, 1148, 1165, 1771
sysAppLaunchCmdFind 22, 1683
sysAppLaunchCmdGoto 22, 25, 78, 1124, 1145, 1157, 1771
sysAppLaunchCmdGoToURL 27, 1172
sysAppLaunchCmdInitDatabase 27
sysAppLaunchCmdLookup 28, 1768
sysAppLaunchCmdNotify 29, 99, 717
sysAppLaunchCmdOpenDB 29
sysAppLaunchCmdPanelCalledFromApp 30, 1768
sysAppLaunchCmdReturnFromPanel 30, 1768
sysAppLaunchCmdSaveData 22, 31
sysAppLaunchCmdSyncNotify 31, 1168
sysAppLaunchCmdSystemLock 32, 1768
sysAppLaunchCmdSystemReset 32, 114
sysAppLaunchCmdTimeChange 33
sysAppLaunchCmdURLParams 33
SysAppLauncherDialog 375
sysAppLaunchFlagNewGlobals 25
sysAppLaunchFlagNewGlobals launch flag 34
sysAppLaunchFlagSubCal launch flag 34
sysAppLaunchFlagUIApp launch flag 34
SysBatteryInfo 843, 1774
SysBatteryInfoV20 844
SysBatteryKind 739
SysBinarySearch 845
SysBroadcastActionCode 847
SysCopyStringResource 847
SysCreateDataBaseList 848
SysCreatePanelList 849
SysCurAppDatabase 850
SysDBListItemType 848
SysDisplayAlarmParamType 16
sysErrLibNotFound 859, 860
sysErrNoFreeLibSlots 860
sysErrNoFreeRAM 860
sysErrOutOfOwnerID 842
sysErrOutOfOwnerIDs 847
sysErrParamErr 842, 847, 861
SysErrString 851
SysEvent.h 67, 1791
SysEvtMgr.h 825
sysExternalConnectorAttachEvent 102
sysExternalConnectorDetachEvent 102
SysFatalAlert 376
sysFileCBtConnectPanelHelper 1353
sysFileCSmsLib 1688
sysFileCUart328 1352
sysFileCUart328EZ 1352
sysFileCUart650 1352
sysFileCVirtIrComm 1353
sysFileCVirtRfComm 1353
sysFileDescStdIn 1272

SysFormPointerArrayToStrings 851, 1152, 1154
sysFtrDefaultBoldFont 653
sysFtrDefaultFont 653
sysFtrNewSerialPresent 1784
sysFtrNewSerialVersion 1785
sysFtrNumEncoding 1781
sysFtrNumIntlMgr 1779
sysFtrNumNotifyMgrVersion 712, 1793
sysFtrNumProcessor328 1777
sysFtrNumProcessorEZ 1777
sysFtrNumProcessorID 1777
sysFtrNumProcessorMask 1777
sysFtrNumROMVersion 1768, 1771, 1775
SysGetOSVersionString 852, 1774
SysGetROMToken 852
SysGetRomToken 1774
SysGetStackInfo 853, 1774
SysGetTrapAddress 854
SysGlueGetTrapAddress 854, 1672
SysGlueTrapExists 1679
SysGraffitiReferenceDialog 376
SysGremlins 855, 1774
SysHandleEvent 103, 108, 110, 115, 718, 856
SysInsertionSort 856
SysKeyboardDialog 858, 1770
SysKeyboardDialogV10 858
SysLibFind 859
SysLibInstall 859
SysLibLoad 860
SysLibRemove 861
sysNotifyAntennaRaisedEvent 103
SysNotifyBroadcast 712
SysNotifyBroadcastDeferred 326, 713
sysNotifyBroadcasterCode 100, 711
SysNotifyBroadcastFromInterrupt 714
sysNotifyCardInsertedEvent 103
sysNotifyCardRemovedEvent 104
sysNotifyDBDeletedEvent 105, 534
SysNotifyDBDeletedType 105
SysNotifyDBInfoType 106
sysNotifyDefaultQueueSize 711, 713
sysNotifyDeleteProtectedEvent 106
sysNotifyDeviceUnlocked 107, 111
SysNotifyDisplayChangeDetailsType 107
sysNotifyDisplayChangeEvent 107, 1061
sysNotifyEarlyWakeupEvent 108
sysNotifyErrBroadcastBusy 712
sysNotifyErrDuplicateEntry 717
sysNotifyErrEntryNotFound 719
sysNotifyErrNoStackSpace 712
sysNotifyErrQueueFull 714, 715
sysNotifyForgotPasswordEvent 108
sysNotifyGotUsersAttention 109
sysNotifyHelperEvent 109, 665, 667
sysNotifyIrDASniffEvent 98
sysNotifyLateWakeupEvent 107, 110
sysNotifyLocaleChangedEvent 111
SysNotifyLocaleChangedType 111
sysNotifyMenuCmdBarOpenEvent 86, 112, 270, 336, 340
sysNotifyNetLibIFMediaEvent 113
SysNotifyNetLibIFMediaType 113
sysNotifyNoDatabaseID 711
sysNotifyNormalPriority 712, 716
SysNotifyParamType 29, 99, 103, 667, 717
sysNotifyPhoneEvent 98
sysNotifyPOSEMountEvent 98
SysNotifyProcPtr 716, 717, 719
SysNotifyRegister 716
sysNotifyResetFinishedEvent 114
sysNotifyRetryEnqueueKey 114
sysNotifySleepNotifyEvent 115, 116
sysNotifySleepRequestEvent 115
sysNotifySyncFinishEvent 32, 117
sysNotifySyncStartEvent 32, 117
sysNotifyTimeChangeEvent 33, 118, 935
SysNotifyUnregister 718
sysNotifyVersionNum 712
sysNotifyVolumeMountedEvent 118
sysNotifyVolumeUnmountedEvent 119
SysQSort 861
SysRandom 863
sysRandomMax 863
SysReset 863
sysResIDExtPrefs 561
sysResTExtPrefs 561

SysSetAutoOffTime 864
SysSetTrapAddress 864
sysSleepAutoOff 116
sysSleepPowerButton 116
sysSleepResumed 117
sysSleepUnknown 117
SysStringByIndex 865
SysTaskDelay 866
system 798
system events
    checking availability 839
system keyboard display 858
SystemMgr.h 3, 841
SystemPreferencesChoice 737, 745
SystemPreferencesType 737
SystemResources.h 1345
SysTicksPerSecond 866
sysTrap.... 854, 864
SysTraps.h 854, 864
SysUIAppSwitch 866
SysUtils.h 841

T

table functions 390–426
table objects
    fields 387
    structure 387
table resource 64
    maximum size 64
Table.h 377
TableAttrType 377
TableColumnAttrType 378
TableDrawItemFuncPtr 390, 407
TableDrawItemFuncType 427
TableItemPtr 380
TableItemStyleType 382
TableItemType 380
TableLoadDataFuncType 390, 407, 428
tableMaxTextItemSize 381
TablePtr 384
TableRowAttrType 385
tables
    setting load data callback 420
    setting save data callback 425
TableSaveDataFuncType 429
TableType 387
tAIB 496
taif 496
task priority 36
TblDrawTable 390
TblEditing 391
tblEnterEvent 93, 94, 271
TblEraseTable 392
tblExitEvent 94
TblFindRowData 392
TblFindRowID 393
TblGetBounds 393
TblGetColumnSpacing 394
TblGetColumnWidth 394
TblGetCurrentField 395, 402
TblGetItemBounds 395
TblGetItemFont 396, 1774
TblGetItemInt 396
TblGetItemPtr 397
TblGetLastUsableRow 398
TblGetNumberOfRows 399
TblGetRowData 399
TblGetRowHeight 400
TblGetRowID 400
TblGetSelection 401
TblGlueGetNumberOfColumns 398, 1673
TblGlueGetTopRow 401, 1673
TblGlueSetSelection 426, 1673
TblGrabFocus 402
TblHandleEvent 93, 94, 403
TblHasScrollBar 404
TblInsertRow 405
TblMarkRowInvalid 405
TblMarkTableInvalid 406
TblRedrawTable 406
TblReleaseFocus 407
TblRemoveRow 408
TblRowInvalid 409
TblRowMasked 409
TblRowSelectable 410
TblRowUsable 410
tblSelectEvent 94, 404

TblSelectItem 411
TblSetBounds 412
TblSetColumnEditIndicator 412
TblSetColumnMasked 413
TblSetColumnSpacing 414
TblSetColumnUsable 414
TblSetColumnWidth 415
TblSetCustomDrawProcedure 415
TblSetItemFont 416, 1774
TblSetItemInt 417
TblSetItemPtr 418
TblSetItemStyle 418
TblSetLoadDataProcedure 420
TblSetRowData 420
TblSetRowHeight 421
TblSetRowID 421
TblSetRowMasked 422
TblSetRowSelectable 423
TblSetRowStaticHeight 424
TblSetRowUsable 424
TblSetSaveDataProcedure 425
TblUnhighlightSelection 426
tblUnusableRow 398
tbmf 496
Tbmp 496
TelCallStateType 1411
TelCancel 1423
TelCfgGetPhoneNumberType 1467
TelCfgGetSmsCenterType 1468
TelCfgSetSmsCenter 1474
TelCgfGetPhoneNumber 1471
TelCgfGetSmsCenter 1473
TelClose 1424
TelClosePhoneConnection 1425
TelDataCallNumberType 1499
TelDtcCallNumber 1504
TelDtcCloseLine 1506
TelDtcReceiveData 1507
TelDtcReceiveDataType 1500
TelDtcSendData 1508
TelDtcSendDataType 1501
TelEmcCall 1510
TelEmcCloseLine 1511
TelEmcGetNumber 1512
TelEmcGetNumberType 1501
TelEmcSelectNumber 1516
TelEmcSetNumber 1517
TelEmcSetNumberType 1502
TelephonyMgr.h 1692, 1696, 1697
TelEventType 1409
TelGetCallState 1426
TelGetEvent 1428
TelGetNumberCount 1514
TelGetTelephonyEvent 1429
TelInfGetInformation 1430
TelInformationType 1413
TelIsCfgServiceAvailable 1432
TelIsDtcServiceAvailable 1433
TelIsEmcServiceAvailable 1434
TelIsInfServiceAvailable 1436
TelIsNwkServiceAvailable 1437
TelIsOemServiceAvailable 1438
TelIsPhbServiceAvailable 1439
TelIsPhoneConnected 1441
TelIsPowServiceAvailable 1442
TelIsSmsServiceAvailable 1443
TelIsSndServiceAvailable 1444
TelIsSpcServiceAvailable 1445
TelIsStyServiceAvailable 1447
TelMatchPhoneDriver 1448
TelNwkGetLocation 1485
TelNwkGetLocationType 1481
TelNwkGetNetworkName 1487
TelNwkGetNetworkNameType 1482
TelNwkGetNetworks 1489
TelNwkGetNetworksType 1483
TelNwkGetNetworkType 1491
TelNwkGetSearchMode 1492
TelNwkGetSelectedNetwork 1097, 1493
TelNwkGetSignalLevel 1495
TelNwkSelectNetwork 1496
TelNwkSetSearchMode 1497
TelOemCall 1449
TelOemCallType 1413
TelOpen 1450
TelOpenPhoneConnection 1452

TelPhbAddEntry 1605
TelPhbDeleteEntry 1606
TelPhbEntryType 1599
TelPhbGetAvailablePhonebooks 1608
TelPhbGetAvailablePhonebooksType 1601
TelPhbGetEntries 1610
TelPhbGetEntriesType 1602
TelPhbGetEntry 1612
TelPhbGetEntryCount 1614
TelPhbGetEntryCountType 1603
TelPhbGetEntryMaxSizes 1615
TelPhbGetEntryMaxSizesType 1603
TelPhbGetSelectedPhonebook 1617
TelPhbSelectPhonebook 1618
TelPowGetBatteryStatus 1453
TelPowGetPowerLevel 1454
TelPowSetPhonePower 1456
TelSendCommandString 1456
TelSendCommandStringType 1414
TelSmsDateTimeType 1539
TelSmsDeleteMessage 1569
TelSmsDeleteMessageType 1540
TelSmsDeliveryAdvancedCDMAType 1541
TelSmsDeliveryAdvancedGSMType 1542
TelSmsDeliveryAdvancedTDMAType 1543
TelSmsDeliveryMessageType 1545
TelSmsExtensionType 1548, 1552, 1553, 1565
TelSmsGetAvailableStorage 1571
TelSmsGetAvailableStorageType 1549
TelSmsGetDataMaxSize 1573
TelSmsGetMessageCount 1574
TelSmsGetMessageCountType 1550
TelSmsGetSelectedStorage 1576
TelSmsGetUniquePartId 1577
TelSmsManualAckType 1551
TelSmsReadMessage 1579
TelSmsReadMessages 1582
TelSmsReadMessagesType 1553
TelSmsReadReport 1584
TelSmsReadReports 1586
TelSmsReadReportsType 1554
TelSmsReadSubmittedMessage 1588
TelSmsReadSubmittedMessages 1590
TelSmsReadSubmittedMessagesType 1555
TelSmsReportType 1556
TelSmsSelectStorage 1592
TelSmsSendManualAcknowledge 1593
TelSmsSendMessage 1595
TelSmsSendMessageType 1558
TelSmsSubmitAdvancedCDMAType 1559
TelSmsSubmitAdvancedGSMType 1560
TelSmsSubmitAdvancedTDMAType 1562
TelSmsSubmitMessageType 1563
TelSmsSubmittedMessageType 1565
TelSndMute 1458
TelSndPlayKeyTone 1459
TelSndPlayKeyToneType 1415
TelSndStopKeyTone 1460
TelSpcAcceptCall 1518
TelSpcCallNumber 1520
TelSpcCloseLine 1522
TelSpcConference 1524
TelSpcGetCallerNumber 1525
TelSpcGetCallerNumberType 1503
TelSpcHoldLine 1527
TelSpcPlayDTMF 1528
TelSpcPlayDTMFType 1503
TelSpcRejectCall 1529
TelSpcRetrieveHeldLine 1530
TelSpcSelectLine 1532
TelSpcSendBurstDTMF 1533
TelSpcStartContinuousDTMF 1535
TelSpcStopContinuousDTMF 1536
TelStyChangeAuthenticationCode 1476
TelStyChangeAuthenticationType 1469
TelStyEnterAuthenticationCode 1477
TelStyGetAuthenticationState 1479
text clipboard 191
text manager 869–908, 1780
text, finding with GetCharCaselessValue 511
TextMgr.h 869
textTableItem 383, 390, 391, 407
textWithNoteTableItem 383, 390, 391, 407
TimAdjust 933
TimDateTimeToSeconds 933
time manager
    structures 913
time system resource 171

time, displaying and selecting 177
TimePtr 922
timeTableItem 384
TimeToAscii 936
TimeType 922
timeZoneStringLength 937
TimGetSeconds 934
TimGetTicks 934
TimSecondsToDateTime 935
TimSetSeconds 118, 935
TimTimeZoneToUTC 938
TimUTCToTimeZone 939
tint 450
title (form) 49
titles
    active area 80
    copying form title 250
transliteration 905
translitOpLowerCase 905
translitOpPreprocess 905
TranslitOpType 905
translitOpUpperCase 905
TsmGlueGetFepMode 1672
TsmGlueSetFepMode 1673
TxtByteAttr 870
TxtCaselessCompare 871
    and StrCaselessCompare 806, 807, 813, 815
TxtCharAttr 872
TxtCharBounds 873
TxtCharEncoding 874
TxtCharIsAlNum 875
TxtCharIsAlpha 876
TxtCharIsCntrl 876
TxtCharIsDelim 877
TxtCharIsDigit 877
TxtCharIsGraph 878
TxtCharIsHardKey 878
TxtCharIsHex 879
TxtCharIsLower 879
TxtCharIsPrint 880
TxtCharIsPunct 880
TxtCharIsSpace 881
TxtCharIsUpper 881
TxtCharIsValid 882
TxtCharSize 882
TxtCharWidth 883
TxtCharXAttr 883
TxtCompare 884
TxtConvertEncoding 886
TxtEncodingName 890
txtErrConvertOverflow 888
txtErrTranslitOverflow 906
txtErrTranslitOverrun 906
txtErrTranslitUnderflow 906
txtErrUnknownEncoding 888
txtErrUnknownTranslitOp 906
TxtFindString 24, 891
    and FindStrInStr 230
TxtGetChar 893
TxtGetNextChar 893, 899
TxtGetPreviousChar 895
TxtGetTruncationOffset 896
TxtGetWordWrapOffset 897
TxtGlueByteAttr 870, 1673
TxtGlueCaselessCompare 872, 1673
TxtGlueCharAttr 873, 1673
TxtGlueCharBounds 874, 1673
TxtGlueCharEncoding 875, 1673
TxtGlueCharIsAlNum 875, 1673
TxtGlueCharIsAlpha 876, 1673
TxtGlueCharIsCntrl 876, 1673
TxtGlueCharIsDelim 877, 1673
TxtGlueCharIsDigit 877, 1673
TxtGlueCharIsGraph 878, 1673
TxtGlueCharIsHex 879, 1673
TxtGlueCharIsLower 879, 1673
TxtGlueCharIsPrint 880, 1673
TxtGlueCharIsPunct 880, 1673
TxtGlueCharIsSpace 881, 1673
TxtGlueCharIsUpper 881, 1673
TxtGlueCharIsValid 882, 1673
TxtGlueCharIsVirtual 1679
TxtGlueCharSize 883, 1673
TxtGlueCharWidth 1674
TxtGlueCharXAttr 884, 1674
TxtGlueCompare 886, 1674
TxtGlueEncodingName 891, 1674
TxtGlueFindString 24, 230, 892, 1674

TxtGlueGetChar 893, 1674
TxtGlueGetHorizEllipsisChar 1680, 1776
TxtGlueGetNextChar 894, 1674
TxtGlueGetNumericSpaceChar 1681, 1776
TxtGlueGetPreviousChar 896, 1674
TxtGlueGetTruncationOffset 896, 1674
TxtGlueLowerChar 1681
TxtGlueLowerStr 1682
TxtGlueMaxEncoding 898, 1674
TxtGlueNextCharSize 900, 1674
TxtGlueParamString 901, 1674
TxtGluePrepFindString 891, 1682
TxtGluePreviousCharSize 902, 1674
TxtGlueReplaceStr 903, 1674
TxtGlueSetNextChar 904, 1674
TxtGlueStrEncoding 905, 1674
TxtGlueStripSpaces 1683
TxtGlueTransliterate 907, 1674
TxtGlueTruncateString 1684
TxtGlueUpperChar 1685
TxtGlueUpperStr 1686
TxtGlueWordBounds 908, 1674
TxtMaxEncoding 898
TxtNameToEncoding 898
TxtNextCharSize 899
TxtParamString 900
TxtPreviousCharSize 901
TxtReplaceStr 902
TxtSetNextChar 903
TxtStrEncoding 904
TxtTransliterate 905
TxtWordBounds 907

U

UDABufferSize 1746
UDAControl 1750
UDADelete 1751
UDAEndOfReader 1752
UDAExchangeReaderNew 1756
UDAExchangeWriterNew 1757
UDAFilterJoin 1752
UDAFilterType 1746
UDAInitiateWrite 1753
UDAMemoryReaderNew 1750, 1758
UDAMoreData 1754
UDAObjectType 1746
UDARead 1754
UDAReaderType 1747
UDAWriterFlush 1755
UDAWriterJoin 1755
UDAWriterType 1748
UI resources 35–65
    custom 51
UIBrightnessAdjust 439
UIColor.h 431
UIColorGetTableEntryIndex 435
UIColorGetTableEntryRGB 436
UIColorSetTableEntry 437
UIColorTableEntries 431
UICommon.h 443
UIContrastAdjust 440
UIControls.h 439
UIPickColor 440
UIPickColorStartPalette 441
UIPickColorStartRGB 441
UIPickColorStartType 440
UIResources.h 337
UIResources.r 561
UnderlineModeType 183, 1014
unitsEnglish 735
unitsMetric 735
user name
    obtaining 1082

V

valid characters 882
vchrCommand 335, 343, 346, 347, 350
vchrHardAntenna 1783
vchrMenu 270, 335, 343, 346, 347, 350, 1791
vchrRadioCoverageFail 1783
vchrRadioCoverageOK 1783
VdrvAPIType structure 1323
VDrvClose 1334
VdrvConfigType 1324
VDrvControl 1335
VdrvCtlOpCodeEnum 1326
VDrvCustomControl 1337
VDrvOpen 1338

VDrvStatus 1340
VDrvWrite 1340
vfprintf 799
VFS Manager 941
VFSAnyMountParamType 118, 942
VFSCustomControl 951
VFSDirCreate 952
VFSDirEntryEnumerate 953
vfsErrBadData 949
vfsErrBadName 949
vfsErrBufferOverflow 949
vfsErrDirectoryNotFound 949
vfsErrDirNotEmpty 949
vfsErrFileAlreadyExists 949
vfsErrFileBadRef 950
vfsErrFileEOF 950
vfsErrFileGeneric 950
vfsErrFileNotFound 950
vfsErrFilePermissionDenied 950
vfsErrFileStillOpen 950
vfsErrIsADirectory 950
vfsErrNameShortened 950
vfsErrNoFileSystem 950
vfsErrNotADirectory 950
vfsErrVolumeBadRef 950
vfsErrVolumeFull 950
vfsErrVolumeStillMounted 950
VFSExportDatabaseToFile 956
VFSExportDatabaseToFileCustom 957
VFSExportProcPtr 1007
vfsFileAttrArchive 948
vfsFileAttrDirectory 948
vfsFileAttrHidden 948
vfsFileAttrLink 948
vfsFileAttrReadOnly 948
vfsFileAttrSystem 948
vfsFileAttrVolumeLabel 948
VFSFileClose 959
VFSFileCreate 959
VFSFileDBGetRecord 961
VFSFileDBGetResource 962
VFSFileDBInfo 964
VFSFileDelete 967
VFSFileEOF 968
VFSFileGetAttributes 969
VFSFileGetDate 970
VFSFileOpen 971
VFSFileRead 972
VFSFileReadData 974
VFSFileRename 975
VFSFileResize 977
VFSFileSeek 978
VFSFileSetAttributes 979
VFSFileSetDate 981
VFSFileSize 982
vfsFilesystemType_AFS 946
vfsFilesystemType_EXT2 946
vfsFilesystemType_FAT 946
vfsFilesystemType_FFS 946
vfsFilesystemType_HFS 946
vfsFilesystemType_HFSPlus 946
vfsFilesystemType_HPFS 946
vfsFilesystemType_MFS 946
vfsFilesystemType_NFS 946
vfsFilesystemType_Novell 946
vfsFilesystemType_NTFS 946
vfsFilesystemType_VFAT 946
VFSFileTell 983
VFSFileWrite 984
VFSGetDefaultDirectory 985
vfsHandledStartPrc 118
vfsHandledUIAppSwitch 118
VFSImportDatabaseFromFile 987
VFSImportDatabaseFromFileCustom 988
VFSImportProcPtr 1008
VFSInstallFSLib 990
vfsModeCreate 947
vfsModeExclusive 947
vfsModeLeaveOpen 947
vfsModeRead 947
vfsModeReadWrite 947
vfsModeTruncate 947
vfsModeVFSLayerOnly 947
vfsModeWrite 947
vfsMountClass_POSE 949
vfsMountClass_Simulator 949

vfsMountClass_SlotDriver 949
VFSPOSEMountParamType 944
VFSRegisterDefaultDirectory 992
VFSRemoveFSLib 993
VFSSlotMountParamType 943
VFSUnregisterDefaultDirectory 994
vfsVolumeAttrHidden 948
vfsVolumeAttrReadOnly 948
vfsVolumeAttrSlotBased 948
VFSVolumeEnumerate 995
VFSVolumeFormat 996
VFSVolumeGetLabel 999
VFSVolumeInfo 1000
VFSVolumeMount 1001
VFSVolumeSetLabel 1004
VFSVolumeSize 1005
VFSVolumeUnmount 1006
Viewer application 1781
virtual character 1679
virtual driver 1785
virtual driver functions 1333
virtual driver queue functions 1341
Virtual File System Manager 941
voltage warning threshold 843, 844
VolumeInfoType 944
vsprintf 800
vsprintf (StrVPrintF) 820

W

WakeupHandlerProc 1382
WakeupHandlerProcPtr 1381
Warning alert 40
Web Clipping Application Viewer 1781
wiCmd... constants 1670
WiCmdEnum 1669
WinClipRectangle 1021
WinCopyRectangle 1022
WinCreateBitmapWindow 1023
WinCreateOffscreenWindow 1024
WinCreateWindow 1026
WinDeleteWindow 1027
WinDirectionType 1070
WinDisplayToWindowPt 1028
window list 259
Window.h 1009
WindowFlagsType 1015
WindowFormatType 1024
windows 1009–1080
  active window 95
  structure 1016
WindowType structure 1016
WinDrawBitmap 1028
WinDrawChar 1029
WinDrawChars 1030
WinDrawGrayLine 1031
WinDrawGrayRectangleFrame 1031
WinDrawInvertedChars 1032
WinDrawLine 1033
WinDrawOperation 1018
WinDrawPixel 1033
WinDrawRectangle 1034
WinDrawRectangleFrame 1034
WinDrawTruncChars 1035
winEnterEvent 95, 283, 344
WinEraseChars 1036
WinEraseLine 1037
WinErasePixel 1037
WinEraseRectangle 1038
WinEraseRectangleFrame 1038
WinEraseWindow 1039
winExitEvent 95, 344
WinFillLine 1039
WinFillRectangle 1040
WinGetActiveWindow 1040
WinGetBitmap 1041
WinGetClip 1042
WinGetDisplayExtent 1042
WinGetDisplayWindow 1043
WinGetDrawWindow 1043
WinGetFirstWindow 1044
WinGetFramesRectangle 1045
WinGetPattern 1045
WinGetPatternType 1046
WinGetPixel 1046
WinGetPixelRGB 1047
WinGetWindowBounds 1041
WinGetWindowExtent 1047

WinGetWindowFrameRect 1048
WinGlueDrawChar 1030, 1674
WinGlueDrawTruncChars 1036, 1674
WinHandle 1020
WinIndexToRGB 1048
WinInvertChars 1049
WinInvertLine 1050
WinInvertPixel 1050
WinInvertRectangle 1051
WinInvertRectangleFrame 1051
WinLineType 1020
WinLockInitType 1065
WinModal 1052
WinPaintBitmap 1052
WinPaintChar 1053
WinPaintChars 1054
WinPaintLine 1055
WinPaintLines 1056
WinPaintPixel 1056
WinPaintPixels 1057
WinPaintRectangle 1058
WinPaintRectangleFrame 1058
WinPalette 107, 1059
WinPopDrawState 1061
WinPtr 1021
WinPushDrawState 1062
WinResetClip 1062
WinRestoreBits 1062
WinRGBToIndex 1063
WinSaveBits 1064
WinScreenLock 1065
WinScreenMode 107, 1066
WinScreenModeOperation 1066
WinScreenUnlock 1070
WinScrollRectangle 1070
WinSetActiveWindow 95, 1071
WinSetBackColor 1071
WinSetBackColorRGB 1072
WinSetBounds 1073
WinSetClip 1073
WinSetDrawMode 1074
WinSetDrawWindow 1074
WinSetForeColor 1075
WinSetForeColorRGB 1075
WinSetPattern 1076
WinSetPatternType 1077
WinSetTextColor 1077
WinSetTextColorRGB 1078
WinSetUnderlineMode 1079
WinUseTableIndexes 1060
WinValidateHandle 1080
WinWindowToDisplayPt 1080
wireless internet feature set 1781
WirelessIndicator.h 1669
word wrap 651
write callback function 1177
WriteBlock 1342
WriteByte 1343
WriteProc 1143, 1177